

Clean Code

Bản dịch song ngữ Anh–Việt · A Handbook of Agile Software Craftsmanship,
Second Edition

Robert C. Martin (nguyên tác)

2026

Mục lục

Foreword — Lời nói đầu	11
Introduction — Lời giới thiệu	18
A Note About Older Chapters — Đôi lời về các chương cũ	21
The Structure of This Book — Cấu trúc của cuốn sách	21
Introduction (from Long Ago) — Lời giới thiệu (từ thuở xa xưa)	23
Chapter 1. Clean Code — Chương 1. Clean Code	26
There Will Be Code — Sẽ luôn có mã	27
Bad Code — Mã tệ	28
Attitude — Thái độ	30
The Primal Conundrum — Nghịch lý nguyên thủy	32
The Art of Clean Code? — Nghệ thuật viết clean code?	32
What Is Clean Code? — Clean code là gì?	33
Putting All This Together — Ghép tất cả lại với nhau	37
Why Should We Be Clean? — Vì sao chúng ta phải sạch?	37
Productivity — Năng suất	40
Livability — Khả năng ở được (Livability)	44
We Read More Than We Write — Ta đọc nhiều hơn ta viết	45
The Boy Scout Rule — Quy tắc Hướng đạo sinh (the Boy Scout Rule)	46
Part I — Phần I	48
Code — Mã nguồn	48
Chapter 2. Clean That Code! — Chương 2. Dọn dẹp đoạn mã đó đi!	49
Future Bob: — Future Bob:	52
: — :	56
The Cleaning Process — Quy trình dọn dẹp	62
Conclusion — Kết luận	68
Postscript: Future Bob Playing with Grok3 — Tái bút: Bob của tương lai nghịch với Grok3	68
Postscript Conclusion — Kết luận cho phần tái bút	71
Chapter 3. First Principles — Chương 3. Những nguyên tắc nền tảng	73
Everything Small, Well Named, Organized, and Ordered — Mọi thứ đều nhỏ, được đặt tên tốt, được tổ chức và sắp xếp	74
Functions — Hàm	74
A More Significant Example — Một ví dụ đáng kể hơn	76
Independent Deployability — Triển khai độc lập được	101

A Final Thought — Một suy nghĩ sau cùng	102
Chapter 4. Meaningful Names — Chương 4. Tên có ý nghĩa	104
Use Intention-Revealing Names — Dùng tên bộc lộ ý định	106
Build a System of Names — Xây dựng một hệ thống tên gọi	107
Avoid Disinformation — Tránh thông tin sai lệch	109
Make Meaningful Distinctions — Tạo ra những khác biệt có ý nghĩa	110
Use Pronounceable Names — Dùng tên đọc lên được	112
Use Searchable Names — Dùng tên tìm kiếm được	113
Use Names of Appropriate Length — Dùng tên có độ dài phù hợp	114
Avoid Encodings — Tránh mã hoá tên	117
Use Appropriate Parts of Speech — Dùng đúng từ loại	119
Consider Keyword Parameters — Cân nhắc tham số kiểu keyword	120
Don't Be Cute — Đừng cố tỏ ra dí dỏm	121
Pick One Word per Concept — Mỗi khái niệm chọn một từ	121
Use Solution Domain Names — Dùng tên thuộc miền lời giải	122
Use Problem Domain Names — Dùng tên thuộc miền bài toán	123
Add Meaningful Context — Thêm bối cảnh có ý nghĩa	123
Don't Add Gratuitous Context — Đừng thêm bối cảnh thừa thãi	125
Final Words — Lời cuối	126
Chapter 5. Comments — Chương 5. Comment	128
Compensating for Failure — Bù đắp cho một thất bại	129
Hidden or Obscured Comments — Comment bị giấu đi hoặc bị làm mờ	130
Lying Comments — Những comment nói dối	130
Comments That Are Too Intimate — Những comment quá thân thuộc	132
Comments Do Not Make Up for Bad Code — Comment không bù đắp được cho mã tệ	132
Explain Your Intent in Code — Hãy giải thích ý định của bạn bằng mã	133
Good Comments — Comment tốt	133
Legal Comments — Comment pháp lý	133
Informative Comments — Comment cung cấp thông tin	134
Explanation of Intent — Giải thích ý định	135
Clarification — Làm rõ	136
Warning of Consequences — Cảnh báo về hậu quả	137
Amplification — Nhấn mạnh	138
Javadocs (and Their ilk) in Public APIs — Javadoc (và họ hàng của nó) trong các API công khai	138
Bad Comments — Comment tệ	139
Mumbling — Lẩm bẩm	139
Redundant Comments — Comment thừa	141
Misleading Comments — Comment gây hiểu nhầm	141
Redundancy and Imprecision — Thừa thãi và thiếu chính xác	142
Mandated Comments — Comment bị bắt buộc	144
Journal Comments — Comment nhật ký	145
Noise Comments — Comment nhiễu	145
Scary Noise — Nhiễu đáng sợ	148
Comments — Comment	148
Use a Function or a Variable Instead of a Comment — Dùng hàm hay biến thay cho comment	149
Position Markers — Dấu mốc vị trí	149
Attributions and Bylines — Ghi công và ký tên	150
Commented-Out Code — Mã bị comment lại	150
HTML Comments — Comment kiểu HTML	151

Nonlocal Information — Thông tin không thuộc chỗ đó	152
Too Much Information — Quá nhiều thông tin	153
Unobvious Connection — Liên hệ không rõ ràng	153
Function Headers — Header của hàm	154
Javadocs in Nonpublic Code — Javadoc trong mã không công khai	154
Example — Ví dụ	154
Conclusion — Kết luận	158
Chapter 6. Formatting — Chương 6. Định dạng	159
The Purpose of Formatting — Mục đích của việc định dạng	160
Vertical Formatting — Định dạng theo chiều dọc	161
Vertical Openness between Concepts — Khoảng thoáng theo chiều dọc giữa các khái niệm	162
Vertical Density — Mật độ theo chiều dọc	163
Vertical Distance — Khoảng cách theo chiều dọc	164
Variable Declarations — Khai báo biến	165
Dependent Functions — Các hàm phụ thuộc nhau	167
Conceptual Affinity — Sự tương cận về khái niệm	169
Horizontal Formatting — Định dạng theo chiều ngang	170
Horizontal Openness and Density — Khoảng thoáng và mật độ theo chiều ngang	171
Horizontal Alignment — Căn chỉnh theo chiều ngang	172
Indentation — Thụt lề	174
Breaking Indentation — Phá vỡ quy tắc thụt lề	175
Team Rules — Luật của nhóm	176
Uncle Bob's Formatting Rules — Các quy tắc định dạng của Uncle Bob	177
Chapter 7. Clean Functions — Chương 7. Hàm sạch	179
Small! — Nhỏ thôi!	180
Well-Written Prose — Văn xuôi viết khéo	181
One Level of Abstraction per Function — Mỗi hàm một mức trừu tượng	182
Reading Code from Top to Bottom: The Stepdown Rule — Đọc mã từ trên xuống: Quy tắc bậc thang	183
Entanglement — Rối bện	185
Switch Statements — Câu lệnh switch	186
Clean Functions: A Deeper Look — Hàm sạch: Nhìn sâu hơn	188
Contextual — Gắn với ngữ cảnh	188
Nameable — Đặt tên được	190
Descriptive — Mô tả rõ	190
Convenient — Tiện dụng	192
Insulated — Cách ly	194
Homogenous — Đồng nhất	198
Pure — Thuần khiết	200
Partial Purity — Thuần khiết một phần	203
Conclusion — Kết luận	204
Chapter 8. Function Heuristics — Chương 8. Kinh nghiệm thực hành về hàm	206
Function Arguments — Đối số của hàm	206
Variadic Arguments — Đối số biến thiên (variadic)	208
More Than Three? — Nhiều hơn ba thì sao?	209
Keyword Arguments — Đối số theo từ khoá	209
Flag Arguments — Đối số cờ (flag)	209
Output Arguments — Đối số đầu ra	211

Error Codes — Mã lỗi	212
Command Query Separation — Tách lệnh khỏi truy vấn	212
Prefer Exceptions to Returning Error Codes — Hãy ưu tiên exception hơn là trả về mã lỗi	214
Caveat Emptor — Liệu mà dùng	215
Extract / Blocks — Extract các khối	216
Error Handling Is One Thing — Xử lý lỗi là một việc	217
The Dependency Magnet of Error Codes — Mã lỗi — thói nam châm hút phụ thuộc	217
DRY: Don't Repeat Yourself — DRY: Đừng lặp lại chính mình	218
Simple Repeated Code — Mã lặp lại đơn giản	219
Similar Code — Mã na ná nhau	220
Loop Duplication — Trùng lặp vòng lặp	224
Accidental versus Essential Duplication — Trùng lặp ngẫu nhiên và trùng lặp bản chất	227
Side Effects — Tác dụng phụ	228
We Are Bad at This — Chúng ta rất dở khoản này	229
Functional Languages — Ngôn ngữ hàm	230
Object-Oriented Languages — Ngôn ngữ hướng đối tượng	232
Structured Programming — Lập trình có cấu trúc	234
Sequence — Tuần tự	235
Selection — Lựa chọn	235
Iteration — Lặp	235
This Is Too Much to Constantly Keep in Mind — Ôm hết chừng này trong đầu thì quá sức	236
Conclusion — Kết luận	237
Chapter 9. The Clean Method — Chương 9. Phương pháp sạch	239
Make It Right — Làm cho nó đúng	240
Example — Ví dụ	243
Considering the Design and Architecture — Ngẫm về thiết kế và kiến trúc	259
Conclusion — Kết luận	265
Chapter 10. One Thing — Chương 10. Một việc duy nhất	267
Extract Method Refactoring — Refactoring Extract Method	268
This Shouldn't Be Controversial — Chuyện này lẽ ra không nên gây tranh cãi	271
1. Drowning — 1. Chết chìm	271
2. Small Functions Don't Obscure Intent — 2. Hàm nhỏ không che mờ ý định	273
3. Performance — 3. Hiệu năng	274
4. Bouncing Around — 4. Nhảy qua nhảy lại	274
5. Entanglement — 5. Vướng víu vào nhau	275
What Are Large Functions Anyway? — Rốt cuộc thì hàm lớn là gì?	276
Extraction and Classes — Trích xuất và class	294
Conclusion — Kết luận	298
Chapter 11. Be Polite — Chương 11. Hãy lịch sự	299
The Newspaper Metaphor — Ẩn dụ tờ báo (the newspaper metaphor)	302
Be Polite — Hãy lịch sự	303
The Stepdown Rule: Once Again — Quy tắc bậc thang: một lần nữa	305
The Abstraction Roller Coaster — Trò tàu lượn trời tượng	305
This Is How We Write, but Not How We Want to Read — Ta viết mã theo kiểu này, nhưng ta không muốn đọc theo kiểu này	306
Chapter 12. Objects and Data Structures — Chương 12. Đối tượng và cấu trúc dữ liệu	308
What Is an Object? — Đối tượng là gì?	309

Data Abstraction — Trừu tượng hoá dữ liệu	311
Data/Object Antisymmetry — Tính phản đối xứng dữ liệu/đối tượng	313
The Law of Demeter — Luật Demeter	316
Trainwrecks — Trainwrecks	317
Hybrids — Thể lai (Hybrids)	318
Hiding Structure — Ẩu cấu trúc	318
Data Transfer Objects — Data Transfer Object	320
The Object/Relational “Impedance Mismatch” — “Impedance Mismatch” giữa đối tượng và cơ sở dữ liệu quan hệ	321
Using Objects and Data Structures — Dùng đối tượng và cấu trúc dữ liệu	322
Statements — Câu lệnh	323
The OO Solution — Giải pháp OO	326
Not So Fast, Johnson — Đừng vội mừng, Johnson	328
The OO/Procedural Trade-off — Đánh đổi giữa OO và thủ tục	329
But What About Performance? — Thế còn hiệu năng thì sao?	330
Conclusion — Kết luận	330
Chapter 13. Clean Classes — Chương 13. Class sạch	332
Classes and Modules versus Files — Class và module so với file	332
What Should a Class Contain? — Một class nên chứa những gì?	334
Characterizing Class Design — Mô tả đặc trưng của thiết kế class	334
Heuristics and Characteristics — Kinh nghiệm thực hành và đặc điểm	336
When Is a Class Too Large? — Khi nào thì một class là quá lớn?	338
Policies in Code — Chính sách trong mã	341
Where Reasons to Change Hide — Lý do thay đổi trốn ở đâu	342
Fixing the Problem — Sửa vấn đề	343
An Overly Open Implementation — Một hiện thực mở toang quá mức	346
Should We Do Anything Now? — Ta có nên làm gì ngay bây giờ không?	348
What About Now? — Thế còn bây giờ thì sao?	348
Closed, Cohesive, Single-Responsibility Classes — Class đóng, gắn kết, một trách nhiệm	352
When Policies Change — Khi các chính sách thay đổi	353
Is This Overengineering? — Vậy có phải là thiết kế quá đà không?	354
Simpler Testing — Kiểm thử đơn giản hơn	356
Enter AI — AI bước vào sân khấu	357
It Will Be Wrong — Nó sẽ sai	358
Chapter 14. Testing Disciplines — Chương 14. Các kỷ luật kiểm thử	360
Discipline 1: Test-Driven Development (TDD) — Kỷ luật 1: Test-Driven Development (TDD)	362
The Three Laws of TDD — Ba định luật của TDD	363
Discipline 2: Test && Commit Revert (TCR) — Kỷ luật 2: Test && Commit Revert (TCR)	364
Discipline 3: Small Bundles — Kỷ luật 3: Những gói nhỏ (Small Bundles)	365
Design — Thiết kế	366
Discipline — Kỷ luật	366
Tedious, Boring, and Slow — Rườm rà, chán ngắt và chậm chạp	367
Debugging — Gỡ lỗi	367
Documentation — Tài liệu	368
Reliability — Độ tin cậy	369
Design — Thiết kế	370
Reprise — Tóm lại	371
The Angel and the Devil — Thiên thần và ác quỷ	371
Muzzling the Devil — Bịt mõm con quỷ	372

Complications and Loopholes — Những rắc rối và kẽ hở	373
Costs and Repercussions — Chi phí và hệ lụy	375
Keeping Tests Clean — Giữ cho test luôn sạch	375
Tests Enable the -ilities — Test mở đường cho các “-ilities”	377
Chapter 15. Clean Tests — Chương 15. Test sạch	379
Domain-Specific Testing Language — Ngôn ngữ kiểm thử chuyên biệt miền	382
Composed Assertions — Composed assertion	383
Composed Test Results — Composed test result	383
Dual Standard — Hai chuẩn mực khác nhau	385
The Single Assert Rule — Quy tắc một assert duy nhất	386
The Single Act Rule — Quy tắc một act duy nhất	386
F.I.R.S.T. — F.I.R.S.T.	387
Fast — Fast — Nhanh	387
Isolated — Isolated — Cô lập	387
Repeatable — Repeatable — Lập lại được	387
Self-Validating — Self-Validating — Tự kiểm chứng	388
Timely — Timely — Đúng lúc	388
Test Design — Thiết kế test	388
Conclusion — Kết luận	389
Chapter 16. Acceptance Testing — Chương 16. Acceptance Testing	390
The Acceptance Testing Discipline — Kỷ luật acceptance testing	391
The Discipline — Kỷ luật	393
The Continuous Build — Continuous build	394
Conclusion — Kết luận	394
Chapter 17. AIs, LLMs, and God Knows What — Chương 17. AI, LLM và có trời mới biết là gì nữa	395
Programming by Prompt — Lập trình bằng prompt	397
Infancy — Thời kỳ trứng nước	402
A SWAG — Một SWAG	403
Conclusion — Kết luận	407
Part II — Phần II	408
Design — Thiết kế	408
Chapter 18. Simple Design — Chương 18. Thiết kế đơn giản	409
YAGNI — YAGNI	411
Covered by Tests — Được test phủ kín	412
An Asymptotic Goal — Một mục tiêu tiệm cận	413
Design? — Thiết kế ư?	413
Maximize Expression — Tối đa hoá khả năng biểu đạt	414
The Underlying Abstraction — Tầng trừu tượng nằm bên dưới	416
Tests: The Other Half of the Problem — Test: nửa còn lại của vấn đề	417
Minimize Duplication — Giảm thiểu trùng lặp	418
Accidental Duplication — Trùng lặp tình cờ	420
Minimize Size — Giảm thiểu kích thước	420
Simple Design — Thiết kế đơn giản	421
Chapter 19. The SOLID Principles — Chương 19. Các nguyên tắc SOLID	422

SRP: The Single Responsibility Principle — SRP: Nguyên tắc Trách nhiệm Đơn nhất	425
Accidental Duplication — Trùng lặp tình cờ	426
Solutions — Các giải pháp	428
Higher Levels — Ở các tầng cao hơn	429
OCP: The Open–Closed Principle — OCP: Nguyên tắc Đóng–Mở	430
A Thought Experiment — Một thí nghiệm tưởng tượng	430
Directional Control — Điều khiển hướng phụ thuộc	435
Information Hiding — Che giấu thông tin	435
Conclusion — Kết luận	436
LSP: The Liskov Substitution Principle — LSP: Nguyên tắc Thay thế Liskov	436
LSP and Design — LSP và thiết kế	438
Taxi Aggregator — Bộ tổng hợp dịch vụ taxi	439
ISP: The Interface Segregation Principle — ISP: Nguyên tắc Phân tách Interface	440
ISP and Language — ISP và ngôn ngữ	442
ISP and Design — ISP và thiết kế	442
DIP: The Dependency Inversion Principle — DIP: Nguyên tắc Đảo ngược Phụ thuộc	443
Stable Abstractions — Trừu tượng ổn định	446
Factories — Factory	448
Concrete Components — Các component cụ thể	450
Chapter 20. Component Principles — Chương 20. Các nguyên tắc về component	451
Chapter 21. Continuous Design — Chương 21. Thiết kế liên tục	486
Continuous Change — Thay đổi liên tục	487
Continuous Design — Thiết kế liên tục	489
Sailing on the Four Cs of Continuous Design — Dong buồm trên bốn chữ C của thiết kế liên tục	490
Clarity — Clarity — Sự rõ ràng	491
Conciseness — Súc tích	498
Confirmability — Tính xác nhận được (Confirmability)	507
Cohesion — Độ gắn kết nội tại (Cohesion)	517
When Else Do We Design? — Còn khi nào nữa thì ta thiết kế?	521
Up-front Design — Thiết kế từ đầu (Up-front Design)	521
Readying for Work — Chuẩn bị vào việc	522
Starting Work — Bắt tay vào việc	523
Doing Work — Làm việc	524
Chapter 22. Concurrency — Chương 22. Đồng thời	525
Part III — Phần III	552
Architecture — Kiến trúc	552
Chapter 23. The Two Values of Software — Chương 23. Hai giá trị của phần mềm	554
Keeping Options Open — Giữ cho các lựa chọn còn ngổ	555
Chapter 24. Independence — Chương 24. Tính độc lập	558
Use Cases — Use case	558
Operation — Vận hành	559
Development — Phát triển	560
Deployment — Triển khai	560
Leaving Options Open — Giữ cho các lựa chọn còn ngổ	561
Chapter 25. Architectural Boundaries — Chương 25. Ranh giới kiến trúc	562

What Lines Do You Draw, and When? — Bạn vạch những đường nào, và vạch khi nào? . . .	563
Plug-in Architecture — Kiến trúc plugin	566
Case Study: FitNesse — Nghiên cứu tình huống: FitNesse	568
Conclusion — Kết luận	570
Part IV — Phần IV	572
Craftmanship — Tinh thần thợ lành nghề (craftmanship)	572
“A Great Number” — “Một số lượng lớn”	573
Eight Decades — Tám thập kỷ	574
Nerds and Savors — Nerd và đáng cứu thế	579
Notoriety — Tiếng tăm	580
Role Models and Villains — Hình mẫu và kẻ ác	582
We Rule the World — Chúng ta cai trị thế giới	583
Catastrophes — Những thảm họa	585
The Oath — Lời thề	587
Chapter 26. Clean Boundaries — Chương 26. Ranh giới sạch	589
Third-Party IoT Framework: Lots o’ Boundaries — Framework IoT của bên thứ ba: ranh giới nhiều vô kể	590
UI/Application Boundary — Ranh giới UI/ứng dụng	596
SOLID and Hexagonal Architecture — SOLID và kiến trúc Hexagonal	599
Exploring and Learning Boundaries — Khám phá và học các ranh giới	601
Using Code That Does Not Yet Exist — Dùng thứ mã còn chưa tồn tại	605
Clean Boundaries — Ranh giới sạch	607
Chapter 27. The Clean Architecture — Chương 27. Clean Architecture	609
The Dependency Rule — Quy tắc Phụ thuộc	610
Entities — Entity	612
Use Cases — Use Case	612
Interface Adapters — Interface Adapter	613
Frameworks and Drivers — Framework và Driver	613
Only Four Circles? — Chỉ có bốn vòng thôi sao?	614
Crossing Boundaries — Vượt qua ranh giới	614
What Data Crosses the Boundaries — Dữ liệu nào vượt qua ranh giới	615
A Typical Scenario — Một kịch bản điển hình	615
Conclusion — Kết luận	617
Chapter 28. Harm — Chương 28. Tổn hại	618
No Harm to Society — Không gây tổn hại cho xã hội	619
Harm to Function — Tổn hại cho chức năng	621
No Harm to Structure — Không gây tổn hại cho cấu trúc	624
Soft — Mềm	626
Tests — Test	628
Chapter 29. No Defect in Behavior or Structure — Chương 29. Không để lại lỗi trong hành vi lẫn cấu trúc	630
Making It Right — Làm cho nó đúng đắn	631
What Is Good Structure? — Thế nào là cấu trúc tốt?	632
Eisenhower’s Matrix — Ma trận Eisenhower	634
Programmers Are Stakeholders — Lập trình viên cũng là bên liên quan	636
Do Your Best — Hãy làm hết sức mình	638

Chapter 30. Repeatable Proof — Chương 30. Bằng chứng lặp lại được	641
Dijkstra — Dijkstra	641
Proving Correctness — Chứng minh tính đúng đắn	642
Structured Programming — Lập trình có cấu trúc	646
Functional Decomposition — Phân rã theo chức năng	649
Test-Driven Development et al. — Test-Driven Development và những thứ tương tự	650
Chapter 31. Small Cycles — Chương 31. Chu kỳ nhỏ	654
The History of Source Code Control — Lịch sử của quản lý mã nguồn	654
Punch Cards — Thẻ đục lỗ	654
Continuous Integration — Tích hợp liên tục	664
Short Cycles — Chu kỳ ngắn	665
Continuous Integration — Tích hợp liên tục	666
Branches versus Toggles — Nhánh hay toggle	667
Continuous Deployment — Triển khai liên tục (Continuous Deployment)	669
Continuous Build — Build liên tục (Continuous Build)	671
Chapter 32. Relentless Improvement — Chương 32. Cải tiến không ngừng	673
Test Coverage — Độ phủ test	673
Mutation Testing — Mutation testing	674
Semantic Stability — Ổn định ngữ nghĩa	675
Cleaning — Dọn dẹp	676
Creations — Những gì ta tạo ra	677
Chapter 33. Maintain High Productivity — Chương 33. Duy trì năng suất cao	678
Viscosity — Độ nhớt	679
Building — Build	680
Testing — Test	680
Debugging — Debug	681
Deploying — Deploy	682
Managing Distractions — Quản lý các thứ gây xao nhãng	682
Meetings — Họp hành	682
Music — Âm nhạc	683
Mood — Tâm trạng	684
The Flow — Trạng thái flow	685
Time Management — Quản lý thời gian	686
Chapter 34. Work as a Team — Chương 34. Làm việc như một đội	688
Collaborative Programming — Lập trình cộng tác	688
Open/Virtual Office — Văn phòng mở / văn phòng ảo	689
Chapter 35. Estimate Honestly and Fairly — Chương 35. Ước lượng trung thực và công bằng	692
Lies — Những lời nói dối	693
Honesty, Accuracy, Precision — Trung thực, đúng, chính xác	694
Lessons from Me — Bài học từ chính tôi	695
Story #1: Vectors — Chuyện số 1: Vector	695
Story #2: pCCU — Chuyện số 2: pCCU	698
The Lesson — Bài học	699
Accuracy and Precision — Độ đúng và độ chính xác	699
Aggregation — Tổng hợp	701

Honesty — Trung thực	702
Pressure — Áp lực	704
Chapter 36. Respect for Fellow Programmers — Chương 36. Tôn trọng đồng nghiệp lập trình viên	706
Chapter 37. Never Stop Learning — Chương 37. Không bao giờ ngừng học	707
Afterword — Lời bạt	709
Appendix. The Clean Code Debate — Phụ lục. Cuộc tranh luận về Clean Code	715
Introductions — Lời giới thiệu	716
Method Length — Độ dài của method	718
Method Length Summary — Tóm lược về độ dài method	734
Comments — Comment	735
Comments Summary — Tổng kết về comment	754
John's Rewrite of — Bản viết lại của John	755
A Tale of Two Programmers — Chuyện hai người lập trình	762
Bob's Rewrite of — Bob viết lại	766
Test-Driven Development — Test-Driven Development	770
TDD Summary — Tóm tắt về TDD	784
Closing Remarks — Lời kết	785
Bibliography	788

Foreword — Lời nói đầu



Micah Daniel Martin

Micah Daniel Martin

You hold in your hand the secrets for building high-quality software—secrets that will make you better than you are, secrets that will help you become a master craftsman.

Bạn đang cầm trên tay những bí quyết để xây dựng phần mềm chất lượng cao — những bí quyết sẽ khiến bạn giỏi hơn con người hiện tại của mình, những bí quyết sẽ giúp bạn trở thành một người thợ bậc thầy.

Like Antonio Stradivari (perhaps the best luthier ever), Uncle Bob has spent a lifetime curating and developing the best techniques for crafting code. Unlike Stradivari, who hoarded his secrets so that only he could **craft** the best violins, Uncle Bob shares his secrets willingly, openly, hoping that you too will craft masterful code.

Giống như Antonio Stradivari (có lẽ là nghệ nhân làm đàn violin vĩ đại nhất mọi thời đại), Uncle Bob đã dành cả đời để sưu tầm và phát triển những kỹ thuật tốt nhất cho việc chế tác mã nguồn. Nhưng khác với Stradivari — người giữ khư khư bí quyết để chỉ mình ông làm ra được những cây violin hay nhất — Uncle Bob chia sẻ bí quyết của mình một cách tự nguyện và cởi mở, với hy vọng rằng bạn cũng sẽ chế tác được thứ mã nguồn bậc thầy.

This is the Way: the Way of **Clean Code**. These are my disciplines, my principles, my techniques. I strive to master them all. Why? Because they make me better. And, this is the way my father taught me.

Đây là Con Đường: Con Đường của Clean Code. Đây là những kỷ luật (discipline), những nguyên tắc, những kỹ thuật của tôi. Tôi nỗ lực để làm chủ tất cả chúng. Vì sao? Vì chúng làm tôi giỏi hơn. Và vì đó là con đường mà cha tôi đã dạy tôi.

Yes, Uncle Bob is my father. He bestowed upon me the honor of writing this foreword. Furthermore, please allow me to introduce my brother, Justin, also a Clean Coder, who wrote the afterword. How appropriate that two second editions of Uncle Bob get to sandwich his second edition of Clean Code . (Yeah . . . that sounded cheesy as I wrote it, but I'm going with it.)

Đúng vậy, Uncle Bob là cha tôi. Ông đã trao cho tôi vinh dự được viết lời nói đầu này. Hơn nữa, xin cho phép tôi giới thiệu anh trai tôi, Justin, cũng là một Clean Coder, người viết lời bạt. Thật hợp lý làm sao khi hai “ấn bản thứ hai” của Uncle Bob được kẹp quanh ấn bản thứ hai cuốn *Clean Code* của ông. (Ừ thì... viết ra thấy hơi sến, nhưng tôi vẫn cứ để vậy.)

Let me tell you a little about Uncle Bob, or at least the version of him that I know as “Dad.”

Hãy để tôi kể cho bạn nghe đôi chút về Uncle Bob, hay ít nhất là về phiên bản của ông mà tôi biết dưới cái tên “Bố”.

One of my earliest memories is riding on my dad's shoulders, playing Robot. The rules are simple. My dad was the robot, and he did whatever I told him to do . . . to a T.

Một trong những ký ức sớm nhất của tôi là ngồi trên vai bố và chơi trò Robot. Luật chơi rất đơn giản. Bố tôi là robot, và ông làm đúng y những gì tôi ra lệnh... không sai một ly.

Me: “Go to my room.”

Tôi: “Đi tới phòng con.”

Dad: “I don't know how to do that.”

Bố: “Tôi không biết làm việc đó.”

Me: Thinking . . . “Turn.”

Tôi: Nghĩ ngợi... “Quay.”

Dad: “Which way?”

Bố: “Quay hướng nào?”

Me: “Turn right.”

Tôi: “Quay phải.”

Dad: Starts turning clockwise. 90° . . . 180° . . . 360° . . . He doesn't stop turning.

Bố: Bắt đầu quay theo chiều kim đồng hồ. 90°... 180°... 360°... Ông không dừng lại.

Me: “STOP!”

Tôi: “DỪNG!”

Dad: Stops. Stands still, facing the absolute wrong direction.

Bố: Dừng. Đứng yên, mặt hướng về đúng cái hướng sai bét.

Me: “Turn right . . . STOP!” Dad: Turns and stops obediently. We are now facing down the hallway, the right direction.

Tôi: “Quay phải... DỪNG!” Bố: Ngoan ngoãn quay rồi dừng. Giờ thì chúng tôi đã nhìn dọc theo hành lang, đúng hướng.

Me: “Start walking.”

Tôi: “Bắt đầu đi.”

Dad: “I don’t know how to walk.”

Bố: “Tôi không biết đi.”

For the next few minutes, I teach the robot how to “walk” by lifting a leg the appropriate amount and falling forward, then repeating with the other leg. We agree to name this activity “walking.”

Suốt mấy phút sau đó, tôi dạy con robot cách “đi” bằng việc nhấc một chân lên vừa đủ rồi đổ người về phía trước, sau đó lặp lại với chân kia. Chúng tôi thống nhất đặt tên cho hoạt động này là “đi”.

Me: “Start walking.”

Tôi: “Bắt đầu đi.”

Dad: Starts walking.

Bố: Bắt đầu đi.

Me: Feeling exhausted by this ignorant robot, but also victorious that I successfully made it do my bidding, I prepare for the final maneuver, a left turn at the end of the hall. “Turn left!”

Tôi: Kiệt sức vì con robot ngu ngơ này, nhưng cũng hả hê vì đã sai khiến được nó, tôi chuẩn bị cho nước cờ cuối cùng: rẽ trái ở cuối hành lang. “Quay trái!”

Dad: “I am still walking” . . . and CRASH we go into the wall. Apparently the robot was single threaded. We bounce off the wall and the robot continues to walk . . . CRASH we go into the wall again . . . and again.

Bố: “Tôi vẫn đang đi”... và RẦM, chúng tôi lao thẳng vào tường. Hoá ra con robot chỉ có một luồng (thread). Chúng tôi bật ra khỏi tường và con robot tiếp tục đi... RẦM, lại đâm vào tường... rồi lại đâm nữa.

Me: Laughing uncontrollably.

Tôi: Cười ngặt nghẽo không kìm được.

At that tender age of 4 or 5 years old, my dad was teaching me how to program with the Robot game. It was fun. Fast-forward 5 years, and my dad brings home a Commodore 64. “What is it?” I inquired. “A computer,” my dad responded. “What is it for?” I asked, still confused. “Well, with a computer you can blah blah blah, blah blah, you can even play games.”

Ở cái tuổi non nớt 4 hay 5 ấy, bố đang dạy tôi lập trình bằng trò Robot. Vui lắm. Tua nhanh 5 năm, bố mang về nhà một chiếc Commodore 64. “Cái gì thế ạ?” tôi hỏi. “Máy tính,” bố đáp. “Để làm gì ạ?” tôi vẫn ngơ ngác. “À, với máy tính thì con có thể bla bla bla bla, bla bla, thậm chí chơi game được nữa.”

“Games? Show me!” I adopted that Commodore 64, or maybe my dad gave it to me . . . regardless, it lived in my room and I played lots of games on it. There was a kid in my neighborhood who knew how to pirate games, and we’d do a weekly floppy disk exchange. Other than loading games from disks, I didn’t really know how to use the computer, and anytime something went wrong, like “PRESS PLAY ON TAPE” . . . I’d shout “DAD! HELP!”

“Game hả? Cho con xem đi!” Tôi nhận nuôi chiếc Commodore 64 đó, hoặc có khi bố cho tôi luôn... dù sao thì nó cũng nằm trong phòng tôi và tôi chơi vô số game trên đó. Trong khu tôi ở có một thằng nhóc biết cách bẻ khoá game, và mỗi tuần chúng tôi lại trao đổi đĩa mềm với nhau. Ngoài việc nạp game từ đĩa ra, tôi chẳng biết dùng máy tính vào việc gì

khác, và mỗi khi có trục trặc gì đó, kiểu như “PRESS PLAY ON TAPE”... là tôi lại hét toáng lên “BỐ ƠI! CỨU CON!”

At one point, he sat me down at the Commodore and introduced me to Logo, a programming language with turtle graphics. He showed me how to make the turtle turn, move forward. It was the Robot game! I played with Logo for a while and it was . . . boring. Back to gaming.

Có lần, bố bắt tôi ngồi xuống trước chiếc Commodore và giới thiệu cho tôi Logo, một ngôn ngữ lập trình với đồ họa rùa. Ông chỉ tôi cách khiến con rùa quay, tiến tới. Đúng là trò Robot ngày nào! Tôi nghịch Logo một hồi và thấy... chán. Quay lại chơi game thôi.

Then, I observed my dad working with Logo for what seemed like forever but was probably just a couple days. When he was finally done monopolizing my computer, he showed me his Lunar Lander game that he'd just built. It was a jaw-dropping moment as I realized this is how games are built. Wow! Lunar Lander was inducted into my game rotation, and I proudly showed it off to my friends.

Rồi tôi thấy bố hì hụi với Logo, cảm giác như dài vô tận nhưng chắc chỉ vài ngày. Khi cuối cùng ông cũng chịu nhả cái máy tính của tôi ra, ông cho tôi xem trò Lunar Lander mà ông vừa viết xong. Đó là khoảnh khắc khiến tôi há hốc mồm khi nhận ra: à, game được làm ra như thế này đây. Wow! Lunar Lander lập tức được nạp vào danh sách game của tôi, và tôi tự hào khoe nó với đám bạn.

A couple of years later, my dad brought home a brand-new Apple Macintosh. It was AMAZING. “What’s that thing?” “This is a mouse.” So novel. And it had more games. Better games! The Mac lived in my dad’s office (basement), but when he was at work, the Mac was mine. Similar to the Lunar Lander incident, my dad spent weeks monopolizing the Mac to create a game he called “Pharaoh.” Pharaoh was much bigger than Lunar Lander. You were the pharaoh, and your goal was to buy enough oxen, plant enough crops, and sustain enough workers every year to build a pyramid before you died. It was a fun game, and I could tell my dad had fun building it.

Vài năm sau, bố mang về nhà một chiếc Apple Macintosh mới cứng. Nó TUYỆT VỜI. “Cái gì kia ạ?” “Đây là chuột.” Lạ lùng thật. Và nó còn có nhiều game hơn. Game hay hơn! Chiếc Mac nằm trong phòng làm việc của bố (dưới tầng hầm), nhưng khi ông đi làm thì chiếc Mac là của tôi. Giống như vụ Lunar Lander, bố tôi lại chiếm dụng chiếc Mac suốt mấy tuần để làm một trò chơi mà ông đặt tên là “Pharaoh”. Pharaoh lớn hơn Lunar Lander rất nhiều. Bạn vào vai pharaoh, và mục tiêu là mỗi năm phải mua đủ bò, trồng đủ mùa màng, nuôi đủ nhân công để xây xong kim tự tháp trước khi chết. Trò đó chơi vui, và tôi biết chắc bố tôi đã rất vui khi làm ra nó.

Programming is so much fun! That’s the conclusion I had when I reached college age. For my whole childhood, if I wasn’t having fun playing on the computers, my dad was having fun programming them. Why would I choose any other major than computer science? Programming in college was . . . not fun. Upon graduating, I was offered a job at ThoughtWorks, and, thank goodness, my dad offered me a job working with him at **Object** Mentor. Of course I accepted my dad’s offer, where I got to work with some of the biggest names and best software guys in the industry. Once again, programming was fun. It was at Object Mentor where I learned the way of Clean Code from my dad, building lots of internal tools and open source projects like FitNesse, that you’ll see later in this book.

Lập trình vui quá đi mất! Đó là kết luận của tôi khi đến tuổi vào đại học. Suốt cả tuổi thơ, nếu tôi không đang vui về nghịch máy tính thì bố tôi cũng đang vui về lập trình chúng. Vậy thì tại sao tôi lại chọn ngành nào khác ngoài khoa học máy tính? Lập trình ở đại học thì... chẳng vui. Khi tốt nghiệp, tôi được ThoughtWorks mời làm việc, và cũng may làm sao, bố mời tôi về làm cùng ông ở Object Mentor. Dĩ nhiên tôi nhận lời bố, và ở đó tôi được làm việc với vài cái tên lớn nhất, vài người giỏi nhất ngành phần mềm. Một lần nữa, lập trình

lại vui. Chính tại Object Mentor, tôi đã học được Con Đường của Clean Code từ bố mình, qua việc xây dựng vô số công cụ nội bộ và các dự án mã nguồn mở như FitNesse, thứ mà bạn sẽ gặp lại ở phần sau của cuốn sách này.

Object Mentor was primarily a training and consulting business. We told people how to build **good** software. We didn't really write the software for them. I'll admit that after several years as a trainer, I got an itch to build software again, instead of teaching how to build software.

Object Mentor chủ yếu là một công ty đào tạo và tư vấn. Chúng tôi chỉ cho người ta cách xây dựng phần mềm tốt. Chúng tôi không thực sự viết phần mềm giúp họ. Tôi phải thú nhận rằng sau vài năm làm giảng viên, tôi bắt đầu ngứa nghề, thèm được xây dựng phần mềm trở lại thay vì chỉ dạy cách xây dựng phần mềm.

An opportunity arose, and Object Mentor started developing software for a client. It was great, and it turned out to be a solid business model. So, I convinced my dad that Object Mentor should do more software development contracts. He agreed. And then I tried to convince him that I should run that branch of the business. Oops . . . that was one step too far. His response hit me like a truck. "If you want to run things, you should start your own business." Wow! Was that his way of putting me in my place? Or was he actually suggesting I start my own business? I concluded it was the latter, so I told him, "I quit."

Một cơ hội xuất hiện, và Object Mentor bắt đầu phát triển phần mềm cho một khách hàng. Mọi thứ rất tuyệt, và hoá ra đó là một mô hình kinh doanh vững chắc. Thế là tôi thuyết phục bố rằng Object Mentor nên nhận thêm nhiều hợp đồng phát triển phần mềm nữa. Ông đồng ý. Rồi tôi thử thuyết phục ông rằng tôi nên là người điều hành nhánh kinh doanh đó. Ối... bước đó là một bước quá đà. Câu trả lời của ông giáng vào tôi như một chiếc xe tải. "Nếu con muốn điều hành, con nên tự lập công ty của riêng mình." Wow! Đó là cách ông dẫn dắt tôi chăng? Hay ông thật sự đang gợi ý tôi khởi nghiệp? Tôi kết luận là vế sau, nên tôi nói với ông: "Con nghỉ việc."

I created a company called 8th Light. It was (and still is) a contract software development company. Clients hired us to build software for them. What was unique about 8th Light was that we practiced the Way of Clean Code. We didn't call it that, because the first edition of Clean Code hadn't been written yet. But we put into practice everything that my dad had taught me. The Uncle Bob Way of Coding doesn't have the same ring. Moreover, 8th Light adopted apprenticeship, which is something we had dabbled with at Object Mentor.

Tôi lập ra một công ty tên là 8th Light. Nó đã (và vẫn) là một công ty phát triển phần mềm theo hợp đồng. Khách hàng thuê chúng tôi xây dựng phần mềm cho họ. Điều khiến 8th Light khác biệt là chúng tôi thực hành Con Đường của Clean Code. Chúng tôi không gọi nó bằng cái tên đó, vì ấn bản đầu tiên của *Clean Code* khi ấy còn chưa được viết ra. Nhưng chúng tôi đưa vào thực hành mọi thứ bố đã dạy tôi. Chữ "Con Đường Viết Mã Của Uncle Bob" nghe đâu có xuôi tai bằng. Hơn nữa, 8th Light còn áp dụng chế độ học việc (apprenticeship), thứ mà chúng tôi từng thử nghiệm sơ sơ ở Object Mentor.

Everyone who joined 8th Light had to go through a 3-to-12-month apprenticeship where they learned the Way of Clean Code. We were ALL-IN on Clean Code. Did it work? Hell yeah! Demand for Clean Code went through the roof. We couldn't train apprentices fast enough. By the time of my departure from 8th Light in 2015, we had nearly a hundred team members, most of them "Clean Code" Craftsmen working on our clients' projects.

Ai gia nhập 8th Light cũng phải trải qua 3 đến 12 tháng học việc để học Con Đường của Clean Code. Chúng tôi TẤT TAY vào Clean Code. Có hiệu quả không? Quá đi chứ! Nhu cầu về Clean Code tăng vọt lên nóc nhà. Chúng tôi đào tạo học việc không kịp so với nhu cầu.

Đến lúc tôi rời 8th Light vào năm 2015, chúng tôi đã có gần một trăm thành viên, phần lớn là những người thợ “Clean Code” đang làm việc trên các dự án của khách hàng.

But maybe 8th Light was a fluke. In 2020, my dad and I decided to start Clean Coders Studio. This is a branch off our existing partnership, Clean Coders, where we sell educational software videos (<https://cleancoders.com>). However, “Studio” is our contract software development branch of the business, and yes, I do run it. Once again, we are ALL-IN on Clean Code, more than ever. Is it working? Hell yeah. Once again, we cannot train apprentices fast enough to meet demand.

Nhưng biết đâu 8th Light chỉ là một sự may mắn tình cờ. Năm 2020, bố và tôi quyết định lập Clean Coders Studio. Đây là một nhánh tách ra từ Clean Coders, quan hệ hợp tác sẵn có giữa hai bố con, nơi chúng tôi bán các video phần mềm giáo dục (<https://cleancoders.com>). Còn “Studio” là nhánh phát triển phần mềm theo hợp đồng, và đúng vậy, tôi là người điều hành nó. Một lần nữa, chúng tôi TẤT TAY vào Clean Code, còn mạnh hơn bao giờ hết. Có hiệu quả không? Quá đi chứ. Và một lần nữa, chúng tôi đào tạo học việc không kịp so với nhu cầu.

The industry wants Clean Code. The industry NEEDS Clean Code. This book is your guide into the world of Clean Code.

Ngành này muốn Clean Code. Ngành này CẦN Clean Code. Cuốn sách này là tấm bản đồ dẫn bạn vào thế giới của Clean Code.

I have some advice for you on how to best use this book.

Tôi có vài lời khuyên dành cho bạn về cách dùng cuốn sách này sao cho hiệu quả nhất.

1. Don’t rush it. There is a lot of ground to cover in these pages. Apprentices at Clean Coders Studio take months to internalize all the material. Don’t feel like you need to cram it all in a week.
2. Use it as a reference. You might want to stick some tabs on some of the pages. For example, **the SOLID principles**; you’ll likely want to jump back to that section when you’re trying to debate a **code smell** with your peers.
3. Read the code examples. When Uncle Bob walks through code changes, like the Video Store, you might be tempted to skip to the end. Don’t. Follow along.
 1. Đừng vội. Có rất nhiều thứ phải cày trong những trang sách này. Học viên học việc ở Clean Coders Studio phải mất hàng tháng mới thấm hết được tài liệu. Đừng cảm thấy bạn buộc phải nhồi tất cả vào trong một tuần.
 2. Dùng nó như tài liệu tra cứu. Bạn có lẽ sẽ muốn dán vài mẫu đánh dấu vào một số trang. Chẳng hạn các nguyên tắc SOLID; nhiều khả năng bạn sẽ muốn lật lại phần đó khi tranh luận với đồng nghiệp về một code smell (mùi mã) nào đó.
 3. Hãy đọc các ví dụ mã. Khi Uncle Bob dẫn bạn đi qua từng bước thay đổi mã, chẳng hạn ví dụ Video Store, bạn có thể sẽ muốn nhảy thẳng tới kết quả cuối. Đừng. Hãy theo cùng ông từng bước.

Ideally, you’d have Uncle Bob sitting next to you, pairing with you on this code. His walk-throughs are the next best thing.

Lý tưởng nhất là có Uncle Bob ngồi ngay cạnh bạn, pair programming với bạn trên đoạn mã đó. Những màn dẫn dắt từng bước của ông là thứ gần với điều đó nhất.

4. Practice. “Calculus is not a spectator sport.” I can still hear my professor recite that when he assigned homework. Coding is not a spectator sport either. You need to practice it to internalize it. Especially **TDD**.
5. Have fun! Uncle Bob has fun when he codes. Trust me, he does. So do I. Clean Code should help you have more fun writing code too.

4. Luyện tập. “Giải tích không phải môn thể thao để ngồi xem.” Tôi vẫn còn nghe văng vẳng giọng ông thầy đọc câu đó mỗi khi giao bài tập về nhà. Viết mã cũng chẳng phải môn thể thao để ngồi xem. Bạn phải luyện tập thì mới thấm được. Nhất là TDD.
5. Vui lên! Uncle Bob thấy vui khi viết mã. Tin tôi đi, ông vui thật. Tôi cũng vậy. Clean Code cũng nên giúp bạn thấy vui hơn khi viết mã.

—Micah Daniel Martin

—Micah Daniel Martin

Introduction — Lời giới thiệu

“Code Quality is a mindset that I’ve seen correlate with wealth and great outcomes over and over again.”

“Chất lượng mã nguồn là một tư duy mà tôi đã thấy đi thấy lại rất nhiều lần rằng nó tương quan với sự giàu có và những kết quả tuyệt vời.”

—David Heinemeier Hansson (DHH)

—David Heinemeier Hansson (DHH)

As I type this, it has been over a decade and a half since *Clean Code* was first published. Since then, I have participated in many discussions, podcasts, interviews, blog postings, and social network flame wars inspired by that first edition. Recently it occurred to me that there are quite a few more things I could say; and several more that I would say differently than I did in 2008.

Lúc tôi gõ những dòng này, đã hơn mười lăm năm kể từ khi *Clean Code* được xuất bản lần đầu. Kể từ đó, tôi đã tham gia vô số cuộc thảo luận, podcast, phỏng vấn, bài blog và cả những trận khẩu chiến trên mạng xã hội, tất cả đều bắt nguồn từ ấn bản đầu tiên ấy. Gần đây tôi chợt nhận ra rằng còn khá nhiều điều nữa tôi có thể nói; và cũng có vài điều tôi sẽ nói khác đi so với cách tôi đã nói năm 2008.

I first proposed the *Clean Code* book in April of 2006. This was before there were iPhones. Mobile internet was mostly GPRS, or maybe EDGE. Nobody was talking about the Internet of Things. YouTube was barely one year old, and hardly anyone had heard of it—fewer took it seriously. Languages like Kotlin, Swift, Go, Dart, Rust, Elm, Elixir, and Clojure had not yet appeared. *Meet the Fockers* was the top-grossing movie up to that point.

Tôi đề xuất viết cuốn *Clean Code* lần đầu vào tháng 4 năm 2006. Đó là thời chưa có iPhone. Internet di động chủ yếu vẫn là GPRS, hoặc may lắm thì EDGE. Chẳng ai nói về Internet of Things. YouTube mới ra đời được hơn một năm, và gần như không ai biết tới nó — số người coi nó là nghiêm túc còn ít hơn nữa. Các ngôn ngữ như Kotlin, Swift, Go, Dart, Rust, Elm, Elixir và Clojure vẫn chưa xuất hiện. *Meet the Fockers* là bộ phim có doanh thu cao nhất tính tới thời điểm đó.

Yeah, that was a long time ago—especially in internet time. So, it’s time to take a new look at the old message of *Clean Code*. The essence of that message has not changed; but after a decade and a half our industry has—and remarkably so. After so much change, it would be irresponsible to just dust off the old manuscript and make a few simple adjustments. So, in light of that decade and a half of progress, learning, and change, I decided to significantly restate and expand the original message.

Đúng vậy, đó là chuyện đã lâu lắm rồi — nhất là nếu tính theo thời gian internet. Vậy nên đã đến lúc nhìn lại thông điệp cũ của *Clean Code* bằng con mắt mới. Cốt lõi của thông điệp đó không đổi; nhưng sau mười lăm năm thì ngành của chúng ta đã đổi — và đổi rất đáng

kể. Sau ngần ấy thay đổi, sẽ là vô trách nhiệm nếu chỉ phủ bụi bản thảo cũ rồi chỉnh sửa qua loa vài chỗ. Thế nên, trước mười lăm năm tiến bộ, học hỏi và đổi thay ấy, tôi quyết định trình bày lại và mở rộng đáng kể thông điệp ban đầu.

I have added many new chapters, new ideas, and new disciplines. I have added newer languages, and newer paradigms. I have also revamped the older chapters with a decade and a half of newer insights and knowledge.

Tôi đã thêm nhiều chương mới, ý tưởng mới và kỷ luật mới. Tôi đã bổ sung những ngôn ngữ mới hơn, những mô hình lập trình mới hơn. Tôi cũng đại tu các chương cũ bằng những hiểu biết và kiến thức tích lũy trong mười lăm năm qua.

I have added chapters on design, **architecture**, and even ethics. These were missing from the first edition, and I felt it necessary to repair that omission. In some cases, these chapters are heavily edited and abridged excerpts from my more recent books, *Clean Architecture* and *Clean **Craftsmanship*** ; others are entirely new.

Tôi đã thêm các chương về thiết kế, kiến trúc (architecture), và thậm chí cả đạo đức nghề nghiệp. Những phần này vắng mặt trong ấn bản đầu, và tôi thấy cần phải bù đắp thiếu sót đó. Trong một số trường hợp, các chương này là những trích đoạn đã được biên tập và rút gọn khá nhiều từ các cuốn sách gần đây hơn của tôi, *Clean Architecture* và *Clean Craftsmanship*; số còn lại là hoàn toàn mới.

In the original edition, I was reluctant to assert my ideas as anything more than one programmer's way of looking at things. Now, many years later, I'm a bit more confident than that. The ideas in that original edition have withstood the **test** of time and have quite a bit more staying power.

Ở ấn bản gốc, tôi khá dè dặt, không dám khẳng định ý tưởng của mình là gì hơn ngoài cách nhìn nhận sự việc của một lập trình viên. Giờ đây, nhiều năm sau, tôi đã tự tin hơn thế một chút. Những ý tưởng trong ấn bản đầu tiên ấy đã vượt qua được phép thử của thời gian và có sức sống bền bỉ hơn hẳn.

On the other hand, there are other very experienced programmers who have different ideas, and I thought it wise to include some of their contrary viewpoints. After all, I'm not the only one who has ideas about what it means for code to be clean. I like my ideas best, but I think all creditable ideas should be heard and considered.

Mặt khác, cũng có những lập trình viên rất dày dặn khác mang ý tưởng khác, và tôi nghĩ sẽ là khôn ngoan nếu đưa vào đây một số quan điểm trái chiều của họ. Suy cho cùng, tôi không phải người duy nhất có ý tưởng về việc thế nào là mã sạch. Tôi thích ý tưởng của mình nhất, nhưng tôi cho rằng mọi ý tưởng đáng tin cậy đều nên được lắng nghe và cân nhắc.

On yet another hand, in the process of reviewing the newer chapters months after they had been written, I was surprised to find that I had issues with some of the examples I had posed. Rather than erasing that surprise by further “cleaning” the examples, I added some Future Bob comments so that you could see my own reaction.

Ở một khía cạnh khác nữa, trong lúc rà lại các chương mới vài tháng sau khi viết xong, tôi ngạc nhiên khi thấy mình có vấn đề với chính vài ví dụ mình đã đưa ra. Thay vì xoá đi sự ngạc nhiên đó bằng cách “dọn dẹp” các ví dụ thêm nữa, tôi thêm vào vài comment của Future Bob để bạn có thể thấy chính phản ứng của tôi.

Why is all this cleanliness important? What benefits do these ideas about clean code bring? Why should we be worried about code cleanliness at all?

Tại sao tất cả sự sạch sẽ này lại quan trọng? Những ý tưởng về clean code mang lại lợi ích gì? Tại sao ta phải bận tâm tới chuyện mã nguồn sạch hay bẩn?

Our civilization depends upon us. Nothing happens in our society without computers being intimately involved in virtually every aspect of our lives. Nowadays we cannot microwave a hot dog, watch TV, make a phone call, drive to the store, wash our dishes or our clothes, buy anything, sell anything, or even tell the time of day without software being smack in the middle of it all.

Nền văn minh của chúng ta phụ thuộc vào chúng ta. Chẳng có chuyện gì xảy ra trong xã hội mà máy tính không dính líu mật thiết tới hầu như mọi khía cạnh đời sống. Ngày nay ta không thể quay lò vi sóng một chiếc xúc xích, xem TV, gọi điện thoại, lái xe ra cửa hàng, rửa bát, giặt quần áo, mua bất cứ thứ gì, bán bất cứ thứ gì, hay thậm chí xem giờ mà không có phần mềm nằm chình ịch ở giữa tất cả những việc đó.

You and I, we programmers, rule the world. Other people think they rule the world, but then they hand those rules to us, and we write the rules—the software—that executes in the machines that sit in the middle of the details of billions of people's lives.

Bạn và tôi, những lập trình viên, chúng ta cai trị thế giới. Người khác tưởng rằng họ mới là người cai trị thế giới, nhưng rồi họ trao những luật lệ ấy cho chúng ta, và chính chúng ta viết ra luật lệ — tức phần mềm — chạy trong những cỗ máy nằm giữa từng chi tiết đời sống của hàng tỷ con người.

And some of the software we wrote has caused two fully loaded passenger jets to slam into the ground at near the speed of sound, cars to accelerate out of control and crash, billions of dollars to be lost in seconds, rockets to explode, spacecraft to crash into Mars, and noxious fumes to be emitted by tens of thousands of cars. Just for starters.

Và một phần trong số phần mềm mà chúng ta viết đã khiến hai chiếc máy bay chở khách đầy người lao thẳng xuống đất ở tốc độ gần bằng tốc độ âm thanh, khiến ô tô tăng tốc mất kiểm soát rồi đâm, khiến hàng tỷ đô-la bốc hơi trong vài giây, khiến tên lửa nổ tung, khiến tàu vũ trụ đâm xuống Sao Hỏa, và khiến hàng chục nghìn chiếc xe thải ra khí độc. Và đó mới chỉ là mở màn.

And with such a burden of responsibility and failure sitting upon us, we have to ask ourselves this question: Are we professionals?

Và với gánh nặng trách nhiệm cùng những thất bại đè lên vai như thế, chúng ta buộc phải tự hỏi mình câu này: Chúng ta có phải là người chuyên nghiệp không?

A **professional** is someone who professes a set of standards, disciplines, and ethics. Do we? Do you? If so, can you recite them?

Người chuyên nghiệp là người tuyên xưng một tập hợp các chuẩn mực, kỷ luật và đạo đức nghề nghiệp. Chúng ta có không? Bạn có không? Nếu có, bạn đọc vanh vách chúng ra được chứ?

My goal for this book is to provide some standards of code cleanliness, some disciplines of code creation and maintenance, and the beginnings of an ethical framework to help us define a true profession. Because if we don't define and adopt that profession, others who are less able but more powerful—the politicians of the world—will define it for us and force it down our throats.

Mục tiêu của tôi với cuốn sách này là đưa ra một số chuẩn mực về sự sạch sẽ của mã nguồn, một số kỷ luật cho việc tạo ra và bảo trì mã, cùng những mầm mống đầu tiên của một khung đạo đức nghề nghiệp giúp chúng ta định nghĩa một nghề thực thụ. Bởi vì nếu chúng ta không tự định nghĩa và không tự nhận lấy cái nghề đó, thì những kẻ khác kém

năng lực hơn nhưng quyền lực hơn — đám chính trị gia trên đời — sẽ định nghĩa nó thay ta rồi nhét thẳng vào cổ họng chúng ta.

I hope there's still time for us to get there before that idea occurs to them.

Tôi hy vọng ta vẫn còn kịp thời gian để tới đích trước khi ý tưởng đó loé lên trong đầu họ.

A Note About Older Chapters — Đôi lời về các chương cũ

I have taken a great deal of liberty with the older chapters for which I was the sole author. In many cases they have been completely rewritten, or very substantially edited.

Tôi đã rất thoải mái trong việc can thiệp vào những chương cũ mà tôi là tác giả duy nhất. Nhiều chương đã được viết lại hoàn toàn, hoặc biên tập lại rất đáng kể.

Most of the authors of the chapters that I did not write have submitted second editions of their chapters to this effort.

Phần lớn tác giả của những chương không do tôi viết cũng đã gửi tới ấn bản thứ hai cho chương của họ để đóng góp vào lần tái bản này.

The Structure of This Book — Cấu trúc của cuốn sách

Possibly the greatest change between the first and second editions is the **structure** of the material. I have divided the book into four parts: Code, Design, Architecture, and Craftsmanship.

Thay đổi lớn nhất giữa ấn bản đầu và ấn bản thứ hai có lẽ nằm ở cấu trúc của nội dung. Tôi đã chia cuốn sách thành bốn phần: Code, Design, Architecture và Craftsmanship.

These parts are not independent of each other.

Các phần này không độc lập với nhau.

Part I : Code strongly depends upon Part II : Design . As you read Part I , you will find many references to design principles and design strategies that are described in Part II . I encourage you to treat Part II as a reference that you can consult while reading Part I . Then, when you are done with Part I , you should read Part II in its entirety.

Phần I: Code phụ thuộc rất nhiều vào Phần II: Design. Khi đọc Phần I, bạn sẽ thấy nhiều chỗ dẫn chiếu tới các nguyên tắc thiết kế và chiến lược thiết kế được mô tả trong Phần II. Tôi khuyến khích bạn coi Phần II như tài liệu tra cứu để giở ra xem trong lúc đọc Phần I. Rồi khi đọc xong Phần I, bạn nên đọc trọn vẹn Phần II.

Part III : Architecture is a set of deeply edited and abridged excerpts from my book Clean Architecture . Again, consider it a reference to assist in your understanding of Part I . However, it also stands alone as a simple introduction to higher-level architectural principles.

Phần III: Architecture là tập hợp các trích đoạn được biên tập và rút gọn rất kỹ từ cuốn Clean Architecture của tôi. Một lần nữa, hãy xem nó như tài liệu tra cứu hỗ trợ bạn hiểu Phần I. Tuy vậy, nó cũng đứng vững một mình như một dẫn nhập đơn giản vào các nguyên tắc kiến trúc ở tầng cao hơn.

Part IV : Craftsmanship is, once again, a set of deeply edited and abridged excerpts from my book Clean Craftsmanship — specifically the chapters that have to do with the ethics of software

development. It is included as a way to underscore that there is an ethical imperative to writing clean code.

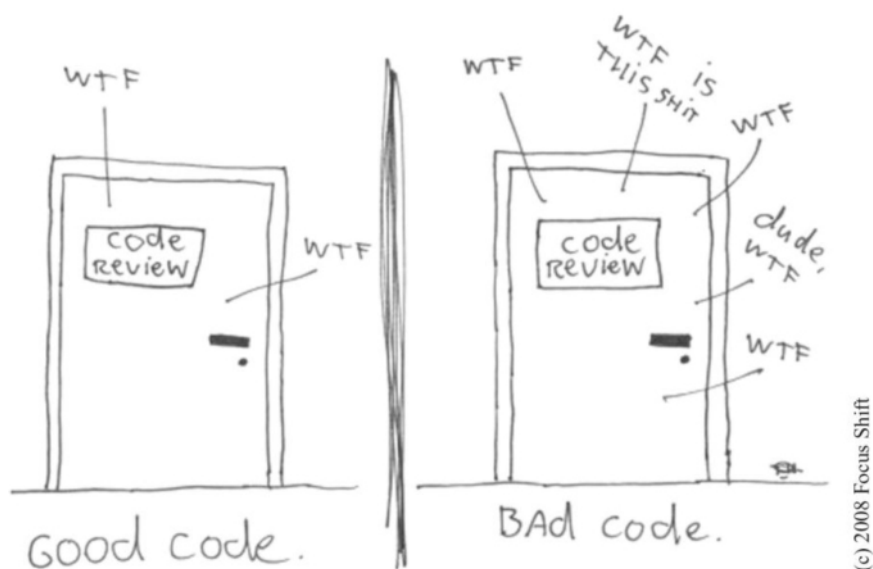
Phần IV: Craftsmanship, cũng lại là tập hợp các trích đoạn được biên tập và rút gọn rất kỹ từ cuốn *Clean Craftsmanship* của tôi — cụ thể là những chương bàn về đạo đức nghề nghiệp trong phát triển phần mềm. Nó được đưa vào đây như một cách nhấn mạnh rằng việc viết mã sạch mang một mệnh lệnh đạo đức.

Introduction (from Long Ago) — Lời giới thiệu (từ thuở xa xưa)

(This is the Introduction to the first edition. Edited and abridged.)

(Đây là Lời giới thiệu của ấn bản đầu tiên. Đã được biên tập và rút gọn.)

The ONLY VALID MEASUREMENT
OF CODE QUALITY: WTFs/MINUTE



Reproduced with the kind permission of Thom Holwerda (www.osnews.com/story/19266/WTFs_m).

Được tái sử dụng với sự cho phép hào phóng của Thom Holwerda (www.osnews.com/story/19266/WTFs_m).

Look at the drawing above. Which door represents your code? Which door represents your team or your company? Why are we in that room? Is this just a normal code review or have we found a stream of horrible problems shortly after going live?

Hãy nhìn bức tranh phía trên. Cánh cửa nào đại diện cho mã của bạn? Cánh cửa nào đại

diện cho đội của bạn hay công ty bạn? Tại sao chúng ta lại ở trong căn phòng đó? Đây chỉ là một buổi code review bình thường, hay chúng ta vừa phát hiện ra cả một chuỗi vấn đề kinh hoàng ngay sau khi hệ thống lên sóng?

Are we debugging in a panic, poring over code that we thought worked? Are customers leaving in droves and managers breathing down our necks? How can we make sure we wind up behind the right door when the going gets tough? The answer is: craftsmanship.

Chúng ta đang hoảng loạn debug, cắm mặt vào đoạn mã mà ta cứ tưởng là chạy tốt? Khách hàng đang bỏ đi hàng loạt còn quản lý thì thở phào sau gáy ta? Làm sao để chắc chắn rằng khi tình hình gay go, ta sẽ đứng sau đúng cánh cửa cần đứng? Câu trả lời là: tay nghề.

There are two parts to learning craftsmanship: knowledge and work. You must gain the knowledge of principles, patterns, practices, and heuristics that a craftsman knows, and you must also grind that knowledge into your fingers, eyes, and gut by working hard and practicing.

Học nghề gồm hai phần: kiến thức và làm việc. Bạn phải thu nạp kiến thức về các nguyên tắc, mẫu (pattern), thực hành và các kinh nghiệm thực hành (heuristic) mà một người thợ lành nghề nắm rõ, đồng thời phải mài thử kiến thức đó vào ngón tay, vào con mắt, vào bản năng của mình bằng cách làm việc cật lực và luyện tập.

I can teach you the physics of riding a bicycle. Indeed, the classical mathematics is relatively straightforward. Gravity, friction, angular momentum, center of mass, and so forth can be demonstrated with less than a page full of equations. Given those formulae, I could prove to you that bicycle riding is practical and give you all the knowledge you needed to make it work.

Tôi có thể dạy bạn vật lý của việc đi xe đạp. Thật ra, phần toán học cổ điển tương đối dễ hiểu. Trọng lực, ma sát, mô-men động lượng, khối tâm, v.v. đều có thể diễn giải trong chưa đầy một trang phương trình. Với những công thức ấy, tôi có thể chứng minh cho bạn thấy việc đi xe đạp là khả thi và trao cho bạn toàn bộ kiến thức cần thiết để làm được điều đó.

And you'd still fall down the first time you climbed on that bike.

Và bạn vẫn sẽ ngã sấp mặt ngay lần đầu leo lên cái xe đạp đó.

Coding is no different. We could write down all the “feel good” principles of clean code and then trust you to do the work (in other words, let you fall down when you get on the bike), but then what kind of teachers would that make us, and what kind of student would that make you?

Viết mã cũng chẳng khác gì. Chúng tôi có thể chép ra hết mọi nguyên tắc “nghe cho sướng tai” về clean code rồi phó mặc bạn tự làm phần việc còn lại (nói cách khác, cứ để bạn ngã khi leo lên xe đạp), nhưng khi đó chúng tôi là loại thầy gì, và bạn là loại trò gì?

No. That's not the way this book is going to work.

Không. Cuốn sách này sẽ không vận hành theo kiểu đó.

Learning to write clean code is hard work. It requires more than just the knowledge of principles and patterns. You must sweat over it. You must practice it yourself, and watch yourself fail. You must watch others practice it and fail. You must see them stumble and retrace their steps. You must see them agonize over decisions and see the price they pay for making those decisions the wrong way.

Học viết clean code là công việc nhọc nhằn. Nó đòi hỏi nhiều hơn là chỉ biết các nguyên tắc và mẫu thiết kế. Bạn phải đổ mồ hôi vì nó. Bạn phải tự mình luyện tập, và nhìn chính mình thất bại. Bạn phải nhìn người khác luyện tập và thất bại. Bạn phải thấy họ vấp ngã rồi lần ngược lại từng bước. Bạn phải thấy họ vật vã trước các quyết định và thấy cái giá họ phải trả khi quyết định sai.

Be prepared to work hard while reading this book. This is not a “feel good” book that you can read on an airplane and finish before you land. This book will make you work, and work hard . What kind of work will you be doing? You’ll be reading code—lots of code. And you will be challenged to think about what’s right about that code and what’s wrong with it. You’ll be asked to follow along as we (the authors) take modules apart and put them back together again. This will take time and effort; but we think it will be worth it.

Hãy sẵn sàng làm việc cật lực khi đọc cuốn sách này. Đây không phải loại sách “đọc cho sướng” mà bạn có thể vừa bay vừa đọc và gấp lại trước khi máy bay hạ cánh. Cuốn sách này sẽ bắt bạn làm việc, và làm việc *cật lực*. Việc gì? Bạn sẽ phải đọc mã — rất nhiều mã. Và bạn sẽ bị thách thức phải suy nghĩ xem đoạn mã đó có gì đúng và có gì sai. Bạn sẽ được yêu cầu theo sát chúng tôi (các tác giả) khi chúng tôi tháo rời các module ra rồi lắp lại. Việc này tốn thời gian và công sức; nhưng chúng tôi tin là nó đáng.

Chapter 1. Clean Code — Chương 1. Clean Code



“Clean’ isn’t a ding at anyone. Our notion of ‘unclean’ is a way of characterizing code that demands more effort from the average developer to understand or maintain.”

“Chữ ‘sạch’ không nhằm đá xoáy ai cả. Khái niệm ‘không sạch’ của chúng tôi chỉ là một cách mô tả thứ mã đòi hỏi lập trình viên trung bình phải bỏ ra nhiều công sức hơn để hiểu hoặc bảo trì.”

—Jeff Langr You are reading this book for two reasons. First, you are a programmer. Second, you want to be a better programmer. **Good.** We need better programmers.

—Jeff Langr Bạn đang đọc cuốn sách này vì hai lý do. Thứ nhất, bạn là lập trình viên. Thứ hai, bạn muốn trở thành một lập trình viên giỏi hơn. Tốt. Chúng ta đang cần những lập trình viên giỏi hơn.

This is a book about good programming. It is filled with code. We are going to look at code from every different direction. We'll look down at it from the top, up at it from the bottom, and through it from the inside out. By the time we are done, we're going to know a lot about code. What's more, we'll be able to tell the difference between good code and **bad code**. We'll know how to write good code. And we'll know how to transform bad code into good code.

Đây là một cuốn sách về lập trình cho tốt. Nó đầy ắp mã. Chúng ta sẽ soi mã từ đủ mọi hướng. Nhìn từ trên xuống, nhìn từ dưới lên, và xuyên qua nó từ trong ra ngoài. Đến lúc khép sách lại, chúng ta sẽ biết rất nhiều về mã. Hơn thế nữa, chúng ta sẽ phân biệt được đâu là mã tốt, đâu là mã tệ. Chúng ta sẽ biết cách viết mã tốt. Và chúng ta sẽ biết cách biến mã tệ thành mã tốt.

There Will Be Code — Sẽ luôn có mã

One might argue that a book about code is somehow behind the times—that code is no longer the issue; that we should be concerned about AI, large language models (LLMs), and requirements instead. Indeed, some have suggested that we are close to the end of code. ¹ That soon, all code will be generated instead of written. That programmers simply won't be needed, because businesspeople will generate programs from specifications—or prompts.

Có người sẽ cãi rằng một cuốn sách về mã thì đã lỗi thời rồi — rằng mã không còn là vấn đề nữa; rằng lẽ ra chúng ta phải bận tâm tới AI, tới các mô hình ngôn ngữ lớn (LLM) và tới yêu cầu phần mềm mới đúng. Quả thật, đã có người bảo rằng chúng ta sắp đi tới ngày tàn của mã. ¹ Rằng chẳng bao lâu nữa, mọi dòng mã sẽ được sinh ra thay vì được viết ra. Rằng lập trình viên đơn giản là sẽ không còn cần thiết, bởi giới kinh doanh sẽ tự sinh ra chương trình từ đặc tả — hoặc từ prompt.

¹ It is hilarious that I wrote that line in 2008, and here it is, 2024. Some things never change.

¹ Buồn cười ở chỗ tôi viết dòng đó vào năm 2008, mà giờ đã là 2024. Có những thứ chẳng bao giờ thay đổi.

Nonsense! We will never be rid of code, because code represents the details of the requirements. At some level, those details cannot be ignored or abstracted; they have to be specified. And specifying requirements in such detail that a machine can execute them is programming. Such a specification is code.

Vớ vẩn! Chúng ta sẽ chẳng bao giờ thoát khỏi mã, bởi mã chính là biểu diễn của các chi tiết trong yêu cầu. Ở một mức nào đó, những chi tiết ấy không thể bị bỏ qua hay trừu tượng hoá đi được; chúng phải được đặc tả. Và việc đặc tả yêu cầu chi tiết đến mức một cỗ máy có thể thi hành được chính là lập trình. Một đặc tả như thế chính là mã.

I expect that the **level of abstraction** of our languages will continue to increase. I also expect that the number of domain-specific languages and powerful AIs will continue to grow. This will be a good thing. But it will not eliminate code. Indeed, all the specifications written in these higher-level and domain-specific languages, and all the prompts written to guide a really good AI, will be code! It will still need to be rigorous, accurate, and so formal and detailed that a machine can understand and execute it.

Tôi cho rằng mức trừu tượng của các ngôn ngữ sẽ còn tiếp tục tăng lên. Tôi cũng cho rằng số lượng ngôn ngữ chuyên biệt miền và số lượng AI mạnh mẽ sẽ còn tiếp tục nhiều lên. Đó là chuyện tốt. Nhưng nó sẽ không xoá sổ mã. Thật ra, mọi đặc tả viết bằng những ngôn ngữ mức cao và chuyên biệt miền đó, cùng mọi prompt viết ra để dẫn dắt một con AI thật

sự giỏi, tất thấy đều sẽ là mã! Chúng vẫn sẽ phải chặt chẽ, chính xác, và hình thức, chi tiết đến mức một cỗ máy có thể hiểu và thi hành được.

The folks who think that code will one day disappear are like “mathematicians”² who hope one day to discover a mathematics that does not have to be formal. They hope one day to discover a way to create machines that can do what they want rather than what they say. These machines will have to be able to understand us so well that they can translate vaguely specified needs into perfectly executing programs that precisely meet those needs.

Những người nghĩ rằng một ngày nào đó mã sẽ biến mất cũng giống như các “nhà toán học”² hy vọng một ngày nào đó khám phá ra một thứ toán học không cần hình thức hoá. Họ hy vọng một ngày nào đó tìm ra cách chế tạo những cỗ máy làm đúng điều họ muốn thay vì đúng điều họ nói. Những cỗ máy ấy sẽ phải hiểu chúng ta giỏi tới mức có thể dịch những nhu cầu mơ hồ thành các chương trình chạy hoàn hảo, đáp ứng chính xác những nhu cầu đó.

² In quotes because no real mathematician would hope for this.

² Để trong ngoặc kép vì không nhà toán học thực thụ nào lại hy vọng điều này.

This will never happen. Not even humans, with all their intuition and creativity, have been able to create successful systems from the vague feelings of their customers. Indeed, if the **discipline** of requirements specification has taught us anything, it is that well-specified requirements are as formal as code and can act as executable tests of that code!

Chuyện đó sẽ không bao giờ xảy ra. Ngay cả con người, với tất cả trực giác và óc sáng tạo, cũng chưa từng dựng nên được hệ thống thành công nào chỉ từ những cảm giác mơ hồ của khách hàng. Thật vậy, nếu bộ môn đặc tả yêu cầu có dạy cho chúng ta điều gì, thì đó là: một yêu cầu được đặc tả tử tế thì cũng hình thức chẳng kém gì mã, và có thể đóng vai trò như những bài test thi hành được cho chính mã đó!

Remember that code is really the language in which we ultimately express the requirements. We may create languages that are closer to the requirements. We may create tools that help us parse and assemble those requirements into formal structures. But we will never eliminate the necessary precision and formality—so there will always be code.

Hãy nhớ rằng mã thực chất là ngôn ngữ mà rốt cuộc chúng ta dùng để diễn đạt yêu cầu. Chúng ta có thể tạo ra những ngôn ngữ gần với yêu cầu hơn. Chúng ta có thể tạo ra công cụ giúp phân tích và lắp ghép các yêu cầu ấy thành những cấu trúc hình thức. Nhưng chúng ta sẽ không bao giờ loại bỏ được sự chính xác và tính hình thức bắt buộc — nên sẽ luôn luôn có mã.

Bad Code — Mã tệ

Long ago I read the preface to Kent Beck’s book **Implementation** Patterns.³ He said, “. . . this book is based on a rather fragile premise: that good code matters. . . .” A fragile premise? I disagree! I think that premise is one of the most robust, supported, and overloaded of all the premises in our **craft** (and I think Kent knows it). We know good code matters because we’ve had to deal for so long with its lack.

Rất lâu trước đây tôi có đọc lời tựa cuốn Implementation Patterns của Kent Beck.³ Ông viết: “. . . cuốn sách này dựa trên một tiền đề khá mong manh: rằng mã tốt là điều quan trọng. . . .” Một tiền đề mong manh ư? Tôi không đồng ý! Tôi cho rằng đó là một trong những tiền đề vững chãi nhất, được chống lưng nhiều nhất và cũng bị nhồi nhét nhiều

nhất trong cái nghề thủ công của chúng ta (và tôi nghĩ Kent thừa biết điều đó). Chúng ta biết mã tốt là quan trọng, bởi chúng ta đã phải sống quá lâu với sự thiếu vắng của nó.

3 [IMP].

3 [IMP].



I know of one company who, in the late '80s, wrote a killer app. It was very popular, and lots of professionals bought and used it. But then the release cycles began to stretch. Bugs were not repaired from one release to the next. Load times grew and crashes increased. I remember the day I shut the product down in frustration and never used it again. The company went out of business a short time after that.

Tôi biết một công ty, cuối thập niên 80, đã viết ra một ứng dụng đình đám. Nó rất được ưa chuộng, và vô số dân chuyên nghiệp đã mua và dùng nó. Nhưng rồi chu kỳ phát hành bắt đầu giãn dần ra. Lỗi từ bản này không được sửa cho tới bản kế tiếp. Thời gian tải tăng lên và số lần treo cũng nhiều thêm. Tôi vẫn nhớ cái ngày mình bức bối tắt sản phẩm đó đi và không bao giờ dùng lại nữa. Ít lâu sau, công ty ấy phá sản.

Two decades later I met one of the early employees of that company and asked him what had happened. The answer confirmed my fears. They had rushed the product to market and had made a huge **mess** in the code. As they added more and more features, the code got worse and worse until they simply could not manage it any longer. It was the bad code that brought the company down .

Hai thập kỷ sau, tôi gặp một trong những nhân viên đời đầu của công ty đó và hỏi rốt cuộc đã xảy ra chuyện gì. Câu trả lời xác nhận đúng nỗi lo của tôi. Họ đã hối hả đẩy sản phẩm ra thị trường và biến mã thành một mớ hỗn độn khổng lồ. Càng thêm nhiều tính năng, mã càng tệ đi, tệ mãi cho tới khi họ đơn giản là không còn quản nổi nữa. Chính mã tệ đã hạ gục cả công ty.

Have you ever been significantly impeded by bad code? If you are a programmer of any experience, you've felt this impediment many times. Indeed, we have a name for it. We call it **wading** and **slogging** . We wade through bad code. We slog through a morass of tangled brambles and hidden pitfalls. We struggle to find our way, hoping for some hint, some clue, of what is going on; but all we see is more and more senseless code.

Bạn từng bị mã tệ cản chân đến mức nghiêm trọng bao giờ chưa? Nếu bạn là lập trình viên có chút kinh nghiệm nào đó, hẳn bạn đã nếm trải sự cản trở này nhiều lần rồi. Thật ra, chúng ta còn có hẳn tên gọi cho nó. Chúng ta gọi đó là lội bì bõm và lê lét. Chúng ta lội bì bõm trong mã tệ. Chúng ta lê lét qua một đầm lầy đầy gai góc rối bời và những hố sụt giấu mặt. Chúng ta vật lộn tìm đường, mong vớ được chút gợi ý, chút manh mối nào đó về chuyện đang diễn ra; nhưng tất cả những gì thấy được chỉ là mã vô nghĩa, ngày một nhiều hơn.

Of course you have been impeded by bad code. So then— why did you write it?

Dĩ nhiên là bạn đã từng bị mã tệ cản chân. Vậy thì — tại sao chính bạn lại viết ra nó?

Were you trying to go fast? Were you in a rush? Probably so. Perhaps you felt that you didn't have time to do a good job; that your boss would be angry with you if you took the time to clean up your code. Perhaps you were just tired of working on this program and wanted it to be over. Or maybe you looked at the backlog of other stuff that you had promised to get done and realized that you needed to slam this **module** together so that you could move on to the next. We've all done it.

Bạn đang cố đi cho nhanh chăng? Bạn đang vội chăng? Chắc là vậy. Có lẽ bạn thấy mình không có thời gian làm cho tử tế; rằng sếp sẽ nổi cáu nếu bạn bỏ thời gian ra dọn dẹp mã. Có lẽ bạn chỉ đơn giản là chán ngấy chương trình này rồi và chỉ mong nó xong cho rảnh nợ. Hoặc có thể bạn ngó vào đồng việc còn tồn mà mình đã trót hứa làm xong, rồi nhận ra rằng phải quăng đại module này lại với nhau cho xong để còn chuyển sang cái tiếp theo. Ai trong chúng ta cũng từng làm thế.

We've all looked at the mess we just made and chosen to leave it for another day. We've all felt the relief of seeing our messy program work and deciding that a working mess is better than nothing. We've all said we'd go back and clean it up later. Of course, in those days we didn't know—or didn't want to know—that later equals never. 4

Ai trong chúng ta cũng từng nhìn mớ hỗn độn mình vừa bày ra và chọn để đó mai tính. Ai cũng từng thở phào khi thấy chương trình bày hầy của mình chạy được và tự nhủ một mớ hỗn độn chạy được vẫn hơn là không có gì. Ai cũng từng nói rằng sẽ quay lại dọn sau. Dĩ nhiên, hồi ấy chúng ta chưa biết — hoặc chưa muốn biết — rằng để sau tức là không bao giờ. 4

4 LeBlanc's law.

4 Định luật LeBlanc.

Attitude — Thái độ

Have you ever waded through a mess so grave that it took weeks to do what should have taken hours? Have you seen what should have been a one-line change, made instead in hundreds of different modules? These symptoms are all too common.

Bạn từng lội qua một mớ hỗn độn trầm trọng đến mức phải mất hàng tuần cho một việc đáng lẽ chỉ tốn vài giờ chưa? Bạn từng chứng kiến một thay đổi đáng lẽ chỉ một dòng mà rốt cuộc phải sửa ở hàng trăm module khác nhau chưa? Những triệu chứng này quá phổ biến.

Why does this happen to code? Why does good code **rot** so quickly into bad code? We have lots of explanations for it. We complain that the requirements changed in ways that thwart the original design. We bemoan the schedules that were too tight to do things right. We blather about stupid managers and intolerant customers and useless marketing types and telephone sanitizers. But the fault, dear Dilbert, is not in our stars, but in ourselves. We are unprofessional.

Vì sao chuyện này lại xảy ra với mã? Vì sao mã tốt lại mục ruỗng nhanh đến thế để thành mã tệ? Chúng ta có cả tá lời giải thích. Chúng ta than yêu cầu thay đổi theo kiểu phá nát thiết kế ban đầu. Chúng ta rên rỉ về những lịch trình quá gấp gáp nên chẳng thể làm cho ra hồn. Chúng ta lải nhải về những ông quản lý ngu ngốc, những khách hàng khó tính, mấy tay marketing vô tích sự và cả những anh chàng lau chùi điện thoại nữa. Nhưng lỗi

lầm, Dilbert thân mến ạ, không nằm ở các vì sao, mà nằm ở chính chúng ta. Chúng ta thiếu chuyên nghiệp.

This may be a bitter pill to swallow. How could this mess be our fault? What about the requirements? What about the schedule? What about the stupid managers and the useless marketing types? Don't they bear some of the blame?

Đây có thể là viên thuốc đắng khó nuốt. Sao cái mớ hỗn độn này lại là lỗi của chúng ta được? Thế còn yêu cầu thì sao? Còn lịch trình thì sao? Còn mấy ông quản lý ngu ngốc và mấy tay marketing vô tích sự thì sao? Chẳng lẽ họ không phải gánh chút trách nhiệm nào?

No. The managers and marketers look to us for the information they need to make promises and commitments; and even when they don't look to us, we should not be shy about telling them what we think. The users look to us to validate the way the requirements will fit into the system. The project managers look to us to help work out the schedule. We are deeply complicit in the planning of the project and share a great deal of the responsibility for any failures, especially if those failures have to do with bad code!

Không. Cánh quản lý và marketing trông vào chúng ta để có thông tin mà đưa ra lời hứa và cam kết; và ngay cả khi họ không hỏi tới chúng ta, chúng ta cũng không nên rụt rè trong việc nói cho họ biết mình nghĩ gì. Người dùng trông vào chúng ta để kiểm chứng xem các yêu cầu sẽ khớp vào hệ thống ra sao. Các quản lý dự án trông vào chúng ta để cùng dựng nên lịch trình. Chúng ta dính líu rất sâu vào việc lập kế hoạch dự án và phải chia sẻ phần lớn trách nhiệm cho mọi thất bại, nhất là khi những thất bại ấy dính dáng tới mã tệ!

"But wait!" you say. "If I don't do what my manager says, I'll be fired." Probably not. Most managers want the truth, even when they don't act like it. Most managers want good code, even when they are obsessing about the schedule. They may defend the schedule and requirements with passion, but that's their job. It's your job to defend the code with equal passion.

"Nhưng khoan đã!" bạn nói. "Nếu tôi không làm theo lời sếp, tôi sẽ bị đuổi việc." Chắc là không đâu. Đa số quản lý muốn nghe sự thật, kể cả khi họ chẳng cư xử như vậy. Đa số quản lý muốn có mã tốt, kể cả khi họ đang ám ảnh về lịch trình. Họ có thể hăng máu bảo vệ lịch trình và yêu cầu, nhưng đó là việc của họ. Còn việc của bạn là bảo vệ mã với sự hăng máu ngang ngửa.

To drive this point home, what if you were a doctor and had a patient who demanded that you stop all the silly handwashing in preparation for surgery because it was taking too much time? 5 Clearly the patient is the boss; and yet the doctor should absolutely refuse to comply. Why? Because the doctor knows more than the patient about the risks of disease and infection. It would be unprofessional (never mind criminal) for the doctor to comply with the patient.

Để đóng đinh luận điểm này, hãy thử tưởng tượng bạn là bác sĩ và có một bệnh nhân đòi bạn dẹp hết cái trò rửa tay ngớ ngẩn trước khi phẫu thuật vì nó tốn quá nhiều thời gian? 5 Rõ ràng bệnh nhân mới là người trả tiền; ấy thế mà bác sĩ nhất định phải từ chối. Vì sao? Bởi bác sĩ hiểu về nguy cơ bệnh tật và nhiễm trùng hơn bệnh nhân. Sẽ là thiếu chuyên nghiệp (chưa nói tới phạm pháp) nếu bác sĩ chiều theo bệnh nhân.

5 When handwashing was first recommended to physicians by Ignaz Semmelweis in 1847, it was rejected on the basis that doctors were too busy and wouldn't have time to wash their hands between patient visits.

5 Khi Ignaz Semmelweis lần đầu khuyến nghị các thầy thuốc rửa tay vào năm 1847, đề xuất này bị bác bỏ với lý do bác sĩ quá bận, không có thời gian rửa tay giữa các lần thăm khám.

So too it is unprofessional for programmers to bend to the will of managers who don't understand the risks of making messes.

Cũng vậy, sẽ là thiếu chuyên nghiệp nếu lập trình viên uốn mình theo ý muốn của những nhà quản lý không hiểu rủi ro của việc bày ra các mớ hỗn độn.

The Primal Conundrum — Nghịch lý nguyên thủy

Programmers face a conundrum of basic values. All developers with more than a few years of experience know that messes slow them down. And yet all developers feel the pressure to make messes in order to meet deadlines. In short, they don't take the time to go fast!

Lập trình viên đối mặt với một nghịch lý về những giá trị nền tảng. Mọi lập trình viên có trên vài năm kinh nghiệm đều biết rằng các mớ hỗn độn làm họ chậm lại. Ấy vậy mà mọi lập trình viên đều cảm thấy áp lực phải bày ra mớ hỗn độn để kịp hạn chót. Nói ngắn gọn, họ không dành thời gian để đi cho nhanh!

True professionals know that the second part of the conundrum is wrong. You will not make the deadline by making the mess. Indeed, the mess will slow you down. It will also slow down everyone else who must deal with it. And that slowness will build and compound and will force you to miss deadline after deadline.

Người chuyên nghiệp thực thụ biết rằng vế thứ hai của nghịch lý này là sai. Bạn sẽ không kịp hạn chót nhờ bày ra mớ hỗn độn đâu. Ngược lại, mớ hỗn độn sẽ làm bạn chậm lại. Nó cũng làm chậm mọi người khác phải đụng tới nó. Và sự chậm chạp ấy sẽ chồng chất, cộng dồn, ép bạn trượt hết hạn chót này tới hạn chót khác.

The only way to make the deadlines—the only way to go fast—is to keep the code clean at all times.

Cách duy nhất để kịp hạn chót — cách duy nhất để đi nhanh — là giữ cho mã luôn sạch, mọi lúc.

The only way to go fast is to go well.

Cách duy nhất để đi nhanh là đi cho tử tế.

The Art of Clean Code? — Nghệ thuật viết clean code?

Let's say you believe that **messy code** is a significant impediment. Let's say that you accept that the only way to go fast is to keep your code clean. Then, you must ask yourself: "How do I write **clean code**?" It's no good trying to write clean code if you don't know what it means for code to be clean!

Giả sử bạn tin rằng mã bầy hầy là một trở ngại đáng kể. Giả sử bạn chấp nhận rằng cách duy nhất để đi nhanh là giữ mã sạch. Vậy thì bạn phải tự hỏi: "Làm sao để viết clean code?" Cố viết clean code mà không biết thế nào là mã sạch thì cũng bằng thừa!

Writing clean code requires the disciplined use of myriad little techniques applied through a painstakingly acquired sense of cleanliness that helps us transform bad code into clean code.

Viết clean code đòi hỏi vận dụng một cách có kỷ luật (discipline) vô số kỹ thuật nho nhỏ, thông qua một cảm quan về sự sạch sẽ phải khổ luyện mới có được — cảm quan giúp ta biến mã tẻ thành clean code.

And it is always a transformation. Nobody writes clean code. We all write messy code. Only those of us who take the extra step to clean it will produce clean code. This follows Kent Beck's law, which you will see over and over in this book:

Và đó luôn luôn là một sự biến đổi. Chẳng ai viết ra clean code ngay được cả. Tất cả chúng ta đều viết ra mã bầy hầy. Chỉ những ai chịu bỏ thêm công dọn dẹp mới cho ra clean code. Điều này tuân theo định luật của Kent Beck, thứ bạn sẽ gặp đi gặp lại trong cuốn sách này:

First, make it work. Then, make it right.

Trước hết, làm cho nó chạy. Sau đó, làm cho nó đúng.

What Is Clean Code? — Clean code là gì?

There are probably as many definitions as there are programmers. So I asked some very well-known and deeply experienced programmers what they thought. I have summarized their responses below.

Có lẽ có bao nhiêu lập trình viên thì có bấy nhiêu định nghĩa. Nên tôi đã đi hỏi vài lập trình viên rất nổi tiếng và cực kỳ dày dặn xem họ nghĩ gì. Tôi tóm lược câu trả lời của họ dưới đây.



“Clean code does one thing well.” —Bjarne Stroustrup, inventor of C++ and author of The C++ Programming Language

“Clean code làm tốt một việc.” —Bjarne Stroustrup, cha đẻ của C++ và tác giả cuốn The C++ Programming Language

This is an old and well-worn maxim. In the '70s, we used to say that a module should do one thing, do it well, and do it only. But what does “one thing” mean? Back in the late '80s, I was the author of a 3,000-line C **function** named `gi`. The name stood for graphic interpreter. If someone had come to me back then and warned me that my long, long function did more than one thing, I would have replied, “No, it interprets graphics.”

Đây là một châm ngôn xưa cũ và đã mòn vẹt. Hồi thập niên 70, chúng tôi hay nói rằng một module nên làm một việc, làm cho tốt, và chỉ làm mỗi việc đó. Nhưng “một việc” nghĩa là gì? Hồi cuối thập niên 80, tôi là tác giả của một hàm C dài 3.000 dòng tên là `gi`. Cái tên đó là viết tắt của graphic interpreter. Nếu hồi ấy có ai tới cảnh báo tôi rằng cái hàm dài dằng dặc kia làm nhiều hơn một việc, hẳn tôi đã đáp: “Không đâu, nó thông dịch đồ hoạ mà.”

The problem with the “one thing” rule is that “one thing” is entirely subjective. But I think I know a way to make it objective. We'll talk about that later in this book. And once I describe it to you, I think

you'll be forced to agree that it is close to, if not entirely, objective. You may not like it; but I wager you'll find no way to substantially disagree.

Vấn đề của quy tắc “một việc” là ở chỗ “một việc” hoàn toàn mang tính chủ quan. Nhưng tôi nghĩ mình biết một cách để làm cho nó trở nên khách quan. Chúng ta sẽ bàn chuyện đó ở phần sau của cuốn sách. Và một khi tôi mô tả xong cho bạn, tôi nghĩ bạn sẽ buộc phải công nhận rằng nó gần như, nếu không muốn nói là hoàn toàn, khách quan. Bạn có thể không thích nó; nhưng tôi cá là bạn sẽ chẳng tìm ra cách nào để phản bác một cách đáng kể.



“Clean code reads like well-written prose.” —Grady Booch, famed author of many software engineering books

“Clean code đọc như một áng văn xuôi viết khéo.” —Grady Booch, tác giả lừng danh của nhiều cuốn sách về công nghệ phần mềm

Now that's one hell of an **assertion**. When is the last time you encountered code that read like “well-written prose”? And yet, as you read on in this book, I'll show you a way to achieve, or at least approach, that very goal.

Chà, đó quả là một khẳng định ghê gớm. Lần cuối bạn gặp một đoạn mã đọc như “áng văn xuôi viết khéo” là khi nào? Ấy vậy mà, khi đọc tiếp cuốn sách này, tôi sẽ chỉ cho bạn một con đường để đạt tới, hoặc ít nhất là tiệm cận, chính cái đích ấy.

Oh, don't get me wrong. Your code is not going to read like a Hemmingway novel. But it is entirely possible to construct code that reads so nicely that even nonprogrammers can make some sense of it.

Ồ, đừng hiểu lầm tôi. Mã của bạn sẽ không đọc như tiểu thuyết của Hemmingway đâu. Nhưng hoàn toàn có thể dựng nên thứ mã đọc mượt mà tới mức ngay cả người không lập trình cũng hiểu được đôi phần về nó.

6 By some definition of the words some and sense.

6 Theo một định nghĩa nào đó của các từ đôi phần và hiểu.



“Clean code always looks like it was written by someone who cares.” —Michael Feathers, author of *Working Effectively with Legacy Code*

“Clean code luôn trông như thể được viết bởi một người có quan tâm.” —Michael Feathers, tác giả cuốn *Working Effectively with Legacy Code*

The words that Michael left out of that wonderful quotation were: “about you.”

Mấy chữ mà Michael bỏ lửng trong câu nói tuyệt vời ấy là: “tới bạn.”

The reason that a programmer will go to the effort of cleaning their code is that the programmer cares about the next person to come along. They care that their code will not mislead, misdirect, or befuddle those who must maintain that code. The word that best defines cleanliness is care .

Lý do khiến một lập trình viên chịu bỏ công dọn dẹp mã của mình là vì họ quan tâm tới người kế tiếp sẽ ghé qua. Họ bận tâm rằng mã của mình sẽ không đánh lừa, không dẫn sai đường, không làm rối trí những người phải bảo trì nó. Từ định nghĩa chuẩn xác nhất cho sự sạch sẽ chính là quan tâm.

The implication of that definition is that code that has not been cleaned was written and released carelessly.

Hệ quả của định nghĩa đó là: thứ mã chưa được dọn dẹp tức là đã được viết ra và phát hành một cách vô tâm.



“You know you are working on clean code when each routine you read turns out to be pretty much what you expected.” —Ward Cunningham, inventor of Wiki, inventor of Fit, co-inventor of Extreme Programming; motive force behind Design Patterns; Smalltalk and OO thought leader; the godfather of all those who care about code

“Bạn biết mình đang làm việc với clean code khi mỗi routine bạn đọc hoá ra gần như đúng y những gì bạn trông đợi.” —Ward Cunningham, cha đẻ của Wiki, cha đẻ của Fit, đồng sáng lập Extreme Programming; động lực đứng sau Design Patterns; nhà tư tưởng dẫn dắt về Smalltalk và OO; bậc bố già của tất cả những ai quan tâm tới mã

Imagine reading code and simply agreeing with it. As you scroll down, you nod in acceptance and affirmation. The act of reading the code is as fluid as the laminar flow of air over an airplane wing. There is no turbulence, no clever tricks, no jarring non sequiturs. You simply read, and agree, from top to bottom.

Hãy hình dung cảnh bạn đọc mã và cứ thế gật gù đồng tình với nó. Cuộn xuống tới đâu, bạn gật đầu chấp nhận và tán thành tới đó. Việc đọc mã trôi chảy như dòng khí chảy tầng lướt trên cánh máy bay. Không có nhiễu loạn, không có mảnh khoé thông minh vặt, không có những đoạn chẳng ăn nhập gì làm bạn khựng lại. Bạn chỉ đơn giản là đọc, và đồng tình, từ trên xuống dưới.

This may seem like an impossible utopian dream; but certainly it is a dream to strive for. In this book, I will show you ways to approach that dream.

Nghe có vẻ như một giấc mơ không tưởng bất khả thi; nhưng chắc chắn đó là giấc mơ đáng để theo đuổi. Trong cuốn sách này, tôi sẽ chỉ cho bạn những cách để tiến gần tới giấc mơ ấy.



“Clean code fits in your head. A function fits on a single screen, involves no more than a handful of moving parts, and has good names. It’s understandable by peers years after it was written. It minimizes surprises, composes well, and is easy to delete.” —Mark Seeman, author of Code That Fits in Your Head, popular blogger on software **craftsmanship** and functional programming

“Clean code vừa vặn trong đầu bạn. Một hàm gọn trong một màn hình, dính dáng tới không quá vài ba bộ phận chuyển động, và có tên gọi tốt. Đồng nghiệp vẫn hiểu được nó nhiều năm sau khi nó được viết ra. Nó giảm thiểu bất ngờ, kết hợp tốt với nhau, và dễ xoá bỏ.”
—Mark Seeman, tác giả cuốn Code That Fits in Your Head, blogger nổi tiếng về tay nghề lập trình và lập trình hàm

That kind of says it all, doesn’t it? In some ways, that quotation is a summary of this book.

Câu đó gần như nói lên tất cả, phải không? Ở một khía cạnh nào đó, trích dẫn ấy chính là bản tóm tắt của cuốn sách này.

Putting All This Together — Ghép tất cả lại với nhau

Clean code will read like well-written prose because it has been carefully partitioned into functions and modules the way prose is partitioned into sentences and paragraphs, each dealing with one thing, one topic, one irreducible concept that flows from one to the next to the next to create the expected whole.

Clean code sẽ đọc như một áng văn xuôi viết khéo, bởi nó đã được chia tách cẩn thận thành các hàm và module đúng theo cách văn xuôi được chia thành câu và đoạn — mỗi phần lo một việc, một chủ đề, một khái niệm không thể nhỏ hơn, và các phần ấy chảy từ cái này sang cái kia rồi cái kia nữa để tạo nên tổng thể như ta trông đợi.

Why Should We Be Clean? — Vì sao chúng ta phải sạch?

Look around you. How many computers are running within 10 feet of you? Do you have a smart phone, a smart watch, smart earbuds, a smart car key? Are you wearing a smart medical device?

Hãy nhìn quanh bạn. Có bao nhiêu máy tính đang chạy trong bán kính ba mét quanh bạn? Bạn có điện thoại thông minh, đồng hồ thông minh, tai nghe thông minh, chìa khoá xe thông minh không? Bạn có đang đeo thiết bị y tế thông minh nào không?

Look around your kitchen. How many computers are in the appliances? Do you have a smart microwave oven, a smart regular oven, a smart stove, a smart blender, a smart refrigerator?

Hãy nhìn quanh gian bếp nhà bạn. Có bao nhiêu máy tính nằm trong đám đồ gia dụng? Bạn có lò vi sóng thông minh, lò nướng thông minh, bếp thông minh, máy xay thông minh, tủ lạnh thông minh không?

Is your TV smart? Is your thermostat smart? Do you have a smart security system or a smart sound system?

TV nhà bạn có thông minh không? Bộ điều nhiệt có thông minh không? Bạn có hệ thống an ninh thông minh hay dàn âm thanh thông minh không?

Do you own a car? How many lines of code are running in that car? Does it have a GPS? Does your phone? Can you even drive to the store without using them?

Bạn có xe hơi không? Có bao nhiêu dòng mã đang chạy trong chiếc xe đó? Nó có GPS không? Điện thoại bạn có không? Bạn có thể lái xe ra cửa hàng mà không cần tới chúng không?

Think about this. Think hard about it. Nearly everything we do, in this modern world, is mediated by software. You can't cook a hot dog without software. You can't watch TV. You can't make a phone call. You can't drive to the store. You can't buy anything. You can't sell anything. No law can be passed. No law can be enforced. No insurance claim can be filed. Without software being smack-dab in the middle of it all.

Hãy nghĩ về điều này. Nghĩ cho thật kỹ. Gần như mọi việc chúng ta làm trong thế giới hiện đại này đều đi qua trung gian là phần mềm. Bạn không thể nướng một cái xúc xích mà không có phần mềm. Bạn không thể xem TV. Bạn không thể gọi điện thoại. Bạn không thể lái xe ra cửa hàng. Bạn không thể mua bất cứ thứ gì. Bạn không thể bán bất cứ thứ gì.

Không đạo luật nào được thông qua. Không đạo luật nào được thực thi. Không yêu cầu bồi thường bảo hiểm nào được nộp. Nếu không có phần mềm nắm chình ình ngay giữa tất cả những chuyện đó.

As I told you in the introduction, very little happens in our society without the involvement of software.

Như tôi đã nói với bạn ở phần mở đầu, rất ít chuyện trong xã hội chúng ta diễn ra được mà không có sự tham gia của phần mềm.

You and I rule the world.

Bạn và tôi thống trị thế giới này.

Others think they rule the world; but then they hand the rules to us and we write the actual rules that run in the machines that govern everything.

Những người khác tưởng rằng họ thống trị thế giới; nhưng rồi họ trao các quy tắc cho chúng ta, và chính chúng ta mới viết ra những quy tắc thật sự chạy trong các cỗ máy đang điều hành mọi thứ.

Our civilization depends upon us; and that dependence is critical. Without us, civilization would grind to a halt overnight. No one is prepared to go back to using index cards to run everything.

Nền văn minh của chúng ta phụ thuộc vào chúng ta; và sự phụ thuộc đó là sống còn. Không có chúng ta, nền văn minh sẽ đứng khựng lại chỉ sau một đêm. Chẳng ai sẵn sàng quay về dùng phiếu bìa để vận hành mọi thứ nữa đâu.

Are we moral, and **professional**, and responsible enough to bear civilization upon our shoulders? Or are we on a course to let everyone down? Are you and I going to be responsible for a major catastrophe?

Chúng ta có đủ đạo đức, đủ chuyên nghiệp và đủ trách nhiệm để gánh cả nền văn minh trên vai không? Hay chúng ta đang trên đà làm tất cả mọi người thất vọng? Liệu bạn và tôi có sẽ phải chịu trách nhiệm cho một thảm hoạ lớn?

We've seen hints of such catastrophes in the recent past. We've seen software drive two jet airliners into the ground at the speed of sound, killing 346 people. We've seen software errors kill dozens, and injure hundreds, when their cars' computers lost their minds and accelerated out of control. We saw Knight Capital lose \$450M in 45 minutes due to a software error. We saw Healthcare.gov stumble and nearly fall over when it was first launched.

Chúng ta đã thấy những dấu hiệu của những thảm hoạ như thế trong quá khứ gần đây. Chúng ta đã thấy phần mềm lao hai chiếc máy bay phản lực chở khách xuống đất ở tốc độ âm thanh, giết chết 346 người. Chúng ta đã thấy lỗi phần mềm giết hàng chục người và làm hàng trăm người bị thương, khi máy tính trên xe của họ mất trí và tăng tốc mất kiểm soát. Chúng ta đã thấy Knight Capital mất 450 triệu đô la trong 45 phút vì một lỗi phần mềm. Chúng ta đã thấy Healthcare.gov loạng choạng và suýt sụp đổ ngay khi vừa ra mắt.

One day, possibly one day soon, there will be a software error that kills tens of thousands of people in a single incident. Nowadays, it's not very hard to imagine a scenario like that. And when that occurs, we programmers will be blamed.

Một ngày nào đó, có thể là một ngày không xa, sẽ có một lỗi phần mềm giết chết hàng chục ngàn người chỉ trong một sự cố. Thời buổi này, tưởng tượng ra một kịch bản như vậy chẳng khó khăn gì. Và khi điều đó xảy ra, chúng ta — những lập trình viên — sẽ bị đổ lỗi.

Don't think it will be your bosses. Don't think it will be the CEOs. Oh, they'll be in hot water too. But the finger of blame will be pointed directly at us—because, after all, it will have been our fingers on the keyboards.

Đừng nghĩ rằng người ta sẽ đổ cho sếp bạn. Đừng nghĩ rằng người ta sẽ đổ cho các CEO.Ồ, họ cũng sẽ vạ lây thôi. Nhưng ngón tay buộc tội sẽ chỉ thẳng vào chúng ta — bởi vì, xét cho cùng, chính ngón tay của chúng ta đã đặt trên bàn phím.

And then imagine the trials. Imagine the testimony of the expert consultants complaining about thousands of global variables, poorly named arguments, mismanaged and missing semaphores, memory leaks, improperly allocated stacks, heap fragmentation, overdependence on performance over quality, 3,000-line functions, modules with duplicated code of which some, but not all, had been repaired.

Rồi hãy hình dung các phiên toà. Hình dung lời khai của những chuyên gia tư vấn phân nân về hàng ngàn biến toàn cục, những đối số đặt tên cầu thả, những semaphore quản lý sai bét hoặc thiếu hẳn, rò rỉ bộ nhớ, stack cấp phát sai cách, phân mảnh heap, việc đặt nặng hiệu năng hơn chất lượng, những hàm 3.000 dòng, những module chứa mã trùng lặp mà chỉ một số — chứ không phải tất cả — được sửa.

We've seen that kind of testimony before—too many times.

Chúng ta đã thấy kiểu lời khai đó trước đây rồi — nhiều lần quá rồi.

Imagine the response of the world's politicians as they try to grapple with the loss of tens of thousands of their citizens.

Hãy hình dung phản ứng của giới chính trị gia trên thế giới khi họ phải vật lộn với cái chết của hàng chục ngàn công dân nước mình.

They would, of course, have to pass new laws.

Dĩ nhiên, họ sẽ phải thông qua những đạo luật mới.

What kinds of laws would they pass? Among other things, they might tell us what languages we have to use, what platforms and frameworks are allowable, what processes we have to follow, what signatures we have to get, what documents we need to write, what review procedures we have to use, what courses we have to take, and what books we have to read.

Họ sẽ thông qua những đạo luật kiểu gì? Trong số nhiều thứ khác, họ có thể bảo chúng ta phải dùng ngôn ngữ nào, được phép dùng nền tảng và framework nào, phải theo quy trình nào, phải xin chữ ký của ai, phải viết những tài liệu gì, phải dùng thủ tục review nào, phải học những khoá học nào, và phải đọc những cuốn sách nào.

I promise you that you and I will not like those new laws. I'm also reasonably certain that those laws will do little to improve the situation.

Tôi cam đoan với bạn rằng cả bạn lẫn tôi đều sẽ không ưa nổi những đạo luật mới ấy. Tôi cũng khá chắc rằng những đạo luật đó sẽ chẳng cải thiện được tình hình bao nhiêu.

So, the first answer to why we should clean our code is that our civilization needs us to behave as professionals, and that if we fail in that responsibility, our governments will force inconvenient and ineffective rules upon us.

Vậy nên, câu trả lời đầu tiên cho câu hỏi vì sao chúng ta phải dọn sạch mã của mình là: nền văn minh của chúng ta cần chúng ta cư xử như những người chuyên nghiệp, và nếu chúng ta thất bại trong trách nhiệm đó, chính quyền sẽ áp lên chúng ta những luật lệ vừa phiền toái vừa vô hiệu.

But there is another reason that we need to clean our code. And this one strikes much closer to home.

Nhưng còn một lý do khác khiến chúng ta cần dọn sạch mã của mình. Và lý do này thì sát sườn hơn nhiều.

Productivity — Năng suất

I've asked you before if you have ever been significantly impeded by bad code. If you are a programmer of any number of years, you most certainly have. Once again, my question is: Why did you write it?

Tôi đã từng hỏi bạn liệu có bao giờ bạn bị mã tệ cản chân nghiêm trọng chưa. Nếu bạn là lập trình viên đã hành nghề được ít năm, chắc chắn là có. Một lần nữa, câu hỏi của tôi là: Tại sao bạn lại viết ra nó?

Why do we write the code that we know will significantly impede us? The answer always is: to go fast. I'll leave you to deal with the logical inconsistency of that answer.

Vì sao chúng ta viết ra thứ mã mà mình biết chắc rồi sẽ cản chân mình nặng nề? Câu trả lời bao giờ cũng là: để đi cho nhanh. Tôi xin nhường bạn tự đối diện với sự phi lý về mặt logic của câu trả lời đó.

Now look, I know all the excuses. They all boil down to: They never give us time. This is false. The only way to go fast is to go well. The best way to meet the schedule, the best way to make the deadline, is to do a good job. If you want to go as fast as you can, don't do the things that slow you down! Clean your code!

Này, tôi biết hết mấy lời bào chữa rồi. Rốt cuộc chúng đều quy về một câu: Họ chẳng bao giờ cho chúng tôi thời gian. Sai. Cách duy nhất để đi nhanh là đi cho tử tế. Cách tốt nhất để bám kịp lịch trình, cách tốt nhất để kịp hạn chót, là làm cho ra hồn. Nếu bạn muốn đi nhanh hết mức có thể, đừng làm những việc khiến bạn chậm lại! Hãy dọn sạch mã của bạn!

A messy module slows down everyone who has to deal with it, every time they have to deal with it. The cost of the mess repeats over and over, whereas the cost of cleaning is a one-time thing.

Một module bầy hầy làm chậm mọi người phải đụng tới nó, mỗi lần họ phải đụng tới nó. Cái giá của mớ hỗn độn lặp đi lặp lại mãi, trong khi cái giá của việc dọn dẹp chỉ phải trả một lần.

A team who allows messes to persist in their code will gradually slow down to a tiny fraction of their potential **productivity**. I've seen teams who have expanded their estimates out to weeks and months for jobs that ought to take a day or two.

Một đội để cho các mớ hỗn độn tồn tại dai dẳng trong mã của mình sẽ dần dần chậm lại tới mức chỉ còn một phần bé xíu so với năng suất tiềm năng. Tôi đã thấy những đội phải giãn ước lượng (estimate) ra thành nhiều tuần, nhiều tháng cho những việc lẽ ra chỉ tốn một hai ngày.

The slower the team goes, the more the pressure builds. The more the pressure builds, the more messes are left uncleaned. Each such mess impedes the team even more; and the slower they all go. It's a vicious cycle, and if you've been a programmer for more than a couple of years, you've likely taken a few turns around it.

Đội càng đi chậm, áp lực càng dồn lên. Áp lực càng dồn lên, càng nhiều mớ hỗn độn bị bỏ mặc không dọn. Mỗi mớ hỗn độn như vậy lại cản chân cả đội thêm nữa; và họ càng đi

chậm hơn. Đó là một vòng luẩn quẩn, và nếu bạn đã làm lập trình viên hơn vài năm, hẳn bạn đã quay vài vòng trong đó rồi.

But it gets worse. Managers, desperate to increase productivity, hire new people. This, of course, has the opposite of the desired effect. The new people, thrown into the mass of messy code, see how things are done around here and—of course—they emulate it. So now there are even more people making messes, which drives the cycle even faster, and drags the productivity even lower.

Nhưng còn tệ hơn nữa. Cánh quản lý, cuống cuồng muốn tăng năng suất, bèn tuyển thêm người mới. Dĩ nhiên, chuyện này cho kết quả ngược hẳn với mong đợi. Đám người mới, bị ném vào giữa khối mã bầy hầy, thấy ở đây người ta làm việc kiểu gì và — tất nhiên — họ bắt chước y hệt. Thế là giờ có thêm cả đồng người bầy ra hỗn loạn, khiến vòng luẩn quẩn quay nhanh hơn nữa, và kéo năng suất xuống thấp hơn nữa.

Managers, astounded that the addition of people did not increase productivity, may try to add even more people into the fracas—because the definition of sanity is to try the same thing over and over and expect different results. In the end, however, the managers have no choice but to consult with the developers to find out why everything is going so slowly.

Cánh quản lý, sửng sốt vì thêm người mà năng suất chẳng tăng, có thể sẽ thử nhét thêm người nữa vào đám hỗn chiến — bởi vì định nghĩa của sự tỉnh táo là làm đi làm lại cùng một việc mà mong kết quả khác đi. Nhưng rốt cuộc, họ chẳng còn lựa chọn nào ngoài việc ngồi xuống hỏi các lập trình viên xem vì sao mọi thứ chậm đến thế.

And the developers have an answer. They've known the answer for months, if not years. And when asked, they are more than willing to volunteer that answer: Redesign the whole system from scratch.

Và các lập trình viên có sẵn câu trả lời. Họ đã biết câu trả lời ấy hàng tháng trời, nếu không muốn nói là hàng năm. Và khi được hỏi, họ sẵn sàng tự nguyện dâng ngay câu trả lời đó: Thiết kế lại toàn bộ hệ thống từ đầu.

Imagine the shock and horror of the managers. The developers have just told them that all their previous work is worthless and has to be scrapped. The developers are also telling them that this new design won't suffer the same fate —that productivity won't decrease again—that everything, this time, will be great.

Hãy hình dung nỗi choáng váng và kinh hoàng của cánh quản lý. Các lập trình viên vừa nói với họ rằng toàn bộ công sức trước đây là vô giá trị và phải vứt đi hết. Các lập trình viên cũng nói với họ rằng thiết kế mới này sẽ không chịu chung số phận — rằng năng suất sẽ không tụt lại lần nữa — rằng lần này thì mọi thứ sẽ tuyệt vời.

Of course the managers don't believe that; and they fight strenuously against the idea, but what choice do they really have? The experts are telling them that the only hope is for the system to be redesigned from scratch. In the end, those poor managers will accede to the demands of the developers. Again, what choice do they have?

Dĩ nhiên cánh quản lý không tin điều đó; và họ chống lại ý tưởng ấy quyết liệt, nhưng thực sự thì họ còn lựa chọn nào khác? Các chuyên gia đang bảo họ rằng hy vọng duy nhất là thiết kế lại hệ thống từ con số không. Rốt cuộc, những nhà quản lý tội nghiệp ấy sẽ nhượng bộ trước đòi hỏi của các lập trình viên. Lại phải hỏi, họ còn lựa chọn nào khác?

And when those managers finally agree, a cheer will go up from the developers. "Hallelujah, we're all going back to the beginning where there's no mess to deal with and life will be good!"

Và khi cánh quản lý cuối cùng cũng đồng ý, các lập trình viên sẽ reo hò. "Hallelujah, chúng ta được quay về vạch xuất phát, nơi chẳng có mớ hỗn loạn nào phải đối phó và đời sẽ tươi đẹp!"

But that's not what actually happens. Instead, the ten best, the Tiger Team, are chosen to redesign the system. These are the people who made the mess in the first place. They are going into a room to work together to take the project in a “new and beautiful” direction. They will chart the course to the future.

Nhưng thực tế lại không diễn ra như vậy. Thay vào đó, mười người giỏi nhất, đội Tiger Team, được chọn ra để thiết kế lại hệ thống. Đây chính là những người đã bày ra mớ hỗn độn ngay từ đầu. Họ kéo nhau vào một căn phòng để cùng đưa dự án theo một hướng “mới mẻ và đẹp đẽ”. Họ sẽ vạch ra con đường tới tương lai.

Meanwhile, the rest of the team is stuck maintaining the old system. After all, that system is part of the business. Bugs have to be fixed. New features have to be added.

Trong khi đó, phần còn lại của đội bị kẹt lại với việc bảo trì hệ thống cũ. Xét cho cùng, hệ thống ấy vẫn là một phần của công việc kinh doanh. Lỗi vẫn phải sửa. Tính năng mới vẫn phải thêm.

Where does the Tiger Team get their requirements from? Is there a requirements document that anyone seriously trusts? Or have there been so many late-night scrambles and midnight modifications that the old requirements are currently hash? No matter, because the requirements are reliably recorded in one definite place—the old code.

Tiger Team lấy yêu cầu từ đâu ra? Có tài liệu yêu cầu nào mà ai đó thực sự tin tưởng không? Hay đã có quá nhiều pha chữa cháy thâu đêm và những sửa đổi lúc nửa khuya khiến mớ yêu cầu cũ giờ thành một mớ bòng bong? Không sao cả, bởi các yêu cầu vẫn được ghi lại đáng tin cậy ở một nơi rõ ràng — mã cũ.

So the Tiger Team is stuck in their room, poring over the old code, trying to figure out what the hell this system actually does. Meanwhile, the rest of the team is changing that code by fixing bugs and adding new features.

Thế là Tiger Team bị nhốt trong phòng, chăm măt vào mã cũ, cố hiểu xem cái hệ thống này rốt cuộc làm cái quái gì. Trong khi đó, phần còn lại của đội vẫn đang thay đổi chính đoạn mã đó bằng cách sửa lỗi và thêm tính năng mới.

And this sets up the race. The Tiger Team has to catch up to where the maintenance team is. But every time the Tiger Team gets to where the maintenance team was, the maintenance team has moved ahead!

Và thế là cuộc đua bắt đầu. Tiger Team phải đuổi kịp chỗ mà đội bảo trì đang đứng. Nhưng cứ mỗi lần Tiger Team tới được chỗ đội bảo trì từng đứng, thì đội bảo trì đã tiến lên phía trước mất rồi!

This is Xeno's paradox—the race between the great Achilles and a tortoise with a head start. Every time Achilles reaches the point where the tortoise was when he began, the tortoise will have moved ahead from that spot. Xeno used that **argument** to prove that all motion was impossible.

Đây chính là nghịch lý Zeno — cuộc chạy đua giữa dũng sĩ Achilles và con rùa được chấp trước một đoạn. Cứ mỗi lần Achilles chạy tới điểm mà con rùa đang đứng lúc anh xuất phát, con rùa lại đã nhích lên khỏi điểm đó. Zeno dùng lập luận ấy để chứng minh rằng mọi chuyển động đều bất khả.

It turns out that if you know enough calculus, you can prove that Achilles will, in fact, pass the tortoise. But that math doesn't always work for software projects.

Hoá ra nếu biết đủ về giải tích, bạn có thể chứng minh rằng Achilles thực ra sẽ vượt qua con rùa. Nhưng thứ toán học đó không phải lúc nào cũng đúng với các dự án phần mềm.

I once worked on a project that was being redesigned from scratch. Ten years later, the “New System” had still not been deployed. The customers had been told, years before, to expect a new and glorious system. But that system never came. Every time the company tried to replace the old systems the customers complained that the new system did not do everything the old system did. Try as they might, the Tiger Team could not catch up to the maintenance team. And since the maintenance team was revenue bearing, they got all the resources they needed, whereas the Tiger Team was always limited.

Tôi từng làm trong một dự án đang được thiết kế lại từ đầu. Mười năm sau, “Hệ thống Mới” vẫn chưa được triển khai. Khách hàng đã được hứa hẹn từ nhiều năm trước rằng sẽ có một hệ thống mới huy hoàng. Nhưng hệ thống ấy chẳng bao giờ tới. Cứ mỗi lần công ty định thay thế các hệ thống cũ, khách hàng lại phàn nàn rằng hệ thống mới không làm được đủ mọi thứ như hệ thống cũ. Cố cách mấy, Tiger Team vẫn không đuổi kịp đội bảo trì. Và vì đội bảo trì mới là nơi mang lại doanh thu, họ được cấp mọi nguồn lực cần thiết, còn Tiger Team thì luôn bị bó buộc.

As the months and years went by, the original members of the Tiger Team were eventually replaced and promoted to work on other projects. After ten years, none of the originals were left.

Tháng ngày trôi qua, rồi năm tháng trôi qua, các thành viên ban đầu của Tiger Team lần lượt bị thay thế hoặc được thăng chức sang dự án khác. Sau mười năm, chẳng còn ai trong nhóm ban đầu ở lại.

In the end, the company, in utter desperation, told their customers that they had no choice but to accept the “new” system. The customers railed and complained; but there was simply no other option. The company could not afford to keep funding two development efforts.

Cuối cùng, trong nỗi tuyệt vọng cùng cực, công ty nói với khách hàng rằng họ không còn lựa chọn nào khác ngoài việc chấp nhận hệ thống “mới”. Khách hàng la ó và phàn nàn; nhưng đơn giản là chẳng còn phương án nào khác. Công ty không kham nổi việc tiếp tục rót tiền cho hai nỗ lực phát triển cùng lúc.

Upon hearing that the new system was going to be deployed, the Tiger Team stood up in horror saying: “You can’t ship this, it’s crap! It’s got to be redesigned!”

Nghe tin hệ thống mới sắp được triển khai, Tiger Team đứng bật dậy kinh hãi mà rằng: “Không thể phát hành cái này được, nó là đồ bỏ! Phải thiết kế lại thôi!”

OK, I’ve embellished this story a bit. Actually, it’s a composite tale from my own experiences and the stories many of my later clients told me. But I know there are many of you, my gentle readers, who identified with it.

Được rồi, tôi có thêm mắm dấm muối cho câu chuyện này một chút. Thật ra, đó là một câu chuyện chắp ghép từ chính trải nghiệm của tôi và từ những gì rất nhiều khách hàng về sau kể lại. Nhưng tôi biết có nhiều bạn, những độc giả thân mến của tôi, đã thấy chính mình trong đó.

So why should we clean our code? Because cleaning our code is the best way for a team to remain productive. If we allow the mess to build, we lay the groundwork for the disaster I just described. But if we keep the code clean, productivity will remain high, the development team will not demand a redesign, and the managers will not desperately try to increase productivity by adding more and more manpower.

Vậy vì sao chúng ta phải dọn sạch mã của mình? Bởi vì dọn sạch mã là cách tốt nhất để một đội duy trì được năng suất. Nếu chúng ta để mớ hỗn độn chồng chất lên, chúng ta đang đặt nền móng cho đúng thảm họa mà tôi vừa mô tả. Nhưng nếu chúng ta giữ cho mã

sạch, năng suất sẽ luôn ở mức cao, đội phát triển sẽ không đòi thiết kế lại, và cảnh quản lý sẽ không phải cố gắng tăng năng suất bằng cách nhét thêm mã nhân lực vào.

Livability — Khả năng ở được (Livability)

7

7

7 <https://blog.cleancoder.com/uncle-bob/2018/08/13/TooClean.html>

7 <https://blog.cleancoder.com/uncle-bob/2018/08/13/TooClean.html>

It would be easy to interpret all of the above comments as aspiring to some kind of Golden Perfection. That is not our goal! Golden Perfection is paralyzing—it affords no room to move. You cannot live within Golden Perfection.

Rất dễ hiểu tất cả những điều nói ở trên như một khát vọng vươn tới thứ Hoàn Hảo Vàng nào đó. Đó không phải mục tiêu của chúng ta! Hoàn Hảo Vàng làm người ta tê liệt — nó chẳng chữa chỗ nào để xoay xở. Bạn không thể sống bên trong Hoàn Hảo Vàng.

Clean code is code that you can live with. It is not perfect code. It is more like a clean house as opposed to a show house. A show house is a house you cannot live in, because the act of living degrades the show-ability of the house. A clean house is a house that you can live in without degrading its livability.

Clean code là mã mà bạn có thể sống chung được. Nó không phải mã hoàn hảo. Nó giống một ngôi nhà sạch sẽ hơn là một ngôi nhà mẫu. Nhà mẫu là ngôi nhà bạn không thể ở, bởi chính việc sinh sống trong đó sẽ phá hỏng vẻ trưng bày của nó. Nhà sạch là ngôi nhà bạn có thể ở mà không làm giảm khả năng ở được của nó.

Clean code is code that you can maintain, expand, enhance, and evolve without degrading its livability. Clean code is not sparkling and shiny; it looks lived in but cared for. It may not read like Dickens, but it's readable. It may not have the perfect design and **architecture**, but it is carefully designed. Concepts may not be perfectly isolated, but they are isolated enough.

Clean code là mã bạn có thể bảo trì, mở rộng, cải thiện và tiến hoá mà không làm giảm khả năng ở được của nó. Clean code không lấp lánh bóng loáng; nó trông có dấu vết sinh hoạt nhưng được chăm chút. Nó có thể không đọc mượt như văn Dickens, nhưng nó đọc được. Nó có thể không có thiết kế và kiến trúc (architecture) hoàn hảo, nhưng nó được thiết kế cẩn thận. Các khái niệm có thể không tách bạch hoàn hảo, nhưng chúng tách bạch đủ dùng.

Clean code is comfortable in the way that a clean but lived-in house is comfortable. There might be some crumbs on the floor beneath the breakfast counter; there may be a drop of milk on that counter; there may be some dog hair on the sofa, the pillows on the love seat might be in disarray, but none of that degrades the livability of the house—so long as it doesn't get out of hand.

Clean code dễ chịu theo đúng cái cách một ngôi nhà sạch nhưng có người ở là dễ chịu. Có thể có vài mẩu vụn bánh rơi trên sàn dưới quầy ăn sáng, có thể có một giọt sữa vương trên mặt quầy, có thể có ít lông chó trên ghế sofa, mấy chiếc gối trên ghế đôi có thể xộc xệch, nhưng chẳng điều nào trong số đó làm giảm khả năng ở được của ngôi nhà — chừng nào chúng chưa vượt khỏi tầm kiểm soát.

Clean code is code that has not gotten out of hand. Those who write and maintain it are not seeking perfection, they don't spend endless hours polishing it, but they do strive to keep the flotsam and detritus of life from accumulating. Or, to invoke Michael Feathers again, they care.

Clean code là mã chưa vượt khỏi tầm kiểm soát. Những người viết và bảo trì nó không đi tìm sự hoàn hảo, họ không bỏ ra vô số giờ để đánh bóng nó, nhưng họ thực sự cố giữ cho rác rưởi và cặn bã của đời sống thường ngày đừng tích tụ lại. Hay, mượn lời Michael Feathers một lần nữa: họ quan tâm.

We Read More Than We Write — Ta đọc nhiều hơn ta viết

Have you ever played back an edit session? In the '80s and '90s, we had editors like Emacs that kept track of every keystroke. You could work for an hour and then play back your whole edit session like a high-speed movie. When I did this, the results were fascinating. The vast majority of the playback was scrolling and navigating to other modules!

Bạn đã bao giờ tua lại một phiên soạn thảo chưa? Vào thập niên 80 và 90, chúng ta có những editor như Emacs ghi lại từng phím gõ. Bạn có thể làm việc suốt một tiếng đồng hồ rồi tua lại toàn bộ phiên soạn thảo như một cuốn phim chiếu nhanh. Khi tôi thử làm thế, kết quả thật thú vị. Phần áp đảo của đoạn tua lại chỉ là cuộn màn hình và nhảy sang các module khác!

1. Bob enters the module.

1. Bob bước vào module.

2. He scrolls down to the function needing change.

3. He pauses, considering his options.

4. Oh, he's scrolling up to the top of the module to check the initialization of a **variable**.

5. Now he scrolls back down and begins to type.

6. Oops, he's erasing what he typed!

7. He types it again.

8. He erases it again!

9. He types half of something else but then erases that!

10. He scrolls down to another function that calls the function he's changing to see how it is called.

11. He scrolls back up and types the same code he just erased.

12. He pauses.

13. He erases that code again!

14. He pops up another window and looks at a subclass. Is that function overridden?

2. Anh ta cuộn xuống tới hàm cần sửa.

3. Anh ta dừng lại, cân nhắc các lựa chọn.

4. Ồ, anh ta cuộn ngược lên đầu module để xem chỗ khởi tạo một biến.

5. Giờ anh ta cuộn trở xuống và bắt đầu gõ.

6. Ối, anh ta xoá đi thứ vừa gõ!

7. Anh ta gõ lại lần nữa.

8. Rồi lại xoá đi!

9. Anh ta gõ nửa chừng một thứ khác, rồi cũng xoá nốt!

10. Anh ta cuộn xuống một hàm khác có gọi tới hàm đang sửa, để xem nó được gọi thế nào.

11. Anh ta cuộn ngược lên và gõ lại đúng đoạn mã vừa xoá.

12. Anh ta dừng lại.

13. Anh ta lại xoá đoạn mã đó!

14. Anh ta bật thêm một cửa sổ và ngó vào một subclass. Liệu hàm đó có bị override không nhỉ?

...

...

You get the drift. Indeed, the ratio of time spent reading versus writing is well over 10:1. We are constantly reading old code as part of the effort to write new code; because the new code has to be kept consistent with all the old code.

Bạn hiểu ý tôi rồi đấy. Quả thực, tỉ lệ thời gian đọc so với thời gian viết cao hơn 10:1 rất nhiều. Chúng ta liên tục đọc mã cũ như một phần của nỗ lực viết mã mới; bởi mã mới phải được giữ cho nhất quán với toàn bộ mã cũ.

Because we read code so much more than we write it, we want the reading of code to be the more efficient operation. And since there's no way to write code without reading it, making it easy to read actually makes it easier to write.

Vì đọc mã nhiều hơn hẳn viết mã, ta muốn việc đọc mới là thao tác hiệu quả hơn. Và vì chẳng có cách nào viết mã mà không đọc mã, làm cho mã dễ đọc thực ra lại khiến việc viết trở nên dễ hơn.

There is no escape from this logic. You cannot write code if you cannot read the surrounding code. The code you are trying to write today will only be easy to write if the surrounding code is easy to read. So, if you want to go fast, if you want to get done quickly, if you want your code to be easy to write, make it easy to read .

Không thoát khỏi cái logic này được đâu. Bạn không thể viết mã nếu không đọc nổi đoạn mã xung quanh. Đoạn mã bạn định viết hôm nay chỉ dễ viết nếu mã xung quanh nó dễ đọc. Vậy nên, nếu muốn đi nhanh, nếu muốn xong sớm, nếu muốn mã của mình dễ viết, hãy làm cho nó dễ đọc.

The Boy Scout Rule — Quy tắc Hướng đạo sinh (the Boy Scout Rule)

It's not enough to write the code well. The code has to be kept clean over time. We've all seen code rot and degrade as time passes. So we must take an active role in preventing this degradation.

Viết mã cho tốt thôi thì chưa đủ. Mã còn phải được giữ sạch theo thời gian. Tất cả chúng ta đều đã thấy mã mục ruỗng và xuống cấp khi năm tháng trôi qua. Vì vậy ta phải chủ động vào cuộc để ngăn sự xuống cấp ấy.

The Boy Scouts of America have a simple rule that we can apply to our profession.

Hội Hướng đạo sinh Hoa Kỳ có một quy tắc đơn giản mà chúng ta có thể áp dụng cho nghề của mình.

Leave the campground cleaner than you found it.

Hãy để lại khu cắm trại sạch hơn lúc bạn đến.

If we all checked in our code a little cleaner than when we checked it out, the code simply could not rot.

Nếu tất cả chúng ta đều check in mã sạch hơn một chút so với lúc check out, thì mã đơn giản là không thể mục ruỗng được.

Check the code in cleaner than you checked it out.

Hãy check in mã sạch hơn lúc bạn check out.

The cleanup doesn't have to be something big. Just do one small random act of kindness. Change one variable name for the better, break up one function that's a little too large, eliminate one small bit of **duplication**, clean up one composite if statement.

Việc dọn dẹp không nhất thiết phải là chuyện gì to tát. Chỉ cần làm một hành động tử tế nho nhỏ, ngẫu hứng thôi. Đổi một tên biến cho khá hơn, chẻ nhỏ một hàm hơi quá dài, khử một chỗ trùng lặp bé tí, dọn gọn một câu lệnh if ghép.

Can you imagine working on a project where everyone on the team practiced this rule? The code would simply get better as time passed. Do you believe that any other option is professional? Indeed, isn't continuous improvement an intrinsic part of **professionalism**—of being a moral human?

Bạn có hình dung nổi cảnh làm việc trong một dự án mà mọi người trong nhóm đều thực hành quy tắc này không? Mã sẽ cứ thế tốt lên theo thời gian. Bạn có tin rằng bất kỳ lựa chọn nào khác là chuyên nghiệp không? Thật vậy, chẳng phải cải tiến không ngừng là một phần cố hữu của tính chuyên nghiệp — của việc làm một con người có đạo đức — hay sao?

Part I — Phần I

Code — Mã nguồn

In Part I, we explore the code-level practices of cleanliness. In the first two chapters, we begin with basics and first principles. In subsequent chapters, we look at comments, **formatting**, and names. Then, we get into the meat of the topic: functions, testing, and partitioning. Throughout, we'll be walking through several worked examples, explained in detail.

Ở Phần I, chúng ta khám phá những thực hành về sự sạch sẽ ở mức mã nguồn. Hai chương đầu bắt đầu từ những điều căn bản và các nguyên lý đầu tiên. Các chương tiếp theo bàn về comment, định dạng (formatting) và cách đặt tên. Rồi chúng ta đi vào phần cốt lõi của chủ đề: hàm, kiểm thử và phân chia trách nhiệm. Xuyên suốt, chúng ta sẽ cùng đi qua vài ví dụ được làm tới nơi tới chốn, giải thích thật chi tiết.

And although this part is not specifically about high-level design concepts, they are never far away.

Và dù phần này không nói riêng về các khái niệm thiết kế ở tầng cao, chúng chưa bao giờ ở xa cả.

While Java predominates in this part, we'll see many other languages, including Python, Go, Ruby, Clojure, JavaScript, and C. What you should take away from that is that the principles of **clean code** are independent of language and platform.

Java chiếm phần lớn trong phần này, nhưng ta cũng sẽ gặp nhiều ngôn ngữ khác, gồm Python, Go, Ruby, Clojure, JavaScript và C. Điều bạn nên rút ra là: các nguyên tắc của clean code không phụ thuộc vào ngôn ngữ hay nền tảng.

Chapter 2. Clean That Code! — Chương 2. Dọn dẹp đoạn mã đó đi!



Clean code! Is that a noun? Or is it a command? Is the word clean an adjective or a verb? In this chapter, and in the book overall, we take the latter view. This is a chapter and a book about cleaning code.

Clean code! Đó là một danh từ? Hay là một mệnh lệnh? Chữ *clean* ở đây là tính từ hay động từ? Trong chương này, và trong cả cuốn sách, chúng tôi chọn cách hiểu thứ hai. Đây là một chương và một cuốn sách nói về việc *dọn dẹp* mã nguồn.

One does not simply write clean code. To create code that is clean one must clean it. Code does not pour from your mind and out through your fingers in a clean state. That's not the way our brains work. This is because cleanliness and functionality are competing orthogonal concerns that we, mere humans, are not particularly **good** at balancing. Instead, our limited minds first focus upon the concern of functionality as we strive to produce code that works. It is only when that goal has been achieved that we can change our focus toward cleaning the **mess** we just made.

Không ai chỉ đơn giản là “viết ra” clean code cả. Muốn có mã sạch thì phải dọn nó. Mã không tự tuôn ra từ đầu bạn, chảy qua mấy đầu ngón tay rồi hiện ra sạch bong. Não chúng ta không hoạt động như vậy. Lý do là sự sạch sẽ và tính đúng chức năng là hai mối bận tâm trực giao và cạnh tranh nhau, mà con người phạm trần như chúng ta lại không giỏi cân bằng chúng cho lắm. Thay vào đó, bộ óc hữu hạn của chúng ta trước hết dồn hết vào chuyện chức năng, cố sao viết ra được đoạn mã *chạy được*. Chỉ khi đạt được mục tiêu đó rồi, ta mới có thể chuyển sự chú ý sang dọn dẹp cái mớ hỗn độn vừa bày ra.

Or again, as Kent Beck so succinctly put it:

Hay nói như Kent Beck đã gói gọn rất khéo:

First, make it work. Then, make it right.

Trước hết, làm cho nó chạy. Sau đó, làm cho nó đúng.

When we do the first part—make it work—we also usually make a mess. This is because making it work takes all of our cognitive energy, and there is none left over to think about anything else.

Khi làm phần thứ nhất — làm cho nó chạy — ta thường cũng bày ra một mớ hỗn độn. Bởi vì việc làm cho nó chạy đã ngốn sạch năng lượng nhận thức của ta, chẳng còn tí nào để nghĩ tới thứ gì khác.

Once we have something that works, then we can pull back, look it over, and think of ways to improve it.

Một khi đã có thứ chạy được, ta mới lùi lại, ngắm nghía nó và nghĩ xem cải thiện thế nào.

You’ve all had the experience of looking over code that you wrote in the past—perhaps only a few days or hours in the past—and then realized that it could be improved by this or that change.

Ai trong các bạn hẳn cũng từng trải qua chuyện này: nhìn lại đoạn mã mình viết trước đó — có khi chỉ vài ngày, thậm chí vài giờ trước — rồi chợt nhận ra nó có thể tốt hơn nếu sửa chỗ này chỗ kia.

You’ve also, very likely, had the same experience with just about anything you’ve ever done. If you’ve ever written a paper, or a story, or an article, you likely created rough drafts and then passed over them once, or twice, or thrice, to improve them. If you’ve ever drawn a picture, you’ve likely spent a lot of time using your eraser. Indeed, I have reread the last few paragraphs several times and have made small improvements each time.

Nhiều khả năng bạn cũng từng trải nghiệm y như vậy với gần như mọi thứ bạn từng làm. Nếu đã từng viết một bài luận, một truyện ngắn hay một bài báo, hẳn bạn đã có bản nháp rồi đọc lại một, hai, ba lượt để làm cho nó hay hơn. Nếu đã từng vẽ tranh, hẳn bạn đã dành khá nhiều thời gian cho cục tẩy. Thật vậy, tôi đã đọc lại vài đoạn vừa rồi mấy lần, và lần nào cũng sửa được đôi chỗ cho tốt hơn.

This is how we humans work. We produce something fairly ugly at first, and then we massage and manipulate it until it is better.

Con người chúng ta làm việc kiểu đó. Ban đầu ta cho ra một thứ khá xấu xí, rồi nhào nặn, chỉnh sửa cho tới khi nó khá hơn.

Perhaps an example will help. I wrote a simple 1 converter from Roman numerals to integers. I did this, with the help of GitHub Copilot, by simply throwing code into a **function** and fumbling around until I thought it worked. I had some unit tests that Copilot and I likewise threw together, but I was frankly not sure they covered all the cases. Here’s the code. I expect that it will make your eyes cross, and cause the blood to drain from your face as you stare in horror.

Có lẽ một ví dụ sẽ giúp ích. Tôi đã viết một bộ chuyển đổi đơn giản 1 từ chữ số La Mã sang số nguyên. Tôi làm việc đó với sự trợ giúp của GitHub Copilot, chỉ bằng cách quảng đại mã vào một hàm rồi mò mẫm loanh quanh cho tới khi thấy nó có vẻ chạy được. Tôi cũng có vài unit test mà Copilot và tôi cùng chấp vá lại, nhưng thú thật tôi chẳng chắc chúng phủ hết các trường hợp. Đây là đoạn mã. Tôi đoán nó sẽ khiến mắt bạn lác đi, và mặt bạn tái mét vì kinh hãi khi nhìn vào.

1 By some definition of that word.

1 Theo một định nghĩa nào đó của từ này.

Listing 2-1

Listing 2-1

```
package fromRoman;
import java.util.Arrays;
public class FromRoman {
    public static int convert(String roman) {
        if (roman.contains("VIV") ||
            roman.contains("IVI") ||
            roman.contains("IXI") ||
            roman.contains("LXL") ||
            roman.contains("XLX") ||
            roman.contains("XCX") ||
            roman.contains("DCD") ||
            roman.contains("CDC") ||

||
            roman.contains("MCM")) {
            throw new InvalidRomanNumeralException(roman);
        }
        roman = roman.replace("IV", "4");
        roman = roman.replace("IX", "9");
        roman = roman.replace("XL", "F");
        roman = roman.replace("XC", "N");
        roman = roman.replace("CD", "G");
        roman = roman.replace("CM", "O");
        if (roman.contains("IIII") ||
            roman.contains("VV") ||
            roman.contains("XXXX") ||
            roman.contains("LL") ||
            roman.contains("CCCC") ||
            roman.contains("DD") ||
            roman.contains("MMMM")) {
            throw new InvalidRomanNumeralException(roman);
        }
        int[] numbers = new int[roman.length()];
        int i = 0;
        for (char digit : roman.toCharArray()) {
            switch (digit) {
                case 'I' -> numbers[i] = 1;
                case 'V' -> numbers[i] = 5;
```

```

        case 'X' -> numbers[i] = 10;
        case 'L' -> numbers[i] = 50;
        case 'C' -> numbers[i] = 100;

        case 'D' -> numbers[i] = 500;
        case 'M' -> numbers[i] = 1000;
        case '4' -> numbers[i] = 4;
        case '9' -> numbers[i] = 9;
        case 'F' -> numbers[i] = 40;
        case 'N' -> numbers[i] = 90;
        case 'G' -> numbers[i] = 400;
        case 'O' -> numbers[i] = 900;
        default -> throw new InvalidRomanNumeralException(roman);
    }
    i++;
}
int lastDigit = 1000;
for (int number : numbers) {
    if (number > lastDigit) {
        throw new InvalidRomanNumeralException(roman);
    }
    lastDigit = number;
}
return Arrays.stream(numbers).sum();
}
public static
class InvalidRomanNumeralException extends RuntimeException {
    public InvalidRomanNumeralException(String roman) {
    }
}
}
}

```

Future Bob: — Future Bob:

I'm reading this code two months later, and I'm laughing hysterically at just how

Tôi đọc lại đoạn mã này hai tháng sau, và tôi cười lăn lộn vì có những chỗ

awful some of it is (like 49FNGO). . . And yet how insightful parts of it are.

tệ hại đến mức khó tin (như 49FNGO). . . Ấy vậy mà cũng có những phần lại sâu sắc đến bất ngờ.

Here are the tests I used. I had my doubts about whether they were complete.

Đây là các test tôi đã dùng. Tôi vẫn ngờ ngợ không biết chúng đã đủ chưa.

```

package fromRoman;
import fromRoman.FromRoman.InvalidRomanNumeralException;
import org.junit.jupiter.api.Test;
import static fromRoman.FromRoman.convert;
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.is;

```

```

import static org.junit.jupiter.api.Assertions.assertThrows;
public class FromRomanTest {
    @Test
    public void valid() throws Exception {
        assertThat(convert(""), is(0));
        assertThat(convert("I"), is(1));
        assertThat(convert("II"), is(2));
        assertThat(convert("III"), is(3));
        assertThat(convert("IV"), is(4));
        assertThat(convert("V"), is(5));
        assertThat(convert("VI"), is(6));
        assertThat(convert("VII"), is(7));

        assertThat(convert("VIII"), is(8));
        assertThat(convert("IX"), is(9));
        assertThat(convert("X"), is(10));
        assertThat(convert("XI"), is(11));
        assertThat(convert("XII"), is(12));
        assertThat(convert("CXC"), is(190));
    }
    @Test
    public void invalid() throws Exception {
        assertInvalid("IIII");
        assertInvalid("VV");
        assertInvalid("XXXX");
        assertInvalid("LL");
        assertInvalid("CCCC");
        assertInvalid("DD");
        assertInvalid("MMMM");
        assertInvalid("IIV");
        assertInvalid("IVI");
        assertInvalid("IXI");
        assertInvalid("VIV");
        assertInvalid("XVX");
        assertInvalid("XIIII");
        assertInvalid("XVV");
        assertInvalid("XIVI");
        assertInvalid("XIXI");
        assertInvalid("XVIV");
        assertInvalid("LXL");

        assertInvalid("XLX");
        assertInvalid("XCX");
        assertInvalid("CDC");
        assertInvalid("DCD");
        assertInvalid("MCMC");
        assertInvalid("MCDM");
    }
    private void assertInvalid(String r) {
        assertThrows(InvalidRomanNumeralException.class, () -> convert
    }
}

```

}

I hope you agree with me that this needs some cleanup. It feels ragged and jumbled. Later in this chapter, and in the chapters to follow, I'll walk through the cleaning procedure. But for now, I just want to show you the result.

Tôi mong bạn cũng đồng ý với tôi rằng chỗ này cần được dọn dẹp. Nó có cảm giác lờm chờm và lộn xộn. Ở phần sau của chương này, và trong các chương kế tiếp, tôi sẽ dẫn bạn đi qua quy trình dọn dẹp. Nhưng lúc này, tôi chỉ muốn cho bạn xem kết quả.

Before I present the result, I want to say that the time it took to clean the code was roughly equal to the time it took to write the code in the first place. Copilot was moderately helpful during the initial writing but significantly less helpful during the cleaning process.

Trước khi trình bày kết quả, tôi muốn nói rằng thời gian bỏ ra để dọn dẹp đoạn mã xấp xỉ bằng thời gian viết ra nó ngay từ đầu. Copilot giúp được tương đối trong lúc viết ban đầu, nhưng giúp ít hơn hẳn trong quá trình dọn dẹp.

During the cleaning process, I found one or two small bugs simply through the effort of organizing my thoughts and the **structure** of the code and tests. It's remarkable what a second look can find 2—and cleaning is the ultimate second look.

Trong quá trình dọn dẹp, tôi tìm ra được một hai lỗi nhỏ, đơn giản là nhờ cái công sắp xếp lại suy nghĩ của mình cùng cấu trúc của mã và test. Thật đáng nể những gì một cái nhìn lần thứ hai có thể phát hiện ra 2 — và dọn dẹp chính là cái nhìn lần thứ hai đúng nghĩa nhất.

2 After I conduct the close-up preflight inspection on my airplane, I stand back, walk around the plane, and take a second look from a farther perspective. It's remarkable what I've found with that approach. There are things that are obvious from a distance that are nearly invisible during closer inspection.

2 Sau khi kiểm tra cận cảnh trước chuyến bay cho chiếc máy bay của mình, tôi lùi lại, đi vòng quanh nó và nhìn lần thứ hai từ khoảng cách xa hơn. Thật đáng nể những gì tôi đã phát hiện ra bằng cách đó. Có những thứ nhìn từ xa thì rõ mồn một nhưng lại gần như vô hình khi soi thật sát.

```
package fromRoman;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
public class FromRoman {
    private String roman;
    private List<Integer> numbers = new ArrayList<>();
    private int charIdx;
    private char nextChar;
    private Integer nextValue;
    private Integer value;
    private int nchars;
    Map<Character, Integer> values = Map.of(
        'I', 1,
        'V', 5,
        'X', 10,
        'L', 50,
        'C', 100,
```

```

        'D', 500,
        'M', 1000);
public FromRoman(String roman) {
    this.roman = roman;
}
public static int convert(String roman) {
    return new FromRoman(roman).doConversion();
}
private int doConversion() {
    checkInitialSyntax();
    convertLettersToNumbers();
    checkNumbersInDecreasingOrder();
    return numbers.stream().reduce(0, Integer::sum);
}
private void checkInitialSyntax() {
    checkForIllegalPrefixCombinations();
    checkForImproperRepetitions();
}
private void checkForIllegalPrefixCombinations() {
    checkForIllegalPatterns(
        new String[]{"VIV", "IVI", "IXI", "IXV", "LXL", "XLX",
                    "XCX", "XCL", "DCD", "CDC", "CMC", "CMD"});
}
private void checkForImproperRepetitions() {
    checkForIllegalPatterns(
        new String[]{"IIII", "VV", "XXXX", "LL", "CCCC", "DD", "MMMM"}
    );
}
private void checkForIllegalPatterns(String[] patterns) {
    for (String badString : patterns)
        if (roman.contains(badString))
            throw new InvalidRomanNumeralException(roman);
}
private void convertLettersToNumbers() {
    char[] chars = roman.toCharArray();
    nchars = chars.length;
    for (charIdx = 0; charIdx < nchars; charIdx++) {
        nextChar = isLastChar() ? 0 : chars[charIdx + 1];
        nextValue = values.get(nextChar);
        char thisChar = chars[charIdx];
        value = values.get(thisChar);
        switch (thisChar) {
            case 'I' -> addValueConsideringPrefix('V', 'X');
            case 'X' -> addValueConsideringPrefix('L', 'C');
            case 'C' -> addValueConsideringPrefix('D', 'M');
            case 'V', 'L', 'D', 'M' -> numbers.add(value);
            default -> throw new InvalidRomanNumeralException(roman);
        }
    }
}
}
}

```

```

private boolean isLastChar() {
    return charIdx + 1 == nchars;
}
private void addValueConsideringPrefix(char p1, char p2) {
    if (nextChar == p1 || nextChar == p2) {
        numbers.add(nextValue - value);
        charIdx++;
    } else numbers.add(value);
}
private void checkNumbersInDecreasingOrder() {
    for (int i = 0; i < numbers.size() - 1; i++)
        if (numbers.get(i) < numbers.get(i + 1))
            throw new InvalidRomanNumeralException(roman);
}
public static
class InvalidRomanNumeralException extends RuntimeException {
    public InvalidRomanNumeralException(String roman) {
        super("Invalid Roman numeral: " + roman);
    }
}
}

```

I think this is much cleaner. What makes it cleaner, in my opinion, are the short, well-named functions, the well-named instance variables, and the fact that the functions are listed in the order that they are called. You can read the code from the top to the bottom and it reads rather like a story.

Tôi cho rằng bản này sạch hơn nhiều. Theo tôi, thứ làm nên sự sạch sẽ đó là các hàm ngắn được đặt tên tốt, các biến instance được đặt tên tốt, và việc các hàm được liệt kê theo đúng thứ tự chúng được gọi. Bạn có thể đọc đoạn mã từ trên xuống dưới và nó đọc lên khá giống một câu chuyện.

Future Bob

Future Bob

: — :

Two months later I'm torn. The first

Hai tháng sau, tôi thấy mình phân vân. Bản

version, ugly as it was, was not as chopped up as this one. It's true that the names and the ordering of the extracted functions read like a story and are a big help in understanding the intent; but there

đầu tiên, xấu thì xấu thật, nhưng không bị băm nhỏ như bản này. Đúng là tên gọi và thứ tự của các hàm được tách ra đọc lên như một câu chuyện và giúp ích rất nhiều để hiểu ý định; nhưng

were several times that I had to scroll back up to the top to assure myself about the types of instance variables. I found the choppiness, and the

đã có vài lần tôi phải cuộn ngược lên đầu để yên tâm về kiểu của các biến instance. Tôi thấy sự vụn vặt đó, cùng với chuyện

scrolling, to be annoying. However, and this is critical, I am reading this cleaned code after having first read the ugly version and having gone

phải cuộn lên cuộn xuống, khá là phiền. Tuy nhiên — và điều này mới là mấu chốt — tôi đang đọc bản mã đã dọn này sau khi đã đọc bản xấu xí trước đó và đã bỏ

through the work of understanding it. So now, as I read this version, I am annoyed because I already

công sức ra để hiểu nó. Nên bây giờ, khi đọc bản này, tôi bực mình vì tôi đã

understand it and find the chopped-up functions and the instance variables redundant.

hiểu rồi và thấy mấy hàm bị băm nhỏ cùng đám biến instance là thừa.

Don't get me wrong, I still think the cleaner version is better. I just wasn't expecting the annoyance. When I first cleaned it, I thought it

Đừng hiểu lầm tôi, tôi vẫn cho rằng bản sạch hơn là bản tốt hơn. Chỉ là tôi không lường trước được cảm giác bực bội đó. Lúc mới dọn xong, tôi cứ tưởng

was going to be annoyance free.

nó sẽ chẳng có gì đáng bực.

I suppose the question you should ask yourself

Tôi cho rằng câu bạn nên tự hỏi mình

is which of these two pieces of code you would rather have read first. Which tells you more about

là: trong hai đoạn mã này, bạn muốn được đọc đoạn nào trước? Đoạn nào nói cho bạn nhiều hơn về

the intent? Which obscures the intent? Certainly the latter is better in that regard.

ý định? Đoạn nào che mờ ý định? Xét ở khía cạnh đó thì chắc chắn bản sau tốt hơn.

This annoyance is an issue that John 3 4 Ousterhout and I have debated. When you understand an algorithm, the artifacts intended to

Sự bực bội này là chuyện mà John 3 4 Ousterhout và tôi đã tranh luận với nhau. Khi bạn đã hiểu một thuật toán, những thứ được bày ra để

help you understand it become annoying. Worse, if you understand an algorithm, the names or comments you write to help others will be biased

giúp bạn hiểu nó lại trở nên phiền phức. Tệ hơn, nếu bạn đã hiểu một thuật toán, thì cái tên hay comment bạn viết ra để giúp người khác sẽ bị thiên lệch

by that understanding and may not help the reader as much as you think they will. A good

bởi chính sự hiểu biết đó, và có thể không giúp người đọc được nhiều như bạn tưởng. Một ví dụ hay

example of that, in this code, is the

cho chuyện đó, trong đoạn mã này, chính là hàm

addValueConsideringPrefix function. That name made perfect sense to me when I understood the algorithm. But it was a bit jarring two months later.

addValueConsideringPrefix. Cái tên ấy hoàn toàn hợp lý với tôi khi tôi đã hiểu thuật toán. Nhưng hai tháng sau thì nó nghe hơi chói.

Perhaps not as jarring as 49FNGO , but still not quite as obvious as I had hoped when I wrote it. It might have been better written as

Có lẽ không đến mức chói như 49FNGO , nhưng vẫn chưa hiển nhiên như tôi từng hy vọng lúc viết ra nó. Có lẽ viết như thế này thì hơn

`numbers.add(decrementValueIfThisCharIsaPrefix);`, since that

would be symmetrical with the `numbers.add(value);` in the nonprefixed case.

sẽ đối xứng với `numbers.add(value);` ở trường hợp không có prefix.

3 [APOD]. 4 See the appendix to read that debate. The bottom line is that your own understanding of what you are cleaning will work against your ability to communicate with the next person to

3 [APOD]. 4 Xem phụ lục để đọc cuộc tranh luận đó. Điểm mấu chốt là: chính sự thấu hiểu của bạn về thứ mà bạn đang dọn dẹp sẽ quay ra chống lại khả năng giao tiếp của bạn với người đến sau

come along. And this will be true whether you are extracting well-named methods, or adding descriptive comments. Therefore, take special care

. Và điều này đúng dù bạn đang tách ra những method được đặt tên tốt, hay đang thêm những comment mô tả. Vì vậy, hãy đặc biệt cẩn trọng

when choosing names and writing comments; and don't be surprised if others are annoyed by your

khi chọn tên và viết comment; và đừng ngạc nhiên nếu người khác thấy phiền vì những choices. Lastly, a look after a few months can be both humbling and profitable.

lựa chọn của bạn. Cuối cùng, nhìn lại sau vài tháng vừa khiến ta khiêm tốn hơn, vừa đem lại lợi ích.

The `convertLettersToNumbers` function is a bit longer than I like; but it hangs together pretty well, and I think that decomposing it into smaller functions would be more obscuring than helpful.

Hàm `convertLettersToNumbers` hơi dài hơn mức tôi muốn; nhưng nó liên lạc khá tốt, và tôi nghĩ việc chẻ nó thành những hàm nhỏ hơn sẽ làm mọi thứ tối nghĩa đi hơn là giúp ích.

Some of you may be looking at this “cleaner” code with a more skeptical eye. Perhaps you have some questions or complaints.

Vài người trong các bạn có thể đang nhìn đoạn mã “sạch hơn” này với ánh mắt hoài nghi hơn. Có lẽ bạn có vài thắc mắc hoặc phàn nàn.

You might complain that it's longer. That's true; but most of that extra length is due to the nicely named functions that explain what's going on.

Bạn có thể phàn nàn rằng nó dài hơn. Đúng vậy; nhưng phần lớn cái độ dài dôi ra đó là do những hàm được đặt tên tử tế, chúng giải thích chuyện gì đang diễn ra.

You might complain that the creation of an **object** is overkill and expensive. It's true that allocating memory, and then garbage-collecting it later, takes extra time. There is also time spent in the calling of all those small methods. If you are writing embedded real-time code with very tight timing constraints, or if you are writing code for a high-performance flight simulator or first-person shooter game, then this decision could seem insane, not to mention unclear. But for my purposes, the extra time is the least of my concerns. I don't need the speed. For my situation, I think trading speed for clarity, within reason, is a good trade to make.

Bạn có thể phàn nàn rằng việc tạo ra một đối tượng (object) là lố và tốn kém. Đúng là cấp phát bộ nhớ rồi sau đó dọn rác tốn thêm thời gian. Cũng mất thời gian cho việc gọi tất cả những method nhỏ đó nữa. Nếu bạn đang viết mã nhúng thời gian thực với ràng buộc thời gian cực ngặt, hoặc viết mã cho một trình mô phỏng bay hiệu năng cao hay một game bắn súng góc nhìn thứ nhất, thì quyết định này nghe có vẻ điên rồ, chưa kể là chẳng sạch sẽ gì. Nhưng với mục đích của tôi, chút thời gian dôi ra là mối bận tâm nhỏ nhất. Tôi không cần tốc độ. Trong hoàn cảnh của tôi, đánh đổi tốc độ lấy sự rõ ràng, trong chừng mực hợp lý, là một cuộc đổi chác đáng làm.

You might be a functional programmer horrified that the functions are not “pure.” But, in fact, the static convert function is as pure as a function can be. The others are just little helpers that operate within a single invocation of that overarching pure function. Those instance variables are very convenient for allowing the individual methods to communicate without having to resort to passing arguments. This shows that one good use for an object is to allow the helper functions that operate within the execution of a pure function to easily communicate through the instance variables of the object.

Bạn có thể là một lập trình viên hàm và thấy kinh hãi vì các hàm này không “thuần khiết”. Nhưng thật ra, hàm static convert thuần khiết hết mức mà một hàm có thể thuần khiết. Những hàm còn lại chỉ là mấy trợ thủ nhỏ hoạt động bên trong một lần gọi của cái hàm thuần khiết bao trùm đó. Các biến instance kia rất tiện để cho từng method giao tiếp với nhau mà không phải viện đến việc truyền đối số. Điều này cho thấy một công dụng tốt của đối tượng: cho phép các hàm trợ giúp chạy bên trong quá trình thực thi của một hàm thuần khiết để dàng trao đổi với nhau qua các biến instance của đối tượng.

So I think this is pretty clean. Am I right? I think I am; but you may disagree. That’s OK. We are allowed to disagree. There is no single standard for what cleanliness is. This is mine, and mine may not be yours. All I’ll ask of you is that you consider mine and develop your own standard. The most important thing is to have a standard that you apply with **discipline**.

Vậy nên tôi cho rằng thế này là khá sạch. Tôi đúng chứ? Tôi nghĩ là đúng; nhưng bạn có thể không đồng ý. Không sao cả. Chúng ta được phép bất đồng. Chẳng có một chuẩn mực duy nhất nào về thế nào là sạch. Đây là chuẩn của tôi, và chuẩn của tôi có thể không phải chuẩn của bạn. Tất cả những gì tôi mong ở bạn là hãy cân nhắc chuẩn của tôi rồi xây dựng chuẩn của riêng mình. Điều quan trọng nhất là có một chuẩn và áp dụng nó với kỷ luật (discipline).

If you are a member of a team, then the team should have a single standard of cleanliness that is negotiated, and then followed and supported, by all members of that team.

Nếu bạn là thành viên của một nhóm, thì cả nhóm nên có một chuẩn duy nhất về sự sạch sẽ, được thương lượng, rồi được mọi thành viên trong nhóm tuân theo và ủng hộ.

5 This might tempt you to create a standards document. I recommend otherwise. The document that represents the standard should be the code itself. The only reason to have a standards document outside of the code is if the code does not currently represent the standard.

5 Điều này có thể khiến bạn muốn soạn một tài liệu quy chuẩn. Tôi khuyên là đừng. Tài liệu đại diện cho chuẩn mực phải chính là mã nguồn. Lý do duy nhất để có một tài liệu quy chuẩn nằm ngoài mã là khi mã hiện tại chưa đại diện được cho chuẩn đó.

By the way, the tests I initially wrote were inadequate. As I worked through the cleaning process, I found a number of situations that needed further testing. For the sake of completeness, I’ve included the final tests below.

Nhân tiện, những test tôi viết ban đầu là chưa đủ. Trong lúc dọn dẹp, tôi phát hiện ra một loạt tình huống cần được kiểm thử thêm. Cho trọn vẹn, tôi để các test cuối cùng ở dưới đây.

```
package fromRoman;
import fromRoman.FromRoman.InvalidRomanNumeralException;
import org.junit.jupiter.api.Test;
import static fromRoman.FromRoman.convert;
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.is;
import static org.junit.jupiter.api.Assertions.assertThrows;
public class FromRomanTest {
    @Test
    public void valid() throws Exception {
        assertThat(convert(""), is(0));
        assertThat(convert("I"), is(1));
        assertThat(convert("II"), is(2));
        assertThat(convert("III"), is(3));
        assertThat(convert("IV"), is(4));
        assertThat(convert("V"), is(5));
        assertThat(convert("VI"), is(6));
        assertThat(convert("VII"), is(7));
        assertThat(convert("VIII"), is(8));
        assertThat(convert("IX"), is(9));
        assertThat(convert("X"), is(10));
        assertThat(convert("XI"), is(11));

        assertThat(convert("XII"), is(12));
        assertThat(convert("XIII"), is(13));
        assertThat(convert("XIV"), is(14));
        assertThat(convert("XV"), is(15));
        assertThat(convert("XVI"), is(16));
        assertThat(convert("XIX"), is(19));
        assertThat(convert("XX"), is(20));
        assertThat(convert("XXX"), is(30));
        assertThat(convert("XL"), is(40));
        assertThat(convert("L"), is(50));
        assertThat(convert("LX"), is(60));
        assertThat(convert("LXXIV"), is(74));
        assertThat(convert("XC"), is(90));
        assertThat(convert("C"), is(100));
        assertThat(convert("CXIV"), is(114));
        assertThat(convert("CXC"), is(190));
        assertThat(convert("CD"), is(400));
        assertThat(convert("D"), is(500));
        assertThat(convert("CDXLIV"), is(444));
        assertThat(convert("DCXCIV"), is(694));
        assertThat(convert("CM"), is(900));
        assertThat(convert("M"), is(1000));
        assertThat(convert("MCM"), is(1900));
        assertThat(convert("MCMXCIX"), is(1999));
    }
}
```

```

        assertThat(convert("MMXXIV"), is(2024));
    }
    @Test
    public void invalid() throws Exception {
        assertInvalid("IIII");
        assertInvalid("VV");
        assertInvalid("XXXX");
        assertInvalid("LL");
        assertInvalid("CCCC");
        assertInvalid("DD");
        assertInvalid("MMMM");
        assertInvalid("XIIII");
        assertInvalid("LXXXX");
        assertInvalid("DCCCC");
        assertInvalid("VIIII");
        assertInvalid("MCCCC");
        assertInvalid("VX");
        assertInvalid("IIV");
        assertInvalid("IVI");
        assertInvalid("IXI");
        assertInvalid("IXV");
        assertInvalid("VIV");
        assertInvalid("XVX");
        assertInvalid("XVV");
        assertInvalid("XIVI");
        assertInvalid("XIXI");
        assertInvalid("XVIV");
        assertInvalid("LXL");
        assertInvalid("XLX");
        assertInvalid("XCX");
        assertInvalid("XCL");

        assertInvalid("CDC");
        assertInvalid("DCD");
        assertInvalid("CMC");
        assertInvalid("CMD");
        assertInvalid("MCMC");
        assertInvalid("MCDM");
    }
    private void assertInvalid(String r) {
        assertThrows(InvalidRomanNumeralException.class,
            () -> convert(r));
    }
}

```

The sheer number of tests is a bit daunting. There is probably some redundancy in the valid tests. I'm reasonably sure that there is no redundancy in the invalid tests.

Riêng số lượng test thôi cũng đã hơi ngợp. Có lẽ vẫn còn dư thừa đâu đó trong nhóm test hợp lệ. Còn với nhóm test không hợp lệ thì tôi khá chắc là không có gì dư.

Future Bob: I am slightly annoyed that I did not

Future Bob: Tôi hơi bực vì đã không

find a more abstract **method** of determining and testing invalid conditions. I'm sure one exists.

tìm ra một cách trừu tượng hơn để xác định và kiểm thử các trường hợp không hợp lệ. Tôi chắc chắn là có một cách như vậy.

The Cleaning Process — Quy trình dọn dẹp

The process I used to first write and then clean that code was relatively simple. I'm going to walk you through each “major” decision; but overall it was just a matter of making smallish improvements while keeping all the tests passing.

Quy trình tôi dùng để viết rồi dọn dẹp đoạn mã đó khá đơn giản. Tôi sẽ dẫn bạn đi qua từng quyết định “lớn”; nhưng nhìn chung thì nó chỉ là chuyện thực hiện những cải thiện nho nhỏ trong khi giữ cho toàn bộ test luôn xanh.

The ugly code in Listing 2-1 was not my first attempt. Instead, I fumbled around with a few bad ideas. One was based on a state machine strategy that I cooked up in the shower. I actually wrote the following code and then threw it away before it got too far.

Đoạn mã xấu xí trong Listing 2-1 không phải là lần thử đầu tiên của tôi. Trước đó tôi đã mò mẫm với vài ý tưởng dở tệ. Một trong số đó dựa trên chiến lược máy trạng thái mà tôi nghĩ ra lúc đang tắm. Tôi thật sự đã viết đoạn mã sau rồi quăng nó đi trước khi nó kịp đi quá xa.

6 Linus Pauling was once asked how he had so many good ideas. His reply was something to the effect of he had lots of ideas and threw away the bad ones.

6 Linus Pauling từng được hỏi vì sao ông có nhiều ý tưởng hay đến thế. Ông trả lời đại ý rằng ông có rất nhiều ý tưởng và ông vứt bỏ những ý tưởng dở.

```
package fromRoman;
public class FromRoman {
    enum state {
        I, V, X, L, C, D, M
    }
    public static int convert(String roman) {
        int currentSum = 0;
        for (char c : roman.toCharArray()) {
            switch (c) {
                case 'I' -> {
                    if (currentSum == 3) {
                        throw new InvalidRomanNumeralException(roman);
                    }
                    currentSum += 1;
                }
                case 'V' -> currentSum += 5;
                case 'X' -> currentSum += 10;
                case 'L' -> currentSum += 50;
                case 'C' -> currentSum += 100;
                case 'D' -> currentSum += 500;
                case 'M' -> currentSum += 1000;
            }
        }
    }
}
```

```

    }
    return currentSum;
}
public static
class InvalidRomanNumeralException extends RuntimeException {
    public InvalidRomanNumeralException(String roman) {
    }
}
}
}

```

I finally settled on the strategy in Listing 2-1 , and then I massaged it into working, with all the tests passing.

Cuối cùng tôi chốt lại chiến lược trong Listing 2-1 , rồi nhào nặn cho tới khi nó chạy được với toàn bộ test đều xanh.

To clean this I looked for some low-hanging fruit. The first I found was the ugly code that checked for illegal prefix combinations. So I extracted that code out into the

Để dọn dẹp chỗ này, tôi đi tìm những quả ở cành thấp, dễ hái nhất. Cái đầu tiên tôi thấy là đoạn mã xấu xí kiểm tra các tổ hợp tiền tố không hợp lệ. Thế là tôi tách đoạn đó ra thành hàm

checkForIllegalPrefixCombinations function. Then, I noticed that there was a similar stretch of code, further down, that checked for improper repetitions of letters. So I extracted that code out as the checkForImproperRepetitions functions.

checkForIllegalPrefixCombinations. Sau đó, tôi để ý thấy có một đoạn mã tương tự ở phía dưới, kiểm tra việc lặp chữ cái sai quy tắc. Thế là tôi tách đoạn đó ra thành các hàm checkForImproperRepetitions.

In a similar fashion, I extracted out the replacement of the digraphs, the conversions of the letters to numbers, and the check for the numbers being in descending order.

Theo cách tương tự, tôi tách ra phần thay thế các digraph, phần chuyển chữ cái thành số, và phần kiểm tra xem các số có theo thứ tự giảm dần hay không.

This left me with the following code.

Việc này để lại cho tôi đoạn mã sau.

```

package fromRoman;
import java.util.Arrays;
public class FromRoman {

    public static int convert(String roman) {
        checkForIllegalPrefixCombinations(roman);
        checkForImproperRepetitions(roman);
        roman = replaceDigraphs(roman);
        int[] numbers = convertLettersToNumbers(roman);
        checkNumbersAreInOrder(numbers, roman);
        return Arrays.stream(numbers).sum();
    }
    private static
    void checkNumbersAreInOrder(int[] numbers, String roman) {
        int lastDigit = 1000;
        for (int number : numbers) {

```

```

        if (number > lastDigit) {
            throw new InvalidRomanNumeralException(roman);
        }
        lastDigit = number;
    }
}

private static int[] convertLettersToNumbers(String roman) {
    int[] numbers = new int[roman.length()];
    int i = 0;
    for (char digit : roman.toCharArray()) {
        switch (digit) {
            case 'I' -> numbers[i] = 1;
            case 'V' -> numbers[i] = 5;
            case 'X' -> numbers[i] = 10;

            case 'L' -> numbers[i] = 50;
            case 'C' -> numbers[i] = 100;
            case 'D' -> numbers[i] = 500;
            case 'M' -> numbers[i] = 1000;
            case '4' -> numbers[i] = 4;
            case '9' -> numbers[i] = 9;
            case 'F' -> numbers[i] = 40;
            case 'N' -> numbers[i] = 90;
            case 'G' -> numbers[i] = 400;
            case 'O' -> numbers[i] = 900;
            default -> throw new InvalidRomanNumeralException(roman);
        }
        i++;
    }
    return numbers;
}

private static String replaceDigraphs(String roman) {
    roman = roman.replace("IV", "4");
    roman = roman.replace("IX", "9");
    roman = roman.replace("XL", "F");
    roman = roman.replace("XC", "N");
    roman = roman.replace("CD", "G");
    roman = roman.replace("CM", "O");
    return roman;
}

private static void checkForImproperRepetitions(String roman) {
    if (roman.contains("IIII") ||
        roman.contains("VV") ||
        roman.contains("XXXX") ||
        roman.contains("LL") ||
        roman.contains("CCCC") ||
        roman.contains("DD") ||
        roman.contains("MMMM")) {
        throw new InvalidRomanNumeralException(roman);
    }
}

```



```

}
private static void checkForIllegalPrefixCombinations(String roma
    if (roman.contains("VIV") ||
        roman.contains("IVI") ||
        roman.contains("IXI") ||
        roman.contains("LXL") ||
        roman.contains("XLX") ||
        roman.contains("XCX") ||
        roman.contains("DCD") ||
        roman.contains("CDC") ||
        roman.contains("MCM")) {
        throw new InvalidRomanNumeralException(roman);
    }
}
public static
class InvalidRomanNumeralException extends RuntimeException {
    public InvalidRomanNumeralException(String roman) {
    }
}
}
}

```

I liked the way this made the convert function read. However, I didn't like passing all those arguments around to the helper functions. So I created the internal object with instance variables so that those helper functions could communicate without the overhead of passing arguments.

Tôi thích cách làm này khiến hàm convert đọc lên xuôi tai hẳn. Tuy nhiên, tôi không thích việc phải chuyển cả đồng đối số qua lại giữa các hàm trợ giúp. Vì vậy tôi tạo ra đối tượng nội bộ với các biến instance để những hàm trợ giúp ấy có thể trao đổi với nhau mà không phải gánh thêm chi phí truyền đối số.

I really didn't like the ad hoc “49FNGO” character replacement for the digraphs. That was just too clever.⁷ So I decided to merge the digraph processing and the conversions of letters to numbers into the same function. I did this one digraph at a time, keeping all the tests passing. This left things as follows.

Tôi thật sự không ưa cái trò thay ký tự “49FNGO” tùy tiện dành cho các digraph. Nó khôn lỏi quá mức.⁷ Vì vậy tôi quyết định gộp phần xử lý digraph và phần chuyển chữ cái thành số vào chung một hàm. Tôi làm từng digraph một, luôn giữ cho toàn bộ test xanh. Kết quả để lại như sau.

⁷ “Rule of thumb: if you think something is clever and sophisticated, beware—it is probably self-indulgence.” [DOET].

⁷ “Quy tắc bỏ túi: nếu bạn thấy thứ gì đó thông minh và tinh vi, hãy cẩn thận — nhiều khả năng đó chỉ là sự tự mãn.” [DOET].

```

package fromRoman;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
public class FromRoman {
    private String roman;
    List<Integer> numbers = new ArrayList<>();

```

```

Map<Character, Integer> values = Map.of(
    'I', 1,
    'V', 5,

    'X', 10,
    'L', 50,
    'C', 100,
    'D', 500,
    'M', 1000);
public FromRoman(String roman) {
    this.roman = roman;
}
public static int convert(String roman) {
    return new FromRoman(roman).doConversion();
}
private int doConversion() {
    checkForIllegalPrefixCombinations();
    checkForImproperRepetitions();
    convertToNumbers();
    checkNumbersAreInOrder();
    return numbers.stream().mapToInt(Integer::intValue).sum();
}
private void checkForIllegalPrefixCombinations() {
    if (roman.contains("VIV") ||
        roman.contains("IVI") ||
        roman.contains("IXI") ||
        roman.contains("LXL") ||
        roman.contains("XLX") ||

        roman.contains("XCX") ||
        roman.contains("DCD") ||
        roman.contains("CDC") ||
        roman.contains("MCM")) {
        throw new InvalidRomanNumeralException(roman);
    }
}
private void checkForImproperRepetitions() {
    for (String badRep : new String[]
        {"IIII", "VV", "XXXX", "LL",
         "CCCC", "DD", "MMMM"}) {
        if (roman.contains(badRep)) {
            throw new InvalidRomanNumeralException(roman);
        }
    }
}
private void convertToNumbers() {
    char[] chars = roman.toCharArray();
    int l = chars.length;
    for (int i = 0; i < l; i++) {
        char nextChar = i + 1 < l ? chars[i + 1] : 0;
        int nextValue = values.get(nextChar);
    }
}

```

```

switch (chars[i]) {
    case 'I' -> {
        if (nextChar == 'V' || nextChar == 'X') {
            numbers.add(nextValue - 1);
            i++;
        } else numbers.add(1);
    }
    case 'V' -> numbers.add(5);
    case 'X' -> {
        if (nextChar == 'L' || nextChar == 'C') {
            numbers.add(nextValue - 10);
            i++;
        } else numbers.add(10);
    }
    case 'L' -> numbers.add(50);
    case 'C' -> {
        if (nextChar == 'D' || nextChar == 'M') {
            numbers.add(nextValue - 100);
            i++;
        } else numbers.add(100);
    }
    case 'D' -> numbers.add(500);
    case 'M' -> numbers.add(1000);
    default -> throw new InvalidRomanNumeralException(roman);
}
}
}
private void checkNumbersAreInOrder() {
    int lastDigit = 1000;
    for (int number : numbers) {
        if (number > lastDigit) {
            throw new InvalidRomanNumeralException(roman);
        }
        lastDigit = number;
    }
}
public static
class InvalidRomanNumeralException extends RuntimeException {
    public InvalidRomanNumeralException(String roman) {
        super("Invalid Roman numeral: " + roman);
    }
}
}
}

```

After that, it was just a matter of reducing all the **duplication** in the cases and removing as many of the magic numbers as possible.

Sau đó, việc còn lại chỉ là giảm bớt mọi trùng lặp (duplication) trong các nhánh case và loại bỏ càng nhiều magic number càng tốt.

Then, I turned to the tests. They were pretty ad hoc. So I looked for patterns in the **test data** and

reorganized them. As part of this process, I found some missing invalid combinations and added them.

Rồi tôi quay sang các test. Chúng khá tùy tiện. Vậy nên tôi tìm các quy luật trong dữ liệu test và sắp xếp lại chúng. Trong quá trình đó, tôi phát hiện ra vài tổ hợp không hợp lệ còn thiếu và bổ sung vào.

That’s pretty much the whole story. I just made one simple change after the next, while keeping all the tests passing. This is pretty much the procedure that Martin Fowler recommends in his wonderful book, **Refactoring** . 8

Câu chuyện gần như chỉ có vậy. Tôi chỉ thực hiện hết thay đổi đơn giản này đến thay đổi đơn giản khác, trong khi luôn giữ cho toàn bộ test xanh. Đây gần như đúng là quy trình mà Martin Fowler khuyến nghị trong cuốn sách tuyệt vời của ông, **Refactoring** . 8

8 [Refactoring].

8 [Refactoring].

Conclusion — Kết luận

First, I made it work. Then, I made it right—or at least better.

Trước hết, tôi làm cho nó chạy. Sau đó, tôi làm cho nó đúng — hoặc chí ít là tốt hơn.

The following chapters in this part of the book are going to expand upon the iterative and incremental techniques I just described. We’ll see many other examples, in several other languages. We’ll look at the details of low-level coding; details like good naming, comments, and formatting. We’ll study higher-level techniques like **TDD**, Collaborative Programming, and Extract Till You Drop. Finally, we’ll look at slightly higher-level concepts such as **module** structure, objects, data structures, classes, and error handling.

Các chương tiếp theo trong phần này của cuốn sách sẽ khai triển thêm những kỹ thuật lặp và tăng dần mà tôi vừa mô tả. Chúng ta sẽ thấy nhiều ví dụ khác nữa, bằng vài ngôn ngữ khác. Chúng ta sẽ xem xét các chi tiết của việc viết mã ở tầng thấp; những chi tiết như đặt tên tốt, comment và định dạng. Chúng ta sẽ nghiên cứu các kỹ thuật ở tầng cao hơn như TDD, Collaborative Programming và Extract Till You Drop. Cuối cùng, chúng ta sẽ xem xét những khái niệm ở tầng cao hơn một chút như cấu trúc module, đối tượng, cấu trúc dữ liệu (data structure), class và xử lý lỗi.

Postscript: Future Bob Playing with Grok3 — Tái bút: Bob của tương lai nghịch với Grok3

All the reviews have come back, and I am now preparing the final manuscript for publication. It is February of 2025, and Grok3 just came out. So I fed my “clean” version of the above code into Grok3 and asked it to make improvements. After a bit of thought, it returned the following code, which compiled and which passed all my tests.

Mọi phản hồi từ khâu review đã về đến nơi, và giờ tôi đang chuẩn bị bản thảo cuối cùng để đưa in. Bây giờ là tháng Hai năm 2025, và Grok3 vừa mới ra mắt. Thế là tôi nạp phiên bản “sạch” của đoạn mã ở trên vào Grok3 rồi bảo nó cải thiện giúp. Sau một hồi ngẫm nghĩ, nó trả về đoạn mã sau — biên dịch được, và vượt qua toàn bộ test của tôi.

```

package fromRoman;
import java.util.Map;
public class FromRoman {
    private String roman;
    private static final String REGEX =
        "^M{0,3}(CM|CD|D?C{0,3})(XC|XL|L?X{0,3})(IX|IV|V?I{0,3})$";
    private static final Map<Character, Integer> VALUES = Map.of(
        'I', 1,
        'V', 5,
        'X', 10,
        'L', 50,
        'C', 100,
        'D', 500,
        'M', 1000
    );
    /** Constructs a FromRoman instance,
        validating the input Roman numeral. */
    public FromRoman(String roman) {
        if (!roman.matches(REGEX)) {
            throw new InvalidRomanNumeralException(roman);
        }
        this.roman = roman;
    }
    /** Converts the Roman numeral to an integer. */
    public int toInt() {
        int sum = 0;
        for (int i = 0; i < roman.length(); i++) {
            int currentValue = VALUES.get(roman.charAt(i));
            if (i < roman.length() - 1) {
                int nextValue = VALUES.get(roman.charAt(i + 1));
                if (currentValue < nextValue) {
                    sum -= currentValue;
                } else {
                    sum += currentValue;
                }
            } else {
                sum += currentValue;
            }
        }
        return sum;
    }
    /** Static method to convert a Roman numeral string to an integ
    public static int convert(String roman) {
        return new FromRoman(roman).toInt();
    }
    public static
    class InvalidRomanNumeralException extends RuntimeException {
        public InvalidRomanNumeralException(String roman) {
            super("Invalid Roman numeral: " + roman);
        }
    }
}

```

```

    }
}
}

```

I'm quite pleased with the improvement to the algorithm. It really reduces things to the bare essence. I'm not a big fan of using regular expressions, but this one is not too hard to understand. I don't find any of the comments helpful, so I'd likely delete them.

Tôi khá hài lòng với phần cải tiến thuật toán. Nó thực sự rút gọn mọi thứ về đúng phần cốt lõi trần trụi. Tôi vốn không mê dùng regular expression cho lắm, nhưng cái này thì cũng không đến nỗi khó hiểu. Còn đám comment thì tôi thấy chẳng cái nào hữu ích cả, nên chắc tôi sẽ xoá hết.

Then, I asked it to improve my tests. Here's what it came up with:

Sau đó, tôi nhờ nó cải thiện luôn mấy cái test của tôi. Và đây là thứ nó nghĩ ra:

```

package fromRoman;

import org.junit.jupiter.api.Test;
import static fromRoman.FromRoman.convert;
import static org.hamcrest.MatcherAssert.assertThat;
import static org.hamcrest.Matchers.is;
import static org.junit.jupiter.api.Assertions.assertThrows;
public class FromRomanTest {
    @Test
    public void valid() {
        // Edge case
        assertThat(convert(""), is(0));
        // Single characters
        assertThat(convert("I"), is(1));
        assertThat(convert("V"), is(5));
        assertThat(convert("X"), is(10));
        assertThat(convert("L"), is(50));
        assertThat(convert("C"), is(100));
        assertThat(convert("D"), is(500));
        assertThat(convert("M"), is(1000));
        // Subtractive notation
        assertThat(convert("IV"), is(4));
        assertThat(convert("IX"), is(9));
        assertThat(convert("XL"), is(40));
        assertThat(convert("XC"), is(90));
        assertThat(convert("CD"), is(400));
        assertThat(convert("CM"), is(900));
        // Complex cases
        assertThat(convert("XIV"), is(14));
        assertThat(convert("CDXLIV"), is(444));
        assertThat(convert("MCMXCIX"), is(1999));
    }
    @Test
    public void invalid() {
        // Too many repeats
        assertInvalid("IIII");
    }
}

```

```

    assertInvalid("VV");
    assertInvalid("MMMM");
    // Invalid subtractive notation
    assertInvalid("VX"); // V can't precede X
    assertInvalid("IIV"); // Incorrect order
    assertInvalid("IXI"); // Invalid sequence
    // Improper structure
    assertInvalid("CMC"); // Multiple subtractive errors
    assertInvalid("XCL"); // Incorrect order
}
private void assertInvalid(String r) {
    assertThrows(FromRoman.InvalidRomanNumeralException.class,
        () -> convert(r));
}
}

```

I like that it organized them so nicely. I remember struggling to find that organization and eventually just deciding to flood the zone. So, kudos to Grok3 for that. On the other hand, the tests are strongly relying on the regular expression and so does, what I consider to be, an inadequate job of testing invalid inputs. For example, it does not check that “IM” is invalid. So I’d likely add a few more tests for things like that.

Tôi thích cách nó sắp xếp chúng gọn gàng đến vậy. Tôi vẫn nhớ mình đã vật lộn thế nào để tìm ra cách tổ chức đó, rồi cuối cùng đành quăng thật nhiều test vào cho phủ kín. Vậy nên, xin ngả mũ trước Grok3 ở điểm này. Mặt khác, các test lại bám quá chặt vào regular expression, và vì thế — theo tôi — chúng kiểm thử các đầu vào không hợp lệ một cách chưa tới nơi tới chốn. Chẳng hạn, chúng không kiểm tra rằng “IM” là không hợp lệ. Nên chắc tôi sẽ thêm vài test nữa cho những trường hợp kiểu đó.

Postscript Conclusion — Kết luận cho phần tái bút

Grok3 did a nice job. One might therefore question whether my effort of cleaning and testing was worth it. On the other hand, I, and not Grok3, am the final arbiter of the code that gets produced. The buck stops with me. I’m pleased that Grok3 was able to help, and I will be happy of that help when I ask for it. But in order to know that Grok3, or any **LLM**/AI, is giving me good code, it remains incumbent upon me to do the work to ensure that I can evaluate what it produces. Doing that work will usually involve writing and cleaning the code that I ask it to improve so that I can properly judge the outcome.

Grok3 đã làm rất tốt. Vậy thì người ta có thể đặt câu hỏi: công sức tôi bỏ ra để dọn dẹp và viết test liệu có đáng không? Mặt khác, người phán quyết cuối cùng về đoạn mã được sinh ra là tôi, chứ không phải Grok3. Trách nhiệm dừng lại ở tôi. Tôi vui vì Grok3 giúp được, và tôi sẽ vui vẻ đón nhận sự trợ giúp đó mỗi khi tôi cần đến. Nhưng để biết được rằng Grok3 — hay bất kỳ LLM/AI nào — đang đưa cho mình mã tốt, thì tôi vẫn phải tự mình làm phần việc bảo đảm rằng tôi đủ sức đánh giá thứ nó tạo ra. Và phần việc ấy thường bao gồm chuyện tự viết rồi tự dọn sạch chính đoạn mã mà tôi nhờ nó cải thiện, để tôi có thể phán xét kết quả một cách đúng đắn.

The trade-off is very similar to using an autopilot in an airplane. The autopilot can vastly reduce the workload. But under no circumstances should it be implicitly trusted. Rather, the pilot must continuously oversee its operation. By the same token, no pilot should ever fly without possessing

the necessary flying skills that they continuously maintain and refine.

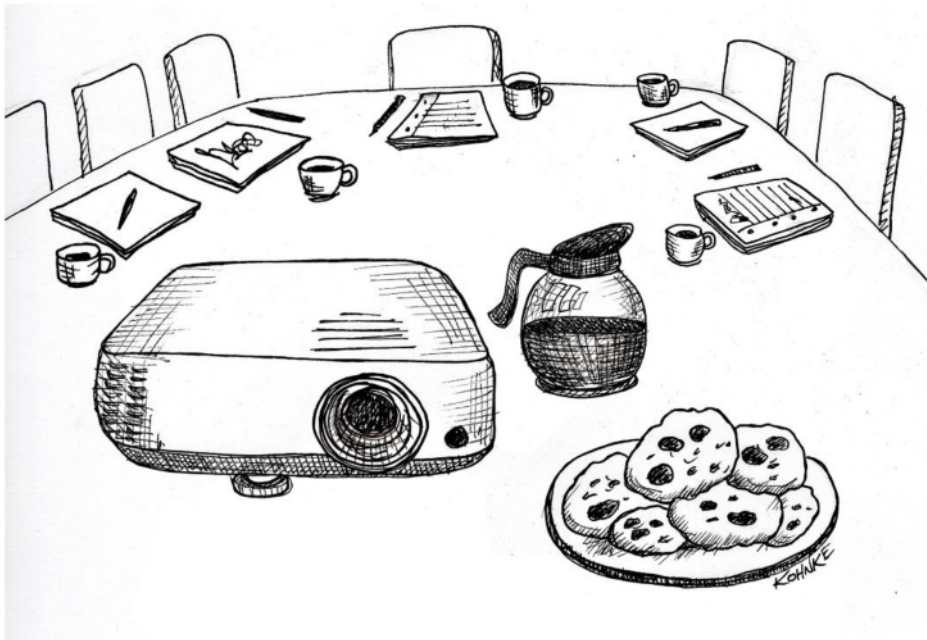
Sự đánh đổi ở đây rất giống với việc dùng chế độ lái tự động trên máy bay. Autopilot có thể giảm tải công việc đi rất nhiều. Nhưng tuyệt đối không được mặc nhiên tin tưởng nó. Trái lại, phi công phải liên tục giám sát hoạt động của nó. Cũng theo lẽ đó, không phi công nào được phép cất cánh mà thiếu những kỹ năng bay cần thiết — những kỹ năng mà họ phải không ngừng duy trì và mài giũa.

So it is with us. We must stay in practice, and keep our skills high, because we must be able to competently oversee the automation that reduces our workload.

Với chúng ta cũng vậy. Chúng ta phải giữ tay nghề luôn ở phong độ và giữ kỹ năng luôn ở mức cao, bởi vì chúng ta cần đủ năng lực để giám sát chính cái công cụ tự động hoá đang giảm tải cho mình.

Chapter 3. First Principles —

Chương 3. Những nguyên tắc nền tảng



If you read this chapter and grasp all the principles within, then you'll be 90% of the way toward cleaning code. The ideas presented here are old and well established. They have stood the **test** of time and should be part of every programmer's regimen of **discipline**.

Nếu bạn đọc chương này và nắm được toàn bộ các nguyên tắc bên trong, bạn đã đi được 90% chặng đường tới chỗ viết mã sạch. Những ý tưởng trình bày ở đây đều cũ và đã được thừa nhận rộng rãi. Chúng đã vượt qua thử thách của thời gian và xứng đáng nằm trong bộ kỷ luật (discipline) rèn luyện của mọi lập trình viên.

Keep in mind that these are not strict laws. They are not rules that must be followed. They are (at the risk of using a trite expression) “more what you'd call guidelines.”¹ Each should be applied in the context of the problem at hand, and every programmer should feel free to take whatever license is necessary to solve that problem.

Hãy nhớ rằng đây không phải là luật lệ cứng nhắc. Chúng không phải những quy tắc bắt buộc phải tuân theo. Chúng (xin mạo muội mượn một câu đã sáo mòn) “đúng hơn thì phải

gọi là chỉ dẫn tham khảo.” 1 Mỗi nguyên tắc cần được áp dụng trong bối cảnh của bài toán trước mắt, và mọi lập trình viên đều có quyền tự do phá lệ đến mức cần thiết để giải xong bài toán đó.

1 Hector Barbossa, *Pirates of the Caribbean*.

1 Hector Barbossa, *Pirates of the Caribbean*.

But, when all else is equal, and all the variables are averaged out, these are the principles that I believe emanate from well-cleaned code.

Nhưng khi mọi thứ khác là như nhau, và mọi biến số đã được san bằng, thì đây chính là những nguyên tắc mà tôi tin là toát ra từ mã đã được dọn dẹp kỹ càng.

If, as you read what follows, you encounter concepts that are foreign or novel, be assured that all will be explained in subsequent chapters. Indeed, when you have finished reading this book, you may wish to complete the circle by rereading this chapter.

Nếu trong lúc đọc phần tiếp theo, bạn gặp những khái niệm xa lạ hoặc mới mẻ, cứ yên tâm rằng tất cả sẽ được giải thích ở các chương sau. Thật vậy, khi đọc xong cuốn sách này, có lẽ bạn sẽ muốn khép lại vòng tròn bằng cách đọc lại chương này.

Everything Small, Well Named, Organized, and Ordered — Mọi thứ đều nhỏ, được đặt tên tốt, được tổ chức và sắp xếp

If you stop now and read nothing more than that heading, you'll have much of the essence of what it takes to **clean code**.

Nếu bạn dừng lại ngay bây giờ và không đọc gì thêm ngoài cái tiêu đề đó, bạn cũng đã nắm được phần lớn tinh thần của việc viết clean code.

Keep everything small. Choose names for those small elements that communicate to other programmers. Define and maintain a **structure** and organization that present the software in such a way that others can easily consume it. Keep the elements within that organization well-ordered so that one concept follows another in a rational and sensible way.

Hãy giữ cho mọi thứ đều nhỏ. Hãy chọn cho những phần tử nhỏ ấy những cái tên nói lên điều gì đó với các lập trình viên khác. Hãy định ra và duy trì một cấu trúc, một cách tổ chức trình bày phần mềm sao cho người khác dễ dàng tiếp thu. Hãy giữ cho các phần tử bên trong cách tổ chức ấy được sắp xếp ngăn nắp, để khái niệm này nối tiếp khái niệm kia một cách hợp lý và có lý lẽ.

Remember that your own understanding of the code will likely bias your attempts at providing **good** names and structures; so work hard to put yourself in the headspace of those who do not understand the code.

Hãy nhớ rằng chính sự hiểu biết của bạn về mã sẽ làm thiên lệch các nỗ lực đặt tên và dựng cấu trúc; vậy nên hãy cố hết sức đặt mình vào đầu óc của những người chưa hiểu gì về đoạn mã đó.

Functions — Hàm

Functions should be small. Most should be just a handful of lines. This allows them to have nice, descriptive names and promotes the separation of concerns. It ensures that each **function** does one,

and only one, thing.

Hàm nên nhỏ. Phần lớn hàm chỉ nên vón vện vài dòng. Điều này cho phép chúng có những cái tên đẹp đẽ, mô tả rõ ràng, đồng thời thúc đẩy việc tách bạch các mối quan tâm. Nó bảo đảm rằng mỗi hàm chỉ làm một việc, và duy nhất một việc.

Let's start with the name. If you see a snippet of lines that does something that can be reasonably named, then that snippet should be moved into a function named for what that snippet does. The name should be a verb that says what the function does and that allows the calling sequence to read "like well-written prose." 2

Hãy bắt đầu từ cái tên. Nếu bạn thấy một mẫu vài dòng mã làm một việc có thể đặt tên được một cách hợp lý, thì mẫu mã đó nên được chuyển vào một hàm mang tên đúng việc nó làm. Cái tên phải là một động từ nói lên hàm làm gì, và phải khiến chuỗi lời gọi đọc lên "như một áng văn xuôi viết khéo." 2

2 Grady Booch, private email.

2 Grady Booch, email riêng.

As an example, consider this function, adapted from the

Lấy ví dụ, hãy xem hàm sau đây, phỏng theo thư viện

JCommon library. It finds the date of a specified weekday before a given date. The weekday codes are integers 1 (Sunday) through 7 (Saturday).

JCommon. Nó tìm ngày ứng với một thứ trong tuần được chỉ định, nằm trước một ngày cho trước. Mã thứ trong tuần là các số nguyên từ 1 (Chủ nhật) đến 7 (Thứ bảy).

```
public static Date getPreviousDayOfWeek(int weekday, Date from) {
    // check arguments...
    if (!isValidWeekdayCode(weekday)) {
        throw new IllegalArgumentException(
            "Invalid day-of-the-week code."
        );
    }
    // find the date...
    final int adjust;
    final int baseDOW = from.getDayOfWeek();
    if (baseDOW > weekday) {
        adjust = Math.min(0, weekday - baseDOW);
    } else {
        adjust = -7 + Math.max(0, weekday - baseDOW);
    }
    return Date.addDays(adjust, from);
}
```

There are at least two snippets of code that we could extract into smaller and better-named functions. And that extraction will lead to some simplifications.

Có ít nhất hai mẫu mã mà ta có thể tách ra thành các hàm nhỏ hơn và được đặt tên tốt hơn. Và việc tách đó sẽ dẫn tới vài chỗ đơn giản đi.

```
public static Date getPreviousDayOfWeek(int weekday, Date from) {
    checkWeekdayArgument(weekday);
    return addDays(-daysBefore(weekday, from), from);
}
```

```

}
private static void checkWeekdayArgument(int weekday) {
    if (!isValidWeekdayCode(weekday))
        throw new IllegalArgumentException("Invalid day-of-the-week cod
}
private static int daysBefore(int targetWeekday, Date from) {
    int fromWeekday = from.getDayOfWeek();
    int diff = fromWeekday - targetWeekday;
    return diff + (diff > 0 ? 0 : 7);
}

```

Notice how the simplified `getPreviousDayOfWeek` function reads. It's short, and it's obvious. If you need to know more, all you have to do is look down to see the extracted functions. However, most often you would not need to know more. You'd simply look at that function, agree with it, and move on. It may not read like the prose of a Crichton novel, but it reads pretty well.

Hãy để ý hàm `getPreviousDayOfWeek` sau khi đơn giản hoá đọc lên thế nào. Nó ngắn, và nó hiển nhiên. Nếu cần biết thêm, bạn chỉ việc nhìn xuống dưới để xem các hàm đã tách ra. Tuy vậy, hầu hết thời gian bạn sẽ chẳng cần biết thêm. Bạn chỉ liếc qua hàm đó, gật đầu đồng ý, rồi đi tiếp. Nó có thể không đọc mượt như văn của một cuốn tiểu thuyết Crichton, nhưng đọc cũng khá xuôi.

Future Bob: I don't like the leading minus sign. It

Future Bob: Tôi không thích cái dấu trừ đứng ở đầu. Lẽ ra

would have been better to create a `subtractDays` function in order to get rid of it. That's a small thing, but small things matter.

tạo một hàm `subtractDays` để tổng khứ nó đi thì hay hơn. Chuyện nhỏ thôi, nhưng chính những chuyện nhỏ mới đáng kể.

A More Significant Example — Một ví dụ đáng kể hơn

Here I present the Uncle Bob Conference Room system. Scan through it. It's not very complicated—probably.

Ở đây tôi giới thiệu hệ thống Phòng họp Uncle Bob. Hãy lướt qua nó. Nó không phức tạp lắm đâu — chắc vậy.

—Statement.java—

```

package ubConferenceCenter;
import java.util.ArrayList;
import java.util.List;
public class Statement {
    public enum CatalogItem {SMALL_ROOM, LARGE_ROOM,
                            PROJECTOR, COFFEE, COOKIES}
    public record RentalItem(CatalogItem type,
                            int days,
                            int unitPrice,
                            int price,
                            int tax) {
    }
}

```

```

public record Totals(int subtotal, int tax) {
}
private String customerName;
private int subtotal = 0;
private int tax = 0;
private List<RentalItem> items = new ArrayList<>();
public Statement(String customerName) {
    this.customerName = customerName;
}
public void rent(CatalogItem item, int days) {
    int unitPrice = switch (item) {
        case SMALL_ROOM -> 100;
        case LARGE_ROOM -> 150;
        case PROJECTOR -> 50;
        case COFFEE -> 10;
        case COOKIES -> 15;
    };
    boolean eligibleForDiscount = switch (item) {
        case SMALL_ROOM, LARGE_ROOM -> days == 5;
        case PROJECTOR, COFFEE, COOKIES -> false;
    };
    int price = unitPrice * days;
    if (eligibleForDiscount) price = (int) Math.round(price * .9);
    subtotal += price;
    int thisTax = switch (item) {
        case SMALL_ROOM, LARGE_ROOM, PROJECTOR ->
            (int) Math.round(price * .05);
        case COFFEE, COOKIES -> 0;
    };
    tax += thisTax;
    items.add(new RentalItem(item, days, unitPrice, price, thisTax));
}
public RentalItem[] getItems() {
    List<RentalItem> items = new ArrayList<>(this.items);
    boolean largeRoomFiveDays = items.stream().anyMatch(
        item -> item.type() == CatalogItem.LARGE_ROOM && item.days() == 5);
    boolean coffeeFiveDays = items.stream().anyMatch(
        item -> item.type() == CatalogItem.COFFEE && item.days() == 5);
    if (largeRoomFiveDays && coffeeFiveDays)
        items.add(new RentalItem(CatalogItem.COOKIES, 5, 0, 0, 0));
    return items.toArray(new RentalItem[0]);
}
public String getCustomerName() {
    return customerName;
}
public Totals getTotals() {
    return new Totals(subtotal, tax);
}
}

```

I'm sure you figured it out. Users can rent small rooms, large rooms, coffee, projectors, and cookies. (How do you rent cookies?) Small rooms rent for \$100 per day. Large rooms are \$150. You get a 10% discount if you rent a room for the whole week. There's a 5% (rounded up) tax on all nonfood items. Coffee is \$10 per day. Cookies are \$15 per day. Projectors are \$50 per day. And if you rent a large room and coffee for a whole week, you get one day of free cookies.

Tôi chắc bạn đã đoán ra rồi. Khách hàng có thể thuê phòng nhỏ, phòng lớn, cà phê, máy chiếu và bánh quy. (Thuê bánh quy kiểu gì nhỉ?) Phòng nhỏ giá 100 đô một ngày. Phòng lớn 150 đô. Bạn được giảm 10% nếu thuê phòng trọn tuần. Mọi món không phải đồ ăn đều chịu thuế 5% (làm tròn lên). Cà phê 10 đô một ngày. Bánh quy 15 đô một ngày. Máy chiếu 50 đô một ngày. Và nếu thuê phòng lớn kèm cà phê trọn tuần, bạn được tặng một ngày bánh quy miễn phí.

The Statement **class** represents a sales statement that includes each item rented, the price of the item, the number of days rented, the subtotal, and the tax. The Statement also contains the subtotals of all the items and all the tax.

Class Statement biểu diễn một hoá đơn bán hàng, gồm từng món đã thuê, giá của món đó, số ngày thuê, thành tiền và thuế. Statement cũng chứa tổng phụ của tất cả các món và tổng thuế.

Easy peasy.

Dễ ợt.

This is not too hard to understand, but you can tell that it was slapped together. It's a bit unkempt, but not terrible. We could probably clean it up a bit, but would that be worth the effort?

Chuyện này không quá khó hiểu, nhưng bạn thấy ngay là nó được ghép lại một cách qua quýt. Hơi luộm thuộm, nhưng chưa đến mức tệ hại. Chắc ta cũng có thể dọn dẹp nó một chút, nhưng liệu có bỏ công không?

But let me ask you. Have you ever seen a system that started out simple and then got much, much more complicated? If you've been in this business for more than a year or so, I know you have. Indeed, if you've been in the business for five years or more, you've likely seen a system that began a bit unkempt but grew into a horrible **mess**, becoming a living hell for the developers and the organization.

Nhưng để tôi hỏi bạn. Bạn đã bao giờ thấy một hệ thống khởi đầu đơn giản rồi trở nên phức tạp hơn rất, rất nhiều chưa? Nếu bạn đã làm nghề này hơn một năm, tôi biết là bạn đã thấy rồi. Thật ra, nếu bạn đã làm nghề từ năm năm trở lên, hẳn bạn từng chứng kiến một hệ thống bắt đầu hơi luộm thuộm rồi lớn lên thành một mớ hỗn độn kinh hoàng, biến thành địa ngục sống cho cả đội phát triển lẫn cả tổ chức.

So let's make this example a bit more real. You don't suppose that this program started out in its current form, do you? No, at first it just rented one small room. Then, many small rooms. Then, large rooms with a different unit price. Then, coffee, with no sales tax. Then, the discount for a full week was added. And then cookies, and the bonus for renting a large room for a week.

Vậy hãy làm cho ví dụ này thực tế hơn một chút. Bạn không nghĩ là chương trình này ngay từ đầu đã có hình hài như bây giờ đấy chứ? Không đâu, thoát tiên nó chỉ cho thuê một phòng nhỏ. Rồi nhiều phòng nhỏ. Rồi phòng lớn với đơn giá khác. Rồi cà phê, không chịu thuế bán hàng. Rồi khoản giảm giá cho thuê trọn tuần được thêm vào. Rồi bánh quy, rồi quà tặng cho việc thuê phòng lớn trọn tuần.

In other words, this program grew.

Nói cách khác, chương trình này đã lớn lên.

Let's suppose that we've just gotten back from an all-hands meeting at which the CEO gave us a glowing report about how the company is expanding. There are new markets, new states, new laws to contend with. There will be new discounts and new promotions, and new tax regulations. It's going to be great for the business; but you look at this little

Hãy giả sử ta vừa từ một buổi họp toàn công ty trở về, nơi CEO đọc một báo cáo hoành tráng về chuyện công ty đang bành trướng ra sao. Sẽ có thị trường mới, bang mới, luật lệ mới phải đối phó. Sẽ có các khoản giảm giá mới, chương trình khuyến mãi mới, quy định thuế mới. Chuyện đó tuyệt vời cho công việc kinh doanh; nhưng rồi bạn nhìn cái

rent function and you think . . . it's going to continue to grow, isn't it? And as it grows, it will become ever more tangled and confusing. It will degrade, like a piece of rotting meat.

hàm rent bé nhỏ này và nghĩ... nó sẽ còn phình to nữa, đúng không? Và càng phình to, nó sẽ càng rối rắm và khó hiểu. Nó sẽ suy đồi, như một miếng thịt đang thối rữa.

So let's stop that from happening. Let's figure out a way to get ahead of the coming business growth and let this function grow without rotting from the inside out. Let's tidy it up first. 3

Vậy hãy ngăn điều đó xảy ra. Hãy tìm cách đi trước một bước so với đà tăng trưởng sắp tới của công việc kinh doanh, để hàm này lớn lên mà không mục ruỗng từ trong ra ngoài. Trước hết, hãy dọn dẹp nó cho gọn gàng. 3

3 [Tidy].

3 [Tidy].

Look back at the rent function and ask yourself what you don't like about it. Personally, I think it's a bit large and disorganized. It does several things that I can pull apart into functions that do one thing.

Hãy nhìn lại hàm rent và tự hỏi bạn không thích điều gì ở nó. Riêng tôi, tôi thấy nó hơi to và lộn xộn. Nó làm vài việc khác nhau mà tôi có thể tách ra thành những hàm mỗi hàm chỉ làm một việc.

So first, let's just use the **extract method refactoring** to create functions that do one thing, and gather together the things that are related and separate the things that are different. 4

Vậy trước hết, hãy dùng refactoring extract method để tạo ra các hàm chỉ làm một việc, gom những thứ liên quan lại với nhau và tách những thứ khác nhau ra. 4

4 The Single Responsibility Principle.

4 Nguyên tắc Trách nhiệm Đơn nhất (the Single Responsibility Principle).

—Statement.java—

...

```
public void rent(CatalogItem item, int days) {
    int unitPrice = getUnitPrice(item);
    int price = calculatePrice(item, days, unitPrice);
    int thisTax = getTax(item, price);
    items.add(new RentalItem(item, days, unitPrice, price, thisTax
    subtotal += price;
    tax += thisTax;
}
private int getUnitPrice(CatalogItem item) {
```



```

    return switch (item) {
        case SMALL_ROOM -> 100;
        case LARGE_ROOM -> 150;
        case PROJECTOR -> 50;
        case COFFEE -> 10;
        case COOKIES -> 15;
    };
}
private int calculatePrice(CatalogItem item, int days, int unitP
    boolean eligibleForDiscount = isEligibleForDiscount(item, days
    int price = unitPrice * days;
    if (eligibleForDiscount) price = (int) Math.round(price * .9);
    return price;
}
private boolean isEligibleForDiscount(CatalogItem item, int days
    return switch (item) {
        case SMALL_ROOM, LARGE_ROOM -> days == 5;
        case PROJECTOR, COFFEE, COOKIES-> false;
    };
}
private int getTax(CatalogItem item, int price) {
    return switch (item) {
        case SMALL_ROOM, LARGE_ROOM, PROJECTOR ->
            (int) Math.round(price * .05);
        case COFFEE, COOKIES -> 0;
    };
}
}

```

Future Bob: Having used Grok3 on the

Future Bob: Đã dùng Grok3 cho

module in the last chapter, I thought I'd do the same here. So I told Grok3 to improve my initial

module ở chương trước, tôi nghĩ sao không làm y như vậy ở đây. Thế là tôi bảo Grok3 cải thiện bản

implementation. It was not particularly surprising that its solution was nearly identical to mine.

hiện thực (implementation) ban đầu của mình. Cũng chẳng có gì đặc biệt bất ngờ khi lời giải của nó gần như giống hệt lời giải của tôi.

Perhaps you look at this and think that it's more code. But it's not more executable code; it's just more names and more structure. That extra structure makes room for growth. For example, if the tax rules become more complicated, it is not the rent function that will grow; instead, in all likelihood, it will be the getTax function that grows.

Có lẽ nhìn vào đây bạn sẽ nghĩ rằng như vậy là nhiều mã hơn. Nhưng nó không phải là nhiều mã thực thi hơn; nó chỉ là nhiều tên gọi hơn và nhiều cấu trúc hơn. Chính phần cấu trúc thêm vào ấy chứa chỗ cho sự tăng trưởng. Chẳng hạn, nếu các quy tắc thuế trở nên rắc rối hơn, thì hàm phình to ra sẽ không phải là rent; mà nhiều khả năng nhất, chính getTax mới là hàm phình to.

This is an example of the Single Responsibility Principle (SRP) 5 in action. The stakeholders of our

system who are most concerned about taxation will most probably be the only ones asking for changes to the `getTax` function, whereas the stakeholders who are interested in discounts will most likely be the only ones affecting the `calculatePrice` and

Đây là một ví dụ về Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle — SRP) 5 đang vận hành. Những bên liên quan (stakeholder) của hệ thống quan tâm nhất đến chuyện thuế má nhiều khả năng sẽ là những người duy nhất yêu cầu thay đổi hàm `getTax`, trong khi các bên liên quan quan tâm đến giảm giá nhiều khả năng sẽ là những người duy nhất tác động tới `calculatePrice` và

`isEligibleForDiscount` functions.

5 See Chapter 19, “**The SOLID Principles**.”

5 Xem Chương 19, “Các nguyên tắc SOLID.”

This is helpful to us because, after this change, we are not very likely to break the discounts when the taxes change, or break the taxes when the price calculation changes. The farther we can keep these different stakeholders’ responsibilities apart and isolated, the safer our code will be when changes are made.

Điều này có ích cho ta, bởi sau thay đổi này, ta rất ít khả năng làm hỏng phần giảm giá khi thuế thay đổi, hay làm hỏng phần thuế khi cách tính giá thay đổi. Càng giữ được trách nhiệm của các bên liên quan khác nhau tách bạch và cô lập, mã của ta càng an toàn mỗi khi có thay đổi.

If we ignore the SRP, then we risk the symptom of fragility. A fragile system breaks in unexpected ways; for example, when a **stakeholder** asks us to modify taxes and we inadvertently break discounts. Such occurrences are terrifying to our stakeholders, managers, and users because it gives them a glimpse of what’s under the hood; and they don’t like what they see.

Nếu phớt lờ SRP, ta chuốc lấy nguy cơ mắc triệu chứng dễ vỡ. Một hệ thống dễ vỡ sẽ hỏng theo những cách không ai ngờ tới; ví dụ, khi một bên liên quan yêu cầu ta sửa phần thuế mà ta lại vô tình làm hỏng phần giảm giá. Những chuyện như thế khiến các bên liên quan, quản lý và người dùng hoảng sợ, vì nó cho họ liếc thấy những gì đang diễn ra dưới nắp ca-pô; và họ không thích thứ họ nhìn thấy.

Our stakeholders have the right to expect that a change to taxes will not break discounts. When we violate that expectation, the only conclusion they can draw is that we’ve lost control of the system and don’t really know what the hell we are doing.

Các bên liên quan có quyền kỳ vọng rằng một thay đổi về thuế sẽ không làm hỏng phần giảm giá. Khi ta phá vỡ kỳ vọng đó, kết luận duy nhất họ có thể rút ra là ta đã mất kiểm soát hệ thống và chẳng thực sự biết mình đang làm cái quái gì.

Now look at that new code. What don’t you like about it? One thing we might focus on are those switch statements.

Giờ hãy nhìn đoạn mã mới. Bạn không thích điều gì ở nó? Một thứ ta có thể để mắt tới là mấy câu lệnh switch kia.

Switch statements aren’t intrinsically bad. However, if the number of cases is likely to grow, then they violate the Open–Closed Principle (OCP).

Câu lệnh switch tự thân nó không xấu. Tuy nhiên, nếu số lượng case có khả năng tăng lên, thì chúng vi phạm Nguyên tắc Đóng–Mở (Open–Closed Principle — OCP).

The OCP suggests that when we add a new feature, we should add that new feature in one place, not in many places. Right now, if we were to add a new `CatalogItem`, like

OCF gợi ý rằng khi thêm một tính năng mới, ta nên thêm tính năng ấy vào một chỗ, chứ không phải nhiều chỗ. Ngay lúc này, nếu ta thêm một `CatalogItem` mới, chẳng hạn

`NotePads`, we'd have to add those changes to as many as five different places in the code. We can reduce this down to two by turning the `CatalogItem`s into **data** structures and replacing the switch statements with accesses of those data structures.

`NotePads`, ta sẽ phải rắc những thay đổi đó vào tận năm chỗ khác nhau trong mã. Ta có thể rút con số ấy xuống còn hai, bằng cách biến các `CatalogItem` thành cấu trúc dữ liệu và thay các câu lệnh switch bằng những lần truy cập vào chính các cấu trúc dữ liệu đó.

—`Statement.java`—

```
...
public enum CatalogItem {
    SMALL_ROOM(100, .05),
    LARGE_ROOM(150, .05),
    PROJECTOR(50, .05),
    COFFEE(10, 0),
    COOKIES(15, 0);
    private double taxRate;
    private int unitPrice;
    CatalogItem(int unitPrice, double taxRate) {
        this.unitPrice = unitPrice;
        this.taxRate = taxRate;
    }
}
...
public void rent(CatalogItem item, int days) {
    int unitPrice = item.unitPrice;
    int price = calculatePrice(item, days, unitPrice);
    int thisTax = (int) Math.round(price * item.taxRate);

    items.add(new RentalItem(item, days, unitPrice, price, thisTax))
    subtotal += price;
    tax += thisTax;
}
...
```

This is better—I have to say that I like that Java's `enum`s are actually classes, and that each enumerator is an instance. Now when we add `NotePads`, we'll only have to change the

Thế này khá hơn — phải nói là tôi thích chuyện `enum` trong Java thực chất là class, và mỗi enumerator là một instance. Giờ khi thêm `NotePads`, ta sẽ chỉ phải sửa

`enum`, and, perhaps, the `getItems` function. However, the OCP tells us to keep new changes out of old modules. So it would be better, from an OCP point of view, to get that `enum` out of the `Statement` module altogether so that when we add

`enum`, và có lẽ cả hàm `getItems`. Tuy nhiên, OCF bảo ta hãy giữ những thay đổi mới ở ngoài các module cũ. Vậy nên, xét theo góc nhìn OCF, sẽ tốt hơn nếu lôi hẳn cái `enum` đó ra khỏi module `Statement`, để khi thêm

NotePads , we can leave the Statement module untouched.

NotePads , ta có thể để nguyên module Statement mà không phải đụng vào.

—CatalogItem.java—

```
package ubConferenceCenter;
public enum CatalogItem {
    SMALL_ROOM(100, .05),
    LARGE_ROOM(150, .05),
    PROJECTOR(50, .05),
    COFFEE(10, 0),
    COOKIES(15, 0);
    public double taxRate;
    public int unitPrice;
    CatalogItem(int unitPrice, double taxRate) {
        this.unitPrice = unitPrice;
        this.taxRate = taxRate;
    }
}
```

We're doing pretty well here. We've isolated the CatalogItem enum s, and we've gotten rid of the switch statements. The SRP and the OCP are pretty happy. But there's another principle that is sounding an alarm.

Ta đang làm khá tốt. Ta đã cô lập được các enum CatalogItem, và đã tổng khứ được mấy câu lệnh switch. SRP và OCP giờ khá hài lòng. Nhưng còn một nguyên tắc khác đang rú còi báo động.

What modules ought to be recompiled if we add NotePads to

Những module nào đáng phải biên dịch lại nếu ta thêm NotePads vào

CatalogItem ? Answer: CatalogItem.java (of course) and probably 6

CatalogItem ? Xin thưa: CatalogItem.java (dĩ nhiên rồi) và có lẽ cả 6

Statement.java because it depends upon CatalogItem.java . Now ask yourself whether Statement.java should have to be recompiled.

Statement.java nữa, vì nó phụ thuộc vào CatalogItem.java . Giờ hãy tự hỏi xem Statement.java có đáng phải bị biên dịch lại hay không.

6 The Java compiler is often smart enough to determine whether or not upstream modules need to be recompiled based on downstream changes. The safe bet, however, is to assume that those modules will be recompiled.

6 Trình biên dịch Java thường đủ thông minh để xác định xem các module ở tầng trên có cần biên dịch lại hay không dựa trên những thay đổi ở tầng dưới. Tuy vậy, đặt cược an toàn nhất vẫn là giả định rằng những module đó sẽ bị biên dịch lại.

I contend that at least the rent function of Statement.java should not have to be recompiled when we add NotePads , because nothing within that function needs to know anything about NotePads .

Tôi cho rằng ít nhất hàm rent của Statement.java không nên phải biên dịch lại khi ta thêm NotePads , bởi chẳng có gì bên trong hàm ấy cần biết bất cứ điều gì về NotePads .

This is a violation of the **Dependency** Inversion Principle (DIP). This principle is violated when high-level policies directly depend on low-level details. The addition of NotePads is a low-level detail that the rent function should not depend upon.

Đây là một vi phạm Nguyên tắc Đảo ngược Phụ thuộc (Dependency Inversion Principle — DIP). Nguyên tắc này bị vi phạm khi các chính sách mức cao phụ thuộc trực tiếp vào các chi tiết mức thấp. Việc thêm NotePads là một chi tiết mức thấp mà hàm rent không nên phụ thuộc vào.

You may not think this is a big issue. Recompiling a module is quick and easy, and why should you care if you compile more modules than is required? For small projects, you could be right to make that decision. But for larger projects, the recompile and redeployment burden can get pretty large. This is especially true for JavaScript applications where redeployments are downloaded into browsers.

Bạn có thể nghĩ đây chẳng phải vấn đề to tát gì. Biên dịch lại một module thì nhanh và dễ, mà việc gì phải bận tâm nếu bạn biên dịch nhiều module hơn mức cần thiết? Với các dự án nhỏ, quyết định như vậy có thể đúng. Nhưng với dự án lớn, gánh nặng biên dịch lại và triển khai lại có thể trở nên khá nặng. Điều này đặc biệt đúng với các ứng dụng JavaScript, nơi mỗi lần triển khai lại là một lần tải về trình duyệt.

There's also the cognitive burden of trying to remember which modules depend on which others. So let's assume that this project is becoming large enough for those issues to be a concern. How can we invert the dependencies to reduce the recompilation, redeployment, and cognitive burdens?

Còn có cả gánh nặng nhận thức khi phải nhớ module nào phụ thuộc vào module nào. Vậy hãy giả định rằng dự án này đang lớn đến mức những vấn đề đó trở thành mối lo. Làm sao đảo ngược được các phụ thuộc (dependency) để giảm gánh nặng biên dịch lại, triển khai lại và gánh nặng nhận thức?

The SRP will help us here. The Statement.java module is doing two major things. The rent **method** adds CatalogItem s, and the

SRP sẽ giúp ta ở đây. Module Statement.java đang làm hai việc lớn. Method rent thêm các CatalogItem , còn

getItems method totals them and computes the cookie bonus. Hmmm, getItems isn't the best name, is it? We'll take care of that momentarily. First, we should separate those two major behaviors.

method getItems thì cộng tổng chúng và tính phần bánh quy khuyến mãi. Hừm, getItems không phải cái tên hay ho gì, đúng không? Lát nữa ta sẽ xử lý chuyện đó. Trước hết, hãy tách hai hành vi lớn ấy ra.

The first step is to pull out the RentalItem record into its own module.

Bước đầu tiên là lôi record RentalItem ra thành một module riêng.

```
—RentalItem.java—
package ubConferenceCenter;
public record RentalItem(CatalogItem type,
                        int days,
                        int unitPrice,
                        int price,
                        int tax) {
}
```

Next, we'll create a new module for the ItemList .

Tiếp theo, ta sẽ tạo một module mới cho ItemList .

—ItemList.java—

```
package ubConferenceCenter;
import java.util.ArrayList;
import java.util.List;

public class ItemList {
    private List<RentalItem> items = new ArrayList<>();
    public void add(CatalogItem item, int days, int unitPrice,
                   int price, int thisTax) {
        items.add(new RentalItem(item, days, unitPrice, price, thisTax));
    }
    public RentalItem[] getItems() {
        boolean largeRoomFiveDays = items.stream().anyMatch(
            item -> item.type() == CatalogItem.LARGE_ROOM && item.days() == 5
        );
        boolean coffeeFiveDays = items.stream().anyMatch(
            item -> item.type() == CatalogItem.COFFEE && item.days() == 5
        );
        if (largeRoomFiveDays && coffeeFiveDays)
            items.add(new RentalItem(CatalogItem.COOKIE, 5, 0, 0, 0));
        return items.toArray(new RentalItem[0]);
    }
}
```

We'll have to come back to this module because that `getItems` method gives me the willies. ⁷ It smells of an SRP violation. But for now, all that remains is to make a few adjustments to the `Statement.java` module.

Ta sẽ còn phải quay lại module này, bởi cái method `getItems` kia khiến tôi rợn cả người (gives me the willies). ⁷ Nó bốc mùi vi phạm SRP. Nhưng tạm thời, tất cả những gì còn lại chỉ là chỉnh sửa đôi chút module `Statement.java`.

⁷ An archaic term that means nervous, uneasy, or creeped out. (According to Grok.)

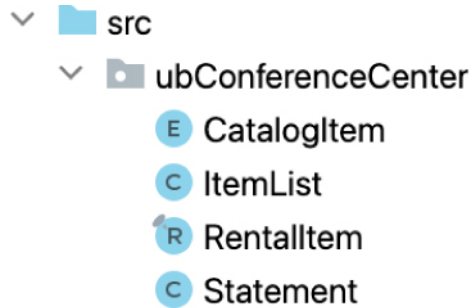
⁷ Một từ cổ, nghĩa là bồn chồn, bất an hoặc nổi da gà. (Theo Grok.)

```
...
public class Statement {
    ...
    private ItemList items = new ItemList();
    ...
    public RentalItem[] getItems() {
        return items.getItems();
    }
    ...
}
```

Again, you may be concerned that we're chopping this up into too many pieces, and that all those pieces will make the code harder to understand. But our assumption has been that this code is going to grow, and that we're trying to make room for that growth. In any case, if you look at the directory structure, you'll see that there's a nice road map that will help anyone understand the organization.

Một lần nữa, có thể bạn lo rằng ta đang băm nhỏ thứ này ra thành quá nhiều mảnh, và tất cả những mảnh đó sẽ khiến mã khó hiểu hơn. Nhưng giả định của ta từ đầu là mã này sẽ lớn lên, và ta đang cố chừa chỗ cho sự tăng trưởng ấy. Dù sao đi nữa, nếu nhìn vào cấu

trúc thư mục, bạn sẽ thấy một tấm bản đồ khá đẹp giúp bất cứ ai cũng hiểu được cách tổ chức.

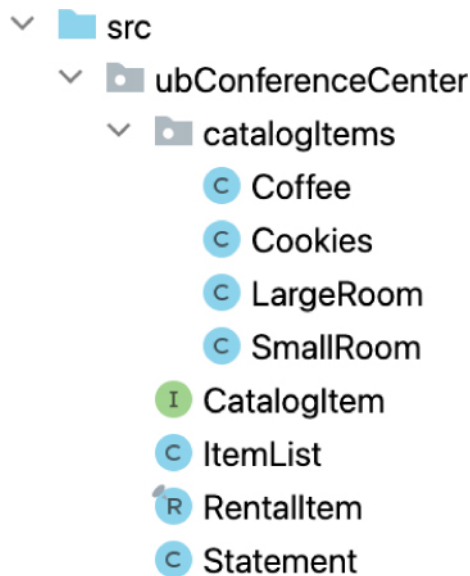


Looking at this, you can see that the names may not be perfect, but we'll come back to that when we understand more about where this reorganization is going. In the meantime, we still have to reduce the recompile and redeployment burden.

Nhìn vào đây, bạn có thể thấy mấy cái tên chưa hẳn đã hoàn hảo, nhưng ta sẽ quay lại chuyện đó khi hiểu rõ hơn cuộc tái tổ chức này đang hướng tới đâu. Trong lúc chờ đợi, ta vẫn phải giảm gánh nặng biên dịch lại và triển khai lại.

To do this, we'll change the CatalogItem from an enum to an **interface**, with each enumeration becoming a derived class. This creates several new classes and makes the directory look like this.

Để làm điều đó, ta sẽ đổi CatalogItem từ một enum thành một interface, với mỗi enumeration trở thành một class dẫn xuất. Việc này sinh ra vài class mới và khiến thư mục trông như thế này.



```
——CatalogItem.java——
package ubConferenceCenter;
public interface CatalogItem {
    boolean isEligibleForDiscount(int days);
    int getUnitPrice();
    double getTaxRate();
    String getName();
}
```

This represents a significant change in the way the application works. Now instead of using switch and if statements to check for enumerators, we defer to the polymorphic methods of the CatalogItem interface. The

Đây là một thay đổi đáng kể trong cách ứng dụng vận hành. Giờ đây, thay vì dùng các câu lệnh switch và if để kiểm tra enumerator, ta giao phó cho các method đa hình của interface CatalogItem. Hai method

getTaxRate and getUnitPrice methods are familiar, but the other two are new. You can see how they are used in the

getTaxRate và getUnitPrice thì đã quen mặt, còn hai method kia là mới. Bạn có thể thấy chúng được dùng ra sao trong

Statement.java and ItemList.java modules.

—Statement.java—

```
package ubConferenceCenter;
public class Statement {
    ...
    public void rent(CatalogItem item, int days) {
        int unitPrice = item.getUnitPrice();
        int price = calculatePrice(item, days, unitPrice);
        int thisTax = (int) Math.round(price * item.getTaxRate());
        items.add(item, days, unitPrice, price, thisTax);
        subtotal += price;
        tax += thisTax;
    }
    private int calculatePrice(CatalogItem item, int days, int unitP
        boolean eligibleForDiscount = item.isEligibleForDiscount(days)
        int price = unitPrice * days;
        if (eligibleForDiscount) price = (int) Math.round(price * .9);
        return price;
    }
    ...
}
```

—ItemList.java—

```
package ubConferenceCenter;
import java.util.ArrayList;
import java.util.List;
public class ItemList {
    private List<RentalItem> items = new ArrayList<>();
    public void add(CatalogItem item, int days, int unitPrice,
        int price, int thisTax) {
        items.add(new RentalItem(item.getName(), days,
            unitPrice, price, thisTax));
    }
    public RentalItem[] getItems() {
        boolean largeRoomFiveDays = items.stream().anyMatch(
            item -> item.type().equals("LARGE_ROOM") && item.days() == 5
        boolean coffeeFiveDays = items.stream().anyMatch(
            item -> item.type().equals("COFFEE") && item.days() == 5);
        if (largeRoomFiveDays && coffeeFiveDays)
```



```

        items.add(new RentalItem("COOKIES", 5, 0, 0, 0));
    return items.toArray(new RentalItem[0]);
}
}

```

Notice that the type field of the RentalItem record has been changed from the enum to a String . This was necessary because, when we abandoned the enum , we needed some kind of token to represent the CatalogItem type; and a String seemed the most obvious choice. However, using that string means that we've had to sacrifice a bit of static type safety. The compiler cannot check that those names are correct. This minor loss of static type safety always accompanies the effort to reduce the recompile and redeployment burden, and the isolation of low-level details from high-level policy.

Hãy để ý rằng trường type của record RentalItem đã được đổi từ enum sang String . Điều này là cần thiết, bởi khi từ bỏ enum , ta cần một thứ token nào đó để biểu diễn kiểu CatalogItem ; và String có vẻ là lựa chọn hiển nhiên nhất. Tuy nhiên, dùng chuỗi ấy nghĩa là ta đã phải hy sinh một chút an toàn kiểu tĩnh (static type safety). Trình biên dịch không thể kiểm tra xem những cái tên đó có đúng hay không. Mất mát nho nhỏ về an toàn kiểu tĩnh này luôn đi kèm với nỗ lực giảm gánh nặng biên dịch lại và triển khai lại, cũng như với việc cô lập chi tiết mức thấp khỏi chính sách mức cao.

——RentalItem.java——

```

package ubConferenceCenter;
public record RentalItem(String type,
                        int days,
                        int unitPrice,
                        int price,
                        int tax) {
}

```

With the **exception** of the getItems method of the ItemList class, the last few listings above represent the high-level policy of the application. We're going to have to do something about that getItems method. For now, let's look at how the low-level details have been isolated in the derivatives of CatalogItem .

Ngoại trừ method getItems của class ItemList, vài listing vừa rồi ở trên chính là phần chính sách mức cao của ứng dụng. Ta sẽ phải làm gì đó với cái method getItems ấy. Còn bây giờ, hãy xem các chi tiết mức thấp đã được cô lập thế nào trong các lớp dẫn xuất của CatalogItem .

——SmallRoom.java——

```

package ubConferenceCenter.catalogItems;
import ubConferenceCenter.CatalogItem;

public class SmallRoom implements CatalogItem {
    public String getName() {
        return "SMALL_ROOM";
    }
    public boolean isEligibleForDiscount(int days) {
        return days == 5;
    }
    public int getUnitPrice() {
        return 100;
    }
    public double getTaxRate() {

```



```

        return 0.05;
    }
}
—LargeRoom.java—
package ubConferenceCenter.catalogItems;
import ubConferenceCenter.CatalogItem;
public class LargeRoom implements CatalogItem {
    public boolean isEligibleForDiscount(int days) {
        return days == 5;
    }
    public int getUnitPrice() {
        return 150;
    }
    public double getTaxRate() {
        return 0.05;
    }
    public String getName() {
        return "LARGE_ROOM";
    }
}
—Coffee.java—
package ubConferenceCenter.catalogItems;
import ubConferenceCenter.CatalogItem;
public class Coffee implements CatalogItem {
    public boolean isEligibleForDiscount(int days) {
        return false;
    }
    public int getUnitPrice() {
        return 10;
    }
    public double getTaxRate() {
        return 0;
    }
    public String getName() {
        return "COFFEE";
    }
}
—Cookies.java—
package ubConferenceCenter.catalogItems;
import ubConferenceCenter.CatalogItem;
public class Cookies implements CatalogItem {
    public boolean isEligibleForDiscount(int days) {
        return false;
    }
    public int getUnitPrice() {
        return 15;
    }
    public double getTaxRate() {
        return 0;
    }
}

```

```
}  
public String getName() {  
    return "COOKIES";  
}
```

One of the people who commented on the first edition of this book looked at code like this and called it “just dreadful.” You might be feeling that now too. But remember, we are expecting business growth, and we have decided that it is necessary to make room in this code for that growth.

Một trong những người bình luận về ấn bản đầu tiên của cuốn sách này nhìn vào mã kiểu như thế này và gọi nó là “kinh khủng khiếp.” Có lẽ giờ bạn cũng đang thấy vậy. Nhưng hãy nhớ, ta đang trông đợi công việc kinh doanh tăng trưởng, và ta đã quyết định rằng cần phải chừa chỗ trong mã này cho sự tăng trưởng đó.

Some folks might charge us with violating **YAGNI**. They think it stands for You Aren’t Going to Need It. But the original intent of YAGNI was to ask yourself: “What if you aren’t going to need it?” It was a way of asking us to count the cost before we invested in making room in our code for things we might not need.

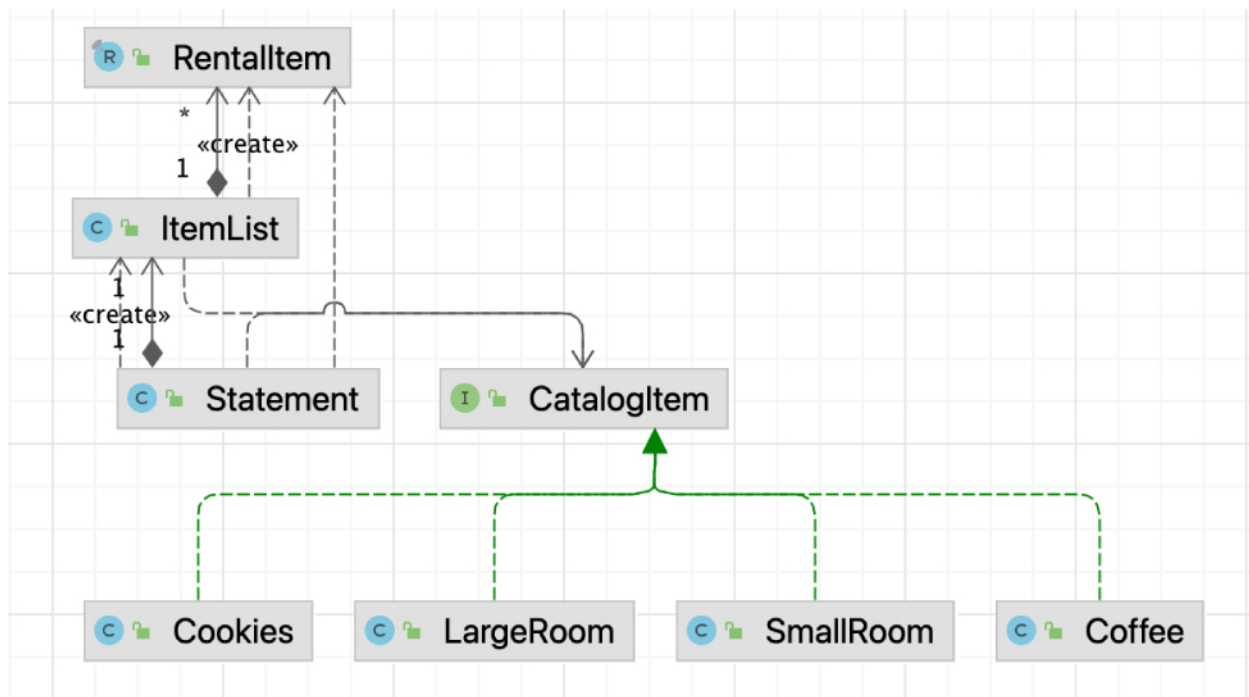
Vài người có thể buộc tội ta vi phạm YAGNI. Họ tưởng nó là viết tắt của You Aren’t Going to Need It. Nhưng ý định ban đầu của YAGNI là bảo bạn tự hỏi: “Thế nếu bạn sẽ không cần đến nó thì sao?” Đó là cách nhắc ta tính trước cái giá phải trả, trước khi đầu tư công sức chừa chỗ trong mã cho những thứ có thể ta chẳng bao giờ cần.

But given the enthusiasm of the CEO and the kinds of markets he is courting, we have decided we need to make this room. So, in our case, we’ve decided to answer YAGNI’s question with: “Yes, we are going to need it.”

Nhưng với sự hào hứng của ngài CEO và những thị trường mà ông ấy đang tán tỉnh, ta đã quyết định rằng cần chừa chỗ này. Vậy nên, trong trường hợp của ta, ta đã quyết định trả lời câu hỏi của YAGNI bằng: “Đúng, chúng ta sẽ cần đến nó.”

Why is this code better? Let’s look at the dependency diagram generated by my IDE.

Vì sao đoạn mã này lại tốt hơn? Hãy nhìn sơ đồ phụ thuộc (dependency) mà IDE của tôi sinh ra.



The four classes that represent our high-level policy—

Bốn class biểu diễn chính sách mức cao (high-level policy) của chúng ta—

`RentalItem`, `ItemList`, `Statement`, and `CatalogItem`—are tied

together with a rat's nest of dependencies. We'll deal with that later. But look at how well isolated the low-level details are. The previous dependencies have been inverted. Nothing within the high-level policy depends upon the low-level details. The low-level details depend only on the `CatalogItem`. Just say this to yourself, over and over: High-level policy should not depend on low-level details.

cùng với một mớ phụ thuộc rối như tổ quạ. Chuyện đó ta sẽ xử lý sau. Nhưng hãy nhìn xem các chi tiết mức thấp (low-level detail) được cô lập tốt đến mức nào. Các phụ thuộc trước đây đã bị đảo ngược. Không có gì bên trong chính sách mức cao phụ thuộc vào chi tiết mức thấp cả. Chi tiết mức thấp chỉ phụ thuộc vào `CatalogItem`. Cứ nhắm đi nhắm lại câu này với chính mình: chính sách mức cao không được phụ thuộc vào chi tiết mức thấp.

This means that if any of the low-level details are changed, nothing in the high-level policy ought necessarily to be recompiled or redeployed. Nor will changes to the high-level policy force recompilation and redeployment of the low-level details.

Điều này nghĩa là nếu bất kỳ chi tiết mức thấp nào thay đổi, sẽ chẳng có gì trong chính sách mức cao nhất thiết phải biên dịch lại hay triển khai lại. Ngược lại, thay đổi trong chính sách mức cao cũng không bắt các chi tiết mức thấp phải biên dịch lại và triển khai lại.

Let me say that differently. If the unit price of `Cookies` changes, or if the tax rate of `Coffee` changes, or if the discount for `SmallRoom` changes, none of the classes in the high-level policy will need to be recompiled or redeployed. Changes to those low-level details are completely isolated from the high-level policy.

Để tôi nói theo cách khác. Nếu đơn giá của `Cookies` thay đổi, hoặc thuế suất của `Coffee` thay đổi, hoặc mức giảm giá cho `SmallRoom` thay đổi, thì không class nào trong chính sách mức cao cần được biên dịch lại hay triển khai lại. Những thay đổi ở các chi tiết mức thấp ấy được cô lập hoàn toàn khỏi chính sách mức cao.

Indeed, the low-level details have become a plug-in to the high-level policy. Or, at least they are in a position to become such a plug-in.

Thật vậy, các chi tiết mức thấp đã trở thành một plug-in cắm vào chính sách mức cao. Hoặc ít nhất chúng cũng đang ở thế sẵn sàng để trở thành một plug-in như vậy.

But now we need to do something about the rat's nest that's caused by the ItemList .

Nhưng giờ ta cần làm gì đó với cái mớ rối như tổ quạ do ItemList gây ra.

We called this class ItemList because initially it contained our list of items, and we needed a place to put that bonus code. Now I want to take that bonus code out of there and do to it what we did to the CatalogItems . So I'll create a Bonus interface and have the CookieBonus implement it. The ItemList can contain a list of Bonus instances and simply iterate through all of them to add all the bonuses. That means that the ItemList finalizes all the bonuses. And that suggests that the ItemList should be renamed something like RentalReceipt , since it represents the final state of the order.

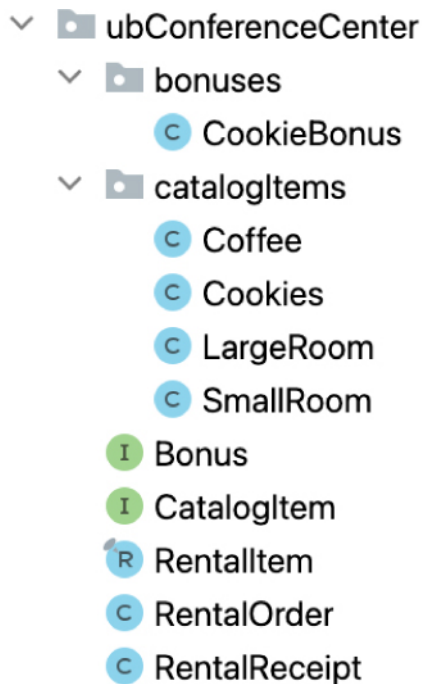
Ta đã gọi class này là ItemList vì ban đầu nó chứa danh sách các item của ta, và ta cần một chỗ để nhét đoạn mã bonus vào. Giờ tôi muốn lôi đoạn mã bonus ấy ra khỏi đó và làm với nó đúng những gì ta đã làm với các CatalogItem . Vậy nên tôi sẽ tạo một interface Bonus và cho CookieBonus hiện thực nó. ItemList có thể chứa một danh sách các instance Bonus rồi chỉ việc duyệt qua tất cả để cộng hết bonus vào. Nghĩa là ItemList chốt lại toàn bộ phần bonus. Và điều đó gợi ý rằng ItemList nên được đổi tên thành đại loại như RentalReceipt , bởi nó biểu diễn trạng thái cuối cùng của đơn thuê.

That name change suggests that Statement should be renamed something like RentalOrder . These name changes are important. As we pick apart the design and start making room for growth, we gain insights into what's really going on. When the problem is small, names can afford to be inconsistent or vague because there aren't that many concepts to keep track of. But as the problem grows, names become much more important because they help us keep the concepts straight in our heads.

Việc đổi tên đó lại gợi ý rằng Statement nên được đổi thành đại loại như RentalOrder . Những thay đổi tên này rất quan trọng. Khi ta mổ xẻ thiết kế và bắt đầu chừa chỗ cho sự phát triển, ta hiểu rõ hơn chuyện gì thực sự đang diễn ra. Lúc bài toán còn nhỏ, tên gọi có thể lỏng lẻo hay mơ hồ cũng chẳng sao, vì đâu có nhiều khái niệm phải theo dõi. Nhưng khi bài toán lớn dần, tên gọi trở nên quan trọng hơn nhiều, bởi chúng giúp ta giữ cho các khái niệm rành mạch trong đầu.

So with those changes in place, here's the directory structure:

Vậy là sau khi áp dụng những thay đổi đó, đây là cấu trúc thư mục:



The diagram looks like this. The RentalOrder depends on the

Sơ đồ trông như thế này. RentalOrder phụ thuộc vào

RentalReceipt , but not vice versa. Both depend upon the Bonus , but the CookieBonus is an independent low-level detail.

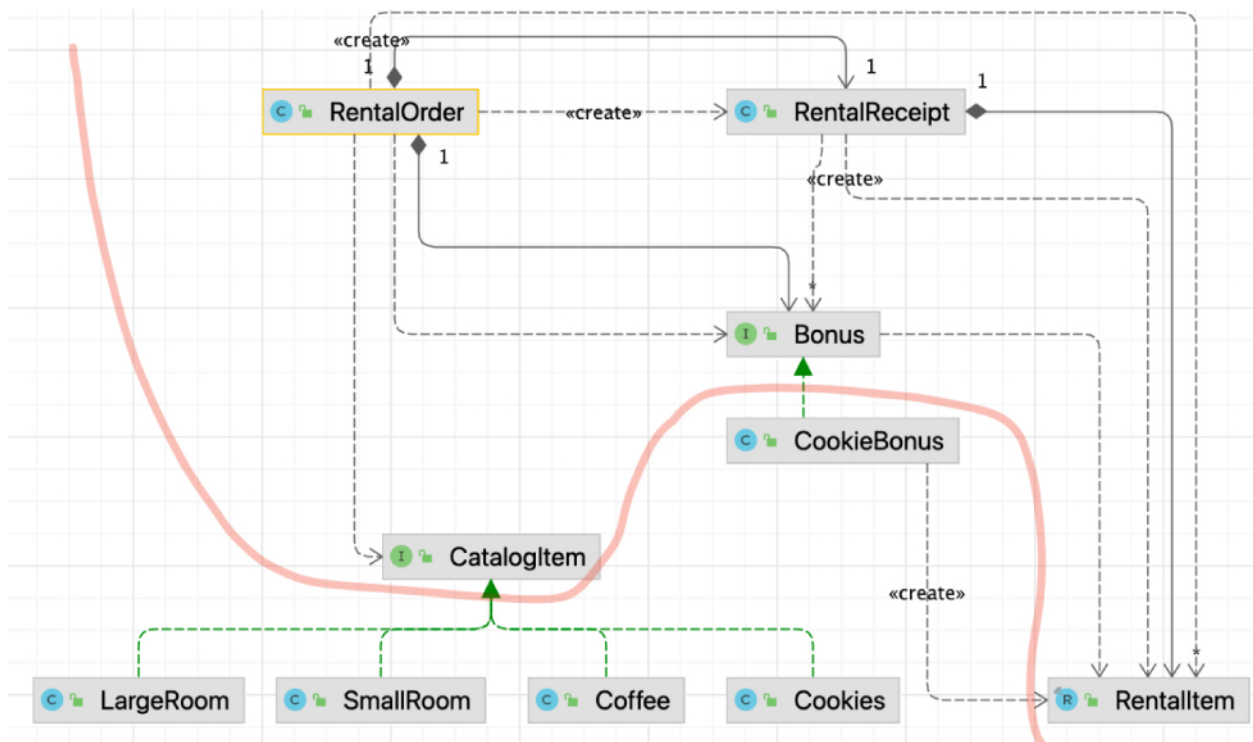
RentalReceipt , chứ không có chiều ngược lại. Cả hai đều phụ thuộc vào Bonus , còn CookieBonus là một chi tiết mức thấp độc lập.

The lowest-level detail is the RentalItem record. That's a bit of a concern because it is so concrete. If it gets changed in any way, then pretty much everything has to recompile and redeploy. There are ways to resolve that—but I think that's a battle for another day.

Chi tiết ở mức thấp nhất là record RentalItem . Chỗ đó hơi đáng lo, vì nó quá cụ thể. Nếu nó bị thay đổi theo bất kỳ cách nào thì gần như mọi thứ đều phải biên dịch lại và triển khai lại. Có những cách để giải quyết chuyện đó — nhưng tôi nghĩ đấy là trận đánh của một ngày khác.

Notice the curvy border line. That line divides the project into two components. One contains the high-level policy, and the other contains the low-level details. All dependencies cross that line in one direction, toward the higher-level **component**. That line is an **architectural boundary** that separates those two components—and the rule for architectural boundaries is that dependencies cross only toward the higher-level side.

Để ý cái đường viền uốn lượn. Đường ấy chia dự án thành hai component. Một bên chứa chính sách mức cao, bên kia chứa các chi tiết mức thấp. Mọi phụ thuộc đều cắt qua đường đó theo một chiều duy nhất, hướng về phía component ở mức cao hơn. Đường ấy là một ranh giới kiến trúc (architectural boundary) ngăn cách hai component — và luật của ranh giới kiến trúc là: phụ thuộc chỉ được cắt qua theo hướng đi về phía mức cao hơn.



First, let's look at the code within the high-level component.

Trước hết, hãy xem mã bên trong component mức cao.

```

package ubConferenceCenter;
import java.util.ArrayList;
import java.util.List;
public class RentalOrder {
    public record Totals(int subtotal, int tax) {
    }
    private String customerName;
    private int subtotal = 0;
    private int tax = 0;
    private RentalReceipt receipt = new RentalReceipt();
    private List<Bonus> bonuses = new ArrayList<>();
    public RentalOrder(String customerName) {
        this.customerName = customerName;
    }
    public void addBonus(Bonus bonus) {
        bonuses.add(bonus);
    }
    public void rent(CatalogItem item, int days) {
        int unitPrice = item.getUnitPrice();
        int price = item.getDiscountedPrice(days);
        int thisTax = (int) Math.round(price * item.getTaxRate());
        receipt.add(new RentalItem(item.getName(), days,
            unitPrice, price, thisTax));
        subtotal += price;
        tax += thisTax;
    }
}

```

```

    }
    public RentalItem[] getReceipt() {
        return receipt.finalize(bonuses);
    }
    public String getCustomerName() {
        return customerName;
    }
    public Totals getTotals() {
        return new Totals(subtotal, tax);
    }
}

```

Notice that virtually all of the business rules have fled this module. The only ones that remain are the tax and total calculations. But the discount and bonus calculations have been moved into the appropriate derivatives of `CatalogItem` and `Bonus`.

Đề ý rằng gần như toàn bộ quy tắc nghiệp vụ (business rule) đã bỏ chạy khỏi module này. Thứ duy nhất còn ở lại là phần tính thuế và tính tổng. Còn phần tính giảm giá và bonus đã được dời vào các dẫn xuất (derivative) thích hợp của `CatalogItem` và `Bonus`.

—RentalReceipt.java—

```

package ubConferenceCenter;
import java.util.ArrayList;
import java.util.List;

public class RentalReceipt {
    private List<RentalItem> items = new ArrayList<>();
    public void add(RentalItem item) {
        items.add(item);
    }
    public RentalItem[] finalize(List<Bonus> bonuses) {
        List<RentalItem> finalItems = new ArrayList<>(this.items);
        finalItems.addAll(addBonuses(bonuses));
        return finalItems.toArray(new RentalItem[0]);
    }
    private List<RentalItem> addBonuses(List<Bonus> bonuses) {
        List<RentalItem> bonusItems = new ArrayList<>();
        for (Bonus bonus : bonuses)
            bonus.checkAndAdd(items, bonusItems);
        return bonusItems;
    }
}

```

The `RentalReceipt` class walks through and applies the bonuses; but it does not know any of the details about those bonuses. It creates a finalized receipt with the bonuses added to all the items.

Class `RentalReceipt` duyệt qua và áp dụng các bonus; nhưng nó chẳng biết chi tiết nào về những bonus ấy. Nó tạo ra một biên nhận đã chốt, với bonus được cộng vào tất cả các item.

Next come the two interfaces that inverted the dependencies between the high-level policy and the low-level detail.

Tiếp theo là hai interface đã đảo ngược các phụ thuộc giữa chính sách mức cao và chi tiết mức thấp.

—CatalogItem.java—

```
package ubConferenceCenter;
public interface CatalogItem {
    int getDiscountedPrice(int days);
    int getUnitPrice();
    double getTaxRate();
    String getName();
}
```

—Bonus.java—

```
package ubConferenceCenter;
import java.util.List;
public interface Bonus {
    void checkAndAdd(List<RentalItem> items, List<RentalItem> bonusIt
}
```

The last module in the high-level component is the

Module cuối cùng trong component mức cao là

RentalItem . This is a simple concrete **data structure**.

RentalItem . Đây là một cấu trúc dữ liệu (data structure) cụ thể và đơn giản.

—RentalItem.java—

```
package ubConferenceCenter;
public record RentalItem(String type,
                        int days,
                        int unitPrice,
                        int price,
                        int tax) {
}
```

As previously stated, this module is somewhat problematic. Any changes made to this data structure will force recompilation and redeployment of both components. And changes to this data structure are not at all unlikely.

Như đã nói ở trên, module này có phần rắc rối. Mọi thay đổi trên cấu trúc dữ liệu này đều buộc cả hai component phải biên dịch lại và triển khai lại. Mà thay đổi trên cấu trúc dữ liệu này thì hoàn toàn không phải chuyện hiếm.

There are ways of dealing with this. For example, we could turn the RentalItem into a hash map. But that's probably going well beyond our current goal of making room for growth. We'll keep that idea in our back pocket for now.

Có những cách để xử lý điều đó. Chẳng hạn, ta có thể biến RentalItem thành một hash map. Nhưng như vậy có lẽ đã vượt xa mục tiêu hiện tại của ta là chừa chỗ cho sự phát triển. Tạm thời cứ giữ ý tưởng ấy trong túi đã.

Now let's look at the low-level component.

Giờ hãy nhìn sang component mức thấp.

—SmallRoom.java—

```
package ubConferenceCenter.catalogItems;
import ubConferenceCenter.CatalogItem;
public class SmallRoom implements CatalogItem {
```



```

public String getName() {
    return "SMALL_ROOM";
}
public int getDiscountedPrice(int days) {
    double discountRate = (days == 5) ? 0.9 : 1.0;
    return (int) Math.round(getUnitPrice() * days * discountRate);
}
public int getUnitPrice() {
    return 100;
}
public double getTaxRate() {
    return 0.05;
}
}

```

—LargeRoom.java—

```

package ubConferenceCenter.catalogItems;
import ubConferenceCenter.CatalogItem;
public class LargeRoom implements CatalogItem {
    public int getDiscountedPrice(int days) {
        double discountRate = (days == 5) ? 0.9 : 1.0;
        return (int) Math.round(getUnitPrice() * days * discountRate);
    }
    public int getUnitPrice() {
        return 150;
    }
    public double getTaxRate() {
        return 0.05;
    }
    public String getName() {
        return "LARGE_ROOM";
    }
}

```

—Coffee.java—

```

package ubConferenceCenter.catalogItems;
import ubConferenceCenter.CatalogItem;
public class Coffee implements CatalogItem {
    public int getDiscountedPrice(int days) {
        return getUnitPrice() * days;
    }
    public int getUnitPrice() {
        return 10;
    }
    public double getTaxRate() {
        return 0;
    }
    public String getName() {
        return "COFFEE";
    }
}

```

—Cookies.java—

```
package ubConferenceCenter.catalogItems;

import ubConferenceCenter.CatalogItem;

public class Cookies implements CatalogItem {
    public int getDiscountedPrice(int days) {
        return getUnitPrice() * days;
    }
    public int getUnitPrice() {
        return 15;
    }
    public double getTaxRate() {
        return 0;
    }
    public String getName() {
        return "COOKIES";
    }
}
```

—CookieBonus.java—

```
package ubConferenceCenter.bonuses;

import ubConferenceCenter.Bonus;
import ubConferenceCenter.RentalItem;
import java.util.List;

public class CookieBonus implements Bonus {
    public void checkAndAdd(List<RentalItem> items,
                           List<RentalItem> bonusItems) {
        boolean largeRoomFiveDays = items.stream().anyMatch(
            item -> item.type().equals("LARGE_ROOM") && item.days() == 5
        );
        boolean coffeeFiveDays = items.stream().anyMatch(
            item -> item.type().equals("COFFEE") && item.days() == 5
        );
        if (largeRoomFiveDays && coffeeFiveDays)
            bonusItems.add(new RentalItem("COOKIES", 5, 0, 0, 0));
    }
}
```

As you can see, all the business rules have been tucked away into their corresponding derivatives. If a new discount is needed for coffee, we can place it in the Coffee class. If a new bonus is needed for small rooms, we can create a Bonus derivative for that. If we want to add NotePads or Projectors, we can do that by adding those classes to the low-level component. None of those changes will cause the high-level component to be recompiled or redeployed. We have achieved that isolation by conforming to the DIP.

Như bạn thấy đây, mọi quy tắc nghiệp vụ đều đã được xếp gọn vào đúng dẫn xuất tương ứng của chúng. Nếu cần một mức giảm giá mới cho coffee, ta đặt nó vào class Coffee. Nếu cần một bonus mới cho phòng nhỏ, ta tạo một dẫn xuất của Bonus cho việc đó. Nếu muốn thêm NotePad hay Projector, ta chỉ việc thêm các class ấy vào component mức thấp. Không thay đổi nào trong số đó khiến component mức cao phải biên dịch lại hay triển khai lại. Ta đạt được sự cô lập ấy nhờ tuân thủ DIP.

You might be concerned about one thing, though. Perhaps you are wondering where all those derivatives are getting instantiated. I never showed you my tests, did I. That's where all that work was done. Here they are.

Dù vậy, có lẽ bạn đang bận tâm một chuyện. Có thể bạn đang tự hỏi tất cả những dẫn xuất kia được khởi tạo ở đâu. Tôi chưa hề cho bạn xem các test của mình, đúng không. Chính ở đó mọi việc ấy được làm. Chúng đây.

——RentalOrderTest.java——

```
package ubConferenceCenter;
import org.junit.jupiter.api.BeforeEach;

import org.junit.jupiter.api.Test;
import ubConferenceCenter.RentalOrder.Totals;
import ubConferenceCenter.bonuses.CookieBonus;
import ubConferenceCenter.catalogItems.Coffee;
import ubConferenceCenter.catalogItems.Cookies;
import ubConferenceCenter.catalogItems.LargeRoom;
import ubConferenceCenter.catalogItems.SmallRoom;
import static org.junit.jupiter.api.Assertions.assertEquals;

public class RentalOrderTest {
    private final SmallRoom SMALL_ROOM = new SmallRoom();
    private final LargeRoom LARGE_ROOM = new LargeRoom();
    private final Coffee COFFEE = new Coffee();
    private final Cookies COOKIES = new Cookies();
    private RentalOrder order;
    @BeforeEach
    void setUp() {
        order = new RentalOrder("Customer Name");
        order.addBonus(new CookieBonus());
    }
    private void assertTotalAndTax(int subtotal, int tax) {
        Totals totals = order.getTotals();
        assertEquals(subtotal, totals.subtotal());
        assertEquals(tax, totals.tax());
    }

    private void assertTypeDaysUnitTotalTax(RentalItem item, String type,
                                             int days, int unitPrice,
                                             int price, int tax) {
        assertEquals(type, item.type());
        assertEquals(days, item.days());
        assertEquals(unitPrice, item.unitPrice());
        assertEquals(price, item.price());
        assertEquals(tax, item.tax());
    }
    @Test
    public void oneSmallRoomForOneDay() throws Exception {
        assertEquals("Customer Name", order.getCustomerName());
        order.rent(SMALL_ROOM, 1);
        RentalItem[] receipt = order.getReceipt();
        assertEquals(1, receipt.length);
        assertTypeDaysUnitTotalTax(receipt[0], "SMALL_ROOM", 1, 100, 10, 10);
        assertTotalAndTax(100, 5);
    }
    @Test
```

```

public void oneSmallRoomForTwoDays() throws Exception {
    order.rent(SMALL_ROOM, 2);
    RentalItem[] receipt = order.getReciept();
    assertEquals(1, receipt.length);
    assertTypeDaysUnitTotalTax(receipt[0], "SMALL_ROOM", 2, 100, 20
    assertTotalAndTax(200, 10);
}
@Test
public void oneLargeRoomForThreeDays() throws Exception {
    order.rent(LARGE_ROOM, 3);
    RentalItem[] receipt = order.getReciept();
    assertEquals(1, receipt.length);
    assertTypeDaysUnitTotalTax(receipt[0], "LARGE_ROOM", 3, 150, 45
    assertTotalAndTax(450, 23);
}
@Test
public void oneSmallRoomForOneWeek() throws Exception {
    order.rent(SMALL_ROOM, 5);
    RentalItem[] receipt = order.getReciept();
    assertEquals(1, receipt.length);
    assertTypeDaysUnitTotalTax(receipt[0], "SMALL_ROOM", 5, 100, 45
    assertTotalAndTax(450, 23); /* 10% discount */
}
@Test
public void twoSmallRoomsForOneDay() throws Exception {
    order.rent(SMALL_ROOM, 1);
    order.rent(SMALL_ROOM, 1);
    RentalItem[] receipt = order.getReciept();
    assertEquals(2, receipt.length);
    assertTypeDaysUnitTotalTax(receipt[0], "SMALL_ROOM", 1, 100, 10
    assertTypeDaysUnitTotalTax(receipt[1], "SMALL_ROOM", 1, 100, 10
    assertTotalAndTax(200, 10);
}
@Test
public void oneSmallRoomAndCoffeeForOneDay() throws Exception {
    order.rent(SMALL_ROOM, 1);
    order.rent(COFFEE, 1);
    RentalItem[] receipt = order.getReciept();
    assertEquals(2, receipt.length);
    assertTypeDaysUnitTotalTax(receipt[0], "SMALL_ROOM", 1, 100, 10
    assertTypeDaysUnitTotalTax(receipt[1], "COFFEE", 1, 10, 10, 0)
    assertTotalAndTax(110, 5); /* No tax on coffee */
}
@Test
public void oneSmallRoomAndCookiesForOneDay() throws Exception {
    order.rent(SMALL_ROOM, 1);
    order.rent(COOKIES, 1);
    RentalItem[] receipt = order.getReciept();
    assertEquals(2, receipt.length);

```

```

    assertTypeDaysUnitTotalTax(receipt[0], "SMALL_ROOM", 1, 100, 10
    assertTypeDaysUnitTotalTax(receipt[1], "COOKIES", 1, 15, 15, 0
    assertTotalAndTax(115, 5); /* No tax on cookies */
}
@Test
public void oneSmallRoomCookiesCoffeeForFiveDays() throws Excepti
    order.rent(SMALL_ROOM, 5);

    order.rent(COFFEE, 5);
    order.rent(COOKIES, 5);
    RentalItem[] receipt = order.getReciept();
    assertEquals(3, receipt.length);
    assertTypeDaysUnitTotalTax(receipt[0], "SMALL_ROOM", 5, 100, 45
    assertTypeDaysUnitTotalTax(receipt[1], "COFFEE", 5, 10, 50, 0)
    assertTypeDaysUnitTotalTax(receipt[2], "COOKIES", 5, 15, 75, 0
    /* 10% discount on room but not on coffee */
    assertTotalAndTax(575, 23);
}
@Test
public void oneLargeRoomAndCoffeeForWeekGetsCookies() throws Exce
    order.rent(LARGE_ROOM, 5);
    order.rent(COFFEE, 5);
    RentalItem[] receipt = order.getReciept();
    assertEquals(3, receipt.length);
    assertTypeDaysUnitTotalTax(receipt[0], "LARGE_ROOM", 5, 150, 67
    assertTypeDaysUnitTotalTax(receipt[1], "COFFEE", 5, 10, 50, 0)
    assertTypeDaysUnitTotalTax(receipt[2], "COOKIES", 5, 0, 0, 0);
}
}

```

These tests were written before any of the refactoring in this chapter had begun. At each step of that refactoring, I kept these tests passing. Some of the changes made to the production code caused changes to the tests, but I was able to minimize them by keeping the tests and production code decoupled and by isolating the details of the production code from the details of the tests. For example, the final fields named for the CatalogItems replaced the old enumerations without too much fuss.

Những test này đã được viết trước khi mọi đợt refactoring trong chương này bắt đầu. Ở từng bước của đợt refactoring ấy, tôi luôn giữ cho chúng pass. Một vài thay đổi trong mã production có kéo theo thay đổi ở test, nhưng tôi đã giảm thiểu được chúng nhờ giữ cho test và mã production tách rời (decoupling) nhau, và nhờ cô lập các chi tiết của mã production khỏi các chi tiết của test. Chẳng hạn, các trường final được đặt tên theo các CatalogItem đã thay thế đám enumeration cũ mà chẳng phải làm um sùm gì nhiều.

Independent Deployability — Triển khai độc lập được

Our design now has a strong architectural boundary that separates it into two components. Those two components can be independently deployed. They can be compiled into two separate jar files. When one component changes, only that jar changes, and the other does not need to be redeployed.

Thiết kế của ta giờ đã có một ranh giới kiến trúc vững chắc, chia nó thành hai component.

Hai component ấy có thể được triển khai độc lập. Chúng có thể được biên dịch thành hai file jar riêng biệt. Khi một component thay đổi, chỉ file jar của nó thay đổi, còn cái kia không cần phải triển khai lại.

Imagine that this code was written in JavaScript as opposed to Java. Imagine that it will be sent to a browser to run there. The fact that we have divided it into two components has a very specific advantage. If one of those two components changes, and if the browser maintains a cache of those components, then only the component that changed needs to be reloaded into the browser. On slow network connections, that could be a big advantage.

Hãy tưởng tượng đoạn mã này được viết bằng JavaScript thay vì Java. Hãy tưởng tượng nó sẽ được gửi tới trình duyệt để chạy ở đó. Việc ta đã chia nó thành hai component mang lại một lợi thế rất cụ thể. Nếu một trong hai component thay đổi, và nếu trình duyệt có cache các component ấy, thì chỉ component nào thay đổi mới cần được tải lại vào trình duyệt. Với đường truyền mạng chậm, đó có thể là một lợi thế lớn.

A Final Thought — Một suy nghĩ sau cùng

This major cleaning will greatly facilitate the growth of the project. The time it took will be paid back many times over simply because there's a clear and obvious place to put the needed changes; and those changes aren't likely to interfere with each other or with the overall flow of the system.

Đợt tổng vệ sinh lớn này sẽ tạo thuận lợi rất nhiều cho sự phát triển của dự án. Thời gian bỏ ra sẽ được hoàn lại gấp nhiều lần, đơn giản vì đã có một chỗ rõ ràng và hiển nhiên để đặt những thay đổi cần thiết; và những thay đổi ấy khó mà giẫm chân lên nhau hay lên dòng chảy tổng thể của hệ thống.

However, the new structure is not cost free. Those who complain that it is harder to read because of all the different modules and all the indirection are not without a point. Yes, adding structure adds complexity—and that complexity is not free. There's a cognitive price to pay. But we knew that the cognitive load was going to grow, and so our goal was to minimize that growth by creating an organization that would accept it gracefully.

Tuy nhiên, cấu trúc mới không phải là miễn phí. Những ai than phiền rằng nó khó đọc hơn vì lắm module và lắm tầng gián tiếp cũng không hẳn là vô lý. Đúng, thêm cấu trúc là thêm độ phức tạp — và độ phức tạp ấy không miễn phí. Có một cái giá về mặt nhận thức phải trả. Nhưng ta biết tải nhận thức rồi sẽ tăng lên, nên mục tiêu của ta là giảm thiểu mức tăng đó bằng cách tạo ra một cách tổ chức có thể tiếp nhận nó một cách duyên dáng.

What about performance? Is the new structure slower than the old one? Probably so. There are a few more references that need to be followed, and switch statements are easier for compilers to optimize than polymorphic dispatch. But the difference in performance is probably so tiny that you could not easily measure it. For most applications, that tiny decrease in performance would be of no concern at all.

Còn hiệu năng thì sao? Cấu trúc mới có chậm hơn cấu trúc cũ không? Có lẽ là có. Có thêm vài tầng tham chiếu phải lần theo, và câu lệnh switch thì dễ cho trình biên dịch tối ưu hơn so với điều phối đa hình (polymorphism). Nhưng chênh lệch hiệu năng có lẽ nhỏ đến mức bạn khó lòng đo được. Với đa số ứng dụng, mức giảm hiệu năng tí hon ấy hoàn toàn chẳng đáng bận tâm.

Future Bob: Grok3, good as it is, did not drive the design of the original code to the level of depth we

Bob tương lai: Grok3, giỏi đến vậy, vẫn không đẩy thiết kế của đoạn mã gốc tới độ sâu mà ta

just saw. Grok3 did not invert the dependencies, nor did it attempt to separate high-level policy
vừa thấy. Grok3 không đảo ngược các phụ thuộc, cũng chẳng buồn tách chính sách mức
cao

from low-level detail. Large language models (LLMs)/AIs have their uses; but they are not human
and are not adept at understanding higher

khỏi chi tiết mức thấp. LLM (mô hình ngôn ngữ lớn)/AI có chỗ hữu dụng của chúng; nhưng
chúng không phải con người và không giỏi nắm bắt những mục tiêu kiến trúc

architectural goals.

ở tầng cao hơn.

This chapter, and the chapter before, were a whirlwind tour through the low-hanging fruit of clean
code. But now that tour is over, and it's time to get down to the basics. And, of course, we begin with
names.

Chương này, và cả chương trước nữa, là một chuyến tham quan chớp nhoáng qua những
trái ngọt dễ hái của clean code. Nhưng giờ chuyến tham quan ấy đã kết thúc, và đã đến
lúc bắt tay vào những điều căn bản. Và, dĩ nhiên, ta bắt đầu bằng tên gọi.

Chapter 4. Meaningful Names — Chương 4. Tên có ý nghĩa

by Tim Ottinger and Uncle Bob

bởi Tim Ottinger và Uncle Bob



Beginning in the late '90s I ran a company named **Object** Mentor Inc. Our website sported a number of technical documents that we offered to the software community. The most downloaded of all those documents was Tim Ottinger's rules for names. What you are about to read is the descendant and natural evolution of Tim's classic document.

Từ cuối những năm 90, tôi điều hành một công ty tên là Object Mentor Inc. Trên website của chúng tôi có một loạt tài liệu kỹ thuật dành tặng cộng đồng phần mềm. Tài liệu được tải về nhiều nhất trong số đó là bộ quy tắc đặt tên của Tim Ottinger. Những gì bạn sắp đọc chính là hậu duệ, là bước tiến hoá tự nhiên của tài liệu kinh điển ấy.

—Uncle Bob

—Uncle Bob

Names are everywhere in software. We name our variables, our functions, our arguments, classes, and packages. We name our source files and the directories that contain them. We name our jar files and dll files and gem files. We name and name and name; and because we do so much of it, we'd better do it well.

Tên có mặt ở khắp nơi trong phần mềm. Ta đặt tên cho biến, cho hàm, cho đối số, cho class và package. Ta đặt tên cho tệp mã nguồn và cho cả thư mục chứa chúng. Ta đặt tên cho tệp jar, tệp dll, tệp gem. Ta đặt tên rồi lại đặt tên, đặt tên nữa; và vì làm việc đó nhiều đến thế, tốt nhất là ta nên làm cho ra hồn.

It might seem that names are so subjective that defining the principles of “goodness” is a fool’s errand. However, despite their subjectivity, we know of **good** restaurants, good movies, good paintings, and good books.

Thoạt nhìn, có vẻ tên gọi là chuyện chủ quan đến mức đi định nghĩa nguyên tắc thế nào là “hay” chỉ là công dã tràng. Nhưng dù chủ quan đến đâu, chúng ta vẫn biết đâu là nhà hàng ngon, phim hay, tranh đẹp và sách tốt.

Subjectivity is tempered by the realization that any work is good if it is accepted by its audience. This means that we aren’t dealing with individual preferences and prejudices, but rather, with the desires of a collective audience.

Tính chủ quan được tiết chế lại bởi một nhận thức: bất kỳ tác phẩm nào cũng là tốt nếu khán giả của nó chấp nhận nó. Nghĩa là ta không bàn về sở thích và định kiến của từng cá nhân, mà bàn về mong muốn của cả một tập thể độc giả.

The audience for this book is the set of programmers who are interested enough in improving their code **craft** to purchase, borrow, or in some other way acquire a copy of this book. If the writing reaches you and helps you, then we’ve written good-enough words in a good-enough order.

Độc giả của cuốn sách này là những lập trình viên đủ quan tâm đến việc nâng cao tay nghề viết mã của mình để bỏ tiền mua, đi mượn, hoặc bằng cách nào đó có được một bản. Nếu những dòng chữ này chạm được tới bạn và giúp ích cho bạn, thì nghĩa là chúng tôi đã viết ra những từ đủ tốt, sắp xếp theo một trật tự đủ tốt.

As an author of code, your audience is the set of people who will work in your codebase. You are not only an author, but also a member of the codebase community. The audience is larger than you are, and they collectively will decide what is welcome and what is unwelcome in the codebase.

Với tư cách người viết mã, độc giả của bạn là tập hợp những người sẽ làm việc trên codebase của bạn. Bạn không chỉ là tác giả, bạn còn là một thành viên của cộng đồng codebase ấy. Đám đông đó lớn hơn bạn, và chính họ sẽ cùng nhau quyết định điều gì được chào đón và điều gì không được chào đón trong codebase.

Write for your audience, not for yourself. You must remember that your experience of writing the code is less important and more transient than their experience of reading that code.

Hãy viết cho độc giả của bạn, đừng viết cho bản thân bạn. Bạn phải nhớ rằng trải nghiệm viết ra đoạn mã của bạn ít quan trọng hơn và cũng chóng qua hơn nhiều so với trải nghiệm đọc đoạn mã đó của họ.

That said, we find there is a fairly consistent band of preferences across many audiences, and this generalized advice is intended to help you find that “sweet spot” with your current software audience.

Dẫu vậy, chúng tôi nhận thấy có một dải sở thích khá nhất quán trải rộng qua nhiều nhóm độc giả khác nhau, và những lời khuyên tổng quát ở đây nhằm giúp bạn tìm ra “điểm ngọt”

đó với nhóm đọc giả phần mềm hiện tại của mình.

Choosing good names takes time, but saves more than it takes. So take care with your names and change them when you find better ones. Everyone who reads your code (including you) will be happier if you do.

Chọn được tên hay thì tốn thời gian, nhưng cái nó tiết kiệm được còn nhiều hơn cái nó lấy đi. Vậy nên hãy chăm chút cho tên gọi của bạn, và hãy đổi chúng khi bạn tìm được cái tên hay hơn. Mọi người đọc mã của bạn (kể cả chính bạn) đều sẽ vui hơn nếu bạn làm thế.

To help you in that effort, here are some of the more useful recommendations we have found.

Để giúp bạn trong nỗ lực đó, dưới đây là một vài khuyến nghị hữu ích nhất mà chúng tôi đúc kết được.

Use Intention-Revealing Names — Dùng tên bộc lộ ý định

The name of a **variable**, **function**, or **class** should answer all the big questions. It should tell its audience why it exists, what it does, and how it is used.

Tên của một biến, một hàm hay một class phải trả lời được mọi câu hỏi lớn. Nó phải nói cho độc giả biết vì sao nó tồn tại, nó làm gì, và nó được dùng ra sao.

If a name requires a **comment** to communicate to its audience, then the name does not reveal its intent.

Nếu một cái tên cần tới comment mới truyền đạt được ý cho độc giả, thì cái tên đó không bộc lộ được ý định của nó.

```
int d; // elapsed time in days
```

The name `d` reveals very little. It does not evoke a sense of elapsed time; specifically, of days. We should choose a name that specifies what is being measured and the unit of that measurement. One of these might be better: `elapsedDays`, `daysSinceCreation`,

Cái tên `d` chẳng tiết lộ được bao nhiêu. Nó không gợi lên cảm giác về một khoảng thời gian đã trôi qua, cụ thể hơn là về số ngày. Ta nên chọn một cái tên nói rõ thứ đang được đo và đơn vị đo của nó. Một trong những cái tên sau có lẽ khá hơn: `elapsedDays`, `daysSinceCreation`,

`daysSinceModification`, `orfileAgeInDays`.

These examples are longer than the original name (`d`), but that doesn't necessarily mean that long names are better than short names, only that clear names are better than unclear ones. For example:

Những ví dụ này dài hơn cái tên ban đầu (`d`), nhưng điều đó không có nghĩa tên dài thì tốt hơn tên ngắn, mà chỉ có nghĩa tên rõ ràng thì tốt hơn tên mù mờ. Ví dụ:

```
int d = getElapsedTimeInDays();
```

Here the name is perfectly clear since it is defined by the context and would be appropriate if the **scope** that contains the name is relatively small. (See “Use Names of Appropriate Length.”)

Ở đây cái tên hoàn toàn rõ ràng vì nó đã được ngữ cảnh định nghĩa sẵn, và nó sẽ là lựa chọn phù hợp nếu phạm vi (scope) chứa nó tương đối nhỏ. (Xem mục “Dùng tên có độ dài phù hợp.”)

Build a System of Names — Xây dựng một hệ thống tên gọi

Choosing names that reveal intent can make it much easier to understand and change code. What follows is an example that breaks this rule. Can you determine the purpose of this code?

Chọn những cái tên bộc lộ ý định có thể khiến việc hiểu và sửa mã dễ hơn rất nhiều. Sau đây là một ví dụ vi phạm quy tắc này. Bạn có đoán được đoạn mã này dùng để làm gì không?

```
public List<int[]> getThem() {  
    List<int[]> list1 = new ArrayList<int[]>();  
  
    for (int[] x : theList)  
        if (x[0] == 4)  
            list1.add(x);  
    return list1;  
}
```

Why is it hard to tell what this code is doing? There are no complex expressions. Spacing and **indentation** are reasonable. There are only three variables and two constants mentioned. There aren't even any fancy classes or polymorphic methods, just a list of arrays (or so it seems).

Vì sao khó nói được đoạn mã này đang làm gì? Chẳng có biểu thức nào phức tạp cả. Khoảng trắng và thụt lề (indentation) đều hợp lý. Chỉ có ba biến và hai hằng số được nhắc đến. Thậm chí cũng không có class cầu kỳ hay method đa hình (polymorphism) nào, chỉ là một danh sách các mảng (hoặc trông có vẻ như vậy).

The problem isn't the simplicity of the code. The problem is that the code provides no context. The code implicitly requires that we know the answers to questions such as

Vấn đề không nằm ở chỗ đoạn mã đơn giản. Vấn đề là đoạn mã chẳng cung cấp chút ngữ cảnh nào. Nó ngầm đòi hỏi ta phải biết sẵn câu trả lời cho những câu hỏi kiểu như

- What kinds of things are in theList ?
- What does the zeroth subscript signify?
- What is the meaning of the value 4?
- How would I use the list being returned?
 - Trong theList có những thứ gì?
 - Chỉ số thứ 0 mang ý nghĩa gì?
 - Giá trị 4 nghĩa là gì?
 - Tôi sẽ dùng cái danh sách được trả về như thế nào?

The answers to these questions are not present in the code sample. However, a good system of names would have made them clear.

Câu trả lời cho những câu hỏi này không hề có trong đoạn mã mẫu. Nhưng một hệ thống tên gọi tốt hẳn đã làm chúng trở nên rõ ràng.

Let's say that code came from the old Minesweeper game. 1 The Minesweeper game board is a list of cells called theList . Let's rename that to gameBoard .

Giả sử đoạn mã đó đến từ trò Minesweeper ngày xưa. 1 Bàn chơi Minesweeper là một danh sách các ô tên là theList . Hãy đổi tên nó thành gameBoard .

1 A game from the '90s that shipped with Windows 3.11.

1 Một trò chơi từ thập niên 90, đi kèm với Windows 3.11.

Each cell on the board is represented by a primitive array. The zeroth subscript is the location of a status value, and a status value of 4 means “flagged.” Just by giving these concepts names we can improve the code considerably:

Mỗi ô trên bàn chơi được biểu diễn bằng một mảng kiểu nguyên thủy. Chỉ số thứ 0 là vị trí chứa giá trị trạng thái, và trạng thái mang giá trị 4 nghĩa là “đã cắm cờ”. Chỉ cần đặt tên cho những khái niệm đó thôi là ta đã cải thiện đoạn mã đáng kể:

```
public List<int[]> getFlaggedCells() {
    List<int[]> flaggedCells = new ArrayList<int[]>();
    for (int[] cell : gameBoard)
        if (cell[STATUS_VALUE] == FLAGGED)
            flaggedCells.add(cell);
    return flaggedCells;
}
```

Notice that the simplicity of the code has not changed. Only the clarity has improved.

Để ý rằng độ đơn giản của đoạn mã không hề thay đổi. Chỉ có độ rõ ràng là tốt lên.

It still has exactly the same number of operators and constants, with exactly the same number of nesting levels. But the code has become much more explicit through naming.

Nó vẫn có đúng chừng ấy toán tử và hằng số, đúng chừng ấy tầng lồng nhau. Nhưng nhờ cách đặt tên, đoạn mã đã trở nên tường minh hơn hẳn.

Note also that if this was the very first bit of code you came across, you would already know that this was a game with a board of cells that had statuses, one of which was “flagged.” A good system of names communicates a great deal about the application as a whole.

Cũng để ý rằng nếu đây là mẫu mã đầu tiên bạn bắt gặp, bạn đã lập tức biết được rằng đây là một trò chơi có bàn chơi gồm các ô, mỗi ô có trạng thái, và một trong các trạng thái đó là “đã cắm cờ”. Một hệ thống tên gọi tốt nói lên rất nhiều điều về toàn bộ ứng dụng.

We can go further and write a simple class for cells instead of using an array of int s. It can include an intention-revealing function (call it isFlagged) to hide the magic numbers. It results in a new version of the function:

Ta còn có thể đi xa hơn: viết hẳn một class đơn giản cho ô thay vì dùng mảng int . Class đó có thể có một hàm bộc lộ ý định (cứ gọi nó là isFlagged) để giấu đi các con số ma thuật. Kết quả là một phiên bản mới của hàm:

```
public List<Cell> getFlaggedCells() {
    List<Cell> flaggedCells = new ArrayList<Cell>();
    for (Cell cell : gameBoard)
        if (cell.isFlagged())
            flaggedCells.add(cell);
    return flaggedCells;
}
```

With these simple name changes, it’s not difficult to understand what’s going on. This is the power of choosing good names.

Chỉ với vài thay đổi tên đơn giản như vậy, hiểu chuyện gì đang diễn ra chẳng còn khó nữa. Đó chính là sức mạnh của việc chọn tên hay.

Avoid Disinformation — Tránh thông tin sai lệch

Programmers must avoid leaving false clues that obscure the meaning of code. For example, do not refer to a grouping of accounts as an `accountList` unless it's actually a `List`.

Lập trình viên phải tránh để lại những manh mối giả làm mờ đi ý nghĩa của mã. Ví dụ, đừng gọi một nhóm tài khoản là `accountList` trừ phi nó thật sự là một `List`.

The word `list` means something specific to programmers. If the container holding the accounts is an array or a JSON document, it may lead to false conclusions. ² So `accountGroup` or `bunchOfAccounts` or just plain `accounts` may be shorter (and thereby easier to take in at a glance) and more honest.

Từ `list` mang một ý nghĩa rất cụ thể đối với lập trình viên. Nếu vật chứa các tài khoản lại là một mảng hay một tài liệu JSON, cái tên ấy có thể dẫn người đọc tới kết luận sai. ² Vậy nên `accountGroup`, `bunchOfAccounts`, hay đơn giản là `accounts`, vừa có thể ngắn hơn (nhờ đó dễ nắm bắt chỉ trong một cái liếc mắt) vừa trung thực hơn.

² As we'll see later on, even if the container is a `List`, it's probably better not to encode the container type into the name.

² Như ta sẽ thấy ở phần sau, ngay cả khi vật chứa đúng là một `List`, có lẽ vẫn tốt hơn nếu đừng mã hoá kiểu của vật chứa vào trong tên.

Beware of using names that vary in small ways. How long does it take to spot the subtle difference between an `XYZControllerForEfficientHandlingOfStrings` in one **module** and, somewhere a little more distant, an `XYZControllerForEfficientStorageOfStrings`? The words have frightfully similar shapes. Would you realize that they were different if they weren't right next to each other?

Hãy cảnh giác với những cái tên chỉ khác nhau chút xíu. Bạn mất bao lâu để nhận ra khác biệt tinh vi giữa một `XYZControllerForEfficientHandlingOfStrings` ở module này và, ở đâu đó xa hơn một chút, một `XYZControllerForEfficientStorageOfStrings`? Hai từ có hình dáng giống nhau đến phát sợ. Liệu bạn có nhận ra chúng khác nhau nếu chúng không nằm sát cạnh nhau?

Those names are doubly difficult because they are long and have similar beginnings. Can you spot the optical illusion below?

Những cái tên đó khó gấp đôi vì chúng vừa dài vừa có phần đầu giống nhau. Bạn có phát hiện ra ảo giác thị giác dưới đây không?

```
XYZControllerForEfficientHandlingOfStrings  
XXZControllerForEfficientHandlingOfStrings
```

Spelling similar concepts similarly is information. Using inconsistent spellings is **disinformation**. With modern environments, we enjoy automatic code completion. We write a few characters of a name and are quickly rewarded with a list of possible completions for that name. Inconsistent spellings will confound that list.

Viết những khái niệm giống nhau theo cách giống nhau là thông tin. Viết không nhất quán là thông tin sai lệch. Với môi trường phát triển hiện đại, ta được hưởng tính năng tự động hoàn thành mã. Ta gõ vài ký tự của một cái tên và lập tức được thưởng bằng một danh sách các gợi ý hoàn thành cho cái tên đó. Cách viết không nhất quán sẽ làm rối tung danh sách ấy.

Modern environments also help by using different fonts, styles, and colors for different syntactic elements. Still, it is wise to be careful of some of the optical illusions inherent in our fonts. For

example, be careful with the use of lowercase L and uppercase O. In certain fonts, they can be indistinguishable from the constants 1 and 0, respectively.

Môi trường hiện đại cũng giúp ta bằng cách dùng font, kiểu chữ và màu sắc khác nhau cho các thành phần cú pháp khác nhau. Dù vậy, khôn ngoan nhất vẫn là cẩn thận với một số ảo giác thị giác vốn có trong các font chữ. Chẳng hạn, hãy cẩn thận khi dùng chữ L thường và chữ O hoa. Ở một số font, chúng gần như không thể phân biệt được với hằng số 1 và 0.

```
int a = 1;
if ( 0 == 1 )
    a = 01;
else
    1 = 01;
```

Combining these letters with numbers and x can make it worse. Consider Oxll as a name.

Trộn những chữ cái đó với chữ số và chữ x thì còn tệ hơn nữa. Thử nghĩ tới cái tên Oxll xem.

Make Meaningful Distinctions — Tạo ra những khác biệt có ý nghĩa

Programmers create problems for themselves when they write code solely to satisfy a compiler or interpreter. For example, when you want to use the same name to refer to two different things in the same scope, you might be tempted to change one name in an arbitrary way to keep them distinct. Sometimes this is done by misspelling one, leading to the surprising situation where correcting spelling errors leads to an inability to compile. 3

Lập trình viên tự tạo rắc rối cho mình khi viết mã chỉ để làm vừa lòng trình biên dịch hay trình thông dịch. Ví dụ, khi bạn muốn dùng cùng một cái tên để chỉ hai thứ khác nhau trong cùng một phạm vi, bạn có thể bị cám dỗ sửa một trong hai cái tên một cách tùy tiện cho chúng khác nhau. Đôi khi việc này được thực hiện bằng cách viết sai chính tả một cái, dẫn tới tình huống dở khóc dở cười là sửa lỗi chính tả xong thì chương trình không biên dịch được nữa. 3

3 Consider, for example, the truly hideous practice of creating a variable named `klass` just because the name `class` was used for something else.

3 Hãy nghĩ tới, chẳng hạn, cái thói quen thật sự gớm ghiếc là tạo một biến tên `klass` chỉ vì cái tên `class` đã bị dùng cho thứ khác.

It is not sufficient to add number series or noise words, even though the compiler is satisfied. If names must be different, then they should also mean something different.

Thêm dãy số hay từ nhiều vào tên là chưa đủ, dù trình biên dịch có hài lòng đi nữa. Nếu các tên buộc phải khác nhau, thì chúng cũng phải mang ý nghĩa khác nhau.

Number-series naming (`a1` , `a2` , ... `aN`) is the opposite of intentional naming. Such names are not dis-informative, they are uninformative; they provide no clue to the author's intention. Consider:

Đặt tên theo dãy số (`a1` , `a2` , ... `aN`) là thứ đối lập với đặt tên có chủ đích. Những cái tên như vậy không đưa thông tin sai lệch, chúng đơn giản là chẳng đưa thông tin gì; chúng không hé lộ chút manh mối nào về ý định của tác giả. Hãy xem:

```
public static void copyChars(char a1[], char a2[]) {
```



```

for (int i = 0; i < a1.length; i++) {
    a2[i] = a1[i];
}
}

```

This function reads much better when source and destination are used for the **argument** names.

Hàm này đọc lên xuôi tai hơn nhiều khi dùng source và destination làm tên đối số.

In general, when you see variables named X1 , X2 , X3 , the code is practically begging you for either an array or a **data structure**. Consider what makes them so similar that they have the same name and only a numeric difference, and look for some meaning that will inspire either different names or different structures entirely.

Nói chung, khi bạn thấy các biến tên X1 , X2 , X3 , đoạn mã gần như đang van nài bạn hãy cho nó một mảng hoặc một cấu trúc dữ liệu (data structure). Hãy nghĩ xem điều gì khiến chúng giống nhau đến mức phải mang cùng một cái tên và chỉ khác nhau ở con số, rồi tìm một ý nghĩa nào đó gợi ra được những cái tên khác nhau, hoặc những cấu trúc hoàn toàn khác nhau.

In the code above, it was just lazy naming. Changing those names to source and destination makes a better solution than trying to create a **structure**, class, or container. Not all problems are deep.

Trong đoạn mã trên, đó chỉ là sự lười biếng trong việc đặt tên. Đổi những cái tên đó thành source và destination là giải pháp tốt hơn so với việc cố nặn ra một cấu trúc, một class hay một vật chứa. Không phải vấn đề nào cũng sâu xa.

Noise words are another meaningless distinction. Imagine that you have a Product class. If you have another called ProductInfo or ProductData , you have made the names different without making them mean anything different. Info and **Data** are indistinct noise words. They add no new information, because every class contains “data” and every class contains “info.”

Từ nhiễu là một kiểu phân biệt vô nghĩa khác. Hãy tưởng tượng bạn có một class Product . Nếu bạn có thêm một class nữa tên ProductInfo hay ProductData , thì bạn đã làm cho các tên khác nhau mà không làm cho chúng mang ý nghĩa khác nhau. Info và Data là những từ nhiễu chẳng phân biệt được gì. Chúng không thêm thông tin mới nào cả, bởi class nào cũng chứa “data” và class nào cũng chứa “info”.

Noise words are redundant. The word variable should never appear in a variable name. The word table should never appear in a table name. How is NameString better than Name ? Would a Name ever be a floating-point number? If so, it likely breaks an earlier rule about disinformation. Imagine finding one class named Customer and another named CustomerObject . What should you understand as the distinction? Which one will represent the best path to a customer’s payment history?

Từ nhiễu là thừa thãi. Từ variable không bao giờ nên xuất hiện trong tên một biến. Từ table không bao giờ nên xuất hiện trong tên một bảng. NameString thì hơn Name ở chỗ nào? Chẳng lẽ có lúc nào đó một Name lại là số dấu phẩy động? Nếu có thì nó đã vi phạm quy tắc phía trên về thông tin sai lệch rồi. Hãy tưởng tượng bạn tìm thấy một class tên Customer và một class khác tên CustomerObject . Bạn phải hiểu sự khác biệt ở đây là gì? Cái nào mới là con đường tốt nhất dẫn tới lịch sử thanh toán của một khách hàng?

Here are some disturbingly meaningless function names we found in one particular application.

Dưới đây là vài cái tên hàm vô nghĩa đến mức đáng lo mà chúng tôi tìm thấy trong một ứng dụng nọ.

```
getActiveAccount();  
getActiveAccounts();  
getActiveAccountInfo();
```

How are the programmers in this project supposed to know which of these functions to call? How many active accounts are there? Which of them are returned by the first function? And what is the difference between the second and third functions? In any universe, this “system” of names is a nightmare.

Làm sao các lập trình viên trong dự án này biết được phải gọi hàm nào trong số đó? Có bao nhiêu tài khoản đang hoạt động? Hàm đầu tiên trả về những tài khoản nào trong số đó? Và hàm thứ hai khác hàm thứ ba ở chỗ nào? Ở bất kỳ vũ trụ nào thì cái “hệ thống” tên gọi này cũng là một cơn ác mộng.

Use Pronounceable Names — Dùng tên đọc lên được

Humans are good at words. A significant part of our brains is dedicated to the concept of words. And words are, by definition, pronounceable. It would be a shame not to take advantage of that huge portion of our brains that has evolved to deal with spoken language. So make your names pronounceable.

Con người rất giỏi về từ ngữ. Một phần đáng kể bộ não của chúng ta được dành riêng cho khái niệm từ ngữ. Và theo định nghĩa, từ ngữ là thứ đọc lên được. Sẽ thật đáng tiếc nếu không tận dụng cái phần khổng lồ đó của bộ não, cái phần đã tiến hoá để xử lý ngôn ngữ nói. Vậy nên hãy đặt những cái tên đọc lên được.

If you can’t pronounce it, you can’t conveniently discuss it. “Well, over here on the bee cee arr three cee enn tee we have a pee ess zee kyew int, see?” Convenience of discussion matters because programming is a social activity.

Nếu bạn không đọc được nó, bạn không thể bàn về nó một cách thuận tiện. “Ừ thì, chỗ này trên cái bê-xê-e-rờ-ba-xê-en-tê ta có một cái pê-ét-dét-kiu kiểu int, thấy chưa?” Sự thuận tiện khi trao đổi là quan trọng, bởi lập trình là một hoạt động mang tính xã hội.

One company had the name genymdhms (generation date, year, month, day, hour, minute, and second), so they walked around saying “gen why emm dee aich emm ess.” This is inconvenient, so the pronunciation gradually evolved into “gen-yah-mudda-hims.” This might sound silly, but they were all in on the joke, so it was fun. Fun or not, they were tolerating poor naming. New developers had to have the variables explained to them, and then they spoke about it in silly, made-up words instead of using proper English terms.

Một công ty nọ có cái tên genymdhms (generation date, year, month, day, hour, minute, and second), thế là họ đi khắp nơi và nói “gen why emm dee aich emm ess”. Đọc như vậy bất tiện quá, nên cách phát âm dần biến thành “gen-yah-mudda-hims”. Nghe thì ngớ ngẩn, nhưng cả đám đều hiểu trò đùa nên nó cũng vui. Vui hay không thì họ cũng đang dung túng cho cách đặt tên tệ. Lập trình viên mới vào phải được giải thích những biến đó là gì, rồi sau đó lại nói về chúng bằng những từ bịa ngớ ngẩn thay vì dùng thuật ngữ tiếng Anh đàng hoàng.

Compare:

Hãy so sánh:

```
class DtaRcrrd102 {  
    private Date genymdhms;
```



```
private Date modymdhms;  
private final String pszqint = "102";
```

to:

với:

```
class Customer {  
    private Date generated;  
    private Date modified;  
    private final String recordId = "102";
```

Intelligent conversation is now possible: “Hey, Mikey, take a look at this record! The generation timestamp is set to tomorrow’s date! How can that be?”

Giờ thì trò chuyện tử tế đã khả thi: “Này Mikey, xem bản ghi này đi! Cái generation timestamp bị đặt thành ngày mai! Sao lại thế được nhỉ?”

Use Searchable Names — Dùng tên tìm kiếm được

Single-letter names and numeric constants have a particular problem in that they are not easy to locate across a body of text. One might easily grep for `MAX_CLASSES_PER_STUDENT`, but the number 7 could be more troublesome. Searches may turn up the digit as part of filenames, as other constant definitions, and in various expressions where the value is used with different intent. It is even worse when a constant is a long number and someone might have transposed digits, thereby creating a bug while simultaneously evading the programmer’s search.

Tên một chữ cái và hằng số dạng số có một vấn đề riêng: chúng không dễ định vị trong cả một khối văn bản. Bạn có thể grep `MAX_CLASSES_PER_STUDENT` một cách dễ dàng, nhưng con số 7 thì rắc rối hơn nhiều. Kết quả tìm kiếm có thể trả về chữ số đó khi nó là một phần của tên tệp, khi nó nằm trong định nghĩa hằng số khác, và trong đủ loại biểu thức mà giá trị ấy được dùng với ý nghĩa hoàn toàn khác. Còn tệ hơn nữa khi hằng số là một con số dài và ai đó lỡ đảo hai chữ số, vừa tạo ra một bug vừa né được luôn lệnh tìm kiếm của lập trình viên.

Likewise, the name `e` is not a searchable name. It is the most common letter in the English language and likely to show up in every passage of text in every program. In general, short names are less searchable than long names, which is one more reason that when it comes to being searchable, longer names can trump shorter names, and any searchable name trumps a constant in code.

Tương tự, cái tên `e` không phải là một cái tên tìm kiếm được. Nó là chữ cái phổ biến nhất trong tiếng Anh và gần như chắc chắn xuất hiện trong mọi đoạn văn bản của mọi chương trình. Nói chung, tên ngắn khó tìm kiếm hơn tên dài, và đó là thêm một lý do nữa để nói rằng, xét về khả năng tìm kiếm, tên dài có thể thắng tên ngắn, và bất kỳ cái tên tìm kiếm được nào cũng thắng một hằng số nằm lơ lửng trong mã.

The name `e` is often used to denote the specific error that was raised in a `try / catch` block, especially in Python and Go. In this case, though, the name is familiar in its context to many programmers, and it occurs in such a small scope as to make searches unnecessary. Short names in limited and well-known circumstances generally don’t confound searches .

Cái tên `e` thường được dùng để chỉ lỗi cụ thể được ném ra trong một khối `try / catch` , nhất là trong Python và Go. Tuy nhiên, trong trường hợp này cái tên đó quen thuộc với nhiều lập trình viên trong đúng ngữ cảnh của nó, và nó nằm trong một phạm vi nhỏ đến mức

chẳng cần phải tìm kiếm gì. Tên ngắn, trong những hoàn cảnh hạn hẹp và ai cũng biết, nhìn chung không làm rối việc tìm kiếm.

Our preference is that very short, or single-letter, names ought only to be used as local variables inside very short scopes. If a variable or constant might be seen or used in multiple places in a body of code, it is best to give it a search-friendly name. Once again, compare:

Chúng tôi thiên về quan điểm rằng những cái tên rất ngắn, hoặc chỉ một chữ cái, chỉ nên dùng cho biến cục bộ bên trong những phạm vi rất ngắn. Nếu một biến hay một hằng số có thể được nhìn thấy hoặc được dùng ở nhiều nơi trong một khối mã, tốt nhất là hãy cho nó một cái tên thân thiện với tìm kiếm. Một lần nữa, hãy so sánh:

```
for (int j=0; j<34; j++) {  
    s += (t[j]*4)/5;  
}
```

to:

với:

```
int realDaysPerIdealDay = 4;  
const int WORK_DAYS_PER_WEEK = 5;  
int sum = 0;  
for (int j=0; j < NUMBER_OF_TASKS; j++) {  
    int realTaskDays = taskEstimate[j] * realDaysPerIdealDay;  
    int realTaskWeeks = (realTaskDays / WORK_DAYS_PER_WEEK);  
    sum += realTaskWeeks;  
}
```

Note that `sum`, above, is not a particularly useful name but at least is searchable. The intentionally named code makes for a longer function, but consider how much easier it will be to find `WORK_DAYS_PER_WEEK` than to find all the places where 5 was used and filter the list down to just the instances with the intended meaning.

Để ý rằng `sum` ở trên không phải cái tên đặc biệt hữu ích, nhưng ít nhất nó tìm kiếm được. Đoạn mã được đặt tên có chủ đích thì hàm dài hơn, nhưng hãy nghĩ xem tìm `WORK_DAYS_PER_WEEK` sẽ dễ hơn bao nhiêu so với việc tìm mọi chỗ có dùng số 5 rồi lọc danh sách đó xuống còn đúng những chỗ mang ý nghĩa mình cần.

Use Names of Appropriate Length — Dùng tên có độ dài phù hợp

Our rules for the length of names may surprise you, but they have withstood the **test** of time.

Các quy tắc của chúng tôi về độ dài của tên có thể sẽ khiến bạn ngạc nhiên, nhưng chúng đã đứng vững qua thử thách của thời gian.

Variable Names

Tên biến

We have a simple rule for the names of variables. The length of a variable's name should be proportional to the length of the scope that contains it. For example:

Chúng tôi có một quy tắc đơn giản cho tên biến. Độ dài tên một biến nên tỉ lệ thuận với độ dài của phạm vi chứa nó. Ví dụ:

```
for (int n=0; n<20; n++)  
    System.out.println(" " + n + " " + n*n);
```

The name `n` is appropriately short because its scope is two lines long. We would find it annoying if that variable were named

Cái tên `n` ngắn như vậy là hợp lý, bởi phạm vi của nó chỉ dài hai dòng. Chúng tôi sẽ thấy khó chịu nếu biến đó được đặt tên là

On the other hand, in a longer scope, such small names can be opaque. We would usually find an instance variable named `n` annoying; especially if `m`, `p`, `i`, and `j` were also instance variables.

Ngược lại, trong một phạm vi dài hơn, những cái tên bé tí như vậy lại trở nên mờ mịt. Chúng tôi thường thấy khó chịu với một biến thành viên tên `n`; nhất là khi `m`, `p`, `i` và `j` cũng là biến thành viên.

As the size of the scope increases, so the length of the variable names within that scope should increase. Arguments and local variables are scoped to a single function and so should have relatively short names; perhaps a word or two. Instance variables are scoped to a whole class. Their names should be relatively longer. Global variable names should be longer still.

Phạm vi càng lớn thì tên các biến bên trong phạm vi đó càng phải dài ra. Đối số và biến cục bộ có phạm vi gói gọn trong một hàm nên tên chúng có thể tương đối ngắn; chừng một hai từ là đủ. Biến thành viên có phạm vi là cả class. Tên của chúng nên dài hơn một chút. Tên biến toàn cục thì còn phải dài hơn nữa.

Function Names

Tên hàm

Our rule for functions is exactly the opposite. The length of their names should be inversely proportional to the length of their scope. Thus, global functions will have short names, while private methods within a class will have relatively long names. Tests, which have the shortest scope of all (effectively zero), have the longest names of all.

Quy tắc của chúng tôi cho hàm thì ngược lại hoàn toàn. Độ dài tên hàm nên tỉ lệ nghịch với độ dài phạm vi của nó. Nghĩa là hàm toàn cục sẽ có tên ngắn, còn method private bên trong một class sẽ có tên tương đối dài. Test, thứ có phạm vi ngắn nhất trong tất cả (gần như bằng không), lại có tên dài nhất trong tất cả.

We justify this in two ways. First, the larger the scope of a function, the more frequently it will be used. So, for the sake of convenience, we'd like the name to be short. We would find it quite annoying if the function that opens a file was named `openFileAndThrowExceptionIfNotFound` instead of `open`.

Chúng tôi biện minh cho điều này bằng hai lý do. Thứ nhất, phạm vi của một hàm càng lớn thì nó càng được dùng thường xuyên. Vậy nên, vì lẽ tiện lợi, ta muốn tên nó ngắn. Chúng tôi sẽ thấy khá bức bối nếu cái hàm mở tệp lại được đặt tên là `openFileAndThrowExceptionIfNotFound` thay vì `open`.

Second, the larger the scope of a function, the more general it is. Again, `open` is a very general function. Methods within a class have a much narrower context and so will likely need more words to explain their intent. Private methods within a class have very small scopes and are called from very few places, or even just one. They are not general at all and may therefore require many words to describe properly. Their names should be correspondingly longer.

Thứ hai, phạm vi của một hàm càng lớn thì nó càng tổng quát. Một lần nữa, `open` là một hàm rất tổng quát. Các method bên trong một class có ngữ cảnh hẹp hơn nhiều, nên nhiều

khả năng sẽ cần thêm từ để nói rõ ý định của chúng. Các method private trong một class có phạm vi rất nhỏ và chỉ được gọi từ vài nơi, thậm chí chỉ một nơi. Chúng chẳng tổng quát chút nào, và vì thế có thể cần khá nhiều từ mới mô tả được cho đúng. Tên của chúng cũng nên dài ra tương ứng.

Class Names

Tên class

Our rule for classes is the same as our rule for functions. The longer the scope, the shorter the name. Classes at the global scope should have relatively short names (a word or two), while derived classes, inner classes, and private classes should have longer names.

Quy tắc của chúng tôi cho class cũng giống quy tắc cho hàm. Phạm vi càng dài thì tên càng ngắn. Class ở phạm vi toàn cục nên có tên tương đối ngắn (một hai từ), còn class dẫn xuất, class lồng bên trong và class private thì nên có tên dài hơn.

Property and Attribute Names

Tên property và attribute

Some languages, like C++ and C#, have elements that look like variables but are actually functions. They look like variables because they can be put on the left of assignment statements. Therefore, they should follow the variable-naming rule. The length of their names should be proportional to the scope that contains them.

Một số ngôn ngữ, như C++ và C#, có những thành phần trông như biến nhưng thực chất là hàm. Chúng trông như biến vì có thể đặt ở vế trái của lệnh gán. Vì vậy, chúng nên tuân theo quy tắc đặt tên biến. Độ dài tên của chúng nên tỉ lệ thuận với phạm vi chứa chúng.

Namespace Names

Tên namespace

Most languages now allow us to put our classes, functions, and variables into namespaces. Namespace names should be descriptive. Thus, they are likely to be longish and are often part of a hierarchy of namespaces. Consider, for example, the namespace from a space war game 4 Uncle Bob wrote a few years back: `spacewar.ui.tactical-scan`. Such long and qualified names are generally inconvenient, which is why most languages that have namespaces have some way to create namespace aliases that can be used within smaller scopes.

Ngày nay, phần lớn ngôn ngữ đều cho phép ta đặt class, hàm và biến vào trong các namespace. Tên namespace nên mang tính mô tả. Vì vậy chúng thường hơi dài dòng và thường là một mắt xích trong cả một cây phân cấp namespace. Chẳng hạn, hãy xem namespace trong một trò chơi chiến tranh không gian 4 mà Uncle Bob viết vài năm trước: `spacewar.ui.tactical-scan`. Những cái tên dài và đầy đủ định danh như vậy nhìn chung khá bất tiện, và đó là lý do hầu hết ngôn ngữ có namespace đều trang bị cách nào đó để tạo bí danh namespace dùng được trong những phạm vi nhỏ hơn.

4 <https://github.com/unclebob/spacewar>

4 <https://github.com/unclebob/spacewar>

Thus, while the length of namespace names is not a function of the scope that contains them, the length of their aliases is. In a very short module, we might use the alias `ts`, whereas in a longer module, we might use `tac-scan`, and in a very long module (which ought to be rare), we might use `tactical-scan`.

Vậy nên, tuy độ dài tên namespace không phụ thuộc vào phạm vi chứa chúng, độ dài bí danh của chúng thì có. Trong một module rất ngắn, ta có thể dùng bí danh ts , còn trong một module dài hơn, ta có thể dùng tac-scan , và trong một module rất dài (điều lẽ ra phải hiếm), ta có thể dùng tactical-scan .

Avoid Encodings — Tránh mã hoá tên

We have enough encodings to deal with without adding more to our burden. **Encoding** type or scope information into names simply adds an extra burden of deciphering. It hardly seems reasonable to require each new employee to learn yet another encoding “language” in addition to learning the (usually considerable) body of code that they’ll be working in. On top of that, encoded names are seldom pronounceable and are easy to mistype.

Chúng ta đã phải xoay sở với đủ thứ mã hoá rồi, đừng chất thêm gánh nặng nữa. Nhờ thông tin về kiểu hay phạm vi (scope) vào trong tên chỉ tạo thêm một lớp phải giải mã. Thật khó mà hợp lý khi bắt mỗi nhân viên mới phải học thêm một “ngôn ngữ” mã hoá nữa, bên cạnh khối lượng mã (thường là khổng lồ) mà họ sắp phải làm việc cùng. Chưa kể, những cái tên đã bị mã hoá thì hiếm khi đọc lên được và rất dễ gõ sai.

In days of old, when we worked in name length–challenged languages, we violated this rule out of necessity and with regret. Fortran forced encodings by making the first letter a code for the type. Early versions of BASIC allowed only a letter plus one digit. Hungarian notation 5 (HN) took this to a whole new level.

Ngày xưa, khi còn làm việc với những ngôn ngữ giới hạn độ dài tên, chúng ta vi phạm quy tắc này vì bắt buộc phải thế, và trong lòng thì tiếc nuối. Fortran ép ta mã hoá tên bằng cách lấy chữ cái đầu tiên làm ký hiệu cho kiểu. Các phiên bản BASIC đời đầu chỉ cho phép một chữ cái cộng thêm một chữ số. Hungarian notation 5 (HN) thì đẩy chuyện này lên một tầm cao mới.

5 So named because it was invented by Charles Simonyi, a Hungarian-born architect at Microsoft during the DOS era.

5 Tên gọi này xuất phát từ việc nó do Charles Simonyi phát minh — một kiến trúc sư người gốc Hungary làm việc tại Microsoft thời DOS.

HN was considered to be pretty important back in the Windows C API, when everything was an integer handle or a long pointer or a void pointer, or one of several implementations of “string” (with different uses and attributes). The C compiler did not check types in those days, so the programmers needed a crutch to help them remember the types.

HN từng được coi là khá quan trọng trong thời Windows C API, khi mọi thứ đều là một integer handle, một long pointer, một void pointer, hoặc một trong vài kiểu hiện thực của “string” (với công dụng và thuộc tính khác nhau). Trình biên dịch C hồi đó không kiểm tra kiểu, nên lập trình viên cần một cái nạng để nhớ được kiểu.

In modern languages, we have much richer type systems, and the compilers remember and enforce the types. What’s more, there is a trend toward smaller classes and shorter functions so that people can usually see the point of declaration of each variable they’re using.

Trong các ngôn ngữ hiện đại, hệ thống kiểu phong phú hơn nhiều, và trình biên dịch tự nhớ rồi tự bắt lỗi kiểu giùm ta. Hơn nữa, xu hướng chung là class nhỏ hơn và hàm ngắn hơn, nên người đọc thường nhìn thấy luôn chỗ khai báo của mỗi biến họ đang dùng.

Today's programmers get enough help from their compilers and IDEs that they don't usually need to encode types within their variable names. So nowadays, HN and other forms of type encoding are simply impediments. They make it harder to change the name or type of a variable, function, or class. They make it harder to read the code. And they create the possibility that the encoding system will mislead the reader.

Lập trình viên ngày nay được trình biên dịch và IDE hỗ trợ đủ nhiều đến mức thường chẳng cần mã hoá kiểu vào tên biến nữa. Vậy nên bây giờ, HN và các dạng mã hoá kiểu khác chỉ còn là vật cản. Chúng khiến việc đổi tên hay đổi kiểu của một biến, một hàm, một class trở nên khó khăn hơn. Chúng khiến mã khó đọc hơn. Và chúng tạo ra khả năng hệ thống mã hoá ấy sẽ đánh lừa người đọc.

```
PhoneNumber phoneString;  
// name not changed when type changed!
```

Member Prefixes

Tiền tố cho biến thành viên

Prefixing member variables with something like `m_` is another form of type encoding that is no longer necessary. Your classes and functions should be small enough that you don't need such a crutch. You should be using an editing environment that highlights or colorizes members to make them distinct.

Gắn tiền tố kiểu `m_` vào biến thành viên là một dạng mã hoá kiểu khác mà nay đã không còn cần thiết. Class và hàm của bạn nên đủ nhỏ để bạn chẳng cần tới cái nạng đó. Và bạn nên dùng một môi trường soạn thảo biết tô sáng hoặc tô màu các thành viên để phân biệt chúng.

```
public class Part {  
    private String m_dsc; // The textual description  
    void setName(String name) {  
        m_dsc = name;  
    }  
}  
  
public class Part {  
    String description;  
    void setDescription(String description) {  
        this.description = description;  
    }  
}
```

Interfaces and Implementations

Interface và hiện thực

We do not recommend prefixing interfaces with `I`, because that's just another kind of type encoding. For example, say you are building an Abstract Factory 6 for the creation of `Shape`s. This factory will be an **interface** and will be implemented by a concrete class. What should we name these classes? `IShapeFactory` and `ShapeFactory`? We prefer to leave interfaces unadorned. The preceding `I` is a distraction at best and too much information at worst. After all, the whole point of an interface is that users don't need to know it's an interface. We just want them to know that it's a `ShapeFactory`. So, if we must encode either the interface or the **implementation**, we prefer to add a suffix to the implementation; for example, `ShapeFactoryImp`.

Chúng tôi không khuyến khích gắn tiền tố `I` vào interface, bởi đó cũng chỉ là một kiểu

mã hoá kiểu khác mà thôi. Ví dụ, giả sử bạn đang xây một Abstract Factory 6 để tạo ra các Shape . Factory này sẽ là một interface và sẽ được hiện thực bởi một class cụ thể. Ta nên đặt tên cho những class đó là gì? IShapeFactory và ShapeFactory ? Chúng tôi thích để interface mặc mặc, không trang sức. Chữ I đứng phía trước, nhẹ thì gây xao nhãng, nặng thì là thừa thông tin. Suy cho cùng, toàn bộ điểm mấu chốt của một interface là người dùng chẳng cần biết nó là interface. Ta chỉ muốn họ biết rằng đó là một ShapeFactory . Vậy nên, nếu buộc phải mã hoá một trong hai — interface hay phần hiện thực — chúng tôi thích thêm hậu tố vào phần hiện thực; ví dụ, ShapeFactoryImp .

6 [GOF95].

6 [GOF95].

Use Appropriate Parts of Speech — Dùng đúng từ loại

Some elements of our code are best represented by nouns, and others by verbs.

Có những thành phần trong mã được thể hiện tốt nhất bằng danh từ, có những thành phần khác lại hợp với động từ.

Class Names Classes and objects should generally have noun or noun phrase names like Customer , WikiPage , Account , and AddressParser . Generally the noun should be singular and not plural. A class that represents customers should be named Customer and not Customers . On the other hand, a class that holds many Customer instances probably should be called Customers .

Tên class Class và đối tượng (object) nói chung nên mang tên là danh từ hoặc cụm danh từ, như Customer , WikiPage , Account , và AddressParser . Nói chung danh từ nên ở dạng số ít chứ không phải số nhiều. Một class biểu diễn khách hàng nên được đặt tên là Customer chứ không phải Customers . Ngược lại, một class chứa nhiều thể hiện Customer thì hẳn nên gọi là Customers .

Avoid noise words like Manager , Processor , Data , or Info in the name of a class. We already know that classes manage and process data and info—you don't need to tell us.

Tránh những từ nhiễu như Manager , Processor , Data hay Info trong tên class. Chúng ta đều đã biết rằng class thì quản lý và xử lý dữ liệu với thông tin rồi — bạn không cần phải nói ra điều đó.

Method Names

Tên method

Methods should generally have verb or verb phrase names like

Method nói chung nên mang tên là động từ hoặc cụm động từ, chẳng hạn

postPayment , deletePage , or save .

Depending on your language, accessors, mutators, and predicates should be named according to the local convention. In Java, they are named for their value and prefixed with get , set , and is according to the JavaBeans standard. 7

Tùy ngôn ngữ, các accessor, mutator và predicate nên được đặt tên theo quy ước bản địa. Trong Java, chúng được đặt tên theo giá trị mà chúng thao tác, kèm tiền tố get , set và is theo chuẩn JavaBeans. 7

7 <http://java.sun.com/products/javabeans/docs/spec.html>

7 <http://java.sun.com/products/javabeans/docs/spec.html>

There is a de facto convention in some languages to name a function after the value it returns, that is, as a noun. For example, the function that returns the name of a customer might be called `name`. We do not prefer this convention, but we will follow it when working with teams that use it.

Ở một số ngôn ngữ có một quy ước bất thành văn là đặt tên hàm theo giá trị nó trả về, tức là dùng danh từ. Ví dụ, hàm trả về tên khách hàng có thể được gọi là `name`. Chúng tôi không chuộng quy ước này, nhưng vẫn sẽ tuân theo khi làm việc với những đội đang dùng nó.

In most cases, it's best if a mutator is named for the mutation it causes, such as `add`, `remove`, `sort`, and so on. These tend to be verb names, and so the standard advice is to “name methods with verb names.”

Trong phần lớn trường hợp, tốt nhất là đặt tên mutator theo đúng phép biến đổi mà nó gây ra, như `add`, `remove`, `sort`, v.v. Đây thường là những cái tên động từ, và vì thế lời khuyên chuẩn mực là “hãy đặt tên method bằng động từ”.

```
string name = employee.getName();
customer.setName("mike");
if (paycheck.isPosted())...
```

C++, Java, and C# have constructors that take the name of the class. This means you cannot give the constructor a **meaningful name**. When such constructors are overloaded, it is better to use static factory methods with names that describe the arguments. For example:

C++, Java và C# có constructor mang đúng tên của class. Điều đó có nghĩa là bạn không thể đặt cho constructor một cái tên có ý nghĩa. Khi những constructor như vậy bị overload, tốt hơn là dùng static factory method với tên mô tả rõ các đối số. Ví dụ:

```
Complex fulcrumPoint = Complex.FromRealNumber(23.0);
```

is generally better than

nhìn chung tốt hơn

```
Complex fulcrumPoint = new Complex(23.0);
```

Consider enforcing their use by making the corresponding constructors private.

Hãy cân nhắc ép buộc dùng chúng bằng cách để các constructor tương ứng ở chế độ `private`.

Consider Keyword Parameters — Cân nhắc tham số kiểu keyword

If your language allows keyword or hash map parameters, such as Python, C#, Ruby, and Clojure do, you may find it best to limit the positional parameters of a function and use explicit keyword-only parameters for most variations.

Nếu ngôn ngữ của bạn cho phép tham số dạng keyword hoặc hash map — như Python, C#, Ruby và Clojure — bạn có thể thấy rằng cách hay nhất là hạn chế số tham số theo vị trí của một hàm, còn hầu hết các biến thể thì dùng tham số keyword-only tường minh.

```
def sort(iterable, *, key=None, reverse=False) :
    ... # body elided
```



```
pets_sorted = sort(pets)
pets_reversed = sort(pets, reverse=True)
pets_by_age = sort(pets, key=lambda x: x.birthdate() )
```

Don't Be Cute — Đừng cố tỏ ra dí dỏm

If names are too clever, they will be memorable only to people who share the author's sense of humor, and only as long as these people remember the joke. Will they know what the function named `HolyHandGrenade` is supposed to do? Sure it's cute, but maybe in this case `DeleteItems` might be a better name. Choose clarity over entertainment value.

Nếu tên quá “thông minh”, chúng sẽ chỉ đáng nhớ với những người cùng khiếu hài hước với tác giả, và chỉ đáng nhớ chừng nào đám người đó còn nhớ được câu đùa. Liệu họ có biết hàm tên `HolyHandGrenade` được cho là làm gì không? Đúng là nó ngộ nghĩnh thật, nhưng có lẽ trong trường hợp này `DeleteItems` mới là cái tên tốt hơn. Hãy chọn sự rõ ràng thay vì giá trị giải trí.

Cuteness in code often appears in the form of colloquialisms or slang. For example, don't use the name `whack()` to mean `kill()` . Don't tell little culture-dependent jokes like `eatMyShorts()` to mean `abort()` .

Sự dí dỏm trong mã thường xuất hiện dưới dạng khẩu ngữ hoặc tiếng lóng. Chẳng hạn, đừng dùng tên `whack()` để nói `kill()` . Đừng pha những trò đùa gắn với văn hoá riêng như `eatMyShorts()` để nói `abort()` .

Say what you mean. Mean what you say.

Hãy nói điều bạn nghĩ. Và hãy nghĩ đúng điều bạn nói.

Pick One Word per Concept — Mỗi khái niệm chọn một từ

Pick one word for one concept, and stick with it. For example, don't use `amount` in one module, `fee` in another, `price` in yet another, and `cost` in still another. Pick a name for each concept and use that name throughout. Make sure that everyone on the team knows those names.

Hãy chọn một từ cho một khái niệm, rồi bám chặt lấy nó. Ví dụ, đừng dùng `amount` ở module này, `fee` ở module kia, `price` ở module nọ, rồi `cost` ở module khác nữa. Hãy chọn một cái tên cho mỗi khái niệm và dùng đúng tên đó ở khắp nơi. Và phải chắc chắn rằng mọi người trong đội đều biết những cái tên ấy.

We will often audit the names in our systems for consistency. This is a small task, and often an IDE will have the option to provide an “outline view” or “object view,” or else “fold/collapse all functions” so that you can look at method and variable names without scrolling. We highly recommend harmonizing all the names in a given class, module, or **test suite**. You will find it satisfying, we have no doubt.

Chúng tôi thường xuyên rà soát lại tên trong hệ thống của mình để kiểm tra tính nhất quán. Đây là việc nhỏ thôi, và IDE thường có tùy chọn “outline view” hay “object view”, hoặc chỉ ít cũng có “fold/collapse all functions” để bạn nhìn được toàn bộ tên method và tên biến mà không phải cuộn màn hình. Chúng tôi rất khuyến khích bạn làm hài hoà toàn bộ tên trong một class, một module hay một bộ test (test suite). Bạn sẽ thấy nó đã lắm, không nghi ngờ gì.

A consistent lexicon (A Ubiquitous Language 8) is a great boon to the programmers who must use your code.

Một bộ từ vựng nhất quán (một Ubiquitous Language 8) là món quà lớn dành cho những lập trình viên buộc phải dùng mã của bạn.

8 [DDD].

8 [DDD].

Use Solution Domain Names — Dùng tên thuộc miền lời giải

Remember that the people who read your code will be programmers. They will be familiar with solution domain terms.

Hãy nhớ rằng người đọc mã của bạn sẽ là lập trình viên. Họ quen thuộc với các thuật ngữ của miền lời giải (solution domain).

When you are underneath the domain level and into pure implementation, go ahead and use computer science (CS) terms, algorithm names, pattern names, math terms, and so forth. It is not wise to draw every name from the problem domain, because we don't want our coworkers to have to run back and forth to the customer asking what every name means when they already know the concept by a different name.

Khi bạn đã ở dưới tầng nghiệp vụ và bước vào phần hiện thực thuần túy, cứ thoải mái dùng thuật ngữ khoa học máy tính (CS), tên thuật toán, tên mẫu thiết kế, thuật ngữ toán học, v.v. Rút mọi cái tên ra từ miền bài toán là không khôn ngoan, bởi ta đâu muốn đồng nghiệp phải chạy tới chạy lui hỏi khách hàng xem mỗi cái tên nghĩa là gì, trong khi họ vốn đã biết khái niệm đó dưới một cái tên khác.

The name AccountVisitor means a great deal to a programmer who is familiar with the Visitor 9 pattern. What programmer would not know what a JobQueue or FIFO is?

Cái tên AccountVisitor nói lên rất nhiều điều với một lập trình viên đã quen mẫu Visitor 9 . Còn lập trình viên nào lại không biết JobQueue hay FIFO là gì?

9 [GOF95].

9 [GOF95].

There are lots of very technical things that programmers have to do. Choosing technical names for doing technical things is usually the most appropriate course.

Có vô số thứ rất kỹ thuật mà lập trình viên phải làm. Chọn tên kỹ thuật cho những việc kỹ thuật thường là con đường thích hợp nhất.

Programs are often best described using programming. You don't have to make everything look like either English prose or mathematical formulae. Use the **level of abstraction** that works best for the algorithm and the audience.

Chương trình thường được mô tả tốt nhất bằng chính ngôn ngữ lập trình. Bạn không nhất thiết phải làm cho mọi thứ trông như văn xuôi tiếng Anh hay như công thức toán học. Hãy dùng mức trừu tượng nào phù hợp nhất với thuật toán và với người đọc.

Use Problem Domain Names — Dùng tên thuộc miền bài toán

Use names from the problem domain. They are likely to be familiar and help set the context for developers on your team. If not, then at least the programmer who maintains your code can ask a domain expert what it means.

Hãy dùng tên lấy từ miền bài toán (problem domain). Chúng nhiều khả năng là quen thuộc và giúp thiết lập bối cảnh cho các lập trình viên trong đội bạn. Còn nếu không quen, thì chỉ ít người bảo trì mã của bạn cũng có thể hỏi một chuyên gia nghiệp vụ xem nó nghĩa là gì.

Separating solution and problem domain concepts is part of the job of a good programmer and designer. The code that has more to do with problem domain concepts should have names drawn from the problem domain. We don't force solution domain names on the problem, nor problem domain names on the solutions.

Tách bạch các khái niệm thuộc miền lời giải và miền bài toán là một phần công việc của lập trình viên và người thiết kế giải. Phần mã nào dính dáng nhiều hơn tới khái niệm của miền bài toán thì nên mang tên rút ra từ miền bài toán. Ta không ép tên miền lời giải lên bài toán, cũng không ép tên miền bài toán lên lời giải.

Add Meaningful Context — Thêm bối cảnh có ý nghĩa

There are a few names that are meaningful in and of themselves— most are not. Instead, you need to place names in context for your reader by enclosing them in well-named classes, functions, or namespaces.

Có một số ít cái tên tự thân đã mang ý nghĩa — phần lớn thì không. Thay vào đó, bạn cần đặt tên vào bối cảnh cho người đọc, bằng cách bọc chúng trong những class, hàm hay namespace được đặt tên tử tế.

Imagine that you have variables named `firstName` , `lastName` , `street` , `houseNumber` , `city` , `state` , and `postcode` . Taken together, it's pretty clear that they form an address. But what if you just saw the `state` variable being used alone in a method? Would you automatically infer that it was part of an address, or would you infer that you are in a finite state machine and tracking which state that machine was currently in?

Hãy tưởng tượng bạn có các biến tên `firstName` , `lastName` , `street` , `houseNumber` , `city` , `state` và `postcode` . Gộp lại với nhau thì khá rõ ràng rằng chúng tạo thành một địa chỉ. Nhưng nếu bạn chỉ thấy mỗi biến `state` đứng một mình trong một method thì sao? Bạn có tự động suy ra rằng nó là một phần của địa chỉ, hay bạn sẽ nghĩ mình đang ở trong một máy trạng thái hữu hạn và đang theo dõi xem cái máy đó hiện đang ở trạng thái nào?

You can remove this ambiguity by creating a class or data structure named `Address` . Then, even the compiler will know that the variables belong to a bigger concept.

Bạn có thể xóa bỏ sự mập mờ này bằng cách tạo ra một class hoặc một cấu trúc dữ liệu (data structure) tên là `Address` . Khi đó, đến cả trình biên dịch cũng biết rằng các biến ấy thuộc về một khái niệm lớn hơn.

Programmers sometimes fall into the **code smell** of Primitive Obsession, 10 avoiding the creation of classes, structures, and module for too long. It is much better, for example, to work with `GlobalLocations` than to deal with primitive variables for latitude and longitude in hours, minutes,

and seconds. It is far easier to work with two “locations” than with 12 floating-point integers, all similarly named.

Lập trình viên đôi khi rơi vào code smell mang tên Primitive Obsession, 10 tức là né tránh việc tạo class, struct và module quá lâu. Chẳng hạn, làm việc với GlobalLocations tốt hơn nhiều so với việc vật lộn với các biến nguyên thủy cho vĩ độ và kinh độ tính theo giờ, phút, giây. Xử lý hai “location” dễ hơn hẳn so với xử lý 12 số dấu chấm động mang tên na ná nhau.

10 One of the code smells in **[Refactoring]**.

10 Một trong các code smell (mùi mã) trong [Refactoring].

As another example, consider the following Python code. Do the variables need a more meaningful context? The function name provides only part of the context; the algorithm provides the rest.

Một ví dụ khác, hãy xem đoạn mã Python dưới đây. Các biến ở đó có cần một bối cảnh có ý nghĩa hơn không? Tên hàm chỉ cung cấp một phần bối cảnh; phần còn lại nằm ở thuật toán.

```
private void printGuessStatistics(char candidate, int count) {
    String number;
    String verb;
    String pluralModifier;
    if (count == 0) {
        number = "no";
        verb = "are";
        pluralModifier = "s";
    } else if (count == 1) {
        number = "1";
        verb = "is";
        pluralModifier = "";
    } else {
        number = Integer.toString(count);
        verb = "are";
        pluralModifier = "s";
    }
    String guessMessage = String.format(
        "There %s %s %s%s", verb, number, candidate, pluralModifier
    );
    print(guessMessage);
}
```

Once you read through the function, you see that the three variables — number , verb , and pluralModifier —are part of the “guess statistics” message. Unfortunately, the context must be inferred. When you first look at the method, the meanings of the variables are opaque.

Đọc hết hàm rồi, bạn mới thấy ba biến — number , verb và pluralModifier — là một phần của thông điệp “guess statistics”. Đáng tiếc, bối cảnh ấy phải suy ra mới biết. Lúc mới nhìn vào method, ý nghĩa của các biến hoàn toàn mờ mịt.

The function is a bit too long and the variables are used throughout. To split the function into smaller pieces we need to create a GuessStatisticsMessage class and make the three variables fields of this class. This provides a clear context for the three variables. They are definitively part of the

GuessStatisticsMessage . The improvement of context also allows the algorithm to be made much cleaner by breaking it into many smaller functions.

Hàm hơi dài quá, và các biến thì được dùng rải rác khắp nơi. Để chẻ hàm thành những mảnh nhỏ hơn, ta cần tạo một class GuessStatisticsMessage và biến ba biến kia thành field của class này. Điều đó tạo ra một bối cảnh rõ ràng cho ba biến. Chúng dứt khoát là một phần của GuessStatisticsMessage . Bối cảnh được cải thiện cũng cho phép làm thuật toán sạch hơn nhiều bằng cách chẻ nó thành nhiều hàm nhỏ.

```
public class GuessStatisticsMessage {
    private String number;
    private String verb;
    private String pluralModifier;
    public String make(char candidate, int count) {
        createPluralDependentMessageParts(count);
        return String.format(
            "There %s %s %s%s",
            verb, number, candidate, pluralModifier );
    }
    private void createPluralDependentMessageParts(int count) {
        if (count == 0) {
            thereAreNoLetters();
        } else if (count == 1) {
            thereIsOneLetter();
        } else {
            thereAreManyLetters(count);
        }
    }
    private void thereAreManyLetters(int count) {
        number = Integer.toString(count);
        verb = "are";
        pluralModifier = "s";
    }
    private void thereIsOneLetter() {
        number = "1";
        verb = "is";
        pluralModifier = "";
    }
    private void thereAreNoLetters() {
        number = "no";
        verb = "are";
        pluralModifier = "s";
    }
}
```

Don't Add Gratuitous Context — Đừng thêm bối cảnh thừa thãi

In an imaginary application called Gas Station Deluxe, it is a bad idea to prefix every class with GSD . Frankly, you are working against your tools. You type G and press the completion key and are rewarded with a mile-long list of every class in the system. Is that wise? Why make it hard for the

IDE to help you?

Trong một ứng dụng tưởng tượng tên là Gas Station Deluxe, gắn tiền tố GSD vào mọi class là một ý tồi. Nói thẳng ra, bạn đang chống lại chính công cụ của mình. Bạn gõ G rồi bấm phím gợi ý hoàn thành, và phần thưởng là một danh sách dài dằng dặc gồm mọi class trong hệ thống. Thế có khôn không? Có gì lại làm khó IDE khi nó đang muốn giúp bạn?

Likewise, say you invented a MailingAddress class in GSD’s accounting module, and you named it GSDAccountAddress. Later, you need a mailing address for your customer contact application. Do you use GSDAccountAddress? Does it sound like the right name? Ten of 17 characters are redundant or irrelevant.

Tương tự, giả sử bạn tạo ra một class MailingAddress trong module kế toán của GSD, và đặt tên nó là GSDAccountAddress. Về sau, bạn cần một địa chỉ thư tín cho ứng dụng liên hệ khách hàng. Bạn có dùng GSDAccountAddress không? Nghe có giống một cái tên đúng đắn không? Mười trên 17 ký tự là thừa hoặc chẳng liên quan.

The names accountAddress and customerAddress are fine names for instances of the class Address but could be poor names for classes. Address is a fine name for a class. If we need to differentiate between MAC addresses, port addresses, and Web addresses, we might consider PostalAddress, MAC, and URI. The resulting names are more precise, which is the point of all naming.

Những cái tên accountAddress và customerAddress là tên tốt cho các thể hiện của class Address, nhưng có thể lại là tên dở cho chính các class. Address là một cái tên tốt cho một class. Nếu cần phân biệt địa chỉ MAC, địa chỉ cổng và địa chỉ Web, ta có thể cân nhắc PostalAddress, MAC và URI. Những cái tên thu được chính xác hơn, mà chính xác vốn là mục đích của mọi việc đặt tên.

Final Words — Lời cuối

The hardest thing about choosing good names is that it requires good descriptive skills and a shared cultural background. It’s not a self-pleasing, universal, learn-one-time-and-run-on-autopilot kind of skill.

Điều khó nhất trong việc chọn tên tốt là nó đòi hỏi khả năng mô tả tốt và một nền tảng văn hoá chung. Đây không phải loại kỹ năng tự sướng một mình, phổ quát cho mọi hoàn cảnh, học một lần rồi chạy tự động mãi mãi.

Each time we change codebases we have to start the process over. Who are the people who comprise our audience now? What does “everyone know” in this space? What is familiar and what seems “weird” or “alien” here?

Mỗi lần đổi codebase là ta lại phải bắt đầu lại quá trình đó từ đầu. Bây giờ độc giả của ta là những ai? Ở không gian này thì “ai cũng biết” cái gì? Ở đây điều gì là quen thuộc, còn điều gì nghe “lạ tai” hay “ngoài hành tinh”?

There is a degree of subjectivity here, as you write for an audience. What you like personally may not be what they need and understand. As a result, many programmers don’t learn to do it very well. It requires contact with your user base, just like any good software development requires contact with real users.

Ở đây có một mức độ chủ quan nhất định, bởi bạn viết cho một lớp độc giả. Cái bạn thích về mặt cá nhân có thể không phải cái họ cần và hiểu được. Hệ quả là nhiều lập trình viên chẳng học được cách làm chuyện này cho ra hồn. Nó đòi hỏi tiếp xúc với chính lớp người

dùng của bạn, hết như mọi hoạt động phát triển phần mềm tử tế đều đòi hỏi tiếp xúc với người dùng thật.

Since it's hard to get it right (even with these fine guidelines), we often have to iterate on our software names. Sometimes other developers are surprised that we, after having written so many times and so many words on naming, will often call a variable by an intentionally bad name temporarily, until a better name comes to mind.

Vì làm cho đúng vốn đã khó (kể cả khi đã có mấy hướng dẫn hay ho này), ta thường phải lặp đi lặp lại việc chỉnh tên trong phần mềm. Đôi khi các lập trình viên khác ngạc nhiên khi thấy chúng tôi — sau ngần ấy lần viết và ngần ấy chữ về chuyện đặt tên — vẫn hay cố tình đặt cho một biến cái tên dở tạm thời, cho tới khi nghĩ ra được cái tên hay hơn.

We rename things. As Josh Kerievsky reminds us, all good writing is based on revision.

Chúng tôi đổi tên. Như Josh Kerievsky nhắc chúng ta, mọi bài viết hay đều dựa trên việc sửa đi sửa lại.

People may defer renaming things for fear that some other developers will object. We do not share that fear and find that we are actually grateful when names change (for the better).

Người ta có thể trì hoãn việc đổi tên vì sợ vài lập trình viên khác phản đối. Chúng tôi không có nỗi sợ đó, và thực tế là còn thấy biết ơn mỗi khi tên được đổi (theo hướng tốt hơn).

Most of the time we don't memorize the names of classes and methods and demand that they remain the same. Memorization is a waste of our time. We use modern tools to deal with details like that. We focus more on whether the code reads like paragraphs and sentences, or at least like equations, tables, and data structures.

Phần lớn thời gian chúng tôi không thuộc lòng tên class và tên method rồi đòi chúng phải giữ nguyên. Học thuộc là phí thời gian. Chúng tôi dùng công cụ hiện đại để lo mấy chi tiết kiểu đó. Chúng tôi tập trung nhiều hơn vào chuyện mã đọc lên có ra đoạn văn và câu chữ hay không, hoặc chí ít thì có ra phương trình, bảng biểu và cấu trúc dữ liệu hay không.

You may surprise someone with a name change, just like you might with any other code improvement. Don't let it stop you in your tracks.

Bạn có thể làm ai đó bất ngờ khi đổi một cái tên, cũng như với bất kỳ cải tiến mã nào khác. Đừng để điều đó chặn bước chân bạn.

Try these guidelines to see if they help in your situation.

Hãy thử các hướng dẫn này xem chúng có giúp ích trong hoàn cảnh của bạn không.

So far, they seem to help most people most of the time. We're confident that you will also find help here.

Cho đến nay, chúng có vẻ giúp ích cho hầu hết mọi người trong hầu hết thời gian. Chúng tôi tin rằng bạn cũng sẽ tìm thấy sự trợ giúp ở đây.

Chapter 5. Comments — Chương 5.

Comment



“Don’t **comment bad code**—rewrite it.”

“Đừng comment cho mã tệ — hãy viết lại nó.”

—Brian W. Kernighan and P. J. Plaugher 1

—Brian W. Kernighan và P. J. Plaugher 1

1 [KP78], p. 144.

1 [KP78], tr. 144.

“Comments are the distracting footnotes of code.”

“Comment là những chú thích chân trang gây xao nhãng của mã.”

—Jeff Langr Nothing can be quite so helpful as a well-placed comment. Nothing can clutter up a **module** more than frivolous dogmatic comments. Nothing can be quite so damaging as an old crufty comment that propagates lies and misinformation.

—Jeff Langr Không gì hữu ích bằng một comment đặt đúng chỗ. Không gì làm một module bừa bộn hơn những comment giáo điều vô bổ. Và không gì tai hại bằng một comment già nua mốc meo cứ thế lan truyền dối trá và thông tin sai lệch.

Compensating for Failure — Bù đắp cho một thất bại

Comments are, at best, a necessary evil. If our programming languages were expressive enough, or if you and I had the talent to subtly wield those languages to express our intent, we would not need comments very much—perhaps not at all.

Comment, giỏi lắm cũng chỉ là một cái ác cần thiết. Nếu ngôn ngữ lập trình của chúng ta đủ sức diễn đạt, hoặc nếu bạn và tôi có đủ tài nghệ để uốn nắn những ngôn ngữ ấy sao cho chúng nói lên đúng ý mình, thì hẳn ta đã chẳng cần đến comment nhiều đến thế — có khi chẳng cần chút nào.

The proper use of comments is to compensate for our failure to express ourselves in code. Note that I used the word failure . I meant it. Comments are always failures of either our languages or our abilities.

Công dụng chính đáng của comment là bù đắp cho việc chúng ta đã thất bại trong việc diễn đạt bằng mã. Để ý là tôi dùng từ *thất bại*. Tôi cố tình dùng từ đó. Comment luôn là thất bại của ngôn ngữ, hoặc của chính năng lực chúng ta.

This doesn't mean that we never write comments. We must sometimes write comments because we cannot always figure out how to express ourselves without them. However, their use is not a cause for celebration.

Điều đó không có nghĩa là ta không bao giờ viết comment. Đôi khi ta buộc phải viết, bởi không phải lúc nào ta cũng nghĩ ra cách diễn đạt mà không cần đến chúng. Có điều, việc dùng comment chẳng phải chuyện đáng ăn mừng.

When you find yourself in a position where you need to write a comment, think it through and see whether there isn't some way to turn the tables and express yourself in code. Every time you express yourself in code you should pat yourself on the back. Every time you write a comment you should grimace and feel the failure of our languages and of our ability to express ourselves in those languages.

Khi thấy mình rơi vào tình huống cần viết một comment, hãy nghĩ kỹ xem liệu có cách nào lật ngược thế cờ và diễn đạt điều đó ngay trong mã hay không. Mỗi lần bạn diễn đạt được ý mình bằng mã, hãy tự vỗ vai khen mình. Mỗi lần bạn viết một comment, hãy nhăn mặt và cảm nhận sự thất bại của ngôn ngữ, cũng như của chính khả năng diễn đạt của chúng ta trong ngôn ngữ ấy.

The comments you write may be necessary; but that necessity is also unfortunate.

Những comment bạn viết có thể là cần thiết; nhưng cái sự cần thiết ấy cũng là điều đáng tiếc.

It has been said that advice like this gives license to younger programmers to avoid writing comments. Shame on any programmers, young or old, who use these recommendations as an excuse for their laziness and/or unwillingness to add proper explanations and context. In the end, I cannot concern myself with how others might abuse my advice. My sole purpose here is help those who will not.

Có người bảo rằng lời khuyên kiểu này chẳng khác nào cấp giấy phép cho đám lập trình viên trẻ khỏi phải viết comment. Xấu hổ thay cho bất kỳ lập trình viên nào, trẻ hay già, lấy những khuyến nghị này làm cớ biện minh cho sự lười biếng và/hoặc thái độ không chịu bổ sung phần giải thích và ngữ cảnh đúng mực. Nói cho cùng, tôi không thể bận tâm chuyện người khác lạm dụng lời khuyên của mình ra sao. Mục đích duy nhất của tôi ở đây là giúp những ai sẽ không làm vậy.

Hidden or Obscured Comments — Comment bị giấu đi hoặc bị làm mờ

Does your IDE hide the first comment in every source file? Mine does. That's because we seldom want to read that comment. We expect that it's some kind of boilerplate license/legalese that offers no insight into the actual intent of the module.

IDE của bạn có giấu comment đầu tiên trong mỗi tệp mã nguồn không? IDE của tôi thì có. Lý do là ta hiếm khi muốn đọc comment đó. Ta đoán chắc nó chỉ là mấy dòng giấy phép/thuật ngữ pháp lý soạn sẵn, chẳng hé lộ tí gì về ý đồ thực sự của module.

But collapsing the first comment is not the only way that our IDEs hide or obscure comments from us.

Nhưng thu gọn comment đầu tiên không phải cách duy nhất mà IDE giấu hay làm mờ comment trước mắt chúng ta.

What color does your IDE use to display comments? For some of you, it will be a nice green—like the grass. And so, like the grass, you can ignore them. They fade into the background like the neighborhood lawns.

IDE của bạn tô comment bằng màu gì? Với một số người, đó sẽ là màu xanh lá dịu mắt — như màu cỏ. Và cũng như cỏ, bạn có thể phớt lờ chúng. Chúng chìm ngấm vào phong nền như mấy bãi cỏ trong khu phố.

For others of you, it will be a nice soft gray; not invisible, but strongly de-emphasized.

Với những người khác, đó sẽ là màu xám nhạt dễ chịu; không đến mức vô hình, nhưng bị chìm xuống rất rõ.

Why? Because we don't want them in the way. We don't really want to see them. We like the fact that they are there; but we don't want them in our face.

Tại sao? Bởi ta không muốn chúng vướng chân. Ta không thực sự muốn nhìn thấy chúng. Ta thích cái cảm giác biết rằng chúng có ở đó; nhưng ta không muốn chúng chường mặt ra trước mũi mình.

I have my IDE paint comments in bright fire-engine red. I do this because I reckon that if someone wrote a comment, I should read it. And then, if that comment does not help me, I delete it on the spot. Or if that comment needs adjusting, I adjust it on the spot.

Còn tôi thì để IDE tô comment bằng màu đỏ chói như xe cứu hoả. Tôi làm vậy vì tôi cho rằng nếu ai đó đã viết một comment thì tôi nên đọc nó. Rồi nếu comment ấy chẳng giúp được gì, tôi xoá nó ngay tại chỗ. Còn nếu nó cần chỉnh lại, tôi chỉnh ngay tại chỗ.

Comments, when they exist, should be visible!

Comment, một khi đã tồn tại, thì phải nhìn thấy được!

Lying Comments — Những comment nói dối

Why am I so down on comments? I'm really not. I'm only down on bad comments. I wholly support **good** comments. The problem, however, is that too many of the comments I come across fall into the former category.

Sao tôi lại ghét comment đến thế? Thật ra thì không. Tôi chỉ ghét comment tệ thôi. Comment tốt thì tôi hoàn toàn ủng hộ. Có điều, vấn đề nằm ở chỗ quá nhiều comment tôi gặp phải rơi vào loại thứ nhất.

First of all, they often lie. The lies aren't intentional, but they are also not infrequent. The older a comment is, and the farther away it is from the code it describes, the more likely it is to be just plain wrong. The reason is simple. Programmers can't realistically maintain them.

Trước hết, chúng thường nói dối. Những lời dối trá ấy không cố ý, nhưng cũng chẳng hiếm gặp. Một comment càng cũ, và càng nằm xa đoạn mã mà nó mô tả, thì càng có khả năng sai bét. Lý do rất đơn giản: lập trình viên không thể bảo trì chúng nổi trên thực tế.

Code will often change and evolve. Chunks of it will move from here to there. Those chunks bifurcate and reproduce and come together again to form hybrids and chimeras. Unfortunately, the comments don't, and can't, always follow them. All too often the comments get separated from the code they describe and become orphaned blurbs of ever-decreasing accuracy. For example, look what has happened to this comment and the line it was intended to describe:

Mã thường thay đổi và tiến hoá. Từng mảng mã dịch chuyển từ chỗ này sang chỗ khác. Những mảng ấy tách đôi, sinh sôi, rồi lại nhập vào nhau tạo thành đủ thứ giống lai và quái vật chấp vá. Đáng tiếc là comment không đi theo được, và cũng không thể lúc nào cũng đi theo. Quá thường xuyên, comment bị tách khỏi đoạn mã mà nó mô tả, trở thành những mẩu vụn mờ côi với độ chính xác ngày một teo tóp. Chẳng hạn, hãy xem chuyện gì đã xảy ra với comment này và dòng mã mà nó vốn định mô tả:

```
MockRequest request;  
private final String HTTP_DATE_REGEX =  
    "[SMTWF][a-z]{2}\\s[0-9]{2}\\s[JFMASOND][a-z]{2}\\s"+  
    "[0-9]{4}\\s[0-9]{2}\\:[0-9]{2}\\:[0-9]{2}\\sGMT";  
private Response response;  
private FitNesseContext context;  
private FileResponder responder;  
private Locale saveLocale;  
// Example: "Tue, 02 Apr 2003 22:18:49 GMT"
```

Other instance variables that were probably added later were interposed between the HTTP_DATE_REGEX constant and its explanatory comment.

Những biến thành viên khác, chắc là được thêm vào về sau, đã chen ngang vào giữa hằng số HTTP_DATE_REGEX và cái comment giải thích cho nó.

You might complain that programmers should be disciplined enough to keep the comments in a high state of repair, relevance, and accuracy. However, I would rather that **discipline** and energy go toward making the code so clear and expressive that it does not need the comments in the first place.

Bạn có thể phàn nàn rằng lập trình viên lẽ ra phải đủ kỷ luật (discipline) để giữ cho comment luôn ở tình trạng tươi tắn, đúng chỗ và chính xác. Nhưng tôi thì thấy cái kỷ luật và năng lượng ấy được dồn vào việc làm cho mã trong sáng và giàu sức diễn đạt đến mức ngay từ đầu đã chẳng cần comment.

Inaccurate comments are far worse than no comments at all. They delude and mislead. They set expectations that will never be fulfilled. They lay down old rules that need not, or should not, be followed any longer.

Comment sai còn tệ hơn nhiều so với không có comment nào. Chúng đánh lừa và dẫn dắt lệch hướng. Chúng gieo những kỳ vọng sẽ chẳng bao giờ thành hiện thực. Chúng đặt ra

những luật lệ cũ kỹ mà nay không cần, hoặc không nên, tuân theo nữa.

Truth can only be certain in one place: the code. Only the code can be trusted to tell you what it does. It is the most reliable source of accurate information. Therefore, though comments are sometimes necessary, we ought to expend significant energy to minimize them.

Sự thật chỉ có thể chắc chắn ở một nơi duy nhất: trong mã. Chỉ có mã mới đáng tin để nói cho bạn biết nó làm gì. Đó là nguồn thông tin chính xác đáng tin cậy nhất. Vì vậy, dù đôi khi comment là cần thiết, ta vẫn nên dốc sức đáng kể để giảm chúng xuống mức tối thiểu.

Comments That Are Too Intimate — Những comment quá thân thuộc

A significant problem with comments is that they are most often written by programmers who have an intimate understanding of the code, and are therefore written from the point of view of that intimacy. Unfortunately, the target reader of the comments does not share that intimacy, and therefore does not have the context to understand the comments.

Một vấn đề lớn với comment là chúng thường được viết bởi những lập trình viên đã hiểu đoạn mã đến mức thân thuộc từng ngóc ngách, và vì thế được viết từ điểm nhìn của sự thân thuộc ấy. Đáng tiếc, người đọc mà comment nhắm tới lại không có được sự thân thuộc đó, nên cũng chẳng có đủ ngữ cảnh để hiểu comment.

It has happened to me, far more than once, that I could not understand a comment so helpfully provided by the author until I understood that author's code. The words in the comment simply didn't make sense to me, nor help me in any way. Once I read through the code and understood it, then the intent of the comment became clear—but not before.

Chuyện này đã xảy ra với tôi, không chỉ một lần: tôi không tài nào hiểu nổi một comment mà tác giả đã hết sức tận tình để lại, cho tới khi tôi hiểu được đoạn mã của chính tác giả đó. Những con chữ trong comment đơn giản là vô nghĩa với tôi, chẳng giúp gì được. Chỉ sau khi đọc hết mã và hiểu ra, ý đồ của comment mới sáng tỏ — chứ không sớm hơn.

Comments Do Not Make Up for Bad Code — Comment không bù đắp được cho mã tệ

One of the more common motivations for writing comments is bad code. We write a module and we know it is confusing and disorganized. We know it's a **mess**. So we say to ourselves, "Ooh, I'd better comment that!"

Một trong những động cơ phổ biến nhất khiến người ta viết comment chính là mã tệ. Ta viết ra một module và biết rõ nó rối rắm, lộn xộn. Ta biết nó là một mớ hỗn độn. Thế là ta tự nhủ: "Ồi, tốt hơn hết là mình comment chỗ đó lại!"

No! You'd better clean it!

Không! Tốt hơn hết là bạn dọn dẹp nó đi!

Clear and expressive code with few comments is far superior to cluttered and complex code with lots of comments. Rather than spend your time writing the comments that explain the mess you've made, spend it cleaning that mess.

Mã trong sáng, giàu sức diễn đạt với vài ba comment thì vượt trội hơn hẳn mã bừa bộn, phức tạp kèm cả đống comment. Thay vì bỏ thời gian viết comment giải thích cái mới hỗn độn bạn vừa tạo ra, hãy dùng thời gian đó để dọn cái mới ấy đi.

Explain Your Intent in Code — Hãy giải thích ý định của bạn bằng mã

There are certainly times when code makes a poor vehicle for explanation. Unfortunately, many programmers have taken this to mean that code is seldom, if ever, a good means for explanation. This is patently false. Which would you rather see? This:

Chắc chắn có những lúc mã là phương tiện giải thích tồi. Đáng tiếc, nhiều lập trình viên lại hiểu điều đó thành ra: mã hiếm khi, nếu không muốn nói là chẳng bao giờ, là cách giải thích tốt. Đó rõ ràng là sai. Bạn thích nhìn thấy cái nào hơn? Cái này:

```
// Check to see if the employee is eligible for full benefits
if ((employee.flags & HOURLY_FLAG) && (employee.age > 65))
or this?
```

hay cái này?

```
if (employee.isEligibleForFullBenefits())
```

It takes only a few seconds of thought to explain most of your intent in code. In many cases, it's simply a matter of creating a **function** that says the same thing as the comment you want to write.

Chỉ cần vài giây suy nghĩ là bạn đã giải thích được phần lớn ý định của mình bằng mã. Trong rất nhiều trường hợp, đơn giản chỉ là tạo ra một hàm nói đúng cái điều mà comment bạn định viết muốn nói.

Good Comments — Comment tốt

Some comments are necessary or beneficial. What follows are a few examples of comments that are worthy of the bits they consume—or are at least not worth deleting.

Một số comment là cần thiết hoặc có ích. Sau đây là vài ví dụ về những comment xứng đáng với chỗ bit mà chúng chiếm — hoặc ít nhất cũng không đáng bị xóa.

Legal Comments — Comment pháp lý

Sometimes our corporate coding standards force us to write certain comments for legal reasons. For example, copyright and authorship statements are necessary and reasonable things to put into a comment at the start of each source file.

Đôi khi chuẩn viết mã của công ty buộc ta phải viết một số comment vì lý do pháp lý. Chẳng hạn, tuyên bố bản quyền và quyền tác giả là những thứ cần thiết và hợp lý để đặt vào một comment ở đầu mỗi tệp mã nguồn.

Here, for example, is the standard comment header that we put at the beginning of every source file in FitNesse. I am happy to say that our IDE hides this comment from acting as clutter by automatically collapsing it.

Ví dụ, đây là phần đầu comment tiêu chuẩn mà chúng tôi đặt ở đầu mọi tệp mã nguồn trong FitNesse. Tôi rất vui được nói rằng IDE của chúng tôi tự động thu gọn comment này lại nên nó không trở thành thứ vướng víu.

```
// Copyright (C) 2003-2005 by Object Mentor, Inc. All rights reserved.  
// Released under the GNU General Public License version 2 or later.
```

Comments like this should not be contracts or legal tomes. Where possible, refer to a standard license or other external document rather than putting all the terms and conditions into the comment.

Những comment kiểu này không nên là hợp đồng hay pho sách luật. Ở đâu có thể, hãy trở tới một giấy phép tiêu chuẩn hoặc một tài liệu bên ngoài, thay vì nhét toàn bộ điều khoản và điều kiện vào comment.

Informative Comments — Comment cung cấp thông tin

It is sometimes useful to provide basic information with a comment. For example, consider this comment that explains the return value of an abstract **method**:

Đôi khi cung cấp thông tin cơ bản bằng một comment cũng có ích. Chẳng hạn, hãy xem comment sau đây, nó giải thích giá trị trả về của một method trừu tượng:

```
// Returns an instance of the Responder being tested.  
protected abstract Responder responderInstance();
```

Why did the author resort to a comment instead of giving the method a better name like `responderBeingTested`? And why would I allow this comment rather than changing the name of the method? That's because the author was using the well-known Singleton 2 design pattern. When you use a design pattern like that, you should follow the canonical form specified by that pattern. In the Singleton pattern, the name of the accessor function always ends in the word `Instance`. So the author decided to fall back on a comment, rather than abandon the canonical form of the pattern.

Vì sao tác giả lại phải viện đến comment thay vì đặt cho method một cái tên tốt hơn, kiểu như `responderBeingTested`? Và vì sao tôi lại chấp nhận comment này thay vì đổi tên method? Đó là vì tác giả đang dùng mẫu thiết kế Singleton 2 nổi tiếng. Khi bạn dùng một mẫu thiết kế như vậy, bạn nên tuân theo dạng thức chuẩn mực mà mẫu đó quy định. Trong mẫu Singleton, tên của hàm truy xuất luôn kết thúc bằng từ `Instance`. Thế nên tác giả quyết định lui về dùng comment, thay vì từ bỏ dạng thức chuẩn mực của mẫu.

2 [GOF95] p. 127.

2 [GOF95] tr. 127.

Here's another case where the comment adds helpful information.

Đây là một trường hợp khác mà comment bổ sung thông tin hữu ích.

```
// format matched kk:mm:ss EEE, MMM dd, yyyy  
Pattern timeMatcher = Pattern.compile(  
    "\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

I would not delete this comment, because regular expressions are always a nightmare to understand. This one is no different, and it was polite of the author to give us a hint with a comment that lets us know that the regular expression is intended to match a timestamp. Any Java programmer would easily infer this from the `kk:mm:ss EEE, MMM dd, yyyy` string that uses the codes from the

Tôi sẽ không xoá comment này, bởi biểu thức chính quy bao giờ cũng là một cơn ác mộng để hiểu. Cái này cũng chẳng khác gì, và tác giả đã rất lịch sự khi cho ta một gợi ý bằng comment để ta biết rằng biểu thức chính quy này nhằm khớp với một dấu thời gian. Bất kỳ lập trình viên Java nào cũng dễ dàng suy ra điều đó từ chuỗi `kk:mm:ss EEE, MMM dd, yyyy` vốn dùng các mã lấy từ

`SimpleDateFormat` class.

However, the comment is a lie—or at least significantly misinformative. That's because the regular expression would also match the following string: `“:: , ,”`. And that is definitely not a timestamp. So I would fix the regular expression and let the comment remain.

Có điều, comment đó là một lời nói dối — hoặc ít nhất cũng sai lệch đáng kể. Bởi biểu thức chính quy này còn khớp cả chuỗi sau: `“:: , ,”`. Và đó dứt khoát không phải là dấu thời gian. Vậy nên tôi sẽ sửa biểu thức chính quy và giữ nguyên comment.

Explanation of Intent — Giải thích ý định

Sometimes a comment goes beyond just useful information about the **implementation** and provides the intent behind a decision. In the following case, we see an interesting decision documented by a comment. When comparing two objects, the author decided that he wanted to sort objects of the containing **class** higher than objects of any other.

Đôi khi một comment vượt ra khỏi phạm vi thông tin hữu ích về phần hiện thực (implementation) và cho ta biết ý định đằng sau một quyết định. Trong trường hợp dưới đây, ta thấy một quyết định thú vị được ghi lại bằng comment. Khi so sánh hai đối tượng (object), tác giả quyết định rằng anh ta muốn xếp các đối tượng thuộc class chứa nó cao hơn đối tượng của mọi class khác.

```
public int compareTo(Object o)
{
    if(o instanceof WikiPagePath)
    {
        WikiPagePath p = (WikiPagePath) o;
        String compressedName = StringUtil.join(names, "");
        String compressedArgumentName = StringUtil.join(p.names, "");
        return compressedName.compareTo(compressedArgumentName);
    }
    return 1; // we are greater because we are the right type.
}
```

Here's an even better example. You might not agree with the programmer's solution to the problem, but at least you know what he was trying to do.

Đây là một ví dụ còn hay hơn nữa. Bạn có thể không đồng tình với cách lập trình viên này giải quyết vấn đề, nhưng ít ra bạn biết anh ta đang cố làm gì.

```
public void testConcurrentAddWidgets() throws Exception {
    WidgetBuilder widgetBuilder = . . .
    //This is our best attempt to get a race condition
    //by creating large number of threads.
    for (int i = 0; i < 25000; i++) {
        WidgetBuilderThread widgetBuilderThread =
            new WidgetBuilderThread(widgetBuilder, text, parent, failFlag)
```

```

        Thread thread = new Thread(widgetBuilderThread);
        thread.start();
    }
    assertEquals(false, failFlag.get());
}

```

I would not delete that comment as part of a code review. However, I'd be tempted to get rid of it by extracting that for loop out into a function named `attemptRaceCondition`.

Tôi sẽ không xóa comment đó khi review mã. Tuy nhiên, tôi sẽ rất muốn khử nó bằng cách tách vòng lặp for kia ra thành một hàm tên là `attemptRaceCondition`.

Future Bob: As an aside, this is a terrible way to force a **race condition**. When you start up a bunch of threads in hopes that they will race, what they will do instead is line up single file and execute without racing. If you want to force a race condition, you either have to set the race up with semaphores, or introduce random delays and hope that the threads will slip past one another and accidentally enter a race configuration.

Future Bob: Nói thêm một chút, đây là một cách cực tệ để ép xảy ra race condition. Khi bạn khởi chạy một đồng luồng (thread) với hy vọng chúng sẽ đua nhau, cái chúng thực sự làm lại là xếp thành một hàng dọc ngay ngắn rồi lần lượt chạy mà chẳng đua tranh gì cả. Nếu muốn ép ra race condition, bạn phải hoặc dàn dựng cuộc đua bằng semaphore, hoặc chèn vào những khoảng trễ ngẫu nhiên rồi cầu mong các luồng sẽ lách qua nhau và vô tình rơi vào thế đua tranh.

Clarification — Làm rõ

Sometimes it is just helpful to translate the meaning of some obscure **argument** or return value into something that's readable. In general, it is better to find a way to make that argument or return value clear in its own right; but when it's part of the standard library, or in code that you cannot alter, a helpful clarifying comment can be useful.

Đôi khi việc dịch ý nghĩa của một đối số hay giá trị trả về khó hiểu nào đó sang thứ gì dễ đọc hơn quả là có ích. Nhìn chung, tốt hơn hết là tìm cách làm cho bản thân đối số hay giá trị trả về ấy tự nó đã rõ ràng; nhưng khi nó thuộc thư viện chuẩn, hoặc nằm trong đoạn mã bạn không thể sửa, thì một comment làm rõ có thể rất hữu dụng.

I wrote the following comments as part of a feature in FitNesse. I was concerned that the long list of `assertTrue` statements would make my readers' eyes cross. Optical impediments and illusions like this are particularly common in code. So I added the comments to be polite.

Tôi đã viết những comment sau đây khi làm một tính năng trong FitNesse. Tôi lo rằng cái danh sách dằng dặc những câu lệnh `assertTrue` sẽ khiến người đọc hoa cả mắt. Những trở ngại và ảo giác thị giác kiểu này đặc biệt hay gặp trong mã. Thế nên tôi thêm comment vào cho phải phép.

```

public void testCompareTo() throws Exception
{
    WikiPagePath a = PathParser.parse("PageA");
    WikiPagePath ab = PathParser.parse("PageA.PageB");
    WikiPagePath b = PathParser.parse("PageB");
    WikiPagePath aa = PathParser.parse("PageA.PageA");
    WikiPagePath bb = PathParser.parse("PageB.PageB");
    WikiPagePath ba = PathParser.parse("PageB.PageA");
}

```



```

assertTrue(a.compareTo(a) == 0);    // a == a
assertTrue(a.compareTo(b) != 0);    // a != b
assertTrue(ab.compareTo(ab) == 0);  // ab == ab
assertTrue(a.compareTo(b) == -1);   // a < b
assertTrue(aa.compareTo(ab) == -1); // aa < ab
assertTrue(ba.compareTo(bb) == -1); // ba < bb
assertTrue(b.compareTo(a) == 1);    // b < a
assertTrue(ab.compareTo(aa) == 1);  // ab > aa
assertTrue(bb.compareTo(ba) == 1);  // bb > ba
}

```

There is a substantial risk, however, that a clarifying comment is incorrect. In fact, one of the comments in the preceding code is wrong. Go through the previous example and see if you can find it; and note how difficult it is to verify them. That difficulty explains why the clarification was necessary and also why it can be risky.

Tuy nhiên, có một rủi ro không nhỏ là comment làm rõ ấy lại sai. Thực tế, một trong những comment ở đoạn mã trên là sai. Hãy xem lại ví dụ vừa rồi và thử tìm ra nó; đồng thời để ý xem việc kiểm chứng chúng khó đến thế nào. Chính cái khó đó vừa giải thích vì sao việc làm rõ là cần thiết, vừa cho thấy vì sao nó có thể nguy hiểm.

Warning of Consequences — Cảnh báo về hậu quả

Sometimes it is useful to warn other programmers about certain consequences. For example, here is a comment that explains why a particular **test** case is turned off:

Đôi khi cảnh báo các lập trình viên khác về một vài hậu quả nào đó là điều hữu ích. Chẳng hạn, đây là một comment giải thích vì sao một test case cụ thể bị tắt đi:

```

// Don't run unless you have some time to kill.
public void _testWithReallyBigFile()
{
    writeLinesToFile(100000000);
    response.setBody(testFile);
    response.readyToSend(this);
    String responseString = output.toString();
    assertSubString("Content-Length: 1000000000", responseString);
    assertTrue(bytesSent > 1000000000);
}

```

Nowadays, of course, we'd turn off the test case by using the `@Ignore` attribute with an appropriate explanatory string: `@Ignore("Takes too long to run")`. But back in the days before JUnit 4, putting an underscore in front of the method name was a common convention. The comment, while flippant, makes the point pretty well.

Tất nhiên, ngày nay ta sẽ tắt test case bằng thuộc tính `@Ignore` kèm một chuỗi giải thích thích hợp: `@Ignore("Takes too long to run")`. Nhưng ở thời trước JUnit 4, đặt một dấu gạch dưới trước tên method là quy ước phổ biến. Cái comment ấy, dù có phần cợt nhả, vẫn nói lên ý của nó khá tốt.

Here's another, more poignant example:

Đây là một ví dụ khác, thậm chí hơn:

```

public static SimpleDateFormat makeStandardHttpDateFormat()
{
    //SimpleDateFormat is not thread safe,
    //so we need to create each instance independently.
    SimpleDateFormat df = new SimpleDateFormat("EEE, dd MMM yyyy HH:mm:
    df.setTimeZone(TimeZone.getTimeZone("GMT"));
    return df;
}

```

You might complain that there are better ways to solve this problem. I might agree with you. But the comment, as given here, is perfectly reasonable. ³ It will prevent some overly eager programmer from using a static initializer in the name of efficiency.

Bạn có thể phàn nàn rằng có những cách hay hơn để giải quyết vấn đề này. Tôi có thể đồng ý với bạn. Nhưng cái comment như đang thấy ở đây thì hoàn toàn hợp lý. ³ Nó sẽ ngăn được một lập trình viên nào đó quá háng hái dùng static initializer nhân danh hiệu năng.

³ You’ve got to wonder how, in the language of the internet, an environment that is intrinsically asynchronous, this function has been allowed to remain **thread** unsafe for so many decades.

³ Bạn phải tự hỏi làm sao mà trong ngôn ngữ của internet — một môi trường vốn dĩ bất đồng bộ — hàm này lại được phép không an toàn luồng (thread unsafe) suốt bao nhiêu thập kỷ như vậy.

Amplification — Nhấn mạnh

A comment may be used to amplify the importance of something that may otherwise seem inconsequential.

Một comment có thể được dùng để nhấn mạnh tầm quan trọng của thứ mà nếu không, ta sẽ tưởng là chẳng đáng bận tâm.

A good example of that is the call to trim() below. The trim function is called so frequently and so frivolously that it’s easy to ignore. The comment makes clear that you should not ignore this one.

Một ví dụ hay cho chuyện đó là lời gọi trim() bên dưới. Hàm trim được gọi thường xuyên đến mức tùy tiện, nên rất dễ bị bỏ qua. Cái comment nói rõ rằng lần này thì bạn đừng có bỏ qua.

```

String listItemContent = match.group(3).trim();
// the trim is real important. It removes the starting
// spaces that could cause the item to be recognized
// as another list.
new ListItemWidget(this, listItemContent, this.level + 1);
return buildList(text.substring(match.end()));

```

Javadocs (and Their ilk) in Public APIs — Javadoc (và họ hàng của nó) trong các API công khai

Nowadays, most language systems have some kind of documentation system that scrapes special document comments from your source files and arranges them in a nicely formatted document. In Java these are called Javadocs.

Ngày nay, hầu hết các hệ ngôn ngữ đều có một dạng hệ thống tài liệu nào đó, chuyên bóc các comment tài liệu đặc biệt ra khỏi tệp mã nguồn của bạn rồi sắp xếp chúng thành một tài liệu được định dạng đẹp đẽ. Trong Java, chúng được gọi là *Javadoc*.

There is nothing quite so helpful and satisfying as a well-described public API. The Javadocs for the standard Java library are a case in point. It would be difficult, at best, to write Java programs without them.

Không gì hữu ích và mãn nguyện bằng một API công khai được mô tả tử tế. Javadoc của thư viện chuẩn Java là một minh chứng. Không có chúng thì viết chương trình Java sẽ khó khăn vô cùng, nếu không muốn nói là bất khả.

If you are writing a public API, then you should certainly write good document comments for it. But keep in mind the rest of the advice in this chapter. Document comments can be just as misleading, nonlocal, and dishonest as any other kind of comment.

Nếu bạn đang viết một API công khai thì chắc chắn bạn nên viết comment tài liệu tốt cho nó. Nhưng hãy nhớ những lời khuyên còn lại trong chương này. Comment tài liệu cũng có thể gây hiểu nhầm, lạc chỗ và gian dối chẳng kém bất kỳ loại comment nào khác.

Bad Comments — Comment tệ

Bad comments are usually crutches or excuses for poor code or justifications for insufficient decisions, amounting to little more than the programmer talking to themselves.

Comment tệ thường là cái nạng chống hoặc lời bào chữa cho mã dở, hoặc lời biện minh cho những quyết định nửa vời, rốt cuộc chẳng hơn gì việc lập trình viên đang lẩm bẩm tự nói với chính mình.

When I see comments in this category, I delete them. What follows are some specific examples of such comments.

Khi thấy comment thuộc loại này, tôi xoá chúng. Sau đây là vài ví dụ cụ thể về những comment như thế.

Mumbling — Lẩm bẩm

Plopping in a comment just because you feel you should or because the process requires it is a hack. If you decide to write a comment, then spend the time necessary to make sure it is the best comment you can write.

Quăng đại một comment vào chỉ vì bạn thấy mình *nên* làm vậy, hoặc vì quy trình bắt buộc, là một trò đối phó. Nếu bạn đã quyết định viết comment thì hãy bỏ ra chừng ấy thời gian cần thiết để chắc chắn đó là comment tốt nhất bạn có thể viết.

Here, for example, is a case I found in FitNesse. It appears to me that the author was in a hurry, or just not paying much attention. His mumbling left behind an enigma:

Chẳng hạn, đây là một trường hợp tôi tìm thấy trong FitNesse. Với tôi, có vẻ tác giả đang vội, hoặc đơn giản là không mấy để tâm. Tiếng lẩm bẩm của anh ta để lại một câu đố:

```
public void loadProperties()  
{  
    try
```

```

{
    String propertiesPath = propertiesLocation + "/" + PROPERTIES_FILE;
    FileInputStream propertiesStream =
        new FileInputStream(propertiesPath);
    loadedProperties.load(propertiesStream);
}
catch(IOException e)
{
    // No properties files means all defaults are loaded
}
}

```

What does that comment in the catch block mean? Clearly it meant something to the author, but the meaning does not come through all that well. Apparently, if we get an `IOException`, it means that there was no “properties” file; and in that case, all the defaults are loaded. But who loads all the defaults? Were they loaded before the call to `loadProperties.load`? Or did `loadProperties.load` catch the **exception**, load the defaults, and then pass the exception on for us to ignore? Or did `loadProperties.load` load all the defaults before attempting to load the file? Was the author trying to comfort himself about the fact that he was leaving the catch block empty? Or—and this is the scary possibility—was the author trying to tell himself to come back here later and write the code that would load the defaults?

Cái comment trong khối catch kia nghĩa là gì? Rõ ràng nó có nghĩa gì đó với tác giả, nhưng cái nghĩa ấy chẳng truyền tải được ra ngoài cho tử tế. Có vẻ như nếu gặp `IOException` thì nghĩa là không có tệp “properties”; và trong trường hợp đó, mọi giá trị mặc định sẽ được nạp. Nhưng ai nạp các giá trị mặc định? Chúng đã được nạp trước lời gọi `loadProperties.load` chưa? Hay là `loadProperties.load` bắt exception, nạp mặc định, rồi ném exception ra ngoài cho ta bỏ qua? Hay `loadProperties.load` nạp toàn bộ mặc định trước khi thử nạp tệp? Phải chăng tác giả đang tự an ủi mình về chuyện anh ta để khối catch trống trơn? Hay là — và đây mới là khả năng đáng sợ — tác giả đang tự nhắc mình quay lại đây sau để viết đoạn mã nạp các giá trị mặc định?

Our only recourse is to examine the code in other parts of the system to find out what’s going on here. Any comment that forces you to look in another module for the meaning of that comment has failed to communicate to you and is not worth the bits it consumes.

Cách duy nhất còn lại của chúng ta là đi soi mã ở những chỗ khác trong hệ thống để tìm hiểu xem ở đây đang xảy ra chuyện gì. Bất kỳ comment nào bắt bạn phải nhìn sang một module khác mới hiểu được nghĩa của chính nó thì đều đã thất bại trong việc giao tiếp với bạn, và không xứng với chỗ bit mà nó chiếm.

Wouldn’t it have been better if the programmer had written the following?

Chẳng phải sẽ tốt hơn nếu lập trình viên viết như thế này sao?

```

catch(IOException e)
{
    LoadedProperties.loadDefaults();
}

```

Redundant Comments — Comment thừa

The following code shows a simple function with a header comment that tells us nothing that the code does not. The comment probably takes longer to read than the code itself.

Đoạn mã sau cho thấy một hàm đơn giản kèm comment đầu hàm chẳng nói lên điều gì mà bản thân mã chưa nói. Đọc cái comment ấy có khi còn tốn thời gian hơn đọc chính đoạn mã.

```
// Utility method that returns when this.closed is true.
// Throws an exception if the timeout is reached.
public synchronized void
waitForClose(long timeoutMillis) throws Exception
{
    if(!closed)
    {
        wait(timeoutMillis);
        if(!closed)
            throw new Exception("MockResponseSender could not be closed");
    }
}
```

What purpose does this comment serve? It's certainly not more informative than the code. It does not justify the code, nor provide intent or rationale. It is not easier to read than the code. Indeed, it is less precise than the code and entices the reader to accept that lack of precision in lieu of true understanding. It is rather like a glad-handing used-car salesman assuring you that you don't need to look under the hood.

Comment này phục vụ mục đích gì? Chắc chắn nó không giàu thông tin hơn đoạn mã. Nó không biện giải cho mã, cũng chẳng cho biết ý định hay lý do. Nó không dễ đọc hơn mã. Thật ra, nó còn kém chính xác hơn mã và dụ người đọc chấp nhận sự thiếu chính xác ấy thay cho việc hiểu thật sự. Nó chẳng khác gì một gã bán xe cũ xởi lởi bắt tay bạn rồi bảo đảm rằng bạn chẳng cần phải mở nắp ca-pô ra xem làm gì.

Misleading Comments — Comment gây hiểu nhầm

Sometimes, with all the best intentions, a programmer makes a statement in their comments that allows misinterpretation. Consider, for another moment, the badly redundant but also subtly misleading comment we saw in the last example.

Đôi khi, với tất cả thiện chí, lập trình viên vẫn đưa ra trong comment một phát biểu có thể bị hiểu sai. Hãy nán lại một chút với cái comment vừa thừa thãi vừa gây hiểu nhầm một cách tinh vi mà ta đã thấy trong ví dụ vừa rồi.

Did you discover how the comment was misleading? The method does not return when this.closed becomes true . It returns if this.closed is true ; otherwise, it waits for a blind timeout and then throws an exception if this.closed is still not true .

Bạn đã nhận ra cái comment ấy gây hiểu nhầm ở chỗ nào chưa? Method này không hề trả về khi this.closed trở thành true. Nó trả về nếu this.closed đang là true; còn không thì nó chờ mù quáng cho hết một khoảng timeout rồi ném exception nếu this.closed vẫn chưa là true.

This subtle bit of misinformation, couched in a comment that is harder to read than the body of the code, could cause another programmer to blithely call this function in the expectation that it will return as soon as `this.closed` becomes `true`. That poor programmer would then find themselves in a debugging session trying to figure out why their code executed so slowly.

Mẫu thông tin sai lệch tinh vi ấy, lại được gói trong một comment còn khó đọc hơn cả thân hàm, có thể khiến một lập trình viên khác hồn nhiên gọi hàm này với kỳ vọng rằng nó sẽ trả về ngay khi `this.closed` trở thành `true`. Rồi anh chàng lập trình viên tội nghiệp đó sẽ thấy mình sa vào một phiên debug, cố tìm hiểu vì sao mã của mình chạy chậm đến thế.

Redundancy and Imprecision — Thừa thãi và thiếu chính xác

Now consider the legion of useless and redundant Javadocs in this code taken long ago from Tomcat.

Giờ hãy xem đạo quân Javadoc vô dụng và thừa thãi trong đoạn mã lấy từ Tomcat cách đây đã lâu.

```
public abstract class ContainerBase
    implements Container, Lifecycle, Pipeline,
    MBeanRegistration, Serializable {
    /**
     * The processor delay for this component.
     */
    protected int backgroundProcessorDelay = -1;
    /**
     * The lifecycle event support for this component.
     */
    protected LifecycleSupport lifecycle =
        new LifecycleSupport(this);
    /**
     * The container event listeners for this Container.
     */
    protected ArrayList listeners = new ArrayList();
    /**
     * The Loader implementation with which this Container is
     * associated.
     */
    protected Loader loader = null;
    /**
     * The Logger implementation with which this Container is
     * associated.
     */
    protected Log logger = null;
    /**
     * Associated logger name.
     */
    protected String logName = null;
    /**
     * The Manager implementation with which this Container is
     * associated.
```

```

    */
protected Manager manager = null;
/**
 * The cluster with which this Container is associated.
 */
protected Cluster cluster = null;
/**
 * The human-readable name of this Container.
 */
protected String name = null;
/**
 * The parent Container to which this Container is a child.
 */
protected Container parent = null;
/**
 * The parent class loader to be configured when we install a
 * Loader.
 */
protected ClassLoader parentClassLoader = null;
/**
 * The Pipeline object with which this Container is
 * associated.
 */
protected Pipeline pipeline = new StandardPipeline(this);
/**
 * The Realm with which this Container is associated.
 */
protected Realm realm = null;
/**
 * The resources DirContext object with which this Container
 * is associated.
 */
protected DirContext resources = null;

```

These comments serve only to clutter and obscure the code. They serve no documentary purpose at all. Look at the first one. The **variable** is named `backgroundProcessorDelay`. The comment is the

Những comment này chỉ tổ làm mã thêm bừa bộn và mờ mịt. Chúng chẳng phục vụ mục đích tư liệu nào hết. Hãy nhìn cái đầu tiên. Biến tên là `backgroundProcessorDelay`. Còn comment thì là

`processor delay for this component..`

The redundancy is painful. The variable name says more than the comment. And what in the world did the author mean by the word **component**?

Sự thừa thãi ấy nhức nhối thật. Tên biến còn nói được nhiều hơn cả comment. Và trên đời này tác giả định ám chỉ điều gì bằng chữ `component` vậy?

Look further down. The same redundancy glares at us from every one of those Javadocs. On top of that, the author seems to use the words `component` and `container` interchangeably. Is that intentional? Is there a difference between `component` and `container`? And why are they sometimes capitalized and sometimes not? And look at `parentClassLoader`. Is the lack of `component` or

container significant?

Nhìn tiếp xuống dưới. Vẫn sự thừa thãi ấy chường ra ở từng cái Javadoc một. Chưa hết, tác giả có vẻ dùng lẫn lộn hai chữ component và container. Đó là cố ý sao? Giữa component và container có gì khác nhau không? Và tại sao lúc thì viết hoa, lúc thì không? Rồi hãy nhìn `parentClassLoader`. Việc thiếu chữ component hay container ở đây có ý nghĩa gì chẳng?

Perhaps you think I'm being picky. I probably am. But code is precise, and imprecision in the comments can be both confusing and misleading. It also makes me wonder if that imprecision carries over into the code.

Có thể bạn nghĩ tôi đang bới lông tìm vết. Chắc là tôi có bới thật. Nhưng mã thì chính xác, còn sự thiếu chính xác trong comment vừa gây bối rối vừa dễ dẫn người ta đi lạc. Nó còn khiến tôi tự hỏi liệu cái sự thiếu chính xác đó có lây sang cả phần mã hay không.

Mandated Comments — Comment bị bắt buộc

It is just plain silly to have a rule that says that every function must have a Javadoc, or every variable must have a comment. Comments like this just clutter up the code, propagate misinformation, and lend to general confusion and disorganization.

Đặt ra một quy định rằng mọi hàm đều phải có Javadoc, hay mọi biến đều phải có comment, thì đúng là ngớ ngẩn hết chỗ nói. Những comment kiểu đó chỉ làm mã thêm bừa bộn, phát tán thông tin sai lệch, rồi góp phần vào sự rối rắm và lộn xộn chung.

For example, required Javadocs for every function lead to abominations like this:

Chẳng hạn, việc bắt buộc mọi hàm phải có Javadoc dẫn tới những thứ quái gở như thế này:

```
/**
 * @param title The title of the CD
 * @param author The author of the CD
 * @param tracks The number of tracks on the CD
 * @param durationInMinutes The duration of the CD in minutes
 */
public void addCD(String title, String author,
                  int tracks, int durationInMinutes) {
    CD cd = new CD();
    cd.title = title;
    cd.author = author;
    cd.tracks = tracks;
    cd.duration = duration;
    cdList.add(cd);
}
```

If you ran the Javadoc tool over this code, and then deleted the comments and ran the tool again, the two HTML files created would be nearly identical. So the clutter of this Javadoc adds nothing and serves only to obfuscate the code and create the potential for misdirection.

Nếu bạn chạy công cụ Javadoc trên đoạn mã này, rồi xoá hết comment đi và chạy lại công cụ đó lần nữa, hai file HTML sinh ra sẽ gần như giống hệt nhau. Vậy nên mở Javadoc bừa bộn này chẳng thêm được gì, chỉ tổ làm mã khó nhìn và tạo cơ hội cho việc dẫn dắt sai đường.

Journal Comments — Comment nhật ký

Sometimes people add a comment to the start of a module every time they edit it. These comments accumulate as a kind of journal, or log, of every change that has ever been made. I have seen some modules with dozens of pages of these run-on journal entries.

Đôi khi người ta cứ mỗi lần sửa một module lại thêm một comment vào đầu module đó. Những comment này tích tụ lại thành một dạng nhật ký, hay bản ghi, của mọi thay đổi từng được thực hiện. Tôi từng thấy có module mang cả mấy chục trang những dòng nhật ký lê thê như vậy.

```
* Changes (from 11-Oct-2001)
* -----
* 11-Oct-2001 : Re-organised the class and moved it to new
*              package com.jrefinery.date (DG);
* 05-Nov-2001 : Added a getDescription() method, and
*              eliminated NotableDate class (DG);
* 12-Nov-2001 : IBD requires setDescription() method, now
*              that NotableDate class is gone (DG);
*              Changed getPreviousDayOfWeek(),
*              getFollowingDayOfWeek() and
*              getNearestDayOfWeek() to correct
*              bugs (DG);
* 05-Dec-2001 : Fixed bug in SpreadsheetDate class (DG);
* 29-May-2002 : Moved the month constants into a separate
*              interface (MonthConstants) (DG);
*
* 27-Aug-2002 : Fixed bug in addMonths() method, thanks to
*              N????levka Petr (DG);
* 03-Oct-2002 : Fixed errors reported by Checkstyle (DG);
* 13-Mar-2003 : Implemented Serializable (DG);
* 29-May-2003 : Fixed bug in addMonths method (DG);
*
* 0Sep-2003 : Implemented Comparable. Updated the isInRange
*              javadocs (DG);
* 05-Jan-2005 : Fixed bug in addYears() method (1096282) (DG);
```

Long ago there was a good reason to create and maintain these log entries at the start of every module. We didn't have source code control systems that did it for us. Nowadays, however, these long journals are just more clutter to obfuscate the module. They should be completely removed. Let the source code control system keep the journal for you.

Ngày xưa, việc tạo và duy trì những dòng nhật ký này ở đầu mỗi module là có lý do chính đáng. Hồi ấy chúng ta chưa có hệ thống quản lý mã nguồn làm hộ việc đó. Nhưng ngày nay, những cuốn nhật ký dài dằng dặc ấy chỉ là thêm rác làm mờ module mà thôi. Nên xóa sạch chúng đi. Cứ để hệ thống quản lý mã nguồn ghi nhật ký hộ bạn.

Noise Comments — Comment nhiễu

Sometimes you see comments that are nothing but noise. They restate the obvious and provide no new information.

Đôi khi bạn gặp những comment chẳng là gì ngoài nhiều. Chúng nhắc lại điều hiển nhiên và không cung cấp thông tin gì mới.

```
/**
 * Default constructor.
 */
protected AnnualDateRule() {
}
```

No, really? Or how about this:

Ồ, thế cơ à? Hay là thế này:

```
/** The day of the month. */
private int dayOfMonth;
```

And then there's this paragon of redundancy:

Rồi còn có kiệt tác của sự thừa thãi này nữa:

```
/**
 * Returns the day of the month.
 *
 * @return the day of the month.
 */
public int getDayOfMonth() {
    return dayOfMonth;
}
```

These comments are so noisy that we learn to ignore them. As we read through the code, our eyes simply skip over them. Eventually, as the code changes, we fail to notice that the comments have begun to lie.

Những comment này nhiều tới mức ta học được cách phớt lờ chúng. Khi đọc mã, mắt ta cứ thế lướt qua. Rồi dần dà, khi mã thay đổi, ta chẳng buồn để ý rằng đám comment đã bắt đầu nói dối.

The first comment below seems appropriate. ⁴ It explains why the catch block is being ignored. But the second comment is pure noise. Apparently the programmer was just so frustrated with writing try / catch blocks in this function that he needed to vent.

Comment đầu tiên bên dưới có vẻ hợp lý. ⁴ Nó giải thích vì sao khối catch bị bỏ qua. Nhưng comment thứ hai thì thuần túy là nhiễu. Có vẻ lập trình viên đã bực bội với việc phải viết các khối try / catch trong hàm này tới mức cần phải xả ra cho hả.

⁴ The current trend for IDEs to check spelling in comments will be a balm for those of us who read a lot of code.

⁴ Xu hướng gần đây của các IDE là kiểm tra chính tả trong comment, và đó sẽ là liều thuốc an ủi cho những ai như chúng ta, những người phải đọc rất nhiều mã.

```
private void startSending()
{
    try
    {
        doSending();
    }
}
```

```

catch(SocketException e)
{
    // normal. someone stopped the request.
}
catch(Exception e)
{
    try
    {
        response.add(ErrorResponder.makeExceptionString(e));
        response.closeAll();
    }
    catch(Exception e1)
    {
        //Give me a break!
    }
}
}
}

```

Rather than venting in a worthless and noisy comment, the programmer should have recognized that his frustration could be resolved by improving the **structure** of his code. He should have redirected his energy to extracting that last try / catch block into a separate function, like this:

Thay vì xả bực vào một comment vô giá trị và đầy nhiễu, lẽ ra lập trình viên nên nhận ra rằng nỗi bực bội đó có thể giải quyết bằng cách cải thiện cấu trúc mã của mình. Anh ta lẽ ra nên dồn năng lượng đó vào việc trích khối try / catch cuối cùng ra thành một hàm riêng, như thế này:

```

private void startSending()
{
    try
    {
        doSending();
    }
    catch(SocketException e)
    {
        // normal. someone stopped the request.
    }
    catch(Exception e)
    {
        addExceptionAndCloseResponse(e);
    }
}
private void addExceptionAndCloseResponse(Exception e)
{
    try
    {
        response.add(ErrorResponder.makeExceptionString(e));
        response.closeAll();
    }
    catch(Exception e1)
    {

```

```
}  
}
```

Replace the temptation to create noise with the determination to clean your code. You'll find it makes you a better and happier programmer.

Hãy thay cái ham muốn tạo ra nhiễu bằng quyết tâm dọn sạch mã của bạn. Bạn sẽ thấy nó khiến bạn thành một lập trình viên giỏi hơn và hạnh phúc hơn.

Scary Noise — Nhiễu đáng sợ

Javadocs can also be noisy. What purpose do the following Javadocs (from a well-known open source library) serve? Answer: nothing. They are just redundant noisy comments written out of some misplaced desire to provide documentation.

Javadoc cũng có thể gây nhiễu. Những Javadoc dưới đây (lấy từ một thư viện mã nguồn mở nổi tiếng) phục vụ mục đích gì? Xin thưa: chẳng gì cả. Chúng chỉ là những comment thừa thãi và nhiễu, viết ra vì một khao khát đặt nhầm chỗ là phải cung cấp tài liệu.

```
/** The name. */  
private String name;  
/** The version. */  
private String version;  
/** The licenceName. */  
private String licenceName;  
/** The version. */  
private String info;
```

Read these comments again more carefully. Do you see the cut-paste error? If authors aren't paying attention when comments are written (or pasted), why should readers be expected to profit from them?

Hãy đọc lại những comment đó kỹ hơn. Bạn có thấy lỗi cắt-dán không? Nếu ngay cả tác giả cũng chẳng buồn để tâm khi viết (hoặc dán) comment, thì có gì lại mong người đọc thu được lợi ích từ chúng?

Comments — Comment

In the first edition of this book, I was in favor of leaving TODO comments in the code as reminders of things that need to be taken care of. I was enamored of the features in my IDE that allowed me to quickly see all the TODO comments in my projects.

Trong lần xuất bản đầu tiên của cuốn sách này, tôi từng ủng hộ việc để lại các comment TODO trong mã như lời nhắc về những việc còn phải xử lý. Tôi mê mẩn tính năng của IDE cho phép mình xem nhanh toàn bộ comment TODO trong các dự án.

```
//TODO-MdM these are not needed  
// We expect this to go away when we do the checkout model  
protected VersionInfo makeVersion() throws Exception  
{  
    return null;  
}
```

I have since changed my mind about this. After a few years of reading code written by others and by myself, I came to understand that TODO means Don't Do . So I changed my rule.

Từ đó tới nay tôi đã đổi ý. Sau vài năm đọc mã do người khác và do chính mình viết, tôi ngộ ra rằng TODO có nghĩa là Đừng Làm. Thế nên tôi đổi quy tắc của mình.

I still use TODO comments, but I will not check them in. I will either do the thing that needs to be done, or eliminate the reason that it needs to be done, or put it into the backlog of tasks that need to be performed.

Tôi vẫn dùng comment TODO, nhưng tôi sẽ không check in chúng. Tôi sẽ hoặc làm luôn việc cần làm, hoặc loại bỏ lý do khiến việc đó cần phải làm, hoặc đưa nó vào backlog những việc cần thực hiện.

Use a Function or a Variable Instead of a Comment — Dùng hàm hay biến thay cho comment

Consider the following stretch of code:

Hãy xem đoạn mã sau:

```
// does the module from the global list <mod> depend on the
// subsystem we are part of?
if (smodule.getDependSubsystems().contains(subSysMod.getSubSystem()))
```

What does this comment mean? What is , and why is it in angle brackets? I put this code into my IDE and extracted out a few explanatory variables. Here is the result.

Comment này nghĩa là gì? là cái gì, và tại sao nó lại nằm trong dấu ngoặc nhọn? Tôi đưa đoạn mã này vào IDE của mình rồi trích ra vài biến giải thích. Kết quả đây.

```
ArrayList moduleDependants = smodule.getDependSubsystems();
String ourSubSystem = subSysMod.getSubSystem();
if (moduleDependants.contains(ourSubSystem))
```

It seems to me that this **refactoring** helps the final if statement read like well-written prose and is easier to understand than the comment (which I still don't understand).

Với tôi, đợt refactor này giúp câu lệnh if cuối cùng đọc lên trôi chảy như văn xuôi viết tốt, và dễ hiểu hơn hẳn so với cái comment kia (thứ mà đến giờ tôi vẫn không hiểu).

Position Markers — Dấu mốc vị trí

Sometimes programmers like to mark a particular position in a source file. For example, I recently found this in a program I was looking through:

Đôi khi lập trình viên thích đánh dấu một vị trí cụ thể trong file mã nguồn. Chẳng hạn, gần đây tôi bắt gặp thứ này trong một chương trình đang xem:

```
// Actions //////////////////////////////////////
```

There are rare times when it makes sense to gather certain functions together beneath a banner like this. But in general, they are clutter that should be avoided—especially the noisy train of slashes at the end.

Có những lúc hiểm hoi mà việc gom vài hàm lại với nhau dưới một tấm biển như thế là hợp lý. Nhưng nhìn chung, chúng là rác nên tránh — nhất là cái đuôi dấu gạch chéo lê thê đầy nhiễu ở cuối.

Think of it this way. A banner is startling and obvious, especially if you don't see banners very often. So use them very sparingly, and only when the benefit is significant. If you overuse banners, they'll fall into the background noise and be ignored—like the little boy who cried “Wolf!”

Hãy nghĩ thế này. Một tấm biển thì gây giật mình và nổi bật, nhất là khi bạn ít khi thấy biển. Vậy nên hãy dùng thật dè sẻn, và chỉ khi lợi ích đủ đáng kể. Nếu lạm dụng biển, chúng sẽ chìm vào đám nhiễu nền và bị phớt lờ — hệt như cậu bé chăn cừu kêu “Sói!”.

Attributions and Bylines — Ghi công và ký tên

/ Added by Rick */*

This is just graffiti. Please don't spray-paint your name all over the source code. Source code control systems are very good at remembering who added what, when. There is no need to pollute the code with little bylines. You might think that such comments would be useful in order to help others know who to talk to about the code. But the reality is that they tend to stay around for years and years, getting less and less accurate and relevant. The same can be said for PR numbers and JIRA tags. Put those in commit comments, not in the source code.

Đây chỉ là vẽ bậy. Xin đừng xịt sơn tên mình khắp mã nguồn. Các hệ thống quản lý mã nguồn rất giỏi trong việc nhớ ai đã thêm gì, vào lúc nào. Không cần thiết phải làm ô nhiễm mã bằng mấy dòng ký tên nhỏ nhặt. Bạn có thể nghĩ rằng những comment như vậy sẽ hữu ích để người khác biết cần trao đổi với ai về đoạn mã đó. Nhưng thực tế là chúng có xu hướng nằm lì năm này qua năm khác, ngày càng sai lệch và ngày càng lạc lõng. Điều tương tự cũng đúng với số hiệu PR và mã JIRA. Hãy để chúng trong comment của commit, đừng để trong mã nguồn.

Commented-Out Code — Mã bị comment lại

Few practices are as odious as commenting-out code. Commented-out code is an abomination before nature and nature's God. Don't check this stuff in!

Ít có thói quen nào đáng ghét bằng việc comment mã lại. Mã bị comment lại là điều ghê tởm trước tự nhiên và trước Đấng tạo ra tự nhiên. Đừng check in cái thứ này!

```
InputStreamResponse response = new InputStreamResponse();
response.setBody(formatter.getResultStream(),
                 formatter.getByteCount());
// InputStream resultsStream = formatter.getResultStream();
// StreamReader reader = new StreamReader(resultsStream);
// response.setContent(reader.read(formatter.getByteCount()));
```

Others who see that commented-out code will be unlikely to have the courage to delete it. They'll think it is there for a reason and is too important to delete. So commented-out code gathers like dregs at the bottom of a bad bottle of wine. Consider this from Apache Commons:

Những người khác thấy đoạn mã bị comment lại đó sẽ khó lòng đủ can đảm để xóa nó đi. Họ sẽ nghĩ nó nằm đó vì một lý do nào đó và quan trọng đến mức không được xóa. Thế là

mã bị comment lại cứ đóng cặn như cặn rượu ở đáy một chai vang dở. Hãy xem ví dụ này từ Apache Commons:

```
this.bytePos = writeBytes(pngIdBytes, 0);
//hdrPos = bytePos;
writeHeader();
writeResolution();
//dataPos = bytePos;
if (writeImageData()) {
    writeEnd();
    this.pngBytes = resizeByteArray(this.pngBytes, this.maxPos);
}
else {
    this.pngBytes = null;
}
return this.pngBytes;
```

Why are those two lines of code commented? Are they important? Were they left as reminders for some imminent change? Or are they just cruft that someone commented out years ago and has simply not bothered to clean up?

Vì sao hai dòng mã kia bị comment lại? Chúng có quan trọng không? Chúng được để lại như lời nhắc cho một thay đổi sắp tới nào đó chăng? Hay chúng chỉ là rác mà ai đó đã comment lại từ mấy năm trước rồi chẳng buồn dọn?

There was a time, back in the sixties, when commenting-out code might have been useful. But we've had good source code control systems for a very long time now. Those systems will remember the code for us. We don't have to comment it out anymore.

Từng có một thời, hồi những năm sáu mươi, việc comment mã lại có lẽ còn hữu ích. Nhưng chúng ta đã có những hệ thống quản lý mã nguồn tốt từ rất lâu rồi. Các hệ thống ấy sẽ nhớ hộ chúng ta đoạn mã đó. Ta không cần comment nó lại nữa.

I still comment out code while debugging from time to time. It can be a useful technique. But my rule is to never check in the commented-out code.

Thỉnh thoảng tôi vẫn comment mã lại khi debug. Đó có thể là một kỹ thuật hữu ích. Nhưng quy tắc của tôi là không bao giờ check in đoạn mã bị comment lại.

HTML Comments — Comment kiểu HTML

HTML in source code comments can be awful, as you can tell by reading the code below. It makes the comments hard to read in the one place where they should be easiest to read: the code. If comments are going to be extracted by some tool (like Javadoc) to appear in a nicely formatted document, then it should be the responsibility of that tool, and not the programmer, to adorn the comments with appropriate **formatting**.

HTML trong comment mã nguồn có thể tệ hại, như bạn có thể thấy khi đọc đoạn mã dưới đây. Nó khiến comment khó đọc ở đúng nơi lẽ ra chúng phải dễ đọc nhất: trong mã. Nếu comment sẽ được một công cụ nào đó (như Javadoc) trích ra để xuất hiện trong một tài liệu được định dạng đẹp đẽ, thì trách nhiệm tô điểm comment bằng định dạng thích hợp phải thuộc về công cụ đó, chứ không phải lập trình viên.

/**

```

* Task to run fit tests.
* This task runs fitness tests and publishes the results.
* <p/>
* <pre>
* Usage:
* <taskdef name="execute-fitness-tests"
*   classname="fitness.ant.ExecuteFitnessTestsTask"
*   classpathref="classpath" />
* OR
* <taskdef classpathref="classpath"
*   resource="tasks.properties" />
* </pre>
* <execute-fitness-tests
*   suitepage="FitNesse.SuiteAcceptanceTests"
*   fitnessreport="8082"
*   resultsdir="{results.dir}"
*   resultshtmlpage="fit-results.html"
*   classpathref="classpath" />
* </pre>
*/

```

Nonlocal Information — Thông tin không thuộc chỗ đó

If you must write a comment, then make sure it describes the code it appears near. Don't offer systemwide information in the context of a local comment. Consider, for example, the Javadoc comment below. Aside from the fact that it is horribly redundant, it also offers information about the default port. And yet the function has absolutely no control over what that default is. The comment is not describing the function, but some other, far-distant part of the system. Of course, there is no guarantee that this comment will be changed when the code containing the default is changed.

Nếu buộc phải viết comment, hãy chắc rằng nó mô tả đoạn mã nằm ngay cạnh nó. Đừng đưa thông tin ở tầm toàn hệ thống vào bối cảnh của một comment cục bộ. Chẳng hạn, hãy xem comment Javadoc dưới đây. Ngoài chuyện nó thừa thãi đến khủng khiếp, nó còn cung cấp thông tin về cổng mặc định. Vậy mà hàm này lại chẳng hề kiểm soát được giá trị mặc định ấy. Comment không mô tả cái hàm, mà mô tả một phần khác, xa lắc, của hệ thống. Và tất nhiên, chẳng có gì bảo đảm comment này sẽ được sửa khi đoạn mã chứa giá trị mặc định kia thay đổi.

```

/**
 * Port on which fitness would run. Defaults to 8082.
 *
 * @param fitnessPort
 */
public void setFitnessPort(int fitnessPort)
{
    this.fitnessPort = fitnessPort;
}

```

Too Much Information — Quá nhiều thông tin

Don't put interesting historical discussions or irrelevant descriptions of details into your comments. The comment below was extracted from a module designed to test that a function could encode and decode base64. Other than the RFC number, someone reading this code has no need for the arcane information contained in the comment.

Đừng nhét những cuộc bàn luận lịch sử thú vị hay những mô tả chi tiết chẳng liên quan vào comment của bạn. Comment dưới đây được trích từ một module viết ra để kiểm thử xem một hàm có mã hoá và giải mã được base64 hay không. Ngoài số hiệu RFC ra, người đọc đoạn mã này chẳng cần đến mớ thông tin bí hiểm chứa trong comment đó.

```
/*
RFC 2045 - Multipurpose Internet Mail Extensions (MIME)
Part One: Format of Internet Message Bodies
section 6.8. Base64 Content-Transfer-Encoding
The encoding process represents 2bit groups of input bits as output
strings of 4 encoded characters. Proceeding from left to right, a
2bit input group is formed by concatenating 3 8-bit input groups.
These 24 bits are then treated as 4 concatenated 6-bit groups, eac
of which is translated into a single digit in the base64 alphabet.
When encoding a bit stream via the base64 encoding, the bit stream
must be presumed to be ordered with the most-significant-bit first
That is, the first bit in the stream will be the high-order bit in
the first 8-bit byte, and the eighth bit will be the low-order bit
in the first 8-bit byte, and so on.
*/
```

Unobvious Connection — Liên hệ không rõ ràng

The connection between a comment and the code it describes should be obvious. If you are going to the trouble to write a comment, then at least you'd like the reader to be able to look at the comment and the code and understand what the comment is talking about. Consider, for example, this comment drawn from Apache Commons:

Mối liên hệ giữa một comment và đoạn mã mà nó mô tả phải rõ ràng. Nếu bạn đã chịu khó viết comment, thì ít nhất bạn cũng muốn người đọc nhìn vào comment và mã rồi hiểu được comment đang nói về cái gì. Chẳng hạn, hãy xem comment này lấy từ Apache Commons:

```
/*
 * start with an array that is big enough to hold all the pixels
 * (plus filter bytes), and an extra 200 bytes for header info
 */
this pngBytesnew byte[((this width + 1) * this height * 3) + 200]

this.pngBytes = new byte[((this.width + 1) * this.height * 3) + 200];
```

What is a filter byte? Did the author mean filler byte? Do those bytes relate to the +1? Or to the *3? Both? Is a pixel a byte? Why 200? The purpose of a comment is to explain code that does not explain itself. It is a pity when a comment needs its own explanation.

Filter byte là cái gì? Tác giả định nói filler byte chẳng? Những byte đó có liên quan đến cái +1 không? Hay đến cái *3? Cả hai? Một pixel có phải một byte không? Sao lại 200? Mục đích của comment là giải thích đoạn mã không tự giải thích được chính nó. Thật đáng tiếc khi một comment lại cần đến lời giải thích cho riêng nó.

Function Headers — Header của hàm

Short functions don't need much description. A well-chosen name and signature for a small function that does one thing is usually better than a comment header. For example, I would prefer:

Hàm ngắn thì không cần mô tả nhiều. Với một hàm nhỏ chỉ làm một việc, một cái tên và chữ ký được chọn khéo thường tốt hơn một comment header. Chẳng hạn, tôi sẽ thích:

```
public static int[] generateNPrimes(int n) {
```

over:

hơn là:

```
/**
 * Computes the first prime numbers; the return value contains the
 * computed primes, in increasing order of size.
 * @param n
 *     How many prime numbers to compute.
 */
public static int[] generate(int n) {
```

Javadocs in Nonpublic Code — Javadoc trong mã không công khai

As useful as Javadocs are for public APIs, they are more of an impediment than a benefit to code that is maintained by a small team and is not intended for public consumption. The team already knows the code and is not likely to run the Javadoc tool to generate Javadoc pages, so the extra formality of the Javadoc comments amounts to little more than cruft and distraction.

Javadoc hữu ích với các API công khai bao nhiêu, thì với đoạn mã do một nhóm nhỏ bảo trì và không nhằm phục vụ công chúng, nó lại là chướng ngại nhiều hơn là lợi ích. Cả nhóm vốn đã hiểu đoạn mã và chẳng mấy khi chạy công cụ Javadoc để sinh ra các trang tài liệu, nên cái sự trịnh trọng thừa thãi của comment Javadoc rốt cuộc chẳng khác gì rác và thứ gây phân tâm.

Example — Ví dụ

I wrote the following module for the first XP Immersion. It was intended to be an example of bad coding and commenting style. Kent Beck then refactored this code into a much more pleasant form in front of several dozen enthusiastic students. Later I adapted the example for my book *Agile Software Development, Principles, Patterns, and Practices* and the first of my *Craftsman* articles published in *Software Development* magazine.

Tôi viết module dưới đây cho khoá XP Immersion đầu tiên. Nó được dựng lên như một ví dụ về phong cách viết mã và viết comment tệ hại. Sau đó Kent Beck đã refactor đoạn mã

này thành một dạng dễ chịu hơn nhiều, ngay trước mặt vài chục học viên đầy hào hứng. Về sau tôi chuyển thể ví dụ này cho cuốn sách Agile Software Development, Principles, Patterns, and Practices của mình và cho bài đầu tiên trong loạt bài Craftsman đăng trên tạp chí Software Development.

What I find fascinating about this module is that there was a time when many of us would have considered it “well documented.” Now we see it as a small mess. See how many different comment problems you can find.

Điều tôi thấy thú vị ở module này là đã từng có một thời mà nhiều người trong chúng ta hẳn sẽ coi nó là “được viết tài liệu tử tế”. Còn giờ ta nhìn nó như một mớ hỗn độn nho nhỏ. Thử xem bạn tìm ra được bao nhiêu vấn đề khác nhau về comment.

```
/**
 * This class Generates prime numbers up to a user specified
 * maximum. The algorithm used is the Sieve of Eratosthenes.
 * <p>
 * Eratosthenes of Cyrene, b. c. 276 BC, Cyrene, Libya --
 * d. c. 194, Alexandria. The first man to calculate the
 * circumference of the Earth. Also known for working on
 * calendars with leap years and ran the library at Alexandria.
 * <p>
 * The algorithm is quite simple. Given an array of integers
 * starting at 2. Cross out all multiples of 2. Find the next
 * uncrossed integer, and cross out all of its multiples.
 * Repeat until you have passed the square root of the maximum
 * value.
 *
 * @author Alphonse
 * @version 13 Feb 2002 atp
 */
import java.util.*;
public class GeneratePrimes
{
    /**
     * @param maxValue is the generation limit.
     */
    public static int[] generatePrimes(int maxValue)
    {
        if (maxValue >= 2) // the only valid case
        {
            // declarations
            int s = maxValue + 1; // size of array
            boolean[] f = new boolean[s];
            int i;
            // initialize array to true.
            for (i = 0; i < s; i++)
                f[i] = true;
            // get rid of known non-primes
            f[0] = f[1] = false;
            // sieve
```

```

int j;
for (i = 2; i < Math.sqrt(s) + 1; i++)
{
    if (f[i]) // if i is uncrossed, cross its multiples.
    {
        for (j = 2 * i; j < s; j += i)
            f[j] = false; // multiple is not prime
    }
}
// how many primes are there?
int count = 0;
for (i = 0; i < s; i++)
{
    if (f[i])
        count++; // bump count.
}
int[] primes = new int[count];
// move the primes into the result
for (i = 0, j = 0; i < s; i++)
{
    if (f[i]) // if prime
        primes[j++] = i;
}
return primes; // return the primes
}
else // maxValue < 2
    return new int[0]; // return null array if bad input.
}
}

```

In the following, you can see a refactored version of the same module. Note that the use of comments is significantly restrained. There are just two comments in the whole module. Both comments are explanatory in nature.

Dưới đây, bạn có thể thấy phiên bản đã refactor của chính module đó. Hãy để ý rằng việc dùng comment đã được tiết chế đáng kể. Cả module chỉ còn đúng hai comment. Cả hai đều mang tính giải thích.

```

/**
 * This class Generates prime numbers up to a user specified
 * maximum. The algorithm used is the Sieve of Eratosthenes.
 * Given an array of integers starting at 2:
 * Find the first uncrossed integer, and cross out all its
 * multiples. Repeat until there are no more multiples
 * in the array.
 */
public class PrimeGenerator
{
    private static boolean[] crossedOut;
    private static int[] result;

    public static int[] generatePrimes(int maxValue)

```

```

{
    if (maxValue < 2)
        return new int[0];
    else
    {
        uncrossIntegersUpTo(maxValue);
        crossOutMultiples();
        putUncrossedIntegersIntoResult();
        return result;
    }
}
private static void uncrossIntegersUpTo(int maxValue)
{
    crossedOut = new boolean[maxValue + 1];
    for (int i = 2; i < crossedOut.length; i++)
        crossedOut[i] = false;
}
private static void crossOutMultiples()
{
    int limit = determineIterationLimit();
    for (int i = 2; i <= limit; i++)
        if (notCrossed(i))
            crossOutMultiplesOf(i);
}
private static int determineIterationLimit()
{
    // Every multiple in the array has a prime factor that
    // is less than or equal to the root of the array size,
    // so we don't have to cross out multiples of numbers
    // larger than that root.
    double iterationLimit = Math.sqrt(crossedOut.length);
    return (int) iterationLimit;
}
private static void crossOutMultiplesOf(int i)
{
    for (int multiple = 2*i;
         multiple < crossedOut.length;
         multiple += i)
        crossedOut[multiple] = true;
}
private static boolean notCrossed(int i)
{
    return crossedOut[i] == false;
}
private static void putUncrossedIntegersIntoResult()
{
    result = new int[numberOfUncrossedIntegers()];
    for (int j = 0, i = 2; i < crossedOut.length; i++)
        if (notCrossed(i))
            result[j++] = i;
}

```



```

    }
    private static int numberOfUncrossedIntegers()
    {
        int count = 0;
        for (int i = 2; i < crossedOut.length; i++)
            if (notCrossed(i))
                count++;
        return count;
    }
}

```

It is easy to argue that the first comment is redundant because it reads very much like the `generatePrimes` function itself. Still, I think the comment serves to ease the reader into the algorithm, so I'm inclined to leave it.

Rất dễ lập luận rằng comment thứ nhất là thừa, bởi nó đọc lên chẳng khác mấy chính hàm `generatePrimes`. Dẫu vậy, tôi cho rằng comment ấy giúp dẫn dắt người đọc bước êm ái vào thuật toán, nên tôi nghiêng về việc giữ nó lại.

The second comment is almost certainly helpful. It explains the rationale behind the use of the square root as the loop limit. I could find no simple variable name, nor any different coding structure that made this point clear.

Comment thứ hai thì gần như chắc chắn là hữu ích. Nó giải thích lý do đằng sau việc lấy căn bậc hai làm giới hạn vòng lặp. Tôi không tìm được cái tên biến đơn giản nào, cũng chẳng có cấu trúc mã nào khác, làm rõ được điểm này.

Conclusion — Kết luận

Comments are not like Schindler's list—they are not “pure good.” They are sometimes utterly essential and other times dangerous and damaging. The only language of “truth” at our disposal is our code. Unfortunately, that language is not always adequate to explain the intention behind our decisions. In those cases, the risk of using an ambiguous, amorphous, and contradictory language like English (or whatever your native language is) can sometimes be worth it. But tread carefully—there dwell dragons.

Comment không giống danh sách của Schindler — chúng không phải là “thuần điều tốt”. Có lúc chúng tuyệt đối thiết yếu, có lúc lại nguy hiểm và gây hại. Thứ ngôn ngữ “chân lý” duy nhất mà ta có trong tay là mã của chính mình. Đáng tiếc, ngôn ngữ ấy không phải lúc nào cũng đủ sức giải thích ý định đằng sau các quyết định của ta. Trong những trường hợp đó, mạo hiểm dùng một thứ ngôn ngữ mơ hồ, không định hình và đầy mâu thuẫn như tiếng Anh (hay tiếng mẹ đẻ của bạn, dù đó là gì) đôi khi cũng đáng. Nhưng hãy bước cẩn thận — nơi đó có rồng ngụ.

Chapter 6. Formatting — Chương 6. Định dạng



I wrote this nearly two decades ago. The sample projects I used in my statistical analysis are well into their third, and approaching their fourth, decade. And yet there is little I needed to change in this chapter. I was tempted to add statistical studies for some other languages, but I decided that would not improve the message of this chapter. Other languages may result in slightly different numbers, but you, gentle reader, are smart enough to account for those minor differences.

Tôi viết chương này cách đây gần hai thập kỷ. Các dự án mẫu mà tôi dùng để phân tích thống kê giờ đã bước sâu vào thập kỷ thứ ba và sắp chạm thập kỷ thứ tư. Vậy mà chẳng có mấy thứ tôi cần sửa trong chương này. Tôi cũng đã định thêm vài khảo sát thống kê cho một số ngôn ngữ khác, nhưng rồi quyết định rằng làm vậy chẳng khiến thông điệp của chương hay hơn. Ngôn ngữ khác có thể cho ra con số hơi khác, nhưng bạn đọc thân mến à, bạn đủ thông minh để tự trừ hao mấy khác biệt nho nhỏ đó.

—Uncle Bob

—Uncle Bob

When people look under the hood, we want them to be impressed with the neatness, consistency, and

attention to detail that they perceive. We want them to be struck by the orderliness. We want their eyebrows to rise as they scroll through the modules. We want them to perceive that professionals have been at work. If instead they see a scrambled mass of code that looks like it was written by a bevy of drunken sailors, then they are likely to conclude that the same inattention to detail pervades every other aspect of the project.

Khi người ta vén nắp ca-pô lên nhìn vào bên trong, ta muốn họ phải trầm trồ vì sự gọn gàng, nhất quán và chín chu đến từng chi tiết mà họ thấy. Ta muốn họ sững người vì sự ngăn nắp. Ta muốn họ nhướn mày khi cuộn qua từng module. Ta muốn họ cảm nhận được rằng ở đây có những người chuyên nghiệp đã làm việc. Ngược lại, nếu thứ họ thấy là một mớ mã lộn xộn trông như do một đám thủy thủ say rượu viết ra, họ nhiều khả năng sẽ kết luận rằng cùng cái sự cẩu thả ấy đang thấm vào mọi khía cạnh khác của dự án.

You should take care that your code is nicely formatted. You should choose a set of simple rules that govern the format of your code, and then you should consistently apply those rules. If you are working on a team, then the team should agree to a single set of **formatting** rules and all members should comply. It helps to have an automated tool that can apply those formatting rules for you.

Bạn nên chăm chút để mã của mình được định dạng đẹp đẽ. Bạn nên chọn ra một bộ quy tắc đơn giản chi phối cách định dạng mã, rồi áp dụng những quy tắc đó một cách nhất quán. Nếu bạn làm việc trong một nhóm, cả nhóm phải thống nhất một bộ quy tắc định dạng duy nhất và mọi thành viên đều tuân theo. Có một công cụ tự động áp dụng giúp bạn những quy tắc ấy thì càng tốt.

The Purpose of Formatting — Mục đích của việc định dạng

Code formatting is important . It is too important to ignore and it is too important to treat religiously. Code formatting is about communication, and communication is the **professional** developer's first order of business.

Định dạng (formatting) mã là chuyện quan trọng. Quan trọng đến mức không thể xem nhẹ, và cũng quan trọng đến mức không nên biến nó thành tín hiệu. Định dạng mã là chuyện giao tiếp, mà giao tiếp lại là nhiệm vụ hàng đầu của một lập trình viên chuyên nghiệp.

Perhaps you thought that “getting it working” was the first order of business for a professional developer. I hope that this book has begun to disabuse you of that idea. The functionality that you create today has a **good** chance of changing in the next release, but the readability of your code will have a profound effect on all the changes that will ever be made. The coding style and readability set precedents that continue to affect maintainability and extensibility long after the original code has been changed beyond recognition. Your style and **discipline** survive, even though your code does not.

Có lẽ bạn từng nghĩ “làm cho nó chạy được” mới là nhiệm vụ hàng đầu của một lập trình viên chuyên nghiệp. Tôi hy vọng cuốn sách này đã bắt đầu gạt bỏ ý nghĩ đó khỏi đầu bạn. Chúc rằng bạn tạo ra hôm nay rất có khả năng sẽ đổi khác ở bản phát hành kế tiếp, nhưng tính dễ đọc của mã lại ảnh hưởng sâu sắc tới mọi thay đổi sẽ được thực hiện sau này. Phong cách viết mã và tính dễ đọc tạo ra những tiền lệ tiếp tục chi phối khả năng bảo trì và mở rộng, rất lâu sau khi mã gốc đã bị sửa tới mức không còn nhận ra. Phong cách và kỷ luật (discipline) của bạn sống sót, dù mã của bạn thì không.

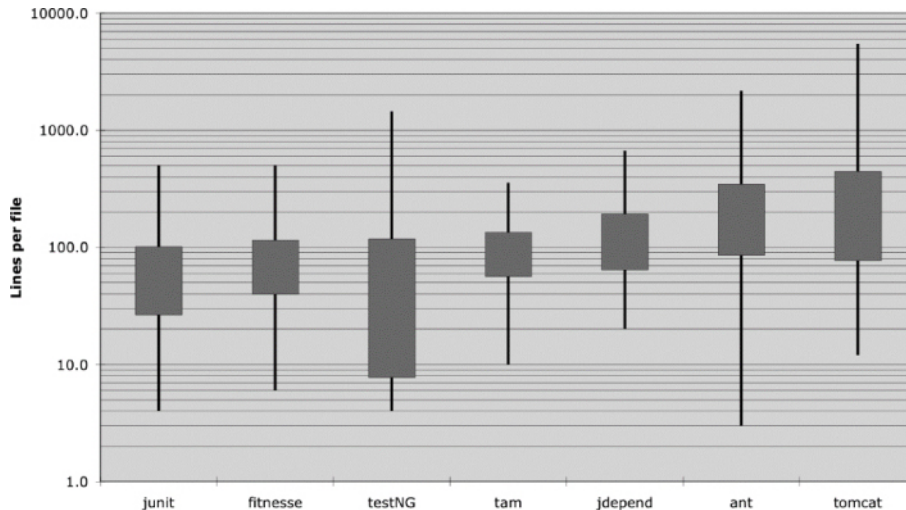
So, what are the formatting issues that help us communicate best?

Vậy những vấn đề định dạng nào giúp ta giao tiếp tốt nhất?

Vertical Formatting — Định dạng theo chiều dọc

Let's start with **vertical** size. How big should a source file be? It turns out that there is a huge range of sizes and some remarkable differences in style. The following shows some of those differences.

Hãy bắt đầu bằng kích thước theo chiều dọc. Một file mã nguồn nên lớn cỡ nào? Hoá ra khoảng dao động rất rộng và có những khác biệt phong cách khá đáng chú ý. Hình dưới đây cho thấy một phần các khác biệt đó.



Seven different projects are depicted: Junit, FitNesse, TestNG, Time and Money, JDepend, Ant, and Tomcat. The lines through the boxes show the minimum and maximum file lengths in each project. The box shows approximately one-third (one standard deviation 1) of the files. The middle of the box is the mean. So the average file size in the FitNesse project is about 65 lines, and about one-third of the files are between 40 and 100+ lines. The largest file in FitNesse is about 400 lines and the smallest is 6 lines. Note that this is a log scale, so the small difference in vertical position implies a very large difference in absolute size.

Có bảy dự án khác nhau được biểu diễn: Junit, FitNesse, TestNG, Time and Money, JDepend, Ant và Tomcat. Các đường xuyên qua hộp cho biết độ dài file nhỏ nhất và lớn nhất trong mỗi dự án. Hộp biểu thị xấp xỉ một phần ba (một độ lệch chuẩn 1) số file. Chính giữa hộp là giá trị trung bình. Vậy nên kích thước file trung bình trong dự án FitNesse là khoảng 65 dòng, và chừng một phần ba số file nằm trong khoảng 40 đến hơn 100 dòng. File lớn nhất trong FitNesse dài khoảng 400 dòng, file nhỏ nhất là 6 dòng. Lưu ý đây là thang logarit, nên chênh lệch nhỏ về vị trí theo chiều dọc thật ra hàm ý một chênh lệch rất lớn về kích thước tuyệt đối.

1 The box shows sigma/2 above and below the mean. Yes, I know that the file length distribution is not normal, and so the standard deviation is not mathematically precise. But we're not trying for precision here. We're just trying to get a feel.

1 Hộp biểu thị khoảng sigma/2 trên và dưới giá trị trung bình. Vâng, tôi biết phân phối độ dài file không phải phân phối chuẩn, nên độ lệch chuẩn ở đây không chính xác về mặt toán học. Nhưng ta đâu có đang cố đạt độ chính xác. Ta chỉ đang cố cảm nhận thôi.

Junit, FitNesse, and Time and Money are composed of relatively small files. None are over 500 lines and most of those files are less than 200 lines. Tomcat and Ant , on the other hand, have some files that are several thousand lines long, and close to half are over 200 lines.

Junit, FitNesse và Time and Money gồm toàn những file tương đối nhỏ. Không file nào quá 500 dòng và phần lớn dưới 200 dòng. Ngược lại, Tomcat và Ant có những file dài tới vài nghìn dòng, và gần một nửa số file vượt quá 200 dòng.

What does that mean to us? It appears to be possible to build significant systems (at the time of this analysis, FitNesse was close to 50,000 lines) out of files that are typically 200 lines long, with an upper limit of 500. Although this should not be a hard-and-fast rule, it should be considered very desirable. Small files are usually easier to understand and maintain than large files.

Điều đó có nghĩa gì với chúng ta? Rõ ràng là hoàn toàn có thể xây những hệ thống đáng kể (vào thời điểm phân tích này, FitNesse đã gần 50.000 dòng) từ những file thường chỉ dài 200 dòng, với giới hạn trên là 500. Dù đây không nên là một quy tắc cứng nhắc, nó vẫn rất đáng mong muốn. File nhỏ thường dễ hiểu và dễ bảo trì hơn file lớn.

Vertical Openness between Concepts — Khoảng thoáng theo chiều dọc giữa các khái niệm

Nearly all code is read left to right and top to bottom. Each line represents an expression or a clause, and each group of lines represents a complete thought. Those thoughts should be separated from each other with blank lines.

Gần như mọi đoạn mã đều được đọc từ trái sang phải và từ trên xuống dưới. Mỗi dòng biểu thị một biểu thức hoặc một mệnh đề, và mỗi nhóm dòng biểu thị một ý trọn vẹn. Những ý đó nên được tách khỏi nhau bằng các dòng trống.

Consider the following code. There are blank lines that separate the **package** declaration, the import(s), and each of the functions. This extremely simple rule has a profound effect on the visual layout of the code. Each blank line is a visual cue that identifies a new and separate concept. As you scan down the listing, your eye is drawn to the first line that follows a blank line.

Hãy xem đoạn mã dưới đây. Có những dòng trống ngăn cách phần khai báo package, các lệnh import và từng hàm. Quy tắc cực kỳ đơn giản này tác động sâu sắc tới bố cục thị giác của mã. Mỗi dòng trống là một tín hiệu thị giác báo hiệu một khái niệm mới và riêng biệt. Khi bạn lướt mắt xuống đoạn liệt kê, mắt bạn bị hút về dòng đầu tiên ngay sau một dòng trống.

```
package fitnessse.wikitext.widgets;
import java.util.regex.*;
public class BoldWidget extends ParentWidget {
    public static final String REGEXP = "''.+?'";
    private static final Pattern pattern = Pattern.compile("''.+?'")
        Pattern.MULTILINE + Pattern.DOTALL
);
public BoldWidget(ParentWidget parent, String text) throws Except
    super(parent);
    Matcher match = pattern.matcher(text);
    match.find();
    addChildWidgets(match.group(1));
}
public String render() throws Exception {
    StringBuffer html = new StringBuffer("<b>");
    html.append(childHtml()).append("</b>");
}
```

```

    return html.toString();
}
}

```

Taking those blank lines out has a remarkably obscuring effect on the readability of the code.

Bỏ những dòng trống ấy đi thì tính dễ đọc của mã lập tức bị che mờ đến mức đáng kinh ngạc.

```

package fitnesse.wikitext.widgets;
import java.util.regex.*;
public class BoldWidget extends ParentWidget {
    public static final String REGEXP = "'.+?'";
    private static final Pattern pattern = Pattern.compile("'(.+?)'
        Pattern.MULTILINE + Pattern.DOTALL);
    public BoldWidget(ParentWidget parent, String text) throws Except
        super(parent);
        Matcher match = pattern.matcher(text);
        match.find();

        addChildWidgets(match.group(1));}
    public String render() throws Exception {
        StringBuffer html = new StringBuffer("<b>");
        html.append(childHtml()).append("</b>");
        return html.toString();
    }
}

```

This effect is even more pronounced when you unfocus your eyes. In the first example, the different groupings of lines pop out at you, whereas the second example looks like a muddle. The difference between these two examples is just a bit of vertical openness.

Hiệu ứng này càng rõ hơn nếu bạn thả cho mắt mất tiêu cự. Ở ví dụ đầu, các cụm dòng khác nhau bật hẳn ra trước mắt bạn, còn ví dụ thứ hai chỉ là một mảng nhoè nhoẹt. Khác biệt giữa hai ví dụ này chỉ là một chút khoảng thoáng theo chiều dọc.

Vertical Density — Mật độ theo chiều dọc

If openness separates concepts, then vertical density implies close association. So lines of code that are tightly related should appear vertically dense. Notice how the useless comments in the code below break the close association of the two instance variables.

Nếu khoảng thoáng dùng để tách các khái niệm, thì mật độ theo chiều dọc lại hàm ý sự gắn bó chặt chẽ. Vì vậy những dòng mã liên quan mật thiết với nhau nên nằm sát nhau theo chiều dọc. Hãy để ý xem mấy comment vô dụng trong đoạn mã dưới đây đã phá vỡ mối liên hệ khăng khít giữa hai biến instance như thế nào.

```

public class ReporterConfig {
    /**
     * The class name of the reporter listener
     */
    private String m_className;
    /**
     * The properties of the reporter listener

```



```

*/
private List<Property> m_properties = new ArrayList<Property>();
private List<Property> m_properties = new ArrayList<Property>();
public void addProperty(Property property) {
    m_properties.add(property);
}
}

```

The code below is much easier to read. It fits in an “eye-full,” or at least it does for me. I can look at it and see that this is a **class** with two variables and a **method**, without having to move my head or eyes much. The previous listing forces me to use much more eye and head motion, not to mention the extra processing of the distracting comments, to achieve the same level of comprehension.

Đoạn mã dưới đây dễ đọc hơn hẳn. Nó gọn trong “một tầm mắt”, ít nhất là với tôi. Tôi chỉ cần nhìn vào là thấy đây là một class có hai biến và một method, mà chẳng phải đảo mắt hay ngoẹo đầu nhiều. Đoạn liệt kê trước đó buộc tôi phải đưa mắt và xoay đầu nhiều hơn hẳn, ấy là chưa kể phần xử lý thêm dành cho mấy comment gây xao nhãng, chỉ để đạt cùng một mức độ thấu hiểu.

```

public class ReporterConfig {
    private String m_className;
    private List<Property> m_properties = new ArrayList<Property>();
    public void addProperty(Property property) {
        m_properties.add(property);
    }
}

```

Vertical Distance — Khoảng cách theo chiều dọc

Have you ever chased your tail through a class, hopping from one **function** to the next, scrolling up and down the source file, trying to divine how the functions relate and operate, only to get lost in a rat’s nest of confusion? Have you ever hunted up the chain of **inheritance** for the definition of a **variable** or function? This is frustrating because you are trying to understand what the system does, but you are spending your time and mental energy on trying to locate and remember where the pieces are.

Đã bao giờ bạn đuổi theo cái đuôi của chính mình xuyên suốt một class, nhảy từ hàm này sang hàm khác, cuộn lên cuộn xuống file mã nguồn, cố đoán xem các hàm liên hệ và vận hành ra sao, để rồi lạc trong một mê cung rối như tổ chuột? Đã bao giờ bạn phải truy ngược cả chuỗi kế thừa (inheritance) chỉ để tìm định nghĩa của một biến hay một hàm? Chuyện đó rất bực bội, bởi bạn đang cố hiểu hệ thống làm gì, nhưng lại tiêu tốn thời gian và sức lực trí óc vào việc định vị và ghi nhớ xem các mảnh ghép nằm ở đâu.

Concepts that are closely related should be kept vertically close to each other. Clearly this rule doesn’t work for concepts that belong in separate files. But then closely related concepts should not be separated into different files unless you have a very good reason. Indeed, this is one of the reasons that protected variables should be avoided.

Những khái niệm liên quan mật thiết với nhau nên được đặt gần nhau theo chiều dọc. Hiển nhiên quy tắc này không áp dụng được cho các khái niệm vốn thuộc về những file khác nhau. Nhưng nói vậy thì các khái niệm liên quan mật thiết cũng không nên bị tách

sang những file khác nhau trừ khi bạn có lý do rất chính đáng. Thật vậy, đây là một trong những lý do nên tránh dùng biến `protected`.

For those concepts that are so closely related that they belong in the same source file, their vertical separation should be a measure of how important each is to the understandability of the other. We want to avoid forcing our readers to hop around through our source files and classes.

Với những khái niệm gắn bó chặt đến mức thuộc về cùng một file mã nguồn, khoảng cách theo chiều dọc giữa chúng nên phản ánh mức độ quan trọng của khái niệm này đối với việc hiểu khái niệm kia. Ta muốn tránh việc bắt người đọc phải nhảy cóc khắp các file mã nguồn và các class của mình.

Variable Declarations — Khai báo biến

Variables should be declared as close to their usage as possible. Because our functions are very short, local variables should appear at the top of each function, as in this longish function from JUnit 4.3.1.

Biến nên được khai báo càng gần nơi sử dụng càng tốt. Vì các hàm của ta rất ngắn, biến cục bộ nên xuất hiện ở đầu mỗi hàm, như trong hàm hơi dài dưới đây lấy từ JUnit 4.3.1.

```
private static void readPreferences() {
    InputStream is= null;
    try {
        is= new FileInputStream(getPreferencesFile());
        setPreferences(new Properties(getPreferences()));
        getPreferences().load(is);
    } catch (IOException e) {
        try {
            if (is != null)
                is.close();
        } catch (IOException e1) {}
    }
}
```

Control variables for loops should usually be declared within the loop statement, as in this cute little function from the same source.

Biến điều khiển vòng lặp thường nên được khai báo ngay trong câu lệnh lặp, như trong hàm nhỏ xinh dưới đây lấy từ cùng nguồn.

```
public int countTestCases() {
    int count= 0;
    for (Test each : tests)
        count += each.countTestCases();
    return count;
}
```

In rare cases, a variable might be declared at the top of a block or just before a loop in a long-ish function. You can see such a variable in this snippet from the midst of a very long function in TestNG.

Trong vài trường hợp hiếm hoi, một biến có thể được khai báo ở đầu một khối hoặc ngay trước một vòng lặp trong một hàm hơi dài. Bạn có thể thấy một biến như vậy trong đoạn trích từ giữa một hàm rất dài trong TestNG.


```

...
for (XmlTest test : m_suite.getTests()) {
    TestRunner tr = m_runnerFactory.newTestRunner(this, test);
    tr.addListener(m_textReporter);
    m_testRunners.add(tr);
    invoker = tr.getInvoker();
for (ITestNGMethod m : tr.getBeforeSuiteMethods()) {
    for (ITestNGMethod m : tr.getBeforeSuiteMethods()) {
        beforeSuiteMethods.put(m.getMethod(), m);
    }
    for (ITestNGMethod m : tr.getAfterSuiteMethods()) {
        afterSuiteMethods.put(m.getMethod(), m);
    }
}
}
...

```

Instance variables, on the other hand, should be declared at either the top or the bottom of a class. This should not increase the vertical distance of these variables, because in a well-designed class, they are used by many, if not all, of the methods of the class.

Ngược lại, biến instance nên được khai báo ở đầu hoặc ở cuối class. Điều này không làm tăng khoảng cách theo chiều dọc của những biến ấy, bởi trong một class được thiết kế tốt, chúng được dùng bởi nhiều method, nếu không muốn nói là tất cả các method của class.

There have been many debates over where instance variables should go. In C++, we commonly practiced the so-called scissors rule, which put all the instance variables at the bottom. The rationale for this rule is that public items should go at the top and private items should go at the bottom.

Đã có vô số cuộc tranh luận về chỗ đứng của biến instance. Trong C++, chúng tôi thường theo cái gọi là quy tắc cây kéo (scissors rule), tức là đặt toàn bộ biến instance ở dưới cùng. Lý lẽ cho quy tắc này là những thứ public nên nằm trên, còn những thứ private nên nằm dưới.

This makes a lot more sense than the Java/C# convention of putting private variables at the top. But since this convention is so pervasive in these languages, it is likely better to follow it than to confuse everyone by appealing to logic and common sense.

Cách đó hợp lý hơn nhiều so với thông lệ của Java/C# là nhét biến private lên trên cùng. Nhưng vì thông lệ ấy quá phổ biến trong các ngôn ngữ này, có lẽ tốt hơn là cứ theo nó, thay vì làm mọi người hoang mang bằng cách viện đến logic và lẽ thường.

The important thing is for the instance variables to be declared in one well-known place. Everybody should know where to go to see the declarations.

Điều quan trọng là biến instance phải được khai báo tại một chỗ ai cũng biết. Mọi người đều phải biết cần tìm đến đâu để xem các khai báo.

Consider, for example, the strange case of the TestSuite class in JUnit 4.3.1. I have greatly attenuated this class to make the point. If you look about halfway down the listing, you will see two instance variables declared there. It would be hard to hide them in a better place. Someone reading this code would have to stumble across the declarations by accident (as I did).

Chẳng hạn, hãy xem trường hợp kỳ quặc của class TestSuite trong JUnit 4.3.1. Tôi đã lược bớt rất nhiều để làm rõ ý. Nếu bạn nhìn vào khoảng giữa đoạn liệt kê, bạn sẽ thấy hai biến

instance được khai báo ở đó. Khó mà giấu chúng ở chỗ nào kín đáo hơn. Người đọc đoạn mã này chỉ có thể vấp phải mấy khai báo đó một cách tình cờ (như tôi đã vấp).

```
public class TestSuite implements Test {
    static public Test createTest(Class<? extends TestCase> theCla
                                String name) {

        ...
    }
    public static Constructor<? extends TestCase>
    getTestConstructor(Class<? extends TestCase> theClass)
    throws NoSuchMethodException {
        ...
    }
    public static Test warning(final String message) {
        ...
    }
    private static String exceptionToString(Throwable t) {
        ...
    }
    private String fName;
    private Vector<Test> fTests= new Vector<Test>(10);
    public TestSuite() {
    }
    public TestSuite(final Class<? extends TestCase> theClass) {
        ...
    }
    public TestSuite(Class<? extends TestCase> theClass, String
        ...
    }
    ... ..
}
```

Dependent Functions — Các hàm phụ thuộc nhau

If one function calls another, they should be vertically close, and the caller should be above 2 the callee, if at all possible. This gives the program a natural flow. If the convention is followed reliably, readers will be able to trust that function definitions will follow shortly after their use. Consider, for example, the snippet from FitNesse below. Notice how the topmost function calls (in bold) those below it. Notice also that the called functions appear in the order that they were called. This makes it easy to find the called functions and greatly enhances the readability of the whole **module**.

Nếu một hàm gọi một hàm khác, chúng nên nằm gần nhau theo chiều dọc, và hàm gọi nên nằm phía trên 2 hàm được gọi, nếu có thể. Điều này tạo cho chương trình một dòng chảy tự nhiên. Nếu thông lệ này được tuân thủ nhất quán, người đọc sẽ tin được rằng định nghĩa của hàm sẽ xuất hiện ngay sau chỗ nó được dùng. Chẳng hạn, hãy xem đoạn trích từ FitNesse dưới đây. Hãy để ý hàm trên cùng gọi (in đậm) những hàm nằm dưới nó. Cũng để ý rằng các hàm được gọi xuất hiện đúng theo thứ tự chúng được gọi. Điều này khiến việc tìm hàm được gọi trở nên dễ dàng và nâng cao đáng kể tính dễ đọc của cả module.

2 Or below in inverted languages like Clojure.

2 Hoặc phía dưới, trong những ngôn ngữ đảo ngược như Clojure.

```
public class WikiPageResponder implements SecureResponder {
    protected WikiPage page;
    protected PageData pageData;
    protected String pageTitle;

    protected Request request;
    protected PageCrawler crawler;
    public Response makeResponse(FitNesseContext context, Request req
        throws Exception {
        String pageName = getPageNameOrDefault(request, "FrontPage");
        loadPage(pageName, context);
        if (page == null)
            return notFoundResponse(context, request);
        else
            return makePageResponse(context);
    }
    private String
    getPageNameOrDefault(Request request, String defaultPageName)
    {
        String pageName = request.getResource();
        if (StringUtil.isBlank(pageName))
            pageName = defaultPageName;
        return pageName;
    }
    protected void loadPage(String resource, FitNesseContext context
        throws Exception {
        WikiPagePath path = PathParser.parse(resource);
        crawler = context.root.getPageCrawler();
        crawler.setDeadEndStrategy(new VirtualEnabledPageCrawler());
        page = crawler.getPage(context.root, path);

        if (page != null)
            pageData = page.getData();
    }
    private Response
    notFoundResponse(FitNesseContext context, Request request)
        throws Exception {
        return new NotFoundResponder().makeResponse(context, request);
    }
    private SimpleResponse makePageResponse(FitNesseContext context)
        throws Exception {
        pageTitle = PathParser.render(crawler.getFullPath(page));
        String html = makeHtml(context);
        SimpleResponse response = new SimpleResponse();
        response.setMaxAge(0);
        response.setContent(html);
        return response;
    }
    ...
}
```

As an aside, this snippet provides a nice example of keeping constants at the appropriate level. The “FrontPage” constant could have been buried in the `getPageNameOrDefault` function, but that would have hidden a well-known and expected constant in an inappropriately low-level function. It was better to pass that constant down from where knowing it makes sense.

Nhân tiện, đoạn trích này là một ví dụ hay về việc giữ hằng số ở đúng cấp độ. Hằng “FrontPage” lẽ ra có thể bị chôn trong hàm `getPageNameOrDefault`, nhưng làm vậy là giấu một hằng số ai cũng biết và ai cũng chờ đợi vào một hàm ở mức quá thấp, không phù hợp. Tốt hơn là truyền hằng số đó xuống từ nơi mà việc biết đến nó là hợp lý.

Conceptual Affinity — Sự tương cận về khái niệm

Certain bits of code want to be near other bits. They have a certain conceptual affinity. The stronger that affinity is, the less vertical distance there should be between them.

Có những mẫu mã muốn được nằm gần những mẫu mã khác. Chúng có một sự tương cận về khái niệm (conceptual affinity). Sự tương cận đó càng mạnh thì khoảng cách theo chiều dọc giữa chúng càng nên nhỏ.

As we have seen, this affinity might be based on a direct dependence, such as one function calling another, or a function using a variable. But there are other possible causes of affinity. Affinity might be caused because a group of functions perform a similar operation. Consider this snippet of code from JUnit 4.3.1:

Như ta đã thấy, sự tương cận này có thể xuất phát từ một phụ thuộc (dependency) trực tiếp, chẳng hạn hàm này gọi hàm kia, hay một hàm dùng một biến. Nhưng còn những nguyên nhân khác nữa. Sự tương cận có thể nảy sinh vì một nhóm hàm cùng thực hiện một thao tác tương tự nhau. Hãy xem đoạn mã sau từ JUnit 4.3.1:

```
public class Assert {
    static public void assertTrue(String message, boolean condition) {
        if (!condition)
            fail(message);
    }
    static public void assertTrue(boolean condition) {
        assertTrue(null, condition);
    }
    static public void assertFalse(String message, boolean condition) {
        assertTrue(message, !condition);
    }
    static public void assertFalse(boolean condition) {
        assertFalse(null, condition);
    }
    ...
}
```

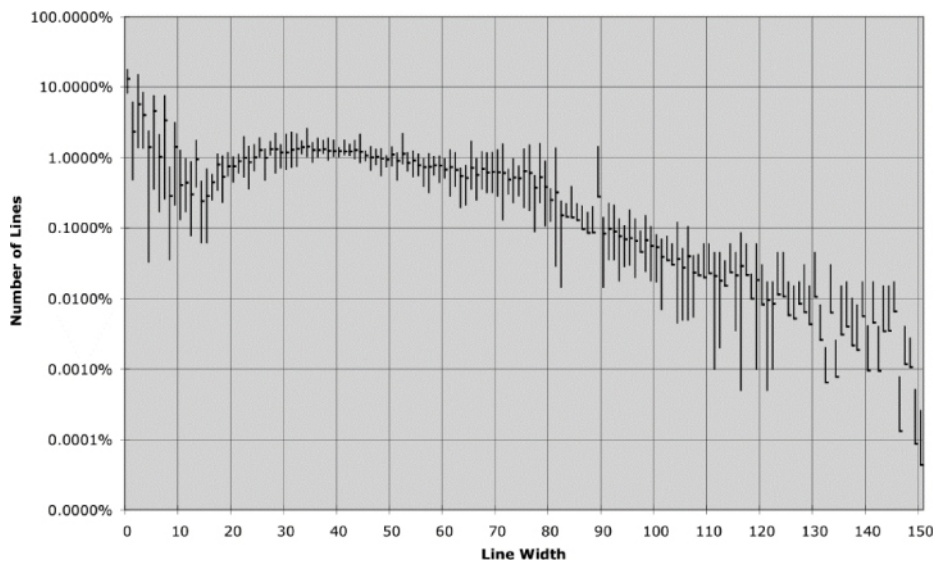
These functions have a strong conceptual affinity because they share a common naming scheme and perform variations of the same basic task. The fact that they call each other is secondary. Even if they didn't, they would still want to be close together.

Những hàm này có sự tương cận về khái niệm rất mạnh, bởi chúng dùng chung một lối đặt tên và cùng thực hiện các biến thể của cùng một tác vụ cơ bản. Việc chúng gọi lẫn nhau chỉ là thứ yếu. Ngay cả khi không gọi nhau, chúng vẫn muốn được nằm cạnh nhau.

Horizontal Formatting — Định dạng theo chiều ngang

How wide should a line be? To answer that, let's look at how wide lines are in typical programs. Again, we examine the seven different projects. The following image shows the distribution of line lengths of all seven projects. The regularity is impressive, especially right around 45 characters. Indeed, every size from 20 to 60 represents about 1% of the total number of lines. That's 40%! Perhaps another 30% are less than ten characters wide. Remember, this is a log scale, so the linear appearance of the drop-off above 80 characters is really very significant. Programmers clearly prefer short lines.

Một dòng nên rộng cỡ nào? Để trả lời, hãy xem các dòng trong những chương trình điển hình rộng bao nhiêu. Một lần nữa, ta khảo sát bảy dự án khác nhau. Hình dưới đây cho thấy phân bố độ dài dòng của cả bảy dự án. Mức độ đều đặn thật ấn tượng, nhất là quanh mốc 45 ký tự. Thật vậy, mỗi kích thước từ 20 đến 60 chiếm khoảng 1% tổng số dòng. Vị chỉ 40%! Có lẽ thêm 30% nữa là những dòng ngắn dưới mười ký tự. Nhớ rằng đây là thang logarit, nên dáng tụt xuống gần như tuyến tính ở phía trên mốc 80 ký tự thật ra rất có ý nghĩa. Rõ ràng lập trình viên thích những dòng ngắn.



This suggests that we should strive to keep our lines short. The old Hollerith limit of 80 is a bit arbitrary, and I'm not opposed to lines edging out to 100 or even 120. But beyond that is probably just careless.

Điều này gợi ý rằng ta nên cố giữ cho các dòng của mình ngắn. Giới hạn Hollerith cũ kỹ ở mốc 80 có phần tùy tiện, và tôi không phản đối việc để dòng nhích lên 100 hay thậm chí 120. Nhưng vượt quá thế thì có lẽ chỉ là cầu thả.

I follow the rule that you should never force your readers to scroll to the right. Given the size and resolution of modern displays I personally set my horizontal limit at 120. I have my IDE draw a soft gray line at that column, and I won't go beyond it.

Tôi theo quy tắc: đừng bao giờ bắt người đọc phải cuộn ngang sang phải. Với kích thước và độ phân giải của màn hình hiện đại, cá nhân tôi đặt giới hạn ngang ở mức 120. Tôi cho IDE vẽ một đường xám mờ ở cột đó, và tôi không vượt qua nó.

Horizontal Openness and Density — Khoảng thoáng và mật độ theo chiều ngang

We use horizontal white space to associate things that are strongly related and disassociate things that are more weakly related. Consider the following function:

Ta dùng khoảng trắng theo chiều ngang để gắn kết những thứ liên quan chặt chẽ và tách rời những thứ liên quan lỏng lẻo hơn. Hãy xem hàm sau:

```
private void measureLine(String line) {  
    lineCount++;  
    int lineSize = line.length();  
    totalChars += lineSize;  
    lineWidthHistogram.addLine(lineSize, lineCount);  
    recordWidestLine(lineSize);  
}
```

I surrounded the assignment operators with white space to accentuate them. Assignment statements have two distinct and major elements: the left side and the right side. The spaces make that separation obvious.

Tôi bao quanh các toán tử gán bằng khoảng trắng để làm nổi bật chúng. Câu lệnh gán có hai thành phần chính và tách biệt: vế trái và vế phải. Khoảng trắng làm cho sự tách biệt đó trở nên rõ ràng.

On the other hand, I didn't put spaces between the function names and the opening parenthesis. This is because the function and its arguments are closely related. Separating them makes them appear disjointed instead of conjoined. I separate arguments within the function call parenthesis to accentuate the comma and show that the arguments are separate.

Mặt khác, tôi không đặt khoảng trắng giữa tên hàm và dấu ngoặc mở. Đó là vì hàm và các đối số của nó gắn bó chặt chẽ với nhau. Tách chúng ra sẽ khiến chúng trông rời rạc thay vì liền mạch. Tôi có tách các đối số bên trong cặp ngoặc lời gọi hàm để làm nổi bật dấu phẩy và cho thấy các đối số là riêng biệt.

Another use for white space is to accentuate the precedence of operators.

Một công dụng khác của khoảng trắng là làm nổi bật độ ưu tiên của toán tử.

```
public class Quadratic {  
    public static double root1(double a, double b, double c) {  
        double determinant = determinant(a, b, c);  
        return (-b + Math.sqrt(determinant)) / (2*a);  
    }  
    public static double root2(int a, int b, int c) {  
        double determinant = determinant(a, b, c);  
        return (-b - Math.sqrt(determinant)) / (2*a);  
    }  
    private static double determinant(double a, double b, double c)  
    {  
        return b*b - 4*a*c;  
    }  
}
```

Notice how nicely the equations read. The factors have no white space between them, because they are high precedence. The terms are separated by white space because addition and subtraction are lower precedence.

Hãy để ý mấy công thức này đọc lên mới thuận làm sao. Các thừa số không có khoảng trắng chen giữa, bởi chúng có độ ưu tiên cao. Các số hạng thì được tách bằng khoảng trắng vì phép cộng và phép trừ có độ ưu tiên thấp hơn.

Unfortunately, most tools for reformatting code are blind to the precedence of operators and impose the same spacing throughout. So subtle spacings like those shown above tend to get lost after you reformat the code.

Đáng tiếc là phần lớn công cụ định dạng lại mã đều mù tịt trước độ ưu tiên của toán tử và áp dụng một cách đặt khoảng trắng cho tất cả. Nên những khoảng trắng tinh tế như trên thường mất sạch sau khi bạn định dạng lại mã.

Horizontal Alignment — Căn chỉnh theo chiều ngang

When I was an assembly language programmer, I used horizontal alignment to accentuate certain structures. When I started coding in C, C++, Java, and eventually Clojure, I continued to try to line up all the variable names in a set of declarations, or all the r-values in a set of assignment statements. My code might have looked like this:

Hồi còn là lập trình viên hợp ngữ, tôi hay dùng căn chỉnh theo chiều ngang để làm nổi bật một số cấu trúc. Khi bắt đầu viết C, C++, Java và cuối cùng là Clojure, tôi vẫn tiếp tục cố xếp thẳng hàng toàn bộ tên biến trong một cụm khai báo, hoặc toàn bộ vế phải trong một cụm câu lệnh gán. Mã của tôi có thể trông như thế này:

3 Who am I kidding? I still am an assembly language programmer. You can take the boy away from the metal, but you can't take the metal out of the boy!

3 Mà tôi đùa ai chứ? Tôi vẫn đang là lập trình viên hợp ngữ đấy. Bạn có thể kéo thẳng nhóc ra khỏi đồng sắt vụn, nhưng không thể lôi đồng sắt vụn ra khỏi thẳng nhóc được đâu!

```
public class FitNesseExpediter implements ResponseSender
{
    private Socket      socket;
    private InputStream input;
    private OutputStream output;

    private Request     request;
    private Response    response;
    private FitNesseContext context;
    protected long      requestParsingTimeLimit;
    private long        requestProgress;
    private long        requestParsingDeadline;
    private boolean     hasError;
    public FitNesseExpediter(Socket s,
                             FitNesseContext context) throws Exce
    {
        this.context = context;
        socket = s;
        input = s.getInputStream();
        output = s.getOutputStream();
    }
}
```



```

    requestParsingTimeLimit = 10000;
}

```

I have found, however, that this kind of alignment is not useful. The alignment seems to emphasize the wrong things and leads my eye away from the true intent. For example, in the list of declarations above, you are tempted to read down the list of variable names without looking at their types. Likewise, in the list of assignment statements, you are tempted to look down the list of r-values without ever seeing the assignment operator.

Tuy nhiên, tôi nhận ra kiểu căn chỉnh này chẳng ích gì. Nó có vẻ nhấn mạnh nhầm thứ và kéo mắt tôi đi xa khỏi ý định thật sự. Chẳng hạn, trong danh sách khai báo phía trên, bạn dễ bị cám dỗ đọc dọc theo cột tên biến mà chẳng buồn nhìn kiểu của chúng. Tương tự, trong danh sách các câu lệnh gán, bạn dễ bị cám dỗ nhìn dọc theo cột vế phải mà không hề để mắt tới toán tử gán.

So, in the end, I don't do this kind of thing anymore. Nowadays, I usually prefer unaligned declarations and assignments, as shown below, because they point out an important deficiency. If I have long lists that need to be aligned, the problem is the length of the lists, not the lack of alignment. The length of the list of declarations in

Nên rất cuộc, tôi không còn làm kiểu đó nữa. Ngày nay tôi thường thích để các khai báo và câu lệnh gán không căn chỉnh, như dưới đây, bởi chúng chỉ ra một khiếm khuyết quan trọng. Nếu tôi có những danh sách dài cần phải căn chỉnh, thì vấn đề nằm ở độ dài của danh sách, chứ không phải ở chuyện thiếu căn chỉnh. Độ dài của danh sách khai báo trong

FitNesseExpediter below suggests that this class should be split up.

FitNesseExpediter dưới đây gợi ý rằng class này nên được tách nhỏ ra.

```

public class FitNesseExpediter implements ResponseSender
{
    private Socket socket;
    private InputStream input;
    private OutputStream output;
    private Request request;
    private Response response;
    private FitNesseContext context;
    protected long requestParsingTimeLimit;
    private long requestProgress;
    private long requestParsingDeadline;
    private boolean hasError;
    public FitNesseExpediter
    (Socket s, FitNesseContext context) throws Exception
    {
        this.context = context;
        socket = s;
        input = s.getInputStream();
        output = s.getOutputStream();
        requestParsingTimeLimit = 10000;
    }
}

```

Indentation — Thụt lề

A source file is a hierarchy rather like an outline. There is information that pertains to the file as a whole, to the individual classes within the file, to the methods within the classes, to the blocks within the methods, and recursively to the blocks within the blocks. Each level of this hierarchy is a **scope** into which names can be declared and in which declarations and executable statements are interpreted.

Một file mã nguồn là một cấu trúc phân cấp, khá giống một dàn ý. Có thông tin thuộc về cả file, có thông tin thuộc về từng class trong file, thuộc về các method trong class, thuộc về các khối trong method, và cứ thế đệ quy xuống các khối bên trong khối. Mỗi cấp của cấu trúc phân cấp này là một phạm vi (scope), nơi các tên có thể được khai báo và nơi các khai báo cùng câu lệnh thực thi được diễn giải.

To make this hierarchy of scopes visible, we indent the lines of source code in proportion to their position in the hierarchy. Statements at the level of the file, such as most class declarations, are not indented at all. Methods within a class are indented one level to the right of the class. Implementations of those methods are implemented one level to the right of the method declaration. Block implementations are implemented one level to the right of their containing block, and so on.

Để cấu trúc phân cấp các phạm vi này hiện ra rõ ràng, ta thụt lề (indentation) các dòng mã tương ứng với vị trí của chúng trong cấu trúc. Câu lệnh ở cấp file, chẳng hạn phần lớn khai báo class, không thụt lề chút nào. Method bên trong một class thì thụt vào một cấp so với class. Phần hiện thực (implementation) của các method đó lại thụt vào một cấp so với khai báo method. Phần hiện thực của một khối thì thụt vào một cấp so với khối chứa nó, và cứ thế.

Programmers rely heavily on this **indentation** scheme. They visually line up the left edge to see what scope each line appears in. This allows them to quickly hop over scopes, such as implementations of if or while statements, that are not relevant to their current situation. They scan the left for new method declarations, new variables, and even new classes. Without indentation, programs would be virtually unreadable by humans.

Lập trình viên phụ thuộc rất nhiều vào lối thụt lề này. Họ đóng mắt theo mép trái để xem mỗi dòng thuộc phạm vi nào. Nhờ đó họ nhảy nhanh qua những phạm vi không liên quan tới việc mình đang làm, chẳng hạn phần thân của lệnh if hay while. Họ quét dọc bên trái để tìm khai báo method mới, biến mới, thậm chí class mới. Không có thụt lề, chương trình gần như không thể đọc nổi đối với con người.

Consider the following programs that are syntactically and semantically identical:

Hãy xem những chương trình dưới đây, chúng giống hệt nhau cả về cú pháp lẫn ngữ nghĩa:

```
public class FitNesseServer implements SocketServer
{ private FitNesseContext context;
public FitNesseServer(FitNesseContext context)
{ this.context = context; } public void
serve(Socket s) { serve(s, 10000); }
public void serve(Socket s, long requestTimeout) { try
{ FitNesseExpediter sender = new FitNesseExpediter(s, context); se
setRequestParsingTimeLimit(requestTimeout);
sender.start(); } catch(Exception e)
{ e.printStackTrace(); } } }
```

```

public class FitNesseServer implements SocketServer {
    private FitNesseContext context;
    public FitNesseServer(FitNesseContext context) {
        this.context = context;
    }
    public void serve(Socket s) {
        serve(s, 10000);
    }
    public void serve(Socket s, long requestTimeout) {
        try {
            FitNesseExpediter sender = new FitNesseExpediter(s, context)

            sender.setRequestParsingTimeLimit(requestTimeout);
            sender.start();
        }
        catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

Your eye can rapidly discern the **structure** of the indented file. You can almost instantly spot the variables, constructors, accessors, and methods. It takes just a few seconds to realize that this is some kind of simple front end to a socket, with a timeout. The unindented version, however, is virtually impenetrable without intense study.

Mắt bạn nhận ra cấu trúc của file có thật là gần như tức thì. Bạn gần như lập tức chỉ ra được đâu là biến, constructor, accessor và method. Chỉ mất vài giây là bạn hiểu ra đây là một lớp vỏ đơn giản bọc quanh một socket, có kèm timeout. Còn phiên bản không thật là thì gần như bất khả xâm nhập nếu không nghiên ngẫm cật lực.

Breaking Indentation — Phá vỡ quy tắc thật là

It is sometimes tempting to break the indentation rule for short if statements, short while loops, or short functions. Whenever I have succumbed to this temptation, I have almost always gone back and put the indentation back in. So I usually avoid collapsing scopes down to one line, like this:

Đôi khi ta bị cám dỗ phá vỡ quy tắc thật là với những câu lệnh if ngắn, vòng lặp while ngắn hay hàm ngắn. Mỗi lần đầu hàng trước cám dỗ đó, gần như lần nào tôi cũng quay lại và đặt phần thật là về chỗ cũ. Nên tôi thường tránh dồn cả một phạm vi xuống một dòng, kiểu như thế này:

```

public class CommentWidget extends TextWidget {
    public static final String REGEXP = "^#[^\\r\\n]*(?:(?:\\r\\n)|\\n|\\r
    public CommentWidget(ParentWidget parent, String text)
    {super(parent, text);}

    public String render() throws Exception {return ""; }
}

```

I prefer to expand and indent the scopes instead, like this:

Tôi thích trải rộng và thật là các phạm vi ra hơn, như thế này:

```

public class CommentWidget extends TextWidget {
    public static final String REGEXP = "^#[^\\r\\n]*(?:(?:\\r\\n)|\\n|\\r
    public CommentWidget(ParentWidget parent, String text) {
        super(parent, text);
    }
    public String render() throws Exception {
        return "";
    }
}

```

There are times, however, when a simple one-liner is more visually appealing than dangling braces.

Tuy vậy, cũng có lúc một dòng đơn giản gọn lỏn lại bắt mắt hơn là mấy cặp ngoặc nhọn lủng lẳng.

```

public int getCount() {return count;}

```

Team Rules — Luật của nhóm

The title of this section is a play on words. Every programmer has their own favorite formatting rules, but if they work in a team, then the team rules.

Tựa đề mục này là một lối chơi chữ. Lập trình viên nào cũng có bộ quy tắc định dạng ruột của riêng mình, nhưng nếu họ làm việc trong một nhóm, thì nhóm mới là bên đặt luật.

A team of developers should agree upon a single formatting style, and then every member of that team should use that style. We want the software to have a consistent style. We don't want it to appear to have been written by a bunch of disagreeing individuals.

Một nhóm lập trình viên nên thống nhất một phong cách định dạng duy nhất, rồi mọi thành viên trong nhóm đều dùng phong cách đó. Ta muốn phần mềm có một phong cách nhất quán. Ta không muốn nó trông như được viết bởi một đám cá nhân bất đồng với nhau.

When I started the FitNesse project back in 2002, I sat down with the team to work out a coding style. This took about ten minutes. We decided where we'd put our braces, what our indent size would be, how we would name classes, variables, and methods, and so forth. Then, we encoded those rules into the code formatter of our IDE and have stuck with them ever since. These were not the rules that I prefer; they were rules decided by the team. As a member of that team, I followed them when writing code in the FitNesse project.

Khi khởi động dự án FitNesse hồi năm 2002, tôi đã ngồi lại với cả nhóm để thống nhất một phong cách viết mã. Việc đó mất khoảng mười phút. Chúng tôi quyết định sẽ đặt dấu ngoặc nhọn ở đâu, độ thụt lề bao nhiêu, đặt tên class, biến và method thế nào, vân vân. Rồi chúng tôi mã hoá những quy tắc ấy vào bộ định dạng mã của IDE và bám theo chúng từ đó tới nay. Đây không phải những quy tắc tôi ưa thích; đây là quy tắc do cả nhóm quyết. Với tư cách một thành viên của nhóm, tôi tuân theo chúng khi viết mã trong dự án FitNesse.

Remember, the source code of a good software system is a set of documents that read nicely. They need to have a consistent and smooth style. The reader needs to be able to trust that the formatting gestures they have seen in one source file will mean the same thing in others. The last thing we want to do is add more complexity to the source code by writing it in a jumble of different individual styles.

Nhớ rằng, mã nguồn của một hệ thống phần mềm tốt là một tập tài liệu đọc lên trôi chảy. Chúng cần có một phong cách nhất quán và mượt mà. Người đọc cần tin được rằng những

cử chỉ định dạng mà họ đã thấy trong file mã nguồn này cũng mang đúng ý nghĩa đó ở những file khác. Thứ cuối cùng ta muốn làm là chất thêm độ phức tạp vào mã nguồn bằng cách viết nó theo một mô phỏng cách cá nhân hồ lớn.

Uncle Bob's Formatting Rules — Các quy tắc định dạng của Uncle Bob

The rules I use personally are very simple and are illustrated by the code below. Consider this an example of how code makes the best coding standard document.

Bộ quy tắc tôi dùng cho cá nhân mình rất đơn giản và được minh họa bằng đoạn mã dưới đây. Hãy xem đây là một ví dụ cho thấy chính mã nguồn mới là tài liệu chuẩn viết mã tốt nhất.

```
public class CodeAnalyzer implements JavaFileAnalysis {
    private int lineCount;
    private int maxLineWidth;
    private int widestLineNumber;
    private LineWidthHistogram lineWidthHistogram;
    private int totalChars;

    public CodeAnalyzer() {
        lineWidthHistogram = new LineWidthHistogram();
    }

    public static List<File> findJavaFiles(File parentDirectory) {
        List<File> files = new ArrayList<File>();
        findJavaFiles(parentDirectory, files);
        return files;
    }

    private static void
    findJavaFiles(File parentDirectory, List<File> files) {
        for (File file : parentDirectory.listFiles()) {
            if (file.getName().endsWith(".java"))
                files.add(file);
            else if (file.isDirectory())
                findJavaFiles(file, files);
        }
    }

    public void analyzeFile(File javaFile) throws Exception {
        BufferedReader br = new BufferedReader(new FileReader(javaFile));
        String line;
        while ((line = br.readLine()) != null)
            measureLine(line);
    }

    private void measureLine(String line) {
        lineCount++;
        int lineSize = line.length();
        totalChars += lineSize;
        lineWidthHistogram.addLine(lineSize, lineCount);
        recordWidestLine(lineSize);
    }
}
```

```

    }
    private void recordWidestLine(int lineSize) {
        if (lineSize > maxLineWidth) {
            maxLineWidth = lineSize;
            widestLineNumber = lineCount;
        }
    }
    public int getLineCount() {
        return lineCount;
    }
    public int getMaxLineWidth() {
        return maxLineWidth;
    }
    public int getWidestLineNumber() {
        return widestLineNumber;
    }
    public LineWidthHistogram getLineWidthHistogram() {
        return lineWidthHistogram;
    }
    public double getMeanLineWidth() {
        return (double)totalChars/lineCount;
    }
    public int getMedianLineWidth() {
        Integer[] sortedWidths = getSortedWidths();
        int cumulativeLineCount = 0;
        for (int width : sortedWidths) {
            cumulativeLineCount += lineCountForWidth(width);
            if (cumulativeLineCount > lineCount/2)
                return width;
        }
        throw new Error("Cannot get here");
    }
    private int lineCountForWidth(int width) {
        return lineWidthHistogram.getLinesForWidth(width).size();
    }
    private Integer[] getSortedWidths() {
        Set<Integer> widths = lineWidthHistogram.getWidths();
        Integer[] sortedWidths = (widths.toArray(new Integer[0]));
        Arrays.sort(sortedWidths);
        return sortedWidths;
    }
}

```

Chapter 7. Clean Functions —

Chương 7. Hàm sạch



In the earliest days of computing, programmers composed their systems of a sequence of routines that performed individual tasks. They quickly discovered that some of those routines were generally useful, and they wrote them in their notebooks and called them subroutines. When needed, they would transcribe those subroutines from their notebooks directly into the program they were creating. There were no calls—those subroutines were simply encoded inline within the program.

Vào những ngày đầu của ngành điện toán, lập trình viên dựng nên hệ thống của mình từ một chuỗi các routine, mỗi routine lo một việc riêng. Họ nhanh chóng nhận ra rằng một số routine ấy nhìn chung là hữu dụng, thế là họ chép chúng vào sổ tay và gọi chúng là subroutine. Khi cần, họ chép nguyên các subroutine đó từ sổ tay thẳng vào chương trình đang viết. Chẳng có lời gọi nào cả — các subroutine ấy đơn giản là được viết thẳng (inline) vào bên trong chương trình.

Later, in the early '50s, instructions were added to the machines that would allow subroutines to be called. At first, programmers were reluctant to use such call instructions because they were

expensive in time. But as machines grew in capability, the call instructions became less expensive, and programmers began to use them more and more frequently.

Về sau, đầu thập niên 50, người ta bổ sung vào máy tính những chỉ thị cho phép gọi subroutine. Ban đầu, lập trình viên còn ngần ngại dùng các lệnh gọi ấy vì chúng đắt đỏ về mặt thời gian. Nhưng khi máy móc mạnh dần lên, lệnh gọi trở nên rẻ hơn, và lập trình viên bắt đầu dùng chúng ngày một nhiều.

Then, in the era of Fortran and PL/1, we started composing our systems primarily of subroutines, and we introduced the special case of functions that take arguments and return values. Nowadays, only the **function** survives. Functions are the first line of organization in any modern program. Writing them well is the topic of this chapter.

Rồi đến thời Fortran và PL/1, chúng ta bắt đầu dựng hệ thống chủ yếu từ các subroutine, và giới thiệu một trường hợp đặc biệt: hàm — thứ nhận đối số và trả về giá trị. Ngày nay, chỉ còn hàm sống sót. Hàm là tuyến tổ chức đầu tiên trong mọi chương trình hiện đại. Viết hàm cho ra hồn chính là chủ đề của chương này.

Small! — Nhỏ thôi!

Over the last six decades, I have written functions of all different sizes. In my early years, I wrote several nasty 3,000-line abominations. Later I wrote scads of functions in the 100-to-300-line range. As I matured, I wrote functions that were 20 to 30 lines long. And in the last two decades, I've shortened that to a dozen lines or less. What those decades have taught me, through long trial and error, is that functions should be very small.

Suốt sáu thập niên qua, tôi đã viết đủ loại hàm với đủ mọi kích cỡ. Những năm đầu, tôi viết vài con quái vật gồm ghê tởm dài 3.000 dòng. Sau đó tôi viết hàng lô hàng lốc hàm dài từ 100 đến 300 dòng. Khi chín chắn hơn, tôi viết những hàm dài 20 đến 30 dòng. Và trong hai thập niên gần đây, tôi rút xuống còn chừng chục dòng hoặc ít hơn. Điều mà mấy thập niên ấy dạy tôi, qua vô số lần thử và sai, là: hàm nên rất nhỏ.

Of course I am not the first person to say this. The idea of keeping functions small is an old one. In the eighties, it was common to hear that a function should be no bigger than a screenful. Of course, this was said at a time when VT100 screens were 24 lines by 80 columns, and our editors used 4 lines for administrative purposes. This effectively put the limit at around twenty lines.

Dĩ nhiên tôi chẳng phải người đầu tiên nói điều này. Ý tưởng giữ cho hàm nhỏ là một ý tưởng xưa rồi. Thập niên 80, người ta thường nghe nói rằng một hàm không nên lớn hơn một màn hình. Tất nhiên, điều đó được nói vào cái thời màn hình VT100 chỉ có 24 dòng × 80 cột, còn trình soạn thảo của chúng ta thì tốn mất 4 dòng cho việc quản trị. Rốt cuộc, giới hạn thực tế rơi vào khoảng hai mươi dòng.

In 1999, I went to visit Kent Beck at his home in Oregon. We sat down and did some programming together so that he could show me the **TDD 1 discipline**. I was a skeptic about TDD and wanted to see it firsthand.

Năm 1999, tôi đến thăm Kent Beck tại nhà ông ở Oregon. Chúng tôi ngồi xuống lập trình cùng nhau để ông chỉ cho tôi kỷ luật (discipline) TDD 1. Tôi vốn hoài nghi TDD và muốn tận mắt chứng kiến.

1 Back in those days, it was called **test**-first programming.

1 Hồi đó, người ta gọi nó là test-first programming.

One failing test at a time, we cobbled together a cute little Java/Swing program that he called Sparkle . It produced a visual effect on the screen very similar to the magic wand of the fairy godmother in the movie Cinderella . As you moved the mouse, the sparkles would drip from the cursor with a satisfying scintillation, falling to the bottom of the window through a simulated gravitational field. As we wrote the code together, making one test pass after another, I was struck by how small the functions were. I was accustomed to functions in Swing programs that took up miles of **vertical** space. Every function in this program was just two, or three, or four lines long. Each had a descriptive name. Each was transparently obvious. Each told a story. And each led you to the next in a compelling order. It was that experience that drove home to me the importance of keeping my functions very small.

Mỗi lần một test hỏng, chúng tôi chấp nối dần thành một chương trình Java/Swing nhỏ xinh mà ông đặt tên là Sparkle. Nó tạo ra trên màn hình một hiệu ứng thị giác rất giống cây đũa thần của bà tiên đỡ đầu trong phim Cinderella. Khi bạn di chuột, những đốm lấp lánh nhỏ giọt từ con trỏ với một vẻ lung linh đầy thoả mãn, rơi xuống đáy cửa sổ qua một trường hấp dẫn mô phỏng. Khi cùng nhau viết mã, cho từng test lần lượt vượt qua, tôi cũng người vì các hàm nhỏ đến thế. Tôi vốn quen với những hàm trong chương trình Swing dài lê thê hàng dặm theo chiều dọc. Vậy mà mọi hàm trong chương trình này chỉ dài hai, ba, hoặc bốn dòng. Hàm nào cũng có một cái tên mô tả rõ ràng. Hàm nào cũng hiển nhiên trong suốt. Hàm nào cũng kể một câu chuyện. Và hàm nào cũng dẫn bạn sang hàm kế tiếp theo một trật tự đầy thuyết phục. Chính trải nghiệm đó đã khắc sâu vào tôi tầm quan trọng của việc giữ cho hàm thật nhỏ.

Well-Written Prose — Văn xuôi viết khéo

Keeping functions very small implies that the blocks within if statements, else statements, while statements, and so on will be only one or two lines long. Often that block will contain a call to another function. Not only does this keep the enclosing function small, but it also adds documentary value because the functions called within the block will have nicely descriptive names. Such if / else / while statements read nicely because the function names flow well with the keyword. They will read like well-written prose.

Giữ cho hàm thật nhỏ cũng hàm ý rằng các khối bên trong câu lệnh if, câu lệnh else, câu lệnh while, v.v. sẽ chỉ dài một hoặc hai dòng. Thường thì khối đó chỉ chứa một lời gọi tới hàm khác. Chuyện này không những giữ cho hàm bao ngoài nhỏ gọn, mà còn thêm giá trị tài liệu, bởi các hàm được gọi bên trong khối sẽ mang những cái tên mô tả rất tử tế. Những câu lệnh if / else / while như thế đọc lên rất xuôi tai, vì tên hàm ăn khớp mượt mà với từ khoá. Chúng sẽ đọc lên như một áng văn xuôi viết khéo.

```
if(employee.shouldBePaidToday())  
    employee.pay();
```

This also implies that small functions will not be large enough to hold deeply nested structures. The indent level of a function will seldom be greater than one or two. This, of course, makes the functions easier to read and understand.

Điều này cũng hàm ý rằng hàm nhỏ sẽ chẳng đủ lớn để chứa những cấu trúc lồng nhau sâu hoắm. Mức thụt lề (indentation) của một hàm hiếm khi vượt quá một hoặc hai. Tất nhiên, nhờ vậy mà hàm dễ đọc và dễ hiểu hơn.

This is not a hard-and-fast rule, and it can be overdone. There are certain functions that read better if they are not decomposed into smaller four-line functions. But these are exceptional cases. In general, keeping functions small is the best strategy.

Đây không phải quy tắc cứng nhắc, và ta cũng có thể làm quá tay. Có những hàm đọc lên hay hơn khi không bị chẻ nhỏ thành các hàm bốn dòng. Nhưng đó là những ca ngoại lệ. Nhìn chung, giữ cho hàm nhỏ vẫn là chiến lược tốt nhất.

One Level of Abstraction per Function — Mỗi hàm một mức trừu tượng

In order to make sure our functions are doing “one thing,” we need to make sure that the statements within our functions are all at the same **level of abstraction**. Look at the following bit of Golang code. 2

Để chắc chắn rằng hàm của ta đang làm “một việc”, ta cần chắc rằng mọi câu lệnh bên trong hàm đều ở cùng một mức trừu tượng (level of abstraction). Hãy nhìn đoạn mã Golang sau đây. 2

2 This is from the Video Store example that we’ll be studying much more thoroughly in Chapter 10 , “One Thing .”

2 Đoạn này lấy từ ví dụ Video Store mà ta sẽ mổ xẻ kỹ lưỡng hơn nhiều ở Chương 10, “Một việc duy nhất”.

```
func (customer *Customer)
statement() string {
    totalAmount := 0.0
    frequentRenterPoints := 0
    result := "Rental Record for " + customer.name + "\n"
    for _, rental := range customer.rentals {
        thisAmount := 0.0

        switch rental.movie.movieType {
        case NewRelease:
            thisAmount += float64(rental.daysRented * 3)
        case Regular:
            thisAmount += 2
            if rental.daysRented > 2 {
                thisAmount += float64(rental.daysRented-2) * 1.5
            }
        case Childrens:
            thisAmount += 1.5
            if rental.daysRented > 3 {
                thisAmount += float64(rental.daysRented-3) * 1.5
            }
        }
        frequentRenterPoints++
        if rental.movie.movieType == NewRelease && rental.daysRented >
            frequentRenterPoints++
    }
    result += fmt.Sprintf("\t%s\t%.1f\n", rental.movie.title, thisA
    totalAmount += thisAmount
}
return result + fmt.Sprintf("Amount owed is %.1f\n" +
```

```

        "You earned %d frequent renter points
        totalAmount, frequentRenterPoints)
    }

```

It is easy to see how this violates the rule. There are concepts in there that are at a very high level of **abstraction**, such as looping over `customer.rentals`. There are other concepts that are at an intermediate level of abstraction, such as the **switch statement** that selects the movie type. And there are still other concepts that are remarkably low level, such as

Chẳng khó gì để thấy đoạn này vi phạm quy tắc ấy ra sao. Trong đó có những khái niệm nằm ở mức trừu tượng rất cao, chẳng hạn việc lặp qua `customer.rentals`. Lại có những khái niệm ở mức trừu tượng trung gian, chẳng hạn câu lệnh `switch` dùng để chọn loại phim. Và còn có những khái niệm thấp lè tè, chẳng hạn

`frequentRenterPoints++` and the code that calculates prices for each movie type.

`frequentRenterPoints++` và đoạn mã tính giá cho từng loại phim.

Mixing levels of abstraction within a function is usually confusing. Readers may not be able to tell whether a particular expression is an essential concept or a detail. Worse, like broken windows, once details are mixed with essential concepts, more and more details tend to accrete within the function.

Trộn lẫn các mức trừu tượng bên trong một hàm thường gây rối trí. Người đọc có thể không phân biệt nổi một biểu thức nào đó là khái niệm cốt yếu hay chỉ là chi tiết vụn vặt. Tệ hơn, giống như những ô cửa kính vỡ, một khi chi tiết đã bị trộn với khái niệm cốt yếu thì ngày càng nhiều chi tiết khác sẽ bám vào hàm đó.

Reading Code from Top to Bottom: The Stepdown Rule — Đọc mã từ trên xuống: Quy tắc bậc thang

We want the code to read like a top-down narrative. 3 We want every function to be followed by those at the next level of abstraction so that we can read the program from top to bottom, descending one level of abstraction at a time. I call this **The Stepdown Rule**. We can see this in the following **refactoring** of the previous example.

Ta muốn mã đọc lên như một mạch kể từ trên xuống. 3 Ta muốn sau mỗi hàm là những hàm ở mức trừu tượng kế tiếp, để có thể đọc chương trình từ trên xuống dưới, mỗi lần hạ xuống đúng một mức trừu tượng. Tôi gọi đây là Quy tắc bậc thang (the Stepdown Rule). Ta thấy rõ điều đó trong bản refactor sau đây của ví dụ vừa rồi.

3 [KP78], p. 37.

3 [KP78], tr. 37.

```

func (statement *RentalStatement)
makeStatement() string {
    statement.clearTotals()
    return statement.makeHeader() +
        statement.makeDetails() +
        statement.makeFooter()
}

func (statement *RentalStatement)
clearTotals() {
    statement.totalAmount = 0.0
}

```

```

    statement.frequentRenterPoints = 0
}
func (statement *RentalStatement)
makeHeader() string {
    return "Rental Record for " + statement.name + "\n"
}
func (statement *RentalStatement)
makeDetails() string {
    rentalDetails := ""
    for _, rental := range statement.rentals {
        rentalDetails += statement.makeDetail(rental)
    }
    return rentalDetails
}
func (statement *RentalStatement)
makeDetail(rental *Rental) string {
    amount := rental.determineAmount()
    statement.frequentRenterPoints += rental.determinePoints()
    statement.totalAmount += amount
    return statement.formatDetail(rental, amount)
}
func (statement *RentalStatement)
formatDetail(rental *Rental, amount float64) string {
    return fmt.Sprintf("\t%s\t%.1f\n", rental.getTitle(), amount)
}
func (statement *RentalStatement)
makeFooter() string {
    return fmt.Sprintf("Amount owed is %.1f\n"+
        "You earned %d frequent renter points",
        statement.totalAmount, statement.frequentRenterPoints)
}

```

To read this we start at the top with the `makeStatement` function. To make a statement we first clear the totals, then make the header, the details, and the footer, and concatenate them all together. All those statements are at the same level of abstraction, one level below the name of the function.

Để đọc đoạn này, ta bắt đầu từ trên cùng với hàm `makeStatement`. Muốn tạo một bản kê, trước hết ta xoá sạch các tổng, rồi tạo phần đầu, phần chi tiết và phần chân, sau đó nối tất cả lại với nhau. Toàn bộ những câu lệnh ấy đều ở cùng một mức trừu tượng, thấp hơn tên hàm đúng một bậc.

To clear the totals we set `totalAmount` and `frequentRenterPoints` to zero. Again, each line is at the same level of abstraction, one level below the name of the function.

Để xoá sạch các tổng, ta đặt `totalAmount` và `frequentRenterPoints` về không. Một lần nữa, mỗi dòng đều ở cùng một mức trừu tượng, thấp hơn tên hàm đúng một bậc.

To make the header we format the name from the rental statement with header text. The single line in that function is one level of abstraction below the name of the function.

Để tạo phần đầu, ta định dạng tên lấy từ bản kê thuê phim cùng với đoạn văn bản tiêu đề. Dòng lệnh duy nhất trong hàm đó nằm thấp hơn tên hàm đúng một mức trừu tượng.

In the '70s and '80s, we called this functional decomposition . It was an important part of the **structured programming** discipline. High-level functions are decomposed into lower-level functions, creating a hierarchy of function calls.

Vào thập niên 70 và 80, chúng tôi gọi việc này là phân rã theo chức năng (functional decomposition). Nó là một phần quan trọng của kỷ luật lập trình có cấu trúc (structured programming). Hàm mức cao được phân rã thành các hàm mức thấp hơn, tạo nên một cây phân cấp các lời gọi hàm.

When you follow this discipline, your functions will generally be very small because each function contains just one level of abstraction and defers to the next level down through another nicely named function call.

Khi bạn theo kỷ luật này, hàm của bạn nói chung sẽ rất nhỏ, bởi mỗi hàm chỉ chứa đúng một mức trừu tượng và nhường phần việc ở mức dưới cho một lời gọi hàm khác được đặt tên tử tế.

The discipline of refactoring is a critical element of this approach. When we initially write the code for a function, we tend to do a depth-first search through the levels of abstraction. The code that results from that depth-first search looks like the first version above. But once we get the function working, it is very easy to **refactor** it by extracting out the different levels of abstraction into nicely named functions.

Kỷ luật refactoring là yếu tố sống còn của lối tiếp cận này. Khi mới viết mã cho một hàm, ta thường có xu hướng đi tìm kiếm theo chiều sâu xuyên qua các mức trừu tượng. Đoạn mã sinh ra từ kiểu tìm kiếm theo chiều sâu đó trông y hệt phiên bản đầu tiên ở trên. Nhưng một khi hàm đã chạy được, việc refactor nó lại rất dễ: chỉ cần rút các mức trừu tượng khác nhau ra thành những hàm có tên tử tế.

So, first, make it work. Then, make it right.

Vậy nên, trước hết hãy làm cho nó chạy. Rồi mới làm cho nó đúng đắn.

Entanglement — Rối bện

One objection to this approach, offered by John Ousterhout, is that when you decompose larger functions into smaller ones, you can sometimes create small functions that are tangled together. Two functions are entangled when, in order to understand one, you have to understand the other.

Một phản biện cho cách tiếp cận này, do John Ousterhout nêu ra, là khi bạn phân rã hàm lớn thành các hàm nhỏ, đôi lúc bạn tạo ra những hàm nhỏ bị rối bện (entangled) vào nhau. Hai hàm bị rối bện khi, để hiểu được hàm này, bạn buộc phải hiểu cả hàm kia.

John is right about this, but he and I differ on the relative cost. I can tolerate a bit of entanglement so long as the functions step down one level of abstraction, and the lower-level functions are positioned after the higher-level functions. John is much more averse to entanglement and would rather two entangled functions be merged into a single function.

John nói đúng, nhưng ông và tôi khác nhau ở chỗ đánh giá cái giá phải trả. Tôi có thể chịu đựng chút rối bện, miễn là các hàm hạ xuống đúng một mức trừu tượng, và các hàm mức thấp hơn được đặt sau các hàm mức cao hơn. John dị ứng với rối bện hơn tôi nhiều, và thà gộp hai hàm rối bện ấy lại thành một hàm duy nhất.

Switch Statements — Câu lệnh switch

It's hard to make a small switch statement. 4 It's also hard to make a switch statement that does one thing. By their nature,

Khó mà viết nổi một câu lệnh switch nhỏ. 4 Cũng khó mà viết nổi một câu lệnh switch chỉ làm một việc. Về bản chất,

switch statements always do N things. Unfortunately, we can't always avoid switch statements, but we can make sure that each switch statement is buried in a low-level **module** and is never repeated. We do this, of course, with **polymorphism**.

câu lệnh switch luôn làm N việc. Đáng tiếc, ta không phải lúc nào cũng tránh được câu lệnh switch, nhưng ta có thể bảo đảm rằng mỗi câu lệnh switch đều bị chôn kỹ trong một module mức thấp và không bao giờ bị lặp lại. Dĩ nhiên, ta làm chuyện đó bằng đa hình (polymorphism).

4 And, of course, I include if / else chains in this.

4 Và dĩ nhiên, tôi tính cả các chuỗi if / else vào đây.

Consider, for example, this switch statement extracted from the example above.

Chẳng hạn, hãy xét câu lệnh switch được rút ra từ ví dụ ở trên.

```
func (rental Rental) determineAmount() float64 {
    amount := 0.0
    switch rental.movie.movieType {
    case NewRelease:
        amount += float64(rental.daysRented * 3)
    case Regular:
        amount += 2
        if rental.daysRented > 2 {
            amount += float64(rental.daysRented-2) * 1.5
        }
    case Childrens:
        amount += 1.5
        if rental.daysRented > 3 {
            amount += float64(rental.daysRented-3) * 1.5
        }
    }
    return amount
}
```

There are several problems with this function.

Hàm này có mấy vấn đề.

1. It's larger than I'd like, and it will grow when new movie types are added.
2. It very clearly does more than one thing.
3. It violates the Single Responsibility Principle 5 (SRP) because there is more than one reason for it to change.
 1. Nó lớn hơn mức tôi mong muốn, và sẽ còn phình ra khi có thêm loại phim mới.
 2. Nó rõ ràng rành rành là làm nhiều hơn một việc.

3. Nó vi phạm Single Responsibility Principle 5 (SRP) vì có nhiều hơn một lý do khiến nó phải thay đổi.

5 See Chapter 19 , “ **The SOLID Principles** .”

5 Xem Chương 19, “Các nguyên tắc SOLID”.

4. It violates the Open–Closed Principle 6 (OCP) because it must change whenever new types are added.

4. Nó vi phạm Open–Closed Principle 6 (OCP) vì nó buộc phải thay đổi mỗi khi có loại mới được thêm vào.

6 Ibid.

6 Như trên.

But possibly the worst problem with this function is that there are an unlimited number of other functions that will have the same **structure**. For example, we already have

Nhưng có lẽ vấn đề tệ nhất của hàm này là: sẽ có vô số hàm khác mang đúng cấu trúc ấy.
Chẳng hạn, ta đã có sẵn

determinePoints , and we might later have applyCoupon or

determinePoints, và sau này có thể ta sẽ có applyCoupon hay

getAgeRestriction . It’s not hard to think of many others. All of which would contain a related switch statement.

getAgeRestriction. Chẳng khó gì để nghĩ ra thêm cả tá cái tương tự. Và tất cả đều sẽ chứa một câu lệnh switch họ hàng với nhau.

One **good** solution to this problem is to bury the switch statement in the basement of an Abstract Factory, 7 and never let anyone see it. The factory uses the switch statement to create appropriate instances of the derivatives of

Một giải pháp tốt cho vấn đề này là chôn câu lệnh switch xuống tận tầng hầm của một Abstract Factory, 7 và đừng bao giờ để ai nhìn thấy nó. Factory dùng câu lệnh switch để tạo ra các instance thích hợp của các lớp dẫn xuất từ

7 [GOF95].

7 [GOF95].

```
type RentalTypeFactory interface {
    make(typeName string) RentalType
}
type RentalTypeFactoryImpl struct{}
func (r RentalTypeFactoryImpl) make(typeName string) RentalType {
    switch typeName {
    case "childrens":
        return ChildrensRental{}
    case "new release":
        return NewReleaseRental{}
    case "regular":
        return RegularRental{}
    }
}
```

```
    return nil
}
```

My general rule for switch statements is that they can be tolerated if they appear only once, are positioned in a concrete module like `main`, are used to create polymorphic objects, and are hidden behind an **interface** so that the rest of the system can't see them.

Quy tắc chung của tôi với câu lệnh `switch` là: có thể chấp nhận được nếu chúng chỉ xuất hiện đúng một lần, được đặt trong một module cụ thể như `main`, được dùng để tạo ra các đối tượng đa hình, và được giấu sau một interface để phần còn lại của hệ thống không nhìn thấy chúng.

Clean Functions: A Deeper Look — Hàm sạch: Nhìn sâu hơn

A clean function should have five attributes. It should be nameable, insulated, homogenous, contextual, and pure. If you reorder that, it spells PINCH—but that's the wrong order in which to present them.

Một hàm sạch nên có năm phẩm chất. Nó phải đặt tên được (nameable), cách ly (insulated), đồng nhất (homogenous), gắn với ngữ cảnh (contextual), và thuần khiết (pure). Nếu xếp lại thứ tự, chúng ghép thành chữ PINCH — nhưng đó lại không phải trật tự nên dùng để trình bày.

Contextual — Gắn với ngữ cảnh

The concept and discipline of modularity was not immediately obvious to the earliest of programmers. It slowly grew out of sheer necessity. In the late 1940s, Grace Hopper wrote down common sequences of code in her notebooks to help her program the Harvard Mark 1. She called these sequences subroutines. They were transcribed into programs, which were then painstakingly punched onto the long paper tapes that drove the execution of the machine.

Khái niệm và kỷ luật về tính module hoá không hiện ra ngay lập tức với những lập trình viên đầu tiên. Nó lớn dần lên từ nhu cầu thuần tuý. Cuối thập niên 1940, Grace Hopper chép lại vào sổ tay những chuỗi mã hay dùng để giúp bà lập trình cho chiếc Harvard Mark 1. Bà gọi những chuỗi đó là subroutine. Chúng được chép vào chương trình, rồi chương trình lại được đục lỗ một cách nhọc nhằn lên những cuộn băng giấy dài — thứ điều khiển việc thực thi của cỗ máy.

Those subroutines were the first modules. From there, the concept of modularity grew into callable subroutines, and finally functions.

Những subroutine ấy là các module đầu tiên. Từ đó, khái niệm module hoá lớn dần thành subroutine gọi được, và cuối cùng là hàm.

By the 1960s, programmers were adopting a discipline called modular programming, which stated that the best programs were decomposed into many different modules. By the late '60s, Dahl and Nygaard advanced the concept of modularity by inventing **object**-oriented programming.⁸ In the early '70s, David Parnas continued the advancement by promoting the benefits of information hiding.

Đến thập niên 1960, lập trình viên đã theo một kỷ luật gọi là modular programming, chủ trương rằng chương trình tốt nhất là chương trình được phân rã thành thật nhiều module khác nhau. Cuối thập niên 60, Dahl và Nygaard đẩy khái niệm module hoá tiến thêm một

bước bằng việc phát minh ra lập trình hướng đối tượng. 8 Đầu thập niên 70, David Parnas tiếp tục mạch tiến bộ đó khi cổ vũ cho lợi ích của việc che giấu thông tin (information hiding).

8 Though they did not give it that name. Object-oriented was coined by Alan Kay a few years later.

8 Dù họ không gọi nó bằng cái tên ấy. Thuật ngữ object-oriented do Alan Kay đặt ra vài năm sau đó.

Nowadays, we understand that every function lives within a context. A context is a cooperative grouping of functions and **data** structures. Every context has an external interface and an internal **implementation**.

Ngày nay, ta hiểu rằng mọi hàm đều sống trong một ngữ cảnh. Ngữ cảnh là một nhóm các hàm và cấu trúc dữ liệu (data structure) cộng tác với nhau. Mỗi ngữ cảnh đều có một interface đối ngoại và một hiện thực (implementation) bên trong.

In OO languages, contexts are most commonly defined with classes. In Java, for example, the public methods of a **class** represent the external interface, while the private data and methods comprise the internal implementation.

Trong các ngôn ngữ OO, ngữ cảnh thường được định nghĩa bằng class. Ví dụ trong Java, các method public của một class chính là interface đối ngoại, còn dữ liệu và method private thì tạo nên hiện thực bên trong.

In C, we use .h header files for the external interface and .c files for the internal implementation.

Trong C, ta dùng tệp header .h cho interface đối ngoại và tệp .c cho hiện thực bên trong.

Some languages provide a rich syntax for defining a context. In other languages, the programmer must define a context by informal conventions such as filenames, directories, and comments.

Một số ngôn ngữ cung cấp cú pháp phong phú để định nghĩa ngữ cảnh. Ở những ngôn ngữ khác, lập trình viên phải tự định nghĩa ngữ cảnh bằng các quy ước không chính thức như tên tệp, thư mục và comment.

But irrespective of the language and the syntax, every function lives within a context—and it is the programmer's job to identify, create, and manage those contexts.

Nhưng bất kể ngôn ngữ và cú pháp ra sao, mọi hàm đều sống trong một ngữ cảnh — và nhiệm vụ của lập trình viên là nhận diện, tạo ra và quản lý những ngữ cảnh đó.

Some of the functions within a context will be public, meaning that they are known and used outside of that context. They are part of the external interface of the context. Other functions within the context will be private, meaning that they are known and used only within the context and are not used outside of it.

Một số hàm trong ngữ cảnh sẽ là public, nghĩa là chúng được biết đến và được dùng từ bên ngoài ngữ cảnh ấy. Chúng là một phần của interface đối ngoại của ngữ cảnh. Những hàm khác trong ngữ cảnh sẽ là private, nghĩa là chúng chỉ được biết và được dùng bên trong ngữ cảnh, không dùng ở ngoài.

Many OO languages provide a syntax to declare functions either public or private. Some have other categories, like

Nhiều ngôn ngữ OO cung cấp cú pháp để khai báo hàm là public hoặc private. Một số ngôn ngữ còn có những loại khác, chẳng hạn

protected or **package** , as ways to create partial restrictions. Other languages depend on the discipline of the programmers and various programmer conventions to achieve the same result.

protected hay package, như những cách tạo ra hạn chế một phần. Các ngôn ngữ khác thì dựa vào kỷ luật của lập trình viên và đủ loại quy ước để đạt cùng kết quả.

A context creates a boundary between the things that are known external to the context and those things that remain private within the context. This boundary, specified by the programmer, describes the programmer's intent regarding the things that are higher level and likely won't change very often and those things that are lower level and likely will experience frequent change.

Ngữ cảnh tạo ra một ranh giới giữa những thứ được biết từ bên ngoài ngữ cảnh và những thứ vẫn ở trạng thái riêng tư bên trong. Ranh giới này, do lập trình viên chỉ định, mô tả ý định của lập trình viên về việc đâu là những thứ ở mức cao và ít khi thay đổi, đâu là những thứ ở mức thấp và nhiều khả năng sẽ thay đổi xoành xoạch.

Nameable — Đặt tên được

There are at least two things to consider when naming a function. The name must be appropriately descriptive within its context, and it must be appropriately convenient.

Có ít nhất hai điều cần cân nhắc khi đặt tên cho một hàm. Cái tên phải mô tả đủ rõ trong ngữ cảnh của nó, và phải đủ tiện dụng.

Descriptive — Mô tả rõ

Every clean function should have a name that describes what that function does. Since the name describes **behavior**, the name should be a verb, a verb phrase, or an implicit verb,

Mọi hàm sạch đều nên có một cái tên mô tả việc mà hàm đó làm. Vì tên mô tả hành vi, nên tên phải là một động từ, một cụm động từ, hoặc một động từ ngầm ẩn,

`likeMath.sign(x)`.

The description should be slightly more abstract than the code within the function. In other words, the name should hide the implementation.

Phần mô tả nên trừu tượng hơn đôi chút so với đoạn mã bên trong hàm. Nói cách khác, cái tên nên che giấu hiện thực.

So, as an absurd counter-example:

Chẳng hạn, hãy xem một phản ví dụ đến mức phi lý sau:

```
public static double addAtoBandDivideBy2(double a, double b) {  
    return (a+b)/2.0;  
}
```

That name certainly tells us what the code does; but it is much too concrete. Not only does it tell us what the function does, but it tells us how it does it. It exposes the implementation. We should abstract it just a bit.

Cái tên đó chắc chắn cho ta biết đoạn mã làm gì; nhưng nó cụ thể quá mức. Nó không chỉ nói cho ta biết hàm làm gì, mà còn nói luôn hàm làm điều đó bằng cách nào. Nó phơi bày hiện thực. Ta nên trừu tượng hoá nó lên một chút.

```
public static double average(double a, double b) {
    return (a+b)/2.0;
}
```

That's nice. The name is an implicit verb (to average) that hides the implementation. That's good because any programmer who sees the name will know what to expect; but any programmer who wants to change the implementation will not have to change the name. For example, expanding the function to use a **variable** number of arguments would not affect the name of the function at all.

Thế mới đẹp. Cái tên là một động từ ngầm ẩn (average — lấy trung bình) và nó che giấu hiện thực. Điều đó tốt, vì bất kỳ lập trình viên nào nhìn thấy cái tên cũng biết mình nên trông đợi điều gì; còn lập trình viên nào muốn đổi hiện thực thì cũng chẳng phải đổi tên. Chẳng hạn, mở rộng hàm để nhận số lượng đối số tùy ý cũng không hề ảnh hưởng đến tên hàm.

```
public static double average(double... ns ) {
    return Arrays.stream(ns).sum()/ns.length;
}
```

A one-word name, like average , may work for some functions, but others will need more words in order to be appropriately descriptive. For example:

Một cái tên chỉ gồm một từ, như average, có thể ổn với một số hàm, nhưng những hàm khác sẽ cần nhiều từ hơn thì mới mô tả đủ rõ. Ví dụ:

```
public void addSalesReceipt(SalesReceipt salesReceipt) {
    salesReceipts.add(salesReceipt);
}
```

The function appears simple enough, but the name is three words long. Why? Perhaps we should look at the context.

Hàm này trông có vẻ đơn giản, vậy mà tên của nó dài tới ba từ. Vì sao? Có lẽ ta nên nhìn vào ngữ cảnh.

```
public class CommissionedEmployee extends Employee {
    private List<SalesReceipt> salesReceipts = new ArrayList<>();
    //...
    public void addSalesReceipt(SalesReceipt salesReceipt) {
        salesReceipts.add(salesReceipt);
    }
}
```

This function is a **method** of a derived class. Derived classes are more detailed than their base classes. Thus, the

Hàm này là một method của lớp dẫn xuất. Lớp dẫn xuất thì chi tiết hơn lớp cơ sở của nó. Bởi vậy, hàm

addSalesReceipt function lives in a detailed context. Detailed contexts require more words to describe. Thus, you should expect that, in more detailed contexts, longer names will be necessary in order to be appropriately descriptive.

addSalesReceipt sống trong một ngữ cảnh chi tiết. Ngữ cảnh càng chi tiết thì càng cần nhiều từ để mô tả. Vậy nên bạn hãy chuẩn bị tinh thần rằng, trong những ngữ cảnh chi tiết hơn, ta sẽ cần những cái tên dài hơn thì mới mô tả đủ rõ.

It is hard to overestimate the value of good names. Remember Ward Cunningham’s principle: “You know you are working on **clean code** when each routine turns out to be pretty much what you expected.” Half the battle to achieving that principle is choosing good names for small functions that do one thing.

Khó mà đánh giá hết giá trị của những cái tên hay. Hãy nhớ nguyên tắc của Ward Cunningham: “Bạn biết mình đang làm việc với clean code khi mỗi routine hoá ra gần đúng như những gì bạn đã trông đợi.” Một nửa cuộc chiến để đạt được nguyên tắc ấy nằm ở việc chọn tên hay cho những hàm nhỏ chỉ làm một việc.

Don’t be afraid to make a name long. A long descriptive name is better than a short enigmatic name. A long descriptive name is often better than a long descriptive **comment**. Use a naming convention like CamelCase , or

Đừng ngại đặt tên dài. Một cái tên dài mà mô tả rõ còn hơn một cái tên ngắn mà bí hiểm. Một cái tên dài mà mô tả rõ thường cũng hơn hẳn một comment dài dòng mô tả. Hãy dùng một quy ước đặt tên như CamelCase, hoặc

snake_case , or kebob-case . This allows multiple words in a function’s name to be easily read. Those words should describe what the function does.

snake_case, hoặc kebob-case. Nhờ vậy, nhiều từ trong tên một hàm sẽ dễ đọc. Những từ ấy phải mô tả được việc mà hàm làm.

Don’t be afraid to spend time choosing a name. Indeed, you should try several different names and read the code with each in place. Modern IDEs make it trivial to change names, so experiment with different names until you find one that is as descriptive as you can make it.

Đừng ngại bỏ thời gian ra chọn tên. Thật vậy, bạn nên thử vài cái tên khác nhau rồi đọc lại đoạn mã với từng cái tên. IDE hiện đại khiến việc đổi tên trở nên dễ như bỡn, nên hãy cứ thử nghiệm cho tới khi tìm được cái tên mô tả rõ nhất trong khả năng của bạn.

Choosing descriptive names will clarify the design of the module in your mind and help you improve it. It is not at all uncommon that hunting for a good name results in a favorable restructuring of the code.

Việc chọn những cái tên mô tả rõ sẽ làm sáng tỏ thiết kế của module trong đầu bạn và giúp bạn cải thiện nó. Chuyện đi săn một cái tên hay rồi dẫn tới một lần tái cấu trúc mã có lợi là chuyện chẳng hiếm chút nào.

Convenient — Tiện dụng

In order to be convenient, the name of a function must be short and memorable. But short and memorable names are not always very descriptive. So there is a trade-off to make. The value of convenience depends on how frequently the function is called. A function that is called a lot should have a short and memorable name. A function that is called just once can afford a longer and much more descriptive name.

Muốn tiện dụng thì tên hàm phải ngắn và dễ nhớ. Nhưng tên ngắn và dễ nhớ lại không phải lúc nào cũng mô tả tốt. Thế nên phải có một sự đánh đổi. Giá trị của tính tiện dụng phụ thuộc vào chuyện hàm đó được gọi thường xuyên đến mức nào. Hàm bị gọi nhiều thì nên có tên ngắn và dễ nhớ. Hàm chỉ được gọi đúng một lần thì thoải mái mang cái tên dài hơn và mô tả kỹ hơn nhiều.

The two conflicting priorities of descriptiveness and convenience conspire to create the simple **heuristic** we discussed in Chapter 4 .

Hai ưu tiên xung khắc nhau — tính mô tả và tính tiện dụng — cùng nhau đẻ ra cái kinh nghiệm thực hành (heuristic) đơn giản mà ta đã bàn ở Chương 4.

The length of the name of a function should be inversely proportional to the size of the containing **scope**.

Độ dài tên của một hàm nên tỉ lệ nghịch với kích thước của phạm vi (scope) chứa nó.

A scope is the span of code in which a name is known and used. For example, the scope of the average function is effectively global. Whereas the scope of `addSalesReceipt` is restricted to the users of the `CommissionedEmployee` class. Thus, the more detailed the context, the smaller the scope. The more general the context, the larger the scope.

Phạm vi là vùng mã mà trong đó một cái tên được biết tới và được dùng. Chẳng hạn, phạm vi của hàm `average` trên thực tế là toàn cục. Còn phạm vi của `addSalesReceipt` thì bị giới hạn trong đám người dùng class `CommissionedEmployee`. Vậy nên bối cảnh càng chi tiết thì phạm vi càng nhỏ. Bối cảnh càng tổng quát thì phạm vi càng lớn.

Functions in a small scope do not require much convenience. So their names can be longer. This is good because functions in a detailed context usually require longer names to be descriptive. Functions in a large scope require more convenience. This is good because those contexts are usually more general and so do not usually require longer names.

Hàm nằm trong phạm vi nhỏ chẳng cần tiện dụng cho lắm. Nên tên chúng có thể dài hơn. Điều đó lại hoá hay, vì hàm trong một bối cảnh chi tiết thường phải có tên dài mới mô tả nổi. Hàm nằm trong phạm vi lớn thì cần tiện dụng hơn. Điều đó cũng hoá hay, vì những bối cảnh như thế thường tổng quát hơn nên thường chẳng cần tên dài.

For example, the function `File.open(String filename)` is short and memorable because it lives at the global scope and is very general. We would find it inconvenient if this function

Chẳng hạn, hàm `File.open(String filename)` vừa ngắn vừa dễ nhớ, bởi nó sống ở phạm vi toàn cục và rất tổng quát. Ta hẳn sẽ thấy bất tiện nếu hàm này

were named `openFileAndThrowExceptionIfNotFound`. By the same token, we would find it somewhat disconcerting if the

token, ta sẽ thấy hơi gợn nếu hàm

`addSalesReceipt` function were named simply `add` .

`addSalesReceipt` lại chỉ được đặt tên cụt lủn là `add` .

So, the bigger the scope, the more general the function and the smaller the name. The smaller the scope, the more detailed the function and the longer the name.

Vậy đó: phạm vi càng lớn thì hàm càng tổng quát và tên càng ngắn. Phạm vi càng nhỏ thì hàm càng chi tiết và tên càng dài.

From this, we would predict that global functions will usually have short, one-word names. Public functions within a context will have slightly longer names. Private functions called by the public functions will have even longer names. And private functions called by other private functions may have very long names. The deeper down you go in the function hierarchy, the longer the names will likely be. This is true of Java, C, Clojure, Python, Ruby, or any other language that allows long names.

Từ đây, ta có thể đoán rằng hàm toàn cục thường sẽ mang tên ngắn, chỉ một từ. Hàm public nằm trong một bối cảnh nào đó sẽ có tên dài hơn chút. Hàm private do các hàm public gọi tới sẽ có tên còn dài hơn nữa. Còn hàm private được các hàm private khác gọi thì có thể mang những cái tên rất dài. Càng đi sâu xuống trong hệ thống thứ bậc của hàm, tên càng có xu hướng dài ra. Điều này đúng với Java, C, Clojure, Python, Ruby, hay bất kỳ ngôn ngữ nào cho phép tên dài.

Insulated — Cách ly

In days long past, we called the notion of insulation low **coupling** or sometimes **encapsulation**. But neither of these terms seem to me to be quite sufficient.

Ngày xưa ngày xưa, chúng tôi gọi khái niệm cách ly này là low coupling, hoặc đôi khi là đóng gói (encapsulation). Nhưng với tôi thì chẳng thuật ngữ nào trong hai cái đó là đủ.

The word complex means more than one strand. One of the factors that make functions complex is the number of inputs and outputs they must deal with. Each of those inputs and outputs is a coupling, and each coupling is another strand weaving its way through that function. Therefore, to manage the complexity of our functions, we seek to limit the number of such couplings.

Từ complex nghĩa gốc là “nhiều hơn một sợi”. Một trong những thứ khiến hàm trở nên phức tạp chính là số lượng đầu vào và đầu ra mà nó phải xử lý. Mỗi đầu vào và đầu ra ấy là một mối kết dính giữa các thành phần (coupling), và mỗi mối kết dính lại là thêm một sợi nữa luồn lách qua hàm đó. Bởi vậy, muốn quản được độ phức tạp của hàm, ta phải tìm cách hạn chế số mối kết dính như thế.

The more arguments a function has, the more it is coupled to its callers. A function with five arguments is likely more coupled than a function with two. Thus, we strive to limit the number of function arguments to some manageable limit. I prefer a limit of three where feasible. I'll talk more about that in the next chapter.

Hàm càng nhiều đối số thì càng bị kết dính chặt với những nơi gọi nó. Một hàm có năm đối số thì gần như chắc chắn kết dính chặt hơn hàm có hai. Vì vậy ta cố gắng giữ số đối số của hàm trong một giới hạn kiểm soát được. Chỗ nào khả dĩ, tôi thích giới hạn là ba. Tôi sẽ nói kỹ hơn về chuyện đó ở chương sau.

The fewer the arguments, the less coupled the function is and the simpler that function is to call. A function with no arguments is very easy to call. However, such functions aren't particularly useful unless they live within a context that has been prepared for them by other functions.

Càng ít đối số thì hàm càng ít kết dính và càng dễ gọi. Hàm không có đối số nào thì gọi cực dễ. Tuy nhiên, những hàm như vậy chẳng mấy hữu dụng trừ phi chúng sống trong một bối cảnh đã được các hàm khác dọn sẵn cho.

For example, you might have a context that represents a finite state machine. That context will contain variables that represent the current state of the machine, as well as the network of state transitions.

Chẳng hạn, bạn có thể có một bối cảnh biểu diễn một máy trạng thái hữu hạn. Bối cảnh đó sẽ chứa các biến mô tả trạng thái hiện tại của máy, cùng với mạng lưới các phép chuyển trạng thái.

Let's use the following state machine. It represents a simple subway turnstile. There are two states: Locked and Unlocked. There are two events: Coin and Pass, which represent the user dropping a

coin in the slot, and passing through the gate, respectively. Finally, there are five actions: alarmOff , alarmOn ,

Hãy dùng máy trạng thái sau đây. Nó biểu diễn một cửa quay (turnstile) đơn giản ở ga tàu điện ngầm. Có hai trạng thái: Locked và Unlocked. Có hai sự kiện: Coin và Pass, lần lượt tương ứng với việc người dùng thả đồng xu vào khe và việc đi qua cổng. Cuối cùng, có năm hành động: alarmOff, alarmOn,

lock , unlock , and thankyou . The state transition table looks like this.

lock , unlock , và thankyou . Bảng chuyển trạng thái trông như thế này.

Locked	Coin	Unlocked	{alarmOff unlock}
Locked	Pass	Locked	alarmOn
Unlocked	Coin	Unlocked	thankyou
Unlocked	Pass	Locked	lock

The logic is very simple.

Logic ở đây rất đơn giản.

- When the machine is in the Locked state and it gets a
 - Khi máy đang ở trạng thái Locked và nhận được

Coin event, it will transition to the Unlocked state and invoke the alarmOff and unlock actions.

sự kiện Coin, nó sẽ chuyển sang trạng thái Unlocked và gọi các hành động alarmOff và unlock.

- When the machine is in the Locked state and it gets a
 - Khi máy đang ở trạng thái Locked và nhận được

Pass event, it will remain in the Locked state and invoke the alarmOn action.

sự kiện Pass, nó sẽ vẫn ở trạng thái Locked và gọi hành động alarmOn.

- When the machine is in the Unlocked state and it gets a
 - Khi máy đang ở trạng thái Unlocked và nhận được

Coin event, it will remain in the Unlocked state and invoke the thankyou 9 action.

sự kiện Coin, nó sẽ vẫn ở trạng thái Unlocked và gọi hành động thankyou 9.

9 Perhaps it lights up a little “Thank You” light, or emits the words “Thank you” from a speaker.

9 Có lẽ nó bật một cái đèn nhỏ ghi “Thank You”, hoặc phát ra câu “Thank you” qua loa.

- When the machine is in the Unlocked state and it gets a
 - Khi máy đang ở trạng thái Unlocked và nhận được

Pass event, it will transition to the Locked state and invoke the lock action.

sự kiện Pass, nó sẽ chuyển sang trạng thái Locked và gọi hành động lock.

From this, it should be obvious that when the system is running, the external software will need to send the Coin and

Từ đây, hẳn đã rõ rằng khi hệ thống chạy, phần mềm bên ngoài sẽ phải gửi các sự kiện Coin và

Pass events. This can be accomplished by calling functions that take no arguments named coin() and pass(). We can capture this in an interface named TurnstileEvents .

Pass. Chuyện này có thể làm được bằng cách gọi những hàm không nhận đối số nào, tên là coin() và pass(). Ta có thể gói chúng vào một interface tên TurnstileEvents .

```
public interface TurnstileEvents {  
    void coin();  
    void pass();  
}
```

The five actions, which also take no arguments, can be captured in an interface named TurnstileActions .

Năm hành động kia, cũng chẳng nhận đối số nào, có thể gói vào một interface tên TurnstileActions .

```
public interface TurnstileActions {  
    void lock();  
    void thankyou();  
    void alarmOn();  
    void unlock();  
    void alarmOff();  
}
```

Then, the logic of the state machine can be captured in an abstract class named TurnstileFSM .

Còn logic của máy trạng thái thì có thể gói gọn trong một abstract class tên TurnstileFSM.

```
public abstract  
class TurnstileFSM implements TurnstileActions, TurnstileEvents {  
    private enum State {Locked, Unlocked}  
    private enum Event {Coin, Pass}  
  
    private State state = State.Locked;  
    private void setState(State s) {  
        state = s;  
    }  
    public void coin() {  
        handleEvent(Event.Coin);  
    }  
    public void pass() {  
        handleEvent(Event.Pass);  
    }  
    private void handleEvent(Event event) {  
        switch (state) {  
            case Locked -> {  
                switch (event) {  
                    case Coin -> {  
                        setState(State.Unlocked);  
                        alarmOff();  
                        unlock();  
                    }  
                    case Pass -> {  
                        setState(State.Locked);  
                    }  
                }  
            }  
        }  
    }  
}
```

```
        alarmOn();
    }
}
case Unlocked -> {
    switch (event) {
        case Coin -> {
            setState(State.Unlocked);
            thankYou();
        }
        case Pass -> {
            setState(State.Locked);
            lock();
        }
    }
}
}
```

The running system that detects and sends events need know only about `TurnstileEvents` . It is completely decoupled from the logic and the actions. The logic of the state machine lives behind a very strong insulation barrier. Nothing outside of `TurnstileFSM` knows about the states or the logic.

Hệ thống đang chạy — cái phần phát hiện và gửi sự kiện — chỉ cần biết tới `TurnstileEvents` mà thôi. Nó hoàn toàn tách rời (decoupling) khỏi logic lẫn các hành động. Logic của máy trạng thái nằm sau một hàng rào cách ly cực kỳ vững chắc. Không thứ gì bên ngoài `TurnstileFSM` biết về các trạng thái hay về logic.

Finally, the TurnstileFSM class is abstract in order to allow the action functions to be implemented in a derived class, thereby **decoupling** the logic of the machine from the implementations of the actions.

Cuối cùng, class `TurnstileFSM` được để ở dạng abstract nhằm cho phép các hàm hành động được hiện thực trong một lớp dẫn xuất, nhờ đó tách rời logic của máy khỏi phần hiện thực của các hành động.

From this example, you can see the contexts that encapsulate the state machine logic, the events, and the actions. You can see how limiting the number of arguments in the functions that cross the boundaries between the contexts helps keep the coupling very low.

Qua ví dụ này, bạn thấy được những bối cảnh đóng gói logic của máy trạng thái, các sự kiện, và các hành động. Bạn cũng thấy được việc hạn chế số đối số ở những hàm bằng qua ranh giới giữa các bối cảnh đã giúp giữ độ kết dính thấp đến mức nào.

Of course, there will be many times when you will need functions that take one, two, or even three or more arguments. But once you pass more than three, you have to ask yourself why you aren't passing in an object or **data structure** that encapsulates them. If you truly need that many arguments passed into a function, it's worth considering whether the data could be separated into separate contexts, and whether the behavior of the function could be split up to live within those contexts.

Đĩ nhiên sẽ có vô số lúc bạn cần những hàm nhận một, hai, thậm chí ba đối số trở lên. Nhưng một khi vượt quá ba, bạn phải tự hỏi vì sao mình không truyền vào một đối tượng (object) hay một cấu trúc dữ liệu (data structure) gói gọn chúng lại. Nếu quả thật bạn cần

chúng ấy đối số truyền vào một hàm, thì cũng đáng cân nhắc xem dữ liệu ấy có thể tách ra thành những bối cảnh riêng biệt hay không, và hành vi của hàm có thể nhỏ để sống bên trong những bối cảnh đó hay không.

Homogenous — Đồng nhất

The lines of a function should all be at the same level of abstraction, which is one level below the name. In the previous section, I called this The Stepdown Rule .

Mọi dòng trong một hàm đều phải ở cùng một mức trừu tượng, thấp hơn cái tên đúng một bậc. Ở mục trước, tôi gọi đây là Quy tắc bậc thang (the Stepdown Rule).

The operation of a function takes place at one particular level of abstraction described by the name of that function. The name of the function represents the abstraction that the lines within the function implement.

Một hàm vận hành ở đúng một mức trừu tượng cụ thể, mức mà tên hàm mô tả. Tên hàm đại diện cho phần trừu tượng hoá (abstraction) mà các dòng bên trong hàm hiện thực.

But what if the lines of a function exist at more than one level of abstraction? For example, consider the following function:

Nhưng nếu các dòng trong một hàm lại nằm ở nhiều mức trừu tượng khác nhau thì sao? Chẳng hạn, hãy xem hàm sau:

```
public static int orientation(Point a, Point b, Point c) {
    double val = (b.x() - a.x()) * (c.y() - a.y()) -
                 (b.y() - a.y()) * (c.x() - a.x());
    if (val == 0) {
        return 0;
    }
    return (val > 0) ? +1 : -1;
}
```

Given three points (a, b, c) on a plane, this function determines whether traveling from a to b to c is a turn to the right (−1), a turn to the left (1), or no turn at all (0).

Cho ba điểm (a, b, c) trên mặt phẳng, hàm này xác định xem đi từ a sang b rồi sang c là rẽ phải (−1), rẽ trái (1), hay chẳng rẽ gì cả (0).

The first part determines the cross product of the two vectors $a \rightarrow b$ and $b \rightarrow c$. The sign of this product determines the answer.

Phần đầu tính tích có hướng của hai vector $a \rightarrow b$ và $b \rightarrow c$. Dấu của tích này quyết định câu trả lời.

The determination of the cross product is appropriately one level of abstraction below the name. But the other four lines are lower level than the first. So let's put those four lines into a function of their own.

Việc tính tích có hướng thì đúng là thấp hơn cái tên một mức trừu tượng. Nhưng bốn dòng còn lại thì ở mức thấp hơn dòng đầu. Vậy ta hãy tổng bốn dòng đó vào một hàm riêng.

```
public static int orientation(Point a, Point b, Point c) {
    double val = (b.x() - a.x()) * (c.y() - a.y()) -
                 (b.y() - a.y()) * (c.x() - a.x());
```

```

    return sign(val);
}
private static int sign(double n) {
    if (n == 0) {
        return 0;
    }
    return (n > 0) ? +1 : -1;
}

```

Now both lines of the original function are at the same level of abstraction. This makes the implementation of the

Giờ thì cả hai dòng của hàm ban đầu đều ở cùng một mức trừu tượng. Điều đó làm cho phần hiện thực của hàm

orientation function homogenous.

orientation trở nên đồng nhất.

But we can do better, because the names are not particularly communicative, and the return values are unnamed and easy to get wrong.

Nhưng ta còn có thể làm tốt hơn, vì mấy cái tên chưa truyền đạt được gì mấy, còn các giá trị trả về thì không có tên và rất dễ nhầm.

```

public enum Chirality {left, right, none};
public static Chirality getChirality(Point a, Point b, Point c) {
    return chirality(crossProduct(a, b, c));
}
private static double crossProduct(Point a, Point b, Point c) {
    double abx = b.x() - a.x();
    double aby = b.y() - a.y();
    double acx = c.x() - a.x();
    double acy = c.y() - a.y();
    return abx * acy - aby * acx;
}
private static Chirality chirality(double n) {
    if (n == 0) {
        return none;
    }
    return (n > 0) ? left : right;
}

```

You might be bothered that this is twice as long as the original; but it seems to me to be much more readable— especially for someone seeing it for the first time. You might also be bothered that this takes a few more CPU cycles than the original. If you have some desperate need to save 50ns, then by all means trim those cycles back. Just keep in mind that someone else is going to have to maintain this after you, and their time has value—possibly a lot more than 50ns will cost you.

Có thể bạn khó chịu vì bản này dài gấp đôi bản gốc; nhưng với tôi nó dễ đọc hơn hẳn — nhất là với người lần đầu nhìn thấy nó. Cũng có thể bạn khó chịu vì nó tốn thêm vài chu kỳ CPU so với bản gốc. Nếu bạn đang khát khao tiết kiệm cho bằng được 50ns, thì cứ việc cắt bớt mấy chu kỳ ấy đi. Chỉ xin nhớ cho rằng sau bạn sẽ có người khác phải bảo trì đồng này, và thời gian của họ cũng có giá — rất có thể còn đắt hơn nhiều so với cái giá 50ns kia.

Pure — Thuần khiết

The behavior of a pure function is dependent upon nothing other than its arguments; and its execution does not cause the behavior of any other function to change. Following are some of the advantages of pure functions.

Hành vi của một hàm thuần khiết (pure function) không phụ thuộc vào bất cứ thứ gì ngoài các đối số của nó; và việc thực thi nó không làm hành vi của bất kỳ hàm nào khác thay đổi. Sau đây là vài lợi thế của hàm thuần khiết.

- A pure function is a true mapping of arguments to return values—and can be replaced with such a mapping in order to trade execution speed for memory.
- Pure functions are much less susceptible to race conditions and problems of concurrent update because they do not change the state of the underlying system.
 - Hàm thuần khiết là một ánh xạ đúng nghĩa từ đối số sang giá trị trả về — và có thể thay thế bằng chính một ánh xạ như vậy để đổi tốc độ thực thi lấy bộ nhớ.
 - Hàm thuần khiết ít bị tổn thương trước race condition và các vấn đề cập nhật đồng thời hơn nhiều, bởi chúng không làm thay đổi trạng thái của hệ thống bên dưới.

10 They are not perfectly immune, because any stateful system can be simulated with pure functions. See [Functional Design].

10 Chúng không hoàn toàn miễn nhiễm, vì bất kỳ hệ thống có trạng thái nào cũng có thể mô phỏng được bằng hàm thuần khiết. Xem [Functional Design].

- Pure functions are easy to combine into networks of functionality. This is because they have no external dependencies and so are free to be called at any time by any other function.
- Pure functions are easy to test. This is because the test setup does not require the coercion of the underlying system into any special state and the test does not need to assert that the underlying state of the system has been changed.
- Pure functions are easy to distribute between multiple processors. Since the output of a pure function depends only on its arguments, it doesn't matter where, or in which processor, the function executes.
 - Hàm thuần khiết rất dễ ghép lại thành những mạng lưới chức năng. Lý do là chúng không có phụ thuộc (dependency) bên ngoài nên có thể được bất kỳ hàm nào gọi vào bất kỳ lúc nào.
 - Hàm thuần khiết rất dễ test. Lý do là phần setup của test không phải ép hệ thống bên dưới vào một trạng thái đặc biệt nào, và test cũng chẳng cần assert rằng trạng thái nền của hệ thống đã đổi.
 - Hàm thuần khiết rất dễ phân tán trên nhiều bộ xử lý. Vì đầu ra của một hàm thuần khiết chỉ phụ thuộc vào đối số của nó, nên hàm chạy ở đâu, trên bộ xử lý nào, cũng chẳng quan trọng.

OK, so they're great. How do you create a pure function? Simple. Don't change the value of any variables; or to paraphrase the famous line from Mommy Dearest : "No Assignment Statements Ever!" Or to say that in yet another way: Pure functions are immutable. 11

Được rồi, chúng tuyệt vời thật. Vậy làm sao tạo ra một hàm thuần khiết? Đơn giản thôi. Đừng thay đổi giá trị của bất kỳ biến nào; hay nói theo kiểu nhại câu thoại nổi tiếng trong Mommy Dearest: "No Assignment Statements Ever!" (Cấm tiệt câu lệnh gán!) Hoặc diễn đạt theo một cách khác nữa: hàm thuần khiết là bất biến. 11

11 [Functional Design], Chapter 1 .

11 [Functional Design], Chương 1.

But how realistic is a rule like that? That depends upon your point of view. For example:

Nhưng một quy tắc như thế thì thực tế đến đâu? Cái đó còn tùy bạn đứng ở góc nhìn nào.
Chẳng hạn:

```
public static double sigma(double... ns) {
    var mu = mean(ns);
    var deviations =
        Arrays.stream(ns).map(x->(x-mu)*(x-mu)).boxed().mapToDouble(x->
double variance = deviations.sum() / ns.length;
    return Math.sqrt(variance);
}
```

The sigma function is pure. The assignment statements within it are initializations, not assignments. They do not alter the state of any existing variable.

Hàm sigma này thuần khiết. Các câu lệnh gán bên trong nó là những phép khởi tạo, chứ không phải phép gán. Chúng chẳng làm thay đổi trạng thái của bất kỳ biến nào đã tồn tại.

But what if we turn them into true assignments by splitting the initializations into declarations and assignments?

Nhưng nếu ta biến chúng thành phép gán thứ thiệt bằng cách tách phần khởi tạo thành khai báo rồi gán thì sao?

```
public static double sigma(double... ns) {
    double mu;
    mu = mean(ns);
    DoubleStream deviations;
    deviations =
        Arrays.stream(ns).map(x->(x-mu)*(x-mu)).boxed().mapToDouble(x->
double variance;
    variance = deviations.sum() / ns.length;
    return Math.sqrt(variance);
}
```

This version of sigma is still pure because even though the values of the variables change, those changes are not seen outside of the function. How far can we push this? Let's use a much more traditional 12 implementation:

Phiên bản sigma này vẫn thuần khiết, bởi tuy giá trị các biến có thay đổi, những thay đổi ấy không hề lộ ra ngoài hàm. Ta có thể đẩy chuyện này đi xa tới đâu? Hãy dùng một bản hiện thực truyền thống 12 hơn nhiều:

12 Of all the sigma implementations shown in this chapter, this is my favorite. No goofy syntax; no strange concatenation of dots, parentheses, and arrows; no annoying conversion functions. Just nice, straightforward code.

12 Trong tất cả các bản hiện thực sigma trình bày ở chương này, đây là bản tôi khoái nhất. Không cú pháp dở hơi; không có mớ dấu chấm, dấu ngoặc và mũi tên nối nhau kỳ quặc; không có hàm chuyển đổi phiền nhiễu. Chỉ là mã đẹp đẽ, thẳng thớm.

```
public static double sigma(double... ns) {
    double mu = mean(ns);
    double variance = 0;
```

```

for (double n : ns) {
    var deviation = n - mu;
    variance += deviation * deviation;
}
variance /= ns.length;
return Math.sqrt(variance);
}

```

This version of sigma is modifying variables all over the place but it is still pure because all those modifications are hidden from any outside observer. The internals are not pure at all, but to the outside observer, the illusion of purity has been preserved.

Phiên bản sigma này sửa biến tứ tung, vậy mà nó vẫn thuần khiết, bởi mọi thay đổi ấy đều bị giấu khỏi mọi người quan sát bên ngoài. Ruột gan bên trong chẳng thuần khiết chút nào, nhưng với người quan sát bên ngoài thì ảo giác thuần khiết vẫn được giữ nguyên.

We can even go so far as to create a mutable object and still keep the sigma function pure.

Ta thậm chí có thể đi xa tới mức tạo ra một đối tượng khả biến mà hàm sigma vẫn thuần khiết.

```

public static double sigma(double... ns) {
    return new SigmaCalculator(ns).invoke();
}
private static class SigmaCalculator {
    private double[] ns;
    private double mu;
    private double variance = 0;
    private double deviation;

    public SigmaCalculator(double... ns) {
        this.ns = ns;
    }

    public double invoke() {
        mu = mean(ns);
        for (double n : ns) {
            deviation = n - mu;
            variance += deviation * deviation;
        }
        variance /= ns.length;
        return Math.sqrt(variance);
    }
}

```

Now we've got instance variables and all manner of variable manipulations. And yet, the sigma function is pure. None of those impure operations are visible outside the sigma function.

Giờ thì ta có cả biến instance lẫn đủ kiểu thao tác lên biến. Vậy mà hàm sigma vẫn thuần khiết. Không một thao tác không thuần khiết nào trong số đó lộ ra bên ngoài hàm sigma.

The bottom line is that purity is an external characteristic of a function, not an internal characteristic. It does not matter how impure the internals of a function are—that function will be pure so long as all the impurity is hidden from all external observers (including other threads).

Rốt cuộc, tính thuần khiết là một đặc tính bên ngoài của hàm, không phải đặc tính bên

trong. Ruột gan của hàm có không thuần khiết đến đâu cũng chẳng sao — hàm đó vẫn thuần khiết chừng nào toàn bộ phần không thuần khiết còn được giấu khỏi mọi người quan sát bên ngoài (kể cả các luồng (thread) khác).

And, by the way, that is how all functional programming languages claim purity. They just hide the impurity of the underlying implementation.

Mà nhân tiện, đó cũng chính là cách mọi ngôn ngữ lập trình hàm tuyên bố mình thuần khiết. Chúng chỉ giấu đi phần không thuần khiết của bản hiện thực bên dưới mà thôi.

Partial Purity — Thuần khiết một phần

As we just saw, purity is an observed quality, not an intrinsic quality. Therefore, whether a function is pure or not depends upon what is being observed.

Như ta vừa thấy, thuần khiết là một phẩm chất được quan sát, chứ không phải phẩm chất nội tại. Bởi vậy, một hàm có thuần khiết hay không còn tùy vào việc ta đang quan sát cái gì.

Consider the C library function `fopen(char* name, char* mode)`. This function has the **side effect** of leaving the named file open. This means that `fopen` is not pure, because there are functions, such as `fgetc` and `fputc`, that will not behave properly unless `fopen` is called first. Indeed, if you call `fopen` with the same arguments twice in a row without an intervening call to `fclose`, the second invocation will likely fail because the file is already open. So `fopen` is not pure, because of the side effect that leaves the system in an altered state.

Hãy xét hàm `fopen(char* name, char* mode)` của thư viện C. Hàm này có tác dụng phụ (side effect) là để lại tệp được nêu tên ở trạng thái đang mở. Nghĩa là `fopen` không thuần khiết, bởi có những hàm như `fgetc` và `fputc` sẽ chẳng chạy cho ra hồn nếu `fopen` không được gọi trước. Thật vậy, nếu bạn gọi `fopen` hai lần liên tiếp với cùng bộ đối số mà không xen một lời gọi `fclose` vào giữa, lần gọi thứ hai nhiều khả năng sẽ thất bại vì tệp đã mở sẵn. Vậy nên `fopen` không thuần khiết, chỉ vì cái tác dụng phụ để hệ thống lại trong trạng thái đã bị đổi.

However, we can make the `fopen` function appear to be pure, at least to certain observers, by ensconcing it in another function that hides the side effect.

Tuy nhiên, ta có thể làm cho hàm `fopen` trông có vẻ thuần khiết, ít nhất là dưới mắt một số người quan sát nhất định, bằng cách nhét nó vào trong một hàm khác che giấu tác dụng phụ đi.

```
void openAndDo(char* fileName, void (*doTo)(FILE*)) {  
    FILE* f = fopen(fileName, "r+");  
    doTo(f);  
    fclose(f);  
}
```

This function opens the file, executes the function pointed at by `doTo`, and then closes the file. This puts the system back into the original state, hiding the impurity of `fopen`. For all intents and purposes, this use of `openAndDo` is pure. We might use it as follows:

Hàm này mở tệp, thực thi hàm mà `doTo` trỏ tới, rồi đóng tệp lại. Việc đó đưa hệ thống trở về trạng thái ban đầu, giấu nhem sự không thuần khiết của `fopen`. Xét về mọi mặt thực dụng, cách dùng `openAndDo` như thế này là thuần khiết. Ta có thể dùng nó như sau:

```

void show(FILE* f) {
    int c;
    while ((c=fgetc(f)) != EOF)
        putchar(c);
}

int main(int ac, char** av) {
    openAndDo("x.x", show);
}

```

This program prints the “x.x” file on standard out. Let’s make this a bit more interesting by modifying the contents of the file. We’ll add a line end, if the file does not end in one already.

Chương trình này in tệp “x.x” ra đầu ra chuẩn. Ta hãy làm cho nó thú vị hơn chút bằng cách sửa nội dung tệp. Ta sẽ thêm một ký tự xuống dòng, nếu tệp chưa kết thúc bằng ký tự đó.

```

void appendLineEnd(FILE* f) {
    int c;
    int lastC;
    while ((c = fgetc(f)) != EOF) {
        lastC = c;
    }
    if (lastC != '\n')
        fputc('\n', f);
}

int main(int ac, char** av) {
    openAndDo("x.x", appendLineEnd);
    openAndDo("x.x", show);
}

```

Now the program adds the line end and then displays the file. This program does leave the system in an altered state, because the contents of the “x.x” file have been changed. However, if we do not care to observe the contents of the

Giờ chương trình thêm ký tự xuống dòng rồi hiển thị tệp. Chương trình này quả có để hệ thống lại trong trạng thái đã bị đổi, vì nội dung tệp “x.x” đã thay đổi. Tuy nhiên, nếu ta không bận tâm quan sát nội dung của tệp

“x.x” file, then the openAndDo function is pure as far as we are concerned.

“x.x”, thì với ta, hàm openAndDo vẫn là thuần khiết.

Conclusion — Kết luận

Clean functions are small, do one thing, and read like well-written prose. Each fits into its place in a hierarchy of functions that lead, one step at a time, from high-level abstractions to low-level details. Clean functions exist within a well-defined context, are isolated from each other, are well and conveniently named, and are pure to all external observers.

Hàm sạch thì nhỏ, làm đúng một việc, và đọc lên như một áng văn xuôi viết khéo. Mỗi hàm nằm đúng chỗ của nó trong một hệ thống thứ bậc các hàm, dẫn ta từng bước một từ những mức trừu tượng cao xuống tới các chi tiết mức thấp. Hàm sạch sống trong một bối

cảnh được định nghĩa rõ ràng, cách ly khỏi nhau, được đặt tên vừa hay vừa tiện, và thuần khiết dưới mắt mọi người quan sát bên ngoài.

Chapter 8. Function Heuristics —

Chương 8. Kinh nghiệm thực hành về hàm



What follows are some heuristics that I have found helpful in keeping my code clean. I don't follow them as hard-and-fast rules, but when all else is equal, these are my biases.

Sau đây là một vài kinh nghiệm thực hành (heuristic) mà tôi thấy hữu ích trong việc giữ cho mã của mình sạch sẽ. Tôi không coi chúng là luật cứng nhắc, nhưng khi mọi thứ khác ngang nhau thì đây là những thiên hướng của tôi.

Function Arguments — Đối số của hàm

Every **argument** you add to a **function** signature makes that function harder to understand. The easiest argument list to understand is the empty argument list.

Mỗi đối số bạn thêm vào chữ ký của một hàm đều khiến hàm đó khó hiểu hơn. Danh sách đối số dễ hiểu nhất là danh sách rỗng.

In the first edition of this book, this statement confused a number of people. Their complaint was that a function with an empty argument list can do little other than return a constant. So, apparently, some further explanation is needed.

Ở lần xuất bản đầu tiên của cuốn sách này, câu nói đó đã làm khá nhiều người bối rối. Họ phản nản rằng một hàm với danh sách đối số rỗng thì chẳng làm được gì nhiều ngoài việc trả về một hằng số. Vậy thì hình như cần giải thích thêm một chút.

In an **object**-oriented program, every **method** of an object has at least one argument. In Python and Go, it is explicitly declared and named `self` by convention. In Java, C#, and C++, it is the implicit `this`. This argument, whether explicit or implicit, represents the enclosing **scope**.

Trong một chương trình hướng đối tượng, mọi **method** của một đối tượng đều có ít nhất một đối số. Trong Python và Go, nó được khai báo tường minh và theo quy ước được đặt tên là `self`. Trong Java, C# và C++, nó là `this` ngầm định. Đối số này, dù tường minh hay ngầm định, đại diện cho phạm vi (scope) bao quanh.

Do functions in C have an implied first argument? Of course they do; it is the enclosing global scope. A C function always has access to that scope, so it plays the same role as `this` or `self`. Thus, every function ever written has an implied argument that represents the enclosing scope.

Vậy các hàm trong C có đối số đầu tiên ngầm định không? Tất nhiên là có; đó chính là phạm vi toàn cục bao quanh. Một hàm C luôn truy cập được phạm vi đó, nên nó đóng đúng vai trò của `this` hay `self`. Như vậy, mọi hàm từng được viết ra đều có một đối số ngầm định đại diện cho phạm vi bao quanh.

Therefore, when I say that the easiest argument list to understand is the empty argument list, I am not counting the implied enclosing scope. The easiest argument list to understand is the argument list with no declared arguments.

Cho nên, khi tôi nói danh sách đối số dễ hiểu nhất là danh sách rỗng, tôi không tính phạm vi bao quanh ngầm định đó. Danh sách đối số dễ hiểu nhất là danh sách không có đối số nào được khai báo.

Do such methods or functions exist? Of course they do. `Stack.pop()`,

Có method hay hàm nào như vậy không? Tất nhiên là có. `Stack.pop()`,

`file.close()`, `semaphore.release()`, `buffer.flush()`: They are very common and very easy to read.

rất dễ đọc.

The next-easiest argument list to understand is the list with one declared argument. The call `f(x)` is pretty easy to understand. There's no chance you'll get the arguments out of order.

Dễ hiểu thứ nhì là danh sách chỉ có một đối số được khai báo. Lời gọi `f(x)` khá dễ hiểu. Chẳng có cơ hội nào để bạn truyền sai thứ tự đối số cả.

Two arguments aren't too hard. Keeping track of which comes first is pretty easy—I mean, there's only one way to get it wrong. You can make this easier by making the two arguments commutative.

Hai đối số cũng chưa đến nỗi khó. Nhớ xem cái nào đứng trước cũng khá dễ — ý tôi là, chỉ có đúng một cách để làm sai thôi. Bạn có thể làm cho việc này còn dễ hơn nữa bằng cách khiến hai đối số có tính giao hoán.

Three arguments starts to get difficult. There are five ways to get them into the wrong order. Yes, I know that the IDEs have gotten quite helpful with this; but still, why tempt fate?

Ba đối số thì bắt đầu khó. Có tới năm cách để xếp sai thứ tự. Vâng, tôi biết các IDE ngày nay đã hỗ trợ khá tốt chuyện này; nhưng dù vậy, việc gì phải thử vận may?

Arguments are couplings and are therefore hard enough as it is without having to worry about their order in the calling sequence. What's more, they are harder to read than they are to write. The IDE will give you a lot of help while writing, but much less while reading. So three is usually my limit.

Đối số là những mối kết dính giữa các thành phần (coupling), nên tự thân chúng đã đủ phiền rồi, khỏi cần phải bận tâm thêm về thứ tự của chúng trong lời gọi. Hơn nữa, đọc chúng còn khó hơn viết chúng. IDE sẽ giúp bạn rất nhiều lúc viết, nhưng giúp ít hơn nhiều lúc đọc. Nên ba thường là giới hạn của tôi.

Yes, a limit of three is arbitrary. You might be comfortable with more. You do you, I'll do me.

Vâng, con số ba là tùy tiện. Bạn có thể thấy thoải mái với nhiều hơn. Bạn cứ theo cách của bạn, tôi theo cách của tôi.

If I need to pass more than three arguments into a function, I try to find a way to group them. I might be able to put them into an object—after all, if four things are so cohesive that they can be passed, as a unit, into a function, then why aren't they already an object? Or, I might put them into a hash map with nice names to identify them.

Nếu cần truyền nhiều hơn ba đối số vào một hàm, tôi sẽ tìm cách gom chúng lại. Có thể tôi sẽ nhét chúng vào một đối tượng (object) — suy cho cùng, nếu bốn thứ gắn kết với nhau đến mức có thể được truyền vào một hàm như một khối thống nhất, thì tại sao chúng chưa phải là một đối tượng? Hoặc, tôi có thể đặt chúng vào một hash map với những cái tên tử tế để nhận diện.

One of the simplest ways to reduce argument lists is to create fields in the current **class** to hold the values that we would otherwise pass as arguments. Indeed, one of the reasons that OO is a powerful tool is that it allows us to create contexts that allow functions to communicate without passing a lot of arguments.

Một trong những cách đơn giản nhất để rút ngắn danh sách đối số là tạo các field trong class hiện tại để giữ những giá trị mà lẽ ra ta phải truyền vào dưới dạng đối số. Quả thật, một trong những lý do khiến OO là công cụ mạnh mẽ chính là nó cho phép ta tạo ra các ngữ cảnh để các hàm giao tiếp với nhau mà không phải truyền cả đống đối số.

Variadic Arguments — Đối số biến thiên (variadic)

Sometimes we want to write functions that take an unlimited number of arguments. In Java, the `String.format` function is like that.

Đôi khi ta muốn viết những hàm nhận số lượng đối số không giới hạn. Trong Java, hàm `String.format` là kiểu như vậy.

```
public String format(String format, Object... args)
```

I think it's safe to assume that this function really just takes two arguments: the `format`, and the array of `Object`s named `args`.

Tôi nghĩ có thể an tâm cho rằng hàm này thực ra chỉ nhận hai đối số: `format`, và mảng các `Object` tên là `args`.

More Than Three? — Nhiều hơn ba thì sao?

Sometimes there is a rationale behind the ordering. For example, it is not hard to keep the arguments of `distance(x1,y1,x2,y2)` in order. On the other hand, if there is no rationale, then there are 23 ways to get the order wrong. Even with the help of the IDE, that's a fairly large risk. It's an even bigger risk while reading the code, since you are not likely to notice when the arguments are out of order. So usually, if there are more than three arguments, I find some other means.

Đôi khi thứ tự có lý do đứng sau nó. Chẳng hạn, chẳng khó gì để nhớ đúng thứ tự các đối số của `distance(x1,y1,x2,y2)`. Ngược lại, nếu không có lý do nào cả thì có tới 23 cách xếp sai thứ tự. Ngay cả khi có IDE trợ giúp, đó vẫn là rủi ro khá lớn. Rủi ro còn lớn hơn nữa lúc đọc mã, vì bạn khó lòng nhận ra khi các đối số bị đảo thứ tự. Vậy nên thường thì nếu có hơn ba đối số, tôi sẽ tìm cách khác.

Keyword Arguments — Đối số theo từ khoá

Some languages allow you to name the arguments in your function calls. For example, in C# you can prefix your arguments with the name of the argument:

Một số ngôn ngữ cho phép bạn nêu tên đối số ngay trong lời gọi hàm. Ví dụ, trong C# bạn có thể đặt tên đối số ở phía trước:

```
createRental(movie: m, daysRented: 3);
```

When using a language feature like that, I don't worry as much about the order of the arguments and might, on occasion, pass more than three.

Khi dùng một tính năng ngôn ngữ như vậy, tôi không lo lắng nhiều về thứ tự đối số, và thỉnh thoảng cũng có thể truyền nhiều hơn ba.

In some languages, hash maps are so easy to define that they provide a nice way to pass named arguments. For example, in Clojure I can pass a hash map as follows:

Ở một số ngôn ngữ, hash map dễ định nghĩa đến mức chúng trở thành một cách hay để truyền đối số có tên. Ví dụ, trong Clojure tôi có thể truyền một hash map như sau:

```
(create-rental {:movie m, :daysRented 3})
```

Ruby provides a similar bit of syntactic sugar:

Ruby cũng có một chút đường cú pháp tương tự:

```
createRental(movie: m, daysRented: 3)
```

In Ruby, if the `createRental` function is declared to take a hash map, then the arguments will automatically be assumed to be keyword/value pairs.

Trong Ruby, nếu hàm `createRental` được khai báo là nhận một hash map thì các đối số sẽ tự động được hiểu là các cặp keyword/value.

Flag Arguments — Đối số cờ (flag)

Flag arguments are ugly. Passing a boolean into a function is generally careless. It immediately complicates the signature of the function, and loudly proclaims that the function does more than

one thing. It does one thing if the flag is true and another if the flag is false . They are also difficult to read. For example:

Đối số cờ thật xấu xí. Truyền một boolean vào hàm nhìn chung là cẩu thả. Nó lập tức làm chữ ký của hàm rối lên, và tuyên bố oang oang rằng hàm này làm nhiều hơn một việc. Nó làm một việc nếu cờ là true và làm việc khác nếu cờ là false . Chúng cũng khó đọc nữa. Ví dụ:

```
String monthNames[] = getMonths(false);
```

What does this mean? Are we getting the months or not?

Câu này nghĩa là gì? Ta có lấy các tháng hay không lấy?

The definition of the function shows us that the boolean argument is used to select between long and short month names. But there's no way to know that by looking at the call. The only way to know it is to read the **implementation** of the function, or at least the ugly Javadoc.

Định nghĩa của hàm cho ta thấy đối số boolean được dùng để chọn giữa tên tháng dạng dài và dạng ngắn. Nhưng chẳng cách nào biết được điều đó khi chỉ nhìn vào lời gọi. Cách duy nhất để biết là đọc phần hiện thực (implementation) của hàm, hoặc chí ít là đọc cái Javadoc xấu xí kia.

```
/**
 * Returns an array of month names.
 *
 * @param shortened a flag indicating that shortened month names should
 *                  be returned.
 * @return an array of month names.
 */
public static String[] getMonths(final boolean shortened) {
    if (shortened) {
        return DATE_FORMAT_SYMBOLS.getShortMonths();
    } else {
        return DATE_FORMAT_SYMBOLS.getMonths();
    }
}
```

Boolean flags can be pretty confusing if you don't know what they mean. That glaring true or false is indicating some kind of option; but if you don't know what that option is, you'll have to click through to the function and read the implementation—and who wants to do that?

Các cờ boolean có thể gây lú kha khá nếu bạn không biết chúng nghĩa là gì. Cái true hay false chình ỉnh đó đang chỉ ra một tùy chọn nào đấy; nhưng nếu bạn không biết tùy chọn đó là gì, bạn sẽ phải bấm vào hàm và đọc phần hiện thực — mà ai lại muốn làm thế chứ?

The solution, where possible, is to split the function into two functions: one that does the true case and one that does the false case.

Giải pháp, khi có thể, là tách hàm thành hai hàm: một hàm lo trường hợp true và một hàm lo trường hợp false .

```
String[] getLongMonthNames() {...}
String[] getShortMonthNames() {...}
```

There are, of course, exceptions to this rule. For example, some objects have binary flags within them that are set with a setFlag(boolean) method. I'm not going to complain about them.

Tất nhiên, quy tắc này cũng có ngoại lệ. Chẳng hạn, một số đối tượng có các cờ nhị phân bên trong, được đặt bằng một method `setFlag(boolean)`. Tôi sẽ không phân nân về chúng đâu.

Output Arguments — Đối số đầu ra

Arguments are most naturally interpreted as inputs to a function. But sometimes people will use arguments to catch output values. This is often because they want to return more than one value from a function. For example, in C# you can use out arguments, in C you can use pointers, and in C++ you can pass references as follows.

Đối số thường được hiểu một cách tự nhiên là đầu vào của hàm. Nhưng đôi khi người ta lại dùng đối số để hứng giá trị đầu ra. Chuyện này thường xảy ra vì họ muốn trả về nhiều hơn một giá trị từ một hàm. Ví dụ, trong C# bạn có thể dùng đối số out, trong C bạn có thể dùng con trỏ, còn trong C++ bạn có thể truyền tham chiếu như sau.

```
void stats(double* ns, int n, double& mean, double& sum) {
    sum=0;
    for (int i=0; i<n; i++)
        sum += ns[i];
    mean=sum/n;
}
```

I try to avoid output arguments like this. They violate the principle of least surprise. 1 People generally don't expect an argument to catch an output, so if they see the following call, they may wonder why we are passing the sum and mean into the function. 1 The Law of Least Astonishment, PL/1 Bulletin, 1967.

Tôi cố tránh những đối số đầu ra kiểu này. Chúng vi phạm nguyên tắc ít gây bất ngờ nhất (principle of least surprise). 1 Người ta thường không mong đợi một đối số lại đi hứng đầu ra, nên khi thấy lời gọi sau đây, họ có thể sẽ thắc mắc vì sao ta lại truyền sum và mean vào hàm. 1 The Law of Least Astonishment, PL/1 Bulletin, 1967.

```
stats(list, listSize, listMean, listSum);
```

Reading a line of code like that can cause the reader to do a double take or to stop and wonder what's going on. That's because they expect **data** to go into functions through the arguments and out from functions through the return value.

Đọc một dòng mã như thế có thể khiến người đọc phải nhìn lại lần nữa, hoặc phải dừng lại tự hỏi chuyện gì đang diễn ra. Đó là vì họ mong dữ liệu đi vào hàm qua các đối số và đi ra khỏi hàm qua giá trị trả về.

In Go, Python, or Ruby, 2 we could return multiple values from a function. 2 Ruby returns a mutable array. Caveat emptor.

Trong Go, Python hay Ruby, 2 ta có thể trả về nhiều giá trị từ một hàm. 2 Ruby trả về một mảng có thể thay đổi được. Liệu mà dùng.

```
mean, sum = stats(list);
```

In other languages, we could return an object with two elements.

Ở các ngôn ngữ khác, ta có thể trả về một đối tượng gồm hai phần tử.

Or, in all cases, we could create two functions: one for sum and the other for mean . All else being equal, that's probably the best solution.

Hoặc, trong mọi trường hợp, ta có thể tạo hai hàm: một cho sum và một cho mean . Khi mọi thứ khác ngang nhau, đó có lẽ là giải pháp tốt nhất.

In the days before object-oriented programming, it was sometimes necessary to have output arguments. However, much of the need for output arguments disappears in OO languages because the object itself can act as the output argument of its methods. In other words, it might be better for stats.calculate to be invoked as follows:

Vào cái thời chưa có lập trình hướng đối tượng, đôi khi đối số đầu ra là thứ bắt buộc phải có. Tuy nhiên, phần lớn nhu cầu dùng đối số đầu ra biến mất trong các ngôn ngữ OO, bởi bản thân đối tượng có thể đóng vai đối số đầu ra cho chính các method của nó. Nói cách khác, có lẽ tốt hơn là gọi stats.calculate như sau:

```
stats.calculate(list);
double mean = stats.getMean();
double sum = stats.getSum();
```

Error Codes — Mã lỗi

Go programmers often use the ability to return multiple values from a function to return error codes along with values:

Lập trình viên Go thường tận dụng khả năng trả về nhiều giá trị từ một hàm để trả về mã lỗi kèm theo giá trị:

```
f, err := os.Open("someFile")
if err != nil {
    log.Fatal(err)
}
```

This strategy can also be used in languages with out arguments.

Chiến lược này cũng dùng được trong các ngôn ngữ có đối số out .

```
File f = os.open("someFile", out error);
```

So long as this convention is applied narrowly and consistently, I think it's a reasonable convention; especially in a language like Go that does not have exceptions.

Miễn là quy ước này được áp dụng trong phạm vi hẹp và nhất quán, tôi thấy nó là một quy ước hợp lý; nhất là trong một ngôn ngữ như Go, vốn không có exception.

Command Query Separation — Tách lệnh khỏi truy vấn

CQS is a convention that many programmers adopt when and where appropriate. I've used it, and it is often very helpful. It should be considered where practical.

CQS là một quy ước mà nhiều lập trình viên áp dụng khi và ở nơi thích hợp. Tôi từng dùng nó, và nó thường rất hữu ích. Nên cân nhắc nó ở những chỗ khả thi.

Under the CQS convention, functions either do something or answer something, but not both. Either a function is a command that creates a **side effect** (i.e., it changes the state of the system), or it is a

query that returns some information. Doing both can lead to confusion. Consider, for example, the following function:

Theo quy ước CQS, hàm hoặc là làm một việc gì đó, hoặc là trả lời một điều gì đó, chứ không làm cả hai. Hoặc hàm đó là một lệnh (command) tạo ra tác dụng phụ (side effect) — tức là nó thay đổi trạng thái của hệ thống — hoặc nó là một truy vấn (query) trả về một thông tin nào đó. Làm cả hai có thể dẫn tới nhầm lẫn. Chẳng hạn, hãy xem hàm sau:

```
public boolean set(String attribute, String value);
```

This function sets the value of a named attribute and returns true if it is successful and false if no such attribute exists. This leads to odd statements like this:

Hàm này đặt giá trị cho một thuộc tính có tên cho trước và trả về true nếu thành công, false nếu không tồn tại thuộc tính đó. Điều này dẫn tới những câu lệnh kỳ quặc như thế này:

```
if (set("username", "unclebob"))...
```

Imagine this from the point of view of the reader. What does it mean? Is it asking whether the "username" attribute was previously set to "unclebob"? Or is it asking whether the "username" attribute was successfully set to "unclebob"? It's hard to infer the meaning from the call because it's not clear whether the word set is a verb or an adjective.

Hãy thử đặt mình vào vị trí người đọc. Câu này nghĩa là gì? Nó đang hỏi rằng thuộc tính "username" trước đó có được đặt thành "unclebob" hay không? Hay nó đang hỏi rằng thuộc tính "username" có được đặt thành "unclebob" thành công hay không? Rất khó suy ra ý nghĩa từ lời gọi, bởi không rõ chữ set ở đây là động từ hay tính từ.

The author intended set to be a verb, but in the context of the if statement, it feels like an adjective. We could try to resolve this by renaming the set function to setAndCheckIfExists, but that doesn't much help the readability of the if statement. The real solution is to separate the command from the query so that the ambiguity cannot occur.

Tác giả muốn set là động từ, nhưng trong ngữ cảnh câu lệnh if, nó lại nghe như tính từ. Ta có thể thử giải quyết bằng cách đổi tên hàm set thành setAndCheckIfExists, nhưng cách đó cũng chẳng giúp câu lệnh if dễ đọc hơn bao nhiêu. Giải pháp thật sự là tách lệnh khỏi truy vấn để sự mập mờ không thể xảy ra.

```
if (attributeExists("username")) {  
    setAttribute("username", "unclebob");  
    ...  
}
```

But, as I said, there are situations where conforming to CQS is impractical or just inconvenient. For example, consider the following:

Nhưng, như tôi đã nói, có những tình huống mà tuân theo CQS là bất khả thi hoặc đơn giản là bất tiện. Ví dụ, hãy xem đoạn sau:

```
int top = stack.pop();
```

That simple line of code violates CQS. It queries the top of the stack and also changes the state of the stack.

Dòng mã đơn giản đó vi phạm CQS. Nó truy vấn đỉnh của stack và đồng thời cũng thay đổi trạng thái của stack.

Is there a cost to violating CQS in convenient situations like this? Yes: **concurrency**. If the pop function is not made atomic with the proper use of semaphores or locks, then it could be possible for a **race condition** to cause the top of the stack to be returned to two different threads before that object was properly removed from the stack.

Vi phạm CQS trong những tình huống tiện lợi như vậy có phải trả giá không? Có: tính đồng thời (concurrency). Nếu hàm pop không được làm cho nguyên tử bằng cách dùng semaphore hay lock đúng cách, thì có thể xảy ra chuyện một race condition khiến đỉnh của stack bị trả về cho hai luồng (thread) khác nhau trước khi đối tượng đó kịp được gỡ khỏi stack đúng cách.

Those of you who are C++ programmers will recognize that the C++ standard template library does not allow you to query and alter a container in a single statement. In that library the above pop operation requires two statements that conform to CQS:

Những ai trong các bạn là lập trình viên C++ hẳn sẽ nhận ra rằng thư viện standard template library của C++ không cho phép bạn vừa truy vấn vừa thay đổi một container trong cùng một câu lệnh. Trong thư viện đó, thao tác pop nói trên đòi hỏi hai câu lệnh tuân thủ CQS:

```
Thing top = myStack.top();  
myStack.pop();
```

In C++, the reason for this, aside from the concurrency argument, is that the Thing class may override the assignment operator, ³ and it might therefore throw an **exception**. If the pop operation both changed the stack and returned the top, then when an exception was thrown by the assignment operator it could leave the stack in an unknown state. ³ That's right, folks, in C++ you can override the = operator for the assignment of objects. So, although I cannot claim to use CQS in every circumstance, I take the advice seriously and violate it only after considering the possible outcomes.

Trong C++, lý do của chuyện này, ngoài lập luận về tính đồng thời (concurrency), là class Thing có thể override toán tử gán, ³ và vì vậy nó có thể ném ra exception. Nếu thao tác pop vừa thay đổi stack vừa trả về phần tử trên đỉnh, thì khi toán tử gán ném exception, nó có thể để stack lại trong một trạng thái không xác định. ³ Đúng thế đấy các bạn, trong C++ bạn có thể override toán tử = dùng cho việc gán đối tượng. Vậy nên, dù không dám nói rằng mình áp dụng CQS trong mọi hoàn cảnh, tôi vẫn xem lời khuyên này là nghiêm túc và chỉ vi phạm nó sau khi đã cân nhắc những hệ quả có thể xảy ra.

Prefer Exceptions to Returning Error Codes — Hãy ưu tiên exception hơn là trả về mã lỗi

Exceptions can be a tricky matter. In most of our modern languages, they are a reliable way to transmit error information up the calling stack. In older languages, most notably C++, that reliability is not guaranteed. And in some languages, like Go, exceptions just don't exist.

Exception có thể là một chuyện rắc rối. Trong hầu hết các ngôn ngữ hiện đại, chúng là cách đáng tin cậy để chuyển thông tin lỗi ngược lên call stack. Ở các ngôn ngữ cũ hơn, đáng chú ý nhất là C++, độ tin cậy đó không được bảo đảm. Còn ở một số ngôn ngữ, như Go, exception đơn giản là không tồn tại.

Caveat Emptor — Liệu mà dùng

In general, I prefer exceptions to error codes where possible. Returning error codes from command functions is a subtle violation of **command query separation**. It promotes commands being used as expressions in the predicates of if statements.

Nhìn chung, tôi thích exception hơn mã lỗi ở những nơi có thể. Trả về mã lỗi từ các hàm lệnh là một vi phạm tinh vi của việc tách lệnh khỏi truy vấn. Nó khuyến khích việc dùng lệnh như biểu thức trong phần điều kiện của câu lệnh if.

```
if (deletePage(page) == E_OK)
```

This does not suffer from verb/adjective confusion but does lead to deeply nested structures. When you return an **error code**, you create the problem that the caller must deal with the error immediately.

Cách này không mắc phải sự nhập nhằng động từ/tính từ, nhưng lại dẫn tới những cấu trúc lồng nhau rất sâu. Khi bạn trả về một mã lỗi, bạn tạo ra vấn đề là bên gọi phải xử lý lỗi đó ngay lập tức.

```
if (deletePage(page) == E_OK) {
    if (registry.deleteReference(page.name) == E_OK) {
        if (configKeys.deleteKey(page.name.makeKey()) == E_OK){
            logger.log("page deleted");
            return E_OK;
        } else {
            logger.log("configKey not deleted");
            return E_ERROR;
        }
    } else {
        logger.log("deleteReference from registry failed");
        return E_ERROR;
    }
} else {
    logger.log("delete failed");
    return E_ERROR;
}
```

Programmers sometimes use guards to pretend that the **structure** is not deeply nested. But the lack of those else clauses doesn't mean they aren't actually there.

Lập trình viên đôi khi dùng guard để giả vờ rằng cấu trúc không hề lồng sâu. Nhưng việc thiếu vắng những mệnh đề else đó không có nghĩa là chúng không thực sự tồn tại.

```
if (deletePage(page) != E_OK) {
    logger.log("delete failed");
    return E_ERROR;
}
if (registry.deleteReference(page.name) != E_OK) {
    logger.log("deleteReference from registry failed");
    return E_ERROR;
}
if (configKeys.deleteKey(page.name.makeKey()) != E_OK){
    logger.log("configKey not deleted");
}
```

```

    return E_ERROR;
}
logger.log("page deleted");
return E_OK;

```

On the other hand, if you use exceptions instead of returned error codes, then the error processing code can be separated from the happy path code and can be simplified:

Mặt khác, nếu bạn dùng exception thay cho mã lỗi trả về, thì phần mã xử lý lỗi có thể được tách khỏi phần mã đi theo đường thuận lợi (happy path) và có thể được đơn giản hoá:

```

try {
    deletePage(page);
    registry.deleteReference(page.name);
    configKeys.deleteKey(page.name.makeKey());
}
catch (Exception e) {
    logger.log(e.getMessage());
}

```

Extract / Blocks — Extract các khối

Try/catch blocks are ugly in their own right. They confuse the structure of the code and mix error processing with normal processing. So it is better to extract the bodies of the try and catch blocks out into functions of their own.

Bản thân các khối try/catch đã xấu xí. Chúng làm rối cấu trúc của mã và trộn lẫn xử lý lỗi với xử lý thông thường. Vậy nên tốt hơn là extract phần thân của khối try và khối catch ra thành những hàm riêng.

```

public void delete(Page page) {
    try {
        deletePageAndAllReferences(page);
    }
    catch (Exception e) {
        logError(e);
    }
}
private void deletePageAndAllReferences(Page page) throws Exception {
    deletePage(page);
    registry.deleteReference(page.name);
    configKeys.deleteKey(page.name.makeKey());
}
private void logError(Exception e) {
    logger.log(e.getMessage());
}

```

In the above, the delete function is all about error processing. It is easy to understand and then ignore. The deletePageAndAllReferences function is all about the processes of fully deleting a page . Error handling can be ignored. This provides a nice separation that makes the code easier to understand and modify.

Trong đoạn trên, hàm delete chỉ lo mỗi việc xử lý lỗi. Nó dễ hiểu, và hiểu xong thì có thể bỏ

qua. Hàm `deletePageAndAllReferences` chỉ lo các bước xoá một page cho trọn vẹn. Chuyện xử lý lỗi có thể bỏ qua. Điều này tạo ra một sự phân tách gọn gàng khiến mã dễ hiểu và dễ sửa hơn.

Error Handling Is One Thing — Xử lý lỗi là một việc

As we have learned, functions should do one thing. Error handling is one thing. Thus, a function that handles errors should do nothing else. This implies (as in the example above) that if the keyword `try` exists in a function, it should be the very first word in the function (after, perhaps, some **variable** declarations) and that there should be nothing after the `catch` / `finally` blocks.

Như ta đã học, hàm nên làm một việc. Xử lý lỗi là một việc. Vậy nên một hàm xử lý lỗi thì không nên làm gì khác. Điều này hàm ý rằng (như trong ví dụ trên) nếu từ khoá `try` xuất hiện trong một hàm thì nó phải là từ đầu tiên trong hàm đó (có lẽ sau vài khai báo biến), và không được có gì nữa sau các khối `catch` / `finally`.

The Dependency Magnet of Error Codes — Mã lỗi — thói nam châm hút phụ thuộc

If you must return error codes, be careful not to turn them into a **dependency** magnet.

Nếu buộc phải trả về mã lỗi, hãy cẩn thận đừng biến chúng thành một thói nam châm hút phụ thuộc (dependency magnet).

In C, we used to create the header file `error.h`. It would look like this (though it would usually be much longer):

Ngày xưa trong C, chúng tôi thường tạo file header `error.h`. Nó trông như thế này (dù thường sẽ dài hơn nhiều):

```
#define E_OK 0
#define E_INVALID 1
#define E_NO_SUCH 2
#define E_LOCKED 3
#define E_OUT_OF_RESOURCES 4
```

Every **module** in the system would `#include error.h`. Anytime a new error code was added to `error.h`, every module in the system had to be recompiled—because the build system used the dates of the source files to determine which were out of date and needed recompilation.

Mọi module trong hệ thống đều `#include error.h`. Mỗi lần thêm một mã lỗi mới vào `error.h` là mọi module trong hệ thống lại phải biên dịch lại — bởi hệ thống build dùng ngày tháng của các file mã nguồn để xác định file nào đã cũ và cần biên dịch lại.

This put a negative pressure on the error codes. Programmers didn't want to add new errors, because then they had to rebuild and redeploy everything, and in those days a rebuild could take hours. So they reused old error codes in “creative” ways instead of adding new ones. And, of course, that led to confusion, disillusion, and heartache.

Điều này tạo ra một áp lực tiêu cực lên mã lỗi. Lập trình viên chẳng muốn thêm lỗi mới, vì thêm là phải build lại và triển khai lại toàn bộ, mà hồi đó một lần build lại có thể mất hàng giờ. Thế là thay vì thêm mã mới, họ tái sử dụng những mã lỗi cũ theo những cách đầy “sáng tạo”. Và tất nhiên, chuyện đó dẫn tới rối rắm, vỡ mộng và đau khổ.

Modern build environments can be smarter than that. For example, in Java you might use enums in a file named `Error.java` :

Môi trường build hiện đại có thể thông minh hơn thế. Chẳng hạn, trong Java bạn có thể dùng enum trong một file tên là `Error.java` :

```
public enum Error {  
    OK,  
    INVALID,  
    NO_SUCH,  
    LOCKED,  
    OUT_OF_RESOURCES;  
}
```

A file like this is still a dependency magnet because many other classes will import and use it. However, when programmers add a new error code to `Error.java` , only those files that are directly impacted by the change will need to be recompiled and redeployed. Files that simply use the previously existing error codes will not.

Một file như vậy vẫn là một thỏi nam châm hút phụ thuộc, vì nhiều class khác sẽ import và dùng nó. Tuy nhiên, khi lập trình viên thêm một mã lỗi mới vào `Error.java` , chỉ những file bị ảnh hưởng trực tiếp bởi thay đổi đó mới cần biên dịch lại và triển khai lại. Những file chỉ dùng các mã lỗi đã có từ trước thì không.

So it's a **good** idea to know how your build system works, and whether dependency magnets will cause thrashing of the build and require redeployment.

Vậy nên hiểu rõ hệ thống build của mình hoạt động ra sao, và liệu các thỏi nam châm hút phụ thuộc có gây ra cảnh build quay cuồng và bắt phải triển khai lại hay không, là điều nên làm.

In languages like Go, `Error` is an **interface**, and any error codes that the programmers want to add implement that interface. That means they can be kept in separate source files and run no risk of becoming a dependency magnet.

Ở những ngôn ngữ như Go, `Error` là một interface, và mọi mã lỗi mà lập trình viên muốn thêm đều hiện thực interface đó. Nghĩa là chúng có thể nằm ở những file mã nguồn riêng biệt và chẳng có nguy cơ nào trở thành thỏi nam châm hút phụ thuộc.

Of course, this approach can be used in any language that supports interfaces, or an equivalent kind of dynamic **polymorphism**. For example, error codes can be independent duck-type classes in Python.

Tất nhiên, cách tiếp cận này dùng được ở bất kỳ ngôn ngữ nào hỗ trợ interface, hoặc một dạng đa hình (polymorphism) động tương đương. Chẳng hạn, trong Python mã lỗi có thể là các class duck-type độc lập.

DRY: Don't Repeat Yourself — DRY: Đừng lặp lại chính mình

We have long felt that **duplication** of code should be avoided. We've heard this advice many times over the years. The authors of *The Pragmatic Programmer* 4 told us this way back in 2000. But they were certainly not the first. Most of Codd's normal forms for organizing databases are techniques for reducing duplication in the data. Bertrand Meyer told us this way back in 1988 when he wrote about the Single Choice Principle. 5 Suffice it to say that the advice is old. 4 [Prag]. 5 [OOSC].

Từ lâu chúng ta đã cảm thấy nên tránh trùng lặp (duplication) mã. Suốt bao năm qua ta đã nghe lời khuyên này không biết bao nhiêu lần. Các tác giả của *The Pragmatic Programmer* 4 đã nói với ta điều đó từ tận năm 2000. Nhưng chắc chắn họ không phải là những người đầu tiên. Hầu hết các dạng chuẩn (normal form) của Codd để tổ chức cơ sở dữ liệu đều là kỹ thuật nhằm giảm trùng lặp trong dữ liệu. Bertrand Meyer đã nói với ta điều đó từ tận năm 1988 khi ông viết về Single Choice Principle. 5 Chỉ cần nói vậy là đủ để thấy lời khuyên này đã cũ lắm rồi. 4 [Prag]. 5 [OOSC].

Simple Repeated Code — Mã lặp lại đơn giản

So, what does it mean to duplicate or repeat code? There's the obvious case where the same L lines of code are repeated over and over in P different places. This is often the result of a programmer being a bit overanxious with the mouse. If both L and P are small, then very little **harm** is done by the duplication, and the effort to eliminate that duplication may do more harm than good.

Vậy trùng lặp (duplication) hay lặp lại mã thực ra nghĩa là gì? Có một trường hợp hiển nhiên: cùng L dòng mã được lặp đi lặp lại ở P chỗ khác nhau. Chuyện này thường là hậu quả của việc lập trình viên hơi quá hăng tay với con chuột. Nếu cả L lẫn P đều nhỏ thì sự trùng lặp đó gây hại chẳng bao nhiêu, và công sức bỏ ra để dẹp nó có khi còn hại nhiều hơn lợi.

For example, if you were to see these two lines repeating twice, it would probably not make much sense to turn them into a function just for the purpose of removing the duplication.

Chẳng hạn, nếu bạn thấy hai dòng này lặp lại đúng hai lần, thì việc gom chúng thành một hàm chỉ để xóa trùng lặp có lẽ chẳng có mấy ý nghĩa.

```
count=0  
list=[]
```

You might extract them into the `initializeList` function as a convenience to your readers. But that's a different matter. The duplication itself is not very harmful.

Bạn có thể tách chúng ra thành hàm `initializeList` cho tiện với người đọc. Nhưng đó lại là chuyện khác. Bản thân sự trùng lặp ở đây không mấy tai hại.

What if L is likely to grow? For example, what if you need to add a new variable to that initialization?

Thế còn nếu L có khả năng phình ra thì sao? Ví dụ, nếu bạn cần thêm một biến mới vào đoạn khởi tạo đó?

```
count=0  
odds=0  
list=[]
```

If P is 2, then you have to make that change in two places. That might be a little inconvenient, but it's a minor issue.

Nếu P bằng 2 thì bạn phải sửa ở hai chỗ. Hơi bất tiện, nhưng chỉ là chuyện nhỏ.

On the other hand, if P is large, then you'd have to find all the duplicated instances and add the new initialization. If, on the other hand, you had extracted the initialization out into the `initializeList` function, you could make that change in one place.

Ngược lại, nếu P lớn thì bạn phải đi lùng cho ra tất cả các bản sao rồi thêm đoạn khởi tạo mới vào từng chỗ. Còn nếu bạn đã tách phần khởi tạo ra thành hàm `initializeList` thì

chỉ cần sửa đúng một nơi.

So, if P is large, even if L is small, it is probably best to extract the duplication into a single, nicely named function.

Vậy nên, nếu P lớn thì dù L có nhỏ, tốt nhất vẫn là tách chỗ trùng lặp ra thành một hàm duy nhất với cái tên tử tế.

What are the odds that a snippet of L lines will need modification later? That's probably a function of L . The larger L is, the larger the odds that the snippet will need to be changed.

Khả năng một đoạn mã dài L dòng sẽ cần sửa về sau là bao nhiêu? Điều đó nhiều khả năng là một hàm của L . L càng lớn thì xác suất đoạn mã ấy phải thay đổi càng cao.

Thus, when L is large, even if P is small, it probably pays to extract the duplication.

Do đó, khi L lớn thì dù P có nhỏ, việc tách chỗ trùng lặp vẫn thường đáng công.

We can describe this with this table.

Ta có thể mô tả điều này bằng bảng sau.

	L is small.	L is large.
P is small.	Maybe extract.	Extract
P is large.	Extract	Extract

Similar Code — Mã na ná nhau

Often we will find repeated snippets of code that are not perfectly duplicated. They may differ by one or two constants or variables. Such snippets often tend to have a larger L , and so should be extracted.

Ta thường bắt gặp những đoạn mã lặp lại nhưng không hoàn toàn giống hệt nhau. Chúng có thể khác nhau ở một hai hằng số hoặc biến. Những đoạn như vậy thường có L khá lớn, và vì thế nên được tách ra.

Here's an example from the first edition of this book—and one that was in the FitNesse 6 project circa 2001. The snippets in bold are similar and could be extracted.

Đây là một ví dụ lấy từ lần xuất bản đầu tiên của cuốn sách này — cũng là một đoạn từng nằm trong dự án FitNesse 6 vào khoảng năm 2001. Các đoạn in đậm na ná nhau và có thể tách ra được.

6 <http://fitnesse.org> ; <https://github.com/unclebob/fitnesse>

6 <http://fitnesse.org> ; <https://github.com/unclebob/fitnesse>

```
public class TestableHtml {
    public static String testableHtml(
        PageData pageData,
        boolean includeSuiteSetup) throws Exception {
        WikiPage wikiPage = pageData.getWikiPage();
        StringBuffer buffer = new StringBuffer();
```

```

if (pageData.hasAttribute("Test")) {
    if (includeSuiteSetup) {
        WikiPage suiteSetup =
            PageCrawlerImpl.getInheritedPage(
                SuiteResponder.SUITE_SETUP_NAME, wikiPage);
        if (suiteSetup != null) {
            WikiPagePath pagePath =
                suiteSetup.getPageCrawler().getFullPath(suiteSetup);
            String pagePathName = PathParser.render(pagePath);
            buffer.append("!include -setup .")
                .append(pagePathName)
                .append("\n");
        }
    }
}

WikiPage setup =
    PageCrawlerImpl.getInheritedPage("SetUp", wikiPage);
if (setup != null) {
    WikiPagePath setupPath =
        setup.getPageCrawler().getFullPath(setup);
    String setupPathName = PathParser.render(setupPath);
    buffer.append("!include - setup.")
        .append(setupPathName)
        .append("\n");
}

buffer.append(pageData.getContent());
if (pageData.hasAttribute("Test")) {
    WikiPage teardown =
        PageCrawlerImpl.getInheritedPage("TearDown", wikiPage);
    if (teardown != null) {
        WikiPagePath tearDownPath =
            teardown.getPageCrawler().getFullPath(teardown);
        String tearDownPathName = PathParser.render(tearDownPath);
        buffer.append("\n")
            .append("!include - teardown.")
            .append(tearDownPathName)
            .append("\n");
    }
}

if (includeSuiteSetup) {
    WikiPage suiteTeardown =
        PageCrawlerImpl.getInheritedPage(
            SuiteResponder.SUITE_TEARDOWN_NAME,
            wikiPage
        );
    if (suiteTeardown != null) {
        WikiPagePath pagePath =
            suiteTeardown.getPageCrawler().getFullPath(suiteTeardown);
        String pagePathName = PathParser.render(pagePath);
        b ffd("!il dtd")

```



```

        buffer.append("!include -teardown .")
            .append(pagePathName)
            .append("\n");
    }
}
}
pageData.setContent(buffer.toString());
return pageData.getHtml();
}
}

```

Usually the extraction of similar snippets like this simply involves adding an argument or two to the extracted function. Let's try this.

Thông thường, việc tách những đoạn na ná nhau kiểu này chỉ đơn giản là thêm một hai đối số vào hàm được tách ra. Ta thử xem.

```

public class TestableHtml {
    public static String testableHtml(
        PageData pageData,
        boolean includeSuiteSetup) throws Exception {
        WikiPage wikiPage = pageData.getWikiPage();
        StringBuffer buffer = new StringBuffer();
        if (pageData.hasAttribute("Test")) {
            if (includeSuiteSetup) {
                includeInheritedPage(SuiteResponder.SUITE_SETUP_NAME,
                    wikiPage, buffer, "!include -setup .");
            }
            includeInheritedPage("SetUp", wikiPage, buffer,
                "!include -setup .");
        }
        buffer.append(pageData.getContent());
        if (pageData.hasAttribute("Test")) {
            includeInheritedPage("TearDown", wikiPage, buffer,
                "\n!include -teardown .");
            if (includeSuiteSetup) {
                includeInheritedPage(SuiteResponder.SUITE_TEARDOWN_NAME,
                    wikiPage, buffer, "!include -teardown .");
            }
        }
        pageData.setContent(buffer.toString());
        return pageData.getHtml();
    }
    private static void
    includeInheritedPage(String pageName, WikiPage wikiPage,
        StringBuffer buffer,
        String includeString) {
        WikiPage inheritedPage =
            PageCrawlerImpl.getInheritedPage(pageName, wikiPage);
        if (inheritedPage != null) {
            WikiPagePath pagePath =

```

```

        inheritedPage.getPageCrawler().getFullPath(inheritedPage);
        String pagePathName = PathParser.render(pagePath);
        buffer.append(includeString)
            .append(pagePathName)
            .append("\n");
    }
}
}

```

This is clearly better. It vastly improves the flow of the `testableHtml` function and it makes sure that any modifications to `includeInheritedPage` will be in one place instead of four. However, the number of arguments to our new function is larger than I like. So let's move some of those variables into fields.

Rõ ràng là khá hơn hẳn. Nó cải thiện rất nhiều mạch chảy của hàm `testableHtml`, và bảo đảm rằng mọi thay đổi đối với `includeInheritedPage` sẽ nằm ở một chỗ thay vì bốn chỗ. Tuy nhiên, số đối số của hàm mới này nhiều hơn mức tôi ưa. Vậy ta hãy chuyển bớt vài biến đó thành field.

```

public class TestableHtml {
    private static StringBuffer buffer;
    private static WikiPage wikiPage;
    public static String testableHtml(
        PageData pageData,
        boolean includeSuiteSetup) throws Exception {
        wikiPage = pageData.getWikiPage();
        buffer = new StringBuffer();

        if (pageData.hasAttribute("Test")) {
            if (includeSuiteSetup) {
                includeInheritedPage(SuiteResponder.SUITE_SETUP_NAME,
                    "!include -setup .");
            }
            includeInheritedPage("SetUp", "!include -setup .");
        }
        buffer.append(pageData.getContent());
        if (pageData.hasAttribute("Test")) {
            includeInheritedPage("TearDown", "\n!include -teardown .");
            if (includeSuiteSetup) {
                includeInheritedPage(SuiteResponder.SUITE_TEARDOWN_NAME,
                    "!include -teardown .");
            }
        }
        pageData.setContent(buffer.toString());
        return pageData.getHtml();
    }
    private static void
    includeInheritedPage(String pageName, String includeString) {
        WikiPage inheritedPage =
            PageCrawlerImpl.getInheritedPage(pageName, wikiPage);
        if (inheritedPage != null) {
            WikiPagePath pagePath =

```

```

        inheritedPage.getPageCrawler().getFullPath(inheritedPage);
        String pagePathName = PathParser.render(pagePath);
        buffer.append(includeString)
            .append(pagePathName)
            .append("\n");
    }
}
}

```

This is nicer still. If you look closely at the result, you ought to be able to detect more duplication. I'll leave the elimination of that to you.

Còn đẹp hơn nữa. Nếu nhìn kỹ kết quả, hẳn bạn sẽ phát hiện thêm chỗ trùng lặp. Việc đẹp nốt chỗ đó tôi xin nhường lại cho bạn.

Loop Duplication — Trùng lặp vòng lặp

Often we find that there are data structures within our application that we must traverse in many different places. The loop that does the traversal is always the same, but the body of that loop is always different.

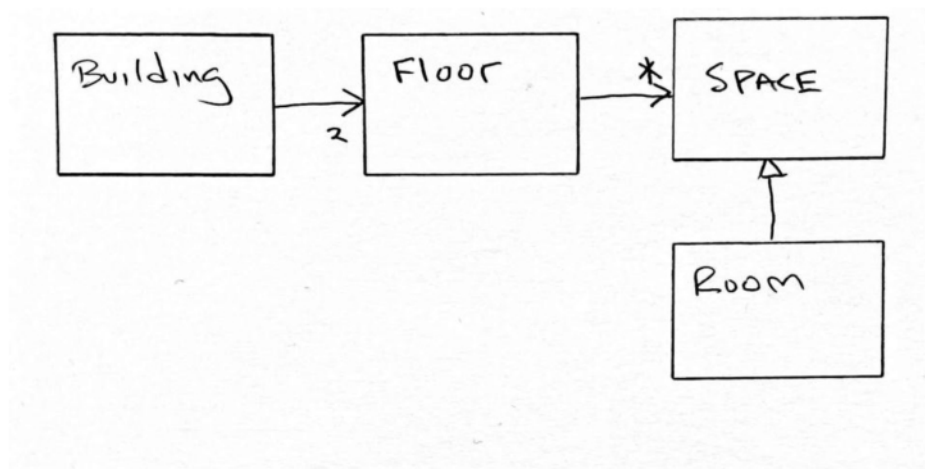
Ta thường thấy trong ứng dụng có những cấu trúc dữ liệu (data structure) mà ta phải duyệt qua ở rất nhiều nơi khác nhau. Vòng lặp làm nhiệm vụ duyệt thì lúc nào cũng y hệt, nhưng thân vòng lặp thì lần nào cũng khác.

Nowadays, the popular way to eliminate that duplication is to use a lambda. Older, and sometimes better options are to use the Command pattern, the Strategy pattern, or the TemplateMethod pattern.

Ngày nay, cách phổ biến để dẹp kiểu trùng lặp đó là dùng lambda. Những lựa chọn cũ hơn — và đôi khi hay hơn — là dùng mẫu Command, mẫu Strategy hoặc mẫu TemplateMethod.

7 [GOF95]. As an example of these techniques, consider a problem my team and I faced way back in the late '90s. We had a **data structure** that described the internals of a building. Below is a vastly simplified diagram of that data structure.

7 [GOF95]. Để minh hoạ cho những kỹ thuật này, hãy xét một bài toán mà tôi và đội của mình đối mặt từ hồi cuối thập niên 1990. Chúng tôi có một cấu trúc dữ liệu mô tả nội thất bên trong một toà nhà. Dưới đây là sơ đồ đã được đơn giản hoá hết mức của cấu trúc dữ liệu ấy.



Our task was to determine several dozen different scorable features of the building by inspecting each of the floors and spaces (and doors and windows and hallways, but never mind that). The scores for these features were determined by a set of C++ classes, each of which had to walk the data structure in order to perform the calculation.

Nhiệm vụ của chúng tôi là xác định vài chục đặc tính tính điểm được của toà nhà bằng cách khảo sát từng tầng và từng không gian (và cả cửa ra vào, cửa sổ, hành lang nữa, nhưng thôi bỏ qua chuyện đó). Điểm của các đặc tính này được tính bởi một tập các class C++, mà mỗi class đều phải đi bộ qua cấu trúc dữ liệu để thực hiện phép tính.

Below is some Ruby code that shows the duplication that could have resulted from all those features.

Dưới đây là một ít mã Ruby cho thấy sự trùng lặp có thể sinh ra từ đồng đặc tính đó.

```

class SpaceFeature
  def score(building)
    spaces = 0
    building.floors.each do |floor|
      floor.spaces.each do |space|
        spaces += 1
      end
    end
    spaces
  end
end

class AreaFeature
  def score(building)
    area = 0
    building.floors.each do |floor|
      floor.spaces.each do |space|
        area += space.width * space.length
      end
    end
    area
  end
end

```

You can see the problem. The nested loops are identical, but the body of the loop is different in each

case. Now imagine this repeated several dozen times with a much more complicated data structure and traversal loop.

Bạn thấy vấn đề rồi đấy. Các vòng lặp lồng nhau giống hệt nhau, chỉ thân vòng lặp là khác ở mỗi trường hợp. Giờ hãy tưởng tượng cảnh này lặp lại vài chục lần với một cấu trúc dữ liệu và vòng lặp duyệt phức tạp hơn nhiều.

We can address this in modern languages by using a lambda (sometimes called a block) as follows.

Trong các ngôn ngữ hiện đại, ta có thể xử lý chuyện này bằng lambda (đôi khi gọi là *block*) như sau.

```
class Feature
  def do_score(building, block)
    value = 0
    building.floors.each do |floor|
      floor.spaces.each do |space|
        value = block.call(value, floor, space)
      end
    end
    value
  end
end

class SpaceFeature < Feature
  def score(building)
    do_score(building, ->(value, floor, space) {return value+1})
  end
end

class AreaFeature < Feature
  def score(building)
    do_score(building, ->(value, floor, space)
      {return value + (space.width * space.length)})
  end
end
```

Now the loop that traverses the data structure exists only in the Feature class. If any changes were made to that data structure, the odds are that only that class would need to be changed. The specific features, like SpaceFeature and AreaFeature , would likely remain unaffected by such changes.

Giờ thì vòng lặp duyệt cấu trúc dữ liệu chỉ tồn tại trong class Feature. Nếu cấu trúc dữ liệu ấy có thay đổi gì, nhiều khả năng chỉ mình class đó cần sửa. Các đặc tính cụ thể như SpaceFeature và AreaFeature gần như chắc chắn sẽ không bị ảnh hưởng.

Another technique would have been to use the TemplateMethod pattern.

Một kỹ thuật khác là dùng mẫu TemplateMethod.

```
class Feature
  def score(building)
    value = 0
    building.floors.each do |floor|
      floor.spaces.each do |space|
        value = score_space(value, floor, space)
      end
    end
  end
```

```

    end
    value
  end
  def score_space(value, floor, space)
    raise "Not Implemented"
  end
end

class SpaceFeature < Feature
  def score_space(value, floor, space)
    value+1
  end
end

class AreaFeature < Feature
  def score_space(value, floor, space)
    value + (space.width * space.length)
  end
end

```

I actually like this better than the lambda solution—and it’s the one we used way back in the ’90s in C++. Back then we called it “Write a Loop Once” because the Design Patterns book had not yet been published.

Thật ra tôi thích cách này hơn cách dùng lambda — và đây cũng chính là cách chúng tôi đã dùng hồi thập niên 1990 với C++. Hồi ấy chúng tôi gọi nó là “Write a Loop Once” vì cuốn *Design Patterns* còn chưa ra đời.

I’ll leave you to check out the Strategy and Command patterns for yourself.

Còn mẫu Strategy và Command thì tôi để bạn tự tìm hiểu.

Accidental versus Essential Duplication — Trùng lặp ngẫu nhiên và trùng lặp bản chất

So far we have been talking about essential duplication. But sometimes two stretches of code appear to be duplicates, when in fact they are not. Those are accidental duplicates.

Từ đầu tới giờ ta vẫn đang nói về trùng lặp bản chất (essential duplication). Nhưng đôi khi hai đoạn mã trông có vẻ trùng lặp mà thực ra thì không. Đó là trùng lặp ngẫu nhiên (accidental duplication).

Essential duplicates change together. When one changes, they all change. They evolve together.

Trùng lặp bản chất thì thay đổi cùng nhau. Một chỗ đổi là tất cả cùng đổi. Chúng tiến hoá cùng nhau.

Accidental duplicates do not change together. They evolve apart from each other. So it is very important not to combine them into a common function.

Trùng lặp ngẫu nhiên thì không thay đổi cùng nhau. Chúng tiến hoá theo những hướng khác nhau. Vì vậy tuyệt đối đừng gộp chúng vào một hàm chung.

You can tell an accidental duplication from an essential duplication by considering the Single Responsibility Principle (SRP). 8 If two similar stretches of code are in two modules that are responsible to different actors, then they are likely to evolve separately as those actors ask for

different changes. If you extract them into a common function, then you are likely to break the system for one actor while you try to satisfy the other. 8 See Chapter 19 , “ **The SOLID Principles** .” On the other hand, if two similar stretches of code are in modules responsible to the same actor, then they are likely to evolve together. If you forget to change one of those stretches, then you will break the system for that actor.

Bạn có thể phân biệt trùng lặp ngẫu nhiên với trùng lặp bản chất bằng cách xét Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle — SRP). 8 Nếu hai đoạn mã na ná nhau nằm trong hai module chịu trách nhiệm trước hai actor khác nhau, thì nhiều khả năng chúng sẽ tiến hoá tách biệt khi các actor đó yêu cầu những thay đổi khác nhau. Nếu bạn tách chúng ra thành một hàm chung, rất có thể bạn sẽ làm hỏng hệ thống với actor này trong lúc cố chiều lòng actor kia. 8 Xem Chương 19, “Các nguyên tắc SOLID”. Ngược lại, nếu hai đoạn mã na ná nhau nằm trong các module cùng chịu trách nhiệm trước một actor, thì nhiều khả năng chúng sẽ tiến hoá cùng nhau. Nếu bạn quên sửa một trong hai đoạn đó, bạn sẽ làm hỏng hệ thống với chính actor ấy.

Side Effects — Tác dụng phụ

In the years since the first edition of this book was written, and with the rise of functional programming, the definition of the term side effect has gone through a transition. Twenty years ago most programmers thought that a function with a side effect had a primary purpose but then also did something else on the side. The example I used in the first edition was a function that checked the validity of a login password but then also initialized the current session if the password was found to be valid.

Trong những năm kể từ khi lần xuất bản đầu tiên của cuốn sách này ra đời, cùng với sự trỗi dậy của lập trình hàm, định nghĩa của thuật ngữ tác dụng phụ (side effect) đã trải qua một cuộc chuyển mình. Hai mươi năm trước, phần lớn lập trình viên cho rằng một hàm có tác dụng phụ là hàm có mục đích chính của nó, nhưng nhân tiện lại làm thêm một việc gì đó bên lề. Ví dụ tôi dùng trong lần xuất bản đầu là một hàm kiểm tra tính hợp lệ của mật khẩu đăng nhập nhưng đồng thời cũng khởi tạo luôn phiên làm việc hiện tại nếu mật khẩu hợp lệ.

Since then, the definition has been generalized. Nowadays, we define a side effect to be any state change that outlives the function, even if that state change is the primary intent of the function. In other words, a function with a side effect is impure.

Kể từ đó, định nghĩa này đã được tổng quát hoá. Ngày nay ta định nghĩa tác dụng phụ là bất kỳ thay đổi trạng thái nào tồn tại lâu hơn bản thân hàm, kể cả khi thay đổi trạng thái đó chính là ý định chính của hàm. Nói cách khác, một hàm có tác dụng phụ là hàm không thuần khiết (impure).

So, for example, if `stack.pop()` removes the first item on the top of the stack , then it leaves that stack in an altered state; the alteration of that state is a side effect.

Chẳng hạn, nếu `stack.pop()` gỡ phần tử trên đỉnh stack ra, thì nó để lại stack ở một trạng thái đã bị thay đổi; sự thay đổi trạng thái ấy chính là một tác dụng phụ.

Side effects create complications for software systems because they introduce an insidious form of **coupling** called temporal coupling —coupling in time. A statement with a side effect separates the code above it from the code below it with a change of state. That change of state means that the order of those lines of code must be preserved.

Tác dụng phụ gây rắc rối cho các hệ thống phần mềm vì chúng đưa vào một dạng kết dính ngầm ngầm gọi là *kết dính thời gian* (temporal coupling) — kết dính về mặt thời gian. Một câu lệnh có tác dụng phụ chia cắt phần mã phía trên nó khỏi phần mã phía dưới nó bằng một thay đổi trạng thái. Thay đổi trạng thái đó có nghĩa là thứ tự của những dòng mã ấy phải được giữ nguyên.

So you can't close a file until you've opened it, and you can't open it if it's already open. You can't release a semaphore until you've seized it. You can't free a block before you've allocated it. You can't release a graphics context until you've taken possession of it. Et cetera.

Vậy nên bạn không thể đóng một tệp trước khi mở nó, và cũng không thể mở nó nếu nó đang mở sẵn. Bạn không thể nhả một semaphore trước khi giành lấy nó. Bạn không thể giải phóng một khối bộ nhớ trước khi cấp phát nó. Bạn không thể trả lại một graphics context trước khi chiếm giữ nó. Vân vân và vân vân.

Side effect functions often come in pairs, like open/close , seize/release , or malloc/free . One creates the side effect and the other undoes it. So side effects are like The Sith: “always two there are.”

Các hàm có tác dụng phụ thường đi thành cặp, kiểu như open/close, seize/release, hay malloc/free. Một cái tạo ra tác dụng phụ, cái kia gỡ nó đi. Vậy nên tác dụng phụ cũng giống người Sith: “luôn luôn có hai, không hơn.”

We Are Bad at This — Chúng ta rất dở khoản này

The proof that we are bad at managing side effects is the fact that almost all of our major languages now feature garbage collection . Garbage collection has been added to our languages to cure the fact that we are just so bloody awful at balancing malloc with free . We've screwed this up so often, and relegated so many systems to be rebooted every midnight, that we finally created this horrible hack of a feature as a desperate measure to relieve us from the burden of cleaning up memory side effects.

Bằng chứng cho thấy chúng ta quản lý tác dụng phụ rất dở chính là việc gần như mọi ngôn ngữ lớn hiện nay đều có *garbage collection*. Garbage collection được thêm vào các ngôn ngữ để chữa cái sự thật rằng chúng ta cân đối malloc với free dở tệ đến phát khiếp. Chúng ta làm hỏng chuyện này nhiều đến mức, và đẩy bao nhiêu hệ thống vào cảnh cứ nửa đêm là phải khởi động lại, đến nỗi cuối cùng phải chế ra cái tính năng chấp vá kinh khủng ấy như một biện pháp tuyệt vọng để tự giải thoát khỏi gánh nặng dọn dẹp tác dụng phụ về bộ nhớ.

How many languages do you know that have gone to great lengths to help programmers manage the ownership of memory blocks? How many have used reference counting, or RAII, 9 or God knows what? And why? Because we poor, feeble programmers cannot seem to figure out how to return the books we've borrowed from the library. 9 Resource Acquisition Is Initialization—the key insight behind the old auto_ptr (now unique_ptr and shared_ptr) facilities in C++. The problem is that facilities like garbage collection only address one of the many Sith-like creatures that we must deal with. We don't have garbage collection for file descriptors, or graphics contexts, or semaphores, or . . . Well, I think you get the point. We are bad at this.

Bạn biết bao nhiêu ngôn ngữ đã lặn lội đủ đường để giúp lập trình viên quản lý quyền sở hữu các khối bộ nhớ? Bao nhiêu ngôn ngữ đã dùng đếm tham chiếu, hay RAII, 9 hay có trời mới biết là gì nữa? Và để làm gì? Bởi vì cánh lập trình viên khổ khổ, yếu ớt chúng ta hình như không tài nào nghĩ ra cách trả lại sách đã mượn của thư viện. 9 Resource Acquisition Is Initialization — cái nhìn cốt lõi đứng sau các tiện ích auto_ptr cũ (nay là unique_ptr và shared_ptr) trong C++. Vấn đề là những tiện ích như garbage collection

chỉ xử lý được đúng một trong vô số sinh vật kiểu Sith mà ta phải đối phó. Ta có garbage collection cho file descriptor không? Cho graphics context? Cho semaphore? Hay... Thôi, tôi nghĩ bạn hiểu ý rồi. Chúng ta rất dở khoản này.

When order matters, temporal couplings can leak out into other parts of the code, creating situations that are as difficult to debug as a memory leak. I'm sure you've had the experience of debugging a misbehaving system only to find that if you swap the order of two adjacent function calls, the system suddenly works—and you have no idea why.

Khi thứ tự có ý nghĩa, các kết dính thời gian có thể rò rỉ ra những phần khác của mã, tạo nên những tình huống khó gỡ chẳng kém gì rò rỉ bộ nhớ. Tôi chắc bạn từng nếm trải cảnh gỡ lỗi một hệ thống dở chứng, để rồi phát hiện rằng chỉ cần hoán đổi thứ tự hai lời gọi hàm nằm cạnh nhau là hệ thống bỗng dưng chạy được — mà bạn chẳng hiểu vì sao.

Temporal couplings created by side effects are at the root of all concurrent update problems, all reentrancy problems, all race conditions, and nearly all initialization and finalization problems. Temporal couplings are big trouble. So how do we avoid them?

Các kết dính thời gian do tác dụng phụ sinh ra chính là gốc rễ của mọi vấn đề cập nhật đồng thời, mọi vấn đề reentrancy, mọi race condition, và gần như mọi vấn đề khởi tạo lẫn kết thúc. Kết dính thời gian là rắc rối lớn. Vậy làm sao để tránh chúng?

Functional Languages — Ngôn ngữ hàm

You can avoid a lot of side effects by using a functional language. This is not a cure-all. It is still possible to create temporal couplings with functional languages—but most functional languages have facilities that make it very obvious when you are creating them.

Bạn có thể tránh được rất nhiều tác dụng phụ bằng cách dùng một ngôn ngữ hàm. Đây không phải thuốc chữa bách bệnh. Vẫn hoàn toàn có thể tạo ra kết dính thời gian trong ngôn ngữ hàm — nhưng phần lớn ngôn ngữ hàm đều có những cơ chế khiến chuyện đó lộ ra rất rõ khi bạn đang làm.

The primary mechanism of functional languages that suppresses side effects is the lack of an assignment statement. In a functional language, you are not allowed to change the state of a variable. In fact, there are no variables in functional languages. Named values cannot be varied.

Cơ chế chính giúp ngôn ngữ hàm dập tắt tác dụng phụ là việc không có câu lệnh gán. Trong một ngôn ngữ hàm, bạn không được phép thay đổi trạng thái của một biến. Thật ra, trong ngôn ngữ hàm chẳng có biến nào cả. Các giá trị có tên thì không thể biến thiên.

Some of you may have used a functional language before, and may now be ripping this page into little pieces out of frustration. Please relax. Let me explain.

Có thể vài người trong số các bạn từng dùng ngôn ngữ hàm, và lúc này đang xé trang sách này thành từng mảnh vì bực. Xin cứ bình tĩnh. Để tôi giải thích.

Functional languages allow names to be assigned to values. The source code may look a lot like an assignment statement. However, in a strictly functional language, while you may be able to reassign the name, you cannot overwrite the value.

Ngôn ngữ hàm cho phép gán tên cho các giá trị. Mã nguồn nhìn có thể rất giống một câu lệnh gán. Tuy nhiên, trong một ngôn ngữ hàm thuần túy nghiêm ngặt, dù bạn có thể gán lại cái tên, bạn không thể ghi đè lên giá trị.

For example, I found the following in a Clojure program I wrote a while ago.

Chẳng hạn, tôi tìm được đoạn sau trong một chương trình Clojure mà tôi viết cách đây một thời gian.

```
(defn update-clouds-age [ms world]
  (let [clouds (:clouds world)
        decay (Math/pow glc/cloud-decay-rate ms)
        clouds (map #(update % :concentration * decay) clouds)
        clouds (filter #(> (:concentration %) 1) clouds)
        clouds (doall clouds)]
    (assoc world :clouds clouds)))
```

The name `clouds` has been assigned and reassigned several times. However, the previous values have not been overwritten—they still exist. The name `clouds` does not represent a slot in memory that has been overwritten by the reassignments. Rather, the four versions of `clouds` refer to four different blocks of memory that retain their values. The name `clouds` does not represent a variable, because nothing has varied.

Cái tên `clouds` đã được gán rồi gán lại vài lần. Tuy nhiên, các giá trị trước đó không hề bị ghi đè — chúng vẫn còn nguyên. Cái tên `clouds` không đại diện cho một ô nhớ bị các lần gán lại ghi đè lên. Đúng hơn, bốn phiên bản của `clouds` trở tới bốn khối bộ nhớ khác nhau, mỗi khối vẫn giữ nguyên giá trị của mình. Cái tên `clouds` không đại diện cho một biến, bởi chẳng có gì biến thiên cả.

Now, go get some tape and repair this page.

Giờ thì đi kiếm băng dính về dán lại trang sách này đi.

Why would functional languages prevent overwriting values? Because overwriting is a side effect. It separates the lines above it and the lines below it by a change of state. It creates a temporal coupling. The order matters.

Tại sao ngôn ngữ hàm lại ngăn việc ghi đè giá trị? Bởi vì ghi đè là một tác dụng phụ. Nó chia cắt những dòng phía trên với những dòng phía dưới bằng một thay đổi trạng thái. Nó tạo ra kết dính thời gian. Thứ tự trở nên có ý nghĩa.

Of course, you can still create code with side effects when using functional languages. Most of the time this is because the language has special facilities for doing so. For example, Clojure has a special data type called an `atom` that represents a variable whose value can be changed using a set of special functions.

Dĩ nhiên, bạn vẫn có thể viết ra mã có tác dụng phụ khi dùng ngôn ngữ hàm. Phần lớn trường hợp là vì ngôn ngữ có sẵn những cơ chế đặc biệt để làm việc đó. Chẳng hạn, Clojure có một kiểu dữ liệu đặc biệt tên là `atom`, đại diện cho một biến mà giá trị có thể thay đổi thông qua một bộ hàm chuyên dụng.

And, of course, there are still operating system facilities that have side effects. Files still need to be opened and closed, for example.

Và dĩ nhiên, vẫn còn đó những tiện ích của hệ điều hành vốn mang tác dụng phụ. Tập thì vẫn cứ phải mở ra rồi đóng lại, chẳng hạn thế.

More generally, it is possible to simulate any system with a functional language—including a system that supports side effects. For example, I could write a Lua execution environment in even the strictest functional language, and then write programs in Lua that had blatant side effects.

Tổng quát hơn, ta hoàn toàn có thể mô phỏng bất kỳ hệ thống nào bằng một ngôn ngữ hàm — kể cả hệ thống có hỗ trợ tác dụng phụ. Chẳng hạn, tôi có thể viết một môi trường thực

thì Lua bằng ngôn ngữ hàm nghiêm ngặt nhất, rồi viết những chương trình Lua đầy rẫy tác dụng phụ lộ liễu.

Now, this is not a book about functional programming, and so I'm not going to continue this at any length. Suffice it to say that using a functional language can help you manage side effects; but it cannot, and will not, eliminate them.

Cuốn sách này không phải sách về lập trình hàm, nên tôi sẽ không kéo dài chuyện này thêm nữa. Chỉ cần nói rằng dùng ngôn ngữ hàm có thể giúp bạn quản lý tác dụng phụ; nhưng nó không thể, và sẽ không, xoá sổ được chúng.

If you want a much more in-depth discussion of these matters, I suggest you read my book *Functional Design*. 10

Nếu bạn muốn một cuộc bàn luận sâu hơn nhiều về những chuyện này, tôi đề nghị bạn đọc cuốn *Functional Design* của tôi. 10

10 [Functional Design].

10 [Functional Design].

Object-Oriented Languages — Ngôn ngữ hướng đối tượng

One of the big complaints that has lately been levied against object-oriented programming languages (OOPs) is that they promote side effects. It's easy to see why this complaint has gotten traction. Objects are containers of state, and the methods of objects often change their state, leading to side effects and temporal couplings.

Một trong những lời than phiền lớn dạo gần đây nhắm vào các ngôn ngữ lập trình hướng đối tượng (OOP) là chúng cổ vũ cho tác dụng phụ. Dễ hiểu vì sao lời than phiền này lại được hưởng ứng. Đối tượng (object) là những cái hộp chứa trạng thái, và method của đối tượng thường thay đổi trạng thái ấy, dẫn tới tác dụng phụ và kết dính thời gian.

However, what is not often argued is the fact that OOPs have very good facilities to suppress and hide side effects. For instance, consider this simplistic and contrived example in C++.

Tuy nhiên, điều ít khi được nhắc đến là các OOP có những cơ chế rất tốt để dập tắt và che giấu tác dụng phụ. Chẳng hạn, hãy xét ví dụ đơn giản đến ngây ngô và hơi khiên cưỡng sau đây bằng C++.

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
class LineCounter {
public:
    static int count(const char* fileName);
private:
    static int nLines;
    static ifstream file;
    static void initialize(const char* fileName);
    static void countTheLines();
    static int finalize();
};
int LineCounter::nLines;
```

```

ifstream LineCounter::file;
void LineCounter::initialize(const char* fileName) {
    nLines = 0;
    file.open(fileName);
}
void LineCounter::countTheLines() {
    string line;
    while (getline (file, line))
        nLines++;
}
int LineCounter::finalize() {
    file.close();
    return nLines;
}
int LineCounter::count(const char* fileName) {
    initialize(fileName);
    countTheLines();
    return finalize();
}

```

What I want you to notice in this example is that every one of the private methods in the `LineCounter` class has a side effect; however, the sole public method, which calls them, does not. This means that none of the users of `LineCounter` are exposed to a temporal coupling. Assuming this is a single-threaded application, the side effects are entirely hidden.

Điều tôi muốn bạn để ý trong ví dụ này là mọi method private trong class `LineCounter` đều có tác dụng phụ; thế nhưng method public duy nhất — cái gọi tới chúng — thì không. Nghĩa là không một người dùng nào của `LineCounter` phải phơi mình trước kết dính thời gian. Với giả định đây là ứng dụng đơn luồng, các tác dụng phụ được giấu kín hoàn toàn.

In this example, we have used the `LineCounter` as a protected namespace. There is no `LineCounter` object. The class is just a place that allows us to hide the variables and functions that might cause temporal couplings.

Trong ví dụ này, ta đã dùng `LineCounter` như một namespace được bảo vệ. Chẳng có đối tượng `LineCounter` nào cả. Class ở đây chỉ là chỗ để ta giấu đi những biến và hàm có thể gây kết dính thời gian.

Notice that I added the single-threaded constraint. In a multithreaded environment, those static variables could be exposed to concurrent update issues. We could solve this by surrounding the count function with a semaphore. Or we could solve it by turning the `LineCounter` class into a real object by getting rid of the static variables and functions and forcing users to create an instance before calling count .

Để ý là tôi đã kèm theo ràng buộc đơn luồng. Trong môi trường đa luồng, những biến static kia có thể phơi ra trước các vấn đề cập nhật đồng thời. Ta có thể giải quyết bằng cách bọc hàm count trong một semaphore. Hoặc có thể giải quyết bằng cách biến class `LineCounter` thành một đối tượng thực thụ: bỏ hết biến và hàm static đi, buộc người dùng phải tạo một instance trước khi gọi count.

The point is that the facilities of most OOPs provide a handy way to hide side effects behind **encapsulation** boundaries, thus preventing temporal couplings from leaking out into the application at large.

Điểm mấu chốt là các cơ chế của hầu hết OOPL đều cho ta một cách tiện lợi để giấu tác dụng phụ sau những ranh giới đóng gói (encapsulation), nhờ đó ngăn kết dính thời gian rò rỉ ra toàn bộ ứng dụng.

Structured Programming — Lập trình có cấu trúc

In 1968, Edsger Dijkstra wrote a paper that stirred up the programming community for the better part of a decade. Back in those days the ideas he expressed in that paper were deeply controversial and hotly debated. Some felt that Dijkstra was a fool. Others thought him a god.

Năm 1968, Edsger Dijkstra viết một bài báo khuấy đảo cộng đồng lập trình suốt gần cả một thập kỷ. Hồi ấy, những ý tưởng ông trình bày trong bài báo đó gây tranh cãi dữ dội và bị đem ra mổ xẻ nảy lửa. Có người cho rằng Dijkstra là một gã ngốc. Người khác lại coi ông như thánh sống.

But by the time the '80s rolled around, Dijkstra's simple idea of **Structured Programming** was well accepted. By the '90s, it was hard to find a language that allowed any other style.

Nhưng đến khi thập niên 1980 gõ cửa, ý tưởng giản dị của Dijkstra về Lập trình có cấu trúc (Structured Programming) đã được chấp nhận rộng rãi. Đến thập niên 1990 thì khó mà tìm nổi một ngôn ngữ cho phép viết theo kiểu khác.

The paper, of course, was “Go To Statement Considered Harmful.” It was published as a letter to the editor in the March 1968 issue of the Communications of the ACM . The title was given to it by the editor at the time: Niklaus Wirth. 11 And that paper changed the world—of programming.

Bài báo ấy, tất nhiên, chính là “Go To Statement Considered Harmful.” Nó được đăng dưới dạng thư gửi ban biên tập trên số tháng Ba năm 1968 của tạp chí *Communications of the ACM*. Cái tiêu đề là do vị biên tập viên thời đó đặt cho: Niklaus Wirth. 11 Và bài báo ấy đã thay đổi thế giới — thế giới của lập trình.

11 The inventor of the Pascal language. The idea behind the paper is very simple. All programs should be written using only three control structures: sequence, selection, and iteration.

11 Cha đẻ của ngôn ngữ Pascal. Ý tưởng đằng sau bài báo rất đơn giản. Mọi chương trình đều nên được viết chỉ bằng ba cấu trúc điều khiển: tuần tự, lựa chọn và lặp.

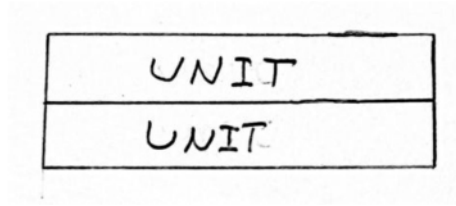
This was revolutionary at the time because most programming languages used jumps (GOTO) as the primary, or even the sole, control structure, and programmers were used to jumping hither and yon within their programs. Such programs often became rats' nests of control flow.

Điều này mang tính cách mạng vào thời đó, bởi hầu hết ngôn ngữ lập trình đều dùng lệnh nhảy (GOTO) làm cấu trúc điều khiển chính, thậm chí là duy nhất, và lập trình viên đã quen nhảy nhót lung tung khắp chương trình của mình. Những chương trình như vậy thường biến thành một ổ chuột chằng chịt các luồng điều khiển.

Dijkstra said that all programs should be ordered as recursively applied units with a single entry and a single exit. The units came in three varieties.

Dijkstra nói rằng mọi chương trình đều nên được tổ chức thành các đơn vị áp dụng đệ quy, mỗi đơn vị có đúng một lối vào và đúng một lối ra. Các đơn vị ấy có ba kiểu.

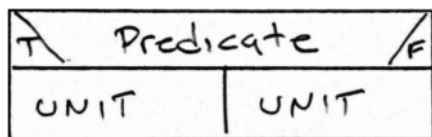
Sequence — Tuần tự



A sequence is two (or more) units that are executed one after the other. Each such unit has a single entry at the top and a single exit at the bottom. A unit might be as simple as a single statement, like `i=0;`, or it might be a function call, or its own sequence of units.

Tuần tự là hai (hoặc nhiều) đơn vị được thực thi lần lượt nối tiếp nhau. Mỗi đơn vị như vậy có một lối vào duy nhất ở trên đỉnh và một lối ra duy nhất ở dưới đáy. Một đơn vị có thể đơn giản như một câu lệnh đơn lẻ, kiểu `i=0;`, hoặc có thể là một lời gọi hàm, hoặc bản thân nó lại là một chuỗi tuần tự các đơn vị.

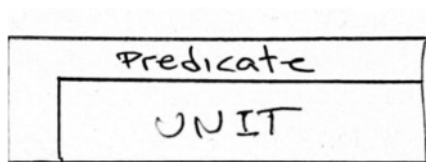
Selection — Lựa chọn



A selection is a boolean predicate that selects one of two units below it. The predicate is any boolean expression, such as `i<0`.

Lựa chọn là một vị từ boolean chọn ra một trong hai đơn vị nằm bên dưới nó. Vị từ ở đây là bất kỳ biểu thức boolean nào, chẳng hạn `i<0`.

Iteration — Lặp

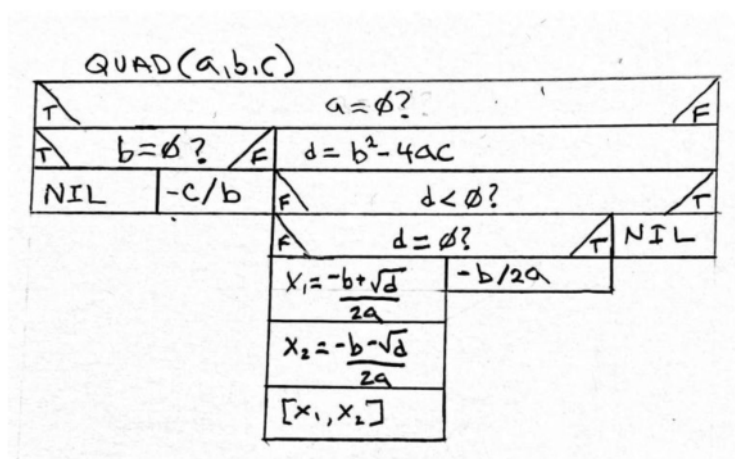


An iteration is a predicate that executes the unit below it until the predicate is false.

Lặp là một vị từ thực thi đơn vị nằm bên dưới nó cho tới khi vị từ đó sai.

These units can be recursively applied into a program as follows.

Những đơn vị này có thể được áp dụng đệ quy để tạo thành một chương trình như sau.



This style of diagram is called a Nassi–Schneiderman flowchart . The diagram itself depicts the flow of control through a quadratic equation solver.

Kiểu sơ đồ này được gọi là *lưu đồ Nassi–Schneiderman*. Bản thân sơ đồ mô tả luồng điều khiển đi qua một chương trình giải phương trình bậc hai.

Dijkstra’s rationale for restricting control flow to these structures was based on the argument that such structures are relatively easy to reason about. You can follow each path and evaluate the outcomes. However, if goto statements are allowed to transfer control from any point in the program to any other point in the program, then following the paths, and therefore reasoning about the problem, becomes impractical.

Lý lẽ của Dijkstra khi giới hạn luồng điều khiển vào những cấu trúc này dựa trên lập luận rằng các cấu trúc như vậy tương đối dễ suy luận. Bạn có thể lần theo từng nhánh và đánh giá kết quả. Nhưng nếu lệnh goto được phép chuyển quyền điều khiển từ bất kỳ điểm nào trong chương trình tới bất kỳ điểm nào khác, thì việc lần theo các nhánh — và do đó việc suy luận về bài toán — trở nên bất khả thi trên thực tế.

Nowadays, our languages restrict control flow for us. Most of us don’t use languages that have a goto statement. Those of us who do, tend not to use it.

Ngày nay, các ngôn ngữ đã tự giới hạn luồng điều khiển giúp chúng ta. Phần lớn chúng ta không dùng ngôn ngữ nào có lệnh goto. Còn số ít vẫn dùng thì thường cũng chẳng đụng tới nó.

When writing clean functions, it is helpful to keep this recursive application of the three structures in mind.

Khi viết những hàm sạch, việc luôn ghi nhớ cách áp dụng đệ quy ba cấu trúc này là rất có ích.

This Is Too Much to Constantly Keep in Mind — Ôm hết chường này trong đầu thì quá sức

Writing software is like any other kind of writing. When you write a paper or an article, you get your thoughts down first, and then you massage it until it reads well. The first draft might be clumsy and disorganized, so you wordsmith it and restructure it and refine it until it reads the way you want it to read.

Viết phần mềm cũng như mọi kiểu viết lách khác. Khi viết một bài luận hay một bài báo, trước hết bạn tổng hết suy nghĩ ra giấy, rồi mới nhào nặn nó cho tới khi đọc xuôi tai. Bản nháp đầu tiên có thể vụng về và lộn xộn, nên bạn gọt câu chữ, sắp xếp lại, tinh chỉnh cho tới khi nó đọc lên đúng như bạn muốn.

When I write functions, they come out long and complicated. They have lots of indenting and nested loops. They have long argument lists. The names are arbitrary, and there is duplicated code, and temporal couplings. But I also have a suite of unit tests that cover every one of those clumsy lines of code.

Khi tôi viết hàm, chúng ra lò dài dòng và rối rắm. Chúng thật lè tùm lum và có vòng lặp lồng nhau. Chúng có danh sách đối số dài lê thê. Tên thì tùy tiện, mã thì trùng lặp, lại còn đầy kết dính thời gian. Nhưng tôi cũng có sẵn một bộ unit test phủ hết từng dòng mã vụng về đó.

So then I massage and refine that code, splitting out functions, changing names, eliminating duplication, and making my functions purer. I shrink the methods and reorder them. Sometimes I break out whole classes, all the while keeping the tests passing.

Rồi sau đó tôi nhào nặn và tinh chỉnh đồng mã ấy: tách hàm ra, đổi tên, dẹp trùng lặp, và làm cho các hàm của mình thuần khiết hơn. Tôi thu nhỏ method lại và sắp xếp lại thứ tự. Đôi khi tôi tách hẳn ra thành những class riêng, mà suốt quá trình đó vẫn giữ cho các test luôn xanh.

In the end, I wind up with functions that follow the attributes and heuristics I've laid down in the last two chapters. I don't write them that way to start. I don't think anyone could.

Cuối cùng, tôi có được những hàm tuân theo đúng các đặc điểm và kinh nghiệm thực hành (heuristic) mà tôi đã trình bày trong hai chương vừa rồi. Ngay từ đầu tôi không viết ra chúng như vậy. Tôi không nghĩ có ai làm được thế.

First, make it work. Then, make it right.

Trước hết, hãy làm cho nó chạy được. Rồi sau đó, hãy làm cho nó đúng đắn.

Conclusion — Kết luận

Every system is built from a domain-specific language designed by the programmers to describe that system. Functions are the verbs of that language, and contexts are the nouns. It is a very old truth that the art of programming is, and has always been, the art of language design.

Mọi hệ thống đều được dựng nên từ một ngôn ngữ chuyên biệt miền (domain-specific language) do chính các lập trình viên thiết kế ra để mô tả hệ thống đó. Hàm là động từ của ngôn ngữ ấy, còn context là danh từ. Có một chân lý xưa như trái đất: nghệ thuật lập trình vốn dĩ, và xưa nay vẫn luôn, là nghệ thuật thiết kế ngôn ngữ.

Master programmers think of systems as stories to be told rather than programs to be written. They use the facilities of their chosen programming language to construct a much richer and more expressive language that can be used to tell that story. Part of that domain-specific language is the hierarchy of functions that describe all the actions that take place within that system. In an artful act of recursion, those actions are written to use the very domain-specific language they define to tell their own small part of the story.

Những lập trình viên bậc thầy nghĩ về hệ thống như những câu chuyện cần được kể, chứ không phải những chương trình cần được viết. Họ tận dụng mọi tiện ích của ngôn ngữ

lập trình mình chọn để dựng lên một ngôn ngữ phong phú và giàu sức biểu đạt hơn hẳn, đủ sức kể câu chuyện đó. Một phần của ngôn ngữ chuyên biệt miền ấy chính là hệ thống phân cấp các hàm mô tả mọi hành động diễn ra bên trong hệ thống. Và trong một màn đệ quy đầy nghệ thuật, bản thân những hành động đó lại được viết bằng đúng cái ngôn ngữ chuyên biệt miền mà chúng định nghĩa, để kể phần chuyện nhỏ bé của riêng mình.

The last two chapters have been about the mechanics of writing functions well. If you follow the ideas herein, your functions will be short, well named, and nicely organized. But never forget that your real goal is to tell the story of the system, and that the functions you write need to fit cleanly together into a clear and precise language to help you with that telling.

Hai chương vừa rồi bàn về kỹ thuật viết hàm sao cho tốt. Nếu bạn đi theo những ý tưởng trong đó, hàm của bạn sẽ ngắn gọn, được đặt tên khéo và tổ chức gọn ghẽ. Nhưng đừng bao giờ quên rằng mục tiêu thật sự của bạn là kể câu chuyện của hệ thống, và những hàm bạn viết cần khớp vào nhau thật sạch sẽ để tạo thành một ngôn ngữ rõ ràng, chính xác, giúp bạn kể được câu chuyện ấy.

Chapter 9. The Clean Method —

Chương 9. Phương pháp sạch



How do we write software? Nowadays, of course, most of us sit at our laptop and type into an IDE. We operate in a loop.

Chúng ta viết phần mềm như thế nào? Ngày nay thì tất nhiên rồi, phần lớn chúng ta ngồi trước laptop và gõ vào một IDE. Chúng ta làm việc theo một vòng lặp.

1. We write a small bit of code (with the possible help of an AI).
 2. We compile and **test** it.
 3. We go back to step 1.
1. Ta viết một mẫu mã nhỏ (có thể có sự trợ giúp của AI).
 2. Ta biên dịch và test nó.
 3. Ta quay lại bước 1.

That loop might take anywhere from five seconds to an hour depending upon the **discipline** you follow.

Vòng lặp đó có thể kéo dài từ năm giây tới cả tiếng đồng hồ, tùy vào kỷ luật (discipline) mà bạn theo.

Of course, that's not how it used to be. In the earliest days of programming, the loop was more like this.

Tất nhiên, trước kia không phải vậy. Vào những ngày đầu của nghề lập trình, vòng lặp giống thế này hơn.

1. We wrote a large bit of code by hand.
2. We desk-checked it by hand.
3. We submitted it to be encoded—by hand—into a machine-readable form.
4. We submitted it for compilation and test.
 1. Ta viết tay một mẫu mã lớn.
 2. Ta rà soát nó bằng tay ngay trên bàn giấy.
 3. Ta gửi nó đi để được mã hoá — bằng tay — sang dạng máy đọc được.
 4. Ta gửi nó đi biên dịch và test.

That loop often took between one and three days. The difference is stark. In the past, the investment per line of code was enormous. Nowadays, that cost is trivial.

Vòng lặp đó thường mất từ một đến ba ngày. Khác biệt thật khủng khiếp. Ngày xưa, khoản đầu tư cho mỗi dòng mã là khổng lồ. Ngày nay, chi phí đó chẳng đáng gì.

When costs are high, we spend a lot of time planning up front in order to avoid waste. When costs are low, it makes more sense to explore by failing. In both cases, the idea of cleaning the code is generally ignored.

Khi chi phí cao, ta dành thật nhiều thời gian lên kế hoạch từ đầu để tránh lãng phí. Khi chi phí thấp, dò dẫm bằng cách cứ thử rồi thất bại lại hợp lý hơn. Trong cả hai trường hợp, ý tưởng dọn dẹp mã nhìn chung đều bị bỏ qua.

Make It Right — Làm cho nó đúng

In the early days, cleaning code was inconvenient because the code was written by hand on sheets of paper. Erasing, cutting, and pasting sheets of paper is very inefficient, and just not fun—especially when small changes can spread through the body of the code in very inconvenient ways.

Thuở ban đầu, dọn dẹp mã là chuyện bất tiện vì mã được viết tay trên giấy. Tẩy xoá, cắt rồi dán những tờ giấy vừa cực kỳ kém hiệu quả, vừa chẳng vui vẻ gì — nhất là khi một thay đổi nhỏ có thể lan ra khắp thân mã theo những cách vô cùng phiền toái.

Nowadays, cleaning the code has been made much more convenient by our IDEs and **refactoring** tools. But we often bypass cleaning the code because that tight loop allows us to make progress so quickly that pausing to clean the code seems like a misuse of time. We drive hard to make the code work—and then we check the code in and move on to the next problem.

Ngày nay, IDE và các công cụ refactoring đã khiến việc dọn dẹp mã tiện lợi hơn nhiều. Nhưng ta vẫn thường bỏ qua việc dọn mã, bởi cái vòng lặp chặt chẽ ấy cho phép ta tiến nhanh đến mức dừng lại để dọn dẹp trông như phí phạm thời gian. Ta lao đầu vào làm cho mã chạy được — rồi check in mã và chuyển sang vấn đề tiếp theo.

Over two decades ago, Kent Beck stated the simple rule that guides this book.

Hơn hai thập kỷ trước, Kent Beck đã phát biểu quy tắc đơn giản dẫn đường cho cuốn sách này.

First, make it work.

Trước hết, hãy làm cho nó chạy.

Then, make it right.

Sau đó, hãy làm cho nó đúng.

There's more to this rule that we'll talk about later. For now, just look at those two lines. Most of us do the first part because we consider it to be the reason we were hired. We were hired to make code work. But many of us bypass the second part because it seems like a waste of time. After all, if we are **good** programmers, we should be writing right code all the time. We shouldn't need to have to stop and make it right.

Quy tắc này còn nhiều điều nữa mà ta sẽ bàn sau. Còn bây giờ, cứ nhìn kỹ hai dòng đó đã. Hầu hết chúng ta làm phần thứ nhất vì ta coi đó là lý do mình được thuê. Ta được thuê để làm cho mã chạy được. Nhưng nhiều người trong chúng ta bỏ qua phần thứ hai vì nó có vẻ phí thời gian. Suy cho cùng, nếu ta là lập trình viên giỏi thì lúc nào ta cũng phải viết ra mã đúng chứ. Đâu cần phải dừng lại để làm cho nó đúng.

Perhaps you are that good. Perhaps the code that comes out of your fingers is right the first time and every time. If so, you can put this book down.

Cũng có thể bạn giỏi đến mức đó thật. Cũng có thể mã tuôn ra từ đầu ngón tay bạn lần nào cũng đúng ngay từ lần đầu. Nếu vậy, bạn có thể gấp cuốn sách này lại.

However, most of us are not particularly adept at keeping two goals in our head at the same time. The primary goal of making the code work pushes aside the secondary goal of making the code right. Most of us find that the mental effort required to get the code to work is so great that we have no mental effort left to simultaneously make the code right.

Tuy nhiên, phần lớn chúng ta không tài tình đến mức giữ được hai mục tiêu trong đầu cùng một lúc. Mục tiêu chính — làm cho mã chạy được — sẽ đẩy mục tiêu phụ — làm cho mã đúng — ra rìa. Đa số chúng ta thấy rằng nỗ lực trí óc để bắt mã chạy được đã lớn tới mức chẳng còn chút sức lực tinh thần nào cho việc đồng thời làm cho mã đúng.

Thus, by the time we get the code to work, the code is also a **mess**. And that's when the second line of Beck's rule comes into play. Once we have the code working, then we will have the mental capacity to clean it—to make it right.

Thế nên, đến lúc mã chạy được thì nó cũng đã là một mớ hỗn độn. Và đó chính là lúc dòng thứ hai trong quy tắc của Beck lên tiếng. Một khi mã đã chạy, ta mới có đủ năng lực trí óc để dọn dẹp nó — để làm cho nó đúng.

Of course, we'd better have a way to ensure that in making the code right we don't break the fact that it works. It is not our goal to create **clean code** that doesn't work. How are we to get that assurance? How will we make sure that the act of cleaning does not break the code?

Tất nhiên, ta cần có cách bảo đảm rằng khi làm cho mã đúng, ta không phá mất chuyện nó đang chạy được. Mục tiêu của ta không phải là tạo ra clean code không chạy. Làm sao có được sự bảo đảm ấy? Làm sao chắc chắn rằng hành động dọn dẹp không làm hỏng mã?

We will talk about testing disciplines in a later chapter. Suffice it to say that if we don't have a **test suite** that we can run quickly and that we implicitly trust to show that the code works, then we are not going to do a lot of cleaning.

Ta sẽ bàn về các kỷ luật kiểm thử ở một chương sau. Ở đây chỉ cần nói rằng nếu ta không có một bộ test (test suite) chạy được thật nhanh và ta ngầm tin tưởng rằng nó chứng minh mã đang chạy đúng, thì ta sẽ chẳng dọn dẹp được bao nhiêu.

Clean code depends on fast, convenient, and comprehensive tests.

Clean code phụ thuộc vào những test nhanh, tiện lợi và toàn diện.

So how do we make the code right? What's the process? It's another tight loop similar to writing the code.

Vậy làm sao để làm cho mã đúng? Quy trình là gì? Đó lại là một vòng lặp chặt chẽ nữa, tương tự vòng lặp viết mã.

1. We clean one little thing.
2. We run the tests.
3. If the tests fail, we revert.
4. Go back to step 1.

1. Ta dọn một thứ nhỏ.
2. Ta chạy test.
3. Nếu test hỏng, ta revert.
4. Quay lại bước 1.

The timing of this loop is typically between five seconds and two minutes. The name of this loop is **refactoring**, and each cleaning step is a **refactoring**.

Vòng lặp này thường kéo dài từ năm giây đến hai phút. Tên của nó là **refactoring**, và mỗi bước dọn dẹp là một **refactoring**.

To quote Martin Fowler, **refactoring** is “a change made to the internal **structure** of software to make it easier to understand and cheaper to modify without changing its observable **behavior**.” ¹

Dẫn lời Martin Fowler, **refactoring** là “một thay đổi thực hiện lên cấu trúc bên trong của phần mềm nhằm khiến nó dễ hiểu hơn và rẻ hơn khi sửa đổi, mà không làm thay đổi hành vi quan sát được của nó.” ¹

¹ [Refactoring], p. 45.

¹ [Refactoring], tr. 45.

How often should we transition between the writing loop and the refactoring loop? As often as possible. This implies an outer loop that includes both the loop to make it work and the loop to make it right.

Ta nên chuyển qua lại giữa vòng lặp viết mã và vòng lặp **refactoring** bao lâu một lần? Càng thường xuyên càng tốt. Điều này ngụ ý một vòng lặp bên ngoài bao trùm cả vòng làm cho nó chạy lẫn vòng làm cho nó đúng.

1. Get something small to work.
 1. Làm cho một thứ nhỏ chạy được.
- a. We write a small bit of code along with tests.
 - a. Ta viết một mẫu mã nhỏ kèm theo test.
- b. We compile and test it.
 - b. Ta biên dịch và test nó.
- c. If it's not too messy, we go back to step 1a. Otherwise. . .
 - c. Nếu chưa quá bầy hầy, ta quay lại bước 1a. Nếu không thì. . .
2. Clean it up.
 2. Dọn dẹp nó.
- a. We clean one little thing.

- a. Ta dọn một thứ nhỏ.
- b. We run the tests.
- b. Ta chạy test.
- c. If the tests fail, we revert and go to step 2a.
- c. Nếu test hỏng, ta revert rồi về bước 2a.
- d. If there is more to clean, we go back to step 2a. Otherwise. . .
- d. Nếu vẫn còn thứ để dọn, ta quay lại bước 2a. Nếu không thì. . .
- 3. Go back to step 1.
- 3. Quay lại bước 1.

The timing of this loop depends on your testing discipline and is on the order of a few minutes to an hour or so.

Thời lượng của vòng lặp này phụ thuộc vào kỷ luật kiểm thử của bạn, cỡ vài phút đến chừng một tiếng.

The bottom line is that, unless you are a multitalented mega being, writing clean code means that you are going to have to sit in that outer loop, patiently cleaning up after yourself, and never allowing the mess to get too big to clean.

Chốt lại là thế này: trừ khi bạn là một siêu nhân đa tài, viết clean code nghĩa là bạn sẽ phải ngồi trong cái vòng lặp bên ngoài đó, kiên nhẫn dọn dẹp sau lưng mình, và không bao giờ để mớ hỗn độn lớn tới mức không dọn nổi.

Be like the sushi chef who never stops making sushi and who never stops cleaning the implements and the environment.

Hãy như người đầu bếp sushi: không ngừng làm sushi và cũng không ngừng lau dọn dụng cụ lẫn chỗ làm việc.

Example — Ví dụ

Perhaps the best way to understand how to write clean code is to watch someone do it. So, in this chapter, we will walk through the procedure for writing a clean **module**. The technique will involve

Có lẽ cách tốt nhất để hiểu cách viết clean code là xem ai đó thực sự làm việc ấy. Vậy nên, trong chương này, chúng ta sẽ đi qua từng bước của quy trình viết một module sạch. Kỹ thuật sẽ bao gồm

- The testing discipline of **TDD**
- Refactoring
- Design principles
- Design patterns We will begin with an imaginary Python application: the Tax Calculator for the mythical state of Bobolia. It's been a long time since I've written any Python, so bear with me.
 - Kỷ luật kiểm thử TDD
 - Refactoring
 - Các nguyên tắc thiết kế

- Các mẫu thiết kế Ta sẽ bắt đầu với một ứng dụng Python tưởng tượng: Tax Calculator cho bang Bobolia trong truyền thuyết. Đã lâu rồi tôi không viết Python, nên mong bạn thông cảm.

Here's the first test.

Đây là test đầu tiên.

```
—bobolia_taxes_test.py—
import unittest
import bobolia_taxes
class bobolia_tax_tests(unittest.TestCase):
    def test_simplest_case(self):
        tax_return = {"income": {"salary": 50000}}
        tax_calculator = bobolia_taxes.tax_calculator(tax_return)
        self.assertEqual(7500, tax_calculator.get_tax())
if __name__ == '__main__':
    unittest.main()
```

In the Great State of Bobolia, the base tax rate is 15%. So someone earning 50,000 bobbles must pay 7,500 bobbles in tax. Note that the tax return is a simple map with an income key that contains yet another map with a salary key.

Tại Đại Bang Bobolia, thuế suất cơ bản là 15%. Vậy người kiếm được 50.000 bobble phải nộp 7.500 bobble tiền thuế. Lưu ý rằng tờ khai thuế là một map đơn giản với khoá income, mà bên trong lại chứa một map khác với khoá salary.

We can get this to pass with a simple degenerate module.

Ta có thể làm cho nó pass bằng một module thoái hoá hết sức đơn giản.

```
—bobolia_taxes.py—
class tax_calculator:
    def __init__(self, tax_return):
        self.tax_return = tax_return

    def get_tax(self):
        return 7500
```

Actually, I made it fail first by returning 0 , and then I made it pass by returning 7500 . This is the TDD way. First we see the test fail, and then we make it pass. If that sounds like a waste of time, let me just say that this simple practice has saved me a lot of debugging time.

Thật ra tôi làm cho nó fail trước bằng cách trả về 0 , rồi mới làm cho nó pass bằng cách trả về 7500 . Đó là cách của TDD. Trước hết ta thấy test fail, rồi ta làm cho nó pass. Nếu nghe có vẻ phí thời gian, tôi chỉ xin nói rằng thực hành đơn giản này đã tiết kiệm cho tôi rất nhiều giờ debug.

But let's move on to the next tax law. Citizens of Bobolia who make no more than 30,000 bobbles per year do not have to pay any tax at all.

Nhưng hãy chuyển sang luật thuế tiếp theo. Công dân Bobolia kiếm không quá 30.000 bobble mỗi năm thì không phải nộp đồng thuế nào.

```
—bobolia_taxes_test.py—
class BoboliaTaxTests(unittest.TestCase):
    def setUp(self):
        self.tax_calculator = bobolia_taxes.TaxCalculator()
```

```

def test_simplest_case(self):
    tax_return = {"income": {"salary": 50000}}
    self.assertEqual(7500, self.tax_calculator.get_tax(tax_ret
def test_30K_limit(self):
    tax_return = {"income": {"salary": 30000}}
    self.assertEqual(0, self.tax_calculator.get_tax(tax_return

```

Here you can see I've already begun to make some design changes. Rather than construct the TaxCalculator with the

Ở đây bạn có thể thấy tôi đã bắt đầu thay đổi thiết kế đôi chút. Thay vì dựng TaxCalculator với

tax_return, I decided to pass the tax_return into the get_tax **function**. I also changed the names of the classes to comport with the PEP 8 standard.

tax_return, tôi quyết định truyền tax_return vào hàm get_tax. Tôi cũng đổi tên các class cho hợp với chuẩn PEP 8.

Making these tests pass was pretty simple.

Làm cho các test này pass khá đơn giản.

```

—bobolia_taxes.py—
class TaxCalculator:
    def __init__(self):
        pass
    def get_tax(self, tax_return):
        total_income = tax_return["income"]["salary"]
        if total_income > 30000:
            return total_income * 0.15
        return 0

```

Now, clearly this isn't fair. You can't have a citizen who earns 30,001 bobbles pay 4,500 in tax. Therefore, no citizen's after-tax income should be less than 30,000.

Rõ ràng thế này chẳng công bằng chút nào. Không thể để một công dân kiếm 30.001 bobble phải nộp 4.500 tiền thuế. Vì vậy, thu nhập sau thuế của bất kỳ công dân nào cũng không được thấp hơn 30.000.

```

—bobolia_taxes_test.py—
class BoboliaTaxTests(unittest.TestCase):
    def setUp(self):
        self.tax_calculator = bobolia_taxes.TaxCalculator()
    def test_15pct_tax_rate(self):
        tax_return = {"income": {"salary": 50000}}
        self.assertEqual(7500, self.tax_calculator.get_tax(tax_ret
    def test_no_tax_for_30K_and_below(self):
        tax_return = {"income": {"salary": 30000}}
        self.assertEqual(0, self.tax_calculator.get_tax(tax_return
    def test_no_after_tax_income_less_than_30K(self):
        tax_return = {"income": {"salary": 30001}}
        self.assertEqual(1, self.tax_calculator.get_tax(tax_return
        tax_return = {"income": {"salary": 35000}}
        self.assertEqual(5000, self.tax_calculator.get_tax(tax_retu

```

No surprise here. I improved the names of the tests. Name improvement is something that often takes place once you know more about the application you are writing. Getting these tests to pass was pretty easy, though it forced me to move most of the older code around a bit.

Không có gì bất ngờ ở đây. Tôi đã cải thiện tên các test. Cải thiện tên là chuyện thường xảy ra một khi bạn hiểu rõ hơn về ứng dụng mình đang viết. Làm cho các test này pass khá dễ, dù nó buộc tôi phải xáo lại khá khá phần mã cũ.

```
—bobolia_taxes.py—
def get_tax(self, tax_return):
    total_income = tax_return["income"]["salary"]
    tax = total_income * 0.15
    after_tax_income = total_income - tax
    if after_tax_income < 30000:
        return total_income - 30000
    else:
        return tax
```

Now then, let's talk about the social demerit system. Bobolia is a free state. We believe that what our neighbors do is none of our darn business. However, there are behaviors that are detrimental to the state of Bobolia, and we believe citizens who engage in these behavior should compensate society for the damage they've done. So all citizens earn badbob points through the course of a year, and they must report these badbobs on their tax returns. (Don't cheat! We have ways. . .)

Giờ thì hãy nói về hệ thống điểm phạt xã hội (social demerit). Bobolia là một bang tự do. Chúng tôi tin rằng hàng xóm làm gì thì chẳng phải việc quái gì của mình. Tuy nhiên, có những hành vi gây hại cho bang Bobolia, và chúng tôi tin rằng công dân nào dính vào những hành vi đó thì phải đền bù cho xã hội phần thiệt hại họ đã gây ra. Vậy nên trong suốt một năm, mọi công dân đều tích lũy điểm badbob, và họ phải khai số badbob này trên tờ khai thuế. (Đừng có gian lận! Chúng tôi có cách. . .)

Badbob points are used in calculating taxes. So we need to add a badbobs key to the tax return. Oh, but that means we have to go back and change all the tests.

Điểm badbob được dùng khi tính thuế. Vậy nên chúng ta cần thêm khoá badbobs vào tờ khai thuế. Ồ, nhưng thế thì lại phải quay lại sửa toàn bộ các test.

Anytime you have to go back and change a bunch of tests, it means there's a flaw in the design of your tests. Let's fix that flaw before we proceed.

Bất cứ khi nào bạn phải quay lại sửa cả một loạt test, điều đó có nghĩa là thiết kế test của bạn có khiếm khuyết. Hãy sửa khiếm khuyết ấy trước khi đi tiếp.

```
—bobolia_taxes_test.py—
def make_return(args):
    tax_return = {"income": {"salary": 0},
                  "badbobs": 0}
    tax_return.update(args)
    return tax_return
class BoboliaTaxTests(unittest.TestCase):
    def setUp(self):
        self.tax_calculator = bobolia_taxes.TaxCalculator()
    def test_15pct_tax_rate(self):
        tax_return = make_return({"income": {"salary": 50000}})
        self.assertEqual(7500, self.tax_calculator.get_tax(tax_ret
```

```

def test_no_tax_for_30K_and_below(self):
    tax_return = make_return({"income": {"salary": 30000}})
    self.assertEqual(0, self.tax_calculator.get_tax(tax_return))
def test_no_after_tax_income_less_than_30K(self):
    tax_return = make_return({"income": {"salary": 30001}})
    self.assertEqual(1, self.tax_calculator.get_tax(tax_return))

    tax_return = make_return({"income": {"salary": 35000}})
    self.assertEqual(5000, self.tax_calculator.get_tax(tax_return))

```

This simple change has decoupled the tests from the expectations of the production code. The `make_return` function will ensure that any omitted fields are given a reasonable default value. Thus, when new fields are added to the tax return, the existing tests are not likely to require changes.

Thay đổi đơn giản này đã tách rời (decoupling) các test khỏi những kỳ vọng về mã production. Hàm `make_return` sẽ bảo đảm rằng mọi trường bị bỏ sót đều nhận một giá trị mặc định hợp lý. Nhờ vậy, khi có trường mới được thêm vào tờ khai thuế, các test hiện có nhiều khả năng sẽ chẳng cần sửa gì.

Now, on with the badbob processing.

Giờ thì tiếp tục với phần xử lý badbob.

The purchase of alcoholic beverages is detrimental to society. So, if you spend 100 bobbles on whiskey, you should earn 100 badbobs. Alcohol badbobs are of **class 1** and will be added to your tax at a rate of 50%. So that 100-bobble bottle of whiskey will cost you another 50 bobbles at tax time.

Việc mua đồ uống có cồn gây hại cho xã hội. Vậy nên nếu bạn tiêu 100 bobble mua whiskey, bạn sẽ lãnh 100 badbob. Badbob do rượu thuộc loại 1 (class 1) và sẽ được cộng vào thuế của bạn theo tỷ lệ 50%. Vậy chai whiskey giá 100 bobble sẽ khiến bạn tốn thêm 50 bobble vào mùa thuế.

—bobolia_taxes_test.py—

```

def make_return(args):
    tax_return = {"income": {"salary": 0},
                  "badbobs": {"class1": ()}}
    tax_return.update(args)
    return tax_return
class BoboliaTaxTests(unittest.TestCase):
    ...
    def test_class_1_badbobs(self):
        tax_return = make_return({"income": {"salary": 50000},
                                   "badbobs": {"class1": (100,)}})
        self.assertEqual(7550, self.tax_calculator.get_tax(tax_return))

```

While writing this test, I realized that the structure of the

Trong lúc viết test này, tôi nhận ra rằng cấu trúc của

badbobs that I had placed in `make_return` was insufficient. So I changed that to add the `class1` key and the empty tuple. The test shows the extra 50 bobbles in tax for the 100 class 1 badbobs. Making this pass was easy, but a little unsettling.

badbobs mà tôi đặt trong `make_return` là chưa đủ. Nên tôi sửa lại, thêm khoá `class1` và một tuple rỗng. Test cho thấy khoản thuế tăng thêm 50 bobble ứng với 100 badbob loại 1.

Làm cho nó pass thì dễ, nhưng hơi khiến người ta bất an.

—bobolia_taxes.py—

```
def get_tax(self, tax_return):
    total_income = tax_return["income"]["salary"]
    tax = total_income * 0.15
    after_tax_income = total_income - tax
    if after_tax_income < 30000:
        tax = total_income - 30000
    tax += sum(tax_return["badbobs"]["class1"]) / 2
    return tax
```

This function is getting messy. We could at least break it up into a couple of nicer functions.

Hàm này đang trở nên bầy hầy. Ít nhất ta cũng có thể tách nó ra thành vài hàm dễ nhìn hơn.

—bobolia_taxes.py—

```
def get_tax(self, tax_return):
    total_income = tax_return["income"]["salary"]
    tax = self.determine_base_tax(total_income)
    tax += self.determine_badbob_adjustment(tax_return)
    return tax
def determine_badbob_adjustment(self, tax_return):
    return sum(tax_return["badbobs"]["class1"]) / 2
def determine_base_tax(self, total_income):
    tax = total_income * 0.15
    after_tax_income = total_income - tax
    if after_tax_income < 30000:
        tax = total_income - 30000
    return tax
```

That's a bit better. I'm not sure why I put these functions into a class. The class is just forcing me to add self everywhere. Maybe I should get rid of that class—but I've got a nagging feeling that the class is going to be important, even if for nothing more than the name space.

Thế là khá hơn một chút. Tôi không chắc vì sao mình lại nhét mấy hàm này vào một class. Cái class chỉ tổ bắt tôi thêm self ở khắp nơi. Có lẽ tôi nên vứt quách nó đi — nhưng tôi cứ có cảm giác day dứt rằng class này rồi sẽ quan trọng, dù chẳng để làm gì ngoài việc tạo một không gian tên (name space).

Perhaps that class will come in handy for allowing the methods to communicate through instance variables rather than arguments. Let's try that.

Có lẽ class đó sẽ có ích khi cho phép các method giao tiếp với nhau qua biến thực thể (instance variable) thay vì qua đối số. Hãy thử xem.

—bobolia_taxes.py—

```
class TaxCalculator:
    def __init__(self):
        self.tax_return = None
    def get_tax(self, tax_return):
        self.tax_return = tax_return
        tax = self.determine_base_tax()
```

```

        tax += self.determine_badbob_adjustment()
    return tax
def determine_badbob_adjustment(self):
    return sum(self.tax_return["badbobs"]["class1"]) / 2

def determine_base_tax(self):
    total_income = self.tax_return["income"]["salary"]
    tax = total_income * 0.15
    after_tax_income = total_income - tax
    if after_tax_income < 30000:
        tax = total_income - 30000
    return tax

```

I think that's a little better. I'm going to keep the class.

Tôi nghĩ thế này khá hơn một chút. Tôi sẽ giữ lại class.

The Great State of Bobolia considers the purchase of gasoline-powered vehicles to be very detrimental to society. Such purchases earn class 2 badbobs at a rate of one badbob per bobble. Class 2 badbobs increase taxes according to the following table:

Đại Bang Bobolia coi việc mua xe chạy xăng là cực kỳ có hại cho xã hội. Những khoản mua sắm như vậy lãnh badbob loại 2 với tỷ lệ một badbob cho mỗi bobble. Badbob loại 2 làm tăng thuế theo bảng sau:

Class 2 badbobs	% of total income
1001–10,000	5%
10,001–50,000	10%
>50,000	15%

—bobolia_taxes_test.py—

```

def make_return(args):
    tax_return = {"income": {"salary": 0},
                  "badbobs": {"class1": (),
                              "class2": ()}}

    if "income" in args:
        tax_return["income"].update(args["income"])
    if "badbobs" in args:
        tax_return["badbobs"].update(args["badbobs"])
    return tax_return

class BoboliaTaxTests(unittest.TestCase):
    ...
    def test_class_2_badbob_purchases_under_1001(self):
        tax_return = make_return({"income": {"salary": 50000},
                                   "badbobs": {"class2": (400, 600)}
                                   )
        self.assertEqual(7500, self.tax_calculator.get_tax(tax_ret
    def test_class_2_badbob_purchases_under_10001(self):

```



```

tax_return = make_return({"income": {"salary": 50000},
                          "badbobs": {"class2": (10000,)}}
)
self.assertEqual(7500 + 2500,
                 self.tax_calculator.get_tax(tax_return))
def test_class_2_badbob_purchases_under_50001(self):
    tax_return = make_return({"income": {"salary": 50000},
                              "badbobs": {"class2": (10000,4000)
                              })
    self.assertEqual(7500 + 5000,
                     self.tax_calculator.get_tax(tax_return))
def test_class_2_badbob_purchases_over_50000(self):
    tax_return = make_return(
        {"income": {"salary": 50000},
         "badbobs": {"class2": (10000,40000,1000)}}
    )
    self.assertEqual(7500 + 7500,
                     self.tax_calculator.get_tax(tax_return))

```

These tests show the four table entries pretty clearly. I made these tests pass one at a time and ended up with the following production code.

Các test này thể hiện khá rõ bốn dòng của bảng. Tôi làm cho từng test pass một, và cuối cùng có được đoạn mã production sau.

—bobolia_taxes.py—

```

class TaxCalculator:
    def __init__(self):
        self.tax_return = None
        self.total_income = None
    def get_tax(self, tax_return):
        self.tax_return = tax_return
        tax = self.determine_base_tax()
        tax += self.determine_badbob_adjustment()
        return tax
    def determine_badbob_adjustment(self):
        class1 = sum(self.tax_return["badbobs"]["class1"])
        class1_adjustment = class1 / 2
        class2 = sum(self.tax_return["badbobs"]["class2"])
        if class2 < 1001:
            if class2 < 1001:
                class2_adjustment = 0
            elif class2 < 10001:
                class2_adjustment = self.total_income * 0.05
            elif class2 < 50001:
                class2_adjustment = self.total_income * 0.1
            else:
                class2_adjustment = self.total_income * 0.15
        return class1_adjustment + class2_adjustment
    def determine_base_tax(self):
        self.total_income = self.tax_return["income"]["salary"]

```

```

tax = self.total_income * 0.15
after_tax_income = self.total_income - tax
if after_tax_income < 30000:
    tax = self.total_income - 30000
return tax

```

This is pretty awful. I expect all those numbers to change over time. I even expect the number of rows in the table to change. Maintaining this is going to be labor intensive and error prone. We can probably do better.

Đoạn này khá kinh khủng. Tôi đoán tất cả những con số kia rồi sẽ thay đổi theo thời gian. Tôi còn đoán cả số dòng trong bảng cũng sẽ thay đổi. Bảo trì thứ này sẽ tốn nhiều công sức và dễ sai. Chắc chắn ta có thể làm tốt hơn.

—bobolia_taxes.py—

```

class TaxCalculator:
    ...
    def get_tax(self, tax_return):
        self.tax_return = tax_return
        tax = self.determine_base_tax()

        tax += BadBobAdjuster(self).get_adjustment()
        return tax
    def determine_base_tax(self):
        self.total_income = self.tax_return["income"]["salary"]
        tax = self.total_income * 0.15
        after_tax_income = self.total_income - tax
        if after_tax_income < 30000:
            tax = self.total_income - 30000
        return tax

class BadBobAdjuster:
    def __init__(self, tax_calculator):
        self.tax_calculator = tax_calculator
    def get_adjustment(self):
        class1_adjustment = self.class1_adjustment()
        class2_adjustment = self.get_class2_badbob_adjustment()
        return class1_adjustment + class2_adjustment
    def get_class2_badbob_adjustment(self):
        class2 = sum(self.tax_calculator.tax_return["badbobs"]["cla
        class2_rate = self.get_class2_rate(class2)
        return class2_rate * self.tax_calculator.total_income
    def get_class2_rate(self, class2):
        class2_rate = 0
        class2_table = ((1001, 0),
                        (10001, 0.05),
                        (50001, 0.1),
                        ("max", 0.15))
        for row in class2_table:
            if row[0] == "max" or class2 < row[0]:
                class2_rate = row[1]
                break
        return class2_rate

```

```
def class1_adjustment(self):
    class1 = sum(self.tax_calculator.tax_return["badbobs"] ["cla
    class1_adjustment = class1 / 2
    return class1_adjustment
```

So I created the table and the loop to extract the rate. Then, I thought that the whole badbob idea should be moved into its own class, just to have a namespace for it.

Vậy là tôi tạo ra bảng cùng vòng lặp để rút ra tỷ lệ. Sau đó, tôi nghĩ rằng toàn bộ ý tưởng badbob nên được chuyển vào class riêng của nó, dù chỉ để có một không gian tên cho nó.

Notice what's happening here. The production code is evolving and morphing to become more generic. Parts of it are being peeled off into their own classes and separated from each other. Meanwhile, the tests are simply growing in a linear fashion. With each new test case the test class becomes more specific.

Hãy để ý điều đang diễn ra ở đây. Mã production đang tiến hoá và biến hình để trở nên tổng quát hơn. Từng phần của nó được bóc ra thành class riêng và tách khỏi nhau. Trong khi đó, các test chỉ đơn giản là dài thêm một cách tuyến tính. Với mỗi test case mới, class test lại trở nên cụ thể hơn.

As the tests get more specific, the code gets more generic.

Test càng cụ thể thì mã càng tổng quát.

This strategy keeps the tests from knowing too much about the production code. When changes to the production code are made, fewer changes need to be made to the tests.

Chiến lược này giữ cho các test không biết quá nhiều về mã production. Khi mã production thay đổi, số thay đổi cần thực hiện trên test sẽ ít hơn.

But now we have a problem. If a citizen saves up to buy an expensive car, they might end up with a tax rate that reduces their after-tax income to significantly less than 30,000 bobbles. In one sense, this is fair. Citizens who **harm** society ought to compensate for that harm. On the other hand, forcing a citizen to live on less than, say, 20,000 bobbles a year would be cruel. So let's limit the effect of the badbobs so that no badbob adjustment will reduce after-tax income to below 20,000 bobbles.

Nhưng giờ ta gặp một vấn đề. Nếu một công dân dành dụm để mua một chiếc xe đắt tiền, họ có thể rơi vào mức thuế khiến thu nhập sau thuế của họ thấp hơn 30.000 bobble rất nhiều. Xét ở một khía cạnh, điều đó công bằng. Công dân gây hại cho xã hội thì phải đền bù cho phần thiệt hại ấy. Mặt khác, ép một công dân sống với chưa tới, chẳng hạn, 20.000 bobble một năm thì thật tàn nhẫn. Vậy hãy giới hạn ảnh hưởng của badbob sao cho không khoản điều chỉnh badbob nào kéo thu nhập sau thuế xuống dưới 20.000 bobble.

—bobolia_taxes_test.py—

```
...
def test_badbobs_do_not_reduce_after_tax_income_below_20000(self):
    tax_return = make_return({"income": {"salary": 20000},
                              "badbobs": {"class2": (10000,)}}
    )
    self.assertEqual(0, self.tax_calculator.get_tax(tax_return))
```

This failed in an unexpected way. It returned a tax of —9,000 bobbles. Negative tax! I bet the citizens would love that. On the other hand, the BRS (Bobolia Revenue Service) would not be pleased. This was a bug. I fixed it by putting in a guard.

Test này fail theo một kiểu bất ngờ. Nó trả về khoản thuế —9.000 bobble. Thuế âm! Tôi cá là công dân sẽ khoái lắm. Nhưng mặt khác, BRS (Sở Thuế vụ Bobolia) hẳn sẽ chẳng vui. Đây là một bug. Tôi sửa nó bằng cách đặt vào một guard.

—bobolia_taxes.py—

```
...
def determine_base_tax(self):
    self.total_income = self.tax_return["income"]["salary"]
    if self.total_income <= 30000:
        return 0
...
```

Yes, TDD is not a magic bullet. You will still write some bugs. Fortunately, as shown in this case, other tests are likely to uncover those bugs. With that fixed, I could get the test to pass pretty easily.

Đúng vậy, TDD không phải viên đạn thần kỳ. Bạn vẫn sẽ viết ra vài con bug. May thay, như trường hợp này cho thấy, các test khác nhiều khả năng sẽ moi ra được chúng. Sửa xong chỗ đó, tôi làm cho test pass khá dễ dàng.

—bobolia_taxes.py—

```
...
def get_tax(self, tax_return):
    self.tax_return = tax_return
    base_tax = self.determine_base_tax()
    badbob_adujustment = BadBobAdjuster(self).get_adjustment()
    total_tax = base_tax + badbob_adujustment
    after_tax_income = self.total_income - total_tax
    if self.total_income < 20000:
        return 0
    elif after_tax_income < 20000:
        total_tax = self.total_income - 20000
    return total_tax
```

Those two guards that I just added are a bit ugly. I think I can just use the max function instead.

Hai cái guard tôi vừa thêm trông hơi xấu. Tôi nghĩ chỉ cần dùng hàm max thay cho chúng là được.

—bobolia_taxes.py—

```
...
def get_tax(self, tax_return):
    self.tax_return = tax_return
    base_tax = self.determine_base_tax()
    badbob_adujustment = BadBobAdjuster(self).get_adjustment()
    total_tax = base_tax + badbob_adujustment
    after_tax_income = self.total_income - total_tax
    if after_tax_income < 20000:
        total_tax = max(0, self.total_income - 20000)
    return total_tax
def determine_base_tax(self):
    self.total_income = self.tax_return["income"]["salary"]
    tax = self.total_income * 0.15
    after_tax_income = self.total_income - tax
```

```

if after_tax_income < 30000:
    tax = max (0, self.total_income - 30000)
return tax

```

That's better.

Thế là khá hơn.

Now then. . . A year has gone by, and everyone is paying their taxes. But after some riots and really mean tweets, we have decided that here in the Great State of Bobolia income inequality is a social harm. Low earners were paying too much, and high earners were paying too little. Those who earn too many bobbles should simply pay a higher tax rate than those who earn less. Not just more taxes, but a higher rate. We've determined that the following table would represent "their fair share":

Giờ thì. . . Một năm đã trôi qua, và mọi người đều đang nộp thuế. Nhưng sau vài cuộc bạo loạn và những dòng tweet thực sự cay nghiệt, chúng tôi quyết định rằng ở Đại Bang Bobolia này, bất bình đẳng thu nhập là một tổn hại (harm) đối với xã hội. Người thu nhập thấp đang nộp quá nhiều, còn người thu nhập cao thì nộp quá ít. Ai kiếm được quá nhiều bobble thì đơn giản là phải chịu thuế suất cao hơn người kiếm ít. Không chỉ nộp nhiều thuế hơn, mà là thuế suất cao hơn. Chúng tôi đã xác định rằng bảng sau đây thể hiện "phần đóng góp công bằng của họ":

Income	Tax rate
0–30,000	0%
30,001–100,000	15% of the amount over 30,000
100,001–250,000	10,500 + 20% of the amount over 100,000
250,001–500,000	40,500 + 30% of the amount over 250,000
>500,000	115,500 + 40% of the amount over 500,000

Of course, this changes the way taxes are calculated, so many of our previous tests are going to need deleting or changing. Since most of our tests use the salary of 50,000, let's create a constant for the tax on 50,000, named `tax_50K`, and then we'll use that in all the tests. That way, if the tax calculation changes again, we only have to change the constant instead of all the tests.

Tất nhiên, điều này làm thay đổi cách tính thuế, nên nhiều test trước đó của ta sẽ phải xóa đi hoặc sửa lại. Vì hầu hết test của ta dùng mức lương 50.000, hãy tạo một hằng số cho khoản thuế trên 50.000, đặt tên là `tax_50K`, rồi dùng nó trong mọi test. Như thế, nếu cách tính thuế lại thay đổi lần nữa, ta chỉ phải sửa hằng số thay vì sửa hết các test.

—bobolia_taxes_test.py—

```

...
tax_50K = 3000
class BoboliaTaxTests(unittest.TestCase):
    def setUp(self):
        self.tax_calculator = bobolia_taxes.TaxCalculator()
        self.tax_return = None

```

```

def make_return(self, args):
    tax_return = {"income": {"salary": 0},
                  "badbobs": {"class1": (),
                              "class2": ()}}

    if "income" in args:
        tax_return["income"].update(args["income"])
    if "badbobs" in args:
        tax_return["badbobs"].update(args["badbobs"])
    self.tax_return = tax_return
def make_simple_return(self, salary):
    self.make_return({"income": {"salary": salary}})
def get_tax(self):
    return self.tax_calculator.get_tax(self.tax_return)
def test_class_1_badbobs(self):
    self.make_return({"income": {"salary": 50000},
                      "badbobs": {"class1": (100,)}}
    )
    self.assertEqual(tax_50K + 50, self.get_tax())
def test_class_2_badbob_purchases_under_1001(self):
    self.make_return({"income": {"salary": 50000},
                      "badbobs": {"class2": (400, 600)}}
    )
    self.assertEqual(tax_50K, self.get_tax())
def test_class_2_badbob_purchases_under_10001(self):
    self.make_return({"income": {"salary": 50000},
                      "badbobs": {"class2": (10000,)}}
    )
    self.assertEqual(tax_50K + 2500, self.get_tax())
def test_class_2_badbob_purchases_under_50001(self):
    self.make_return({"income": {"salary": 50000},
                      "badbobs": {"class2": (10000, 40000)}}
    )
    self.assertEqual(tax_50K + 5000, self.get_tax())
def test_class_2_badbob_purchases_over_50000(self):
    self.make_return({"income": {"salary": 50000},
                      "badbobs": {"class2": (10000, 40000, 1000
    )
    )
    self.assertEqual(tax_50K + 7500, self.get_tax())
def test_badbobs_do_not_reduce_after_tax_income_below_20000(self):
    self.make_return({"income": {"salary": 20000},
                      "badbobs": {"class2": (10000,)}}
    )
    self.assertEqual(0, self.get_tax())
def test_0_30_tax_bracket(self):
    self.make_simple_return(15000)
    self.assertEqual(0, self.get_tax())

    self.make_simple_return(30000)
    self.assertEqual(0, self.get_tax())
def test_30_100_tax_bracket(self):

```

```

        self.make_simple_return(30001)
        self.assertEqual(0, self.get_tax())
        self.make_simple_return(50000)
        self.assertEqual(tax_50K, self.get_tax())
        self.make_simple_return(100000)
        self.assertEqual(10500, self.get_tax())
    def test_100_250_tax_bracket(self):
        self.make_simple_return(100001)
        self.assertEqual(10500, self.get_tax())
        self.make_simple_return(200000)
        self.assertEqual(30500, self.get_tax())
        self.make_simple_return(250000)
        self.assertEqual(40500, self.get_tax())
    def test_250_500_tax_bracket(self):
        self.make_simple_return(250001)
        self.assertEqual(40500, self.get_tax())
        self.make_simple_return(500000)
        self.assertEqual(115500, self.get_tax())
    def test_500_up_tax_bracket(self):
        self.make_simple_return(500001)
        self.assertEqual(115500, self.get_tax())

        self.make_simple_return(1000000)
        self.assertEqual(315500, self.get_tax())

```

I made quite a few changes to the tests to try to keep the noise down. I moved the `make_return` **method** into the test class, and I added `make_simple_return` and `get_tax`. I also made the `tax_return` into an instance **variable** of the test class so that I didn't have to keep assigning and passing it.

Tôi đã thực hiện khá nhiều thay đổi trên các test để giảm bớt nhiễu. Tôi chuyển method `make_return` vào trong class test, rồi thêm `make_simple_return` và `get_tax`. Tôi cũng biến `tax_return` thành một biến thực thể của class test để khỏi phải gán rồi truyền nó đi suốt.

The production code is a relatively simple change to the

Còn mã production thì chỉ là một thay đổi tương đối đơn giản ở

`determine_base_tax` method.

—bobolia_taxes.py—

```

...
def determine_base_tax(self):
    self.total_income = self.tax_return["income"]["salary"]
    if self.total_income <= 30000:
        return 0
    elif self.total_income <= 100000:
        return round(0.15 * (self.total_income - 30000))
    elif self.total_income <= 250000:
        return round(0.2 * (self.total_income - 100000) + 10500)
    elif self.total_income <= 500000:
        return round(0.3 * (self.total_income - 250000) + 40500)
    else:
        return round(0.4 * (self.total_income - 500000) + 115500)
...

```


Those tests are pretty ugly. Let's clean them up by using composed assertions.

Mấy cái test đó trông khá xấu xí. Hãy dọn dẹp chúng bằng cách dùng assertion ghép lại (composed assertions).

—bobolia_taxes_test.py—

```
...
def assert_tax_with_badbobs(self, income, badbobs, tax):
    self.make_return({"income": {"salary": income},
                      "badbobs": badbobs})
    self.assertEqual(tax, self.get_tax())
def test_class_1_badbobs(self):
    self.assert_tax_with_badbobs(50000,
                                  {"class1": (100,)},
                                  tax_50K + 50)
def test_class_2_badbob_purchases_under_1001(self):
    self.assert_tax_with_badbobs(50000,
                                  {"class2": (400, 600)},
                                  tax_50K)
def test_class_2_badbob_purchases_under_10001(self):
    self.assert_tax_with_badbobs(50000,
                                  {"class2": (10000,)},
                                  tax_50K + 2500)
def test_class_2_badbob_purchases_under_50001(self):
    self.assert_tax_with_badbobs(50000,
                                  {"class2": (10000, 40000)},
                                  tax_50K + 5000)
def test_class_2_badbob_purchases_over_50000(self):
    self.assert_tax_with_badbobs(50000,
                                  {"class2": (10000, 40000, 1000)},
                                  tax_50K + 7500)
def test_badbobs_do_not_reduce_after_tax_income_below_20000(self):
    self.assert_tax_with_badbobs(20000,
                                  {"class2": (10000,)},
                                  0)
def assert_tax_for(self, income, tax):
    self.make_simple_return(income)
    self.assertEqual(tax, self.get_tax())
def test_0_30_tax_bracket(self):
    self.assert_tax_for(15000, 0)
    self.assert_tax_for(30000, 0)
def test_30_100_tax_bracket(self):
    self.assert_tax_for(30001, 0)
    self.assert_tax_for(50000, tax_50K)
    self.assert_tax_for(100000, 10500)
def test_100_250_tax_bracket(self):
    self.assert_tax_for(100001, 10500)
    self.assert_tax_for(200000, 30500)
    self.assert_tax_for(250000, 40500)
def test_250_500_tax_bracket(self):
```

```

        self.assert_tax_for(250001, 40500)
        self.assert_tax_for(500000, 115500)
    def test_500_up_tax_bracket(self):
        self.assert_tax_for(500001, 115500)
        self.assert_tax_for(1000000, 315500)
    ...

```

That's much better. Those tests are now in a position to become parametric (table driven) should the need arise— but I don't think we're there yet.

Thế là khá hơn nhiều. Giờ mấy test này đã ở vị thế có thể chuyển thành dạng tham số hoá (điều khiển bằng bảng) nếu có nhu cầu — nhưng tôi nghĩ chúng ta chưa tới mức đó.

On the other hand, the production code is crying out for a table-driven approach. You don't think we've seen the last of the tax brackets, do you? So let's take care of that.

Ngược lại, mã production thì đang kêu gào đòi một cách tiếp cận điều khiển bằng bảng. Bạn không nghĩ rằng mình đã gặp các bậc thuế lần cuối cùng đấy chứ? Vậy thì hãy lo chuyện đó luôn.

—bobolia_taxes.py—

```

...
def determine_base_tax(self):
    tax_brackets = ((30000, 0),
                    (100000, .15),
                    (250000, .20),
                    (500000, .30),
                    (None, .40))
    self.total_income = self.tax_return["income"]["salary"]
    previous_maximum = 0
    offset = 0
    for maximum, rate in tax_brackets:
        if maximum is None or self.total_income <= maximum:
            return round(
                rate *
                (self.total_income - previous_maximum) +
                offset)
        else:
            offset = (maximum - previous_maximum) * rate + offset
            previous_maximum = maximum
    ...

```

OK, that's better. Any new bracket can be added easily.

Được rồi, thế là khá hơn. Bậc thuế mới nào cũng có thể thêm vào dễ dàng.

At this point, you might ask whether the tests ought to use the same `tax_brackets` table. I'm usually not in favor of allowing the tests to know the details of the production code. The whole point of tests is that they are a second statement of intent. It makes little sense to use the production code to test the production code. So I prefer to do the calculations manually and enter them into the tests.

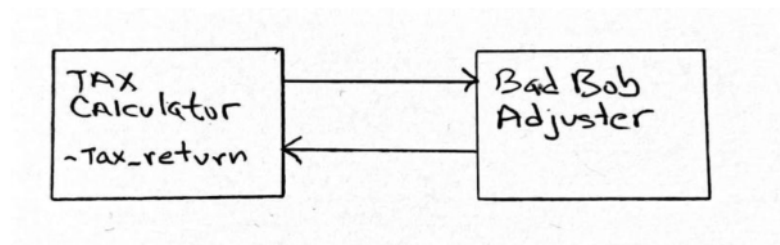
Tới đây, có lẽ bạn sẽ hỏi liệu các test có nên dùng chính bảng `tax_brackets` đó không. Thường thì tôi không ủng hộ việc để test biết chi tiết của mã production. Toàn bộ điểm

mẫu chốt của test là chúng là một lời phát biểu ý định thứ hai. Chẳng có mấy ý nghĩa khi dùng mã production để kiểm thử chính mã production. Vì vậy tôi thích tự tính tay rồi nhập kết quả vào test hơn.

Considering the Design and Architecture — Ngẫm về thiết kế và kiến trúc

Now let's take a step back. So far you have watched me write a small module using a process that keeps it “clean” and test as it grows and is changed. I hope the process has been illuminating. The module has grown into two classes that have the following relationships.

Bây giờ hãy lùi lại một bước. Từ đầu tới giờ bạn đã theo dõi tôi viết một module nhỏ bằng một quy trình giữ cho nó luôn “sạch” và luôn được test trong lúc nó lớn dần lên và thay đổi. Tôi hy vọng quy trình đó đã cho bạn nhiều điều sáng tỏ. Module đã lớn lên thành hai class có quan hệ như sau.

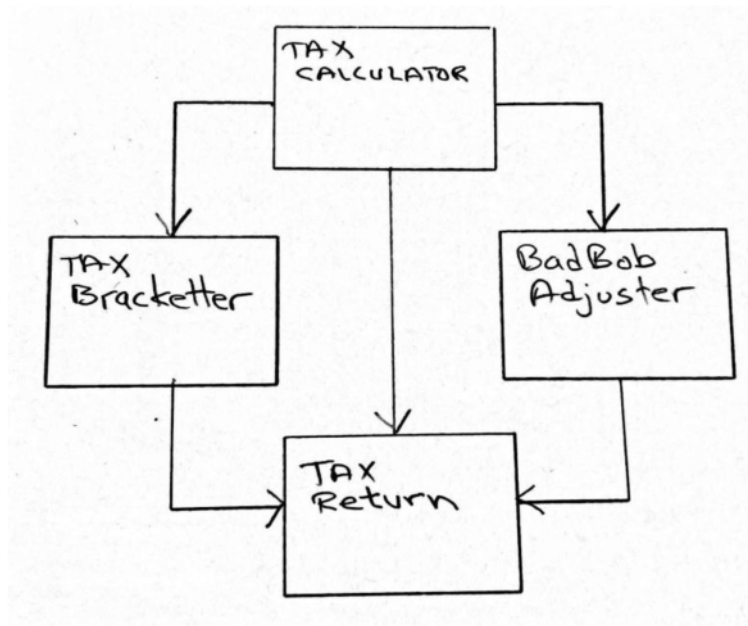


First of all, I don't like that reciprocal relationship. It only exists because the BadBobAdjuster uses the tax_return field of the TaxCalculator. Second, I think this is a bit asymmetric. Why does the BadBobAdjuster deserve its own special class whereas the tax bracket calculations are buried in the

Trước hết, tôi không thích mối quan hệ qua lại đó. Nó tồn tại chỉ vì BadBobAdjuster dùng field tax_return của TaxCalculator. Thứ hai, tôi thấy chỗ này hơi bất đối xứng. Tại sao BadBobAdjuster lại xứng đáng có class riêng của mình, trong khi phần tính bậc thuế thì bị chôn vùi trong

TaxCalculator? Let's clean that up a bit. I'd like it to look like this.

TaxCalculator? Hãy dọn dẹp chỗ đó một chút. Tôi muốn nó trông như thế này.



This change requires only minor changes to the tests, so our test design is holding up.

Thay đổi này chỉ đòi hỏi vài chỉnh sửa nhỏ trong test, nên thiết kế test của chúng ta vẫn đứng vững.

—bobolia_taxes_test.py—

```

...
def make_return(self, args):
    tax_return_data = {"income": {"salary": 0},
                       "badbobs": {"class1": (),
                                    "class2": ()}}

    if "income" in args:
        tax_return_data["income"].update(args["income"])
    if "badbobs" in args:
        tax_return_data["badbobs"].update(args["badbobs"])

    self.tax_return = bobolia_taxes.TaxReturn(tax_return_data)
...

```

—bobolia_taxes.py—

```

class TaxCalculator:
    def get_tax(self, tax_return):
        total_income = tax_return.get_total_income()
        base_tax = TaxBracketter(tax_return).determine_base_tax()
        badbob_adjustment = BadBobAdjuster(tax_return).get_adjustme
        total_tax = base_tax + badbob_adjustment
        after_tax_income = total_income - total_tax
        if after_tax_income < 20000:
            total_tax = max(0, total_income - 20000)
        return total_tax

class TaxBracketter:
    tax_brackets = ((30000, 0),
                    (100000, .15),
                    (250000, .20),

```

```

        (500000, .30),
        (None, .40))
def __init__(self, tax_return):
    self.tax_return = tax_return
def determine_base_tax(self):
    total_income = self.tax_return.get_total_income()
    previous_maximum = 0
    offset = 0
    for maximum, rate in self.tax_brackets:
        if maximum is None or total_income <= maximum:
            return round(rate *
                          (total_income - previous_maximum) +
                          offset)
        else:
            offset = (maximum - previous_maximum) * rate + off
            previous_maximum = maximum
class BadBobAdjuster:
    def __init__(self, tax_return):
        self.tax_return = tax_return
        self.total_income = tax_return.get_total_income()
    def get_adjustment(self):
        class1_adjustment = self.class1_adjustment()
        class2_adjustment = self.get_class2_badbob_adjustment()
        return class1_adjustment + class2_adjustment
    def get_class2_badbob_adjustment(self):
        class2 = self.tax_return.get_total_class2_badbobs()
        class2_rate = self.get_class2_rate(class2)
        return class2_rate * self.total_income
class2_table = ((1001, 0),
                 (10001, 0.05),
                 (50001, 0.1),
                 (None, 0.15))
def get_class2_rate(self, class2):
    for limit, rate in self.class2_table:
        if limit is None or class2 < limit:
            return rate
    return None
def class1_adjustment(self):
    class1 = self.tax_return.get_total_class1_badbobs()
    return class1 / 2
class TaxReturn:
    def __init__(self, tax_return_data):
        self.tax_return_data = tax_return_data
    def get_total_income(self):
        return self.tax_return_data["income"]["salary"]
    def get_total_class1_badbobs(self):
        return sum(self.tax_return_data["badbobs"]["class1"])
    def get_total_class2_badbobs(self):
        return sum(self.tax_return_data["badbobs"]["class2"])

```

At this point, you might be accusing this design of classitis 2 —the tendency to create too many classes and thereby complicate the design. I disagree. The three new classes do not add any complexity that wasn't already there. They just move that complexity into nicely named places.

Tới đây, có thể bạn đang buộc tội thiết kế này mắc chứng classitis 2 — thói quen tạo ra quá nhiều class và vì thế làm thiết kế rối rắm. Tôi không đồng ý. Ba class mới không thêm vào chút phức tạp nào mà trước đó chưa có. Chúng chỉ dời cái phức tạp ấy vào những chỗ được đặt tên tử tế.

2 [APOD], p. 26.

2 [APOD], tr. 26.

A place for everything, and everything in its place!

Mọi thứ đều có chỗ của nó, và mọi thứ đều nằm đúng chỗ!

One major advantage of the TaxReturn class is that it hides the internal **data structure** of the tax_return_data from the rest of the application. When there are changes to that **data** structure, the other modules in the application will not need to be changed.

Một lợi thế lớn của class TaxReturn là nó che giấu cấu trúc dữ liệu (data structure) nội bộ của tax_return_data khỏi phần còn lại của ứng dụng. Khi cấu trúc dữ liệu đó thay đổi, các module khác trong ứng dụng sẽ không cần phải sửa.

In any case, the advantage of those classes should become evident soon; because it's time for us to take yet another step back and think about the overall **architecture** of this system.

Dù sao đi nữa, lợi thế của những class ấy sẽ sớm hiện ra rõ ràng thôi; vì đã đến lúc chúng ta lùi thêm một bước nữa và nghĩ về kiến trúc (architecture) tổng thể của hệ thống này.

It seems to me that this system has two axes of change so far. There's the computation of the base tax, which is done by the TaxBracketter ; and then there's the computation of the social demerit score, accomplished by the BadBobAdjuster . If we expect this system to grow (and what tax system doesn't grow?), then we may want to establish these axes of change within the architecture of our system. Or, to say that differently, we may wish to draw architectural boundaries that isolate and protect them.

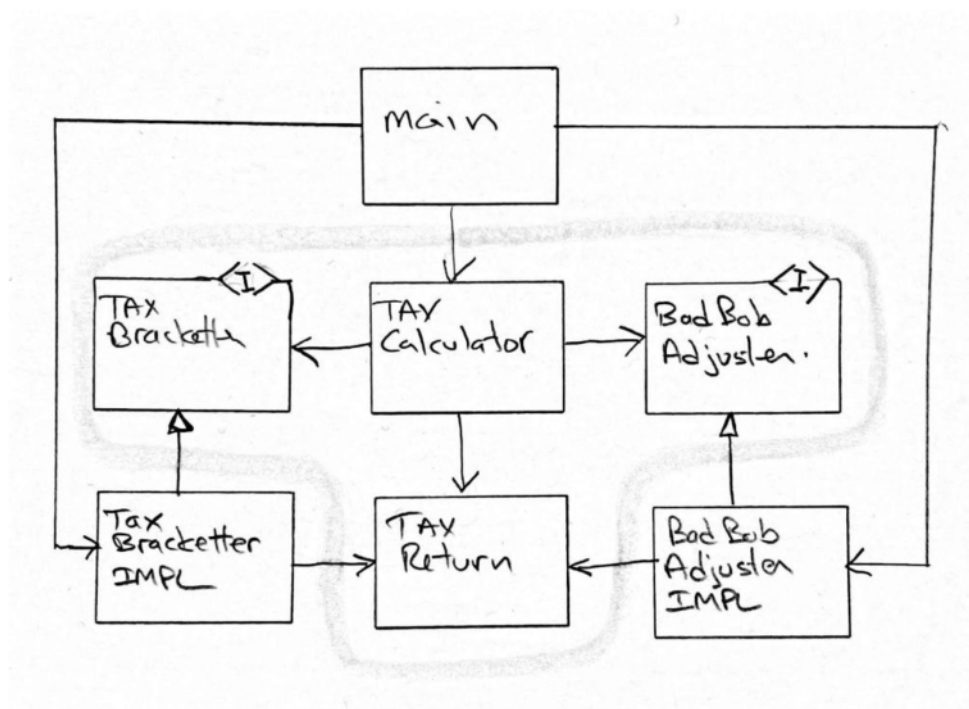
Theo tôi thấy thì cho tới lúc này hệ thống có hai trục thay đổi. Một là việc tính thuế cơ sở, do TaxBracketter đảm nhiệm; hai là việc tính điểm phạt xã hội, do BadBobAdjuster lo. Nếu chúng ta dự liệu rằng hệ thống này sẽ còn lớn thêm (mà hệ thống thuế nào lại chẳng lớn thêm?), thì có lẽ ta nên thiết lập sẵn hai trục thay đổi ấy ngay trong kiến trúc (architecture) của hệ thống. Hay nói cách khác, ta sẽ muốn vẽ ra những ranh giới kiến trúc (architectural boundary) để cô lập và bảo vệ chúng.

We'll use the Single Responsibility Principle (SRP) to do that, and we'll use the **Dependency** Inversion Principle (DIP) to create a structure like this. 3

Chúng ta sẽ dùng Single Responsibility Principle (SRP) để làm điều đó, và dùng Dependency Inversion Principle (DIP) để tạo ra một cấu trúc như thế này. 3

3 See Chapter 19 , “ **The SOLID Principles** .”

3 Xem Chương 19 , ” Các nguyên tắc SOLID .”



Note the **architectural boundary** drawn around the four central classes. Notice also that every arrow that crosses that boundary points inward, following the Dependency Rule 4 of architecture. Inside that boundary is the high-level policy of this system. Outside are lower-level details. The main **component** is the lowest level in the system. It creates the implementations of the TaxBracketter and BadBobAdjuster and hands them to the TaxCalculator prior to any taxes being calculated.

Hãy để ý ranh giới kiến trúc được vẽ quanh bốn class trung tâm. Cũng hãy để ý rằng mọi mũi tên cắt qua ranh giới ấy đều chỉ vào trong, đúng theo Dependency Rule 4 của kiến trúc. Bên trong ranh giới đó là chính sách mức cao (high-level policy) của hệ thống. Bên ngoài là những chi tiết ở mức thấp hơn. Component main là mức thấp nhất trong hệ thống. Nó tạo ra các hiện thực (implementation) của TaxBracketter và BadBobAdjuster rồi trao chúng cho TaxCalculator trước khi bất kỳ khoản thuế nào được tính.

4 See Chapter 26, “**The Clean Architecture**.” The TaxCalculator uses the Strategy 5 pattern to delegate the bracket and demerit calculations to the appropriate objects. The canonical form of Strategy calls for the strategy implementations to derive from interfaces. However, in Python, interfaces do not need to exist as independent source code entities. Python is dynamically typed, and so polymorphic dispatch is through duck types. So the “interfaces” shown above are just the method signatures that the TaxCalculator expects to invoke.

4 Xem Chương 26, “**Clean Architecture**.” TaxCalculator dùng mẫu Strategy 5 để uỷ thác việc tính bậc thuế và tính điểm phạt cho những đối tượng thích hợp. Dạng chuẩn mực của Strategy đòi hỏi các hiện thực strategy phải dẫn xuất từ interface. Tuy nhiên, trong Python, interface không nhất thiết phải tồn tại như những thực thể mã nguồn độc lập. Python định kiểu động, nên việc điều phối đa hình (polymorphism) diễn ra thông qua duck type. Vậy nên những “interface” ở hình trên chỉ là các method signature mà TaxCalculator kỳ vọng sẽ gọi tới.

5 [GOF95].

5 [GOF95].

The role of the main component will be played by our tests.

Vai trò của component main sẽ do các test của chúng ta đảm nhiệm.

—bobolia_taxes_test.py—

```
def setUp(self):
    self.tax_bracketter = tax_bracketter.TaxBracketter()
    self.badbob_adjuster = badbob_adjuster.BadBobAdjuster()
    self.tax_calculator = tax_calculator.TaxCalculator(
        self.tax_bracketter, self.badbob_adjuster)
    self.tax_return = None
```

The TaxCalculator and TaxReturn exist in their own module.

TaxCalculator và TaxReturn nằm trong module riêng của chúng.

—tax_calculator.py—

```
class TaxCalculator:
    def __init__(self, tax_bracketter, badbob_adjuster):
        self.tax_bracketter = tax_bracketter
        self.badbob_adjuster = badbob_adjuster
    def get_tax(self, tax_return):
        total_income = tax_return.get_total_income()
        base_tax = self.tax_bracketter.determine_base_tax(tax_return)
        badbob_adjustment =
            self.badbob_adjuster.get_adjustment(tax_return)
        total_tax = base_tax + badbob_adjustment
        after_tax_income = total_income - total_tax
        if after_tax_income < 20000:
            total_tax = max(0, total_income - 20000)
        return total_tax
class TaxReturn:
    def __init__(self, tax_return_data):
        self.tax_return_data = tax_return_data
    def get_total_income(self):
        return self.tax_return_data["income"]["salary"]
    def get_total_class1_badbobs(self):
        return sum(self.tax_return_data["badbobs"]["class1"])
    def get_total_class2_badbobs(self):
        return sum(self.tax_return_data["badbobs"]["class2"])
```

Then come the TaxBracketter and BadBobAdjuster modules.

Rồi tới các module TaxBracketter và BadBobAdjuster.

—tax_bracketter.py—

```
class TaxBracketter:
    tax_brackets = ((30000, 0),
                    (100000, .15),
                    (250000, .20),
                    (500000, .30),
                    (None, .40))
    def determine_base_tax(self, tax_return):
```

```

total_income = tax_return.get_total_income()
previous_maximum = 0
offset = 0
for maximum, rate in self.tax_brackets:
    if maximum is None or total_income <= maximum:
        return round(rate *
                      (total_income - previous_maximum) +
                      offset)
    else:
        offset = (maximum - previous_maximum) * rate + off
        previous_maximum = maximum
—badbob_adjuster.py—
class BadBobAdjuster:
    def __init__(self):
        self.total_income = None
        self.tax_return = None
    def get_adjustment(self, tax_return):
        self.tax_return = tax_return
        self.total_income = tax_return.get_total_income()
        class1_adjustment = self.class1_adjustment()
        class2_adjustment = self.get_class2_badbob_adjustment()

        return class1_adjustment + class2_adjustment
    def get_class2_badbob_adjustment(self):
        class2 = self.tax_return.get_total_class2_badbobs()
        class2_rate = self.get_class2_rate(class2)
        return class2_rate * self.total_income
    class2_table = ((1001, 0),
                    (10001, 0.05),
                    (50001, 0.1),
                    (None, 0.15))
    def get_class2_rate(self, class2):
        for limit, rate in self.class2_table:
            if limit is None or class2 < limit:
                return rate
        return None
    def class1_adjustment(self):
        class1 = self.tax_return.get_total_class1_badbobs()
        return class1 / 2

```

Conclusion — Kết luận

Is this clean? I'm pretty happy with it, though I'm not a Python programmer, so I've likely violated a lot of conventions and probably have not used the most efficient techniques. Still, I think some major goals have been accomplished. We've separated the system into at least four major components: main , TaxCalculator , TaxBracketter , and

Thế này đã sạch chưa? Tôi khá hài lòng với nó, dù tôi không phải dân Python, nên nhiều khả năng tôi đã vi phạm cả đồng quy ước và hẳn là chưa dùng những kỹ thuật hiệu quả nhất. Dù vậy, tôi nghĩ chúng ta đã đạt được vài mục tiêu quan trọng. Chúng ta đã tách hệ

thống thành ít nhất bốn component chính: `main` , `TaxCalculator` , `TaxBracketter` , và `BadBobAdjuster` . And we placed those components behind appropriate architectural boundaries.

`BadBobAdjuster` . Và chúng ta đã đặt các component đó sau những ranh giới kiến trúc thích hợp.

Our tests have survived a significant amount of change and have showed themselves to be resilient. The same is true of our basic algorithms. Changes to one are very unlikely to affect the others. We've also built in some flexibility by using tables to drive the algorithms.

Các test của chúng ta đã sống sót qua một lượng thay đổi đáng kể và tự chứng tỏ được sức bền của mình. Các thuật toán cơ bản cũng vậy. Thay đổi ở cái này rất khó lòng ảnh hưởng tới những cái kia. Chúng ta cũng đã xây sẵn một chút linh hoạt bằng cách dùng bảng để điều khiển thuật toán.

As the tests grew in specificity, the production code grew in generality.

Khi các test lớn dần về mức cụ thể, thì mã production lớn dần về mức tổng quát.

Some folks might complain about the names. I dunno, I kinda like `TaxBracketter 6` and `BadBobAdjuster` . But if you play with this code, you might find some better names.

Sẽ có người phàn nàn về mấy cái tên. Chẳng biết nữa, tôi thì thấy `TaxBracketter 6` và `BadBobAdjuster` cũng hay hay. Nhưng nếu bạn nghịch thử mã này, biết đâu bạn lại tìm ra vài cái tên khác hơn.

6 Despite the spelling error.

6 Dù có lỗi chính tả.

I have to say that I was pleased with IntelliJ's Python plug-in and Copilot's ability to help someone who hadn't written Python in over 20 years. I also have to say that I was impressed with the responsiveness of the tests. I'm used to JVM startup times, and it's a joy to have the test results pop up on the screen before my fingers have left the test button.

Tôi phải nói rằng tôi rất hài lòng với plug-in Python của IntelliJ và khả năng của Copilot trong việc trợ giúp một kẻ đã hơn 20 năm không viết Python. Tôi cũng phải nói rằng tôi ấn tượng với độ nhạy của các test. Tôi vốn quen với thời gian khởi động của JVM, nên thật là sung sướng khi thấy kết quả test hiện lên màn hình trước cả lúc ngón tay tôi rời khỏi nút chạy test.

Chapter 10. One Thing — Chương 10. Một việc duy nhất



Do you remember what Bjarne Stroustrup said about **clean code**? He said, “Clean code does one thing well.” And, indeed, many other software authors have said things like that over the years. The problem with that statement is: What does one thing mean?

Bạn còn nhớ Bjarne Stroustrup đã nói gì về clean code không? Ông bảo: “Clean code làm tốt một việc duy nhất.” Và quả thật, suốt bao năm qua có rất nhiều tác giả phần mềm khác cũng nói những điều tương tự. Vấn đề của câu nói ấy là: một việc duy nhất nghĩa là gì?

As I said in a previous chapter, I was once responsible for a C **function** that was 3,000 lines long. It was named `gi`, which stood for graphic interpreter. Had anyone challenged me about whether my 3,000-line function did one thing, I would have confidently replied: “Certainly! It interprets graphics.”

Như tôi đã kể ở một chương trước, có thời tôi phụ trách một hàm C dài 3.000 dòng. Nó tên là `gi`, viết tắt của graphic interpreter. Nếu khi đó có ai chất vấn rằng cái hàm 3.000 dòng của tôi có làm một việc duy nhất hay không, tôi đã đáp lại đầy tự tin: “Chắc chắn rồi! Nó thông dịch đồ họa.”

Of course, a 3,000-line function must be doing a lot of things while it interprets graphics. I’m quite convinced that the problem of interpreting graphics can be broken down into quite a few things.

Dĩ nhiên, một hàm dài 3.000 dòng ắt phải làm khối thứ trong lúc thông dịch đồ họa. Tôi khá chắc rằng bài toán thông dịch đồ họa có thể chia nhỏ thành khá việc.

So how do we resolve this issue? If the advice is to be followed, we must have some reasonable definition of one thing. I think I know what that definition is, and I’m going to tell you now.

Vậy gỡ chuyện này ra sao? Nếu muốn làm theo lời khuyên đó, ta phải có một định nghĩa hợp lý cho một việc duy nhất. Tôi nghĩ mình biết định nghĩa ấy là gì, và tôi sẽ nói cho bạn ngay bây giờ.

But, like Deep Thought said in Hitchhiker’s Guide to the Galaxy: “You’re not going to like it.”

Nhưng, như Deep Thought đã nói trong Hitchhiker’s Guide to the Galaxy : “Bạn sẽ không thích nó đâu.”

Extract Method Refactoring — Refactoring Extract Method

Does your IDE have a **Refactor** menu? Mine does. One of the menu items is **Extract Method** . IDEs have had this menu item for two decades now. I’m sure you’ve all used it. But just in case, let me describe it for you.

IDE của bạn có menu Refactor không? IDE của tôi thì có. Một trong các mục của menu ấy là Extract Method . Các IDE đã có mục menu này suốt hai thập kỷ nay rồi. Tôi chắc rằng ai cũng từng dùng nó. Nhưng phòng khi chưa, để tôi mô tả cho bạn nghe.

If you see a function that’s a little too big, and you decide you are going to split it into two functions, you can use your mouse to select the code you want to pull out (extract) from the parent function. 1 Then, you select the Extract **Method refactoring** and up pops a dialog box that asks you what name you want for the new function. That box also tells you what arguments it plans to pass into the new function, and also what it plans to return. The dialog box gives you the option to change a few of these things. When you are ready, you hit the Refactor button on that dialog box, and the IDE cuts the selected code out of the original function and replaces it with a call to the new function. Then, it pastes the selected code into that new function and creates the glue code that allows it to be called.

Nếu bạn thấy một hàm hơi phình to, và bạn quyết định tách nó thành hai hàm, bạn có thể dùng chuột bôi đen đoạn mã muốn lôi (extract) ra khỏi hàm cha. 1 Rồi bạn chọn refactoring Extract Method, và một hộp thoại hiện lên hỏi bạn muốn đặt tên gì cho hàm mới. Hộp thoại đó cũng cho bạn biết nó định truyền những đối số nào vào hàm mới, và nó định trả về cái gì. Hộp thoại còn cho phép bạn chỉnh sửa vài thứ trong số đó. Khi đã sẵn sàng, bạn bấm nút Refactor trên hộp thoại, và IDE cắt đoạn mã đã chọn ra khỏi hàm gốc rồi thay bằng một lời gọi tới hàm mới. Sau đó, nó dán đoạn mã ấy vào hàm mới và tạo luôn phần mã kết nối để hàm đó gọi được.

1 In 1974, no programmer could have made sense of that sentence. Forty years ago, some might have had a glimmer.

1 Vào năm 1974, không lập trình viên nào hiểu nổi câu vừa rồi. Bốn mươi năm trước, có lẽ vài người đã lờ mờ hình dung được.

I mean . . . it’s magic!

Ý tôi là... đúng là phép màu!

So, what does one thing mean?

Vậy, một việc duy nhất nghĩa là gì?

A function does one thing if you cannot meaningfully extract another function from it.

Một hàm làm một việc duy nhất nếu bạn không thể extract thêm một hàm nào có ý nghĩa ra khỏi nó nữa.

OK, now disregard the word meaningfully for just a moment and think about that statement. Can you refute it? I don’t think you can. If you can split one function into two, then the original function did two things. So, a function that does one thing is a function from which you cannot extract another.

Được rồi, giờ hãy tạm gạt chữ có ý nghĩa sang một bên và ngẫm về câu vừa nói. Bạn bác bỏ được không? Tôi không nghĩ là được. Nếu bạn tách được một hàm thành hai, thì hàm

bạn đầu đã làm hai việc. Vậy, một hàm làm một việc duy nhất là hàm mà từ đó bạn không extract ra được hàm nào nữa.

OK, now, what does the word meaningfully mean in this context? Consider this function:

Được rồi, thế thì chữ có ý nghĩa mang nghĩa gì trong ngữ cảnh này? Hãy xem hàm sau:

```
public void addRental (Rental rental) {  
    rentals.addElement (rental);  
}
```

It is not meaningful to extract the body of that function into a new function:

Việc extract phần thân của hàm đó thành một hàm mới chẳng có ý nghĩa gì:

```
public void addRental (Rental rental) {  
    doTheAdd(rental);  
}  
public void doTheAdd (Rental rental) {  
    rentals.addElement (rental);  
}
```

There is no point in extracting the entire body of a function, leaving behind a pure delegator. Such an extraction is not meaningful. 2

Chẳng ích gì khi extract toàn bộ thân hàm, để lại đằng sau một kẻ chỉ biết uỷ quyền thuần tuý. Một phép extract như thế là vô nghĩa. 2

2 Except in those rare circumstances where you are creating a delegator so that you can move the extracted function to a new **module**.

2 Trừ những trường hợp hiếm hoi khi bạn cố tình tạo ra một delegator để có thể chuyển hàm vừa extract sang một module khác.

Or consider this function:

Hoặc hãy xét hàm này:

```
void clearTotals() {  
    totalAmount=0;  
    renterPoints=0;  
}
```

It would almost certainly be meaningless to extract those two lines into two different functions:

Gần như chắc chắn sẽ là vô nghĩa nếu extract hai dòng đó thành hai hàm khác nhau:

```
void clearAmount() {totalAmount=0;}  
void clearPoints() {renterPoints=0;}
```

This has taken extraction too far. The clearAmount and

Thế này là extract quá đà rồi. Hai hàm clearAmount và

clearPoints functions have names that are indistinguishable from their implementations. A meaningful **abstraction** is one that follows **The Stepdown Rule**, where the name of a function is more abstract than the **implementation**.

clearPoints có tên không thể phân biệt được với chính phần hiện thực (implementation) của chúng. Một trừu tượng hoá (abstraction) có ý nghĩa là trừu tượng hoá tuân theo Quy

tắc bậc thang (the Stepdown Rule), trong đó tên hàm phải trừu tượng hơn phần hiện thực của nó.

I'll leave the rest of the interpretation of the word meaningful up to you. Suffice it to say that you can extract too little, and you can extract too much.

Phần diễn giải còn lại về chữ có ý nghĩa, tôi xin nhường cho bạn. Chỉ cần nói rằng bạn có thể extract quá ít, và bạn cũng có thể extract quá nhiều.

So how do we ensure that all our functions do one thing? We extract, and extract, until we cannot meaningfully extract anymore. Or, in other words:

Vậy làm sao để chắc rằng mọi hàm của ta đều làm một việc duy nhất? Ta cứ extract, rồi extract nữa, cho tới khi không thể extract thêm gì có ý nghĩa. Hay nói cách khác:

Extract till you drop!

Extract cho tới khi gục ngã!

OK, perhaps that's a little cheeky. I don't really want you to drop. On the other hand, I do want you to consider as many extractions as you can. You may not choose to do them all, but you should at least think about them.

Được rồi, câu ấy có lẽ hơi láu cá. Tôi đâu thật lòng muốn bạn gục ngã. Mặt khác, tôi thật sự muốn bạn cân nhắc càng nhiều phép extract càng tốt. Bạn có thể chọn không làm hết, nhưng ít nhất bạn nên nghĩ tới chúng.

As a consequence, the majority of your functions will be pretty small. How small? A few lines. That number might be three, or four, or six. It might be larger for certain switch statements, or certain **formatting** statements, or in certain languages. But, in general, following this recommendation will cause you to create a lot of small functions.

Hệ quả là phần lớn hàm của bạn sẽ khá nhỏ. Nhỏ cỡ nào? Vài dòng. Con số đó có thể là ba, bốn, hay sáu. Nó có thể lớn hơn với một số câu lệnh switch, hoặc một số câu lệnh định dạng, hoặc trong một số ngôn ngữ nhất định. Nhưng nhìn chung, làm theo khuyến nghị này sẽ khiến bạn tạo ra rất nhiều hàm nhỏ.

And what will you be extracting? Often you'll be extracting the bodies of if / else statements and for / while loops. You'll also likely be extracting the predicates out of those statements so that those statements will be composed of little more than the keywords and the function calls.

Và bạn sẽ extract những gì? Thường thì bạn sẽ extract thân của các câu lệnh if / else và các vòng lặp for / while. Nhiều khả năng bạn cũng sẽ extract luôn các biểu thức điều kiện ra khỏi những câu lệnh đó, để chúng chỉ còn lại vốn vẹn mấy từ khoá và các lời gọi hàm.

For example:

Ví dụ:

```
if (shouldDeleteRecord(r))  
    deleteRecord(r);
```

If you read that carefully, you'll see that it reads like well-written prose .

Nếu đọc kỹ, bạn sẽ thấy nó đọc lên nghe như một áng văn xuôi được viết chẵn chu .

This Shouldn't Be Controversial — Chuyện này lẽ ra không nên gây tranh cãi

Over the years, this particular recommendation, from the first edition of this book, has been the most controversial. But why? There are a lot of reasons.

Bao năm qua, chính khuyến nghị này, kể từ lần xuất bản đầu tiên của cuốn sách, lại là điều gây tranh cãi nhiều nhất. Nhưng vì sao? Có khối lý do.

1. People are afraid that if they follow this rule, they will drown beneath a sea of tiny little functions.
2. People are afraid that following this rule will obscure the intent of a function because they won't be able to see it all in one place.
3. People worry that the performance penalty of calling all those tiny little functions will cause their applications to run like molasses in the wintertime.
4. People think they're going to have to be bouncing around, from function to function, in order to understand anything.
 1. Người ta sợ rằng nếu làm theo quy tắc này, họ sẽ chết chìm dưới một biển hàm bé tí.
 2. Người ta sợ rằng làm theo quy tắc này sẽ che mờ ý định của một hàm, vì họ không còn nhìn thấy toàn bộ nó ở một chỗ nữa.
 3. Người ta lo rằng cái giá hiệu năng của việc gọi ngần ấy hàm bé tí sẽ khiến ứng dụng chạy chậm như mật đường giữa mùa đông.
 4. Người ta nghĩ họ sẽ phải nhảy qua nhảy lại, từ hàm này sang hàm khác, chỉ để hiểu được một chuyện gì đó.
5. People are concerned that the resulting functions will be too entangled to understand in isolation.
 5. Người ta e rằng các hàm tạo ra sẽ vướng víu vào nhau tới mức không thể hiểu được khi tách riêng.

Let's address these one at a time.

Hãy giải quyết từng điểm một.

1. Drowning — 1. Chết chìm

Long ago our languages were very primitive. The first version of Fortran, for example, allowed you to write one function. And it wasn't called a function. It was just called a program. There were no modules, no functions, and no subroutines. As I said, it was primitive.

Xa xưa, các ngôn ngữ của chúng ta rất thô sơ. Chẳng hạn phiên bản Fortran đầu tiên chỉ cho phép bạn viết đúng một hàm. Mà nó còn chẳng được gọi là hàm. Nó chỉ được gọi là chương trình. Không có module, không có hàm, cũng chẳng có subroutine. Như tôi đã nói, nó thô sơ lắm.

COBOL wasn't much better. You could jump around from one paragraph to the next using a goto statement; but the idea of functions that you could call with arguments was still a decade in the future.

COBOL cũng chẳng khá hơn là bao. Bạn có thể nhảy từ paragraph này sang paragraph khác bằng một câu lệnh goto; nhưng ý tưởng về những hàm mà bạn có thể gọi kèm đối số thì vẫn còn cách đó cả một thập kỷ nữa.

When Algol, and later, C, came along, the idea of functions burst upon us like a wave. Suddenly we could create modules that could be individually compiled. Modular programming became all the rage. 3

Khi Algol, rồi sau đó là C, xuất hiện, ý tưởng về hàm ập đến với chúng ta như một con sóng. Bỗng chốc ta có thể tạo ra những module biên dịch riêng lẻ được. Lập trình theo module trở thành một thịnh hành. 3

3 And as with all things in software, it was badly abused. Some managers insisted that no module should be longer than 500 bytes. So programmers were forced to put a goto at the end of their modules in order to call the next.

3 Và như mọi thứ trong phần mềm, nó bị lạm dụng tệ hại. Vài vị quản lý khẳng rằng không module nào được dài quá 500 byte. Thế là lập trình viên buộc phải nhét một lệnh goto ở cuối module để gọi sang module tiếp theo.

But as the years went by, and our programs got larger and larger, we began to realize that functions weren't enough. Our names started to collide. We were having trouble keeping them unique. And we really didn't like the idea of prefixing every name with the name of its module. I mean, nobody likes `io_files_open` as a function name.

Nhưng năm tháng trôi qua, chương trình của ta ngày một phình to, và ta bắt đầu nhận ra rằng chỉ có hàm thôi thì chưa đủ. Tên bắt đầu đụng nhau. Ta chật vật giữ cho chúng là duy nhất. Mà ta thì thật sự không ưa cái ý tưởng gắn tên module vào trước mọi cái tên. Ý tôi là, chẳng ai thích `io_files_open` làm tên hàm cả.

So we added classes and namespaces to our languages. These gave us lovely little cubbyholes that had names into which we could place our functions and our variables. Moreover, these namespaces and classes quickly became hierarchical. We could have functions within namespaces that were within yet other namespaces.

Thế là ta thêm class và namespace vào ngôn ngữ. Chúng cho ta những cái ngăn kéo xinh xắn có tên hẳn hoi để cất hàm và biến vào. Hơn nữa, những namespace và class ấy nhanh chóng trở nên phân cấp. Ta có thể có hàm nằm trong namespace, mà namespace đó lại nằm trong namespace khác.

And this is the solution for the “sea of tiny little functions” fear. You aren't going to drown beneath that sea, because there won't be a sea. Instead, you'll have a nicely arranged and nicely named hierarchy of namespaces and classes into which you can tuck your nicely named functions. Those nice names will be the guideposts that other programmers will use to locate your functions. Better still, those guideposts will provide the context that other programmers will use to understand your functions.

Và đó chính là lời giải cho nỗi sợ “biển hàm bé tí”. Bạn sẽ không chết chìm dưới biển ấy đâu, bởi vì sẽ chẳng có biển nào cả. Thay vào đó, bạn sẽ có một hệ phân cấp namespace và class được sắp xếp gọn gàng, đặt tên tử tế, để nhét những cái hàm đã được đặt tên tử tế của bạn vào. Những cái tên tử tế ấy sẽ là cột mốc chỉ đường để lập trình viên khác tìm ra hàm của bạn. Hay hơn nữa, những cột mốc đó sẽ cung cấp ngữ cảnh để lập trình viên khác hiểu được hàm của bạn.

Let me say this a different way. Some of us really like working on largish functions. There's something pleasing about the shape of those functions. We like the undulating in and out of the indentations. Something in our hindbrain finds comfort in it.

Để tôi nói theo một cách khác. Vài người trong chúng ta thật sự thích làm việc với những hàm to to. Có gì đó dễ chịu ở hình dáng của những hàm ấy. Ta thích cái nhịp nhấp nhô ra vào của các mức thụt lề. Có thứ gì đó trong phần não cổ của ta thấy an ủi khi nhìn nó.

Why? Perhaps because, if you rotate the listing to the right by 90°, it looks like the horizon. Those undulations become peaks and valleys. And we, hominids who evolved on the savannah, used those shapes on the horizon to determine our location. We knew that between the two peaks to the east there was a watering hole; and the valley on the right was home to a ravenous saber-toothed tiger.

Vì sao? Có lẽ vì, nếu bạn xoay đoạn mã sang phải 90°, nó trông như đường chân trời. Những gợn nhấp nhô ấy hoá thành đỉnh núi và thung lũng. Còn chúng ta, những sinh vật họ người tiến hoá trên thảo nguyên, đã dùng chính những hình dáng trên đường chân trời để xác định vị trí của mình. Ta biết rằng giữa hai đỉnh núi phía đông có một vũng nước; và thung lũng bên phải là nhà của một con hổ răng kiếm đói mồi.

Remember my gi function? I knew that function like the back of my hand. I knew it geographically ! If someone asked me to change the scaling on the x-axis, I would have said to myself: “The scaling on the x-axis . . . that’s in the third indent after the big **comment** block.” And I would have scrolled down to the big comment block and counted 1..2..3 and there it would be—the code that set the scaling on the x-axis.

Còn nhớ hàm gi của tôi chứ? Tôi thuộc cái hàm đó như lòng bàn tay. Tôi thuộc nó theo kiểu địa lý ! Nếu ai đó nhờ tôi đổi tỉ lệ trên trục x , tôi sẽ tự nhủ: “Tỉ lệ trên trục x . . . cái đó nằm ở tầng thụt lề thứ ba, ngay sau khối comment to đừng.” Rồi tôi sẽ cuộn xuống tới khối comment to đừng ấy, đếm 1..2..3, và nó nằm đó — đoạn mã đặt tỉ lệ cho trục x .

Now take a new programmer who’d never seen gi . Ask that poor sod to change the scaling on the x-axis. Where the hell should he go? He doesn’t know where the watering hole is. He’s got no idea that there’s a saber-toothed tiger on the loose in that valley over there. He’s just going to fumble around inside that 3,000-line morass of code until, by luck, he stumbles upon something that looks like it might set the scaling on the x-axis.

Giờ hãy lấy một lập trình viên mới toanh chưa từng thấy gì bao giờ. Hãy bảo anh chàng tội nghiệp ấy đổi tỉ lệ trên trục x . Anh ta biết đi đường nào? Anh ta đâu biết vũng nước nằm ở đâu. Anh ta chẳng may biết rằng có một con hổ răng kiếm đang rình trong cái thung lũng đằng kia. Anh ta sẽ chỉ mò mẫm loay quanh trong đống 3.000 dòng mã sinh lầy đó cho tới khi, nhờ may mắn, vấp phải một thứ gì đó trông có vẻ như đang đặt tỉ lệ cho trục x.

Wouldn’t it be great for that poor programmer if there were a function named setScalingOnX ? Wouldn’t it be just spectacular if that function were located in the Axis namespace? I hope I’ve made my point. So, on to point 2.

Chẳng phải sẽ tuyệt vời cho anh lập trình viên tội nghiệp ấy nếu có một hàm tên là setScalingOnX sao? Chẳng phải sẽ ngoạn mục hết sức nếu hàm đó nằm trong namespace Axis sao? Tôi hy vọng mình đã nói rõ ý. Vậy, sang điểm 2.

2. Small Functions Don’t Obscure Intent — 2. Hàm nhỏ không che mờ ý định

The answer to point 2 is the same as the answer to point 1. You cannot obscure the intent of the code by putting that code into a nicely named organizational **structure**. A well-designed and well-named structure of namespaces, classes, and functions reveals intent. It does not obscure it. More on this later.

Câu trả lời cho điểm 2 cũng chính là câu trả lời cho điểm 1. Bạn không thể che mờ ý định của mã bằng cách đặt mã đó vào một cấu trúc tổ chức được đặt tên tử tế. Một cấu trúc

namespace, class và hàm được thiết kế tốt và đặt tên tốt sẽ bộc lộ ý định. Nó không che mờ ý định. Chuyện này sẽ bàn thêm sau.

3. Performance — 3. Hiệu năng

OK, what about point 3? Performance: This is a real fear. But it is real only for those programmers who must preserve and protect every nanosecond under their control. If you are writing a first-person shooter game, or a high-speed trading app, or the guidance system for a hypersonic missile, then nanoseconds can be pretty important.

Được rồi, thế còn điểm 3? Hiệu năng: Đây là nỗi sợ có thật. Nhưng nó chỉ có thật với những lập trình viên phải nâng niu và bảo vệ từng nano giây trong tầm kiểm soát của mình. Nếu bạn đang viết một game bắn súng góc nhìn thứ nhất, hay một ứng dụng giao dịch tốc độ cao, hay hệ thống dẫn đường cho tên lửa siêu thanh, thì nano giây có thể quan trọng thật.

On the other hand, if you are writing a travel website, or a payroll system, or even the autopilot for a small airplane, nanoseconds are not your concern.

Ngược lại, nếu bạn đang viết một website du lịch, một hệ thống tính lương, hay thậm chí hệ thống lái tự động cho một chiếc máy bay nhỏ, thì nano giây không phải mối bận tâm của bạn.

Long ago a function call took several microseconds. Machines were slow back then. So we were careful about how many functions we called. But nowadays, function calls take a few nanoseconds. There's almost no way that you can appreciably slow your application down by extracting methods. And, if your measurements show you that there is a slowdown, all you have to do is inline the most critical of those functions back again.

Xa xưa, một lời gọi hàm tốn vài micro giây. Máy móc hồi đó chậm chạp. Nên ta phải dè chừng xem mình gọi bao nhiêu hàm. Nhưng ngày nay, lời gọi hàm chỉ tốn vài nano giây. Gần như không đời nào bạn có thể làm ứng dụng của mình chậm đi thấy rõ chỉ vì extract method. Và nếu số đo cho thấy có chậm đi thật, tất cả những gì bạn cần làm là inline lại vài hàm trọng yếu nhất trong số đó.

And remember, **good** compilers will often do that inlining for you, without your knowledge. If a good compiler sees that you've called a function from just one place, it's very likely to simply replace the call with the code within the function itself.

Và hãy nhớ, các compiler tốt thường sẽ tự inline giúp bạn mà bạn chẳng hay biết. Nếu một compiler tốt thấy rằng bạn chỉ gọi một hàm từ đúng một chỗ, rất có khả năng nó sẽ thay luôn lời gọi đó bằng chính phần mã bên trong hàm.

4. Bouncing Around — 4. Nhảy qua nhảy lại

OK, then. Point 4: bouncing around. I agree with this point— or at least I would agree if the functions in question were randomly scattered hither and yon throughout the code. But who would do a thing like that? It'd be very rude.

Được rồi. Điểm 4: nhảy qua nhảy lại. Tôi đồng ý với điểm này — hay ít nhất tôi sẽ đồng ý nếu những hàm đang bàn tới bị rải rác lung tung khắp nơi trong mã. Nhưng ai lại đi làm chuyện đó chứ? Thế thì bất lịch sự quá.

I like to organize my extracted functions in the order that they are called. So, when one function calls another, all you have to do is scroll down to see the called function—right there. If function A calls B and C, then their order in the source file should be A, B, C. 4 How hard is that?

Tôi thích sắp xếp các hàm đã extract theo đúng thứ tự chúng được gọi. Nhờ vậy, khi một hàm gọi hàm khác, bạn chỉ cần cuộn xuống là thấy hàm được gọi — ngay đó. Nếu hàm A gọi B và C, thì thứ tự của chúng trong file mã nguồn nên là A, B, C. 4 Khó gì đâu?

4 In languages like Clojure, it would be C, B, A.

4 Trong các ngôn ngữ như Clojure, thứ tự sẽ là C, B, A.

So, no, if the author is polite, you are not going to be bouncing all around inside the code.

Vậy nên, không đâu, nếu tác giả lịch sự thì bạn sẽ chẳng phải nhảy loạn xạ khắp nơi trong mã.

Instead, you'll be able to read down the length of the code, from high level to low level. Not only that, but the high-level code will be organized and separated from the low-level code—because when you extract functions, you virtually always extract lower-level code from higher-level code.

Thay vào đó, bạn sẽ đọc xuôi từ trên xuống dưới suốt chiều dài của mã, từ mức cao xuống mức thấp. Không chỉ vậy, phần mã mức cao còn được tổ chức và tách bạch khỏi phần mã mức thấp — bởi khi extract hàm, hầu như bao giờ bạn cũng đang lôi mã mức thấp ra khỏi mã mức cao.

Indeed, I'll show you an example of that later in this chapter. But first I want to talk about one last thing.

Thật vậy, tôi sẽ cho bạn xem một ví dụ về chuyện đó ở phần sau của chương này. Nhưng trước hết tôi muốn bàn về một điều cuối cùng.

5. Entanglement — 5. Vướng víu vào nhau

Of all the complaints, this is the most reasonable, and it is the point that John Ousterhout makes in the appendix. It is undeniably true that when you extract one function from another, you may be entangling the logic of those two functions. To say that differently, the lower-level function might not be independent of the operation of the higher-level function. To understand the lower-level function you must have a good understanding of the higher-level function.

Trong tất cả những lời than phiền, đây là lời hợp lý nhất, và cũng chính là điểm mà John Ousterhout nêu ra trong phần phụ lục. Không thể chối cãi rằng khi bạn extract một hàm ra khỏi hàm khác, bạn có thể đang làm logic của hai hàm đó vướng víu vào nhau. Nói cách khác, hàm mức thấp có thể không độc lập với hoạt động của hàm mức cao. Để hiểu hàm mức thấp, bạn phải hiểu rõ hàm mức cao.

If this entanglement is severe—that is if there are several different facts established by the higher-level function that you must keep in mind while reading the lower-level function—then there is a reasonable chance that the extraction might not be worth doing. However, if the entanglement is minimal, then the fact that the extracted function has a nice descriptive name and that fact that it appears directly after its parent function will likely make the extraction worthwhile.

Nếu sự vướng víu này nghiêm trọng — tức là có vài dữ kiện khác nhau do hàm mức cao thiết lập mà bạn phải luôn ghi nhớ trong lúc đọc hàm mức thấp — thì khả năng cao là phép extract ấy không đáng làm. Tuy nhiên, nếu sự vướng víu là tối thiểu, thì việc hàm

vừa extract có một cái tên mô tả tử tế, cộng thêm việc nó nằm ngay sau hàm cha của nó, nhiều khả năng sẽ khiến phép extract đó đáng giá.

Or to say this even more differently: It's a judgment call. Choose wisely.

Hay nói theo cách khác hẳn: Đây là chuyện phải tự cân nhắc mà quyết. Hãy chọn cho khôn ngoan.

What Are Large Functions Anyway? — Rốt cuộc thì hàm lớn là gì?

Once again, imagine my gi function from long ago. Let's imagine that it was written in Java and that it began like this:

Một lần nữa, hãy hình dung hàm gi của tôi thuở xa xưa. Hãy tưởng tượng nó được viết bằng Java và bắt đầu như thế này:

```
public class GraphicInterpreter {
    public void gi() {
        int i;
        int j;
        ...
    }
}
```

As we scroll down through the 3,000 lines of the gi function, we notice an if statement:

Khi cuộn xuống đọc 3.000 dòng của hàm gi, ta bắt gặp một câu lệnh if:

```
if (...) {
    ...
    i++;
    j++;
    ...
}
```

We decide we'd like to extract the body of that if statement into a new method. So we select the body with our mouse, and we invoke the Extract Method menu item. But the IDE complains that it cannot extract that method because the code modifies two local variables. Had the code modified only one local **variable**, the IDE could have returned the new value from the extracted method. But the IDE has no good way to return two variables. (This is Java and not Go, after all).

Ta quyết định muốn extract phần thân của câu lệnh if đó thành một method mới. Thế là ta bôi đen phần thân bằng chuột rồi chọn mục Extract Method trên menu. Nhưng IDE lại phàn nàn rằng nó không thể extract method đó, vì đoạn mã có sửa đổi hai biến cục bộ. Nếu đoạn mã chỉ sửa một biến cục bộ thôi, IDE đã có thể trả giá trị mới về từ method vừa extract. Nhưng IDE thì chẳng có cách nào hay ho để trả về hai biến. (Suy cho cùng, đây là Java chứ không phải Go.)

So, what should we do? How are we going to extract the body of that if statement into a new method? Well, what if we did this:

Vậy ta phải làm gì? Làm sao để extract phần thân của câu lệnh if đó thành một method mới? Ừm, thế nếu ta làm thế này thì sao:

```

public class GraphicInterpreter {
    private int i;
    private int j;
    public void gi() {
        ...
        if (...) {
            ...
            i++;
            j++;
            ...
        }
    }
}

```

I know what you are thinking: “Oh no! You promoted local variables into instance variables! You are violating **encapsulation**!” True. I am. But now I can extract the body of my if statement into a new function because the two variables remain in **scope** within the **class**:

Tôi biết bạn đang nghĩ gì: “Ôi không! Anh vừa nâng biến cục bộ lên thành biến instance! Anh đang vi phạm đóng gói (encapsulation)!” Đúng thế. Tôi đang làm vậy thật. Nhưng giờ tôi có thể extract phần thân của câu lệnh if thành một hàm mới, bởi vì hai biến đó vẫn nằm trong phạm vi (scope) của class:

```

public class GraphicInterpreter {
    private int i;
    private int j;
    public void gi() {
        ...
        if (...) {
            f();
        }
    }

    private void f() {
        ...
        i++;
        j++;
        ...
    }
}

```

Let’s keep doing this. We find more and more if and while statements whose bodies manipulate i and j, and that we can extract. Indent after indent we extract and extract and extract. And what is the result?

Cứ tiếp tục như thế đi. Ta tìm thấy ngày càng nhiều câu lệnh if và while mà phần thân của chúng thao tác trên i và j, và ta có thể extract được. Hết tầng thụt lề (indentation) này đến tầng thụt lề khác, ta extract, extract rồi lại extract. Và kết quả là gì?

The result is a whole bunch of methods that use the instance variables i and j. What do you call it when you’ve got a bunch of methods that manipulate a set of variables? You call that a class, don’t you?

Kết quả là cả một đống method cùng dùng hai biến instance i và j. Bạn gọi là gì khi có một

đồng method thao tác trên cùng một tập biến? Bạn gọi đó là một class, đúng không nào?

Now, think about this carefully.

Bây giờ, hãy ngẫm thật kỹ điều này.

Every large function is really hiding a class (or more) inside it.

Mỗi hàm lớn thực ra đang giấu một class (hoặc nhiều hơn) bên trong nó.

Because every large function has a set of local variables and a set of indented regions that manipulate those variables. Those variables should really be fields of a class, and those indented regions should be methods of that class.

Bởi vì mỗi hàm lớn đều có một tập biến cục bộ và một tập các vùng thụt lề thao tác trên đám biến ấy. Những biến đó lẽ ra phải là field của một class, còn những vùng thụt lề kia lẽ ra phải là method của class đó.

But to channel George Carlin, perhaps I've gone a bit too far, a bit too fast. So let's look at an example.

Nhưng nói theo kiểu George Carlin thì có lẽ tôi đi hơi xa và hơi nhanh rồi. Vậy hãy cùng xem một ví dụ.

Allow me to introduce the famous Video Store example from the first edition of Martin Fowler's wonderful book, Refactoring . Back in 1999 Martin wrote this little example in Java. Here, we're going to see it in Go. But never fear, if you don't know Go, it's pretty easy to understand.

Xin phép được giới thiệu ví dụ Video Store lừng danh trong ấn bản đầu tiên của cuốn sách tuyệt vời Refactoring của Martin Fowler. Hồi năm 1999, Martin viết ví dụ nhỏ này bằng Java. Ở đây, ta sẽ xem nó bằng Go. Nhưng đừng lo, nếu bạn không biết Go thì cũng khá dễ hiểu thôi.

First come the tests, which Martin did not include in his book so long ago.

Trước hết là các test, thứ mà ngày xưa Martin không đưa vào sách của ông.

```
——customer_test.go——
package GoVideoStore
import (
    "testing"
)
var customer *Customer
func setup() {
    customer = NewCustomer("Fred")
}
func assertEquals(t *testing.T, expected string, statement string)
    if statement != expected {
        t.Errorf("\nExpected:\n%s\n\nGot:\n%s", expected, statement)
    }
}
func TestSingleNewReleaseStatement(t *testing.T) {
    setup()
    customer.addRental(NewRental(NewMovie("The Cell", NewRelease), 3
    assertEquals(t,
        "Rental Record for Fred\n"+
        "\tThe Cell\t9.0\n"+
        "Amount owed is 9.0\n"+
```

```

        "You earned 2 frequent renter points",
        customer.statement())
    }
func TestDualNewReleaseStatement(t *testing.T) {
    setup()
    customer.addRental(NewRental(NewMovie("The Cell", NewRelease), 3)
    customer.addRental(NewRental(
        NewMovie("The Tigger Movie", NewRelease), 3)
    assertEquals(t,
        "Rental Record for Fred\n"+
        "\tThe Cell\t9.0\n"+
        "\tThe Tigger Movie\t9.0\n"+
        "Amount owed is 18.0\n"+
        "You earned 4 frequent renter points",
        customer.statement())
    }
func TestSingleChildrensStatement(t *testing.T) {
    setup()
    customer.addRental(NewRental(
        NewMovie("The Tigger Movie", Childrens), 3)
    assertEquals(t,
        "Rental Record for Fred\n"+
        "\tThe Tigger Movie\t1.5\n"+
        "Amount owed is 1.5\n"+
        "You earned 1 frequent renter points",
        customer.statement())
    }
func TestMultipleRegularStatement(t *testing.T) {
    setup()
    customer.addRental(NewRental(
        NewMovie("Plan 9 from Outer Space", Regular
        1))
    customer.addRental(NewRental(NewMovie("8 1/2", Regular), 2))
    customer.addRental(NewRental(NewMovie("Eraserhead", Regular), 3)
    assertEquals(t,
        "Rental Record for Fred\n"+
        "\tPlan 9 from Outer Space\t2.0\n"+
        "\t8 1/2\t2.0\n"+
        "\tEraserhead\t3.5\n"+
        "Amount owed is 7.5\n"+
        "You earned 3 frequent renter points",
        customer.statement())
    }
}

```

And then there's the code that passes these tests.

Và rồi đây là đoạn mã làm cho các test đó pass.

```

—Customer.go—
package GoVideoStore
import "fmt"

```

```

type Customer struct {
    name    string
    rentals []*Rental
}
func NewCustomer(name string) *Customer {
    return &Customer{name: name}
}
func (customer *Customer) addRental(rental *Rental) {
    customer.rentals = append(customer.rentals, rental)
}
func (customer *Customer) statement() string {
    totalAmount := 0.0
    frequentRenterPoints := 0
    result := "Rental Record for " + customer.name + "\n"
    for _, rental := range customer.rentals {
        thisAmount := 0.0
        switch rental.movie.movieType {
        case NewRelease:
            thisAmount += float64(rental.daysRented * 3)
        case Regular:
            thisAmount += 2
            if rental.daysRented > 2 {
                thisAmount += float64(rental.daysRented-2) * 1.5
            }
        case Childrens:
            thisAmount += 1.5
            if rental.daysRented > 3 {
                thisAmount += float64(rental.daysRented-3) * 1.5
            }
        }
        frequentRenterPoints++
        if rental.movie.movieType == NewRelease && rental.daysRented >
            frequentRenterPoints++
        }
        result += fmt.Sprintf("\t%s\t%.1f\n", rental.movie.title, thisA
        totalAmount += thisAmount
    }
    return result +
        fmt.Sprintf(
            "Amount owed is %.1f\nYou earned %d frequent renter points",
            totalAmount, frequentRenterPoints)
}
——Rental.go——
package GoVideoStore
type Rental struct {
    movie    *Movie
    daysRented int
}
func NewRental(movie *Movie, daysRented int) *Rental {

```

```

    return &Rental{movie: movie, daysRented: daysRented}
}
—Movie.go—
package GoVideoStore

const (
    Regular int = iota
    NewRelease
    Childrens
)
type Movie struct {
    title      string
    movieType int
}
func NewMovie(title string, movieType int) *Movie {
    return &Movie{title: title, movieType: movieType}
}

```

The first thing we're going to do is attack the tests. The tests need a better design, because those tests are a **mess** ! Here's why.

Việc đầu tiên ta sẽ làm là tấn công đám test. Chúng cần một thiết kế tốt hơn, bởi vì đám test đó đúng là một mớ hỗn độn! Đây là lý do.

- The tests are all conducted through the UI. The statement strings are the UI, and they are volatile. Marketing people will change those strings on a whim. For example, if a marketing person says that the title of each statement should be Rental Statement for... instead of Rental Record for ... , every **test** will break and will have to be changed. That makes the tests fragile. So, instead of testing the format of the statement many times, we should only test it once. The other tests should focus on the values being computed.
 - Toàn bộ test đều được tiến hành thông qua UI. Các chuỗi statement chính là UI, và chúng rất hay thay đổi. Dân marketing sẽ đổi mấy chuỗi đó bất chợt theo hứng. Chẳng hạn, nếu một người bên marketing bảo rằng tiêu đề của mỗi statement phải là Rental Statement for... thay vì Rental Record for..., thì mọi test sẽ vỡ và đều phải sửa. Điều đó khiến các test trở nên mong manh. Vậy nên, thay vì kiểm tra định dạng của statement nhiều lần, ta chỉ nên kiểm tra nó đúng một lần. Các test còn lại nên tập trung vào những giá trị được tính ra.
- Names like Fred and The Cell tell us nothing about what is being tested. Those names should be changed to help the reader of the tests understand what the tests are about. In general, tests should not contain production **data**. They should contain data that explains the intent of the test.
- The tests are wordy. We should be able to tighten them up a bit by using some variables for the movies, and a composed **assertion** .
 - Những cái tên như Fred và The Cell chẳng cho ta biết gì về thứ đang được kiểm thử. Nên đổi những cái tên đó để giúp người đọc test hiểu các test nói về chuyện gì. Nhìn chung, test không nên chứa dữ liệu production. Chúng nên chứa dữ liệu giải thích được ý định của test.
 - Các test quá dài dòng. Ta có thể siết chúng gọn lại đôi chút bằng cách dùng vài biến cho các bộ phim, cùng một assertion được ghép lại (composed assertion) .

—customer_test.go—

```

package GoVideoStore
import (
    "testing"
)
var customer *Customer
var newRelease1 *Movie
var newRelease2 *Movie
var childrensMovie *Movie
var regular1 *Movie
var regular2 *Movie
var regular3 *Movie
func setup() {
    customer = NewCustomer("Customer Name")
    newRelease1 = NewMovie("New Release 1", NewRelease)
    newRelease2 = NewMovie("New Release 2", NewRelease)
    childrensMovie = NewMovie("Childrens", Childrens)

    regular1 = NewMovie("Regular 1", Regular)
    regular2 = NewMovie("Regular 2", Regular)
    regular3 = NewMovie("Regular 3", Regular)
}
func assertStringsEqual(t *testing.T, expected string, actual string) {
    if actual != expected {
        t.Errorf("\nExpected:\n%s\n\nGot:\n%s", expected, actual)
    }
}
func assertIntsEqual(t *testing.T, expected int, actual int) {
    if actual != expected {
        t.Errorf("\nExpected:%d Got:%d", expected, actual)
    }
}
func assertFloatsEqual(t *testing.T, expected float64, actual float64) {
    if actual != expected {
        t.Errorf("\nExpected:%f Got:%f", expected, actual)
    }
}
func assertOwedAndPoints(t *testing.T, owed float64, points int) {
    customer.statement()
    assertFloatsEqual(t, owed, customer.totalAmount)
    assertIntsEqual(t, points, customer.frequentRenterPoints)
}
func TestTotalsForOneNewRelease(t *testing.T) {
    setup()
    customer.addRental(NewRental(newRelease1, 3))
    assertOwedAndPoints(t, 9.0, 2)
}
func TestTotalsForTwoNewReleases(t *testing.T) {
    setup()
    customer.addRental(NewRental(newRelease1, 3))
    customer.addRental(NewRental(newRelease2, 3))
}

```

```

    assertOwedAndPoints(t, 18.0, 4)
}
func TestTotalsForOneChildrensMovie(t *testing.T) {
    setup()
    customer.addRental(NewRental(childrensMovie, 3))
    assertOwedAndPoints(t, 1.5, 1)
}
func TestTotalsForManyRegularMovies(t *testing.T) {
    setup()
    customer.addRental(NewRental(regular1, 1))
    customer.addRental(NewRental(regular2, 2))
    customer.addRental(NewRental(regular3, 3))
    assertOwedAndPoints(t, 7.5, 3)
}

func TestStatementFormat(t *testing.T) {
    setup()
    customer.addRental(NewRental(regular1, 1))
    customer.addRental(NewRental(regular2, 2))
    customer.addRental(NewRental(regular3, 3))
    assertStringsEqual(t,
        "Rental Record for Customer Name\n"+
        "\tRegular 1\t2.0\n"+
        "\tRegular 2\t2.0\n"+
        "\tRegular 3\t3.5\n"+
        "Amount owed is 7.5\n"+
        "You earned 3 frequent renter points",
        customer.statement())
}

```

Remember, in the discussion about gi, how I broke encapsulation by moving some local variables into fields of the containing class? The changes I made to the tests caused me to do the same thing to the production code. I moved the `totalAmount` and `frequentRenterPoints` local variables into instance variables of the `Customer` class.

Bạn còn nhớ trong phần bàn về gi, chuyện tôi phá vỡ đóng gói bằng cách chuyển vài biến cục bộ thành field của class chứa nó chứ? Những thay đổi tôi vừa làm với đám test buộc tôi phải làm đúng như vậy với mã production. Tôi chuyển hai biến cục bộ `totalAmount` và `frequentRenterPoints` thành instance variable của class `Customer`.

—Customer.go—

```

...
type Customer struct {
    name            string
    rentals         []*Rental
    totalAmount     float64
    frequentRenterPoints int
}
...

```

You may recall that the reason I said we needed to break encapsulation was in order to support extraction. So let's get to that extraction. You're going to see me extract till I drop . Here's the first

pass.

Chắc bạn còn nhớ lý do tôi nói ta cần phá vỡ đóng gói là để phục vụ việc extract. Vậy thì hãy bắt tay vào extract thôi. Bạn sắp được thấy tôi extract tới lúc gục ngã. Đây là lượt đầu tiên.

—Customer.go—

```
package GoVideoStore
import "fmt"
type Customer struct {
    name            string
    rentals         []*Rental
    totalAmount     float64
    frequentRenterPoints int
}
func NewCustomer(name string) *Customer {
    return &Customer{name: name}
}
func (customer *Customer) addRental(rental *Rental) {
    customer.rentals = append(customer.rentals, rental)
}
func (customer *Customer) makeStatement() string {
    customer.clearTotals()

    return
        customer.makeHeader() +
        customer.makeDetails() +
        customer.makeFooter()
}
func (customer *Customer) clearTotals() {
    customer.totalAmount = 0.0
    customer.frequentRenterPoints = 0
}
func (customer *Customer) makeHeader() string {
    return "Rental Record for " + customer.name + "\n"
}
func (customer *Customer) makeDetails() string {
    rentalDetails := ""
    for _, rental := range customer.rentals {
        rentalDetails += customer.makeDetail(rental)
    }
    return rentalDetails
}
func (customer *Customer) makeDetail(rental *Rental) string {
    amount := customer.determineAmount(rental)
    customer.frequentRenterPoints += customer.determinePoints(rental)
    customer.totalAmount += amount
    return customer.formatDetail(rental, amount)
}
func (customer *Customer) determineAmount(rental *Rental) float64
    amount := 0.0
```



```

switch rental.movie.movieType {
case NewRelease:
    amount += float64(rental.daysRented * 3)
case Regular:
    amount += 2
    if rental.daysRented > 2 {
        amount += float64(rental.daysRented-2) * 1.5
    }
case Childrens:
    amount += 1.5
    if rental.daysRented > 3 {
        amount += float64(rental.daysRented-3) * 1.5
    }
}
return amount
}
func (customer *Customer) determinePoints(rental *Rental) int {
    if rental.movie.movieType == NewRelease && rental.daysRented > 1
        return 2
    }
    return 1
}
func (customer *Customer)
formatDetail(rental *Rental, amount float64) string {
    return fmt.Sprintf("\t%s\t%.1f\n", rental.getTitle(), amount)
}
func (customer *Customer) makeFooter() string {
    return fmt.Sprintf("Amount owed is %.1f\n"+
        "You earned %d frequent renter points",
        customer.totalAmount, customer.frequentRenterPoints)
}

```

I imagine that some of you who are fluent in Golang are now running away in horror. 5 If so, I apologize—I am not an accomplished Golang programmer. Others of you might be screaming as you run down the hall because of all those little tiny functions . Aaaarrgggghhh!

Tôi hình dung rằng vài người trong số các bạn thông thạo Golang giờ đang bỏ chạy trong kinh hãi. 5 Nếu vậy thì tôi xin lỗi — tôi không phải một lập trình viên Golang lành nghề. Số khác có lẽ đang vừa chạy dọc hành lang vừa hét lên vì đồng hàm bé tí ti kia. Aaaarrgggghhh!

5 See the Future Bob note near the end of this example.

5 Xem ghi chú Future Bob ở gần cuối ví dụ này.

But wait. Go back, right now, and read that makeStatement function. I'll wait.

Nhưng khoan đã. Hãy quay lại, ngay bây giờ, và đọc kỹ hàm makeStatement. Tôi chờ đấy.

How do you make a statement? You clear the totals and then compose the header, details, and footer together into a single string.

Bạn tạo ra một statement như thế nào? Bạn xóa các tổng, rồi ghép phần đầu, phần chi tiết và phần chân lại thành một chuỗi duy nhất.

Now, if you think about it, that's the high-level procedure . Indeed, that's the high-level procedure for most reports— because most reports are composed of a header, a set of details, and a footer.

Giờ thì, nếu ngẫm kỹ, đó chính là quy trình ở mức cao. Thật vậy, đó là quy trình mức cao của hầu hết mọi báo cáo — bởi vì đa số báo cáo đều gồm một phần đầu, một tập các dòng chi tiết, và một phần chân.

So, to those of you who feared that all this extracting would hide the intent of the code, I suggest quite the opposite has happened. The extraction has exposed the high-level intent of the code and simply moved the lower-level details into other functions, where they cannot distract you. If you have been told to fix a bug in the footer of this statement, you now know exactly where to go: the `makeFooter` function.

Vậy nên, với những ai từng lo rằng ngần ấy phép `extract` sẽ che mờ ý định của đoạn mã, tôi cho rằng điều hoàn toàn ngược lại đã xảy ra. Việc `extract` đã phơi bày ý định ở mức cao của đoạn mã, và chỉ đơn giản là dời những chi tiết ở mức thấp hơn sang các hàm khác, nơi chúng không thể làm bạn phân tâm. Nếu bạn được giao sửa một lỗi ở phần chân của bản statement này, giờ bạn biết chính xác phải đi đâu: hàm `makeFooter`.

This code is polite. It is polite because, when the poor reader looks at it, there's little to decode. The reader does not have to pore and ponder over strings and variables and loops and

Đoạn mã này lịch sự. Nó lịch sự vì khi người đọc tội nghiệp nhìn vào, chẳng có mấy thứ phải giải mã. Người đọc không phải cắm cúi và vò đầu bứt tai với đủ thứ chuỗi, biến, vòng lặp và

if statements. Instead, the reader sees a simple statement of policy with directions pointing to the answers to any questions they might have.

câu lệnh `if`. Thay vào đó, người đọc thấy một lời phát biểu chính sách đơn giản, kèm những chỉ dẫn trở tới câu trả lời cho bất cứ thắc mắc nào họ có thể có.

Because this code is polite, it's fair to say that the original code was rude. And we don't want to be rude, do we?

Vì đoạn mã này lịch sự, ta có thể nói công bằng rằng đoạn mã gốc thì bất lịch sự. Và chúng ta đâu muốn bất lịch sự, phải không?

Let's continue. It's pretty easy to see how to `clearTotals` . It's also pretty obvious how to make the header and the footer. Making the details is just a loop over all the rentals that simply calls `makeDetail` .

Đi tiếp nào. Khá dễ để thấy `clearTotals` nên làm gì. Cũng khá hiển nhiên chuyện tạo phần đầu và phần chân. Việc tạo phần chi tiết chỉ là một vòng lặp qua toàn bộ rental, gọi `makeDetail` là xong.

How do you make a detail? You determine the amount, then you determine the points, and then you format the detail. Yikes! How obvious can you get?

Bạn tạo một detail như thế nào? Bạn xác định amount, rồi xác định points, rồi định dạng detail. Chao ôi! Còn có thể hiển nhiên đến mức nào nữa?

Formatting the detail is pretty obvious too. But those two

Việc định dạng detail cũng khá hiển nhiên. Nhưng hai hàm `determine` functions are a bit problematic.

`determine` kia thì hơi có vấn đề.

Look through all those methods. The first **argument** is always customer `*Customer` . This is just Go's way of saying that the function is part of the Customer class. Now look at the bodies of all the functions. Notice that they all use the

Hãy lướt qua toàn bộ đám method đó. Đối số đầu tiên luôn là customer `*Customer`. Đây chỉ là cách Go nói rằng hàm này thuộc về class Customer. Giờ hãy nhìn vào phần thân của tất cả các hàm. Để ý rằng chúng đều dùng đối số

customer argument—except for `formatDetail` , `determineAmount` , and

`customer` — ngoại trừ `formatDetail`, `determineAmount`, và

`determinePoints` . We'll deal with `formatDetail` later. For now, there's an awful lot of code in those two determine functions that doesn't deal with the Customer class that they belong to. So . . . maybe they don't belong to that class.

`determinePoints`. Ta sẽ xử lý `formatDetail` sau. Còn lúc này, có quá nhiều mã trong hai hàm `determine` đó chẳng dính dáng gì tới class Customer mà chúng đang thuộc về. Vậy thì... có lẽ chúng không thuộc về class đó.

What else gets passed into those determine functions? The

Còn thứ gì khác được truyền vào hai hàm `determine` ấy? Chính là

`Rental` ! So maybe those two determine functions belong in the `Rental` class. Let's move them there.

`Rental` ! Vậy có lẽ hai hàm `determine` ấy thuộc về class `Rental`. Hãy chuyển chúng sang đó.

—`Customer.go`—

```
func (customer *Customer) makeDetail(rental *Rental) string {
    amount := rental.determineAmount()
    customer.frequentRenterPoints += rental.determinePoints()
    customer.totalAmount += amount
    return customer.formatDetail(rental, amount)
}
```

—`Rental.go`—

```
Package GoVideoStore
type Rental struct {
    movie      *Movie
    daysRented int
}
func (rental Rental) getTitle() string {
    return rental.movie.title
}
func (rental Rental) determineAmount() float64 {
    amount := 0.0
    switch rental.movie.movieType {
    case NewRelease:
        amount += float64(rental.daysRented * 3)
    case Regular:
        amount += 2
        if rental.daysRented > 2 {
            amount += float64(rental.daysRented-2) * 1.5
        }
    case Childrens:
```

```

    amount += 1.5
    if rental.daysRented > 3 {
        amount += float64(rental.daysRented-3) * 1.5
    }
}
return amount
}
func (rental Rental) determinePoints() int {
    if rental.movie.movieType == NewRelease && rental.daysRented > 1
        return 2
    }
    return 1
}
func NewRental(movie *Movie, daysRented int) *Rental {
    return &Rental{movie: movie, daysRented: daysRented}
}

```

The tests still pass, unchanged. The tests have no idea that this fundamental change in design has happened. That means the tests have a design that tolerates change!

Các test vẫn pass, không cần sửa gì. Đám test chẳng hề hay biết rằng một thay đổi thiết kế mang tính nền tảng vừa diễn ra. Điều đó có nghĩa là các test này có một thiết kế chịu được thay đổi!

However, something even more magical has taken place. All the business rules that were in the Customer class have fled to the Rental . The Customer class is now all about the formatting of the statement—the business calculations, other than a few sums, are now done in the Rental .

Tuy nhiên, còn một điều kỳ diệu hơn thế vừa diễn ra. Toàn bộ quy tắc nghiệp vụ (business rule) từng nằm trong class Customer đã chạy sang Rental . Class Customer giờ chỉ còn nói về việc định dạng bản statement — các phép tính nghiệp vụ, ngoài vài phép cộng dồn, nay đều được thực hiện trong Rental .

This calls into question why the Customer class was given that name. It's not about the customer anymore—if it ever was to begin with. Now it's all about the rental statement. So that's what its name should be: RentalStatement .

Điều này khiến ta phải đặt dấu hỏi vì sao class Customer lại mang cái tên đó. Nó chẳng còn nói về customer nữa — nếu như nó từng nói về customer thật. Giờ nó hoàn toàn nói về bản rental statement. Vậy thì tên của nó phải đúng như thế: RentalStatement.

—RentalStatement.go—

```

...
type RentalStatement struct {
    name            string
    rentals          []*Rental
    totalAmount      float64
    frequentRenterPoints int
}
...

```

But now let's consider that Rental class. The determine functions are strongly coupled to the Rental class in that they use both the movie and the daysRented fields. That suggests a strong **cohesion**. However, the determine functions are very strongly coupled to the Movie class as well, and that's a

bit troubling.

Nhưng giờ hãy nghĩ về class Rental. Các hàm determine kết dính chặt (coupling) với class Rental ở chỗ chúng dùng cả field movie lẫn daysRented. Điều đó gợi ý một độ gắn kết nội tại (cohesion) mạnh. Tuy nhiên, các hàm determine cũng kết dính rất chặt với class Movie, và điều đó thì hơi đáng lo.

The **coupling** to the Movie is all about the movieType . That type field is used to select the various business rules for calculating the amount and the points. So why are the type fields and the type constants in the Movie class?

Sự kết dính với Movie hoàn toàn xoay quanh movieType . Cái field type đó được dùng để chọn ra các quy tắc nghiệp vụ khác nhau nhằm tính amount và points. Vậy tại sao các field type và các hằng type lại nằm trong class Movie?

The original tests contain a hint. If you look back, you'll see that “The Tigger Movie” was considered both a NewRelease and a

Các test ban đầu có để lại một manh mối. Nếu bạn ngó lại, sẽ thấy “The Tigger Movie” được coi vừa là phim NewRelease vừa là phim

Childrens movie. That suggests that a movie can be rented as a new release and later rented as a children's movie. And that suggests that the type field actually belongs to the

Childrens. Điều đó gợi ý rằng một bộ phim có thể được thuê với tư cách new release, rồi sau đó lại được thuê với tư cách phim thiếu nhi. Và điều đó gợi ý rằng field type thực ra thuộc về

Rental . Let's try that.

Rental . Hãy thử xem sao.

—Rental.go—

```
...
type Rental struct {
    movie      *Movie
    daysRented int
    rentalType int
}
const (
    Regular int = iota
    NewRelease
    Childrens
)
...
func NewRental(movie *Movie, rentalType int, daysRented int) *Rental {
    return &Rental{
        movie:      movie,
        rentalType: rentalType,
        daysRented: daysRented
    }
}
...
```

This change affected the tests because the NewRental constructor was changed to include the type. But overall this was a pretty quick change. However, it begs another question. Whenever you have a **switch statement** that acts on that type code, you might want to consider a polymorphic dispatch.

Thay đổi này có ảnh hưởng tới đám test, vì hàm khởi tạo NewRental được sửa để nhận thêm type. Nhưng nhìn chung đây là một thay đổi khá nhanh gọn. Tuy vậy, nó lại kéo theo một câu hỏi khác. Mỗi khi bạn có một câu lệnh switch hành xử dựa trên mã kiểu (type code) như thế, có lẽ bạn nên cân nhắc dùng polymorphic dispatch.

Why? Because switch statements are like gerbils—given enough time they’ll reproduce all over your code and fill it to the brim. Indeed, there’s already a baby switch statement growing in the determinePoints method.

Tại sao ư? Bởi vì các câu lệnh switch giống như chuột nhảy — cho chúng đủ thời gian, chúng sẽ sinh sôi khắp mã của bạn và lấp đầy tới tận miệng. Thật vậy, đã có một con switch con đang lớn dần trong method determinePoints rồi đấy.

In Chapter 12, “Objects and Data Structures,” we’ll talk about the pros and cons of switch statements. For now, let’s see if we can get rid of these by creating a type hierarchy.

Ở Chương 12, “Đối tượng và cấu trúc dữ liệu”, ta sẽ bàn về ưu và nhược điểm của câu lệnh switch. Còn lúc này, hãy xem ta có thể loại bỏ chúng bằng cách tạo ra một cây phân cấp kiểu hay không.

```
——RentalType.go——
package GoVideoStore
type RentalType interface {
    determineAmount(daysRented int) float64
    determinePoints(daysRented int) int
}
——Rental.go——
package GoVideoStore
type Rental struct {
    movie      *Movie
    daysRented int
    rentalType RentalType
}
func (rental Rental) getTitle() string {
    return rental.movie.title
}

func (rental Rental) determineAmount() float64 {
    return rental.rentalType.determineAmount(rental.daysRented)
}
func (rental Rental) determinePoints() int {
    return rental.rentalType.determinePoints(rental.daysRented)
}
func NewRental(movie *Movie,
               rentalType RentalType,
               daysRented int) *Rental {
    return &Rental{
        movie:      movie,
        rentalType: rentalType,
        daysRented: daysRented}
}
——NewReleaseRental.go——
package GoVideoStore
```

```

type NewReleaseRental struct{}
func (rentalType NewReleaseRental)
determineAmount(daysRented int) float64 {
    return float64(daysRented * 3)
}

func (rentalType NewReleaseRental) determinePoints(daysRented int)
    if daysRented > 1 {
        return 2
    }
    return 1
}

——ChildrensRental.go——
package GoVideoStore
type ChildrensRental struct{}
func (rentalType ChildrensRental)
determineAmount(daysRented int) float64 {
    amount := 1.5
    if daysRented > 3 {
        amount += float64(daysRented-3) * 1.5
    }
    return amount
}
func (rentalType ChildrensRental) determinePoints(daysRented int) i
    return 1
}

——RegularRental.go——
package GoVideoStore
type RegularRental struct{}
func (rentalType RegularRental) determineAmount(daysRented int) flo
    amount := 2.0
    if daysRented > 2 {
        amount += float64(daysRented-2) * 1.5
    }
    return amount
}
func (rentalType RegularRental) determinePoints(daysRented int) int
    return 1
}

```

That's a nice polymorphic dispatch. Now all the business rules for each type are safely encapsulated within their own type, and the Rental couldn't care less which one it is using. Of course, the tests had to change, but only a little bit. Here's just one to give you the idea.

Đó là một polymorphic dispatch đẹp đẽ. Giờ toàn bộ quy tắc nghiệp vụ của mỗi kiểu đã được đóng gói an toàn bên trong chính kiểu đó, còn Rental thì chẳng bận tâm nó đang dùng kiểu nào. Dĩ nhiên các test phải đổi, nhưng chỉ chút xíu thôi. Đây là một cái để bạn hình dung.

```

——RentalStatementTest.go——
func TestTotalsForOneNewRelease(t *testing.T) {
    setup()
}

```



```
statement.addRental(NewRental(newRelease1, NewReleaseRental{}, 3
assertOwedAndPoints(t, 9.0, 2)
}
```

The Movie class is virtually empty at this point.

Đến lúc này thì class Movie gần như rỗng.

```
—Movie.go—
package GoVideoStore
type Movie struct {
    title string
}
func NewMovie(title string) *Movie {
    return &Movie{title: title}
}
```

And the RentalStatement class is concerned only with the formatting of the statement—which is why the FormatDetail function belongs there even though it doesn’t use any of the

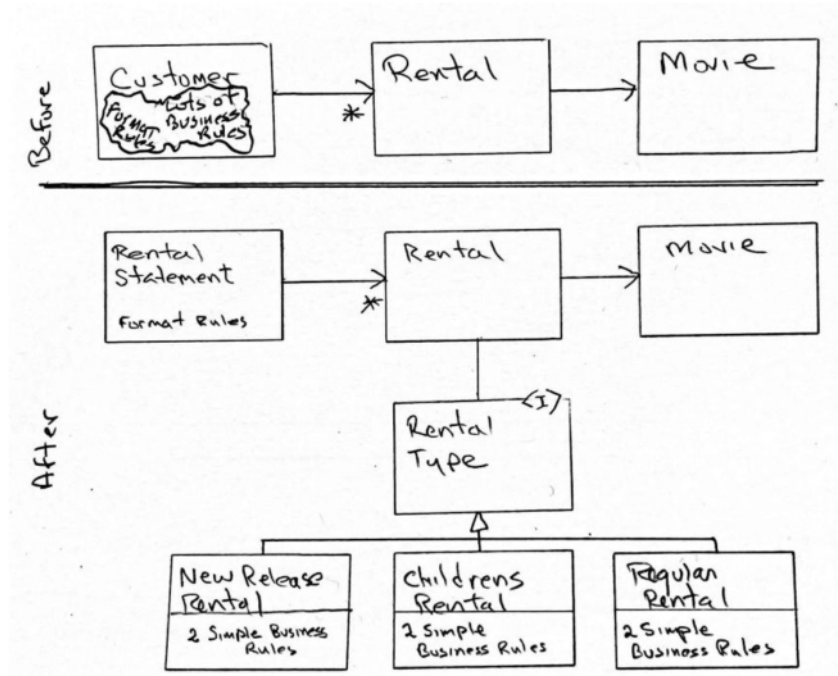
Còn class RentalStatement thì chỉ còn quan tâm tới việc định dạng bản statement — đó chính là lý do hàm FormatDetail thuộc về nơi này, dù nó chẳng dùng field nào của

RentalStatement’s fields.

RentalStatement cả.

Now, let’s consider what we’ve just done. The diagram below shows a before-and-after picture.

Giờ hãy ngẫm lại xem ta vừa làm gì. Sơ đồ dưới đây cho thấy bức tranh trước và sau.



Our cleanup procedure extracted the business rules and separated them from the formatting rules. Then, it moved those business rules, first into the Rental and then decomposed them down into the polymorphic hierarchy. Nice! And in the midst of all that, we realized that some names and concepts

were wrong—and that’s pretty typical. When you clean things up, you see things that were hidden before.

Quy trình dọn dẹp của ta đã bóc các quy tắc nghiệp vụ ra và tách chúng khỏi các quy tắc định dạng. Rồi nó chuyển đám quy tắc nghiệp vụ ấy, trước hết vào Rental, sau đó phân rã chúng xuống cây phân cấp đa hình (polymorphism). Tuyệt! Và giữa lúc làm tất cả những việc đó, ta nhận ra rằng vài cái tên và vài khái niệm đã sai — chuyện này khá điển hình. Khi bạn dọn dẹp mọi thứ, bạn sẽ thấy những điều trước đây bị che khuất.

Finally, notice that there is an **architectural boundary** forming below the RentalType . I didn’t draw it, but you ought to be able to sense it. All those little **business rule** details are isolated from the higher-level policies in RentalStatement and Rental . Now when the requirements change, we can add lots of new RentalType s; and none of this code will need to change to accommodate those changes. That’s the Open–Closed Principle (OCP) at work.

Cuối cùng, hãy để ý rằng có một ranh giới kiến trúc (architectural boundary) đang hình thành ngay bên dưới RentalType . Tôi không vẽ nó ra, nhưng lẽ ra bạn phải cảm nhận được. Toàn bộ những chi tiết quy tắc nghiệp vụ vụn vặt kia đã được cách ly khỏi các chính sách ở mức cao hơn trong RentalStatement và Rental . Giờ đây, khi yêu cầu thay đổi, ta có thể thêm thật nhiều RentalType mới; và chẳng đoạn mã nào trong số này cần phải đổi để đáp ứng những thay đổi ấy. Đó chính là OCP — Nguyên tắc Đóng–Mở (Open–Closed Principle) đang phát huy tác dụng.

So, remember when I said that large functions are always hiding classes? That’s what we found here. That one large

Vậy, còn nhớ lúc tôi nói rằng các hàm lớn luôn giấu class bên trong chứ? Đó đúng là thứ ta vừa tìm thấy ở đây. Cái hàm

statement function in the original code got broken up and distributed into three new classes and one existing class.

statement to dùng duy nhất trong mã gốc đã bị chẻ nhỏ và phân bổ vào ba class mới cùng một class sẵn có.

Future Bob: Robert Laszczak happened to be perusing GitHub and saw the above meager

Future Bob: Robert Laszczak tình cờ dạo GitHub và trông thấy cái nỗ lực Golang

attempt at Golang. He has kindly offered a blog post with improvements to this example that are more consistent with common Golang style. You

còn cõ ở trên. Anh ấy đã nhiệt tình viết một bài blog với những cải tiến cho ví dụ này, sao cho gần với phong cách Golang thông dụng hơn. Bạn

can see this at <https://threedots.tech/clean-code/> . For my part, as a Golang novice, I agree with most of his changes, but I prefer to keep my names a bit

có thể xem tại <https://threedots.tech/clean-code/>. Về phần mình, với tư cách một tay mơ Golang, tôi đồng ý với hầu hết thay đổi của anh ấy, nhưng tôi vẫn thích tên của mình hơi

longer, and prefix my getters with get.

dài hơn một chút, và thích gắn tiền tố get cho các getter.

Extraction and Classes — Trích xuất và class

Consider the following Java code for solving the quadratic formula. The class is just a convenient namespace for the

Hãy xem đoạn mã Java sau đây để giải phương trình bậc hai. Class ở đây chỉ đóng vai trò một namespace tiện lợi cho hàm

solve function.

solve.

```
import java.util.Arrays;
import java.util.List;
import static java.util.Arrays.asList;
public class QuadraticFormula {
    public static List<Double> solve(double a, double b, double c) {
        if (a == 0) {
            (a 0) {
                if (b == 0) {
                    return asList();
                }
                return asList(-c / b);
            } else {
                double d = b * b - 4 * a * c;
                if (d > 0) {
                    double sqrtd = Math.sqrt(d);
                    double x1 = (-b + sqrtd) / (2 * a);
                    double x2 = (-b - sqrtd) / (2 * a);
                    return asList(x1, x2);
                } else if (d == 0) {
                    return asList(-b / (2 * a));
                } else {
                    return asList();
                }
            }
        }
    }
}
```

This code is pretty straightforward, but let's pretend that it's more complicated and involved than shown here. Let's also assume that it is going to grow with time. We might improve the readability by doing a few simple extractions and some renaming.

Đoạn mã này khá đơn giản, nhưng hãy cứ giả vờ rằng nó phức tạp và rối rắm hơn những gì đang thấy. Cũng hãy giả sử rằng theo thời gian nó sẽ còn phình to ra. Ta có thể cải thiện tính dễ đọc bằng vài phép trích xuất đơn giản và một chút đổi tên.

```
import java.util.List;
import static java.util.Arrays.asList;
public class QuadraticFormula {
    public static List<Double> solve(double a, double b, double c) {
        if (a == 0)
            return linearSolution(b, c);
    }
}
```

```

    return quadraticSolution(a, b, c);
}
private static List<Double> linearSolution(double b, double c) {
    if (b == 0)
        return asList();
    return asList(-c / b);
}
private static
List<Double> quadraticSolution(double a, double b, double c) {
    double discriminant = b * b - 4 * a * c;
    if (discriminant < 0)
        return complexSolution();
    if (discriminant == 0)
        return singleQuadraticSolution(a, b);
    return twoQuadraticSolutions(a, b, discriminant);
}
private static List<Double> complexSolution() {
    return asList();
}
private static
List<Double> singleQuadraticSolution(double a, double b) {
    return asList(-b / (2 * a));
}
private static List<Double>
twoQuadraticSolutions(double a, double b, double discriminant) {
    double sqrted = Math.sqrt(discriminant);
    double x1 = (-b + sqrted) / (2 * a);
    double x2 = (-b - sqrted) / (2 * a);
    return asList(x1, x2);
}
}

```

Now again, for the sake of argument, we are pretending that this is a much more involved system that is likely to grow. I hope you agree that the extractions have lent some organization to the code that might make something more complicated a bit easier to read. But having done this, we are missing the opportunity of taking advantage of the class. Rather than passing arguments between all those extracted functions, we can load those values into variables within the class.

Một lần nữa, cứ cho là như vậy đi, ta đang giả vờ rằng đây là một hệ thống rối rắm hơn nhiều và nhiều khả năng sẽ còn lớn lên. Tôi hy vọng bạn đồng ý rằng những phép trích xuất vừa rồi đã mang lại chút trật tự cho mã, đủ để một thứ phức tạp hơn trở nên dễ đọc hơn một chút. Nhưng làm xong chừng đó, ta lại đang bỏ lỡ cơ hội tận dụng chính class này. Thay vì truyền đối số qua lại giữa tất cả những hàm vừa trích xuất, ta có thể nạp các giá trị ấy vào biến bên trong class.

```

import java.util.List;
import static java.util.Arrays.asList;
public class QuadraticFormula {
    private static double a, b, c, discriminant;
    public static List<Double> solve(double a, double b, double c) {
        QuadraticFormula.a = a;
    }
}

```

```

QuadraticFormula b = b;
    QuadraticFormula.b = b;
    QuadraticFormula.c = c;
    if (a == 0)
        return linearSolution();
    return quadraticSolution();
}
private static List<Double> linearSolution() {
    if (b == 0)
        return asList();
    return asList(-c / b);
}
private static List<Double> quadraticSolution() {
    discriminant = b * b - 4 * a * c;
    if (discriminant < 0)
        return complexSolution();
    if (discriminant == 0)
        return singleQuadraticSolution();
    return twoQuadraticSolutions();
}
private static List<Double> complexSolution() {
    return asList();
}
private static List<Double> singleQuadraticSolution() {
    return asList(-b / (2 * a));
}
}

}
private static List<Double> twoQuadraticSolutions() {
    double sqrtd = Math.sqrt(discriminant);
    double x1 = (-b + sqrtd) / (2 * a);
    double x2 = (-b - sqrtd) / (2 * a);
    return asList(x1, x2);
}
}
}

```

This makes things just a little bit cleaner. In a more complex module, this could make things a lot cleaner. However, there is a risk. Those static variables could get corrupted in a multithreaded environment. This is easy enough to correct by changing all the functions and variables from static to instance methods and variables. But I'm not going to worry about that for this example.

Cách này làm mọi thứ sạch hơn một chút. Trong một module phức tạp hơn, nó có thể làm mọi thứ sạch hơn rất nhiều. Tuy nhiên, vẫn có một rủi ro. Những biến static kia có thể bị hỏng dữ liệu trong môi trường đa luồng. Chuyện đó dễ khắc phục thôi: chuyển toàn bộ hàm và biến từ static sang method và biến của instance. Nhưng trong ví dụ này tôi sẽ không bận tâm tới điều đó.

The bottom line is that sometimes when we extract functions, it is convenient and clean to allow those functions to communicate through variables in the scope (the class) that contains them.

Điểm mấu chốt là: đôi khi, sau khi trích xuất các hàm, việc để chúng trao đổi với nhau qua các biến nằm trong phạm vi (scope) chứa chúng — tức là class — vừa tiện lợi vừa sạch sẽ.

So now let's look at this problem in a functional language like Clojure.

Giờ hãy nhìn bài toán này trong một ngôn ngữ hàm như Clojure.

```
(defn quad [a b c]
  (let [discriminant (- (* b b) (* 4 a c))]
    (cond
      (zero? a)
      (if (zero? b)
        []
        [(/ (- c) b)])
      (zero? discriminant)
      [(/ (- b) (* 2 a))]
      (neg? discriminant)
      []
      :else
      (let [sqrt-desc (Math/sqrt discriminant)
            x1 (/ (+ (- b) sqrt-desc) (* 2 a))
            x2 (/ (- (- b) sqrt-desc) (* 2 a))]
        [x1 x2])))))
```

This is very pretty. I would not be tempted to improve this by extracting. But again, let's pretend that this is a much larger and more involved function. How might we extract functions to improve it?

Đẹp thật. Tôi sẽ chẳng thấy có gì thôi thúc mình cải thiện nó bằng cách trích xuất cả. Nhưng lại một lần nữa, hãy giả vờ rằng đây là một hàm lớn hơn và rối rắm hơn nhiều. Ta sẽ trích xuất hàm thế nào để cải thiện nó?

```
(defn quad [a b c]
  (let [discriminant (- (* b b) (* 4 a c))
        linear-solution
        (fn []
          (if (zero? b)
            []
            [(/ (- c) b)]))
        single-solution
        (fn []
          [(/ (- b) (* 2 a))])
        two-solutions
        (fn []
          (let [sqrt-desc (Math/sqrt discriminant)
                x1 (/ (+ (- b) sqrt-desc) (* 2 a))
                x2 (/ (- (- b) sqrt-desc) (* 2 a))]
            [x1 x2]))]
    (cond
      (zero? a) (linear-solution)
      (zero? discriminant) (single-solution)
      (neg? discriminant) []
      :else (two-solutions))))
```

OK, now for the last time, keep in mind that we are pretending that this module is larger and

more complicated. That last `cond` statement is pretty nice. Notice that we're not passing arguments to the extracted functions. That's because a function in Clojure creates a kind of **object** that has variables and functions scoped within it. Notice also that those variables are not static, and so our **concurrency** worries disappear.

Được rồi, lần cuối cùng nhé, hãy nhớ rằng ta đang giả vờ module này lớn hơn và phức tạp hơn. Câu lệnh `cond` cuối cùng đó khá là ổn. Để ý rằng ta không truyền đối số nào cho các hàm được trích xuất. Đó là vì một hàm trong Clojure tạo ra một kiểu đối tượng (object) có biến và hàm nằm trong phạm vi của nó. Cũng để ý rằng những biến ấy không static, nên mọi lo lắng về đồng thời (concurrency) của ta biến mất.

Conclusion — Kết luận

So, should you practice Extract Till You Drop? And if so, how much?

Vậy, bạn có nên thực hành Extract Till You Drop không? Và nếu có thì tới mức nào?

To some extent this is a matter of style and preference. You may like your functions a bit bigger than I like mine. Perhaps you are more comfortable with 20-line functions or 25-line functions. That's up to you.

Ở một chừng mực nào đó, đây là chuyện phong cách và sở thích. Bạn có thể thích hàm của mình to hơn hàm của tôi một chút. Có lẽ bạn thấy thoải mái với những hàm 20 dòng hay 25 dòng. Đó là quyền của bạn.

However, remember what you just saw. It was the extraction of all those methods in the Video Store example that allowed us to see that some of that code did not belong in the class it was in. That was not evident before we did the extractions. So, if you are going to leave your functions larger, make sure you don't miss those kinds of opportunities. In fact, you may find it worthwhile to extract first and then do a little strategic inlining once you have determined where everything actually belongs.

Tuy nhiên, hãy nhớ những gì bạn vừa chứng kiến. Chính việc trích xuất tất cả những method đó trong ví dụ Video Store đã cho phép ta nhìn ra rằng một phần mã ấy chẳng thuộc về class mà nó đang nằm trong đó. Điều này không hề hiển hiện trước khi ta thực hiện các phép trích xuất. Vậy nên, nếu bạn định để hàm của mình lớn hơn, hãy chắc rằng bạn không bỏ lỡ những cơ hội kiểu ấy. Thật ra, bạn có thể thấy đáng công khi trích xuất trước, rồi inline lại đôi chỗ một cách có chiến lược sau khi đã xác định được mọi thứ thực sự thuộc về đâu.

But remember, when you inline a function, you destroy its name.

Nhưng hãy nhớ, khi bạn inline một hàm, bạn phá huỷ cái tên của nó.

Chapter 11. Be Polite — Chương 11.

Hãy lịch sự



We want our modules to be polite. Polite modules follow Ward Cunningham’s definition of cleanliness. As you read the code, you find it to be “pretty much what you expected”. A **module** is a bounded set of functions and variables. This might be composed of one **class**, several classes, or no classes at all. What matters are the functions and variables. A module may also be contained in a single source file or several source files. What matters is the **cohesion** of the functions and variables, and not the artificial boundaries that separate them. Another common word for a module is a **component**.

Chúng ta muốn các module của mình thật lịch sự. Một module lịch sự là module đáp ứng đúng định nghĩa về sự sạch sẽ của Ward Cunningham: khi đọc mã, bạn thấy nó “gần như đúng những gì bạn đã trông đợi”. Module là một tập hợp có ranh giới rõ ràng gồm các hàm và biến. Nó có thể gồm một class, vài class, hoặc chẳng có class nào cả. Điều quan trọng là các hàm và biến. Một module cũng có thể nằm gọn trong một file mã nguồn hoặc trải ra nhiều file. Điều quan trọng là độ gắn kết nội tại (cohesion) giữa các hàm và biến, chứ không phải những ranh giới nhân tạo ngăn cách chúng. Một từ thông dụng khác để chỉ module là component.

If you follow something roughly akin to Extract Till You Drop, then your modules will be composed of a few public functions and the many private functions that were extracted from the public functions. Again, these may be organized into classes so that your module contains a few public classes and several private classes.

Nếu bạn theo một cách làm đại khái giống Extract Till You Drop, thì các module của bạn sẽ gồm vài hàm public và rất nhiều hàm private được extract ra từ các hàm public đó. Một lần nữa, chúng có thể được tổ chức thành các class, để module của bạn chứa vài class public và vài class private.

The public functions, possibly organized into public classes, represent the **interface** of your module. The private functions, also possibly organized into private classes, represent the **implementation** of your module.

Các hàm public — có thể được tổ chức thành các class public — chính là interface của module. Các hàm private — cũng có thể được tổ chức thành các class private — chính là hiện thực (implementation) của module.

To borrow a concept from John Ousterhout's wonderful book *A Philosophy of Software Design*, 1 that module has a narrow interface and a deep implementation—and that's a **good** thing.

Mượn một khái niệm từ cuốn sách tuyệt vời *A Philosophy of Software Design* của John Ousterhout, 1 module đó có interface hẹp và phần hiện thực sâu — và đó là điều tốt.

1 [APOD].

1 [APOD].

Now, pick one of those public functions. It performs a task at the highest level of the module. The name of that **function** should identify that task in abstract terms. For example, consider the `makeStatement` function from the `RentalStatement` class in the previous chapter.

Bây giờ, hãy chọn ra một trong những hàm public đó. Nó thực hiện một công việc ở mức cao nhất của module. Tên của hàm đó phải chỉ ra công việc ấy bằng những từ ngữ trừu tượng. Chẳng hạn, hãy xem hàm `makeStatement` trong class `RentalStatement` ở chương trước.

```
func (statement *RentalStatement) makeStatement() string {
    statement.clearTotals()
    return
        statement.makeHeader() +
statement makeDetails() +
        statement.makeDetails() +
        statement.makeFooter()
}
```

The name of the function is abstract. It tells you what the function does, but not how the function does it. Looking inside the `makeStatement` function we see four calls to private functions that were extracted from it. Each of those names is also abstract, but at a lower level than `makeStatement`. Let's look at `clearTotals`.

Tên hàm mang tính trừu tượng. Nó cho bạn biết hàm làm gì, chứ không cho biết hàm làm điều đó bằng cách nào. Nhìn vào bên trong hàm `makeStatement`, ta thấy bốn lời gọi tới các hàm private đã được extract ra từ nó. Mỗi cái tên ấy cũng trừu tượng, nhưng ở mức thấp hơn `makeStatement`. Hãy xem `clearTotals`.

```
func (statement *RentalStatement) clearTotals() {
    statement.totalAmount = 0.0
    statement.frequentRenterPoints = 0
}
```

We can go no further. We've reached the end of the chain of function calls. The implementation of `clearTotals` tells us exactly how the totals are cleared, in excruciating detail. There is no meaningful lower level to descend to.

Ta không thể đi xa hơn được nữa. Ta đã tới cuối chuỗi lời gọi hàm. Phần hiện thực của `clearTotals` nói cho ta biết chính xác các tổng được xoá về không như thế nào, chi tiết đến từng ly. Chẳng còn mức thấp hơn nào đáng để đi xuống nữa.

Let's look instead at `makeDetails`.

Thay vào đó, hãy xem `makeDetails`.

```
func (statement *RentalStatement) makeDetails() string {
    rentalDetails := ""
    for _, rental := range statement.rentals {
        rentalDetails += statement.makeDetail(rental)
    }
    return rentalDetails
}
```

The name of this function tells us what the function does at this **level of abstraction**. The implementation creates a loop that calls yet another private **method**, `makeDetail`, which is one level of **abstraction** lower.

Tên của hàm này cho ta biết hàm làm gì ở mức trừu tượng của nó. Phần hiện thực dựng nên một vòng lặp gọi tới một hàm private khác nữa là `makeDetail`, vốn nằm thấp hơn một mức trừu tượng.

Thus, every public function creates a tree of calls to private functions at lower levels of abstraction.

Như vậy, mỗi hàm public tạo ra một cây các lời gọi tới những hàm private ở mức trừu tượng thấp hơn.

Here, again, we see **The Stepdown Rule**. Each such descending call goes one level of abstraction down. Not two, not three. Just one. That's why the body of the loop within

Ở đây, một lần nữa, ta lại gặp Quy tắc bậc thang (the Stepdown Rule). Mỗi lần đi xuống như vậy chỉ hạ đúng một mức trừu tượng. Không phải hai, không phải ba. Chỉ một. Đó là lý do vì sao thân vòng lặp bên trong

`makeDetails` calls the next function down. If I had placed the implementation of `makeDetail` within `makeDetails`, then

`makeDetails` lại gọi hàm ở mức kế tiếp bên dưới. Nếu tôi nhét phần hiện thực của `makeDetail` vào thẳng trong `makeDetails`, thì

`makeDetails` would have contained at least two levels of abstraction.

`makeDetails` sẽ chứa ít nhất hai mức trừu tượng.

Do I always follow that rule? No, sometimes I allow two levels of abstraction within a function. Generally, however, I consider that to be rude.

Tôi có luôn tuân theo quy tắc đó không? Không. Đôi khi tôi cho phép hai mức trừu tượng nằm trong cùng một hàm. Nhưng nhìn chung, tôi coi đó là bất lịch sự.

The Newspaper Metaphor — Ẩn dụ tờ báo (the newspaper metaphor)

Think of a well-written newspaper article. You read it vertically. At the top, you expect a headline that will tell you what the story is about and allow you to decide whether it is something you want to read. The first paragraph gives you a synopsis of the whole story, hiding all the details while giving you the broad-brush concepts. As you continue downward, the details increase until you have all the dates, names, quotes, claims, and other minutiae.

Hãy nghĩ tới một bài báo được viết tốt. Bạn đọc nó theo chiều dọc. Trên cùng, bạn trông đợi một cái tít cho biết bài viết nói về chuyện gì, để bạn quyết định có muốn đọc hay không. Đoạn đầu tiên tóm lược toàn bộ câu chuyện, giấu đi mọi chi tiết mà vẫn phác cho bạn những ý niệm tổng quát. Càng đi xuống, chi tiết càng dày lên, cho tới khi bạn có đủ ngày tháng, tên người, trích dẫn, tuyên bố và mọi thứ vụn vặt khác.

Most of us never read the entirety of an article. We start at the headline, and if it is interesting we read on. If that paragraph interests us we read on. And we stay in that very simple loop:

Phần lớn chúng ta chẳng bao giờ đọc trọn một bài báo. Ta bắt đầu từ cái tít, thấy thú vị thì đọc tiếp. Đoạn đó hấp dẫn thì đọc tiếp nữa. Và ta cứ ở trong cái vòng lặp rất đơn giản ấy:

```
while (interested) readMore;
```

As soon as the article becomes boring, we stop. Articles that allow us to use that loop are polite—they let us out early. Articles that force us to waste time reading a lot of unnecessary detail are rude.

Ngay khi bài báo trở nên nhạt nhẽo, ta dừng. Những bài báo cho phép ta dùng cái vòng lặp đó là những bài lịch sự—chúng để ta thoát ra sớm. Những bài buộc ta phải phí thời gian đọc cả đồng chi tiết không cần thiết là những bài bất lịch sự.

A newspaper is composed of many articles; most are very short. Some are a bit longer. Very few contain as much text as a page can hold. This makes the newspaper usable. If the newspaper were just one long story containing a disorganized agglomeration of facts, dates, and names, then we simply would not read it.

Một tờ báo gồm nhiều bài; phần lớn rất ngắn. Vài bài dài hơn chút. Rất hiếm bài chứa nhiều chữ đến mức lấp đầy cả một trang. Chính điều đó làm cho tờ báo dùng được. Nếu tờ báo chỉ là một câu chuyện dài dằng dặc gom góp lộn xộn đủ thứ sự kiện, ngày tháng và tên người, thì đơn giản là ta sẽ không đọc nó.

We would like the source code of a module to be like a newspaper article. The module's name should be simple but explanatory. The name, by itself, should be sufficient to tell us whether we are in the right module or not. The topmost parts of the module should provide the high-level concepts and algorithms. Detail should increase as we move downward, 2 until at the end we find the lowest-level functions and details in the module.

Ta muốn mã nguồn của một module giống như một bài báo. Tên module phải đơn giản nhưng có tính giải thích. Chỉ riêng cái tên thôi cũng đủ cho ta biết mình có đang ở đúng module hay không. Những phần trên cùng của module phải trình bày các khái niệm và thuật toán ở mức cao. Chi tiết tăng dần khi ta đi xuống, 2 cho tới cuối cùng, nơi ta gặp những hàm và chi tiết ở mức thấp nhất của module.

2 Some languages, such as Clojure, invert the **vertical** ordering because they require that declarations appear before their use. Thus, low-level details go at the top and the higher-level policies go toward the bottom.

2 Một số ngôn ngữ, chẳng hạn Clojure, đảo ngược thứ tự theo chiều dọc vì chúng đòi hỏi khai báo phải xuất hiện trước chỗ dùng. Vì vậy, chi tiết mức thấp nằm ở trên, còn các chính sách mức cao thì nằm gần cuối.

A newspaper article that was a mixture of two different stories would be an abomination. So too is a module that contains two or more different topics. 3 So the classes and functions within a module should all be related to a single topic—a single responsibility—with a nice **structure** that proceeds from high-level policy to low-level detail.

Một bài báo trộn lẫn hai câu chuyện khác nhau sẽ là một thứ quái đản. Một module chứa hai chủ đề khác nhau trở lên cũng vậy. 3 Cho nên mọi class và hàm trong một module đều phải thuộc về cùng một chủ đề — một trách nhiệm duy nhất — với cấu trúc gọn gàng đi từ chính sách mức cao xuống chi tiết mức thấp.

3 The Single Responsibility Principle (SRP).

3 SRP — Nguyên tắc Trách nhiệm Đơn nhất (the Single Responsibility Principle).

Be Polite — Hãy lịch sự

When you write functions that encapsulate a single level of abstraction, it allows your readers to escape early. They don't have to read a lot of code to understand what the function does at that level. Sometimes that level is all they care about, and they don't want to be bothered by all the details from the lower levels.

Khi bạn viết những hàm đóng gói đúng một mức trừu tượng, bạn cho phép người đọc thoát ra sớm. Họ không phải đọc cả đống mã mới hiểu hàm làm gì ở mức đó. Đôi khi mức đó là tất cả những gì họ quan tâm, và họ chẳng muốn bị làm phiền bởi mọi chi tiết ở các mức thấp hơn.

For example, I consider the makeStatement function above to be polite. The reader can glance at it and see what it does. If the reader is not worried about anything else, then they are done and go back to whatever else they were doing.

Chẳng hạn, tôi coi hàm makeStatement ở trên là lịch sự. Người đọc chỉ cần liếc qua là thấy nó làm gì. Nếu họ không bận tâm điều gì khác nữa, thế là xong, họ quay lại với việc họ đang làm.

Now consider the original code that makeStatement evolved from.

Bây giờ hãy xem đoạn mã gốc mà makeStatement đã tiến hoá từ đó.

```
func (customer *Customer) statement() string {
    totalAmount := 0.0
    frequentRenterPoints := 0
    result := "Rental Record for " + customer.name + "\n"
    for _, rental := range customer.rentals {
        thisAmount := 0.0
        switch rental.movie.movieType {
        case NewRelease:
            thisAmount += float64(rental.daysRented * 3)
```

```

case Regular:
    thisAmount += 2
    if rental.daysRented > 2 {
        thisAmount += float64(rental.daysRented-2) * 1.5
    }
case Childrens:
    thisAmount += 1.5
    if rental.daysRented > 3 {
        thisAmount += float64(rental.daysRented-3) * 1.5
    }
}
frequentRenterPoints++
if rental.movie.movieType == NewRelease && rental.daysRented >
    frequentRenterPoints++
}
result += fmt.Sprintf("\t%s\t%.1f\n", rental.movie.title, thisA
totalAmount += thisAmount
}
return result +
    fmt.Sprintf(
        "Amount owed is %.1f\nYou earned %d frequent renter points",
        totalAmount, frequentRenterPoints)
}

```

I consider this code to be rude. I consider the author to have been inconsiderate. To understand the high-level intent of this code I must deal with all the low-level details—and that takes time.

Tôi coi đoạn mã này là bất lịch sự. Tôi coi tác giả của nó là người thiếu ý tứ. Để hiểu được ý định mức cao của đoạn mã này, tôi buộc phải đánh vật với toàn bộ chi tiết mức thấp — và điều đó ngốn thời gian.

Think of it this way. Let's say that you were reading code somewhere and you came across a call to the polite

Hãy hình dung thế này. Giả sử bạn đang đọc mã ở đâu đó và bắt gặp một lời gọi tới hàm lịch sự

makeStatement(). The name of the function tells you what it does, but perhaps you'd like to check that the statement includes the footer. So you click over to the implementation of makeStatement and you see four simple lines of code that tell you exactly what you need to know. Two seconds later you click back to where you were and continue reading, suffering no feeling of interruption—just a simple question asked and answered in two seconds.

makeStatement(). Tên hàm cho bạn biết nó làm gì, nhưng có lẽ bạn muốn kiểm tra xem bản kê có kèm phần footer hay không. Thế là bạn nhảy sang phần hiện thực của makeStatement và thấy bốn dòng mã đơn giản nói cho bạn đúng những gì bạn cần biết. Hai giây sau, bạn nhảy ngược về chỗ cũ và đọc tiếp, chẳng hề thấy bị ngắt mạch — chỉ là một câu hỏi đơn giản được đặt ra và trả lời trong hai giây.

Now let's say that you were reading code somewhere and you came across a call to the rude statement() function. The name is a noun, which describes no action. You have to infer that the function builds the statement. Again, you're not sure if the statement includes the footer. So you click over to the implementation and are faced with 30 lines of deeply detailed code. Is the footer in

there? Does the code that builds the footer get called at the right time? A minute or two later you've satisfied yourself that the footer is intact, but now you've forgotten why you were worried about that. You click back to where you were, and you've lost your train of thought and have to backtrack several lines, and several more minutes, to remind yourself of what you were doing and re-create that train of thought in your mind.

Bây giờ hãy giả sử bạn đang đọc mã ở đâu đó và bắt gặp một lời gọi tới hàm `statement()` bất lịch sự. Cái tên là một danh từ, chẳng mô tả hành động nào cả. Bạn phải tự suy ra rằng hàm này dừng nên bản kê. Một lần nữa, bạn không chắc bản kê có kèm phần footer hay không. Thế là bạn nhảy sang phần hiện thực và đối diện với 30 dòng mã chi tiết đến ngộp thở. Footer có nằm trong đó không? Đoạn mã dựng footer có được gọi đúng lúc không? Một hai phút sau, bạn đã yên tâm rằng footer vẫn còn nguyên, nhưng giờ bạn lại quên bém mắt vì sao mình phải lo chuyện đó. Bạn nhảy ngược về chỗ cũ, và mạch suy nghĩ đã đứt: bạn phải lần ngược lại mấy dòng — thêm mấy phút nữa — chỉ để nhắc mình nhớ mình đang làm gì và dựng lại mạch suy nghĩ ấy trong đầu.

That's rude. It's as rude as the guy who bursts into your cubicle and throws a question at you without waiting for you to acknowledge him.

Thế là bất lịch sự. Bất lịch sự y như gã xộc vào chỗ ngồi của bạn rồi quăng ngay một câu hỏi mà chẳng buồn đợi bạn ngừng lên đáp lời.

The Stepdown Rule: Once Again — Quy tắc bậc thang: một lần nữa

So, the rule I like to follow, in order to be polite, is to descend one level of abstraction at a time. Each function has a name that defines what the function does, and each function consists of a set of lines that describe the implementation and that are all at the same level of abstraction. Each of those lines, if it is not already at the lowest level, calls functions at the next level down.

Vậy nên, quy tắc mà tôi thích tuân theo để giữ phép lịch sự là: mỗi lần chỉ đi xuống một mức trừu tượng. Mỗi hàm có một cái tên định nghĩa hàm đó làm gì, và mỗi hàm gồm một nhóm dòng mô tả phần hiện thực, tất cả đều nằm ở cùng một mức trừu tượng. Mỗi dòng trong đó, nếu chưa ở mức thấp nhất, sẽ gọi các hàm ở mức kế tiếp bên dưới.

How do you determine how big one level of abstraction is? All I can say to that is to use your best judgment. For me, it's pretty simple. If I can meaningfully extract, then I do the extraction and check that it makes sense.

Làm sao xác định được một mức trừu tượng thì “to” cỡ nào? Tất cả những gì tôi có thể nói là hãy dùng phán đoán tốt nhất của bạn. Với tôi thì khá đơn giản: nếu tôi có thể extract ra một cách có ý nghĩa, thì tôi extract rồi kiểm lại xem nó có hợp lý không.

This is a polite practice, and it helps avoid the abstraction roller coaster.

Đây là một thói quen lịch sự, và nó giúp tránh được trò tàu lượn trừu tượng.

The Abstraction Roller Coaster — Trò tàu lượn trừu tượng

Look again at that rude 30-line `statement` function. Right at the top, why do `totalAmount` and `frequentRenterPoints` need to be set to zero? What are we doing? This is the first line of a high-level

function, and we are confronted with the lowest-level kind of detail. Then, we pop up a level to deal with the

Hãy nhìn lại hàm statement 30 dòng bất lịch sự kia. Ngay ở trên cùng, tại sao totalAmount và frequentRenterPoints lại cần được gán về không? Chúng ta đang làm gì vậy? Đây là dòng đầu tiên của một hàm mức cao, vậy mà ta bị dí thẳng vào mặt thứ chi tiết ở mức thấp nhất. Rồi ta nhảy vọt lên một mức để xử lý

result, which we must infer to be the header, and then there's a loop. The first line of the loop descends immediately to another low-level detail and then enters a

result — thứ mà ta phải tự suy ra là phần header — rồi tới một vòng lặp. Dòng đầu tiên của vòng lặp lập tức lao xuống một chi tiết mức thấp khác, rồi bước vào một

switch statement.

câu lệnh switch.

Can you feel the roller coaster? We are going up and down through levels of abstraction. Sometimes we are high, as we set up the loop through the rentals, and sometimes we are low, as we zero out totals and check individual cases of the switch.

Bạn có cảm nhận được trò tàu lượn không? Ta cứ leo lên tụt xuống qua các mức trừu tượng. Có lúc ta ở trên cao, khi thiết lập vòng lặp duyệt qua các lượt thuê; có lúc ta ở dưới thấp, khi xoá các tổng về không và kiểm từng nhánh của switch.

Riding that roller coaster is not fun for the reader. The reason is simple. Every time you go from a line at a high level to a line at a low level, you have to push your mental model up on your mental stack to deal with that low-level concern. Once you understand it, then you pop your mental stack back to pick up where you left off.

Ngồi trên chuyến tàu lượn đó chẳng vui chút nào với người đọc. Lý do rất đơn giản. Mỗi lần bạn đi từ một dòng ở mức cao xuống một dòng ở mức thấp, bạn phải push mô hình tư duy hiện tại lên ngăn xếp trong đầu để xử lý mối bận tâm mức thấp kia. Hiểu xong rồi, bạn pop ngăn xếp ấy ra để nối lại chỗ đang dở.

The problem with that is that we do not have a mental stack. When we push our train of thought aside to deal with a detail, more often than not we lose that train of thought. Again, it's like that guy who bursts into your cubicle and shouts a question at you.

Vấn đề là chúng ta làm gì có ngăn xếp trong đầu. Khi ta gạt mạch suy nghĩ sang một bên để xử lý một chi tiết, thường thì ta mất luôn mạch suy nghĩ đó. Lại vẫn là gã xộc vào chỗ ngồi của bạn rồi hét vào mặt bạn một câu hỏi.

This Is How We Write, but Not How We Want to Read — Ta viết mã theo kiểu này, nhưng ta không muốn đọc theo kiểu này

The problem is that we write code on the roller coaster. We all do. We all must. Imagine the person writing that original rude statement function. Perhaps the first line they wrote was the loop, but then they remembered to zero out the totalAmount. Later, after they wrote the switch, they realized they needed to deal with the frequent renter points, and so they went back to the top and added that line.

Vấn đề là ta viết mã ngay trên chuyến tàu lượn. Ai cũng vậy. Ai cũng buộc phải vậy. Hãy hình dung người đã viết ra hàm statement bất lịch sự ban đầu kia. Có lẽ dòng đầu tiên họ viết là vòng lặp, nhưng rồi họ sực nhớ phải xoá totalAmount về không. Về sau, viết xong

switch, họ nhận ra mình còn phải xử lý điểm thuê thường xuyên (frequent renter points), thế là quay ngược lên đầu và thêm dòng đó vào.

In short, we construct our code by rolling through the undulating track of abstractions and details. Our code comes out looking like the roller coaster. And this is where Kent Beck's advice becomes critical:

Nói ngắn gọn, ta dựng nên mã của mình bằng cách lăn qua lăn lại trên đường ray nhấp nhô giữa trừu tượng và chi tiết. Và mã của ta rốt cuộc trông đúng như một chuyến tàu lượn. Đây chính là lúc lời khuyên của Kent Beck trở nên tối quan trọng:

First, make it work. Then, make it right.

Trước hết, hãy làm cho nó chạy. Rồi hãy làm cho nó đúng đắn.

You are not done when the code works. You are done when the code reads well. You are done when you have extracted and organized and laid out the tree of functions descending from the public functions in a manner convenient to the reader.

Bạn chưa xong khi mã chạy được. Bạn xong khi mã đọc lên trôi chảy. Bạn xong khi đã extract, sắp xếp và bày biện cái cây hàm toả xuống từ các hàm public theo cách thuận tiện cho người đọc.

Now, again, I don't follow these rules religiously or dogmatically. There are situations in which following them is inappropriate. But these rules are my default. I will follow them unless I have a good reason not to.

Và một lần nữa, tôi không tuân theo những quy tắc này một cách sùng tín hay giáo điều. Có những tình huống mà tuân theo chúng là không phù hợp. Nhưng đây là mặc định của tôi. Tôi sẽ theo chúng, trừ khi có lý do chính đáng để không theo.

Chapter 12. Objects and Data Structures — Chương 12. Đối tượng và cấu trúc dữ liệu



Niklaus Wirth once wrote a lovely book titled Algorithms + **Data** Structures = Programs . 1 Though it wasn't his intent, the title very neatly captures the fundamental concept behind **object**-oriented programming: that there is a deep relation between functions and the data they manipulate.

Niklaus Wirth từng viết một cuốn sách rất hay mang tựa Algorithms + Data Structures = Programs.¹ Dù đó không phải chủ ý của ông, cái tựa ấy đã tóm gọn cực kỳ khéo léo khái niệm nền tảng đứng sau lập trình hướng đối tượng: giữa các hàm và dữ liệu mà chúng thao tác có một mối quan hệ sâu sắc.

1 [A+DS=P].

1 [A+DS=P].

Object orientation was invented 2 /discovered by Ole-Johann Dahl and Kristen Nygaard in the mid-to late '60s through the gradual evolution of their Simula language, culminating in Simula 67. Bjarne Stroustrup and Alan Kay both used Simula 67 in the early '70s and then extended the ideas in C++

and Smalltalk, respectively.

Hướng đối tượng được Ole-Johann Dahl và Kristen Nygaard phát minh² — hay khám phá ra — vào khoảng giữa đến cuối thập niên 1960, qua quá trình tiến hoá dần dần của ngôn ngữ Simula do hai ông tạo ra, mà đỉnh cao là Simula 67. Bjarne Stroustrup và Alan Kay đều từng dùng Simula 67 hồi đầu thập niên 1970, rồi mở rộng những ý tưởng đó lần lượt trong C++ và Smalltalk.

2 The story of this invention is fascinating. I wrote about it in [We Programmers].

2 Câu chuyện về phát minh này rất hấp dẫn. Tôi đã viết về nó trong [We Programmers].

From there, C++ evolved into Java and C# and then Go, while Smalltalk evolved into Objective-C, Ruby, and Python. Both of those lines of evolution borrowed from each other, causing a lot of cross-pollination. Today, virtually all of our languages are object oriented in one sense or another.

Từ đó, C++ tiến hoá thành Java và C# rồi tới Go, còn Smalltalk tiến hoá thành Objective-C, Ruby và Python. Hai nhánh tiến hoá này vay mượn lẫn nhau, tạo ra rất nhiều sự lai tạp qua lại. Ngày nay, hầu như mọi ngôn ngữ của chúng ta đều hướng đối tượng theo nghĩa này hay nghĩa khác.

What Is an Object? — Đối tượng là gì?

An object is a bundle of data manipulated by dedicated functions, some of which are public. From the outside looking in, only those public functions are visible. 3 The **structure** of the data within the object is hidden.

Một đối tượng (object) là một bó dữ liệu được thao tác bởi những hàm chuyên trách, trong đó một số hàm là public. Nhìn từ bên ngoài vào, chỉ những hàm public ấy là thấy được.³ Cấu trúc của dữ liệu bên trong đối tượng thì bị giấu đi.

3 Some languages enforce that visibility with keywords like public and protected . Other languages depend upon programmers to adopt conventions to determine which functions are public and private.

3 Một số ngôn ngữ cưỡng chế tính khả kiến đó bằng các từ khoá như public và protected. Những ngôn ngữ khác thì trông cậy vào việc lập trình viên tự tuân theo quy ước để xác định hàm nào là public, hàm nào là private.

An object-oriented program is composed of a group of such objects, each with its own responsibilities and each collaborating with the others by calling each other's public functions.

Một chương trình hướng đối tượng được tạo thành từ một nhóm những đối tượng như vậy, mỗi đối tượng có trách nhiệm riêng của mình và cộng tác với các đối tượng khác bằng cách gọi các hàm public của nhau.

Alan Kay once imagined this using a biology analogy—a set of cells collaborating by passing molecular messages to each other. The cells do not know of each other's internals, and may not even know the particular cells they are communicating with. They simply emit messages (molecules) that other cells detect and respond to. Notice that cells never ask other cells questions. They send messages but do not expect answers.

Alan Kay từng hình dung điều này qua một phép loại suy sinh học — một tập hợp các tế bào cộng tác với nhau bằng cách trao đổi những thông điệp phân tử. Các tế bào không biết gì về nội tạng của nhau, thậm chí có thể còn không biết cụ thể mình đang liên lạc với tế bào nào. Chúng chỉ phát ra thông điệp (phân tử) để những tế bào khác nhận biết và phản

ứng lại. Hãy để ý rằng tế bào không bao giờ hỏi han tế bào khác. Chúng gửi thông điệp nhưng không chờ đợi câu trả lời.

The message-passing concept is a very powerful principle for organizing **behavior**. If you only tell an object what to do and never ask that object anything, then the data in that object is not relevant to you.

Khái niệm truyền thông điệp là một nguyên tắc rất mạnh để tổ chức hành vi. Nếu bạn chỉ bảo một đối tượng phải làm gì mà không bao giờ hỏi nó điều gì, thì dữ liệu bên trong đối tượng ấy chẳng liên quan gì đến bạn.

Avoiding all queries is impractical in software; but as we shall see, there are ways to keep such queries to a minimum. This lack of dependence upon data means that object-oriented designs focus primarily on behavior. Objects are constructed and related so as to support the flow of messages that elicit the required behaviors of the system. The internal structure of the data within the individual objects is hidden from, and therefore nearly irrelevant to, the majority of that behavior.

Tránh hoàn toàn mọi truy vấn là điều bất khả thi trong phần mềm; nhưng như ta sẽ thấy, vẫn có cách giữ những truy vấn đó ở mức tối thiểu. Việc không phụ thuộc vào dữ liệu này có nghĩa là thiết kế hướng đối tượng tập trung chủ yếu vào hành vi. Các đối tượng được dựng lên và liên kết với nhau sao cho hỗ trợ được dòng chảy thông điệp làm phát sinh những hành vi mà hệ thống cần. Cấu trúc nội tại của dữ liệu bên trong từng đối tượng bị giấu khỏi phần lớn hành vi ấy, và do đó gần như chẳng còn liên quan gì tới nó.

But OO is not the only game in town.

Nhưng OO không phải cuộc chơi duy nhất trong thị trấn.

In 1970, Edgar Codd published A Relational Model of Data for Large Shared Databanks . 4 From there he, Raymond Boyce, and Chris Date derived/discovered the concept that currently drives all relational databases. The overarching concept is that the structure of the data is critical and visible, and all programs that use it are subservient to it.

Năm 1970, Edgar Codd công bố A Relational Model of Data for Large Shared Databanks.4 Từ đó, ông cùng Raymond Boyce và Chris Date rút ra — hay khám phá ra — cái khái niệm hiện đang vận hành toàn bộ các cơ sở dữ liệu quan hệ. Khái niệm bao trùm ở đây là: cấu trúc của dữ liệu mới là thứ tối quan trọng và phải lộ ra, còn mọi chương trình dùng nó đều phải phục tùng nó.

4 Communications of the ACM 13, no. 6 (June 1970), 377–387.

4 Communications of the ACM 13, no. 6 (June 1970), 377–387.

A large enterprise database is a complex, sprawling **data structure**, whereas the programs that manipulate it are little more than minions that surround it. The schema of the database dominates those programs.

Một cơ sở dữ liệu doanh nghiệp lớn là một cấu trúc dữ liệu (data structure) phức tạp, đồ sộ, còn những chương trình thao tác trên nó thì chẳng hơn gì mấy đám tùy tùng vây quanh. Schema của cơ sở dữ liệu thống trị đám chương trình đó.

This is the opposite of OO. To recap: An object is a set of public functions with data hidden deep inside. A database is a published set of data structures with no intrinsic functions. Again, they are very nearly opposites. And, as we shall see, that opposing nature goes very deep indeed.

Đây là điều ngược hẳn với OO. Tóm lại: một đối tượng là một tập hợp các hàm public với dữ liệu giấu tít bên trong. Một cơ sở dữ liệu là một tập hợp cấu trúc dữ liệu được công bố

công khai mà bản thân không có hàm nào. Xin nhắc lại, chúng gần như trái ngược nhau. Và như ta sẽ thấy, cái bản chất đối nghịch ấy còn ăn sâu hơn nhiều.

Data Abstraction — Trừu tượng hoá dữ liệu

There is a reason that we tend to keep our variables private. We don't want anyone else to depend on them. We want to keep the freedom to change their type or **implementation** on a whim or an impulse. Why, then, do so many programmers automatically add getters and setters to their objects, exposing their private variables as if they were public?

Có một lý do khiến ta thường để các biến của mình ở chế độ private. Ta không muốn ai khác phụ thuộc vào chúng. Ta muốn giữ quyền tự do đổi kiểu hay đổi hiện thực (implementation) của chúng bất cứ lúc nào, chỉ vì hứng lên. Vậy thì tại sao rất nhiều lập trình viên lại theo phản xạ mà nhét getter và setter vào đối tượng của mình, phơi những biến private đó ra như thể chúng là public?

Consider the difference between Point 1 and Point 2 below. Both represent the data of a point on the Cartesian plane. And yet one exposes its implementation and the other completely hides it.

Hãy xét khác biệt giữa Point 1 và Point 2 dưới đây. Cả hai đều biểu diễn dữ liệu của một điểm trên mặt phẳng Descartes. Vậy mà một cái phơi bày hiện thực của nó, còn cái kia thì giấu tiệt đi.

```
---Point 1---
public class Point {
    public double x;
    public double y;
}
---Point 2---
public interface Point {
    double getX();
    double getY();
    void setCartesian(double x, double y);
    double getR();
    double getTheta();

    void setPolar(double r, double theta);
}
```

The beautiful thing about Point 2 is that there is no way you can tell whether the implementation is in rectangular or polar coordinates. It might be neither! The **interface** represents an abstract data structure. We can tell that there must be a data structure in there somewhere, but we do not know what form that data structure takes.

Điều đẹp đẽ ở Point 2 là bạn không tài nào biết được hiện thực của nó dùng toạ độ vuông góc hay toạ độ cực. Có khi chẳng phải cái nào cả! Interface ở đây biểu diễn một cấu trúc dữ liệu trừu tượng. Ta biết chắc bên trong phải có một cấu trúc dữ liệu nào đó, nhưng không biết cấu trúc đó mang hình hài gì.

But Point 2 represents more than just a data structure. It also represents behavioral policies. Despite the fact that the getter methods allow you to access individual coordinate values, the two setter methods enforce the policy that the two coordinate values are set together as an atomic operation.

Nhưng Point 2 còn biểu diễn nhiều hơn một cấu trúc dữ liệu. Nó còn biểu diễn cả các chính sách về hành vi. Dù các method getter cho phép bạn truy cập từng giá trị toạ độ riêng lẻ, hai method setter lại cưỡng chế chính sách rằng hai giá trị toạ độ phải được gán cùng lúc như một thao tác nguyên tử.

Point 1, on the other hand, is very clearly implemented in rectangular coordinates, and it forces us to manipulate the coordinate values independently. This exposes implementation. Indeed, it would expose implementation even if the variables were private and we were using single-**variable** getters and setters.

Point 1, ngược lại, hiện thực rõ mồn một bằng toạ độ vuông góc, và nó buộc ta phải thao tác từng giá trị toạ độ một cách riêng lẻ. Như vậy là phơi bày hiện thực. Thật ra, nó vẫn phơi bày hiện thực ngay cả khi các biến là private và ta dùng getter với setter cho từng biến một.

Hiding implementation is not just a matter of putting a layer of functions between the variables. Hiding implementation is about abstractions! A **class** does not simply push its variables out through getters and setters. Rather, it exposes abstract interfaces that allow its users to manipulate the essence of the data, without having to know how that data is implemented.

Giấu hiện thực không đơn thuần là kê một lớp hàm chắn trước các biến. Giấu hiện thực là chuyện của trừu tượng hoá (abstraction)! Một class không chỉ đơn giản đẩy các biến của nó ra ngoài qua getter và setter. Đúng hơn, nó phơi ra những interface trừu tượng cho phép người dùng thao tác trên phần cốt lõi của dữ liệu mà không cần biết dữ liệu ấy được hiện thực ra sao.

Consider Vehicle 1 and Vehicle 2 below. The first uses concrete terms to communicate the fuel level of a vehicle, whereas the second does so with the **abstraction** of percentage. In the concrete case, you can be pretty sure that these are just accessors of variables. In the abstract case, you have no clue at all about the form of the data.

Hãy xem Vehicle 1 và Vehicle 2 dưới đây. Cái thứ nhất dùng những thuật ngữ cụ thể để truyền đạt mức nhiên liệu của một chiếc xe, còn cái thứ hai làm việc đó bằng mức trừu tượng là phần trăm. Với trường hợp cụ thể, bạn khá chắc rằng đó chỉ là mấy accessor của biến. Với trường hợp trừu tượng, bạn chẳng có manh mối nào về hình hài của dữ liệu.

```
---Vehicle 1---
public interface Vehicle {
    double getFuelTankCapacityInGallons();
    double getGallonsOfGasoline();
}
---Vehicle 2---
public interface Vehicle {
    double getPercentFuelRemaining();
}
```

In both of the above cases, the second option is preferable. We do not want to expose the details of our data. Rather, we want to express our data in abstract terms. This is not merely accomplished by using interfaces and/or getters and setters. Serious thought needs to be put into the best way to represent the data that an object contains. The worst option is to blithely add getters and setters.

Trong cả hai trường hợp trên, lựa chọn thứ hai đều đáng chọn hơn. Ta không muốn phơi bày chi tiết dữ liệu của mình. Đúng hơn, ta muốn diễn đạt dữ liệu bằng những thuật ngữ trừu tượng. Điều đó không đơn thuần đạt được bằng cách dùng interface và/hoặc getter

và setter. Cần suy nghĩ nghiêm túc về cách tốt nhất để biểu diễn dữ liệu mà một đối tượng chứa đựng. Lựa chọn tệ nhất là hồn nhiên nhét thêm getter và setter.

Data/Object Antisymmetry — Tính phản đối xứng dữ liệu/đối tượng

These two examples show the difference between objects and data structures. Objects hide their data behind abstractions and expose functions that operate on that data. Data structures expose their data and have no meaningful functions. Go back and read that again. Notice the complementary nature of the two definitions. They are virtual opposites. This difference may seem trivial, but it has far-reaching implications.

Hai ví dụ này cho thấy khác biệt giữa đối tượng và cấu trúc dữ liệu. Đối tượng giấu dữ liệu của mình sau các lớp trừu tượng và phơi ra những hàm thao tác trên dữ liệu ấy. Cấu trúc dữ liệu phơi bày dữ liệu của mình và không có hàm nào đáng kể. Hãy đọc lại câu đó lần nữa. Để ý bản chất bổ trợ cho nhau của hai định nghĩa này. Chúng gần như trái ngược nhau. Khác biệt này nghe có vẻ vụn vặt, nhưng hệ quả của nó thì vươn rất xa.

Consider, for example, the Procedural Shapes example below. The Geometry class operates on the three shape classes. The shape classes are simple data structures without any behavior. All the behavior is in the Geometry class.

Chẳng hạn, hãy xét ví dụ Procedural Shapes dưới đây. Class Geometry thao tác trên ba class hình. Các class hình chỉ là những cấu trúc dữ liệu đơn giản, không có hành vi nào. Toàn bộ hành vi nằm trong class Geometry.

```
---Procedural Shapes---
public class Square {
    public Point topLeft;
    public double side;
}
public class Rectangle {
    public Point topLeft;
    public double height;
    public double width;
}
public class Circle {
    public Point center;
    public double radius;
}
public class Geometry {
    public final double PI = 3.141592653589793;
    public double area(Object shape) throws NoSuchShapeException
    {
        if (shape instanceof Square) {
            Square s = (Square)shape;
            return s.side * s.side;
        }
        else if (shape instanceof Rectangle) {
            Rectangle r = (Rectangle)shape;
```



```

        return r.height * r.width;
    }
    else if (shape instanceof Circle) {
        Circle c = (Circle)shape;
        return PI * c.radius * c.radius;
    }
    throw new NoSuchShapeException();
}
}

```

Object-oriented programmers might wrinkle their noses at this and complain that it is procedural—and they'd be right. But the sneer may not be warranted. Consider what would happen if a `perimeter()` **function** were added to `Geometry`. The shape data structures would be unaffected! Any other modules that depended upon those shapes would also be unaffected! On the other hand, if I add a new shape, I must change all the functions in `Geometry` to deal with it. Again, read that over. When you add a new function, the data structures remain unchanged; but when you add a new data structure, the functions must all be changed.

Đám lập trình viên hướng đối tượng có thể nhăn mũi mà phàn nàn rằng cái này mang tính thủ tục — và họ nói đúng. Nhưng cái bĩu môi ấy chưa chắc đã có cơ sở. Hãy thử nghĩ xem chuyện gì sẽ xảy ra nếu ta thêm hàm `perimeter()` vào `Geometry`. Các cấu trúc dữ liệu hình sẽ chẳng hề hấn gì! Mọi module khác phụ thuộc vào những hình đó cũng chẳng hề hấn gì! Ngược lại, nếu tôi thêm một hình mới, tôi phải sửa toàn bộ các hàm trong `Geometry` để xử lý nó. Lại một lần nữa, xin đọc kỹ đoạn này. Khi bạn thêm một hàm mới, các cấu trúc dữ liệu vẫn nguyên vẹn; nhưng khi bạn thêm một cấu trúc dữ liệu mới, tất cả các hàm đều phải sửa.

Now consider the OO Shapes example below. Here the `area()` **method** is polymorphic. No `Geometry` class is necessary. So, if I add a new shape, none of the existing functions are affected, but if I add a new function, all of the shapes must be changed! 5

Giờ hãy xem ví dụ OO Shapes bên dưới. Ở đây method `area()` là đa hình (polymorphism). Không cần class `Geometry` nào cả. Thế nên, nếu tôi thêm một hình mới, chẳng hàm nào hiện có bị ảnh hưởng, nhưng nếu tôi thêm một hàm mới thì tất cả các hình đều phải sửa! 5

5 There are ways around this that are well known to experienced object-oriented designers: Visitor, or dual-dispatch, for example. But these techniques carry costs of their own and generally return the structure to that of a procedural program.

5 Có những cách lách chuyện này mà dân thiết kế hướng đối tượng lão luyện đều biết rõ: Visitor, hay dual-dispatch chẳng hạn. Nhưng các kỹ thuật đó cũng có cái giá riêng của chúng, và nhìn chung là kéo cấu trúc quay về đúng dạng của một chương trình thủ tục.

```

--OO Shapes---
public class Square implements Shape {
    private Point topLeft;
    private double side;
    public double area() {
        return side*side;
    }
}

public class Rectangle implements Shape {
    private Point topLeft;
    private double height;

```

```

    private double width;
    public double area() {
        return height * width;
    }
}
public class Circle implements Shape {
    private Point center;
    private double radius;
    public final double PI = 3.141592653589793;
    public double area() {
        return PI * radius * radius;
    }
}

```

Again we see the complementary nature of these two definitions; they are virtual opposites! This exposes the fundamental dichotomy between objects and data structures: Procedural code (code using data structures) makes it easy to add new functions without changing the existing data structures. OO code, on the other hand, makes it easy to add new data structures (classes) without changing existing functions.

Lại một lần nữa ta thấy bản chất bổ trợ cho nhau của hai định nghĩa này; chúng gần như trái ngược nhau! Điều đó phơi bày sự phân đôi căn bản giữa đối tượng và cấu trúc dữ liệu: mã thủ tục (mã dùng cấu trúc dữ liệu) khiến việc thêm hàm mới trở nên dễ dàng mà không phải sửa các cấu trúc dữ liệu sẵn có. Ngược lại, mã OO khiến việc thêm cấu trúc dữ liệu mới (các class) trở nên dễ dàng mà không phải sửa các hàm sẵn có.

The complement is also true: Procedural code makes it hard to add new data structures, because all the functions must change. OO code makes it hard to add new functions, because all the classes must change.

Điều ngược lại cũng đúng: mã thủ tục khiến việc thêm cấu trúc dữ liệu mới trở nên khó khăn, vì tất cả các hàm đều phải đổi. Mã OO khiến việc thêm hàm mới trở nên khó khăn, vì tất cả các class đều phải đổi.

So, the things that are hard for OO are easy for procedures, and the things that are hard for procedures are easy for OO.

Vậy nên, những gì khó với OO thì lại dễ với thủ tục, còn những gì khó với thủ tục thì lại dễ với OO.

In any complex system, there are going to be times when we want to add new data types rather than new functions. For these cases, objects and OO are most appropriate. On the other hand, there will also be times when we'll want to add new functions as opposed to data types. In these cases, procedural code and data structures will be more appropriate.

Trong bất kỳ hệ thống phức tạp nào, sẽ có lúc ta muốn thêm kiểu dữ liệu mới chứ không phải hàm mới. Với những trường hợp đó, đối tượng và OO là phù hợp nhất. Ngược lại, cũng sẽ có lúc ta muốn thêm hàm mới thay vì kiểu dữ liệu. Trong những trường hợp đó, mã thủ tục và cấu trúc dữ liệu sẽ hợp hơn.

Mature programmers know that the idea that everything is an object is a myth. Sometimes you really do want simple data structures with procedures operating on them.

Lập trình viên chín chắn biết rằng cái ý tưởng “mọi thứ đều là đối tượng” chỉ là chuyện hoang đường. Đôi khi thứ bạn thực sự cần chính là những cấu trúc dữ liệu đơn giản với

các thủ tục thao tác trên chúng.

The Law of Demeter — Luật Demeter

There is a well-known **heuristic** called **the Law of Demeter** ⁶ that recommends that a **module** should not know about the innards of the objects it manipulates. As we saw in the last section, objects hide their data and expose operations. This means that an object should not expose its internal structure through accessors, because to do so is to expose, rather than to hide, its internal structure.

Có một kinh nghiệm thực hành (heuristic) nổi tiếng tên là Luật Demeter (the Law of Demeter)⁶ khuyên rằng một module không nên biết về nội tạng của những đối tượng mà nó thao tác. Như ta đã thấy ở phần trước, đối tượng giấu dữ liệu và phơi ra các thao tác. Điều này có nghĩa là một đối tượng không nên phơi cấu trúc nội tại của mình ra qua các accessor, vì làm vậy là phơi bày chứ không phải giấu đi cấu trúc nội tại.

⁶ http://en.wikipedia.org/wiki/Law_of_Demeter More precisely, the Law of Demeter says that a method *f* of a class *C* should only call the methods of the following:

⁶ http://en.wikipedia.org/wiki/Law_of_Demeter Chính xác hơn, Luật Demeter nói rằng một method *f* của class *C* chỉ nên gọi method của những thứ sau:

- *C*
- An object created by *f*
- An object passed as an **argument** to *f*
- An object held in an instance variable of *C*
 - *C*
 - Một đối tượng do *f* tạo ra
 - Một đối tượng được truyền làm đối số cho *f*
 - Một đối tượng được giữ trong một biến instance của *C*

The method should not invoke methods on objects that are returned by any of the allowed functions. In other words, be shy, talk to friends, and don't talk to strangers.

Method đó không nên gọi method trên những đối tượng do bất kỳ hàm hợp lệ nào kể trên trả về. Nói cách khác: hãy nhút nhát, chỉ trò chuyện với bạn bè, và đừng bắt chuyện với người lạ.

The following code ⁷ appears to violate the Law of Demeter (among other things) because it calls the `getScratchDir()` function on the return value of `getOptions()` and then calls

Đoạn mã sau đây⁷ có vẻ vi phạm Luật Demeter (cùng vài thứ khác nữa), vì nó gọi hàm `getScratchDir()` trên giá trị trả về của `getOptions()`, rồi lại gọi

`getAbsolutePath()` on the return value of `getScratchDir()`.

`getAbsolutePath()` trên giá trị trả về của `getScratchDir()`.

⁷ Found somewhere in the Apache framework.

⁷ Tìm thấy đâu đó trong framework Apache.

```
final String outputDir =
    ctxt.getOptions().getScratchDir().getAbsolutePath();
```

Trainwrecks — Trainwrecks

This kind of code is often called a trainwreck because it look like a bunch of coupled train cars. Chains of calls like this are generally considered to be sloppy style and should be avoided. It is usually best to split them up as follows:

Kiểu mã này thường được gọi là trainwreck (vụ trật đường ray) vì trông nó chẳng khác gì một dây toa tàu móc nối vào nhau. Những chuỗi lời gọi như vậy nhìn chung bị coi là văn phong luộm thuộm và nên tránh. Thường thì tốt nhất là tách chúng ra như sau:

```
Options opts = ctxt.getOptions();
File scratchDir = opts.getScratchDir();
final String outputDir = scratchDir.getAbsolutePath();
```

Are these two snippets of code violations of the Law of Demeter? Certainly the containing module knows that the

Hai đoạn mã này có vi phạm Luật Demeter không? Chắc chắn module chứa chúng biết rằng

ctxt object contains options, which contain a scratch directory, which has an absolute path. That's a lot of knowledge for one function to know. The calling function knows how to navigate through a lot of different objects.

đối tượng ctxt chứa options, trong options có một scratch directory, và scratch directory ấy có một absolute path. Đó là quá nhiều hiểu biết cho một hàm duy nhất. Hàm gọi biết rành rẽ đường đi nước bước xuyên qua cả đồng đối tượng khác nhau.

Whether this is a violation of Demeter depends on whether or not ctxt, Options, and ScratchDir are objects or data structures. If they are objects, then their internal structure should be hidden rather than exposed, and so knowledge of their innards is a clear violation of the Law of Demeter. On the other hand, if ctxt, Options, and ScratchDir are just data structures with no behavior, then they naturally expose their internal structure, and so Demeter does not apply.

Chuyện này có phải là vi phạm Demeter hay không còn tùy vào việc ctxt, Options và ScratchDir là đối tượng hay cấu trúc dữ liệu. Nếu chúng là đối tượng thì cấu trúc nội tại của chúng lẽ ra phải được giấu đi chứ không phơi ra, và như vậy việc biết nội tạng của chúng rõ ràng là vi phạm Luật Demeter. Ngược lại, nếu ctxt, Options và ScratchDir chỉ là những cấu trúc dữ liệu không có hành vi, thì chúng phơi bày cấu trúc nội tại là chuyện hiển nhiên, và Demeter không áp dụng ở đây.

The use of accessor functions in the examples above confuses the issue. If the code had been written as follows, then we probably wouldn't be worried about Demeter violations.

Việc dùng các hàm accessor trong những ví dụ trên làm vấn đề trở nên rối rắm. Nếu đoạn mã được viết như sau, hẳn ta đã chẳng phải lo lắng gì về chuyện vi phạm Demeter.

```
final String outputDir = ctxt.options.scratchDir.absolutePath;
```

This issue would be a lot less confusing if we adopted the convention that data structures had public variables and no functions, whereas objects had private variables and public functions that abstracted the implementation.

Vấn đề này sẽ bớt rối rắm đi rất nhiều nếu ta theo quy ước rằng cấu trúc dữ liệu thì có biến public và không có hàm nào, còn đối tượng thì có biến private và những hàm public trừu tượng hoá phần hiện thực.

Hybrids — Thể lai (Hybrids)

This confusion sometimes leads to unfortunate hybrid structures that are half object and half data structure. They have functions that do significant things, and they also have either public variables or public accessors and mutators that, for all intents and purposes, make the private variables public, tempting other external functions to use those variables the way a procedural program would use a data structure. 8

Sự lẫn lộn này đôi khi dẫn tới những cấu trúc lai đáng tiếc, nửa đối tượng nửa cấu trúc dữ liệu. Chúng có những hàm làm việc hệ trọng, đồng thời lại có biến public hoặc những accessor và mutator public mà xét đến cùng thì biến hết các biến private thành public, dụ đổ các hàm bên ngoài dùng những biến đó y như cách một chương trình thủ tục dùng cấu trúc dữ liệu.⁸

8 This is sometimes called feature envy in **[Refactoring]**.

8 Trong **[Refactoring]**, hiện tượng này đôi khi được gọi là feature envy.

Such hybrids make it hard to add new functions but also make it hard to add new data structures. They are the worst of both worlds. Avoid creating them. They are indicative of a muddled design whose authors are unsure of—or worse, ignorant of—whether they need protection from functions or types.

Những thể lai như vậy vừa khiến việc thêm hàm mới trở nên khó, vừa khiến việc thêm cấu trúc dữ liệu mới cũng khó nốt. Chúng gom hết cái dở của cả hai thế giới. Đừng tạo ra chúng. Chúng là dấu hiệu của một thiết kế lộn xộn mà tác giả không chắc — hoặc tệ hơn, không hề biết — mình cần được bảo vệ khỏi hàm hay khỏi kiểu.

Hiding Structure — Giấu cấu trúc

What if `ctxt`, `options`, and `scratchDir` are objects with real behavior? Then, because objects are supposed to hide their internal structure, we should not be able to navigate through them. How, then, would we get the absolute path of the scratch directory?

Vậy nếu `ctxt`, `options` và `scratchDir` là những đối tượng có hành vi thật sự thì sao? Khi đó, vì đối tượng lẽ ra phải giấu cấu trúc nội tại của mình, ta đâu thể rong ruổi xuyên qua chúng. Vậy làm sao lấy được absolute path của scratch directory đây?

```
ctxt.getAbsolutePathOfScratchDirectoryOption();
```

or

hoặc

```
ctx.getScratchDirectoryOption().getAbsolutePath()
```

The first option could lead to an explosion of methods in the

Lựa chọn thứ nhất có thể dẫn tới sự bùng nổ số lượng method trong

`ctxt` object. The second presumes that

đối tượng `ctxt`. Lựa chọn thứ hai thì mặc định rằng

`getScratchDirectoryOption()` returns a data structure, not an object. Neither option feels **good**.

`getScratchDirectoryOption()` trả về một cấu trúc dữ liệu chứ không phải một đối tượng. Chẳng lựa chọn nào cho cảm giác ổn cả.

If ctxt is an object, we should be telling it to do something; we should not be asking it about its internals. So why did we want the absolute path of the scratch directory? What were we going to do with it? Consider this code from (many lines farther down in) the same module:

Nếu ctxt là một đối tượng, ta nên bảo nó làm gì đó, chứ không nên hỏi han nội tạng của nó. Vậy tại sao ta lại muốn có absolute path của scratch directory? Ta định làm gì với nó? Hãy xem đoạn mã sau, nằm mãi tí bên dưới trong cùng module ấy:

```
String outFile = outputDir + "/" + className.replace('.', '/') + "  
FileOutputStream fout = new FileOutputStream(outFile);  
BufferedOutputStream bos = new BufferedOutputStream(fout);
```

The admixture of different levels of detail is a bit troubling. Dots, slashes, file extensions, and File objects should not be so carelessly mixed together, and mixed with the enclosing code. Ignoring that, however, we see that the intent of getting the absolute path of the scratch directory was to create a scratch file of a given name.

Cái mớ trộn lẫn nhiều mức chi tiết khác nhau này hơi đáng lo. Dấu chấm, dấu gạch chéo, phần mở rộng tệp và đối tượng File lẫn ra không nên bị trộn chung một cách cẩu thả như vậy, lại còn trộn lẫn với mã bao quanh. Nhưng tạm bỏ qua chuyện đó, ta thấy rằng ý định lấy absolute path của scratch directory hoá ra là để tạo một scratch file mang một cái tên cho trước.

So, what if we told the ctxt object to do this?

Vậy nếu ta bảo thẳng đối tượng ctxt làm việc đó thì sao?

```
BufferedOutputStream bos =  
    ctxt.createScratchFileStream(className);
```

That seems like a reasonable thing for an object to do! This allows ctxt to hide its internals and prevents the current function from having to violate the Law of Demeter by navigating through objects it shouldn't know about.

Nghe có vẻ là việc hoàn toàn hợp lý cho một đối tượng làm! Cách này cho phép ctxt giấu nội tạng của mình và tránh cho hàm hiện tại khỏi phải vi phạm Luật Demeter bằng việc rong ruổi qua những đối tượng mà lẽ ra nó chẳng nên biết tới.

So, we can often fix trainwrecks and Law of Demeter violations by telling objects to do what we want, as opposed to traversing the network of objects to gather all the resources. This advice is called Tell the Other Guy .

Vậy nên, ta thường có thể chữa được trainwreck và các vi phạm Luật Demeter bằng cách bảo đối tượng làm điều ta muốn, thay vì tự mình rong ruổi qua mạng lưới đối tượng để gom góp tài nguyên. Lời khuyên này được gọi là Tell the Other Guy — hãy bảo anh bạn kia làm.

Long ago Dave Thomas and Andy Hunt wrote a lovely article titled “OO in One Sentence: Keep it **DRY**, Shy, and Tell the Other Guy.” 9 DRY, of course, is Don't Repeat Yourself, and being shy refers to the Law of Demeter. Telling the other guy is a reference to Alan Kay's message-passing idea or command/query separation (CQS). Good advice all around.

Từ lâu lắm rồi, Dave Thomas và Andy Hunt đã viết một bài báo rất hay mang tựa “OO in One Sentence: Keep it DRY, Shy, and Tell the Other Guy.”⁹ DRY tất nhiên là Don't Repeat Yourself, còn “shy” — nhút nhát — thì nhắc tới Luật Demeter. “Tell the other guy” là nói tới ý tưởng truyền thông điệp của Alan Kay, hay tách lệnh khỏi truy vấn (command query separation — CQS). Toàn lời khuyên hay cả.

9 IEEE Software 21, no. 3 (May/June 2004), 101–103.

9 IEEE Software 21, no. 3 (May/June 2004), 101–103.

Data Transfer Objects — Data Transfer Object

The quintessential form of a data structure is a context, class, or struct with public variables and no functions. This is sometimes called a data transfer object, or **DTO**. DTOs are very useful structures, especially when communicating with databases, parsing messages from sockets, and so on. They often become the first in a series of translation stages that convert raw data in a database into objects in the application code.

Dạng tiêu biểu nhất của một cấu trúc dữ liệu là một context, class hay struct với các biến public và không có hàm nào. Thứ này đôi khi được gọi là data transfer object, hay DTO. DTO là những cấu trúc rất hữu ích, đặc biệt khi giao tiếp với cơ sở dữ liệu, phân tích thông điệp từ socket, v.v. Chúng thường trở thành chặng đầu tiên trong một loạt bước chuyển đổi biến dữ liệu thô trong cơ sở dữ liệu thành các đối tượng trong mã ứng dụng.

Early in the Java era, the “bean” form shown below was popular. Beans have private variables manipulated by getters and setters. The quasi-**encapsulation** of beans seems to make some OO purists feel better but usually provides no other benefit. The Address DTO might as well have public fields.

Thuở đầu của kỷ nguyên Java, dạng “bean” trình bày dưới đây rất thịnh hành. Bean có các biến private được thao tác qua getter và setter. Kiểu đóng gói (encapsulation) nửa vời của bean có vẻ làm mấy tín đồ OO thuần túy thấy dễ chịu hơn, nhưng thường thì chẳng đem lại lợi ích nào khác. Cái DTO Address kia để field public cũng chẳng sao.

```
public class Address {
    private String street;
    private String streetExtra;
    private String city;
    private String state;
    private String zip;
    public Address(String street, String streetExtra,
                  String city, String state, String zip) {
        this.street = street;
        this.streetExtra = streetExtra;
        this.city = city;
        this.state = state;
        this.zip = zip;
    }
    public String getStreet() {
        return street;
    }
    public String getStreetExtra() {
        return streetExtra;
    }
    public String getCity() {
        return city;
    }
}
```



```

public String getState() {
    return state;
}
public String getZip() {
    return zip;
}
}

```

The Object/Relational “Impedance Mismatch” — “Impedance Mismatch” giữa đối tượng và cơ sở dữ liệu quan hệ

In the late '80s, as OO grew in popularity, the difference between the OO concept of public behavior and private data and the relational concept of public data and no behavior became clear. Programmers sought a merger of the two ideas; but they were disappointed with the result. For a while, object-oriented databases were tried; but these never really solved the issue. In the end, we fell back on object-relational mappers (ORMs).

Cuối thập niên 1980, khi OO ngày càng phổ biến, khác biệt giữa quan niệm OO — hành vi public, dữ liệu private — và quan niệm quan hệ — dữ liệu public, không hành vi — trở nên rõ mồn một. Các lập trình viên tìm cách hợp nhất hai ý tưởng đó; nhưng họ thất vọng với kết quả. Có một dạo người ta thử cơ sở dữ liệu hướng đối tượng; nhưng thứ đó chưa bao giờ thực sự giải quyết được vấn đề. Rốt cuộc, ta lùi về với các object-relational mapper (ORM).

The hope was that ORMs would allow programmers who were using object-oriented languages to conveniently access the data within a relational database. The ORM would query the database, unpack the data from the resulting table rows, and repackage the data into the data structures used by the objects in their programs.

Hy vọng đặt vào ORM là chúng sẽ cho phép lập trình viên dùng ngôn ngữ hướng đối tượng truy cập dữ liệu trong cơ sở dữ liệu quan hệ một cách tiện lợi. ORM sẽ truy vấn cơ sở dữ liệu, bóc dữ liệu ra khỏi các hàng của bảng kết quả, rồi đóng gói lại dữ liệu đó vào những cấu trúc dữ liệu mà các đối tượng trong chương trình sử dụng.

This works well enough as a way to convert tables into a more convenient form, but it does not resolve the dilemma, because objects are more about functions than data. In the end, there is no way to map a relational database to an object, because databases contain data structures, and data structures and objects are orthogonal. All we can really manage to do is extract data from a database and place it into a form that an object can internalize. ORMs don't create objects, they just load data structures.

Cách này chạy khá ổn nếu xem nó như một cách chuyển bảng sang một dạng tiện dùng hơn, nhưng nó không giải được thế lưỡng nan, bởi đối tượng nói về hàm nhiều hơn là về dữ liệu. Rốt cuộc, chẳng có cách nào ánh xạ một cơ sở dữ liệu quan hệ sang một đối tượng, vì cơ sở dữ liệu chứa cấu trúc dữ liệu, mà cấu trúc dữ liệu với đối tượng thì trực giao với nhau. Tất cả những gì ta thực sự làm được là rút dữ liệu khỏi cơ sở dữ liệu và đặt nó vào một dạng mà đối tượng có thể tiếp nhận. ORM không tạo ra đối tượng, chúng chỉ nạp cấu trúc dữ liệu mà thôi.

Using Objects and Data Structures — Dùng đối tượng và cấu trúc dữ liệu

So why do we have these two opposite concepts? Why do we sometimes want the data structure to be hidden and the behavior exposed and other times want the behavior to be hidden while the data structure is exposed?

Vậy tại sao ta lại có hai khái niệm đối nghịch nhau này? Tại sao có lúc ta muốn giấu cấu trúc dữ liệu và phơi hành vi ra, lúc khác lại muốn giấu hành vi trong khi phơi cấu trúc dữ liệu ra?

It all comes down to which is more likely to grow and change.

Tất cả quy về chuyện bên nào có nhiều khả năng phình ra và thay đổi hơn.

If there are likely to be new behaviors, and if existing behaviors are likely to change, then we want to hide the volatile behaviors while exposing the static data structure. If, on the other hand, it is the data structures that are likely to grow and change, then we want to expose the static behavior and hide the volatile data structures.

Nếu nhiều khả năng sẽ có hành vi mới, và những hành vi sẵn có nhiều khả năng sẽ đổi, thì ta muốn giấu phần hành vi hay biến động ấy đi trong khi phơi ra phần cấu trúc dữ liệu ổn định. Ngược lại, nếu chính cấu trúc dữ liệu mới là thứ nhiều khả năng phình ra và thay đổi, thì ta muốn phơi ra phần hành vi ổn định và giấu đi những cấu trúc dữ liệu hay biến động.

We hide the volatile so that the changes don't affect the static. We build a barrier that protects the static from the volatile.

Ta giấu cái hay biến động để các thay đổi không ảnh hưởng tới cái ổn định. Ta dựng lên một hàng rào che chắn cho cái ổn định khỏi cái hay biến động.

Example 1: The structure of a large enterprise database is relatively static. The data itself may change as new records are added and old records are updated or deleted, but the structure changes very infrequently. However, the behaviors of the programs that use that database change frequently. Business rules are volatile, but business structure is stable in comparison.

Ví dụ 1: Cấu trúc của một cơ sở dữ liệu doanh nghiệp lớn thì tương đối ổn định. Bản thân dữ liệu có thể thay đổi khi bản ghi mới được thêm vào và bản ghi cũ được cập nhật hay xóa đi, nhưng cấu trúc thì rất hiếm khi đổi. Trong khi đó, hành vi của những chương trình dùng cơ sở dữ liệu ấy lại đổi xoành xoạch. Quy tắc nghiệp vụ (business rule) hay biến động, còn cấu trúc nghiệp vụ thì ổn định hơn hẳn khi so sánh.

Example 2: The behavior of an IDE is relatively static. It is an editor of text files. But the structure of the data changes every time a new language is added to its capabilities. Users expect the IDE to behave consistently, whether programming in Ruby, Python, Java, or Clojure; and yet those languages have remarkably different structures. The overall data structure is volatile compared to the behavior.

Ví dụ 2: Hành vi của một IDE thì tương đối ổn định. Nó là một trình soạn thảo tệp văn bản. Nhưng cấu trúc dữ liệu thì thay đổi mỗi lần có thêm một ngôn ngữ mới được đưa vào danh sách năng lực của nó. Người dùng mong IDE hành xử nhất quán, dù họ đang lập trình bằng Ruby, Python, Java hay Clojure; vậy mà những ngôn ngữ đó lại có cấu trúc khác nhau một trời một vực. Cấu trúc dữ liệu tổng thể hay biến động hơn hẳn so với hành vi.

Determining whether data or behavior is the more static is a decision that requires some thought and experience. It is also a decision that may have to be made differently in different parts of a system.

Xác định xem dữ liệu hay hành vi mới là thứ ổn định hơn là một quyết định đòi hỏi phải suy nghĩ và có kinh nghiệm. Đó cũng là quyết định có thể phải đưa ra theo những cách khác nhau ở những phần khác nhau của cùng một hệ thống.

A complex system can have components of both kinds. Some components may have behavior that is static relative to the data structure, and others may have data structures that are static relative to the behavior.

Một hệ thống phức tạp có thể có component thuộc cả hai loại. Một số component có thể có hành vi ổn định so với cấu trúc dữ liệu, số khác lại có cấu trúc dữ liệu ổn định so với hành vi.

Oddly enough, all this boils down to the decision to use

Kỳ lạ thay, tất cả chuyện này rốt cuộc quy về quyết định có nên dùng hay không switch statements.

câu lệnh switch.

Statements — Câu lệnh

Switch statements and their siblings, if / else chains, are considered by some to be bad form and by others to be utterly essential. The reason behind this disagreement is the same as the decision over static behavior or data structures. Consider this simple program.

Câu lệnh switch và người anh em của nó, chuỗi if / else , bị một số người coi là văn phong tồi tệ, còn số khác lại cho là tối cần thiết. Nguyên do của bất đồng này cũng chính là nguyên do đằng sau quyết định nên giữ cố định phần hành vi hay phần cấu trúc dữ liệu. Hãy xem chương trình đơn giản sau.

```
pj
enum ShapeType {CIRCLE, SQUARE}
public class Shape {
    public ShapeType type;
}
—Square.java—
class Square extends Shape {
    public int side;
    public Point topLeft;
}
—Circle.java—
class Circle extends Shape {
    public int radius;
    public Point center;
}
—ShapeManager.java—
public class ShapeManager {
    public static void renderShapes(List<Shape> shapes) {
        for (Shape shape : shapes) {
            switch (shape.type) {
                case CIRCLE:
                    renderCircle((Circle) shape);
                    break;
```

```

        case SQUARE:
            renderSquare((Square) shape);
break
        break;
    }
}
private static void renderSquare(Square square) {
    //...
}
private static void renderCircle(Circle circle) {
    //...
}
}

```

If you are a Java programmer, this might look a little odd to you. If you are a C programmer, this might look quite normal to you.

Nếu bạn là lập trình viên Java, đoạn này trông có lẽ hơi lạ mắt. Còn nếu bạn là lập trình viên C, nó trông khá bình thường.

Now consider the consequence of adding a Triangle .

Giờ hãy xét hệ quả của việc thêm một Triangle .

The first file that needs to change is Shape.java , because

File đầu tiên phải sửa là Shape.java , bởi vì

TRIANGLE has to be added to the enum. Next, of course, the

TRIANGLE phải được thêm vào enum. Kế đến, dĩ nhiên, ta phải viết

Triangle data structure (class) must be written. And then, finally, the Triangle case needs to be added to the **switch statement** in the ShapeManager class.

cấu trúc dữ liệu (class) Triangle . Và rồi, cuối cùng, phải thêm case Triangle vào câu lệnh switch trong class ShapeManager .

This design forces several modules to change, and others to be added, in response to one simple change to the requirements. This violates the Single Responsibility Principle (SRP) and the Open–Closed Principle (OCP). The number of changes also means that this is a rigid design. The more changes that are required, the more rigid the design becomes.

Thiết kế này buộc nhiều module phải đổi, và buộc phải thêm những module khác, chỉ để đáp lại một thay đổi đơn giản trong yêu cầu. Nó vi phạm Nguyên tắc Trách nhiệm Đơn nhất (SRP — Single Responsibility Principle) và Nguyên tắc Đóng–Mở (OCP — Open–Closed Principle). Số lượng thay đổi ấy cũng có nghĩa đây là một thiết kế cứng nhắc. Càng phải sửa nhiều chỗ, thiết kế càng cứng nhắc.

But it's worse than that, because that renderShapes method in

Nhưng còn tệ hơn thế nữa, bởi cái method renderShapes trong

ShapeManager is not the only method of that class. To save space I didn't write them all, but there's dragShapes ,

ShapeManager không phải là method duy nhất của class đó. Để tiết kiệm chỗ, tôi đã không viết ra hết, nhưng còn có dragShapes ,

eraseShapes , rotateShapes , scaleShapes , and a menagerie of other functions that manipulate shapes. And every one of them has that very same switch statement in it. And every one of them is going to need the TRIANGLE case added to it.

eraseShapes , rotateShapes , scaleShapes và cả một bầy hàm khác nữa chuyên thao tác trên hình. Cái nào cũng có đúng cái câu lệnh switch ấy bên trong. Và cái nào cũng sẽ cần được thêm case TRIANGLE vào.

Except that some of those methods don't use switch statements. Some of them use if / else chains. What's more, clever programmers may use logical optimizations to eliminate or modify some of the switch statements and if / else chains. For example, the switch statement in rotateShapes may not contain a CIRCLE case, because a clever programmer realized that circles have rotational symmetry.

Chỉ có điều một số method trong đó không dùng câu lệnh switch. Vài method dùng chuỗi if / else . Hơn nữa, những lập trình viên khôn ngoan có thể đã dùng các tối ưu logic để loại bỏ hoặc biến đổi một số câu lệnh switch và chuỗi if / else. Chẳng hạn, câu lệnh switch trong rotateShapes có thể không hề có case CIRCLE, vì một lập trình viên tinh ý nhận ra rằng hình tròn có tính đối xứng quay.

So, we are going to have to find all those switch / if / else statements, decode all the logical optimizations, and figure out some way to get them all to properly deal with the new

Vậy là ta sẽ phải lùng cho ra tất cả những câu lệnh switch / if / else đó, giải mã mọi tối ưu logic, rồi nghĩ ra cách nào đó bắt tất cả chúng xử lý cho đúng

TRIANGLE case. And, in large systems, we'll almost certainly miss one . . . or more.

case TRIANGLE mới. Và trong các hệ thống lớn, gần như chắc chắn ta sẽ bỏ sót một chỗ... hoặc vài chỗ.

A design that forces many modifications to be made within modules when only one simple requirement is changed or added is both rigid and fragile—it's hard to change and it breaks easily. The more changes that are required, the more fragile the design is.

Một thiết kế buộc phải sửa đổi nhiều chỗ bên trong các module chỉ vì một yêu cầu đơn giản bị đổi hoặc được thêm vào thì vừa cứng nhắc vừa mong manh — nó khó thay đổi và dễ vỡ. Càng phải sửa nhiều, thiết kế càng mong manh.

But it's worse than that. Because, you see, our business has been trying to sell this shape management system for several months now, and without a lot of success. We were all starting to worry about our jobs. Life at a startup rests on a tenuous **thread**. Fortunately, our CEO has just made a critical announcement. He knows what the problem is, and he knows how to solve it.

Nhưng còn tệ hơn thế nữa. Bởi vì, bạn thấy đấy, mấy tháng nay công ty ta vẫn đang cố bán cái hệ thống quản lý hình này mà chẳng mấy thành công. Tất cả bọn ta đã bắt đầu lo cho chỗ làm của mình. Đời sống ở một startup treo trên sợi chỉ mảnh. May thay, CEO của chúng ta vừa đưa ra một thông báo mang tính sống còn. Ông biết vấn đề nằm ở đâu, và ông biết cách giải quyết.

"Our problem," he says, "is that we've been forcing our customers to deal with circles and squares. Customers don't want that. They want a shape management system that allows them to choose whether they use circles or squares. They don't want to pay for circles if they only use squares. And who can blame them?"

“Vấn đề của chúng ta,” ông nói, “là chúng ta đang ép khách hàng phải ôm cả hình tròn lẫn hình vuông. Khách hàng đâu muốn thế. Họ muốn một hệ thống quản lý hình cho phép họ tự chọn dùng hình tròn hay hình vuông. Họ không muốn trả tiền cho hình tròn nếu họ chỉ dùng hình vuông. Mà trách họ sao được?”

“So,” he continues, “We are going to give the ShapeManager facility away for free. Customers can download it from our website. But. . . He pauses here for effect, because this is the insight that will save the company. “. . . We’ll make them pay to download circles. And we’ll make them pay to download squares. And if they want other shapes, we’ll make them pay for those as well. Instead of selling one big system that manages shapes, we are going to sell the shapes themselves—and we’re going to make a blithering fortune!”

“Cho nên,” ông tiếp, “chúng ta sẽ cho không cái ShapeManager. Khách hàng có thể tải nó về từ website của chúng ta. Nhưng...” Ông ngừng lại lấy hiệu ứng, bởi đây chính là tia sáng sẽ cứu cả công ty. “...Chúng ta sẽ bắt họ trả tiền để tải hình tròn. Và bắt họ trả tiền để tải hình vuông. Còn nếu họ muốn những hình khác, chúng ta cũng sẽ bắt họ trả tiền cho những hình đó nữa. Thay vì bán một hệ thống to dùng chuyên quản lý hình, chúng ta sẽ bán chính các hình — và chúng ta sẽ hốt bạc đến ngu người!”

And amid all the cheers, balloons, and popping of campaign corks, you realize that the CEO’s dream is dead on arrival. It is dead because Circle and Square are woven through the entire fabric of this system. All those switch statements, all those if/else statements, all those clever logical optimizations: They all deal specifically with Circle and Square. There’s just no way the **architecture** of this system is going to allow the CEO to give the ShapeManager away for free without including the Circle and Square with it. They are inextricably bound together.

Và giữa tràng hoan hô, bóng bay và tiếng nút champagne bật tung, bạn nhận ra giấc mơ của CEO đã chết ngay từ trong trứng. Nó chết bởi vì Circle và Square được dệt xuyên suốt toàn bộ tấm vải của hệ thống này. Tất cả những câu lệnh switch đó, tất cả những câu lệnh if/else đó, tất cả những tối ưu logic khôn ngoan đó: chúng đều xử lý riêng cho Circle và Square. Kiến trúc của hệ thống này không đời nào cho phép CEO tặng không ShapeManager mà không kèm theo cả Circle lẫn Square. Chúng bị trói chặt vào nhau, không tài nào gỡ ra được.

A design that prevents its parts from being separated and deployed independently from its other parts violates the **Dependency Inversion Principle (DIP)** and is immobile. The fewer parts that are **independently deployable**, the more immobile it is.

Một thiết kế không cho phép các phần của nó được tách ra và triển khai độc lập với những phần khác là vi phạm Nguyên tắc Đảo ngược Phụ thuộc (DIP — Dependency Inversion Principle), và nó bất động. Càng ít phần có thể triển khai độc lập, nó càng bất động.

This design is rigid, fragile, and immobile. It’s trash. It’s a horror scene. It’s going to take the company down, and all of us with it!

Thiết kế này cứng nhắc, mong manh và bất động. Nó là rác. Nó là một hiện trường kinh dị. Nó sẽ kéo công ty xuống đáy, và kéo theo tất cả chúng ta!

The OO Solution — Giải pháp OO

We hire a famous consultant who flies in, makes a lot of noise, craps all over everything, and then flies home. 10 His message is: “Your design is procedural. It should have been object oriented. Here, let me show you.” And what he shows is this:

Chúng ta thuê một chuyên gia tư vấn nổi tiếng; ông ta bay tới, làm ầm ĩ một hồi, ị bừa lên mọi thứ, rồi bay về. 10 Thông điệp của ông ta là: “Thiết kế của các anh là thủ tục. Đáng lẽ nó phải hướng đối tượng. Đây, để tôi chỉ cho.” Và đây là những gì ông ta chỉ:

10 These are sometimes known as seagull consultants.

10 Kiểu người này đôi khi được gọi là seagull consultant — tư vấn viên mòng biển.

```
—Shape.java—
interface Shape {
    void render();
}
—Square.java—
class Square implements Shape {
    public int side;
    public Point topLeft;
    public void render() {
        //...
    }
}
—Circle.java—
class Circle implements Shape {
    public int radius;
    public Point center;
    public void render() {
        public void render() {
            //...
        }
    }
}
—ShapeManager.java—
public class ShapeManager {
    public static void renderShapes(List<Shape> shapes) {
        for (Shape shape : shapes) {
            shape.render();
        }
    }
}
```

Not a switch statement in sight. Now consider what happens when you add the TRIANGLE case.

Chẳng thấy bóng dáng câu lệnh switch nào. Giờ hãy xem chuyện gì xảy ra khi bạn thêm case TRIANGLE.

- Shape.java does not need to change.
- Square.java does not need to change.
- Circle.java does not need to change.
- ShapeManager.java does not need to change.
 - Shape.java không cần đổi.
 - Square.java không cần đổi.
 - Circle.java không cần đổi.
 - ShapeManager.java không cần đổi.

Nothing needs to be changed; except, perhaps, for the main program. But none of these modules need changing. And that means the design is not rigid.

Chẳng có gì cần đổi cả; ngoại trừ, có lẽ, chương trình main. Nhưng không module nào trong số này phải sửa. Và điều đó nghĩa là thiết kế này không cứng nhắc.

There are no switch / if / else statements to find and change either. All the methods, like erase , drag , scale , and rotate , are contained within the Shape classes. This design is not fragile.

Cũng chẳng có câu lệnh switch / if / else nào phải đi lòng rồi sửa. Mọi method, như erase , drag , scale và rotate , đều nằm gọn bên trong các class Shape. Thiết kế này không mong manh.

Best of all, we can deploy Circle and Square separately from

Tuyệt nhất là, ta có thể triển khai Circle và Square tách rời khỏi

ShapeManager . We could put ShapeManager , Circle , and Square into their own jar files and distribute them individually!

ShapeManager . Ta có thể đóng ShapeManager , Circle và Square vào các file jar riêng của chúng rồi phân phối từng cái một!

This design conforms to the OCP and the DIP. It is mobile, flexible, and robust. Hallelujah! We are saved!

Thiết kế này tuân thủ OCP và DIP. Nó cơ động, linh hoạt và bền bỉ. Hallelujah! Chúng ta được cứu rồi!

Not So Fast, Johnson — Đừng vội mừng, Johnson

What does this design protect us from? It protects us from new shapes. Whenever a customer wants a new shape, all we need to do is write a new derivative of the Shape class and put it into its own jar file that can be independently distributed and deployed.

Thiết kế này che chắn cho ta khỏi cái gì? Nó che chắn cho ta khỏi những hình mới. Mỗi khi khách hàng muốn một hình mới, tất cả những gì ta phải làm là viết một class dẫn xuất mới của class Shape rồi bỏ nó vào file jar riêng của nó, để có thể phân phối và triển khai độc lập.

But what if the customers don't ask for a new shape? What if they ask instead for a new method on Shape that renders drop shadows?

Nhưng nếu khách hàng chẳng đòi hình mới thì sao? Nếu thay vào đó họ đòi một method mới trên Shape để vẽ bóng đổ thì sao?

Now our Shape abstraction doesn't help us at all. We're going to have to take that lovely OO design and touch every single one of those classes. Shape , Circle , and Square will all need a

Lúc này cái trừu tượng Shape của ta chẳng giúp được gì hết. Ta sẽ phải lôi cái thiết kế OO đáng yêu kia ra và sờ vào từng class một trong đó. Shape , Circle và Square đều sẽ cần một

renderDropShadow method; and the ShapeManager will have to call it. That Shape abstraction gave us no benefit at all.

method renderDropShadow ; còn ShapeManager thì sẽ phải gọi nó. Cái trừu tượng Shape kia rốt cuộc chẳng đem lại cho ta chút lợi ích nào.

But now look back at that first solution, the one with the

Nhưng giờ hãy ngoái lại nhìn giải pháp đầu tiên, cái giải pháp có switch statement. All we have to do to that one is add the

câu lệnh switch. Với nó, tất cả những gì ta phải làm là thêm các hàm renderDropShadow functions to the ShapeManager . We don't have to touch the Circle or Square at all!

renderDropShadow vào ShapeManager . Ta chẳng phải động tới Circle hay Square một chút nào!

The OO/Procedural Trade-off — Đánh đổi giữa OO và thủ tục

If our design interprets changes to the requirements as new types, then OO is a good choice because it is easy to add new types, like Triangle , without changing anything else. However, if our design interprets changes as new functions, then the procedural switch statement is best because we don't have to change any of the types.

Nếu thiết kế của ta diễn giải những thay đổi trong yêu cầu thành các kiểu mới, thì OO là lựa chọn tốt, vì thêm kiểu mới — như Triangle — mà không phải đổi thứ gì khác là chuyện dễ dàng. Ngược lại, nếu thiết kế của ta diễn giải thay đổi thành các hàm mới, thì câu lệnh switch kiểu thủ tục mới là hay nhất, vì ta chẳng phải sửa kiểu nào cả.

Or, to say this differently, we use OO when we want to add new data types to existing functions, and we use procedural switch statements when we want to add new functions to existing data types.

Hay nói cách khác, ta dùng OO khi muốn thêm kiểu dữ liệu mới vào những hàm sẵn có, và ta dùng câu lệnh switch kiểu thủ tục khi muốn thêm hàm mới vào những kiểu dữ liệu sẵn có.

Again we see this interesting opposition. Hiding data and exposing functions allows the data to change without significant impact. Hiding functions and exposing data allows the functions to be changed without significant impact.

Một lần nữa ta lại thấy thế đối lập thú vị này. Ẩn dữ liệu và phơi bày hàm cho phép dữ liệu thay đổi mà không gây ảnh hưởng đáng kể. Ẩn hàm và phơi bày dữ liệu cho phép các hàm thay đổi mà không gây ảnh hưởng đáng kể.

So which is it? Which would we rather change, functions or data? And the answer to that is that it depends on how your application and your design interpret changes to that application.

Vậy rốt cuộc chọn đằng nào? Ta muốn thay đổi cái gì hơn: hàm hay dữ liệu? Câu trả lời là: còn tùy vào cách ứng dụng của bạn và thiết kế của bạn diễn giải những thay đổi đối với ứng dụng ấy.

In large enterprise models, we often write new applications against the existing data schema. Whereas when we add new capabilities to a given application, we often encapsulate that behavior into new data types.

Trong các mô hình doanh nghiệp lớn, ta thường viết ứng dụng mới dựa trên schema dữ liệu sẵn có. Còn khi thêm năng lực mới cho một ứng dụng nào đó, ta lại thường đóng gói hành vi mới ấy vào những kiểu dữ liệu mới.

Personally, I prefer the latter. Where possible, I create designs that interpret changes as new data types. And that means I prefer an OO design. For the most part, I find that works pretty well.

Cá nhân tôi thích vẻ sau hơn. Ở đâu có thể, tôi tạo ra những thiết kế diễn giải thay đổi thành kiểu dữ liệu mới. Và điều đó nghĩa là tôi thiên về thiết kế OO. Phần lớn thời gian, tôi thấy cách ấy hoạt động khá tốt.

Thus, my general rule for switch statements and their ugly duckling sibling, if / else chains, is that they should be located at the periphery of the system (e.g., the main module); there should be, at most, one for each switchable type; and the cases within them must create the polymorphic objects that the rest of the system uses.

Vì vậy, quy tắc chung của tôi cho câu lệnh switch và người anh em vịt con xấu xí của nó, chuỗi if / else, là: chúng nên nằm ở vùng rìa của hệ thống (ví dụ module main); với mỗi kiểu cần switch thì nhiều nhất chỉ nên có một cái; và các case bên trong chúng phải tạo ra những đối tượng đa hình (polymorphic) mà phần còn lại của hệ thống sử dụng.

However, that preference and that rule do not mean that I won't use a switch statement in those parts of the system where behavior is overwhelmingly more volatile than data. I won't force an OO round peg into a procedural square hole, or vice versa.

Tuy nhiên, sở thích đó và quy tắc đó không có nghĩa là tôi sẽ không dùng câu lệnh switch ở những phần của hệ thống nơi hành vi biến động dữ dội hơn dữ liệu rất nhiều. Tôi sẽ không ép cái chốt tròn OO vào cái lỗ vuông thủ tục, hay ngược lại.

But What About Performance? — Thế còn hiệu năng thì sao?

It is fair to say that today's compilers can take advantage of the local nature of switch statements and specific processor architectures to create executable code that runs somewhat faster than a polymorphic dispatch. Thus, OO designs can incur a slight performance penalty. If you are working on a project where a few nanoseconds here or there are critical, then it may be that you will have to abandon the OCP and the DIP and use switch statements instead of **polymorphism**.

Cũng công bằng mà nói, các compiler ngày nay có thể tận dụng tính cục bộ của câu lệnh switch cùng những kiến trúc bộ xử lý cụ thể để sinh ra mã thực thi chạy nhanh hơn đôi chút so với một lời gọi đa hình (polymorphic dispatch). Vì vậy, thiết kế OO có thể phải chịu một cái giá nho nhỏ về hiệu năng. Nếu bạn đang làm một dự án mà dăm ba nano giây chỗ này chỗ kia cũng mang tính sống còn, thì có lẽ bạn sẽ phải vứt bỏ OCP và DIP để dùng câu lệnh switch thay cho đa hình.

Most programmers have no need to care about stray nanoseconds. But if you do, be careful to concentrate them where they do you the most good. Don't abandon OO design throughout the entire system just because a few time-critical inner loops must use switches.

Phần lớn lập trình viên chẳng cần bận tâm tới dăm ba nano giây lạc lõng. Nhưng nếu bạn có bận tâm, hãy cẩn thận dồn chúng vào nơi mang lại lợi ích lớn nhất. Đừng vứt bỏ thiết kế OO trên toàn hệ thống chỉ vì vài vòng lặp bên trong nhạy cảm với thời gian buộc phải dùng switch.

Conclusion — Kết luận

Objects expose behavior and hide data. This makes it easy to add new kinds of objects without changing existing behaviors. It also makes it hard to add new behaviors to existing objects. Data

structures expose data and have no significant behavior. This makes it easy to add new behaviors to existing data structures but makes it hard to add new data structures to existing functions.

Đối tượng phơi bày hành vi và giấu dữ liệu. Điều đó khiến việc thêm những loại đối tượng mới trở nên dễ dàng mà không phải đổi các hành vi sẵn có. Nó cũng khiến việc thêm hành vi mới cho các đối tượng sẵn có trở nên khó khăn. Cấu trúc dữ liệu thì phơi bày dữ liệu và hầu như không có hành vi gì đáng kể. Điều đó khiến việc thêm hành vi mới cho các cấu trúc dữ liệu sẵn có trở nên dễ dàng, nhưng lại khiến việc thêm cấu trúc dữ liệu mới vào các hàm sẵn có trở nên khó khăn.

In any given system, we will sometimes want the flexibility to add new data types, and so we prefer objects for that part of the system. Other times we will want the flexibility to add new behaviors, and so in that part of the system, we prefer data types and procedures. Good software developers understand these issues without prejudice and choose the approach that is best for the job at hand.

Trong bất kỳ hệ thống nào, đôi khi ta muốn sự linh hoạt để thêm kiểu dữ liệu mới, và ở phần đó của hệ thống ta chuộng đối tượng. Những lúc khác ta lại muốn sự linh hoạt để thêm hành vi mới, và ở phần đó của hệ thống ta chuộng kiểu dữ liệu cùng các thủ tục. Người phát triển phần mềm giỏi hiểu những vấn đề này mà không mang định kiến, và chọn cách tiếp cận phù hợp nhất cho công việc trước mắt.

Chapter 13. Clean Classes — Chương 13. Class sạch

by Jeff Langr

tác giả Jeff Langr

In Chapter 7, “Clean Functions,” you learned how to best organize code statements and expressions in the form of functions or methods. Hopefully you learned these important things about clean functions:

Trong Chương 7, “Hàm sạch”, bạn đã học cách tổ chức các câu lệnh và biểu thức dưới dạng hàm hoặc method sao cho tốt nhất. Hy vọng bạn đã rút ra được những điều quan trọng sau về hàm sạch:

- Functions provide a way to declare, invoke, and reuse pieces of logical **behavior**, or concepts.
- While nothing prevents you from aggregating multiple concepts into a single **function**, you’re much better off if you compose methods that capture small, discrete concepts.
 - Hàm cho ta một cách để khai báo, gọi và tái sử dụng những mẫu hành vi logic, hay nói cách khác là những khái niệm.
 - Tuy chẳng có gì ngăn bạn nhồi nhiều khái niệm vào một hàm duy nhất, bạn sẽ khá hơn rất nhiều nếu soạn ra các method gói gọn từng khái niệm nhỏ và tách bạch.

In this chapter, you’ll hear echoes of the advice from Chapter 7 at the next-higher-up organizational construct: classes or modules.

Trong chương này, bạn sẽ nghe lại tiếng vọng của những lời khuyên ở Chương 7, nhưng ở cấu trúc tổ chức cao hơn một bậc: class hoặc module.

Your programming language allows you to group concepts into a single construct—a **class**, if you’re coding with an **object**-oriented language. If you program in a primarily functional language like JavaScript, Elixir, or F#, you might refer to a **module** instead.

Ngôn ngữ lập trình của bạn cho phép gom các khái niệm vào một cấu trúc duy nhất — là class, nếu bạn đang viết bằng ngôn ngữ hướng đối tượng. Nếu bạn lập trình chủ yếu bằng ngôn ngữ hàm như JavaScript, Elixir hay F#, có lẽ bạn sẽ gọi nó là module.

Classes and Modules versus Files — Class và module so với file

In some object-oriented languages, such as Java, a file can declare only a single, publicly available class. Other languages, such as C++, TypeScript, and Ruby, allow you to declare multiple classes within a single file, or even split a single class between multiple files.

Ở một số ngôn ngữ hướng đối tượng, chẳng hạn Java, mỗi file chỉ được khai báo đúng một class công khai. Những ngôn ngữ khác như C++, TypeScript và Ruby lại cho phép bạn khai báo nhiều class trong cùng một file, thậm chí chẻ một class ra thành nhiều file.

To make matters more confusing, some languages (both OO and functional) allow you to declare namespaces. These can help you avoid name clashes across a larger system. They can also help you organize related concepts.

Rối hơn nữa, một số ngôn ngữ (cả hướng đối tượng lẫn hàm) còn cho phép khai báo namespace. Chúng giúp bạn tránh đụng độ tên trong một hệ thống lớn, và cũng giúp bạn tổ chức các khái niệm có liên quan với nhau.

Most functional languages—for example, Clojure, Elixir, and Haskell—allow you to organize functions into multiple, separate modules within a file. Even for languages that don’t directly support modules/namespaces within the language (JavaScript comes to mind), you can organize multiple “virtual” modules within a single source file.

Phần lớn ngôn ngữ hàm — ví dụ Clojure, Elixir và Haskell — cho phép bạn tổ chức các hàm thành nhiều module riêng biệt trong cùng một file. Ngay cả với những ngôn ngữ không hỗ trợ trực tiếp module/namespace ngay trong ngôn ngữ (JavaScript là cái tên bật ra ngay), bạn vẫn có thể tổ chức nhiều module “ảo” trong một file mã nguồn.

Just because you can doesn’t mean you should. Generally, stick to a single module or class per source file. You’ll create fewer headaches for yourself and your dev team.

Nhưng làm được không có nghĩa là nên làm. Nhìn chung, hãy giữ mỗi file mã nguồn một module hoặc một class thôi. Bạn sẽ đỡ đau đầu, và cả nhóm phát triển của bạn cũng vậy.

Everything is a trade-off in software development, of course, and thus, exceptions always exist. You might find it useful to declare other very closely related classes within the same source file, such as small structs (e.g., records in Java) or **exception** types. Perhaps they are consumed only by other types within the same file, or they always appear in client code that also involves the file’s primary type.

Dĩ nhiên trong phát triển phần mềm mọi thứ đều là đánh đổi, nên bao giờ cũng có ngoại lệ. Bạn có thể thấy hữu ích khi khai báo vài class có quan hệ cực kỳ mật thiết trong cùng một file mã nguồn, chẳng hạn các struct nhỏ (ví dụ record trong Java) hoặc các kiểu exception. Có lẽ chúng chỉ được dùng bởi các kiểu khác trong cùng file đó, hoặc chúng luôn xuất hiện trong mã phía client vốn cũng dùng đến kiểu chính của file.

(And, of course, all this means little to those devious Smalltalk programmers who have the audacity to not organize their code into files at all.)

(Và tất nhiên, tất cả những điều này chẳng có mấy ý nghĩa với đám lập trình viên Smalltalk ranh mãnh, những người gan đến mức không thèm tổ chức mã của họ vào file nào cả.)

But, yeah, for the most part, class $\Leftrightarrow$ file, or module $\Leftrightarrow$ file, or whatever. Consider the file part of things mostly not relevant in our discussions in this chapter. We’re going to focus instead on how to properly group concepts in classes and modules, irrespective of how they are deployed into files.

Nhưng đúng vậy, phần lớn thời gian thì class $\Leftrightarrow$ file, hoặc module $\Leftrightarrow$ file, hay gì đó tương tự. Cứ xem như chuyện file phần lớn là không liên quan trong các thảo luận ở chương này. Thay vào đó, chúng ta sẽ tập trung vào cách gom nhóm các khái niệm vào class và module cho đúng, bất kể chúng được đặt vào file thể nào.

What Should a Class Contain? — Một class nên chứa những gì?

You could conceivably lump together your system’s entire set of methods into a single class—a monolith. You could, similarly perversely, create a separate class for each and every **method**. Both are outrageous ideas, though I’ve done the former for very small systems, and perhaps the second isn’t so outrageous for some Smalltalkers.

Về lý thuyết, bạn có thể dồn toàn bộ các method của hệ thống vào một class duy nhất — một khối monolith. Bạn cũng có thể, theo kiểu quái đản không kém, tạo một class riêng cho từng method một. Cả hai đều là ý tưởng trời ơi, dù tôi từng làm cách thứ nhất cho những hệ thống rất nhỏ, và có lẽ với vài tin đồ Smalltalk thì cách thứ hai cũng chẳng đến nỗi trời ơi.

Classes provide ways to help you keep your sanity as your system grows. Without sensible organization, you will

Class cho bạn những cách để giữ được sự tỉnh táo khi hệ thống lớn dần. Không có cách tổ chức hợp lý, bạn sẽ

- Have a harder time finding the code you’re looking for.
 - Bloat your system with considerable redundancy; doubling it in size is a likely possibility.
 - Create defects around inconsistent behaviors, due to the redundancies.
 - Create defects due to increasing complexity.
 - Spend extensive amounts of time writing automated tests for it.
- Vất vả hơn khi tìm đoạn mã mình cần.
 - Phình hệ thống lên với lượng dư thừa đáng kể; gấp đôi kích thước là chuyện hoàn toàn có thể.
 - Sinh ra lỗi (defect) quanh những hành vi không nhất quán, do chính sự dư thừa đó.
 - Sinh ra lỗi do độ phức tạp ngày càng tăng.
 - Tốn cực nhiều thời gian để viết test tự động cho nó.

All those outcomes increase in likelihood with growth, in turn increasing your costs and frustrations.

Tất cả những hệ quả đó càng lúc càng dễ xảy ra khi hệ thống lớn lên, và đến lượt mình, chúng đẩy chi phí lẫn nỗi bức dọc của bạn lên cao.

We’ll talk about clean class design throughout this chapter. In your head, translate that to “doing all the things that should make our development lives easier.”

Chúng ta sẽ bàn về thiết kế class sạch xuyên suốt chương này. Trong đầu bạn, hãy dịch cụm đó thành “làm mọi thứ khiến đời sống phát triển phần mềm của chúng ta dễ thở hơn”.

“Clean” isn’t a ding at anyone. Our notion of “unclean” is a way of characterizing code that demands more effort from the average developer to understand or maintain.

“Sạch” không phải là lời xỏ xiên ai cả. Khái niệm “không sạch” của chúng ta chỉ là một cách mô tả thứ mã đòi hỏi nhiều công sức hơn từ một lập trình viên trung bình để hiểu hoặc bảo trì.

Characterizing Class Design — Mô tả đặc trưng của thiết kế class

The quality of a design can only truly be assessed in the face of change; change that occurs each time the system must take on new features. Is it getting increasingly costly to introduce new features as the system grows and ages? Is the number of defects increasing?

Chất lượng của một thiết kế chỉ thực sự được đánh giá khi đối diện với thay đổi; thay đổi xảy ra mỗi lần hệ thống phải gánh thêm tính năng mới. Việc thêm tính năng mới có ngày càng đắt đỏ hơn khi hệ thống lớn lên và già đi không? Số lỗi có tăng lên không?

Across seven decades of software development, we've gathered empirical **data** from our experiences with accommodating change. We've gleaned valuable insights from that data, and we've continually extrapolated those learnings into larger and newer contexts.

Qua bảy thập kỷ phát triển phần mềm, chúng ta đã thu thập được dữ liệu thực nghiệm từ chính trải nghiệm ứng phó với thay đổi. Chúng ta đã đãi ra được nhiều hiểu biết quý giá từ dữ liệu ấy, và liên tục ngoại suy những bài học đó sang các bối cảnh lớn hơn, mới hơn.

In the late 1940s, a typical system was implemented like our degenerate example, with the entire set of code essentially stored as a single function to be read and executed top to bottom. Let's call this a monolith.

Cuối thập niên 1940, một hệ thống điển hình được hiện thực (implementation) đúng như ví dụ thoái hoá ở trên: toàn bộ mã về cơ bản được lưu như một hàm duy nhất, đọc và thực thi từ trên xuống dưới. Hãy gọi đó là monolith.

John von Neumann and Herman Goldstine's article, "Planning and Coding of Problems for an Electronic Computing Instrument,"¹ explored the premise of a subroutine:

Bài báo "Planning and Coding of Problems for an Electronic Computing Instrument"¹ của John von Neumann và Herman Goldstine đã khai phá ý niệm về subroutine:

1 H. H. Goldstine and J. von Neumann, "Planning and Coding of Problems for an Electronic Computing Instrument," Institute for Advanced Study, Princeton, New Jersey, 1947, archived at <https://www.ias.edu/sites/default/files/library/pdfs/ecp/planningcodingof0103inst.pdf>.

1 H. H. Goldstine và J. von Neumann, "Planning and Coding of Problems for an Electronic Computing Instrument," Institute for Advanced Study, Princeton, New Jersey, 1947, lưu trữ tại <https://www.ias.edu/sites/default/files/library/pdfs/ecp/planningcodingof0103inst.pdf>.

"We call the coded sequence of a problem a routine, and one which is formed with the purpose of possible substitution into other routines, a subroutine."

"Chúng tôi gọi chuỗi mã hoá của một bài toán là một routine, và routine nào được tạo ra với mục đích có thể thay thế vào các routine khác thì gọi là subroutine."

The premise was that subroutines (particularly for smaller, common operations) would cut down on the amount of code by allowing reuse, and would also help reduce defects.

Ý tưởng đặt ra là subroutine (nhất là với những thao tác nhỏ, dùng đi dùng lại) sẽ cắt giảm lượng mã nhờ cho phép tái sử dụng, đồng thời giúp giảm lỗi.

Pioneers Maurice Wilkes and David Wheeler, inspired by ideas found in the seminal article, implemented the concept of subroutines in the EDSAC computer in 1951.² In short, they believed that code modularization brought about by subroutines would lead to less costly software development. They didn't wait for research (though it did come, slowly, across later decades).

Hai người tiên phong Maurice Wilkes và David Wheeler, được truyền cảm hứng từ những ý tưởng trong bài báo mở đường ấy, đã hiện thực khái niệm subroutine trên máy tính EDSAC vào năm 1951.² Nói ngắn gọn, họ tin rằng việc module hoá mã mà subroutine mang lại sẽ khiến việc phát triển phần mềm bớt tốn kém hơn. Họ không ngồi chờ nghiên cứu chứng minh (dù rốt cuộc nghiên cứu cũng đến, chậm chạp, qua nhiều thập kỷ sau đó).

² [PPEDC].

Since the advent of subroutines, we’ve extrapolated the associated claimed value of modularization to larger contexts. We learned to organize collections of subroutines—functions or methods—into modules or classes. That, in turn, presented new challenges around the composition of such modules and classes, as well as the interrelationships (dependencies) between them. We began to derive various heuristics to guide other developers.

Kể từ khi subroutine ra đời, chúng ta đã ngoại suy giá trị được cho là của việc module hoá sang những bối cảnh lớn hơn. Chúng ta học cách tổ chức các tập hợp subroutine — tức hàm hoặc method — thành module hay class. Điều đó, đến lượt nó, lại đặt ra những thách thức mới quanh việc cấu thành các module và class ấy, cũng như các mối quan hệ qua lại (phụ thuộc (dependency)) giữa chúng. Chúng ta bắt đầu đúc kết đủ loại kinh nghiệm thực hành (heuristic) để dẫn đường cho các lập trình viên khác.

Heuristics and Characteristics — Kinh nghiệm thực hành và đặc điểm

Heuristics help you identify the steps to take, and also the steps to avoid, to arrive at well-designed classes.

Kinh nghiệm thực hành giúp bạn nhận ra những bước nên làm, và cả những bước nên tránh, để đi tới những class được thiết kế tốt.

- Positive heuristics—steps to take. Kent Beck’s four rules of emergent design, 3 for example, tell you to do the following.
 - Kinh nghiệm thực hành thuận (positive heuristic) — những bước nên làm. Chẳng hạn, bốn quy tắc thiết kế nảy sinh (emergent design) của Kent Beck 3 bảo bạn hãy làm những điều sau.

3 See Chapter 18, “**Simple Design**.” See also <https://martinfowler.com/bliki/BeckDesignRules.html>.

3 Xem Chương 18, “Thiết kế đơn giản”. Xem thêm <https://martinfowler.com/bliki/BeckDesignRules.html>.

- Ensure that all code is testable as you go.
- Eliminate logical redundancies.
- Ensure that all programmatic elements are clearly and concisely named.
- Minimize the number of elements.
- Negative heuristics—steps to avoid. Antipatterns describe bad paths to take while developing code. Copy-paste programming, for example, tells us to not habitually create new logic by first duplicating existing code and then changing it. (A more memorable name for this antipattern might be “grab and stab.” Or “snatch and patch.” Or “copy and corrupt.” Or . . . oh, that’s enough for now. Just avoid it.) Characteristics help you assess the current quality of your classes.
- Positive characteristics—desired traits. Bob Martin’s collection of five class design principles, known as SOLID, have withstood around three decades of scrutiny, and can be practically applied to functional code as well.
- Negative characteristics—undesired traits. Martin Fowler’s code smells 4 are a collection of things to avoid in code, each identified with a memorable name. Shotgun surgery, for example, means that your code forces you to make updates to numerous classes in order to effect simple changes.
 - Bảo đảm mọi đoạn mã đều test được ngay khi bạn viết ra.

- Loại bỏ mọi trùng lặp (duplication) về mặt logic.
- Bảo đảm mọi thành phần trong chương trình đều được đặt tên rõ ràng và súc tích.
- Giảm thiểu số lượng thành phần.
- Kinh nghiệm thực hành nghịch (negative heuristic) — những bước nên tránh. Antipattern mô tả những lối đi sai lầm khi phát triển mã. Ví dụ, lập trình kiểu copy-paste nhắc ta đừng biến việc tạo logic mới bằng cách chép lại mã sẵn có rồi sửa đi thành thói quen. (Một cái tên dễ nhớ hơn cho antipattern này có lẽ là “vơ và đâm”. Hay “chộp rồi vá”. Hay “chép rồi phá”. Hay... thôi, thế đủ rồi. Cứ tránh nó ra là được.) Các đặc điểm thì giúp bạn đánh giá chất lượng hiện tại của những class mình đang có.
- Đặc điểm thuận — những nét mong muốn. Bộ năm nguyên tắc thiết kế class của Bob Martin, quen gọi là SOLID, đã trụ vững qua khoảng ba thập kỷ soi xét, và cũng có thể áp dụng rất thực tế cho mã theo phong cách hàm.
- Đặc điểm nghịch — những nét không mong muốn. Bộ code smell (mùi mã) của Martin Fowler 4 là một tập hợp những thứ cần tránh trong mã, mỗi thứ gắn với một cái tên dễ nhớ. Ví dụ, shotgun surgery nghĩa là mã của bạn bắt bạn phải sửa hàng loạt class chỉ để thực hiện một thay đổi đơn giản.

4 [Refactoring] and <https://martinfowler.com/bliki/CodeSmell.html> .

4 [Refactoring] và <https://martinfowler.com/bliki/CodeSmell.html> .

Over time, folks in the software development arena have codified numerous collections of heuristics and characteristics. We'll refer to any of these as a design perspective.

Theo thời gian, giới phát triển phần mềm đã hệ thống hoá vô số tập hợp các kinh nghiệm thực hành và đặc điểm như thế. Chúng ta sẽ gọi chung mỗi tập hợp đó là một góc nhìn thiết kế (design perspective).

Design perspectives can describe design at both the method (micro) level and the class (macro) level (or even higher levels). You might be able to consider some characteristics as heuristics, and vice versa; the categorization isn't terribly important.

Góc nhìn thiết kế có thể mô tả thiết kế ở cả cấp method (vi mô) lẫn cấp class (vĩ mô), hoặc thậm chí ở những cấp cao hơn. Bạn có thể xem một số đặc điểm như là kinh nghiệm thực hành và ngược lại; chuyện phân loại không quan trọng lắm.

For much of this chapter, we'll focus on the characteristics of an ideally sized class, because the idea is that you can glean them from looking at the code.

Phần lớn chương này, chúng ta sẽ tập trung vào các đặc điểm của một class có kích thước lý tưởng, bởi ý tưởng là bạn có thể nhận ra chúng chỉ bằng cách nhìn vào mã.

Such an ideal class is typically small. Its name concisely summarizes the small numbers of behaviors gathered within. And it is defined cohesively: Its methods are closely related and focused, and it performs a single, well-defined task (or set of related tasks) effectively. As a result, the class exhibits the characteristic of adhering to the Single Responsibility Principle (SRP): It has one reason to change.

Một class lý tưởng như vậy thường là nhỏ. Tên của nó tóm tắt súc tích một số ít hành vi được gom vào bên trong. Và nó được định nghĩa một cách gắn kết: các method của nó liên quan chặt chẽ với nhau và có trọng tâm, và nó thực hiện hiệu quả một nhiệm vụ duy nhất, được định nghĩa rõ ràng (hoặc một nhóm nhiệm vụ liên quan). Nhờ vậy, class thể hiện đặc điểm tuân thủ SRP — Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle): nó chỉ có một lý do để thay đổi.

Design perspectives most certainly overlap. Notably, outcomes for the more overarching perspectives—SOLID, Kent Beck’s simple design, and Martin Fowler’s code smells, for example— are comparable. They all foster small, cohesive modules and functions.

Các góc nhìn thiết kế chắc chắn có phần chồng lấn nhau. Đáng chú ý là kết quả của những góc nhìn bao quát hơn — SOLID, thiết kế đơn giản (simple design) của Kent Beck, và code smell của Martin Fowler chẳng hạn — khá tương đồng. Tất cả đều nuôi dưỡng những module và hàm nhỏ, gắn kết.

In nearly all of these perspectives, there’s an acknowledgment, whether stated outright or implied, of the need for balance: Virtually no principle is absolute. Kent Beck’s four rules of simple design, for example, drive us toward small, single responsibility classes. The resulting classes are easier to **test** (rule 1), they can help us eliminate redundancies more easily (rule 2), and they promote clear, intention-revealing names (rule 3). But rule 4—minimize the number of elements, for example, modules and functions— suggests that we can go too far and create tiny classes that add no advantages to the system.

Ở gần như mọi góc nhìn ấy đều có một sự thừa nhận, dù nói thẳng ra hay chỉ ngầm hiểu, về nhu cầu cân bằng: hầu như chẳng nguyên tắc nào là tuyệt đối. Ví dụ, bốn quy tắc thiết kế đơn giản của Kent Beck đẩy chúng ta về phía những class nhỏ, một trách nhiệm. Những class kết quả sẽ dễ test hơn (quy tắc 1), giúp ta loại bỏ trùng lặp dễ hơn (quy tắc 2), và khuyến khích những cái tên rõ ràng, bộc lộ ý định (intention-revealing name) (quy tắc 3). Nhưng quy tắc 4 — giảm thiểu số lượng thành phần, chẳng hạn module và hàm — lại gợi ý rằng ta có thể đi quá đà và tạo ra những class tí hon chẳng đem lại lợi ích gì cho hệ thống.

Future Bob: Indeed, this is one of John Ousterhout’s fears regarding the “One Thing” rule.

Bob của tương lai: Quả thật, đây chính là một trong những nỗi lo của John Ousterhout về quy tắc “Một Việc”.

Most of us needn’t worry: The typical system has moved too far in the wrong direction, containing large, muddled, multipurpose modules and functions. Still, keep in mind that there are trade-offs for every choice in software. Choose the design perspective(s) you like, and focus on the outcome of creating code that is easy to maintain.

Phần lớn chúng ta chẳng cần bận tâm chuyện đó: hệ thống điển hình đã trượt quá xa về phía sai, chứa đầy những module và hàm to đùng, rối rắm, ôm đồm đủ thứ mục đích. Dù vậy, hãy nhớ rằng trong phần mềm, mọi lựa chọn đều có cái giá phải đánh đổi. Hãy chọn (những) góc nhìn thiết kế bạn thích, và tập trung vào kết quả cuối cùng là tạo ra thứ mã dễ bảo trì.

When Is a Class Too Large? — Khi nào thì một class là quá lớn?

Let’s take a look at the HoldingService class, used to manage the holdings (materials, such as books or physical audio recordings) of a library system and its patrons. Here are the public behaviors defined on HoldingService :

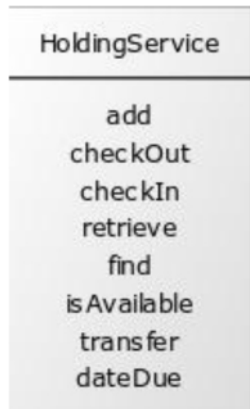
Hãy cùng nhìn vào class HoldingService , dùng để quản lý các holding (tài liệu, chẳng hạn sách hay bản ghi âm vật lý) của một hệ thống thư viện và các độc giả của nó. Đây là những hành vi công khai được định nghĩa trên HoldingService :

```
boolean isAvailable(String barCode)
Holding find(String barCode)
```

```
String add(String sourceId, String branchId)
void transfer(String barcode, String branchScanCode)
Date dateDue(String barCode)
void checkOut(String patronId, String barCode, Date date)
int checkIn(String barCode, Date date, String branchScanCode)
```

The following figure depicts that HoldingService **interface** (we'll be looking at more pictures later), using Unified Modeling Language (UML) diagramming guidelines:

Hình dưới đây mô tả interface HoldingService đó (lát nữa chúng ta sẽ xem thêm nhiều hình khác), dùng quy ước vẽ sơ đồ Unified Modeling Language (UML):



The method parameters aren't usually interesting when we start looking at class-level design. We'll omit them until they become useful.

Tham số của method thường chẳng có gì thú vị khi ta bắt đầu xem xét thiết kế ở cấp class. Chúng ta sẽ bỏ qua chúng cho tới khi chúng trở nên hữu ích.

Our picture shows that the HoldingService allows holdings to be added to the system, borrowed ("checked out") by a patron, returned ("checked in") to a library location (branch), and transferred from one branch to another. The HoldingService also supports a few queries, including the ability to retrieve holding details, answer whether it's available for a patron to check out, and determine the date when the material is due to be returned.

Hình vẽ cho thấy HoldingService cho phép thêm holding vào hệ thống, cho độc giả mượn ("check out"), trả về ("check in") một chi nhánh thư viện (branch), và chuyển từ chi nhánh này sang chi nhánh khác. HoldingService cũng hỗ trợ vài truy vấn, gồm khả năng lấy chi tiết của holding, trả lời xem một độc giả có mượn được nó không, và xác định ngày phải trả tài liệu.

The picture suggests that the core (single?) responsibility of the HoldingService is to manage the disposition of holdings. Each of the behaviors depicted directly supports that interest.

Hình vẽ gợi ý rằng trách nhiệm cốt lõi (duy nhất?) của HoldingService là quản lý việc luân chuyển các holding. Mỗi hành vi được vẽ ra đều trực tiếp phục vụ mối quan tâm đó.

The SRP tells us to be a bit pickier, however: An SRP-compliant class should have only one reason to change. It's the opposite of a monolith, in which every single change requires updating the sole class in the system.

Tuy nhiên, SRP bảo chúng ta phải khó tính hơn một chút: một class tuân thủ SRP chỉ nên có đúng một lý do để thay đổi. Đó là mặt đối lập của monolith, nơi mọi thay đổi dù nhỏ đến đâu cũng đòi phải sửa cái class duy nhất của hệ thống.

Why might a single-responsibility class be desirable?

Vì sao một class chỉ có một trách nhiệm lại đáng mong muốn?

- Its name can concisely reflect the behaviors contained within, making those behaviors easier to locate.
 - It creates real possibilities for reuse.
 - Its likelihood of adhering to the Open–Closed Principle (OCP; see Chapter 19), and thus being closed to future changes, increases.
 - It's easier to test.
 - It makes your system more flexible.
- Tên của nó có thể phản ánh súc tích những hành vi chứa bên trong, khiến những hành vi đó dễ tìm hơn.
 - Nó tạo ra khả năng tái sử dụng thực sự.
 - Khả năng nó tuân thủ OCP — Nguyên tắc Đóng–Mở (Open–Closed Principle; xem Chương 19), và nhờ đó đóng lại trước những thay đổi tương lai, sẽ tăng lên.
 - Nó dễ test hơn.
 - Nó làm hệ thống của bạn linh hoạt hơn.

Each of the behaviors in `HoldingService` implements a policy—a set of rules established by the library system and implemented in the code. The `dateDue` policy, for example, specifies that books are due 21 days after they are checked out and that DVDs are due seven days after checkout.

Mỗi hành vi trong `HoldingService` đều hiện thực một chính sách (policy) — một tập quy tắc do hệ thống thư viện đặt ra và được cài đặt trong mã. Ví dụ, chính sách `dateDue` quy định sách phải trả sau 21 ngày kể từ lúc mượn, còn DVD thì phải trả sau bảy ngày.

The `checkIn` policy is more involved: When a book is checked in, it's marked as available for other patrons to check out. The patron returning the book is assessed a fine if material is late, that is, when it is returned after its due date. The fine amount varies depending on the material type and other possible factors.

Chính sách `checkIn` thì rắc rối hơn: khi một cuốn sách được trả, nó được đánh dấu là sẵn sàng cho độc giả khác mượn. Độc giả trả sách sẽ bị phạt tiền nếu tài liệu bị trễ, tức là khi nó được trả sau ngày hạn. Mức phạt thay đổi tùy loại tài liệu và có thể tùy vài yếu tố khác nữa.

Policies do change, and it's possible that the policy for check-outs might need to change exclusively from the policy for check-ins. Such independent variance suggests implementing each of the policies in its own class— `CheckOutService` , `CheckInService` , and so on—at least if you want to be SRP compliant. At that point, the only job of the `HoldingService` would be as a consolidated interface for holdings interests that delegated to other objects to do the work.

Chính sách thì có thay đổi, và hoàn toàn có khả năng chính sách cho việc check-out phải thay đổi độc lập với chính sách cho việc check-in. Kiểu biến thiên độc lập như vậy gợi ý rằng nên hiện thực mỗi chính sách trong một class riêng — `CheckOutService` , `CheckInService` , v.v. — ít nhất là nếu bạn muốn tuân thủ SRP. Đến lúc đó, việc duy nhất của `HoldingService` sẽ là làm một interface hợp nhất cho mọi mối quan tâm liên quan tới holding, còn phần việc thực sự thì uỷ thác cho các đối tượng khác.

Future Jeff: Creating new classes is surprisingly easy in an IDE, yet we often resist as developers. It takes only moments and has few downsides, but we often pair with developers who do everything they can to avoid clicking a New Class menu item. Just do it.

Jeff của tương lai: Tạo class mới trong IDE dễ đến bất ngờ, vậy mà cánh lập trình viên

chúng ta lại thường ngần ngại. Chuyện đó chỉ mất vài giây và chẳng mấy bất lợi, nhưng chúng tôi vẫn thường xuyên pair với những lập trình viên làm đủ mọi cách để khỏi phải bấm vào mục menu New Class. Cứ bấm đi.

We might not want a “ravioli” system where every class contains only one method, but there’s nothing wrong with tiny classes containing three or two methods, or even one.

Có thể chúng ta không muốn một hệ thống kiểu “ravioli” nơi mỗi class chỉ chứa đúng một method, nhưng chẳng có gì sai với những class tí hon chứa ba hay hai method, thậm chí một.

While the SRP tells us to be picky, it also doesn’t tell us to speculate about the nexus of change. Your systems probably rampantly violate the SRP. Rather than go find code to change, wait for the next demand for change and ensure that you take that opportunity to shape your system to be more compliant.

SRP bảo ta phải khó tính, nhưng nó cũng không bảo ta phải ngồi đoán mò xem thay đổi sẽ ập tới ở đâu. Hệ thống của bạn có lẽ đang vi phạm SRP tràn lan. Thay vì đi lùng mã để sửa, hãy chờ yêu cầu thay đổi kế tiếp và bảo đảm rằng bạn tận dụng cơ hội đó để nắn hệ thống của mình về phía tuân thủ hơn.

Policies in Code — Chính sách trong mã

Let’s assume that the core library policies for check-in and check-out are stable. It’s easy to think of numerous reasons for

Giả sử các chính sách cốt lõi của thư viện về check-in và check-out là ổn định. Vẫn rất dễ nghĩ ra vô số lý do khiến

HoldingService to potentially change. An example: The library wants to allow patrons to rent DVDs for a longer period—14 days instead of seven—because they’re an older technology and are in low demand nowadays.

HoldingService có thể phải thay đổi. Một ví dụ: thư viện muốn cho độc giả thuê DVD lâu hơn — 14 ngày thay vì bảy — vì đó là công nghệ cũ và ngày nay ít người mượn.

Will HoldingService be impacted by that change? While looking at UML models can be very helpful, we can’t spot all change reasons by looking at UML alone.

Liệu HoldingService có bị ảnh hưởng bởi thay đổi đó không? Nhìn vào mô hình UML có thể rất hữu ích, nhưng ta không thể phát hiện mọi lý do thay đổi nếu chỉ nhìn UML.

The interface of a class only tells you about the behaviors it directly publicizes. To understand fully the extent to which a class violates the SRP, you have to open up the source file and peruse the code. It doesn’t take long to spot more reasons why a class might need to change.

Interface của một class chỉ cho bạn biết những hành vi mà nó công khai trực tiếp. Muốn hiểu trọn vẹn một class vi phạm SRP tới mức nào, bạn phải mở file mã nguồn ra và đọc kỹ. Chẳng mất bao lâu để phát hiện thêm nhiều lý do khác khiến class ấy có thể phải thay đổi.

Here’s the `dateDue` method:

Đây là method `dateDue` :

```
public Date dateDue(String barCode) {  
    var holding = find(barCode);  
    if (holding == null)
```



```

        throw new HoldingNotFoundException();
    return holding.dateDue();
}

```

The service delegates the responsibility for calculating a due date to the Holding class. As such, the DVD rental period change won't impact the HoldingService class, but it will impact the Holding class (or maybe another class it depends on).

Class dịch vụ này uỷ thác trách nhiệm tính ngày hạn trả cho class Holding . Nhờ vậy, thay đổi về thời hạn thuê DVD sẽ không đụng tới class HoldingService , nhưng nó sẽ đụng tới class Holding (hoặc có thể là một class khác mà Holding phụ thuộc vào).

“Managing dates due” is a responsibility that could disappear, because some libraries have actually eliminated fines. If and when that occurs, we'll look to shield HoldingService from similar, subsequent changes.

“Quản lý ngày hạn trả” là một trách nhiệm có thể biến mất, bởi vài thư viện thực sự đã bỏ hẳn tiền phạt. Nếu và khi điều đó xảy ra, chúng ta sẽ tìm cách che chắn HoldingService khỏi những thay đổi tương tự về sau.

Where Reasons to Change Hide — Lý do thay đổi trốn ở đâu

Large classes tend to contain and obscure numerous responsibilities. Often, change reasons are buried within the body of a long method. Let's look at an example.

Class lớn thường chứa và che giấu vô số trách nhiệm. Lý do thay đổi hay bị chôn trong thân của một method dài. Hãy xem một ví dụ.

The HoldingService method checkIn does a pretty **good** job of declaring the policy for returning materials:

Method checkIn của HoldingService làm khá tốt việc phát biểu chính sách trả tài liệu:

```

public int checkIn(String barCode, Date date, String branchScanCode)
    var branch = new BranchService().find(branchScanCode);
    var holding = find(barCode);
    if (holding == null)
        throw new HoldingNotFoundException();

    holding.checkIn(date, branch);
    var foundPatron = findPatronWith(holding);
    removeBookFromPatron(foundPatron, holding);
    if (holding.dateLastCheckedIn().after(holding.dateDue())) { //late
        foundPatron.addFine(calculateLateFine(holding));
        return holding.daysLate();
    }
    return 0;
}

```

While rich in policy, the checkIn method still begs for a bit of cleanup. One line of glaring **implementation** detail stands out as a potential problem—the if statement that determines whether the check-in is late:

Dù đậm đặc chính sách, method checkIn vẫn còn kêu gào đòi được dọn dẹp một chút. Có một dòng chi tiết hiện thực lộ liễu nổi lên như một vấn đề tiềm tàng — câu lệnh if xác định

xem lần trả này có trễ hay không:

```
if (holding.dateLastCheckedIn().after(holding.dateDue())) { //late?
```

That conditional represents a policy decision: A holding is late if it's returned after the date due. However, it's possible that the policy will become considerably more complex. Librarians might imagine no end of new policy changes: VIPs (very important patrons) might be exempt from fines, the library might offer grace periods, and so on. As a result, that seemingly innocuous single line of code represents another reason for HoldingService to change.

Điều kiện đó thể hiện một quyết định về chính sách: một holding là trễ nếu nó được trả sau ngày hạn. Tuy nhiên, hoàn toàn có khả năng chính sách này sẽ trở nên phức tạp hơn nhiều. Các thủ thư có thể nghĩ ra hàng hà sa số thay đổi chính sách mới: VIP (very important patron — độc giả rất quan trọng) có thể được miễn phạt, thư viện có thể cho thời gian ân hạn, v.v. Kết quả là cái dòng mã tưởng như vô hại ấy lại là thêm một lý do nữa để HoldingService phải thay đổi.

Because it exposes implementation detail, the conditional also slows comprehension time for readers. The detail demands closer inspection: We must carefully read it, then mentally assemble it into a singular concept.

Vì phơi bày chi tiết hiện thực, câu điều kiện đó cũng làm chậm thời gian tiếp thu của người đọc. Chi tiết ấy đòi phải soi kỹ: ta phải đọc cẩn thận, rồi tự lắp ghép trong đầu thành một khái niệm duy nhất.

A **comment** might help, but it's not the right solution, and can be a lie anyway. It's best if we can glean intent from scanning the code itself, rather than having to look elsewhere. Comments are the distracting footnotes of code.

Một comment có thể giúp được, nhưng đó không phải giải pháp đúng, mà dù sao thì nó cũng có thể nói dối. Tốt nhất là ta rút ra được ý định ngay khi lướt qua chính đoạn mã, thay vì phải nhìn đi chỗ khác. Comment là những cái chú thích chân trang gây phân tâm của mã.

Fixing the Problem — Sửa vấn đề

We want to move the specifics of “is it late” out of the

Ta muốn dời phần chi tiết cụ thể của chuyện “có trễ hay không” ra khỏi class

HoldingService class. Where to? We could move it to a new class such as LateHoldingService , but the obvious short-term candidate appears to be the Holding class.

HoldingService . Dời đi đâu? Ta có thể dời nó sang một class mới, chẳng hạn LateHoldingService , nhưng ứng viên hiển nhiên trong ngắn hạn có vẻ là class Holding .

Step 1, however, is abstracting the concept of “is it late” by creating a new method that contains just the conditional. Yep —this notion of extracting methods should be familiar from Chapter 10 , “One Thing.” Extract till you drop!

Tuy nhiên, bước 1 là trừu tượng hoá (abstraction) khái niệm “có trễ không” bằng cách tạo một method mới chỉ chứa câu điều kiện ấy. Đúng vậy — ý niệm extract method này hẳn đã quen thuộc với bạn từ Chương 10 , “ Một Việc ”. Cứ extract cho tới khi gục!

```
public int checkIn(String barCode, Date date, String branchScanCode)
// ...
```

```

    if (isLate(holding)) {
        foundPatron.addFine(calculateLateFine(holding));
        return holding.daysLate();
    }
    return 0;
}
private boolean isLate(Holding holding) {
    return holding.dateLastCheckedIn().after(holding.dateDue());
}

```

The silly line-level comment `//late?` has disappeared. The new query method's name, `isLate`, precisely imparts the same information. The implementation detail is encapsulated within.

Cái comment ngớ ngẩn ở mức dòng `//late?` đã biến mất. Tên của method truy vấn mới, `isLate`, truyền đạt chính xác cùng thông tin đó. Chi tiết hiện thực được đóng gói (encapsulation) bên trong.

Going forward, if you feel you need a comment to guide readers through a line or five of related code, consider extracting it into its own method. The “guiding” comment often provides the basis for the new method's name, as it does here. A clear, concise method name obviates the need for a comment.

Từ giờ trở đi, nếu bạn thấy mình cần một comment để dẫn dắt người đọc qua một hay năm dòng mã liên quan nhau, hãy cân nhắc extract chúng thành một method riêng. Cái comment “dẫn đường” đó thường chính là cơ sở cho tên của method mới, đúng như trường hợp ở đây. Một cái tên method rõ ràng, súc tích sẽ làm cho comment trở nên thừa thãi.

Predicates are particularly ripe for extraction. In fact, when faced with the challenge of trying to understand a long, daunting method, one of the best first steps you can take is to ensure that you understand the various paths through it (sometimes so that you can know which smaller piece you can focus on). Often the conditionals that control these pathways are complex, depending on combinations of method calls and variables— `if (x(a) && y || !z(c, d))`, for example. Those details don't make it clear why we're heading into a certain piece of code. An extracted predicate's method name can impart immediate understanding.

Các biểu thức predicate đặc biệt chín muồi để đem đi extract. Thật vậy, khi phải đối mặt với thử thách hiểu một method dài dằng dặc đáng sợ, một trong những bước đầu tiên tốt nhất bạn có thể làm là bảo đảm mình nắm được các nhánh đi khác nhau xuyên qua nó (đôi khi là để biết mình có thể tập trung vào mẫu nhỏ nào). Những câu điều kiện điều khiển các nhánh đó thường phức tạp, phụ thuộc vào tổ hợp của các lời gọi method và biến — ví dụ `if (x(a) && y || !z(c, d))`. Đám chi tiết ấy chẳng làm rõ được vì sao ta lại rẽ vào một đoạn mã nào đó. Tên của một method predicate đã được extract có thể mang lại sự thấu hiểu tức thì.

Once isolated within `HoldingService`, the `isLate` method reveals a glaring **code smell** known as feature envy 5: The method, apparently envious of the `Holding` class, asks it multiple questions (“What's the date checked in? What's the date due?”) in order to compute a result. The `isLate` method also shows disinterest in the `HoldingService` class on which it's defined.

Một khi đã được tách riêng bên trong `HoldingService`, method `isLate` lại để lộ một code smell chói mắt mang tên feature envy 5: method này, có vẻ như đang ghen tị với class `Holding`, hỏi nó hết câu này đến câu khác (“Ngày trả là gì? Ngày hạn là gì?”) chỉ để tính ra một kết quả. Method `isLate` cũng tỏ ra thờ ơ với chính class `HoldingService` nơi nó được định nghĩa.

5 [Refactoring].

5 [Refactoring].

We can soothe the method's envy by moving it to the Holding class, where it can talk directly to its new peers:

Ta có thể xoa dịu cơn ghen của method này bằng cách dời nó sang class Holding , nơi nó có thể trò chuyện trực tiếp với những người bạn mới:

```
public class HoldingService {
// ...
    public int checkIn(
        String barCode, Date date, String branchScanCode) {
        // ...

        if (holding.isLate()) {
            foundPatron.addFine(calculateLateFine(holding));
            return holding.daysLate();
        }
        return 0;
    }
}

public class Holding {
// ...
    public boolean isLate() {
        return dateLastCheckedIn().after(dateDue());
    }
}
```

The calling code (in the checkIn method) now concisely states a piece of the overall policy:

Đoạn mã gọi (trong method checkIn) giờ phát biểu súc tích một mẫu của chính sách tổng thể:

```
if (holding.isLate())
```

The conditional is now immediately digestible. It doesn't require us to stop and pause. Instead, it helps us quickly understand the method's control flow, which, in turn, helps us know where we need to look next.

Câu điều kiện giờ tiêu hoá được ngay lập tức. Nó không bắt ta phải dừng lại và ngẫm nghĩ. Ngược lại, nó giúp ta nhanh chóng hiểu luồng điều khiển của method, và điều đó lại giúp ta biết cần nhìn tiếp vào đâu.

After moving the checkIn method to Holding , we look again at the method in its new context. Does the Holding class now contain too many reasons to change? Should we move the method again, perhaps this time to a brand-new class? What can be simplified within the method itself?

Sau khi dời method checkIn sang Holding , ta lại nhìn method ấy trong bối cảnh mới của nó. Liệu class Holding bây giờ có chứa quá nhiều lý do để thay đổi không? Ta có nên dời method đó lần nữa, lần này có lẽ sang một class hoàn toàn mới? Bên trong bản thân method có thể đơn giản hoá được gì?

With every change, we continue to ask these kinds of questions as part of **continuous design**.

Với mỗi thay đổi, chúng ta tiếp tục đặt ra những câu hỏi kiểu này, như một phần của thiết kế liên tục (continuous design).

It might seem like we just passed the buck by moving a problem from `HoldingService` into `Holding`. But toward our primary interest in SRP-compliant classes, we've removed one reason to change from `HoldingService`. We've also replaced unnecessary detail with an **abstraction**, which, in turn, helps us more easily consider the remainder of `HoldingService`.

Có vẻ như ta vừa đá quả bóng trách nhiệm đi chỗ khác, chuyển một vấn đề từ `HoldingService` sang `Holding`. Nhưng xét theo mối quan tâm chính của chúng ta là những class tuân thủ SRP, ta đã bỏ được một lý do thay đổi khỏi `HoldingService`. Ta cũng đã thay chi tiết không cần thiết bằng một trừu tượng hoá, và điều đó, đến lượt nó, giúp ta xem xét phần còn lại của `HoldingService` dễ dàng hơn.

An Overly Open Implementation — Một hiện thực mở toang quá mức

Opportunities to extract and move are rampant in a typical codebase. Extracting implementation detail from a method often makes it obvious that the code belongs elsewhere. Do this enough, and eventually most code makes it to the right place, and each such place is reasonably small, cohesive, and easy to work with. Another line in the `checkIn` method similarly suggests feature envy:

Cơ hội để extract rồi dời đi nhan nhản trong một codebase điển hình. Việc trích chi tiết hiện thực ra khỏi một method thường làm lộ rõ rằng đoạn mã ấy thuộc về nơi khác. Làm chuyện này đủ nhiều, rốt cuộc phần lớn mã sẽ về đúng chỗ của nó, và mỗi chỗ như vậy đều nhỏ vừa phải, gắn kết và dễ làm việc cùng. Một dòng khác trong method `checkIn` cũng bốc mùi feature envy y hệt:

```
foundPatron.addFine(calculateLateFine(holding));
```

We're calling a local method, `calculateFine`, that interacts with code defined in `Holding`, again asking it questions in order to calculate a fine. We move the method into `Holding` (with only two keystrokes in IDEA! 6):

Ta đang gọi một method cục bộ, `calculateFine`, và nó lại tương tác với mã được định nghĩa trong `Holding`, lại hỏi han `Holding` để tính ra khoản phạt. Ta chuyển method này vào `Holding` (chỉ với hai lần gõ phím trong IDEA! 6):

6 The IDE from IntelliJ.

6 IDE của IntelliJ.

```
foundPatron.addFine(holding.calculateLateFine());
```

Here's what `calculateLateFine` looks like in its new home:

Và đây là dáng vẻ của `calculateLateFine` tại ngôi nhà mới:

```
public int calculateLateFine() {
    var daysLate = daysLate();
    var fineBasis = getMaterial().materialType().dailyFine();
    var fine = 0;
    switch (getMaterial().materialType()) {
        case BOOK, NEW_RELEASE_DVD:
            fine = fineBasis * daysLate;

            break;
        case AUDIO_CASSETTE, VINYL_RECORDING, MICRO_FICHE,
```

```

        AUDIO_CD, SOFTWARE_CD, DVD, BLU_RAY, VIDEO_CASSETTE:
        fine = Math.min(1000, 100 + fineBasis * daysLate);
        break;
    }
    return fine;
}

```

Ugh, a **switch statement**, and an old-school one at that—the kind where control flow can accidentally drip from one case to the next if you’re careless.

Ồi, một câu lệnh switch, mà lại còn là kiểu switch cổ lỗ sĩ — kiểu mà luồng điều khiển có thể vô tình rỉ từ case này sang case kế tiếp nếu bạn bất cẩn.

Even after being moved to Holding, the method still doesn’t look at home. While calculateLateFine does interact directly with the Holding (asking it for daysLate), it predominantly interacts with the Material object returned by getMaterial.

Ngay cả sau khi đã dọn về Holding, method này vẫn chưa thấy thoải mái như ở nhà. Đúng là calculateLateFine có tương tác trực tiếp với Holding (hỏi nó về daysLate), nhưng phần lớn thời gian nó lại tương tác với đối tượng Material do getMaterial trả về.

The calculateLateFine method is the sort that will likely suffer many changes over time. It has at least three reasons to change: the addition of new material types (e-books, puzzles, STEM kits, etc.), new schemes to encourage patrons to return materials in high demand, and new rates to cover material price increases.

calculateLateFine thuộc loại method nhiều khả năng sẽ phải chịu vô số thay đổi theo thời gian. Nó có ít nhất ba lý do để thay đổi: bổ sung loại tài liệu mới (e-book, trò chơi ghép hình, bộ kit STEM...), các cơ chế mới nhằm khuyến khích bạn đọc trả lại những tài liệu đang được nhiều người mượn, và các mức phí mới để bù cho việc giá tài liệu tăng.

Such a method violates another SOLID principle—we want to minimize “opening up” existing classes to make changes, and instead find ways to enhance a system by adding new, single-purpose classes. This is the OCP.

Một method như thế vi phạm một nguyên tắc SOLID khác — ta muốn hạn chế tối đa việc “mở toang” các class sẵn có để sửa đổi, và thay vào đó tìm cách nâng cấp hệ thống bằng cách thêm những class mới, mỗi class chỉ làm một việc. Đó chính là OCP — Nguyên tắc Đóng-Mở.

Open classes introduce risk and cost. Holding is open. Changes to any of the three responsibilities in calculateLateFine carry the risk of breaking existing behaviors in Holding. With a drippy

Class mở toang thì kéo theo rủi ro và chi phí. Holding đang mở toang. Mọi thay đổi với bất kỳ trách nhiệm nào trong ba trách nhiệm của calculateLateFine đều mang theo rủi ro làm hỏng những hành vi sẵn có trong Holding. Mà nếu có thêm một câu lệnh

switch statement in the mix, anything can happen.

switch rỉ luồng điều khiển góp mặt nữa thì chuyện gì cũng có thể xảy ra.

Should We Do Anything Now? — Ta có nên làm gì ngay bây giờ không?

Typical systems being what they are, we already missed most of the opportunities to take advantage of the OCP. Our codebase is unashamed about its openness. Virtually no classes are closed. Rampant change greets everyone who must touch the code. If change is a given, the best approach is to wait for it. Speculative cleanup creates unnecessary risk from improvements no one yet needs. Let's wait for the next change.

Với bản chất của các hệ thống điển hình, ta đã bỏ lỡ gần hết cơ hội tận dụng OCP rồi. Codebase của ta chẳng thấy ngượng ngùng gì về sự mở toang của mình. Hầu như chẳng class nào đóng cửa. Ai buộc phải động vào mã cũng được chào đón bằng thay đổi tràn lan. Nếu thay đổi là điều chắc chắn xảy ra, cách tốt nhất là đợi nó đến. Dọn dẹp theo kiểu phỏng đoán chỉ tạo ra rủi ro không cần thiết từ những cải tiến chưa ai cần. Vậy hãy chờ thay đổi kế tiếp.

What About Now? — Thế còn bây giờ thì sao?

OK, that didn't take too long. We were told, moments ago (between the prior paragraph and this one), that indeed we must support a couple of new material types. Specifically, we need to allow jigsaw puzzles and board games to be borrowed. They'll fall under a new fine scheme the library folks want to try.

Được rồi, chờ cũng chẳng lâu la gì. Vừa mới đây thôi (giữa đoạn văn trước và đoạn này), người ta báo với ta rằng quả thật phải hỗ trợ thêm vài loại tài liệu mới. Cụ thể, ta cần cho phép mượn trò chơi ghép hình và board game. Chúng sẽ rơi vào một cơ chế tính phạt mới mà mấy vị bên thư viện muốn thử nghiệm.

We'll prepare for the new requirement by first factoring the code (prefactoring it) so that the change effort is smoother and has minimal impact.

Ta sẽ chuẩn bị cho yêu cầu mới bằng cách sắp xếp lại mã trước đã (prefactor), để công sức bỏ ra cho thay đổi được mượn mà và ít gây xáo trộn nhất.

Each of the two switch branches calculates an appropriate fine value using one of two strategies. We can extract the tiny bits of calculation logic to a couple of Strategy 7 classes, each implementing a common interface:

Mỗi nhánh trong hai nhánh của switch tính ra một khoản phạt phù hợp bằng một trong hai chiến lược. Ta có thể trích những mẩu logic tính toán tí hon ấy ra thành một cặp class Strategy 7, mỗi class hiện thực một interface chung:

7 [GOF95].

7 [GOF95].

```
public interface LateStrategy {
    int calculateFine(int daysLate);
}
public class ConstrainedFineStrategy implements LateStrategy{
    public static final short BASE_FEE = 100;

    public static final short MAX_FINE = 1000;
    private final int fineBasis;
```



```

    public ConstrainedFineStrategy(int fineBasis) {
        this.fineBasis = fineBasis;
    }
    @Override
    public int calculateFine(int daysLate) {
        return Math.min(MAX_FINE, BASE_FEE + fineBasis * daysLate);
    }
}
public class DaysLateStrategy implements LateStrategy {
    private final int fineBasis;
    public DaysLateStrategy(int fineBasis) {
        this.fineBasis = fineBasis;
    }
    @Override
    public int calculateFine(int daysLate) {
        return fineBasis * daysLate;
    }
}

```

What cute, tiny little classes! Here's the updated

Mấy class bé xíu dễ thương làm sao! Còn đây là bản cập nhật của calculateLateFine method.

```

public int calculateLateFine() {
    var daysLate = daysLate();
    var fineBasis = getMaterial().materialType().dailyFine();
    var fine = 0;
    switch (getMaterial().materialType()) {
        case BOOK, NEW_RELEASE_DVD:
            fine =
                new DaysLateStrategy(fineBasis).calculateFine(daysLate);
            break;
        case AUDIO_CASSETTE, VINYL_RECORDING, MICRO_FICHE,
            AUDIO_CD, SOFTWARE_CD, DVD, BLU_RAY, VIDEO_CASSETTE:
            fine = new ConstrainedFineStrategy(fineBasis)
                .calculateFine(daysLate);
            break;
    }
    return fine;
}

```

Note that calculateLateFine first retrieves the material type (e.g.,

Lưu ý rằng calculateLateFine trước hết lấy về loại tài liệu (ví dụ, BOOK) từ material, in order to decide which strategy to apply.

BOOK) từ material, để quyết định xem nên áp dụng chiến lược nào.

Here's what MaterialType looks like:

Đây là dáng vẻ của MaterialType:

```

public enum MaterialType {

```

```

    BOOK(21, 10),
    AUDIO_CASSETTE(14, 10),
    VINYL_RECORDING(14, 10),
    MICRO_FICHE(7, 200),
    AUDIO_CD(7, 100),

    SOFTWARE_CD(7, 500),
    DVD(3, 100),
    NEW_RELEASE_DVD(1, 200),
    BLU_RAY(3, 200),
    VIDEO_CASSETTE(7, 10);
    private final int checkoutPeriod;
    private final int dailyFine;
    MaterialType(int checkoutPeriod, int dailyFine) {
        this.checkoutPeriod = checkoutPeriod;
        this.dailyFine = dailyFine;
    }
    public int dailyFine() {
        return dailyFine;
    }
    public int checkoutPeriod() {
        return checkoutPeriod;
    }
}

```

If each material type can be initialized with the checkout period and late fine amounts, it can also be initialized with a Strategy object:

Nếu mỗi loại tài liệu có thể được khởi tạo cùng với thời hạn mượn và các mức phạt trễ hạn, thì nó cũng có thể được khởi tạo cùng với một đối tượng Strategy:

```

import domain.core.ConstrainedFineStrategy;
import domain.core.DaysLateStrategy;
import domain.core.LateStrategy;

public enum MaterialType {
    BOOK(21, new DaysLateStrategy(10)),
    AUDIO_CASSETTE(14, new ConstrainedFineStrategy(10)),
    VINYL_RECORDING(14, new ConstrainedFineStrategy(10)),
    MICRO_FICHE(7, new ConstrainedFineStrategy(200)),
    AUDIO_CD(7, new ConstrainedFineStrategy(100)),
    SOFTWARE_CD(7, new ConstrainedFineStrategy(500)),
    DVD(3, new ConstrainedFineStrategy(100)),
    NEW_RELEASE_DVD(1, new DaysLateStrategy(200)),
    BLU_RAY(3, new ConstrainedFineStrategy(200)),
    VIDEO_CASSETTE(7, new ConstrainedFineStrategy(10));
    private final int checkoutPeriod;
    private final LateStrategy lateStrategy;
    MaterialType(int checkoutPeriod, LateStrategy lateStrategy) {
        this.checkoutPeriod = checkoutPeriod;
        this.lateStrategy = lateStrategy;
    }
    public int checkoutPeriod() {

```

```

        return checkoutPeriod;
    }
    public int calculateFine(int daysLate) {
        return lateStrategy.calculateFine(daysLate);
    }
}

```

(Yes we could use Singletons 8 for each LateStrategy derivative, particularly since the subclasses contain no data. We could also specify a class type for each enum value, then use reflection to instantiate the Strategy object. But why make things more complicated?)

(Đúng vậy, ta có thể dùng Singleton 8 cho từng dẫn xuất của LateStrategy, nhất là khi các subclass chẳng chứa dữ liệu gì. Ta cũng có thể chỉ định một kiểu class cho mỗi giá trị enum, rồi dùng reflection để khởi tạo đối tượng Strategy. Nhưng làm mọi thứ phức tạp thêm lên để làm gì?)

8 [GOF95].

8 [GOF95].

MaterialType can then supply a calculateFine implementation that delegates to the lateStrategy object:

Khi đó MaterialType có thể cung cấp một hiện thực calculateFine chỉ việc uỷ thác cho đối tượng lateStrategy :

```

public int calculateFine(int daysLate) {
    return lateStrategy.calculateFine(dailyFine, daysLate);
}

```

The best part? At this point, the Holding method calculateLateFine can simplify to a single line of code that delegates to

Phần hay nhất ư? Đến lúc này, method calculateLateFine của Holding có thể rút gọn còn đúng một dòng mã uỷ thác cho

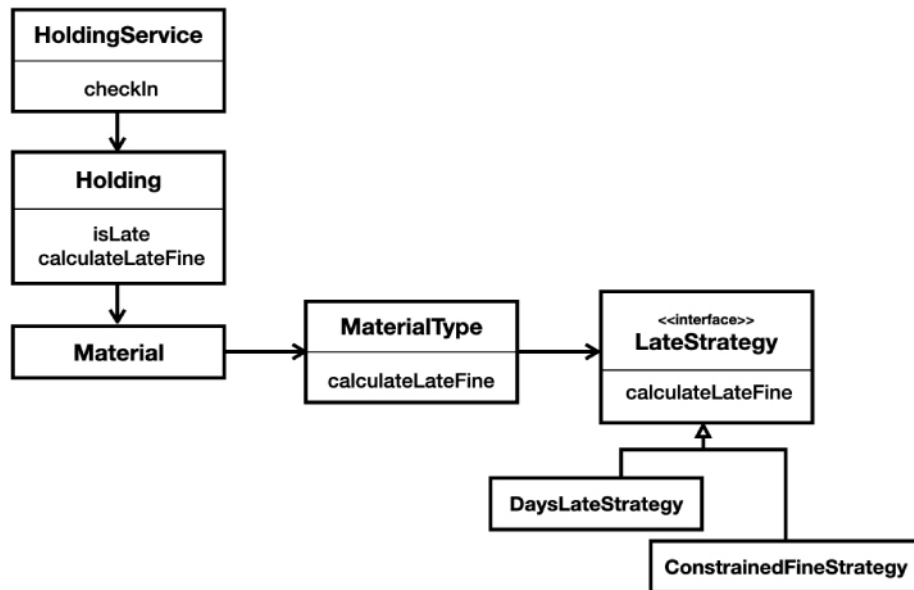
```

MaterialType:
public int calculateLateFine() {
    return getMaterial().materialType().calculateFine(daysLate());
}

```

The following shows a picture of the updated solution:

Hình dưới đây cho thấy giải pháp sau khi cập nhật:



Closed, Cohesive, Single-Responsibility Classes — Class đóng, gắn kết, một trách nhiệm

We've been tasked with supporting a new feature. For jigsaw puzzles and board games, the library board has decided to test out a new fine scheme. We test-drive the corresponding Strategy class:

Ta được giao nhiệm vụ hỗ trợ một tính năng mới. Với trò chơi ghép hình và board game, ban quản lý thư viện đã quyết định thử nghiệm một cơ chế tính phạt mới. Ta test-drive class Strategy tương ứng:

```

public class DegradingFineStrategy implements LateStrategy {
    private final int fineBasis;
    private final double degradationRate;
    public DegradingFineStrategy(
        int fineBasis, double degradationRate) {
        this.fineBasis = fineBasis;
        this.degradationRate = degradationRate;
    }
    @Override
    public int calculateFine(int daysLate) {
        double total = fineBasis *
            (1 - Math.pow(1 - degradationRate, daysLate)) /
            degradationRate;
        return (int) Math.round(total);
    }
}

```

Here are the tests, in case you thought we were only pretending to write them:

Còn đây là các test, phòng khi bạn tưởng rằng chúng tôi chỉ làm bộ viết test cho có:

```

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

```

```

class DegradatingFineStrategyTest {
    int initialFine = 1000;
    double degradationRate = 0.1d;
    DegradatingFineStrategy strategy =
        new DegradatingFineStrategy(initialFine, degradationRate);
    @Test
    void firstDayIsFineBasis() {
        assertEquals(initialFine, strategy.calculateFine(1));
    }
    @Test
    void nextDayReducedUsingDegradationRate() {
        assertEquals(1000 + 900, strategy.calculateFine(2));
    }
    @Test
    void multipleDays() {
        assertEquals(1000 + 900 + 810 + 729 + 656,
            strategy.calculateFine(5));
    }
}

```

For now, we must add a line to the `MaterialType` class to accommodate the new category:

Hiện tại, ta phải thêm một dòng vào class `MaterialType` để chứa hạng mục mới:

```

public enum MaterialType {
    GAMES_AND_PUZZLES(21, new DegradatingFineStrategy(500, 0.20)),
    BOOK(21, new DaysLateStrategy(10)),
    AUDIO_CASSETTE(14, new ConstrainedFineStrategy(10)),
    // ...
}

```

As a result of having to modify an existing class, our system isn't quite fully closed with respect to accommodating new fine schemes. But we realize that it wouldn't be difficult to replace the `MaterialType` enum with a class that reads the necessary information from a database. Once built, our system would be fully OCP compliant: Code a new class, seed the database with sufficient configuration data, and go—no changes to existing classes required!

Vì buộc phải sửa một class sẵn có, hệ thống của ta vẫn chưa đóng hoàn toàn đối với việc tiếp nhận các cơ chế tính phạt mới. Nhưng ta nhận ra rằng thay enum `MaterialType` bằng một class đọc thông tin cần thiết từ cơ sở dữ liệu cũng chẳng khó. Làm xong thứ đó, hệ thống của ta sẽ tuân thủ OCP trọn vẹn: viết một class mới, nạp đủ dữ liệu cấu hình vào cơ sở dữ liệu, rồi chạy — không cần thay đổi bất kỳ class sẵn có nào!

When Policies Change — Khi các chính sách thay đổi

Many service classes must orchestrate numerous disparate behaviors. They interact with business logic regarding various domains (holdings, patrons, branches), they might persist or retrieve data, and they might interact with external services. The `HoldingServiceClass` is one example of an orchestrating class.

Nhiều class dịch vụ phải điều phối hàng loạt hành vi rời rạc. Chúng tương tác với logic nghiệp vụ của đủ thứ miền khác nhau (holdings, patrons, branches), chúng có thể lưu

hoặc đọc dữ liệu, và chúng có thể gọi tới các dịch vụ bên ngoài. `HoldingServiceClass` là một ví dụ của class điều phối.

Some services might have minimal orchestration needs. An example is a collection of behaviors concerned with date calculations.

Một số dịch vụ có nhu cầu điều phối rất ít. Ví dụ là một tập hợp các hành vi lo việc tính toán ngày tháng.

A well-defined SRP-compliant service, as such, should either:

Vì thế, một dịch vụ được định nghĩa tốt và tuân thủ SRP — Nguyên tắc Trách nhiệm Đơn nhất — nên hoặc là:

- Contain only orchestration logic that declares policy and delegates to other classes for implementation specifics.
- Contain only implementation specifics for related domain or utility behaviors.
 - Chỉ chứa logic điều phối: khai báo chính sách rồi uỷ thác các chi tiết hiện thực cho những class khác.
 - Chỉ chứa chi tiết hiện thực cho các hành vi nghiệp vụ hoặc tiện ích có liên quan với nhau.

In other words, don't mix policy with implementation detail. Doing so is costlier to understand, navigate, and maintain. It's also not SRP compliant: A policy change is one reason to make a code change, and implementation details about a step in the policy is another reason.

Nói cách khác, đừng trộn chính sách với chi tiết hiện thực. Trộn vào nhau thì tốn kém hơn để hiểu, để dò tìm và để bảo trì. Nó cũng không tuân thủ SRP: thay đổi chính sách là một lý do để sửa mã, còn chi tiết hiện thực của một bước trong chính sách lại là một lý do khác.

Is This Overengineering? — Vậy có phải là thiết kế quá đà không?

What would happen if we took the SRP and the OCP to the extreme? Virtually everything in our system would be “plug and play.” In a service, we wouldn't have a single controller class with a half dozen routes; we'd create a separate class for each route. It would register itself (perhaps like what an

Chuyện gì sẽ xảy ra nếu ta đẩy SRP và OCP tới mức cực đoan? Gần như mọi thứ trong hệ thống sẽ trở thành “cắm vào là chạy”. Trong một service, ta sẽ không có một class controller duy nhất ôm nửa tá route; ta sẽ tạo một class riêng cho từng route. Nó sẽ tự đăng ký chính mình (có lẽ giống như thứ mà annotation

`@PostMapping` annotation accomplishes in Spring Boot). Its sole job would be to delegate requests to a similarly single-purpose service class (or to reject them as a result of validation, which is likewise implemented elsewhere). Each microservice facet would represent either a policy—whereby all specifics are delegated to—or a chunk of code that accomplished a singular need.

`@PostMapping` làm được trong Spring Boot). Việc duy nhất của nó là uỷ thác request cho một class service cũng chỉ làm đúng một việc (hoặc từ chối request vì không qua được validation, mà phần validation ấy cũng được hiện thực ở nơi khác). Mỗi mặt của microservice sẽ hoặc đại diện cho một chính sách — nơi mọi chi tiết cụ thể đều được uỷ thác đi chỗ khác — hoặc là một mẫu mã hoàn thành đúng một nhu cầu duy nhất.

And so on for each class down the line. Pretty much all conditionals would be implemented polymorphically; that is, both branches would be implemented in separate classes, much like states

in a State 9 pattern. Each step in a policy (a workflow) would be implemented in a separate class as well.

Và cứ thế cho từng class ở các tầng bên dưới. Gần như mọi câu lệnh điều kiện sẽ được hiện thực theo lối đa hình (polymorphism); tức là cả hai nhánh đều được hiện thực trong những class tách biệt, khá giống các trạng thái trong mẫu State 9. Mỗi bước trong một chính sách (một workflow) cũng sẽ được hiện thực trong một class riêng.

9 [GOF95].

9 [GOF95].

Ridiculous? It may sound like it, but such a system would truly support the OCP, whereby all new behavior was accomplished by dropping a new class in place. All classes would be small and easily tested in isolation. You'd write tests for each new class and perhaps would update an end-to-end test. Otherwise, you'd make no changes to the system.

Nghe nực cười? Có thể nghe là như vậy thật, nhưng một hệ thống như thế mới thực sự hỗ trợ OCP, khi mà mọi hành vi mới đều đạt được bằng cách thả một class mới vào đúng chỗ. Mọi class đều nhỏ và dễ test độc lập. Bạn sẽ viết test cho mỗi class mới và có lẽ cập nhật thêm một test đầu-cuối. Ngoài ra, bạn chẳng phải thay đổi gì trong hệ thống cả.

Almost all systems don't look like this, and they are harder to work with as well as riskier.

Gần như chẳng hệ thống nào trông như vậy, và chúng vừa khó làm việc cùng hơn vừa nhiều rủi ro hơn.

A highly SRP/OCP-compliant system can be initially challenging for developers not familiar with the idea—it takes a while to figure out where things are at first. (“There’s all these objects and none of ‘em are doing anything!”) Long term, however, it dramatically streamlines the change process.

Một hệ thống tuân thủ SRP/OCP ở mức cao lúc đầu có thể gây khó cho những lập trình viên chưa quen với ý tưởng này — phải mất một lúc mới định vị được mọi thứ nằm ở đâu. (“Toàn là object, mà chẳng thằng nào làm gì cả!”) Tuy nhiên, về dài hạn, nó làm quy trình thay đổi trở nên trơn tru hơn hẳn.

It's not a new idea. We worked with systems approaching this ideal 30 years ago in Smalltalk. Most classes were small, and the majority of methods were one to three lines of code each. Life was bliss.

Đây không phải ý tưởng mới. Ba mươi năm trước, chúng tôi đã làm việc với những hệ thống gần đạt tới lý tưởng đó bằng Smalltalk. Phần lớn class đều nhỏ, và đa số method chỉ dài một đến ba dòng mã. Cuộc đời khi ấy thật đẹp.

As with most principles, consider both the SRP and the OCP as ideals: You want to almost always move in their direction, not away from them. It might be OK that you'll never hit the ideal. The typical system, however, moves in the opposite direction, degrading toward larger, more monolithic classes. That's why they're harder to work with as they grow.

Như với hầu hết các nguyên tắc, hãy xem cả SRP lẫn OCP là những lý tưởng: hầu như lúc nào bạn cũng muốn tiến về phía chúng, chứ không phải rời xa chúng. Không bao giờ chạm tới lý tưởng cũng chẳng sao. Vấn đề là hệ thống điển hình lại đi theo hướng ngược lại, thoái hoá dần thành những class to hơn, khối nạc hơn. Đó là lý do chúng càng lớn thì càng khó làm việc cùng.

For now, forgive your system for its flaws. Moving forward, assess each change you make—how many existing classes did you have to open for change? Think of a reshaped, SRP/OCP-compliant system that would have instead accommodated that new feature as an extension of the existing code. Expend at least some effort to move your system in that direction.

Hiện tại, hãy tha thứ cho hệ thống của bạn vì những khiếm khuyết của nó. Nhưng từ giờ trở đi, hãy đánh giá từng thay đổi bạn thực hiện — bạn đã phải mở bao nhiêu class sẵn có ra để sửa? Hãy hình dung một hệ thống được nắn lại theo hướng tuân thủ SRP/OCP, nơi tính năng mới ấy sẽ được tiếp nhận như một phần mở rộng của mã sẵn có. Hãy bỏ ra ít nhất một chút công sức để đẩy hệ thống của bạn về hướng đó.

Seek to create smaller, single-purpose classes. They're easier to work with (test, understand, change), and they're easier to close.

Hãy tìm cách tạo ra những class nhỏ hơn, mỗi class chỉ làm một việc. Chúng dễ làm việc cùng hơn (dễ test, dễ hiểu, dễ sửa), và cũng dễ đóng lại hơn.

Seek to close classes. They're something we can consider “out of sight, out of mind.” We seek to have less code that we must concern ourselves with.

Hãy tìm cách đóng các class lại. Chúng là thứ ta có thể coi như “khuất mắt trông coi”. Ta muốn có ít mã phải bận tâm tới hơn.

Simpler Testing — Kiểm thử đơn giản hơn

You could choose to retain whatever tests you had in place for

Bạn có thể chọn giữ lại bất kỳ test nào bạn đang có cho

HoldingService . When adding the new behavior for fines, you'd add to these tests. That's one acceptable route.

HoldingService . Khi thêm hành vi mới về tiền phạt, bạn sẽ bổ sung thêm vào những test ấy. Đó là một hướng đi chấp nhận được.

However, you can also distribute the testing as well. Yes, the individual fine policies aren't publicly exposed to the ultimate client, and might be considered “private behavior.” But the policies are also isolated, closed concepts that can be reused— and more easily tested than within the larger context of

Tuy nhiên, bạn cũng có thể phân tán luôn cả việc kiểm thử. Đúng là từng chính sách tính phạt riêng lẻ không được phơi ra công khai cho client cuối cùng, và có thể bị coi là “hành vi riêng tư”. Nhưng các chính sách ấy đồng thời cũng là những khái niệm biệt lập, đã đóng, có thể tái sử dụng — và dễ test hơn nhiều so với khi nằm trong ngữ cảnh lớn hơn của

HoldingService . They are also public in the sense that they could conceivably be consumed by a different client.

HoldingService . Chúng cũng công khai theo nghĩa là hoàn toàn có thể được một client khác dùng đến.

Creating unit tests focused directly on a now-public fine policy Strategy class makes it officially documented. Such focused unit tests describe all the intentional behavior that developers coded in the class. Clean (clear, concise, and well-abstracted) unit tests provide far more useful documentation than method-level doc comments: We can trust them, as long as the tests pass, and we can see exactly how to interact with the class as a direct client.

Việc tạo unit test nhắm thẳng vào class Strategy tính phạt — thứ giờ đã công khai — biến nó thành tài liệu chính thức. Những unit test tập trung như vậy mô tả toàn bộ hành vi mà lập trình viên cố ý viết ra trong class đó. Unit test sạch (rõ ràng, súc tích và trừu tượng hoá tốt) cung cấp tài liệu hữu ích hơn nhiều so với doc comment ở mức method: ta có thể tin

chúng, chừng nào chúng còn pass, và ta thấy chính xác cách tương tác với class ấy với tư cách một client trực tiếp.

We avoid testing implementation details. We test the outcomes of (small) behavioral concepts instead. Small, closed units should be concepts that can be moved about and work in any context where they're useful. A days-late fine policy can be tested in isolation without exposing any implementation specifics. It is a “sub-behavioral unit.”

Ta tránh test chi tiết hiện thực. Thay vào đó, ta test kết quả đầu ra của những khái niệm hành vi (nhỏ). Các đơn vị nhỏ, đã đóng nên là những khái niệm có thể mang đi mang lại và hoạt động được trong bất kỳ ngữ cảnh nào cần tới chúng. Một chính sách phạt theo số ngày trễ có thể được test biệt lập mà không cần phơi ra bất kỳ chi tiết hiện thực nào. Nó là một “đơn vị dưới mức hành vi”.

Corollary: If you extract to a small, single-purpose class, but don't view it as useful in any other context, don't make it publicly accessible. You'll still need to write tests that indirectly involve these classes at a higher level. Test such private details directly only as a last resort.

Hệ quả: nếu bạn trích ra một class nhỏ chỉ làm một việc, nhưng lại thấy nó chẳng hữu ích trong ngữ cảnh nào khác, thì đừng cho nó công khai. Bạn vẫn sẽ phải viết test chạm tới các class đó một cách gián tiếp ở mức cao hơn. Chỉ test trực tiếp những chi tiết riêng tư như vậy khi đã hết cách.

Our new fine strategy took less than 20 minutes to test-drive— we created both unit tests and the production code in that time. We didn't have to worry about changing tests for, or adding tests to, an existing class.

Chiến lược tính phạt mới của chúng tôi mất chưa tới 20 phút để test-drive — trong khoảng thời gian đó, chúng tôi tạo ra cả unit test lẫn mã sản phẩm. Chúng tôi chẳng phải lo sửa test của một class sẵn có, cũng chẳng phải lo thêm test vào đó.

Enter AI — AI bước vào sân khấu

We (humans) are and have been advancing by orders of magnitude in many ways, technologically, every five to ten years or so for the past 90 years. AI will likely continue that amazing trend, accelerating us toward the singularity by the 2040s. Per Ray Kurzweil, “We are now entering the steep part of the exponential.” 10

Chúng ta (loài người) đang và đã tiến bộ theo cấp số nhân trên nhiều phương diện công nghệ, cứ khoảng năm tới mười năm một lần, suốt 90 năm qua. AI nhiều khả năng sẽ tiếp nối xu hướng kinh ngạc ấy, đẩy nhanh chúng ta về phía điểm kỳ dị (singularity) vào những năm 2040. Theo lời Ray Kurzweil, “Chúng ta đang bước vào đoạn dốc đứng của đường cong hàm mũ.” 10

10 [Singularity], pp. 164–165.

10 [Singularity], tr. 164–165.

Today, we can ask an **LLM** to produce code at the class level, and it will do an OK job, maybe getting 80% of the code correct. (There's that good, old 80/20 rule again. 11) We can help an LLM do a better job by:

Ngày nay, ta có thể nhờ một LLM (mô hình ngôn ngữ lớn) sinh mã ở mức class, và nó sẽ làm tạm ổn, có lẽ đúng chừng 80% lượng mã. (Lại là quy tắc 80/20 quen thuộc đấy. 11) Ta có thể giúp LLM làm tốt hơn bằng cách:

11 <https://www.investopedia.com/terms/1/80-20-rule.asp>

11 <https://www.investopedia.com/terms/1/80-20-rule.asp>

- Providing it with examples
 - Providing it with a simple style that tells it to (a) produce small, single-purpose modules/classes and small, single-purpose functions/methods, (b) produce more-functional solutions with less state management and side effects, and (c) emphasize clarity What's cool is that this simple style matches how we've been told to code all along. It's also the style that has the broadest support, with few serious detractors. We have concepts like SOLID and an exhaustively examined body of work in design patterns; these things have been around for decades now.
- Cung cấp cho nó các ví dụ
 - Cung cấp cho nó một phong cách đơn giản, bảo nó (a) sinh ra các module/class nhỏ chỉ làm một việc và các hàm/method nhỏ chỉ làm một việc, (b) sinh ra những giải pháp thiên về hàm hơn, ít quản lý trạng thái và ít tác dụng phụ (side effect) hơn, và (c) đề cao sự rõ ràng. Điều thú vị là phong cách đơn giản này trùng khớp với những gì người ta vẫn dạy ta viết mã bấy lâu nay. Nó cũng là phong cách được ủng hộ rộng rãi nhất, với rất ít người phản đối nghiêm túc. Ta đã có những khái niệm như SOLID và cả một kho tàng công trình về design pattern đã được mổ xẻ cặn kẽ; những thứ này đã tồn tại hàng thập kỷ rồi.

It Will Be Wrong — Nó sẽ sai

Unfortunately, we cannot trust that an LLM will get 100% of the code correct. The easy solution is to have it generate tests from the examples provided, then vet the tests for fidelity with our examples, then run them.

Đáng tiếc là ta không thể tin rằng một LLM sẽ viết đúng 100% lượng mã. Giải pháp dễ nhất là bảo nó sinh test từ các ví dụ ta đã cung cấp, rồi soát lại xem test có trung thành với ví dụ của ta không, rồi chạy chúng.

In fact, comprehensive testing for behavioral intent is essential when using AI. AI will quickly generate more code than has been produced by humans to this point in history. It should create fear in everyone that 20% of that code is likely to be defective.

Thật ra, kiểm thử toàn diện cho ý định hành vi là điều thiết yếu khi dùng AI. AI sẽ nhanh chóng sinh ra nhiều mã hơn cả lượng mã loài người từng viết ra tính đến thời điểm này của lịch sử. Việc 20% số mã đó nhiều khả năng có lỗi đáng để mọi người phải sợ.

When the AI gets the code wrong—and it will—we need to have a conversation with it. That conversation is far more effective with a clean design: The source of the problem will be easier to pinpoint, and the resolution can involve focusing on that one small class or method. (If it's a method, sometimes the best route is to have the AI extract what's known as a method object—a new class that embodies the single troubled method. 12)

Khi AI viết sai mã — mà nó sẽ sai — ta cần nói chuyện với nó. Cuộc trò chuyện đó hiệu quả hơn nhiều khi thiết kế sạch sẽ: nguồn cơn của vấn đề sẽ dễ khoanh vùng hơn, và cách khắc phục có thể chỉ tập trung vào đúng một class hay một method nhỏ. (Nếu là method, đôi khi hướng tốt nhất là bảo AI trích ra thứ gọi là method object — một class mới hiện thân cho đúng cái method rắc rối ấy. 12)

12 [WELC].

12 [WELC].

While the current trend is to view AI as most useful as a line-by-line coding assistant, it's already at the point where it's more effective to have it produce code at a modular level. At this point, the code content of the module matters far less. Need to change a class? Pull up the list of examples, update them, and regenerate both the tests and production code. That's not so easy if the modules aren't SRP compliant.

Trong khi xu hướng hiện nay xem AI hữu dụng nhất ở vai trò trợ lý viết mã từng dòng, thì nó đã tới mức mà để nó sinh mã ở cấp module còn hiệu quả hơn. Đến lúc đó, nội dung mã bên trong module trở nên ít quan trọng hơn nhiều. Cần sửa một class ư? Mở danh sách ví dụ ra, cập nhật chúng, rồi sinh lại cả test lẫn mã sản phẩm. Chuyện đó chẳng dễ nếu các module không tuân thủ SRP.

Don't get us wrong. We love mucking with code on a line-by-line basis. We are gratified by streamlining functions to make them elegant—that is, clear and concise. But ultimately, the contents of any well-composed function are irrelevant once we begin generating wholesale modules.

Đừng hiểu lầm chúng tôi. Chúng tôi mê mẩn chuyện hì hục với mã trên từng dòng. Chúng tôi thấy đã đời khi tinh gọn một hàm cho nó trở nên thanh thoát — tức là rõ ràng và súc tích. Nhưng rốt cuộc, nội dung của bất kỳ hàm nào được soạn khéo cũng trở nên vô nghĩa một khi ta bắt đầu sinh ra nguyên cả module.

Our focus begins to shift. We concern ourselves with the overall **architecture**/design of the system, and we seek to create a system based on the OCP and SRP. When we need new behavior, we generate the appropriate module using AI, we vet the tests it generates, and we drop it into place.

Trọng tâm của ta bắt đầu dịch chuyển. Ta bận tâm tới kiến trúc/thiết kế tổng thể của hệ thống, và ta tìm cách tạo ra một hệ thống dựa trên OCP và SRP. Khi cần hành vi mới, ta dùng AI sinh ra module phù hợp, soát lại đám test mà nó sinh ra, rồi thả module đó vào đúng chỗ.

Our system designs once again take center stage. We find value in employing time-honored design patterns that directly speak toward pluggable designs: Functional Pipeline, Command, Factory, Decorator, Strategy, Composite, Chain of Responsibility, and Bridge.

Thiết kế hệ thống của ta lại một lần nữa chiếm vị trí trung tâm sân khấu. Ta thấy giá trị ở việc dùng những design pattern đã được thời gian kiểm chứng và nói thẳng tới các thiết kế kiểu cắm-rút: Functional Pipeline, Command, Factory, Decorator, Strategy, Composite, Chain of Responsibility, và Bridge.

Our focus on **clean code** remains important for the near future. At some point, AI will be able to completely implement an entire system from a set of examples/tests. We'll be happy to have retired by then. You'll still have opportunities as someone who knows how to design and deliver a system as an orchestration of well-tested, AI-implemented pieces.

Việc tập trung vào clean code vẫn còn quan trọng trong tương lai gần. Đến một lúc nào đó, AI sẽ có thể hiện thực trọn vẹn cả một hệ thống từ một bộ ví dụ/test. Khi ấy thì chúng tôi hẳn đã vui vẻ về hưu rồi. Còn bạn thì vẫn sẽ có cơ hội, với tư cách một người biết cách thiết kế và bàn giao hệ thống như một dàn nhạc gồm những mảnh ghép đã được test kỹ do AI hiện thực.

Chapter 14. Testing Disciplines —

Chương 14. Các kỷ luật kiểm thử



Our profession has come a long way in the last three decades. In 1997, no one had heard of testing disciplines such as **TDD** 1 or TCR. 2 For the vast majority of us, unit tests were short bits of throwaway code that we wrote to make sure our programs “worked.” We would painstakingly write our classes and methods, and then we might concoct some ad hoc code to **test** them. Typically, this would involve some kind of simple driver program that would allow us to manually interact with the program we had written.

Ngành của chúng ta đã đi được một quãng đường rất dài trong ba thập kỷ qua. Năm 1997, chưa ai từng nghe tới những kỷ luật (discipline) kiểm thử như TDD 1 hay TCR. 2 Với đại đa số chúng ta, unit test chỉ là mấy mẫu mã dùng xong rồi vứt, viết ra để chắc rằng chương trình “chạy được”. Chúng ta cặm cụi viết class và method, rồi chế ra ít mã tạm bợ để thử chúng. Thường thì đó là một chương trình driver đơn giản nào đó cho phép ta tương tác thủ công với chương trình vừa viết.

1 Test-driven development. 2 Test && commit || revert.

1 Test-driven development. 2 Test && commit || revert.

I remember writing a C++ program for an embedded real-time system back in the mid-’90s. The program was a simple timer with the following signature:

Tôi nhớ hồi giữa những năm 90 mình có viết một chương trình C++ cho một hệ thống nhúng thời gian thực. Chương trình là một bộ đếm giờ đơn giản với signature như sau:

```
void Timer::ScheduleCommand(Command* theCommand, int milliseconds)
```

The idea was simple; the execute **method** of the Command 3 would be executed in a new **thread** after the specified number of milliseconds. The problem was how to test it.

Ý tưởng thì đơn giản: method execute của Command 3 sẽ được chạy trong một luồng (thread) mới sau đúng số milliseconds đã chỉ định. Vấn đề là làm sao test được nó.

3 See the Command pattern in [GOF95].

3 Xem mẫu Command trong [GOF95].

I cobbled together a simple driver program that listened to the keyboard. Every time a character key was pressed, it would schedule a command that would print the same character five seconds later. Then, I tapped out a rhythmic melody on the keyboard and waited for that melody to replay on the screen five seconds later.

Tôi chấp vá một chương trình driver đơn giản biết lắng nghe bàn phím. Mỗi lần một phím ký tự được nhấn, nó lên lịch một command in ra đúng ký tự đó sau năm giây. Rồi tôi gõ một giai điệu có nhịp điệu trên bàn phím và chờ giai điệu ấy phát lại trên màn hình năm giây sau.

“I . . . want-a-girl . . . just . . . like-the-girl-who-marr . . . ied . . . dear . . . old . . . dad.”

“I . . . want-a-girl . . . just . . . like-the-girl-who-marr . . . ied . . . dear . . . old . . . dad.”

I actually sang that melody while typing the “.” key, and then I sang it again as the dots appeared on the screen.

Tôi hát giai điệu đó thật, vừa hát vừa gõ phím “.”, rồi hát lại lần nữa khi những dấu chấm lần lượt hiện lên màn hình.

That was my test! Once I saw it work and demonstrated it to my colleagues, I threw the test code away.

Đó là test của tôi đấy! Sau khi thấy nó chạy được và trình diễn cho đồng nghiệp xem, tôi vứt luôn đăm mã test đi.

As I said, our profession has come a long way.

Như tôi đã nói, nghề của chúng ta đã đi được một quãng đường rất dài.

Nowadays, I write tests that make sure every nook and cranny of the code works as I expect it to. I isolate my code from the operating system rather than creating hard dependencies upon it. If I were to write the above example today, I would use test doubles to “**mock**” out the OS timing functions so that I had absolute control over the time. I would schedule commands that set boolean flags, and then I would use my tests to step the time forward, watching those flags and ensuring that they went from false to true just as I changed the time to the right value.

Ngày nay, tôi viết test để đảm bảo mọi góc ngách của mã đều hoạt động đúng như tôi mong đợi. Tôi cô lập mã của mình khỏi hệ điều hành thay vì tạo ra những phụ thuộc (dependency) cứng nhắc vào nó. Nếu hôm nay phải viết lại ví dụ trên, tôi sẽ dùng test double để “mock” các hàm định thời của OS, nhờ đó nắm quyền kiểm soát tuyệt đối với thời gian. Tôi sẽ lên lịch những command đặt cờ boolean, rồi dùng test để đẩy thời gian tiến lên, quan sát các cờ ấy và chắc chắn rằng chúng chuyển từ false sang true đúng vào lúc tôi chỉnh thời gian tới giá trị thích hợp.

Nowadays, once I get a set of tests to pass, I clean those tests to make sure that they are well designed and are strongly decoupled from the production code. I also ensure that they are convenient to run for anyone else who needs to work with the code. I keep the code and tests together in the same source **package**.

Ngày nay, khi đã làm cho một bộ test pass, tôi dọn sạch chính đám test đó để chắc rằng chúng được thiết kế tử tế và tách rời (decoupling) triệt để khỏi mã production. Tôi cũng đảm bảo rằng bất kỳ ai khác cần đụng tới mã đều có thể chạy chúng một cách thuận tiện. Tôi để mã và test nằm cạnh nhau trong cùng một source package.

In the early days of Extreme Programming and Agile, we only talked about one testing **discipline**: TDD. When I wrote the first edition of this book, TDD was the only testing discipline that I presented. In the intervening years, it has become clear that there are other testing disciplines that are as effective, or nearly as effective, as TDD. It has also become clear that there are many programmers who prefer those other disciplines for one reason or another.

Vào những ngày đầu của Extreme Programming và Agile, chúng tôi chỉ nói tới một kỷ luật kiểm thử duy nhất: TDD. Khi viết ấn bản đầu của cuốn sách này, TDD cũng là kỷ luật kiểm thử duy nhất tôi trình bày. Những năm sau đó, dần dần lộ ra rằng còn có những kỷ luật kiểm thử khác cũng hiệu quả, hoặc gần như hiệu quả, ngang với TDD. Và cũng lộ ra rằng có rất nhiều lập trình viên, vì lý do này hay lý do khác, lại thích những kỷ luật kia hơn.

And so while I will remain an adherent to the discipline of TDD, I feel that it is appropriate to talk about two other disciplines that are compatible with writing **clean code**. And, although I am only discussing three such disciplines, there may well be others.

Thế nên, dù vẫn là tín đồ của kỷ luật TDD, tôi thấy nên bàn thêm về hai kỷ luật khác cũng tương thích với việc viết clean code. Và dù ở đây tôi chỉ bàn ba kỷ luật, hẳn là còn nhiều kỷ luật khác nữa.

Discipline 1: Test-Driven Development (TDD) — Kỷ luật 1: Test-Driven Development (TDD)

This discipline was proposed by Kent Beck in the mid- to late '90s and became strongly associated with Extreme Programming and Agile development.

Kỷ luật này được Kent Beck đề xuất vào khoảng giữa tới cuối những năm 90 và gắn chặt với Extreme Programming cùng phong trào phát triển Agile.

By now, everyone knows that TDD asks us to write unit tests first, before we write production code. But that rule is just the tip of the iceberg.

Đến giờ thì ai cũng biết TDD yêu cầu ta viết unit test trước, trước cả khi viết mã production. Nhưng quy tắc đó mới chỉ là phần nổi của tảng băng.

TDD is not just writing tests first. TDD is an intricate discipline, not a simple rule of thumb. We practitioners do not write a whole batch of tests up front and then, one by one, make them pass. The discipline is far more intricate and intimate than that.

TDD không đơn thuần là viết test trước. TDD là một kỷ luật tinh vi, không phải một mẹo bỏ túi đơn giản. Những người thực hành chúng tôi không viết sẵn cả mớ test rồi mới lần lượt làm cho từng cái pass. Kỷ luật này tinh vi và gần gũi hơn thế nhiều.

The Three Laws of TDD — Ba định luật của TDD

What follows are the three laws of TDD. These laws are the basic foundation of the discipline. Following them is very hard, especially at first. Following them also requires some skill and knowledge that are hard to come by. If you try to follow these laws without that skill and knowledge, you will almost certainly become frustrated and abandon the discipline. We will address that skill and knowledge in subsequent chapters. For the moment, be warned. Following these laws without proper preparation can be very difficult.

Sau đây là ba định luật của TDD. Chúng là nền móng cơ bản của kỷ luật này. Tuân theo chúng rất khó, nhất là lúc mới bắt đầu. Tuân theo chúng còn đòi hỏi một số kỹ năng và kiến thức không dễ có. Nếu bạn cố tuân theo mà thiếu kỹ năng và kiến thức ấy, gần như chắc chắn bạn sẽ bực bội rồi bỏ cuộc. Chúng ta sẽ bàn về kỹ năng và kiến thức đó trong các chương sau. Còn lúc này, xin cảnh báo trước: tuân theo các định luật này mà không chuẩn bị đàng hoàng thì rất gian nan.

The First Law

Định luật thứ nhất

Write no production code until you have first written a test that fails due to the lack of that production code.

Không viết mã production nào cho tới khi bạn đã viết được một test thất bại chính vì thiếu đoạn mã production đó.

If you are a programmer of any years' experience, this law may seem foolish. You might wonder: What test am I supposed to write if there's no code to test? This question comes from the common expectation that tests are written after code. But, if you think about it, you'll realize that if you can write the production code, you can also write the code that tests the production code. It may seem out of order, but there's no lack of information preventing you from writing the test first.

Nếu bạn là lập trình viên đã có vài năm kinh nghiệm, định luật này nghe có vẻ ngớ ngẩn. Bạn sẽ thắc mắc: Tôi viết test gì được khi chưa có mã nào để test? Câu hỏi ấy xuất phát từ kỳ vọng quen thuộc rằng test được viết sau mã. Nhưng nếu ngẫm kỹ, bạn sẽ nhận ra: nếu bạn viết được mã production, thì bạn cũng viết được đoạn mã dùng để test mã production đó. Nghe có vẻ ngược đời, nhưng chẳng thiếu thông tin nào ngăn bạn viết test trước cả.

The Second Law

Định luật thứ hai

Write no more of a test than is sufficient to fail, or fail to compile. Resolve the failure by writing some production code.

Không viết test nhiều hơn mức vừa đủ để nó thất bại, hoặc không biên dịch được. Hãy khắc phục thất bại ấy bằng cách viết một ít mã production.

Again, if you are an experienced programmer, then you likely realize that the very first line of the test will fail to compile because that first line will be written to interact with code that does not yet exist. And that means, of course, that you will often be unable to write more than one line of a test before having to switch over to writing production code.

Một lần nữa, nếu là lập trình viên có kinh nghiệm, hẳn bạn nhận ra rằng ngay dòng đầu tiên của test đã không biên dịch được, bởi dòng đó viết ra để tương tác với đoạn mã còn chưa tồn tại. Và điều đó, dĩ nhiên, có nghĩa là bạn thường không thể viết quá một dòng test trước khi phải quay sang viết mã production.

The Third Law

Định luật thứ ba

Write no more production code than will resolve the currently failing test. Once the test passes, write more test code.

Không viết mã production nhiều hơn mức đủ để làm cho test đang thất bại pass. Khi test đã pass, hãy viết tiếp mã test.

And now the cycle is complete. It should be obvious to you that these three laws **lock** you into a cycle that is just a few seconds long. It looks like this:

Thế là vòng lặp khép kín. Hẳn bạn thấy rõ rằng ba định luật này khoá bạn vào một chu kỳ chỉ dài vài giây. Nó trông như thế này:

- You write a line of test code, but it doesn't compile (of course).
- You write a line of production code that makes the test compile.
- You write another line of test code that doesn't compile.
- You write another line or two of production code that makes the test compile.
- You write another line or two of test code that compiles, but fails an **assertion**.
- You write another line or two of production code that passes the assertion.
 - Bạn viết một dòng mã test, và dĩ nhiên nó không biên dịch được.
 - Bạn viết một dòng mã production khiến test biên dịch được.
 - Bạn viết thêm một dòng mã test nữa, lại không biên dịch được.
 - Bạn viết thêm một hai dòng mã production khiến test biên dịch được.
 - Bạn viết thêm một hai dòng mã test, biên dịch được nhưng lại rớt một assertion.
 - Bạn viết thêm một hai dòng mã production khiến assertion đó pass.

You saw this cycle being played out in the previous chapter. Watching that cycle is the best way to understand the tight feedback loop. I wrote a book sometime back titled Clean **Craftsmanship**. It has many examples of the TDD cycle. There are also some videos of those examples that you can see here: <>.

Bạn đã thấy chu kỳ này diễn ra ở chương trước. Quan sát chu kỳ ấy là cách tốt nhất để hiểu vòng phản hồi chặt chẽ này. Cách đây một thời gian tôi có viết cuốn Clean Craftsmanship. Trong đó có rất nhiều ví dụ về chu kỳ TDD. Cũng có vài video về các ví dụ đó, bạn có thể xem tại đây: <>.

Discipline 2: Test && Commit || Revert (TCR) — Kỷ luật 2: Test && Commit || Revert (TCR)

This discipline was proposed 4 by Kent Beck in 2018. TCR does not insist that tests be written first. Rather, it insists that code that passes tests should be immediately committed, and code that fails tests should be immediately reverted. This is generally enforced with a script that runs the tests whenever source files are saved and automatically executes the commit or revert based on the test results.

Kỷ luật này được Kent Beck đề xuất 4 vào năm 2018. TCR không đòi hỏi phải viết test trước. Thay vào đó, nó đòi hỏi rằng mã nào làm test pass thì phải commit ngay lập tức, còn mã nào làm test rớt thì phải revert ngay lập tức. Thông thường điều này được ép buộc bằng một script chạy test mỗi khi tệp nguồn được lưu, rồi tự động commit hay revert tùy theo kết quả test.

4 https://medium.com/@kentbeck_7670/test-commit-revert-870bbd756864

4 https://medium.com/@kentbeck_7670/test-commit-revert-870bbd756864

The immediate reversion on failure forces the programmers to move very carefully, and in very small steps. The programmer can write as much test and production code as they like, but they are liable to lose what they wrote if they have not written tests that pass. Thus, they will write tests and code together in very short cycles. Those cycles are as small as those in TDD.

Việc bị revert ngay khi thất bại buộc lập trình viên phải đi rất thận trọng, từng bước rất nhỏ. Lập trình viên muốn viết bao nhiêu mã test và mã production tùy thích, nhưng họ có nguy cơ mất trắng những gì vừa viết nếu chưa viết được test pass. Vì thế, họ sẽ viết test và mã cùng nhau trong những chu kỳ cực ngắn. Các chu kỳ đó nhỏ chẳng kém gì chu kỳ của TDD.

Personally, I find TCR to be stressful, while I find TDD to be relaxing. But others may find TCR to be better for their mindset because they are not forced into writing tests first.

Cá nhân tôi thấy TCR gây căng thẳng, còn TDD thì thư thái. Nhưng người khác có thể thấy TCR hợp với đầu óc họ hơn, vì họ không bị ép phải viết test trước.

Discipline 3: Small Bundles — Kỷ luật 3: Những gói nhỏ (Small Bundles)

This discipline was proposed to me by John Ousterhout while I was in the process of writing this second edition. It forces neither the ordering of the tests and code, nor the intensely small cycle of TDD and TCR. The cycle time is longer, probably by an order of magnitude. Minutes as opposed to seconds.

Kỷ luật này do John Ousterhout đề xuất với tôi trong lúc tôi đang viết ấn bản thứ hai này. Nó không ép thứ tự giữa test và mã, cũng không ép chu kỳ siêu ngắn như TDD và TCR. Thời gian mỗi chu kỳ dài hơn, có lẽ dài hơn cả một bậc độ lớn: tính bằng phút chứ không phải bằng giây.

The idea is simple. Programmers write a small bundle of code and tests to achieve a certain result. They might write the code first. They might write the tests first. They might intersperse the writing of the code and tests. But they will conclude with that small bundle having high **test coverage** and all tests passing.

Ý tưởng rất đơn giản. Lập trình viên viết một gói nhỏ gồm mã và test để đạt được một kết quả nhất định. Họ có thể viết mã trước. Họ có thể viết test trước. Họ có thể viết xen kẽ mã và test. Nhưng họ sẽ kết thúc với một gói nhỏ có độ phủ test (test coverage) cao và toàn bộ test đều pass.

If you are used to TDD or TCR, you might feel that the bundling discipline is a watered-down compromise. I disagree. It is perhaps a softer ritual; but it can be just as effective. Any discipline can be abused, and any discipline can be honored. If programmers honor this discipline by keeping bundles small and testing every branch and condition, then they will produce a **test suite** that they can trust with their lives. Therefore, it is entirely compatible with writing clean code.

Nếu quen với TDD hay TCR, bạn có thể cảm thấy kỷ luật đóng gói này là một thỏa hiệp bị pha loãng. Tôi không nghĩ vậy. Có lẽ nó là một nghi thức mềm mỏng hơn, nhưng có thể hiệu quả không kém. Kỷ luật nào cũng có thể bị lạm dụng, và kỷ luật nào cũng có thể được tôn trọng. Nếu lập trình viên tôn trọng kỷ luật này bằng cách giữ cho các gói thật nhỏ và test mọi nhánh, mọi điều kiện, thì họ sẽ tạo ra một bộ test (test suite) đáng tin đến mức có thể gửi gắm cả tính mạng. Vậy nên nó hoàn toàn tương thích với việc viết clean code.

Design — Thiết kế

You might **object** that disciplines like these are too tactical because they make no room for strategic design. However, testing disciplines are not intended to supplant strategic thinking. I, for example, am a big believer in hammock-driven development. 5 Thinking up front is vital in all software endeavors—so long as that up-front thinking is brought back to reality on a frequent basis. Months of strategic planning, without any coding, is suicide. Coding without any strategic planning is equally suicidal.

Bạn có thể phản đối rằng những kỷ luật kiểu này quá thiên về chiến thuật, vì chúng chẳng chứa chỗ nào cho thiết kế mang tính chiến lược. Tuy nhiên, các kỷ luật kiểm thử không nhằm thay thế tư duy chiến lược. Chẳng hạn, bản thân tôi rất tin vào hammock-driven development. 5 Suy nghĩ trước là điều sống còn trong mọi công cuộc phần mềm — miễn là suy nghĩ trước ấy thường xuyên được kéo về đối chiếu với thực tế. Lên kế hoạch chiến lược hàng tháng trời mà không viết một dòng mã nào là tự sát. Viết mã mà không có chút kế hoạch chiến lược nào cũng tự sát y như thế.

5 <https://www.youtube.com/watch?v=f84n5oFoZBc>

5 <https://www.youtube.com/watch?v=f84n5oFoZBc>

After an appropriate amount of strategic design, programmers must become tactical. After all, the lines of code have to get written, and writing them is tactical. The testing disciplines described above are tactical disciplines—they augment the tactic of writing code.

Sau một lượng thiết kế chiến lược vừa đủ, lập trình viên phải chuyển sang chiến thuật. Suy cho cùng, những dòng mã vẫn phải được viết ra, và việc viết chúng là chuyện chiến thuật. Các kỷ luật kiểm thử mô tả ở trên là những kỷ luật chiến thuật — chúng hỗ trợ cho chiến thuật viết mã.

Moreover, these disciplines test the design of the code by imposing the viewpoint of users—after all, the tests are a kind of user.

Hơn nữa, các kỷ luật này còn kiểm tra chính thiết kế của mã bằng cách áp đặt góc nhìn của người dùng — vì suy cho cùng, test cũng là một dạng người dùng.

This imposition of viewpoint has led many practitioners to consider these disciplines to be design disciplines as well as coding disciplines. Personally, I find that TDD assists in verifying my lowest-level designs, but it does not help much in higher-level design and **architecture**. I have seen some pretty bad designs implemented with TDD. Some are in this very book.

Việc áp đặt góc nhìn ấy khiến nhiều người thực hành coi các kỷ luật này vừa là kỷ luật viết mã, vừa là kỷ luật thiết kế. Riêng tôi thấy TDD giúp kiểm chứng những thiết kế ở tầng thấp nhất của mình, nhưng chẳng giúp được bao nhiêu ở tầng thiết kế và kiến trúc (architecture) cao hơn. Tôi từng thấy vài thiết kế khá tệ được hiện thực bằng TDD. Có vài cái nằm ngay trong cuốn sách này.

Discipline — Kỷ luật

Disciplines like these should be taken seriously; but they are not universally applicable and should not be followed blindly. For example, I am a pilot; I follow the discipline of using checklists instead of committing many complex procedures to memory. I take that discipline very seriously. There are, however, certain procedures that I must commit to memory because time is of the essence, and the

time it would take to pull out and follow a checklist would be unsafe. Pilots call these procedures memory items, and the learning and refreshing of memory items is yet another discipline.

Những kỷ luật như thế này nên được xem trọng; nhưng chúng không phải lúc nào cũng áp dụng được và không nên tuân theo một cách mù quáng. Ví dụ, tôi là phi công; tôi tuân theo kỷ luật dùng checklist thay vì học thuộc lòng hàng loạt quy trình phức tạp. Tôi rất nghiêm túc với kỷ luật đó. Tuy vậy, có những quy trình tôi buộc phải thuộc nằm lòng, vì thời gian là yếu tố sống còn, và khoảng thời gian bỏ ra để rút checklist rồi làm theo sẽ là không an toàn. Phi công gọi những quy trình đó là memory items, và việc học rồi ôn lại các memory items lại là một kỷ luật khác nữa.

Tedious, Boring, and Slow — Rườm rà, chán ngắt và chậm chạp

Any programmer who is unfamiliar with these disciplines, and who has been a programmer for more than a few years, may find these descriptions horrifying. They may sound tedious, boring, and slow. You might infer that you'd never be able to think through a problem. With TDD and TCR, it even seems likely that you'd never be able to write a little

Bất kỳ lập trình viên nào chưa quen với những kỷ luật này, mà lại đã hành nghề hơn dăm ba năm, hẳn sẽ thấy các mô tả trên thật kinh hoàng. Nghe cứ rườm rà, chán ngắt và chậm chạp. Bạn có thể suy ra rằng mình sẽ chẳng bao giờ nghĩ thấu đáo được một vấn đề. Với TDD và TCR, thậm chí có vẻ như bạn sẽ chẳng bao giờ viết nổi một

while loop or if statement without interrupting yourself. And all three will certainly break your concentration and interrupt your flow. In short, you could conclude that these disciplines are stupid.

vòng lặp while hay một câu lệnh if bé tẹo nào mà không tự cắt ngang chính mình. Và cả ba kỷ luật ấy chắc chắn sẽ phá vỡ sự tập trung và làm đứt mạch làm việc của bạn. Nói ngắn gọn, bạn có thể kết luận rằng những kỷ luật này thật ngớ ngẩn.

But consider:

Nhưng hãy thử ngẫm xem.

Debugging — Gỡ lỗi

Imagine a room full of programmers following one of these disciplines. Walk up to any programmer you like, at any time you like. Everything they are working on executed and passed their tests sometime in the last few minutes. And it doesn't matter who you pick or when you pick them: Everything worked a few minutes ago.

Hãy hình dung một căn phòng đầy lập trình viên đang tuân theo một trong các kỷ luật này. Bạn cứ tiến tới bất kỳ ai bạn thích, vào bất kỳ lúc nào bạn thích. Mọi thứ họ đang làm đều đã chạy và pass test vào một thời điểm nào đó trong vài phút vừa qua. Và chẳng quan trọng bạn chọn ai hay chọn lúc nào: mọi thứ đều đã chạy tốt cách đây vài phút.

What would your life be like if everything worked a few minutes ago—and this was always true? How much debugging would you do?

Cuộc sống của bạn sẽ ra sao nếu mọi thứ đều đã chạy tốt cách đây vài phút — và điều đó luôn luôn đúng? Bạn sẽ phải gỡ lỗi bao nhiêu?

It's hard to believe you would debug much if everything worked so recently. The most probable debugging technique would be to revert back to the last working version and just try again.

Khó tin nổi là bạn sẽ phải gỡ lỗi nhiều, khi mà mọi thứ vừa mới chạy tốt cách đây chốc lát. Kỹ thuật gỡ lỗi khả dĩ nhất chắc là quay ngược về phiên bản chạy được gần nhất rồi thử lại.

Those of us who follow these disciplines have little use for debuggers. Oh, I'll fire one up from time to time. I mean, this is still software, and it's still hard. Weird things still happen. But the time I spend in the debugger is vanishingly small. So small, in fact, that I don't remember the hot keys.

Những ai trong chúng tôi theo các kỷ luật này hầu như chẳng dùng đến debugger. Ồ, thỉnh thoảng tôi vẫn bật nó lên. Ý tôi là, đây vẫn là phần mềm, và nó vẫn khó. Chuyện quái đản vẫn xảy ra. Nhưng thời gian tôi ngồi trong debugger nhỏ đến mức gần như bằng không. Nhỏ tới mức tôi chẳng nhớ nổi phím tắt của nó.

Every time I try to use the debugger I have to reacquaint myself with how to operate it.

Cứ mỗi lần định dùng debugger, tôi lại phải làm quen lại từ đầu xem nó vận hành ra sao.

Following one these disciplines (TDD) has reduced my debugging time by a very significant factor. I contend that any of these disciplines will do the same for every adherent who masters them.

Tuân theo một trong các kỷ luật này (TDD) đã giảm thời gian gỡ lỗi của tôi đi rất đáng kể. Tôi cho rằng bất kỳ kỷ luật nào trong số đó cũng sẽ làm điều tương tự cho mọi người theo đuổi và làm chủ được nó.

Documentation — Tài liệu

Have you ever integrated a third-party package? Perhaps you downloaded a zip file from a website. Upon unzipping, you find the artifacts you need to integrate, but also a PDF manual with instructions to help with the integration. At the end of that manual, there is an ugly appendix with all the code examples. And, of course, that's the first place you go.

Bạn từng tích hợp một package của bên thứ ba chưa? Có lẽ bạn tải một tệp zip từ website nào đó về. Giải nén ra, bạn thấy các artifact cần tích hợp, kèm theo một quyển hướng dẫn PDF chỉ dẫn cách tích hợp. Ở cuối quyển hướng dẫn ấy có một phụ lục xấu xí chứa toàn bộ ví dụ mã. Và dĩ nhiên, đó là chỗ đầu tiên bạn tìm tới.

You go to the code examples because you hope that the example code is current and that it will tell you more truth than the words within the manual. If you are very lucky, you can copy and paste that code from the manual into your project and fiddle it into working.

Bạn tìm tới phần ví dụ mã vì hy vọng rằng mã ví dụ là mã mới nhất và nói cho bạn nhiều sự thật hơn đám chữ nghĩa trong quyển hướng dẫn. Nếu cực kỳ may mắn, bạn có thể copy rồi paste đoạn mã đó từ quyển hướng dẫn vào dự án của mình và mày mò cho nó chạy.

When you follow a testing discipline, you write a series of tiny little tests. Each one of those tests describes how a part of the system works. If you want to know how to create an object, there are tests that create that object every way it can be created. If you need to know how to call a certain API, there are tests that call that API every way it can be called. These tests are the code examples for the whole system.

Khi tuân theo một kỷ luật kiểm thử, bạn viết ra một loạt test bé xíu. Mỗi test trong số đó mô tả cách một phần của hệ thống hoạt động. Nếu bạn muốn biết cách tạo một đối tượng (object), sẽ có những test tạo đối tượng đó theo mọi cách có thể tạo. Nếu bạn cần biết cách gọi một API nào đó, sẽ có những test gọi API đó theo mọi cách có thể gọi. Chính những test này là bộ ví dụ mã cho toàn bộ hệ thống.

What's more, each one of those tests is isolated from all the others. The tests themselves are not a system. Each is understandable in its own right.

Hơn nữa, mỗi test lại tách biệt khỏi tất cả các test khác. Bản thân đám test không phải là một hệ thống. Mỗi test đều tự nó dễ hiểu.

The tests produced are a document that describes the low-level design of the system. And that document is written in a language that you intimately understand. It is utterly unambiguous, it is so formal that it executes, and it cannot get out of sync with the application that it describes. The tests are an almost perfect form of low-level documentation.

Đám test sinh ra chính là một tài liệu mô tả thiết kế tầng thấp của hệ thống. Và tài liệu ấy được viết bằng thứ ngôn ngữ mà bạn hiểu tới tận chân tơ kẽ tóc. Nó tuyệt đối không mập mờ, nó hình thức tới mức chạy được, và nó không thể lệch pha với ứng dụng mà nó mô tả. Test là một dạng tài liệu tầng thấp gần như hoàn hảo.

That's not to say that they are high-level documentation—they are not. But at the lowest level of the system, the tests are about as **good** a document as can be created.

Nói vậy không có nghĩa chúng là tài liệu tầng cao — chúng không phải. Nhưng ở tầng thấp nhất của hệ thống, test gần như là thứ tài liệu tốt nhất có thể tạo ra.

Reliability — Độ tin cậy

Imagine a programmer who does not follow one of these disciplines. He has just finished writing a **module** and has run it through a battery of manual tests. It seems to work just fine. But then the process proctor reminds him that he needs to write unit tests. He feels that this is redundant make-work since he's already tested the module, so his effort is less than stellar.

Hãy hình dung một lập trình viên không theo kỷ luật nào trong số này. Anh ta vừa viết xong một module và đã cho nó chạy qua một loạt test thủ công. Có vẻ nó chạy ngon lành. Nhưng rồi người giám sát quy trình nhắc anh ta rằng anh cần viết unit test. Anh ta thấy đây là việc thừa thãi vì đã test module rồi, nên nỗ lực bỏ ra chẳng có gì xuất sắc.

He writes a few tests here and there, and they pass just fine. But then he encounters the module that's hard to test. It's hard to test because it was not designed to be testable. It is coupled to things that should not be executed in the context of a test.

Anh ta viết dăm ba test chỗ này chỗ kia, và chúng pass ngon lành. Nhưng rồi anh ta đụng phải cái module khó test. Nó khó test vì nó không được thiết kế để test được. Nó bị kết dính (coupling) với những thứ đáng lẽ không nên được thực thi trong ngữ cảnh của một test.

He could redesign that module to make it testable; but that would take a lot of time that he doesn't think he has. And besides, his manual tests proved to him that everything already works. So he walks away without writing that test.

Anh ta có thể thiết kế lại module đó để nó test được; nhưng làm vậy sẽ tốn rất nhiều thời gian mà anh ta cho là mình không có. Vả lại, đám test thủ công đã chứng minh với anh rằng mọi thứ vẫn chạy tốt. Thế là anh ta bỏ đi mà không viết cái test ấy.

Now, if you've been a programmer for more than a few years, you may recognize this scenario. Many of us have written tests after the fact; and many of us have walked away from a difficult test because we didn't want to change a "working" design.

Nếu bạn đã đi làm lập trình được vài năm, hẳn bạn nhận ra kịch bản này. Nhiều người trong chúng ta từng viết test sau khi mọi chuyện đã rồi; và nhiều người trong chúng ta từng bỏ ngang một test khó vì không muốn động vào một thiết kế “đang chạy tốt”.

Unfortunately, this leaves a hole in the test suite. And if you have left holes in the test suite, you know everyone else has left some holes as well. And so everyone knows that the test suite is full of holes.

Đáng tiếc là chuyện đó để lại một lỗ hổng trong bộ test. Và nếu chính bạn từng để lại lỗ hổng trong bộ test, bạn biết rằng ai khác cũng để lại vài lỗ hổng như vậy. Thế là ai cũng biết bộ test thủng lỗ chỗ.

What happens when you run such a test suite and see it pass? Everyone smiles knowingly and no other action is taken. When you know that the test suite is full of holes, there is no safe decision you can make when it passes. All you can do is smile, knowing all too well that the passing of the test suite does not mean that the system works properly.

Điều gì xảy ra khi bạn chạy một bộ test như thế và thấy nó pass? Ai nấy đều mỉm cười đầy hiểu ý và chẳng làm thêm gì cả. Khi đã biết bộ test thủng lỗ chỗ, bạn chẳng thể ra quyết định an toàn nào dựa trên việc nó pass. Tất cả những gì bạn có thể làm là mỉm cười, thừa hiểu rằng bộ test pass không có nghĩa là hệ thống chạy đúng.

When you follow a testing discipline, things are very different. When the tests pass, you know with a high degree of confidence that the system works as it should. And that allows you to make critical decisions.

Khi bạn tuân theo một kỷ luật kiểm thử, mọi chuyện rất khác. Khi test pass, bạn biết với mức độ tin cậy cao rằng hệ thống hoạt động đúng như nó phải hoạt động. Và điều đó cho phép bạn ra những quyết định trọng yếu.

For example, you could decide to deploy the system. I’ve worked on systems like this. When the tests passed, we deployed because we had very high confidence in our test suite.

Chẳng hạn, bạn có thể quyết định triển khai hệ thống. Tôi từng làm trên những hệ thống như vậy. Khi test pass, chúng tôi triển khai luôn, vì chúng tôi rất tin bộ test của mình.

Even if deployment is not an option, the passing of the tests could allow you to move the system into a new stage. For example, you could move it from DEV to QA.

Ngay cả khi triển khai không phải là lựa chọn, việc test pass vẫn có thể cho phép bạn đưa hệ thống sang một giai đoạn mới. Ví dụ, bạn có thể chuyển nó từ DEV sang QA.

Design — Thiết kế

Something else happens when you follow these disciplines. You cannot write the module that’s hard to test. By writing the test along with the code, you are automatically designing every module to be easy to test.

Còn một chuyện nữa xảy ra khi bạn theo các kỷ luật này. Bạn không thể viết ra cái module khó test kia. Bằng cách viết test song song với mã, bạn tự động thiết kế mọi module sao cho dễ test.

A module that is easy to test is decoupled. It is the **decoupling** that makes the module easy to test. So the end result is a less-coupled design. The mere act of writing the tests along with the code improves the overall design of the system.

Một module dễ test là một module đã được tách rời. Chính sự tách rời làm cho module dễ test. Vậy nên kết quả cuối cùng là một thiết kế ít kết dính hơn. Chỉ riêng hành động viết test song song với mã đã cải thiện thiết kế tổng thể của hệ thống.

Reprise — Tóm lại

So, by following these disciplines, you will in all likelihood:

Vậy đây, bằng cách theo các kỷ luật này, nhiều khả năng bạn sẽ:

- Significantly reduce your debugging time.
 - Create a stream of nearly perfect low-level documentation.
 - Create a test suite that you trust so much that you can deploy when it passes.
 - Create a system with a less coupled design.
- Giảm đáng kể thời gian gỡ lỗi.
 - Tạo ra một dòng tài liệu tầng thấp gần như hoàn hảo.
 - Tạo ra một bộ test đáng tin đến mức có thể triển khai ngay khi nó pass.
 - Tạo ra một hệ thống có thiết kế ít kết dính hơn.

And while these four things are good—very good, in fact— they are not the reason that we follow a testing discipline.

Và tuy bốn điều đó đều tốt — thực ra là rất tốt — chúng vẫn không phải lý do khiến chúng ta theo một kỷ luật kiểm thử.

The Angel and the Devil — Thiên thần và ác quỷ

In Chapter 1 , “ Clean Code ,” we discussed the cost of making a **mess**. I described the way that the ever-increasing mess degrades **productivity**. I introduced **the Boy Scout Rule** and told you that keeping the code clean was the only way to prevent that loss.

Ở Chương 1 , “ Clean Code “, chúng ta đã bàn về cái giá của việc tạo ra một mớ hỗn độn. Tôi đã mô tả cách mà mớ hỗn độn cứ ngày một phình to bào mòn năng suất. Tôi đã giới thiệu Quy tắc Hướng đạo sinh (the Boy Scout Rule) và nói với bạn rằng giữ cho mã sạch là cách duy nhất để ngăn tổn thất đó.

So how do you keep the code clean?

Vậy làm sao để giữ cho mã sạch?

Imagine that you are sitting at your computer, scrolling through some code you need to work on. As you scroll, the blood drains from your face and you realize that this code is an undodly mess that is going to be hell to work with.

Hãy hình dung bạn đang ngồi trước máy tính, cuộn qua một đoạn mã mà bạn cần làm việc cùng. Càng cuộn, mặt bạn càng tái đi, và bạn nhận ra đây là một mớ hỗn độn kinh khủng, làm việc với nó chẳng khác nào xuống địa ngục.

Just then, a little angel appears on your right shoulder and whispers in a still small voice: “You could clean it.” But then, in a gout of flame, a devil appears on your left shoulder and screams: “NO! Don’t touch it! If you touch it, you will break it; and if you break it, it will become yours FOREVER. Muahahahahahaha!”

Ngay lúc đó, một thiên thần bé nhỏ hiện ra trên vai phải bạn và thì thầm bằng giọng nhỏ nhẹ: “Bạn có thể dọn dẹp nó mà.” Nhưng rồi, trong một quãng lửa, một con quỷ hiện ra trên vai trái bạn và gào lên: “KHÔNG! Đừng động vào! Động vào là mày làm hỏng nó; mà hỏng rồi thì nó thành của mày VĨNH VIỄN. Mua-ha-ha-ha-ha-ha!”

And so you ignore the angel and back away from the code. You leave it in its current state. You leave it for someone else to deal with. You leave it to fester and to **rot** and to pull down the productivity of the team.

Thế là bạn phớt lờ thiên thần và lùi xa khỏi đoạn mã. Bạn để nguyên nó như hiện trạng. Bạn để đó cho người khác xử lý. Bạn để nó mưng mủ, mục ruỗng và kéo tụt năng suất của cả đội.

This is a fear reaction. You are afraid to clean the code.

Đây là một phản ứng sợ hãi. Bạn sợ phải dọn dẹp mã.

I want you to think very carefully about how wildly unprofessional and irresponsible it is to have allowed the code to get so far out of your control that you are afraid of it. You react to it, rather than it reacting to you. You defer to its demands, rather than forcing it to conform to yours. You, the creator, have lost control of your creation and have become a minion of its whims.

Tôi muốn bạn suy nghĩ thật kỹ xem việc để cho mã tuột khỏi tầm kiểm soát tới mức bạn phải sợ nó là chuyện thiếu chuyên nghiệp và vô trách nhiệm đến nhường nào. Bạn phản ứng theo nó, thay vì nó phản ứng theo bạn. Bạn nhún nhường trước những đòi hỏi của nó, thay vì bắt nó chiều theo đòi hỏi của bạn. Bạn, kẻ sáng tạo, đã mất quyền kiểm soát tạo vật của mình và trở thành tay sai cho những cơn đồng bóng của nó.

And because of that fear, the only thing that can happen to the code in that system is that it must rot. No one dares do the one thing that might reverse that rot—cleaning. And so down it will go, with every month getting worse and worse, and dragging the productivity of the team down with it.

Và vì nỗi sợ đó, điều duy nhất có thể xảy đến với mã trong hệ thống ấy là nó phải mục ruỗng. Chẳng ai dám làm cái việc duy nhất có thể đảo ngược sự mục ruỗng đó — dọn dẹp. Thế là nó cứ trượt dốc, mỗi tháng một tệ hơn, và kéo cả năng suất của đội tụt theo.

And if you think that’s an exaggeration, talk to some of the older programmers you know. They’ll tell you.

Và nếu bạn nghĩ tôi đang nói quá, hãy thử hỏi vài lập trình viên già dặn mà bạn quen. Họ sẽ kể cho bạn nghe.

Muzzling the Devil — Bịt mõm con quỷ

Let’s revisit that scenario again. But this time let’s say that the team has been practicing a good testing discipline and has built up a very good test suite.

Hãy quay lại kịch bản đó lần nữa. Nhưng lần này hãy giả sử cả đội đã thực hành một kỷ luật kiểm thử tử tế và xây dựng được một bộ test rất tốt.

There you are, looking at your screen, scrolling through the code. You see that **bad code** and the blood drains from your face as you stare into the depths of its horror. And, as expected, the little angel whispers in that still small voice: “Clean it!”

Bạn vẫn ngồi đó, nhìn màn hình, cuộn qua đoạn mã. Bạn thấy đám mã tệ hại và mặt bạn tái đi khi nhìn thẳng vào chiều sâu kinh hoàng của nó. Và đúng như dự đoán, thiên thần

bé nhỏ thì thầm bằng giọng nhỏ nhẹ ấy: “Dọn nó đi!”

And the devil does not appear. The devil has no power. Because you have the test suite.

Còn con quỷ thì không xuất hiện. Con quỷ chẳng còn quyền năng gì. Bởi vì bạn có bộ test.

So you make a small change, perhaps just the name of a **variable**. And you run the tests. And the tests pass and you know you haven't broken anything. So you make another small change, perhaps splitting a longer **function** in two. And again, the tests pass. So you move one of those new functions to a different **class**, and the tests fail! So you quickly revert and see the tests pass. Then, you realize your mistake and you move the function again, but this time correctly, and the tests pass.

Thế là bạn thực hiện một thay đổi nhỏ, có lẽ chỉ là tên một biến. Rồi bạn chạy test. Test pass, và bạn biết mình chưa làm hỏng thứ gì. Thế là bạn thực hiện một thay đổi nhỏ nữa, có lẽ là tách một hàm dài thành hai. Và một lần nữa, test pass. Rồi bạn chuyển một trong hai hàm mới đó sang một class khác, và test rớt! Bạn revert ngay lập tức và thấy test pass trở lại. Sau đó bạn nhận ra mình sai ở đâu, chuyển hàm ấy lần nữa nhưng lần này làm đúng, và test pass.

If you have a test suite, and if you trust that test suite, and if that test suite runs quickly, then you will clean the code. You will, because cleaning the code is virtually risk free. And it won't be just you cleaning the code—everyone on the team will clean the code. Everyone on the team will follow the Boy Scout Rule and continuously improve the quality of the code, keeping productivity high.

Nếu bạn có một bộ test, nếu bạn tin bộ test đó, và nếu bộ test đó chạy nhanh, thì bạn sẽ dọn dẹp mã. Bạn sẽ làm, bởi dọn dẹp mã gần như chẳng còn rủi ro gì. Và không chỉ mình bạn dọn — mọi người trong đội đều sẽ dọn dẹp mã. Cả đội sẽ tuân theo Quy tắc Hướng đạo sinh và liên tục cải thiện chất lượng mã, giữ cho năng suất luôn cao.

Complications and Loopholes — Những rắc rối và kẽ hở

If that last discussion sounded too good to be true, it is. Oh, it can be true for a very large fraction of a system; but the devil still hides in the details. There are situations where writing tests is either impossible or at least impractical.

Nếu phần vừa rồi nghe hay tới mức khó tin, thì đúng là vậy thật. Ồ, nó có thể đúng với phần rất lớn của một hệ thống; nhưng con quỷ vẫn còn nấp trong các chi tiết. Có những tình huống mà viết test là bất khả thi, hoặc ít nhất là bất khả thi trên thực tế.

For example, it is impractical, if not downright impossible, to write tests for code that communicates outside the hardware boundary of the machine. For example, unless you somehow use a camera that's staring at the screen, it is impossible to test the final output of a graphical user **interface**. The same holds true for testing the movement of the mouse, or the **data** going in and out over a socket. Automated tests for these things are not quite impossible; but in most cases, writing them is deeply impractical.

Chẳng hạn, viết test cho đoạn mã giao tiếp ra ngoài ranh giới phần cứng của máy là chuyện phi thực tế, nếu không muốn nói là hoàn toàn bất khả thi. Ví dụ, trừ phi bạn xoay xở dùng một cái camera chĩa vào màn hình, bằng không không thể test đầu ra cuối cùng của một giao diện đồ họa. Điều tương tự đúng với việc test chuyển động của chuột, hay dữ liệu đi vào đi ra qua một socket. Test tự động cho những thứ này không hẳn là bất khả thi; nhưng trong đa số trường hợp, viết chúng là chuyện cực kỳ phi thực tế.

Therefore, seasoned developers will design their systems such that those untestable stretches of code are very thin. They will move as much intelligence as possible away from the hardware boundaries

and ensure that the code that does finally touch those boundaries is as anemic as possible.

Vì vậy, những lập trình viên dày dạn sẽ thiết kế hệ thống sao cho các dải mã không test được ấy thật mỏng. Họ sẽ đẩy càng nhiều phần “thông minh” ra xa ranh giới phần cứng càng tốt, và đảm bảo rằng đoạn mã cuối cùng thực sự chạm vào những ranh giới đó càng thiếu máu càng tốt.

This technique is called The Humble Object Pattern . 6 I talk in much more detail about these kinds of situations in my book Clean Craftsmanship .

Kỹ thuật này gọi là The Humble Object Pattern . 6 Tôi bàn kỹ hơn nhiều về những tình huống kiểu này trong cuốn Clean Craftsmanship của mình.

6 <http://xunitpatterns.com/Humble%20Object.html>

6 <http://xunitpatterns.com/Humble%20Object.html>

Another complication can occur when using third-party frameworks. If a framework was not written to be testable, then the code that uses that framework may also be impractical to test. Again, good designers will anticipate this and attempt to push as much intelligence away from the framework as possible and build an isolation layer between the framework and the code that needs to be tested. Care should be taken when committing to frameworks, because they have the potential to prevent very large swathes of code from being properly tested.

Một rắc rối khác có thể xảy ra khi dùng framework của bên thứ ba. Nếu một framework không được viết ra để test được, thì đoạn mã dùng framework đó cũng có thể trở nên phi thực tế để test. Một lần nữa, những người thiết kế giỏi sẽ lường trước điều này và cố đẩy càng nhiều phần thông minh ra xa framework càng tốt, đồng thời dựng một lớp cô lập giữa framework và đoạn mã cần được test. Cần cẩn trọng khi quyết định gắn bó với một framework nào đó, vì chúng hoàn toàn có khả năng khiến những mảng mã rất lớn không thể test tử tế.

Experience Report: I recently wrote a significant application that used the Java Swing framework.

Báo cáo trải nghiệm: Gần đây tôi viết một ứng dụng khá lớn dùng framework Java Swing.

Trying to test any of the code that touched Swing was impractical. I had to separate the application into a tested **component** and an untested (Swing)

Cố test bất kỳ đoạn mã nào chạm tới Swing đều là chuyện phi thực tế. Tôi buộc phải tách ứng dụng thành một component có test và một component (Swing)

component. That untested component grew fairly large because the Swing API forces the construction of large, interconnected data

không có test. Cái component không có test ấy phình khá to, vì Swing API buộc ta phải dựng nên những cấu trúc dữ liệu

structures that contain callbacks. This left my application with ~70% test coverage. I could easily

lớn, chằng chịt liên kết với nhau và chứa đầy callback. Kết cục là ứng dụng của tôi chỉ đạt độ phủ test khoảng 70%. Tôi có thể dễ dàng

refactor the tested component; but touching the Swing component was always very risky. I won't use Swing again.

refactor phần component có test; nhưng động vào component Swing thì luôn rất rủi ro. Tôi sẽ không dùng Swing nữa.

Still another complication occurs when you write a function, but you don't know the correct results of that function. For example, it is hard to test whether a font looks “right,” or whether a field is positioned “correctly,” or whether the format of a report looks “good.” In all of these cases, the results are subjective. The only way to ascertain whether the code is correct is to look at the results and make a judgment. There's no way to write a test for that. So there's no way to write the code and tests together. It's also doubtful that writing a test long after the fact is very helpful, since all it can manage to do is cement a subjective decision.

Còn một rắc rối nữa xảy ra khi bạn viết một hàm nhưng lại không biết kết quả đúng của hàm đó là gì. Ví dụ, khó mà test được một font chữ trông đã “chuẩn” chưa, một trường đã được đặt “đúng chỗ” chưa, hay định dạng của một báo cáo trông đã “đẹp” chưa. Trong tất cả những trường hợp này, kết quả đều mang tính chủ quan. Cách duy nhất để xác định mã có đúng hay không là nhìn vào kết quả rồi phán đoán. Chẳng có cách nào viết test cho chuyện đó. Vậy nên cũng chẳng có cách nào viết mã và test cùng nhau. Cũng khó tin rằng viết test sau đó thật lâu sẽ giúp ích được gì, bởi tất cả những gì nó làm được chỉ là đóng đinh một quyết định chủ quan.

These three cases are the only situations that I have encountered in which tests are impractical. There are, however, likely to be others. For example, how do you test the results of an AI?

Ba trường hợp trên là những tình huống duy nhất tôi từng gặp mà ở đó test là phi thực tế. Tuy nhiên, hẳn còn có những tình huống khác nữa. Chẳng hạn, bạn test kết quả của một AI bằng cách nào?

In the end, when faced with areas of the system that are impractical to test, try to make those areas as small and simple as possible, and protect the rest of the system from them.

Rốt cuộc, khi đối mặt với những vùng của hệ thống mà test là phi thực tế, hãy cố làm cho các vùng đó nhỏ nhất và đơn giản nhất có thể, rồi bảo vệ phần còn lại của hệ thống khỏi chúng.

Costs and Repercussions — Chi phí và hệ lụy

If we follow these disciplines, we will write dozens of tests every day, hundreds of tests every month, and thousands of tests every year. Those tests will cover virtually all of our production code. The sheer bulk of those tests, which can rival the size of the production code itself, can present a daunting management problem.

Nếu tuân theo các kỷ luật này, mỗi ngày chúng ta sẽ viết hàng chục test, mỗi tháng hàng trăm test, và mỗi năm hàng ngàn test. Số test đó sẽ phủ gần như toàn bộ mã production của chúng ta. Riêng khối lượng khổng lồ của đám test ấy, vốn có thể sánh ngang kích thước của chính mã production, đã có thể đặt ra một bài toán quản lý đáng ngại.

Keeping Tests Clean — Giữ cho test luôn sạch

Many years ago I was asked to coach a team who had explicitly decided that their test code should not be maintained to the same standards of quality as their production code. They gave one another license to break all the rules of clean code and good design in their unit tests. “Quick and dirty” were the watchwords. Their variables did not have to be well named; their test functions did not need to be short and descriptive. Their test code did not need to be well designed and thoughtfully partitioned. So long as the test code worked, and so long as it covered the production code, it was good enough.

Nhiều năm trước, tôi được mời huấn luyện một nhóm đã dứt khoát quyết định rằng mã test của họ không cần được giữ ở cùng chuẩn chất lượng với mã production. Họ cho phép nhau phá bỏ mọi quy tắc của clean code và thiết kế tốt khi viết unit test. Khẩu hiệu là “nhanh và bẩn”. Biến của họ không cần đặt tên cho tử tế; hàm test của họ không cần ngắn gọn và mô tả rõ ý. Mã test của họ không cần được thiết kế đàng hoàng và phân chia có suy nghĩ. Chỉ cần mã test chạy được, và chỉ cần nó phủ được mã production, thế là đủ tốt.

Some of you reading this might sympathize with that decision. Perhaps, long in the past, you wrote tests of the kind that I wrote for that Timer class mentioned at the beginning of this chapter. It's a huge step from writing that kind of throwaway test to writing a suite of well-designed, well-partitioned, automated unit tests. So, like the team I was coaching, you might decide that having dirty tests is better than having no tests.

Vài người trong số các bạn đang đọc dòng này có lẽ thấy đồng cảm với quyết định ấy. Có thể đã lâu lắm rồi, bạn từng viết những cái test kiểu như tôi đã viết cho class Timer nhắc tới ở đầu chương này. Từ chỗ viết loại test dùng-một-lần-rồi-vứt đến chỗ viết cả một bộ unit test tự động, được thiết kế tốt và phân chia gọn gàng, là một bước nhảy khổng lồ. Thế nên, giống như nhóm tôi đang huấn luyện, bạn có thể kết luận rằng có test bẩn vẫn hơn là không có test nào.

What that team did not realize was that having dirty tests is equivalent to, if not worse than, having no tests. The problem is that tests must change as the production code evolves. The dirtier the tests, the harder they are to change. The more tangled the test code, the more likely it is that you will spend more time cramming new tests into the suite than it takes to write the new production code. As you modify the production code, old tests start to fail, and the mess in the test code makes it hard to get those tests to pass again. So the tests become viewed as an ever-increasing liability.

Điều nhóm đó không nhận ra là: có test bẩn cũng chẳng khác gì không có test, nếu không muốn nói là còn tệ hơn. Vấn đề nằm ở chỗ test phải thay đổi theo sự tiến hoá của mã production. Test càng bẩn thì càng khó sửa. Mã test càng rối bời thì khả năng bạn tốn nhiều thời gian nhồi nhét test mới vào bộ test hơn cả thời gian viết mã production mới càng cao. Khi bạn chỉnh sửa mã production, các test cũ bắt đầu fail, và mớ hỗn độn trong mã test khiến việc làm cho chúng xanh trở lại trở nên khó khăn. Thế là bộ test dần bị nhìn nhận như một khoản nợ ngày càng phình to.

From release to release the cost of maintaining this team's test suite rose. Eventually, it became the single biggest complaint among the developers. When managers asked why their estimates were getting so large, the developers blamed the tests. In the end, they were forced to discard the test suite entirely.

Qua từng bản phát hành, chi phí bảo trì bộ test của nhóm này cứ thế tăng lên. Cuối cùng, nó trở thành lời than phiền lớn nhất trong đội ngũ lập trình viên. Khi các quản lý hỏi tại sao ước lượng (estimate) của họ lại phình to đến vậy, lập trình viên đổ lỗi cho đám test. Rốt cuộc, họ buộc phải vứt bỏ toàn bộ bộ test.

But, without a test suite, they lost the ability to make sure that changes to their codebase worked as expected. Without a test suite, they could not ensure that changes to one part of their system did not break other parts of their system. So their **defect** rate began to rise. As the number of unintended defects rose, the devil's voice grew louder, and they started to fear making changes. They stopped cleaning their production code because they feared the changes would do more **harm** than good. Their production code began to rot. In the end, they were left with no tests, tangled and bug-riddled production code, frustrated customers, and the feeling that their testing effort had failed them.

Nhưng không còn bộ test, họ mất luôn khả năng bảo đảm rằng những thay đổi trên

codebase vẫn chạy đúng như mong đợi. Không còn bộ test, họ không thể chắc rằng thay đổi ở một phần hệ thống không làm hỏng các phần khác. Thế là tỷ lệ lỗi (defect) bắt đầu leo thang. Số lỗi ngoài ý muốn càng tăng, giọng của con quỷ càng to, và họ bắt đầu sợ phải thay đổi bất cứ thứ gì. Họ ngừng dọn dẹp mã production vì sợ rằng sửa vào chỉ tổ hại nhiều hơn lợi. Mã production của họ bắt đầu mục ruỗng. Cuối cùng, thứ còn lại là: không có test, mã production rồi như canh hẹ và đầy bug, khách hàng bức bối, cùng cảm giác rằng công sức bỏ vào việc kiểm thử đã phản bội họ.

In a way, they were right. Their testing effort had failed them. But it was their decision to allow the tests to be messy that was the seed of that failure. Had they kept their tests clean, their testing effort would not have failed. I can say this with some certainty because I have participated in, and coached, many teams who have been successful with clean unit tests.

Xét ở một khía cạnh, họ nói đúng. Công sức kiểm thử đã phản bội họ thật. Nhưng chính quyết định để cho test bầy hầy mới là hạt giống của thất bại đó. Nếu họ giữ test luôn sạch, công sức kiểm thử đã không phản bội họ. Tôi dám nói điều này với ít nhiều chắc chắn, bởi tôi từng tham gia và huấn luyện rất nhiều nhóm đã thành công với những unit test sạch sẽ.

The moral of the story is simple: Test code is just as important as production code . It is not a second-class citizen. It requires thought, design, and care. It must be kept as clean as production code.

Bài học rút ra từ câu chuyện này rất đơn giản: Mã test quan trọng ngang mã production . Nó không phải công dân hạng hai. Nó đòi hỏi suy nghĩ, thiết kế và chăm chút. Nó phải được giữ sạch như mã production.

Tests Enable the -ilities — Test mở đường cho các “-ilities”

If you don't keep your tests clean, you will lose them. And without them, you lose the very thing that keeps your production code flexible. Yes, you read that correctly. It is tests that keep our code flexible, maintainable, and reusable. The reason is simple. If you have tests, you do not fear improving the code! Without tests, every change is a possible bug. No matter how flexible your architecture is, no matter how nicely partitioned your design is—without tests, you will be reluctant to make changes due to the fear that you will introduce undetected bugs. Without tests, you fear to clean the code.

Nếu không giữ test cho sạch, bạn sẽ mất chúng. Và mất chúng rồi, bạn mất luôn chính thứ giữ cho mã production của mình còn linh hoạt. Vâng, bạn đọc không nhầm đâu. Chính test mới là thứ giữ cho mã của chúng ta linh hoạt, dễ bảo trì và dùng lại được. Lý do rất đơn giản. Có test thì bạn không sợ cải thiện mã! Không có test, mỗi thay đổi đều là một con bug tiềm tàng. Kiến trúc (architecture) của bạn linh hoạt đến đâu, thiết kế của bạn phân chia đẹp đến mấy cũng chẳng ăn thua — không có test, bạn sẽ ngần ngại đụng vào bất cứ thứ gì vì sợ gieo thêm những con bug không ai phát hiện ra. Không có test, bạn sợ dọn dẹp mã.

But with tests, that fear virtually disappears. The higher your test coverage, the lower your fear. You can make changes with near impunity to code that has a less-than-stellar architecture and a tangled and opaque design. Indeed, you can improve that architecture and design without fear!

Nhưng có test rồi thì nỗi sợ đó gần như tan biến. Độ phủ test (test coverage) càng cao, nỗi sợ càng thấp. Bạn gần như có thể sửa thoải mái mà chẳng bị trừng phạt, ngay cả với đoạn mã có kiến trúc chẳng lấy gì làm rực rỡ và thiết kế thì rối rắm, tối mù. Thật vậy, bạn còn có thể cải thiện chính cái kiến trúc và thiết kế đó mà không hề sợ hãi!

So, having an automated suite of tests that cover the production code is the key to keeping your design and architecture as clean as possible. Tests enable all **the -ilities**, because tests enable change.

Vậy nên, có một bộ test tự động phủ được mã production chính là chìa khoá để giữ cho thiết kế và kiến trúc của bạn sạch nhất có thể. Test mở đường cho mọi “-ilities”, bởi test mở đường cho sự thay đổi.

So, if your tests are dirty, then your ability to change your code is hampered, and you begin to lose the ability to improve the **structure** of that code. The dirtier your tests are, the dirtier your code becomes. Eventually, you lose the tests, and your code rots.

Ngược lại, nếu test của bạn bẩn thì khả năng thay đổi mã bị cản trở, và bạn bắt đầu đánh mất khả năng cải thiện cấu trúc của mã đó. Test càng bẩn, mã của bạn càng bẩn. Đến một lúc nào đó, bạn mất luôn đám test, và mã của bạn mục ruỗng.

Chapter 15. Clean Tests — Chương 15. Test sạch



Clean tests are readable, fast, isolated, repeatable, self-verifying, timely, and designed. Readability is perhaps even more important in unit tests than it is in production code.

Test sạch thì dễ đọc, nhanh, cô lập, lặp lại được, tự kiểm chứng, đúng lúc, và có thiết kế hẳn hoi. Tính dễ đọc trong unit test có lẽ còn quan trọng hơn cả trong mã production.

What makes tests readable? The same things that make all code readable: clarity, simplicity, and density of expression. In a **test**, you want to say a lot with as few expressions as reasonable.

Điều gì làm cho test dễ đọc? Vẫn là những thứ làm cho mọi đoạn mã dễ đọc: rõ ràng, đơn giản, và cô đọng trong cách diễn đạt. Trong một test, bạn muốn nói thật nhiều bằng số biểu thức ít nhất có thể chấp nhận được.

Consider this code from FitNesse. These three tests are difficult to understand and can certainly be improved. First, there is a terrible amount of duplicate code in the repeated calls to `addPage` and `assertSubString`. More importantly, this code is just loaded with details that interfere with the expressiveness of the test.

Hãy xem đoạn mã sau lấy từ FitNesse. Ba test này rất khó hiểu và chắc chắn còn cải thiện được. Trước hết, có một lượng mã trùng lặp (duplication) kinh khủng trong những lời gọi lặp đi lặp lại tới addPage và assertSubString. Quan trọng hơn, đoạn mã này chất đầy những chi tiết cản trở khả năng biểu đạt của test.

```
public void testGetPageHierarchyAsXml() throws Exception {
    crawler.addPage(root, PathParser.parse("PageOne"));
    crawler.addPage(root, PathParser.parse("PageOne.ChildOne"));
    crawler.addPage(root, PathParser.parse("PageTwo"));
    request.setResource("root");
    request.addInput("type", "pages");
    Responder responder = new SerializedPageResponder();
    SimpleResponse response =
        (SimpleResponse) responder.makeResponse(
            new FitNesseContext(root), request);
    String xml = response.getContent();
    assertEquals("text/xml", response.getContentType());
    assertSubString("<name>PageOne</name>", xml);
    assertSubString("<name>PageTwo</name>", xml);
    assertSubString("<name>ChildOne</name>", xml);
}

public void testGetPageHierarchyAsXmlDoesntContainSymbolicLinks
throws Exception {
    WikiPage pageOne =
        crawler.addPage(root, PathParser.parse("PageOne"));
    crawler.addPage(root, PathParser.parse("PageOne.ChildOne"));
    crawler.addPage(root, PathParser.parse("PageTwo"));
    PageData data = pageOne.getData();
    WikiPageProperties properties = data.getProperties();
    WikiPageProperty symLinks =
        properties.set(SymbolicPage.PROPERTY_NAME);
    symLinks.set("SymPage", "PageTwo");
    pageOne.commit(data);
    request.setResource("root");
    request.addInput("type", "pages");
    Responder responder = new SerializedPageResponder();
    SimpleResponse response =
        (SimpleResponse) responder.makeResponse(
            new FitNesseContext(root), request);
    String xml = response.getContent();
    assertEquals("text/xml", response.getContentType());
    assertSubString("<name>PageOne</name>", xml);
    assertSubString("<name>PageTwo</name>", xml);
    assertSubString("<name>ChildOne</name>", xml);
    assertNotSubString("SymPage", xml);
}

public void testGetDataAsHtml() throws Exception {
    crawler.addPage(root, PathParser.parse("TestPageOne"), "test");
    request.setResource("TestPageOne");
    request.addInput("type", "data");
}
```

```

Responder responder = new SerializedPageResponder();
SimpleResponse response =
    (SimpleResponse) responder.makeResponse(
        new FitNesseContext(root), request);
String xml = response.getContent();
assertEquals("text/xml", response.getContentType());
assertSubString("test page", xml);
assertSubString("<Test", xml);
}

```

For example, look at the PathParser calls. They transform strings into PagePath instances used by the crawlers. This transformation is completely irrelevant to the test at hand and serves only to obfuscate the intent. The details surrounding the creation of the responder and the gathering and casting of the response are also just noise. Then there's the ham-handed way that the request URL is built from a resource and an **argument**. (I helped write this code 20 years ago, so I feel free to roundly criticize it.)

Chẳng hạn, hãy nhìn các lời gọi PathParser. Chúng biến chuỗi thành các instance PagePath mà crawler sử dụng. Phép biến đổi này hoàn toàn chẳng liên quan gì tới test đang bàn, nó chỉ có tác dụng làm mờ ý định đi mà thôi. Đám chi tiết quanh việc tạo responder rồi thu thập và ép kiểu response cũng chỉ là nhiễu. Rồi còn cái kiểu vụng về mà URL của request được ghép lại từ một resource và một argument nữa. (Hai mươi năm trước tôi cũng góp tay viết đoạn mã này, nên tôi cứ thoải mái phê phán nó cho đã.)

In the end, this code was not designed to be read. The poor reader is inundated with a swarm of details that must be understood before the tests make any real sense.

Rốt cuộc, đoạn mã này không hề được thiết kế để đọc. Người đọc tội nghiệp bị chìm trong một đám chi tiết mà họ phải hiểu hết thì mấy cái test mới bắt đầu có nghĩa.

Now consider the improved tests below. These tests do the exact same thing, but they have been refactored into a much cleaner and more explanatory form.

Bây giờ hãy xem phiên bản test đã cải thiện dưới đây. Chúng làm y hệt những việc như trên, nhưng đã được refactor thành một dạng sạch sẽ và dễ hiểu hơn nhiều.

```

public void testGetPageHierarchyAsXml() throws Exception {
    makePages("PageOne", "PageOne.ChildOne", "PageTwo");
    submitRequest("root", "type:pages");
    assertResponseIsXML();
    assertResponseContains(
        "<name>PageOne</name>",
        "<name>PageTwo</name>",
        "<name>ChildOne</name>");
}

public void
testSymbolicLinksAreNotInXmlPageHierarchy() throws Exception {
    WikiPage page = makePage("PageOne");
    makePages("PageOne.ChildOne", "PageTwo");
    addLinkTo(page, "PageTwo", "SymPage");
    submitRequest("root", "type:pages");
    assertResponseIsXML();
    assertResponseContains(
        "<name>PageOne</name>",

```

```

        "<name>PageTwo</name>",
        "<name>ChildOne</name>"
    );
    assertResponseDoesNotContain( "SymPage" );
}
public void testGetDataAsXml() throws Exception {
    makePageWithContent( "TestPageOne", "test page" );
    submitRequest( "TestPageOne", "type:data" );
    assertResponseIsXML();
    assertResponseContains( "test page", "<Test" );
}

```

The Arrange/Act/Assert (AAA) 1 pattern is made obvious by the **structure** of these tests. Each of the tests is clearly split into three parts. The first part (Arrange) builds up the test **data**, the second part (Act) operates on that test data, and the third part (Assert) checks that the operation yielded the expected results.

Mẫu Arrange/Act/Assert (AAA) 1 hiện lên rõ mồn một qua cấu trúc của những test này. Mỗi test được chia rành mạch thành ba phần. Phần đầu (Arrange) dựng dữ liệu test, phần thứ hai (Act) thao tác trên dữ liệu đó, và phần thứ ba (Assert) kiểm tra xem thao tác có cho ra kết quả như mong đợi hay không.

1 <https://automationpanda.com/2020/07/07/arrange-act-assert-a-pattern-for-writing-good-tests/>

1 <https://automationpanda.com/2020/07/07/arrange-act-assert-a-pattern-for-writing-good-tests/>

Notice that the vast majority of annoying detail has been eliminated. The tests get right to the point and use only the data types and functions that they truly need. Anyone who reads these tests should be able to work out what they do very quickly, without being misled or overwhelmed by details.

Để ý rằng phần lớn đám chi tiết phiền nhiễu đã bị loại bỏ. Các test đi thẳng vào vấn đề và chỉ dùng đúng những kiểu dữ liệu cùng những hàm chúng thật sự cần. Bất kỳ ai đọc mấy test này cũng có thể hình dung rất nhanh chúng làm gì, mà không bị chi tiết đánh lạc hướng hay nhấn chìm.

Domain-Specific Testing Language — Ngôn ngữ kiểm thử chuyên biệt miền

The tests above demonstrate the technique of building a domain-specific language for your tests. Rather than using the APIs that programmers use to manipulate the system, we build up a set of functions and utilities that make use of those APIs and that make the tests more convenient to write and easier to read. These functions and utilities become a specialized API used by the tests. They are a testing language that programmers use to help them write their tests and to help those who must read those tests later on.

Những test ở trên minh họa kỹ thuật xây dựng một ngôn ngữ chuyên biệt miền dành riêng cho test của bạn. Thay vì dùng chính những API mà lập trình viên vẫn dùng để điều khiển hệ thống, ta dựng lên một tập các hàm và tiện ích sử dụng những API đó, sao cho test viết ra tiện hơn và đọc vào dễ hơn. Những hàm và tiện ích ấy trở thành một API chuyên dụng cho test. Chúng là một ngôn ngữ kiểm thử mà lập trình viên dùng để tự giúp mình viết test, và giúp cả những người sau này phải đọc lại đồng test đó.

Composed Assertions — Composed assertion

The functions `assertResponseIsXML` and `assertResponseContains` are part of that **domain-specific testing language**. These kinds of **assertion** functions are called composed assertions .

một phần của ngôn ngữ kiểm thử chuyên biệt miền ấy. Những hàm assertion kiểu này được gọi là composed assertion .

Testing APIs like these are not designed up front; rather, they evolve from the continued **refactoring** of test code that has gotten too tainted by obfuscating detail. Just as you saw me **refactor** the original tests, so too will disciplined developers refactor their test code into more succinct and expressive forms.

Các API phục vụ test như thế này không được thiết kế sẵn từ đầu; đúng hơn, chúng tiến hoá dần từ việc liên tục refactor mã test đã bị chi tiết rườm rà làm cho vẩn đục. Y như bạn vừa thấy tôi refactor những test ban đầu, các lập trình viên có kỷ luật (discipline) cũng sẽ refactor mã test của họ thành những dạng ngắn gọn và giàu sức biểu đạt hơn.

Composed Test Results — Composed test result

Another technique for creating a domain-specific testing API is to create composed test results. These are used when the test results are needlessly complex and can be compressed down into a smaller form that is easy to understand.

Một kỹ thuật khác để tạo ra API kiểm thử chuyên biệt miền là tạo ra các composed test result — kết quả test được gộp lại. Chúng dùng đến khi kết quả test phức tạp một cách không cần thiết và có thể nén xuống thành một dạng nhỏ gọn, dễ hiểu.

Consider the tests I wrote as part of an environment control system I was prototyping. Without going into the details, you can tell that this test checks that the low-temperature alarm, the heater, and the blower are all turned on when the temperature is “way too cold.”

Hãy xem những test tôi viết khi làm bản prototype cho một hệ thống điều khiển môi trường. Không cần đi vào chi tiết, bạn vẫn có thể nhận ra rằng test này kiểm tra xem báo động nhiệt độ thấp, bộ sưởi và quạt thổi có được bật hết lên hay không khi nhiệt độ ở mức “lạnh quá thế”.

```
@
public void turnOnLoTempAlarmAtThreshold() throws Exception {
    hw.setTemp(WAY_TOO_COLD);
    controller.tic();
    assertTrue(hw.heaterState());
    assertTrue(hw.blowerState());
    assertFalse(hw.coolerState());
    assertFalse(hw.hiTempAlarm());
    assertTrue(hw.loTempAlarm());
}
```

There are, of course, lots of details here. For example, what is that `tic` **function** all about? In fact, I'd rather you not worry about that while reading this test. I'd rather you just worry about whether you agree that the end state of the system is consistent with the temperature being “way too cold.”

Đương nhiên ở đây có cả đồng chi tiết. Chẳng hạn, cái hàm tic kia rốt cuộc là gì vậy? Thật ra, khi đọc test này tôi mong bạn đừng bận tâm tới nó. Tôi chỉ mong bạn bận tâm xem mình có đồng ý rằng trạng thái cuối cùng của hệ thống là hợp lý với việc nhiệt độ đang “lạnh quá thế” hay không.

Notice, as you read the test, that your eye needs to bounce back and forth between the name of the state being checked and the sense of the state being checked. You see

Để ý mà xem, khi đọc test này, mắt bạn phải nhảy qua nhảy lại giữa tên của trạng thái được kiểm tra và sắc thái của trạng thái được kiểm tra. Bạn nhìn thấy

heaterState , and then your eyes glissade left to assertTrue . You see coolerState, and your eyes must track left to assertFalse . This is tedious and unreliable. It makes the test hard to read.

heaterState , rồi mắt bạn phải trượt sang trái tới assertTrue . Bạn nhìn thấy coolerState, rồi mắt lại phải dò sang trái tới assertFalse . Kiểu này vừa mệt vừa dễ sai. Nó khiến test khó đọc.

To make matters worse, this is one of 14 very similar tests. Imagine how your eyes will cross as you try to read all that.

Tệ hơn nữa, đây mới chỉ là một trong 14 test na ná nhau. Cứ tưởng tượng mắt bạn sẽ đảo đến mức nào khi phải đọc hết chùng ấy.

I improved the reading of these tests greatly by transforming them as follows.

Tôi đã cải thiện đáng kể khả năng đọc của mấy test này bằng cách biến đổi chúng như sau.

```
@Test
public void turnOnLoTempAlarmAtThreshold() throws Exception {
    wayTooCold();
    assertEquals("HBchL", hw.getState());
}
```

Of course, I hid the detail of the tic function by creating a

Đương nhiên, tôi đã giấu chi tiết của hàm tic đi bằng cách tạo ra hàm

wayTooCold function. But the thing to note is the strange string in the assertEquals . Uppercase means “on,” lowercase means “off,” and the letters are always in the following order: {heater, blower, cooler, hi-temp-alarm, lo-temp-alarm}.

wayTooCold . Nhưng thứ đáng chú ý ở đây là cái chuỗi kỳ quặc trong assertEquals . Chữ hoa nghĩa là “bật”, chữ thường nghĩa là “tắt”, và các chữ cái luôn theo đúng thứ tự sau: {heater, blower, cooler, hi-temp-alarm, lo-temp-alarm}.

Once you know the meaning, your eyes glide across that string and you can quickly interpret the results. Reading the test becomes almost a pleasure. Just take a look at a few of these tests and see how easy they are to comprehend.

Một khi đã biết ý nghĩa của nó, mắt bạn lướt qua chuỗi ấy là hiểu ngay kết quả. Đọc test bỗng trở thành một thú vui. Cứ thử liếc qua vài test dưới đây mà xem chúng dễ hiểu tới mức nào.

```
@Test
public void turnOnCoolerAndBlowerIfTooHot() throws Exception {
    tooHot();
    assertEquals("hBChl", hw.getState());
}
```

```

@Test
public void turnOnHeaterAndBlowerIfTooCold() throws Exception {
    tooCold();
    assertEquals("HBchl", hw.getState());
}
@Test
public void turnOnHiTempAlarmAtThreshold() throws Exception {
    wayTooHot();
    assertEquals("hBCHl", hw.getState());
}
@Test
public void turnOnLoTempAlarmAtThreshold() throws Exception {
    wayTooCold();
    assertEquals("HBchL", hw.getState());
}

```

The `getState` function that creates the composed test result is shown below.

Hàm `getState` tạo ra composed test result được trình bày dưới đây.

```

public String getState() {
    String state = "";
    state += heater ? "H" : "h";
    state += blower ? "B" : "b";
    state += cooler ? "C" : "c";
    state += hiTempAlarm ? "H" : "h";
    state += loTempAlarm ? "L" : "l";
    return state;
}

```

Dual Standard — Hai chuẩn mực khác nhau

You might complain that composing the test result in this fashion is wasteful of computing resources. After all, I didn't even use a `StringBuffer` or `StringBuilder` to do the concatenation.

Có thể bạn sẽ phàn nàn rằng gộp kết quả test kiểu này là phí phạm tài nguyên tính toán. Nói cho cùng, tôi thậm chí còn chẳng thèm dùng `StringBuffer` hay `StringBuilder` để nối chuỗi.

This application is clearly an embedded real-time system, and it is likely that computer and memory resources are very constrained. However, the test environment runs in my laptop and is not constrained at all.

Ứng dụng này rõ ràng là một hệ thống nhúng thời gian thực, và nhiều khả năng tài nguyên máy tính lẫn bộ nhớ đều rất eo hẹp. Nhưng môi trường test thì chạy trên laptop của tôi, chẳng bị bó buộc gì cả.

That is the nature of the dual standard between test and production. There are things that you might never do in a production environment that are perfectly fine in a test environment. Usually they involve issues of memory or CPU efficiency. But they never involve issues of cleanliness.

Đó chính là bản chất của hai chuẩn mực khác nhau giữa test và production. Có những thứ bạn sẽ không bao giờ làm trong môi trường production nhưng lại hoàn toàn ổn trong môi

trường test. Chúng thường liên quan tới hiệu quả bộ nhớ hoặc CPU. Nhưng chúng không bao giờ liên quan tới chuyện sạch sẽ.

The Single Assert Rule — Quy tắc một assert duy nhất

This rule states that a good test asserts one and only one thing. This is often misunderstood to mean that a test should have only a single assertion statement. In reality, a test can have many assertion statements and still assert one logical fact.

Quy tắc này nói rằng một test tốt thì khẳng định một và chỉ một điều. Người ta thường hiểu nhầm nó thành: một test chỉ được có đúng một câu lệnh assertion. Thực tế, một test có thể có nhiều câu lệnh assertion mà vẫn chỉ khẳng định một sự thật logic duy nhất.

For example, consider the first set of environment controller tests above. They may be hard to read with all those assertions, but they are asserting a single logical fact. Indeed, that is why we could refactor them with composed test results. Or, consider the refactored assertions in in the

Chẳng hạn, hãy xem lại nhóm test đầu tiên của bộ điều khiển môi trường ở trên. Chúng có thể khó đọc vì chất đầy assertion, nhưng chúng đang khẳng định một sự thật logic duy nhất. Thật ra, chính vì vậy mà ta mới refactor được chúng bằng composed test result. Hoặc hãy xem những assertion đã refactor trong ví dụ

pageHierarchy example above. They are asserting the single logical fact that the response is an XML document that has certain contents.

pageHierarchy ở trên. Chúng đang khẳng định một sự thật logic duy nhất: rằng response là một tài liệu XML có những nội dung nhất định.

You might find this confusing, or at least subjective. After all, the contents of the document and the type of the document are different facts, aren't they?

Có thể bạn thấy điều này khó hiểu, hoặc ít nhất là mang tính chủ quan. Nói cho cùng, nội dung của tài liệu và kiểu của tài liệu là hai sự thật khác nhau mà, đúng không?

The confusion arises because the Single Assert Rule is poorly named.

Sự lẩn cẩn đó nảy sinh vì Quy tắc một assert duy nhất bị đặt tên dở.

The Single Act Rule — Quy tắc một act duy nhất

The rule should really be called the Single Act Rule. The Arrange-Act-Assert pattern says that every test should be composed of three elements: the arrangement, the action, and the assertion. The first arranges the data to be tested. The second is the action upon that data that we wish to test. The third is the assertion that the data has been properly transformed by the action.

Đúng ra quy tắc ấy phải được gọi là Quy tắc một act duy nhất. Mẫu Arrange-Act-Assert nói rằng mọi test đều nên gồm ba thành phần: phần sắp đặt, phần hành động, và phần khẳng định. Phần đầu sắp đặt dữ liệu cần kiểm thử. Phần thứ hai là hành động tác động lên dữ liệu đó — thứ ta muốn kiểm thử. Phần thứ ba là lời khẳng định rằng dữ liệu đã được hành động kia biến đổi đúng như mong muốn.

The goal of the Single Assert Rule is to make sure that each test is testing one and only one action. We want to avoid tests that arrange, act, assert, act, assert. We also want to avoid tests that arrange,

act, act, assert, assert. We want every action tested individually because we don't want downstream assertions to be corrupted by upstream actions. We want every test to stand alone.

Mục tiêu của Quy tắc một assert duy nhất là bảo đảm mỗi test chỉ kiểm thử một và chỉ một hành động. Ta muốn tránh những test kiểu sắp đặt, hành động, khẳng định, rồi lại hành động, lại khẳng định. Ta cũng muốn tránh những test kiểu sắp đặt, hành động, hành động, khẳng định, khẳng định. Ta muốn mọi hành động đều được kiểm thử riêng rẽ, bởi ta không muốn những khẳng định phía sau bị các hành động phía trước làm nhiễu. Ta muốn mỗi test đều đứng độc lập được.

F.I.R.S.T. — F.I.R.S.T.

Here is a simple memory aid for the desired characteristics of tests.

Đây là một mẹo ghi nhớ đơn giản cho những đặc tính mà ta mong muốn ở test.

Fast — Fast — Nhanh

Tests should be fast. They should run quickly. When tests run slow, you won't want to run them frequently. If you don't run them frequently, you won't find problems early enough to fix them easily. You won't feel as free to clean up the code. Eventually the code will begin to **rot**. So consider the speed of the tests to be a design imperative. Design your tests to be fast. Very fast.

Test phải nhanh. Chúng phải chạy vào một cái là xong. Khi test chạy chậm, bạn sẽ chẳng buồn chạy chúng thường xuyên. Mà không chạy thường xuyên thì bạn sẽ không phát hiện vấn đề đủ sớm để sửa cho dễ. Bạn cũng sẽ không còn thấy thoải mái mà dọn dẹp mã nữa. Rốt cuộc, mã sẽ bắt đầu mục ruỗng. Vậy nên hãy coi tốc độ của test là một mệnh lệnh thiết kế. Hãy thiết kế test của bạn cho nhanh vào. Thật nhanh.

Isolated — Isolated — Cô lập

This is a restatement of the Single Act Rule. Tests should not depend on each other. One test should not set up the conditions for the next test. You should be able to run each test independently and run the tests in any order you like. When tests depend on each other, the first one to fail causes a cascade of downstream failures, making diagnosis difficult and hiding downstream defects.

Đây chính là Quy tắc một act duy nhất nói lại theo cách khác. Các test không được phụ thuộc lẫn nhau. Test này không được dựng sẵn điều kiện cho test kia. Bạn phải có thể chạy từng test một cách độc lập và chạy chúng theo bất kỳ thứ tự nào bạn thích. Khi các test phụ thuộc nhau, test đầu tiên hỏng sẽ kéo theo cả một dây chuyền hỏng phía sau, khiến việc chẩn đoán trở nên khó khăn và che giấu luôn những lỗi (defect) ở phía sau đó.

Repeatable — Repeatable — Lặp lại được

Tests should be repeatable in any environment. You should be able to run the tests in the production environment, in the QA environment, and on your laptop while riding home on the train without a network. If your tests aren't repeatable in any environment, then you'll always have an excuse for why they fail. You'll also find yourself unable to run the tests when the environment isn't available.

Test phải lặp lại được trong mọi môi trường. Bạn phải chạy được test trong môi trường production, trong môi trường QA, và cả trên laptop của mình khi đang ngồi tàu về nhà mà không có mạng. Nếu test của bạn không lặp lại được ở mọi môi trường, thì bạn sẽ luôn có sẵn một cái cớ để giải thích vì sao chúng hỏng. Bạn cũng sẽ rơi vào cảnh không chạy nổi test khi môi trường không sẵn sàng.

Self-Validating — Self-Validating — Tự kiểm chứng

The tests should have a boolean output. Either they pass or they fail. You should not have to read through a log file to tell whether the tests pass. You should not have to manually compare two different text files to see whether the tests pass. If the tests aren't self-validating, then failure can become subjective and running the tests can require a long manual evaluation.

Test phải cho ra kết quả boolean. Hoặc là pass, hoặc là fail. Bạn không phải lần mò đọc file log mới biết test có pass hay không. Bạn không phải ngồi so sánh thủ công hai file văn bản để xem test có pass hay không. Nếu test không tự kiểm chứng được, thì việc hỏng hay không hỏng sẽ trở thành chuyện cảm tính, và chạy test sẽ kéo theo cả một màn đánh giá thủ công dài dòng.

Timely — Timely — Đúng lúc

The tests need to be written in a timely fashion. Unit tests should be written along with the production code that makes them pass. If you write tests long after the production code, then you may find the production code to be too hard to test. Timely tests cause the design of the production code to be testable.

Test cần được viết đúng lúc. Unit test nên được viết song song với đoạn mã production làm cho chúng pass. Nếu bạn viết test rất lâu sau khi đã viết mã production, bạn có thể sẽ thấy mã production ấy quá khó để kiểm thử. Test viết đúng lúc sẽ khiến thiết kế của mã production trở nên kiểm thử được.

Test Design — Thiết kế test

As we saw in Chapter 9 , “The Clean **Method**,” tests need to be designed to reduce their **coupling** to the production code. This makes sure that changes to the requirements affect as few tests as possible.

Như ta đã thấy ở Chương 9 , “Method sạch”, test cần được thiết kế sao cho giảm được độ kết dính giữa các thành phần (coupling) của chúng với mã production. Điều này bảo đảm rằng thay đổi trong yêu cầu sẽ ảnh hưởng tới càng ít test càng tốt.

Suppose you are working on a large system composed of hundreds of modules. Suppose a single change to that system forces you to make changes to a majority of those modules. That system is poorly designed—by definition.

Giả sử bạn đang làm việc trên một hệ thống lớn gồm hàng trăm module. Giả sử một thay đổi duy nhất trong hệ thống đó buộc bạn phải sửa phần lớn số module ấy. Hệ thống đó — theo đúng định nghĩa — là hệ thống được thiết kế tồi.

The principles and rules of software design, when carefully followed, prevent—or at least mitigate—that nightmare scenario.

Các nguyên tắc và quy tắc thiết kế phần mềm, nếu được tuân thủ cẩn thận, sẽ ngăn chặn — hoặc chí ít cũng làm dịu bớt — cơn ác mộng đó.

Now imagine you have a well-designed system with hundreds of modules and a **test suite** with thousands of unit tests. Suppose a single change to that system breaks hundreds of those tests. That is a very poorly designed suite of tests.

Giờ hãy hình dung bạn có một hệ thống được thiết kế tốt với hàng trăm module và một bộ test (test suite) gồm hàng nghìn unit test. Giả sử một thay đổi duy nhất trong hệ thống làm hỏng hàng trăm test trong số đó. Đó là một bộ test được thiết kế cực kỳ tồi.

The principles and rules of software design apply to the tests as much as they apply to the production code. Coupling must be minimized. One change to the signature of a function in the production code must not cause hundreds of tests to be altered, just as it must not cause hundreds of other modules to be altered.

Các nguyên tắc và quy tắc thiết kế phần mềm áp dụng cho test y như áp dụng cho mã production. Coupling phải được giảm tới mức tối thiểu. Một thay đổi trong signature của một hàm ở mã production không được phép kéo theo việc phải sửa hàng trăm test, cũng như nó không được phép kéo theo việc phải sửa hàng trăm module khác.

Conclusion — Kết luận

We have barely scratched the surface of this topic. My book Clean **Craftsmanship** goes very deep into the topic of clean and well-designed tests. Tests are as important to the health of a project as the production code is. Perhaps they are even more important, because tests preserve and enhance the flexibility, maintainability, and reusability of the production code. So keep your tests constantly clean and well designed. Work to make them expressive and succinct. Invent testing APIs that act as domain-specific languages that help you write the tests.

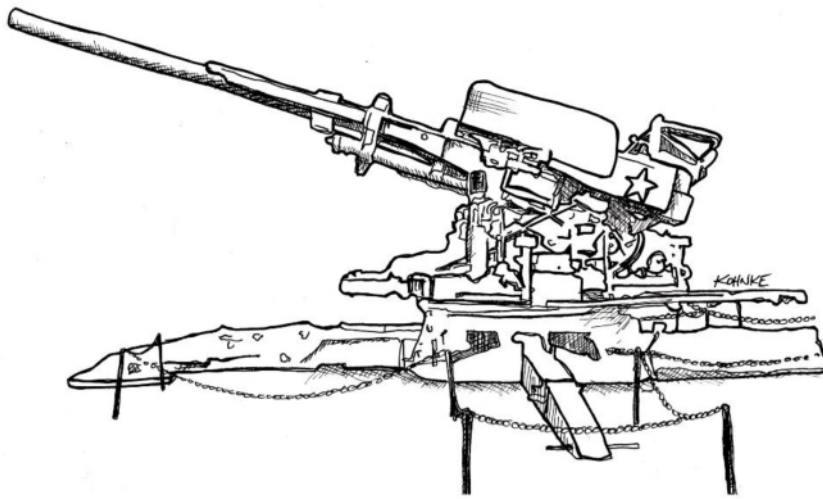
Chúng ta mới chỉ chạm nhẹ vào bề mặt của chủ đề này. Cuốn Clean Craftsmanship của tôi đào rất sâu vào chuyện test sạch và test được thiết kế tốt. Test quan trọng với sức khỏe của một dự án chẳng kém gì mã production. Có lẽ chúng còn quan trọng hơn, bởi test là thứ gìn giữ và nâng cao tính linh hoạt, khả năng bảo trì và khả năng tái sử dụng của mã production. Vậy nên hãy giữ cho test của bạn luôn sạch và được thiết kế tử tế. Hãy gắng làm cho chúng giàu sức biểu đạt và cô đọng. Hãy phát minh ra những API kiểm thử đóng vai trò như các ngôn ngữ chuyên biệt miền giúp bạn viết test.

If you let the tests rot, then your code will rot too. Maintain your disciplines and keep your tests clean.

Nếu bạn để test mục ruỗng, thì mã của bạn cũng sẽ mục ruỗng theo. Hãy giữ vững kỷ luật và giữ cho test của bạn luôn sạch.

Chapter 16. Acceptance Testing —

Chương 16. Acceptance Testing



Of all the disciplines of **clean code**, this is the one that programmers have the least control over. Fulfilling this **discipline** requires the participation of the business. Unfortunately, many businesses have, so far, proven unwilling to properly engage.

Trong tất cả các kỷ luật (discipline) của clean code, đây là kỷ luật mà lập trình viên ít quyền kiểm soát nhất. Muốn thực hiện được nó thì phải có sự tham gia của phía kinh doanh. Đáng tiếc là cho tới nay, nhiều doanh nghiệp vẫn tỏ ra không sẵn lòng nhập cuộc một cách nghiêm túc.

How do you know when a system is ready to deploy? Organizations around the world frequently make this decision by engaging a QA department or group to “bless” the deployment. Typically, this means that the QA folks run a rather large battery of manual tests that walk through the various behaviors of the system until they are convinced that the system behaves as specified. When those tests “pass,” the system may be deployed.

Làm sao bạn biết một hệ thống đã sẵn sàng để triển khai? Các tổ chức trên khắp thế giới thường ra quyết định này bằng cách nhờ một bộ phận hoặc một nhóm QA “ban phước” cho đợt triển khai. Thông thường, điều đó có nghĩa là đám QA chạy một loạt khá đồ sộ các test thủ công, rà qua đủ mọi hành vi của hệ thống cho tới khi họ tin rằng hệ thống hoạt động đúng như đặc tả. Khi những test đó “pass”, hệ thống có thể được triển khai.

This means that the true requirements of the system are those tests. It does not matter what the requirements document says, it is only the tests that matter. If QA signs off after running their tests,

the system is deployed. Therefore, it is those tests that are the requirements.

Điều này có nghĩa là yêu cầu thực sự của hệ thống chính là những test đó. Tài liệu yêu cầu viết gì không quan trọng, chỉ có các test mới quan trọng. Nếu QA ký duyệt sau khi chạy xong test của họ, hệ thống được triển khai. Vậy nên chính những test đó mới là yêu cầu.

The Acceptance Testing Discipline — Kỷ luật acceptance testing

The discipline of acceptance testing recognizes this simple fact and recommends that all requirements be specified as tests. Those tests should be written by business analysts (BA) and QA on a feature-by-feature basis, shortly before each feature is implemented. QA is not responsible for running those tests. Rather, that task is left to the programmers; and therefore, the programmers will very likely automate those tests.

Kỷ luật acceptance testing (kiểm thử chấp nhận) thừa nhận sự thật đơn giản này và khuyến nghị rằng mọi yêu cầu đều nên được đặc tả dưới dạng test. Những test đó phải do các business analyst (BA) và QA viết, theo từng feature một, ngay trước khi feature đó được hiện thực. QA không chịu trách nhiệm chạy những test này. Nhiệm vụ ấy được giao cho lập trình viên; và do đó, rất nhiều khả năng lập trình viên sẽ tự động hoá chúng.

No programmer in their right mind wants to manually **test** the system over and over again. Programmers automate things. Thus, if the programmers are responsible for running the tests, the programmers will automate those tests.

Không lập trình viên nào còn tỉnh táo lại muốn ngồi test hệ thống bằng tay hết lần này đến lần khác. Lập trình viên thì tự động hoá mọi thứ. Vì thế, nếu lập trình viên chịu trách nhiệm chạy test, lập trình viên sẽ tự động hoá đám test đó.

However, since BA and QA author the tests, the programmers must be able to prove to BA and QA that the automation actually performs the tests that were authored. Therefore, the language in which the tests are automated must be a language that BA and QA understand. Indeed, BA and QA ought to be able to write the tests in that automation language.

Tuy nhiên, vì BA và QA mới là tác giả của các test, lập trình viên phải chứng minh được cho BA và QA thấy rằng phần tự động hoá thực sự chạy đúng những test đã được viết ra. Do đó, ngôn ngữ dùng để tự động hoá test phải là ngôn ngữ mà BA và QA hiểu được. Thật ra, BA và QA phải có khả năng tự viết test bằng chính ngôn ngữ tự động hoá đó.

There are several tools that have been invented over the years to help with this problem: FitNesse, 1 JBehave, SpecFlow, Cucumber, and others. But tools are not really the issue. The specification of software **behavior** is always a simple **function** of specifying input **data**, the action to perform, and the expected output data. This is the well-known AAA 2 pattern of Arrange/Act/Assert, or the Given-When-Then discipline of behavior-driven development (BDD).

Nhiều năm qua đã có vài công cụ được phát minh để giúp giải quyết vấn đề này: FitNesse, 1 JBehave, SpecFlow, Cucumber, và những công cụ khác. Nhưng công cụ không thực sự là vấn đề. Việc đặc tả hành vi phần mềm luôn là một hàm đơn giản của ba thứ: đặc tả dữ liệu đầu vào, hành động cần thực hiện, và dữ liệu đầu ra mong đợi. Đây chính là mẫu AAA 2 nổi tiếng — Arrange/Act/Assert — hay kỷ luật Given-When-Then của behavior-driven development (BDD).

1 <https://fitnesse.org/> 2 Credit: Bill Wake.

1 <https://fitnesse.org/> 2 Ghi công: Bill Wake.

All tests begin by arranging the input data for the test. Then, the test causes the tested action to be performed. Finally, the test asserts that the output data from that action matches the expectation.

Mọi test đều bắt đầu bằng việc sắp đặt (arrange) dữ liệu đầu vào cho test. Sau đó, test khiến hành động cần kiểm tra được thực hiện. Cuối cùng, test khẳng định (assert) rằng dữ liệu đầu ra của hành động đó khớp với kỳ vọng.

These three elements can be specified in a variety of different ways; but the easiest approach is a simple tabular format.

Ba thành phần này có thể được đặc tả theo nhiều cách khác nhau; nhưng cách dễ nhất là một định dạng bảng đơn giản.

The following figure, for example, is a portion of one of the acceptance tests within the FitNesse tool. FitNesse is a wiki, and this test checks that the various markup gestures are properly translated into HTML. The action to be performed is “ widget should render ,” the input data is the “ wiki text ,” and the output is the “ html text .”

Chẳng hạn, hình dưới đây là một phần của một trong các acceptance test bên trong công cụ FitNesse. FitNesse là một wiki, và test này kiểm tra xem các cú pháp markup khác nhau có được dịch đúng sang HTML hay không. Hành động cần thực hiện là “ widget should render ”, dữ liệu đầu vào là “ wiki text ”, còn đầu ra là “ html text ”.

widget should render		
wiki text	html text	
normal text	normal text	
this is "italic" text	this is <i>italic</i> text	italic widget
this is "'bold'" text	this is bold text	bold widget
!c This is centered text	<center>This is centered text</center>	
!1 header	header	
!2 header	header	
!3 header	header	
!4 header	header	
!5 header	header	
!6 header	header	
http://files/x	http://files/x	file link
http://fitnesse.org	http://fitnesse.org	http link
SomePage	SomePage\[?\] 	missing wiki word

Following is an example of the Given-When-Then style:

Sau đây là một ví dụ theo phong cách Given-When-Then:

Given a page with the wiki text: !1 header
When that page is rendered.
Then the page will contain: <h1>header</h1>

It should be clear that these formalisms, whether they are written in an acceptance testing tool or whether they are written in a simple spreadsheet or text editor, are relatively easy to automate.

Hẳn bạn cũng thấy rõ rằng những dạng hình thức hoá này — dù được viết trong một công cụ acceptance testing, hay chỉ viết trong một bảng tính hoặc trình soạn thảo văn bản đơn giản — đều tương đối dễ tự động hoá.

The Discipline — Kỷ luật

In the strictest form of the discipline, the acceptance tests are written by BA and QA. BA focuses on the happy path scenarios, while QA focuses on exploring the myriad of ways that the system can fail.

Ở dạng nghiêm ngặt nhất của kỷ luật này, acceptance test do BA và QA viết. BA tập trung vào các kịch bản happy path, còn QA tập trung khai phá muôn vàn cách mà hệ thống có thể hỏng.

These tests are written at the same time, or just before, the features they test are developed. In an Agile project, divided up into sprints or iterations, the tests are written during the first few days of the sprint. They should all pass by the end of the sprint.

Những test này được viết cùng lúc, hoặc ngay trước khi, các feature mà chúng kiểm tra được phát triển. Trong một dự án Agile chia thành các sprint hoặc iteration, test được viết trong vài ngày đầu của sprint. Tất cả chúng đều phải pass vào cuối sprint.

BA and QA provide the programmers with these tests, and the programmers automate them in a manner that keeps BA and QA engaged.

BA và QA cung cấp các test này cho lập trình viên, và lập trình viên tự động hoá chúng theo cách vẫn giữ được sự tham gia của BA và QA.

These tests become the definition of done. A feature is not complete until all its acceptance tests pass. And when all the acceptance tests pass, the feature is done.

Những test này trở thành định nghĩa của “xong”. Một feature chưa hoàn tất chừng nào toàn bộ acceptance test của nó chưa pass. Và khi tất cả acceptance test đều pass, feature đó đã xong.

This, of course, puts a huge responsibility on BA and QA. The tests that they write must be full specifications of the features being tested. The suite of acceptance tests is the requirements document for the entire system. By writing those tests, BA and QA are certifying that when they pass, the specified features are done and working.

Dĩ nhiên, điều này đặt một trách nhiệm khổng lồ lên vai BA và QA. Các test mà họ viết phải là đặc tả đầy đủ cho những feature được kiểm tra. Bộ acceptance test chính là tài liệu yêu cầu cho toàn bộ hệ thống. Bằng việc viết ra những test ấy, BA và QA đang chứng nhận rằng khi chúng pass thì các feature được đặc tả đã xong và chạy được.

In some cases, BA and QA may not be accustomed to writing such formal and detailed documents. In that case, the programmers may wish to write the acceptance tests with guidance from BA and QA. The intermediate goal is to create tests that BA and QA can read and bless. The ultimate goal is to get BA and QA comfortable enough to write the tests.

Trong một số trường hợp, BA và QA có thể chưa quen viết những tài liệu hình thức và chi tiết đến vậy. Khi đó, lập trình viên có thể muốn tự viết acceptance test dưới sự hướng dẫn của BA và QA. Mục tiêu trung gian là tạo ra những test mà BA và QA có thể đọc và ban phước. Mục tiêu tối hậu là làm cho BA và QA đủ thoải mái để tự viết test.

The Continuous Build — Continuous build

Once an **acceptance test** passes, it goes into the suite of tests that is run during the continuous build.

Một khi acceptance test đã pass, nó gia nhập bộ test được chạy trong quá trình continuous build.

The continuous build is an automated procedure that runs every time a programmer checks code into the source code control system. This procedure builds the system from source and then runs the suite of automated programmer unit tests and the suite of automated acceptance tests. The results of that run are visibly posted, often in an email to every programmer and interested party. The state of the continuous build is something everyone should be continuously aware of.

Continuous build là một quy trình tự động, chạy mỗi lần 3 có lập trình viên check code vào hệ thống quản lý mã nguồn. Quy trình này build hệ thống từ mã nguồn rồi chạy bộ unit test tự động của lập trình viên cùng bộ acceptance test tự động. Kết quả của lần chạy đó được công bố công khai, thường là qua email gửi tới mọi lập trình viên và mọi bên quan tâm. Trạng thái của continuous build là thứ mà tất cả mọi người nên liên tục để mắt tới.

3 Within a few minutes.

3 Trong vòng vài phút.

The continuous running of all these tests ensures that subsequent changes to the system do not break working features. If a previously passing acceptance test fails during the continuous build, the team must immediately respond and repair it before making any other changes. Allowing failures to accumulate in the continuous build is suicidal.

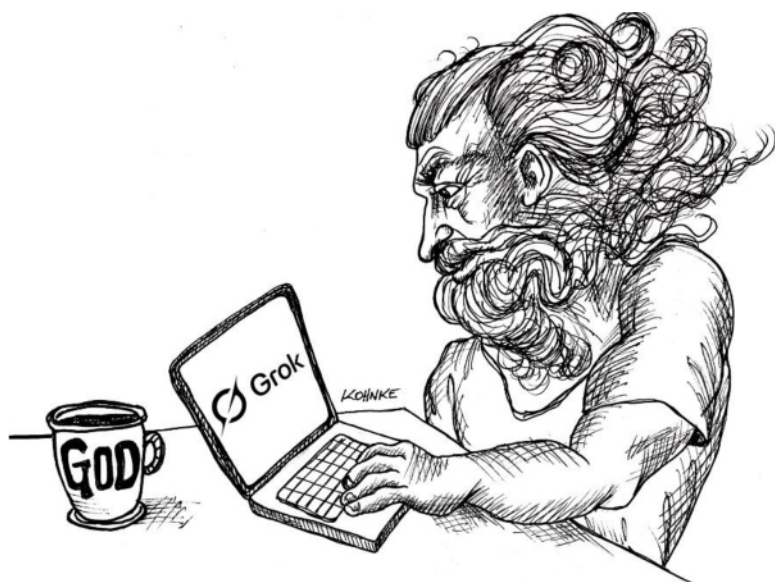
Việc liên tục chạy tất cả những test này đảm bảo rằng các thay đổi về sau không làm hỏng những feature đang chạy tốt. Nếu một acceptance test trước đó vẫn pass bỗng fail trong continuous build, cả nhóm phải phản ứng ngay lập tức và sửa nó trước khi thực hiện bất kỳ thay đổi nào khác. Để cho các lỗi fail chất đống trong continuous build là hành vi tự sát.

Conclusion — Kết luận

Acceptance testing is one of the weakest aspects of software development in the first half of the twenty-first century. This must change. The businesses and the developers must learn to collaborate to create a language and a procedure that allow requirements to be formally specified by BA and QA and that allow programmers to quickly verify that those requirements have been satisfied. The advent of AI makes this need doubly important.

Acceptance testing là một trong những mảng yếu nhất của ngành phát triển phần mềm trong nửa đầu thế kỷ hai mươi mốt. Điều này phải thay đổi. Phía kinh doanh và phía lập trình viên phải học cách hợp tác để tạo ra một ngôn ngữ và một quy trình cho phép BA và QA đặc tả yêu cầu một cách hình thức, đồng thời cho phép lập trình viên nhanh chóng kiểm chứng rằng những yêu cầu đó đã được thoả mãn. Sự xuất hiện của AI khiến nhu cầu này trở nên quan trọng gấp đôi.

Chapter 17. AIs, LLMs, and God Knows What — Chương 17. AI, LLM và có trời mới biết là gì nữa



In the late 1940s, Grace Hopper wrote her first lines of code. Each line looked something like this 1 :

Cuối thập niên 1940, Grace Hopper viết những dòng mã đầu tiên của mình. Mỗi dòng trông đại khái thế này 1 :

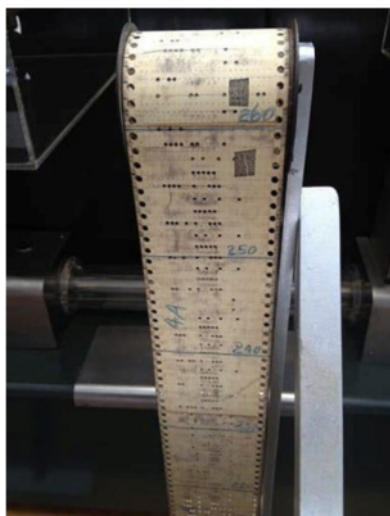
1 [We Programmers].

1 [We Programmers].

| 521 | 53 | 2 |

Each digit represented a particular hole on a paper tape that looked like this:

Mỗi chữ số đại diện cho một lỗ nhất định trên cuộn băng giấy trông như thế này:



Each row of that paper tape was an instruction for the Harvard Mark I to execute. The instructions were very simple —something like “Add Register 10 to Register 12.”

Mỗi hàng trên cuộn băng giấy đó là một chỉ thị để chiếc Harvard Mark I thực thi. Các chỉ thị này rất đơn giản — kiểu như “Cộng Register 10 vào Register 12.”

I’ll leave you to imagine just how hard programming was back then.

Tôi để bạn tự hình dung xem lập trình hồi ấy khó đến mức nào.

Within a few years, it became clear that a computer program could be used to generate codes like that from a more abstract textual representation like “add 10, 12.” Hopper called this automatic programming .

Chỉ vài năm sau, người ta thấy rõ rằng có thể dùng một chương trình máy tính để sinh ra những mã như vậy từ một biểu diễn văn bản trừu tượng hơn, chẳng hạn “add 10, 12”. Hopper gọi đó là lập trình tự động (automatic programming).

Before a decade had passed from Hopper’s first line of code, John Backus and his team had produced the first Fortran compiler, which used statements like “ $a = b + c$.” Algol came a handful of years later, and within a decade, Dahl and Nygaard had created Simula-67, the first **object**-oriented language. C, Unix, Smalltalk, and C++ were born within a few years. And within the next few, Java and C# appeared.

Chưa đầy một thập kỷ kể từ dòng mã đầu tiên của Hopper, John Backus và nhóm của ông đã cho ra đời trình biên dịch Fortran đầu tiên, dùng những câu lệnh như “ $a = b + c$ ”. Algol xuất hiện vài năm sau đó, và trong vòng một thập kỷ, Dahl cùng Nygaard tạo ra Simula-67, ngôn ngữ hướng đối tượng đầu tiên. C, Unix, Smalltalk và C++ ra đời chỉ trong vài năm. Rồi vài năm nữa, Java và C# xuất hiện.

At each one of those transitions, the **productivity** of programmers increased. The first leap in productivity was the greatest. In the ’50s, the productivity of programmers using a Fortran-like compiler was shown to be 45 times that of programmers using machine code.

Ở mỗi bước chuyển đó, năng suất của lập trình viên lại tăng lên. Bước nhảy đầu tiên là lớn nhất. Vào thập niên 1950, người ta đo được năng suất của lập trình viên dùng trình biên dịch kiểu Fortran cao gấp 45 lần so với lập trình viên viết mã máy.

Each productivity leap thereafter was smaller than the previous. C was probably twice that of Fortran. C++ was likely a 20% improvement on C. Java/C# was perhaps on the order of another 20%.

Ruby/Python are likely 20% better than Java, and Clojure likely adds another 10%.

Mỗi bước nhảy năng suất sau đó đều nhỏ hơn bước trước. C có lẽ chỉ gấp đôi Fortran. C++ chắc chỉ cải thiện được 20% so với C. Java/C# có lẽ thêm được chừng 20% nữa. Ruby/Python nhiều khả năng tốt hơn Java khoảng 20%, và Clojure có lẽ cộng thêm 10%.

Each of these transitions in language was driven by an increase in **abstraction**. Each new language in that sequence was another step further removed from Grace Hopper’s holes in that paper tape. And each of those steps was accompanied by the fear that programming was being ruined.

Mỗi bước chuyển ngôn ngữ ấy đều được thúc đẩy bởi một mức trừu tượng cao hơn. Mỗi ngôn ngữ mới trong chuỗi đó lại là một bước rời xa thêm những cái lỗ trên cuộn băng giấy của Grace Hopper. Và mỗi bước như vậy đều đi kèm nỗi sợ rằng lập trình đang bị huỷ hoại.

Yes, you read that correctly. At every stage, there were those who predicted the end of programming. Books were published that predicted the doom of the industry. They claimed that programmers would become as rare as dodo birds.

Vâng, bạn đọc không nhầm đâu. Ở mọi giai đoạn đều có những người tiên đoán ngày tàn của nghề lập trình. Sách được xuất bản để báo trước hồi kết của cả ngành. Họ bảo rằng lập trình viên rồi sẽ hiếm như chim dodo.

Every time we made programming a little easier, those of us who were invested in the current complexity were liable to fear that unwashed masses would take our jobs. And so we predicted the doom of the profession. And every time, those fears and predictions were the exact opposite of the actual outcomes. With every new advance, the number of projects and the demand for programmers grew.

Cứ mỗi lần chúng ta làm cho lập trình dễ hơn một chút, những ai đang đầu tư vào mức độ phức tạp hiện hành lại dễ dâm ra sợ rằng đám đông “chưa tắm rửa” sẽ cướp mất việc của mình. Thế là chúng ta lại tiên đoán ngày tàn của nghề. Và lần nào cũng vậy, những nỗi sợ và lời tiên đoán ấy hoá ra hoàn toàn trái ngược với thực tế. Cứ mỗi bước tiến mới, số lượng dự án và nhu cầu về lập trình viên lại tăng lên.

Today, we face yet another such transition—the transition of AI. And there is, of course, rampant fear that programming is dead and that programmers will soon go extinct.

Hôm nay, chúng ta lại đối mặt với một bước chuyển như thế — bước chuyển của AI. Và tất nhiên, lại tràn lan nỗi sợ rằng lập trình đã chết và lập trình viên sắp tuyệt chủng đến nơi.

Baloney! This is just another increase in the **level of abstraction** of our languages—another step away from the holes in the paper tape. And as we come to grips with it, the number of projects and the demand for programmers will continue to grow.

Nhảm nhí! Đây chỉ là một lần nữa nâng mức trừu tượng của ngôn ngữ mà thôi — thêm một bước rời xa những cái lỗ trên băng giấy. Và khi chúng ta thuần thục được nó, số lượng dự án cùng nhu cầu về lập trình viên sẽ tiếp tục tăng.

Programming by Prompt — Lập trình bằng prompt

This is a very new technology, and we don’t have a **good** handle on how to use it yet. This is reminiscent of our first forays into Java and the Web. Does anybody remember applets? It took several years before we worked out the rudiments of how to use Java/C# well. It took several more to refine those rudiments.

Đây là một công nghệ rất mới, và chúng ta vẫn chưa thực sự nắm được cách dùng nó. Chuyện này gợi nhớ những lần chúng ta lần mò với Java và Web thuở ban đầu. Có ai còn nhớ applet không? Phải mất mấy năm chúng ta mới rút ra được những nguyên tắc sơ đẳng để dùng Java/C# cho tốt. Rồi mất thêm mấy năm nữa để tinh chỉnh những nguyên tắc sơ đẳng đó.

The same happened with Ruby and Python. At first we stumbled and bumbled around, and then we gradually figured out the best ways to proceed. And believe me, when Fortran first came out, there was plenty of fear, anguish, doubt, and bumbling and stumbling.

Với Ruby và Python cũng vậy. Ban đầu chúng ta loay hoay vấp vấp, rồi dần dần mới tìm ra cách làm tốt nhất. Và tin tôi đi, khi Fortran mới xuất hiện, cũng đầy rẫy sợ hãi, khắc khoải, hoài nghi cùng đủ thứ vấp vấp loay hoay.

And so it will be with AIs. At this point, we have no idea what to do. We are just now barely able to bumble and stumble our way to write programs with prompts. We have no idea how to do it well.

Với AI rồi cũng sẽ như thế. Ở thời điểm này, chúng ta chẳng biết phải làm gì. Chúng ta mới chỉ vừa đủ khả năng loay hoay vấp vấp để viết chương trình bằng prompt. Chúng ta hoàn toàn chưa biết làm sao cho tốt.

So, for example, consider the following **prompt** that I gave to Grok 2 a few days ago:

Chẳng hạn, hãy xem cái prompt mà tôi đưa cho Grok cách đây vài hôm 2 :

2 Grok 3 beta (19 March, 2025).

2 Grok 3 beta (19 tháng Ba, 2025).

```
write a clojure program that finds all sentences
in a given file that contain a given substring.
Choose one of those sentences at random and find
Choose one of those sentences at random and find
the next word in the substring, or a . if no word follows.
Add the found word to the substring and remove the
first if the substring is longer than three words.
Repeat the entire process until the substring
ends in a . or has exceeded 50 words.
Accumulate all the words that were found and
return them appended to the initial substring.
```

Here's the code that Grok generated. It works—after a fashion.

Và đây là mã Grok sinh ra. Nó chạy được — theo một kiểu nào đó.

```
(ns markov
  (:require [clojure.string :as str]))
(defn read-file [filename]
  "Reads the content of a file and splits it into sentences."
  (try
    (let [content (slurp filename)
          sentences (filter not-empty
                           (map str/trim
                                (str/split content #"[.!?]"))))]
      sentences)
    (catch Exception e
```

```

        (throw (Exception. (str "Error reading file: " (.getMessage e
(defn find-sentences-with-substring [sentences substring]
  "Finds all sentences containing the substring."
  (filter #(str/includes? (str/lower-case %) (str/lower-case subst
    sentences)))
(defn get-next-word [sentence substring]
  "Finds the next word after the substring
  in the sentence, or '.' if none."
  (let [index (str/index-of (str/lower-case sentence)
    (str/lower-case substring))]
    (if index
      (let [post-substring (subs sentence (+ index (count substring)
        words (str/split (str/trim post-substring) #"\s+"))]
        (if (seq words)
          (first words)
          "."))
      "."))))
(defn update-substring [substring next-word]
  "Updates the substring by adding the next word
  and trimming if longer than 3 words."
  (let [words (str/split substring #"\s+")
    new-words (conj (vec words) next-word)
    trimmed-words (if (> (count new-words) 3)
      (subvec new-words 1)
      new-words)]
    (str/join " " trimmed-words)))
(defn sentence-walk [filename initial-substring]
  "Main function to perform the sentence walking process."
  (let [sentences (read-file filename)]
    (loop [substring initial-substring
      accumulated-words []
      iterations 0]
      (if (or (= (last (str/split substring #"\s+")) ".")
        (> iterations 50))
        (str/join " " (concat (str/split initial-substring #"\s+")
          accumulated-words))
        (let [matching-sentences (find-sentences-with-substring
          sentences substring)]
          (if (empty? matching-sentences)
            (str/join " " (concat (str/split initial-substring #"\s+")
              accumulated-words))
            (let [random-sentence (rand-nth matching-sentences)
              next-word (get-next-word random-sentence substr
                (recur (update-substring substring next-word)
                  (conj accumulated-words next-word)
                  (inc iterations))))))))))
(defn -main [& args]
  (if (< (count args) 2)
    (println "Usage: program <filename> <initial-substring>")

```

```
(let [filename (first args)
      initial-substring (second args)]
  (try
    (println (sentence-walk filename initial-substring))
    (catch Exception e
      (println (.getMessage e))))))
```

It is easy to be impressed by this. I certainly am. But it also terrifies me. I am not terrified by the power of the AI to generate code. I am terrified by the **structure** of that horrible prompt.

Rất dễ để thấy ấn tượng với chuyện này. Bản thân tôi cũng ấn tượng thật. Nhưng nó cũng làm tôi khiếp sợ. Tôi không sợ khả năng sinh mã của AI. Tôi sợ cái cấu trúc của cái prompt kinh khủng kia.

Look carefully at that prompt. What is the definition of

Hãy nhìn kỹ prompt đó. Đây là định nghĩa của

sentence ? What is the definition of word ? What does the clause

sentence? Đây là định nghĩa của word? Còn mệnh đề

the found word mean? Are those dot characters punctuation or nouns? What does it mean to accumulate words ?

the found word thì nghĩa là gì? Mấy ký tự chấm kia là dấu câu hay là danh từ? accumulate words nghĩa là làm gì?

Yes, Grok figured all this out well enough for this little experiment. But the program that resulted was by no means ready for production. It makes lots of dumb little mistakes, most of which have to do with the questions I raised in the previous paragraph.

Đúng là Grok đã tự suy ra được tất cả những thứ đó đủ tốt cho thí nghiệm nhỏ này. Nhưng chương trình sinh ra thì còn lâu mới sẵn sàng lên production. Nó mắc vô số lỗi vụn vặt ngớ ngẩn, mà phần lớn đều liên quan đến chính những câu hỏi tôi vừa nêu ở đoạn trên.

Here, for example, is a sample run:

Chẳng hạn, đây là một lần chạy thử:

```
>clj -M -m markov
~/Documents/Books/Clean\ Code\ 2ded\ 20250311.txt "The Single"
The Single Responsibility Principle, and the Dependency
relationships and wind up back at that first solution, the one wit
the project you were working on it every day business and persona
```

Notice the double spaces in the third and fourth lines. Why is it doing that?

Để ý mấy khoảng trắng đôi ở dòng thứ ba và thứ tư. Sao nó lại làm vậy?

I could fix those mistakes by adding clarification to the prompt. For example, I might add:

Tôi có thể sửa những lỗi đó bằng cách bổ sung làm rõ vào prompt. Ví dụ, tôi có thể thêm:

Make sure all accumulated words are separated by just one space.

But there are a number of problems with this approach. First, I haven't figured out why those double spaces are there, so this change to the prompt is just a patch to fix the symptom. Second, am I sure that the clause all accumulated words will properly refer to the return value?

Nhưng cách làm này có khá nhiều vấn đề. Thứ nhất, tôi vẫn chưa tìm ra vì sao lại có mấy khoảng trắng đôi đó, nên thay đổi này trong prompt chỉ là một miếng vá chữa triệu chứng mà thôi. Thứ hai, tôi có chắc rằng mệnh đề *all accumulated words* sẽ trở đúng vào giá trị trả về hay không?

I could try it to find out; but that causes another problem. With each new change to the prompt, Grok regenerates the program from scratch as opposed to modifying the existing program. With all the ambiguities in the prompt, how can I be sure that the regenerated program doesn't break something that worked in the previous one? Will it use the same definitions of word and sentence that it did before? Will it interpret the found word the same as it did before? Given the statistical and fuzzy nature of the **LLM** algorithms, there's a good chance it won't!

Tôi có thể thử để biết; nhưng thế lại sinh ra một vấn đề khác. Mỗi lần thay đổi prompt, Grok sinh lại chương trình từ đầu chứ không sửa chương trình sẵn có. Với tất cả những chỗ mập mờ trong prompt, làm sao tôi dám chắc chương trình vừa sinh lại không làm hỏng thứ vốn đang chạy tốt ở bản trước? Liệu nó có dùng đúng định nghĩa word và sentence như lần trước không? Liệu nó có hiểu the found word y như lần trước không? Với bản chất thống kê và mờ nhòe của các thuật toán LLM, khả năng cao là không!

Here is what Grok generated for that modified prompt. Notice that the two programs look like they were written by two different programmers.

Đây là những gì Grok sinh ra cho prompt đã sửa. Để ý xem, hai chương trình trông cứ như do hai lập trình viên khác nhau viết vậy.

```
(ns sentence-walker
  (:require [clojure.string :as str]
            [clojure.java.io :as io]))
(defn get-sentences [filename]
  "Read file and split into sentences"
  (-> (slurp filename)
      (str/replace #"[\.!?]\s+" "$0|")
      (str/split #"\\")
      (->> (map str/trim)
           (filter not-empty))))
(defn find-matching-sentences [sentences substring]
  "Find all sentences containing the substring"
  (filter #(str/includes? (str/lower-case %)
                          (str/lower-case substring))
          sentences))
(defn get-next-word [sentence substring]
  "Find word following substring in sentence, or . if at end"
  (let [idx (str/index-of (str/lower-case sentence)
                          (str/lower-case substring))
        substr-end (+ idx (count substring))
        remainder (subs sentence substr-end)]
    (if (empty? (str/trim remainder))
        "."
        (let [next-word (first (str/split (str/trim remainder) #"\\s+"))
              (if (or (nil? next-word) (empty? next-word))
                  "."
                  next-word))]))))
```

```

(defn update-substring [substring next-word]
  "Add next word and trim if longer than 3 words"
  (let [words (str/split substring #"\s+")
        new-words (conj (vec words) next-word)]
    (if (> (count new-words) 3)
      (str/join " " (subvec new-words (- (count new-words) 3)))
      (str/join " " new-words))))
(defn walk-sentences [filename initial-substring]
  "Main function to walk through sentences"
  (let [sentences (get-sentences filename)]
    (loop [current-substr initial-substring
          accumulated-words [initial-substring]
          iterations 0]
      (if (or (str/ends-with? current-substr ".")
              (> iterations 50))
        (str/join " " accumulated-words)
        (let [matching-sentences (find-matching-sentences
                                   sentences current-substr)]
          (if (empty? matching-sentences)
            (str/join " " accumulated-words)
            (let [random-sentence (rand-nth matching-sentences)
                  next-word (get-next-word random-sentence current
                                             new-substr (update-substring current-substr next
                                                                              (recur new-substr
                                                            (conj accumulated-words next-word)
                                                            (inc iterations)))))))]))))))
;; Example usage:
;; (walk-sentences "sample.txt" "the")

```

This new program solves the double-space problem but creates a new problem at the end of the sentence. It also doesn't deal with uppercase and lowercase very well.

Chương trình mới này giải quyết được vấn đề khoảng trắng đôi nhưng lại để ra một vấn đề mới ở cuối câu. Nó cũng xử lý chữ hoa chữ thường chẳng ra gì.

clean code is hard Writing a system that handles one user at a time

Infancy — Thời kỳ trứng nước

These are problems related to the infancy of the approach. In order to effectively program large systems by prompt, we are going to need AI tools that allow for incremental changes. We are also going to need ways to eliminate the ambiguities in our prompts.

Đây là những vấn đề của một hướng tiếp cận còn trong trứng nước. Để lập trình các hệ thống lớn bằng prompt một cách hiệu quả, chúng ta sẽ cần những công cụ AI cho phép thay đổi tăng dần. Chúng ta cũng sẽ cần cách để loại bỏ những chỗ mập mờ trong prompt của mình.

In short, we are going to need a formalism—a prompt language that is unambiguous and precise and that cannot be misinterpreted by the statistical and fuzzy nature of the LLM algorithm. We'll need to come up with a way to precisely say what we mean.

Nói ngắn gọn, chúng ta sẽ cần một hệ hình thức (formalism) — một ngôn ngữ prompt không mập mờ, chính xác, và không thể bị bản chất thống kê, mờ nhoè của thuật toán LLM diễn giải sai. Chúng ta sẽ cần tìm ra cách nói chính xác điều mình muốn nói.

We'll also need to come up with a way to mean what we say. Or, in other words, even if we have a perfectly precise language, it is easy to precisely say something that is wrong. To ensure that our precise statements are accurate descriptions of our intent, we should state our intent in at least two different ways (just as I did in this paragraph!).

Chúng ta cũng sẽ cần tìm ra cách muốn đúng điều mình nói. Nói cách khác, ngay cả khi đã có một ngôn ngữ hoàn toàn chính xác, việc phát biểu thật chính xác một điều sai vẫn dễ như bỡn. Để bảo đảm những phát biểu chính xác của mình mô tả đúng ý định, chúng ta nên phát biểu ý định đó theo ít nhất hai cách khác nhau (đúng như tôi vừa làm trong đoạn này!).

When you say something in two different ways, you are overloading the concept you are trying to communicate. Such overloading is a way to ensure that any error in either of the two statements will be detected by the other. We error-prone humans absolutely require overloading.

Khi bạn nói một điều theo hai cách khác nhau, bạn đang nạp chồng (overload) cái khái niệm mình muốn truyền đạt. Kiểu nạp chồng như vậy là một cách bảo đảm rằng mọi sai sót ở một trong hai phát biểu sẽ bị phát biểu còn lại phát hiện. Loài người dễ sai sót như chúng ta thì tuyệt đối cần đến nạp chồng.

Overloading is the strategy behind double-entry bookkeeping and the testing disciplines described in earlier chapters. It would be supremely irresponsible to trust a single set of precise statements written by a human if those statements aren't overloaded. What we are going to need is a way to create precise and overloaded prompts and a way to quickly verify their consistency.

Nạp chồng chính là chiến lược nằm sau ghi sổ kép (double-entry bookkeeping) và các kỷ luật (discipline) kiểm thử đã mô tả ở những chương trước. Sẽ là cực kỳ vô trách nhiệm nếu tin vào một tập phát biểu chính xác duy nhất do con người viết ra mà không được nạp chồng. Thứ chúng ta sẽ cần là một cách tạo ra những prompt vừa chính xác vừa được nạp chồng, cùng một cách kiểm chứng nhanh tính nhất quán của chúng.

A SWAG — Một SWAG

So let me make a Scientific Wild-Axx Guess.

Vậy hãy để tôi đưa ra một Scientific Wild-Axx Guess (một phỏng đoán liều mạng nhưng nghe cho ra vẻ khoa học).

Imagine a prompt language that contains formal definitions and constraints overloaded by a set of BDD-style 3 Given - When -

Hãy hình dung một ngôn ngữ prompt chứa các định nghĩa và ràng buộc hình thức, được nạp chồng bằng một tập kịch bản kiểu BDD 3 Given - When -

Then scenarios.

Then.

3 [RSpec].

3 [RSpec].

Let's give that a try with a slightly simpler problem. Notice how much it looks like COBOL. That similarity might give us pause.

Hãy thử điều đó với một bài toán đơn giản hơn một chút. Để ý xem nó giống COBOL đến thế nào. Sự tương đồng ấy có lẽ nên khiến chúng ta chững lại đôi chút.

Language: Clojure

Name: Markov

Command line arguments:

 f a text file.

 s a text string.

Definitions:

 Word: A string of alphabetic characters.

 Sentence: One or more words separated by spaces and ending with a period, question mark, or exclamation mark.

Constraints:

 Ignore case in all comparisons.

Description:

 Find all sentences in f that contain s.

 Choose a sentence c at random with equal probability.

 Return the word in c that immediately follows s;

 if none return a period.

Scenarios to be tested with spec1j:

Given f contains the following lines:

 Oh, Hello there Bob, how are you today? I'm fine.

 Hello there Bill, I trust you are well. How's the wife?

Ah yeshello there Johngreat day!Kowabunga!

 Ah yes, hello there John, great day! Kowabunga!

When s is "not in any sentence"

 Return "."

When s is "hello"

 Then return "there"

When s is "are well"

 Then return "."

When s is "hello there"

 Then return "Bob", "Bill", or "John".

Assert a uniform distribution after 100 iterations.

Grok was quite happy to generate the following code.

Grok rất vui vẻ sinh ra đoạn mã sau đây.

```
(ns markov.core
  (:require [clojure.string :as str]
             [clojure.java.io :as io]))
(defn get-sentences [file]
  (->> (slurp file)
        (re-seq #"[A-Za-z\s]+[.!?]" )
        (map str/trim)))
(defn words [sentence]
  (str/split (str/lower-case sentence) #"\\s+"))
(defn find-following-word [sentences search-str]
```



```

(let [search-str (str/lower-case search-str)
      matching-sentences (filter
                           #(str/includes? (str/lower-case %)
                                             search-str)
                           sentences)]
  (if (empty? matching-sentences)
      "."
      (let [chosen (rand-nth matching-sentences)
            word-seq (words chosen)
            search-words (str/split search-str #"\s+")
            idx (.indexOf word-seq (last search-words))]
        (if (or (< idx -1) (>= (+ idx 1) (count word-seq)))
            "."
            (nth word-seq (+ idx 1))))))
(defn -main [f s]
  (let [sentences (get-sentences f)
        result (find-following-word sentences s)]
    (println result)))

```

And here are the tests Grok generated from the scenarios.

Và đây là các test mà Grok sinh ra từ những kịch bản đó.

```

;; spec!j tests
(ns markov.core-spec
  (:require [spec!j.core :refer :all]
             [markov.core :refer :all]
             [clojure.java.io :as io]))

(def test-content
  "Oh, Hello there Bob, how are you today? I'm fine.
  Hello there Bill, I trust you are well.  How's the wife?
  Ah yes, hello there John, great day!  Kowabunga!")
(describe "Markov"
  (before-all
    (spit "test.txt" test-content))
  (after-all
    (.delete (io/file "test.txt")))
  (it "returns period when string not found"
    (should= "." (find-following-word (get-sentences "test.txt")
                                       "not in any sentence")))
  (it "returns 'there' when searching for 'hello'"
    (should= "there" (find-following-word (get-sentences "test.txt")
                                           "hello")))
  (it "returns period when searching for 'are well'"
    (should= "." (find-following-word (get-sentences "test.txt")
                                       "are well")))
  (it "returns Bob, Bill, or John for 'hello there'
      with uniform distribution"
    (let [results (repeatedly 100
                              #(find-following-word
                                (get-sentences "test.txt") "hello there"))]

```

```

    freqs (frequencies results)
    expected ["bob" "bill" "john"]]
(should= 3 (count freqs))
(should (every? #(contains? freqs %) expected))
(doseq [name expected]
  (let [count (get freqs name)]
    ;; Check if count is reasonably close to 33 (100/3)
    ;; with some tolerance
    (should (> count 20))
    (should (< count 50))))))

```

Despite a valiant effort by Grok, these tests do not pass. That should drive home just how important it is to overload a prompt with testing scenarios!

Bất chấp nỗ lực đáng khen của Grok, những test này không pass. Điều đó cho thấy rõ mồn một việc nạp chồng một prompt bằng các kịch bản kiểm thử quan trọng đến mức nào!

Failures:

- 1) Markov returns 'there' when searching for 'hello'
Expected: "there"
got: "." (using =)
- 2) Markov returns period when searching for 'are well'
Expected: "."
got: "i" (using =)
- 3) Markov returns Bob, Bill, or John for
'hello there' with uniform distribution
Expected: 3
got: 1 (using =)

Debugging the code is relatively simple; but can you debug the prompt? And even if you can “fix” the prompt, are you sure it will generate code and tests that pass if you apply the prompt more than once?

Debug mã thì tương đối đơn giản; nhưng bạn có debug được prompt không? Và ngay cả khi bạn “sửa” được prompt, liệu bạn có chắc nó sẽ sinh ra mã và test pass nếu bạn chạy lại prompt đó thêm lần nữa?

The fact that Grok wrote the tests and yet could not use them to infer the desired **behavior** exposes the limitation of the LLM approach. LLMs are statistical in nature; they create their responses through probability and not inference. It is true that many inferences are trained into the statistical model; but the evaluation of the prompt does not infer in real time. Inference is the primary operation of reason. If LLMs do not actively infer, then LLMs do not reason!

Việc Grok viết ra các test mà lại không dùng được chính chúng để suy ra hành vi mong muốn đã phơi bày giới hạn của hướng tiếp cận LLM. LLM về bản chất là thống kê; chúng tạo ra câu trả lời bằng xác suất chứ không phải bằng suy luận. Đúng là nhiều suy luận đã được huấn luyện sẵn vào mô hình thống kê; nhưng lúc đánh giá prompt thì nó không suy luận theo thời gian thực. Suy luận mới là thao tác cốt lõi của lý trí. Nếu LLM không chủ động suy luận, thì LLM không lập luận!

If you debug the code that Grok produced, you will find that Grok apparently ignored the provided definition of “word” and did not properly interpret the case insensitivity constraint. The fact that Grok got confused by a formal prompt overloaded with **test** scenarios is not encouraging. It seems obvious, at least to me, that without overloaded formality, the prospect of using AIs to write our

systems is doomed to spectacular failure. After all, programming is formal, and humans require overloading.

Nếu bạn debug đoạn mã Grok tạo ra, bạn sẽ thấy Grok rõ ràng đã phớt lờ định nghĩa “word” được cung cấp và không diễn giải đúng ràng buộc phân biệt hoa thường. Chuyện Grok bị rối bởi một prompt hình thức đã được nạp chồng bằng các kịch bản test là điều chẳng lấy gì làm phần khởi. Với tôi thì có vẻ đã quá rõ: nếu thiếu tính hình thức được nạp chồng, viễn cảnh dùng AI để viết hệ thống cho chúng ta chắc chắn sẽ thất bại thảm hại. Suy cho cùng, lập trình là chuyện hình thức, còn con người thì cần nạp chồng.

Conclusion — Kết luận

In the end, I expect that the LLMs and AIs will improve to the point that long, formal specifications will be within their grasp. I also expect that we programmers will design formal and overloaded specification languages that the AIs can consume to produce complex systems.

Rốt cuộc, tôi cho rằng LLM và AI sẽ cải thiện đến mức nắm bắt được những đặc tả hình thức dài dòng. Tôi cũng cho rằng chính chúng ta, những lập trình viên, sẽ thiết kế ra các ngôn ngữ đặc tả hình thức và được nạp chồng để AI tiêu thụ mà tạo ra những hệ thống phức tạp.

And who will learn and apply these formalisms? Who will provide the ability to reason that the AIs lack? Programmers, of course. We programmers will become the lawyers of AI, responsible (as we always have been) to reason through and draw up the precise and accurate formal requirements and contracts for the applications our clients need.

Và ai sẽ học rồi áp dụng các hệ hình thức ấy? Ai sẽ cung cấp cái năng lực lập luận mà AI còn thiếu? Lập trình viên, tất nhiên rồi. Chúng ta, những lập trình viên, sẽ trở thành luật sư của AI, chịu trách nhiệm (như xưa nay vẫn thế) lập luận thấu đáo rồi soạn ra những yêu cầu và hợp đồng hình thức chính xác, đúng đắn cho các ứng dụng mà khách hàng của chúng ta cần.

Part II — Phần II

Design — Thiết kế

In this part of the book, we’re going to raise our perspective slightly above coding issues and look at the principles of clean design. You’ll still see plenty of code—but the level will continuously rise as each chapter goes by. We’ll cover the basics of lower-level boundaries and systems, Kent Beck’s rules of **simple design**, the **SOLID principles**, and the principles of **component** design.

Ở phần này của cuốn sách, chúng ta sẽ nâng tầm nhìn lên cao hơn một chút so với các vấn đề thuần về viết mã, để bàn tới những nguyên tắc của thiết kế sạch. Bạn vẫn sẽ thấy rất nhiều mã — nhưng mức độ sẽ liên tục được nâng lên qua từng chương. Chúng ta sẽ đi qua những điều cơ bản về ranh giới ở mức thấp và về hệ thống, các quy tắc thiết kế đơn giản (simple design) của Kent Beck, các nguyên tắc SOLID, và các nguyên tắc thiết kế component.

Portions of the following chapters are edited and abridged excerpts from my books **Clean Craftsmanship** and **Clean Architecture**.

Một số phần trong các chương sau là trích đoạn đã được biên tập và rút gọn từ hai cuốn sách của tôi: *Clean Craftsmanship* và *Clean Architecture*.

Chapter 18. Simple Design —

Chương 18. Thiết kế đơn giản



Kent Beck

[Kent Beck](#)

This chapter is an abridged and edited excerpt from the book *Clean Craftsmanship* .

Chương này là trích đoạn đã được rút gọn và biên tập từ cuốn *Clean Craftsmanship*.

It should be obvious, on the face of it, that the best design for a system is the simplest design that supports all the required features of that system while simultaneously affording the greatest flexibility for change. However, that leaves us to ponder the meaning of simplicity. 1 Simple does not mean easy. Simple means untangled; and untangling things is hard .

Thoạt nhìn thì hiển nhiên rằng thiết kế tốt nhất cho một hệ thống là thiết kế đơn giản nhất mà vẫn đỡ được toàn bộ tính năng hệ thống đó cần, đồng thời vẫn dành ra độ linh hoạt lớn nhất cho việc thay đổi. Có điều, điều đó lại buộc chúng ta phải ngẫm xem “đơn giản” nghĩa là gì. 1 Đơn giản không có nghĩa là dễ. Đơn giản nghĩa là không rối; mà gỡ rối thì khó.

1 In 2012, Rich Hickey gave a wonderful talk titled “Simplicity Made Easy.” I encourage you to listen

to it.

1 Năm 2012, Rich Hickey có một bài nói tuyệt vời tên là “Simplicity Made Easy”. Tôi khuyến khích bạn nghe thử.

What things get tangled in software systems? The most expensive and significant entanglements are those that convolve high-level policies with low-level details. You create terrible complexities when you conjoin SQL with HTML, or frameworks with core values, or the format of a report with the business rules that calculate the reported values. These entanglements are easy to write, but they make it hard to add new features, hard to fix bugs, and hard to improve and clean the design.

Trong các hệ thống phần mềm thì những thứ gì bị rối vào nhau? Những mối rối dằng dặc và nghiêm trọng nhất là những chỗ trộn lẫn chính sách mức cao với chi tiết mức thấp. Bạn tạo ra sự phức tạp kinh khủng khi ghép SQL với HTML, hoặc ghép framework với các giá trị cốt lõi, hoặc ghép định dạng của một báo cáo với các quy tắc nghiệp vụ (business rule) tính ra chính những con số trong báo cáo đó. Viết ra những mối rối này thì dễ, nhưng chúng khiến việc thêm tính năng mới trở nên khó, sửa lỗi khó, và cải thiện hay dọn dẹp thiết kế cũng khó.

A simple design is a design in which high-level policies are ignorant of low-level details. Those high-level policies are sequestered and isolated from low-level details such that changes to the low-level details have no impact upon the high-level policies. 2

Một thiết kế đơn giản là thiết kế trong đó các chính sách mức cao không hề hay biết gì về chi tiết mức thấp. Những chính sách mức cao ấy được tách riêng và cô lập khỏi chi tiết mức thấp, sao cho thay đổi ở chi tiết mức thấp không hề tác động đến chính sách mức cao.
2

2 I write a great deal about this in [Clean Arch].

2 Tôi viết rất nhiều về điều này trong [Clean Arch].

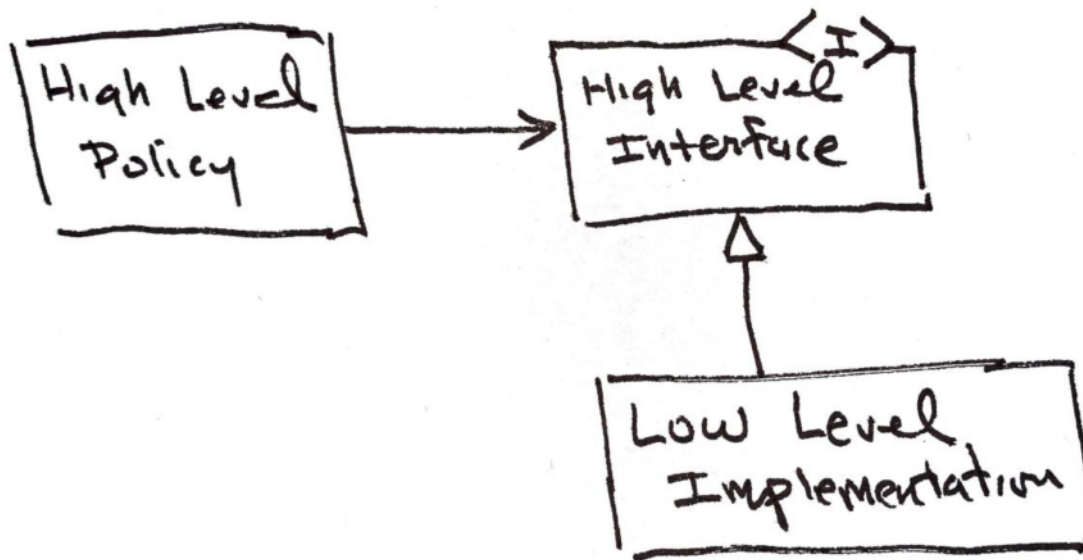
The primary means for creating this separation and isolation is **abstraction**. Abstraction is the amplification of the essential and the elimination of the irrelevant. High-level policies are essential, so they are amplified. Low-level details are irrelevant at that level, and so they are isolated and sequestered.

Phương tiện chính để tạo ra sự tách bạch và cô lập đó là *trừu tượng hoá (abstraction)*. Trừu tượng hoá là khuếch đại cái thiết yếu và loại bỏ cái không liên quan. Chính sách mức cao là thiết yếu, nên chúng được khuếch đại. Chi tiết mức thấp là thứ không liên quan ở mức đó, nên chúng bị cô lập và tách riêng ra.

One of the physical means we employ for this abstraction is **polymorphism**. We arrange high-level policies to use polymorphic interfaces to manage the low-level details. Then, we arrange the low-level details as implementations of those polymorphic interfaces. This keeps all source code dependencies pointing from low-level details to high-level policies and keeps high-level policies ignorant of the implementations of the low-level details. Low-level details can be changed without affecting the high-level policies. This is the essence of the **Dependency** Inversion Principle (DIP).

Một trong những phương tiện vật lý mà ta dùng để trừu tượng hoá là đa hình (polymorphism). Ta bố trí sao cho các chính sách mức cao dùng interface đa hình để điều khiển chi tiết mức thấp. Rồi ta bố trí chi tiết mức thấp thành các hiện thực (implementation) của những interface đa hình đó. Cách này giữ cho mọi phụ thuộc (dependency) trong mã nguồn đều chứa từ chi tiết mức thấp về phía chính sách mức cao, và giữ cho chính sách mức cao không hay biết gì về hiện thực của chi tiết mức thấp. Chi

tiết mức thấp có thể thay đổi mà không ảnh hưởng tới chính sách mức cao. Đó chính là tinh thần của DIP — Nguyên tắc Đảo ngược Phụ thuộc (Dependency Inversion Principle).



Pursuant to the goal of creating **good** software designs, we will first discuss **YAGNI** and then Kent Beck's four principles of simple design:

Hướng tới mục tiêu tạo ra những thiết kế phần mềm tốt, trước hết ta sẽ bàn về YAGNI, rồi tới bốn nguyên tắc thiết kế đơn giản của Kent Beck:

1. Covered by tests
 2. Reveals intent
 3. Minimizes **duplication**
 4. Minimizes size
1. Được test phủ kín
 2. Bộc lộ ý định
 3. Giảm thiểu trùng lặp
 4. Giảm thiểu kích thước

YAGNI — YAGNI

What if you aren't gonna need it ?

Nhờ đâu *bạn sẽ chẳng cần đến nó* thì sao?

In 1999, I was teaching an Extreme Programming course with Martin Fowler, Kent Beck, Ron Jeffries, and a host of others. The topic turned to the dangers of overdesign, and premature generalization. Someone wrote YAGNI on the whiteboard and said, "You aren't gonna need it." Kent Beck interrupted and said something to the effect of you might, in fact, need it; but you should ask yourself: "What if you aren't going to need it?"

Năm 1999, tôi dạy một khoá Extreme Programming cùng Martin Fowler, Kent Beck, Ron Jeffries và một loạt người khác. Câu chuyện chuyển sang mối nguy của việc thiết kế quá tay và tổng quát hoá quá sớm. Ai đó viết YAGNI lên bảng trắng rồi nói: “You aren’t gonna need it” (Bạn sẽ chẳng cần đến nó đâu). Kent Beck ngắt lời và nói đại ý rằng thật ra thì bạn *có thể* cần đến nó thật; nhưng bạn nên tự hỏi mình: “Nhờ đâu mình sẽ chẳng cần đến nó thì sao?”

That was the original question that YAGNI posed: Every time you thought to yourself “I’m going to need this hook,” you then asked yourself what would happen if you didn’t put the hook in. If the cost of leaving the hook out was tolerable, then you probably shouldn’t put it in. If the cost of carrying the hook in the design year after year would be high while the odds that you’d eventually need that hook were low, you probably shouldn’t put that hook in.

Đó mới là câu hỏi gốc mà YAGNI đặt ra: mỗi lần bạn nghĩ bụng “mình sẽ cần cái móc này”, hãy tự hỏi xem điều gì sẽ xảy ra nếu bạn không gắn cái móc đó vào. Nếu cái giá phải trả khi bỏ qua cái móc là chấp nhận được, thì có lẽ bạn không nên gắn nó vào. Nếu cái giá phải mang vác cái móc đó trong thiết kế hết năm này qua năm khác là cao, trong khi khả năng rốt cuộc bạn cần đến nó lại thấp, thì có lẽ bạn không nên gắn nó vào.

It is always wise to think of the future, and there are times when putting a particular hook in is a good idea. But be sure to count the costs and advantages on both sides of the **argument**. Make sure you ask yourself the question: What if I’m not gonna need it?

Nghĩ tới tương lai bao giờ cũng là khôn ngoan, và có những lúc gắn một cái móc cụ thể nào đó là ý hay. Nhưng hãy chắc chắn rằng bạn đã cân đo chi phí và lợi ích ở cả hai phía. Hãy chắc chắn rằng bạn đã tự hỏi: Nhờ đâu mình sẽ chẳng cần đến nó thì sao?

This kind of discernment is one of the most important disciplines of good software design. If you have a good suite of tests and you are skilled at the **discipline of refactoring**, then the cost of adding a new feature and updating the design to support that new feature will very often be smaller than the cost of implementing and maintaining all the hooks you might need one day.

Kiểu phân định này là một trong những kỷ luật (discipline) quan trọng nhất của thiết kế phần mềm tốt. Nếu bạn có một bộ test (test suite) tử tế và bạn thành thục kỷ luật refactor, thì chi phí để thêm một tính năng mới rồi cập nhật thiết kế cho khớp với tính năng đó thường sẽ nhỏ hơn nhiều so với chi phí hiện thực và bảo trì tất cả những cái móc mà một ngày nào đó biết đâu bạn sẽ cần.

Covered by Tests — Được test phủ kín

What is a good **test coverage** number: 80%? 90%? Many teams treat such numbers as laudable goals. I disagree. The only reasonable coverage goal is 100%.

Bao nhiêu phần trăm độ phủ test (test coverage) thì gọi là tốt: 80%? 90%? Nhiều đội xem những con số ấy là mục tiêu đáng khen. Tôi không đồng ý. Mục tiêu phủ test hợp lý duy nhất là 100%.

What other number could possibly make sense? If you are happy with 80% coverage, it means you don’t know if 20% of your code works. How could you possibly be happy with that? How could your customers be happy with that?

Chứ còn con số nào khác có thể hợp lý được nữa? Nếu bạn hài lòng với độ phủ 80%, nghĩa là bạn không biết 20% mã của mình có chạy đúng hay không. Sao bạn có thể hài lòng với điều đó được? Sao khách hàng của bạn có thể hài lòng với điều đó được?

An Asymptotic Goal — Một mục tiêu tiệm cận

You might complain that 100% is an unreachable goal. I might even agree with you. Achieving 100% line and branch coverage is no mean feat. It may, in fact, be impractical depending on the situation. But that does not mean that the coverage you currently have cannot be improved.

Bạn có thể phàn nàn rằng 100% là mục tiêu không thể với tới. Thậm chí tôi có khi còn đồng ý với bạn. Đạt 100% độ phủ dòng lệnh và nhánh không phải chuyện đơn giản. Thật ra, tùy tình huống mà nó có thể là bất khả thi. Nhưng điều đó không có nghĩa là độ phủ bạn đang có không thể cải thiện được.

Think of 100% as an asymptotic goal. You may never reach it, but that's no excuse for not trying to get closer and closer with every check-in.

Hãy coi 100% như một mục tiêu tiệm cận. Bạn có thể chẳng bao giờ chạm tới nó, nhưng đó không phải cái cớ để không cố tiến gần hơn, gần hơn nữa sau mỗi lần check-in.

I have personally participated in projects that grew to many tens of thousands of lines of code while constantly keeping the code coverage percentage in the very high 90s.

Đích thân tôi từng tham gia những dự án phình lên tới hàng chục nghìn dòng mã mà vẫn giữ được tỷ lệ phủ mã ở mức trên 90% rất cao suốt chặng đường.

Design? — Thiết kế ư?

But what does high code coverage have to do with simple design? Why is coverage the first rule?

Nhưng độ phủ mã cao thì liên quan gì tới thiết kế đơn giản? Vì sao độ phủ lại là quy tắc đầu tiên?

Testable code is decoupled code.

Mã kiểm thử được là mã đã được tách rời (decoupling).

In order to achieve high line and branch coverage of each individual part of the code, each of those parts must be made accessible to the **test** code. That means those parts must be so well decoupled from the rest of the code that they can be isolated and invoked from an individual test. Therefore, those tests are not only tests of **behavior**, they are also tests of **decoupling**. The act of writing isolated tests is an act of design, because the code being tested must be designed to be tested.

Để đạt độ phủ dòng lệnh và nhánh cao cho từng phần riêng lẻ của mã, mỗi phần ấy phải tiếp cận được từ mã test. Nghĩa là những phần đó phải được tách rời khỏi phần còn lại của mã đủ tốt để có thể cô lập và gọi ra từ một test riêng lẻ. Vì thế, những test đó không chỉ kiểm thử hành vi, chúng còn kiểm thử mức độ tách rời. Hành động viết test cô lập chính là một hành động thiết kế, bởi mã được kiểm thử buộc phải được thiết kế sao cho kiểm thử được.

Tests don't just drive you to create decoupled and robust designs. They also allow you to improve those designs with time. As we have discussed many times before in these pages, a trusted suite of tests vastly reduces the fear of change. If you have such a suite, and if that suite executes quickly, then you can improve the design of the code every time you find a better approach. When the requirements change in a way that the current design does not easily afford, the tests will allow you to fearlessly shift the design to better match those new requirements.

Test không chỉ thúc bạn tạo ra những thiết kế tách rời và vững chãi. Chúng còn cho phép bạn cải thiện những thiết kế đó theo thời gian. Như đã bàn nhiều lần trong những trang

sách này, một bộ test đáng tin cậy làm giảm mạnh nỗi sợ thay đổi. Nếu bạn có một bộ test như vậy, và nếu bộ test đó chạy nhanh, thì bạn có thể cải thiện thiết kế của mã mỗi khi tìm ra một cách làm tốt hơn. Khi yêu cầu thay đổi theo hướng mà thiết kế hiện tại không dễ đáp ứng, các test sẽ cho phép bạn không chút sợ hãi mà xoay chuyển thiết kế cho khớp hơn với những yêu cầu mới ấy.

And this is why this rule is the first and most important rule of simple design. Without a suite of tests that covers the system, the other three rules become impractical, because those rules are best applied after the fact. Those other three rules are rules that involve refactoring. And refactoring is virtually impossible without a good, comprehensive suite of tests.

Và đó là lý do quy tắc này là quy tắc đầu tiên và quan trọng nhất của thiết kế đơn giản. Không có một bộ test phủ được hệ thống thì ba quy tắc còn lại trở nên bất khả thi, bởi những quy tắc đó phát huy tác dụng tốt nhất khi được áp dụng *sau khi mã đã có*. Ba quy tắc còn lại đều là những quy tắc dính tới refactor. Mà refactor thì gần như không thể làm nổi nếu thiếu một bộ test tốt và toàn diện.

Maximize Expression — Tối đa hoá khả năng biểu đạt

In the early decades of programming, the code we wrote could not reveal intent. Indeed, the very name “code” suggests that intent is obscured. Back in the late ’60s, code looked like this:

Trong những thập niên đầu của nghề lập trình, mã chúng ta viết ra không thể bộc lộ ý định. Thật vậy, chính cái tên “code” (mật mã) đã ngụ ý rằng ý định bị che khuất. Hồi cuối thập niên 1960, mã trông như thế này:

```

-----

/Routine TO TYPE A MESSAGE                                PAL3-V10D NO DATE    PAGE 1

                /ROUTINE TO TYPE A MESSAGE
00200 0200      *200
00200 7600      MONADR=7600
00200 7300  START,  CLA CLL          /CLEAR ACCUMULATOR AND LINK
00201 6046      TLS                /CLEAR TERMINAL FLAG
00202 1216      TAD BUFADR          /SET UP POINTER
00203 3217      DCA PNTR            /FOR GETTING CHARACTERS
00204 6041  NEXT,  TSF              /SKIP IF TERMINAL FLAG SET
00205 5204      JMP .-1             /NO: CHECK AGAIN
00206 1617      TAD I PNTR          /GET A CHARACTER
00207 6046      TLS                /PRINT A CHARACTER
00210 2217      ISZ PNTR            /DONE YET?
00211 7300      CLA CLL            /CLEAR ACCUMULATOR AND LINK
00212 1617      TAD I PNTR          /GET ANOTHER CHARACTER
00213 7640      SZA CLA             /JUMP ON ZERO AND CLEAR
00214 5204      JMP NEXT            /GET READY TO PRINT ANOTHER
00215 5631      JMP I MON           /RETURN TO MONITOR
00216 0220  BUFADR,  BUFF           /BUFFER ADDRESS
00217 0220  PNTR,    BUFF           /POINTER
00220 0215  BUFF,    215;212;"H;"E;"L;"L;"O;"!";0
00221 0212
00222 0310
00223 0305
00224 0314
00225 0314
00226 0317
00227 0241
00230 0000
00231 7600  MON,      MONADR        /MONITOR ENTRY POINT

```

Notice the ubiquitous comments. These were absolutely necessary because the code itself revealed nothing at all about the intent of the program.

Để ý mấy dòng comment nhan nhản khắp nơi. Chúng tuyệt đối cần thiết, bởi bản thân mã chẳng hé lộ chút gì về ý định của chương trình.

However, we do not work in the 1960s anymore. The languages that we use are immensely expressive. With the proper discipline, we can produce code that reads like “well-written prose.”

Tuy nhiên, chúng ta đâu còn làm việc ở thập niên 1960 nữa. Những ngôn ngữ ta dùng ngày nay có sức biểu đạt vô cùng lớn. Với kỷ luật đúng đắn, ta có thể tạo ra thứ mã đọc lên như “văn xuôi viết khéo”.

As an example of such code, consider this little bit of Java:

Làm ví dụ cho thứ mã như thế, hãy xem đoạn Java nho nhỏ này:

```

public class RentalCalculator {
    private List<Rental> rentals = new ArrayList<>();
    public void addRental(String title, int days) {
        rentals.add(new Rental(title, days));
    }
}

```

```

public int getRentalFee() {
    int fee = 0;
    for (Rental rental : rentals)
        fee += rental.getFee();
    return fee;
}
public int getRenterPoints() {
    int points = 0;
    for (Rental rental : rentals)
        points += rental.getPoints();
    return points;
}

```

If you were not a programmer on this project, you might not understand everything that is going on in this code. However, after even the most cursory glance, the basic intent of the designer is easy to identify. The names of the variables, functions, and types are deeply descriptive. The **structure** of the algorithm is easy to see. This code is expressive. This code is simple.

Nếu bạn không phải lập trình viên trong dự án này, có thể bạn sẽ không hiểu hết mọi chuyện đang diễn ra trong đoạn mã. Tuy nhiên, chỉ cần liếc qua hết sức qua loa, ý định căn bản của người thiết kế cũng dễ nhận ra. Tên các biến, hàm và kiểu đều mô tả rất sát. Cấu trúc của thuật toán dễ thấy. Đoạn mã này có sức biểu đạt. Đoạn mã này đơn giản.

The Underlying Abstraction — Tầng trừu tượng nằm bên dưới

Lest you think that expressivity is solely a matter of nice names for functions and variables, I should point out that there is another concern: the separation of levels and the exposition of the underlying abstraction.

Kéo bạn lại tưởng sức biểu đạt chỉ đơn thuần là chuyện đặt tên hay cho hàm và biến, tôi phải chỉ ra rằng còn một mối bận tâm nữa: sự tách bạch các mức và việc phơi bày tầng trừu tượng nằm bên dưới.

A software system is expressive if each line of code, each **function**, and each **module** lives in a well-defined partition that clearly depicts the level of the code and its place in the overall abstraction.

Một hệ thống phần mềm có sức biểu đạt nếu mỗi dòng mã, mỗi hàm và mỗi module đều nằm trong một phân vùng được xác định rõ ràng, phân vùng ấy khắc họa minh bạch mức của đoạn mã và vị trí của nó trong bức tranh trừu tượng tổng thể.

You may have found that last sentence difficult to parse, so let me be a bit clearer by being much more long-winded.

Chắc bạn thấy câu vừa rồi hơi khó nuốt, nên để tôi nói dài dòng hơn nhiều cho rõ ràng hơn một chút.

Imagine an application that has a complex set of requirements. The example I like to use is a payroll system.

Hãy hình dung một ứng dụng có tập yêu cầu phức tạp. Ví dụ tôi thích dùng là một hệ thống trả lương.

- Hourly employees are paid every Friday, based upon the time cards they have submitted. They are paid time-and-a-half for every hour they work after 40 hours in a week.

- Commissioned employees are paid on the first and third Friday of every month. They are paid a base salary plus a commission on the sales receipts they have submitted.
 - Nhân viên tính theo giờ được trả lương vào thứ Sáu hằng tuần, dựa trên các phiếu chấm công họ nộp. Họ được trả gấp rưỡi cho mỗi giờ làm vượt quá 40 giờ trong tuần.
 - Nhân viên ăn hoa hồng được trả lương vào thứ Sáu tuần thứ nhất và tuần thứ ba mỗi tháng. Họ được trả lương cơ bản cộng thêm hoa hồng trên các phiếu bán hàng họ nộp.
- Salaried employees are paid on the last day of the month. They are paid a fixed monthly salary.
 - Nhân viên ăn lương cứng được trả vào ngày cuối cùng của tháng. Họ được trả một mức lương tháng cố định.

It should not be hard for you to imagine a set of functions with a complex **switch statement** or if / else chain that captures these requirements. However, such a set of functions is likely to obscure the underlying abstraction. What is that underlying abstraction?

Hẳn bạn không khó hình dung ra một mớ hàm với câu lệnh switch phức tạp hoặc một chuỗi if / else để hứng trọn đồng yêu cầu này. Có điều, một mớ hàm như vậy nhiều khả năng sẽ che khuất tầng trừu tượng nằm bên dưới. Tầng trừu tượng đó là gì?

```
public List<Paycheck> run(Database db) {
    Calendar now = SystemTime.getCurrentDate();
    List<Paycheck> paychecks = new ArrayList<>();
    for (Employee e : db.getAllEmployees()) {
        if (e.isPayDay(now))
            paychecks.add(e.calculatePay());
    }
    return paychecks;
}
```

Notice that there is no mention of any of the hideous details that dominate the requirements. The underlying truth of this application is that we need to pay all employees on their payday. Separating the high-level policy from the low-level detail is the most fundamental part of making a design simple and expressive.

Để ý rằng chẳng có bóng dáng nào của những chi tiết gớm ghiếc đang thống trị phần yêu cầu. Sự thật cốt lõi của ứng dụng này là ta cần trả lương cho tất cả nhân viên vào ngày lĩnh lương của họ. Tách chính sách mức cao khỏi chi tiết mức thấp là phần nền tảng nhất để làm cho một thiết kế trở nên đơn giản và giàu sức biểu đạt.

Tests: The Other Half of the Problem — Test: nửa còn lại của vấn đề

When Kent Beck originally wrote this rule, he phrased it this way:

Khi Kent Beck viết ra quy tắc này lần đầu, ông diễn đạt nó thế này:

The system (code and tests) must communicate everything you want to communicate.

Hệ thống (mã và test) phải truyền đạt được mọi thứ bạn muốn truyền đạt.

There is a reason he phrased it that way: No matter how expressive you make the production code, it cannot communicate the context in which it is used. That's the job of the tests.

Ông diễn đạt như vậy là có lý do: dù bạn có làm cho mã sản phẩm giàu sức biểu đạt đến đâu, nó cũng không thể truyền đạt được bối cảnh mà nó được dùng. Đó là việc của test.

Every test you write, especially if those tests are isolated and decoupled, is a demonstration of how the production code is intended to be used. Well-written tests are example use cases for the parts of the code that they test.

Mỗi test bạn viết, nhất là khi những test đó được cô lập và tách rời, đều là một màn trình diễn về cách mã sản phẩm được kỳ vọng sẽ dùng ra sao. Test viết khéo chính là những use case mẫu cho phần mã mà chúng kiểm thử.

Thus, taken together, the code and the tests are an expression of what each element of the system does and how each element of the system should be used.

Vì vậy, *gộp cả hai lại*, mã và test là một biểu đạt về việc mỗi thành phần của hệ thống làm gì và mỗi thành phần của hệ thống nên được dùng thế nào.

What does this have to do with design? Everything, of course. Because the primary goal we wish to achieve with our designs is to make it easy for other programmers to understand, improve, and upgrade our systems. And there is no better way to achieve that goal than to demonstrate the intended usage of the system's abstractions through a comprehensive suite of executable tests.

Chuyện này thì liên quan gì tới thiết kế? Liên quan tất tật, dĩ nhiên rồi. Bởi mục tiêu hàng đầu ta muốn đạt được với thiết kế của mình là làm cho các lập trình viên khác dễ hiểu, dễ cải thiện và dễ nâng cấp hệ thống của ta. Và chẳng có cách nào đạt mục tiêu đó tốt hơn là trình diễn cách dùng đã dự tính cho các tầng trừu tượng của hệ thống thông qua một bộ test chạy được, đầy đủ và toàn diện.

Minimize Duplication — Giảm thiểu trùng lặp

As I have told you in several previous chapters, in the earliest days of software, there was no way to call one program from another. If you wanted to do something more than once, you had to duplicate the code. Programmers wrote code snippets into their notebooks so that they could copy them into their programs when needed. They called those snippets subroutines.

Như tôi đã kể với bạn trong vài chương trước, vào những ngày đầu của phần mềm, chẳng có cách nào để gọi chương trình này từ chương trình kia. Nếu muốn làm một việc gì đó hơn một lần, bạn phải sao chép mã ra. Lập trình viên chép các mẫu mã vào sổ tay để khi cần thì chép chúng vào chương trình của mình. Họ gọi những mẫu mã đó là *subroutine*.

In the 1950s, it became possible for programs to call subroutines and have those subroutines return to the point of call. In those days, we had no source code editors at all.

Đến thập niên 1950, các chương trình đã có thể gọi subroutine và những subroutine ấy có thể quay về đúng điểm gọi. Thời đó, chúng tôi chẳng có trình soạn thảo mã nguồn nào cả.

We wrote our code, using #2 pencils, on preprinted coding forms. We therefore avoided most duplication because it was easier for us to create a single instance of a code snippet and call it. The calls took many CPU cycles, but the lack of duplication saved a lot of memory. It was often a good trade-off.

Chúng tôi viết mã bằng bút chì số 2, lên những tờ biểu mẫu viết mã in sẵn. Vì thế chúng tôi tránh gần như mọi sự trùng lặp, bởi tạo ra một bản duy nhất của mẫu mã rồi gọi nó thì dễ hơn nhiều. Việc gọi tốn khá nhiều chu kỳ CPU, nhưng không trùng lặp lại tiết kiệm được nhiều bộ nhớ. Đó thường là một sự đánh đổi hời.

Then came the source code editors. And with those editors came copy/paste operations. Suddenly it became easy to copy a snippet of code from one routine and paste it into a new routine and then fiddle with it until it worked. With memory becoming cheaper and bigger, the cost of duplication became less of a concern. Thus, as the years went by, more and more systems exhibited massive amounts of duplication in the code.

Rồi các trình soạn thảo mã nguồn xuất hiện. Và kèm theo chúng là thao tác copy/paste. Bổng chốc, chép một mẩu mã từ routine này rồi dán vào một routine mới và vọc vạch cho tới khi nó chạy được trở nên quá dễ. Bộ nhớ ngày càng rẻ và ngày càng lớn, nên cái giá của trùng lặp cũng bớt đáng lo. Thế là năm này qua năm khác, ngày càng nhiều hệ thống phơi bày ra lượng mã trùng lặp khổng lồ.

Duplication is usually problematic. Two or more similar stretches of code will often need to be modified together. Finding those similar stretches is hard. Properly modifying them is even harder because they exist in different contexts. Thus, duplication leads to fragility.

Trùng lặp (duplication) thường là chuyện phiền toái. Hai hay nhiều đoạn mã giống nhau thì thường sẽ phải sửa cùng lúc. Tìm ra những đoạn giống nhau ấy đã khó. Sửa chúng cho đúng còn khó hơn, bởi chúng nằm trong những bối cảnh khác nhau. Thế nên trùng lặp dẫn tới sự mong manh dễ vỡ.

In general, it is best to reduce similar stretches of code into a single instance by abstracting the code into a new function and providing it with appropriate arguments that communicate any differences in context.

Nhìn chung, tốt nhất là gom các đoạn mã giống nhau về một bản duy nhất bằng cách trừu tượng hoá đoạn mã đó thành một hàm mới, rồi cấp cho nó những đối số phù hợp để truyền đạt mọi khác biệt về bối cảnh.

Sometimes that strategy is not workable. For example, sometimes the duplication is in code that traverses a complex **data structure**. Many different parts of the system may wish to traverse that structure, and they will use the same looping and traversal code only to then operate on the **data** structure in the body of that code.

Đôi khi chiến lược đó không dùng được. Chẳng hạn, đôi khi phần trùng lặp nằm trong đoạn mã duyệt một cấu trúc dữ liệu (data structure) phức tạp. Nhiều phần khác nhau của hệ thống có thể muốn duyệt cấu trúc đó, và chúng sẽ dùng cùng một đoạn mã vòng lặp và duyệt, chỉ để rồi thao tác lên cấu trúc dữ liệu ở phần thân của đoạn mã ấy.

As the structure of the data changes over time, programmers will have to find all the duplications of the traversal code and properly update them. The more the traversal code is duplicated, the higher the risk of fragility.

Khi cấu trúc của dữ liệu đổi thay theo thời gian, lập trình viên sẽ phải lần ra tất cả các bản sao của đoạn mã duyệt và cập nhật chúng cho đúng. Đoạn mã duyệt càng bị nhân bản nhiều thì rủi ro mong manh dễ vỡ càng cao.

The duplications of the traversal code can be eliminated by encapsulating it in once place, and using lambdas, Command objects, the Strategy pattern, or even the Template **Method** pattern 3 to pass the necessary operations into the traversal.

Có thể loại bỏ những bản sao của mã duyệt bằng cách đóng gói (encapsulation) nó vào một chỗ duy nhất, rồi dùng lambda, đối tượng Command, mẫu Strategy, hay thậm chí mẫu Template Method 3 để truyền các thao tác cần thiết vào bên trong quá trình duyệt.

3 [GOF95].

Accidental Duplication — Trùng lặp tình cờ

Not all duplication should be eliminated. There are situations in which two stretches of code may be very similar, even identical, but will change for very different reasons. ⁴ I call this accidental duplication. Accidental duplicates should not be eliminated. The duplication should be allowed to persist. As the requirements change, the duplicates will evolve separately and the accidental duplication will dissolve.

Không phải mọi trùng lặp đều nên bị loại bỏ. Có những tình huống mà hai đoạn mã trông rất giống nhau, thậm chí y hệt nhau, nhưng lại sẽ thay đổi vì những lý do rất khác nhau. ⁴ Tôi gọi đó là *trùng lặp tình cờ*. Không nên loại bỏ những bản trùng lặp tình cờ. Hãy để sự trùng lặp đó tồn tại. Khi yêu cầu thay đổi, các bản sao ấy sẽ tiến hoá theo những hướng riêng và sự trùng lặp tình cờ sẽ tự tan biến.

⁴ See “SRP: The Single Responsibility Principle” in Chapter 19, “The SOLID Principles.”

⁴ Xem “SRP — Nguyên tắc Trách nhiệm Đơn nhất” trong Chương 19, “*Các nguyên tắc SOLID*”.

It should be clear that managing duplication is nontrivial. Identifying which duplications are real and which are accidental and then encapsulating and isolating the real duplications requires a significant amount of thought and care.

Hẳn đã rõ rằng quản lý trùng lặp không phải chuyện vặt. Nhận ra đâu là trùng lặp thật, đâu là trùng lặp tình cờ, rồi đóng gói và cô lập những trùng lặp thật, đòi hỏi rất nhiều suy nghĩ và cẩn trọng.

Determining the real duplications from the accidental duplications depends strongly on how well the code expresses its intent. Accidental duplications have divergent intent. Real duplications have convergent intent.

Việc phân định trùng lặp thật với trùng lặp tình cờ phụ thuộc rất lớn vào chuyện mã bộc lộ ý định của nó tốt đến đâu. Trùng lặp tình cờ có ý định phân kỳ. Trùng lặp thật có ý định hội tụ.

Encapsulating and isolating the real duplications, using abstraction, lambdas, and design patterns, involves a substantial amount of refactoring. And refactoring requires a good, solid suite of tests.

Đóng gói và cô lập những trùng lặp thật — bằng trừu tượng hoá, lambda và các mẫu thiết kế — kéo theo một lượng refactor đáng kể. Mà refactor thì cần một bộ test tốt và chắc chắn.

Therefore, eliminating duplication is third in the priority list of the rules of simple design. First come the tests, then the expression, and then the duplication.

Vì thế, loại bỏ trùng lặp đứng thứ ba trong thứ tự ưu tiên của các quy tắc thiết kế đơn giản. Trước hết là test, rồi tới sức biểu đạt, rồi mới tới trùng lặp.

Minimize Size — Giảm thiểu kích thước

A simple design is composed of as few simple elements as possible without compromising test coverage and expression. Simple elements are small. The last rule of simple design suggests that after you’ve gotten all the tests to pass, and after you have made the code as expressive as possible,

and after you have minimized duplication, then you should work to decrease the number of modules, classes, functions, and lines.

Một thiết kế đơn giản được ghép nên từ càng ít thành phần đơn giản càng tốt, miễn là không hy sinh độ phủ test và sức biểu đạt. Thành phần đơn giản thì nhỏ. Quy tắc cuối cùng của thiết kế đơn giản gợi ý rằng sau khi bạn đã làm cho toàn bộ test pass, sau khi bạn đã làm cho mã giàu sức biểu đạt hết mức có thể, và sau khi bạn đã giảm thiểu trùng lặp, thì bạn nên bắt tay vào giảm bớt số lượng module, class, hàm và dòng mã.

Simple Design — Thiết kế đơn giản

Many years back, Kent Beck and I were having a discussion on the principles of design. He said something that has always stuck with me. He said that if you followed these four things as diligently as possible, all other design principles would be satisfied—the principles of design can be reduced to coverage, expression, singularization, and reduction.

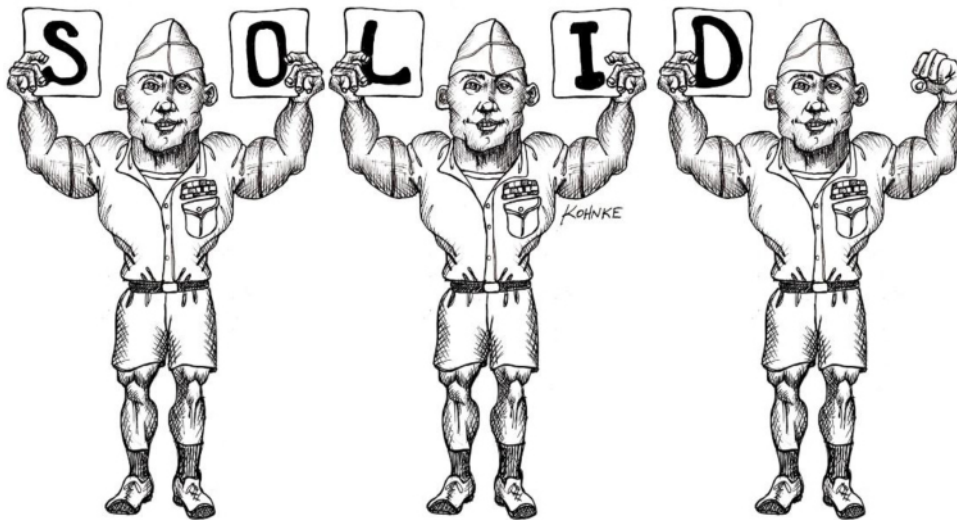
Nhiều năm trước, Kent Beck và tôi có một cuộc trò chuyện về các nguyên tắc thiết kế. Ông nói một câu mà tôi nhớ mãi. Ông bảo rằng nếu bạn theo đuổi bốn điều này chuyên cần hết mức có thể, thì mọi nguyên tắc thiết kế khác đều sẽ được thoả mãn — các nguyên tắc thiết kế có thể rút gọn về bốn chữ: độ phủ, sức biểu đạt, gom về một mối, và tinh giản.

I don't know if this is true or not. I don't know if a perfectly covered, expressed, singularized, and reduced program necessarily conforms to the Open–Closed Principle (OCP) or the Single Responsibility Principle (SRP). What I am very sure of, however, is that knowing and studying the principles of good design and good architecture (e.g., the SOLID principles) makes it much easier to create well-partitioned and simple designs.

Tôi không biết điều này có đúng hay không. Tôi không biết một chương trình được phủ test hoàn hảo, biểu đạt hoàn hảo, gom về một mối hoàn hảo và tinh giản hoàn hảo thì có nhất thiết tuân theo OCP — Nguyên tắc Đóng–Mở (Open–Closed Principle) hay SRP — Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle) hay không. Tuy nhiên, điều tôi rất chắc chắn là: hiểu và nghiền ngẫm các nguyên tắc thiết kế tốt và kiến trúc tốt (chẳng hạn các nguyên tắc SOLID) sẽ khiến việc tạo ra những thiết kế đơn giản và phân vùng gọn ghẽ dễ dàng hơn nhiều.

Chapter 19. The SOLID Principles —

Chương 19. Các nguyên tắc SOLID



This chapter is a significantly abridged and edited set of excerpts from my book **Clean Architecture**. It is here to give you a quick reference to the principles mentioned elsewhere in the book, and to provide you with directions for further study.

Chương này là một tập trích đoạn đã được rút gọn và biên tập lại đáng kể từ cuốn **Clean Architecture** của tôi. Nó có mặt ở đây để bạn có một chỗ tra cứu nhanh những nguyên tắc được nhắc tới ở các chương khác, và để chỉ cho bạn hướng tìm hiểu sâu hơn.

Good software systems begin with **clean code**. If the bricks aren't well made, the architecture of the building doesn't matter much. On the other hand, you can make a substantial **mess** with well-made bricks. This is where **the SOLID principles** come in.

Hệ thống phần mềm tốt bắt đầu từ **clean code**. Nếu viên gạch làm không ra hồn thì kiến trúc (architecture) của toà nhà cũng chẳng còn mấy ý nghĩa. Ở chiều ngược lại, bạn hoàn toàn có thể dựng nên một mớ hỗn độn đồ sộ từ những viên gạch tử tế. Đó chính là chỗ các nguyên tắc **SOLID** bước vào.

The SOLID principles tell us how to arrange our functions and **data** structures into classes, and how those classes should be interconnected. The use of the word **class** does not imply that these principles are only applicable to **object-oriented** software. A class is simply a coupled grouping of functions and data. Every software system has such groupings, whether they are called classes or not. The SOLID principles apply to those groupings.

Các nguyên tắc SOLID cho ta biết nên sắp xếp hàm và cấu trúc dữ liệu (data structure) vào các class ra sao, và các class đó nên nối kết với nhau thế nào. Việc dùng chữ class không hàm ý rằng những nguyên tắc này chỉ áp dụng cho phần mềm hướng đối tượng. Class đơn giản là một nhóm hàm và dữ liệu gắn kết với nhau. Hệ thống phần mềm nào cũng có những nhóm như thế, dù chúng có được gọi là class hay không. Các nguyên tắc SOLID áp dụng cho những nhóm đó.

The goal of the principles is the creation of mid-level software structures that

Mục tiêu của các nguyên tắc này là tạo ra những cấu trúc phần mềm tầm trung mà

- Tolerate change
- Are easy to understand
- Are the basis of components that can be used in other software systems
 - Chịu được thay đổi
 - Dễ hiểu
 - Là nền tảng cho các component có thể tái sử dụng trong những hệ thống phần mềm khác

The term mid-level refers to the fact that these principles are applied by programmers working at the **module** level. They are applied just above the level of functions and help define the kinds of software structures used within modules and components.

Chữ tầm trung ám chỉ chuyện các nguyên tắc này được lập trình viên áp dụng ở mức module. Chúng được áp dụng ngay phía trên mức hàm, và giúp định hình những kiểu cấu trúc phần mềm dùng bên trong module và component.

Just as it is possible to create a substantial mess with well-made bricks, it is also possible to create a systemwide mess with well-designed mid-level components. So, once we have covered the SOLID principles, we will move on to their counterparts in the **component** world, and then, in Part III, to the principles of high-level architecture.

Cũng như bạn có thể dựng một mớ hỗn độn đồ sộ từ những viên gạch tử tế, bạn hoàn toàn có thể tạo ra một mớ hỗn độn cỡ toàn hệ thống từ những component tầm trung được thiết kế tốt. Vì vậy, sau khi đi qua các nguyên tắc SOLID, chúng ta sẽ chuyển sang những nguyên tắc tương ứng trong thế giới component, rồi tới Phần III là các nguyên tắc kiến trúc mức cao.

The history of the SOLID principles is long. I began to assemble them in the late '80s while debating software design principles with others on Usenet (an early social media platform). Over the years, the principles have shifted and changed. Some were deleted. Others were merged. Still others were added. The final grouping stabilized in the early 2000s, though I presented them in a different order.

Các nguyên tắc SOLID có một lịch sử dài. Tôi bắt đầu gộp nhặt chúng vào cuối thập niên 80, trong lúc tranh luận về các nguyên tắc thiết kế phần mềm với mọi người trên Usenet (một nền tảng mạng xã hội thời kỳ đầu). Qua năm tháng, các nguyên tắc dịch chuyển và biến đổi. Một số bị bỏ đi. Một số được gộp lại. Một số khác được thêm vào. Nhóm cuối cùng ổn định vào đầu những năm 2000, dù khi đó tôi trình bày chúng theo thứ tự khác.

Then, in 2004 or thereabouts, Michael Feathers sent me an email saying that if I rearranged the principles, their first letters of their initialisms would spell the word solid. And thus, the SOLID principles were born. The sections that follow describe each principle in detail. Here is the executive summary:

Rồi khoảng năm 2004, Michael Feathers gửi email cho tôi, bảo rằng nếu tôi sắp xếp lại thứ tự các nguyên tắc thì chữ cái đầu của các tên viết tắt sẽ ghép thành chữ solid. Và thế là các

nguyên tắc SOLID ra đời. Những mục tiếp theo mô tả từng nguyên tắc một cách chi tiết. Còn đây là bản tóm tắt cho người bận rộn:

- SRP—The Single Responsibility Principle: The best **structure** for a software system is heavily influenced by the social structure of the organization that uses it, such that each software module has one, and only one, reason to change.
- OCP—The Open–Closed Principle: Bertrand Meyer made this famous in the '80s. The gist is that in order for software systems to be easy to change, they must be designed to allow the **behavior** of those systems to be changed by adding new code, rather than by changing existing code.
- LSP—The Liskov Substitution Principle: This is Barbara Liskov's famous definition of subtypes, from 1988. In short, this principle says that in order to build software systems from interchangeable parts, those parts must adhere to a contract that allows them to be substituted one for another.
- ISP—The **Interface** Segregation Principle: This principle advises software designers to avoid depending on things that they don't use.
 - SRP — Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle): Cấu trúc tốt nhất cho một hệ thống phần mềm chịu ảnh hưởng nặng nề từ cấu trúc xã hội của tổ chức sử dụng nó, sao cho mỗi module phần mềm chỉ có một, và duy nhất một, lý do để thay đổi.
 - OCP — Nguyên tắc Đóng–Mở (Open–Closed Principle): Bertrand Meyer làm nguyên tắc này nổi tiếng vào thập niên 80. Cốt lõi của nó là: để hệ thống phần mềm dễ thay đổi, chúng phải được thiết kế sao cho hành vi của hệ thống có thể được thay đổi bằng cách thêm mã mới, chứ không phải bằng cách sửa mã đang có.
 - LSP — Nguyên tắc Thay thế Liskov (Liskov Substitution Principle): Đây là định nghĩa nổi tiếng của Barbara Liskov về kiểu con, từ năm 1988. Nói ngắn gọn, nguyên tắc này bảo rằng để xây dựng hệ thống phần mềm từ những bộ phận có thể hoán đổi cho nhau, các bộ phận đó phải tuân theo một hợp đồng cho phép chúng thay thế lẫn nhau.
 - ISP — Nguyên tắc Phân tách Interface (Interface Segregation Principle): Nguyên tắc này khuyên người thiết kế phần mềm tránh phụ thuộc vào những thứ mình không dùng tới.
- DIP—The **Dependency** Inversion Principle: This principle says that the code that implements high-level policy should not depend on the code that implements low-level detail. Rather, low-level details should depend upon high-level policies.
 - DIP — Nguyên tắc Đảo ngược Phụ thuộc (Dependency Inversion Principle): Nguyên tắc này nói rằng mã hiện thực chính sách mức cao không nên phụ thuộc vào mã hiện thực chi tiết mức thấp. Ngược lại mới đúng: chi tiết mức thấp phải phụ thuộc vào chính sách mức cao.

These principles have been described in detail in many different publications ¹ over the years. The sections that follow are not meant to replace those detailed writings. If you are not already familiar with these principles, what follows will be a good introduction, but insufficient for a deep understanding. You would be well advised to study them in the footnoted documents.

Những nguyên tắc này đã được mô tả chi tiết trong rất nhiều ấn phẩm khác nhau ¹ suốt bao năm qua. Các mục tiếp theo không nhằm thay thế những bài viết chi tiết ấy. Nếu bạn chưa quen với các nguyên tắc này, phần dưới đây sẽ là một lời giới thiệu tốt, nhưng chưa đủ để hiểu sâu. Bạn nên tìm đọc chúng trong các tài liệu ghi ở chú thích chân trang.

¹ For example, [PPP02], <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod> , and [https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)) (or just google SOLID).

1 Ví dụ, [PPP02], <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod> , và [https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)) (hoặc cứ google SOLID).

SRP: The Single Responsibility Principle — SRP: Nguyên tắc Trách nhiệm Đơn nhất

Of all the SOLID principles, this might be the least well understood. That's likely because it has a particularly inappropriate name. It is too easy for programmers to hear the name and then assume that it means that every module should do just one thing.

Trong tất cả các nguyên tắc SOLID, đây có lẽ là nguyên tắc bị hiểu sai nhiều nhất. Nhiều khả năng vì nó mang một cái tên đặc biệt không phù hợp. Lập trình viên quá dễ nghe cái tên rồi mặc định rằng nó có nghĩa là mỗi module chỉ nên làm đúng một việc.

There is a principle like that, as we have seen in previous chapters. It is true that a **function** should do one, and only one, thing. We use that principle when we are **refactoring** large functions into smaller functions. We use that principle at the lowest levels. But it is not one of the SOLID principles. It is not the SRP.

Có một nguyên tắc như vậy thật, như ta đã thấy ở các chương trước. Đúng là một hàm chỉ nên làm một, và duy nhất một, việc. Ta dùng nguyên tắc đó khi refactor những hàm to thành các hàm nhỏ. Ta dùng nó ở tầng thấp nhất. Nhưng nó không phải là một trong các nguyên tắc SOLID. Nó không phải SRP.

Historically, the SRP has been described this way:

Trong lịch sử, SRP được mô tả thế này:

A module should have one, and only one, reason to change.

Một module chỉ nên có một, và duy nhất một, lý do để thay đổi.

Software systems are changed in order to satisfy users and stakeholders. So those users and stakeholders are the “reason to change” that the principle is talking about. Indeed, we can rephrase the principle to say this:

Hệ thống phần mềm thay đổi là để làm hài lòng người dùng và các bên liên quan (stakeholder). Vậy nên chính những người dùng và bên liên quan ấy mới là “lý do để thay đổi” mà nguyên tắc đang nói tới. Thực vậy, ta có thể phát biểu lại nguyên tắc như sau:

A module should be responsible to one, and only one, user or **stakeholder**.

Một module chỉ nên chịu trách nhiệm trước một, và duy nhất một, người dùng hoặc bên liên quan.

Unfortunately, the words user and stakeholder aren't really the right words to use here. This is because there will likely be more than one user or stakeholder who wants the system changed in the same way. So we're really referring to a group of one or more people who require changes. We shall refer to that group as an actor .

Đáng tiếc, người dùng và bên liên quan không hẳn là những từ đúng để dùng ở đây. Lý do là rất có thể sẽ có nhiều hơn một người dùng hay bên liên quan cùng muốn hệ thống thay đổi theo một hướng. Vậy nên thứ ta thực sự nhắc tới là một nhóm gồm một hoặc nhiều người cùng yêu cầu thay đổi. Ta sẽ gọi nhóm đó là một tác nhân (actor).

Thus, the final version of the SRP is:

Vậy phiên bản cuối cùng của SRP là:

A module should be responsible to one, and only one, actor.

Một module chỉ nên chịu trách nhiệm trước một, và duy nhất một, tác nhân.

Now, what do we mean by the word module ? A module is just a cohesive set of functions and data structures. That word cohesive implies the SRP. **Cohesion** is the force that binds together the code responsible to a single actor.

Giờ thì, chữ module ở đây nghĩa là gì? Module chỉ đơn giản là một tập hàm và cấu trúc dữ liệu gắn kết với nhau. Chính chữ gắn kết đó hàm ý SRP. Độ gắn kết nội tại (cohesion) là lực buộc chặt vào nhau những đoạn mã cùng chịu trách nhiệm trước một tác nhân duy nhất.

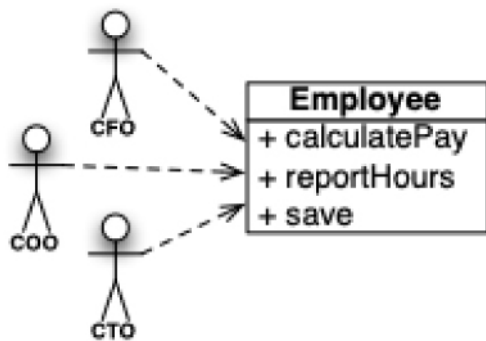
Perhaps the best way to understand this principle is by looking at one of the many symptoms that result from violating it.

Có lẽ cách tốt nhất để hiểu nguyên tắc này là nhìn vào một trong vô số triệu chứng sinh ra khi ta vi phạm nó.

Accidental Duplication — Trùng lặp tình cờ

Consider the Employee class from a payroll application. It has

Hãy xem class Employee trong một ứng dụng tính lương. Nó có three methods: `calculatePay()`, `reportHours()`, and `save()`.



This class violates the SRP because those three methods are responsible to three very different actors.

Class này vi phạm SRP, bởi ba method đó chịu trách nhiệm trước ba tác nhân rất khác nhau.

- The `calculatePay()` **method** is specified by the accounting department, which reports up to the CFO.
- The `reportHours()` method is specified and used by human resources, which reports up to the COO.
- The `save()` method is specified by the database administrators (DBAs), who report up to the CTO.
 - Method `calculatePay()` do phòng kế toán đặc tả, mà phòng này thì báo cáo lên CFO.
 - Method `reportHours()` do phòng nhân sự đặc tả và sử dụng, mà phòng này thì báo cáo lên COO.
 - Method `save()` do các quản trị viên cơ sở dữ liệu (DBA) đặc tả, mà họ thì báo cáo lên CTO.

By putting the source code for these three methods into a single Employee class, the developers have coupled each of these actors to the others. This **coupling** can cause the actions of the CFO's team to affect something that the COO's team depends upon.

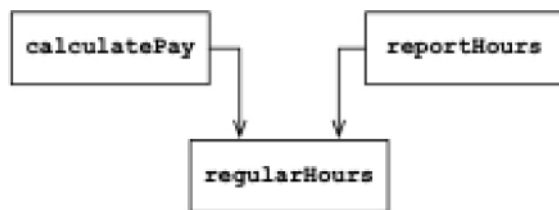
Bằng cách nhét mã nguồn của cả ba method vào chung một class Employee, các lập trình viên đã kết dính từng tác nhân này với các tác nhân kia. Sự kết dính đó có thể khiến hành động của nhóm CFO ảnh hưởng tới thứ mà nhóm COO đang dựa vào.

For example, let's say that the calculatePay() function and the

Chẳng hạn, giả sử hàm calculatePay() và hàm

reportHours() function share a common algorithm for calculating nonovertime hours. Let's also say that the developers, who are careful not to duplicate code, put that algorithm into a function named regularHours() .

reportHours() dùng chung một thuật toán để tính số giờ làm không tăng ca. Lại giả sử các lập trình viên, vốn rất cẩn thận không để mã bị trùng lặp, đã đưa thuật toán đó vào một hàm tên là regularHours() .



Now let's say that the CFO's team decides that the way nonovertime hours are calculated needs to be tweaked. But the COO's team in HR does not want that particular tweak, because they use nonovertime hours for a different purpose.

Giờ giả sử nhóm của CFO quyết định rằng cách tính giờ làm không tăng ca cần chỉnh lại đôi chút. Nhưng nhóm của COO bên nhân sự lại không muốn chỉnh sửa đó, vì họ dùng số giờ không tăng ca cho một mục đích khác.

A developer is tasked to make the change and sees the convenient regularHours() function called by the calculatePay() method. Unfortunately, that developer does not notice that the function is also called by the reportHours() function.

Một lập trình viên được giao thực hiện thay đổi và trông thấy hàm regularHours() tiện lợi đang được method calculatePay() gọi. Đáng tiếc, anh ta không để ý rằng hàm này cũng được hàm reportHours() gọi.

The developer makes the required change and carefully tests it. The CFO's team validates that the new function works as desired, and the system is deployed.

Lập trình viên đó thực hiện thay đổi được yêu cầu và test cẩn thận. Nhóm của CFO xác nhận hàm mới chạy đúng như mong muốn, và hệ thống được triển khai.

Of course, the COO's team doesn't know that this is happening. The COO's team continues to use the reports generated by the reportHours() function—but now they contain incorrect numbers. Eventually the problem is discovered, and the COO is livid because the bad data has cost his budget millions of dollars.

Dĩ nhiên, nhóm của COO chẳng hay biết gì về chuyện đang diễn ra. Họ vẫn tiếp tục dùng các báo cáo do hàm reportHours() sinh ra — có điều giờ đây chúng chứa những con số sai.

Rốt cuộc thì vấn đề cũng bị phát hiện, và COO nổi trận lôi đình vì đồng dữ liệu sai đã ngốn mất hàng triệu đô la trong ngân sách của ông ta.

We've all seen things like this happen. It occurs because we put code that different actors depend upon into close proximity. The SRP says to separate the code that different actors depend upon .

Tất cả chúng ta đều từng chứng kiến những chuyện kiểu này. Nó xảy ra vì ta đặt sát cạnh nhau những đoạn mã mà các tác nhân khác nhau cùng phụ thuộc vào. SRP bảo hãy tách riêng những đoạn mã mà các tác nhân khác nhau phụ thuộc vào.

Solutions — Các giải pháp

There are many different solutions to this problem. Each moves the functions into different classes.

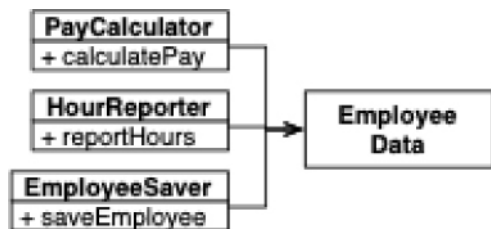
Có rất nhiều giải pháp khác nhau cho vấn đề này. Giải pháp nào cũng chuyển các hàm sang những class khác nhau.

Perhaps the most obvious way to do this is to separate the data from the functions. The three classes share access to

Có lẽ cách hiển nhiên nhất để làm chuyện đó là tách dữ liệu ra khỏi các hàm. Ba class cùng chia sẻ quyền truy cập vào

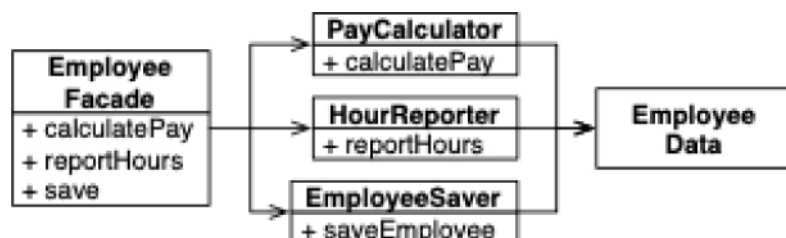
EmployeeData , which is a simple **data structure** with no methods. Each class holds only the source code necessary for its particular function. The three classes are not allowed to know about each other. Thus, any accidental **duplication** is avoided.

EmployeeData , một cấu trúc dữ liệu đơn giản không có method nào. Mỗi class chỉ giữ đúng phần mã nguồn cần cho chức năng riêng của nó. Ba class này không được phép biết gì về nhau. Nhờ vậy, mọi trùng lặp tình cờ đều bị loại trừ.



Of course, the downside of this solution is that the developers now have three classes that they have to instantiate and keep track of. A common solution to this dilemma is to use the Facade pattern.

Dĩ nhiên, mặt trái của giải pháp này là các lập trình viên giờ có tới ba class phải khởi tạo và theo dõi. Một cách xử lý thường gặp cho thể lưỡng nan này là dùng mẫu Facade.

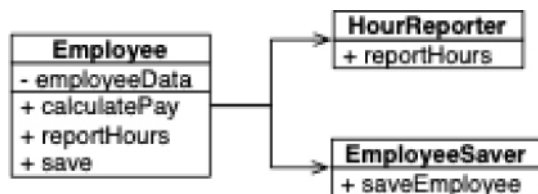


The EmployeeFacade contains very little code. It is responsible for instantiating and delegating to the classes with the functions.

EmployeeFacade chứa rất ít mã. Nó chỉ chịu trách nhiệm khởi tạo và uỷ nhiệm cho các class chứa những hàm kia.

Some developers prefer to keep the most important business rules closer to the data. This can be done by keeping the most important method in the original Employee class and then using that class as a facade for the lesser functions.

Một số lập trình viên thích giữ những quy tắc nghiệp vụ quan trọng nhất ở gần dữ liệu. Có thể làm điều đó bằng cách giữ method quan trọng nhất trong class Employee ban đầu, rồi dùng chính class ấy làm facade cho những hàm ít quan trọng hơn.



You might object to these solutions on the basis that every class would just contain one function. This is hardly the case. The number of functions required to calculate pay, generate a report, or save the data is likely to be large in each case. Each of those classes would likely have many other public methods, and many more private methods in them.

Bạn có thể phản đối những giải pháp này với lý do mỗi class tốt cuộc chỉ chứa đúng một hàm. Thực tế hiếm khi như vậy. Số hàm cần để tính lương, sinh báo cáo hay lưu dữ liệu nhiều khả năng đều rất lớn trong từng trường hợp. Mỗi class đó nhiều khả năng sẽ còn nhiều public method khác, và còn nhiều private method hơn nữa bên trong.

Higher Levels — Ở các tầng cao hơn

The SRP is about functions and classes. But the principle reappears in a different form at two more levels.

SRP nói về hàm và class. Nhưng nguyên tắc này còn tái xuất dưới dạng khác ở hai tầng nữa.

As we shall see, at the level of components it becomes the Common Closure Principle. And at the architectural level, it becomes the axis of change responsible for the creation of architectural boundaries. We'll be studying all of this in the chapters to come.

Như ta sẽ thấy, ở tầng component nó trở thành Nguyên tắc Đóng chung (Common Closure Principle). Còn ở tầng kiến trúc, nó trở thành trục thay đổi — thứ chịu trách nhiệm tạo ra các ranh giới kiến trúc (architectural boundary). Chúng ta sẽ tìm hiểu tất cả những điều này ở các chương sắp tới.

OCP: The Open–Closed Principle — OCP: Nguyên tắc Đóng–Mở



The OCP was coined in 1988 by Bertrand Meyer. 2 It says:

OCP được Bertrand Meyer đặt ra năm 1988. 2 Nó nói rằng:

2 [OOSC], p. 23.

2 [OOSC], tr. 23.

A software artifact should be open for extension but closed for modification.

Một tạo tác phần mềm nên mở cho việc mở rộng nhưng đóng với việc sửa đổi.

In other words, the behavior of a software artifact ought to be extendible, without having to modify that artifact.

Nói cách khác, hành vi của một tạo tác phần mềm phải có thể mở rộng được mà không cần sửa chính tạo tác đó.

This, of course, is the most fundamental reason that we study software design. When simple extensions to the requirements force massive changes to the software, the architects of that software system have engaged in a spectacular failure.

Đây, dĩ nhiên, là lý do căn bản nhất khiến chúng ta nghiên cứu thiết kế phần mềm. Khi những mở rộng đơn giản về yêu cầu lại buộc phần mềm phải thay đổi ồ ạt, thì kiến trúc sư của hệ thống ấy đã thất bại một cách ngoạn mục.

Most students of software design recognize the OCP as a principle that guides them in the design of classes and modules. But the principle takes on even greater significance when we consider the level of architectural components.

Phần lớn người học thiết kế phần mềm nhận ra OCP như một nguyên tắc dẫn đường khi thiết kế class và module. Nhưng nguyên tắc này còn mang ý nghĩa lớn hơn nữa khi ta xét tới tầng component kiến trúc.

A Thought Experiment — Một thí nghiệm tưởng tượng

A thought experiment will make this clear.

Một thí nghiệm tưởng tượng sẽ làm rõ chuyện này.

Imagine, for a moment, that we have a system that displays a financial summary on a web page. The data on the page is scrollable, and negative numbers are rendered in red.

Hãy hình dung trong chốc lát rằng ta có một hệ thống hiển thị bản tóm tắt tài chính trên trang web. Dữ liệu trên trang có thể cuộn, và số âm được tô đỏ.

Now imagine that the stakeholders ask that this same information be produced into a report to be printed on a black-and-white printer. The report should be properly paginated, with appropriate page headers, page footers, and column labels. Negative numbers should be surrounded by parentheses.

Giờ hãy hình dung các bên liên quan yêu cầu đúng thông tin đó phải được xuất thành một báo cáo in trên máy in đen trắng. Báo cáo phải được phân trang tử tế, có tiêu đề đầu trang, chân trang và nhãn cột phù hợp. Số âm phải được đặt trong ngoặc đơn.

Clearly there will be some new code to write. But how much old code will have to change?

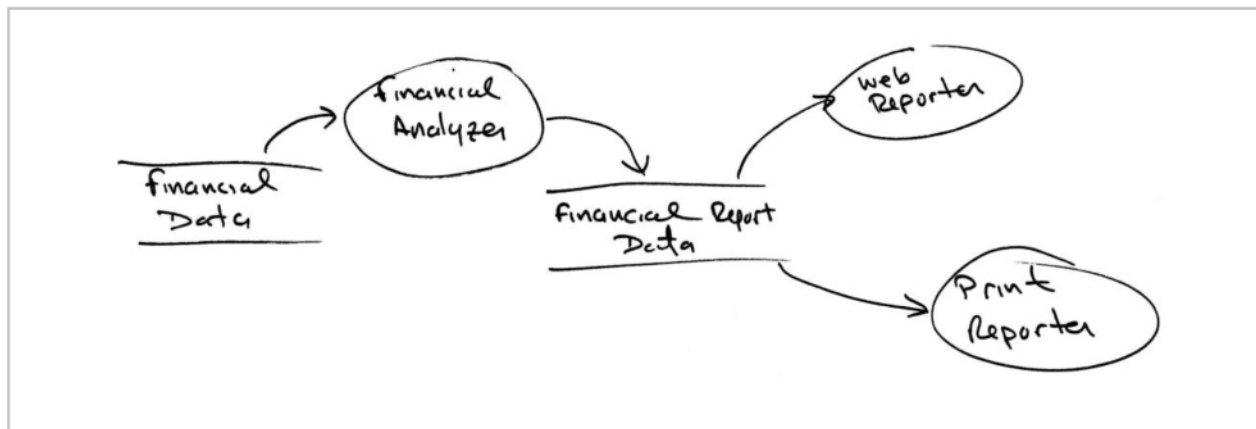
Rõ ràng sẽ có một ít mã mới phải viết. Nhưng bao nhiêu mã cũ sẽ phải sửa?

A good software architecture would reduce the amount of changed code to the barest minimum; ideally, zero.

Một kiến trúc phần mềm tốt sẽ giảm lượng mã phải sửa xuống mức tối thiểu; lý tưởng nhất là bằng không.

How? By properly separating the things that change for different reasons (the SRP), and then organizing the dependencies between those things properly (the DIP).

Bằng cách nào? Bằng cách tách bạch đúng đắn những thứ thay đổi vì những lý do khác nhau (SRP), rồi tổ chức các phụ thuộc (dependency) giữa chúng cho hợp lý (DIP).



Applying the SRP, we might come up with the data-flow view above. Some analysis procedure inspects the financial data and produces reportable data, which is then formatted appropriately by the two reporter processes.

Áp dụng SRP, ta có thể đi tới góc nhìn luồng dữ liệu ở trên. Một thủ tục phân tích nào đó xem xét dữ liệu tài chính và tạo ra dữ liệu có thể báo cáo, rồi hai tiến trình reporter sẽ định dạng dữ liệu đó cho phù hợp.

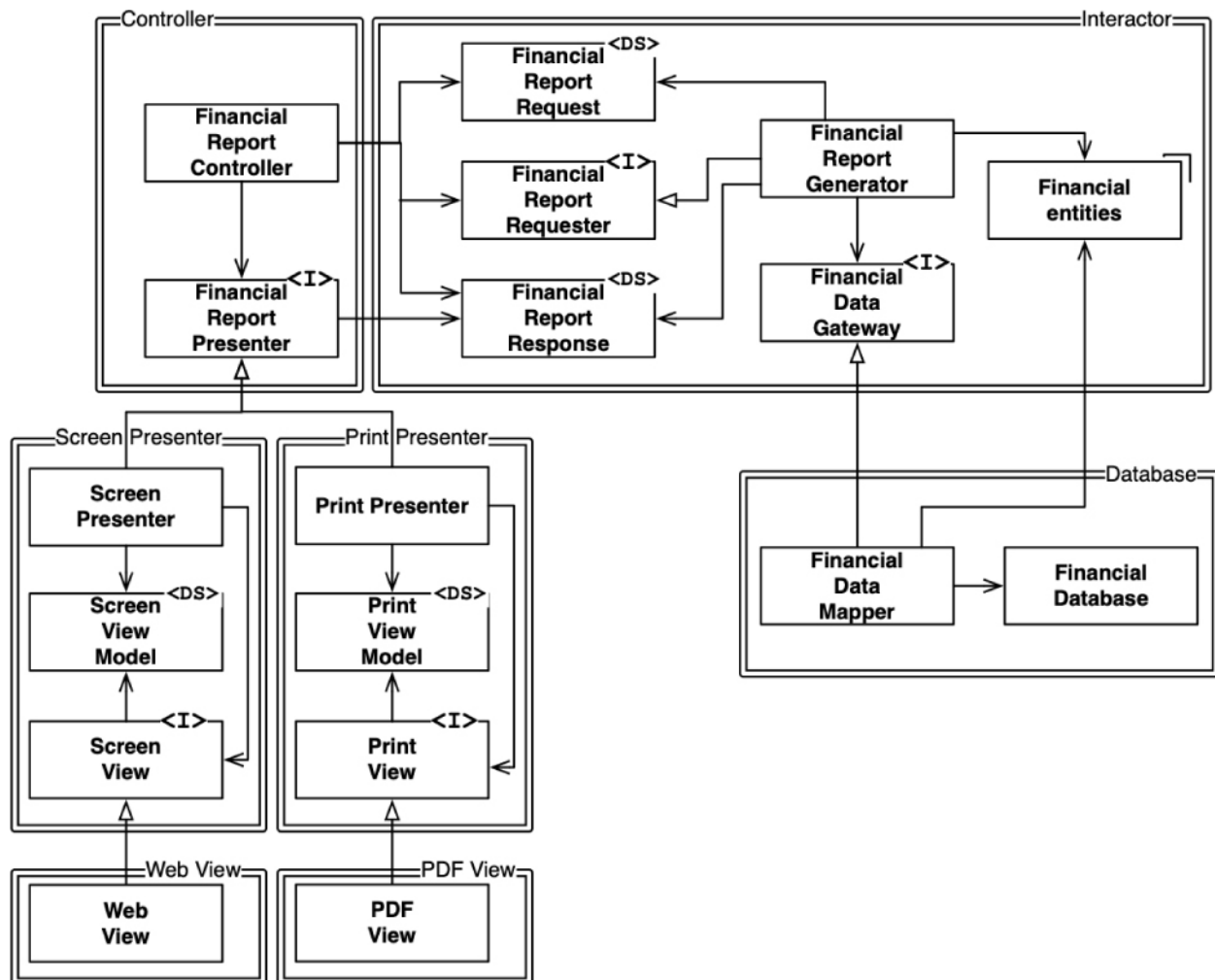
The essential insight here is that the report consists of two separate responsibilities: the calculation of the reported data and the presentation of that data into a web and printer form.

Điểm mấu chốt ở đây là báo cáo gồm hai trách nhiệm tách biệt: tính toán dữ liệu được báo cáo, và trình bày dữ liệu đó dưới dạng web và dạng in.

Sau khi đã tách như vậy, ta cần tổ chức các phụ thuộc trong mã nguồn sao cho thay đổi ở một trách nhiệm không kéo theo thay đổi ở trách nhiệm kia, và sao cho hành vi có thể mở rộng mà không cần sửa đổi.

We accomplish this by partitioning the processes into classes, and separating those classes into components as shown by the double lines in the following diagram.

Ta đạt được điều đó bằng cách chia các tiến trình thành class, rồi tách các class ấy vào những component khác nhau, như thể hiện bằng các đường kẻ đôi trong sơ đồ dưới đây.



Bob tương lai: Nói cho rõ, tôi chẳng bao giờ trông đợi bạn

vẽ một sơ đồ như thế này. Bản thân tôi chắc chắn cũng không vẽ khi đang suy nghĩ về những vấn đề kiểu này. Tôi có thể vẽ

432

the classes within those components would be

bản rút gọn mà bạn thấy dưới đây lên bảng trắng để cả nhóm cùng bàn. Phần lớn các class bên trong những component đó sẽ

so obvious to the team that drawing them would be superfluous. If any team member had a particular question, someone else on the team

hiển nhiên tới mức vẽ chúng ra là thừa. Nếu có thành viên nào trong nhóm thắc mắc điều gì cụ thể, một người khác trong nhóm

could scribble a portion of the diagram on the whiteboard or on a napkin.

có thể nguệch ngoạc một phần sơ đồ lên bảng trắng hoặc lên tờ giấy ăn.

The component at the upper left is the Controller. On the upper right we have the Interactor. On the lower right there is the Database. And on the lower left there are four components that represent the Presenters and the Views.

Component ở góc trên bên trái là Controller. Góc trên bên phải là Interactor. Góc dưới bên phải là Database. Còn góc dưới bên trái có bốn component đại diện cho các Presenter và các View.

Classes marked with are interfaces and with are data structures. Open arrowheads are using relationships. Closed arrowheads are implements or **inheritance** relationships.

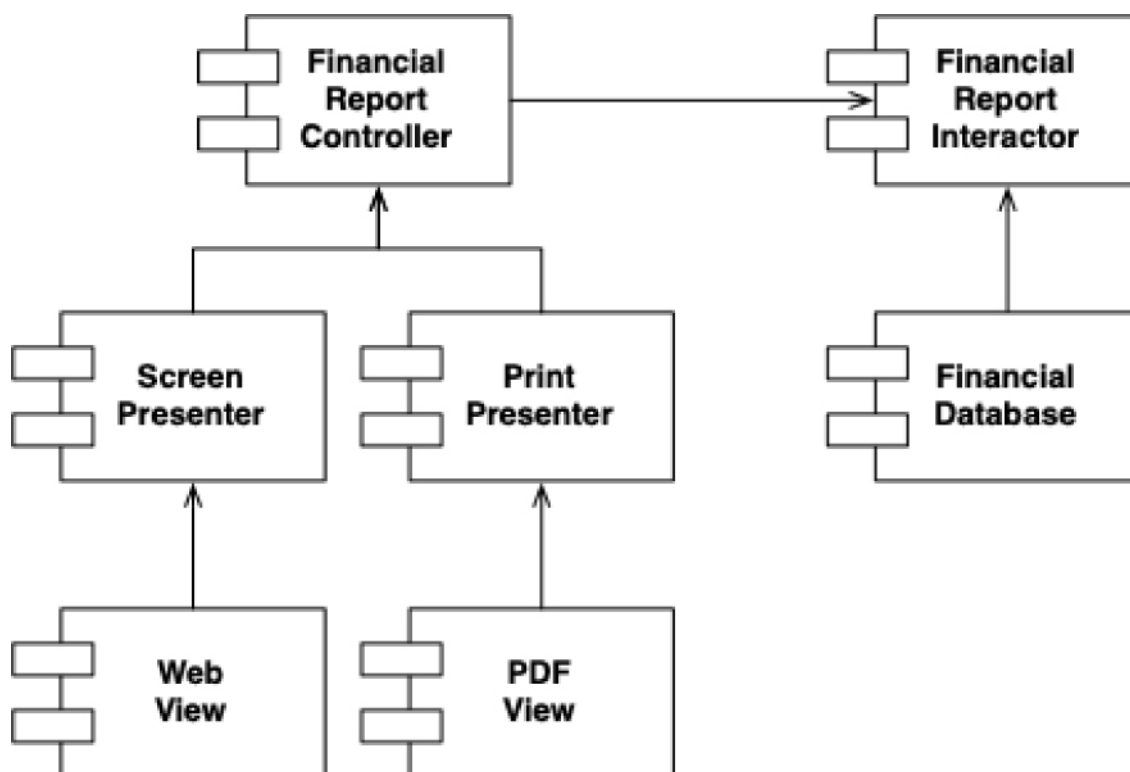
Các class được đánh dấu là interface, còn là cấu trúc dữ liệu. Đầu mũi tên rỗng biểu thị quan hệ sử dụng. Đầu mũi tên đặc biểu thị quan hệ implements hoặc kế thừa.

The first thing to notice is that all the dependencies are source code dependencies. An arrow pointing from class A to class B means that the source code of class A mentions the name of class B; but class B mentions nothing about class A. So, in the diagram above, FinancialDataMapper knows about FinancialDataGateway with an implements relationship, but FinancialDataGateway knows nothing at all about

Điều đầu tiên cần để ý là mọi phụ thuộc ở đây đều là phụ thuộc trong mã nguồn. Một mũi tên đi từ class A tới class B nghĩa là mã nguồn của class A có nhắc tên class B; còn class B thì không nhắc gì tới class A. Vậy nên, trong sơ đồ trên, FinancialDataMapper biết tới FinancialDataGateway qua quan hệ implements, nhưng FinancialDataGateway thì hoàn toàn không biết gì về

The next thing to notice is that each double line is crossed in one direction only . This means that all component relationships are unidirectional, as shown in the following diagram.

Điều tiếp theo cần để ý là mỗi đường kẻ đôi chỉ bị cắt qua theo một chiều duy nhất. Nghĩa là mọi quan hệ giữa các component đều một chiều, như trong sơ đồ dưới đây.



These arrows point toward the components that we want to protect from change.

Những mũi tên này trở về phía các component mà ta muốn bảo vệ khỏi thay đổi.

Let me say that again. If component A should be protected from changes in component B, then component B should depend upon component A.

Để tôi nói lại lần nữa. Nếu component A cần được bảo vệ khỏi những thay đổi trong component B, thì component B phải phụ thuộc vào component A.

We want to protect the Controller from changes in the Presenters. We want to protect the Presenters from changes in the Views. We want to protect the Interactor from changes in—anything.

Ta muốn bảo vệ Controller khỏi những thay đổi trong các Presenter. Ta muốn bảo vệ các Presenter khỏi những thay đổi trong các View. Ta muốn bảo vệ Interactor khỏi những thay đổi trong — bất cứ thứ gì.

The Interactor is in the position that best conforms to the OCP. Changes to the Database, or the Controller, or the Presenters, or the Views will have no impact upon the Interactor.

Interactor nằm ở vị trí tuân thủ OCP tốt nhất. Thay đổi ở Database, hay Controller, hay các Presenter, hay các View đều không tác động gì tới Interactor.

Why should the Interactor hold such a privileged position? Because it contains the business rules. The Interactor contains the highest-level policies of the application. All the other components are dealing with peripheral concerns. The Interactor deals with the central concern.

Tại sao Interactor lại được hưởng vị trí đặc quyền đó? Bởi nó chứa các quy tắc nghiệp vụ. Interactor chứa những chính sách ở tầng cao nhất của ứng dụng. Tất cả component còn lại đều đang lo những chuyện ngoại vi. Interactor lo chuyện cốt lõi.

Even though the Controller is peripheral to the Interactor, it is nevertheless central to the Presenters and Views. And while the Presenters might be peripheral to the Controller, they are central to the

Views.

Dù Controller là ngoại vi đối với Interactor, nó vẫn là trung tâm đối với các Presenter và các View. Và tuy các Presenter có thể là ngoại vi đối với Controller, chúng lại là trung tâm đối với các View.

Notice how this creates a hierarchy of protection based upon the notion of “level.” Interactors are the highest-level concept, so they are the most protected. Views are among the lowest-level concepts, so they are the least protected. Presenters are higher level than Views, but lower level than the Controller or the Interactor.

Hãy để ý cách điều này tạo nên một hệ thứ bậc bảo vệ dựa trên khái niệm “tầng”. Interactor là khái niệm ở tầng cao nhất, nên được bảo vệ nhiều nhất. View nằm trong nhóm khái niệm tầng thấp nhất, nên được bảo vệ ít nhất. Presenter ở tầng cao hơn View, nhưng thấp hơn Controller hay Interactor.

This is how the OCP works at the architectural level. Architects separate functionality based on how, why, and when it changes, and then they organize that separated functionality into a hierarchy of components. Higher-level components in that hierarchy are protected from the changes made to lower-level components.

Đó là cách OCP hoạt động ở tầng kiến trúc. Kiến trúc sư tách các chức năng dựa trên việc chúng thay đổi như thế nào, vì sao và khi nào, rồi tổ chức những chức năng đã tách ấy thành một hệ thứ bậc các component. Component ở tầng cao hơn trong hệ thứ bậc đó được bảo vệ khỏi những thay đổi ở các component tầng thấp hơn.

Directional Control — Điều khiển hướng phụ thuộc

If you recoiled in horror from the class design above, look again. Much of the complexity in that diagram was to make sure that the dependencies between the components pointed in the correct direction.

Nếu bạn rùng mình kinh hãi trước thiết kế class ở trên, hãy nhìn lại lần nữa. Phần lớn độ phức tạp trong sơ đồ đó chỉ nhằm bảo đảm rằng các phụ thuộc giữa những component trở đúng hướng.

For example, the FinancialDataGateway interface between the

Chẳng hạn, interface FinancialDataGateway nằm giữa

FinancialReportGenerator and theFinancialDataMapper exists in

order to invert the dependency that would otherwise have pointed from the Interactor component to the Database component. The same is true of the FinancialReportPresenter interface and the two View interfaces.

là để đảo ngược phụ thuộc mà nếu không có nó sẽ trở từ component Interactor sang component Database. Interface FinancialReportPresenter và hai interface View cũng vậy.

Information Hiding — Che giấu thông tin

The FinancialReportRequester interface serves a different purpose. It is there to protect the FinancialReportController from knowing too much about the internals of the Interactor. If that

interface were not there, then the Controller would have transitive dependencies upon the FinancialEntities .

Interface FinancialReportRequester lại phục vụ một mục đích khác. Nó ở đó để bảo vệ FinancialReportController khỏi việc biết quá nhiều về nội tình của Interactor. Nếu không có interface ấy, Controller sẽ có những phụ thuộc bắc cầu vào các FinancialEntities .

Transitive dependencies are a violation of the general principle that software entities should not depend upon things they don't directly use. We'll see that principle covered later when we talk about the ISP and the CRP.

Phụ thuộc bắc cầu là sự vi phạm nguyên tắc chung rằng các thực thể phần mềm không nên phụ thuộc vào những thứ chúng không dùng trực tiếp. Ta sẽ gặp lại nguyên tắc đó khi bàn về ISP và CRP.

So, even though our first priority is to protect the Interactor from changes to the Controller; we also want to protect the Controller from changes to the Interactor by hiding the internals of the Interactor.

Vậy nên, dù ưu tiên số một của ta là bảo vệ Interactor khỏi những thay đổi ở Controller, ta cũng muốn bảo vệ Controller khỏi những thay đổi ở Interactor, bằng cách che giấu nội tình của Interactor.

Conclusion — Kết luận

The OCP is one of the driving forces behind the design of systems. The goal is to make the system easy to extend without incurring a high impact of change. This is accomplished by partitioning the system into components and arranging them into a dependency hierarchy that protects higher-level components from changes in lower-level components.

OCP là một trong những động lực chính chi phối thiết kế hệ thống. Mục tiêu là làm cho hệ thống dễ mở rộng mà không phải gánh chịu tác động lớn từ thay đổi. Ta đạt được điều đó bằng cách chia hệ thống thành các component rồi sắp xếp chúng thành một hệ thống phân cấp phụ thuộc, sao cho các component ở mức cao được bảo vệ khỏi những thay đổi ở các component mức thấp.

LSP: The Liskov Substitution Principle — LSP: Nguyên tắc Thay thế Liskov

In 1988, Barbara Liskov wrote the following as a way of defining subtypes.

Năm 1988, Barbara Liskov đã viết những dòng sau như một cách định nghĩa subtype.

What is wanted here is something like the following substitution property: If for each object o1 of type S there is an object o2 of type T such that for all programs P defined in terms of T, the behavior of P is unchanged when o1 is substituted for o2 then S is a subtype of T. 3

Thứ ta cần ở đây là một tính chất thay thế đại loại như sau: Nếu với mỗi đối tượng o1 thuộc kiểu S đều tồn tại một đối tượng o2 thuộc kiểu T sao cho với mọi chương trình P được định nghĩa theo T, hành vi của P không đổi khi thay o2 bằng o1, thì S là một subtype của T. 3

3 Barbara Liskov, “**Data Abstraction** and Hierarchy,” ACM SIGPLAN Notices 23, no. 5 (May 1988): 17–34.

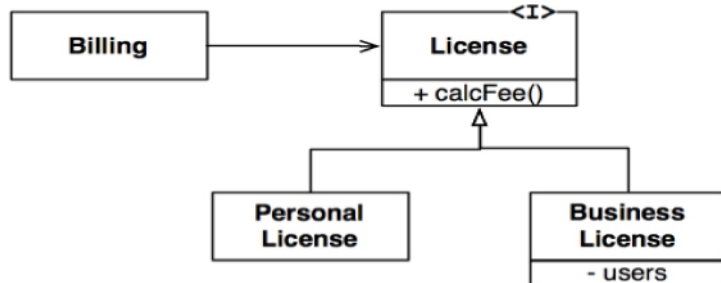
3 Barbara Liskov, “Data Abstraction and Hierarchy,” ACM SIGPLAN Notices 23, số 5 (tháng 5, 1988): 17–34.

For example, imagine that we have a class named `License`, as shown below. This class has a method named `calcFee()` that is called by the Billing application. There are two “subtypes”

Chẳng hạn, hãy hình dung ta có một class tên là `License`, như dưới đây. Class này có một method tên `calcFee()` được ứng dụng Billing gọi đến. Có hai “subtype”

of `License`: `PersonalLicense` and `BusinessLicense`. They use different algorithms to calculate the license fee.

các thuật toán khác nhau để tính phí license.



This design conforms to the LSP because the behavior of the

Thiết kế này tuân thủ LSP, bởi hành vi của

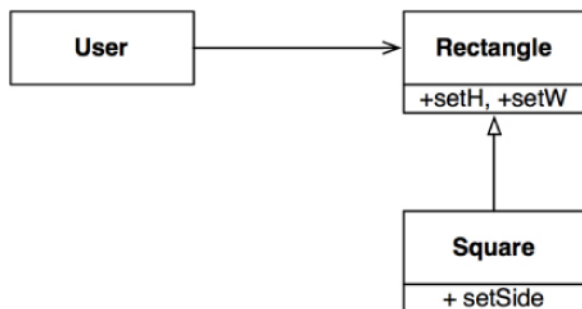
Billing application does not depend, in any way, upon which of the two subtypes it uses. Both of the subtypes are substitutable for the `License` type.

ứng dụng Billing không hề phụ thuộc vào chuyện nó đang dùng subtype nào trong hai subtype đó. Cả hai subtype đều thay thế được cho type `License`.

The canonical example of a violation of the LSP is the famed

Ví dụ kinh điển về một vi phạm LSP chính là bài toán trừ danh

Square/Rectangle problem.



In this example, `Square` is not a proper subtype of `Rectangle`, because the height and width of the `Rectangle` are independently mutable, but the height and width of the

Trong ví dụ này, `Square` không phải là một subtype đúng nghĩa của `Rectangle`, vì chiều cao và chiều rộng của `Rectangle` có thể thay đổi độc lập với nhau, còn chiều cao và chiều rộng của

Square must change together. Since the User believes it is communicating with a Rectangle , it could easily get confused. The following code shows why.

Square thì buộc phải thay đổi cùng lúc. Vì User tin rằng mình đang giao tiếp với một Rectangle , nó rất dễ bị rối. Đoạn mã sau cho thấy vì sao.

```
Rectangle r = . . .  
r.setW(5);  
r.setH(2);  
assert(r.area() == 10);
```

If the ... produced a Square , then the **assertion** would fail.

Nếu chỗ ... tạo ra một Square , thì assertion sẽ thất bại.

The only way to defend against this kind of LSP violation is to add mechanisms to the User (such as an if statement) that detect whether the Rectangle is, in fact, a Square . Since the behavior of the User depends on the types it uses, those types are not substitutable.

Cách duy nhất để phòng vệ trước kiểu vi phạm LSP này là thêm vào User một cơ chế nào đó (chẳng hạn một câu lệnh if) để phát hiện xem Rectangle kia thực chất có phải là Square hay không. Vì hành vi của User phụ thuộc vào các type nó dùng, những type đó không thay thế được cho nhau.

Putting an if statement like that into the User violates the OCP. Thus, every violation of the LSP is a latent violation of the OCP.

Nhét một câu lệnh if như vậy vào User là vi phạm OCP. Bởi vậy, mọi vi phạm LSP đều là một vi phạm OCP còn tiềm ẩn.

LSP and Design — LSP và thiết kế

In the early years of the object-oriented revolution, we thought of this principle as a way to guide the use of inheritance, as shown in the previous section. However, over the years, we have morphed the LSP into a broader principle of software design that pertains to interfaces and implementations.

Vào những năm đầu của cuộc cách mạng hướng đối tượng, chúng tôi xem nguyên tắc này như một chỉ dẫn cho việc dùng kế thừa (inheritance), đúng như phần trước đã trình bày. Tuy nhiên, qua nhiều năm, chúng tôi đã biến LSP thành một nguyên tắc thiết kế phần mềm rộng hơn, liên quan đến interface và hiện thực (implementation).

The interfaces in question can be of many forms. We might have a Java-style interface, implemented by several classes. Or we might have several Ruby classes that share the same method signatures. Or we might have a set of services that all respond to the same REST interface.

Những interface đang bàn tới ở đây có thể mang nhiều hình hài. Đó có thể là một interface kiểu Java, được nhiều class hiện thực. Hoặc có thể là vài class Ruby cùng chia sẻ một bộ chữ ký method. Hoặc cũng có thể là một tập các service cùng đáp ứng một REST interface.

In all of these situations, and more, the LSP is applicable because there are users who depend upon well-defined interfaces and upon the substitutability of the implementations of those interfaces.

Trong tất cả những tình huống ấy, và còn nhiều nữa, LSP đều áp dụng được, bởi luôn có những người dùng phụ thuộc vào các interface được định nghĩa rõ ràng và vào tính thay thế được của các hiện thực của những interface đó.

Taxi Aggregator — Bộ tổng hợp dịch vụ taxi

The best way to understand the LSP from a software design viewpoint is to look at what happens to the design of a system when the principle is violated.

Cách tốt nhất để hiểu LSP dưới góc nhìn thiết kế phần mềm là nhìn xem chuyện gì xảy ra với thiết kế của một hệ thống khi nguyên tắc này bị vi phạm.

Let's assume we that we are building an aggregator for many taxi dispatch services. Customers use our website to find the most appropriate taxi to use, regardless of the taxi company. Once the customer decides, our system dispatches the taxi by using a RESTful service.

Giả sử ta đang xây một bộ tổng hợp cho nhiều dịch vụ điều phối taxi. Khách hàng dùng website của ta để tìm chiếc taxi phù hợp nhất, bất kể nó thuộc hãng nào. Khi khách đã chọn xong, hệ thống của ta điều xe đến bằng một RESTful service.

Now let's assume that the URI for the RESTful dispatch service is part of the information contained in the driver database. Once our system has chosen a driver appropriate for the customer, it gets that URI from the driver record and then uses it to dispatch the driver.

Bây giờ giả sử URI của RESTful service điều phối là một phần thông tin nằm trong cơ sở dữ liệu tài xế. Sau khi hệ thống đã chọn được tài xế phù hợp với khách, nó lấy URI đó ra từ bản ghi tài xế rồi dùng nó để điều tài xế đi.

For example, let's say that Driver Bob has a dispatch URI that looks like this:

Ví dụ, giả sử tài xế Bob có một URI điều phối trông như thế này:

Our system will append the dispatch information onto this URI and send it with a PUT , as follows:

Hệ thống của ta sẽ nối thông tin điều phối vào URI này rồi gửi đi bằng PUT , như sau:

```
purplecab.com/driver/Bob  
/pickupAddress/24 Maple st.  
/pickupTime/1530  
/destination/ORD
```

Clearly this means that all the dispatch services for all the different companies must conform to the same REST interface. They must treat the pickupAddress , pickupTime , and

Rõ ràng điều này có nghĩa là mọi dịch vụ điều phối của tất cả các hãng khác nhau đều phải tuân theo cùng một REST interface. Chúng phải xử lý các trường pickupAddress , pickupTime và

destination fields identically.

destination y hệt như nhau.

Now let's say the Acme taxi company hired some programmers who didn't read the spec very carefully, and they abbreviated the destination field to just dest . And let's say that Acme was the largest taxi company in our area, and Acme's CEO's ex-wife is our CEO's new wife, and . . . Well, you get the picture. What would happen to the architecture of our system?

Giờ giả sử hãng taxi Acme thuê mấy lập trình viên đọc spec không kỹ lắm, và họ viết tắt trường destination thành dest . Lại giả sử thêm rằng Acme là hãng taxi lớn nhất khu vực của chúng ta, mà vợ cũ của CEO bên Acme lại đang là vợ mới của CEO bên ta, và . . . Thôi, chắc bạn hình dung ra rồi đấy. Chuyện gì sẽ xảy ra với kiến trúc hệ thống của chúng ta?

Clearly we'd need to add a special case. The dispatch request for any Acme driver would have to be constructed using a different set of rules from all the other drivers.

Rõ ràng ta sẽ phải thêm một trường hợp đặc biệt. Yêu cầu điều phối cho bất kỳ tài xế nào của Acme sẽ phải được dựng lên theo một bộ quy tắc khác với tất cả tài xế còn lại.

The simplest way to accomplish this would be to add an if statement to the module that constructed the dispatch command.

Cách đơn giản nhất để làm chuyện đó là thêm một câu lệnh if vào module dựng câu lệnh điều phối.

```
if (driver.getDispatchUri().startsWith("acme.com"))...
```

But, of course, no developer worth their salt would allow such a construction to exist in the system. Putting the word acme into the code itself creates an opportunity for all kinds of horrible and mysterious errors, not to mention security breaches.

Nhưng dĩ nhiên, chẳng lập trình viên nào ra hồn lại để một cấu trúc như vậy tồn tại trong hệ thống. Nhét chữ acme thẳng vào mã nguồn mở đường cho đủ loại lỗi khủng khiếp và khó hiểu, chưa kể tới các lỗ hổng bảo mật.

For example, what if Acme got very successful and bought the Purple Taxi company? What if they maintained the brand and the website but unified all their systems? Would we have to add another if statement for the word purple ?

Chẳng hạn, sẽ ra sao nếu Acme ăn nên làm ra và mua luôn hãng Purple Taxi? Sẽ ra sao nếu họ vẫn giữ thương hiệu cùng website nhưng hợp nhất toàn bộ hệ thống? Chẳng lẽ ta lại phải thêm một câu lệnh if nữa cho chữ purple ?

So, our architect would have to insulate the system from bugs like this by creating some kind of dispatch command creation module that was driven by a configuration database keyed by the dispatch URI. The configuration data might look something like this:

Thế là kiến trúc sư của ta sẽ phải cách ly hệ thống khỏi những con bug kiểu này bằng cách tạo ra một module dựng câu lệnh điều phối nào đó, được điều khiển bởi một cơ sở dữ liệu cấu hình lấy URI điều phối làm khoá. Dữ liệu cấu hình đó có thể trông đại loại thế này:

URI	Dispatch format
Acme.com	/pickupAddress/%s/pickupTime/%s/dest/%s
,	/pickupAddress/%s/pickupTime/%s/destination/%s

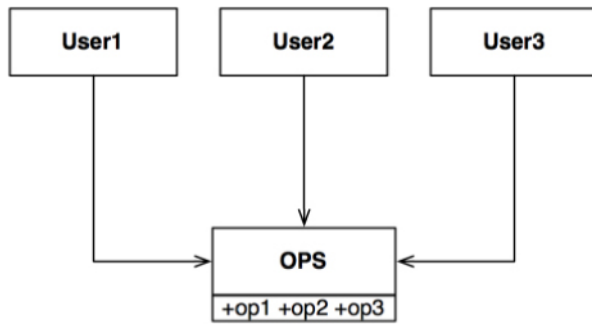
The point of all this is that a simple violation of the LSP for a RESTful service can cause our architecture to be polluted with a significant number of extra mechanisms.

Mấu chốt của tất cả những chuyện này là: chỉ một vi phạm LSP đơn giản ở một RESTful service cũng đủ khiến kiến trúc của ta bị ô nhiễm bởi cả đồng cơ chế phụ trợ.

ISP: The Interface Segregation Principle — ISP: Nguyên tắc Phân tách Interface

The ISP derives its name from the following:

ISP lấy tên từ tình huống sau:



In the situation above, there are several users that use the operations of the OPS class. Let's assume that User1 only uses

Trong tình huống trên, có vài người dùng sử dụng các thao tác của class OPS . Giả sử User1 chỉ dùng

op1 , User2 only uses op2 , and User3 only uses op3 .

op1 , User2 chỉ dùng op2 , còn User3 chỉ dùng op3 .

Now imagine that OPS is a class written in a language like C++. 4 Clearly, in that case, the source code of User1 will inadvertently depend upon op2 and op3 , even though it doesn't call them. This dependence means that a change to the source code of op2 in OPS will force User1 to be recompiled and redeployed, even though nothing that it cared about has actually changed.

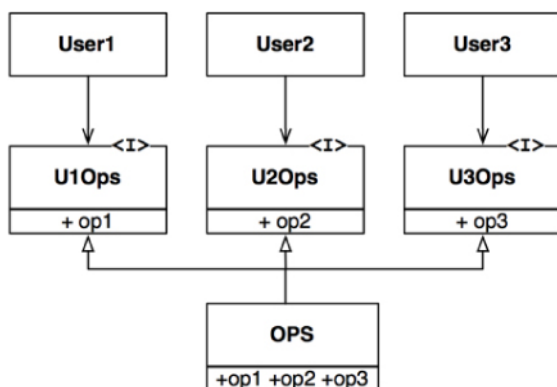
Giờ hãy hình dung OPS là một class được viết bằng ngôn ngữ kiểu như C++. 4 Rõ ràng khi đó mã nguồn của User1 sẽ vô tình phụ thuộc vào op2 và op3 , dù nó chẳng hề gọi tới chúng. Sự phụ thuộc này có nghĩa là một thay đổi trong mã nguồn của op2 bên trong OPS sẽ buộc User1 phải được biên dịch lại và triển khai lại, dù chẳng có thứ gì nó quan tâm thực sự thay đổi cả.

4 The situation in Java and C# is a bit more complicated. The compiler does not recompile modules based solely upon dates. Rather, it looks for changed declarations.

4 Tình hình trong Java và C# thì phức tạp hơn một chút. Trình biên dịch không biên dịch lại các module chỉ dựa vào ngày tháng. Thay vào đó, nó tìm những khai báo đã thay đổi.

This can be resolved by segregating the operations into interfaces, as shown below.

Có thể giải quyết chuyện này bằng cách phân tách các thao tác thành những interface riêng, như dưới đây.



Again, if we imagine that this is implemented in a statically typed language like C++, then the source code of User1 will depend upon U1Ops and op1 , but it will not depend upon OPS . Thus, a change to OPS that User1 does not care about will not cause User1 to be recompiled and redeployed.

Một lần nữa, nếu ta hình dung chuyện này được hiện thực bằng một ngôn ngữ định kiểu tĩnh như C++, thì mã nguồn của User1 sẽ phụ thuộc vào U1Ops và op1 , nhưng sẽ không phụ thuộc vào OPS . Nhờ vậy, một thay đổi trong OPS mà User1 không quan tâm sẽ không khiến User1 phải biên dịch lại và triển khai lại.

ISP and Language — ISP và ngôn ngữ

Statically typed languages like C++ force programmers to create declarations that users must `#include` . It is these included declarations in the source code that create the source code dependencies that can force recompilation and redeployment.

Các ngôn ngữ định kiểu tĩnh như C++ buộc lập trình viên phải tạo ra những khai báo mà người dùng phải `#include` . Chính những khai báo được include trong mã nguồn ấy đã tạo ra các phụ thuộc mã nguồn có thể buộc phải biên dịch lại và triển khai lại.

In languages like Java and C#, the dependencies are softer but not absent. The compilers follow the dependency tree using individual declarations rather than source files. So recompilation and redeployment are only necessary if declarations change.

Trong những ngôn ngữ như Java và C#, các phụ thuộc mềm hơn nhưng không hề biến mất. Trình biên dịch lần theo cây phụ thuộc dựa trên từng khai báo riêng lẻ chứ không dựa trên file nguồn. Vì vậy chỉ khi khai báo thay đổi thì mới cần biên dịch lại và triển khai lại.

In dynamically typed languages like Ruby and Python, interfaces don't exist in source code. Rather, they are inferred at runtime. Thus, there are no source code dependencies nor any interface declarations to force recompilation and redeployment. This is one reason that dynamically typed languages create systems that are more flexible and less coupled than statically typed languages.

Trong các ngôn ngữ định kiểu động như Ruby và Python, interface không tồn tại trong mã nguồn. Thay vào đó, chúng được suy ra lúc chạy. Do đó không có phụ thuộc ở mức mã nguồn, cũng chẳng có khai báo interface nào buộc phải biên dịch lại và triển khai lại. Đây là một lý do khiến các ngôn ngữ định kiểu động tạo ra những hệ thống linh hoạt hơn và ít kết dính hơn so với ngôn ngữ định kiểu tĩnh.

This fact could lead you to conclude that the ISP is a language issue and not a design issue.

Sự thật này có thể khiến bạn kết luận rằng ISP là chuyện của ngôn ngữ chứ không phải chuyện của thiết kế.

ISP and Design — ISP và thiết kế

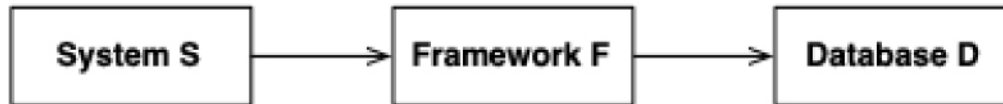
However, if you take a step back and look at the root motivations of the ISP, you can see that there is a deeper concern lurking there. In general, it is harmful to depend upon modules that contain more than you need. This is obviously true for source code dependencies that can force unnecessary recompilation and redeployment. However, it is also true at a much higher level.

Tuy nhiên, nếu lùi lại một bước và nhìn vào động cơ gốc rễ của ISP, bạn sẽ thấy có một mối bận tâm sâu xa hơn đang ẩn nấp ở đó. Nói chung, việc phụ thuộc vào những module

chứa nhiều hơn thứ bạn cần là có hại. Điều này hiển nhiên đúng với các phụ thuộc ở mức mã nguồn, vốn có thể buộc phải biên dịch lại và triển khai lại một cách không cần thiết. Nhưng nó cũng đúng ở một mức cao hơn nhiều.

Consider, for example, a developer working on a system, S . He wants to include a certain framework, F , in the system. Now suppose that the authors of F have bound it to a particular database, D . So S depends upon F , which depends upon D .

Chẳng hạn, hãy xét một lập trình viên đang làm việc trên hệ thống S . Anh ta muốn đưa một framework F nào đó vào hệ thống. Giờ giả sử các tác giả của F đã trói nó vào một cơ sở dữ liệu D cụ thể. Vậy là S phụ thuộc vào F , còn F thì phụ thuộc vào D .



Now suppose that D contains features that F does not use and that therefore S does not care about. Changes to those features within D may well force the redeployment of F , and therefore S . Worse, a failure of one of the features within D may cause failures in F and S .

Bây giờ giả sử D chứa những tính năng mà F không dùng tới, và do đó S cũng chẳng bận tâm. Những thay đổi ở các tính năng ấy bên trong D rất có thể sẽ buộc F phải triển khai lại, và kéo theo cả S . Tệ hơn nữa, một sự cố ở một trong các tính năng đó của D có thể gây ra sự cố cho cả F lẫn S .

The lesson here is that depending upon something that carries baggage that you don't need can cause you troubles that you didn't expect.

Bài học ở đây là: phụ thuộc vào một thứ mang theo hành lý mà bạn không cần có thể gây cho bạn những rắc rối bạn chẳng ngờ tới.

The bottom line is:

Tóm lại là:

Don't depend on things you don't need.

Đừng phụ thuộc vào những thứ bạn không cần.

We'll see more about that when we discuss the Common Reuse Principle in Chapter 20 , “ Component Principles .”

Ta sẽ bàn thêm về chuyện đó khi nói tới Common Reuse Principle ở Chương 20 , “ Các nguyên tắc về component .”

DIP: The Dependency Inversion Principle — DIP: Nguyên tắc Đảo ngược Phụ thuộc

“It's better to depend on something you control than on something you don't control, lest it end up controlling you.”

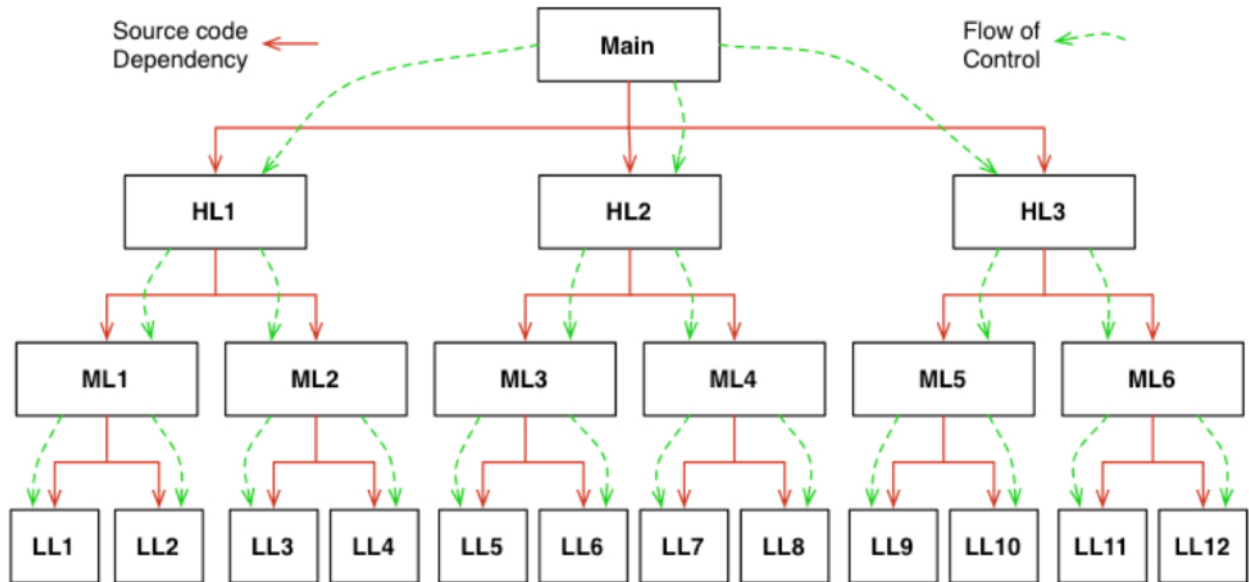
“Phụ thuộc vào thứ bạn kiểm soát được vẫn hơn là phụ thuộc vào thứ bạn không kiểm soát nổi, kẻo rốt cuộc chính nó lại kiểm soát bạn.”

—James Grenning

—James Grenning

In the '60s and '70s, virtually all programs were built according to the following pattern.

Trong thập niên 60 và 70, gần như mọi chương trình đều được xây theo khuôn mẫu sau.

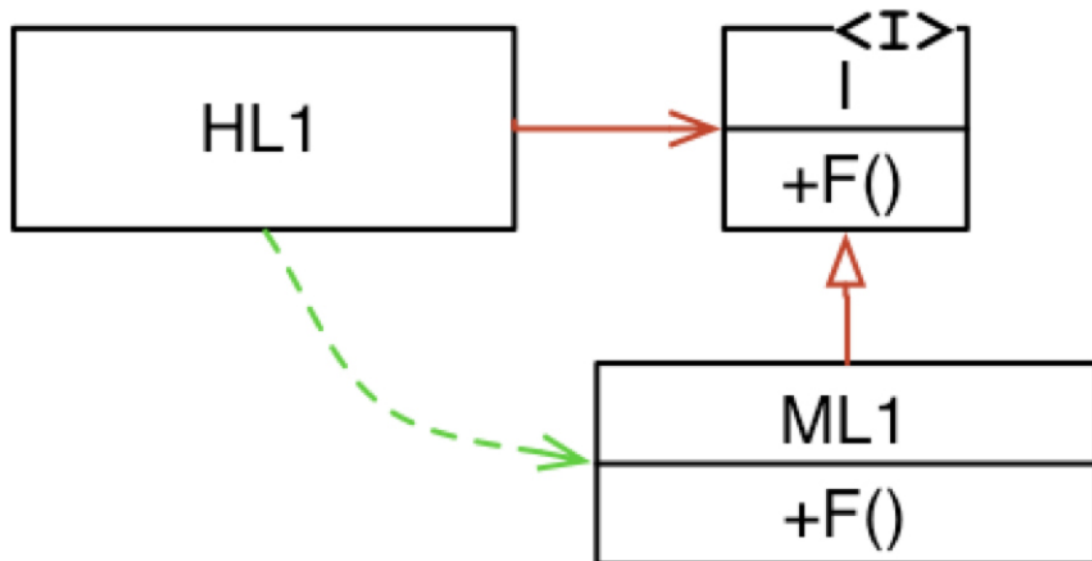


The source code dependencies followed the flow of control, without **exception**. High-level modules depended upon low-level modules because the high-level modules called the low-level modules.

Các phụ thuộc ở mức mã nguồn đi theo luồng điều khiển, không có ngoại lệ. Module mức cao phụ thuộc vào module mức thấp bởi vì module mức cao gọi module mức thấp.

OO gave us a different option: polymorphic interfaces.

Lập trình hướng đối tượng cho ta một lựa chọn khác: interface đa hình (polymorphism).



The insertion of an interface into the calling path inverts the source code dependency against the flow of control. In the above diagram, the high-level module HL1 calls the f function in the lower-level module ML1 ; but it does so through the interface I . The source code dependency from ML1 to I points against the flow of control.

Việc chèn một interface vào đường gọi làm đảo ngược phụ thuộc mã nguồn so với luồng điều khiển. Trong sơ đồ trên, module mức cao HL1 gọi hàm f trong module mức thấp hơn ML1 ; nhưng nó gọi thông qua interface I . Phụ thuộc mã nguồn từ ML1 tới I chỉ ngược chiều luồng điều khiển.

The ability to invert dependencies in this way gives the software designer absolute control over every source code dependency in the system. If any are problematic (and they often are), they can be quickly inverted by inserting an interface.

Khả năng đảo ngược phụ thuộc theo cách này trao cho người thiết kế phần mềm quyền kiểm soát tuyệt đối với mọi phụ thuộc mã nguồn trong hệ thống. Nếu có phụ thuộc nào gây rắc rối (mà chuyện đó xảy ra thường xuyên), ta có thể nhanh chóng đảo ngược nó bằng cách chèn thêm một interface.

This ability is power!

Khả năng này chính là sức mạnh!



It allows us to create plug-in architectures that allow high-level policies, like the business rules, to be independent of low-level details, like the UI and the database.

Nó cho phép ta tạo ra các kiến trúc plugin, nhờ đó những chính sách mức cao như quy tắc nghiệp vụ (business rule) trở nên độc lập với các chi tiết mức thấp như UI và cơ sở dữ liệu.



In the above diagram, all source code dependencies (solid arrows) point from lower-level modules to higher-level modules. Both UI and Database depend upon and implement interfaces within BusinessRules . In other words, the concrete details depend upon abstractions. This tells us that the most flexible systems are those in which source code dependencies refer only to abstractions, not to concretions.

Trong sơ đồ trên, mọi phụ thuộc mã nguồn (các mũi tên liền nét) đều trở từ module mức thấp lên module mức cao. Cả UI lẫn Database đều phụ thuộc vào và hiện thực các interface nằm bên trong BusinessRules . Nói cách khác, các chi tiết cụ thể phụ thuộc vào những trừu tượng. Điều này cho ta biết rằng những hệ thống linh hoạt nhất là những hệ thống mà phụ thuộc mã nguồn chỉ trở tới cái trừu tượng, chứ không trở tới cái cụ thể.

In a statically typed language, like Java, this means that the

Trong một ngôn ngữ định kiểu tĩnh như Java, điều này có nghĩa là các `use`, `import`, or `include` statements should refer only to source modules containing interfaces, abstract classes, or some other kind of abstract declaration. Nothing concrete should be depended upon.

câu lệnh `use`, `import` hay `include` chỉ nên trỏ tới những module nguồn chứa interface, abstract class, hoặc một dạng khai báo trừu tượng nào đó. Không nên phụ thuộc vào bất cứ thứ gì cụ thể.

The same rule applies for dynamically typed languages, like Ruby or Python. Source code dependencies should not refer to concrete modules. However, in these languages, it is a bit harder to define what a concrete module is. In particular, it is any module in which the functions being called are implemented.

Quy tắc ấy cũng áp dụng cho các ngôn ngữ định kiểu động như Ruby hay Python. Phụ thuộc mã nguồn không nên trỏ tới những module cụ thể. Có điều, trong các ngôn ngữ này, định nghĩa thế nào là một module cụ thể lại khó hơn đôi chút. Cụ thể, đó là bất kỳ module nào mà những hàm được gọi đến thực sự nằm ở đó.

Treating this as a hard-and-fast rule is unrealistic. There are many concrete facilities that software systems must depend upon. For example, the `String` class in Java is concrete. It would be unrealistic to try to force it to be abstract. The source code dependency upon the concrete `java.lang.string` cannot, and should not, be avoided.

Coi đây là một quy tắc cứng nhắc bất di bất dịch thì lại thiếu thực tế. Có rất nhiều tiện ích cụ thể mà hệ thống phần mềm buộc phải phụ thuộc vào. Chẳng hạn, class `String` trong Java là cụ thể. Ép nó thành trừu tượng thì thật viển vông. Phụ thuộc mã nguồn vào `java.lang.string` cụ thể là thứ không thể, và cũng không nên, né tránh.

On the other hand, the `String` class is very stable. Changes to that class are very rare and tightly controlled. Programmers and architects do not have to worry about frequent and capricious changes to `String`.

Mặt khác, class `String` rất ổn định. Những thay đổi ở class đó cực hiếm và được kiểm soát chặt chẽ. Lập trình viên và kiến trúc sư chẳng phải lo lắng chuyện `String` bị đổi liên tục hay đổi tùy hứng.

And so we tend to ignore the stable background of operating system and platform facilities when it comes to the DIP. We tolerate those concrete dependencies because we know we can rely upon them not to change.

Thế nên, khi bàn về DIP, ta thường bỏ qua cái nền ổn định gồm các tiện ích của hệ điều hành và nền tảng. Ta chấp nhận những phụ thuộc cụ thể ấy bởi ta biết mình có thể tin rằng chúng sẽ không đổi.

It is the volatile concrete elements of our system that we want to avoid depending upon. It is those modules that we are actively developing and that are undergoing frequent change.

Thứ ta muốn tránh phụ thuộc vào chính là những thành phần cụ thể hay biến động trong hệ thống. Đó là những module ta đang tích cực phát triển và đang thay đổi xoành xoạch.

Stable Abstractions — Trừu tượng ổn định

Every change to an abstract interface corresponds to a change to its concrete implementations. On the other hand, changes to concrete implementations do not always, or even usually, require

changes to the interfaces that they implement. Therefore, interfaces are generally less volatile than implementations.

Mỗi thay đổi ở một interface trừu tượng đều tương ứng với một thay đổi ở các hiện thực cụ thể của nó. Ngược lại, thay đổi ở các hiện thực cụ thể thì không phải lúc nào — thậm chí thường là không — đòi hỏi phải sửa những interface mà chúng hiện thực. Bởi vậy, interface nhìn chung ít biến động hơn hiện thực.

Indeed, good software designers and architects work hard to reduce the volatility of interfaces. They try to find ways to add functionality to implementations without making changes to the interfaces. This is Software Design 101.

Thật vậy, những người thiết kế và kiến trúc sư phần mềm giỏi luôn nỗ lực giảm bớt độ biến động của interface. Họ tìm cách bổ sung chức năng cho các hiện thực mà không phải đụng vào interface. Đây là Thiết kế Phần mềm nhập môn.

The implication, therefore, is that stable software designs are those that avoid depending on volatile concretions, and favor the use of stable abstract interfaces.

Hệ quả rút ra là: những thiết kế phần mềm ổn định là những thiết kế tránh phụ thuộc vào các thứ cụ thể hay biến động, và ưu tiên dùng các interface trừu tượng ổn định.

This implication boils down to a set of very specific coding practices.

Hệ quả này quy về một tập các thực hành viết mã rất cụ thể.

- Don't refer to volatile concrete classes. Refer to abstract interfaces instead. This rule applies in all languages, whether statically or dynamically typed. It also puts severe constraints on the creation of objects and generally enforces the use of Abstract Factories.
- Don't derive from volatile concrete classes. This is a corollary to the previous rule, but it bears special mention. In statically typed languages, inheritance is the strongest and most rigid of all the source code relationships, and it should be used with great care. In dynamically typed languages, inheritance is less of a problem; but it is still a dependency, and caution is always the wisest choice.
- Don't override concrete functions. Concrete functions often require source code dependencies. When you override those functions, you do not eliminate those dependencies—indeed, you inherit them. To manage those dependencies you should make the function abstract and create multiple implementations.
 - Đừng tham chiếu tới những class cụ thể hay biến động. Hãy tham chiếu tới interface trừu tượng. Quy tắc này áp dụng cho mọi ngôn ngữ, dù định kiểu tĩnh hay động. Nó cũng đặt ra những ràng buộc rất ngặt nghèo cho việc tạo đối tượng và nhìn chung buộc ta phải dùng Abstract Factory.
 - Đừng dẫn xuất từ những class cụ thể hay biến động. Đây là hệ quả của quy tắc trên, nhưng đáng được nhắc riêng. Trong các ngôn ngữ định kiểu tĩnh, kế thừa là quan hệ mạnh nhất và cứng nhắc nhất trong mọi quan hệ ở mức mã nguồn, nên phải dùng hết sức thận trọng. Trong các ngôn ngữ định kiểu động, kế thừa đỡ phiền hơn; nhưng nó vẫn là một phụ thuộc, và cần trọng luôn là lựa chọn khôn ngoan.
 - Đừng override những hàm cụ thể. Hàm cụ thể thường kéo theo các phụ thuộc mã nguồn. Khi bạn override chúng, bạn không hề loại bỏ được những phụ thuộc đó — thực ra bạn còn thừa hưởng luôn. Muốn quản lý các phụ thuộc ấy, bạn nên biến hàm đó thành trừu tượng rồi tạo nhiều hiện thực khác nhau.
- Never mention the name of anything concrete and volatile. This is really just a restatement of the principle itself.

- Đừng bao giờ nhắc tới tên của bất cứ thứ gì vừa cụ thể vừa biến động. Đây thực chất chỉ là cách phát biểu lại chính nguyên tắc này.

While these are written as rules, they are actually more like warnings. A pragmatic programmer will violate this principle frequently. But a wise programmer will consider the warnings first.

Tuy được viết dưới dạng quy tắc, chúng thực ra giống lời cảnh báo hơn. Một lập trình viên thực dụng sẽ vi phạm nguyên tắc này khá thường xuyên. Nhưng một lập trình viên khôn ngoan sẽ cân nhắc những lời cảnh báo trước đã.

Slavishly obeying this principle can lead to an explosion of interfaces that no one really needs. On the other hand, blithely ignoring this principle can cause your software to become tangled and rigid.

Tuân thủ nguyên tắc này một cách mù quáng có thể dẫn tới sự bùng nổ của cả đồng interface mà chẳng ai thực sự cần. Ngược lại, phớt lờ nó một cách vô tư lại có thể khiến phần mềm của bạn trở nên rối rắm và cứng nhắc.

Finally, wise programmers will conform to this principle gradually, as the system evolves and the needs arise. With a **test suite** that they trust with their lives, they know that when interfaces need to be added in order to invert dependencies, the changes will be relatively easy to make and free of risk.

Cuối cùng, những lập trình viên khôn ngoan sẽ tuân theo nguyên tắc này một cách *dần dần*, khi hệ thống tiến hoá và nhu cầu nảy sinh. Với một bộ test (test suite) mà họ tin tưởng đến mức đặt cược cả tính mạng, họ biết rằng khi cần thêm interface để đảo ngược phụ thuộc, việc thay đổi sẽ khá dễ dàng và không rủi ro.

Factories — Factory

In order to comply with the DIP, the creation of volatile concrete objects may require special handling. This is because, in virtually all languages, the creation of an object requires a source code dependency upon the concrete definition of that object.

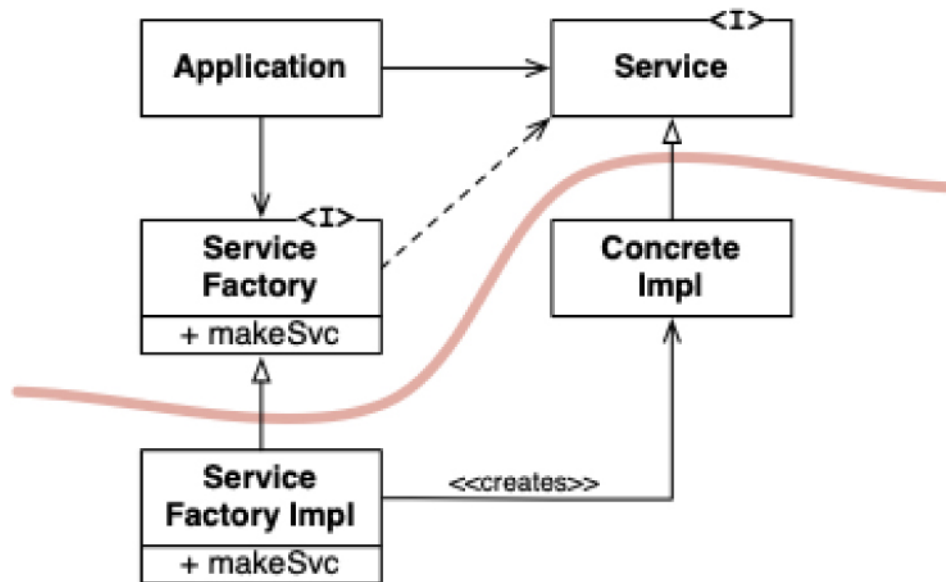
Để tuân thủ DIP, việc tạo ra các đối tượng cụ thể dễ biến động (volatile concrete objects) có thể cần được xử lý đặc biệt. Lý do là ở hầu như mọi ngôn ngữ, muốn tạo một đối tượng thì mã nguồn buộc phải phụ thuộc vào định nghĩa cụ thể của đối tượng đó.

In most OO language, like Java, we would use an Abstract Factory ⁵ to eliminate this undesirable dependency.

Trong hầu hết các ngôn ngữ OO, chẳng hạn Java, chúng ta sẽ dùng Abstract Factory ⁵ để loại bỏ sự phụ thuộc không mong muốn ấy.

⁵ [GOF95].

⁵ [GOF95].



The preceding diagram shows the structure. The Application uses the ConcreteImpl through the Service interface. However, the Application must somehow create instances of the

Sơ đồ ở trên cho thấy cấu trúc. Application sử dụng ConcreteImpl thông qua interface Service. Tuy nhiên, bằng cách nào đó Application vẫn phải tạo ra các instance của

ConcreteImpl. To achieve this without creating a source code dependency upon the ConcreteImpl, the Application calls the

ConcreteImpl. Để làm được điều đó mà không tạo ra phụ thuộc mã nguồn vào ConcreteImpl, Application gọi

makeSvc method of the ServiceFactory interface. This method is implemented by the ServiceFactoryImpl class that derives from

method makeSvc của interface ServiceFactory. Method này được hiện thực bởi class ServiceFactoryImpl, vốn dẫn xuất từ

ServiceFactory. That **implementation** instantiates the

ServiceFactory. Hiện thực đó khởi tạo

ConcreteImpl and returns it as a Service.

ConcreteImpl rồi trả nó về dưới dạng Service.

Note the curved line. This is an **architectural boundary**. It separates the abstract from the concrete. All source code dependencies cross that line pointing in the same direction: toward the abstract side.

Hãy để ý đường cong. Đó là một ranh giới kiến trúc (architectural boundary). Nó tách phần trừu tượng khỏi phần cụ thể. Mọi phụ thuộc (dependency) trong mã nguồn đều cắt qua đường đó và cùng chỉ về một hướng: về phía trừu tượng.

The line divides the system into two components: one abstract and the other concrete. The abstract component contains all the high-level business rules of the application. The concrete component contains all the implementation details that those business rules manipulate.

Đường ấy chia hệ thống thành hai component: một trừu tượng, một cụ thể. Component trừu tượng chứa toàn bộ các quy tắc nghiệp vụ (business rule) mức cao của ứng dụng.

Component cụ thể chứa toàn bộ các chi tiết hiện thực (implementation) mà những quy tắc nghiệp vụ đó thao tác lên.

Note that the flow of control crosses the line in the opposite direction of the source code dependencies. The source code dependencies are inverted against the flow of control. That is why we refer to this principle as dependency inversion .

Hãy để ý rằng luồng điều khiển cắt qua đường đó theo chiều ngược lại với các phụ thuộc mã nguồn. Phụ thuộc mã nguồn bị đảo ngược so với luồng điều khiển. Đó chính là lý do ta gọi nguyên tắc này là đảo ngược phụ thuộc (dependency inversion) .

Concrete Components — Các component cụ thể

The concrete component in the above diagram contains a single dependency that violates the DIP. This is typical. DIP violations cannot be entirely removed; but they can be gathered into a small number of concrete components and kept separate from the rest of the system.

Component cụ thể trong sơ đồ trên chứa đúng một phụ thuộc vi phạm DIP. Chuyện đó là bình thường. Không thể loại bỏ hoàn toàn các vi phạm DIP; nhưng ta có thể gom chúng lại vào một số ít component cụ thể và giữ chúng tách biệt khỏi phần còn lại của hệ thống.

Most systems will contain at least one such concrete component, often called `main` , because it contains the `main` function. In the above case, the `main` function would instantiate the `ServiceFactoryImpl` and place that instance in a global **variable** of type `ServiceFactory` . The Application would then access the factory through that global.

Hầu hết hệ thống sẽ chứa ít nhất một component cụ thể kiểu như vậy, thường được gọi là `main` , bởi nó chứa hàm `main` . Trong trường hợp trên, hàm `main` sẽ khởi tạo `ServiceFactoryImpl` rồi đặt instance đó vào một biến toàn cục có kiểu `ServiceFactory` . Application sau đó sẽ truy cập tới factory thông qua biến toàn cục ấy.

6 That is, the function that is invoked by the operating system when the application is first started up.

6 Tức là hàm được hệ điều hành gọi tới khi ứng dụng vừa khởi động.

As we proceed in this book and cover higher-level architectural principles, the DIP will show up again and again. It will be the most visible organizing principle in our architecture diagrams. The curved line in the diagram above will become the architectural boundaries in later chapters. The way the dependencies cross that line in one direction and toward more abstract entities will become a new rule that we will call the Dependency Rule.

Càng đi tiếp trong cuốn sách này và càng bàn tới những nguyên tắc kiến trúc (architecture) ở mức cao hơn, DIP sẽ còn xuất hiện lặp đi lặp lại. Nó sẽ là nguyên tắc tổ chức dễ thấy nhất trong các sơ đồ kiến trúc của chúng ta. Đường cong trong sơ đồ ở trên sẽ trở thành các ranh giới kiến trúc ở những chương sau. Còn cái cách các phụ thuộc cắt qua đường đó theo một chiều duy nhất, hướng về phía những thực thể trừu tượng hơn, sẽ trở thành một quy tắc mới mà ta sẽ gọi là Dependency Rule.

Chapter 20. Component Principles —

Chương 20. Các nguyên tắc về component



If **the SOLID principles** tell us how to arrange the bricks into walls and rooms, then the **component** principles tell us how to arrange the rooms into buildings. Large software systems, like large buildings, are built out of smaller components.

Nếu các nguyên tắc SOLID chỉ cho ta cách xếp gạch thành tường và thành phòng, thì các nguyên tắc về component chỉ cho ta cách ghép các phòng thành toà nhà. Hệ thống phần mềm lớn, cũng như những toà nhà lớn, được dựng lên từ những component nhỏ hơn.

In this chapter, we will discuss what software components are, what elements they should be composed of, and how they should be composed together into systems.

Trong chương này, chúng ta sẽ bàn xem component phần mềm là gì, chúng nên được cấu thành từ những phần tử nào, và nên được ghép lại thành hệ thống ra sao.

This chapter is an overview. The principles are described in detail in [PPP02] and again in [Clean Arch].

Chương này chỉ là bản tổng quan. Các nguyên tắc được mô tả chi tiết trong [PPP02] và một lần nữa trong [Clean Arch].

Components

Component

Components are the units of deployment. They are the smallest entities that can be deployed as part of a system. In Java, they are jar files. In Ruby, they are gem files. In .NET, they are DLLs. In compiled languages, they are aggregations of binary files. In interpreted languages, they are aggregations of source files. In all languages, they are the granule of deployment .

Component là đơn vị triển khai. Chúng là những thực thể nhỏ nhất có thể được triển khai như một phần của hệ thống. Trong Java, đó là các file jar. Trong Ruby, đó là các file gem. Trong .NET, đó là các DLL. Với ngôn ngữ biên dịch, chúng là tập hợp các file nhị phân. Với ngôn ngữ thông dịch, chúng là tập hợp các file mã nguồn. Trong mọi ngôn ngữ, chúng là hạt triển khai.

Components can be linked together into a single executable. Or they can be aggregated together into a single archive such as a .war file. Or they can be independently deployed as separate, dynamically loaded plug-ins, such as .jar or .dll or .exe files. However, regardless of how they are eventually deployed, well-designed components always retain the ability to be **independently deployable** and therefore independently developable.

Các component có thể được link lại với nhau thành một file thực thi duy nhất. Hoặc được gộp chung vào một archive duy nhất, chẳng hạn file .war. Hoặc được triển khai độc lập dưới dạng các plug-in riêng biệt nạp động, như file .jar, .dll hay .exe. Tuy nhiên, dù rốt cuộc được triển khai theo cách nào, component được thiết kế tốt luôn giữ được khả năng triển khai độc lập, và nhờ đó phát triển độc lập được.

A Brief History of Components

Lược sử của component

In the early years of software development, programmers controlled the memory location and layout of their programs. One of the first lines of code in a program would have been the origin statement, which declared the address at which the program was to be loaded. Consider this simple PDP-8 program. It consists of a subroutine named GETSTR , which inputs a string from the keyboard and saves it in a buffer. It also has a little **unit test** program to exercise GETSTR .

Vào những năm đầu của ngành phát triển phần mềm, lập trình viên tự tay kiểm soát vị trí và bố cục bộ nhớ của chương trình mình viết. Một trong những dòng mã đầu tiên của chương trình thường là câu lệnh origin, khai báo địa chỉ mà chương trình sẽ được nạp vào. Hãy xem chương trình PDP-8 đơn giản sau đây. Nó gồm một subroutine tên GETSTR , nhận một chuỗi từ bàn phím rồi lưu vào buffer. Kèm theo đó là một chương trình unit test nho nhỏ để chạy thử GETSTR .

```
*200
TLS
START, CLA
      TAD BUFR
      JMS GETSTR
      CLA
      TAD BUFR
      JMS PUTSTR
      JMP START
```



```

BUFR,    3000
GETSTR,  0
        DCA PTR
NXTCH,   KSF
        JMP  .-1
        KRB
        DCA I PTR
        TAD I PTR
        AND K177

        ISZ PTR
        TAD MCR
        SZA
        JMP NXTCH
K177,    177
MCR,     -15

```

Note the *200 at the start of this program. This tells the compiler to generate code that will be loaded at address 200 8 . 1

Hãy để ý dòng *200 ở đầu chương trình này. Nó bảo compiler sinh ra mã sẽ được nạp vào địa chỉ 200 8 . 1

1 For example, [PPP02], <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod> , and [https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)) (or just google SOLID). This is a foreign concept for most programmers today. They don't usually have to think about where a program is loaded in the memory of the computer. But in the early days, this was one of the first decisions a programmer needed to make. In those days, programs were not relocatable.

1 Chẳng hạn [PPP02], <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod> , và [https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)) (hoặc cứ google SOLID). Với phần lớn lập trình viên ngày nay, đây là một khái niệm xa lạ. Thường thì họ chẳng phải bận tâm chương trình của mình được nạp vào chỗ nào trong bộ nhớ máy tính. Nhưng vào những ngày đầu, đó lại là một trong những quyết định đầu tiên mà lập trình viên phải đưa ra. Hồi ấy, chương trình không tái định vị được.

How did you access a library **function** in those days? The code above shows you how we did it. We included the source code of the library functions with our application code and compiled them all as a single program. 2 Libraries were kept in source, not in binary. 2 My first employer kept several dozen decks of the subroutine library source code on a shelf. When you wrote a new program, you simply grabbed one of those decks and slapped it onto the end of your deck. The problem with this was that devices in those days were slow, and memory was expensive and therefore limited. Compilers needed to make several passes over the source code, but memory was too limited to keep all the source code resident. So the compiler had to read in the source code several times using those slow devices.

Hồi ấy bạn dùng một hàm thư viện bằng cách nào? Đoạn mã trên cho thấy chúng tôi đã làm thế nào. Chúng tôi đưa luôn mã nguồn của các hàm thư viện vào cùng mã ứng dụng rồi biên dịch tất cả thành một chương trình duy nhất. 2 Thư viện được lưu ở dạng mã nguồn, chứ không phải dạng nhị phân. 2 Nơi làm việc đầu tiên của tôi để trên giá vài chục tập bìa đục lỗ chứa mã nguồn của thư viện subroutine. Khi viết một chương trình mới, bạn chỉ việc vớ lấy một tập rồi đập nó vào cuối tập bìa của mình. Cái dở là thiết bị thời đó rất chậm, còn bộ nhớ thì đắt nên hạn hẹp. Compiler phải quét mã nguồn nhiều lượt, mà bộ nhớ lại quá ít để giữ toàn bộ mã nguồn thường trú. Thế nên compiler buộc phải đọc đi

đọc lại mã nguồn nhiều lần từ những thiết bị chậm chạp ấy.

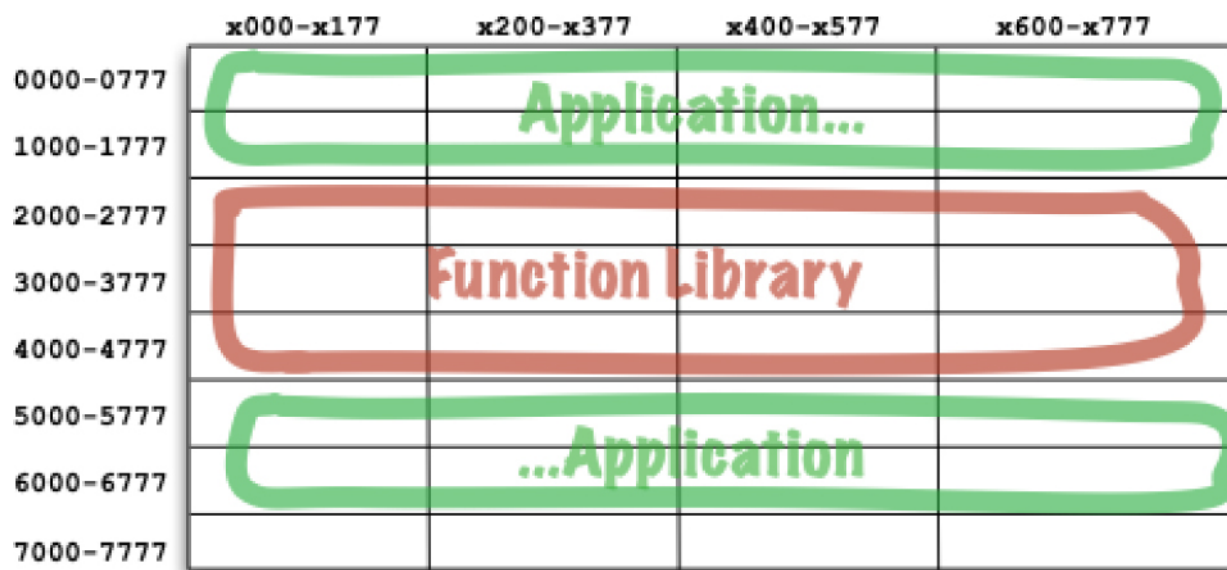
This took a long time; and the larger your function library was, the longer the compiler took. Compiling a large program could take hours. 3 In order to shorten the compile times, we separated the source code of the function library from the applications. We compiled the function library separately and loaded the binary at a known address; say, 2000 . We created a symbol table 8 for the function library and compiled that with our application code. When we wanted to run an application, we would load the binary function library, 4 and then load the application. 3 This video, which I uploaded to YouTube in 2022, shows the process: <https://www.youtube.com/playlist?list=PLINHgpdX-Cq0bImYF-IWVO8U2XERZuit> . 4 Actually, most of those old machines used core memory, which did not get erased when you powered the computer down. So we often just left the function library loaded for days at a time. Memory looked like this:

Việc này tốn rất nhiều thời gian; thư viện hàm càng lớn thì compiler càng lâu. Biên dịch một chương trình lớn có thể mất hàng giờ. 3 Để rút ngắn thời gian biên dịch, chúng tôi tách mã nguồn của thư viện hàm ra khỏi ứng dụng. Chúng tôi biên dịch thư viện hàm riêng rồi nạp phần nhị phân vào một địa chỉ đã biết trước, chẳng hạn 2000 8 . Chúng tôi tạo một symbol table cho thư viện hàm rồi biên dịch nó cùng mã ứng dụng. Khi muốn chạy một ứng dụng, chúng tôi nạp thư viện hàm dạng nhị phân, 4 rồi nạp ứng dụng. 3 Video này, do tôi tải lên YouTube năm 2022, cho thấy quy trình đó: <https://www.youtube.com/playlist?list=PLINHgpdX-Cq0bImYF-IWVO8U2XERZuit> . 4 Thật ra, phần lớn những cỗ máy cũ ấy dùng core memory, loại bộ nhớ không bị xóa khi tắt máy. Vì thế chúng tôi thường cứ để nguyên thư viện hàm đã nạp trong máy suốt nhiều ngày liền. Bộ nhớ trông như thế này:

	x000-x177	x200-x377	x400-x577	x600-x777
0000-0777	Application			
1000-1777				
2000-2777	Function Library			
3000-3777				
4000-4777				
5000-5777				
6000-6777				
7000-7777				

This worked fine so long as the application could fit between 0000 8 and 1777 . But soon applications grew to be larger than the space allotted. So 8 then, programmers had to split their applications into two address segments, jumping around the function library.

Cách này chạy tốt chừng nào ứng dụng còn nhét vừa khoảng từ 0000 8 tới 1777 . Nhưng chẳng bao lâu, ứng dụng phình to hơn cả khoảng không gian được cấp. Thế là 8 lập trình viên buộc phải xé ứng dụng của mình thành hai đoạn địa chỉ, nhảy vòng qua thư viện hàm.



It should be clear that this was not a sustainable situation. As we added more functions to the function library, it exceeded its bounds, and we had to allocate more space for it near 7000 8 . And this fragmentation of programs and libraries necessarily continued as computer memory grew.

Hẳn bạn cũng thấy rõ đây không phải là tình thế bền vững. Cứ thêm hàm vào thư viện hàm là nó lại vượt quá giới hạn của mình, và chúng tôi lại phải cấp thêm chỗ cho nó ở quanh 7000 8 . Và sự phân mảnh giữa chương trình với thư viện ấy tất yếu cứ thế tiếp diễn khi bộ nhớ máy tính ngày một lớn thêm.

Clearly, something had to be done.

Rõ ràng là phải làm gì đó.

Relocatability

Khả năng tái định vị

The solution was relocatable binaries. The idea was very simple. We changed the compiler to output binary code based at address zero. The loader would be told where to load the relocatable code, and the loader would add the load address to all the addresses in the binary code. This was a little more complicated than it sounds, but we found ways to make it work.

Lời giải là các file nhị phân tái định vị được. Ý tưởng rất đơn giản. Chúng tôi sửa compiler để nó xuất ra mã nhị phân lấy gốc là địa chỉ 0. Loader sẽ được cho biết cần nạp đoạn mã tái định vị được ấy vào đâu, rồi nó cộng địa chỉ nạp vào mọi địa chỉ trong mã nhị phân. Chuyện này phức tạp hơn nghe qua một chút, nhưng chúng tôi đã tìm được cách làm cho nó chạy.

Now we could tell the loader where to load the function library and where to load the application. In fact, the loader would accept several binary inputs and simply load them in memory one right after the other, relocating them as it loaded them. This allowed us to load only those functions that we needed.

Giờ thì chúng tôi có thể bảo loader nạp thư viện hàm vào đâu và nạp ứng dụng vào đâu. Thực tế, loader chấp nhận nhiều đầu vào nhị phân rồi cứ thế nạp chúng vào bộ nhớ nối tiếp nhau, vừa nạp vừa tái định vị. Nhờ đó chúng tôi chỉ phải nạp đúng những hàm mình cần.

The compiler was also changed to emit the names of the functions as metadata in the relocatable binary. If a program called a library function, the compiler would emit that name as an external reference. If a program defined a function, the compiler would emit that name as an external definition. Then, the loader could link the external references to the external definitions once it had determined where it had loaded those definitions.

Compiler cũng được sửa để phát ra tên các hàm dưới dạng metadata trong mã nhị phân tái định vị được. Nếu một chương trình gọi hàm thư viện, compiler sẽ phát tên đó ra như một external reference. Nếu chương trình định nghĩa một hàm, compiler phát tên đó ra như một external definition. Sau đó, loader có thể nối các external reference với các external definition, một khi nó đã xác định được mình nạp những definition ấy vào đâu.

And the linking loader was born.

Và thế là linking loader ra đời.

Linkers

Linker

The linking loader allowed us to divide our programs onto segments that can be separately compiled and loaded. This worked great when we had relatively small programs being linked with relatively small libraries. However, in the late '60s and early '70s, we got more ambitious, and programs got a lot bigger.

Linking loader cho phép chúng tôi chia chương trình thành các đoạn có thể biên dịch và nạp riêng rẽ. Cách này hoạt động rất tốt khi chương trình còn tương đối nhỏ và được link với những thư viện cũng tương đối nhỏ. Nhưng đến cuối thập niên 1960 và đầu 1970, chúng tôi tham vọng hơn, và chương trình thì phình to ra rất nhiều.

Eventually, the linking loaders were too slow to tolerate. This was because our function libraries were stored on slow devices such as a magnetic tape. Even the disks back then were quite slow. Using these relatively slow devices, the linking loaders had to read dozens, if not hundreds, of binary libraries in order to resolve the external references. As programs grew larger and larger, and more library functions accumulated in our libraries, a linking loader could take well over an hour just to load the program.

Rốt cuộc, linking loader chậm tới mức không chịu nổi. Nguyên do là thư viện hàm được lưu trên những thiết bị chậm như băng từ. Ngay cả đĩa hồi ấy cũng khá chậm. Với những thiết bị chậm chạp như thế, linking loader phải đọc hàng chục, nếu không muốn nói là hàng trăm, thư viện nhị phân chỉ để phân giải các external reference. Chương trình càng lúc càng lớn, thư viện càng lúc càng chất chồng thêm hàm, và một linking loader có thể ngốn hơn cả tiếng đồng hồ chỉ để nạp chương trình.

So the loader and the linker were separated into two phases. We took the slow part, the part that did that linking, and we put it into a separate application called the linker. The output of the linker was a linked relocatable that a relocating loader could load very quickly. This allowed us to prepare an executable using the slow linker; but then we could load it quickly, anytime we wanted to.

Thế là loader và linker được tách thành hai pha. Chúng tôi lấy phần chậm chạp — phần lo chuyện link — rồi đẩy nó vào một ứng dụng riêng gọi là linker. Đầu ra của linker là một khối tái định vị được đã link xong, thứ mà relocating loader có thể nạp cực nhanh. Nhờ vậy chúng tôi có thể dùng linker chậm chạp để chuẩn bị sẵn một file thực thi; sau đó thì nạp nó thật nhanh, bất cứ lúc nào muốn.

Then came the '80s. We were working in C or some other high-level language. As our ambitions grew, so did our programs. Programs containing hundreds of thousands of lines of code were not unusual.

Rồi thập niên 1980 tới. Chúng tôi làm việc bằng C hoặc một ngôn ngữ bậc cao nào đó. Tham vọng lớn lên thì chương trình cũng lớn theo. Chương trình dài hàng trăm nghìn dòng mã không còn là chuyện lạ.

We compiled our source modules from .c files into .o files, and fed them into the linker to create executable files that could be quickly loaded. Compiling each individual **module** was relatively fast; but compiling all the modules took a bit of time. And then the linker would take the most time. Turnaround had again grown to an hour or more in many cases.

Chúng tôi biên dịch từng module mã nguồn từ file .c thành file .o, rồi đưa chúng vào linker để tạo ra file thực thi nạp được thật nhanh. Biên dịch riêng từng module thì tương đối nhanh; nhưng biên dịch toàn bộ module thì mất khá thời gian. Và rồi linker mới là kẻ ngốn nhiều thời gian nhất. Trong nhiều trường hợp, vòng quay (turnaround) lại leo lên một tiếng hoặc hơn.

It seemed we were doomed to endlessly chase our tail. Throughout the '60s, '70s, and '80s, all the changes we had made to speed up our workflow were thwarted by our ambitions and the size of the programs we wrote. We could not seem to escape from the hourlong turnaround times. Loading time remained fast; but compile-link times were the bottleneck.

Xem ra chúng tôi bị kết án phải đuổi theo cái đuôi của chính mình đến vô tận. Suốt các thập niên 1960, 1970 và 1980, mọi thay đổi chúng tôi thực hiện để tăng tốc quy trình làm việc đều bị chính tham vọng và kích thước chương trình mình viết ra chặn đứng. Chúng tôi dường như không thoát nổi cái vòng quay dài cả tiếng ấy. Thời gian nạp thì vẫn nhanh; nhưng thời gian biên dịch cộng link mới là nút thắt cổ chai.

We were, of course, experiencing Murphy's law of program size:

Tất nhiên, cái chúng tôi đang nếm trải chính là định luật Murphy về kích thước chương trình:

Programs will grow to fill all available compile and link time.

Chương trình sẽ phình lên cho tới khi ngốn hết mọi thời gian biên dịch và link sẵn có.

But Murphy was not the only contender in town. Along came Moore, 5 and in the late '80s, the two battled it out. Moore won that battle. Disks started to shrink and get exponentially faster. Computer memory started to get so ridiculously cheap that much of the **data** on disk could be cached in RAM. Computer clock rates increased from 1MHz to 100MHz. 5 Moore's law: Computer speed, memory, and density double every 18 months. This law held from the 1950s to 2000; but then stopped cold. By the mid-'90s, the time spent linking had begun to shrink faster than our ambitions could make programs grow. In many cases, link time dropped to a matter of seconds. For small jobs, the idea of a linking loader became feasible again.

Nhưng Murphy không phải tay đua duy nhất trong thị trấn. Moore xuất hiện, 5 và đến cuối thập niên 1980, hai bên so găng. Moore thắng trận đó. Đĩa bắt đầu nhỏ đi và nhanh lên theo cấp số nhân. Bộ nhớ máy tính rẻ tới mức lỗ bịch, đến nỗi phần lớn dữ liệu trên đĩa có thể được cache trong RAM. Xung nhịp máy tính tăng từ 1MHz lên 100MHz. 5 Định luật Moore: tốc độ, bộ nhớ và mật độ của máy tính tăng gấp đôi sau mỗi 18 tháng. Định luật này đúng từ thập niên 1950 tới năm 2000; rồi đột ngột dừng lại. Đến giữa thập niên 1990, thời gian dành cho việc link bắt đầu co lại nhanh hơn tốc độ mà tham vọng của chúng tôi làm chương trình phình ra. Trong nhiều trường hợp, thời gian link rút xuống còn vài giây. Với những công việc nhỏ, ý tưởng linking loader lại trở nên khả thi.

This was the era of ActiveX, shared libraries, and the beginnings of .jar and .dll files. Computers and devices had gotten so fast that we could once again do the linking at load time. We could link together

several .jar files or several shared libraries in a matter of seconds, and execute the resulting program.

Đó là thời của ActiveX, của thư viện chia sẻ, và của những file .jar, .dll đầu tiên. Máy tính và thiết bị đã nhanh tới mức chúng tôi lại có thể link ngay lúc nạp. Chúng tôi có thể link vài file .jar hay vài shared library với nhau chỉ trong vài giây, rồi chạy chương trình thu được.

And so the component plug-in **architecture** was born.

Và thế là kiến trúc plugin dạng component ra đời.

Today, we routinely ship .jar files or DLL s or shared libraries as plug-ins to existing applications. If you want to create a mod to Minecraft, for example, you simply include your custom .jar files in a certain folder. If you want to plug ReSharper into Visual Studio, you simply include the appropriate DLL s.

Ngày nay, việc phát hành file .jar, DLL hay shared library dưới dạng plug-in cho các ứng dụng có sẵn là chuyện thường ngày. Chẳng hạn, nếu muốn tạo một mod cho Minecraft, bạn chỉ cần đặt file .jar tùy biến của mình vào một thư mục nhất định. Nếu muốn cắm ReSharper vào Visual Studio, bạn chỉ cần thêm những DLL thích hợp.

These dynamically linked files, which can be plugged together at runtime, are the software components of our architectures. It has taken 50 years, but we have arrived at a place where component plug-in architecture can be the casual default as opposed to the herculean effort it once was.

Những file được link động, có thể cắm ghép với nhau lúc chạy ấy, chính là các component phần mềm trong kiến trúc (architecture) của chúng ta. Phải mất 50 năm, nhưng cuối cùng chúng ta đã tới được chỗ mà kiến trúc plugin dạng component có thể là lựa chọn mặc định nhẹ tênh, thay vì một nỗ lực phi thường như xưa.

Component Cohesion

Độ gắn kết của component

Which classes belong in which components? This is an important decision, and it requires guidance from **good** software engineering principles. Unfortunately, over the years, this decision has been made in an ad hoc manner based almost entirely on context.

Class nào thì thuộc về component nào? Đây là một quyết định quan trọng, và nó cần được dẫn dắt bởi những nguyên tắc kỹ nghệ phần mềm đúng đắn. Đáng tiếc là suốt nhiều năm qua, quyết định này thường được đưa ra một cách tùy tiện, gần như hoàn toàn dựa vào ngữ cảnh.

In this section, we will discuss the three principles of component **cohesion**:

Trong mục này, chúng ta sẽ bàn về ba nguyên tắc của độ gắn kết (cohesion) trong component:

- REP: The Reuse/Release Equivalence Principle
- CCP: The Common Closure Principle
- CRP: The Common Reuse Principle
 - REP: Nguyên tắc Tương đương Tái sử dụng/Phát hành (The Reuse/Release Equivalence Principle)
 - CCP: Nguyên tắc Đóng kín Chung (The Common Closure Principle)
 - CRP: Nguyên tắc Tái sử dụng Chung (The Common Reuse Principle)

The Reuse/Release Equivalence Principle (REP)

Nguyên tắc Tương đương Tái sử dụng/Phát hành (REP)

The granule of reuse is the granule of release. The past decade has seen the rise of a menagerie of module management tools, such as Maven, Leiningen, the tools.deps library, and RVM. These tools have grown in importance because, during that time, a vast number of reusable components and component libraries have been created. We are now living in the age of software reuse—a fulfillment of one of the oldest promises of OO.

Hạt tái sử dụng chính là hạt phát hành. Thập kỷ vừa qua chứng kiến sự trỗi dậy của cả một vườn thú công cụ quản lý module: Maven, Leiningen, thư viện tools.deps, RVM... Những công cụ này ngày càng quan trọng, bởi trong quãng thời gian đó, vô số component và thư viện component tái sử dụng được đã ra đời. Chúng ta đang sống trong thời đại tái sử dụng phần mềm — một trong những lời hứa xưa cũ nhất của OO cuối cùng cũng thành hiện thực.

The REP is a principle that seems obvious, in hindsight. People who want to reuse software components cannot, and will not, do so unless those components are tracked through a release process and are given release numbers.

Nhìn lại thì REP có vẻ hiển nhiên. Người muốn tái sử dụng component phần mềm sẽ không thể, và sẽ không chịu, làm điều đó trừ phi những component ấy được theo dõi qua một quy trình phát hành và được gán số hiệu phát hành.

This is not simply because, without release numbers, there would be no way to ensure that all the reused components are compatible with each other. It is also because software developers need to know when new releases are coming and what changes those new releases will bring.

Không chỉ vì nếu thiếu số hiệu phát hành thì chẳng có cách nào đảm bảo mọi component được tái sử dụng tương thích với nhau. Còn vì lập trình viên cần biết khi nào có bản phát hành mới và bản mới ấy mang theo những thay đổi gì.

It is not uncommon for developers to be alerted about a new release and decide, based on the changes made in that release, to continue to use the old release instead. Therefore, the release process must produce the appropriate notifications and release documentation so that users can make informed decisions about when and whether to integrate.

Chuyện lập trình viên được báo có bản phát hành mới rồi, sau khi xem những thay đổi trong đó, quyết định vẫn tiếp tục dùng bản cũ là hoàn toàn bình thường. Vì thế, quy trình phát hành phải tạo ra thông báo và tài liệu phát hành phù hợp, để người dùng có đủ thông tin mà quyết định khi nào và có nên tích hợp hay không.

From a software design and architecture point of view, this principle means that the classes and modules that are formed into a component must belong to a cohesive group. The component cannot simply consist of a random hodgepodge of classes and modules. There must be some overarching theme or purpose that those modules all share.

Xét từ góc độ thiết kế và kiến trúc phần mềm, nguyên tắc này có nghĩa là các class và module được gộp thành một component phải thuộc về một nhóm gắn kết. Component không thể chỉ là một mớ class và module hổ lốn nhặt ngẫu nhiên. Phải có một chủ đề hay mục đích bao trùm mà tất cả các module ấy cùng chia sẻ.

Of course, this should be obvious. However, there is another way to look at this that is perhaps not quite so obvious. Classes and modules that are grouped together into a component should be releasable together. The fact that they share the same version number and the same release tracking and are included under the same release documentation should make sense both to the author and the users.

Tất nhiên, điều này thì hiển nhiên rồi. Nhưng còn một cách nhìn khác, có lẽ không hiển nhiên đến thế. Các class và module được gom vào cùng một component thì phải phát hành cùng nhau được. Việc chúng dùng chung một số hiệu phiên bản, chung một cơ chế theo dõi phát hành và nằm chung trong một tài liệu phát hành phải là điều hợp lý với cả tác giả lẫn người dùng.

Clearly this is weak advice. Saying that something should “make sense” is just a way of waving your hands in the air and trying to sound authoritative. The advice is weak because it is hard to precisely explain the glue that holds the classes and modules together into a single component. Weak though the advice may be, the principle itself is important, because violations are easy to detect—they don’t “make sense.” If you violate the REP, your users will know, and will not be impressed with your architectural skills.

Rõ ràng đây là lời khuyên yếu ớt. Bảo rằng cái gì đó phải “hợp lý” thì chẳng khác nào khua tay trong không khí mà cố tỏ ra đầy uy tín. Lời khuyên này yếu vì rất khó giải thích chính xác thứ keo nào gắn các class và module lại thành một component duy nhất. Dù lời khuyên có yếu, bản thân nguyên tắc vẫn quan trọng, bởi vi phạm nó thì rất dễ phát hiện — nó “không hợp lý”. Nếu bạn vi phạm REP, người dùng sẽ nhận ra, và họ sẽ chẳng nề phục tài kiến trúc của bạn đâu.

The weakness of this principle is more than compensated for by the strength of the next two. Indeed, the next two principles strongly constrain this principle.

Điểm yếu của nguyên tắc này được bù đắp thừa thãi bởi sức mạnh của hai nguyên tắc tiếp theo. Thực vậy, hai nguyên tắc sau ràng buộc nó rất chặt.

The Common Closure Principle (CCP)

Nguyên tắc Đóng kín Chung (CCP)

Gather into components those classes that change for the same reasons and at the same times. Separate into different components those classes that change for different reasons or at different times.

Hãy gom vào cùng một component những class thay đổi vì cùng lý do và vào cùng thời điểm. Hãy tách sang các component khác nhau những class thay đổi vì lý do khác nhau hoặc vào thời điểm khác nhau.

This is the Single Responsibility Principle (SRP) restated for components. Just as the SRP says that a **class** should not contain multiples reasons to change, this principle says that a component should not have multiple reasons to change.

Đây chính là SRP — Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle) — phát biểu lại cho component. Cũng như SRP nói rằng một class không được chứa nhiều lý do để thay đổi, nguyên tắc này nói rằng một component không được có nhiều lý do để thay đổi.

For most applications, maintainability is more important than reusability. If the code in an application must change, you would rather the changes occur all in one component than being distributed through many components. If changes are focused into a single component, then we need only redeploy the one changed component. Other components that don’t depend upon the changed component do not need to be revalidated or redeployed.

Với đa số ứng dụng, khả năng bảo trì quan trọng hơn khả năng tái sử dụng. Nếu mã trong ứng dụng buộc phải đổi, bạn sẽ muốn mọi thay đổi dồn cả vào một component hơn là rải rác khắp nhiều component. Nếu thay đổi tập trung vào một component duy nhất, ta

chỉ cần triển khai lại đúng component đã đổi. Các component khác không phụ thuộc vào component đó thì chẳng cần kiểm định lại hay triển khai lại.

The CCP prompts us to gather together in one place all the classes that are likely to change for the same reasons. If two classes are so tightly bound, either physically or conceptually, that they always change together, then they belong in the same component. This minimizes the workload related to releasing, revalidating, and redeploying the software.

CCP thúc chúng ta gom về cùng một chỗ tất cả những class có khả năng thay đổi vì cùng những lý do. Nếu hai class ràng buộc chặt với nhau, dù về mặt vật lý hay khái niệm, đến mức luôn thay đổi cùng nhau, thì chúng thuộc về cùng một component. Điều này giảm thiểu khối lượng công việc liên quan tới phát hành, kiểm định lại và triển khai lại phần mềm.

This principle is closely associated with the Open–Closed Principle (OCP). For it is “closure” in the OCP sense of the word that this principle is dealing with. The OCP states that classes should be closed for modification but open for extension. But 100% closure is not attainable. Closure must be strategic. We design our classes such that they are closed to the most common kinds of changes that we expect or have experienced.

Nguyên tắc này gắn bó chặt chẽ với OCP — Nguyên tắc Đóng–Mở (Open–Closed Principle). Bởi cái mà nguyên tắc này bàn tới chính là “sự đóng kín” theo đúng nghĩa của OCP. OCP nói rằng class phải đóng với việc sửa đổi nhưng mở cho việc mở rộng. Nhưng đóng kín 100% là điều bất khả. Sự đóng kín phải mang tính chiến lược. Chúng ta thiết kế class sao cho chúng đóng với những loại thay đổi phổ biến nhất mà ta dự đoán hoặc đã từng nếm trải.

The CCP amplifies this by gathering together into the same component those classes that are closed to the same types of changes. Thus, when a change in requirements comes along, that change has a good chance of being restricted to a minimal number of components.

CCP khuếch đại điều đó lên bằng cách gom vào cùng một component những class cùng đóng với cùng một loại thay đổi. Nhờ vậy, khi một thay đổi về yêu cầu ập tới, thay đổi ấy có nhiều khả năng bị giới hạn trong số component tối thiểu.

Similarity with SRP

Nét tương đồng với SRP

As I stated above, the CCP is the component form of the SRP. The SRP tells us to separate methods into different classes if they change for different reasons. The CCP tells us to separate classes into different components if they change for different reasons.

Như tôi đã nói ở trên, CCP là dạng component của SRP. SRP bảo ta tách các method sang những class khác nhau nếu chúng thay đổi vì những lý do khác nhau. CCP bảo ta tách các class sang những component khác nhau nếu chúng thay đổi vì những lý do khác nhau.

Different Times?

Thời điểm khác nhau thì sao?

The caption at the start of this section implies that a component should not have elements that change at different times, even though they might change for the same reasons. Some changes are very high priority. Other changes are lower priority. Some changes are volatile with a high frequency. Some have a much lower volatility. We’d like to avoid components that have both high- and low-volatility elements.

Câu dẫn ở đầu mục này hàm ý rằng một component không nên chứa những phần tử thay đổi vào các thời điểm khác nhau, ngay cả khi chúng thay đổi vì cùng một lý do. Có những thay đổi rất gấp. Có những thay đổi ít gấp hơn. Có những thay đổi biến động với tần suất cao. Có những thay đổi biến động thấp hơn nhiều. Chúng ta muốn tránh những component chứa lẫn lộn cả phần tử biến động cao lẫn phần tử biến động thấp.

The Common Reuse Principle (CRP)

Nguyên tắc Tái sử dụng Chung (CRP)

Don't force users of a component to depend on things they don't need.

Đừng ép người dùng của một component phải phụ thuộc vào những thứ họ không cần.

This is yet another principle that helps us decide which classes and modules should be placed into a component. It states that classes and modules that tend to be reused together belong in the same component.

Đây lại là một nguyên tắc nữa giúp ta quyết định nên đặt class và module nào vào component nào. Nó phát biểu rằng những class và module hay được tái sử dụng cùng nhau thì thuộc về cùng một component.

Classes are seldom reused in isolation. Generally, reusable classes collaborate with other classes that are part of the reusable **abstraction**. The CRP states that these classes belong together in the same component. In such a component, we would expect to see classes that have lots of dependencies upon each other.

Class hiếm khi được tái sử dụng đơn độc. Nhìn chung, các class tái sử dụng được luôn cộng tác với những class khác cùng thuộc về phần trừu tượng hoá (abstraction) tái sử dụng ấy. CRP nói rằng những class này thuộc về cùng một component. Trong một component như vậy, ta sẽ thấy các class phụ thuộc lẫn nhau rất nhiều.

A simple example might be a container class and its associated iterators. These classes are reused together because they are tightly coupled to each other. Thus, they ought to be in the same component.

Một ví dụ đơn giản là một class container và các iterator đi kèm nó. Những class này được tái sử dụng cùng nhau vì chúng kết dính chặt với nhau. Vì thế, chúng nên nằm trong cùng một component.

But the CRP tells us more than just what classes to put together into a component. It also tells us what classes not to keep together in a component.

Nhưng CRP không chỉ cho ta biết nên gom những class nào vào một component. Nó còn cho ta biết những class nào thì không nên để chung trong một component.

When one component uses another, a **dependency** is created between the components. It may be that the using component only uses one module within the used component; but that doesn't weaken the dependency at all. The using component still depends upon the used component.

Khi một component dùng một component khác, một phụ thuộc (dependency) hình thành giữa hai component. Có thể component đi dùng chỉ dùng đúng một module bên trong component được dùng; nhưng điều đó chẳng làm phụ thuộc nhẹ đi chút nào. Component đi dùng vẫn cứ phụ thuộc vào component được dùng.

Because of that dependency, every time the used component is changed, the using component will likely need corresponding changes. Even if no changes are necessary to the using component, it will

likely still need to be recompiled, revalidated, and redeployed. This may even be true if the change to the used component is something that the using component doesn't care about.

Vì phụ thuộc đó, mỗi lần component được dùng thay đổi, component đi dùng nhiều khả năng cũng phải thay đổi theo cho khớp. Cho dù nó chẳng cần sửa gì, nó vẫn nhiều khả năng phải được biên dịch lại, kiểm định lại và triển khai lại. Điều này có thể đúng ngay cả khi thay đổi ở component được dùng là thứ mà component đi dùng chẳng bận tâm.

Thus, we want to make sure that when we depend upon a component, we depend upon the majority of classes in that component. To say this another way, we want to make sure that the classes that we put into a component are inseparable; that it is unlikely that we can depend upon some and not the others. Otherwise, we will be redeploying more components than is necessary, and wasting significant effort.

Vì thế, ta muốn chắc rằng khi đã phụ thuộc vào một component thì ta phụ thuộc vào phần lớn các class trong component đó. Nói cách khác, ta muốn chắc rằng những class ta bỏ vào một component là không thể tách rời; rằng khó có chuyện ta phụ thuộc vào vài class trong đó mà không phụ thuộc vào số còn lại. Nếu không, ta sẽ phải triển khai lại nhiều component hơn mức cần thiết, lãng phí công sức đáng kể.

Therefore, the CRP tells us more about what classes shouldn't be together than what classes should be together. The CRP says that classes that are not tightly bound to each other should not be in the same component.

Do đó, CRP nói cho ta biết nhiều hơn về những class nào không nên ở chung, hơn là về những class nào nên ở chung. CRP nói rằng những class không ràng buộc chặt với nhau thì không nên nằm trong cùng một component.

Relation to ISP

Quan hệ với ISP

The CRP is the generic version of the ISP. The ISP advises us not to depend upon classes that have methods we don't use. The CRP advises us not to depend upon components that have modules we don't use. All of this advice can be reduced to a single sound bite:

CRP là phiên bản tổng quát của ISP. ISP — Nguyên tắc Phân tách Interface (Interface Segregation Principle) — khuyên ta đừng phụ thuộc vào những class có method mà ta không dùng. CRP khuyên ta đừng phụ thuộc vào những component có module mà ta không dùng. Toàn bộ lời khuyên này có thể rút gọn thành một câu gọn lỏn:

Don't depend on things you don't need.

Đừng phụ thuộc vào những thứ bạn không cần.

The Tension Diagram for Component Cohesion

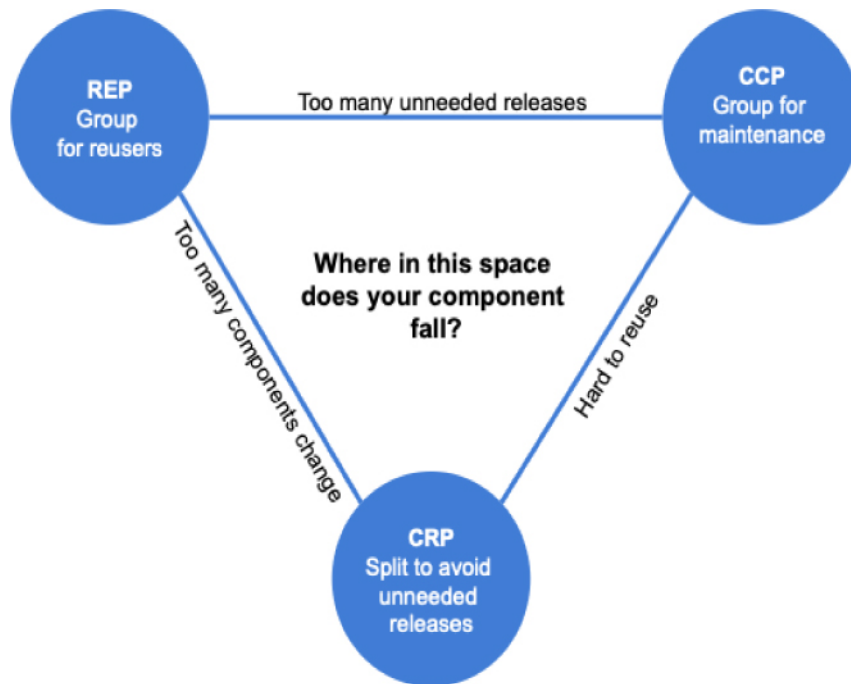
Sơ đồ căng kéo cho độ gắn kết của component

You may have already realized that the three cohesion principles tend to fight each other. The REP and CCP are inclusive principles. They both tend to make components larger. The CCP is an exclusive principle, driving components to be smaller. It is the tension between these principles that we seek to resolve.

Có lẽ bạn đã nhận ra rằng ba nguyên tắc về độ gắn kết này thường xung khắc nhau. REP và CCP là những nguyên tắc mang tính bao gộp. Cả hai đều có xu hướng làm component to ra. CCP là nguyên tắc mang tính loại trừ, đẩy component nhỏ lại. Chính sự căng kéo giữa các nguyên tắc này là thứ ta phải hoá giải.

Below is a tension diagram 6 that shows how the three principles of cohesion interact with each other. The edges of the diagram describe the cost of abandoning the principle on the opposite vertex.

Dưới đây là một sơ đồ căng kéo 6 cho thấy ba nguyên tắc về độ gắn kết tương tác với nhau ra sao. Các cạnh của sơ đồ mô tả cái giá phải trả khi từ bỏ nguyên tắc nằm ở đỉnh đối diện.



So, a programmer who focuses on the REP and the CRP will find that too many components are impacted when simple changes are made. An architect who focuses too strongly on the CCP and the REP will cause too many unneeded releases to be made.

Vậy nên, lập trình viên chỉ chăm chăm vào REP và CRP sẽ thấy quá nhiều component bị ảnh hưởng mỗi khi có một thay đổi đơn giản. Còn kiến trúc sư nghiêng quá mạnh về CCP và REP sẽ khiến quá nhiều bản phát hành không cần thiết bị tung ra.

The goal is to find a position in that tension triangle that meets the current concerns of the development team, and shift that position as the system evolves over time. For example, early in the development of a project, the CCP is much more important than the REP, because developability is more important than reuse.

Mục tiêu là tìm được một vị trí trong tam giác căng kéo đó đáp ứng mối bận tâm hiện tại của nhóm phát triển, rồi dịch chuyển vị trí ấy khi hệ thống tiến hoá theo thời gian. Chẳng hạn, ở giai đoạn đầu của một dự án, CCP quan trọng hơn REP nhiều, bởi khả năng phát triển quan trọng hơn khả năng tái sử dụng.

Generally, projects tend to start on the right-hand side of the triangle, where the only sacrifice is reuse. As the project matures and other projects begin to draw from it, the project will slide over to the left.

Nhìn chung, các dự án thường khởi đầu ở phía bên phải tam giác, nơi thứ duy nhất bị hy sinh là khả năng tái sử dụng. Khi dự án chín muồi và các dự án khác bắt đầu lấy đồ từ nó, dự án sẽ trượt dần sang trái.

This means that the component **structure** of a project can vary with time and maturity. It has more to do with the way that project is developed and used than with what the project actually does.

Điều này nghĩa là cấu trúc component của một dự án có thể biến thiên theo thời gian và theo độ chín. Nó liên quan tới cách dự án được phát triển và sử dụng nhiều hơn là tới việc dự án thực sự làm gì.

In Summary . . .

Tóm lại . . .

In the past, our view of cohesion was much simpler than the last three principles have implied. We used to think that cohesion was simply the attribute of a module to perform one, and only one, function. However, the three principles of component cohesion describe a much more complex variety of cohesion. In choosing the classes to group together into components, we must consider the opposing forces involved in reusability and developability. Balancing these forces with the needs of the application is nontrivial. Moreover, the balance is almost always dynamic. That is, the partitioning that is appropriate today might not be appropriate next year. Thus, the composition of the components will likely jitter and evolve with time as the focus of the project changes from developability to reusability.

Trước đây, quan niệm của chúng ta về độ gắn kết đơn giản hơn nhiều so với những gì ba nguyên tắc vừa rồi hàm ý. Chúng ta từng nghĩ độ gắn kết chỉ đơn thuần là thuộc tính của một module khi nó thực hiện một, và chỉ một, chức năng. Nhưng ba nguyên tắc về độ gắn kết của component mô tả một kiểu gắn kết phức tạp hơn thế nhiều. Khi chọn class nào để gom vào component nào, ta phải cân nhắc những lực đối nghịch giữa khả năng tái sử dụng và khả năng phát triển. Cân bằng những lực ấy với nhu cầu của ứng dụng chẳng hề tầm thường. Hơn nữa, thế cân bằng ấy gần như luôn động. Nghĩa là cách phân chia hợp lý hôm nay có thể không còn hợp lý vào năm sau. Vì thế, thành phần của các component nhiều khả năng sẽ rung lắc và tiến hoá theo thời gian, khi trọng tâm của dự án chuyển từ khả năng phát triển sang khả năng tái sử dụng.

This means that the component structure of the system cannot be determined up front, and must incrementally evolve as the system grows and changes. And this, of course, means that the changes to the component structure will be small but frequent. They become a normal part of any Agile iterative cycle.

Điều này nghĩa là cấu trúc component của hệ thống không thể xác định ngay từ đầu, mà phải tiến hoá dần từng bước khi hệ thống lớn lên và thay đổi. Và tất nhiên, nó cũng nghĩa là những thay đổi lên cấu trúc component sẽ nhỏ nhưng thường xuyên. Chúng trở thành một phần bình thường của bất kỳ chu kỳ lặp Agile nào.

Component **Coupling**

Độ kết dính của component

The next three principles deal with the relationships between components. Here again we will run into the tension between developability and logical design. The forces that impinge upon the architecture of a component structure are technical, political, and volatile.

Ba nguyên tắc tiếp theo bàn về quan hệ giữa các component. Ở đây, một lần nữa ta sẽ vấp phải sự căng kéo giữa khả năng phát triển và thiết kế logic. Những lực tác động lên kiến trúc của một cấu trúc component vừa mang tính kỹ thuật, vừa mang tính chính trị, lại vừa hay biến động.

The Acyclic Dependencies Principle (ADP)

Nguyên tắc Phụ thuộc Phi chu trình (ADP)

Allow no cycles in the component dependency graph. Have you ever worked all day, gotten some stuff working and then gone home, only to arrive the next morning to find that your stuff no longer works? Why doesn't it work? Because somebody stayed later than you and changed something you depend upon! I call this the morning-after syndrome.

Không cho phép chu trình nào tồn tại trong đồ thị phụ thuộc giữa các component. Đã bao giờ bạn làm việc cả ngày, chạy được vài thứ rồi về nhà, để rồi sáng hôm sau đến nơi thì phát hiện thứ của mình không còn chạy nữa chưa? Sao lại không chạy? Vì có người ở lại muộn hơn bạn và đã sửa mất một thứ mà bạn phụ thuộc vào! Tôi gọi đây là hội chứng sáng-hôm-sau (the morning-after syndrome).

The morning-after syndrome occurs in development environments where many developers are modifying the same source files. In relatively small projects with just a few developers, it isn't too big a problem. But as the sizes of the project and the development team grow, the mornings after can get pretty nightmarish. It is not uncommon for weeks to go by without being able to build a stable version of the project. Instead, everyone keeps on changing and changing their code, trying to make it work with the last changes that someone else made.

Hội chứng sáng-hôm-sau xảy ra trong những môi trường phát triển nơi nhiều lập trình viên cùng sửa chung một tập file mã nguồn. Ở những dự án tương đối nhỏ chỉ vài người, đó chưa phải vấn đề lớn. Nhưng khi quy mô dự án và nhóm phát triển phình ra, những buổi sáng hôm sau có thể trở thành ác mộng. Chuyện cả tuần trôi qua mà không build nổi một phiên bản ổn định của dự án là không hiếm. Thay vào đó, ai nấy cứ sửa rồi lại sửa mã của mình, cố làm cho nó chạy được với những thay đổi mới nhất mà người khác vừa thực hiện.

Eliminating Dependency Cycles

Loại bỏ chu trình phụ thuộc

The solution to this problem is to partition the development environment into releasable components. The components become units of work, which can be the responsibility of a developer or a team of developers. When developers get a component working, they release it for use by the other developers. They give it a release number and move it into a directory for other teams to use. They then continue to modify their component in their own private areas. Everyone else uses the released version.

Lời giải cho vấn đề này là chia môi trường phát triển thành các component phát hành được. Component trở thành đơn vị công việc, có thể giao cho một lập trình viên hoặc một nhóm lập trình viên chịu trách nhiệm. Khi làm cho component của mình chạy được, họ phát hành nó cho các lập trình viên khác dùng. Họ gán cho nó một số hiệu phát hành rồi đưa vào một thư mục để các nhóm khác sử dụng. Sau đó họ tiếp tục sửa component của mình trong khu vực riêng. Mọi người khác dùng phiên bản đã phát hành.

As new releases of a component are made, other teams can decide whether or not to immediately adopt the new release. If they decide not to, they simply continue using the old release. Once they decide that they are ready, they begin to use the new release.

Khi có bản phát hành mới của một component, các nhóm khác có thể quyết định có nhận bản mới ngay hay không. Nếu quyết định không, họ cứ tiếp tục dùng bản cũ. Khi nào thấy đã sẵn sàng, họ mới bắt đầu dùng bản mới.

Thus, none of the teams are at the mercy of the others. Changes made to one component do not need to have an immediate effect on other teams. Each team can decide for itself when to adapt its

components to new releases of the components they use. Moreover, integration happens in small increments. There is no single point in time when all developers must come together and integrate everything they are doing.

Nhờ vậy, không nhóm nào bị phó mặc cho nhóm khác định đoạt. Thay đổi ở một component không nhất thiết tác động ngay lập tức tới các nhóm khác. Mỗi nhóm tự quyết định khi nào thì thích ứng component của mình với bản phát hành mới của những component mà họ dùng. Hơn nữa, việc tích hợp diễn ra theo từng bước nhỏ. Không có một thời điểm duy nhất nào mà tất cả lập trình viên phải xúm lại và tích hợp mọi thứ họ đang làm.

This is a very simple and rational process; and it is widely used. However, to make it work you must manage the dependency structure of the components. There can be no cycles .

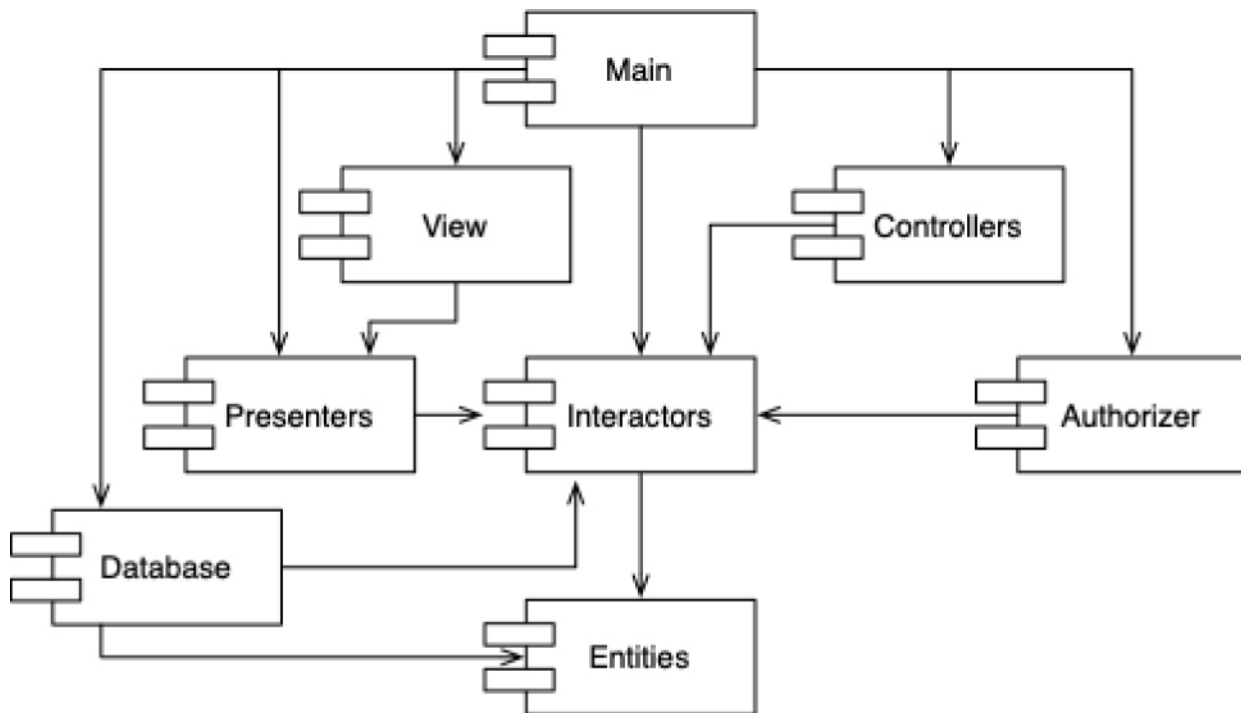
Đây là một quy trình rất đơn giản và hợp lý; và nó được dùng rộng rãi. Tuy nhiên, để nó chạy được, bạn phải quản lý cấu trúc phụ thuộc giữa các component. Không được có chu trình nào cả.

If there are cycles in the dependency structure, then the morning-after syndrome cannot be avoided.

Nếu cấu trúc phụ thuộc có chu trình thì không thể nào tránh được hội chứng sáng-hôm-sau.

Consider the component diagram below.

Hãy xem sơ đồ component dưới đây.



Here we see a rather typical structure of components assembled into an application. The function of this application is unimportant for the purpose of this example. What is important is the dependency structure of the components. Notice that this structure is a directed graph. The components are the nodes, and the dependency relationships are the directed edges.

Ở đây ta thấy một cấu trúc khá điển hình gồm các component lắp ghép thành một ứng dụng. Ứng dụng này làm gì thì không quan trọng với ví dụ này. Điều quan trọng là cấu trúc phụ

thuộc giữa các component. Để ý rằng cấu trúc này là một đồ thị có hướng. Component là các nút, còn quan hệ phụ thuộc là các cạnh có hướng.

Now notice one more thing. Regardless of which component you begin at, it is impossible to follow the dependency relationships and wind up back at that component. This structure has no cycles. It is a directed acyclic graph (DAG).

Giờ để ý thêm một điều nữa. Dù bạn xuất phát từ component nào, cũng không thể lần theo các quan hệ phụ thuộc mà quay về đúng component đó. Cấu trúc này không có chu trình. Nó là một đồ thị có hướng phi chu trình (directed acyclic graph — DAG).

Now notice what happens when the team responsible for Presenters makes a new release of their component. It is easy to find out who is affected by this release; you just follow the dependency arrows backward. Thus, View and Main are both going to be affected. The developers currently working on those components will have to decide when they should integrate with the new release of Presenters .

Bây giờ hãy để ý chuyện gì xảy ra khi nhóm phụ trách Presenters phát hành bản mới của component họ. Rất dễ biết ai bị ảnh hưởng bởi bản phát hành này; bạn chỉ việc lần ngược theo các mũi tên phụ thuộc. Vậy là View và Main đều sẽ bị ảnh hưởng. Những lập trình viên đang làm trên hai component đó sẽ phải quyết định khi nào thì tích hợp với bản phát hành mới của Presenters .

Notice also that when Presenters is released, it has utterly no effect upon many of the other components in the system. They don't know about Presenters , and they don't care when it changes. This is nice. It means that the impact of releasing Main is relatively small.

Cũng để ý rằng khi Presenters được phát hành, nó hoàn toàn chẳng ảnh hưởng gì tới nhiều component khác trong hệ thống. Chúng không biết Presenters là ai, và chẳng bận tâm khi nó thay đổi. Thế thì hay quá. Nghĩa là tác động của việc phát hành Main tương đối nhỏ.

When the developers working on the Presenters component would like to run a **test** of that component, all they need to do is build their version of Presenters with the versions of the Interactors and Entities components that they are currently using. None of the other components in the system need be involved. This is nice. It means that the developers working on Presenters have relatively little work to do to set up a test and that there are relatively few variables for them to consider.

Khi các lập trình viên làm trên component Presenters muốn chạy test cho component đó, tất cả những gì họ cần làm là build phiên bản Presenters của mình với phiên bản Interactors và Entities mà họ đang dùng. Không component nào khác trong hệ thống phải dính vào. Thế thì hay quá. Nghĩa là những người làm trên Presenters chỉ phải bỏ ra tương đối ít công sức để dựng một bài test, và họ cũng có tương đối ít biến số phải cân nhắc.

When it is time to release the whole system, it is done from the bottom up. First, the Entities component is compiled, tested, and released; then, Database and Interactors . These are followed by Presenters , View , Controllers , and then Authorizer . Main is last. This process is very clear and easy to deal with. We know how to build the system because we understand the dependencies between its parts.

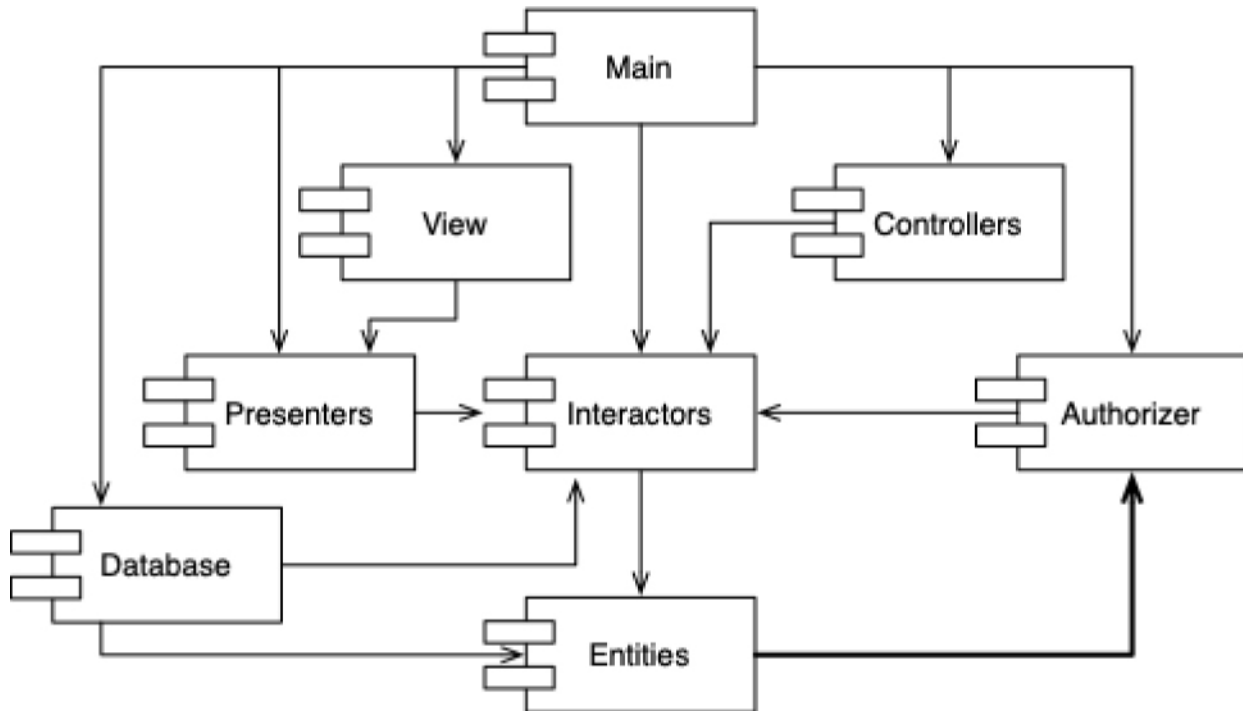
Khi tới lúc phát hành cả hệ thống, việc đó được làm từ dưới lên. Trước hết, component Entities được biên dịch, test và phát hành; rồi tới Database và Interactors . Tiếp theo là Presenters , View , Controllers , rồi Authorizer . Main sau cùng. Quy trình này rất rõ ràng và dễ xử lý. Chúng ta biết cách build hệ thống vì chúng ta hiểu các phụ thuộc giữa những phần của nó.

The Effect of a Cycle in the Component Dependency Graph

Hệ quả của một chu trình trong đồ thị phụ thuộc component

Let us say that a new requirement forces us to change one of the modules in Entities such that it makes use of a module in Authorizer . For example, let's say that the User class in Entities uses the Permissions class in Authorizer . This creates a dependency cycle, as shown below.

Giả sử một yêu cầu mới buộc ta phải sửa một module trong Entities sao cho nó dùng tới một module trong Authorizer . Chẳng hạn, giả sử class User trong Entities dùng class Permissions trong Authorizer . Việc này tạo ra một chu trình phụ thuộc, như minh hoạ dưới đây.



This cycle effectively combines Interactors , Entities , and Authorizer into a single component. Any component that uses one depends upon them all. And this makes building and testing much more difficult. It also begs the question of the correct build order. You can't build Entities first, because it depends upon Authorizer . But Authorizer depends upon Entities through Interactors . So there is no correct build order . Any system built without a correct build order functions by accident rather than by plan.

Trên thực tế, chu trình này đã gộp Interactors , Entities và Authorizer thành một component duy nhất. Bất kỳ component nào dùng một trong ba thì phụ thuộc vào cả ba. Và điều đó khiến việc build cùng việc test khó hơn nhiều. Nó cũng đặt ra câu hỏi về thứ tự build đúng đắn. Bạn không thể build Entities trước, vì nó phụ thuộc vào Authorizer . Nhưng Authorizer lại phụ thuộc vào Entities thông qua Interactors . Vậy là chẳng có thứ tự build nào đúng cả. Bất kỳ hệ thống nào được build mà không có thứ tự build đúng thì chạy được là nhờ may rủi chứ không phải nhờ tính toán.

Breaking the Cycle

Phá vỡ chu trình

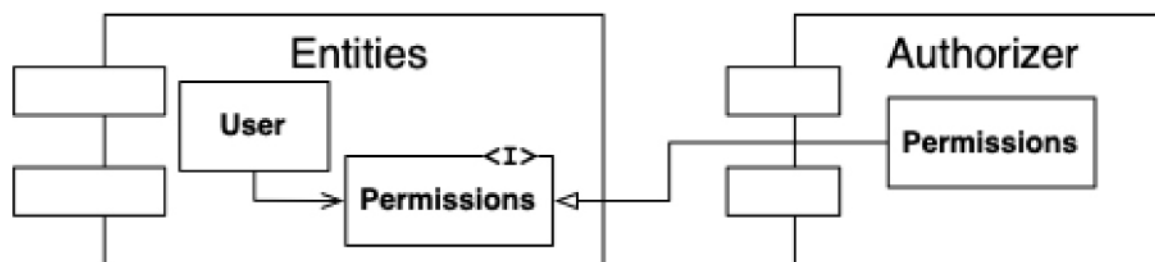
It is always possible to break a cycle of components and reinstate the dependency graph as a DAG.

There are two primary mechanisms.

Luôn luôn có thể phá vỡ một chu trình giữa các component và đưa đồ thị phụ thuộc trở lại thành DAG. Có hai cơ chế chính.

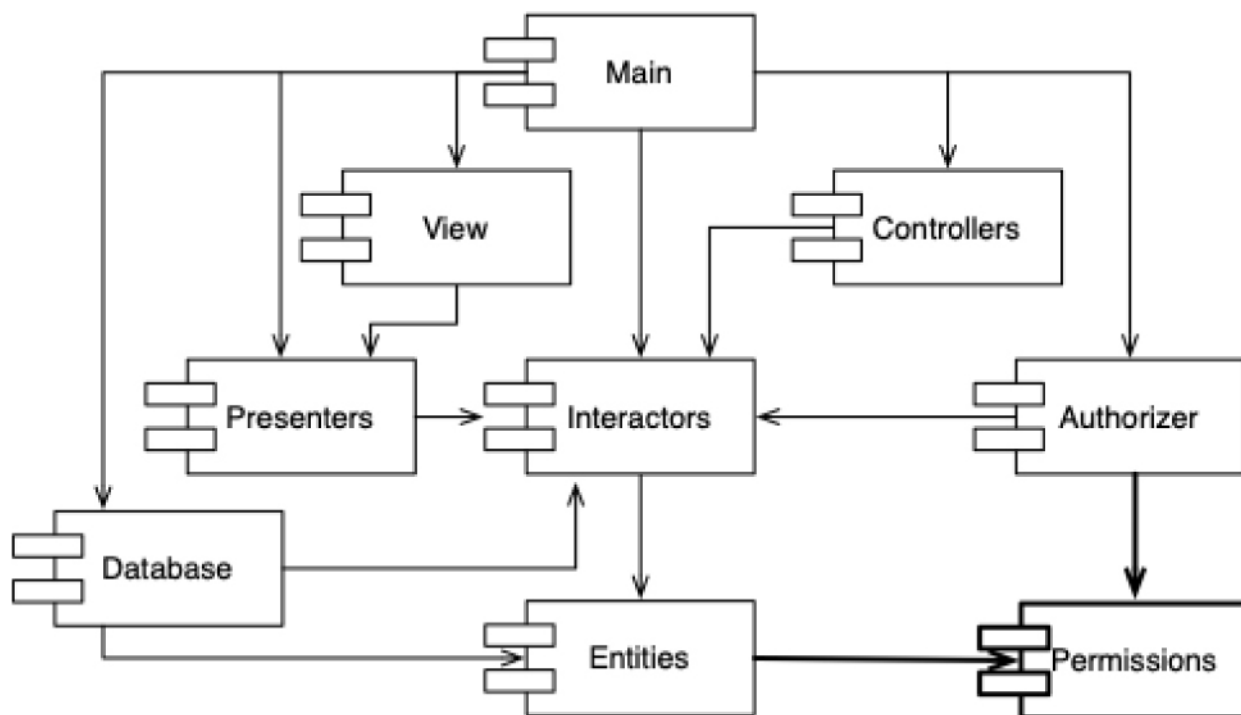
1. Apply the Dependency Inversion Principle (DIP). In the case below, we could create an **interface** that has the methods that User needs. We could then put that interface into Entities and inherit it into Authorizer . This inverts the dependency between Entities and Authorizer , thus breaking the cycle. See the before-and-after view below.

1. Áp dụng DIP — Nguyên tắc Đảo ngược Phụ thuộc (Dependency Inversion Principle). Trong trường hợp dưới đây, ta có thể tạo một interface chứa những method mà User cần. Rồi ta đặt interface đó vào Entities và cho Authorizer kế thừa nó. Cách này đảo ngược phụ thuộc giữa Entities và Authorizer , nhờ đó phá vỡ chu trình. Xem hình so sánh trước-sau bên dưới.



2. Create a new component that both Entities and Authorizer depend upon. Move the class(es) that they both depend upon into that new component. See below.

2. Tạo một component mới mà cả Entities lẫn Authorizer đều phụ thuộc vào. Chuyển những class mà cả hai cùng phụ thuộc sang component mới đó. Xem hình dưới.



The “Jitters”

Những cơn “rung lắc”

The second solution implies that the component structure is volatile in the presence of changing requirements. Indeed, as the application grows, the component dependency structure jitters and grows. Thus, the dependency structure must always be monitored for cycles. When cycles occur, they must be broken somehow. Sometimes this will mean creating new components, making the dependency structure grow.

Giải pháp thứ hai hàm ý rằng cấu trúc component sẽ biến động khi yêu cầu thay đổi. Thật vậy, khi ứng dụng lớn lên, cấu trúc phụ thuộc giữa các component vừa rung lắc vừa phình ra. Vì thế, cấu trúc phụ thuộc phải luôn được theo dõi để phát hiện chu trình. Khi chu trình xuất hiện, phải phá vỡ chúng bằng cách nào đó. Đôi khi điều đó có nghĩa là phải tạo thêm component mới, làm cấu trúc phụ thuộc lớn thêm.

Top-Down Design

Thiết kế từ trên xuống

The issues we have discussed so far lead to an inescapable conclusion. The component structure cannot be designed from the top down. This means that it is not one of the first things about the system that is designed. Indeed, it evolves as the system grows and changes.

Những vấn đề ta bàn tới từ đầu tới giờ dẫn đến một kết luận không thể tránh khỏi. Cấu trúc component không thể được thiết kế từ trên xuống. Nghĩa là nó không phải một trong những thứ đầu tiên của hệ thống được đem ra thiết kế. Thực vậy, nó tiến hoá cùng với việc hệ thống lớn lên và thay đổi.

Some of you may find this to be counterintuitive. We have come to expect that large-grained decompositions, like components, are also high-level functional decompositions.

Một số bạn có thể thấy điều này đi ngược trực giác. Chúng ta đã quen nghĩ rằng những phân rã ở mức thô, chẳng hạn như component, cũng đồng thời là phân rã chức năng ở

mức cao.

When we see a large-grained grouping, such as a component dependency structure, we feel that the components ought to somehow represent the functions of the system. Yet this does not seem to be an attribute of component dependency diagrams.

Khi nhìn thấy một cách nhóm ở mức thô, ví dụ một cấu trúc phụ thuộc giữa các component, ta có cảm giác rằng các component ấy hẳn phải biểu diễn các chức năng của hệ thống theo cách nào đó. Vậy mà điều này lại chẳng phải là thuộc tính của sơ đồ phụ thuộc component.

In fact, component dependency diagrams have very little to do with describing the function of the application. Instead, they are a map to the buildability and maintainability of the application. This is why they aren't designed at the start of the project. There is no software to build or maintain, and so there is no need for a build and maintenance map. But as more and more modules accumulate in the early stages of **implementation** and design, there is a growing need to manage the dependencies so that the project can be developed without the morning-after syndrome. Moreover, we want to keep changes as localized as possible, so we start paying attention to the SRP and the CCP and collocate classes that are likely to change together.

Thực ra, sơ đồ phụ thuộc component gần như chẳng liên quan gì tới việc mô tả chức năng của ứng dụng. Thay vào đó, chúng là tấm bản đồ về khả năng build được và khả năng bảo trì của ứng dụng. Đó là lý do người ta không thiết kế chúng ngay lúc khởi động dự án. Lúc ấy chưa có phần mềm nào để build hay bảo trì, nên cũng chẳng cần bản đồ build và bảo trì làm gì. Nhưng khi ngày càng nhiều module tích tụ lại trong giai đoạn đầu của hiện thực (implementation) và thiết kế, nhu cầu quản lý các phụ thuộc (dependency) cũng lớn dần lên, để dự án có thể phát triển mà không dính hội chứng sáng-hôm-sau. Hơn nữa, chúng ta muốn giữ mọi thay đổi cục bộ nhất có thể, nên bắt đầu để mắt tới SRP và CCP, gom những class có khả năng thay đổi cùng nhau về cùng một chỗ.

One of the overriding concerns of this dependency structure is the isolation of volatility. We don't want components that change frequently and for capricious reasons to affect components that otherwise ought to be stable. For example, we don't want cosmetic changes to the GUI to have an impact on our business rules. We don't want the addition or modification of reports to have an impact on our highest-level policies. So the component dependency graph is created and molded by developers to protect stable high-value components from volatile lower-level components.

Một trong những mối bận tâm bao trùm của cấu trúc phụ thuộc này là cô lập sự biến động (volatility). Chúng ta không muốn những component thay đổi xoành xoạch vì đủ thứ lý do đồng đánh lại tác động tới những component lẽ ra phải ổn định. Ví dụ, ta không muốn một thay đổi thẩm mỹ trên GUI lại ảnh hưởng tới các quy tắc nghiệp vụ (business rule). Ta không muốn việc thêm hay sửa báo cáo lại ảnh hưởng tới các chính sách ở mức cao nhất. Vì vậy đồ thị phụ thuộc component được lập trình viên tạo ra và nhào nặn để bảo vệ những component ổn định, giá trị cao khỏi những component cấp thấp hay biến động.

As the application continues to grow, we start becoming concerned about creating reusable elements. That is when the CRP begins to influence the composition of the components. Finally, as cycles appear, the ADP is applied and the component dependency graph jitters and grows.

Khi ứng dụng tiếp tục lớn lên, chúng ta bắt đầu quan tâm tới việc tạo ra những thành phần dùng lại được. Đó là lúc CRP bắt đầu chi phối cách cấu thành các component. Cuối cùng, khi các chu trình xuất hiện, ADP được đem ra áp dụng và đồ thị phụ thuộc component lại rung lắc rồi phình ra.

If we were to try to design the component dependency structure before we had designed any classes, we would likely fail rather badly. We would not know much about common closure, we would be

unaware of any reusable elements, and we would almost certainly create components that produced dependency cycles. Thus, the component dependency structure grows and evolves with the logical design of the system.

Nếu cố thiết kế cấu trúc phụ thuộc component trước khi thiết kế bất kỳ class nào, nhiều khả năng chúng ta sẽ thất bại thảm hại. Ta sẽ chẳng biết mấy về common closure, chẳng hay biết gì về những thành phần dùng lại được, và gần như chắc chắn sẽ tạo ra những component sinh ra chu trình phụ thuộc. Bởi vậy, cấu trúc phụ thuộc component lớn lên và tiến hoá cùng với thiết kế logic của hệ thống.

The Stable Dependencies Principle (SDP)

SDP — Nguyên tắc Phụ thuộc Ổn định (The Stable Dependencies Principle)

Depend in the direction of stability. Designs cannot be completely static. Some volatility is necessary if the design is to be maintained. We accomplish this by conforming to the CCP. Using this principle, we create components that are sensitive to certain kinds of changes but immune to others. Some of these components are designed to be volatile. We expect them to change.

Hãy phụ thuộc theo hướng ổn định. Thiết kế không thể hoàn toàn tĩnh. Cần có một mức biến động nhất định thì thiết kế mới bảo trì được. Chúng ta đạt được điều đó bằng cách tuân thủ CCP. Với nguyên tắc này, ta tạo ra những component nhạy cảm với một số loại thay đổi nhưng miễn nhiễm với những loại khác. Một vài component trong đó được thiết kế để hay thay đổi. Ta trông đợi chúng thay đổi.

Any component that we expect to be volatile should not be depended upon by a component that is difficult to change! Otherwise, the volatile component will also be difficult to change.

Bất kỳ component nào ta trông đợi sẽ hay thay đổi thì đừng để một component khó thay đổi phụ thuộc vào nó! Bằng không, chính component hay thay đổi kia cũng sẽ trở nên khó thay đổi.

It is the perversity of software that a module that you have designed to be easy to change can be made hard to change by someone else who simply hangs a dependency upon it. Not a line of source code in your module need change; and yet your module will suddenly be hard to change. By conforming to the SDP, we ensure that modules that are intended to be easy to change are not depended upon by modules that are harder to change.

Cái oái oăm của phần mềm là ở chỗ: một module bạn thiết kế cho dễ thay đổi có thể bị người khác biến thành khó thay đổi, chỉ đơn giản bằng việc treo lên nó một phụ thuộc. Không một dòng mã nguồn nào trong module của bạn cần đổi; vậy mà module của bạn bỗng dưng trở nên khó thay đổi. Tuân thủ SDP là cách bảo đảm rằng những module vốn được làm ra để dễ thay đổi sẽ không bị các module khó thay đổi hơn phụ thuộc vào.

Stability

Tính ổn định (Stability)

What is meant by stability? Stand a penny on its side. Is it stable in that position? You'd likely say that it was not. However, unless disturbed, it will remain in that position for a very long time. Thus, stability has nothing directly to do with frequency of change. The penny is not changing, but it is hard to think of it as stable.

Ổn định nghĩa là gì? Hãy dựng một đồng xu đứng trên cạnh của nó. Ở tư thế đó nó có ổn định không? Hẳn bạn sẽ bảo là không. Vậy mà nếu chẳng ai đụng vào, nó sẽ đứng nguyên như thế rất lâu. Vậy nên tính ổn định chẳng liên quan trực tiếp gì tới tần suất thay đổi. Đồng xu không hề thay đổi, nhưng thật khó mà xem nó là ổn định.

Webster says that something is stable if it is “not easily moved.” Stability is related to the amount of work required to make a change. The penny is not very stable, because it requires very little work to topple it. On the other hand, a table is very stable because it takes a considerable amount of effort to turn it over.

Từ điển Webster nói rằng một thứ là ổn định nếu nó “không dễ bị dịch chuyển”. Tính ổn định gắn với lượng công sức phải bỏ ra để tạo một thay đổi. Đồng xu không mấy ổn định, vì chỉ cần rất ít công sức là làm nó đổ. Ngược lại, một cái bàn thì rất ổn định, vì phải tốn khá nhiều sức mới lật được nó.

Stability is a continuum, not a boolean value.

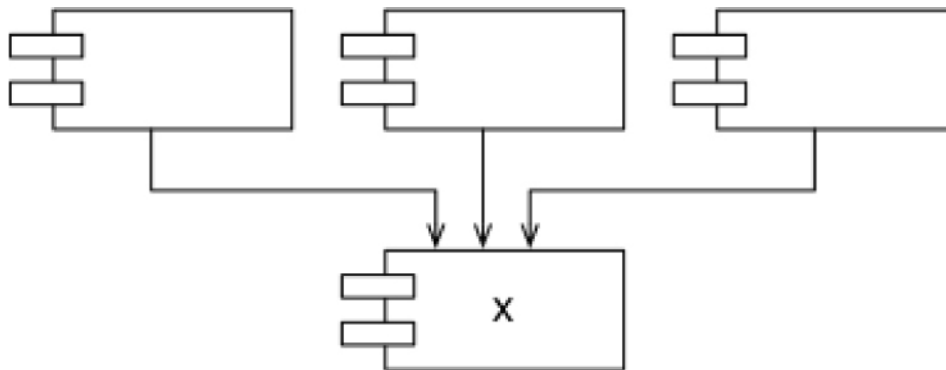
Tính ổn định là một dải liên tục, không phải một giá trị boolean.

How does this relate to software? There are many factors that make a software component hard to change—its size, complexity, clarity, and so on. We are going to ignore all those factors and focus upon something different. One sure way to make a software component difficult to change is to make lots of other software components depend upon it. A component with lots of incoming dependencies is very stable because it requires a great deal of work to reconcile any changes with all the dependent components.

Chuyện này liên quan gì tới phần mềm? Có rất nhiều yếu tố khiến một component phần mềm trở nên khó thay đổi — kích thước, độ phức tạp, độ rõ ràng, v.v. Chúng ta sẽ bỏ qua hết những yếu tố đó và tập trung vào một thứ khác. Một cách chắc chắn để khiến một component phần mềm khó thay đổi là làm cho thật nhiều component khác phụ thuộc vào nó. Một component có nhiều phụ thuộc đi vào thì rất ổn định, bởi phải tốn rất nhiều công mới dung hoà được mọi thay đổi với tất cả các component phụ thuộc vào nó.

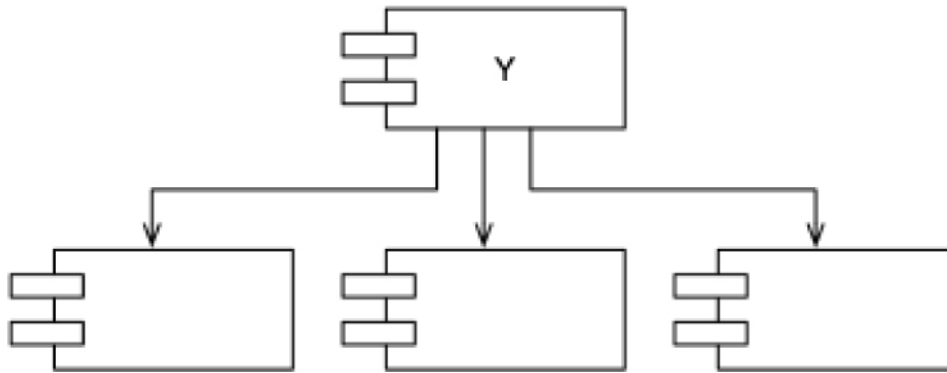
The following diagram shows X, a stable component. This component has three components depending upon it, and therefore it has three good reasons not to change. We say that it is responsible to those three components. On the other hand, X depends upon nothing, so it has no external influence to make it change. We say that X is responsible and independent: It is an adult.

Sơ đồ dưới đây cho thấy X, một component ổn định. Component này có ba component phụ thuộc vào nó, vì thế nó có ba lý do chính đáng để không thay đổi. Ta nói rằng nó có trách nhiệm với ba component đó. Mặt khác, X chẳng phụ thuộc vào thứ gì cả, nên không có ảnh hưởng bên ngoài nào bắt nó phải đổi. Ta nói X vừa có trách nhiệm vừa độc lập: nó là một người trưởng thành.



The next diagram, on the other hand, shows Y, a very unstable component. Y has no other components depending upon it; we say that it is irresponsible. Y also has three components that it depends upon, so changes may come from three external sources. We say that Y is dependent and irresponsible: It is a teenager.

Ngược lại, sơ đồ tiếp theo cho thấy Y, một component rất bất ổn định. Y không có component nào phụ thuộc vào nó; ta nói nó vô trách nhiệm. Y lại phụ thuộc vào ba component khác, nên thay đổi có thể ập đến từ ba nguồn bên ngoài. Ta nói Y vừa lệ thuộc vừa vô trách nhiệm: nó là một cậu thiếu niên.



Stability Metrics

Các thước đo tính ổn định

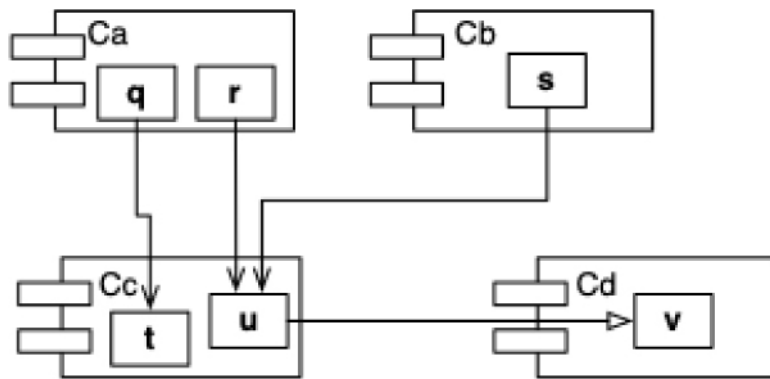
How can we measure the stability of a component? One way is to count the number of dependencies that enter and leave that component. These counts will allow us to calculate the positional stability of the component.

Làm sao đo được tính ổn định của một component? Một cách là đếm số phụ thuộc đi vào và đi ra khỏi component đó. Những con số này cho phép ta tính ra tính ổn định theo vị trí (positional stability) của component.

- Fan-in: Incoming Couplings—This is the number of modules outside this component that depend upon modules within this component.
- Fan-out: Outgoing Couplings—This is the number of modules inside this component that depend upon modules outside this component.
- I: Instability: $I = \text{Fan-out} \div (\text{Fan-in} + \text{Fan-out})$. This metric has the range $[0,1]$. $I = 0$ indicates a maximally stable component. $I = 1$ indicates a maximally unstable component.
 - Fan-in: Coupling đi vào — Đây là số module nằm ngoài component đang xét mà phụ thuộc vào các module bên trong component đó.
 - Fan-out: Coupling đi ra — Đây là số module nằm trong component đang xét mà phụ thuộc vào các module bên ngoài component đó.
 - I: Tính bất ổn định (Instability): $I = \text{Fan-out} \div (\text{Fan-in} + \text{Fan-out})$. Thước đo này nằm trong khoảng $[0,1]$. $I = 0$ hàm ý component ổn định tối đa. $I = 1$ hàm ý component bất ổn định tối đa.

The Fan-in and Fan-out metrics are calculated by counting the number of modules outside the component in question that have dependencies on the modules inside the component in question. Consider the example below: In previous publications I used the names Efferent and Afferent couplings (Ce, and Ca) for Fan-out and Fan-in respectively. That was just hubris on my part... I liked the metaphor of the central nervous system.

Hai thước đo Fan-in và Fan-out được tính bằng cách đếm số module nằm ngoài component đang xét mà có phụ thuộc vào các module bên trong component đó. Hãy xét ví dụ dưới đây: Trong các ấn phẩm trước tôi dùng tên Efferent và Afferent couplings (Ce và Ca) lần lượt cho Fan-out và Fan-in. Đó chỉ là thói ngạo mạn của tôi thôi... tôi khoái cái ẩn dụ về hệ thần kinh trung ương.



Let's say we want to calculate the stability of the component Cc. We find that there are three classes outside Cc that depend upon classes in Cc. Thus, Fan-in = 3. Moreover, there is one class outside Cc that classes in Cc depend upon. Thus, Fan-out = 1, and $I = 1/4$.

Giả sử ta muốn tính tính ổn định của component Cc. Ta thấy có ba class nằm ngoài Cc phụ thuộc vào các class trong Cc. Vậy Fan-in = 3. Ngoài ra, có một class nằm ngoài Cc mà các class trong Cc phụ thuộc vào. Vậy Fan-out = 1, và $I = 1/4$.

In C++, these dependencies are typically represented by `#include` statements. Indeed, the I metric is easiest to calculate when you have organized your source code such that there is one class in each source file. In Java, the I metric can be calculated by counting import statements and qualified names. Other languages have similar mechanisms such as `using` or `require`.

Trong C++, những phụ thuộc này thường được thể hiện bằng các câu lệnh `#include`. Thật vậy, thước đo I dễ tính nhất khi bạn tổ chức mã nguồn sao cho mỗi file nguồn chỉ chứa một class. Trong Java, có thể tính I bằng cách đếm các câu lệnh `import` và các tên đầy đủ (qualified name). Các ngôn ngữ khác cũng có cơ chế tương tự, chẳng hạn `using` hay `require`.

When the I metric is 1, it means that no other component depends upon this component (Fan-in = 0); and this component depends upon other components (Fan-out > 0). Thus, it is as unstable as a component can get; it is a teenager, irresponsible and dependent. Its lack of dependents gives it no reason not to change, and the components that it depends upon may give it ample reason to change.

Khi I bằng 1, nghĩa là không component nào khác phụ thuộc vào component này (Fan-in = 0); và bản thân component này lại phụ thuộc vào những component khác (Fan-out > 0). Vậy là nó bất ổn định tới mức tối đa; nó là một cậu thiếu niên, vô trách nhiệm và lệ thuộc. Việc chẳng ai phụ thuộc vào nó khiến nó không có lý do gì để dừng thay đổi, còn những component mà nó phụ thuộc vào lại cho nó vô số lý do để phải thay đổi.

On the other hand, when the I metric is zero, it means that the component is depended upon by other components (Fan-in > 0), but it does not itself depend upon any other components (Fan-out = 0). It is a responsible and independent adult. Such a component is as stable as it can get. Its dependents make it hard to change, and it has no dependencies that might force it to change.

Ngược lại, khi I bằng 0, nghĩa là có những component khác phụ thuộc vào component này (Fan-in > 0), còn bản thân nó thì không phụ thuộc vào component nào cả (Fan-out = 0). Đó là một người trưởng thành có trách nhiệm và độc lập. Một component như vậy ổn định tới mức tối đa. Những kẻ phụ thuộc vào nó khiến nó khó thay đổi, còn nó thì chẳng có phụ thuộc nào có thể ép nó phải đổi.

The SDP says that the I metric of a component should be larger than the I metrics of the components that it depends upon.

SDP nói rằng thước đo I của một component phải lớn hơn thước đo I của những component mà nó phụ thuộc vào.

I metrics should decrease in the direction of dependency.

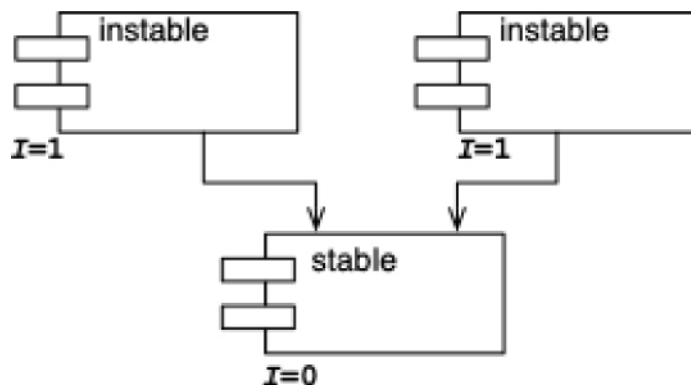
Giá trị I phải giảm dần theo hướng phụ thuộc.

Not All Components Should Be Stable

Không phải component nào cũng nên ổn định

If all the components in a system were maximally stable, the system would be unchangeable. This is not a desirable situation. Indeed, we want to design our component structure so that some components are unstable and some are stable. The following diagram shows a conformant configuration for a system with three components.

Nếu mọi component trong một hệ thống đều ổn định tối đa thì hệ thống ấy sẽ không sao thay đổi được. Đó chẳng phải tình cảnh đáng mong muốn. Thật vậy, chúng ta muốn thiết kế cấu trúc component sao cho có những component bất ổn định và có những component ổn định. Sơ đồ dưới đây cho thấy một cấu hình hợp lệ cho hệ thống gồm ba component.

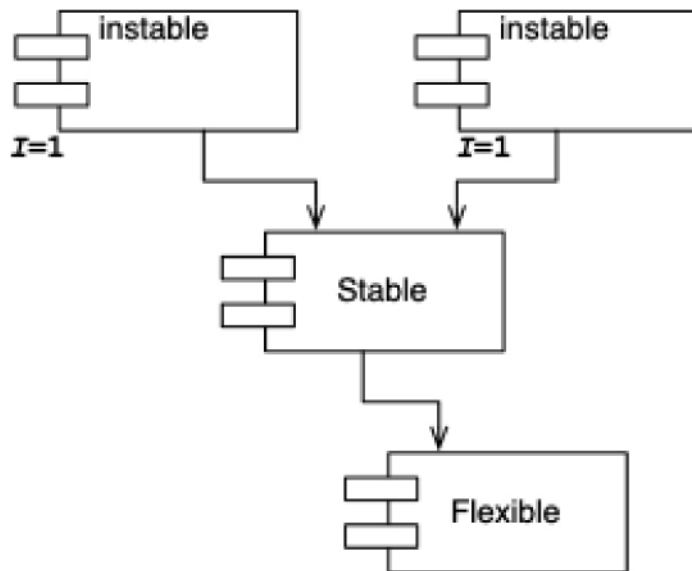


The changeable components are on top and depend upon the stable component at the bottom. Putting the unstable components at the top of the diagram is a useful convention since any arrow that points up is violating the SDP (and, as we shall see in the next chapter, the ADP).

Các component dễ thay đổi nằm ở trên và phụ thuộc vào component ổn định nằm ở dưới. Đặt các component bất ổn định lên phía trên sơ đồ là một quy ước hữu ích, vì khi đó mọi mũi tên chỉ lên trên đều là đang vi phạm SDP (và, như ta sẽ thấy ở chương sau, vi phạm cả ADP).

The next diagram shows how the SDP can be violated.

Sơ đồ tiếp theo cho thấy SDP có thể bị vi phạm như thế nào.



Flexible is a component that we have designed to be easy to change. We want Flexible to be unstable. However, some developer, working in the component named Stable , has hung a dependency upon Flexible . This violates the SDP since the I metric for Stable is much lower than the I metric for Flexible . As a result, Flexible will no longer be easy to change. A change to Flexible will force us to deal with Stable and all its dependents.

Flexible là component mà ta đã thiết kế cho dễ thay đổi. Ta muốn Flexible bất ổn định. Thế nhưng một lập trình viên nào đó, khi làm việc trong component tên là Stable , đã treo một phụ thuộc lên Flexible . Điều này vi phạm SDP, vì thước đo I của Stable thấp hơn nhiều so với I của Flexible . Hệ quả là Flexible không còn dễ thay đổi nữa. Một thay đổi trong Flexible sẽ buộc chúng ta phải xử lý cả Stable lẫn mọi thứ phụ thuộc vào nó.

To fix this, we somehow have to break the dependence of Stable upon Flexible . Why does this dependency exist? Let's assume that there is a class C within Flexible that another class U within Stable needs to use.

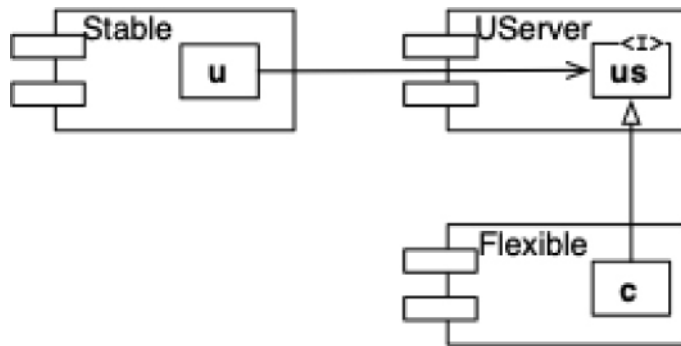
Để sửa, bằng cách nào đó ta phải cắt đứt phụ thuộc của Stable vào Flexible . Vì sao phụ thuộc này lại tồn tại? Giả sử trong Flexible có một class C mà một class U trong Stable cần dùng.



We can fix this by employing the DIP. We create an interface class called US and put it in a component named UServer . We make sure that this interface declares all the methods that U needs to use. We then make C implement this interface, as shown below. This breaks the dependency of Stable upon Flexible , and it forces both components to be dependent upon UServer . UServer is very stable (I = 0), and Flexible retains its necessary instability (I = 1). All the dependencies now flow in the direction of decreasing I.

Ta có thể sửa việc này bằng cách vận dụng DIP. Ta tạo một class interface tên là US và đặt nó vào một component tên là UServer . Ta bảo đảm interface này khai báo đủ mọi method mà U cần dùng. Sau đó ta cho C hiện thực interface ấy, như hình dưới. Cách làm này cắt đứt phụ thuộc của Stable vào Flexible , và buộc cả hai component cùng phụ thuộc vào

UServer . UServer thì rất ổn định (I = 0), còn Flexible vẫn giữ được tính bất ổn định cần thiết của nó (I = 1). Giờ đây mọi phụ thuộc đều chảy theo hướng I giảm dần.



Abstract Components

Component trừu tượng

You may find it strange that we have created a component, like UService , that contains nothing but an interface. Such a component contains no executable code! It turns out, however, that this is a very common, and necessary, tactic when using statically typed languages like Java or C#. These abstract components are very stable and are therefore ideal targets for less stable components to depend upon.

Bạn có thể thấy lạ khi chúng ta tạo ra một component như UService , bên trong chẳng chứa gì ngoài một interface. Một component như thế không hề có mã thực thi! Nhưng hoá ra đây lại là chiến thuật rất phổ biến, và cần thiết, khi dùng những ngôn ngữ định kiểu tĩnh như Java hay C#. Các component trừu tượng này rất ổn định, và vì vậy là mục tiêu lý tưởng để những component kém ổn định hơn phụ thuộc vào.

When using dynamically typed language like Ruby or Python, these abstract components don't exist at all. Nor do the dependencies that would have targeted them. Dependency structures in these languages are much simpler because dependency inversion does not require either the declaration or the **inheritance** of interfaces.

Khi dùng ngôn ngữ định kiểu động như Ruby hay Python, những component trừu tượng kiểu này hoàn toàn không tồn tại. Các phụ thuộc lẽ ra trở tới chúng cũng vậy. Cấu trúc phụ thuộc trong những ngôn ngữ đó đơn giản hơn nhiều, bởi việc đảo ngược phụ thuộc không đòi hỏi phải khai báo hay kế thừa interface.

The Stable Abstractions Principle (SAP)

SAP — Nguyên tắc Trừu tượng Ổn định (The Stable Abstractions Principle)

A component should be as abstract as it is stable. What follows is going to get a bit mathematical and theoretical. Don't let that scare you off. The theory is important, even if the practice is only loosely bound to it. Read through to the end and you'll see what I mean.

Một component nên trừu tượng đúng bằng mức nó ổn định. Phần tiếp theo sẽ hơi nặng tính toán và lý thuyết. Đừng vì thế mà chùn bước. Lý thuyết là quan trọng, dù thực hành chỉ gắn với nó một cách lỏng lẻo. Cứ đọc cho hết rồi bạn sẽ hiểu ý tôi.

Where Do We Put the High-Level Policy?

Đặt chính sách mức cao ở đâu?

Some software in the system should not change very often. This software represents high-level architecture and policy decisions. We don't want these business and architectural decisions to be volatile. Thus, the software that encapsulates the high-level policies of the system should be placed into stable components ($I = 0$). Unstable components ($I = 1$) should contain only the software that is volatile—software that we want to be able to quickly and easily change.

Một số phần mềm trong hệ thống lẽ ra không nên thay đổi thường xuyên. Chúng đại diện cho kiến trúc (architecture) mức cao và các quyết định về chính sách. Ta không muốn những quyết định nghiệp vụ và kiến trúc ấy hay biến động. Vì thế, phần mềm đóng gói các chính sách mức cao của hệ thống nên được đặt vào những component ổn định ($I = 0$). Còn các component bất ổn định ($I = 1$) chỉ nên chứa những phần mềm hay biến động — thứ mà ta muốn có thể thay đổi thật nhanh và thật dễ.

However, if the high-level policies are placed into stable components, then the source code that represents those policies will be difficult to change. This could make the overall architecture inflexible. How can a component that is maximally stable ($I = 0$) be flexible enough to withstand change? The answer is to be found in the OCP. This principle tells us that it is possible and desirable to create classes that are flexible enough to be extended without requiring modification. What kinds of classes conform to this principle? Abstract classes.

Tuy nhiên, nếu đặt các chính sách mức cao vào những component ổn định thì mã nguồn biểu diễn các chính sách ấy sẽ khó thay đổi. Điều này có thể khiến toàn bộ kiến trúc trở nên cứng nhắc. Làm sao một component ổn định tối đa ($I = 0$) lại có thể đủ linh hoạt để chịu được thay đổi? Câu trả lời nằm ở OCP. Nguyên tắc này cho ta biết rằng hoàn toàn có thể — và nên — tạo ra những class đủ linh hoạt để mở rộng mà không cần sửa đổi. Loại class nào tuân thủ nguyên tắc ấy? Class trừu tượng (abstract class).

The SAP sets up a relationship between stability and abstractness. It says that a stable component should also be abstract so that its stability does not prevent it from being extended. On the other hand, it says that an unstable component should be concrete since its instability allows the concrete code within it to be easily changed.

SAP thiết lập một quan hệ giữa tính ổn định và độ trừu tượng (abstractness). Nó nói rằng một component ổn định thì cũng nên trừu tượng, để tính ổn định của nó không ngăn cản việc mở rộng. Mặt khác, nó nói rằng một component bất ổn định thì nên cụ thể, bởi chính tính bất ổn định cho phép mã cụ thể bên trong nó dễ dàng được thay đổi.

Thus, if a component is to be stable, it should also consist of interfaces and abstract classes so that it can be extended. Stable components that are extensible are flexible and do not overly constrain the architecture.

Vậy nên, nếu một component muốn ổn định thì nó cũng nên gồm các interface và class trừu tượng để có thể mở rộng được. Những component ổn định mà mở rộng được thì linh hoạt và không trói buộc kiến trúc quá mức.

The combination of the SAP and the SDP amounts to the DIP for components. This is true because the SDP says that dependencies should run in the direction of stability, and the SAP says that stability implies abstraction. Thus, dependencies run in the direction of abstraction.

Kết hợp SAP với SDP thì chẳng khác nào DIP ở cấp component. Đúng như vậy, bởi SDP nói rằng các phụ thuộc phải chạy theo hướng ổn định, còn SAP nói rằng ổn định kéo theo trừu tượng. Vậy nên các phụ thuộc chạy theo hướng trừu tượng.

Measuring Abstraction The A metric is a measure of the abstractness of a component. Its value is simply the ratio of interfaces and abstract classes in a component to the total number of classes in

the component.

Đo độ trừu tượng Thuộc đo A đo mức trừu tượng của một component. Giá trị của nó đơn giản là tỉ lệ giữa số interface và class trừu tượng trong component với tổng số class trong component đó.

- N_c : The number of classes in the component.
- N_a : The number of abstract classes and interfaces in the component.
- A: Abstractness.
 - N_c : Số class trong component.
 - N_a : Số class trừu tượng và interface trong component.
 - A: Độ trừu tượng.

$A = N_a \div N_c$. The A metric ranges from 0 to 1. Zero implies that the component has no abstract classes at all. A value of 1 implies that the component contains nothing but abstract classes and/or interfaces.

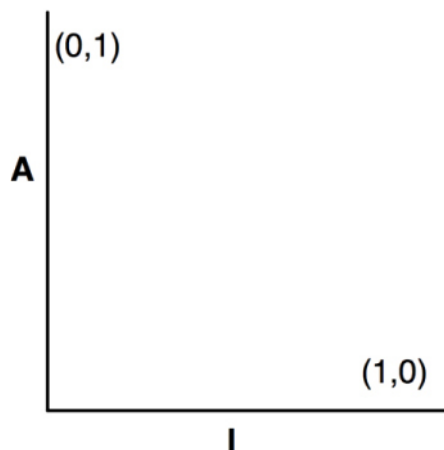
$A = N_a \div N_c$. Thuộc đo A nằm trong khoảng từ 0 đến 1. Giá trị 0 hàm ý component chẳng có class trừu tượng nào cả. Giá trị 1 hàm ý component không chứa gì ngoài class trừu tượng và/hoặc interface.

The Main Sequence

Dãy chính

We are now in a position to define the relationship between stability (I) and abstractness (A). We can create a graph with A on the **vertical** axis and I on the horizontal axis. If we plot the two “good” kinds of components on this graph, we will find the components that are maximally stable and abstract (the adults) at the upper left at (0,1). The components that are maximally unstable and concrete (the teenagers) are at the lower right at (1,0).

Giờ ta đã đủ điều kiện để định nghĩa quan hệ giữa tính ổn định (I) và độ trừu tượng (A). Ta dựng một đồ thị với A trên trục tung và I trên trục hoành. Nếu vẽ hai loại component “tốt” lên đồ thị này, ta sẽ thấy những component ổn định và trừu tượng tối đa (những người trưởng thành) nằm ở góc trên bên trái, tại (0,1). Còn những component bất ổn định và cụ thể tối đa (những cậu thiếu niên) nằm ở góc dưới bên phải, tại (1,0).

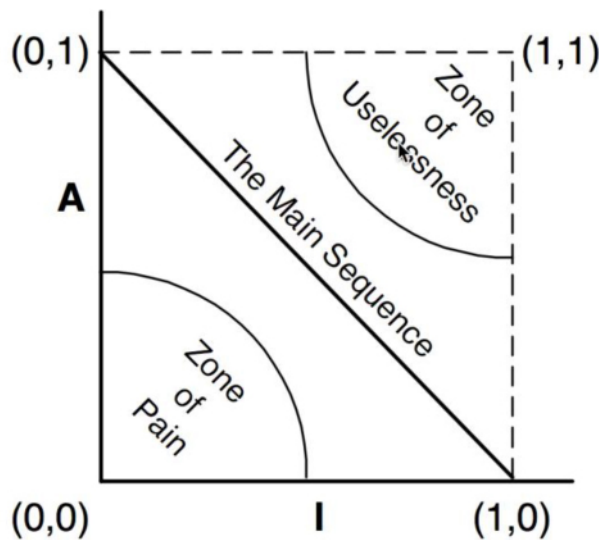


Not all components can fall into one of these two positions. Components have degrees of abstraction and stability. For example, it is very common for one abstract class to derive from another abstract class. The derivative is an abstraction that has a dependency. Thus, though it is maximally abstract, it will not be maximally stable. Its dependency will decrease its stability.

Không phải component nào cũng rơi vào một trong hai vị trí đó. Component có những mức trừu tượng và ổn định khác nhau. Chẳng hạn, chuyển một class trừu tượng dẫn xuất từ một class trừu tượng khác là hết sức bình thường. Lớp dẫn xuất là một trừu tượng nhưng lại có phụ thuộc. Vì thế, dù trừu tượng tối đa, nó sẽ không ổn định tối đa. Phụ thuộc của nó sẽ làm giảm tính ổn định.

Since we cannot enforce that all components sit at either (0,1) or (1,0), we must assume that there is a locus of points on the A/I graph that defines reasonable positions for components. We can infer what that locus is by finding the areas where components should not be—that is, zones of exclusion.

Vì không thể ép mọi component phải nằm ở (0,1) hoặc (1,0), ta buộc phải giả định rằng tồn tại một quỹ tích các điểm trên đồ thị A/I xác định những vị trí hợp lý cho component. Có thể suy ra quỹ tích đó bằng cách tìm những vùng mà component không nên nằm vào — tức là các vùng loại trừ (zone of exclusion).



The Zone of Pain

Vùng Đau khổ (The Zone of Pain)

Consider a component in the area of (0,0). This is a highly stable and concrete component. Such a component is not desirable, because it is rigid. It cannot be extended, because it is not abstract. And it is very difficult to change because of its stability. Thus, we do not normally expect to see well-designed components sitting near (0,0). The area around (0,0) is a zone of exclusion called the Zone of Pain.

Hãy xét một component nằm ở vùng (0,0). Đây là một component vừa rất ổn định vừa rất cụ thể. Một component như vậy chẳng đáng mong muốn chút nào, vì nó cứng nhắc. Nó không thể mở rộng, bởi nó không trừu tượng. Và nó rất khó thay đổi, bởi nó quá ổn định. Vì thế, thông thường ta không trông đợi thấy những component được thiết kế tốt nằm gần (0,0). Vùng quanh (0,0) là một vùng loại trừ, gọi là Vùng Đau khổ.

It should be noted that certain software entities do, in fact, fall within the Zone of Pain. An example would be a database schema. Database schemas are notoriously volatile, extremely concrete, and highly depended upon. This is one of the reasons that the interface between OO applications and databases is so difficult, and that schema updates are generally painful.

Cũng cần lưu ý rằng trên thực tế có những thực thể phần mềm quả thật rơi vào Vùng Đau khổ. Một ví dụ là schema cơ sở dữ liệu. Schema cơ sở dữ liệu nổi tiếng là hay biến động,

cực kỳ cụ thể, và bị vô số thứ phụ thuộc vào. Đây là một trong những lý do khiến việc kết nối giữa ứng dụng OO với cơ sở dữ liệu lại khó đến thế, và khiến việc cập nhật schema nói chung rất đau đớn.

Another example of software that sits on (0,0) is a concrete utility library. Although such a library has an I metric of 1, it may in fact be nonvolatile. Consider the String component, for example. Even though all the classes within it are concrete, it is so commonly used that changing it would create chaos. Therefore, String is nonvolatile.

Một ví dụ khác về phần mềm nằm ở (0,0) là thư viện tiện ích cụ thể. Dù thư viện kiểu này có thước đo I bằng 1, trên thực tế nó có thể chẳng biến động chút nào. Chẳng hạn hãy xét component String. Dù mọi class bên trong nó đều là class cụ thể, nó được dùng phổ biến tới mức thay đổi nó sẽ gây ra hỗn loạn. Bởi vậy, String không hề biến động.

Nonvolatile components are harmless in the (0,0) zone since they are not likely to be changed. For that reason, it is only software components that are volatile that are also problematic in the Zone of Pain. The more volatile a component in the Zone of Pain is, the more “painful” it is. Indeed, we might consider volatility to be a third axis of the graph; and then the graph above shows only the most painful plane, where volatility = 1.

Những component không biến động thì vô hại khi nằm ở vùng (0,0), bởi chúng khó lòng bị thay đổi. Vì lẽ đó, chỉ những component phần mềm hay biến động mới thực sự là vấn đề khi rơi vào Vùng Đau khổ. Một component trong Vùng Đau khổ càng hay biến động thì càng “đau”. Thật vậy, ta có thể coi tính biến động là trục thứ ba của đồ thị; khi đó đồ thị bên trên chỉ thể hiện mặt phẳng đau đớn nhất, nơi volatility = 1.

The Zone of Uselessness

Vùng Vô dụng (The Zone of Uselessness)

Consider a component near (1,1). This location is undesirable because it is maximally abstract and yet has no dependents. This is like an interface that has no implementation. Such components are useless. Thus, this is called the Zone of Uselessness.

Hãy xét một component nằm gần (1,1). Vị trí này chẳng đáng mong muốn, vì component ấy trừu tượng tối đa mà lại không có ai phụ thuộc vào. Nó giống như một interface không có hiện thực nào. Những component như thế là vô dụng. Bởi vậy vùng này được gọi là Vùng Vô dụng.

The software entities that inhabit this region are a kind of detritus. They are often leftover abstract classes that no one ever implemented. We find them in systems from time to time, sitting in the codebase, unused.

Các thực thể phần mềm cư ngụ ở vùng này là một dạng rác thừa. Chúng thường là những class trừu tượng còn sót lại mà chẳng ai từng hiện thực. Thỉnh thoảng ta lại bắt gặp chúng trong hệ thống, nằm chình ình trong codebase, không ai dùng đến.

Or they may be superfluous interfaces—interfaces that a few classes implemented but no class ever used.

Hoặc chúng có thể là những interface dư thừa — interface mà vài class có hiện thực nhưng chẳng class nào từng dùng tới.

A component that has a position deep within the Zone of Uselessness must contain a significant fraction of such entities. The presence of such useless entities is undesirable.

Một component nằm sâu trong Vùng Vô dụng ắt phải chứa một tỉ lệ đáng kể những thực thể kiểu đó. Sự hiện diện của các thực thể vô dụng như vậy là điều không mong muốn.

The Main Sequence

Dãy chính

We'd like our most volatile components to be as far from both zones of exclusion as possible. The locus of points that is maximally distant from each zone is the line that connects (1,0) and (0,1). I call this line the main sequence. 8

Chúng ta muốn những component hay biến động nhất nằm càng xa cả hai vùng loại trừ càng tốt. Quỹ tích các điểm cách xa mỗi vùng nhất chính là đường thẳng nối (1,0) với (0,1). Tôi gọi đường này là dãy chính (main sequence). 8

8 The author begs the reader's indulgence for the arrogance of borrowing such an important term from astronomy. A component that sits on the main sequence is not "too abstract" for its stability, nor is it "too unstable" for its abstractness. It is neither useless nor particularly painful. It is depended upon to the extent that it is abstract, and it depends upon others to the extent that it is concrete.

8 Tác giả xin bạn đọc lượng thứ cho sự ngạo mạn khi mượn một thuật ngữ quan trọng đến thế từ thiên văn học. Một component nằm trên dãy chính thì không "quá trừu tượng" so với mức ổn định của nó, cũng chẳng "quá bất ổn định" so với mức trừu tượng của nó. Nó chẳng vô dụng mà cũng chẳng gây đau đớn gì đặc biệt. Nó được kẻ khác phụ thuộc vào đúng bằng mức nó trừu tượng, và nó phụ thuộc vào kẻ khác đúng bằng mức nó cụ thể.

The most desirable positions for a component are at one of the two endpoints of the main sequence. We strive to position the majority of components at those endpoints. However, in my experience, some small fraction of the components in a large system are neither perfectly abstract nor perfectly stable. Those components have the best characteristics if they are on or close to the main sequence.

Vị trí đáng mong muốn nhất của một component là một trong hai đầu mút của dãy chính. Chúng ta cố gắng đặt phần lớn component vào hai đầu mút đó. Tuy nhiên, theo kinh nghiệm của tôi, trong một hệ thống lớn luôn có một phần nhỏ component không hoàn toàn trừu tượng mà cũng không hoàn toàn ổn định. Những component ấy sẽ có đặc tính tốt nhất khi nằm trên hoặc gần dãy chính.

Distance from the Main Sequence

Khoảng cách tới dãy chính

This leads us to our last metric. If it is desirable for components to be on or close to the main sequence, we can create a metric that measures how far away a component is from this ideal.

Điều này dẫn ta tới thước đo cuối cùng. Nếu component nên nằm trên hoặc gần dãy chính, ta có thể tạo ra một thước đo cho biết một component cách cái lý tưởng ấy bao xa.

- D 9 : Distance. $D = |A+I-1|$. This metric ranges from [0,1]. Zero indicates that the component is directly on the main sequence. One indicates that the component is as far away as possible from the main sequence.
- D 9 : Khoảng cách. $D = |A+I-1|$. Thước đo này nằm trong khoảng [0,1]. Giá trị 0 hàm ý component nằm ngay trên dãy chính. Giá trị 1 hàm ý component cách dãy chính xa nhất có thể.

9 In previous publications, I called this metric D'. I see no reason to continue that practice. Given this metric, a design can be analyzed for its overall conformance to the main sequence. The D metric for each component can be calculated. Any component that has a D value that is not near zero can be reexamined and restructured.

9 Trong các ấn phẩm trước, tôi gọi thước đo này là D' . Tôi thấy chẳng có lý do gì để tiếp tục như thế nữa. Có thước đo này rồi, ta có thể phân tích xem một thiết kế nhìn chung bám sát dãy chính đến đâu. Có thể tính D cho từng component. Component nào có D không gần 0 thì nên được xem xét lại và tái cấu trúc.

Conclusion

Kết luận

If your head is reeling at this point, I suggest you breathe. I am not suggesting that you should measure and calculate all these metrics. It's good to know that $F = ma$, but you don't often do that math when you drive your car. Theory is one thing. Practice is another.

Nếu đến đây đầu bạn đã quay mòng mòng thì tôi khuyên bạn hãy hít một hơi. Tôi không hề bảo rằng bạn phải đo đạc và tính toán tất cả những thước đo này. Biết $F = ma$ thì tốt, nhưng bạn đâu có thường xuyên làm phép tính ấy lúc lái xe. Lý thuyết là một chuyện. Thực hành lại là chuyện khác.

I certainly don't make a practice of gathering the necessary statistics, doing all the math, and generating plots. In most projects, I can see these relationships in my head and do a simple mental evaluation—and I don't mean that I'm doing the math in my head either. Rather, I simply envision the relationships and infer how stable and how abstract each component is. If you wish, consider it a “feeling.”

Bản thân tôi chắc chắn không có thói quen đi thu thập số liệu, làm hết mấy phép tính rồi vẽ đồ thị. Ở hầu hết dự án, tôi có thể hình dung những quan hệ này trong đầu và làm một phép đánh giá đơn giản trong óc — mà tôi cũng chẳng có ý bảo rằng mình nhầm toán trong đầu đâu nhé. Đúng hơn là tôi chỉ mường tượng ra các quan hệ ấy rồi suy ra mỗi component ổn định và trừu tượng đến mức nào. Nếu muốn, bạn cứ coi đó là “cảm giác”.

It's good to understand the above theory. There are cases where it may even be beneficial to do the above calculations. There have been a few extreme cases where I certainly have. But in most cases, understanding the theory and using it to “feel” the relationships is good enough.

Hiểu được lý thuyết bên trên là điều tốt. Có những trường hợp thực sự làm các phép tính kia còn mang lại lợi ích. Đã có vài tình huống cực đoan mà tôi chắc chắn đã làm như vậy. Nhưng trong phần lớn trường hợp, hiểu lý thuyết và dùng nó để “cảm” được các quan hệ là đã đủ tốt.

You may find that your IDE does many of these calculations for you. I've found it helpful, in larger projects, to look over the numbers that the IDE has calculated. But my “feel” is pretty good, and I am seldom surprised.

Bạn có thể nhận ra IDE của mình làm giúp bạn khá nhiều phép tính này. Ở những dự án lớn, tôi thấy việc ngó qua các con số mà IDE đã tính là có ích. Nhưng “cảm giác” của tôi cũng khá chuẩn, và tôi hiếm khi bị bất ngờ.

Chapter 21. Continuous Design —

Chương 21. Thiết kế liên tục



By Jeff Langr

Tác giả: Jeff Langr

Product owners (and others responsible for helping to define a software product) determine the set of end goals that a system must accomplish for its users. Developers, in turn, build support for each of these goals in the system, in the form of code and configuration. As Jack W. Reeves taught us over three decades ago, this code (and configuration 1) is the ultimate and definitive representation of “the design.” 2

Product owner (và những người khác có trách nhiệm giúp định hình một sản phẩm phần mềm) xác định tập các mục tiêu cuối cùng mà hệ thống phải đạt được cho người dùng của nó. Đến lượt mình, lập trình viên xây dựng phần hỗ trợ cho từng mục tiêu đó trong hệ thống, dưới dạng mã và cấu hình. Như Jack W. Reeves đã dạy chúng ta hơn ba thập kỷ trước, chính mã này (và cấu hình 1) mới là biểu diễn tối hậu và dứt khoát của “thiết kế”. 2

1 For example, [PPP02], <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod> , and [https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)) (or just google SOLID). 2 <https://www.developerdotst.com> . Republished in [PPP02].

1 Ví dụ, [PPP02], <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod> , và

[https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)) (hoặc cứ google chữ SOLID).
2 https://www.developerdotstar.com/mag/articles/reeves_design.html . Đăng lại trong [PPP02].

The word design classically has two primary meanings:

Theo nghĩa cổ điển, từ *design* có hai nghĩa chính:

- The plan for building something
- The form and functionality exuded by the current state of that something
 - Bản kế hoạch để xây dựng một thứ gì đó
 - Hình hài và chức năng toát ra từ trạng thái hiện tại của chính thứ đó

Continuous design, also abbreviated as . . . Wait, that abbreviation is already taken. We could just utter the full phrase “continuous design” throughout this chapter. Ultimately, we’ll just call it “design,” because that’s how we best build software systems: one conscious design decision after another.

Thiết kế liên tục (continuous design), còn được viết tắt là... Khoan đã, cái viết tắt đó có người xí mất rồi. Vậy thì cả chương này ta cứ nói đủ cụm “thiết kế liên tục” vậy. Mà rốt cuộc, ta sẽ chỉ gọi nó là “thiết kế” thôi, bởi đó chính là cách tốt nhất để xây dựng hệ thống phần mềm: từng quyết định thiết kế có ý thức, nối tiếp nhau.

As we were saying: Continuous design means that we continually plan how to accommodate new behaviors and that our system has a new design each time we add a new **behavior**. Most of that planning occurs as we begin writing the code that will provide the new behaviors.

Như đã nói: thiết kế liên tục nghĩa là chúng ta liên tục lên kế hoạch để dung nạp các hành vi mới, và mỗi lần thêm một hành vi mới là hệ thống lại có một thiết kế mới. Phần lớn việc lên kế hoạch đó diễn ra ngay khi ta bắt đầu viết đoạn mã sẽ cung cấp các hành vi mới ấy.

Adding new code or updating existing code always changes the design, often only in a very small way.

Thêm mã mới hay sửa mã cũ thì luôn làm thay đổi thiết kế, dù thường chỉ ở mức rất nhỏ.

Design is not a nebulous phase of development that occurs at the outset of a project. It is not solely the activity of drawing high-level sketches about what the system needs to look like.

Thiết kế không phải một pha mờ mịt nào đó của quá trình phát triển, xảy ra ở đầu dự án. Nó cũng không đơn thuần là hoạt động vẽ mấy bản phác thảo mức cao về hình dung hệ thống sẽ trông ra sao.

Design is a continuous activity. Our system’s design should command our constant attention, because it changes continuously, by definition. Sounds daunting, but in reality, it’s easier than the alternative of generating an initial set of design models, then pretending we can ignore design after that.

Thiết kế là một hoạt động liên tục. Thiết kế của hệ thống phải được ta để mắt tới thường trực, bởi tự bản chất, nó thay đổi không ngừng. Nghe thì nản, nhưng thực tế lại dễ hơn nhiều so với lựa chọn kia: vẽ ra một mô hình thiết kế ban đầu, rồi giả vờ rằng sau đó có thể quên chuyện thiết kế đi.

Continuous Change — Thay đổi liên tục

While software is made up of code, it’s more useful to say that software is made of thousands of small behaviors (sometimes called units), each implemented in the form of code. Our job as programmers

is to create and orchestrate these behaviors in a way that supports meeting the end goals of a system's users.

Dù phần mềm được tạo nên từ mã, sẽ hữu ích hơn nếu nói rằng phần mềm được tạo nên từ hàng ngàn hành vi nhỏ (đôi khi gọi là các *unit*), mỗi hành vi được hiện thực dưới dạng mã. Công việc của lập trình viên chúng ta là tạo ra và phối hợp các hành vi đó sao cho đáp ứng được những mục tiêu cuối cùng của người dùng hệ thống.

We must first translate end goals into a number of smaller software behaviors. Given the goal “present a random list of flash cards for learning Czech nouns,” we might implement the following sequence of behaviors.

Trước hết ta phải chuyển các mục tiêu cuối cùng thành một loạt hành vi phần mềm nhỏ hơn. Với mục tiêu “trình bày một danh sách ngẫu nhiên các thẻ flash card để học danh từ tiếng Séc”, ta có thể hiện thực chuỗi hành vi sau.

1. Retrieve a random subset of all nouns.
2. For each noun:
 1. Lấy ra một tập con ngẫu nhiên từ toàn bộ danh từ.
 2. Với mỗi danh từ:
 - a. Present a question: A fill-in-the-blank sentence along with a multiple-choice listing of the four possible noun forms.
 - a. Trình bày một câu hỏi: một câu điền vào chỗ trống kèm danh sách trắc nghiệm gồm bốn dạng có thể có của danh từ đó.
 - b. Capture the user's choice.
 - b. Ghi nhận lựa chọn của người dùng.
 - c. Show the user the correct answer if their choice is incorrect.
 - c. Cho người dùng thấy đáp án đúng nếu họ chọn sai.
3. Repeat steps 2a through 2c for each question that the user chose incorrectly.
 3. Lặp lại các bước 2a đến 2c cho từng câu hỏi mà người dùng đã trả lời sai.

There are usually many ways to skin a cat, so we might instead support an alternate set of steps. For example, the user might be able to continue selecting an answer for a single question until they get it correct. We might present hints with each failed answer. We might replace multiple-choice questions with fill-in-the blank questions as the user improves in skill.

Thường thì có trăm đường để đến cùng một đích, nên ta cũng có thể chọn một tập bước khác hẳn. Chẳng hạn, người dùng có thể được phép chọn đi chọn lại đáp án cho một câu hỏi cho tới khi đúng. Ta có thể đưa gợi ý sau mỗi lần trả lời sai. Ta có thể thay câu hỏi trắc nghiệm bằng câu hỏi điền vào chỗ trống khi trình độ người dùng khá lên.

If there are many design choices for an application itself, there are orders of magnitude more choices for how we write the underlying code. No, that's not right. We have infinite design choices. Most of those choices will be suboptimal.

Nếu bản thân một ứng dụng đã có nhiều lựa chọn thiết kế đến thế, thì số lựa chọn về cách viết đoạn mã bên dưới còn nhiều hơn cả mấy bậc độ lớn. Không, nói vậy vẫn chưa đúng. Ta có vô hạn lựa chọn thiết kế. Và phần lớn trong số đó là những lựa chọn dưới mức tối ưu.

Our product managers and owners shape our software products to meet the perceived needs of the marketplace. Most software products continue to demand constant change in the form of new needs, and enhancements to existing needs.

Product manager và product owner nhào nặn sản phẩm phần mềm của ta để đáp ứng những nhu cầu mà họ nhìn thấy trên thị trường. Đa số sản phẩm phần mềm luôn đòi hỏi thay đổi thường xuyên, dưới dạng nhu cầu mới hoặc cải tiến cho nhu cầu sẵn có.

In turn, we must constantly be able to meet the demand by shaping and enhancing existing software. A software system is a unique kind of product: It must not only continuously present a design that supports all current needs to date, but also accept that this design will need to be different tomorrow.

Đến lượt mình, ta phải luôn có khả năng đáp ứng đòi hỏi đó bằng cách nhào nặn và bồi đắp phần mềm hiện có. Hệ thống phần mềm là một loại sản phẩm rất đặc biệt: nó không những phải liên tục trình ra một thiết kế đáp ứng mọi nhu cầu tính đến hôm nay, mà còn phải chấp nhận rằng ngày mai thiết kế ấy sẽ phải khác đi.

We might be able to build a slick, useful flash card system in a short period of time, perhaps a month or two. After an initial release, users will demand new behaviors. We're selling new features—great! But we must also continuously return to the current system's inherent design—its codebase. In order to add new features, we're forced to figure out how to accommodate new concepts into the existing design, things that might have never been considered before.

Ta có thể xây được một hệ thống flash card bóng bẩy và hữu dụng trong thời gian ngắn, chừng một hai tháng. Sau bản phát hành đầu tiên, người dùng sẽ đòi hỏi những hành vi mới. Ta bán được tính năng mới — tuyệt vời! Nhưng ta cũng phải liên tục quay lại với thiết kế nội tại của hệ thống hiện hành — tức là codebase của nó. Để thêm tính năng mới, ta buộc phải nghĩ cách nhét những khái niệm mới vào thiết kế sẵn có, những thứ mà có lẽ trước đây chưa ai từng tính tới.

Continuous Design — Thiết kế liên tục

When it comes to making choices in software, we have only a few kinds of primary building blocks: modules (classes), functions (methods), **data** references (variables), and statements/expressions. The first three of these constructs can be named, to give us humans a better chance of understanding what concepts exist in the codebase, and also a better chance of finding the code we need to change or extend.

Khi phải đưa ra lựa chọn trong phần mềm, ta chỉ có vài loại viên gạch cơ bản: module (class), hàm (method), tham chiếu dữ liệu (biến), và câu lệnh/biểu thức. Ba loại đầu tiên có thể được đặt tên, cho loài người chúng ta thêm cơ hội hiểu được trong codebase đang tồn tại những khái niệm gì, và cũng thêm cơ hội tìm ra đúng đoạn mã cần sửa hay cần mở rộng.

How we choose to manage and organize these building blocks is the activity of software design. Our system's design is the collection of choices we've made to this point in time.

Cách ta chọn để quản lý và tổ chức những viên gạch ấy chính là hoạt động thiết kế phần mềm. Thiết kế của hệ thống là tập hợp mọi lựa chọn mà ta đã đưa ra cho tới thời điểm này.

We can easily make many bad choices when assembling these constructs, and in a very short time too. After making changes to our flash card app, it might still work, but our changes might have unnecessarily increased the cost for any future modifications.

Khi ráp các cấu trúc đó lại, ta rất dễ đưa ra hàng loạt lựa chọn tồi, mà lại tồi rất nhanh. Sau khi sửa ứng dụng flash card, nó vẫn có thể chạy tốt, nhưng những thay đổi của ta có thể đã làm tăng chi phí một cách không cần thiết cho mọi lần chỉnh sửa về sau.

If changing our flash card app results in unorganized, convoluted, and unclear code, accommodating new needs will be slow. We'll expend too much time navigating code to find all the places it must change. Once we find those locations (usually plural), we'll expend too much time understanding what the current code does. We'll then expend too much time carefully inserting our changes and ensuring that all the existing behaviors remain unbroken.

Nếu việc sửa ứng dụng flash card để lại một đống mã lộn xộn, rối rắm và tối nghĩa, thì việc dung nạp nhu cầu mới sẽ chậm chạp. Ta sẽ mất quá nhiều thời gian lần mò trong mã để tìm hết những chỗ cần đổi. Tìm được rồi (thường là nhiều chỗ chứ chẳng phải một), ta lại mất quá nhiều thời gian để hiểu mã hiện tại đang làm gì. Rồi ta lại mất quá nhiều thời gian để cẩn thận chèn thay đổi vào và bảo đảm mọi hành vi sẵn có không bị vỡ.

Each design choice we make impacts the cost for future choices. We succeed only if we minimize the negative design impact of each choice. This is the critical concept known as continuous design .

Mỗi lựa chọn thiết kế đều tác động lên chi phí của những lựa chọn sau này. Ta chỉ thành công nếu giảm thiểu được tác động tiêu cực về thiết kế của từng lựa chọn. Đây chính là khái niệm cốt tử mang tên *thiết kế liên tục*.

Sailing on the Four Cs of Continuous Design — Dong buồm trên bốn chữ C của thiết kế liên tục

An Unclean Code system makes it harder to

Một hệ thống không-Clean-Code sẽ khiến ta khó

- Find code you're looking for.
 - Understand where you need to make changes.
 - Make changes without creating defects.
 - Write tests to verify its behaviors.
- Tìm ra đoạn mã mình đang cần.
 - Hiểu được mình phải sửa ở đâu.
 - Sửa mà không để ra lỗi (defect).
 - Viết test để kiểm chứng các hành vi của nó.

You might recognize these characteristics from Chapter 13 , “ Clean Classes .”

Có lẽ bạn đã nhận ra những đặc điểm này từ Chương 13, “Class sạch”.

We have many design perspectives to help guide us toward a **Clean Code** system. As we change our system, we continue to reflect on one or more of these perspectives— SOLID, code smells, **simple design**, design patterns, and others, including various one-off principles such as **the Law of Demeter** or Tell-Don't-Ask.

Chúng ta có rất nhiều góc nhìn thiết kế để dẫn đường tới một hệ thống Clean Code. Khi thay đổi hệ thống, ta liên tục soi lại một hoặc vài trong số các góc nhìn ấy — SOLID, code smell (mùi mã), thiết kế đơn giản (simple design), mẫu thiết kế (design pattern), và những thứ khác nữa, kể cả các nguyên tắc lẻ như Luật Demeter (the Law of Demeter) hay Tell-Don't-Ask.

For the concept of continuous design, we'll try—as many have before—to codify it all in a simple, hopefully memorable set of rules. Consider this YADP: Yet Another Design Perspective.

Với khái niệm thiết kế liên tục, ta sẽ thử — như bao người đi trước — gói tất cả vào một bộ quy tắc đơn giản, và hy vọng là dễ nhớ. Cứ coi đây là YADP: Yet Another Design Perspective (Lại thêm một góc nhìn thiết kế nữa).

Our four Cs design considerations are

Bốn chữ C trong danh sách cân nhắc thiết kế của chúng ta là

- **Clarity:** The programmer's intents in the system are all clearly stated.
- **Conciseness:** The programmer's intents in the system are implemented in a minimal amount of code.
- **Confirmability:** All unit behaviors in the system can be easily tested, in a way that also provides “living documentation” of the behavioral choices that were made.
- **Cohesion:** Each **module** in the system has a high level of cohesion—all its elements maximally relate to one another.
 - Clarity (rõ ràng): mọi ý định của lập trình viên trong hệ thống đều được phát biểu rành mạch.
 - Conciseness (súc tích): mọi ý định của lập trình viên trong hệ thống được hiện thực bằng lượng mã tối thiểu.
 - Confirmability (kiểm chứng được): mọi hành vi đơn vị trong hệ thống đều dễ dàng test được, theo cách đồng thời cung cấp “tài liệu sống” về những lựa chọn hành vi đã được đưa ra.
 - Cohesion (độ gắn kết nội tại): mỗi module trong hệ thống đều có độ gắn kết nội tại (cohesion) cao — mọi phần tử của nó liên quan tới nhau ở mức tối đa.

These considerations don't live in isolation. We'll talk about them one by one, but as we discuss, think about how they can both support and oppose each other.

Những điều cần cân nhắc này không tồn tại tách biệt. Ta sẽ bàn từng cái một, nhưng vừa bàn vừa nghĩ xem chúng có thể vừa nâng đỡ nhau lại vừa chống lại nhau ra sao.

Clarity — Clarity — Sự rõ ràng

Getting code working is always our first step. We're challenged to add a small, new piece of behavior to the code. We think about how we want to solve the problem for a small amount of time, then quickly splash some code into our editor. We whittle the code a bit until we finally manage to get it working, then move on to the next slice of logic needed.

Làm cho mã chạy được luôn là bước đầu tiên. Ta được giao thêm một mẫu hành vi nhỏ, mới toanh vào mã. Ta nghĩ về cách giải quyết vấn đề trong một khoảnh khắc ngắn, rồi mau chóng đổ ào một mớ mã vào trình soạn thảo. Ta gọt tĩa đôi chút cho tới khi rốt cuộc nó chạy được, rồi chuyển sang lát logic kế tiếp.

If we never had to read that code again, we might consider ourselves “done.” Code complete! It ain't broke, and so we ain't gonna try 'n fix what's already working. Why risk breaking things?

Nếu không bao giờ phải đọc lại đoạn mã ấy, hẳn ta đã có thể tự nhận là “xong”. Code complete! Nó có hỏng đâu, nên ta chẳng việc gì phải đi sửa cái đang chạy ngon lành. Rước lấy rủi ro làm hỏng để làm gì?

But we will have to read that code again, when we're later required to change nuances of the logic! Or when we're asked "What does that code do in this case?" Or when we're required to add new related behaviors. Or in a half hour when we have to return to the code because it's not quite done yet.

Nhưng ta sẽ phải đọc lại đoạn mã đó, khi sau này phải chỉnh sửa những chi tiết tinh vi của logic! Hoặc khi bị hỏi "trong trường hợp này thì đoạn mã đó làm gì?". Hoặc khi phải thêm những hành vi mới có liên quan. Hoặc nửa tiếng nữa thôi, khi ta phải quay lại vì nó vẫn chưa xong hẳn.

For each of these needs, we want the code to be as clearly stated as possible. Anything unclear wastes time.

Với mọi nhu cầu đó, ta đều muốn mã được phát biểu rõ ràng hết mức có thể. Bất cứ chỗ nào tối nghĩa đều là thời gian bị phí phạm.

A challenge: Decipher all the nuances of the retrieveWords **function**. It might take you a half minute or two.

Một thử thách nhé: hãy giải mã cho hết mọi sắc thái của hàm retrieveWords . Có khi bạn mất tới nửa phút, thậm chí một hai phút.

```
export const retrieveWords = async words => {
  const wordPrefix = words.length === 1 ? 'word' : 'words'
  let wordList = "";
  for (let i = 0; i < words.length; i++) {
    const currentWord = words[i];
    const quotedWord = "'" + currentWord + "'";
    wordList += quotedWord;
    if (i !== words.length - 1) {
      wordList += ",";
    }
  }
  const finalPrompt = `Given a list of nouns separated by commas,
    `provide appropriate Czech language information for the ` +
    `${wordPrefix} ${wordList}`
  const response = await sendPrompt(`${format} ${finalPrompt}`)
  const startIndex = response.indexOf('[')
  const endIndex = response.lastIndexOf(']') + 1
  const jsonText = response.slice(startIndex, endIndex)
  return JSON.parse(jsonText)
}
```

This is not necessarily **bad code**, but it could easily be better. It took more time for you to decipher than necessary, so we'll call it unclean code.

Đây chưa hẳn là mã tệ, nhưng rõ ràng nó có thể tốt hơn nhiều một cách dễ dàng. Bạn đã tốn nhiều thời gian giải mã nó hơn mức cần thiết, nên ta sẽ gọi đây là mã không sạch.

A continual focus on code clarity tells us to produce code that we can all consume quickly. In the following refactored code, retrieveWords appears as the bottommost function. Start your reading there.

Việc chú tâm liên tục vào sự rõ ràng của mã bảo ta hãy tạo ra thứ mã mà ai cũng tiêu hoá được thật nhanh. Trong đoạn mã đã refactor dưới đây, retrieveWords nằm ở vị trí hàm

dưới cùng. Hãy bắt đầu đọc từ đó.

```
const sliceJSONArrayFrom = response => {
  const startIndex = response.indexOf('[')
  const endIndex = response.lastIndexOf(']') + 1

  return response.slice(startIndex, endIndex)
}
const joinQuoted = words =>
  words.map(word => `"${word}"`).join(',')
const pluralizeIfMany = (word, list) =>
  list.length === 1 ? word : `${word}s`
export const retrieveWords = async words => {
  const finalPrompt = `Given a list of nouns separated by commas,
    provide appropriate Czech language information for the ` +
    `${(pluralizeIfMany('word', words))} ${(joinQuoted(words))}`
  const response = await sendPrompt(`${format} ${finalPrompt}`)
  return JSON.parse(sliceJSONArrayFrom(response))
}
```

If we're tasked with making changes to `retrieveWords`, understanding its overall intent and flow now occurs more quickly. The function initializes a `finalPrompt` as a long text string with a couple of embedded concepts:

Nếu được giao việc sửa `retrieveWords`, giờ đây việc nắm bắt ý định tổng thể và luồng chạy của nó diễn ra nhanh hơn hẳn. Hàm này khởi tạo `finalPrompt` là một chuỗi văn bản dài với vài khái niệm nhúng bên trong:

- Either the text “word” or “words,” depending on the size of the words array
- A joined string of the quoted words in that array
 - Hoặc chữ “word”, hoặc chữ “words”, tùy theo kích thước của mảng `words`
 - Một chuỗi ghép từ các từ đã được đặt trong dấu nháy trong mảng đó

Those two concepts are now calls to tiny, isolated functions.

Hai khái niệm ấy giờ đã trở thành lời gọi tới những hàm tí hon, tách bạch.

We then send a longer **prompt** string to an **LLM**, receiving a text response. Finally, we slice out the JSON text from the response string (which exists because we sent the LLM a desired format as part of the prompt), then parse it.

Sau đó ta gửi một chuỗi prompt dài hơn tới một LLM (mô hình ngôn ngữ lớn) và nhận về một phản hồi dạng văn bản. Cuối cùng, ta cắt lấy phần văn bản JSON ra khỏi chuỗi phản hồi (phần này có mặt là vì ta đã gửi kèm định dạng mong muốn trong prompt), rồi parse nó.

We can understand such a function in about fifteen seconds; it is largely a statement of policy. It uses extracted functions with names that capture their intent. Each extracted function is implemented in one to three lines of code. If we needn't change any of those implementations, that's one to three lines of hidden detail we can ignore for now and maybe forever.

Ta có thể hiểu một hàm như vậy trong khoảng mười lăm giây; nó gần như chỉ là một tuyên bố chính sách. Nó dùng các hàm đã được trích ra với những cái tên nói lên đúng ý định của chúng. Mỗi hàm được trích ra chỉ được hiện thực trong một tới ba dòng mã. Nếu ta không cần đụng tới các hiện thực đó, thì đấy là một tới ba dòng chi tiết được giấu đi mà ta có thể lờ luôn — lúc này, và có khi là mãi mãi.

If we need to change one of the helper functions, the streamlined policy declaration allows us to find it quickly and focus on a tiny bit of **implementation** detail. We can make our changes without considering any other code.

Nếu cần sửa một trong các hàm phụ trợ, lời tuyên bố chính sách gọn ghẽ kia cho phép ta tìm ra nó rất nhanh và chỉ phải tập trung vào một mẫu chi tiết hiện thực tí xíu. Ta có thể sửa mà không phải bận tâm tới bất kỳ đoạn mã nào khác.

Editing Our Code

Biên tập lại mã của chúng ta

Crafting code with high clarity demonstrates that we care enough about our teammates (and ourselves) to spend a few moments editing our code before moving on.

Chăm chút cho mã đạt độ rõ ràng cao chứng tỏ ta đủ quan tâm tới đồng đội (và tới chính mình) để bỏ ra vài khoảnh khắc biên tập lại mã trước khi bước tiếp.

As developers, we're **good** about getting things to work—our #1 job. We need to also remind ourselves that we're writers as well, and that others must consume what we write. Once we capture our ideas on paper, good writing demands that we revisit our spewage and edit it to emphasize clarity.

Là lập trình viên, ta rất giỏi chuyện làm cho mọi thứ chạy được — nhiệm vụ số 1 của ta mà. Nhưng ta cũng cần tự nhắc rằng mình còn là người viết nữa, và người khác sẽ phải tiêu hoá thứ ta viết ra. Một khi đã đổ được ý tưởng lên giấy, kỹ năng viết tốt đòi hỏi ta phải quay lại với mớ vừa phun ra ấy và biên tập nó cho bật lên sự rõ ràng.

The sad fact is that we're not typically taught to edit our code. The vast majority of code out there shows it, and wastes copious amounts of your time as a result. WTFs per minute are high.

Sự thật đáng buồn là chẳng mấy ai dạy chúng ta biên tập mã. Phần áp đảo mã ngoài kia phơi bày điều đó, và hệ quả là ngón của bạn vô số thời gian. Số lần WTF mỗi phút cao ngất.

In fact, we're often told expressly not to edit code. "If it ain't broke, don't fix it." There's that lame, misapplied mantra again.

Thực tế, ta còn thường bị dặn dò dứt khoát là *đừng* biên tập mã. "Nó có hỏng đâu mà sửa." Lại cái câu thần chú cùn mòn, bị vận dụng sai chỗ ấy.

If other people can't make quick sense of your code, it is broken.

Nếu người khác không hiểu nhanh được mã của bạn, thì nó *đang hỏng*.

If you picked up a poorly written book, one that required you to reread sentences and paragraphs before they made sense, you'd consider it a bad, broken book (we bet you had at least one of these at university).

Nếu cầm phải một cuốn sách viết dở, loại bắt bạn đọc đi đọc lại từng câu từng đoạn mới hiểu, hẳn bạn sẽ coi đó là một cuốn sách tệ, một cuốn sách hỏng (chúng tôi cá là hồi đại học bạn đã gặp ít nhất một cuốn như thế).

The mantra exists because we fear breaking things. Yes, code is a brittle material. It's pretty easy to make a dumb coding mistake in just about any programming language, yet not even spot it. As a result, we're inclined to avoid cleaning things up, despite how easy it usually is.

Câu thần chú kia tồn tại vì ta sợ làm hỏng mọi thứ. Đúng, mã là thứ vật liệu giòn. Trong hầu như bất kỳ ngôn ngữ lập trình nào, phạm một lỗi ngớ ngẩn cũng dễ như bỡn, mà lại còn chẳng phát hiện ra. Hệ quả là ta có khuynh hướng né việc dọn dẹp, bất kể việc đó thường dễ đến mức nào.

Fearing making changes to our code is a quality smell. There are simple paths for creating the controls that tell us, with every tiny change, whether or not we've broken already-working logic. Visit Chapter 14, “Testing Disciplines,” to learn how.

Sợ phải đụng vào mã của chính mình là một mùi chất lượng. Có những con đường đơn giản để dựng nên các chốt kiểm soát báo cho ta biết, với từng thay đổi bé xíu, rằng ta đã làm hỏng logic vốn đang chạy tốt hay chưa. Hãy ghé Chương 14, “Các kỷ luật kiểm thử”, để biết cách làm.

Quick Steps to Clarity

Vài bước nhanh để đạt sự rõ ràng

- Extract implementation detail from multipurpose functions to create new functions with concise names.
- Move functions as appropriate to other modules. This results in increased cohesion, which helps readers find code where they might expect it.
- Replace comments with clear declarations.
- Write tests that double as documentation on all behavioral intents coded into the system.
 - Trích chi tiết hiện thực ra khỏi các hàm đa mục đích để tạo thành những hàm mới có tên gọn ghẽ.
 - Chuyển hàm sang module khác khi thích hợp. Việc này làm tăng độ gắn kết nội tại, nhờ đó người đọc tìm thấy mã ngay ở nơi họ trông đợi.
 - Thay comment bằng những khai báo rõ nghĩa.
 - Viết test đóng luôn vai trò tài liệu cho mọi ý định hành vi đã được cài vào hệ thống.

Objections to Clarity

Những phản bác dành cho sự rõ ràng

Seasoned programmers are able to immediately digest short code phrases. An experienced JavaScript developer can quickly comprehend the following couple of examples:

Lập trình viên lão luyện có thể tiêu hoá ngay lập tức các cụm mã ngắn. Một lập trình viên JavaScript nhiều kinh nghiệm sẽ hiểu nhanh hai ví dụ sau:

```
words.map(word => `${word}`).join(',')  
list.length === 1 ? word : `${word}s`
```

Such phrases still demand careful reading, though. We must mentally assemble the tokens involved into the behavioral concept they represent. A small potential for misinterpretation exists, as does a small potential for a **defect**. The following line of seemingly idiomatic code should give us pause:

Dù vậy, những cụm mã như thế vẫn đòi hỏi phải đọc kỹ. Ta phải tự lắp ráp trong đầu các token liên quan thành khái niệm hành vi mà chúng biểu diễn. Vẫn tồn tại một khả năng nhỏ là hiểu sai, cũng như một khả năng nhỏ là có lỗi. Dòng mã tưởng như rất “chuẩn phong cách” sau đây lẽ ra phải khiến ta khựng lại:

```
for (let i = 0; i <= words.length; i++)
```

Yet for many, it does not. That's not the idiom; this is:

Vậy mà với nhiều người, nó lại không. Đó không phải cách viết chuẩn; đây mới là:

```
for (let i = 0; i < words.length; i++)
```

The first for loop is likely a defect.

Vòng for đầu tiên nhiều khả năng là một lỗi.

In any case, extracting small implementation details into intention-revealing functions streamlines code in

Dù thế nào đi nữa, việc trích các chi tiết hiện thực nhỏ ra thành những hàm bộc lộ ý định (intention-revealing function) cũng giúp mã gọn ghẽ hẳn lên, chẳng hạn trong

```
retrieveWords:
const joinQuoted = words =>
  words.map(word => `"${word}"`).join(',')
const pluralizeIfMany = (word, list) =>
  list.length === 1 ? word : `${word}s`
export const retrieveWords = async words => {
  const finalPrompt =
    `Given a list of English nouns, separated by commas, ` +
    `provide appropriate Czech language information for the ` +
    `${pluralizeIfMany('word', words)} ${joinQuoted(words)} `
  // ...
}
```

Declare Intent, Don't Ooze Details

Hãy tuyên bố ý định, đừng rỉ ra chi tiết

We use functional pipelines partly because they allow us to replace details with declarations:

Một phần lý do ta dùng pipeline theo phong cách hàm là vì chúng cho phép ta thay chi tiết bằng những lời tuyên bố:

```
words.map(word => `"${word}"`).join(',')
```

That's loads easier to follow than the old-school procedural equivalent:

Cái đó dễ theo dõi hơn hẳn so với bản tương đương viết theo lối thủ tục kiểu cũ:

```
let wordList = "";
for (let i = 0; i < words.length; i++) {
  const currentWord = words[i];
  const quotedWord = '"' + currentWord + '"';
  wordList += quotedWord;
  if (i !== words.length - 1) {
    wordList += ",";
  }
}
```

The old-school code also has a dramatically increased possibility of hiding defects (hearken back to the for loop example).

Đoạn mã kiểu cũ còn có khả năng che giấu lỗi cao hơn hẳn (nhớ lại ví dụ vòng for lúc nãy đi).

Keeping Pipelines Streamlined

Giữ cho pipeline luôn gọn ghẽ

While it's reasonable to code short in-line functions within a pipeline:

Việc viết các hàm ngắn ngay trong lòng một pipeline là hợp lý:

```
export const mostExpensiveHighlyRatedBookInEachCategory = (categories) =>
  categories.flatMap(category =>
```

```
category.books
  .filter(book => !!book.title)
  .filter(book => book.rating > 4.0)
  .sort((a, b) => b.price - a.price)
  .map((book) =>
    ({ category: book.category, title: book.title }))
  .slice(0, 1))
```

they break the flow and require careful stepwise reading.

nhưng chúng làm đứt mạch và buộc người đọc phải dò từng bước một cách cẩn thận.

Replacing all the in-line lambdas—even the very short functions—with named functions does wonders for our ability to quickly follow the entire flow:

Thay toàn bộ các lambda viết in-line — kể cả những hàm cực ngắn — bằng các hàm có tên sẽ tạo ra khác biệt kỳ diệu cho khả năng theo dõi nhanh toàn bộ luồng chạy:

```
export const
mostExpensiveHighlyRatedBookInEachCategory = (categories) => {
  const byPrice = (a, b) => b.price - a.price
  const highlyRated = book => book.rating > 4.0
  const hasTitle = book => !!book.title
  const categoryAndTitle = (book) =>
    ({ category: book.category, title: book.title })
  return categories.flatMap(category =>
    category.books
      .filter(hasTitle)
      .filter(highlyRated)
      .sort(byPrice)
      .map(categoryAndTitle)
      .slice(0, 1))
}
```

When reading the larger function, we scan past the one-liner function declarations and focus on the return statement. We're able to read and mentally assemble what's going on more quickly now, because the pipeline is abstracted and not full of detail.

Khi đọc hàm lớn hơn, ta lướt qua các khai báo hàm một dòng và tập trung vào câu lệnh return. Giờ ta đọc và lắp ráp trong đầu chuyện đang diễn ra nhanh hơn hẳn, bởi pipeline đã được trừu tượng hoá chứ không còn ngồn ngộn chi tiết.

Hmm. The updated solution contains an idiomatic expression: `.slice(0, 1)`. It returns the first element of an array (less idiomatic) or an empty array if the array is empty (yes idiomatic). We don't eliminate our idioms unless they're causing readers to come to a grinding halt. If so, we find a way to abstract them. (Here, we might create a `firstOrDefault` function to supplant the slice call.)

Hừm. Lời giải mới lại chứa một cách viết đặc thù: `.slice(0, 1)`. Nó trả về phần tử đầu tiên của một mảng (chỗ này thì kém đặc thù) hoặc một mảng rỗng nếu mảng ban đầu rỗng (chỗ này thì đúng là đặc thù). Ta không loại bỏ các lối viết đặc thù trừ khi chúng khiến người đọc phải phanh gấp. Nếu vậy, ta tìm cách trừu tượng hoá chúng đi. (Ở đây, ta có thể tạo một hàm `firstOrDefault` để thế chỗ lời gọi slice.)

Note that as we extract functions, we find insufficient cohesion in the form of misplaced bits of logic. Typical. A couple of functions extracted from this example's pipeline might find more appropriate

homes in a book module.

Để ý rằng khi trích các hàm ra, ta phát hiện độ gắn kết nội tại chưa đủ, dưới dạng những mẫu logic bị đặt nhầm chỗ. Chuyện thường. Vài hàm được trích ra từ pipeline trong ví dụ này có lẽ sẽ tìm được mái nhà thích hợp hơn ở một module book.

Conciseness — Súc tích

A functional pipeline can provide code that is not only very clear, but also concise. The solution that requires the fewest possible tokens is maximally concise—but this is almost never what we really want.

Một pipeline theo phong cách hàm có thể cho ra đoạn mã không chỉ rất rõ ràng mà còn rất súc tích. Lời giải dùng ít token nhất có thể là lời giải súc tích tối đa — nhưng gần như chẳng bao giờ đó là thứ ta thực sự muốn.

Code that is maximally concise without consideration for clarity is obfuscated. The only legitimate time when we might deliberately obscure code is as part of a post-development deployment process (perhaps for compression or security interests), or as challenge/entertainment (see <https://www.ioccc.org>).

Mã súc tích tối đa mà bất chấp độ rõ ràng là mã bị làm rối. Thời điểm duy nhất chính đáng để cố tình làm mã trở nên khó đọc là trong quy trình triển khai sau khi đã phát triển xong (có lẽ vì mục đích nén hoặc bảo mật), hoặc để thách đố/giải trí (xem <https://www.ioccc.org>).

The absolute need for clarity means we must find a balance between conciseness and clarity. Finding the balance isn't tough, though—just ask your teammates. They'll let you know the moment something doesn't make sense. Watch them read through your code, and it'll usually be obvious when they're struggling with your ultra-optimized logic.

Nhu cầu tuyệt đối về độ rõ ràng nghĩa là ta phải tìm được điểm cân bằng giữa súc tích và rõ ràng. Mà tìm điểm cân bằng ấy cũng chẳng khó — cứ hỏi đồng đội của bạn. Họ sẽ báo ngay khi có chỗ nào đọc không hiểu. Hãy quan sát họ đọc mã của bạn; thường thì rất dễ nhận ra lúc họ đang vật lộn với thứ logic được tối ưu quá đà của bạn.

Let's start with an opposite example, however—code that's clear but not concise. The following generateCard function, along with a couple of helper functions, assembles a text-based flash card (complete with the answer on it—how curious!).

Tuy nhiên, hãy bắt đầu bằng một ví dụ ngược lại — đoạn mã rõ ràng nhưng không súc tích. Hàm generateCard dưới đây, cùng vài hàm phụ trợ, lắp ráp một tấm thẻ ghi nhớ dạng văn bản (kèm luôn cả đáp án trên đó — lạ chưa!).

```
// generate integer from 0 to n-1
export const randomInt = n => {
  const randomValue = Math.random(); // Generate a random decimal
  const scaledValue = randomValue * n; // scale to desired range
  const flooredValue = Math.floor(scaledValue); // floor to int
  return flooredValue;
}
// return a word in its plural form if needed
const pluralizeIf = (word, isPlural) => {
  if (isPlural) {
```

```

    return `${word}s`; // Append 's' to make the word plural
  } else {
    return word; // Return the original word if not plural
  }
}
// generates prompt text for a language card game
const generateCard = (adjectives, nouns) => {
  // Select a random adjective and noun from the provided lists
  const randomAdjectiveIndex = randomInt(adjectives.length);
  const randomNounIndex = randomInt(nouns.length);

  const adjective = adjectives[randomAdjectiveIndex];
  const noun = nouns[randomNounIndex];
  // Determine the case and number randomly
  const nominativeOrAccusative =
    randomInt(2) === 0 ? 'nominative' : 'accusative';
  const isPlural = randomInt(2) === 0;
  const singularOrPlural = isPlural ? 'plural' : 'singular';
  // Get correct forms of the noun & adjective based on case/number
  const correctNoun = noun[singularOrPlural][nominativeOrAccusative]
  const nounGender = noun.gender.toLowerCase();
  const correctAdjective =
    adjective[nominativeOrAccusative][singularOrPlural][nounGender]
  // Construct the prompt text for the game card
  const promptText = `Adjective: ${adjective.word}
Noun: ${noun.word}
Determine the ${singularOrPlural} ${nominativeOrAccusative} case.
Correct answer: ${correctAdjective} ${correctNoun}`;
  // Return the constructed prompt
  return promptText;
}

```

The code appears typical to us, and that's not a good thing. The core function, `generateCard`, is a top-to-bottom affair. It's not terribly long, but it includes comments to guide readers through what's happening in the next few statements. Each of the function's statements uses clear **variable** names and is readily comprehensible. We can carefully read through the code statement by statement, mentally assembling groups of statements into unit behaviors. What's not to like?

Đoạn mã trên trông thật điển hình với chúng ta, mà đó lại chẳng phải chuyện hay. Hàm cốt lõi, `generateCard`, là một mạch chạy thẳng từ trên xuống dưới. Nó không dài khủng khiếp, nhưng có kèm comment để dẫn người đọc qua những gì đang xảy ra ở vài câu lệnh kế tiếp. Mỗi câu lệnh trong hàm đều dùng tên biến rõ ràng và dễ hiểu ngay. Ta có thể đọc kỹ từng câu lệnh một, rồi lắp ráp trong đầu từng nhóm câu lệnh thành các hành vi đơn vị. Có gì mà chê được nào?

Indeed, the above example is how many developers happily continue coding for most of their career. But it's suboptimal for numerous reasons.

Quả thật, ví dụ trên chính là cách rất nhiều lập trình viên vui vẻ tiếp tục viết mã suốt phần lớn sự nghiệp. Nhưng nó chưa tối ưu, vì vô số lý do.

A reworked version results in additional small helper functions.

Bản viết lại sinh thêm vài hàm phụ trợ nhỏ.

(Again, start your reading at generateCard at the bottom.)

(Một lần nữa, hãy bắt đầu đọc từ generateCard ở dưới cùng.)

```
export const randomInt = n => Math.floor(Math.random() * n)
const pluralizeIf = (word, isPlural) => isPlural ? `${word}s` : word
const randomElement = (array) => array[randomInt(array.length)]
const cases = ["nominative", "accusative"]
const numbers = ["singular", "plural"]
const generateRandomCardData = (adjectives, nouns) => ({
  adjective: randomElement(adjectives),
  noun: randomElement(nouns),
  grammaticalCase: randomElement(cases),
  number: randomElement(numbers)
})

const formatCard =
  (adjective, noun, number, grammaticalCase,
   correctAdjective, correctNoun) =>
  `${adjective.word} ${pluralizeIf(noun.word, number === 'plural')}
  Provide the ${number} ${grammaticalCase} case.
  Correct answer: ${correctAdjective} ${correctNoun}`

const generateCard = (adjectives, nouns) => {
  const { adjective, noun, grammaticalCase, number } =
    generateRandomCardData(adjectives, nouns)
  const correctNoun = noun[number][grammaticalCase]
  const correctAdjective = adjective[grammaticalCase]
    [number]
    [noun.gender.toLowerCase()]
  return formatCard(adjective, noun, number, grammaticalCase,
    correctAdjective, correctNoun)
}
```

Clarity is increased in a few ways. Distilling generateCard into three top-level chunks allows readers to quickly identify the overall policy.

Độ rõ ràng tăng lên theo vài hướng. Việc chưng cất generateCard thành ba mảng lớn ở tầng trên cùng giúp người đọc nhanh chóng nhận ra chính sách tổng thể.

- Generate random flash card elements using the adjectives and nouns passed in.
 - Sinh ra các thành phần ngẫu nhiên của tấm thẻ từ danh sách adjective và noun được truyền vào.
- Determine the correct answers to the challenge on the flash card by dereferencing the noun and adjective structures passed in.
- Return a formatted card containing all the relevant information.
 - Xác định đáp án đúng cho câu đố trên tấm thẻ bằng cách tra ngược các cấu trúc noun và adjective được truyền vào.
 - Trả về một tấm thẻ đã định dạng chứa toàn bộ thông tin liên quan.

We can immediately understand and trust each supporting function or piece of data in this solution. We might not even need to look at any supporting code or data.

Ta hiểu và tin được ngay từng hàm phụ trợ hay từng mẫu dữ liệu trong lời giải này. Thậm chí có khi chẳng cần nhìn vào mã hay dữ liệu hỗ trợ nào cả.

The separate functions enhance clarity by allowing us to focus on small, understandable chunks. And while it would help to see the data structures involved (individual noun and adjective objects specifically), every statement otherwise stands on its own. No statements demand a **comment**. Comments would only increase the development, comprehension, and maintenance costs for this code.

Các hàm tách riêng làm tăng độ rõ ràng vì cho phép ta tập trung vào từng mảnh nhỏ, dễ hiểu. Và dù có thấy được các cấu trúc dữ liệu liên quan (cụ thể là từng đối tượng noun và adjective) thì cũng tốt, mọi câu lệnh còn lại đều tự đứng vững một mình. Không câu lệnh nào đòi hỏi một comment. Comment lúc này chỉ làm tăng chi phí phát triển, chi phí đọc hiểu và chi phí bảo trì cho đoạn mã.

As for understanding the data structures, that's what unit tests are for. Still, we might make a case for extracting a couple of functions to get just an ounce more isolation from implementation specifics in generateCard :

Còn chuyện hiểu các cấu trúc dữ liệu ư? Unit test sinh ra chính là để làm việc đó. Dù vậy, ta vẫn có thể lập luận cho việc trích ra đôi ba hàm nữa, để generateCard được cách ly thêm một chút xít khỏi các chi tiết hiện thực:

```
const generateCard = (adjectives, nouns) => {
  const { adjective, noun, grammaticalCase, number } =
    generateRandomCardData(adjectives, nouns)
  const correctNoun = selectNoun(noun, number, grammaticalCase)
  const correctAdjective =
    selectAdjective(adjective, grammaticalCase, number, noun)
  return formatCard(
    adjective,
    noun,
    number,
    grammaticalCase,
    correctAdjective,
    correctNoun)
}
```

We also spotted opportunities to cohere seemingly disparate elements, by introducing a new **abstraction**—the notion of an object that captures the underlying data elements of a random card. Yet there's still something amiss with our function. The excessive number of parameters returned and passed about not only muddles the solution, but also decreases its concision.

Chúng tôi cũng phát hiện cơ hội gắn kết những phần tử tưởng chừng rời rạc, bằng cách đưa vào một trừu tượng hoá (abstraction) mới — khái niệm về một đối tượng (object) gói trọn các phần tử dữ liệu nền tảng của một tấm thẻ ngẫu nhiên. Ấy thế mà hàm của ta vẫn còn chỗ chưa ổn. Số lượng tham số bị trả về và chuyển qua chuyển lại quá nhiều không chỉ làm rối lời giải, mà còn làm giảm sự súc tích của nó.

Let's make a final cleanup pass.

Hãy làm một lượt dọn dẹp cuối cùng.

```
const generateRandom = (adjectives, nouns) => ({
  adjective: randomElement(adjectives),
```

```

    noun: randomElement(nouns),
  })
const formatCard =
  (phrase, caseAgreement, correctAdjective, correctNoun) =>
    `${phrase.adjective.word} ${pluralizeIf(
      phrase.noun.word,
      caseAgreement.number === 'plural')
    }
    Provide the ${caseAgreement.number} ` +
      `${caseAgreement grammaticalCase} case.
    Correct answer: ${correctAdjective} ${correctNoun}`

const selectNoun = (noun, caseAgreement) =>
  noun[caseAgreement.number][caseAgreement.grammaticalCase]
const selectAdjective = (adjective, gender, caseAgreement) =>
  adjective[caseAgreement.grammaticalCase]
    [caseAgreement.number]
    [gender.toLowerCase()]
const cases = ["nominative", "accusative"]
const numbers = ["singular", "plural"]
const generateCaseAgreementChallenge = () => ({
  grammaticalCase: randomElement(cases),
  number: randomElement(numbers)
})
const generateCard = (adjectives, nouns) => {
  const phrase = generateRandom(adjectives, nouns)
  const caseAgreement = generateCaseAgreementChallenge()
  const correctAdjective = selectAdjective(
    phrase.adjective, phrase.noun.gender, caseAgreement)
  const correctNoun = selectNoun(phrase.noun, caseAgreement)
  return formatCard(
    phrase, caseAgreement, correctAdjective, correctNoun)
}

```

Slightly more-thoughtful data groupings make generateCard seem less of a haphazard bustle of wayward local variables. Each of its handful of steps emphasizes the key elements involved. We also reworked selectAdjective to accommodate a gender parameter instead of a noun. Our choice emphasizes that a noun's gender, specifically, is key to determining the proper adjective.

Việc gom nhóm dữ liệu chẵn chu hơn một chút khiến generateCard bớt vẻ một mớ biến cục bộ lang thang xô bồ. Mỗi bước trong dăm bước ít ỏi của nó đều làm nổi bật những phần tử then chốt có liên quan. Chúng tôi cũng chỉnh lại selectAdjective để nhận tham số gender thay vì một noun. Lựa chọn ấy nhấn mạnh rằng chính giống của danh từ mới là chìa khoá để xác định adjective phù hợp.

Code **Duplication**

Trùng lặp mã

Being concise in code means implementing concepts once and only once. We often find small chunks of implementation detail replicated through a module or codebase—usually from a couple to a handful of lines that are oft repeated. Here's a commonly found example—repetitive error handling:

Súc tích trong mã nghĩa là hiện thực mỗi khái niệm một lần và chỉ một lần duy nhất. Ta

thường bắt gặp những mẫu chi tiết hiện thực nhỏ được nhân bản khắp một module hay cả codebase — thường là từ vài dòng đến dăm bảy dòng cứ lặp đi lặp lại. Đây là một ví dụ hay gặp — xử lý lỗi lặp lại:

```
export const postItem = (request, response) => {
  const itemDetails = retrieveItem(request.body.upc)
  if (!itemDetails) {
    response.status = 400
    const errorMessage = 'unrecognized UPC code';
    logError(errorMessage)
    return response.send({error: errorMessage})
  }
  const checkout = Checkouts.retrieve(request.params.id)
  if (!checkout) {
    response.status = 400
    const errorMessage = 'nonexistent checkout';
    logError(errorMessage)
    return response.send({error: errorMessage})
  }
  const newCheckoutItem = Checkouts.addItem(checkout, itemDetails)
  sendResponse(response, newCheckoutItem, 201)
}
```

Repetition increases costs for a number of reasons. The essence of the controller function `postItem` is obscured by repetitious error handling code. This duplication also decreases clarity, by mixing implementation details into policy flow.

Sự lặp lại làm tăng chi phí vì nhiều lý do. Bản chất của hàm controller `postItem` bị che khuất bởi đoạn mã xử lý lỗi lặp đi lặp lại. Sự trùng lặp này còn làm giảm độ rõ ràng, do trộn lẫn chi tiết hiện thực vào luồng chính sách.

```
const sendRequestError = (response, message) => {
  response.status = 400
  logError(errorMessage)
  response.send({error: message})
}
export const postItem = (request, response) => {
  const itemDetails = retrieveItem(request.body.upc)
  if (!itemDetails)
    return sendRequestError(response, 'unrecognized UPC code')
  const checkout = Checkouts.retrieve(request.params.id)
  if (!checkout)
    return sendRequestError(response, 'nonexistent checkout')
  const newCheckoutItem = Checkouts.addItem(checkout, itemDetails)
  sendResponse(response, newCheckoutItem, 201)
}
```

Often, eliminating duplication involves replacing concrete details with abstractions. It's generally a win-win-win-win situation (increased conciseness, clarity, and ease of confirmability, plus a springboard to increased cohesion), but we don't go overboard by obsessing over two lines of code that happen to look the same. We seek to find and eradicate duplicate implementations of concepts,

not incidentally common lines of code.

Thường thì loại bỏ trùng lặp đồng nghĩa với việc thay các chi tiết cụ thể bằng trừu tượng hoá. Nhìn chung đó là tình huống thẳng-thẳng-thẳng-thẳng (súc tích hơn, rõ ràng hơn, dễ kiểm chứng hơn, cộng thêm bàn đạp để tăng độ gắn kết nội tại (cohesion)), nhưng ta đừng làm quá đến mức ám ảnh vì hai dòng mã tình cờ trông giống nhau. Thứ ta muốn truy lùng và diệt trừ là những bản hiện thực trùng lặp của cùng một khái niệm, chứ không phải mấy dòng mã giống nhau một cách ngẫu nhiên.

Duplication fosters many increases in cost/effort:

Trùng lặp làm chi phí/công sức tăng lên theo rất nhiều cách:

- Time to read and comprehend cost
- Time to search code in general (we're always **wading** through increased amounts of irrelevant results)
- Time to **test**
- Effort to find all points of change for a replicated concept
- Risk of not finding all necessary points of change
- Time to analyze whether variances found in duplicated code are deliberate or if they represent defects
- Effort to change duplicated code
- Cost to **refactor** code
- Propagation of defect costs, when common code is defective
 - Thời gian đọc và hiểu
 - Thời gian tìm kiếm trong mã nói chung (ta luôn phải lội bì bõm qua ngày càng nhiều kết quả không liên quan)
 - Thời gian test
 - Công sức tìm cho ra mọi điểm cần sửa của một khái niệm bị nhân bản
 - Rủi ro không tìm ra hết những điểm cần sửa
 - Thời gian phân tích xem những khác biệt trong mã trùng lặp là cố ý hay là lỗi (defect)
 - Công sức sửa mã trùng lặp
 - Chi phí refactor mã
 - Chi phí lỗi lan truyền, khi đoạn mã dùng chung có lỗi

We've worked on many systems that were two-to-three times as large as they might have been due to rampant code duplication. Oh, the days of getting paid by the line of code! (Just kidding. It never happened for any of us.) One example included an operating room scheduling system, which by definition needed to do a lot of work with dates and timestamps. Among many other amusements, we found a common five-line piece of (old-school) Java date handling logic. These five lines were repeated in over fifty places throughout the codebase. Our costs increased dramatically when we sought to replace the old-school logic with a modern library.

Chúng tôi từng làm với nhiều hệ thống to gấp hai đến ba lần mức đáng có, chỉ vì mã trùng lặp tràn lan. Ôi, cái thời được trả tiền theo số dòng mã! (Đùa thôi. Chưa ai trong chúng tôi được hưởng chuyện đó bao giờ.) Một ví dụ là hệ thống xếp lịch phòng mổ, mà theo đúng bản chất của nó thì phải xử lý rất nhiều ngày tháng và dấu thời gian. Giữa vô vàn trò vui khác, chúng tôi tìm thấy một đoạn năm dòng logic xử lý ngày tháng kiểu Java đời cũ. Năm dòng ấy lặp lại ở hơn năm mươi chỗ khắp codebase. Chi phí của chúng tôi tăng vọt khi muốn thay thứ logic đời cũ đó bằng một thư viện hiện đại.

Extract-and-move is once again our trustworthy workhorse for tackling duplication problems.

Trích rồi dời (extract-and-move) một lần nữa lại là con ngựa thồ đáng tin cậy để xử lý vấn

đề trùng lặp.

- Identify a clump of implementation detail that represents a singular concept.
- Extract it to a function.
- Focus on the new function in isolation. Move it to another module if appropriate.
 - Xác định một cụm chi tiết hiện thực thể hiện đúng một khái niệm duy nhất.
 - Trích nó ra thành một hàm.
 - Tập trung vào hàm mới một cách biệt lập. Dời nó sang module khác nếu thấy hợp lý.

As awareness of the existence of a new module increases, we start to spot more opportunities to move additional related behaviors into it. The cohesion of our modules increases as a result.

Khi mọi người dần biết đến sự tồn tại của module mới, ta bắt đầu thấy thêm nhiều cơ hội để dời các hành vi liên quan khác vào đó. Kết quả là độ gắn kết nội tại của các module tăng lên.

Conciseness Nits

Những chuyện vặt về sự súc tích

Every unnecessary token adds to the clutter of our code. We can and should often use things like idiomatic constructs and syntactic sugar to reduce the amount of unnecessary code that we must scan and mentally assemble. A few pet nits of ours:

Mỗi token thừa đều góp thêm vào mớ bừa bộn trong mã. Ta có thể và nên tận dụng những thứ như cấu trúc đặc thù của ngôn ngữ hay đường cú pháp (syntactic sugar) để giảm bớt lượng mã thừa mà mắt ta phải quét và đầu ta phải lắp ráp lại. Vài chuyện vặt mà chúng tôi đặc biệt để bụng:

Return Boolean Expressions

Trả về biểu thức boolean

Learning a language well means we must not only learn its syntax, but also learn the most concise way of representing common concepts. Some of these streamlined constructs might go so far as to be classified as idiomatic. (Yes, we've used that word a number of times; we guess we should finally define it: Idiomatic code isn't obvious the first time you see it.)

Học tốt một ngôn ngữ nghĩa là không chỉ học cú pháp của nó, mà còn phải học cách diễn đạt súc tích nhất cho những khái niệm quen thuộc. Một số cấu trúc gọn gàng kiểu đó thậm chí có thể xếp vào loại idiomatic. (Vâng, chúng tôi đã dùng từ này nhiều lần rồi; chắc cuối cùng cũng nên định nghĩa nó: mã idiomatic là mã không hiển nhiên ngay lần đầu bạn nhìn thấy.)

A ternary operator is idiomatic: In an isolated context, it doesn't inherently provide all the information you need to immediately decipher it. The second or third time you see it, however, you'll know how to interpret it—it has become obvious to you. It's like riding a bicycle; you won't likely ever need to be reminded after that.

Toán tử ba ngôi là idiomatic: đặt trong một ngữ cảnh biệt lập, bản thân nó không cung cấp đủ thông tin để bạn giải mã ngay lập tức. Nhưng đến lần thứ hai hay thứ ba nhìn thấy, bạn sẽ biết đọc nó thế nào — nó đã trở nên hiển nhiên với bạn. Giống như đi xe đạp; sau đó gần như chẳng bao giờ bạn cần ai nhắc lại nữa.

Sometimes we see anti-idioms:

Đôi khi ta gặp phải những thứ phản-idiom:

```
const hasTitle = book => {
  if (book.title !== null) {
    return true
  } else {
    return false
  }
}
```

Anti-idiom? Despite the fact that this construct demands five precious **vertical** source lines, we usually digest it as a single chunk when we see it (because unfortunately, we see it a lot). Sometimes we even notice when a chucklehead has sneakily reversed the order of the true and false returns.

Phản-idiom ư? Mặc dù cấu trúc này ngắn tới năm dòng mã quý giá theo chiều dọc, ta thường vẫn tiêu hoá nó gọn trong một nhịp khi nhìn thấy (bởi vì, thật đáng buồn, ta thấy nó nhiều lắm). Đôi khi ta còn nhận ra cả những lúc một gã đãng trí nào đó lén đảo ngược thứ tự của nhánh trả về true và false.

We rarely say that things are flat-out wrong in programming, but this is one of those cases.

Chúng tôi rất hiếm khi nói rằng thứ gì đó trong lập trình là sai bét, nhưng đây là một trong những trường hợp như vậy.

Simply (and we don't use that word lightly) return the boolean expression:

Chỉ cần (và chúng tôi không dùng chữ “chỉ cần” một cách hời hợt) trả về chính biểu thức boolean:

```
const hasTitle = book => book.title !== null
```

Unnecessary Else Maybe we needn't embrace the ternary operator—worse things than if-else statements pollute our coding world—but if we're going to code a conditional return:

Else thừa thãi Có lẽ ta không nhất thiết phải ôm lấy toán tử ba ngôi — thế giới lập trình của chúng ta còn bị ô nhiễm bởi lắm thứ tệ hơn câu lệnh if-else — nhưng nếu đã định viết một lệnh return có điều kiện:

```
const pluralizeIf = (word, isPlural) => {
  if (isPlural) {
    return `${word}s`
  } else {
    return word
  }
}
```

we should at least have the decency to not include the unnecessary else keyword. An early return suffices:

thì ít nhất ta cũng nên tử tế mà đừng nhét thêm từ khoá else thừa thãi vào. Một lệnh return sớm là đủ:

```
const pluralizeIf = (word, isPlural) => {
  if (isPlural) {
    return `${word}s`;
  }
  return word
}
```

No, we really don't need the braces, though odds are good our team has adopted a coding standard that insists on them. But it's really just overprotective clutter. It's hard to **mess** up if you start with a short, single-line if statement:

Không, ta thật sự chẳng cần cặp ngoặc nhọn, dù nhiều khả năng đội của ta đã chọn một chuẩn viết mã khăng khăng đòi phải có. Nhưng đó thật ra chỉ là thứ rác bảo hộ quá mức. Rất khó làm hỏng nếu bạn bắt đầu bằng một câu lệnh if ngắn gọn nằm gọn trên một dòng:

```
const pluralizeIf = (word, isPlural) => {  
  if (isPlural) return `${word}s`  
  return word  
}
```

We write tests if we remain worried. In any case, there's honestly not much good reason to eschew a ternary for this example:

Nếu vẫn còn lo thì ta viết test. Dù sao đi nữa, thành thật mà nói cũng chẳng có mấy lý do chính đáng để né toán tử ba ngôi trong ví dụ này:

```
const pluralizeIf = (word, isPlural) => isPlural ? `${word}s` : wo
```

We get it, though, if you object on aesthetic grounds.

Nhưng chúng tôi cũng hiểu, nếu bạn phản đối vì lý do thẩm mỹ.

And So On

Và cứ thế

We could dedicate an entire book to clutter in code. Most of it, however, you'll figure out on your own. We don't seek clever code; we instead seek elegant code. Elegant code says exactly what it needs, without too few or too many words. It is both clear and concise.

Chúng tôi có thể dành hẳn một cuốn sách cho chuyện rác rưởi trong mã. Tuy nhiên, phần lớn trong số đó bạn sẽ tự ngộ ra. Chúng ta không tìm kiếm mã khôn lỏi; chúng ta tìm kiếm mã thanh lịch. Mã thanh lịch nói đúng những gì cần nói, không thiếu chữ nào mà cũng chẳng thừa chữ nào. Nó vừa rõ ràng vừa súc tích.

Confirmability — Tính xác nhận được (Confirmability)

Our continuous design journey requires that we know what our code is intended to do. Without that knowledge, any change is unsafe. A system may contain many thousands of behavioral units. An unnoticed change in behavior to any one of them can allow us to unleash a costly defect in production.

Hành trình thiết kế liên tục (continuous design) đòi hỏi ta phải biết mã của mình được kỳ vọng làm gì. Thiếu hiểu biết đó, mọi thay đổi đều không an toàn. Một hệ thống có thể chứa hàng nghìn đơn vị hành vi. Chỉ cần một trong số chúng đổi hành vi mà ta không để ý là đủ để ta tung ra production một lỗi (defect) đắt giá.

Fortunately, we can write thousands of small, fast unit tests that verify whether we've created any regressions. These tests can tell us within seconds the moment we broke something. We'll need other kinds of tests at higher levels, of course, such as end-to-end functional tests, performance tests, load tests, and contract tests. But if we want to move fast, we need to be able to rapidly create and manage tests around the unit implementations.

May thay, ta có thể viết hàng nghìn unit test nhỏ và nhanh để kiểm chứng xem mình có gây ra hồi quy (regression) nào không. Chỉ trong vài giây, chúng báo cho ta biết đúng khoảng khắc ta làm hỏng thứ gì đó. Dĩ nhiên ta vẫn cần các loại test khác ở mức cao hơn: test chức năng end-to-end, test hiệu năng, test tải, contract test. Nhưng nếu muốn đi nhanh, ta phải tạo và quản lý được thật mau lẹ các test quanh phần hiện thực (implementation) ở mức đơn vị.

Fear Degrades Design In contrast, the lack of such tests gradually slows us down. As the amount of (not unit-tested) code increases, our costs to verify it increase. We can manually step through only so many test cases.

Nỗi sợ làm suy đồi thiết kế Ngược lại, việc thiếu những test như thế sẽ dần dần làm ta chậm lại. Lượng mã (không có unit test) càng phình ra thì chi phí kiểm chứng nó càng lớn. Ta chỉ có thể lần tay qua chừng đó test case, không hơn.

We can write integration tests, but they're generally costlier to build and maintain. They also create a slower feedback loop. Finally, it's unrealistic to expect that you'll be able to cover all the thousands of intentional decisions and logic variants that went into the codebase by virtue of integration tests (which also can increase the challenge of pinpointing the defect when they do fail).

Ta có thể viết integration test, nhưng loại này nhìn chung tốn kém hơn để xây và bảo trì. Chúng cũng tạo ra vòng phản hồi chậm hơn. Cuối cùng, thật phi thực tế khi trông đợi rằng chỉ nhờ integration test là bạn phủ hết được hàng nghìn quyết định có chủ đích và biến thể logic đã đi vào codebase (chưa kể khi chúng thất bại, việc chỉ đích danh lỗi cũng khó hơn nhiều).

Without fast feedback in the form of unit tests, we relegate ourselves to that “it ain't broke, don't fix it” mentality. Our codebase degrades by definition. We don't have the confidence to ensure that the code remains cohesive, clear, and concise. Instead, we habitually slap out “first drafts”: We get our code working, then immediately move on to the next thing without making any edits.

Không có phản hồi nhanh dưới dạng unit test, ta tự đẩy mình vào nỗi trạng “chưa hỏng thì đừng sửa”. Codebase của ta mục ruỗng — theo đúng định nghĩa. Ta không đủ tự tin để bảo đảm mã vẫn gắn kết, rõ ràng và súc tích. Thay vào đó, ta quen tay quăng ra những “bản nháp đầu tiên”: cho mã chạy được, rồi lập tức nhảy sang việc kế tiếp mà chẳng chỉnh sửa gì.

It seems fast at first, yet the results of fear-driven coding quickly become apparent in the codebase. Developers learn to do the worst possible thing to a system, rather than what they know to be the best thing for the system.

Thoạt đầu có vẻ nhanh, nhưng hậu quả của lối viết mã bị nỗi sợ dẫn dắt lộ ra trong codebase rất mau. Lập trình viên học cách làm điều tệ nhất có thể cho hệ thống, thay vì điều mà chính họ biết là tốt nhất cho nó.

A real example from a high-performance, historically famous large system: Developers needed to create variants of behaviors existing within typically long (100+ lines) C++ member functions. Shipping defective versions of these existing behaviors could cost the company millions of dollars per minute.

Một ví dụ có thật từ một hệ thống lớn, hiệu năng cao, từng nổi danh trong lịch sử: lập trình viên cần tạo các biến thể của những hành vi vốn nằm sẵn trong các member function C++ thường rất dài (hơn 100 dòng). Ship ra bản lỗi của những hành vi này có thể khiến công ty mất hàng triệu đô mỗi phút.

As a result, developers habitually replicated (i.e., copied/pasted) entire member functions, then made their changes within these copies. They didn't change the original functions—no one wanted to be the one who broke code that was already working. (“Why did you even touch that code?”)

Kết quả là lập trình viên quen tay nhân bản (tức copy/paste) nguyên cả member function, rồi sửa đổi bên trong bản sao đó. Họ không đụng đến hàm gốc — chẳng ai muốn làm kẻ phá vỡ đoạn mã vốn đang chạy tốt. (“Sao anh lại còn đi động vào đoạn mã đó làm gì?”)

Fear significantly increases code duplication and costs.

Nỗi sợ làm trùng lặp (duplication) và chi phí tăng lên đáng kể.

Conflated Units Imply Tough Tests

Đơn vị bị trộn lẫn kéo theo test khó khăn

If we write code like the following postCheckoutTotal function, writing tests for it will be daunting:

Nếu ta viết mã như hàm postCheckoutTotal dưới đây, viết test cho nó sẽ là một cực hình:

```
export const postCheckoutTotal = (request, response) => {
  const checkoutId = request.params.id
  const checkout = Checkouts.retrieve(checkoutId)
  if (!checkout) {
    response.status = 400
    response.send({error: 'nonexistent checkout'})
    return
  }
  const messages = []
  const discount = checkout.member ? checkout.discount : 0
  let totalOfDiscountedItems = 0
  let total = 0
  let totalSaved = 0
  checkout.items.forEach(item => {
    let price = item.price
    const isExempt = item.exempt
    if (!isExempt && discount > 0) {
      const discountAmount = discount * price

      const discountedPrice = price * (1.0 - discount)
      // add into total
      totalOfDiscountedItems += discountedPrice
      let text = item.description
      // format percent
      const amount = parseFloat(
        (Math.round(price * 100) / 100).toString()).toFixed(2)
      const amountWidth = amount.length
      let textWidth = LineWidth - amountWidth
      messages.push(pad(text, textWidth) + amount)
      total += discountedPrice
      // discount line
      const discountFormatted = '-' + parseFloat(
        (Math.round(discountAmount * 100) / 100).toString()).toFixed(2)
      textWidth = LineWidth - discountFormatted.length
      text = `    ${discount * 100}% mbr disc`
    }
  })
}
```



```

    messages.push(`${pad(text, textWidth)}${discountFormatted}`)
    totalSaved += discountAmount
  }
  else {
    total += price
    const text = item.description

    const amount = parseFloat(
      (Math.round(price * 100) / 100).toString()).toFixed(2)
    const amountWidth = amount.length
    const textWidth = LineWidth - amountWidth
    messages.push(pad(text, textWidth) + amount)
  }
})
total = Math.round(total * 100) / 100
// append total line
const formattedTotal = parseFloat(
  (Math.round(total * 100) / 100).toString()).toFixed(2)
const formattedTotalWidth = formattedTotal.length
const textWidth = LineWidth - formattedTotalWidth
messages.push(pad('TOTAL', textWidth) + formattedTotal)
if (totalSaved > 0) {
  const formattedTotal = parseFloat(
    (Math.round(totalSaved * 100) / 100).toString()).toFixed(2)
  console.log(`formattedTotal: ${formattedTotal}`)
  const formattedTotalWidth = formattedTotal.length
  const textWidth = LineWidth - formattedTotalWidth
  messages.push(pad('*** You saved:', textWidth) + formattedTotal)
}
totalOfDiscountedItems =
  Math.round(totalOfDiscountedItems * 100) / 100
totalSaved = Math.round(totalSaved * 100) / 100
response.status = 200
// send total saved instead
response.send(
  { id: checkoutId,
    total,
    totalOfDiscountedItems,
    messages,
    totalSaved })
}

```

Capturing all the nuances and writing tests for all relevant cases might take a few hours. Also, there's a good possibility that we don't cover all the cases, which increases our risk of shipping defects.

Nắm bắt hết mọi sắc thái rồi viết test cho tất cả các trường hợp liên quan có thể tốn vài giờ. Mà nhiều khả năng ta vẫn không phủ hết mọi trường hợp, làm tăng rủi ro ship ra lỗi.

Small, Isolated Units Imply Simple Tests

Đơn vị nhỏ và biệt lập kéo theo test đơn giản

It is much easier to write small, simple tests as we **craft** the logic for assembling a checkout receipt.

No doubt you can spot a few of the bits of duplication in the midst of

Viết những test nhỏ và đơn giản dễ hơn nhiều khi ta gọt giữa logic lắp ráp một hoá đơn thanh toán. Chắc chắn bạn có thể chỉ ra vài mẫu trùng lặp nằm ngay giữa

postCheckoutTotal , as well as the number of opportunities for extracting conceptual units to new functions.

postCheckoutTotal , cũng như hàng loạt cơ hội trích các đơn vị khái niệm ra thành hàm mới.

Here's an improved solution, a work in progress.

Đây là một giải pháp cải tiến, vẫn đang trong quá trình hoàn thiện.

```
const sendSuccessResponse = (response, body) => {
  response.status = 200
  response.send(body)
}

const sendErrorResponse = (response, message) => {
  response.status = 400
  response.send({ error: message })
}

export const postCheckoutTotal = (request, response) => {
  const checkoutId = request.params.id
  const checkout = Checkouts.retrieve(checkoutId)
  if (!checkout)
    return sendErrorResponse(response, 'nonexistent checkout')
  const { totals, messages } = createReceipt(checkout)
  sendSuccessResponse(response, {
    id: checkoutId,
    messages,
    total: round2(totals.total),
    totalOfDiscountedItems: round2(totals.totalOfDiscountedItems),
    totalSaved: round2(totals.totalSaved)
  })
}
```

The function postCheckoutTotal , now a declaration of policy, can be covered with a couple of “end-to-end” tests.

Hàm postCheckoutTotal , giờ đã là một lời tuyên bố chính sách, có thể được phủ bằng đôi ba test “end-to-end”.

The module receipt.js exposes code that can be directly and easily unit tested:

Module receipt.js phơi ra phần mã có thể unit test một cách trực tiếp và dễ dàng:

```
import { pad } from '../util/stringutil'
import { calculateTotal,
  calculateTotalOfDiscountedItems,
  calculateTotalSaved } from './checkout-model'

const LineWidth = 45
const Indent = ` `
export const round2 = amount => Math.round(amount * 100) / 100
```

```

const formatAmount = amount => parseFloat(round2(amount)).toFixed(2)
const shouldApplyDiscount = (checkout, item) =>
  !item.exempt && memberDiscountPct(checkout) > 0
const calculateDiscountAmount = (checkout, item) =>
  memberDiscountPct(checkout) * item.price
const memberDiscountPct =
  checkout => checkout.member ? checkout.discount : 0
const lineItem = (text, lineAmount) => {
  const amount = formatAmount(lineAmount)
  const amountWidth = amount.length
  const textWidth = LineWidth - amountWidth
  return pad(text, textWidth) + amount
}

export const createReceiptMessages = (checkout, totals) => {
  const messages = []
  checkout.items.forEach(item => {
    messages.push(lineItem(item.description, item.price))
    if (shouldApplyDiscount(checkout, item)) {
      messages.push(lineItem(
        `${Indent}${memberDiscountPct(checkout) * 100}% mbr disc`,
        -calculateDiscountAmount(checkout, item)))
    }
  })
  messages.push(lineItem('TOTAL', round2(totals.total)))
  if (totals.totalSaved > 0)
    messages.push(lineItem('*** You saved:', totals.totalSaved))
  return messages
}

export const createReceipt = checkout => {
  const totals = ({
    total: calculateTotal(checkout),
    totalSaved: calculateTotalSaved(checkout),
    totalOfDiscountedItems: calculateTotalOfDiscountedItems(checko
  })
  return {
    totals,
    messages: createReceiptMessages(checkout, totals)
  }
}

```

By separating out receipt.js , we've also increased cohesion. The postCheckoutTotal function is primarily concerned with orchestrating the creation of a response to a request. It delegates the calculation specifics of assembling a receipt and totals to the receipt.js module.

Bằng việc tách riêng receipt.js , chúng tôi cũng đã tăng độ gắn kết. Hàm postCheckoutTotal giờ chủ yếu lo việc điều phối chuyện tạo ra phản hồi cho một request. Nó uỷ thác các chi tiết tính toán để lắp ráp hoá đơn và các khoản tổng cho module receipt.js .

The receipt.js still includes a few utility/helper functions that could be again moved, then tested in isolation (notably

receipt.js vẫn còn vài hàm tiện ích/hỗ trợ có thể dời đi lần nữa rồi test biệt lập (đáng chú

ý là

`round2 andformatAmount).`

Here's the final cohesive module, `checkout-model.js` :

Và đây là module gắn kết sau cùng, `checkout-model.js` :

```
export const calculateTotal = checkout => {
  let total = 0
  checkout.items.forEach(item => {
    if (!item.exempt && (checkout.member ? checkout.discount : 0) >
      total += item.price * 1.0 -
        (checkout.member ? checkout.discount : 0))
    Else
      total += item.price
  })
  return total
}
export const calculateTotalSaved = checkout => {
  let totalSaved = 0
  checkout.items.forEach(item => {
    if (!item.exempt && (checkout.member ? checkout.discount : 0) >
      totalSaved +=
        (checkout.member ? checkout.discount : 0) * item.price
  })
  return totalSaved
}
export const calculateTotalOfDiscountedItems = checkout =>
  checkout.items
    .filter(item => !item.exempt &&
      (checkout.member ? checkout.discount : 0) > 0)
    .reduce((total, item) => total + item.price *
      (1.0 - (checkout.member ? checkout.discount : 0)), 0)
```

Now that it is isolated, we can focus on improving clarity and conciseness within these calculate functions. We can put tests in place in short order. It took ten minutes to create these tests for `calculateTotal` :

Giờ khi nó đã đứng riêng, ta có thể tập trung cải thiện độ rõ ràng và súc tích bên trong các hàm calculate này. Ta cũng đặt được test vào chỗ chỉ trong chốc lát. Mất mười phút để tạo ra những test sau cho `calculateTotal` :

```
import {calculateTotal} from "../checkout-model";
describe('calculate total', () => {
  it('is zero when no items exist', () => {
    const checkout = {
      items: []
    }
    const result = calculateTotal(checkout)
    expect(result).toBe(0)
  })
  it('is price of single undiscounted item', () => {
```

```

const checkout = {
  items: [{ price: 42 }]
}
const result = calculateTotal(checkout)
expect(result).toBe(42)
})
it('discounts item when member attached & item not exempt', () => {
  const checkout = {
    member: {},
    discount: 0.1,
    items: [{
      exempt: false,
      price: 100
    }]
  }
  const result = calculateTotal(checkout)
  expect(result).toBe(90)
})
it('does not discount item when exempt', () => {
  const checkout = {
    member: {},
    discount: 0.1,
    items: [{
      exempt: true,
      price: 100
    }]
  }
  const result = calculateTotal(checkout)
  expect(result).toBe(100)
})
it('does not discount item when no member attached', () => {
  const checkout = {
    discount: 0.1,
    items: [{
      exempt: true,
      price: 100
    }]
  }
  const result = calculateTotal(checkout)
  expect(result).toBe(100)
})
it('totals discounted and non-discounted items', () => {
  const checkout = {
    discount: 0.1,
    member: {},
    items: [
      { exempt: true, price: 5 },
      { exempt: true, price: 10 },
      { exempt: false, price: 100 },
    ]
  }

```

```

    { exempt: false, price: 200 },
  ]
}
const result = calculateTotal(checkout)
expect(result).toBe(5 + 10 + 90 + 180)
})
})

```

The tests gave us the confidence to fearlessly refactor

Các test cho ta đủ tự tin để refactor không chút sợ hãi

calculateTotal to a clearer solution:

calculateTotal thành một giải pháp rõ ràng hơn:

```

const memberDiscountPct = checkout =>
  checkout.member ? checkout.discount : 0
const shouldApplyDiscount = (checkout, item) =>
  !item.exempt && memberDiscountPct(checkout) > 0
const calculateDiscountedPrice = (checkout, item) =>
  item.price * (1.0 - memberDiscountPct(checkout))
export const calculateTotal = checkout => {
  const totalDiscountable = checkout.items
    .filter(item => shouldApplyDiscount(checkout, item))
  reduce((total, item) => total +
    .reduce((total, item) => total +
      calculateDiscountedPrice(checkout, item), 0);
  const totalNonDiscountable = checkout.items
    .filter(item => !shouldApplyDiscount(checkout, item))
    .reduce((total, item) => total + item.price, 0);
  return totalDiscountable + totalNonDiscountable;
}

```

It looks like our **refactoring** increased the lines of code, for

Có vẻ đợt refactor vừa rồi làm số dòng mã tăng lên, ít nhất là với

calculateTotal at least. However, the overall number of lines in the module decreases by a handful after we made similar changes to the other calculation functions as well.

calculateTotal. Tuy nhiên, tổng số dòng của cả module lại giảm được một ít sau khi ta áp dụng những thay đổi tương tự cho các hàm tính toán còn lại.

When Should We Ensure Confirmability?

Khi nào ta nên bảo đảm tính xác nhận được?

Many developers prefer to write a significant chunk of code, then later come back and meet what they view as their obligation to achieve code coverage mandates. The mandate for code coverage—the measure of what percent of code is exercised by tests—becomes a self-fulfilling prophecy: We get what we ask for. If a development team is mandated a minimum coverage metric of 75%, we'll get that and no more.

Nhiều lập trình viên thích viết một mảng mã khá khá trước, rồi sau đó mới quay lại làm cho tròn thứ mà họ xem là nghĩa vụ: đạt chỉ tiêu độ phủ test (test coverage). Chỉ tiêu độ phủ mã — thước đo xem bao nhiêu phần trăm mã được test chạy qua — trở thành một

lời tiên tri tự ứng nghiệm: ta đòi gì thì được nấy. Nếu một đội phát triển bị áp chỉ tiêu tối thiểu 75%, ta sẽ nhận đúng 75%, không hơn.

Seventy-five percent coverage doesn't sound bad until you invert it: One-quarter of our system has insufficient unit tests. They're also often the more complex parts of our system: They're harder to test, and when meeting a metric we don't really believe in, we'll do it in the most expeditious way we can imagine.

Bảy mươi lăm phần trăm nghe cũng không đến nỗi, cho tới khi bạn lật ngược nó lại: một phần tư hệ thống của ta không có đủ unit test. Mà đó lại thường là những phần phức tạp hơn cả: chúng khó test hơn, và khi phải chạy theo một chỉ số mà bản thân ta chẳng thực sự tin, ta sẽ làm theo cách chống chế nhanh nhất nghĩ ra được.

The reason so much of our code is harder to test is because testing it wasn't a paramount concern while we were coding it. Our primary interest was to just get the damn thing working. As a result, the complex parts and even the not-so-complex parts are tougher to test. Our code is not easily confirmable.

Lý do khiến phần lớn mã của ta khó test là vì lúc viết nó, chuyện test không phải mối bận tâm hàng đầu. Mối quan tâm chính của ta chỉ là làm cho cái của nợ đó chạy được. Kết quả là những phần phức tạp — và cả những phần chẳng phức tạp gì mấy — đều khó test hơn. Mã của ta không dễ xác nhận.

Writing tests after the fact is harder because the existing code

Viết test khi mọi chuyện đã rồi thì khó hơn, bởi đoạn mã sẵn có

- Lacks clarity. It is overly complex, because it wasn't edited. It's tougher to figure out the basic question of what the functions do.
- Lacks cohesion. It becomes increasingly difficult to discern all the actual developer intents, because they aren't all in one place, and they aren't decoupled from other intents.
- Lacks conciseness. It contains excessive code, as well as excessively repeated constructs that must be retested in multiple contexts.
- Contains deep **dependency** chains. As a result, tests demand increased effort to set up the appropriate data/context. The existence of what are typically private dependencies means that we must reshape the code to allow tests to inject the dependencies.
 - Thiếu rõ ràng. Nó phức tạp quá mức, vì chẳng ai buồn chỉnh sửa. Trả lời câu hỏi cơ bản nhất — mấy hàm này làm gì — cũng đã khó.
 - Thiếu gắn kết. Càng lúc càng khó nhận ra hết mọi ý định thực sự của lập trình viên, bởi chúng không nằm gọn một chỗ và cũng không tách rời khỏi các ý định khác.
 - Thiếu súc tích. Nó chứa quá nhiều mã, cùng những cấu trúc lặp đi lặp lại quá mức mà ta phải test lại trong nhiều ngữ cảnh khác nhau.
 - Chứa những chuỗi phụ thuộc sâu. Hệ quả là test đòi hỏi nhiều công sức hơn để dựng dữ liệu/ngữ cảnh phù hợp. Vì các phụ thuộc (dependency) kiểu này thường là private, ta buộc phải nắn lại mã để test có thể tiêm chúng vào.

One more problem with that 75% or 80% or whatever code metric: The remaining portion of our code is a 20%+ blob of “who really knows what this stuff does.” Good tests not only provide us with the ability to edit code without fear, but they also document all the developer intents.

Còn một vấn đề nữa với con số 75% hay 80% hay bất cứ chỉ số nào: phần mã còn lại là một khối 20%+ mang tên “có trời mới biết đám này làm gì”. Test tốt không chỉ cho ta khả năng sửa mã mà không sợ hãi, chúng còn ghi lại tài liệu cho toàn bộ ý định của lập trình viên.

If writing tests later creates so many problems (or, more positively stated, doesn't help us sufficiently enough), the obvious answer is to start with tests. With each new behavior we must add, we write a test that captures that behavior. Our test's name summarizes that behavior (which helps us later find the behaviors we seek). Our test's code provides an example that pins down the design for how we effect that behavior.

Nếu viết test về sau gây ra lắm vấn đề đến thế (hoặc nói theo hướng tích cực hơn: chẳng giúp ta được bao nhiêu), thì câu trả lời hiển nhiên là hãy bắt đầu bằng test. Với mỗi hành vi mới cần thêm, ta viết một test nắm bắt hành vi đó. Tên test tóm tắt hành vi ấy (nhờ vậy về sau ta dễ tìm lại đúng hành vi mình cần). Mã của test cung cấp một ví dụ ghim chặt thiết kế về cách ta hiện thực hành vi đó.

Our code coverage becomes a self-fulfilling prophecy: We attain complete coverage on all the behaviors we needed and designed into the system. We get high confidence to continuously shape its design to be clear, cohesive, and concise. Our system is eminently confirmable.

Độ phủ mã của ta trở thành một lời tiên tri tự ứng nghiệm — lần này theo nghĩa tốt: ta đạt độ phủ trọn vẹn trên mọi hành vi mà ta cần và đã chủ đích thiết kế vào hệ thống. Ta có đủ tự tin để liên tục nắn thiết kế cho rõ ràng, gắn kết và súc tích. Hệ thống của ta xác nhận được đến từng ngóc ngách.

You might be wondering about what that coverage metric mandate should be. The right answer: (a) Mandates are a bad idea, and (b) the coverage number is never what's important. What's important is that developers are sold on the value of delivering what we know to be working software.

Có thể bạn đang thắc mắc vậy rốt cuộc chỉ tiêu độ phủ nên là bao nhiêu. Câu trả lời đúng: (a) áp chỉ tiêu là một ý tồi, và (b) con số độ phủ chưa bao giờ là thứ quan trọng. Điều quan trọng là lập trình viên phải thực sự tin vào giá trị của việc bàn giao thứ phần mềm mà ta biết chắc là chạy được.

Cohesion — Độ gắn kết nội tại (Cohesion)

It was infeasible to get to this last C criterion for design without previously mentioning cohesion. Our continuous design criteria necessarily intertwine to support each other, and at times they can be at odds with one another. Increasing clarity might require diminishing conciseness, for example.

Không thể nào đi tới chữ C cuối cùng trong bộ tiêu chí thiết kế mà lại chưa từng nhắc tới độ gắn kết. Các tiêu chí thiết kế liên tục tất yếu đan xen và nâng đỡ lẫn nhau, đôi lúc lại xung khắc với nhau. Chẳng hạn, tăng độ rõ ràng có khi đồng nghĩa với giảm độ súc tích.

But here we are, and cohesion deserves its own focused discussion.

Nhưng dù sao ta cũng đã tới đây rồi, và độ gắn kết xứng đáng có một phần bàn luận riêng.

We're building Czecher , a flash card application designed to help us ingrain challenging aspects of the Czech language. As a first step, we've created the module words.js to allow us to add new nouns to our collection of words to practice:

Chúng ta đang xây Czecher , một ứng dụng flash card được thiết kế để giúp ta khắc sâu những khía cạnh học búa của tiếng Séc. Ở bước đầu tiên, ta đã tạo module words.js để có thể thêm danh từ mới vào bộ sưu tập từ dùng để luyện tập:

```
import { retrieveWord } from '../prompts/languageClient'
const definitions = {}
export const clearWords = () =>
```

```

Object.keys(definitions).forEach(key => delete definitions[key])
export const loadDefinition = (word, definition) =>
  definitions[word] = { ...definition, word }
export const addWord = async word => {
  if (definitions[word]) return
  const definition = await retrieveWord(word)
  definitions[word] = { ...definition, word }
}
export const definition = word => definitions[word]
export const allDefinitions = () =>
  Object.values(definitions)

```

The addWord function takes an English word as an **argument** (e.g., “dog”). If a definition has already been loaded for the word, the function does nothing. Otherwise, addWord calls

Hàm addWord nhận một từ tiếng Anh làm đối số (ví dụ “dog”). Nếu định nghĩa của từ đó đã được nạp sẵn, hàm không làm gì cả. Ngược lại, addWord gọi

retrieveWord to look up information using a third-party resource (OpenAI in this case). The resulting definition is added to a local store. Calling the definition function with a word returns the detailed information.

retrieveWord để tra cứu thông tin qua một nguồn của bên thứ ba (ở đây là OpenAI). Định nghĩa thu được sẽ được thêm vào kho lưu trữ cục bộ. Gọi hàm definition với một từ sẽ trả về thông tin chi tiết.

Some tests might add a little color:

Vài test có thể tô thêm chút màu sắc:

```

import { when } from 'jest-when'
import { addWord, clearWords, definition,
  allDefinitions, loadDefinition }
  from './words'
import { retrieveWord } from '../prompts/languageClient'
jest.mock('../prompts/languageClient')
describe('addWord', () => {
  const orangeDefinition = {
    word: 'orange',
    gender: 'MI',
    singular: { nominative: 'pomeranč', accusative: 'pomeranč' },
    plural: { nominative: 'pomeranče', accusative: 'pomeranče' }
  }
  const dogDefinition = {
    word: 'dog',
    gender: 'MA',
    singular: { nominative: 'pes', accusative: 'psa' },
    plural: { nominative: 'psi', accusative: 'psy' }
  }
  beforeEach(() => {
    clearWords()
    jest.resetAllMocks()
  })

```

```

it('retrieves and saves definition on add', async () => {
  when(retrieveWord).calledWith('orange')
    .mockResolvedValueOnce(orangeDefinition)
  await addWord('orange')

  expect(definition('orange')).toEqual(orangeDefinition)
})
it('does nothing if already added', async () => {
  loadDefinition('orange', orangeDefinition)
  const word = definition('orange')
  expect(word).toEqual(orangeDefinition)
  expect(retrieveWord).not.toHaveBeenCalled()
})
const add = async (word, definition) => {
  when(retrieveWord).calledWith(word)
    .mockResolvedValueOnce(definition)
  await addWord(word)
}
it('persists multiple words', async () => {
  await add('orange', orangeDefinition)
  await add('dog', dogDefinition)
  const definitions = allDefinitions()
  expect(definitions).toEqual([orangeDefinition, dogDefinition])
})
})

```

The words module is our first incremental step toward an MVP. ³ As a quick measure, we chose to persist the words in an in-memory JavaScript object (`{}`), where the keys are the words and the values provide the language detail we need. The constant `orangeDefinition` provides one example.

Module words là bước tăng dần đầu tiên của ta hướng tới một MVP. ³ Để cho nhanh, ta chọn lưu các từ trong một object JavaScript nằm trong bộ nhớ (`{}`), trong đó key là các từ, còn value cung cấp thông tin ngôn ngữ mà ta cần. Hằng `orangeDefinition` là một ví dụ.

3 Minimum Viable Product.

3 Minimum Viable Product — sản phẩm khả dụng tối thiểu.

Here's our continuous design report card.

Đây là phiếu điểm thiết kế liên tục của ta.

- Clarity: Pass. All the operations appear in short, JavaScript-idiomatic functions, without any C-for- Cleverness (for which we would immediately get a failing mark).
- Confirmability: Pass. All code was driven into existence by behavioral tests.
- Conciseness: Pass. The code seems about as short as it could get.
- Cohesion: Fail. Wait, fail?
 - Rõ ràng (Clarity): Đạt. Mọi thao tác đều nằm trong các hàm ngắn, đậm chất JavaScript, không dính chút C nào của Cleverness — “khôn lỏi” (mà nếu có thì ta rớt ngay lập tức).
 - Xác nhận được (Confirmability): Đạt. Toàn bộ mã đều được các test hành vi dẫn dắt vào tồn tại.
 - Súc tích (Conciseness): Đạt. Mã có vẻ đã ngắn hết mức có thể.
 - Gắn kết (Cohesion): Trượt. Khoan, trượt á?

Our intent is to replace the JavaScript object with a proper persistence mechanism. Unfortunately, we've conflated our little bit of definition-management logic with logic supporting storage of the definitions. When we do change, we'll have to open up the words module and poke through most of its methods to update implementations.

Ý định của ta là thay object JavaScript bằng một cơ chế lưu trữ đáng hoàng. Đáng tiếc, ta đã trộn lẫn phần logic quản lý định nghĩa bé xiu của mình với logic lo việc lưu trữ chính các định nghĩa đó. Khi thay đổi, ta sẽ phải mở module words ra và chọc vào gần như mọi method của nó để cập nhật hiện thực.

You'll likely recognize that we're describing a Single Responsibility Principle (SRP) violation. SRP and cohesion both say the same thing, minus some nuances. Cohesion is how well the elements of a module align to the same purpose.

Hẳn bạn nhận ra ngay ta đang mô tả một vi phạm SRP — Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle). SRP và độ gắn kết nói cùng một điều, bỏ qua vài sắc thái nhỏ. Độ gắn kết là mức độ mà các thành phần trong một module cùng phục vụ một mục đích.

Had we adhered to a cohesive solution, we might have found it a bit easier to build the solution in the first place. We'd also have been ready to immediately accommodate changes, rather than have to refactor our code to support the change.

Nếu ngay từ đầu ta bám vào một giải pháp gắn kết, có lẽ dựng giải pháp ấy cũng đã dễ hơn đôi chút. Ta cũng sẵn sàng đón nhận thay đổi ngay lập tức, thay vì phải refactor — nắn lại mã trước — để đỡ được thay đổi đó.

We'll refactor the code now, to demonstrate how a cohesive solution increases the clarity of the code.

Giờ ta sẽ refactor đoạn mã, để thấy một giải pháp gắn kết làm tăng độ rõ ràng của mã như thế nào.

```
import { retrieveWord } from '../prompts/languageClient'
import * as Data from '../persistence/database'
export const clearWords = () =>
  Data.deleteAll()
export const loadDefinition = (word, definition) =>
  Data.add(word, definition)
export const addWord = async word => {
  if (Data.containsKey(word)) return
  const definition = await retrieveWord(word)
  Data.add(word, { word, ...definition })
}
export const definition = word => Data.get(word)
export const allDefinitions = () => Data.allValues()
```

The implementation specifics around JavaScript objects have been replaced with abstractions: add, get, allValues,

Các chi tiết hiện thực xoay quanh object JavaScript đã được thay bằng những trừu tượng: add, get, allValues,

containsKey, and deleteAll. The underlying implementation could change from an object, to a list, to interaction with a third-party key/value store or relational database. The now-cohesive words module wouldn't be impacted by these changes.

containsKey , và deleteAll . Hiện thực bên dưới có thể đổi từ một object sang một list, rồi sang việc tương tác với một kho key/value của bên thứ ba hay một cơ sở dữ liệu quan hệ. Module words giờ đã gắn kết sẽ không bị những thay đổi đó tác động tới.

The module database.js provides an **interface** that abstracts the notion for persisting data, with nothing exported that betrays the underlying **data structure**. Future changes to its implementation don't impact other code in the system.

Module database.js cung cấp một interface trừu tượng hoá khái niệm lưu trữ dữ liệu, không export bất cứ thứ gì để lộ cấu trúc dữ liệu (data structure) bên dưới. Những thay đổi hiện thực trong tương lai sẽ không ảnh hưởng tới phần mã còn lại của hệ thống.

```
const data = {}
export const containsKey = key => data[key]
export const deleteAll = () =>
  Object.keys(data).forEach(key => delete data[key])
export const add = (key, value) =>
  data[key] = value
export const get = word => data[word]
export const allValues = () => Object.values(data)
```

When Else Do We Design? — Còn khi nào nữa thì ta thiết kế?

With a continuous design mentality, we seek to keep costs low by reviewing our code's adherence to the four Cs, a collection of code criteria that we can continuously consult. When do we do that? Why, continuously of course, or at least continually.

Với não trạng thiết kế liên tục, ta tìm cách giữ chi phí ở mức thấp bằng việc rà lại xem mã của mình có bám sát bốn chữ C hay không — một bộ tiêu chí về mã mà ta có thể tra cứu liên tục. Khi nào thì làm chuyện đó? Thì liên tục chứ sao nữa, hoặc chí ít cũng phải thường xuyên.

But we don't only consider design when we're writing production code. It's a part of virtually every step of software development.

Nhưng ta không chỉ nghĩ tới thiết kế lúc đang viết mã production. Nó là một phần của gần như mọi bước trong phát triển phần mềm.

Up-front Design — Thiết kế từ đầu (Up-front Design)

We initiate a project by determining what new capabilities our product will bring to its users. Necessarily this requires some level of understanding about the design of the system —what it accomplishes and what its constraints are, for example.

Ta khởi động một dự án bằng việc xác định sản phẩm sẽ mang lại những năng lực mới nào cho người dùng. Điều này tất yếu đòi hỏi một mức hiểu biết nhất định về thiết kế của hệ thống — chẳng hạn nó làm được gì và bị ràng buộc bởi những gì.

As part of answering our analysis and design efforts, we might first produce summary diagrams that capture salient elements of the system's current design. We would then also create speculative sketches for what the design will need to look like in order to accommodate the desired new behaviors.

Trên đường tìm lời giải cho công việc phân tích và thiết kế, trước tiên ta có thể vẽ ra vài sơ đồ tóm lược nắm bắt những phần nổi bật trong thiết kế hiện tại của hệ thống. Sau đó ta phác thảo mang tính phỏng đoán xem thiết kế sẽ phải trông ra sao để đỡ được các hành vi mới mà ta mong muốn.

Stakeholders usually want to know when the project will be done at this point in the planning process. Estimates are a design consideration: Not only must we have a sense of how much time it would take to develop a new feature, but we also must know the extent to which the current design resists the new feature. Ideally, the amount of extra effort needed is “zero,” which can be true if we’ve adhered to the four Cs well enough.

Ở giai đoạn lập kế hoạch này, các bên liên quan (stakeholder) thường muốn biết bao giờ dự án xong. Ước lượng (estimate) là một yếu tố thuộc về thiết kế: ta không chỉ cần cảm nhận được sẽ mất bao lâu để phát triển tính năng mới, mà còn phải biết thiết kế hiện tại kháng cự tính năng đó tới mức nào. Lý tưởng nhất, lượng công sức phát sinh thêm là “không” — điều hoàn toàn có thể đạt được nếu ta bám bốn chữ C đủ tốt.

The estimates themselves might feed back into the project plan. The powers that be might decide to trim their wishlist due to unexpectedly high estimates. Given these changes to their desired **scope**, we reconsider the design and provide updated estimates.

Bản thân các ước lượng có thể phản hồi ngược vào kế hoạch dự án. Các đồng僚 trên có thể quyết định cắt bớt danh sách mong muốn vì ước lượng cao ngoài dự tính. Trước những thay đổi về phạm vi ấy, ta xem lại thiết kế và đưa ra ước lượng cập nhật.

Our project estimates at this time cover larger initiatives that might take weeks or months to complete. The estimates are high level as a result; we invest enough time in them to get a ballpark sense of the initiative. We might use relative sizes—small, medium, large, extra-large—to figure out which ones we can guarantee we’ll deliver in the next quarter.

Ước lượng dự án ở thời điểm này bao trùm những sáng kiến lớn, có thể mất hàng tuần hay hàng tháng mới xong. Vì vậy chúng ở mức tổng quan; ta chỉ đầu tư vừa đủ thời gian để có được cảm nhận áng chừng về sáng kiến đó. Ta có thể dùng cỡ tương đối — nhỏ, vừa, lớn, cực lớn — để xác định cái nào chắc chắn giao được trong quý tới.

As we ready for development, we seek to slice these larger challenges into thinner pieces of work that we can start and deliver within shorter periods of time (perhaps hours or days). An understanding of the system’s design is once again required at this point, to ensure that our ideas around slicing up work align with the realities of the system.

Khi đã sẵn sàng bước vào phát triển, ta tìm cách xẻ những thách thức lớn ấy thành các mảnh việc mỏng hơn, có thể bắt đầu và hoàn tất trong khoảng thời gian ngắn hơn (có lẽ vài giờ hoặc vài ngày). Tại điểm này, một lần nữa ta lại cần hiểu thiết kế của hệ thống, để chắc rằng ý tưởng cắt lát công việc của mình khớp với thực tế của hệ thống.

Readying for Work — Chuẩn bị vào việc

As we get ready to code a feature, the product folk (product owners, product managers, business analysts, UX designers, and others) are working to ensure that they’ve captured all the actual specifics needed for us to get started. When they meet with us to describe the new feature, we ask questions, and we dig a bit again into the system’s design.

Khi ta chuẩn bị code một tính năng, những người bên sản phẩm (product owner, product manager, business analyst, UX designer và nhiều vai trò khác) đang làm việc để bảo đảm

họ đã gom đủ mọi chi tiết thực tế cần cho ta bắt đầu. Lúc họ gặp ta để mô tả tính năng mới, ta đặt câu hỏi, và ta lại đào thêm một chút vào thiết kế của hệ thống.

Sometimes we uncover the devil when the product people reveal the details for the feature. We discover that things aren't as ideal as we'd speculated earlier. It's also possible that the system has changed enough in the interim to make things more difficult.

Đôi khi con quỷ chỉ lộ mặt lúc những người làm sản phẩm hé lộ chi tiết của tính năng. Ta phát hiện mọi thứ chẳng lý tưởng như mình phỏng đoán trước đó. Cũng có khả năng trong quãng thời gian ở giữa, hệ thống đã thay đổi đủ nhiều để làm mọi chuyện khó hơn.

Sometimes answers to our questions reveal hidden complexity. “Oh, I didn't think that bell or whistle would be a big deal,” they might tell us. It might not be, from their perspective; from ours, it might be something that will be very difficult to fit into the existing design.

Đôi khi lời đáp cho câu hỏi của ta lại phơi ra độ phức tạp ẩn giấu. “Ồ, tôi đâu nghĩ cái thứ râu ria đó lại là chuyện to tát,” họ có thể bảo vậy. Từ góc nhìn của họ thì có lẽ đúng là không to tát; từ góc nhìn của ta, nó có thể là thứ cực kỳ khó nhét vừa vào thiết kế hiện có.

Starting Work — Bắt tay vào việc

We break down our work first into smaller behavioral units. If our work is to add support for adverbs to the Czecher app, we might break that effort into the following smaller slices.

Trước hết ta chia nhỏ công việc thành các đơn vị hành vi bé hơn. Nếu việc của ta là bổ sung hỗ trợ trạng từ cho ứng dụng Czecher, ta có thể tách nỗ lực đó thành những lát nhỏ sau.

- Support adding a single adverb to the word list.
- Bulk-load a CSV file of adverbs.
 - Hỗ trợ thêm một trạng từ đơn lẻ vào danh sách từ.
 - Nạp hàng loạt một file CSV chứa trạng từ.
- Add flash card examples containing adverbs.
 - Thêm các ví dụ flash card có chứa trạng từ.

Those slices might still be a little too large. We might split “support adding an adverb” into a few technical challenges.

Những lát ấy có lẽ vẫn hơi lớn. Ta có thể chẻ “hỗ trợ thêm một trạng từ” thành vài thách thức kỹ thuật.

- Create and incorporate LLM prompt text to specify the format for adverbs.
- Add persistence support for adverbs.
- Rework card building logic to provide examples involving adverbs.
 - Tạo và tích hợp đoạn prompt cho LLM để quy định định dạng cho trạng từ.
 - Thêm hỗ trợ lưu trữ cho trạng từ.
 - Chỉnh lại logic dựng thẻ để đưa ra ví dụ có trạng từ.

Such decomposition necessarily involves an awareness of design, and may also spur a discussion around how the current and/or speculative design must change.

Kiểu phân rã như vậy tất yếu đòi hỏi ý thức về thiết kế, và cũng có thể châm ngòi cho một cuộc thảo luận xem thiết kế hiện tại và/hoặc thiết kế phỏng đoán phải thay đổi ra sao.

Doing Work — Làm việc

We write tests to drive in each unit behavior. A **unit test** describes the behavior and provides a “live” example demonstrating how clients interact with the system to effect that behavior. The test, as a result, is our primary mechanism for designing and documenting the interface to the system.

Ta viết test để dẫn dắt từng hành vi đơn vị vào tồn tại. Một unit test mô tả hành vi và cung cấp một ví dụ “sống” minh họa cách client tương tác với hệ thống để tạo ra hành vi đó. Nhờ vậy, test chính là cơ chế chủ yếu để ta thiết kế và ghi lại tài liệu cho interface của hệ thống.

For each test we write, we code the behavior that we hope realizes the need specified in the test. We run our tests, correcting the code (i.e., the design) until the tests pass. We revisit the coded behavior, and we edit it for clarity, conciseness, and cohesion before moving on to the next test.

Với mỗi test viết ra, ta code phần hành vi mà ta hy vọng sẽ đáp ứng nhu cầu test đã nêu. Ta chạy test, sửa mã (tức là sửa thiết kế) cho tới khi test pass. Ta quay lại đoạn hành vi vừa code và chỉnh sửa nó cho rõ ràng, súc tích và gắn kết trước khi chuyển sang test tiếp theo.

We ship when we’ve written tests for all the behaviors, as well as gotten all the tests to pass.

Ta ship khi đã viết test cho mọi hành vi, và mọi test đều pass.

Design is omnipresent.

Thiết kế hiện diện ở khắp mọi nơi.

Chapter 22. Concurrency — Chương 22. Đồng thời

By Brett L. Schuchert

Tác giả: Brett L. Schuchert



“Objects are abstractions of processing. Threads are abstractions of schedule.” —James O. Coplien 1

“Object là sự trừu tượng hoá của xử lý. Thread là sự trừu tượng hoá của lịch trình.” —James O. Coplien 1

1 For example, [PPP02], <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod> , and [https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)) (or just google “SOLID”).

1 Ví dụ, [PPP02], <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod> , và [https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)) (hoặc cứ google “SOLID”).

Future Bob: Everything in this chapter applies equally to services and microservices.

Bob của tương lai: Mọi điều trong chương này đều áp dụng y hệt cho service và microservice.

Writing clean concurrent programs is hard—very hard. It is much easier to write code that executes in a single **thread**. It is also easy to write multithreaded code that looks fine on the surface but is broken at a deeper level. Such code works fine until the system is placed under stress.

Viết chương trình đồng thời (concurrency) sạch sẽ là chuyện khó — rất khó. Viết mã chạy trên một luồng (thread) duy nhất dễ hơn nhiều. Và cũng dễ không kém là viết mã đa luồng trông thì ổn ở bề mặt nhưng hỏng ở tầng sâu. Loại mã đó chạy ngon lành cho tới khi hệ thống bị đặt dưới áp lực.

In this chapter, we discuss the need for concurrent programming and the difficulties it presents. We then present several recommendations for dealing with those difficulties and writing clean concurrent code. Finally, we conclude with issues related to testing concurrent code.

Trong chương này, chúng ta bàn về nhu cầu lập trình đồng thời và những khó khăn mà nó đặt ra. Sau đó chúng tôi đưa ra vài khuyến nghị để đối phó với những khó khăn ấy và viết mã đồng thời sạch. Cuối cùng, chúng ta khép lại bằng các vấn đề liên quan đến việc test mã đồng thời.

Clean **concurrency** is a complex topic, worthy of a book by itself. Our strategy in this chapter is to present an overview.

Tính đồng thời sạch sẽ là một chủ đề phức tạp, xứng đáng có hẳn một cuốn sách riêng. Chiến lược của chúng tôi trong chương này chỉ là phác ra một cái nhìn tổng quan.

Why Concurrency?

Tại sao lại cần đồng thời?

Concurrency is a **decoupling** strategy. It helps us decouple what gets done from when it gets done. In single-threaded applications, what and when are so strongly coupled that the state of the entire application can often be determined by looking at the stack backtrace. A programmer who debugs such a system can set a breakpoint, or a sequence of breakpoints, and know the state of the system by which breakpoints are hit.

Đồng thời là một chiến lược tách rời (decoupling). Nó giúp ta tách *cái gì được làm khỏi khi nào nó được làm*. Trong các ứng dụng đơn luồng, *cái gì* và *khi nào* dính chặt vào nhau đến mức trạng thái của toàn bộ ứng dụng thường có thể xác định được chỉ bằng cách nhìn vào stack backtrace. Lập trình viên gỡ lỗi một hệ thống như vậy có thể đặt một breakpoint, hoặc một chuỗi breakpoint, rồi biết được trạng thái hệ thống qua việc breakpoint nào bị chạm tới.

Decoupling what from when can dramatically improve both the throughput and **structure** of an application. From a structural point of view, the application looks like many little collaborating computers rather than one big main loop. This can make the system easier to understand and offers some powerful ways to separate concerns.

Tách *cái gì* khỏi *khi nào* có thể cải thiện đáng kể cả thông lượng lẫn cấu trúc của một ứng dụng. Xét về mặt cấu trúc, ứng dụng trông giống một tập hợp nhiều máy tính nhỏ hợp tác với nhau hơn là một vòng lặp chính khổng lồ. Điều này có thể khiến hệ thống dễ hiểu hơn và mở ra vài cách rất mạnh để phân tách mối quan tâm.

Consider, for example, the standard “servlet” model of Web applications. These systems run under the umbrella of a Web server that partially manages concurrency for you. The servlets are executed asynchronously whenever Web requests come in. The servlet programmer does not have to manage all the incoming requests. In principle, each servlet execution lives in its own little world and is decoupled from all the other servlet executions.

Chẳng hạn, hãy xét mô hình “servlet” tiêu chuẩn của các ứng dụng Web. Các hệ thống này chạy dưới sự che chở của một Web server, nơi đã lo giùm bạn một phần chuyện đồng thời. Servlet được thực thi bất đồng bộ mỗi khi có request Web đi tới. Lập trình viên servlet không phải tự quản lý toàn bộ các request đến. Về *nguyên tắc*, mỗi lần thực thi servlet sống trong thế giới nhỏ của riêng nó và tách rời khỏi mọi lần thực thi servlet khác.

Of course, if it were that easy, this chapter wouldn’t be necessary. In fact, the decoupling provided by Web servers is far less than perfect. Servlet programmers have to be very aware, and very careful, to make sure their concurrent programs are correct. Still, the structural benefits of the servlet model are significant.

Tất nhiên, nếu mọi chuyện dễ đến thế thì chương này đã chẳng cần tồn tại. Thực tế, mức độ tách rời mà Web server cung cấp còn xa mới hoàn hảo. Lập trình viên servlet phải rất tỉnh táo, rất cẩn thận, để bảo đảm chương trình đồng thời của mình là đúng đắn. Dù vậy, lợi ích về mặt cấu trúc của mô hình servlet là rất đáng kể.

But structure is not the only motive for adopting concurrency. Some systems have response time and throughput constraints that require hand-coded concurrent solutions.

Nhưng cấu trúc không phải động cơ duy nhất để chọn tính đồng thời. Một số hệ thống có ràng buộc về thời gian phản hồi và thông lượng đến mức buộc phải có giải pháp đồng thời viết tay.

For example, consider a single-threaded information aggregator that acquires information from many different websites and merges that information into a daily summary. Because this system is single threaded, it hits each website in turn, always finishing one before starting the next. The daily run needs to be executed in less than 24 hours. However, as more and more websites are added, the time grows until it takes more than 24 hours to gather all the **data**. The single thread involves a lot of waiting at Web sockets for IO to complete. We could improve the performance by using a multithreaded algorithm that hits more than one website at a time.

Ví dụ, hãy hình dung một bộ tổng hợp thông tin đơn luồng, thu thập thông tin từ rất nhiều website khác nhau rồi gộp lại thành một bản tóm tắt hằng ngày. Vì hệ thống này chạy đơn luồng, nó ghé từng website một, luôn xong cái này rồi mới bắt đầu cái kia. Lướt chạy hằng ngày cần hoàn tất trong vòng chưa tới 24 giờ. Thế nhưng càng thêm nhiều website, thời gian càng phình ra cho tới lúc phải mất hơn 24 giờ mới gom đủ dữ liệu. Luồng đơn ấy dành rất nhiều thời gian chỉ để ngồi chờ IO hoàn tất trên các Web socket. Ta có thể cải thiện hiệu năng bằng một thuật toán đa luồng ghé nhiều website cùng lúc.

Or consider a system that handles one user at a time and requires only one second of time per user. This system is fairly responsive for a few users, but as the number of users increases, the system’s response time increases. No user wants to get in line behind 150 others! We could improve the response time of this system by handling many users concurrently.

Hoặc hãy xét một hệ thống phục vụ mỗi lần một người dùng và chỉ cần một giây cho mỗi người. Hệ thống này phản hồi khá nhanh khi có vài người dùng, nhưng khi số người dùng tăng lên, thời gian phản hồi của hệ thống cũng tăng theo. Chẳng ai muốn xếp hàng sau 150 người khác cả! Ta có thể cải thiện thời gian phản hồi của hệ thống này bằng cách phục vụ nhiều người dùng đồng thời.

Or consider a system that interprets large data sets but can only give a complete solution after processing all of them. Perhaps each data set could be processed on a different computer so that many data sets are being processed in parallel.

Hoặc hãy xét một hệ thống diễn giải những tập dữ liệu lớn nhưng chỉ đưa ra lời giải hoàn

chính sau khi đã xử lý hết tất cả. Có lẽ mỗi tập dữ liệu có thể được xử lý trên một máy tính khác nhau, để nhiều tập dữ liệu được xử lý song song.

Myths and Misconceptions

Những huyền thoại và ngộ nhận

And so there are compelling reasons to adopt concurrency. However, as we said before, concurrency is hard. If you aren't very careful, you can create some very nasty situations. Consider these common myths and misconceptions.

Vậy là có những lý do rất thuyết phục để chọn tính đồng thời. Tuy nhiên, như đã nói ở trên, đồng thời là chuyện *khó*. Nếu không thật cẩn thận, bạn có thể tạo ra vài tình huống cực kỳ khó chịu. Hãy xem những huyền thoại và ngộ nhận phổ biến sau đây.

- Concurrency always improves performance. Concurrency can sometimes improve performance, but only when there is a lot of wait time that can be shared between multiple threads or multiple processors. Neither situation is trivial.
- Design does not change when writing concurrent programs. In fact, the design of a concurrent algorithm can be remarkably different from the design of a single-threaded system. The decoupling of what from when usually has a huge effect on the structure of the system.
- Understanding concurrency issues is not important when working with a Web server. In fact, you'd better know just what your server is doing and how to guard against the issues of concurrent update and **deadlock**, described later in this chapter.
 - Đồng thời luôn cải thiện hiệu năng. Đồng thời đôi khi *có thể* cải thiện hiệu năng, nhưng chỉ khi có nhiều thời gian chờ mà nhiều luồng hoặc nhiều bộ xử lý có thể chia nhau. Cả hai tình huống ấy đều không hề tầm thường.
 - Thiết kế không thay đổi khi viết chương trình đồng thời. Thực tế, thiết kế của một thuật toán đồng thời có thể khác một trời một vực so với thiết kế của hệ thống đơn luồng. Việc tách *cái gì* khỏi *khi nào* thường có ảnh hưởng rất lớn tới cấu trúc của hệ thống.
 - Hiểu các vấn đề đồng thời là không quan trọng khi làm việc với một Web server. Thực tế, tốt hơn hết là bạn nên biết chính xác server của mình đang làm gì và làm sao để phòng các vấn đề cập nhật đồng thời và deadlock, sẽ được mô tả ở phần sau chương này.

Here are a few more balanced sound bites regarding writing concurrent software.

Dưới đây là vài phát biểu cân bằng hơn về việc viết phần mềm đồng thời.

- Concurrency incurs some overhead, both in performance as well as writing additional code.
- Correct concurrency is complex, even for simple problems.
- Concurrency bugs aren't usually repeatable, so they are often ignored as one-offs 2 instead of actual defects.
 - Đồng thời phải trả một khoản phí tổn, cả về hiệu năng lẫn lượng mã phải viết thêm.
 - Đồng thời cho đúng là chuyện phức tạp, ngay cả với những bài toán đơn giản.
 - Bug về đồng thời thường không lặp lại được, nên chúng hay bị bỏ qua như những sự cố ngẫu nhiên 2 thay vì được coi là lỗi (defect) thật sự.

2 Cosmic rays, glitches, and so on.

2 Tia vũ trụ, trục trặc vặt, vân vân.

- Concurrency often requires a fundamental change in design strategy.
 - Đồng thời thường đòi hỏi một thay đổi căn bản trong chiến lược thiết kế.

Challenges

Những thách thức

What makes concurrent programming so difficult? Consider the following trivial **class**:

Điều gì khiến lập trình đồng thời khó đến vậy? Hãy xét class tầm thường sau:

```
public class X {  
    private int lastIdUsed;  
  
    public int getNextId() {  
        return ++lastIdUsed;  
    }  
}
```

Let's say we create an instance of X , set the lastIdUsed field to 42 , and then share the instance between two threads. Now suppose that both of those threads call the **method** getNextId() ; there are three possible outcomes:

Giả sử ta tạo một instance của X , gán trường lastIdUsed bằng 42 , rồi chia sẻ instance đó giữa hai luồng. Bây giờ giả sử cả hai luồng đều gọi method getNextId() ; có ba kết cục khả dĩ:

1. Thread one gets the value 43 , thread two gets the value 44 , lastIdUsed is 44
2. Thread one gets the value 44 , thread two gets the value 43 , lastIdUsed is 44
3. Thread one gets the value 43 , thread two gets the value 43 , lastIdUsed is 43 The surprising third result occurs when the two threads step on each other. This happens because there are many possible paths that the two threads can take through that one line of code, and some of those paths generate incorrect results. How many different paths are there? To really answer that question, we need to understand what the compiler does with the generated bytecode, and understand what the Java memory model considers to be atomic.

1. Luồng một nhận giá trị 43 , luồng hai nhận giá trị 44 , lastIdUsed là 44
2. Luồng một nhận giá trị 44 , luồng hai nhận giá trị 43 , lastIdUsed là 44
3. Luồng một nhận giá trị 43 , luồng hai cũng nhận giá trị 43 , lastIdUsed là 43 Kết quả thứ ba gây bất ngờ ấy xảy ra khi hai luồng giẫm chân lên nhau. Chuyện này xảy ra vì có rất nhiều đường đi khả dĩ mà hai luồng có thể men theo xuyên qua đúng một dòng mã đó, và một số đường đi trong đó sinh ra kết quả sai. Có bao nhiêu đường đi khác nhau? Để trả lời thật sự câu hỏi ấy, ta cần hiểu trình biên dịch làm gì với bytecode được sinh ra, và hiểu mô hình bộ nhớ của Java coi cái gì là nguyên tử (atomic).

A quick answer, working with just the generated bytecode, is that there are 12,870 different possible execution paths for those two threads executing within the getNextId method. If the type of lastIdUsed is changed from int to long , the number of possible paths increases to 2,704,156. Of course, most of those paths generate valid results. The problem is that some of them don't .

Câu trả lời nhanh, chỉ xét trên bytecode được sinh ra, là có 12.870 đường thực thi khả dĩ khác nhau cho hai luồng cùng chạy bên trong method getNextId . Nếu kiểu của lastIdUsed đổi từ int sang long , số đường đi khả dĩ tăng lên 2.704.156. Tất nhiên, phần lớn những đường đi ấy cho ra kết quả hợp lệ. Vấn đề là một số thì *không* .

Concurrency Defense Principles

Các nguyên tắc phòng thủ trước tính đồng thời

What follows is a series of principles and techniques for defending your systems from the problems of concurrent code.

Sau đây là một loạt nguyên tắc và kỹ thuật để bảo vệ hệ thống của bạn khỏi những vấn đề của mã đồng thời.

Single Responsibility Principle

Nguyên tắc Trách nhiệm Đơn nhất

The SRP states that a given method/class/**component** should have a single reason to change. Concurrency design is complex enough to be a reason to change in its own right and therefore deserves to be separated from the rest of the code. Unfortunately, it is all too common for concurrency **implementation** details to be embedded directly into other production code. Here are a few things to consider.

SRP — Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle) phát biểu rằng một method/class/component chỉ nên có một lý do duy nhất để thay đổi. Thiết kế cho tính đồng thời đủ phức tạp để tự nó là một lý do thay đổi, và vì thế xứng đáng được tách khỏi phần mã còn lại. Đáng tiếc, việc nhét thẳng chi tiết hiện thực (implementation) của đồng thời vào giữa mã sản phẩm khác lại quá phổ biến. Sau đây là vài điều đáng cân nhắc.

- Concurrency-related code has its own life cycle of development, change, and tuning.
- Concurrency-related code has its own challenges, which are different from and often more difficult than nonconcurrency-related code.
- The number of ways in which miswritten concurrency-based code can fail makes it challenging enough without the added burden of surrounding application code.
 - Mã liên quan tới đồng thời có vòng đời phát triển, thay đổi và tinh chỉnh riêng của nó.
 - Mã liên quan tới đồng thời có những thách thức riêng, khác biệt và thường khó hơn nhiều so với mã không liên quan tới đồng thời.
 - Riêng số cách mà mã đồng thời viết sai có thể hỏng đã đủ khiến nó khó nhằn rồi, không cần thêm gánh nặng của đám mã ứng dụng vây quanh.

Recommendation: Keep your concurrency-related code separate from other code.

Khuyến nghị: Hãy giữ mã liên quan tới đồng thời tách biệt khỏi phần mã còn lại.

Corollary: Limit the **Scope** of Data

Hệ quả: Hạn chế phạm vi của dữ liệu

As we saw, two threads modifying the same field of a shared object can interfere with each other, causing unexpected **behavior**. One solution is to use something like Java's synchronized keyword to protect a critical section in the code that uses the shared object. It is important to restrict the number of such critical sections. The more places **shared data** can get updated, the more likely

Như ta đã thấy, hai luồng cùng sửa một trường của một đối tượng (object) dùng chung có thể can thiệp lẫn nhau, gây ra hành vi ngoài ý muốn. Một giải pháp là dùng thứ gì đó như từ khoá synchronized của Java để bảo vệ vùng tới hạn trong đoạn mã sử dụng đối tượng dùng chung ấy. Điều quan trọng là hạn chế số lượng những vùng tới hạn như vậy. Càng nhiều nơi có thể cập nhật dữ liệu dùng chung thì càng nhiều khả năng

- You will forget to protect one or more of those places—effectively breaking all code that modifies that shared data.
- There will be **duplication** of effort required to make sure everything is effectively guarded (a violation of **DRY 3**).
 - Bạn sẽ quên bảo vệ một hoặc vài nơi trong số đó — và thế là làm hỏng toàn bộ mã có sửa đổi dữ liệu dùng chung ấy.

- Sẽ có sự trùng lặp (duplication) công sức để bảo đảm mọi thứ đều được canh giữ đàng hoàng (vi phạm DRY 3).

3 [Prag].

3 [Prag].

- It will be difficult to determine the source of failures, which are already hard enough to find.
- Sẽ rất khó xác định nguồn cơn của các sự cố, vốn dĩ đã đủ khó tìm rồi.

Recommendation: Take data **encapsulation** to heart; severely limit the access of any data that may be shared.

Khuyến nghị: Hãy khắc cốt ghi tâm chuyện đóng gói (encapsulation) dữ liệu; hạn chế thật gắt gao quyền truy cập vào bất kỳ dữ liệu nào có thể bị dùng chung.

Future Uncle Bob: Consider using a functional language, or at least a functional style, when writing concurrent modules.

Uncle Bob của tương lai: Hãy cân nhắc dùng một ngôn ngữ hàm, hoặc chí ít là một phong cách hàm, khi viết các module đồng thời.

Corollary: Use Copies of Data

Hệ quả: Hãy dùng bản sao của dữ liệu

A **good** way to avoid shared data is to avoid sharing the data in the first place. In some situations, it is possible to copy objects and treat them as read-only. In other cases, it might be possible to copy objects, collect results from multiple threads in these copies, and then merge the results in a single thread.

Một cách hay để tránh dữ liệu dùng chung là ngay từ đầu đừng dùng chung dữ liệu. Trong vài tình huống, ta có thể sao chép đối tượng và đối xử với chúng như chỉ-đọc. Trong các trường hợp khác, có thể sao chép đối tượng, thu kết quả từ nhiều luồng vào những bản sao ấy, rồi gộp kết quả lại trong một luồng duy nhất.

If there is an easy way to avoid sharing objects, the resulting code will be far less likely to cause problems. You might be concerned about the cost of all the extra object creation. It is worth experimenting to find out if this is, in fact, a problem. However, if using copies of objects allows the code to avoid synchronizing, the savings in avoiding the intrinsic **lock** will likely make up for the additional creation and garbage collection overhead.

Nếu có cách dễ dàng để tránh dùng chung đối tượng, đoạn mã kết quả sẽ ít có khả năng gây rắc rối hơn hẳn. Có thể bạn lo về cái giá của toàn bộ số đối tượng tạo thêm ấy. Đáng để thử nghiệm xem liệu đó có thật sự là vấn đề hay không. Tuy nhiên, nếu việc dùng bản sao của đối tượng cho phép mã né được chuyện đồng bộ hoá (synchronization), thì khoản tiết kiệm nhờ tránh được lock nội tại nhiều khả năng sẽ bù lại phần chi phí tạo đối tượng và thu gom rác tăng thêm.

Corollary: Threads Should Be as Independent as Possible

Hệ quả: Các luồng nên độc lập nhất có thể

Consider writing your threaded code such that each thread exists in its own world, sharing no data with any other thread. Each thread processes one client request, with all of its required data coming from an unshared source and stored as local variables. This makes each of those threads behave as if it were the only thread in the world and there were no **synchronization** requirements.

Hãy cân nhắc viết mã đa luồng sao cho mỗi luồng tồn tại trong thế giới riêng của nó, không dùng chung dữ liệu với bất kỳ luồng nào khác. Mỗi luồng xử lý một request của client, với toàn bộ dữ liệu cần thiết đến từ một nguồn không dùng chung và được lưu dưới dạng biến cục bộ. Điều này khiến từng luồng hành xử như thể nó là luồng duy nhất trên đời và chẳng có yêu cầu đồng bộ hoá nào cả.

For example, classes that subclass from `HttpServlet` receive all of their information as parameters passed in to the `doGet` and `doPost` methods. This makes each Servlet act as if it has its own machine. So long as the code in the Servlet uses only local variables, there is no chance that the Servlet will cause synchronization problems. Of course, most applications using Servlets eventually run into shared resources such as database connections.

Ví dụ, các class kế thừa từ `HttpServlet` nhận toàn bộ thông tin của chúng dưới dạng tham số truyền vào các method `doGet` và `doPost`. Điều này khiến mỗi Servlet hành xử như thể nó có máy riêng. Chẳng nào mã trong Servlet chỉ dùng biến cục bộ thì không có cơ hội nào để Servlet gây ra vấn đề đồng bộ hoá. Tất nhiên, hầu hết ứng dụng dùng Servlet rất cuộc đều đụng phải tài nguyên dùng chung, chẳng hạn kết nối cơ sở dữ liệu.

Recommendation: Attempt to partition data into independent subsets that can be operated on by independent threads, possibly in different processors.

Khuyến nghị: Hãy cố chia dữ liệu thành những tập con độc lập, để các luồng độc lập — có thể trên các bộ xử lý khác nhau — thao tác trên đó.

Know Your Language and Library

Hãy hiểu rõ ngôn ngữ và thư viện của bạn

Here are a few more tips.

Sau đây là thêm vài mẹo.

- Use provided **thread-safe** collections.
- Use the threading frameworks (e.g., `Executor` in Java) for executing unrelated tasks.
- Use nonblocking solutions when possible.
- Know which library modules are and are not thread safe.
 - Hãy dùng các collection an toàn luồng (thread-safe) đã có sẵn.
 - Hãy dùng các framework luồng (ví dụ `Executor` trong Java) để thực thi những tác vụ không liên quan tới nhau.
 - Hãy dùng giải pháp nonblocking khi có thể.
 - Hãy biết module thư viện nào an toàn luồng và module nào thì không.

Thread-Safe Collections

Collection an toàn luồng

When Java was young, Doug Lea wrote the seminal book *4 Concurrent Programming in Java*. Along with the book, he developed several thread-safe collections, which later became part of the JDK in the `java.util.concurrent` **package**. The collections in that package are safe for multithreaded situations and they perform well. In fact, the `ConcurrentHashMap` implementation performs better than `HashMap` in nearly all situations. It also allows for simultaneous concurrent reads and writes, and it has methods supporting common composite operations that are otherwise not thread safe. If Java is the deployment environment, start with `ConcurrentHashMap`. 4 [Lea99]. There are several other kinds of classes added to support advanced concurrency design. Here are a few examples in Java:

Hồi Java còn non trẻ, Doug Lea viết cuốn sách khai phá 4 *Concurrent Programming in Java*. Cùng với cuốn sách, ông phát triển vài collection an toàn luồng, về sau trở thành một phần của JDK trong package `java.util.concurrent`. Các collection trong package đó an toàn cho tình huống đa luồng và chúng chạy rất tốt. Thực tế, hiện thực `ConcurrentHashMap` chạy tốt hơn `HashMap` trong gần như mọi tình huống. Nó còn cho phép đọc và ghi đồng thời cùng lúc, và có những method hỗ trợ các thao tác tổ hợp thông dụng vốn không an toàn luồng nếu tự làm. Nếu môi trường triển khai là Java, hãy bắt đầu với `ConcurrentHashMap`. 4 [Lea99]. Còn vài loại class khác được bổ sung để hỗ trợ thiết kế đồng thời nâng cao. Sau đây là vài ví dụ trong Java:

ReentrantLock	A lock that can be acquired in one method and released in another.
Semaphore	An implementation of the classic semaphore, a lock with a count.
CountDownLatch	A lock that waits for a number of events before releasing all threads waiting on it. This allows all threads to have a fair chance of starting at about the same time.

Recommendation: Review the classes available to you. In the case of Java,

Khuyến nghị: Hãy rà lại những class sẵn có cho bạn. Với Java thì là:

become familiar with `java.util.concurrent`, `java.util.concurrent.atomic`, and `java.util.concurrent.locks`.

Know Your Execution Models

Hãy hiểu rõ các mô hình thực thi

There are several different ways to partition behavior in a concurrent application. To discuss them we need to understand some basic definitions.

Có vài cách khác nhau để phân chia hành vi trong một ứng dụng đồng thời. Để bàn về chúng, ta cần hiểu vài định nghĩa cơ bản.

Bound resources	Resources of a fixed size or number used in a concurrent environment. Examples include database connections and fixed-size read/write buffers.
Mutual exclusion	Only one thread can access shared data or a shared resource at a time.
Starvation	One thread or a group of threads is prohibited from proceeding for an excessively long time—or forever. For example, always letting fast-running threads through first could starve out longer-running threads if there is no end to the fast-running threads.
Deadlock	Two or more threads are waiting for each other to finish. Each thread has a resource that the other thread requires, and neither can finish until it gets the other resource.
Livelock	Threads are in lockstep, each trying to do work but finding another “in the way.” Due to resonance, threads continue trying to make progress but are unable to for an excessively long time—or forever.

Given these definitions, we can now discuss the various execution models used in concurrent programming.

Với những định nghĩa trên, giờ ta có thể bàn về các mô hình thực thi khác nhau được dùng trong lập trình đồng thời.

Producer-Consumer 5

Producer-Consumer 5

5 <http://en.wikipedia.org/wiki/Producer-consumer> One or more producer threads create some work and place it in a buffer or queue. One or more consumer threads acquire that work from the queue and complete it. The queue between the producers and consumers is a bound resource. This means producers must wait for free space in the queue before writing and consumers must wait until there is something in the queue to consume. Coordination between the producers and consumers via the queue involves producers and consumers signaling each other. The producers write to the queue and signal that the queue is no longer empty. Consumers read from the queue and signal that the queue is no longer full. Both potentially wait to be notified when they can continue.

5 <http://en.wikipedia.org/wiki/Producer-consumer> Một hoặc nhiều luồng producer tạo ra công việc và đặt nó vào một buffer hay hàng đợi. Một hoặc nhiều luồng consumer lấy công việc đó ra khỏi hàng đợi và hoàn tất nó. Hàng đợi giữa producer và consumer là một tài nguyên có giới hạn. Nghĩa là producer phải chờ có chỗ trống trong hàng đợi rồi mới ghi được, còn consumer phải chờ tới khi hàng đợi có thứ gì đó để tiêu thụ. Việc phối hợp giữa producer và consumer thông qua hàng đợi đòi hỏi hai bên phát tín hiệu cho nhau. Producer ghi vào hàng đợi rồi báo rằng hàng đợi không còn rỗng. Consumer đọc từ hàng đợi rồi báo rằng hàng đợi không còn đầy. Cả hai đều có khả năng phải chờ được thông báo để tiếp tục.

Readers-Writers 6

Readers-Writers 6

6 http://en.wikipedia.org/wiki/Readers-writers_problem When you have a shared resource that primarily serves as a source of information for readers, but which is occasionally updated by writers, throughput is an issue. Emphasizing throughput can cause **starvation** and the accumulation of stale information. Allowing updates can impact throughput. Coordinating readers so that they do not read something a writer is updating, and vice versa, is a tough balancing act. Writers tend to block many readers for a long period of time, thus causing throughput issues.

6 http://en.wikipedia.org/wiki/Readers-writers_problem Khi bạn có một tài nguyên dùng chung mà chủ yếu đóng vai nguồn thông tin cho các reader, nhưng thỉnh thoảng lại bị writer cập nhật, thì thông lượng trở thành vấn đề. Nhấn mạnh thông lượng có thể gây ra starvation và tích tụ thông tin cũ mèm. Cho phép cập nhật lại có thể ảnh hưởng tới thông lượng. Phối hợp các reader sao cho chúng không đọc phải thứ mà một writer đang cập nhật, và ngược lại, là một màn giữ thăng bằng gian nan. Writer có xu hướng chặn nhiều reader trong thời gian dài, do đó gây vấn đề về thông lượng.

The challenge is to balance the needs of both readers and writers to satisfy correct operation, provide reasonable throughput, and avoid starvation. A simple strategy makes writers wait until there are no readers before allowing the writer to perform an update. If there are continuous readers, however, the writers will be starved. On the other hand, if there are frequent writers and they are given priority, throughput will suffer. Finding that balance and avoiding concurrent update issues is what the problem addresses.

Thách thức là cân bằng nhu cầu của cả reader lẫn writer để vừa vận hành đúng, vừa cho thông lượng hợp lý, vừa tránh starvation. Một chiến lược đơn giản là bắt writer chờ tới khi không còn reader nào rồi mới cho nó cập nhật. Nhưng nếu reader cứ đến liên tục thì

writer sẽ bị bỏ đói. Ngược lại, nếu writer đến thường xuyên và được ưu tiên thì thông lượng sẽ chịu thiệt. Tìm ra điểm cân bằng đó và tránh các vấn đề cập nhật đồng thời chính là điều bài toán này nhằm tới.

Dining Philosophers 7

Dining Philosophers 7

7 http://en.wikipedia.org/wiki/Dining_philosophers_problem Imagine a number of philosophers sitting around a circular table. A fork is placed to the left of each philosopher. There is a big bowl of spaghetti in the center of the table. The philosophers spend their time thinking until they get hungry. Once hungry, they pick up the forks on either side of them and eat. A philosopher cannot eat unless he is holding two forks. If the philosopher to his right or left is already using one of the forks he needs, he must wait until that philosopher finishes eating and puts the forks back down. Once a philosopher eats, he puts both his forks back down on the table and waits until he is hungry again.

7 http://en.wikipedia.org/wiki/Dining_philosophers_problem Hãy hình dung một nhóm triết gia ngồi quanh chiếc bàn tròn. Bên trái mỗi triết gia đặt một cái nĩa. Giữa bàn có một tô mì spaghetti to. Các triết gia dành thời gian để suy tư cho tới khi đói bụng. Để đói, họ nhặt hai cái nĩa ở hai bên mình lên và ăn. Một triết gia không thể ăn nếu không cầm đủ hai cái nĩa. Nếu triết gia bên phải hoặc bên trái đang dùng một trong hai cái nĩa mà ông ta cần, ông ta phải chờ tới khi vị kia ăn xong và đặt nĩa xuống. Ăn xong, triết gia đặt cả hai cái nĩa xuống bàn rồi chờ tới khi đói lại.

Replace “philosophers” with “threads” and “forks” with “resources” and this problem is similar to many enterprise applications in which processes compete for resources. Unless carefully designed, systems that compete in this way can experience deadlock, **livelock**, throughput, and efficiency degradation.

Thay “triết gia” bằng “luồng” và “nĩa” bằng “tài nguyên”, thế là bài toán này giống hệt vô số ứng dụng doanh nghiệp nơi các tiến trình tranh giành tài nguyên. Nếu không được thiết kế cẩn thận, những hệ thống tranh giành kiểu này có thể gặp deadlock, livelock, cùng sự xuống cấp về thông lượng và hiệu quả.

Recommendation: Most concurrent problems you will likely encounter will be some variation of these three problems. Study these algorithms and write solutions using them on your own so that when you come across concurrent problems, you’ll be more prepared to solve them.

Khuyến nghị: Hầu hết bài toán đồng thời mà bạn có khả năng gặp phải đều là biến thể của ba bài toán trên. Hãy nghiên cứu các thuật toán này và tự tay viết lời giải dùng chúng, để khi đụng phải vấn đề đồng thời, bạn đã sẵn sàng hơn để giải quyết.

Beware Dependencies between Synchronized Methods

Coi chừng phụ thuộc giữa các method synchronized

Dependencies between synchronized methods cause subtle bugs in concurrent code. The Java language has the notion of synchronized , which protects an individual method. However, if there is more than one synchronized method on the same shared class, then your system may be written incorrectly.

Phụ thuộc (dependency) giữa các method synchronized gây ra những bug rất tinh vi trong mã đồng thời. Ngôn ngữ Java có khái niệm synchronized , thứ bảo vệ một method riêng lẻ. Tuy nhiên, nếu có nhiều hơn một method synchronized trên cùng một class dùng chung, thì hệ thống của bạn có thể đã được viết sai.

Recommendation: Avoid using more than one method on a shared object.

Khuyến nghị: Tránh dùng nhiều hơn một method trên một đối tượng dùng chung.

There will be times when you must use more than one method on a shared object. When this is the case, there are three ways to make the code correct.

Sẽ có lúc bạn buộc phải dùng nhiều hơn một method trên một đối tượng dùng chung. Khi rơi vào tình huống đó, có ba cách để đoạn mã vẫn đúng.

- Client-based locking: Have the client lock the server before calling the first method, and make sure the lock's extent includes code calling the last method.
 - Khoá phía client (client-based locking): Để client lock server trước khi gọi method đầu tiên, và bảo đảm phạm vi của lock bao trùm cả đoạn mã gọi method cuối cùng.
- Server-based locking: Within the server, create a method that locks the server, calls all the methods, and then unlocks. Have the client call the new method.
- Adapted server: Create an intermediary that performs the locking. This is an example of server-based locking, where the original server cannot be changed.
 - Khoá phía server (server-based locking): Bên trong server, tạo một method tự lock server, gọi hết tất cả các method, rồi mở khoá. Để client gọi method mới ấy.
 - Server chuyển thể (adapted server): Tạo một lớp trung gian đảm nhiệm việc khoá. Đây là một dạng khoá phía server, dùng khi không thể sửa được server gốc.

Keep Synchronized Sections Small

Giữ các vùng synchronized thật nhỏ

The synchronized keyword introduces a lock. All sections of code guarded by the same lock are guaranteed to have only one thread executing through them at any given time. Locks are expensive because they create delays and add overhead. So we don't want to litter our code with synchronized statements. On the other hand, critical sections must be guarded. So we want to design our code with as few critical sections as possible. A critical section is any section of code that must be protected from simultaneous use for the program to be correct. Some naive programmers try to achieve this by making their critical sections very large. However, extending synchronization beyond the minimal critical section increases contention and degrades performance.

Từ khoá synchronized tạo ra một lock. Mọi đoạn mã được canh giữ bởi cùng một lock được bảo đảm rằng tại bất kỳ thời điểm nào cũng chỉ có đúng một luồng chạy xuyên qua chúng. Lock đắt đỏ vì chúng tạo ra độ trễ và thêm chi phí. Vậy nên ta không muốn rải câu lệnh synchronized vương vãi khắp mã. Mặt khác, vùng tới hạn phải được canh giữ. Vì vậy ta muốn thiết kế mã sao cho có càng ít vùng tới hạn càng tốt. Vùng tới hạn (critical section) là bất kỳ đoạn mã nào phải được bảo vệ khỏi việc dùng đồng thời thì chương trình mới đúng. Vài lập trình viên ngây thơ cố đạt được điều đó bằng cách làm cho vùng tới hạn của họ thật lớn. Tuy nhiên, kéo dài phạm vi đồng bộ hoá vượt quá vùng tới hạn tối thiểu sẽ làm tăng tranh chấp và làm giảm hiệu năng.

Recommendation: Keep your synchronized sections as small as possible.

Khuyến nghị: Hãy giữ các vùng synchronized nhỏ nhất có thể.

Writing Correct Startup and Shutdown Code Is Hard

Viết đúng mã khởi động và tắt máy là chuyện khó

Writing a system that is meant to stay alive and run forever is different from writing something that works for a while and then shuts down gracefully.

Viết một hệ thống được thiết kế để sống mãi và chạy mãi khác với viết một thứ chạy một hồi rồi tắt êm đẹp.

Graceful shutdown can be hard to get correct. Common problems involve deadlock, with threads waiting for a signal to continue that never comes.

Tắt êm đẹp là chuyện khó làm cho đúng. Các vấn đề thường gặp liên quan tới deadlock, với những luồng cứ chờ mãi một tín hiệu cho phép đi tiếp mà tín hiệu ấy không bao giờ tới.

For example, imagine a system with a parent thread that spawns several child threads and then waits for them all to finish before it releases its resources and shuts down. What if one of the spawned threads is deadlocked? The parent will wait forever, and the system will never shut down.

Ví dụ, hãy hình dung một hệ thống có luồng cha sinh ra vài luồng con rồi chờ tất cả chúng kết thúc mới giải phóng tài nguyên và tắt. Chuyện gì xảy ra nếu một trong các luồng con bị deadlock? Luồng cha sẽ chờ mãi mãi, và hệ thống sẽ chẳng bao giờ tắt được.

Or consider a similar system that has been instructed to shut down. The parent tells all the spawned children to abandon their tasks and finish. But what if two of the children were operating as a producer/consumer pair. Suppose the producer receives the signal from the parent and quickly shuts down. The consumer might have been expecting a message from the producer and be blocked in a state where it cannot receive the shutdown signal. It could get stuck waiting for the producer and never finish, preventing the parent from finishing as well.

Hoặc hãy xét một hệ thống tương tự vừa được lệnh tắt. Luồng cha bảo tất cả luồng con hãy bỏ dở tác vụ và kết thúc. Nhưng nếu hai trong số các luồng con đang hoạt động như một cặp producer/consumer thì sao? Giả sử producer nhận tín hiệu từ luồng cha và tắt cái rụp. Consumer có thể đang chờ một thông điệp từ producer và bị chặn ở trạng thái không thể nhận tín hiệu tắt máy. Nó có thể mắc kẹt trong lúc chờ producer và không bao giờ kết thúc, khiến luồng cha cũng chẳng kết thúc nổi.

Situations like this are not at all uncommon and apply equally to startup scenarios. So, if you must write concurrent code that involves starting up and shutting down gracefully, expect to spend much of your time getting those scenarios to happen correctly.

Những tình huống như thế này chẳng hiếm chút nào, và cũng đúng y hệt với các kịch bản khởi động. Vậy nên, nếu buộc phải viết mã đồng thời có khởi động và tắt êm đẹp, hãy chuẩn bị tinh thần dành phần lớn thời gian để làm cho những kịch bản ấy diễn ra đúng đắn.

Recommendation: Think about startup and shutdown early, and get it working early. It's going to take longer than you expect. Review existing algorithms, because this is probably harder than you think.

Khuyến nghị: Hãy nghĩ về khởi động và tắt máy từ sớm, và làm cho nó chạy được từ sớm. Nó sẽ tốn nhiều thời gian hơn bạn tưởng. Hãy rà lại các thuật toán sẵn có, vì chuyện này có lẽ khó hơn bạn nghĩ.

Testing Threaded Code

Test mã đa luồng

Proving that code is correct is impractical. Testing does not guarantee correctness. However, good testing can minimize risk. This is all true in a single-threaded solution. As soon as there are two or more threads using the same code and working with shared data, things get substantially more complex.

Chứng minh mã là đúng thì bất khả thi trên thực tế. Test không bảo đảm được tính đúng đắn. Tuy nhiên, test tốt có thể giảm thiểu rủi ro. Tất cả những điều đó đều đúng trong một giải pháp đơn luồng. Nhưng ngay khi có từ hai luồng trở lên cùng dùng một đoạn mã và làm việc với dữ liệu dùng chung, mọi thứ trở nên phức tạp hơn hẳn.

Recommendation: Write tests that have the potential to expose problems, and then run them frequently, with different programmatic configurations and system configurations and load. If tests ever fail, track down the failure. Don't ignore a failure just because the tests pass on a subsequent run.

Khuyến nghị: Hãy viết những test có tiềm năng phơi bày vấn đề, rồi chạy chúng thường xuyên, với các cấu hình chương trình, cấu hình hệ thống và mức tải khác nhau. Nếu test có lần nào thất bại, hãy truy cho ra nguyên nhân. Đừng phớt lờ một lần thất bại chỉ vì lần chạy sau test lại pass.

That is a whole lot to take into consideration. Here are a few, more fine-grained recommendations.

Ngân ấy thứ đã là rất nhiều để cân nhắc. Sau đây là vài khuyến nghị chi tiết hơn.

- Treat spurious failures as candidate threading issues.
- Get your nonthreaded code working first.
- Make your threaded code pluggable.
- Make your threaded code tunable.
- Run with more threads than processors.
- Run on different platforms.
- Instrument your code to try and force failures.
 - Hãy coi những thất bại vu vơ là ứng viên của vấn đề đa luồng.
 - Hãy làm cho mã không-đa-luồng chạy đúng trước đã.
 - Hãy làm cho mã đa luồng cắm-tháo được.
 - Hãy làm cho mã đa luồng tinh chỉnh được.
 - Hãy chạy với số luồng nhiều hơn số bộ xử lý.
 - Hãy chạy trên nhiều nền tảng khác nhau.
 - Hãy gắn đo đạc vào mã để cố ép nó thất bại.

Treat Spurious Failures as Candidate Threading Issues

Hãy coi những thất bại vu vơ là ứng viên của vấn đề đa luồng

Threaded code causes things to fail that “simply cannot fail.” Most developers do not have an intuitive feel for how threading interacts with other code (authors included). Bugs in threaded code might exhibit their symptoms once in a thousand, or a million, executions. Attempts to repeat the systems can be frustrating. This often leads developers to write off the failure as a cosmic ray, a hardware glitch, or some other kind of “one-off.” It is best to assume that one-offs do not exist. The longer these one-offs are ignored, the more code is built on top of a potentially faulty approach.

Mã đa luồng khiến những thứ “đơn giản là không thể hỏng” lại hỏng. Phần lớn lập trình viên không có trực giác về cách luồng tương tác với phần mã còn lại (kể cả các tác giả cuốn sách này). Bug trong mã đa luồng có thể chỉ lộ triệu chứng một lần trong một nghìn, hay một triệu, lượt chạy. Cố tái hiện lại có thể khiến người ta phát điên. Điều đó thường dẫn tới việc lập trình viên gạt phăng thất bại đó đi, đổ cho tia vũ trụ, trục trặc phần cứng, hay một kiểu “sự cố ngẫu nhiên” nào đó. Tốt nhất hãy giả định rằng sự cố ngẫu nhiên không tồn tại. Càng phớt lờ những “sự cố ngẫu nhiên” ấy lâu, càng có nhiều mã được xây chồng lên một cách tiếp cận có thể đang sai.

Recommendation: Do not ignore system failures as one-offs.

Khuyến nghị: Đừng phớt lờ các thất bại hệ thống như thể chúng là sự cố ngẫu nhiên.

Get Your Nonthreaded Code Working First

Hãy làm cho mã không-đa-luồng chạy đúng trước đã

This may seem obvious, but it doesn't hurt to reinforce it. Make sure code works outside of its use in threads. Generally, this means creating simple modules that are called by your threads. Since these modules are not thread aware, they can be tested outside of the threaded environment. The more of your system you can place in such modules, the better.

Điều này nghe có vẻ hiển nhiên, nhưng nhắc lại cũng chẳng hại gì. Hãy bảo đảm mã chạy đúng khi nằm ngoài ngữ cảnh dùng luồng. Nói chung, điều đó nghĩa là tạo ra những module đơn giản để luồng của bạn gọi tới. Vì những module này không biết gì về luồng, chúng có thể được test bên ngoài môi trường đa luồng. Càng đặt được nhiều phần hệ thống vào các module như vậy càng tốt.

Recommendation: Do not try to chase down nonthreading bugs and threading bugs at the same time. Make sure your nonthreaded code works outside of threads.

Khuyến nghị: Đừng cố truy lùng bug không-đa-luồng và bug đa luồng cùng một lúc. Hãy bảo đảm mã không-đa-luồng của bạn chạy đúng khi ở ngoài luồng.

Make Your Threaded Code Pluggable

Hãy làm cho mã đa luồng cắm-tháo được

Write the concurrency-supporting code such that it can be run in several configurations:

Hãy viết phần mã hỗ trợ đồng thời sao cho nó chạy được với nhiều cấu hình khác nhau:

- Write as one thread, several threads, or varied as it executes.
- Threaded code interacts with something that can be both real or a **test** double.
- Execute with test doubles that run quickly, slowly, **variable**.
- Execute with tests that run many iterations.
 - Viết sao cho chạy được với một luồng, nhiều luồng, hoặc thay đổi trong lúc đang chạy.
 - Mã đa luồng tương tác với một thứ có thể là thật hoặc là test double.
 - Chạy với test double nhanh, chậm, hoặc lúc nhanh lúc chậm.
 - Chạy với những test lặp thật nhiều vòng.

Recommendation: Make your thread-based code especially pluggable so that you can run it in various configurations.

Khuyến nghị: Hãy làm cho mã dựa trên luồng của bạn đặc biệt dễ cắm-tháo, để bạn có thể chạy nó với đủ kiểu cấu hình.

Make Your Threaded Code Tunable

Hãy làm cho mã đa luồng tinh chỉnh được

Getting the right balance of threads typically requires trial and error. Early on, find ways to time the performance of your system under different configurations. Allow the number of threads to be easily tuned. Consider allowing it to change while the system is running. Consider allowing self-tuning based on throughput and system utilization.

Tìm ra số luồng cân bằng thường đòi hỏi thử-và-sai. Ngay từ sớm, hãy tìm cách đo thời gian chạy của hệ thống dưới các cấu hình khác nhau. Hãy cho phép dễ dàng tinh chỉnh số luồng. Hãy cân nhắc cho phép thay đổi con số ấy ngay trong lúc hệ thống đang chạy. Hãy cân nhắc cho phép tự tinh chỉnh dựa trên thông lượng và mức sử dụng hệ thống.

Run with More Threads Than Processors

Hãy chạy với số luồng nhiều hơn số bộ xử lý

Things happen when the system switches between tasks. To encourage task swapping, run with more threads than processors or cores. The more frequently your tasks swap, the more likely you'll encounter code that is missing a critical section or causes deadlock.

Đủ thú chuyện xảy ra khi hệ thống chuyển đổi giữa các tác vụ. Để khuyến khích việc hoán đổi tác vụ, hãy chạy với số luồng nhiều hơn số bộ xử lý hoặc số core. Tác vụ càng hoán đổi thường xuyên, bạn càng dễ đụng phải đoạn mã thiếu vùng tới hạn hoặc gây deadlock.

Run on Different Platforms

Hãy chạy trên nhiều nền tảng khác nhau

In the middle of 2007, we developed a course on concurrent programming. The course development ensued primarily under OS X. The class was presented using Windows XP running under a VM. Tests written to demonstrate failure conditions did not fail as frequently in an XP environment as they did running on OS X.

Giữa năm 2007, chúng tôi phát triển một khoá học về lập trình đồng thời. Việc phát triển khoá học chủ yếu diễn ra trên OS X. Lớp học thì được trình bày trên Windows XP chạy trong máy ảo. Những test viết ra để minh hoạ các điều kiện gây lỗi lại không thất bại thường xuyên trong môi trường XP như khi chạy trên OS X.

In all cases, the code under test was known to be incorrect. This just reinforced the fact that different operating systems have different threading policies, each of which impacts the code's execution. Multithreaded code behaves differently in different environments. 9 You should run your tests in every potential deployment environment.

Trong mọi trường hợp, đoạn mã đem ra test đều đã biết chắc là sai. Điều này chỉ càng khẳng định một thực tế: các hệ điều hành khác nhau có chính sách quản lý luồng khác nhau, và mỗi chính sách lại tác động tới cách mã được thực thi. Mã đa luồng hành xử khác nhau trong những môi trường khác nhau. 9 Bạn nên chạy test của mình trong mọi môi trường triển khai tiềm năng.

9 Did you know that the threading model in Java does not guarantee preemptive threading? Modern OSes support preemptive threading, so you get that “for free.” Even so, it is not guaranteed by the JVM. Recommendation: Run your threaded code on all target platforms early and often.

9 Bạn có biết rằng mô hình luồng trong Java không bảo đảm luồng có tính preemptive không? Các hệ điều hành hiện đại đều hỗ trợ luồng preemptive, nên bạn được hưởng điều đó “miễn phí”. Dù vậy, JVM không hề bảo đảm chuyện ấy. Khuyến nghị: Hãy chạy mã đa luồng của bạn trên tất cả nền tảng đích, từ sớm và thường xuyên.

Instrument Your Code to Try and Force Failures

Hãy gắn đo đạc vào mã để cố ép nó thất bại

It is normal for flaws in concurrent code to hide. Simple tests often don't expose them. Indeed, they often hide during normal processing. They might show up once every few hours, or days, or weeks!

Chuyện khiếm khuyết trong mã đồng thời ẩn mình là bình thường. Những test đơn giản thường không phơi bày được chúng. Thật vậy, chúng hay ẩn mình suốt quá trình xử lý bình thường. Có khi vài giờ, vài ngày, hay vài tuần chúng mới lộ mặt một lần!

The reason that threading bugs can be infrequent, sporadic, and hard to repeat is that only a very few pathways out of the many thousands of possible pathways through a vulnerable section actually

fail. So the probability that a failing pathway is taken can be startlingly low. This makes detection and debugging very difficult.

Lý do khiến bug đa luồng thừa thớt, thất thường và khó tái hiện là chỉ một số rất ít đường đi trong hàng nghìn đường đi khả dĩ xuyên qua một vùng dễ tổn thương mới thật sự hỏng. Vì vậy xác suất đi đúng vào một đường hỏng có thể thấp đến mức giạt mình. Điều đó khiến việc phát hiện và gỡ lỗi trở nên vô cùng khó khăn.

How might you increase your chances of catching such rare occurrences? You can instrument your code and force it to run in different orderings by adding calls to

Làm sao tăng cơ hội bắt được những trường hợp hiếm hoi ấy? Bạn có thể gắn đo đạc vào mã và ép nó chạy theo những thứ tự khác nhau bằng cách thêm lời gọi tới

methods like `Object.wait()`, `Object.sleep()`, `Object.yield()`, and `Object.priority()`.

Each of these methods can affect the order of execution, thereby increasing the odds of detecting a flaw. It's better when broken code fails as early and as often as possible.

Mỗi method trong số đó đều có thể tác động tới thứ tự thực thi, qua đó tăng khả năng phát hiện khiếm khuyết. Mã hỏng mà hỏng càng sớm và càng thường xuyên thì càng tốt.

There are two options for code instrumentation:

Có hai lựa chọn để gắn đo đạc vào mã:

- Hand-coded
- Automated
 - Viết tay
 - Tự động

Hand-Coded

Viết tay

You can insert calls to `wait()`, `sleep()`, `yield()`, and `priority()` in your code by hand. It might be just the thing to do when you're testing a particularly thorny piece of code.

Bạn có thể tự tay chèn các lời gọi `wait()`, `sleep()`, `yield()` và `priority()` vào mã của mình. Đó có thể đúng là việc nên làm khi bạn đang test một đoạn mã đặc biệt gai góc.

Here is an example of doing just that:

Dưới đây là một ví dụ làm đúng như vậy:

```
public synchronized String nextUrlOrNull() {
    if(hasNext()) {
        String url = urlGenerator.next();
        Thread.yield(); // inserted for testing.
        updateHasNext();
        return url;
    }
    return null;
}
```

The inserted call to `yield()` will change the execution pathways taken by the code and possibly cause the code to fail where it did not fail before. If the code does break, it was not because you added a call to `yield()`.¹⁰ Rather, your code was broken and this simply made the failure evident.¹⁰ This is not strictly the case. Since the JVM does not guarantee preemptive threading, a particular algorithm

might always work on an OS that does not preempt threads. The reverse is also possible, but for different reasons. There are many problems with this approach.

Lời gọi `yield()` được chèn vào sẽ làm thay đổi các đường đi thực thi mà mã đi qua, và có thể khiến mã thất bại ở chỗ trước đó nó không hề thất bại. Nếu mã có hỏng thật, thì đó không phải vì bạn đã thêm một lời gọi `yield()`. 10 Đúng hơn, mã của bạn vốn đã hỏng sẵn, và việc này chỉ đơn giản làm cho thất bại ấy lộ ra. 10 Nói cho thật chặt chẽ thì điều này không hoàn toàn đúng. Vì JVM không bảo đảm luồng có tính preemptive, một thuật toán cụ thể nào đó có thể luôn chạy đúng trên hệ điều hành không preempt luồng. Chiều ngược lại cũng có thể xảy ra, nhưng vì những lý do khác. Cách làm này có rất nhiều vấn đề.

- You have to manually find appropriate places to do this.
- How do you know where to put the call and what kind of call to use?
- Leaving such code in a production environment unnecessarily slows the code down.
- It's a shotgun approach. You may or may not find flaws. Indeed, the odds aren't with you.
 - Bạn phải tự tay tìm ra những chỗ thích hợp để làm việc này.
 - Làm sao bạn biết nên đặt lời gọi ở đâu và dùng loại lời gọi nào?
 - Để lại đoạn mã như vậy trong môi trường production sẽ làm mã chậm đi một cách không cần thiết.
 - Đây là kiểu bắn súng săn hú họa. Bạn có thể tìm ra khiếm khuyết, mà cũng có thể không. Thật ra thì xác suất chẳng đứng về phía bạn.

What we need is a way to do this during testing but not in production. We also need to easily mix up configurations between different runs, which results in increased chances of finding errors in the aggregate.

Thứ chúng ta cần là một cách làm việc này trong lúc test nhưng không làm trong production. Chúng ta cũng cần dễ dàng đảo trộn các cấu hình giữa những lần chạy khác nhau, nhờ đó tăng cơ hội tìm ra lỗi trên tổng thể.

Clearly, if we divide our system into simple modules that know nothing of threading and others that control the threading, it will be easier to find appropriate places to instrument the code. Moreover, we could create many different test jigs that invoke the modules under different regimens of calls to `sleep`, `yield`, and so on.

Rõ ràng, nếu ta chia hệ thống thành những module đơn giản chẳng biết gì về luồng (thread) và những module khác lo chuyện điều khiển luồng, thì việc tìm chỗ thích hợp để gắn đo đạc vào mã sẽ dễ hơn nhiều. Hơn nữa, ta có thể dựng ra nhiều bộ gá test khác nhau để gọi các module đó dưới những chế độ gọi `sleep`, `yield` ... khác nhau.

Automated

Tự động

You could use tools like CGLIB or ASM to programmatically instrument your code with calls to a jiggle. For example, you could use a class with a single method:

Bạn có thể dùng những công cụ như CGLIB hay ASM để gắn đo đạc vào mã bằng chương trình, chèn vào đó các lời gọi tới một jiggle (bộ rung lắc thứ tự thực thi). Ví dụ, bạn có thể dùng một class chỉ có duy nhất một method:

```
public class ThreadJigglePoint {  
    public static void jiggle() {  
    }  
}
```

You can add calls to this in various places within your code:

Bạn có thể thêm lời gọi tới nó ở nhiều chỗ khác nhau trong mã:

```
public synchronized String nextUrlOrNull() {
    if(hasNext()) {
        ThreadJigglePoint.jiggle();
        String url = urlGenerator.next();
        ThreadJigglePoint.jiggle();
        updateHasNext();
        ThreadJigglePoint.jiggle();
        return url;
    }
    return null;
}
```

Imagine that the ThreadJigglePoint class has two implementations. The first implements jiggle to do nothing and is used in production. The second generates a random number to choose between sleeping, yielding, or just falling through. If you run your tests a thousand times with random jiggling, you may root out some flaws. If the tests pass, at least you can say you’ve done due diligence. Though a bit simplistic, this could be a reasonable option in lieu of a more sophisticated tool.

Hãy hình dung class ThreadJigglePoint có hai hiện thực (implementation). Bản thứ nhất hiện thực jiggle sao cho chẳng làm gì cả và được dùng trong production. Bản thứ hai sinh một số ngẫu nhiên để chọn giữa ngủ một chút, nhường lượt, hoặc đơn giản là chạy thẳng qua. Nếu bạn chạy test cả nghìn lần với việc rung lắc ngẫu nhiên, có thể bạn sẽ moi ra được vài khiếm khuyết. Nếu test đều pass, ít nhất bạn cũng nói được rằng mình đã làm hết trách nhiệm. Tuy hơi thô sơ, đây vẫn có thể là lựa chọn hợp lý khi chưa có công cụ tinh vi hơn.

The point is to jiggle the code so that threads run in different orderings at different times. The combination of well-written tests and jiggling can dramatically increase the chance of finding errors.

Điểm mấu chốt là rung lắc mã để các luồng chạy theo những thứ tự khác nhau vào những thời điểm khác nhau. Kết hợp test viết tốt với việc rung lắc có thể tăng vọt cơ hội tìm ra lỗi.

Recommendation: Use jiggling strategies to ferret out errors .

Khuyến nghị: Hãy dùng các chiến lược rung lắc (jiggling) để moi lỗi ra ánh sáng.

2025 Update and Report from the Field

Cập nhật 2025 và báo cáo từ thực địa

Modern stacks handle multiple users well when those users do not share data. The following are some cases where knowledge of parallel/threading issues is still good to keep in mind.

Các stack hiện đại xử lý tốt chuyện nhiều người dùng, miễn là những người dùng đó không chia sẻ dữ liệu với nhau. Dưới đây là vài trường hợp cho thấy hiểu biết về các vấn đề song song/đa luồng vẫn rất đáng ghi nhớ.

A little background: Modern development stacks often purport to take care of many of the issues we used to hand-roll back when the first edition of **Clean Code** was written. However, many of the original problems and issues still exist in modern systems, even ones that handle “all the multiuser stuff” for you.

Một chút bối cảnh: Các stack phát triển hiện đại thường tự nhận là đã lo giúp bạn nhiều vấn đề mà hồi ấn bản đầu của Clean Code ra đời chúng ta phải tự tay xử lý. Tuy vậy, nhiều vấn đề và rắc rối thuở ban đầu vẫn còn nguyên trong các hệ thống hiện đại, kể cả những hệ thống lo hết “mọi thứ đa người dùng” cho bạn.

Here is a collection of examples that we’ve come across coincidentally just in the year before this second edition of Clean Code was released.

Sau đây là một bộ sưu tập ví dụ mà chúng tôi tình cờ gặp phải chỉ trong vòng một năm trước khi ấn bản thứ hai của Clean Code này ra mắt.

Data Integrity

Tính toàn vẹn dữ liệu

In the past few years, we’ve come across a number of situations where a multiuser system had potential data integrity issues. All of these systems have multiple simultaneous users (parallel) doing something that updates shared data.

Vài năm qua, chúng tôi đã gặp không ít tình huống mà một hệ thống đa người dùng tiềm ẩn vấn đề về tính toàn vẹn dữ liệu (data integrity). Tất cả những hệ thống ấy đều có nhiều người dùng đồng thời (song song) cùng làm việc gì đó cập nhật dữ liệu dùng chung.

These examples come from two different multiuser systems based in React Native. They use AWS Lambda for a backend, and storage in some datastore. There’s nothing inherently problematic with these kinds of systems. If two users do not share data, then there are likely no data integrity issues. However, what happens when multiple users do depend on shared data?

Những ví dụ này đến từ hai hệ thống đa người dùng khác nhau, cùng dựa trên React Native. Chúng dùng AWS Lambda cho backend và lưu trữ trong một datastore nào đó. Bản thân loại hệ thống này chẳng có gì đáng lo. Nếu hai người dùng không chia sẻ dữ liệu thì gần như chắc chắn chẳng có vấn đề toàn vẹn dữ liệu nào. Nhưng chuyện gì xảy ra khi nhiều người dùng cùng phụ thuộc vào dữ liệu dùng chung?

The examples are

Các ví dụ là

- Joining a game
- Working on a shared design
- Players working in parallel to create a deck of cards
- Double-deleting of resources on rerender using a 2D library in React Native
- Loading icon resources
 - Tham gia một ván chơi
 - Cùng làm việc trên một bản thiết kế dùng chung
 - Người chơi làm song song để tạo một bộ bài
 - Xoá tài nguyên hai lần khi render lại bằng một thư viện 2D trong React Native
 - Nạp tài nguyên icon

Joining a Game

Tham gia một ván chơi

Imagine a mobile games platform used for training workshops. Students log in to a game. One game had a limit of six players, so after the sixth player joins, subsequent attempts to join the game are rejected. Each player is assigned a role, numbered 1 to 6.

Hãy hình dung một nền tảng game di động dùng cho các workshop đào tạo. Học viên đăng nhập vào một ván chơi. Có ván giới hạn sáu người, nên sau khi người thứ sáu vào rồi thì mọi nỗ lực tham gia tiếp theo đều bị từ chối. Mỗi người chơi được gán một vai, đánh số từ 1 tới 6.

The backend system uses DynamoDB, which gives “eventually consistent” results.

Hệ thống backend dùng DynamoDB, vốn cho kết quả “nhất quán cuối cùng” (eventually consistent).

For a number of months, we’ve had an occasional kerfuffle where it seems we were losing players. It turned out to be an issue with when we decided to assign player roles, as well as having players immediately join a game.

Suốt mấy tháng liền, thi thoảng lại có một phen nhốn nháo: hình như chúng tôi đang làm mất người chơi. Hoá ra vấn đề nằm ở thời điểm chúng tôi quyết định gán vai cho người chơi, cộng thêm việc để người chơi vào thẳng ván chơi ngay lập tức.

In the original implementation, we combined logging in with joining a game and assigning roles to players. Assigning players to games and roles, along with eventual consistency in DynamoDB, led to users getting the same roles, or games overfilling.

Trong hiện thực ban đầu, chúng tôi gộp việc đăng nhập với việc vào ván chơi và gán vai cho người chơi. Việc gán người chơi vào ván và vào vai, cộng với tính nhất quán cuối cùng của DynamoDB, dẫn tới chuyện nhiều người dùng nhận cùng một vai, hoặc ván chơi bị nhồi quá số.

While it was possible to rewrite how we interacted with DynamoDB, there was another desired change that predated discovering this **defect**. That change would make the original problem disappear by separating logging in with joining a game, and subsequent role assignment.

Dù hoàn toàn có thể viết lại cách chúng tôi tương tác với DynamoDB, vẫn còn một thay đổi mong muốn khác đã được nghĩ tới từ trước khi phát hiện ra lỗi (defect) này. Thay đổi đó sẽ khiến vấn đề ban đầu tự biến mất, bằng cách tách việc đăng nhập khỏi việc vào ván chơi và khỏi việc gán vai sau đó.

The original issue was assigning roles to multiple users joining a game from multiple mobile clients, that is, in parallel. To fix this, we first expanded the domain. Rather than players joining a game at login time, players join a waiting room. Later, the administrator creates games and then issues a request to assign all waiting players to games.

Vấn đề gốc là gán vai cho nhiều người dùng cùng vào một ván chơi từ nhiều client di động, tức là song song. Để sửa, trước hết chúng tôi mở rộng miền nghiệp vụ. Thay vì vào ván chơi ngay lúc đăng nhập, người chơi vào một phòng chờ. Sau đó, người quản trị tạo các ván chơi rồi phát một yêu cầu để gán tất cả người đang chờ vào các ván.

This took something that was not safe without transaction isolation, both joining a game and role assignment, from being executed potentially in parallel to being executed later from a single thread/issued by a single user. The original implementation was not an essential part of how we wanted the game to work; it was one of the first ideas on how to handle multiple games going on at the same time.

Việc này đưa một thứ vốn không an toàn nếu thiếu cô lập giao dịch (transaction isolation) — cả việc vào ván chơi lẫn việc gán vai — từ chỗ có thể bị thực thi song song sang chỗ được thực thi sau đó, từ một luồng duy nhất/do một người dùng duy nhất phát ra. Hiện thực ban đầu không phải phần cốt yếu trong cách chúng tôi muốn trò chơi vận hành; nó chỉ là một trong những ý tưởng đầu tiên về cách xử lý nhiều ván chơi diễn ra cùng lúc.

One of the side effects of that approach was that the administrator would create games ahead of time, and then tell specific people to log in to specific games. Users would enter their name, a game name, and a game password. It was common for communication issues, causing people to join the wrong game.

Một tác dụng phụ (side effect) của cách làm đó là người quản trị phải tạo sẵn ván chơi từ trước, rồi bảo từng người cụ thể đăng nhập vào ván cụ thể. Người dùng phải nhập tên mình, tên ván và mật khẩu ván. Trục trặc trong khâu truyền đạt là chuyện thường tình, khiến người ta vào nhầm ván.

After the change, all players join a waiting room, the name of which is the administrator's email address, provided in a pull-down list. So, rather than game name, username, and password, players selected a waiting room name and provided their name. It made the UX simpler and less error prone.

Sau thay đổi, mọi người chơi đều vào một phòng chờ mà tên phòng chính là địa chỉ email của người quản trị, chọn từ một danh sách xổ xuống. Thế là thay vì tên ván, tên người dùng và mật khẩu, người chơi chỉ việc chọn tên phòng chờ rồi nhập tên mình. Trải nghiệm người dùng nhờ vậy đơn giản hơn và ít sinh lỗi hơn.

This separated the multithread-safe part of the work—getting into the system—from the not multithread-safe part of the work—joining a specific game and assigning user roles.

Cách này tách phần công việc an toàn với đa luồng — vào được hệ thống — khỏi phần công việc không an toàn với đa luồng — vào một ván chơi cụ thể và gán vai cho người dùng.

Working on a Shared Design Next up: a system allowing the shared creation of a design for the installation/updating of a commercial security system. For this system, we used a relational database as the data store.

Cùng làm việc trên một bản thiết kế dùng chung Tiếp theo: một hệ thống cho phép nhiều người cùng tạo ra bản thiết kế để lắp đặt/nâng cấp một hệ thống an ninh thương mại. Với hệ thống này, chúng tôi dùng cơ sở dữ liệu quan hệ làm nơi lưu dữ liệu.

Such a plan has dozens of images, and each image might have several elements that might be connected. How do you make sure that multiple people can work on the same design at the same time, without conflict?

Một bản thiết kế như vậy có hàng chục hình ảnh, và mỗi hình lại có thể chứa vài phần tử có thể nối với nhau. Làm sao bảo đảm nhiều người cùng làm việc trên một bản thiết kế vào cùng một lúc mà không xung đột?

For this particular domain, the number of users on a design is generally less than five, and the users tend to work on different parts of an overall design. So, while the possibility of conflict was there, given the typical use of the system, the likelihood of it happening was small.

Với miền nghiệp vụ cụ thể này, số người dùng trên một bản thiết kế thường ít hơn năm, và họ có xu hướng làm ở những phần khác nhau của tổng thể. Vậy nên tuy khả năng xung đột là có, xét theo cách hệ thống thường được dùng thì xác suất xảy ra là nhỏ.

Preexisting in the system was the need to select an element to edit it. This became a natural place to lock an element. We use a flag on the element for whether it is locked. Any attempt to lock when already locked fails, and informs the user that the element is locked. Upon deselection, the element's lock is released. We chose a flag on an element, rather than locking the element in the database, because the latter would require long-lived transactions.

Hệ thống vốn đã đòi hỏi phải chọn một phần tử thì mới sửa được nó. Đây hoá ra là chỗ tự nhiên để khoá phần tử. Chúng tôi dùng một cờ trên phần tử để đánh dấu nó có đang bị

khoá hay không. Mọi nỗ lực khoá khi phần tử đã bị khoá đều thất bại, và người dùng được báo rằng phần tử đang bị khoá. Khi bỏ chọn, lock của phần tử được giải phóng. Chúng tôi chọn dùng cờ trên phần tử thay vì khoá phần tử ngay trong cơ sở dữ liệu, vì cách sau sẽ đòi hỏi những giao dịch sống rất lâu.

With proper transaction isolation, this is a clean, safe solution to the particular problem.

Với mức cô lập giao dịch phù hợp, đây là một giải pháp sạch sẽ và an toàn cho vấn đề cụ thể ấy.

Players Working in Parallel to Create a Deck of Cards

Người chơi làm song song để tạo một bộ bài

This is an example of an anti-case, that is, we chose to not solve the problem. Imagine a second mobile game, this one allowing small groups of players to compete in creating a deck of cards with certain characteristics.

Đây là ví dụ về một phản-trường hợp, tức là chúng tôi đã chọn không giải quyết vấn đề. Hãy hình dung một game di động thứ hai, lần này cho phép những nhóm nhỏ người chơi thi nhau tạo ra một bộ bài có những đặc tính nhất định.

In this case, we have up to four people creating a deck of cards. Those players might be remote, local, collocated, or not. What happens if two players create the same card?

Ở đây, có tới bốn người cùng tạo một bộ bài. Những người chơi đó có thể ở xa, ở gần, ngồi cùng chỗ hoặc không. Chuyện gì xảy ra nếu hai người tạo ra cùng một lá bài?

Players attempt to create a deck with certain characteristics. If they happen to get duplicate cards, the instructor rejects their deck. However, when this happens, the team can clean up their deck and resubmit it.

Người chơi cố tạo một bộ bài với những đặc tính nhất định. Nếu chẳng may có lá trùng, người hướng dẫn sẽ loại bộ bài đó. Nhưng khi chuyện này xảy ra, cả nhóm có thể dọn lại bộ bài của mình rồi nộp lại.

Additionally, that experience also leads to good discussions about collaboration and communication, so trying to make sure it doesn't happen reduces the value of the experience.

Thêm nữa, chính trải nghiệm đó lại dẫn tới những cuộc thảo luận hay ho về cộng tác và giao tiếp, nên cố ngăn nó xảy ra hoá ra lại làm giảm giá trị của trải nghiệm.

In this case, there's a shared resource: the deck of cards. The addition of cards is done in parallel. There can be a "data integrity" issue, duplicated work, but the end result is easy to recover from, all done through human interaction.

Trong trường hợp này có một tài nguyên dùng chung: bộ bài. Việc thêm bài diễn ra song song. Có thể xuất hiện vấn đề "toàn vẹn dữ liệu", tức là làm trùng việc, nhưng kết quả cuối cùng thì dễ khắc phục, và tất cả đều được xử lý qua tương tác giữa người với người.

Early on, we realized this was a potential issue. When we thought through solutions, we decided to leave the potential in the game. An instructor can run the game in different ways, with different rules. So, in this case, the solution was to leave it as is.

Từ sớm, chúng tôi đã nhận ra đây là một vấn đề tiềm tàng. Khi cân nhắc các giải pháp, chúng tôi quyết định để nguyên khả năng đó trong trò chơi. Người hướng dẫn có thể điều hành trò chơi theo nhiều cách, với những luật khác nhau. Nên trong trường hợp này, giải pháp là cứ để y như vậy.

Double-Deleting of Resources on Rerender Using a 2D Library in React Native We were using React Native, with React Native Web, to produce a web and mobile application for a customer. We additionally used a 2D graphics library called Skia.

Xoá tài nguyên hai lần khi render lại bằng một thư viện 2D trong React Native Chúng tôi dùng React Native, kèm React Native Web, để làm một ứng dụng web và di động cho khách hàng. Ngoài ra, chúng tôi còn dùng một thư viện đồ hoạ 2D tên là Skia.

Skia is a powerful library, and it has its own rendering engine. This means there are at least two rendering engines running at the same time—React Native and Skia— and those two rendering engines are not synchronized. We did not notice an issue until we added asynchronous loading of designs.

Skia là một thư viện mạnh, và nó có engine render riêng. Điều đó nghĩa là có ít nhất hai engine render chạy cùng lúc — React Native và Skia — mà hai engine ấy lại chẳng đồng bộ với nhau. Chúng tôi không nhận ra vấn đề gì cho tới khi thêm phần nạp bản thiết kế theo kiểu bất đồng bộ.

When a user opens a design, we load it asynchronously while moving into the design view of the application. When we started doing this, we found that Skia was frequently crashing.

Khi người dùng mở một bản thiết kế, chúng tôi nạp nó bất đồng bộ trong lúc chuyển sang màn hình thiết kế của ứng dụng. Vừa làm vậy, chúng tôi phát hiện Skia sập liên tục.

What ultimately caused this was rendering the Skia view before we had finished loading. In React Native, this is normal. It rerenders the view. However, since Skia has its own rendering thread, when React Native rerendered the view at an unexpected time, Skia would start to delete, then redelete the same resources.

Nguyên nhân sâu xa là view của Skia được render trước khi chúng tôi nạp xong. Trong React Native, chuyện đó là bình thường. Nó render lại view. Nhưng vì Skia có luồng render riêng, nên khi React Native render lại view vào một thời điểm không lường trước, Skia sẽ bắt đầu xoá, rồi xoá lại chính những tài nguyên ấy.

Deleting already-deleted memory killed Skia and required restarting the application. This wasn't an issue with Skia; it was how we were using it.

Xoá vùng nhớ vốn đã bị xoá làm Skia chết và buộc phải khởi động lại ứng dụng. Đây không phải lỗi của Skia; đó là do cách chúng tôi dùng nó.

The solution for this was to not render the Skia view until loading was done. We had well-defined locations where we needed Skia to be rendering, and where it was not required. We make sure to

Giải pháp là không render view của Skia cho tới khi nạp xong. Chúng tôi đã xác định rõ những chỗ nào cần Skia render và những chỗ nào không. Chúng tôi bảo đảm rằng:

- Only render the Skia view when loading was complete; otherwise, we rendered an empty view.
- First clear a setting anytime we issued a change that caused a rerender of the Skia view. This would cause the Skia view to stop being rendered.
- Set a new value, causing everything to be rendered fresh.
 - Chỉ render view của Skia khi việc nạp đã hoàn tất; nếu chưa, chúng tôi render một view rỗng.
 - Trước tiên xoá một thiết lập mỗi khi phát ra một thay đổi khiến view của Skia phải render lại. Việc này khiến view của Skia ngừng được render.
 - Đặt giá trị mới, khiến mọi thứ được render lại từ đầu.

The end solution resulted in cleaner code overall. The solution was to allow for “parallel” rendering threads only when a design was loaded. No design? No Skia rendering. Design? Skia rendering. Anytime we changed designs, we first set state to stop rendering Skia, loaded the design, and upon completion, began using Skia again.

Giải pháp cuối cùng khiến toàn bộ mã sạch hơn. Cách làm là chỉ cho phép các luồng render “song song” khi một bản thiết kế đã nạp xong. Không có thiết kế? Không render Skia. Có thiết kế? Render Skia. Mỗi lần đổi thiết kế, trước hết chúng tôi đặt trạng thái để ngừng render Skia, nạp thiết kế, và khi xong xuôi thì bắt đầu dùng Skia trở lại.

Loading Icon Resources

Nạp tài nguyên icon

We had an issue with the order in which icons were loaded. It was possible to load a design into the system before the code to load the icons had started executing. When this happened, we’d have to reload the design, or have several missing icons.

Chúng tôi gặp vấn đề với thứ tự nạp các icon. Có khả năng một bản thiết kế được nạp vào hệ thống trước cả khi đoạn mã nạp icon kịp chạy. Khi điều đó xảy ra, chúng tôi phải nạp lại bản thiết kế, hoặc chấp nhận thiếu vài icon.

Since the resources were needed by other parts of the system, we moved them into a so-called provider in React Native. Then, any code that might cause a design to be loaded is always executed under that provider; that is, there was a parent-child relationship between the provider and anything using icons. The parent initializes first, guaranteeing the order of execution.

Vì các tài nguyên đó còn cần cho những phần khác của hệ thống, chúng tôi chuyển chúng vào một thứ gọi là provider trong React Native. Từ đó, mọi đoạn mã có thể làm một bản thiết kế được nạp đều luôn chạy bên dưới provider ấy, tức là có quan hệ cha-con giữa provider và mọi thứ dùng icon. Cha khởi tạo trước, nhờ đó bảo đảm được thứ tự thực thi.

As with the previous example, this resulted in cleaner code. Things that were related moved closer together under a single place, and we removed duplication.

Cũng như ví dụ trước, việc này cho ra mã sạch hơn. Những thứ liên quan tới nhau được dồn lại gần nhau dưới một chỗ duy nhất, và chúng tôi loại bỏ được trùng lặp (duplication).

Sharing Data In all of these examples, there’s some issue with sharing data:

Chia sẻ dữ liệu Trong tất cả những ví dụ trên đều có vấn đề nào đó với việc chia sẻ dữ liệu:

- Allowing players to join a specific game, and get a role assigned in parallel, with an eventually consistent database
- Allowing editing of a shared design, by multiple users in parallel
- Allowing players to create cards in parallel
- Multiple, dependent rendering loops that are not coordinated, with other asynchronous activity impacting them
- Not controlling the execution/loading of resources explicitly in an asynchronous environment
 - Cho phép người chơi vào một ván chơi cụ thể và được gán vai một cách song song, trên một cơ sở dữ liệu nhất quán cuối cùng
 - Cho phép nhiều người dùng cùng song song sửa một bản thiết kế dùng chung
 - Cho phép người chơi tạo bài song song
 - Nhiều vòng lặp render phụ thuộc nhau mà không được phối hợp, lại còn bị các hoạt động bất đồng bộ khác tác động

- Không kiểm soát tường minh việc thực thi/nạp tài nguyên trong một môi trường bất đồng bộ

We took different approaches for each of these problems.

Chúng tôi đã chọn những cách tiếp cận khác nhau cho từng vấn đề.

- Isolate the nonthread-safe code and execute it later in a single thread (isolate and separate).
- Lock objects with a flag, using a relational database.
- Do nothing. A human can choose to reject or accept based on the current rules of the game.
- Avoid using the second rendering thread until it's actually needed, and clear it before clearing everything else.
- Make the loading part of a parent–child relationship such that the child doesn't render until the parent is ready.
 - Cô lập phần mã không an toàn luồng rồi thực thi nó sau, trong một luồng duy nhất (cô lập và tách riêng).
 - Khoá đối tượng bằng một cờ, dựa trên cơ sở dữ liệu quan hệ.
 - Không làm gì cả. Con người có thể chọn loại hay chấp nhận, tùy theo luật hiện hành của trò chơi.
 - Tránh dùng luồng render thứ hai cho tới khi thật sự cần, và dọn nó trước khi dọn mọi thứ khác.
 - Đưa việc nạp vào một quan hệ cha–con sao cho con không render cho tới khi cha đã sẵn sàng.

Once you've discovered an issue with shared data, there are many approaches you might take to address it. We've seen five examples and five different approaches to specific problems.

Một khi đã phát hiện vấn đề với dữ liệu dùng chung, bạn có nhiều cách để xử lý nó. Chúng ta vừa xem năm ví dụ và năm cách tiếp cận khác nhau cho những vấn đề cụ thể.

Conclusion

Kết luận

Concurrent code is difficult to get right. Code that is simple to follow can become nightmarish when multiple threads and shared data get into the mix. If you are faced with writing concurrent code, you need to write clean code with rigor or else face subtle and infrequent failures.

Mã đồng thời (concurrency) rất khó làm cho đúng. Một đoạn mã dễ theo dõi có thể biến thành ác mộng khi nhiều luồng và dữ liệu dùng chung nhảy vào cuộc. Nếu buộc phải viết mã đồng thời, bạn cần viết clean code một cách nghiêm ngặt, bằng không sẽ phải đối mặt với những thất bại tinh vi và hiếm hoi.

First and foremost, follow the Single Responsibility Principle. Break your system into simple modules that separate thread-aware code from thread-ignorant code. Make sure when you are testing your thread-aware code that you are only testing it and nothing else. This suggests that your thread-aware code should be small and focused.

Trước hết và trên hết, hãy tuân theo SRP — Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle). Hãy chia hệ thống thành những module đơn giản, tách mã có ý thức về luồng khỏi mã chẳng biết gì về luồng. Khi test phần mã có ý thức về luồng, hãy bảo đảm bạn chỉ test đúng nó chứ không test gì khác. Điều đó ngụ ý rằng mã có ý thức về luồng nên nhỏ và tập trung.

Know the possible sources of concurrency issues: multiple threads operating on shared data, or using a common resource pool. Boundary cases, such as starting up and shutting down cleanly or finishing

the iteration of a loop, can be especially thorny.

Hãy nắm được các nguồn có thể sinh ra vấn đề đồng thời: nhiều luồng thao tác trên dữ liệu dùng chung, hoặc cùng dùng một pool tài nguyên chung. Các trường hợp biên, chẳng hạn khởi động và tắt sao cho gọn ghẽ hay kết thúc một vòng lặp, có thể đặc biệt gai góc.

Learn your library and know the fundamental algorithms. Understand how some of the features offered by the library support solving problems similar to the fundamental algorithms.

Hãy học thư viện của bạn và nắm các thuật toán nền tảng. Hãy hiểu xem một số tính năng thư viện cung cấp hỗ trợ giải những bài toán tương tự các thuật toán nền tảng đó ra sao.

Learn how to find regions of code that must be locked, and lock them. Do not lock regions of code that do not need to be locked. Avoid calling one locked section from another. This requires a deep understanding of whether something is or is not shared. Keep the number of shared objects and the scope of the sharing as narrow as possible. Change designs of the objects with shared data to accommodate clients rather than forcing clients to manage shared state.

Hãy học cách tìm ra những vùng mã cần được khoá, và khoá chúng lại. Đừng khoá những vùng mã không cần khoá. Tránh gọi một vùng đã khoá từ bên trong một vùng khoá khác. Việc này đòi hỏi hiểu thật sâu xem thứ gì được chia sẻ và thứ gì không. Hãy giữ số đối tượng dùng chung và phạm vi (scope) chia sẻ hẹp nhất có thể. Hãy sửa thiết kế của những đối tượng mang dữ liệu dùng chung để chúng phục vụ client, thay vì bắt client phải tự quản lý trạng thái dùng chung.

Issues will crop up. The ones that do not crop up early are often written off as a one-time occurrence. These so-called one-offs typically only happen under load or at seemingly random times. Therefore, you need to be able to run your thread-related code in many configurations on many platforms repeatedly and continuously. Testability, which comes naturally from following a good testing **discipline**, implies some level of plug-ability, which offers the support necessary to run code in a wider range of configurations.

Vấn đề rồi sẽ nảy sinh. Những vấn đề không lộ ra sớm thường bị gạt đi như một sự cố chỉ xảy ra một lần rồi thôi. Máy thử “một lần rồi thôi” ấy thường chỉ xuất hiện khi hệ thống chịu tải, hoặc vào những thời điểm tưởng như ngẫu nhiên. Vì vậy, bạn cần có khả năng chạy mã liên quan tới luồng của mình trong nhiều cấu hình, trên nhiều nền tảng, lặp đi lặp lại và liên tục. Khả năng kiểm thử — thứ đến một cách tự nhiên khi bạn theo một kỷ luật (discipline) test tốt — kéo theo một mức độ khả năng cắm-thay nào đó, và chính điều đó cho bạn chỗ dựa cần thiết để chạy mã trong một dải cấu hình rộng hơn.

You will greatly improve your chances of finding erroneous code if you take the time to instrument your code. You can either do so by hand or use some kind of automated technology. Invest in this early. You want to be running your thread-based code as long as possible before you put it into production.

Bạn sẽ tăng đáng kể cơ hội tìm ra mã sai nếu chịu bỏ thời gian gần đo đạc vào mã. Bạn có thể làm bằng tay hoặc dùng một công nghệ tự động nào đó. Hãy đầu tư vào việc này từ sớm. Bạn muốn chạy mã dựa trên luồng của mình càng lâu càng tốt trước khi đưa nó lên production.

If you take a clean approach, your chances of getting it right increase drastically.

Nếu bạn theo một cách tiếp cận sạch sẽ, cơ hội làm đúng của bạn sẽ tăng vọt.

Part III — Phần III

Architecture — Kiến trúc

The **architecture** of a software system is the shape given to that system by those who build it. The form of that shape is in the division of that system into components, the arrangement of those components, and the ways in which those components communicate with each other.

Kiến trúc (architecture) của một hệ thống phần mềm là hình hài mà những người xây dựng nó đã trao cho nó. Hình hài ấy nằm ở cách chia hệ thống thành các component, ở cách sắp xếp các component đó, và ở cách chúng giao tiếp với nhau.

The purpose of that shape is to facilitate the development, deployment, operation, and maintenance of the software system contained within it.

Mục đích của hình hài ấy là làm cho việc phát triển, triển khai, vận hành và bảo trì hệ thống phần mềm nằm bên trong nó trở nên dễ dàng hơn.

The strategy behind that facilitation is to leave as many options open as possible, for as long as possible.

Chiến lược đằng sau chuyện “làm cho dễ dàng hơn” đó là: giữ cho càng nhiều lựa chọn còn ngỏ càng tốt, và giữ được càng lâu càng tốt.

Perhaps this statement has surprised you. Perhaps you thought that the goal of software architecture was to make the system work properly. Certainly we want the system to work properly; and certainly the architecture of the system must support that as one of its highest priorities.

Có lẽ câu vừa rồi làm bạn ngạc nhiên. Có lẽ bạn vẫn nghĩ mục tiêu của kiến trúc phần mềm là làm cho hệ thống chạy đúng. Dĩ nhiên chúng ta muốn hệ thống chạy đúng; và dĩ nhiên kiến trúc của hệ thống phải xem đó là một trong những ưu tiên cao nhất của mình.

However, the architecture of a system has very little bearing on whether that system works. There are many systems out there, with terrible architectures, that work just fine. Their troubles are not in their operation; they are in deployment, maintenance, and ongoing development.

Tuy vậy, kiến trúc của một hệ thống lại chẳng mấy liên quan đến chuyện hệ thống đó có chạy được hay không. Ngoài kia có vô số hệ thống với kiến trúc tệ hại mà vẫn chạy ngon lành. Rắc rối của chúng không nằm ở khâu vận hành; nó nằm ở khâu triển khai, bảo trì và phát triển tiếp.

This is not to say that architecture plays no role in supporting the proper **behavior** of the system. It certainly does, and the role is critical. But the role is passive and cosmetic, not active or essential. There are few, if any, behavioral options that the architecture of a system can leave open.

Nói vậy không có nghĩa là kiến trúc chẳng đóng vai trò gì trong việc hỗ trợ hành vi đúng đắn của hệ thống. Nó có vai trò, và vai trò đó rất quan trọng. Nhưng đó là vai trò thụ động và mang tính hình thức, chứ không chủ động hay thiết yếu. Số lựa chọn về hành vi mà kiến trúc của một hệ thống có thể để ngỏ là rất ít, nếu như có.

The primary purpose of architecture is to support the lifecycle of the system. **Good** architecture makes the system easy to understand, easy to develop, easy to maintain, and easy to deploy. The ultimate goal is to minimize the lifetime cost of the system and maximize programmer **productivity**.

Mục đích chính của kiến trúc là hỗ trợ vòng đời của hệ thống. Kiến trúc tốt làm cho hệ thống dễ hiểu, dễ phát triển, dễ bảo trì và dễ triển khai. Mục tiêu tối hậu là giảm thiểu chi phí trọn đời của hệ thống và tối đa hoá năng suất của lập trình viên.

Note: This chapter is an overview. For a more complete treatment of the topic, see [Clean Arch].

Lưu ý: Chương này chỉ là một cái nhìn tổng quan. Muốn tìm hiểu chủ đề này một cách đầy đủ hơn, xem [Clean Arch].

Chapter 23. The Two Values of Software — Chương 23. Hai giá trị của phần mềm



There are two values to software. There is the value of its behavior, and there is the value of its **structure**. The second of these is the greater of the two because it is this value that makes software soft .

Phần mềm có hai giá trị: giá trị của hành vi và giá trị của cấu trúc. Trong hai giá trị đó, giá trị thứ hai mới là giá trị lớn hơn, bởi chính nó là thứ khiến phần mềm *mềm* được — đúng như chữ *soft* trong *software*.

Software was invented because we needed a way to quickly and easily change the behavior of our machines. But that flexibility depends critically on the shape of the system, the arrangement of its components, and the way those components are interconnected.

Phần mềm được phát minh ra vì chúng ta cần một cách để thay đổi hành vi của máy móc thật nhanh và thật dễ. Nhưng sự linh hoạt đó phụ thuộc một cách sống còn vào hình hài của hệ thống, vào cách sắp xếp các component của nó, và vào cách những component ấy nối kết với nhau.

Keeping Options Open — Giữ cho các lựa chọn còn ngỏ

The way you keep software soft is to leave as many options open as possible, for as long as possible. What are the options that we need to leave open? They are the details that don't matter.

Cách để giữ cho phần mềm còn mềm là để ngỏ càng nhiều lựa chọn càng tốt, và để ngỏ càng lâu càng tốt. Vậy những lựa chọn nào cần được để ngỏ? Đó là những chi tiết không quan trọng.

All software systems can be decomposed into two major elements: policy and detail. The policy element embodies all the business rules and procedures. The policy is where the true value of the system lives.

Mọi hệ thống phần mềm đều có thể phân rã thành hai thành phần lớn: chính sách (policy) và chi tiết (detail). Phần chính sách hàm chứa toàn bộ quy tắc nghiệp vụ (business rule) và các quy trình. Chính sách là nơi giá trị thật sự của hệ thống trú ngụ.

The details are those things that are necessary to enable humans, other systems, and programmers to communicate with the policy, but do not impact the behavior of the policy at all. These include IO devices, databases, Web systems, servers, frameworks, and communication protocols, among others.

Chi tiết là những thứ cần thiết để con người, các hệ thống khác và lập trình viên có thể giao tiếp với chính sách, nhưng lại hoàn toàn không tác động gì đến hành vi của chính sách. Chúng bao gồm thiết bị IO, cơ sở dữ liệu, hệ thống Web, máy chủ, framework, giao thức truyền thông, và nhiều thứ khác nữa.

The goal of architecture is to imbue the system with a shape that recognizes policy as the most essential element of the system while making the details irrelevant to that policy. This allows decisions about those details to be delayed and deferred .

Mục tiêu của kiến trúc là trao cho hệ thống một hình hài biết thừa nhận chính sách là thành phần thiết yếu nhất của hệ thống, đồng thời làm cho các chi tiết trở nên vô can đối với chính sách đó. Nhờ vậy, những quyết định về các chi tiết ấy có thể được trì hoãn và để dành lại sau.

For example:

Ví dụ:

- It is not necessary to choose a database system in the early days of development, because the high-level policy should not care what kind of database will be used. Indeed, if you are careful, the high-level policy will not care if the database is relational, distributed, hierarchical, or just plain flat files.
- It is not necessary to choose a Web server early in development, because the high-level policy should not know that it is being delivered over the Web. If the high-level policy is unaware of HTML, CSS, AJAX, JSP, JSF, or any of the rest of the alphabet soup of web development, then you don't need to decide which Web system to use until much later in the project. Indeed, you don't even have to decide if the system will be delivered over the Web .
- It is not necessary to adopt REST early in development, because the high-level policy should be agnostic about the **interface** to the outside world. Nor is it necessary to adopt a microservices framework, or an SOA framework, or REACT . . . Again, the high-level policies should not care about these things.
- It is not necessary to adopt a **dependency** injection framework early in development, because the high-level policy should not care how dependencies are resolved.

- Không cần phải chọn hệ quản trị cơ sở dữ liệu ngay những ngày đầu phát triển, bởi chính sách mức cao lẽ ra không nên bận tâm sẽ dùng loại cơ sở dữ liệu nào. Thật vậy, nếu bạn cẩn thận, chính sách mức cao sẽ chẳng buồn quan tâm cơ sở dữ liệu là quan hệ, phân tán, phân cấp, hay chỉ là mấy file phẳng bình thường.
- Không cần phải chọn máy chủ Web từ sớm, bởi chính sách mức cao lẽ ra không nên biết rằng nó đang được phân phối qua Web. Nếu chính sách mức cao chẳng hay biết gì về HTML, CSS, AJAX, JSP, JSF, hay bất kỳ món nào khác trong mớ bong bóng chữ viết tắt của giới lập trình web, thì bạn chẳng cần quyết định dùng hệ thống Web nào cho tới tận rất muộn về sau trong dự án. Thật ra, bạn thậm chí còn chưa cần quyết định xem hệ thống có được phân phối qua Web hay không.
- Không cần phải chọn REST từ sớm, bởi chính sách mức cao lẽ ra phải trung dung với giao diện hướng ra thế giới bên ngoài. Cũng chẳng cần chọn một framework microservices, hay một framework SOA, hay REACT... Một lần nữa, các chính sách mức cao lẽ ra không nên bận tâm đến những thứ này.
- Không cần phải chọn một framework dependency injection từ sớm, bởi chính sách mức cao lẽ ra không nên quan tâm các phụ thuộc (dependency) được giải quyết bằng cách nào.

I think you get the point. If you can develop the high-level policy without committing to the details that surround it, you can delay and defer decisions about those details for a long time. And the longer you wait to make those decisions, the more information you have with which to make them properly.

Tôi nghĩ bạn đã nắm được ý. Nếu bạn có thể phát triển chính sách mức cao mà không phải cam kết với đám chi tiết bao quanh nó, thì bạn có thể trì hoãn và để dành các quyết định về những chi tiết đó thật lâu. Và càng chờ lâu mới ra quyết định, bạn càng có nhiều thông tin để quyết định cho đúng.

This also leaves you the option to try different experiments. If you have a portion of the high-level policy working and it is agnostic about the database, you could try connecting it to several different databases in order to check applicability and performance. The same is true with Web systems, Web frameworks, or even the Web itself.

Điều này còn để lại cho bạn lựa chọn thử nghiệm nhiều phương án khác nhau. Nếu bạn đã có một phần chính sách mức cao chạy được và nó trung dung với cơ sở dữ liệu, bạn có thể thử nối nó vào vài cơ sở dữ liệu khác nhau để kiểm tra mức độ phù hợp và hiệu năng. Với hệ thống Web, framework Web, hay thậm chí bản thân Web, chuyện cũng y hệt.

The longer you leave options open, the more experiments you can run, the more things you can try, and the more information you will have when those decisions can no longer be deferred.

Càng để ngỏ các lựa chọn lâu, bạn càng chạy được nhiều thử nghiệm, càng thử được nhiều thứ, và càng có nhiều thông tin trong tay khi những quyết định đó không thể trì hoãn thêm được nữa.

What if the decisions have already been made by someone else? What if your company has made a commitment to a certain database, or a certain Web server, or a certain framework? I suggest that you pretend that those decisions have not been made, and give the system a shape that allows those decisions to be deferred for as long as possible. You never know when decisions like that might suddenly change.

Thế còn nếu các quyết định đã được người khác chốt sẵn rồi thì sao? Nếu công ty bạn đã cam kết dùng một cơ sở dữ liệu nào đó, một máy chủ Web nào đó, hay một framework nào đó thì sao? Tôi đề nghị bạn cứ giả vờ như những quyết định ấy chưa hề được đưa ra, và hãy trao cho hệ thống một hình hài cho phép trì hoãn chúng lâu nhất có thể. Bạn chẳng

bao giờ biết được khi nào những quyết định kiểu đó đột nhiên đổi khác.

A good architecture maximizes the number of decisions not made.

Một kiến trúc tốt là kiến trúc tối đa hoá số quyết định *chưa* được đưa ra.

Chapter 24. Independence —

Chương 24. Tính độc lập



A **good architecture** must support

Một kiến trúc (architecture) tốt phải hỗ trợ được

- The use cases of the system
 - The operation of the system
 - The development of the system
 - The deployment of the system
- Các use case của hệ thống
 - Việc vận hành hệ thống
 - Việc phát triển hệ thống
 - Việc triển khai hệ thống

Use Cases — Use case

The first bullet—use cases—means that the architecture of the system must support the intent of the system. If the system is a shopping cart application, then the architecture must support shopping cart use cases. Indeed, this is the first priority of the architecture. The architecture must support the use cases.

Gạch đầu dòng thứ nhất — các use case — có nghĩa là kiến trúc của hệ thống phải hỗ trợ được ý đồ của hệ thống. Nếu hệ thống là một ứng dụng giỏ hàng, thì kiến trúc phải hỗ trợ các use case của giỏ hàng. Thật vậy, đây là ưu tiên số một của kiến trúc. Kiến trúc phải hỗ trợ các use case.

However, as we discussed previously, architecture does not wield much influence over the **behavior** of the system. There are very few behavioral options that the architecture can leave open. But influence isn't everything. The most important thing a good architecture can do to support behavior is to clarify and expose that behavior so that the intent of the system is visible at the architectural level.

Tuy nhiên, như đã bàn ở phần trước, kiến trúc không có mấy ảnh hưởng lên hành vi của hệ thống. Có rất ít lựa chọn về hành vi mà kiến trúc có thể để ngỏ. Nhưng ảnh hưởng không phải là tất cả. Điều quan trọng nhất mà một kiến trúc tốt có thể làm để hỗ trợ hành vi là làm rõ và phơi bày hành vi đó ra, sao cho ý đồ của hệ thống hiện lên ngay ở mức kiến trúc.

A shopping cart application with a good architecture will look like a shopping cart application. The use cases of that system will be plainly visible within the **structure** of that system. Developers will not have to hunt for behaviors, because those behaviors will be first-class elements visible at the top level of the system. Those elements will be classes or functions or modules that have prominent positions within the architecture and will have names that clearly describe their **function**.

Một ứng dụng giỏ hàng có kiến trúc tốt sẽ trông giống một ứng dụng giỏ hàng. Các use case của hệ thống đó sẽ hiện ra rành rành trong chính cấu trúc của hệ thống. Lập trình viên sẽ không phải đi truy lùng các hành vi, bởi những hành vi ấy là các thành phần hạng nhất, nhìn thấy được ngay ở tầng trên cùng của hệ thống. Chúng sẽ là những class, hàm hoặc module nằm ở vị trí nổi bật trong kiến trúc và mang những cái tên mô tả rõ ràng chức năng của chúng.

Operation — Vận hành

Architecture plays a more substantial, and less cosmetic, role in supporting the operation of the system. If the system must handle 100,000 transactions per second, the architecture must support that kind of throughput and response time for each **use case** that demands it. If the system must query big **data** cubes in milliseconds, then the architecture must be structured to allow this kind of operation.

Kiến trúc đóng một vai trò thực chất hơn, ít mang tính hình thức hơn, trong việc hỗ trợ vận hành hệ thống. Nếu hệ thống phải xử lý 100.000 giao dịch mỗi giây, kiến trúc phải hỗ trợ được mức thông lượng và thời gian phản hồi đó cho từng use case có yêu cầu như vậy. Nếu hệ thống phải truy vấn những khối dữ liệu lớn trong vài mili-giây, thì kiến trúc phải được tổ chức sao cho cho phép kiểu vận hành ấy.

For some systems, this will mean arranging the processing elements of the system into an array of little services that can be run in parallel on many different servers. For other systems, it will mean a plethora of little lightweight threads sharing the address space of a single process within a single processor. Still other systems will need just a few processes running in isolated address spaces. And some systems can even survive as simple monolithic programs running in a single process.

Với một số hệ thống, điều này có nghĩa là sắp xếp các thành phần xử lý của hệ thống thành một dãy các dịch vụ nhỏ có thể chạy song song trên nhiều máy chủ khác nhau. Với những hệ thống khác, nó lại có nghĩa là hàng loạt luồng (thread) nhỏ, nhẹ cùng chia sẻ không gian địa chỉ của một tiến trình duy nhất trong một bộ xử lý duy nhất. Lại có những hệ thống

chỉ cần vài tiến trình chạy trong các không gian địa chỉ tách biệt. Và một số hệ thống thậm chí vẫn sống tốt dưới dạng chương trình nguyên khối đơn giản chạy trong một tiến trình duy nhất.

As strange as it may seem, this decision is one of the options that a good architecture leaves open. A system that is written as a monolith and that depends upon that monolithic structure cannot easily be upgraded to multiple processes, multiple threads, or microservices should the need arise. On the other hand, an architecture that maintains the proper isolation of its components and does not assume the means of communication between those components will be much easier to transition through the spectrum of threads, processes, and services as the operational needs of the system change over time.

Nghe có vẻ lạ, nhưng quyết định này lại chính là một trong những lựa chọn mà một kiến trúc tốt để ngỏ. Một hệ thống được viết như một khối nguyên khối và phụ thuộc vào chính cấu trúc nguyên khối đó thì không dễ gì nâng cấp lên nhiều tiến trình, nhiều luồng hay microservice khi nhu cầu phát sinh. Ngược lại, một kiến trúc duy trì được sự cô lập đúng mức giữa các component của nó, và không mặc định trước phương tiện giao tiếp giữa những component ấy, sẽ dễ dàng chuyển dịch qua lại trong dải luồng — tiến trình — dịch vụ hơn rất nhiều khi nhu cầu vận hành của hệ thống thay đổi theo thời gian.

Development — Phát triển

Architecture plays a significant role in supporting the development environment. This is where Conway's law and the Single Responsibility Principle (SRP) come into play. A system that must be developed by an organization with many teams and many concerns must have an architecture that facilitates independent actions by those teams so that the teams do not interfere with each other during development.

Kiến trúc đóng vai trò đáng kể trong việc hỗ trợ môi trường phát triển. Đây là chỗ định luật Conway và SRP — Nguyên tắc Trách nhiệm Đơn nhất (Single Responsibility Principle) phát huy tác dụng. Một hệ thống phải được phát triển bởi một tổ chức có nhiều đội và nhiều mối bận tâm khác nhau thì buộc phải có kiến trúc tạo điều kiện cho các đội đó hành động độc lập, sao cho họ không giẫm chân nhau trong lúc phát triển.

This is accomplished by properly partitioning the system into well-isolated, independently developable components. Those components can then be allocated to teams that can work independently of each other.

Điều này đạt được bằng cách phân hoạch hệ thống một cách hợp lý thành các component được cô lập tốt và có thể phát triển độc lập. Khi đó, các component ấy có thể được phân bổ cho những đội làm việc độc lập với nhau.

Deployment — Triển khai

Finally, the architecture plays a huge role in the ease with which the system is deployed. The goal is “immediate deployment.” A good architecture does not rely on dozens of little configuration scripts and property file tweaks. It does not require manual creation of directories or files that must be arranged just so. A good architecture helps the system be immediately deployable after each build.

Cuối cùng, kiến trúc đóng vai trò cực lớn trong việc hệ thống được triển khai dễ dàng đến đâu. Mục tiêu là “triển khai tức thì”. Một kiến trúc tốt không dựa vào hàng tá script cấu hình lặt vặt và những lần chỉnh sửa file thuộc tính. Nó không đòi hỏi phải tạo tay các thư

mục hay file rồi sắp đặt chúng đúng y một kiểu nào đó. Một kiến trúc tốt giúp hệ thống có thể triển khai được ngay sau mỗi lần build.

Again, this is achieved through the proper partitioning and isolation of the components of the system, including those master components that tie the whole system together and ensure that each **component** is properly started, integrated, and supervised.

Một lần nữa, điều này đạt được nhờ phân hoạch và cô lập đúng cách các component của hệ thống, bao gồm cả những component chủ đạo có nhiệm vụ gắn kết toàn hệ thống lại với nhau và bảo đảm mỗi component được khởi động, tích hợp và giám sát đúng cách.

Leaving Options Open — Giữ cho các lựa chọn còn ngỏ

A good architecture balances all the above concerns with a component structure that mutually satisfies them all. Sounds easy, right? Well, it's easy for me to write that.

Một kiến trúc tốt cân bằng được tất cả những mối bận tâm nói trên bằng một cấu trúc component thoả mãn đồng thời tất cả chúng. Nghe dễ nhỉ? Ừ thì, viết ra câu đó quả là dễ với tôi.

The reality is that this is pretty hard. The problem is that most of the time we don't know what all the use cases are, nor do we know the operational constraints, the team structure, or the deployment requirements. Worse, even if we did know them, they will all change as the system moves through its lifecycle. In short, the goals we must meet are indistinct and inconstant. Welcome to the real world.

Thực tế là chuyện này khá khó. Vấn đề nằm ở chỗ hầu hết thời gian chúng ta không biết hết các use case là những gì, cũng chẳng biết các ràng buộc vận hành, cơ cấu đội ngũ, hay yêu cầu triển khai. Tệ hơn nữa, cho dù có biết đi chẳng nữa thì tất cả chúng cũng sẽ thay đổi khi hệ thống đi qua vòng đời của nó. Nói ngắn gọn: những mục tiêu ta phải đáp ứng vừa mờ mịt vừa không đứng yên. Chào mừng bạn đến với thế giới thực.

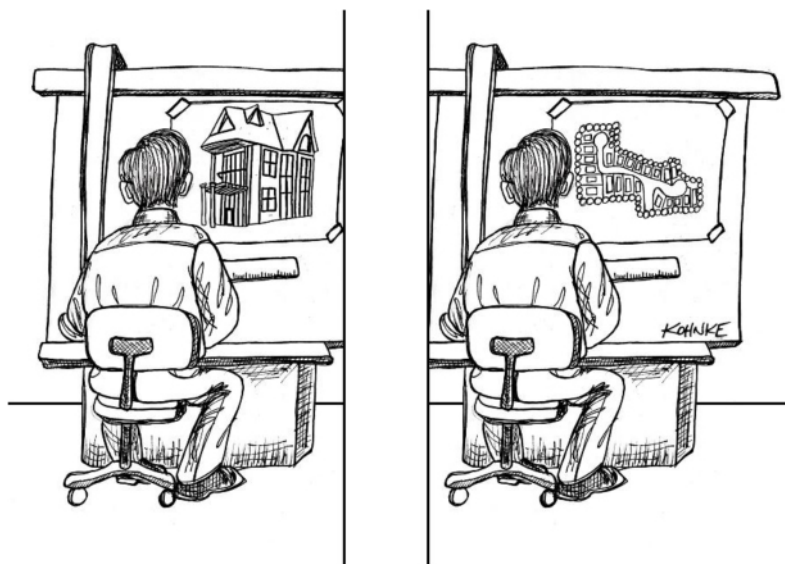
But all is not lost. There are principles of architecture that are relatively inexpensive to implement and that can help balance those concerns, even when you don't have a clear picture of the goals you have to hit. Those principles help us partition our systems into well-isolated components that allow us to leave as many options open as possible, for as long as possible.

Nhưng chưa mất hết đâu. Có những nguyên tắc kiến trúc tương đối rẻ để áp dụng mà vẫn giúp cân bằng các mối bận tâm đó, ngay cả khi bạn chưa hình dung rõ những mục tiêu mình phải trúng. Những nguyên tắc ấy giúp ta phân hoạch hệ thống thành các component được cô lập tốt, cho phép ta giữ cho càng nhiều lựa chọn còn ngỏ càng tốt, và giữ được càng lâu càng tốt.

A good architecture makes the system easy to change, in all the ways that it must change, by leaving options open.

Một kiến trúc tốt làm cho hệ thống dễ thay đổi, theo mọi cách mà nó buộc phải thay đổi, bằng cách giữ cho các lựa chọn còn ngỏ.

Chapter 25. Architectural Boundaries — Chương 25. Ranh giới kiến trúc



Software **architecture** is the art of drawing lines that I call boundaries . Those boundaries separate software elements from one another, and restrict those on one side from knowing about those on the other. Some of those lines are drawn very early in a project's life—even before any code is written. Others are drawn much later. Those that are drawn early are drawn for the purpose of deferring decisions for as long as possible, and of keeping those decisions from polluting the core business logic.

Kiến trúc (architecture) phần mềm là nghệ thuật vạch ra những đường kẻ mà tôi gọi là ranh giới (boundary). Những ranh giới ấy tách các thành phần phần mềm ra khỏi nhau, và ngăn không cho bên này biết về bên kia. Có những đường được vạch từ rất sớm trong đời một dự án — thậm chí trước khi viết dòng mã nào. Có những đường được vạch muộn hơn nhiều. Những đường vạch sớm là để trì hoãn các quyết định càng lâu càng tốt, và để những quyết định đó không làm ô nhiễm phần logic nghiệp vụ cốt lõi.

Remember that the goal of an architecture is to minimize the manpower required to build and maintain the required system. What is it that saps manpower? **Coupling** ; and especially, coupling to premature decisions.

Hãy nhớ rằng mục tiêu của một kiến trúc là giảm thiểu nhân lực cần bỏ ra để xây dựng và

bảo trì hệ thống. Vậy cái gì bảo mòn nhân lực? Chính là độ kết dính giữa các thành phần (coupling); và đặc biệt là kết dính vào những quyết định vội vàng.

What kinds of decisions are premature? Decisions that have nothing to do with the business requirements—the use cases—of the system. These include decisions about frameworks, databases, Web servers, utility libraries, **dependency** injection, and the like. A **good** system architecture is one in which decisions like these are rendered ancillary and deferrable. A good system architecture does not depend on those decisions. A good system architecture allows those decisions to be made at the latest possible moment, without significant impact.

Những quyết định nào là vội vàng? Đó là các quyết định chẳng liên quan gì tới yêu cầu nghiệp vụ — tới các use case — của hệ thống. Chúng bao gồm quyết định về framework, cơ sở dữ liệu, Web server, thư viện tiện ích, dependency injection, và những thứ tương tự. Một kiến trúc hệ thống tốt là kiến trúc biến những quyết định kiểu đó thành thứ yếu và có thể hoãn lại. Một kiến trúc hệ thống tốt không phụ thuộc vào chúng. Một kiến trúc hệ thống tốt cho phép ra những quyết định đó vào thời điểm muộn nhất có thể, mà không gây ảnh hưởng đáng kể.

What Lines Do You Draw, and When? — Bạn vạch những đường nào, và vạch khi nào?

You draw lines to separate things that matter from things that don't. The GUI doesn't matter to the business rules, so there should be a line between them. The database doesn't matter to the GUI, so there should be a line between them. The database doesn't matter to the business rules, so there should be a line between them.

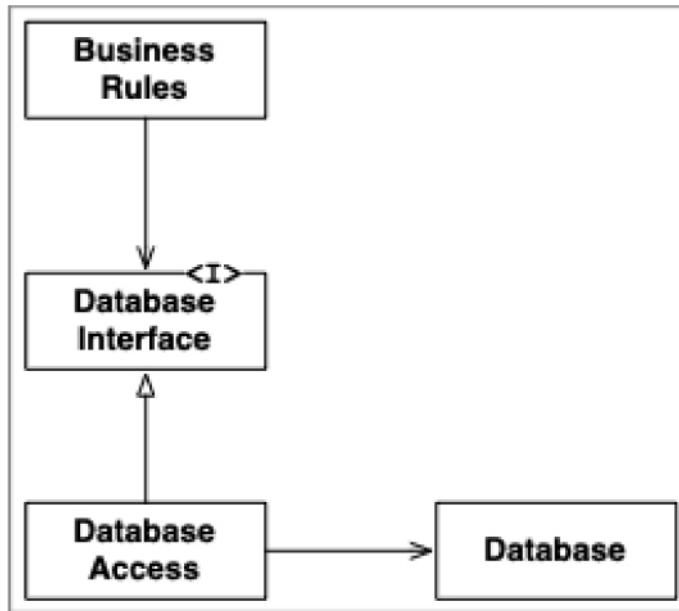
Bạn vạch đường để tách những thứ quan trọng khỏi những thứ không quan trọng. GUI không quan trọng đối với các quy tắc nghiệp vụ (business rule), vậy nên giữa chúng phải có một đường kẻ. Cơ sở dữ liệu không quan trọng đối với GUI, vậy nên giữa chúng phải có một đường kẻ. Cơ sở dữ liệu không quan trọng đối với các quy tắc nghiệp vụ, vậy nên giữa chúng phải có một đường kẻ.

Some of you may have rejected one or more of those statements; especially the part about the business rules not caring about the database. Many of us have been taught to believe that the database is inextricably connected to the business rules. Some of us have even been convinced that the database is the embodiment of the business rules.

Có thể vài người trong các bạn không chấp nhận một hoặc nhiều câu vừa rồi; nhất là cái đoạn bảo rằng quy tắc nghiệp vụ chẳng bận tâm tới cơ sở dữ liệu. Nhiều người trong chúng ta được dạy để tin rằng cơ sở dữ liệu gắn chặt không thể tách rời với quy tắc nghiệp vụ. Một số còn bị thuyết phục rằng cơ sở dữ liệu chính là hiện thân của quy tắc nghiệp vụ.

But this idea is misguided. The database is a tool that the business rules can use indirectly. The business rules don't need to know about the schema, or the query language, or any of the other details about the database. All the business rules need to know is that there is a set of functions that can be used to fetch or save **data**. This allows us to put the database behind an **interface**.

Nhưng ý nghĩ đó là lầm lạc. Cơ sở dữ liệu là một công cụ mà quy tắc nghiệp vụ có thể sử dụng một cách gián tiếp. Quy tắc nghiệp vụ không cần biết về schema, về ngôn ngữ truy vấn, hay bất kỳ chi tiết nào khác của cơ sở dữ liệu. Tất cả những gì quy tắc nghiệp vụ cần biết là có một tập hàm để lấy hoặc lưu dữ liệu. Điều đó cho phép chúng ta đặt cơ sở dữ liệu ra sau một interface.



You can see this clearly in the diagram above. The

Bạn có thể thấy rõ điều này trong sơ đồ phía trên. Các

BusinessRules use the DatabaseInterface to load and save data. The DatabaseAccess implements the interface and directs operation of the actual Database .

BusinessRules dùng DatabaseInterface để nạp và lưu dữ liệu. DatabaseAccess hiện thực interface đó và điều khiển Database thật sự.

The classes and interfaces in this diagram are symbolic. In a real application, there would be many **business rule** classes, many database interface classes, and many database access implementations. But they would all follow roughly the same pattern.

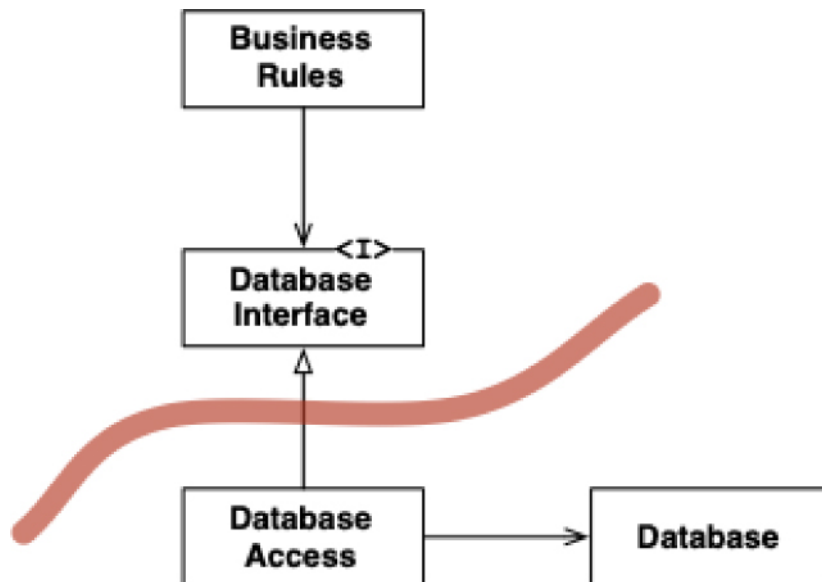
Các class và interface trong sơ đồ này chỉ mang tính tượng trưng. Trong một ứng dụng thật, sẽ có rất nhiều class quy tắc nghiệp vụ, rất nhiều class interface cơ sở dữ liệu, và rất nhiều hiện thực (implementation) truy cập cơ sở dữ liệu. Nhưng tất cả đều đi theo đại thể cùng một khuôn mẫu.

Where is the boundary line? The boundary is drawn across the **inheritance**/implements relationship, just below the

Đường ranh giới nằm ở đâu? Ranh giới được vạch cắt ngang quan hệ kế thừa (inheritance)/implements, ngay bên dưới

DatabaseInterface , as shown in the following diagram.

DatabaseInterface, như trong sơ đồ sau.



Note the two arrows leaving the DatabaseAccess **class**. Those two arrows point away from the DatabaseAccess class. That means that none of these classes knows that the

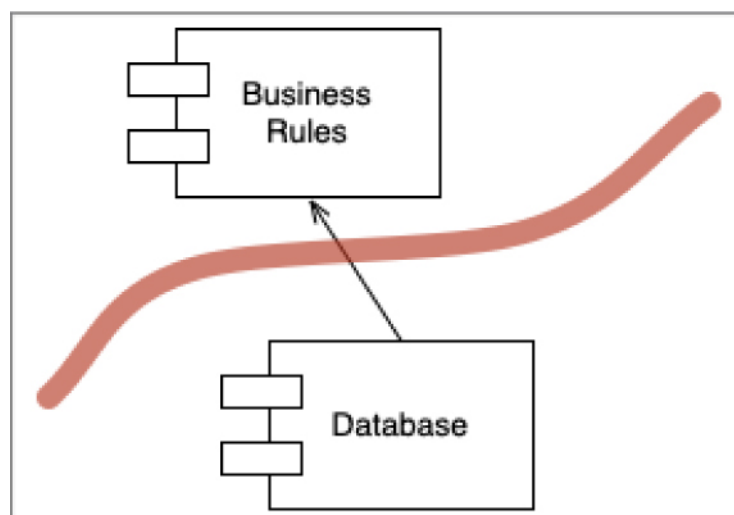
Hãy để ý hai mũi tên đi ra khỏi class DatabaseAccess. Cả hai mũi tên đó đều trở ra xa khỏi class DatabaseAccess. Điều đó có nghĩa là không class nào trong số này biết rằng class

DatabaseAccess class exists. That's the line.

DatabaseAccess tồn tại. Đó chính là đường ranh giới.

Now let's pull back a bit. We'll look at the **component** that contains many business rules, and the component that contains the database and all its access classes.

Giờ hãy lùi ra xa một chút. Chúng ta sẽ nhìn vào component chứa nhiều quy tắc nghiệp vụ, và component chứa cơ sở dữ liệu cùng toàn bộ các class truy cập của nó.



Note the direction of the arrow. The Database knows about the

Hãy để ý hướng của mũi tên. Database biết về

BusinessRules . The BusinessRules do not know about the Database . This implies that the

DatabaseInterface classes live in the

BusinessRules. Còn BusinessRules thì không hề biết về Database. Điều này hàm ý rằng các class DatabaseInterface sống trong component

BusinessRules component, and the DatabaseAccess classes live in the Database component.

BusinessRules, còn các class DatabaseAccess sống trong component Database.

The direction of this arrow is important. It shows that the

Hướng của mũi tên này rất quan trọng. Nó cho thấy

Database does not matter to the BusinessRules . But the Database cannot exist without the BusinessRules .

Database không quan trọng đối với BusinessRules. Nhưng Database lại không thể tồn tại nếu không có BusinessRules.

If that seems strange to you, just remember: That Database component contains the code that translates the calls made by the BusinessRules into the query language of the database. It is that translation code that knows about the BusinessRules .

Nếu điều đó nghe có vẻ lạ, hãy nhớ lại: component Database chứa đoạn mã chuyển các lời gọi từ BusinessRules thành ngôn ngữ truy vấn của cơ sở dữ liệu. Chính đoạn mã chuyển đổi đó mới là thứ biết về BusinessRules.

Having drawn this boundary line between the two components, and having set the direction of the arrow toward the BusinessRules , we can now see that the

Sau khi đã vạch đường ranh giới này giữa hai component, và đã đặt hướng mũi tên trở về phía BusinessRules, giờ ta có thể thấy rằng

BusinessRules could use any kind of database at all. The Database component could be replaced with many different implementations. The BusinessRules don't care.

BusinessRules có thể dùng bất kỳ loại cơ sở dữ liệu nào cũng được. Component Database có thể được thay bằng vô số hiện thực (implementation) khác nhau. BusinessRules chẳng bận tâm.

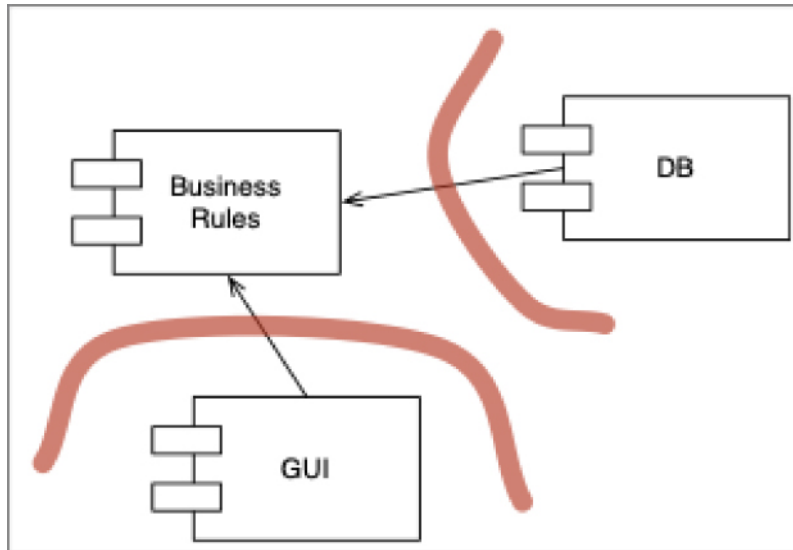
The database could be implemented with Oracle, or MySQL, or Couch, or Datomic, or even flat files. The business rules don't care at all. And that means that the database decision can be deferred and you can focus on getting the business rules written and tested before you have to make the database decision.

Cơ sở dữ liệu có thể được hiện thực bằng Oracle, hay MySQL, hay Couch, hay Datomic, hay thậm chí là các tệp phẳng. Quy tắc nghiệp vụ hoàn toàn không quan tâm. Và điều đó có nghĩa là quyết định về cơ sở dữ liệu có thể hoãn lại, còn bạn thì tập trung viết và kiểm thử xong quy tắc nghiệp vụ trước khi buộc phải chọn cơ sở dữ liệu.

Plug-in Architecture — Kiến trúc plugin

Look at the two boundaries in the diagram below. They separate the Database and the GUI from the BusinessRules . The arrows point from the lower-level components to the high-level BusinessRules component.

Hãy nhìn hai đường ranh giới trong sơ đồ dưới đây. Chúng tách Database và GUI ra khỏi BusinessRules. Các mũi tên trở từ những component mức thấp về phía component BusinessRules mức cao.



The GUI and DB are plug-ins to the BusinessRules. This is a pattern that can be repeated for many different kinds of components. The core business rules can be kept separate from, and independent of, any components that either are optional or can be implemented in many different forms.

GUI và DB là các plug-in cắm vào BusinessRules. Đây là một khuôn mẫu có thể lặp lại cho rất nhiều loại component khác nhau. Phần quy tắc nghiệp vụ cốt lõi có thể được giữ tách biệt và độc lập với mọi component vốn hoặc là tùy chọn, hoặc có thể hiện thực theo nhiều hình thức khác nhau.

So, because the user interface in this design is considered to be a plug-in, we have made it possible to plug in many different kinds of user interfaces. They could be Web based, client/server based, SOA based, console based, or based on any other kind of user interface technology.

Vậy nên, vì giao diện người dùng trong thiết kế này được xem là một plug-in, chúng ta đã mở đường để cắm vào đủ loại giao diện người dùng khác nhau. Chúng có thể dựa trên Web, dựa trên client/server, dựa trên SOA, dựa trên console, hay dựa trên bất kỳ công nghệ giao diện nào khác.

The same is true of the database. Since we have chosen to treat it as a plug-in, we can replace it with any of the various SQL databases, or a NoSQL database, or a file system-based database, or any other kind of database technology we might deem necessary in the future.

Với cơ sở dữ liệu cũng vậy. Vì đã chọn đối xử với nó như một plug-in, chúng ta có thể thay nó bằng bất kỳ cơ sở dữ liệu SQL nào, hay một cơ sở dữ liệu NoSQL, hay một cơ sở dữ liệu dựa trên hệ thống tệp, hay bất kỳ công nghệ cơ sở dữ liệu nào khác mà sau này ta thấy cần.

These replacements might not be trivial. If the initial deployment of our system was Web based, then writing the plug-in for a client/server UI could be challenging. It is likely that some of the communications between the BusinessRules and the new UI would have to be reworked. But by starting with the presumption of a plug-in **structure**, we have, at the very least, made such a change practical.

Những cuộc thay thế đó có thể không hề đơn giản. Nếu bản triển khai đầu tiên của hệ

thống là dựa trên Web, thì viết plug-in cho một UI client/server có thể là chuyện khó nhằn. Nhiều khả năng một phần giao tiếp giữa BusinessRules và UI mới sẽ phải làm lại. Nhưng bằng cách khởi đầu với giả định về một cấu trúc plug-in, ít nhất chúng ta cũng đã khiến một thay đổi như thế trở nên khả thi.

Case Study: FitNesse — Nghiên cứu tình huống: FitNesse

My son, Micah, and I started work on FitNesse in 2001. The idea was to create a simple wiki that wrapped Ward Cunningham's FIT tool for writing acceptance tests. We wrote FitNesse in Java.

Con trai tôi, Micah, và tôi bắt tay vào làm FitNesse năm 2001. Ý tưởng là tạo ra một wiki đơn giản bọc quanh công cụ FIT của Ward Cunningham để viết acceptance test (kiểm thử chấp nhận). Chúng tôi viết FitNesse bằng Java.

I was adamant that anything we produced should not require people to download more than one jar file. I called this rule Download and Go . This rule drove many of our decisions.

Tôi kiên quyết rằng bất cứ thứ gì chúng tôi làm ra cũng không được bắt người dùng tải về quá một tệp jar. Tôi gọi quy tắc này là Download and Go. Quy tắc đó chi phối rất nhiều quyết định của chúng tôi.

One of the first was to write our own Web server, specific to the needs of FitNesse . This might sound absurd. Even in 2001 there were plenty of open source Web servers that we could have used. Yet writing our own turned out to be a really good decision because a bare-bones Web server is a very simple piece of software to write, and it allowed us to postpone any Web framework decision until much later. 1

Một trong những quyết định đầu tiên là tự viết lấy Web server riêng, do ni đóng giày cho nhu cầu của FitNesse. Nghe thì có vẻ vô lý. Ngay cả năm 2001 cũng đã có ê hề Web server mã nguồn mở mà chúng tôi có thể dùng. Vậy mà tự viết lại hoá ra là một quyết định rất tốt, bởi một Web server trần trụi là thứ phần mềm cực kỳ đơn giản để viết, và nó cho phép chúng tôi hoãn mọi quyết định về Web framework tới tận rất lâu sau đó. 1

1 Many years later we were able to slip the Velocity framework into FitNesse.

1 Nhiều năm sau, chúng tôi mới lồng được framework Velocity vào FitNesse.

Another early decision was to avoid thinking about a database. We had MySQL in the back of our minds, but we purposely delayed that decision by employing a design that made the decision irrelevant. That design was simply to put an interface between all data accesses and the data repository itself.

Một quyết định sớm khác là tránh nghĩ tới cơ sở dữ liệu. Trong đầu chúng tôi vẫn lớn vồn MySQL, nhưng chúng tôi cố ý trì hoãn quyết định đó bằng cách dùng một thiết kế khiến nó trở nên vô nghĩa. Thiết kế ấy đơn giản là đặt một interface giữa mọi thao tác truy cập dữ liệu và bản thân kho dữ liệu.

We put the data access methods into an interface named

Chúng tôi đặt các method truy cập dữ liệu vào một interface tên là

WikiPage . Those methods provided all the functionality we needed to find, fetch, and save pages. Of course, we didn't implement those methods at first; we simply stubbed them out while we worked on features that didn't involve fetching and saving the data.

WikiPage. Những method đó cung cấp đủ mọi chức năng chúng tôi cần để tìm, nạp và lưu các trang. Tất nhiên, ban đầu chúng tôi không hiện thực chúng; chúng tôi chỉ để chúng dưới dạng stub trong lúc làm những tính năng không dính dáng gì tới việc nạp và lưu dữ liệu.

Indeed, for three months we simply worked on translating wiki text into HTML. This didn't require any kind of data storage, so we created a class named MockWikiPage that simply left the data access methods stubbed.

Quả thật, suốt ba tháng chúng tôi chỉ làm mỗi việc chuyển văn bản wiki thành HTML. Việc này chẳng cần bất kỳ dạng lưu trữ dữ liệu nào, nên chúng tôi tạo một class tên là MockWikiPage, chỉ đơn giản để nguyên các method truy cập dữ liệu dưới dạng stub.

Eventually, of course, those stubs were insufficient for the features we wanted to write. We needed real data access, not stubs. So we created a new derivative of WikiPage, named

Rồi tất nhiên, đến lúc những stub đó không còn đủ cho các tính năng chúng tôi muốn viết. Chúng tôi cần truy cập dữ liệu thật, không phải stub. Thế là chúng tôi tạo một dẫn xuất mới của WikiPage, tên là

InMemoryPage. This derivative implemented the data access **method** to manage a hash table of wiki pages, which we kept in RAM.

InMemoryPage. Dẫn xuất này hiện thực method truy cập dữ liệu để quản lý một hash table các trang wiki, giữ trong RAM.

This allowed us to write feature after feature for a full year. In fact, we got the whole first version of the FitNesse program working this way. We could create pages, link to other pages, do all the fancy wiki **formatting**, and even run tests with FIT. What we couldn't do was save any of our work.

Cách đó cho phép chúng tôi viết hết tính năng này tới tính năng khác suốt trọn một năm. Thực tế, chúng tôi đã làm cho cả phiên bản đầu tiên của chương trình FitNesse chạy được theo kiểu ấy. Chúng tôi có thể tạo trang, liên kết sang trang khác, làm đủ mọi kiểu định dạng wiki hoa mỹ, thậm chí chạy test bằng FIT. Thứ duy nhất chúng tôi không làm được là lưu lại bất cứ thành quả nào.

So, when it came time to implement persistence, we thought again about MySQL, but we decided that wasn't necessary in the short term because it was really easy to write the hash tables out to flat files. So we implemented

Rồi khi đến lúc phải hiện thực việc lưu trữ lâu dài, chúng tôi lại nghĩ đến MySQL, nhưng rồi quyết định rằng trước mắt chưa cần, bởi việc ghi các hash table ra tệp phẳng thật sự quá dễ. Thế là chúng tôi hiện thực

FileSystemWikiPage, which just moved the functionality out to flat files, and then we continued developing more features.

FileSystemWikiPage, thứ chỉ việc chuyển chức năng đó ra các tệp phẳng, rồi chúng tôi tiếp tục phát triển thêm tính năng.

Three months later we reached the conclusion that the flat-file solution was good enough, and we decided to abandon the idea of MySQL altogether. We deferred that decision into nonexistence, and never looked back.

Ba tháng sau, chúng tôi đi tới kết luận rằng giải pháp tệp phẳng là đủ tốt, và quyết định dẹp hẳn ý tưởng MySQL. Chúng tôi đã hoãn quyết định đó tới mức nó không còn tồn tại nữa, và chẳng bao giờ ngoái đầu nhìn lại.

That would be the end of the story if it weren't for a customer of ours who decided that he needed to put the wiki into MySQL for his own purposes. So we showed him the architecture of WikiPages that had allowed us to defer the decision. He came back a day later with the whole system working in MySQL . He simply wrote a MySqlWikiPage derivative and got it working.

Câu chuyện đã kết thúc ở đó, nếu không có một khách hàng của chúng tôi quyết định rằng anh ta cần đưa wiki vào MySQL cho mục đích riêng. Thế là chúng tôi cho anh ta xem kiến trúc WikiPages, thứ đã giúp chúng tôi hoãn được quyết định. Một ngày sau anh ta quay lại với cả hệ thống chạy trên MySQL. Anh ta chỉ việc viết một dẫn xuất MySqlWikiPage và cho nó chạy.

We used to bundle that option with FitNesse , but nobody else ever used it, so eventually we dropped it. Even he eventually dropped it.

Có dạo chúng tôi còn đóng gói kèm tùy chọn đó vào FitNesse, nhưng chẳng ai khác thêm dùng, nên cuối cùng chúng tôi bỏ nó đi. Rốt cuộc thì đến chính anh ta cũng bỏ.

Early in the development of FitNesse , we drew a boundary line between business rules and databases. That line prevented the business rules from knowing anything at all about the database, other than the simple data access methods. That decision allowed us to defer the choice and **implementation** of the database for well over a year. It allowed us to try the file system option. And it allowed us to change direction when we saw a better solution. And yet it did not prevent, or even impede, the original direction, MySQL , when someone wanted it.

Ngay từ giai đoạn đầu phát triển FitNesse, chúng tôi đã vạch một đường ranh giới giữa quy tắc nghiệp vụ và cơ sở dữ liệu. Đường kẻ đó ngăn quy tắc nghiệp vụ biết bất cứ điều gì về cơ sở dữ liệu, ngoài mấy method truy cập dữ liệu đơn giản. Quyết định ấy cho phép chúng tôi hoãn việc chọn và hiện thực cơ sở dữ liệu suốt hơn một năm trời. Nó cho phép chúng tôi thử phương án hệ thống tệp. Và nó cho phép chúng tôi đổi hướng khi thấy có giải pháp tốt hơn. Vậy mà nó không hề ngăn cản, thậm chí không hề làm chậm, hướng đi ban đầu là MySQL, khi có người muốn dùng.

The fact that we did not have a database running for 18 months of development meant that, for 18 months, we did not have schema issues, query issues, database server issues, password issues, connection time issues, or all the other nasty issues you get when you fire up a database. It also meant that all our tests ran fast, because there was no database to slow them down.

Việc suốt 18 tháng phát triển chúng tôi không hề chạy một cơ sở dữ liệu nào có nghĩa là, suốt 18 tháng đó, chúng tôi không dính vấn đề schema, vấn đề truy vấn, vấn đề database server, vấn đề mật khẩu, vấn đề thời gian kết nối, hay tất cả những rắc rối khó chịu khác mà bạn lãnh đủ khi khởi động một cơ sở dữ liệu. Nó còn có nghĩa là mọi test của chúng tôi chạy nhanh, bởi chẳng có cơ sở dữ liệu nào làm chúng chậm lại.

In short, drawing the boundary lines that helped us delay and defer decisions saved us an enormous amount of manpower and headaches. And that's what a good architecture should do.

Nói ngắn gọn, việc vạch những đường ranh giới giúp chúng tôi hoãn và trì hoãn được các quyết định đã tiết kiệm cho chúng tôi vô số nhân lực lẫn đau đầu. Và đó chính là điều mà một kiến trúc tốt phải làm được.

Conclusion — Kết luận

This is how you draw boundary lines in a software architecture. You partition the system into components. Some of those components are core business rules. Some are plug-ins that contain

necessary functions that are not directly related to the core business. Then, you arrange the code in those components such that the arrows between them point in one direction—toward the core business.

Đó là cách bạn vạch những đường ranh giới trong một kiến trúc phần mềm. Bạn chia hệ thống thành các component. Một số component là quy tắc nghiệp vụ cốt lõi. Một số là plug-in chứa những chức năng cần thiết nhưng không liên quan trực tiếp tới nghiệp vụ cốt lõi. Rồi bạn sắp xếp mã trong các component đó sao cho mũi tên giữa chúng chỉ trở về một hướng — hướng về phía nghiệp vụ cốt lõi.

You should recognize this as an application of the Dependency Inversion Principle (DIP) and the Stable Abstractions Principle (SAP). Dependency arrows are arranged to point from lower-level details to higher-level abstractions.

Bạn hẳn nhận ra đây là một ứng dụng của DIP — Nguyên tắc Đảo ngược Phụ thuộc (Dependency Inversion Principle) và SAP — Nguyên tắc Trừu tượng Ổn định (Stable Abstractions Principle). Các mũi tên phụ thuộc (dependency) được sắp sao cho trở từ chi tiết mức thấp về phía trừu tượng hoá (abstraction) mức cao.

And that is the Dependency Rule of Architecture.

Và đó chính là Quy tắc Phụ thuộc của Kiến trúc.

Dependencies that cross architectural boundaries must always point toward the higher-level side.

Mọi phụ thuộc cắt qua ranh giới kiến trúc (architectural boundary) đều phải luôn trở về phía mức cao hơn.

Part IV — Phần IV

Craftmanship — Tinh thần thợ lành nghề (craftmanship)



The following is an abridged and edited excerpt from my book **Clean Craftmanship** .

Phần dưới đây là một trích đoạn đã được rút gọn và biên tập lại từ cuốn **Clean Craftmanship** của tôi.

The profession of software began, inauspiciously, in the summer of 1935 when Alan Turing began work on his paper.

Nghề phần mềm khởi đầu một cách chẳng lấy gì làm hứa hẹn, vào mùa hè năm 1935, khi Alan Turing bắt tay viết bài báo của mình.

His goal was to resolve a mathematical dilemma that had perplexed mathematicians for a decade or more: the Entscheidungsproblem . The decision problem.

Mục tiêu của ông là giải quyết một nan đề toán học đã làm giới toán học bối rối suốt cả chục năm trời hoặc hơn: Entscheidungsproblem. Bài toán quyết định.

In that goal, he was successful; but he had no idea, at the time, that his paper would spawn a globe-spanning industry upon which we would all depend and that now forms the lifeblood of our entire civilization.

Ở mục tiêu đó, ông đã thành công; nhưng vào lúc ấy ông không hề hình dung được rằng bài báo của mình sẽ khai sinh ra một ngành công nghiệp trải khắp địa cầu mà tất cả chúng

ta đều phụ thuộc vào, và ngày nay là mạch máu của cả nền văn minh.

In 1945, Turing wrote code for the Automatic Computing Engine (ACE). He wrote this code in binary machine language, using base 2 numbers. Fewer than a dozen people had ever written code like this before, so Turing had to invent and implement concepts like subroutines, stacks, and floating-point numbers.

Năm 1945, Turing viết mã cho chiếc Automatic Computing Engine (ACE). Ông viết bằng ngôn ngữ máy nhị phân, dùng hệ cơ số 2. Trước đó chưa tới một tá người từng viết mã theo kiểu này, nên Turing phải tự nghĩ ra và hiện thực (implementation) những khái niệm như subroutine, stack và số dấu chấm động.

“A Great Number” — “Một số lượng lớn”

After several months of inventing the basics and using them to solve mathematical problems, he wrote a report that stated the following conclusions.

Sau vài tháng trời phát minh ra những thứ căn bản rồi dùng chúng để giải các bài toán, ông viết một bản báo cáo nêu ra những kết luận sau đây.

“We shall need a great number of mathematicians of ability, because there will probably be a **good** deal of work of this kind to be done.”

“Chúng ta sẽ cần một số lượng lớn các nhà toán học có năng lực, bởi vì nhiều khả năng sẽ có rất nhiều công việc thuộc loại này phải làm.”

“A great number.” How did he know? In reality, he had no idea just how prescient that statement was; he could not have envisioned the great number we have now.

“Một số lượng lớn.” Làm sao ông biết được? Thực ra, ông chẳng thể ngờ câu nói đó tiên tri đến mức nào; ông không thể nào lường tượng nổi cái “số lượng lớn” mà chúng ta có ngày hôm nay.

But what was that other thing he said? “Mathematicians of ability.” Do you consider yourself to be a mathematician of ability? Turing did.

Nhưng còn về kia ông nói là gì nhỉ? “Các nhà toán học có năng lực.” Bạn có tự coi mình là một nhà toán học có năng lực không? Turing thì có đấy.

In the same report, he went on to write:

Cũng trong bản báo cáo ấy, ông viết tiếp:

“One of our difficulties will be the maintenance of an appropriate **discipline**, so that we do not lose track of what we are doing.” Discipline! Alan Turing looked forward through eight decades and saw that our problem would be discipline. And with that, he laid the first stone in the framework of software **professionalism**. He said that we should be mathematicians of ability who maintain an appropriate discipline.

“Một trong những khó khăn của chúng ta sẽ là duy trì một kỷ luật (discipline) thích đáng, để không đánh mất dấu vết của những gì mình đang làm.” Kỷ luật! Alan Turing nhìn xuyên qua tám thập kỷ và thấy rằng vấn đề của chúng ta sẽ là kỷ luật. Và với câu đó, ông đặt viên đá đầu tiên cho khung sườn của tính chuyên nghiệp trong nghề phần mềm. Ông nói rằng chúng ta phải là những nhà toán học có năng lực biết duy trì một kỷ luật thích đáng.

Is that who we are? Is that who you are?

Chúng ta có phải là những người như thế không? Bạn có phải là người như thế không?

Eight Decades — Tám thập kỷ

One person's lifetime. That's how old our profession is as of this writing. And what has happened in those fourscore years?

Một đời người. Nghề của chúng ta mới già chừng ấy tính đến lúc tôi viết những dòng này. Và điều gì đã xảy ra trong tám mươi năm đó?

In 1945, there were a handful of computers in the world, and an equally small number of programmers. In computer science terms, this is on the order of one: $O(1)$. These numbers grew rapidly in those first years. But let's use this as our origin.

Năm 1945, cả thế giới chỉ có dăm ba cái máy tính, và số lập trình viên cũng ít ỏi chẳng kém. Nói theo ngôn ngữ khoa học máy tính thì con số này ở bậc một: $O(1)$. Những con số ấy tăng nhanh chóng trong các năm đầu tiên. Nhưng ta hãy lấy đây làm mốc gốc.

1945. Computers: $O(1)$. Programmers $O(1)$.

1945. Máy tính: $O(1)$. Lập trình viên: $O(1)$.

In the decade that followed, the reliability, consistency, and power usage of vacuum tubes improved dramatically. This made it possible for larger and more powerful computers to be built.

Trong thập kỷ tiếp theo, độ tin cậy, tính ổn định và mức tiêu thụ điện của đèn điện tử chân không được cải thiện ngoạn mục. Điều đó khiến người ta có thể chế tạo những máy tính lớn hơn và mạnh hơn.

By 1960, IBM had sold 140 of its 700 series computers. These were huge, expensive behemoths that could only be afforded by the military, the government, and very large corporations. They were, by our standards, slow, impoverished, and fragile.

Đến năm 1960, IBM đã bán được 140 máy thuộc dòng 700. Đó là những cỗ máy khổng lồ, đắt đỏ, chỉ có quân đội, chính phủ và các tập đoàn rất lớn mới kham nổi. Theo tiêu chuẩn ngày nay, chúng chậm chạp, nghèo nàn và mong manh.

It was in the '50s that Grace Hopper invented the concept of a higher-level language and coined the term compiler. By 1960, her work led to COBOL.

Chính trong thập niên 1950, Grace Hopper đã nghĩ ra khái niệm ngôn ngữ bậc cao và đặt ra thuật ngữ compiler. Đến năm 1960, công trình của bà dẫn tới sự ra đời của COBOL.

In 1952, John Backus submitted the FORTRAN specification. This was followed rapidly by the development of ALGOL. By 1958, John McCarthy had developed LISP. So the proliferation of the language zoo had begun.

Năm 1952, John Backus đệ trình bản đặc tả FORTRAN. Ngay sau đó là sự phát triển của ALGOL. Đến năm 1958, John McCarthy đã tạo ra LISP. Thế là vườn bách thú ngôn ngữ bắt đầu sinh sôi nảy nở.

In those days, there were no operating systems, no frameworks, no **function** libraries. If something executed on your computer, it was because you wrote it. So, in those days, it took a staff of a dozen or more programmers just to keep one computer running.

Thời đó chẳng có hệ điều hành, chẳng có framework, chẳng có thư viện hàm nào cả. Nếu có thứ gì chạy được trên máy tính của bạn, ấy là vì chính bạn đã viết ra nó. Vậy nên hồi

ấy, phải cần cả một đội cả tá lập trình viên trở lên chỉ để giữ cho một chiếc máy tính hoạt động.

By 1960, 15 years after Turing, there were $O(100)$ computers in the world. The number of programmers was an order of magnitude greater: $O(1000)$.

Đến năm 1960, tức 15 năm sau Turing, thế giới có $O(100)$ máy tính. Số lập trình viên thì lớn hơn một bậc: $O(1000)$.

Who were these programmers? They were people like Grace Hopper, Edsger Dijkstra, John von Neumann, John Backus, and Jean Jennings. They were scientists and mathematicians and engineers. Most were people who already had careers and already understood the businesses and disciplines they were employed by. Many, if not most, were in their 30s, 40s, and 50s.

Những lập trình viên ấy là ai? Họ là những người như Grace Hopper, Edsger Dijkstra, John von Neumann, John Backus và Jean Jennings. Họ là các nhà khoa học, nhà toán học và kỹ sư. Phần lớn là những người đã có sẵn sự nghiệp và đã hiểu rõ ngành nghề, lĩnh vực mà họ đang phục vụ. Nhiều người, nếu không muốn nói là đa số, đang ở tuổi ba mươi, bốn mươi, năm mươi.

The 1960s was the decade of the transistor. Bit by bit these small, simple, inexpensive, and reliable devices replaced the vacuum tube. And the effect on computers was a game changer.

Thập niên 1960 là thập kỷ của transistor. Từng chút một, những linh kiện nhỏ bé, đơn giản, rẻ tiền và đáng tin cậy này thay thế đèn điện tử chân không. Và tác động lên máy tính thì thay đổi hoàn toàn cuộc chơi.

By 1965, IBM had produced over 10,000 transistor-based 1401 computers. They rented for about \$2,500 per month, putting them within the reach of thousands of medium-sized businesses.

Đến năm 1965, IBM đã sản xuất hơn 10.000 máy 1401 dùng transistor. Chúng được cho thuê với giá khoảng 2.500 đô la một tháng, tức là trong tầm với của hàng ngàn doanh nghiệp cỡ vừa.

These machines were programmed in Assembler, Fortran, COBOL, and RPG. And all those companies who rented those machines needed staffs of programmers to write their applications.

Những cỗ máy này được lập trình bằng Assembler, Fortran, COBOL và RPG. Và tất cả các công ty thuê những cỗ máy ấy đều cần đội ngũ lập trình viên để viết ứng dụng cho mình.

IBM wasn't the only company making computers at the time; so we'll just say that by 1965, there were $O(1E4)$ computers in the world. And if each computer needed ten programmers to keep it running, there must have been $O(1E5)$ programmers.

Hồi đó IBM không phải là công ty duy nhất làm máy tính; nên ta cứ tạm nói rằng đến năm 1965, thế giới có $O(1E4)$ máy tính. Và nếu mỗi máy cần mười lập trình viên để duy trì hoạt động, thì hẳn phải có $O(1E5)$ lập trình viên.

Twenty years after Turing, there must have been several hundred thousand programmers in the world. Where did these programmers come from? There weren't enough mathematicians, scientists, and engineers to cover the need. And there weren't any computer science graduates coming out of the universities, because there weren't any computer science degree programs—anywhere.

Hai mươi năm sau Turing, hẳn thế giới đã có vài trăm ngàn lập trình viên. Số lập trình viên ấy ở đâu ra? Không có đủ nhà toán học, nhà khoa học và kỹ sư để đáp ứng nhu cầu. Cũng chẳng có cử nhân khoa học máy tính nào ra trường, bởi vì chẳng nơi nào — chẳng một nơi nào — có chương trình đào tạo ngành khoa học máy tính cả.

So companies drew from the best and brightest of their accountants, clerks, planners—anyone with some proven technical aptitude. And they found lots.

Thế là các công ty tuyển từ số kế toán, nhân viên văn phòng, người lập kế hoạch giỏi giang và sáng dạ nhất của mình — bất cứ ai tỏ ra có chút năng khiếu kỹ thuật. Và họ tìm được khối người.

And, again, these were people who were already professionals in another field. They were in their 30s, 40s, and 50s. They already understood deadlines and commitments, what to leave in, what to leave out. 1 And, although these people were not mathematicians per se, they were disciplined professionals. Turing would likely have approved.

Và một lần nữa, đó là những người vốn đã là dân chuyên nghiệp trong một lĩnh vực khác. Họ ở tuổi ba mươi, bốn mươi, năm mươi. Họ đã hiểu thế nào là hạn chót và cam kết, cái gì nên giữ lại, cái gì nên bỏ đi. 1 Và, tuy những người này không phải là nhà toán học theo đúng nghĩa, họ vẫn là những người chuyên nghiệp có kỷ luật. Hẳn Turing sẽ gật đầu tán thành.

1 Apologies to Bob Seeger.

1 Xin lỗi Bob Seeger.

But the crank kept on turning. By 1966, IBM was producing 1,000 System/360s every month. These computers were popping up everywhere. They were immensely powerful for the day. The model 30 could address 64K bytes of memory and execute 35,000 instructions per second.

Nhưng cỗ máy vẫn tiếp tục quay. Đến năm 1966, mỗi tháng IBM sản xuất 1.000 chiếc System/360. Những máy tính này mọc lên khắp nơi. Vào thời đó chúng mạnh khủng khiếp. Model 30 có thể định địa chỉ 64K byte bộ nhớ và thực thi 35.000 lệnh mỗi giây.

It was during this period, in the mid-'60s, when Ole-Johan Dahl and Kristen Nygaard invented SIMULA 67, the first **object**-oriented language.

Cũng chính trong giai đoạn này, giữa thập niên 1960, Ole-Johan Dahl và Kristen Nygaard đã tạo ra SIMULA 67, ngôn ngữ hướng đối tượng đầu tiên.

It was also during this period that Edsger Dijkstra invented **structured programming**.

Cũng trong giai đoạn này, Edsger Dijkstra phát minh ra lập trình có cấu trúc (structured programming).

And it was also during this time that Ken Thompson and Dennis Ritchie invented C and Unix.

Và cũng trong quãng thời gian ấy, Ken Thompson và Dennis Ritchie tạo ra C và Unix.

And still the crank turned. In the early '70s, the integrated circuit came into regular use. These little chips could hold dozens, hundreds, even thousands of transistors. They allowed electronic circuits to be massively miniaturized.

Cỗ máy vẫn quay. Đầu thập niên 1970, mạch tích hợp bắt đầu được dùng phổ biến. Những con chip nhỏ xíu ấy có thể chứa hàng chục, hàng trăm, thậm chí hàng ngàn transistor. Chúng cho phép thu nhỏ mạch điện tử ở quy mô khổng lồ.

And so the minicomputer was born.

Và thế là máy tính mini ra đời.

In the late '60s and into the '70s, Digital Equipment Corporation sold 50,000 PDP8 systems and hundreds of thousands of PDP11 systems.

Cuối thập niên 1960 và sang thập niên 1970, Digital Equipment Corporation bán được 50.000 hệ thống PDP8 và hàng trăm ngàn hệ thống PDP11.

And they weren't alone! The minicomputer market exploded. By the mid-'70s, there were dozens and dozens of companies selling minicomputers.

Và họ không đơn độc! Thị trường máy tính mini bùng nổ. Đến giữa thập niên 1970, có hàng chục, hàng chục công ty bán máy tính mini.

And so by 1975, 30 years after Turing, there were $O(1E6)$ computers in the world. And how many programmers were there? The ratio was starting to change. The number of computers was approaching the number of programmers. So, by 1975, there were $O(1E6)$ programmers.

Thế là đến năm 1975, tức 30 năm sau Turing, thế giới có $O(1E6)$ máy tính. Còn bao nhiêu lập trình viên? Tỷ lệ bắt đầu thay đổi. Số máy tính đang tiến sát số lập trình viên. Vậy nên đến năm 1975, có $O(1E6)$ lập trình viên.

Where did these millions of programmers come from? Who were they?

Hàng triệu lập trình viên ấy từ đâu ra? Họ là ai?

They were me. Me and my buddies. Me and my cohort of young, energetic, geeky boys.

Họ là tôi. Tôi và đám bạn của tôi. Tôi và cả lứa những gã trai trẻ, tràn trề năng lượng, một công nghệ.

Tens of thousands of new EE and CS grads—we were all young. We were all smart. We, in the United States, were all concerned about the draft. And we were almost all male.

Hàng chục ngàn cử nhân EE (kỹ thuật điện) và CS (khoa học máy tính) mới ra trường — tất cả chúng tôi đều trẻ. Tất cả đều thông minh. Ở Mỹ, tất cả chúng tôi đều nơm nớp lo lệnh quân dịch. Và gần như tất cả đều là nam.

Oh, it's not that women were leaving the field in any number—yet. That didn't start until the mid-'80s. No, it's just that many more boys (and we were boys) were entering the field.

Ồ, không phải là phụ nữ đang rời bỏ ngành hàng loạt — chưa đâu. Chuyện đó mãi giữa thập niên 1980 mới bắt đầu. Không, chỉ đơn giản là có quá nhiều thằng con trai (mà chúng tôi đúng là bọn con trai thật) tràn vào ngành.

In my first job as a programmer, in 1969, there were a couple of dozen programmers. They were all in their 30s or 40s, and one-third to one-half were women.

Ở công việc lập trình đầu tiên của tôi, năm 1969, có chừng vài chục lập trình viên. Tất cả đều ở tuổi ba mươi hay bốn mươi, và một phần ba đến một nửa là phụ nữ.

Ten years later, I was working at a company with about fifty programmers, and perhaps three were women.

Mười năm sau, tôi làm ở một công ty có khoảng năm mươi lập trình viên, và có lẽ chỉ ba người là nữ.

So, 30 years after Turing, the demographics of programming had shifted dramatically toward very young men. Hundreds of thousands of twenty-something males.

Vậy là 30 năm sau Turing, cơ cấu nhân khẩu của nghề lập trình đã dịch chuyển mạnh mẽ sang phía đàn ông rất trẻ. Hàng trăm ngàn nam giới tuổi đôi mươi.

We were typically not what Turing would have described as disciplined mathematicians.

Nhìn chung chúng tôi không phải là kiểu “nhà toán học có kỷ luật” mà Turing từng mô tả.

But businesses had to have programmers. The demand was through the roof. And what very young men lack in discipline, they make up for with energy.

Nhưng các doanh nghiệp thì buộc phải có lập trình viên. Nhu cầu cao ngất trời. Và cái mà đám đàn ông rất trẻ thiếu về kỷ luật thì họ bù lại bằng năng lượng.

And we were also cheap. Despite the high starting salaries of programmers today, back then companies could pick up programmers pretty inexpensively. My starting salary in 1969 was \$7,200 per year.

Chúng tôi lại còn rẻ nữa. Bất chấp mức lương khởi điểm cao ngất của lập trình viên ngày nay, hồi ấy các công ty tuyển được lập trình viên khá rẻ. Lương khởi điểm của tôi năm 1969 là 7.200 đô la một năm.

And this has been the trend ever since. Young men and women have been pouring out of computer science programs every year; and industry seems to have an insatiable appetite for them.

Và xu hướng đó vẫn kéo dài từ bấy đến nay. Năm nào cũng có những chàng trai, cô gái trẻ ùn ùn tốt nghiệp các chương trình khoa học máy tính; còn ngành công nghiệp thì dường như có một cơn thèm khát không bao giờ nguôi đối với họ.

In the 30 years between 1945 and 1975, the number of programmers grew by at least a factor of a million. In the 40 years since then, that growth rate has slowed a bit, but it is still very high.

Trong 30 năm từ 1945 đến 1975, số lập trình viên tăng ít nhất một triệu lần. Trong 40 năm kể từ đó, tốc độ tăng có chậm lại đôi chút, nhưng vẫn còn rất cao.

How many programmers do you think are in the world this year, 2025? If you include the VBA programmers, I think the number must be $O(1E8)$. There must be hundreds of millions of programmers in the world today.

Bạn nghĩ năm nay, năm 2025, thế giới có bao nhiêu lập trình viên? Nếu tính cả dân lập trình VBA, tôi cho rằng con số hẳn phải là $O(1E8)$. Ngày nay hẳn phải có hàng trăm triệu lập trình viên trên thế giới.

This is clearly exponential growth. Exponential growth curves have a doubling rate. You can do the math. Hey Albert, what's the doubling rate if we go from 1 to $1E8$ in 80 years?

Đây rõ ràng là tăng trưởng theo hàm mũ. Đường cong tăng trưởng hàm mũ thì có chu kỳ nhân đôi. Bạn có thể tính lấy. Này Albert, chu kỳ nhân đôi là bao nhiêu nếu ta đi từ 1 lên $1E8$ trong 80 năm?

The log to the base 2 of one hundred million is approximately 27—divide that into 80 and you get about 2.96. So perhaps the number of programmers doubled roughly every three-ish years.

Log cơ số 2 của một trăm triệu xấp xỉ 27 — lấy 80 chia cho nó ta được khoảng 2,96. Vậy có lẽ số lập trình viên nhân đôi sau mỗi khoảng ba năm.

Actually, as we saw earlier, the rate was higher in the first decades and has slowed down a bit now. My guess is that the doubling rate is about five years. Every five years the number of programmers in the world doubles.

Thực ra, như ta đã thấy ở trên, tốc độ này cao hơn trong những thập kỷ đầu và giờ đã chậm lại chút ít. Tôi đoán chu kỳ nhân đôi vào khoảng năm năm. Cứ năm năm thì số lập trình viên trên thế giới lại tăng gấp đôi.

The implications of that hypothesis are staggering. If the number of programmers in the world doubles every five years, it means that half the programmers in the world have less than five years

of experience; and this will always be true so long as that doubling rate continues. This leaves the programming industry in the precarious position of— perpetual inexperience.

Hệ quả của giả thuyết đó thì choáng váng. Nếu cứ năm năm số lập trình viên trên thế giới lại nhân đôi, điều đó có nghĩa là một nửa số lập trình viên trên thế giới có chưa tới năm năm kinh nghiệm; và điều đó sẽ luôn đúng chừng nào chu kỳ nhân đôi ấy còn tiếp diễn. Nó đặt ngành lập trình vào một vị thế công chên: — non kinh nghiệm vĩnh viễn.

Nerds and Savors — Nerd và đấng cứu thế

Perpetual inexperience. Oh, don't worry, this doesn't mean that you are perpetually inexperienced. It just means that once you gain five years of experience, the number of programmers will have doubled. By the time you gain ten years of experience, the number of programmers will have quadrupled.

Non kinh nghiệm vĩnh viễn. Ồ, đừng lo, điều này không có nghĩa là bạn sẽ non kinh nghiệm mãi mãi. Nó chỉ có nghĩa là khi bạn tích được năm năm kinh nghiệm thì số lập trình viên đã nhân đôi. Đến lúc bạn có mười năm kinh nghiệm thì số lập trình viên đã gấp bốn lần.

Perhaps you think AI will slow this rate. Don't be deluded. Every new software technology that was supposed to slow that rate has only increased it. Pick a new software technology. Fortran, C, Unix, OO, C++, Java, C#, Ruby, Functional . . . Not one of them decreased the demand for new programmers. If anything, the demand accelerated because each of those technologies made the field accessible to larger and larger groups of people—mostly young.

Có lẽ bạn nghĩ AI sẽ làm chậm nhịp độ này lại. Đừng ảo tưởng. Mọi công nghệ phần mềm mới ra phải làm chậm nhịp độ ấy thì rốt cuộc đều chỉ làm nó tăng thêm. Hãy chọn thử một công nghệ phần mềm mới xem. Fortran, C, Unix, OO, C++, Java, C#, Ruby, Functional... Chẳng cái nào làm giảm nhu cầu về lập trình viên mới cả. Nếu có gì thay đổi thì là nhu cầu ấy tăng tốc, bởi mỗi công nghệ nói trên lại mở cửa ngành này cho những nhóm người ngày một đông hơn — mà phần lớn là người trẻ.

People look at the number of young people in programming and conclude that it's a young person's profession. They ask, "Where are all the old people?"

Người ta nhìn vào số lượng người trẻ trong ngành lập trình rồi kết luận rằng đây là nghề của người trẻ. Họ hỏi: "Vậy chứ đám già đâu hết rồi?"

We're all still here! We haven't gone anywhere. There just weren't that many of us 50 years ago.

Chúng tôi vẫn còn đây cả mà! Có đi đâu đâu. Chỉ là 50 năm trước, chúng tôi không đông đến thế.

The problem is that there aren't enough of us old guys to teach the new programmers coming in. For every programmer with 30 years of experience, there are ~63 programmers who need to learn something from her (or him); 32 of whom are brand new.

Vấn đề nằm ở chỗ đám già chúng tôi không đủ người để dạy lứa lập trình viên mới đang tràn vào. Cứ mỗi lập trình viên có 30 năm kinh nghiệm thì có khoảng 63 lập trình viên cần học được điều gì đó từ cô ấy (hoặc anh ấy); trong đó 32 người hoàn toàn mới toanh.

So, perpetual inexperience, with insufficient mentors to correct the problem. And so the same old mistakes get repeated over, and over, and over again.

Vậy là non kinh nghiệm vĩnh viễn, mà lại thiếu người dẫn dắt để sửa chữa vấn đề. Thế nên vẫn những sai lầm cũ rích ấy cứ lặp đi, lặp lại, lặp lại mãi.

Notoriety — Tiếng tăm

But something else has happened in the last 80 years. Programmers have gained something that I am sure Alan Turing never anticipated: notoriety.

Nhưng còn một chuyện khác đã xảy ra trong 80 năm qua. Lập trình viên đã có được một thứ mà tôi chắc chắn Alan Turing chưa bao giờ lường trước: tiếng tăm.

Back in the '50s and '60s, nobody knew what a programmer was. There weren't enough of us to have a social impact. Programmers did not live next door to very many people.

Hồi thập niên 1950 và 1960, chẳng ai biết lập trình viên là cái giống gì. Chúng tôi không đủ đông để tạo ra một tác động xã hội. Chẳng mấy ai có lập trình viên làm hàng xóm.

That started to change in the '70s. By then, fathers were advising their sons (and sometimes their daughters) to get degrees in computer science. There were enough programmers in the world so that everybody knew somebody who knew one. And the image of the nerdy, Twinkie-eating geek was born.

Điều đó bắt đầu thay đổi vào thập niên 1970. Đến lúc ấy, các ông bố đã khuyên con trai (và đôi khi cả con gái) đi lấy bằng khoa học máy tính. Số lập trình viên trên thế giới đã đủ đông để ai cũng quen một người quen một lập trình viên. Và hình ảnh gã mọt sách lập dị, suốt ngày nhai bánh Twinkie ra đời từ đó.

Few people had seen a computer; but virtually everyone had heard about them. Computers showed up in TV shows, like *Star Trek*, and in movies, like *2001: A Space Odyssey* and *Colossus: The Forbin Project*. All too often in those shows, the computers themselves were cast as villains.

Ít người từng tận mắt thấy máy tính; nhưng gần như ai cũng nghe nói về chúng. Máy tính xuất hiện trong các chương trình truyền hình như *Star Trek*, và trong phim như *2001: A Space Odyssey* hay *Colossus: The Forbin Project*. Và quá thường xuyên, trong những tác phẩm ấy, chính máy tính bị gán vai kẻ ác.

Notice, however, that in each of these cases, the programmer is not a significant character. Society didn't know what to make of programmers back then. They were shadowy, hidden, and somehow insignificant compared to the machines themselves.

Tuy nhiên, hãy để ý rằng trong từng trường hợp nói trên, lập trình viên không phải là một nhân vật đáng kể. Xã hội hồi đó chẳng biết phải xếp lập trình viên vào đâu. Họ mờ ám, khuất lấp, và bằng cách nào đó trở nên vô nghĩa so với chính những cỗ máy.

I have fond memories of one television commercial from this era. A wife and her husband, a nerdy little guy with glasses, a pocket protector, and a calculator, were comparing prices at a grocery store. Mrs. Olsen described him as “a computer genius” and proceeded to school the wife and husband alike on the benefits of a particular brand of coffee.

Tôi vẫn nhớ như in một mẫu quảng cáo truyền hình thời ấy. Một cặp vợ chồng — người chồng là gã mọt sách nhỏ thó, đeo kính, cài bút túi áo, cầm máy tính bỏ túi — đang so giá trong một cửa hàng tạp hoá. Bà Olsen giới thiệu anh ta là “một thiên tài máy tính” rồi thao thao giảng giải cho cả hai vợ chồng về những ưu điểm của một nhãn hiệu cà phê nọ.

The computer programmer in that commercial was naive, bookish, and inconsequential. Someone smart, but with no wisdom or common sense. Not someone you'd invite to parties.

Anh lập trình viên trong mẫu quảng cáo ấy ngây ngô, mọt sách và chẳng có trọng lượng gì. Một kẻ thông minh, nhưng không có sự khôn ngoan hay óc thường thức. Không phải kiểu người bạn muốn mời tới dự tiệc.

Indeed, computer programmers were seen as the kind of people who got beaten up a lot at school.

Thật vậy, lập trình viên bị nhìn nhận như đám người hay bị đánh hội đồng ở trường.

By 1983, personal computers started to appear; and it was clear that teenagers were interested in them for lots of reasons. By this time, a rather large number of people knew at least one computer programmer. We were considered professionals; but still mysterious.

Đến năm 1983, máy tính cá nhân bắt đầu xuất hiện; và rõ ràng là đám thiếu niên hứng thú với chúng vì đủ mọi lý do. Đến lúc này, khá đông người đã quen ít nhất một lập trình viên. Chúng tôi được xem là dân chuyên nghiệp; nhưng vẫn đầy bí ẩn.

That year, the movie War Games depicted a young Matthew Broderick as a computer-savvy teenager and hacker. He hacks into WOPR, the US weapons control system, thinking it's a video game and starts the countdown to thermonuclear war. At the end of the movie, he saves the world by convincing the computer that the only winning move is not to play.

Cũng năm đó, bộ phim War Games khắc họa Matthew Broderick trẻ măng trong vai một cậu thiếu niên rành máy tính kiêm hacker. Cậu ta đột nhập vào WOPR, hệ thống điều khiển vũ khí của Mỹ, tưởng đó là trò chơi điện tử, rồi khởi động đồng hồ đếm ngược tới chiến tranh nhiệt hạch. Cuối phim, cậu cứu thế giới bằng cách thuyết phục máy tính rằng nước cờ duy nhất để thắng là không chơi.

The computer and the programmer had switched roles. Now the computer was the childlike, naive character and the programmer the conduit, if not the source, of wisdom.

Máy tính và lập trình viên đã đổi vai cho nhau. Giờ đây máy tính mới là nhân vật ngây thơ như trẻ con, còn lập trình viên là kênh dẫn truyền, nếu không muốn nói là cội nguồn, của sự khôn ngoan.

We saw something similar in 1986 in the movie Short Circuit in which the computerized robot known as #5 is childlike and innocent but learns wisdom with the help of its bumbling creator and his girlfriend.

Ta thấy điều tương tự vào năm 1986 trong phim Short Circuit, ở đó con robot máy tính có tên #5 hồn nhiên như đứa trẻ nhưng học được sự khôn ngoan nhờ người tạo ra nó — một gã hậu đậu — và cô bạn gái của anh ta.

By 1993, things had changed dramatically. In the film Jurassic Park, the programmer was the villain and the computer was not a character at all. It was just a tool.

Đến năm 1993, mọi thứ đã thay đổi triệt để. Trong phim Jurassic Park, lập trình viên là kẻ ác còn máy tính thì chẳng phải là nhân vật gì cả. Nó chỉ là một công cụ.

Society was beginning to understand who we were and the role we played. We had graduated from nerd to teacher to villain in just 20 years.

Xã hội bắt đầu hiểu chúng ta là ai và vai trò chúng ta đảm nhận. Chỉ trong 20 năm, chúng ta đã thăng hạng từ một sách thành thầy giáo rồi thành kẻ ác.

But the vision changed again. In the 1999 film The Matrix, the main characters were both programmers and saviors. Indeed, their godlike powers came from their ability to read and understand “the code.”

Nhưng hình dung đó lại thay đổi lần nữa. Trong bộ phim The Matrix năm 1999, các nhân vật chính vừa là lập trình viên vừa là đấng cứu thế. Thật vậy, quyền năng như thần thánh của họ đến từ khả năng đọc và hiểu “the code.”

Our roles were changing fast. Villain to savior in just a few years. Society at large was beginning to understand the power we have for both good and evil.

Vai trò của chúng ta thay đổi rất nhanh. Từ kẻ ác thành đấng cứu thế chỉ trong vài năm. Xã hội nói chung đã bắt đầu hiểu ra thứ quyền lực mà chúng ta nắm giữ, cho cả điều thiện lẫn điều ác.

Role Models and Villains — Hình mẫu và kẻ ác

Fifteen years later, in 2014, I visited the Mojang office in Stockholm to do give lectures on **clean code** and **test-driven development (TDD)**.

Mười lăm năm sau, vào năm 2014, tôi ghé văn phòng Mojang ở Stockholm để giảng về clean code và test-driven development (TDD).

Mojang, in case you didn't know, is the company that produced the game Minecraft .

Mojang, nếu bạn chưa biết, là công ty làm ra trò chơi Minecraft.

Afterward, since the weather was nice, the Mojang programmers and I sat outside at a beer garden, chatting.

Xong việc, vì trời đẹp nên tôi và các lập trình viên Mojang ngồi ngoài trời ở một quán bia, tán gẫu.

All of a sudden a young boy, perhaps 12, ran up to the fence and called out to one of the programmers: “Are you Jeb?”

Đột nhiên một cậu bé chừng 12 tuổi chạy tới hàng rào và gọi với vào một trong các lập trình viên: “Anh có phải là Jeb không ạ?”

He was referring to Jens Bergensten, one of the lead programmers at Mojang.

Cậu bé muốn nói tới Jens Bergensten, một trong những lập trình viên chủ chốt ở Mojang.

The lad asked Jens for his autograph and peppered him with questions. He had eyes for no one else.

Thằng bé xin Jens chữ ký rồi hỏi dồn dập đủ thứ. Nó chẳng buồn để mắt tới ai khác.

Programmers have become role models and idols for our children. They dream of growing up to be like Jeb, or Dinnerbone, or Notch.

Lập trình viên đã trở thành hình mẫu và thần tượng của lũ trẻ nhà chúng ta. Chúng mơ lớn lên sẽ được như Jeb, hay Dinnerbone, hay Notch.

Programmers, real-life programmers, are heroes.

Lập trình viên, những lập trình viên bằng xương bằng thịt, là các anh hùng.

But where there are heroes, there are also villains.

Nhưng ở đâu có anh hùng, ở đó cũng có kẻ ác.

In October of 2015, Michael Horn, the CEO of Volkswagen North America, testified before the Congress of the United States regarding the software in their cars that was cheating the EPA testing devices.

Tháng 10 năm 2015, Michael Horn, CEO của Volkswagen Bắc Mỹ, ra điều trần trước Quốc hội Hoa Kỳ về phần mềm trong xe của hãng đã gian lận các thiết bị kiểm định của EPA.

When asked why the company did this, he blamed programmers. He said, “This was a couple of software engineers who put this in for whatever reasons.”

Khi bị hỏi vì sao công ty lại làm vậy, ông ta đổ lỗi cho lập trình viên. Ông ta nói: “Đây là chuyện của vài kỹ sư phần mềm đã đưa nó vào vì lý do gì đó.”

Of course he was lying about the “whatever reasons.” He knew perfectly well what the reasons were, and so did the Volkswagen company at large. His feeble attempt to shift blame onto the programmers was pretty transparent.

Dĩ nhiên ông ta nói dối về cái “lý do gì đó.” Ông ta thừa biết lý do là gì, và cả tập đoàn Volkswagen cũng vậy. Nỗ lực yếu ớt nhằm đẩy tội sang cho đám lập trình viên của ông ta lộ liễu đến mức ai cũng thấy.

On the other hand, he was exactly right. It was some programmers who wrote that lying, cheating code.

Mặt khác, ông ta nói đúng phóc. Chính vài lập trình viên đã viết ra đoạn mã dối trá, gian lận đó.

And those programmers, whoever they were, gave us all a bad name. If we had a true **professional** organization, their recognition as programmers would, and should, be revoked. They betrayed us all. They besmirched the honor of our profession.

Và những lập trình viên ấy, dù họ là ai, đã bôi nhọ danh tiếng của tất cả chúng ta. Nếu chúng ta có một tổ chức nghề nghiệp thực thụ, tư cách lập trình viên của họ hẳn sẽ — và nên — bị tước bỏ. Họ đã phản bội tất cả chúng ta. Họ đã làm hoen ố danh dự của nghề chúng ta.

And so we’ve graduated. It’s taken 80 years. But we’ve gone from nothing, to nerds, to role models, to heroes and villains in that time.

Và thế là chúng ta đã tốt nghiệp. Mất 80 năm đấy. Nhưng trong quãng thời gian đó, chúng ta đã đi từ chỗ chẳng là gì cả, thành đám một sách, thành hình mẫu, rồi thành anh hùng và kẻ ác.

Society has begun—just begun—to understand who we are and the threats and promises that we represent. (For more on that, see the section “Why Should We Be Clean?” in Chapter 1, “Clean Code.”)

Xã hội đã bắt đầu — mới chỉ bắt đầu thôi — hiểu ra chúng ta là ai, cùng những mối đe dọa và những hứa hẹn mà chúng ta đại diện. (Muốn biết thêm về chuyện này, xem mục “Tại sao chúng ta nên sạch?” trong Chương 1, “Clean Code.”)

We Rule the World — Chúng ta cai trị thế giới

But society doesn’t understand everything yet. Indeed, neither do we. As I told you in the introduction, you and I, we programmers; we rule the world.

Nhưng xã hội thì chưa hiểu hết mọi chuyện. Mà thật ra, chính chúng ta cũng vậy. Như tôi đã nói với bạn ở phần mở đầu: bạn và tôi, những lập trình viên chúng ta, chúng ta cai trị thế giới.

That may seem like an exaggerated statement. But consider. There are more computers in the world right now than there are people. And the computers that outnumber us perform a myriad of essential tasks for us.

Nghe có vẻ như một phát biểu phóng đại. Nhưng thử ngẫm mà xem. Ngay lúc này, số máy tính trên thế giới đã nhiều hơn số con người. Và đám máy tính đông hơn chúng ta ấy đang làm vô số công việc thiết yếu cho chúng ta.

They keep track of our reminders. They manage our calendars. They deliver our Facebook messages and keep our photo albums. They connect our phone calls and deliver our text messages.

Chúng theo dõi những lời nhắc của ta. Chúng quản lý lịch của ta. Chúng chuyển tin nhắn Facebook và giữ album ảnh cho ta. Chúng kết nối các cuộc gọi và gửi tin nhắn của ta.

They control the engines in our cars, as well as the brakes, the accelerator, and sometimes even the steering wheels.

Chúng điều khiển động cơ trong xe của ta, cả phanh, chân ga, và đôi khi cả vô lăng nữa.

We can't cook without them. We can't wash clothes without them. They keep our houses warm in winter. They sweep and mop our floors. They entertain us when we're bored.

Không có chúng ta không nấu ăn được. Không có chúng ta không giặt được quần áo. Chúng giữ ấm nhà ta trong mùa đông. Chúng quét và lau sàn cho ta. Chúng mua vui cho ta khi ta buồn chán.

They keep track of our bank records and our credit cards. They help us pay our bills.

Chúng theo dõi sổ sách ngân hàng và thẻ tín dụng của ta. Chúng giúp ta thanh toán hoá đơn.

In fact, most people in the western world interact with some software system every waking minute of every day. Some even continue interacting while they sleep.

Thực tế, phần lớn người phương Tây tương tác với một hệ thống phần mềm nào đó trong từng phút thức giấc của mỗi ngày. Có người còn tiếp tục tương tác cả khi đang ngủ.

The point is that nothing happens in our society without software. No product gets bought or sold. No law gets enacted or enforced. No car drives. No Amazon products get delivered. No phone connects. No power comes out of outlets. No food gets delivered to stores. No water come out of faucets. No gas gets piped to furnaces. Without software monitoring and coordinating it all.

Vấn đề là chẳng có gì trong xã hội chúng ta diễn ra được nếu không có phần mềm. Chẳng món hàng nào được mua hay bán. Chẳng điều luật nào được ban hành hay thực thi. Chẳng chiếc xe nào chạy. Chẳng kiện hàng Amazon nào được giao. Chẳng cuộc gọi nào kết nối. Chẳng dòng điện nào chảy ra khỏi ổ cắm. Chẳng thực phẩm nào được đưa tới cửa hàng. Chẳng giọt nước nào chảy ra từ vòi. Chẳng khí gas nào được dẫn tới lò sưởi. Nếu không có phần mềm giám sát và điều phối tất cả.

And WE write that software. And that makes us the rulers of the world.

Và CHÚNG TA viết ra phần mềm đó. Chính điều đó biến chúng ta thành những kẻ cai trị thế giới.

Oh, other people think they make the rules—but then they hand those rules to us, and WE write the rules that execute in the machines that monitor and coordinate every aspect of our lives.

Ồ, người khác thì tưởng chính họ đặt ra luật lệ — nhưng rồi họ trao những luật lệ ấy cho chúng ta, và CHÚNG TA mới là người viết ra các luật lệ chạy trong những cỗ máy giám sát và điều phối mọi khía cạnh của đời sống chúng ta.

Society does not quite understand this yet. Not quite. Not yet. But the day is coming soon when our society will understand it all too well.

Xã hội vẫn chưa hiểu thấu điều này. Chưa hẳn. Chưa đâu. Nhưng cái ngày mà xã hội hiểu ra điều đó đến tận xương tủy đang tới rất gần.

We, programmers, don't quite understand this yet either. Not really. But again, the day is coming when it will be savagely driven home to us.

Chúng ta, những lập trình viên, cũng chưa hiểu thấu điều này. Chưa thật sự hiểu. Nhưng lại một lần nữa, cái ngày mà điều đó bị giáng vào mặt chúng ta một cách tàn nhẫn đang tới gần.

Catastrophes — Những thảm họa

We've seen plenty of software catastrophes over the years. Some have been pretty spectacular.

Nhiều năm qua, chúng ta đã chứng kiến không ít thảm họa phần mềm. Một số vụ khá là ngoạn mục.

Between 2018 and 2019, two 737 MAX aircraft dove headlong into the ground at near the speed of sound, killing 346 people. The reason: a software weakness exposed by a hardware failure.

Giữa năm 2018 và 2019, hai chiếc 737 MAX lao thẳng xuống đất với vận tốc gần bằng tốc độ âm thanh, cướp đi sinh mạng của 346 người. Nguyên nhân: một điểm yếu trong phần mềm bị phơi bày bởi một hỏng hóc phần cứng.

In 2016, we lost the Schiaparelli Mars lander and rover due to a software issue that caused the lander to believe it had already landed, when it was actually nearly 4 kilometers above the surface.

Năm 2016, chúng ta mất tàu đổ bộ và xe tự hành Schiaparelli trên sao Hỏa vì một vấn đề phần mềm khiến tàu đổ bộ “tin” rằng nó đã hạ cánh xong, trong khi thực tế nó còn cách bề mặt gần 4 kilômét.

In 1999, we lost the Mars Climate Orbiter due to a ground-based software error that transmitted **data** to the orbiter using English units (pound-seconds) rather than metric units (newton-seconds). This error caused the orbiter to descend too far into the Martian atmosphere, where it was torn to pieces.

Năm 1999, chúng ta mất tàu Mars Climate Orbiter vì một lỗi phần mềm dưới mặt đất: nó truyền dữ liệu lên tàu bằng đơn vị Anh (pound-giây) thay vì đơn vị mét (newton-giây). Sai sót đó khiến con tàu hạ xuống quá sâu vào khí quyển sao Hỏa và bị xé tan.

In 1996, the Ariane 5 launch vehicle and payload were destroyed 37 seconds after launch due to an integer overflow **exception** when a 64-bit floating-point number underwent an unchecked conversion to a 16-bit integer. The exception crashed the onboard computers, and the vehicle self-destructed.

Năm 1996, tên lửa đẩy Ariane 5 cùng toàn bộ tải trọng nổ tung 37 giây sau khi phóng vì một exception tràn số nguyên, xảy ra khi một số dấu phẩy động 64-bit bị chuyển đổi sang số nguyên 16-bit mà không hề được kiểm tra. Exception đó làm sập máy tính trên khoang, và tên lửa tự huỷ.

Should we talk about the Therac-25 radiation therapy machine that, due to a **race condition**, killed three people and injured three others by blasting them with a high-powered electron beam?

Có nên nhắc tới cỗ máy xạ trị Therac-25 nữa không? Chỉ vì một race condition, nó đã giết ba người và làm ba người khác bị thương bằng cách nã vào họ một chùm electron công suất cao.

Or maybe we should talk about Knight Capital, who lost \$460 million in 45 minutes because they reused a flag that activated dead code inadvertently left in the system.

Hay có lẽ ta nên nói về Knight Capital, công ty đã mất 460 triệu đô la trong 45 phút chỉ vì họ tái sử dụng một cờ (flag), và cái cờ ấy vô tình kích hoạt đoạn mã chết còn sót lại trong hệ thống.

Or perhaps we should talk about the Toyota stack overflow bug that caused cars to accelerate out of control—killing perhaps as many as 89 people.

Hoặc có lẽ ta nên nói về lỗi tràn stack của Toyota, thứ đã khiến những chiếc xe tăng tốc ngoài tầm kiểm soát — có thể đã giết tới 89 người.

Or maybe we should talk about Healthcare.gov and the software failure that nearly overturned a new and controversial US healthcare law.

Hay có lẽ ta nên nói về Healthcare.gov và sự cố phần mềm suýt nữa đã lật đổ một đạo luật y tế mới và đầy tranh cãi của nước Mỹ.

These disasters have cost billions of dollars and many lives. And they were caused by programmers.

Những thảm họa ấy đã tốn hàng tỉ đô la và cướp đi rất nhiều sinh mạng. Và chúng do lập trình viên gây ra.

We, programmers, through the code that we write, are killing people.

Chúng ta, những lập trình viên, thông qua đoạn mã mình viết ra, đang giết người.

Now I know you didn't get into this business in order to kill people. Probably you became a programmer because you wrote an infinite loop that printed your name once, and you experienced that joyous feeling of power.

Tôi biết bạn không bước vào nghề này để giết người. Nhiều khả năng bạn trở thành lập trình viên vì có lần bạn viết một vòng lặp vô tận in ra tên mình, và bạn nếm trải cái cảm giác quyền năng sung sướng ấy.

But facts are facts. We are now in a position in our society where our actions can destroy fortunes, livelihoods, and lives.

Nhưng sự thật vẫn là sự thật. Chúng ta giờ đang ở một vị trí trong xã hội mà hành động của mình có thể phá tan gia sản, kế sinh nhai và mạng sống của người khác.

One day, probably not too long from now, some poor programmer is going to do something just a little dumb, and tens of thousands of people will die.

Một ngày nào đó, có lẽ chẳng còn xa lắm, một lập trình viên xấu số nào đó sẽ làm một điều hơi ngu ngốc một chút, và hàng chục nghìn người sẽ chết.

This isn't wild speculation—it's just a matter of time.

Đây không phải là suy diễn hoang đường — đó chỉ là chuyện sớm muộn.

And when this happens, the politicians of the world will demand an accounting. They will demand that we show them how we will prevent this in the future.

Và khi điều đó xảy ra, giới chính trị gia trên khắp thế giới sẽ đòi chúng ta phải giải trình. Họ sẽ đòi chúng ta chỉ cho họ thấy chúng ta sẽ ngăn chặn chuyện này trong tương lai bằng cách nào.

And if we show up without a statement of ethics; if we show up without any standards, or defined disciplines; if we show up and whine about how our bosses set unreasonable schedules and deadlines ...

Và nếu chúng ta xuất hiện mà không có lấy một bản tuyên ngôn đạo đức nghề nghiệp; nếu chúng ta xuất hiện mà không có bất kỳ chuẩn mực nào, không có kỷ luật (discipline) nào được định nghĩa rõ ràng; nếu chúng ta xuất hiện và chỉ biết than vãn rằng sắp đặt ra lịch trình và deadline vô lý ...

Then we will be found GUILTY.

Thì chúng ta sẽ bị tuyên là CÓ TỘI.

The Oath — Lời thề

And so, to begin the definition of our ethics as software developers, I offer the following oath.

Vậy nên, để bắt đầu định nghĩa nền đạo đức nghề nghiệp của chúng ta với tư cách những người phát triển phần mềm, tôi xin đưa ra lời thề sau đây.

In order to defend and preserve the honor of the profession of computer programmers, I promise that, to the best of my ability and judgment:

Để bảo vệ và gìn giữ danh dự của nghề lập trình máy tính, tôi xin hứa rằng, trong khả năng và phán đoán tốt nhất của mình:

1. I will not produce harmful code.
 2. The code that I produce will always be my best work. I will not knowingly allow code that is defective either in **behavior** or in **structure** to accumulate.
 3. I will produce, with each release, a quick, sure, and repeatable demonstration that every element of the code works as it should.
 4. I will make frequent, small releases so that I do not impede the progress of others.
 5. I will fearlessly and relentlessly improve my creations at every opportunity. I will never degrade them.
 6. I will do all that I can to keep the **productivity** of myself and others as high as possible. I will do nothing that decreases that productivity.
 7. I will continuously ensure that others can cover for me and that I can cover for them.
 8. I will produce estimates that are honest both in magnitude and precision. I will not make promises without reasonable certainty.
 9. I will respect my fellow programmers for their ethics, standards, disciplines, and skill. No other attribute or characteristic will be a factor in my regard for my fellow programmers.
 10. I will never stop learning and improving my **craft**.
1. Tôi sẽ không tạo ra đoạn mã gây tổn hại (harm).
 2. Đoạn mã tôi tạo ra sẽ luôn là phần việc tốt nhất của tôi. Tôi sẽ không cố tình để cho mã bị lỗi (defect) — dù về hành vi hay về cấu trúc — được phép tích tụ.
 3. Với mỗi lần phát hành, tôi sẽ tạo ra một minh chứng nhanh chóng, chắc chắn và lặp lại được rằng mọi thành phần của mã đều hoạt động đúng như nó phải thế.
 4. Tôi sẽ phát hành thường xuyên, theo từng phần nhỏ, để không cản trở tiến độ của người khác.
 5. Tôi sẽ không sợ hãi và không ngừng cải tiến những gì mình tạo ra, ở mọi cơ hội. Tôi sẽ không bao giờ làm chúng tệ đi.
 6. Tôi sẽ làm mọi điều có thể để giữ năng suất của bản thân và của người khác ở mức cao nhất. Tôi sẽ không làm bất cứ điều gì làm giảm năng suất ấy.

7. Tôi sẽ liên tục bảo đảm rằng người khác có thể gánh việc thay tôi, và tôi có thể gánh việc thay họ.
8. Tôi sẽ đưa ra những ước lượng (estimate) trung thực cả về độ lớn lẫn độ chính xác. Tôi sẽ không hứa hẹn khi chưa có đủ mức chắc chắn hợp lý.
9. Tôi sẽ tôn trọng các đồng nghiệp lập trình viên vì đạo đức nghề nghiệp, chuẩn mực, kỷ luật và kỹ năng của họ. Không một thuộc tính hay đặc điểm nào khác được phép trở thành yếu tố chi phối cách tôi nhìn nhận đồng nghiệp của mình.
10. Tôi sẽ không bao giờ ngừng học hỏi và mài giũa tay nghề của mình.

Chapter 26. Clean Boundaries —

Chương 26. Ranh giới sạch

by James Grenning

tác giả James Grenning



We seldom control all the software in our systems. Sometimes we buy third-party packages or use open source. Other times we depend on teams in our own company to produce components or subsystems for us. Somehow we must cleanly integrate this foreign code with our own. In this section, we look at practices and techniques to keep the boundaries of our software clean.

Hiếm khi chúng ta kiểm soát được toàn bộ phần mềm trong hệ thống của mình. Có lúc ta mua package của bên thứ ba hoặc dùng mã nguồn mở. Lúc khác ta lại phụ thuộc vào các nhóm khác trong chính công ty mình để họ làm ra component hay hệ thống con cho ta. Bằng cách nào đó, ta phải tích hợp thứ mã ngoại lai ấy vào mã của mình một cách sạch sẽ. Trong phần này, chúng ta sẽ xem xét những thực hành và kỹ thuật giúp giữ cho ranh giới (boundary) của phần mềm luôn sạch.

Third-Party IoT Framework: Lots o' Boundaries — Framework IoT của bên thứ ba: ranh giới nhiều vô kể

My brother had an idea for automating the manual process that users of his product perform. The product is used by a technician testing a high-capacity water pump. The pump is tested by flowing water and manually measuring and recording water pressure at several nozzles. Given the geometry of the nozzles, pressure is converted into flow rate.

Anh trai tôi nảy ra ý tưởng tự động hoá cái quy trình thử công mà người dùng sản phẩm của anh vẫn phải làm. Sản phẩm này dành cho kỹ thuật viên kiểm tra máy bơm nước công suất lớn. Người ta kiểm tra máy bơm bằng cách cho nước chảy qua rồi đo và ghi lại áp suất nước tại vài vòi phun — tất cả đều bằng tay. Biết được hình học của vòi phun, ta quy đổi áp suất ra lưu lượng.

The pump is permanently installed in a commercial or industrial building; water flows outside, onto the curb, and down a storm drain. Pump capacities are measured in thousands of gallons per minute. A manual **test** can take 20 to 30 minutes, and a lot of water is wasted in the process. Typically, pressure measurements are made with a physical gauge at several locations: inside the building next to the pump, and outside at the nozzles, with records kept manually on a clipboard.

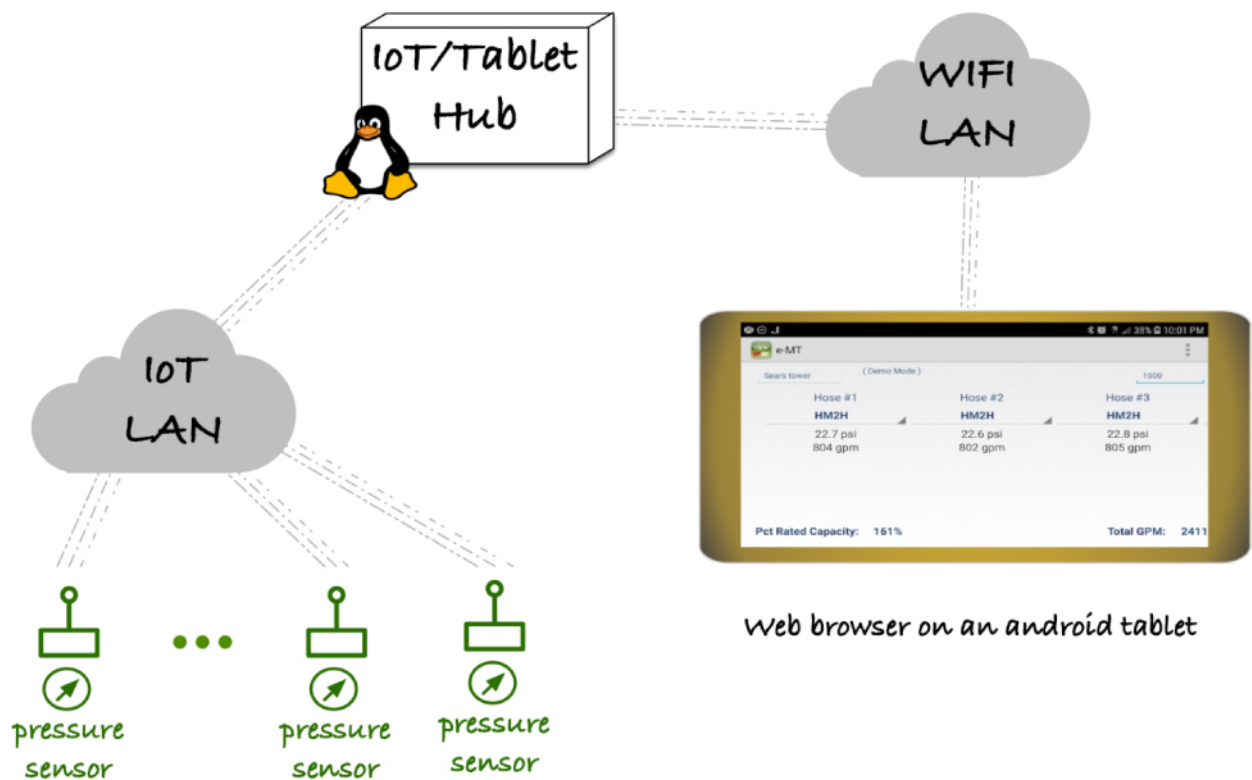
Máy bơm được lắp cố định trong toà nhà thương mại hoặc công nghiệp; nước chảy ra ngoài, tràn xuống lề đường rồi xuống cống thoát. Công suất bơm được đo bằng hàng nghìn gallon mỗi phút. Một lượt kiểm tra thử công có thể mất 20 đến 30 phút, và rất nhiều nước bị phí phạm trong quá trình đó. Thông thường, áp suất được đo bằng đồng hồ cơ tại vài vị trí: bên trong toà nhà, ngay cạnh máy bơm, và bên ngoài tại các vòi phun, với số liệu ghi tay lên tấm kẹp giấy.

The big idea was to measure pressure with an electronic gauge that could report its pressure once per second and have the readings shown on a tablet computer or smart phone. The first design used Bluetooth to communicate from the sensor to the tablet. It worked great in the lab, but in the field there was always a brick wall (or worse) in the way. Bluetooth could not penetrate the brick wall. So we dreamed up the following idea.

Ý tưởng lớn là đo áp suất bằng một đồng hồ điện tử có khả năng báo áp suất mỗi giây một lần, rồi hiển thị số đọc lên máy tính bảng hoặc điện thoại thông minh. Thiết kế đầu tiên dùng Bluetooth để truyền từ cảm biến sang máy tính bảng. Trong phòng lab thì tuyệt vời, nhưng ngoài hiện trường bao giờ cũng có một bức tường gạch (hoặc thứ gì tệ hơn) chắn ngang. Bluetooth không xuyên nổi tường gạch. Thế là chúng tôi nghĩ ra ý tưởng sau.

With an off-the-shelf Internet of Things (IoT) infrastructure, we could get the water pressure measurements through brick walls.

Với một hạ tầng Internet of Things (IoT) mua sẵn, chúng tôi có thể lấy số đo áp suất nước xuyên qua tường gạch.



We selected an IoT vendor with infrastructure that let us build our application in Python. Their existing radio system, along with our production code in Python, would make it possible for us to get a proof-of-concept prototype developed quickly.

Chúng tôi chọn một nhà cung cấp IoT có hạ tầng cho phép xây dựng ứng dụng bằng Python. Hệ thống radio sẵn có của họ, cộng với mã production viết bằng Python của chúng tôi, sẽ giúp dựng nhanh một nguyên mẫu chứng minh khả thi.

We set off to build a prototype, knowing we might need a different vendor once we went to market. Why did we doubt this vendor? At the prototype stage, we weren't that concerned with cost. That could change by the time we went to market. Also, in the rapidly growing IoT market, this vendor may go out of business or get eaten by another IoT supplier. We wanted to preserve our business logic investment, so in our prototype, we needed **clean code**, separation of concerns, and a limited **dependency** on our vendor's infrastructure.

Chúng tôi bắt tay dựng nguyên mẫu, và biết rõ rằng có thể sẽ cần một nhà cung cấp khác khi ra thị trường. Vì sao chúng tôi nghi ngờ nhà cung cấp này? Ở giai đoạn nguyên mẫu, chúng tôi không mấy bận tâm đến chi phí. Nhưng chuyện đó có thể đổi khác khi sản phẩm ra thị trường. Ngoài ra, trong một thị trường IoT đang phình to nhanh chóng, nhà cung cấp này có thể phá sản hoặc bị một hãng IoT khác nuốt chửng. Chúng tôi muốn bảo toàn khoản đầu tư vào business logic của mình, nên ngay trong nguyên mẫu, chúng tôi cần clean code, cần tách bạch mối quan tâm, và cần hạn chế phụ thuộc (dependency) vào hạ tầng của nhà cung cấp.

Nicely, our vendor provided a simple "hello world" IoT application to get started. Their "hello world" demo allowed us to send messages between a small Linux box (it was kind of like an industrial Raspberry Pi) and the little microcontroller-driven radios that would be married to our pressure sensors. We could morph their "hello world" example code into a layer that would contain all the dependencies on the vendor infrastructure.

May thay, nhà cung cấp có sẵn một ứng dụng IoT “hello world” đơn giản để khởi động. Bản demo “hello world” của họ cho phép chúng tôi gửi thông điệp qua lại giữa một hộp Linux nhỏ (kiểu như một chiếc Raspberry Pi công nghiệp) và mấy con radio chạy vi điều khiển sẽ được gắn với cảm biến áp suất của chúng tôi. Chúng tôi có thể nhào nặn mã ví dụ “hello world” của họ thành một lớp chứa trọn mọi phụ thuộc vào hạ tầng nhà cung cấp.

Something I learned the hard way leads me to prefer starting with a working system and focusing on small changes to grow functionality and not break stuff. You know, it's easier to keep a system working than to fix it after you break it! Test-driven development (**TDD**) allowed me to see the value of small, verified changes.

Một bài học tôi phải trả giá mới có được khiến tôi ưa bắt đầu từ một hệ thống chạy được rồi tập trung vào những thay đổi nhỏ để mở rộng tính năng mà không làm hỏng thứ gì. Bạn biết đấy, giữ cho một hệ thống tiếp tục chạy dễ hơn nhiều so với sửa nó sau khi đã làm nó hỏng! Test-driven development (TDD) đã giúp tôi thấy được giá trị của những thay đổi nhỏ và đã được kiểm chứng.

The vendor had an elaborate application infrastructure we could take advantage of. It had a lot of capabilities. We just would not need everything it offered. We wanted to use the radios for some simple message passing, not marry the vendor (until death do us part). Consequently, we used very little of the vendor's very wide **interface**.

Nhà cung cấp có cả một hạ tầng ứng dụng công phu mà chúng tôi có thể tận dụng. Nó lắm khả năng lắm. Chỉ có điều chúng tôi không cần đến tất cả những gì nó mời chào. Chúng tôi muốn dùng đám radio cho vài việc truyền thông điệp đơn giản, chứ không phải cưới nhà cung cấp (cho tới khi cái chết chia lìa đôi ta). Vì thế, chúng tôi chỉ dùng một phần rất nhỏ trong cái interface rất rộng của họ.

Let's start with the sensor and the hub getting to know each other. The vendor makes this very easy. The hub and radios interact using a remote procedure call (RPC) mechanism. The hub has a full Python environment while the radio is limited to MicroPython.

Hãy bắt đầu từ lúc cảm biến và hub làm quen với nhau. Nhà cung cấp làm chuyện này rất dễ dàng. Hub và các radio tương tác với nhau qua cơ chế remote procedure call (RPC). Hub có môi trường Python đầy đủ, còn radio thì chỉ giới hạn ở MicroPython.

The project is code-named Hydra, after the many-headed monster. The **class** HydraNetwork encapsulates all dependencies in the hub on the IoT vendor. The functions are low complexity and delegate to the communications infrastructure. Starting at the beginning, initialization, the hub announces itself to all the radios listening, then polls the framework for replies.

Dự án có tên mã là Hydra, đặt theo con quái vật nhiều đầu. Class HydraNetwork đóng gói (encapsulation) toàn bộ phụ thuộc của hub vào nhà cung cấp IoT. Các hàm ở đây có độ phức tạp thấp và chỉ uỷ nhiệm cho hạ tầng truyền thông. Bắt đầu từ đầu, tức phần khởi tạo: hub tự giới thiệu với mọi radio đang lắng nghe, rồi hỏi vòng framework để lấy hồi đáp.

```
# In the hub
class HydraNetwork:
    ...
    def announceHub(self):
        self.broadcast('announceHub', '')
        self.poll_a_second()
```

Through the magic of the infrastructure RPC mechanism, the announceHub() **function** is called in

the radio. The radio remembers the hub's address and sends an acknowledgment to the hub.

Nhờ phép màu của cơ chế RPC trong hạ tầng, hàm `announceHub()` được gọi bên phía radio. Radio ghi nhớ địa chỉ của hub rồi gửi một acknowledgment về cho hub.

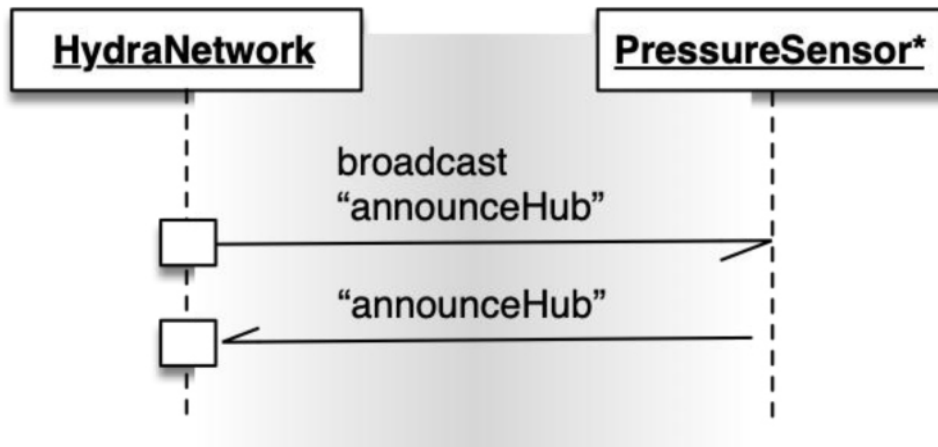
```
# In the radio
def announceHub():
    global hub_addr
    hub_addr = rpcSourceAddr()
    info = nodeIdentity()
    sendAck(hub_addr, 'announceHub', info)
```

This sequence chart shows the asynchronous communications between the HydraNetwork hub and its

Biểu đồ tuần tự này cho thấy quá trình liên lạc bất đồng bộ giữa hub HydraNetwork và các

PressureSensors on the IoT LAN.

PressureSensor của nó trên mạng LAN IoT.



* Every PressureSensor on the LAN replies

Now, going back to the hub, the infrastructure RPC facility calls the hub's `HydraNetwork.processAck()` function.

Giờ quay lại phía hub: cơ chế RPC của hạ tầng gọi hàm `HydraNetwork.processAck()` của hub.

```
# In the hub
class HydraNetwork:
    ...
    def processAck(self, command, payload):
        sensor_id = self.source_addr()
        self.ack_handler.handle_reply(sensor_id, command, payload)
```

The `ack_handler` is an injected dependency, which happens to be an instance of `HydraDataSource`. The **method** `handle_reply()` passes the received bookkeeping and payload to other parts of the Hydra system. As it turns out, there is nothing to do for the 'announceHub' ACK from the radio.

ack_handler là một phụ thuộc được inject vào, mà ở đây tình cờ là một instance của HydraDataSource . Method handle_reply() chuyển phần thông tin quản lý cùng payload nhận được sang các phần khác của hệ thống Hydra. Hoá ra với ACK ‘announceHub’ từ radio thì chẳng có gì phải làm cả.

The HydraDataSource initiates a network PSI 1 read when its

HydraDataSource khởi động một lượt đọc PSI 1 qua mạng khi

flow_update() method is called. The flow_update() method is called once per second during a flow test. Each update has a

flow_update() của nó được gọi. Method flow_update() được gọi mỗi giây một lần trong suốt bài test lưu lượng. Mỗi lượt cập nhật đều mang theo một

sync_token so that asynchronous measurement replies can be grouped together by second. When a new second starts, a report is saved from the previous second. More on that later.

sync_token để các phản hồi đo bất đồng bộ có thể được gom nhóm theo từng giây. Khi một giây mới bắt đầu, báo cáo của giây trước đó được lưu lại. Chuyện này sẽ nói kỹ hơn ở phần sau.

1 For example, [PPP02], <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod> , and [https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)) (or just google “SOLID”).

1 Ví dụ, [PPP02], <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod> , và [https://en.wikipedia.org/wiki/SOLID_\(object-oriented_design\)](https://en.wikipedia.org/wiki/SOLID_(object-oriented_design)) (hoặc cứ google “SOLID”).

In the hub

class HydraDataSource:

```
...
def flow_update(self):
    self.sync_token.next()
    self.network.broadcast(
        'initiateRead',
        self.sync_token.value())
    self.report = self.reply_dispatcher.report()
```

When ‘initiateRead’ is broadcast to the sensor network, each radio replies asynchronously. Again, via the IoT framework,

Khi ‘initiateRead’ được broadcast xuống mạng cảm biến, mỗi radio sẽ trả lời một cách bất đồng bộ. Một lần nữa, thông qua framework IoT,

initiateRead() is called in the PressureSensor **module**.

initiateRead() được gọi trong module PressureSensor.

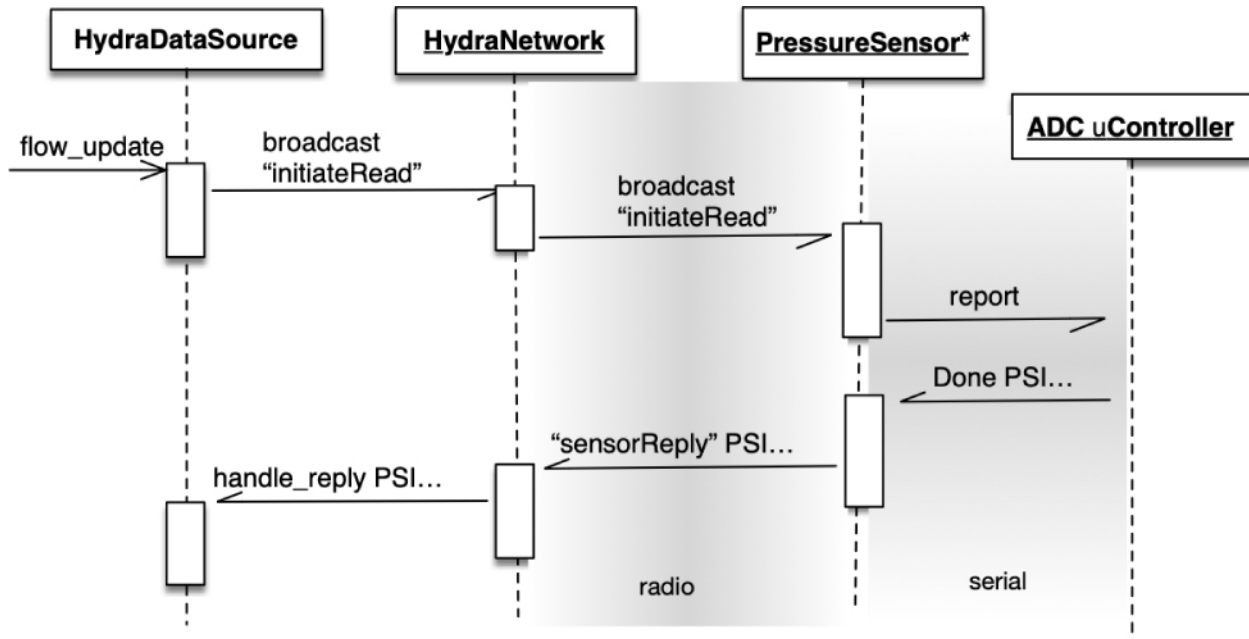
First, initiateRead() responds with an ACK and then calls

Trước hết, initiateRead() đáp lại bằng một ACK rồi gọi reportMeasurement(sync_token) to get the latest PSI value.

reportMeasurement(sync_token) để lấy giá trị PSI mới nhất.

I neglected to tell you about another boundary. There is a separate microcontroller responsible for reading the analog-to-digital converter (ADC) and doing the needed math. The call to reportMeasurement() writes to a serial connection asking the ADC microcontroller for the next measurement.

Tôi quên chưa kể cho bạn về một ranh giới nữa. Có một vi điều khiển riêng chịu trách nhiệm đọc bộ chuyển đổi analog sang số (ADC) và làm các phép tính cần thiết. Lờ gọi `reportMeasurement()` ghi ra một kết nối serial để hỏi vi điều khiển ADC lấy số đo kế tiếp.



```

# In the radio
def initiateRead(sync_token):
    global hub_addr
    hub_addr = rpcSourceAddr()
    sendAck(hub_addr, 'initiateRead', sync_token)
    reportMeasurement(sync_token)
def reportMeasurement(sync_token):
    print 'RA ' + sync_token
@setHook(HOOK_STDIN)
def measurementDone(reply):
    p y
    stdin_info = getInfo(STDIN_INFO)
    if stdin_info != STDIN_RECEIVED:
        reply = 'Error: Invalid STDIN_INFO'+
                str(stdin_info)
    sendAck(hub_addr, 'SensorReply', reply[:MAX_SIZE])
    
```

It takes a little time to get a new ADC value and convert it to PSI. When the measurement is ready, the ADC microcontroller sends back a CSV string using the serial connection. The HOOK_STDIN notifies `measurementDone()`, providing a reply. If there is no serial communication error, the reply includes the `sync_token` and sensor configuration. The reply is sent as a `SensorReply` ACK to the hub.

Phải mất chút thời gian mới lấy được giá trị ADC mới và quy đổi nó ra PSI. Khi số đo đã sẵn sàng, vi điều khiển ADC gửi trả một chuỗi CSV qua kết nối serial. HOOK_STDIN báo cho `measurementDone()`, kèm theo phần hồi đáp. Nếu không có lỗi truyền serial, hồi đáp sẽ chứa `sync_token` và cấu hình cảm biến. Hồi đáp này được gửi tới hub dưới dạng ACK `SensorReply`.

Meanwhile, back at the hub, the hub is collecting ACK messages. ACK messages are sent to

handle_reply() . A

Trong lúc đó, ở phía hub, hub đang thu gom các thông điệp ACK. Thông điệp ACK được gửi tới handle_reply() . Một

‘SensorReply’ ACK is given to the

ACK ‘SensorReply’ được chuyển cho

reply_dispatcher.sensor_update(). Thereply_dispatcher knows the

message format, and where to send it once it is parsed. The payload is a simple set of CSVs. The reply_dispatcher and collaborators are test driven and free from IoT dependencies.

định dạng thông điệp, và nơi cần gửi nó tới sau khi đã phân tích cú pháp. Payload chỉ là một tập CSV đơn giản. reply_dispatcher cùng những thành phần cộng tác với nó đều được viết theo lối test driven và hoàn toàn không dính phụ thuộc IoT nào.

class HydraDataSource:

```
...
def handle_reply(self, sensor_id,
                  command, payload):
    sync_token = self.sync_token.value
    if command == 'SensorReply':
        result = self.reply_dispatcher.
            sensor_update(sensor_id,
                          sync_token,
                          payload)
```

Because HydraNetwork encapsulated all IoT framework dependencies, we could substitute a different vendor’s IoT infrastructure.

Vì HydraNetwork đã đóng gói trọn mọi phụ thuộc vào framework IoT, chúng tôi có thể thay bằng hạ tầng IoT của một nhà cung cấp khác.

UI/Application Boundary — Ranh giới UI/ứng dụng

Next, let’s look at the UI/application boundary. The user interface of the system is a Web browser on a tablet or smart phone. We used the popular Python/Flask Web server framework. The application runs on a small Linux box as two processes.

Tiếp theo, hãy nhìn sang ranh giới giữa UI và ứng dụng. Giao diện người dùng của hệ thống là một trình duyệt Web trên máy tính bảng hoặc điện thoại thông minh. Chúng tôi dùng framework máy chủ Web Python/Flask vốn rất phổ biến. Ứng dụng chạy trên một hộp Linux nhỏ dưới dạng hai tiến trình.

The Collector process collects measurements from the

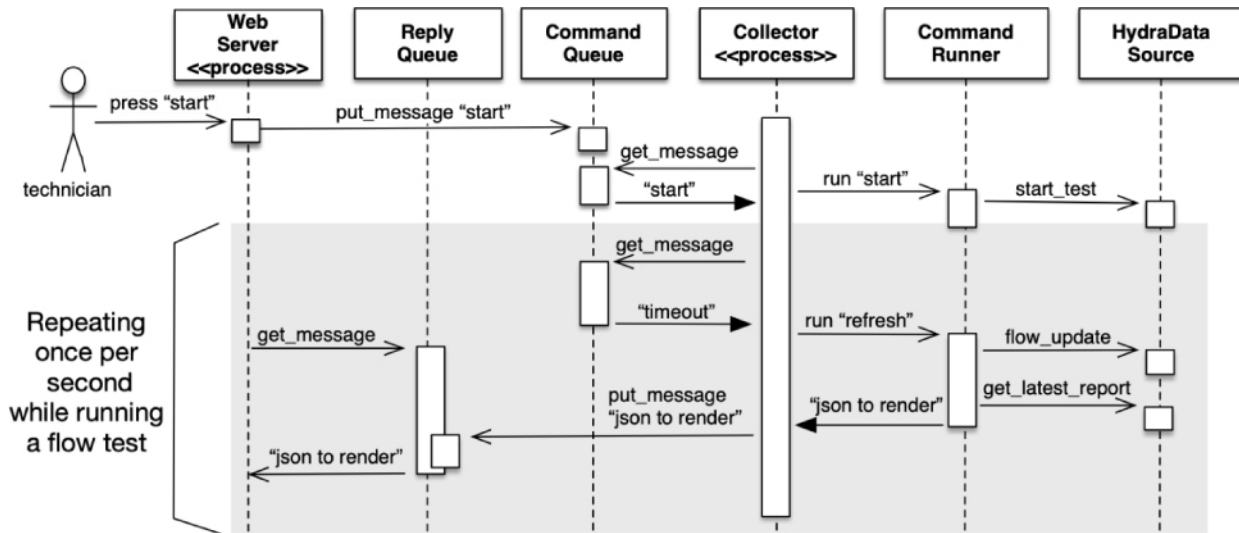
Tiến trình Collector thu thập các số đo từ

HydraNetwork . The WebServer process runs the customized Flask Web server in the main **thread**. The user has buttons to start and stop the flow test, which comes to the Collector in the

HydraNetwork . Tiến trình WebServer chạy máy chủ Web Flask đã tùy biến trong luồng (thread) chính. Người dùng có các nút để bắt đầu và dừng bài test lưu lượng; lệnh đó đi tới Collector thông qua

CommandQueue . Measurement updates are sent asynchronously to the ReplyQueue once a synchronized group of measurements is collected.

CommandQueue . Các cập nhật số đo được gửi bất đồng bộ tới ReplyQueue mỗi khi gom đủ một nhóm số đo đã đồng bộ.



The Collector process entry point is the `measurement_loop()` function, which waits on the CommandQueue with a one-second timeout. Each timeout initiates a refresh/flow_update of the measurement **data** all the way down to the ADC microcontroller (not shown in this diagram).

Điểm vào của tiến trình Collector là hàm `measurement_loop()` , hàm này chờ trên CommandQueue với thời gian chờ tối đa một giây. Mỗi lần hết thời gian chờ sẽ kích hoạt một lượt `refresh/flow_update` dữ liệu đo, đi suốt xuống tận vi điều khiển ADC (không vẽ trong sơ đồ này).

```

# Collector process entry function
def measurement_loop(command_q, reply_q,
                    command_runner, update_interval):
    command = None
    while command != HYDRA_EXIT:
        command = get_message(command_q, update_interval)
        if command == None:
            Command = HYDRA_REFRESH
            reply = command_runner.run(command)
            put_message(reply_q, reply)
  
```

There are a number of application objects not shown, but then they are not at the boundary. We'll look at a bigger picture soon.

Còn khá nhiều đối tượng ứng dụng không được vẽ ra, nhưng dù sao chúng cũng không nằm ở ranh giới. Chúng ta sẽ xem bức tranh rộng hơn ngay sau đây.

Let's take a quick look at `hydra_run()` . This function is called from `main()` with its `data_source` injected. In production, a

Hãy nhìn nhanh vào `hydra_run()` . Hàm này được gọi từ `main()` với `data_source` được inject vào. Trong production, một

HydraDataSource , with its HydraNetwork , is injected. During development and testing, we have a simulated data source where a network is not needed.

HydraDataSource , kèm theo HydraNetwork của nó, sẽ được inject vào. Còn trong lúc phát triển và kiểm thử, chúng tôi dùng một nguồn dữ liệu mô phỏng, chẳng cần mạng gì cả.

The hydra_run() function puts everything in motion and is called from main() . The following code starts the Collector process running the Web server in the main thread of execution.

Hàm hydra_run() khởi động mọi thứ và được gọi từ main() . Đoạn mã sau khởi chạy tiến trình Collector, còn máy chủ Web thì chạy trong luồng thực thi chính.

```
def hydra_run(port, data_source, polling_rate,
              logger):
    logger.info("Hydra start")
    command_runner = CommandRunner(data_source,
                                    logger)

    collector = Process(
        target=measurement_loop,
        args=(command_q, reply_q,
              command_runner, polling_rate))
    collector.start()
    sleep(1)
    logger.info("Hydra collector started")
    run_hydra_web_server(port, command_q, reply_q)
    logger.info("Hydra web server exited")
    command_q.put(HYDRA_EXIT)
    collector.join()
    li f ("H dh t d")

    logger.info("Hydra shut down")
```

This is a little messy, as all the dependencies come together here. On the plus side, the code is not complex and probably won't change much.

Chỗ này hơi bầy hầy một chút, vì tất cả phụ thuộc đều tụ về đây. Điểm cộng là mã không phức tạp và nhiều khả năng sẽ chẳng thay đổi mấy.

Let's review some of the important attributes of our clean boundaries.

Hãy điểm lại vài đặc tính quan trọng của những ranh giới sạch mà chúng ta vừa dựng.

About the IoT isolation:

Về việc cô lập IoT:

- The business logic is free from the radio system knowledge.
- IoT-dependent functions are low complexity.
- IoT-dependent functions are unlikely to change often or much.
 - Business logic hoàn toàn không biết gì về hệ thống radio.
 - Các hàm phụ thuộc IoT có độ phức tạp thấp.
 - Các hàm phụ thuộc IoT khó mà thay đổi thường xuyên hay thay đổi nhiều.

About the UI isolation:

Về việc cô lập UI:

- Business logic is separate from the UI.

- Communications between the application to the Web server rely on a simple queuing mechanism.
 - Business logic tách bạch khỏi UI.
 - Việc liên lạc giữa ứng dụng và máy chủ Web chỉ dựa trên một cơ chế hàng đợi đơn giản.

About **concurrency**:

Về tính đồng thời (concurrency):

- None of the business logic is concerned with concurrency.
- Concurrency mechanisms are in low-complexity functions with a low risk of changes.
 - Không phần business logic nào phải bận tâm tới đồng thời.
 - Các cơ chế đồng thời nằm trong những hàm có độ phức tạp thấp và ít rủi ro thay đổi.

About object creation and binding:

Về việc tạo và ràng buộc đối tượng (object):

- Object creation is separate from object usage.
- During initialization, the concrete objects are created and bound to processes and to each other.
- Binding is based on a need-to-know basis.
 - Việc tạo đối tượng tách rời khỏi việc sử dụng đối tượng.
 - Trong lúc khởi tạo, các đối tượng cụ thể được tạo ra rồi gắn vào các tiến trình và gắn với nhau.
 - Việc gắn kết chỉ dựa trên nguyên tắc cần-biết-tới-đâu-thì-biết.

SOLID and Hexagonal Architecture — SOLID và kiến trúc Hexagonal

We've looked at the little pieces; now let's look at a bigger picture and the relationship between the classes in the design. Following **the SOLID principles** and isolating dependencies of the outside world naturally leads to something that Alistair Cockburn calls the Hexagonal **architecture** 2 (aka Ports and Adapters).

Chúng ta đã xem những mảnh nhỏ; giờ hãy nhìn bức tranh rộng hơn và quan hệ giữa các class trong thiết kế. Việc tuân theo các nguyên tắc SOLID và cô lập những phụ thuộc vào thế giới bên ngoài sẽ tự nhiên dẫn tới thứ mà Alistair Cockburn gọi là kiến trúc Hexagonal 2 (còn gọi là Ports and Adapters).

2 [Hex].

2 [Hex].

Inside the hexagon is all the product's logic and **behavior**. Instead of a big ball of mud inside, responsibilities are allocated to separate classes, functions, and domain objects. Outside the hexagon is the execution environment. The application depends on service **abstraction** layers and the edge of the hexagon.

Bên trong hình lục giác là toàn bộ logic và hành vi của sản phẩm. Thay vì một quả cầu bùn khổng lồ nằm bên trong, trách nhiệm được phân bổ cho các class, hàm và đối tượng miền riêng biệt. Bên ngoài hình lục giác là môi trường thực thi. Ứng dụng phụ thuộc vào các lớp trừu tượng hoá dịch vụ và vào chính rìa của hình lục giác.

There are two MessageQueue instances: the CommandQueue and the

Có hai instance của MessageQueue : CommandQueue và

ReplyQueue . The UI, through the CommandQueue , drives the system. The user starts a test with a button on the Web page, which writes the “start” command to the CommandQueue . The start command works its way to the sensors that start collecting measurements. The CommandRunner does a timed wait for new UI commands. Each one-second timeout causes the

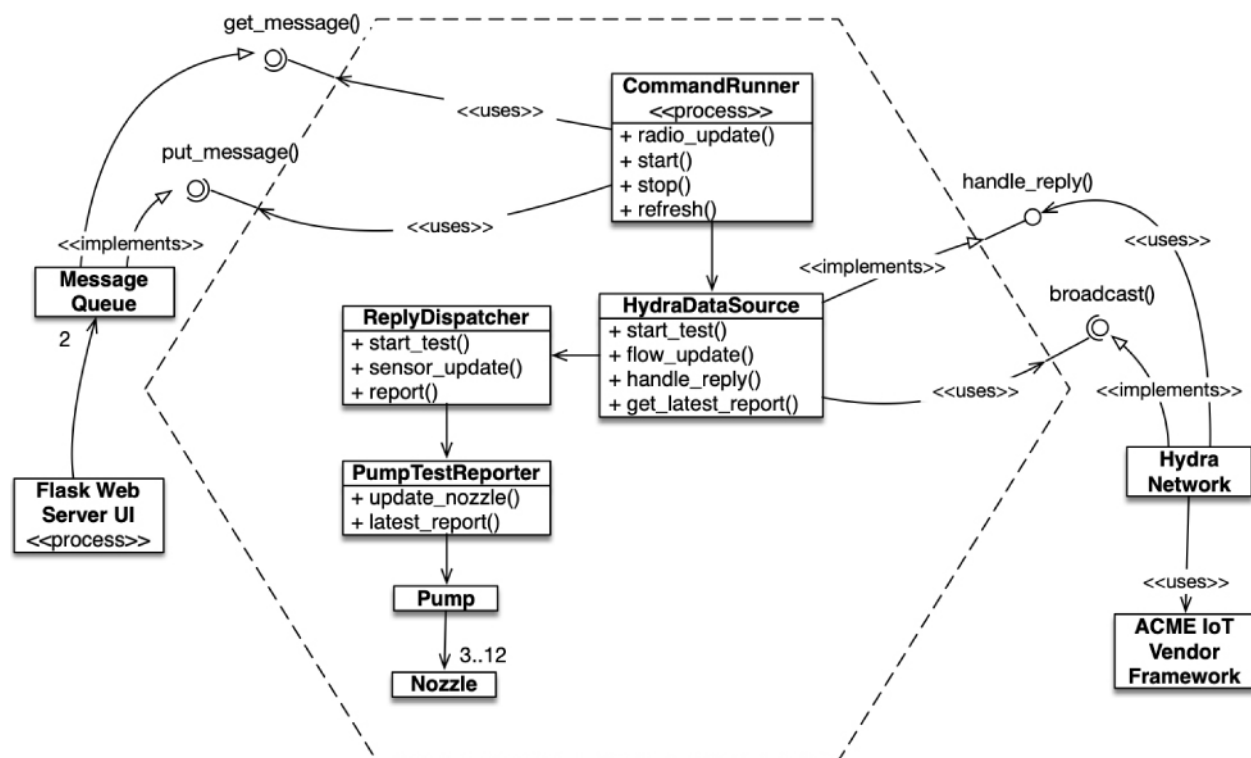
ReplyQueue . UI, thông qua CommandQueue , là thứ điều khiển cả hệ thống. Người dùng bắt đầu một bài test bằng một nút bấm trên trang Web, và nút đó ghi lệnh “start” vào CommandQueue . Lệnh start men theo đường của nó tới các cảm biến, và chúng bắt đầu thu thập số đo. CommandRunner chờ lệnh mới từ UI theo kiểu chờ có hẹn giờ. Mỗi lần hết một giây chờ, nó lại khiến

CommandRunner to initiate a new update from the sensors. The previous second’s sensor readings are bundled into a report that is sent to the ReplyQueue . The following diagram shows how that message starts the flow test. We also saw the connection of the CommandRunner to the queues in the

CommandRunner khởi động một lượt cập nhật mới từ các cảm biến. Số đọc cảm biến của giây trước đó được gói lại thành một báo cáo rồi gửi sang ReplyQueue . Sơ đồ dưới đây cho thấy thông điệp ấy khởi động bài test lưu lượng ra sao. Chúng ta cũng đã thấy mối nối giữa CommandRunner với các hàng đợi trong

measurement_loop() in a code snippet earlier.

measurement_loop() ở một đoạn mã phía trước.



The HydraNetwork encapsulates the dependencies on the ACME IoT vendor framework. HydraNetwork.broadcast() tells all sensors in the network to report their latest measurements. As individual radio replies are received, HydraNetwork calls

HydraNetwork đóng gói mọi phụ thuộc vào framework IoT của nhà cung cấp ACME. HydraNetwork.broadcast() bảo mọi cảm biến trong mạng báo về số đo mới nhất của chúng. Mỗi khi nhận được hồi đáp từ một radio, HydraNetwork lại gọi

handle_reply() , feeding the latest PressureSensor messages into the Hydra application. Hydra knows nothing about the ACME IoT.

handle_reply() , đẩy những thông điệp PressureSensor mới nhất vào ứng dụng Hydra. Hydra chẳng biết gì về ACME IoT cả.

The interfaces of broadcast() and handle_reply() have more to do with what the application wants from the ACME IoT than what the ACME IoT offers. HydraDataSource knows it has a network of sensors, but only needs it to broadcast the need for an update and to receive the updates. It's important that the interfaces at the boundary of the hexagon are about the Hydra application's needs and not about ACME's offerings. The ACME IoT has facilities that are very close to our needs, so the adapter is very thin and has low complexity.

Interface của broadcast() và handle_reply() gắn với những gì ứng dụng cần ở ACME IoT nhiều hơn là với những gì ACME IoT chào mời. HydraDataSource biết rằng mình có một mạng cảm biến, nhưng nó chỉ cần mạng đó broadcast yêu cầu cập nhật và nhận về các cập nhật. Điều quan trọng là interface ở ranh giới của hình lục giác phải nói về nhu cầu của ứng dụng Hydra chứ không phải về những thứ ACME bày ra. ACME IoT có sẵn những tiện ích rất sát với nhu cầu của chúng tôi, nên lớp adapter rất mỏng và có độ phức tạp thấp.

As our product, or the IoT vendor offerings, evolve, we may want to change vendors. If the time comes where we need to replace the ACME IoT or support multiple vendors, the

Khi sản phẩm của chúng tôi, hoặc những gì nhà cung cấp IoT chào bán, tiến hoá, có thể chúng tôi sẽ muốn đổi nhà cung cấp. Nếu tới lúc phải thay thế ACME IoT hoặc phải hỗ trợ nhiều nhà cung cấp cùng lúc, thì

HydraNetwork class would have to be evolved.

class HydraNetwork sẽ là thứ phải tiến hoá theo.

Imagine some new IoT vendor, BrandX, who could provide significant cost savings. Unfortunately, BrandX does not have a broadcast built in, as ACME does. Instead, each radio has to be addressed individually. The BrandX HydraNetwork **implementation** could implement broadcast() , but it would have knowledge of each sensor connection to do so. That layer might become nontrivial, but we could change the vendor without the risk of touching the Hydra core application.

Hãy hình dung có một nhà cung cấp IoT mới, BrandX, hứa hẹn tiết kiệm chi phí đáng kể. Khổ nỗi, BrandX không có sẵn cơ chế broadcast như ACME. Thay vào đó, phải gọi tới từng radio một. Bản hiện thực HydraNetwork cho BrandX vẫn có thể hiện thực broadcast() , nhưng để làm được vậy nó phải biết từng kết nối cảm biến. Lớp đó có thể trở nên không hề tầm thường, nhưng chúng tôi vẫn đổi được nhà cung cấp mà không phải mạo hiểm đụng vào lõi ứng dụng Hydra.

Exploring and Learning Boundaries — Khám phá và học các ranh giới

Third-party code helps us get more functionality delivered in less time. Where do we start when we want to utilize some third-party **package**? It's not our job to test the third-party code, but it may be in our best interest to write tests for the third-party code we use.

Mã của bên thứ ba giúp chúng ta giao được nhiều tính năng hơn trong ít thời gian hơn. Nhưng bắt đầu từ đâu khi ta muốn dùng một package của bên thứ ba? Kiểm thử mã của bên thứ ba không phải việc của chúng ta, nhưng viết test cho phần mã bên thứ ba mà mình dùng thì có thể lại rất có lợi cho chính chúng ta.

Suppose it is not clear how to use our third-party library. We might spend a day or two (or more) reading the documentation and deciding how we are going to use it. Then, we might write our code to use the third-party code and see whether it does what we think. We would not be surprised to find ourselves bogged down in long debugging sessions trying to figure out whether the bugs we are experiencing are in our code or theirs.

Giả sử ta chưa rõ phải dùng thư viện bên thứ ba đó thế nào. Ta có thể mất một hai ngày (hoặc hơn) để đọc tài liệu và quyết định sẽ dùng nó ra sao. Rồi ta viết mã của mình để gọi mã bên thứ ba và xem nó có làm đúng như ta nghĩ không. Chẳng có gì lạ nếu ta thấy mình sa lầy trong những buổi debug dài dằng dặc, chỉ để cố tìm xem mấy con bug đang gặp nằm trong mã của ta hay mã của họ.

Learning the third-party code is hard. Integrating the third-party code is hard too. Doing both at the same time is doubly hard.

Học mã của bên thứ ba đã khó. Tích hợp mã của bên thứ ba cũng khó. Làm cả hai cùng lúc thì khó gấp đôi.

What if we took a different approach? Instead of experimenting and trying out the new stuff in our production code, we could write some tests to explore our understanding of the third-party code. Jim Newkirk calls such tests learning tests . 3

Nếu ta chọn một cách tiếp cận khác thì sao? Thay vì thí nghiệm và thử thứ mới ngay trong mã production, ta có thể viết vài test để khám phá xem mình hiểu mã bên thứ ba tới đâu. Jim Newkirk gọi những test như vậy là learning test . 3

3 [BeckTDD], pp. 136–137.

3 [BeckTDD], tr. 136–137.

In learning tests, we call the third-party API, as we expect to use it in our application. We're essentially doing controlled experiments that check our understanding of that API. The tests focus on what we want out of the API.

Trong learning test, ta gọi API của bên thứ ba đúng theo cách ta dự định dùng nó trong ứng dụng của mình. Về bản chất, ta đang làm những thí nghiệm có kiểm soát để kiểm chứng cách hiểu của mình về API đó. Các test này tập trung vào thứ ta muốn lấy ra từ API.

Not having done concurrent programming in Python before, I thought it would be a **good** idea to learn how the concurrency and communications mechanisms work before using them. I could simply read the docs and integrate their usage into my production code, or I could experiment and learn from the comfort of test cases.

Chưa từng lập trình đồng thời trong Python bao giờ, tôi nghĩ tốt nhất là học xem các cơ chế đồng thời và truyền thông hoạt động ra sao trước khi dùng chúng. Tôi có thể chỉ việc đọc tài liệu rồi nhét chúng thẳng vào mã production, hoặc tôi có thể thí nghiệm và học trong sự thoải mái an toàn của các test case.

From looking at the documentation, I decided to test-drive two functions, `put_message()` and `get_message()` , that put and get messages from a queue.

Sau khi ngó qua tài liệu, tôi quyết định test-drive hai hàm, `put_message()` và `get_message()` , làm nhiệm vụ đưa thông điệp vào và lấy thông điệp ra khỏi một hàng đợi.

Here are the tests that helped me know for sure that I understood how Python's Queue works.

Đây là những test đã giúp tôi chắc chắn rằng mình hiểu Queue của Python hoạt động thế nào.

```
class QueueTest(unittest.TestCase):
    def test_get_message_returns_message(self):
        q = Queue(maxsize=1)
        self.assertTrue(put_message(q, 'hey'))
    ifl( h(0))

    self.assertEqual('hey', get_message(q, 1.0))
    def test_get_message_None_when_timeout(self):
        q = Queue(maxsize=1)
        self.assertEqual(None, get_message(q, 0.01))
    def test_get_message_None_empty_no_wait(self):
        q = Queue(maxsize=1)
        self.assertEqual(None, get_message(q, 0.0))
    def test_put_message_True_for_success(self):
        q = Queue(maxsize=1)
        self.assertTrue(put_message(q, 'hey'))
        self.assertEqual('hey', get_message(q, 1.0))
    def test_put_message_False_when_full(self):
        q = Queue(maxsize=1)
        self.assertTrue(put_message(q, 'hey1'))
        self.assertFalse(put_message(q, 'hey2'))
        self.assertEqual('hey1', get_message(q, 1.0))
```

Now that we know how Queue functions work, let's use them to communicate with a Process . First, we define a function,

Giờ đã biết các hàm của Queue hoạt động thế nào, hãy dùng chúng để liên lạc với một Process . Trước hết, ta định nghĩa một hàm,

worker() , that will be executed on a separate Process . It gets from in_q , puts to out_q , and then exits. At the time, we expected to use this approach to separate the Hydra core from the WebServer .

worker() , sẽ được chạy trên một Process riêng. Nó lấy từ in_q , đẩy vào out_q , rồi thoát. Ở thời điểm đó, chúng tôi dự tính dùng cách này để tách lõi Hydra khỏi WebServer .

```
def worker(in_q, out_q):
    message = get_message(in_q, 0.05)
    if message:
        put_message(out_q, message)
class ProcessLearningTest(unittest.TestCase):
    def setUp(self):
        self.command_q = Queue(maxsize=1)
        self.reply_q = Queue(maxsize=1)
        self.process = Process(target=worker,
                                args=(self.command_q,
                                      self.reply_q,))
        self.process.daemon = True
    def test_thread_q_message_echos(self):
        put_message(self.command_q, 'Do this')
```

```

self.process.start()
reply = self.reply_q.get(True, 0.1)
self.process.join()
self.assertEqual('Do this', reply)
def test_thread_message_timeout(self):
    self.process.start()
    reply = get_message(self.reply_q, 0.1)
    self.process.join()
    self.assertEqual(None, reply))

```

These learning tests cost nothing. We have to learn mechanics anyway, and writing those tests was an easy and isolated way to get that knowledge. The learning tests were precise experiments that helped increase our understanding.

Những learning test này chẳng tốn gì. Chẳng nào ta cũng phải học cơ chế vận hành, và viết mấy test đó là cách dễ dàng, biệt lập để có được hiểu biết ấy. Learning test chính là những thí nghiệm chính xác giúp ta hiểu sâu hơn.

Not only are learning tests free, they have a positive return on investment. When there are new releases of the third-party package, we run the learning tests to see whether there are behavioral differences.

Learning test không chỉ miễn phí, chúng còn sinh lời. Khi package bên thứ ba ra bản mới, ta chạy lại learning test để xem hành vi có gì khác đi không.

Learning tests verify that the third-party packages we are using work the way we expect them to. Once integrated, there are no guarantees that the third-party code will stay compatible with our needs. The original authors will have pressures to change their code to meet new needs of their own. They will fix bugs and add new capabilities. With each release comes new risk. If the third-party package changes in some way that is incompatible with our tests, we will find out right away.

Learning test xác nhận rằng những package bên thứ ba ta đang dùng vẫn hoạt động đúng như ta kỳ vọng. Một khi đã tích hợp, không có gì bảo đảm mã bên thứ ba sẽ mãi tương thích với nhu cầu của ta. Các tác giả gốc sẽ chịu sức ép phải sửa mã của họ cho hợp với những nhu cầu mới của chính họ. Họ sẽ sửa bug và bổ sung khả năng mới. Mỗi bản phát hành đều kèm theo rủi ro mới. Nếu package bên thứ ba đổi theo hướng nào đó không còn tương thích với test của ta, ta sẽ biết ngay lập tức.

In this case, who expects Python to change? So, there is no future payback. Well, actually, Python did change. While converting the Hydra code from Python 2 to Python 3, the packaging changed; some edits were needed to compile and run the tests. Nicely, our learning tests showed us the behavior was preserved.

Mà trong trường hợp này, ai lại nghĩ Python sẽ thay đổi chứ? Vậy nên chẳng có khoản hoàn vốn nào trong tương lai đâu. À, thật ra thì Python có thay đổi. Trong lúc chuyển mã Hydra từ Python 2 sang Python 3, cách đóng gói đã đổi; phải chỉnh sửa đôi chỗ mới biên dịch và chạy được test. May thay, các learning test cho thấy hành vi vẫn được giữ nguyên.

Regardless of whether you need the learning provided by the learning tests, a **clean boundary** should be supported by a set of boundary tests that exercise the interface the same way the production code does. Without these boundary tests to ease the migration, we might be tempted to stay with the old version longer than we should.

Dù bạn có cần đến phần kiến thức mà learning test mang lại hay không, một ranh giới sạch vẫn nên được chống lưng bởi một bộ test ranh giới, thử vận hành interface đúng theo cách

mã production vận hành nó. Không có những test ranh giới ấy để làm nhẹ việc di trú, ta rất dễ bị cám dỗ bám lấy phiên bản cũ lâu hơn mức đáng ra nên làm.

Using Code That Does Not Yet Exist — Dùng thứ mã còn chưa tồn tại

There is another kind of boundary, one that separates the known from the unknown. There are often places in the code where our knowledge seems to drop off the edge.

Còn một loại ranh giới nữa, loại ngăn cách cái đã biết với cái chưa biết. Trong mã thường có những chỗ mà hiểu biết của chúng ta như rơi tòm xuống vực.

Sometimes what is on the other side of the boundary is unknowable (at least right now). Sometimes we choose to look no further than the boundary.

Đôi khi thứ nằm bên kia ranh giới là bất khả tri (ít nhất là tại thời điểm này). Đôi khi chính chúng ta chọn không nhìn xa hơn ranh giới đó.

A number of years back I was part of a team developing software for a radio communications system. There was a subsystem, the “Transmitter,” that we knew little about, and the people responsible for the subsystem had not gotten to the point of defining their interface. We did not want to be blocked, so we started our work away from the unknown part of the code.

Nhiều năm trước, tôi tham gia một nhóm phát triển phần mềm cho hệ thống liên lạc vô tuyến. Có một hệ thống con tên là “Transmitter” mà chúng tôi biết rất ít, còn những người phụ trách nó thì vẫn chưa đi đến bước định nghĩa interface. Chúng tôi không muốn bị chặn lại, nên bắt đầu làm phần việc của mình ở xa cái vùng mã chưa biết ấy.

We had a pretty good idea of where our world ended and the new world began. As we worked, we sometimes bumped up against this boundary. Though mists and clouds of ignorance obscured our view beyond the boundary, our work made us aware of what we wanted the boundary interface to be. We wanted to tell the transmitter something like this:

Chúng tôi khá rõ thế giới của mình kết thúc ở đâu và thế giới mới bắt đầu từ đâu. Trong lúc làm, thỉnh thoảng chúng tôi lại va vào ranh giới này. Dù sương mù và mây mờ của sự thiếu hiểu biết che khuất tầm nhìn sang phía bên kia, công việc vẫn giúp chúng tôi nhận ra mình muốn interface ở ranh giới đó trông như thế nào. Chúng tôi muốn nói với transmitter đại loại thế này:

Key the transmitter on the provided frequency and emit an analog representation of the data coming from this stream.

Hãy kích hoạt transmitter ở tần số được cung cấp và phát ra biểu diễn analog của dữ liệu đến từ stream này.

We had no idea how that would be done, because the hardware and the API had not been designed yet. So we would have to work out the details later.

Chúng tôi hoàn toàn không biết việc đó sẽ được làm ra sao, bởi phần cứng lẫn API đều chưa được thiết kế. Nên chi tiết đành để tính sau.

To keep from being blocked, we defined our own interface to the nonexistent hardware. We called it something catchy, like Transmitter. We gave it a method called transmit that took a frequency and a data stream. This was the interface we wished we had.

Để khỏi bị chặn, chúng tôi tự định nghĩa interface của riêng mình cho phần cứng chưa tồn tại kia. Chúng tôi đặt cho nó một cái tên bắt tai, kiểu như Transmitter . Chúng tôi cho nó một method tên là transmit nhận vào một tần số và một luồng dữ liệu. Đây chính là interface mà chúng tôi ước gì mình đã có.

One good thing about writing the interface is that it's under our control. This helps keep client code more readable and focused on what it is trying to accomplish. On the client side of the boundary, we are not burdened by the many implementation details. If we had a hardware abstraction layer (HAL), we probably would have started depending on that. HALs usually are not all that abstract from the application perspective. Not knowing the HAL, we had no way to depend on it. Instead of a liability, for now it was an advantage.

Một điểm hay của việc tự viết interface là nó nằm trong tầm kiểm soát của ta. Điều này giúp mã phía client dễ đọc hơn và tập trung vào đúng thứ nó đang muốn hoàn thành. Ở phía client của ranh giới, chúng ta không bị đè nặng bởi vô số chi tiết hiện thực (implementation). Nếu đã có sẵn một tầng trừu tượng hoá phần cứng (HAL), có lẽ chúng tôi đã bắt đầu phụ thuộc vào nó. Mà HAL thì thường chẳng trừu tượng đến thế dưới góc nhìn của ứng dụng. Vì không biết gì về HAL, chúng tôi chẳng có cách nào phụ thuộc vào nó. Thay vì là một gánh nặng, lúc này đó lại là một lợi thế.

In the following diagram, you can see that we insulated the

Trong sơ đồ dưới đây, bạn có thể thấy chúng tôi đã cách ly các class

CommunicationsController classes from the transmitter API (which was out of our control and currently undefined). By using our own application-specific interface, we kept our

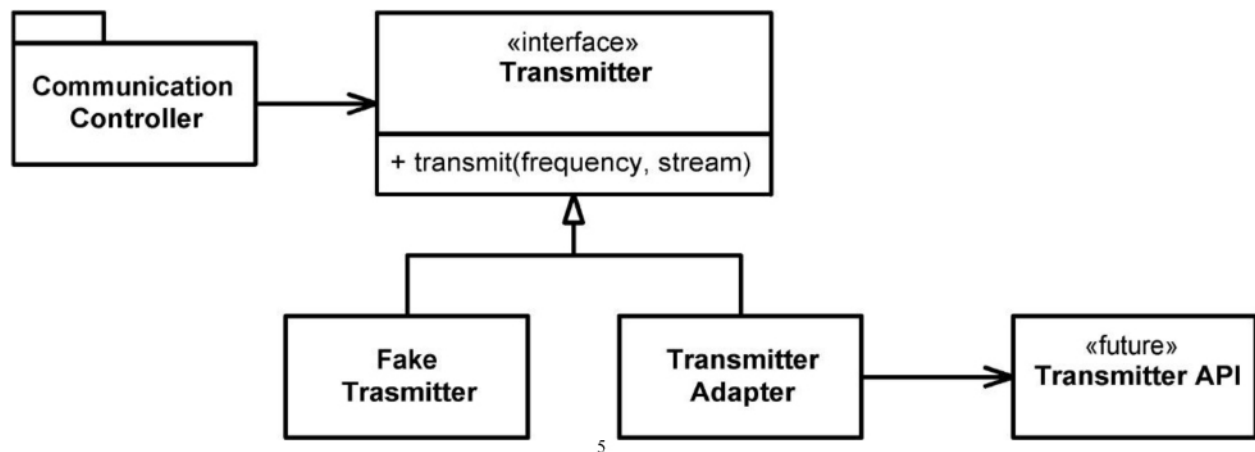
CommunicationsController khỏi transmitter API (thứ nằm ngoài tầm kiểm soát của chúng tôi và khi ấy còn chưa được định nghĩa). Bằng cách dùng interface riêng đặc thù cho ứng dụng của mình, chúng tôi giữ cho mã

CommunicationsController code clean and expressive. Once the transmitter API was defined, we wrote the TransmitterAdapter to bridge the gap. The Adapter 4 encapsulated the interaction with the API and provides a single place to change when the API evolves.

CommunicationsController sạch sẽ và giàu tính biểu đạt. Khi transmitter API được định nghĩa xong, chúng tôi viết TransmitterAdapter để bắc cầu qua khoảng trống đó. Adapter 4 đóng gói (encapsulation) toàn bộ tương tác với API và cung cấp một chỗ duy nhất cần sửa mỗi khi API thay đổi.

4 See the Adapter pattern in [GOF95].

4 Xem mẫu Adapter trong [GOF95].



This design also gives us a very convenient seam 5 in the code for testing. Using a suitable FakeTransmitter , we can test the CommunicationsController classes. Once we have the

Thiết kế này còn cho chúng ta một seam 5 rất tiện lợi trong mã để phục vụ việc test. Với một FakeTransmitter phù hợp, chúng ta có thể test các class CommunicationsController . Khi đã có

TransmitterAPI , we can also create boundary tests that make sure we are using the API correctly.

TransmitterAPI , chúng ta cũng có thể tạo những boundary test để bảo đảm mình đang dùng API đúng cách.

5 See more about seams in [WELC].

5 Đọc thêm về seam trong [WELC].

Clean Boundaries — Ranh giới sạch

Interesting things happen at boundaries. Change is one of those things. Good software designs accommodate change without huge investments and rework. When we use code that is out of our control, special care must be taken to protect our investment and make sure future change is not too costly.

Những chuyện thú vị luôn xảy ra ở ranh giới. Thay đổi là một trong số đó. Thiết kế phần mềm tốt đón nhận được thay đổi mà không đòi hỏi đầu tư khổng lồ hay làm lại từ đầu. Khi dùng mã nằm ngoài tầm kiểm soát của mình, ta phải đặc biệt cẩn thận để bảo vệ khoản đầu tư của mình và bảo đảm rằng thay đổi trong tương lai không quá đắt đỏ.

Code at the boundaries needs clear separation and tests that define expectations. We should avoid letting too much of our code know about the third-party particulars. It's better to depend on something you control than on something you don't control, lest it end up controlling you.

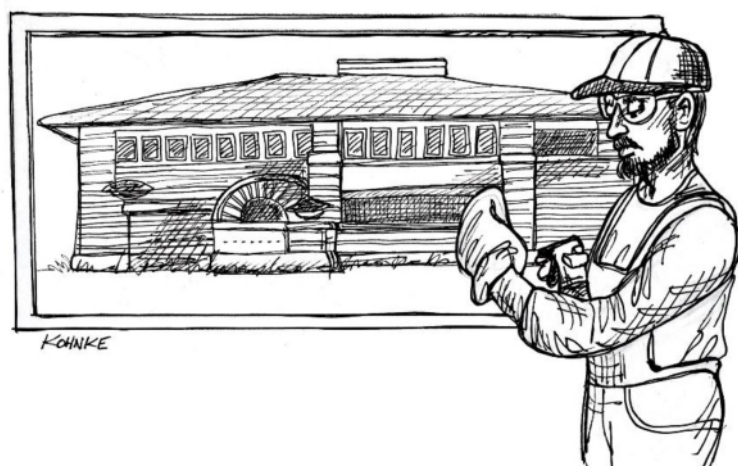
Mã ở ranh giới cần được tách bạch rõ ràng và cần những test định nghĩa rõ kỳ vọng. Chúng ta nên tránh để quá nhiều phần mã của mình biết về những chi tiết riêng của thư viện bên thứ ba. Thà phụ thuộc vào thứ bạn kiểm soát được còn hơn phụ thuộc vào thứ bạn không kiểm soát nổi, kéo rối cuộc chính nó lại kiểm soát bạn.

We manage third-party boundaries by having very few places in the code that refer to them. We may wrap third-party dependencies in an ADAPTER to convert from our application-centered interface to the provided interface. Either way, our code speaks to us better, promotes internally consistent usage across the boundary, and has fewer maintenance points when the third-party code changes.

Chúng ta quản lý các ranh giới với bên thứ ba bằng cách chỉ để rất ít chỗ trong mã tham chiếu tới chúng. Ta có thể bọc các phụ thuộc (dependency) bên thứ ba trong một ADAPTER để chuyển đổi từ interface lấy ứng dụng làm trung tâm của mình sang interface mà họ cung cấp. Dù theo cách nào đi nữa, mã của chúng ta cũng nói với ta rõ ràng hơn, khuyến khích cách dùng nhất quán trong nội bộ khi đi qua ranh giới, và có ít điểm phải bảo trì hơn khi mã bên thứ ba thay đổi.

Chapter 27. The Clean Architecture

— Chương 27. Clean Architecture



Over the last several decades, we've seen a whole range of ideas regarding the **architecture** of systems. These include

Suốt mấy thập kỷ vừa qua, chúng ta đã chứng kiến hàng loạt ý tưởng về kiến trúc (architecture) của hệ thống. Trong số đó có

- Hexagonal architecture (aka Ports and Adapters) by Alistair Cockburn, and adopted by Steve Freeman and Nat Pryce in their wonderful book Growing **Object**- Oriented Software, Guided by Tests (Addison-Wesley, 2009)
- **Data**, Context, and Interaction (DCI) architecture by James Coplien and Trygve Reenskaug
- Boundary–Control–**Entity** (BCE) architecture by Ivar Jacobson from his book Object-Oriented Software Engineering: A Use-Case Driven Approach (Addison- Wesley, 1992)
 - Hexagonal architecture (còn gọi là Ports and Adapters) của Alistair Cockburn, được Steve Freeman và Nat Pryce áp dụng trong cuốn sách tuyệt vời Growing Object-Oriented Software, Guided by Tests (Addison-Wesley, 2009)
 - Kiến trúc Data, Context, and Interaction (DCI) của James Coplien và Trygve Reenskaug
 - Kiến trúc Boundary–Control–Entity (BCE) của Ivar Jacobson trong cuốn Object-Oriented Software Engineering: A Use-Case Driven Approach (Addison-Wesley, 1992)

Though these architectures all vary somewhat in their details, they are very similar. They all have the same objective, which is the separation of concerns. They all achieve this separation by dividing the

software into layers. Each has at least one layer for business rules and another for user and system interfaces.

Tuy chi tiết của các kiến trúc này có khác nhau đôi chút, chúng rất giống nhau. Tất cả đều nhằm tới cùng một mục tiêu: phân tách các mối quan tâm (separation of concerns). Và tất cả đều đạt được sự phân tách đó bằng cách chia phần mềm thành các tầng. Kiến trúc nào cũng có ít nhất một tầng dành cho quy tắc nghiệp vụ (business rule) và một tầng khác dành cho giao diện người dùng cùng giao diện hệ thống.

Each of these architectures produce systems that are

Mỗi kiến trúc kể trên đều tạo ra những hệ thống

- Independent of frameworks. The architecture does not depend on the existence of some library of feature-laden software. This allows you to use such frameworks as tools, rather than forcing you to cram your system into their limited constraints.
 - Testable. The business rules can be tested without the UI, database, Web server, or any other external element.
 - Independent of UI. The UI can change easily, without changing the rest of the system. A Web UI could be replaced with a console UI, for example, without changing the business rules.
 - Independent of database. You can swap out Oracle or SQL Server for Mongo, Bigtable, CouchDB, or something else. Your business rules are not bound to the database.
 - Independent of any external agency. In fact, your business rules simply don't know anything at all about the interfaces to the outside world.
- Độc lập với framework. Kiến trúc không phụ thuộc vào sự tồn tại của một thư viện phần mềm nhồi đầy tính năng nào đó. Nhờ vậy bạn dùng framework như công cụ, thay vì bị ép nhét cả hệ thống của mình vào những ràng buộc chặt hẹp của chúng.
 - Kiểm thử được. Các quy tắc nghiệp vụ có thể được test mà không cần UI, cơ sở dữ liệu, Web server hay bất kỳ thành phần bên ngoài nào khác.
 - Độc lập với UI. UI có thể thay đổi dễ dàng mà không phải đụng tới phần còn lại của hệ thống. Chẳng hạn, một UI trên Web có thể được thay bằng UI dòng lệnh mà quy tắc nghiệp vụ vẫn nguyên vẹn.
 - Độc lập với cơ sở dữ liệu. Bạn có thể tháo Oracle hay SQL Server ra và lắp Mongo, Bigtable, CouchDB hay thứ gì khác vào. Quy tắc nghiệp vụ của bạn không bị trói vào cơ sở dữ liệu.
 - Độc lập với mọi tác nhân bên ngoài. Thật ra, quy tắc nghiệp vụ của bạn hoàn toàn chẳng biết gì về các interface hướng ra thế giới bên ngoài.

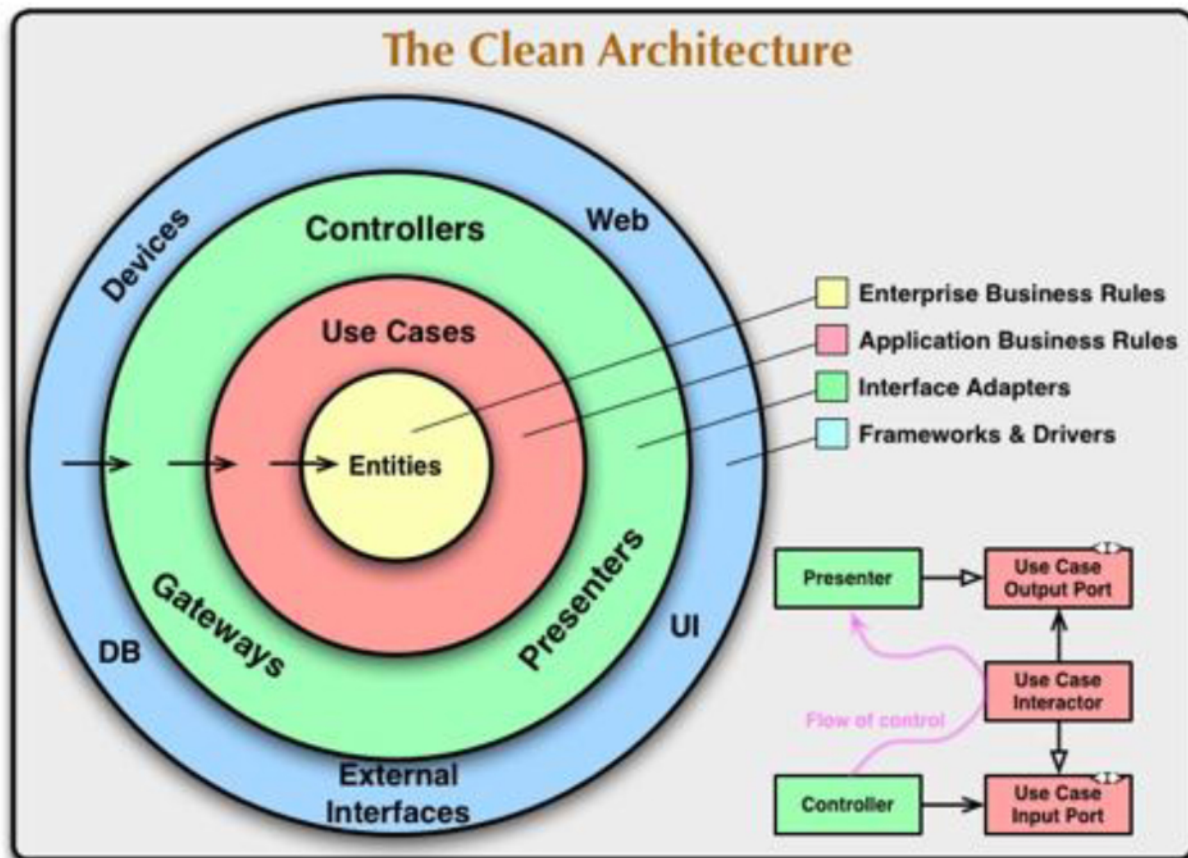
The Dependency Rule — Quy tắc Phụ thuộc

The diagram that follows is an attempt at integrating all these architectures into a single actionable idea.

Sơ đồ dưới đây là một nỗ lực tích hợp tất cả những kiến trúc đó thành một ý tưởng duy nhất, dùng được ngay.

The concentric circles represent different areas of software. In general, the further in you go, the higher level the software becomes. The outer circles are mechanisms. The inner circles are policies.

Các vòng tròn đồng tâm biểu diễn những vùng khác nhau của phần mềm. Nhìn chung, càng đi vào trong thì phần mềm càng ở mức cao hơn. Vòng ngoài là cơ chế (mechanism). Vòng trong là chính sách (policy).



The overriding rule that makes this architecture work is the **Dependency** Rule. This rule says:

Quy tắc bao trùm khiến kiến trúc này vận hành được là Quy tắc Phụ thuộc (the Dependency Rule). Quy tắc đó nói rằng:

Source code dependencies must only point inward, toward higher-level policies.

Các phụ thuộc (dependency) trong mã nguồn chỉ được phép trở vào trong, hướng về những chính sách ở mức cao hơn.

Nothing in an inner circle can know anything at all about something in an outer circle. In particular, the name of something declared in an outer circle must not be mentioned by the code in an inner circle. That includes functions, classes, variables, or any other named software entity.

Không thứ gì ở vòng trong được biết bất cứ điều gì về thứ ở vòng ngoài. Cụ thể, tên của một thứ được khai báo ở vòng ngoài tuyệt đối không được xuất hiện trong mã ở vòng trong. Điều đó áp dụng cho hàm, class, biến, hay bất kỳ thực thể phần mềm có tên nào khác.

By the same token, data formats declared in an outer circle should not be used by an inner circle, especially if those formats are generated by a framework in an outer circle. We don't want anything in an outer circle to impact the inner circles.

Tương tự, các định dạng dữ liệu khai báo ở vòng ngoài cũng không nên được vòng trong sử dụng, nhất là khi những định dạng ấy do một framework ở vòng ngoài sinh ra. Chúng ta không muốn bất cứ thứ gì ở vòng ngoài tác động tới các vòng trong.

Entities — Entity

Entities encapsulate enterprise-wide critical business rules. An entity can be an object with methods, or it can be a set of data structures and functions. It doesn't matter so long as the entities can be used by many different applications in the enterprise.

Entity đóng gói (encapsulation) những quy tắc nghiệp vụ trọng yếu ở cấp toàn doanh nghiệp. Một entity có thể là một đối tượng (object) có method, mà cũng có thể là một tập hợp các cấu trúc dữ liệu (data structure) và hàm. Điều đó không quan trọng, miễn là các entity ấy dùng được cho nhiều ứng dụng khác nhau trong doanh nghiệp.

If you don't have an enterprise and are just writing a single application, then these entities are the business objects of the application. They encapsulate the most general and high-level rules. They are the least likely to change when something external changes. For example, you would not expect these objects to be affected by a change to page navigation or security. No operational change to any particular application should affect the entity layer.

Nếu bạn không có cả một doanh nghiệp mà chỉ đang viết một ứng dụng đơn lẻ, thì các entity này chính là những đối tượng nghiệp vụ của ứng dụng. Chúng đóng gói những quy tắc tổng quát nhất, ở mức cao nhất. Chúng cũng là thứ ít có khả năng thay đổi nhất khi có gì đó bên ngoài thay đổi. Chẳng hạn, bạn sẽ không mong đợi những đối tượng này bị ảnh hưởng bởi một thay đổi về cách điều hướng trang hay về bảo mật. Không một thay đổi vận hành nào của một ứng dụng cụ thể được phép ảnh hưởng tới tầng entity.

Use Cases — Use Case

The software in this layer contains application-specific business rules. It encapsulates and implements all of the use cases of the system. These use cases orchestrate the flow of data to and from the entities, and direct those entities to use their critical business rules to achieve the goals of the **use case**.

Phần mềm ở tầng này chứa các quy tắc nghiệp vụ riêng của ứng dụng. Nó đóng gói và hiện thực (implementation) toàn bộ use case của hệ thống. Những use case này điều phối dòng dữ liệu đi vào và đi ra khỏi các entity, đồng thời chỉ đạo các entity ấy dùng những quy tắc nghiệp vụ trọng yếu của chúng để đạt được mục tiêu của use case.

We do not expect changes in this layer to affect the entities. We also do not expect this layer to be affected by changes to externalities such as the database, the UI, or any of the common frameworks. This layer is isolated from such concerns.

Chúng ta không mong những thay đổi ở tầng này ảnh hưởng tới các entity. Chúng ta cũng không mong tầng này bị ảnh hưởng bởi những thay đổi ở bên ngoài như cơ sở dữ liệu, UI hay bất kỳ framework phổ biến nào. Tầng này được cách ly khỏi những mối bận tâm đó.

We do, however, expect that changes to the operation of the application will affect the use cases and therefore the software in this layer. If the details of a use case change, then some code in this layer will certainly be affected.

Nhưng chúng ta có mong rằng thay đổi trong cách vận hành của ứng dụng sẽ ảnh hưởng tới các use case, và do đó ảnh hưởng tới phần mềm ở tầng này. Nếu chi tiết của một use case thay đổi, chắc chắn sẽ có mã ở tầng này bị ảnh hưởng.

Interface Adapters — Interface Adapter

The software in this layer is a set of adapters that convert data from the format most convenient for the use cases and entities to the format most convenient for some external agency such as the database or the Web. It is this layer, for example, that will wholly contain the MVC architecture of a GUI. The presenters, views, and controllers all belong in here. The models are likely just data structures that are passed from the controllers to the use cases, and then back from the use cases to the presenters and views.

Phần mềm ở tầng này là một tập hợp các adapter, chuyển dữ liệu từ định dạng thuận tiện nhất cho use case và entity sang định dạng thuận tiện nhất cho một tác nhân bên ngoài nào đó, chẳng hạn cơ sở dữ liệu hay Web. Ví dụ, chính tầng này chứa trọn vẹn kiến trúc MVC của một GUI. Presenter, view và controller đều thuộc về đây. Model thì nhiều khả năng chỉ là các cấu trúc dữ liệu được truyền từ controller sang use case, rồi lại từ use case quay về presenter và view.

Similarly, data is converted in this layer from the form most convenient for entities and use cases into the form most convenient for whatever persistence framework is being used (i.e., the database). No code inward of this circle should know anything at all about the database. If the database is a SQL database, then all the SQL should be restricted to this layer, and in particular to the parts of this layer that have to do with the database.

Tương tự, chính tại tầng này dữ liệu được chuyển từ dạng thuận tiện nhất cho entity và use case sang dạng thuận tiện nhất cho framework lưu trữ đang dùng (tức là cơ sở dữ liệu). Không một dòng mã nào nằm phía trong vòng tròn này được biết bất cứ điều gì về cơ sở dữ liệu. Nếu cơ sở dữ liệu là loại SQL, thì toàn bộ SQL phải bị giới hạn trong tầng này, và cụ thể hơn là trong những phần của tầng này liên quan tới cơ sở dữ liệu.

Also in this layer is any other adapter necessary to convert data from some external form, such as an external service, to the internal form used by the use cases and entities.

Cũng nằm ở tầng này là mọi adapter khác cần thiết để chuyển dữ liệu từ một dạng bên ngoài nào đó, chẳng hạn từ một dịch vụ ngoài, sang dạng nội bộ mà use case và entity sử dụng.

Frameworks and Drivers — Framework và Driver

The outermost layer is generally composed of frameworks and tools such as the database and the Web framework. Generally, you don't write much code in this layer other than glue code that communicates to the next circle inward.

Tầng ngoài cùng thường gồm các framework và công cụ như cơ sở dữ liệu hay Web framework. Nói chung bạn không viết nhiều mã ở tầng này, ngoài phần mã keo dính để giao tiếp với vòng tròn kế tiếp phía trong.

This layer is where all the details go. The Web is a detail. The database is a detail. We keep these things on the outside, where they can do little **harm**.

Tầng này là nơi mọi chi tiết được đẩy tới. Web là một chi tiết. Cơ sở dữ liệu là một chi tiết. Chúng ta giữ những thứ đó ở bên ngoài, nơi chúng ít gây tổn hại (harm) nhất.

Only Four Circles? — Chỉ có bốn vòng thôi sao?

No, the circles are schematic. You may find that you need more than just these four. There's no rule that says you must always have just these four. However, the Dependency Rule always applies. Source code dependencies always point inward. As you move inward, the **level of abstraction** and policy increases. The outermost circle is low-level concrete detail. As you move inward, the software grows more abstract and encapsulates higher-level policies. The innermost circle is the most general and highest level.

Không, các vòng tròn chỉ mang tính minh họa. Bạn có thể thấy mình cần nhiều hơn bốn vòng này. Chẳng có quy tắc nào bắt bạn phải luôn có đúng bốn vòng cả. Tuy nhiên, Quy tắc Phụ thuộc thì luôn luôn áp dụng. Các phụ thuộc trong mã nguồn luôn trở vào trong. Càng đi vào trong, mức trừu tượng và mức chính sách càng tăng. Vòng ngoài cùng là chi tiết cụ thể ở mức thấp. Càng vào trong, phần mềm càng trừu tượng hơn và đóng gói những chính sách ở mức cao hơn. Vòng trong cùng là tổng quát nhất và ở mức cao nhất.

Crossing Boundaries — Vượt qua ranh giới

At the lower right of the diagram is an example of how we cross the circle boundaries. It shows the Controllers and

Ở góc dưới bên phải sơ đồ là một ví dụ về cách chúng ta vượt qua ranh giới giữa các vòng tròn. Nó cho thấy Controller và

Presenters communicating with the Use Cases in the next layer. Note the flow of control. It begins in the controller, moves through the use case, and then winds up executing in the presenter. Note also the source code dependencies. Each one of them points inward toward the use cases.

Presenter giao tiếp với Use Case ở tầng kế tiếp. Hãy để ý luồng điều khiển. Nó khởi đầu trong controller, đi xuyên qua use case, rồi cuối cùng chạy trong presenter. Cũng hãy để ý các phụ thuộc trong mã nguồn. Phụ thuộc nào cũng trở vào trong, hướng về các use case.

We usually resolve this apparent contradiction by using the Dependency Inversion Principle (DIP). In a language like Java, for example, we would arrange interfaces and implements/**inheritance** relationships such that the source code dependencies oppose the flow of control at just the right points across the boundary.

Chúng ta thường hoá giải mâu thuẫn bề ngoài này bằng DIP — Nguyên tắc Đảo ngược Phụ thuộc (Dependency Inversion Principle). Chẳng hạn với một ngôn ngữ như Java, ta sẽ sắp xếp các interface cùng quan hệ implements/kế thừa (inheritance) sao cho phụ thuộc trong mã nguồn đi ngược lại luồng điều khiển, đúng tại những điểm cần thiết khi băng qua ranh giới.

For example, consider that the use case needs to call the presenter. However, this call must not be direct, because that would violate the Dependency Rule: No name in an outer circle can be mentioned by an inner circle. So we have the use case call an **interface** (shown here as Use Case Output Port) in the inner circle, and have the presenter in the outer circle implement it.

Ví dụ, hãy xét trường hợp use case cần gọi presenter. Nhưng lời gọi này không được phép trực tiếp, vì như thế là vi phạm Quy tắc Phụ thuộc: không tên nào ở vòng ngoài được vòng trong nhắc đến. Vậy nên ta để use case gọi một interface (ở đây được thể hiện là Use Case Output Port) nằm trong vòng trong, còn presenter ở vòng ngoài thì hiện thực interface đó.

The same technique is used to cross all the boundaries in the architectures. We take advantage of dynamic **polymorphism** to create source code dependencies that oppose the flow of control so that we can conform to the Dependency Rule no matter in what direction the flow of control is going.

Cùng một kỹ thuật ấy được dùng để vượt mọi ranh giới trong các kiến trúc này. Ta tận dụng đa hình (polymorphism) động để tạo ra những phụ thuộc trong mã nguồn đi ngược luồng điều khiển, nhờ đó vẫn tuân thủ Quy tắc Phụ thuộc bất kể luồng điều khiển chạy theo hướng nào.

What Data Crosses the Boundaries — Dữ liệu nào vượt qua ranh giới

Typically, the data that crosses the boundaries are simple data structures. You can use basic structs or simple data transfer objects (DTOs) if you like. Or the data can simply be arguments in **function** calls. Or you can pack it into a hash map or construct it into an object. The important thing is that isolated, simple data structures are passed across the boundaries. We don't want to cheat and pass entities or database rows. We don't want the data structures to have any kind of dependency that violates the Dependency Rule.

Thông thường, dữ liệu vượt qua ranh giới là những cấu trúc dữ liệu đơn giản. Bạn có thể dùng struct cơ bản hoặc DTO (data transfer object) đơn giản nếu muốn. Hoặc dữ liệu đơn giản chỉ là các đối số trong lời gọi hàm. Hoặc bạn nhét nó vào một hash map, hoặc dựng nó thành một đối tượng. Điều quan trọng là những cấu trúc dữ liệu đơn giản, biệt lập mới là thứ được truyền qua ranh giới. Chúng ta không muốn gian lận bằng cách truyền entity hay hàng dữ liệu của cơ sở dữ liệu. Chúng ta không muốn các cấu trúc dữ liệu ấy mang bất kỳ phụ thuộc nào vi phạm Quy tắc Phụ thuộc.

For example, many database frameworks return a convenient data format in response to a query. We might call this a row **structure**. We don't want to pass that row structure inward across a boundary. That would violate the Dependency Rule because it would force an inner circle to know something about an outer circle.

Chẳng hạn, nhiều framework cơ sở dữ liệu trả về một định dạng dữ liệu rất tiện lợi để đáp lại một truy vấn. Ta có thể gọi đó là một cấu trúc row. Chúng ta không muốn truyền cấu trúc row đó vào trong, băng qua một ranh giới. Làm vậy là vi phạm Quy tắc Phụ thuộc, vì nó buộc một vòng trong phải biết điều gì đó về vòng ngoài.

So, when we pass data across a boundary, it is always in the form that is most convenient for the inner circle.

Vậy nên, khi truyền dữ liệu qua một ranh giới, dữ liệu ấy luôn ở dạng thuận tiện nhất cho vòng trong.

A Typical Scenario — Một kịch bản điển hình

The following shows a typical scenario for a Web-based system using a database. The Web server gathers input data from the user and hands it to the Controller on the upper left. The Controller packages that data into a plain old object and passes it through the InputBoundary to the UseCaseInteractor. The

Phần dưới đây mô tả một kịch bản điển hình cho hệ thống chạy trên Web có dùng cơ sở dữ liệu. Web server thu thập dữ liệu đầu vào từ người dùng rồi giao nó cho Controller ở

góc trên bên trái. Controller đóng gói dữ liệu ấy vào một plain old object và truyền qua InputBoundary tới UseCaseInteractor . Đến lượt mình,

UseCaseInteractor interprets that data and uses it to control the dance of the Entities . It also uses the DataAccessInterface to bring the data used by those Entities into memory from the

UseCaseInteractor diễn giải dữ liệu đó và dùng nó để điều phối điệu múa của các Entities . Nó cũng dùng DataAccessInterface để nạp vào bộ nhớ phần dữ liệu mà các Entities ấy cần, lấy từ

Database . Upon completion, the UseCaseInteractor gathers data from the Entities and constructs the OutputData as another plain old object. The OutputData is then passed through the

Database . Khi xong việc, UseCaseInteractor gom dữ liệu từ các Entities và dựng nên OutputData , cũng lại là một plain old object nữa. Sau đó OutputData được truyền qua interface

OutputBoundary interface to the Presenter .

OutputBoundary tới Presenter .

The job of the Presenter is to repackage the OutputData into viewable form as the ViewModel , which is yet another plain old object. The ViewModel contains mostly Strings and flags that the View uses to display the data. Whereas the OutputData may contain Date objects, the Presenter will load the ViewModel with corresponding Strings already formatted properly for the user. The same is true of Currency objects, or any other business-related data. Button and MenuItem names are placed in the ViewModel . So are flags that tell the View whether those

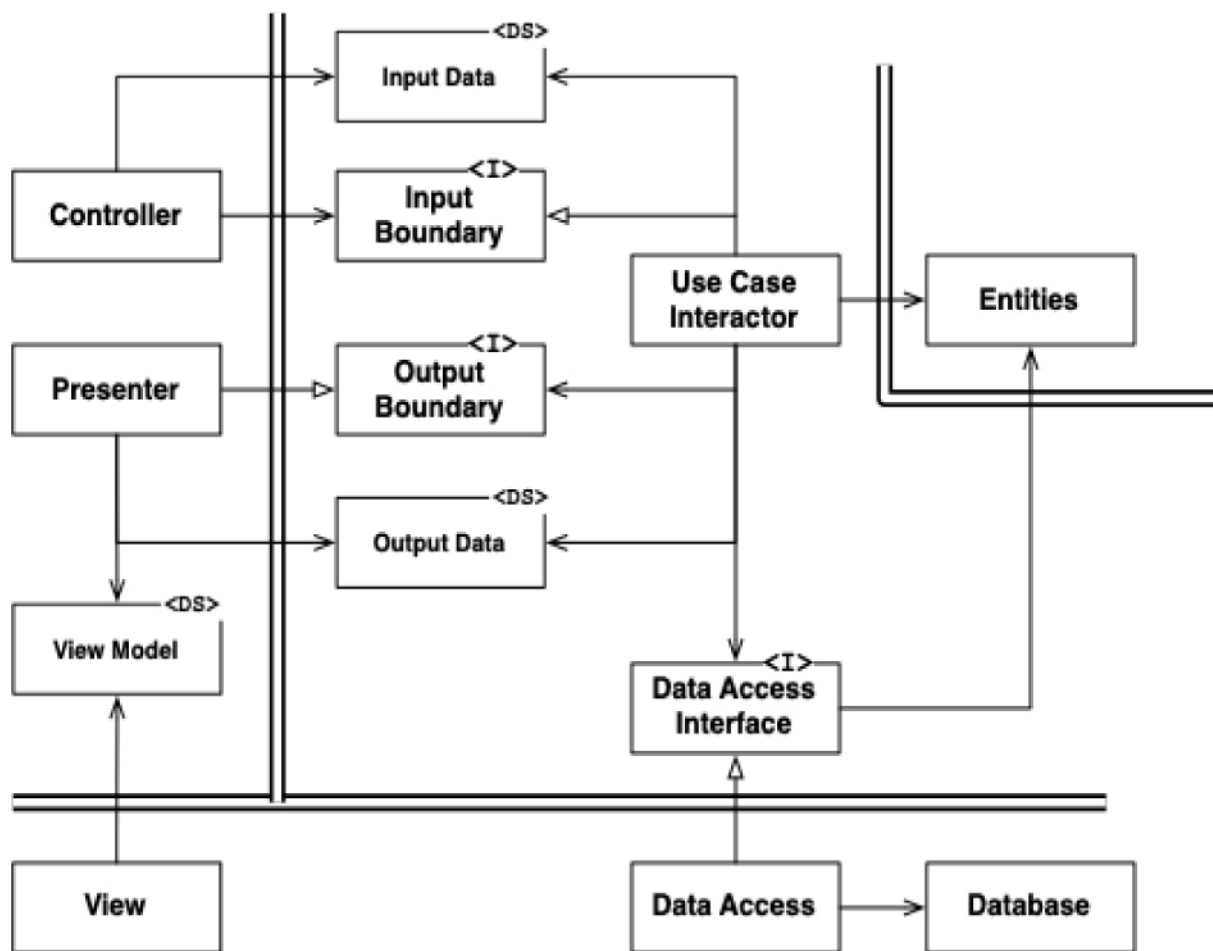
Việc của Presenter là đóng gói lại OutputData thành dạng hiển thị được, tức ViewModel — thêm một plain old object nữa. ViewModel chủ yếu chứa các String và các cờ (flag) mà View dùng để hiển thị dữ liệu. Trong khi OutputData có thể chứa các đối tượng Date , Presenter sẽ nạp vào ViewModel những String tương ứng đã được định dạng sẵn cho người dùng. Với các đối tượng Currency , hay bất kỳ dữ liệu nghiệp vụ nào khác, cũng vậy. Tên của Button và MenuItem được đặt vào ViewModel . Các cờ báo cho View biết những

Buttons and Menus should be gray.

Buttons và Menus đó có nên bị làm xám hay không cũng nằm ở đấy luôn.

This leaves the View with almost nothing to do other than move the data from the ViewModel into the HTML page.

Thành ra View gần như chẳng còn việc gì để làm, ngoài chuyện chuyển dữ liệu từ ViewModel vào trang HTML.



Note the directions of the dependencies. All dependencies cross the boundary lines pointing inward, following the Dependency Rule.

Hãy để ý hướng của các phụ thuộc. Mọi phụ thuộc đều băng qua các đường ranh giới và trở vào trong, đúng theo Quy tắc Phụ thuộc.

Conclusion — Kết luận

Conforming to these simple rules is not hard, and will save you a lot of headaches going forward. By separating the software into layers and conforming to the Dependency Rule, you will create a system that is intrinsically testable, with all the benefits that implies. When any of the external parts of the system become obsolete, like the database or the Web framework, you can replace those obsolete elements with a minimum of fuss.

Tuân theo mấy quy tắc đơn giản này chẳng khó khăn gì, mà lại giúp bạn đỡ đau đầu rất nhiều về sau. Bằng cách tách phần mềm thành các tầng và tuân thủ Quy tắc Phụ thuộc, bạn sẽ tạo ra một hệ thống tự thân đã kiểm thử được, kèm theo tất cả lợi ích mà điều đó mang lại. Khi bất kỳ phần bên ngoài nào của hệ thống trở nên lỗi thời — như cơ sở dữ liệu hay Web framework — bạn có thể thay thế những thành phần lỗi thời ấy mà chẳng phải nhọc công bao nhiêu.

Chapter 28. Harm — Chương 28. Tồn hại

1. I will not produce harmful code.

1. Tôi sẽ không tạo ra mã gây tổn hại (harm).

The first promise of the software **professional** is: DO NO **HARM**! And that means that your code must not harm your users, your employers, your managers, or your fellow programmers.

Lời hứa đầu tiên của một người làm phần mềm chuyên nghiệp là: KHÔNG GÂY TỔN HẠI!
Nghĩa là mã của bạn không được làm hại người dùng, chủ lao động, quản lý, hay đồng nghiệp lập trình viên của bạn.

You must know what your code does. You must know that it works. And you must know that it is clean.

Bạn phải biết mã của mình làm gì. Bạn phải biết nó chạy đúng. Và bạn phải biết nó sạch.

Sometime back, it was discovered that some programmers at Volkswagen wrote some code that purposely thwarted EPA emissions tests. Those programmers wrote harmful code.

Cách đây ít lâu, người ta phát hiện một số lập trình viên ở Volkswagen đã viết mã cố tình qua mặt các bài kiểm tra khí thải của EPA. Những lập trình viên đó đã viết mã gây tổn hại.

It was harmful because it was deceitful. That code fooled the EPA into allowing cars to be sold that emitted 20 times the amount of harmful nitrous oxides as the EPA deemed safe. Therefore, that code potentially harmed the health of everyone living where those cars were driven.

Nó gây tổn hại vì nó lừa dối. Đoạn mã ấy đánh lừa EPA để những chiếc xe thải ra lượng nitơ oxit độc hại gấp 20 lần ngưỡng EPA cho là an toàn vẫn được phép bán ra thị trường. Vì thế, đoạn mã đó có nguy cơ làm hại sức khỏe của mọi người sống ở nơi những chiếc xe ấy lăn bánh.

What should happen to those programmers? Did they know the purpose of that code? Should they have known?

Chuyện gì nên xảy đến với những lập trình viên đó? Họ có biết mục đích của đoạn mã ấy không? Họ có nên biết không?

I'd fire them and prosecute them. Because whether they knew or not, they should have known. Hiding behind requirements written by others is no excuse. It's our fingers on the keyboard. It's our code. We must know what it does!

Tôi sẽ sa thải và truy tố họ. Bởi vì dù họ có biết hay không, họ *đáng lẽ phải* biết. Núp sau những yêu cầu do người khác viết ra không phải là lý do bào chữa. Ngón tay đặt trên bàn

phím là ngón tay của chúng ta. Mã đó là mã của chúng ta. Chúng ta phải biết nó làm gì!

That's a tough one, isn't it? We write the code that makes our machines work. And those machines are often in positions to do tremendous harm. Since we will be held responsible for any harm our code does, we must be responsible for knowing what our code will do.

Khó nuốt nhĩ? Chúng ta viết ra đoạn mã khiến máy móc vận hành. Và những cỗ máy ấy thường ở vào vị thế có thể gây ra tổn hại khủng khiếp. Vì chúng ta sẽ phải chịu trách nhiệm cho mọi tổn hại mà mã của mình gây ra, chúng ta buộc phải có trách nhiệm biết mã của mình sẽ làm gì.

Each programmer should be held accountable based on their level of experience and responsibility. As you advance in experience and position, your responsibility for your actions and the actions of those under your charge increases.

Mỗi lập trình viên phải chịu trách nhiệm tương xứng với mức độ kinh nghiệm và chức trách của mình. Kinh nghiệm và vị trí càng cao thì trách nhiệm với hành động của bản thân — và với hành động của những người dưới quyền — càng lớn.

Clearly, we can't hold junior programmers as responsible as team leads. We can't hold team leads as responsible as senior developers. But those senior people ought to be held to a very high standard, and be ultimately responsible for those whom they direct.

Rõ ràng, ta không thể bắt một lập trình viên junior chịu trách nhiệm ngang với team lead. Ta cũng không thể bắt team lead chịu trách nhiệm ngang với senior developer. Nhưng những người senior ấy thì phải được đo bằng một chuẩn rất cao, và cuối cùng phải chịu trách nhiệm cho những người mà họ dẫn dắt.

That doesn't mean that all the blame goes to the senior developers or the managers. Every programmer is responsible for knowing what the code does to the level of their maturity and understanding. Every programmer is responsible for the harm their code does.

Điều đó không có nghĩa là mọi lời trách cứ đều đổ hết lên đầu các senior developer hay các quản lý. Mỗi lập trình viên đều có trách nhiệm biết mã mình viết làm gì, ở mức tương xứng với độ chín và hiểu biết của mình. Mỗi lập trình viên đều chịu trách nhiệm về tổn hại mà mã của mình gây ra.

No Harm to Society — Không gây tổn hại cho xã hội

First, you will do no harm to the society in which you live.

Trước hết, bạn sẽ không gây tổn hại cho xã hội mà bạn đang sống trong đó.

This is the rule that the VW programmers broke. Their software might have benefited their employer—Volkswagen. However, it harmed society in general. And we, programmers, must never do that.

Đây chính là quy tắc mà các lập trình viên của VW đã vi phạm. Phần mềm của họ có thể đã có lợi cho chủ lao động — Volkswagen. Nhưng nó làm hại xã hội nói chung. Và chúng ta, những lập trình viên, tuyệt đối không được làm điều đó.

But how do you know if you are harming society or not? For example, is building software that controls weapons systems harmful to society? What about gambling software? What about violent or sexist video games? What about pornography?

Nhưng làm sao bạn biết mình có đang làm hại xã hội hay không? Ví dụ, viết phần mềm điều khiển hệ thống vũ khí có phải là gây hại cho xã hội? Còn phần mềm cờ bạc thì sao? Còn game bạo lực hay kỳ thị giới tính? Còn phim khiêu dâm?

If your software is within the law, might it still be harmful to society?

Nếu phần mềm của bạn không phạm luật, liệu nó vẫn có thể gây hại cho xã hội không?

Frankly, that's a matter for your own judgment. You'll just have to make the best call you can. Your conscience is going to have to be your guide. Choose wisely!

Thành thật mà nói, đó là chuyện thuộc về phán đoán của chính bạn. Bạn chỉ có thể đưa ra lựa chọn tốt nhất trong khả năng của mình. Lương tâm bạn sẽ phải là kim chỉ nam. Hãy chọn cho khôn ngoan!

Another example of harm to society was Hhealthcare.gov, the website that was mandated by the Affordable Care Act. Although in this case the harm was unintentional.

Một ví dụ khác về tổn hại cho xã hội là Healthcare.gov, trang web được luật Affordable Care Act yêu cầu xây dựng. Dù trong trường hợp này, tổn hại là ngoài ý muốn.

The Affordable Care Act was a bill passed by the Congress of the United States and signed into law by the president. Among its many directives was the demand that a website be created and activated on October 1, 2013.

Affordable Care Act là một đạo luật được Quốc hội Hoa Kỳ thông qua và được tổng thống ký ban hành. Trong vô số chỉ thị của nó có yêu cầu phải tạo ra và đưa vào hoạt động một trang web đúng ngày 1 tháng 10 năm 2013.

Now, never mind the insanity of specifying, by law, a date by which a whole new massive software system must be activated. The real problem was that on October 1, 2013, they actually turned it on.

Thôi thì cứ bỏ qua cái sự điên rồ của việc dùng luật để ấn định ngày mà cả một hệ thống phần mềm đồ sộ hoàn toàn mới phải được kích hoạt. Vấn đề thật sự là vào ngày 1 tháng 10 năm 2013, họ đã bật nó lên thật.

Do you think, maybe, there were some programmers hiding under their desks that day?

Bạn có nghĩ rằng hôm đó có vài lập trình viên đang trốn dưới gầm bàn không?

Oh man, I think they turned it on.

Trời ơi, hình như họ bật lên thật rồi.

Yeah, yeah, they really shouldn't be doing that.

Ừ, ừ, đáng lẽ họ không nên làm thế.

Oh, my poor mother. Whatever will she do?

Ôi mẹ tôi tội nghiệp. Rồi bà ấy sẽ xoay xở thế nào đây?

I won't bore you with a laundry list of the problems that ensued. Suffice it to say that the website did not work—at all—for months. This is a case where a technical screwup put a huge new public policy at risk. The law was very nearly overturned because of these failures. And no matter what you think about the politics of the situation, that was harm to the society.

Tôi sẽ không làm bạn chán bằng cái danh sách dài dằng dặc những rắc rối kéo theo sau đó. Chỉ cần nói rằng trang web không chạy — không chạy chút nào — suốt nhiều tháng trời. Đây là trường hợp một cú hỏng việc kỹ thuật đặt cả một chính sách công mới mẻ và

khổng lồ vào vòng nguy hiểm. Đạo luật ấy suýt nữa đã bị lật đổ vì những thất bại này. Và dù bạn nghĩ gì về khía cạnh chính trị của câu chuyện, đó vẫn là tổn hại cho xã hội.

Who was responsible for that harm? Every programmer, team lead, manager, and director who knew that system wasn't ready and who nevertheless remained silent.

Ai chịu trách nhiệm cho tổn hại đó? Mọi lập trình viên, team lead, quản lý và giám đốc từng biết hệ thống ấy chưa sẵn sàng nhưng vẫn im lặng.

That harm upon society was perpetrated by every software developer who maintained a passive-aggressive attitude toward their management. Everyone who said: "I'm just doing my job—it's their problem."

Tổn hại cho xã hội đó do chính những người phát triển phần mềm giữ thái độ thụ động-hằn học với cấp quản lý của mình gây ra. Tất cả những ai đã nói: "Tôi chỉ làm việc của tôi thôi — đó là vấn đề của họ."

Every software developer who knew something was wrong, and yet did nothing to stop the deployment of that system, shares part of the blame.

Mọi người phát triển phần mềm biết có gì đó không ổn mà chẳng làm gì để chặn việc triển khai hệ thống ấy đều gánh một phần lỗi.

Because here's the thing. One of the prime reasons you were hired as a programmer was because you know when things are going wrong. You have the knowledge to identify trouble before it happens. And therefore, you have the responsibility to speak up before something terrible happens.

Bởi vì vấn đề là thế này. Một trong những lý do chính khiến bạn được thuê làm lập trình viên là vì bạn biết khi nào mọi thứ đang đi chệch hướng. Bạn có kiến thức để nhận ra rắc rối trước khi nó xảy ra. Và vì vậy, bạn có trách nhiệm lên tiếng trước khi điều tồi tệ ập đến.

Harm to Function — Tổn hại cho chức năng

You must KNOW that your code works. You must KNOW that the functioning of your code will not do harm to your company, your users, or your fellow programmers.

Bạn phải BIẾT rằng mã của mình chạy đúng. Bạn phải BIẾT rằng việc mã của bạn hoạt động sẽ không gây tổn hại cho công ty, cho người dùng, hay cho đồng nghiệp lập trình viên của bạn.

On August 1, 2012, some technicians at Knight Capital loaded their servers with new software. Unfortunately, they loaded only seven of the eight servers, leaving the eighth running with the older version.

Ngày 1 tháng 8 năm 2012, vài kỹ thuật viên ở Knight Capital nạp phần mềm mới lên các máy chủ của họ. Không may, họ chỉ nạp lên bảy trong tám máy chủ, để máy thứ tám chạy phiên bản cũ.

Why they made this mistake is anybody's guess. Somebody got sloppy.

Vì sao họ phạm sai lầm này thì chỉ có trời biết. Ai đó đã cẩu thả.

Knight Capital ran a trading system. They traded stocks on the NYSE. Part of their operation was to take large "parent" trades and break them up into many smaller "child" trades in order to prevent other traders from seeing the size of the initial parent trade and adjusting prices accordingly.

Knight Capital vận hành một hệ thống giao dịch. Họ giao dịch cổ phiếu trên sàn NYSE. Một phần trong hoạt động của họ là nhận các lệnh “parent” cỡ lớn rồi xé nhỏ thành nhiều lệnh “child” để những nhà giao dịch khác không nhìn thấy quy mô của lệnh parent ban đầu mà điều chỉnh giá theo.

Eight years earlier, a simple version of this parent–child algorithm, named Power-Peg, was disabled and replaced with something much better, called SMARS. Oddly, however, the old Power-Peg code was not removed from the system. It was simply disabled with a flag.

Tám năm trước đó, một phiên bản đơn giản của thuật toán parent–child này, tên là Power-Peg, đã bị vô hiệu hoá và thay bằng thứ tốt hơn nhiều, gọi là SMARS. Nhưng lạ thay, đoạn mã Power-Peg cũ lại không bị gỡ khỏi hệ thống. Nó chỉ đơn giản bị vô hiệu hoá bằng một cờ (flag).

That flag had been used to regulate the parent–child process. When the flag was on, child trades were made. When enough child trades had been made to satisfy the parent trade, the flag was turned off.

Cái cờ đó vốn được dùng để điều tiết quá trình parent–child. Khi cờ bật, các lệnh child được tạo ra. Khi đã có đủ lệnh child để khớp lệnh parent, cờ được tắt đi.

They disabled the Power-Peg code by simply leaving the flag off.

Họ vô hiệu hoá mã Power-Peg bằng cách đơn giản là cứ để cờ ở trạng thái tắt.

Unfortunately, the new software update that made it onto only seven of the eight servers repurposed that flag. They turned it on. And that eighth server started making child trades in a high-speed infinite loop.

Không may, bản cập nhật phần mềm mới — thứ chỉ lên được bảy trong tám máy chủ — đã tái sử dụng cái cờ đó cho mục đích khác. Họ bật nó lên. Và máy chủ thứ tám bắt đầu tạo ra các lệnh child trong một vòng lặp vô hạn tốc độ cao.

They knew something had gone wrong, but they didn’t know exactly what. It took them 45 minutes to shut down that errant server; 45 minutes while it was making bad trades in an infinite loop.

Họ biết có gì đó đã hỏng, nhưng không biết chính xác là gì. Họ mất 45 phút để tắt được cái máy chủ hư hỏng ấy; 45 phút trong lúc nó vẫn đang đặt những lệnh giao dịch tai hại theo vòng lặp vô hạn.

The bottom line is that, in those first 45 minutes of trading, Knight Capital had unintentionally purchased over \$7 billion worth of stock that it did not want, and had to sell it off at a \$460 million loss. Worse, they only had \$360M in cash. They were bankrupt.

Kết cục là, trong 45 phút giao dịch đầu tiên ấy, Knight Capital đã vô tình mua hơn 7 tỷ đô la cổ phiếu mà họ chẳng hề muốn, rồi phải bán tháo với khoản lỗ 460 triệu đô. Tệ hơn, họ chỉ có 360 triệu tiền mặt. Họ phá sản.

Forty-five minutes; one dumb mistake; \$460 million.

Bốn mươi lăm phút; một sai lầm ngớ ngẩn; 460 triệu đô la.

And what was that mistake? The mistake was that they did not KNOW what their system was going to do.

Và sai lầm đó là gì? Sai lầm là họ đã không BIẾT hệ thống của mình sắp làm gì.

At this point, you may be worried that I’m demanding that programmers have perfect knowledge about the **behavior** of their code. Of course, perfect knowledge is not achievable. There will always be a knowledge deficit of some kind.

Đến đây, có thể bạn lo rằng tôi đang đòi hỏi lập trình viên phải hiểu biết hoàn hảo về hành vi của mã mình viết. Dĩ nhiên, hiểu biết hoàn hảo là chuyện bất khả. Sẽ luôn có một khoảng hụt kiến thức nào đó.

The issue isn't the perfection of knowledge. Rather, the issue is to KNOW that there will be no harm.

Vấn đề không nằm ở sự hoàn hảo của kiến thức. Vấn đề là phải BIẾT rằng sẽ không có tổn hại nào xảy ra.

Those poor guys at Knight Capital had a knowledge deficit that was horribly harmful—and given what was at stake, it was one they should not have allowed to exist.

Mấy anh chàng tội nghiệp ở Knight Capital có một khoảng hụt kiến thức gây tổn hại kinh hoàng — và với những gì đang bị đặt cược, đó là khoảng hụt lẽ ra họ không được phép để tồn tại.

Or let's take the case of Toyota, and the software system that made cars accelerate uncontrollably.

Hoặc lấy trường hợp Toyota, với hệ thống phần mềm khiến xe tăng tốc mất kiểm soát.

As many as 89 people were killed by that software. And more were injured.

Có tới 89 người đã chết vì phần mềm đó. Và còn nhiều người khác bị thương.

Imagine that you are driving in a crowded downtown business district. Imagine that your car suddenly begins to accelerate, and your brakes stop working. Within seconds, you are rocketing through stop lights and crosswalks, with no way to stop.

Hãy tưởng tượng bạn đang lái xe giữa khu trung tâm thương mại đông đúc. Hãy tưởng tượng chiếc xe của bạn bỗng dưng tăng tốc, và phanh thì mất tác dụng. Chỉ trong vài giây, bạn lao như tên bắn qua các đèn đỏ và vạch sang đường, không cách nào dừng lại.

That's what investigators found that the Toyota software could do—and probably had done.

Đó chính là điều mà các nhà điều tra phát hiện phần mềm của Toyota có thể gây ra — và nhiều khả năng đã gây ra.

The software was killing people.

Phần mềm đó đang giết người.

The programmers who wrote that code did not KNOW that their code would not kill. Notice the double negative. They did not KNOW that their code would NOT kill. And they should have known that. They should have known that their code would NOT kill.

Những lập trình viên viết đoạn mã ấy đã không BIẾT rằng mã của họ sẽ không giết người. Để ý cái phủ định kép nhé. Họ đã không BIẾT rằng mã của họ sẽ KHÔNG giết người. Và lẽ ra họ phải biết điều đó. Lẽ ra họ phải biết rằng mã của họ sẽ KHÔNG giết người.

Now again, this is all about risk. When the stakes are high, you want to drive your knowledge as near to perfection as you can get it. If lives are at stake, you have to KNOW that your code won't kill anyone. If fortunes are at stake, you have to KNOW that your code won't lose them.

Một lần nữa, tất cả chuyện này là về rủi ro. Khi mức đặt cược cao, bạn phải đẩy hiểu biết của mình tới gần mức hoàn hảo nhất có thể. Nếu mạng người đang bị đặt cược, bạn phải BIẾT rằng mã của mình sẽ không giết ai. Nếu gia sản đang bị đặt cược, bạn phải BIẾT rằng mã của mình sẽ không làm chúng bốc hơi.

On the other hand, if you are writing a chat application or a simple shopping cart website, neither fortunes nor lives are at stake . . .

Mặt khác, nếu bạn đang viết một ứng dụng chat hay một trang web giỏ hàng đơn giản thì chẳng có gia sản hay mạng người nào bị đặt cược cả...

... or are they?

... hay là có?

What if someone using your chat application starts having a medical emergency and types: “HELP ME. CALL 911” on your app? What if your app malfunctions and drops that message?

Nếu ai đó đang dùng ứng dụng chat của bạn bỗng gặp tình huống y tế khẩn cấp và gõ vào app: “CỨU TÔI VỚI. GỌI 911” thì sao? Nếu app của bạn trục trặc và làm rơi mất tin nhắn đó thì sao?

What if your website leaks personal information to hackers who use it to steal identities?

Nếu website của bạn để lộ thông tin cá nhân cho hacker, rồi chúng dùng nó để đánh cắp danh tính thì sao?

And what if the poor functioning of your code drives customers away from your employer and to a competitor?

Và nếu mã chạy tệ khiến khách hàng bỏ chủ lao động của bạn mà chạy sang đối thủ thì sao?

The point is that it's easy to underestimate the harm that can be done by software. It's comforting to think that your software can't harm anyone, because it's not important enough.

Vấn đề là người ta rất dễ đánh giá thấp mức tổn hại mà phần mềm có thể gây ra. Thật dễ chịu khi nghĩ rằng phần mềm của mình chẳng thể làm hại ai, bởi nó có quan trọng gì đâu.

But you forget that software is very expensive to write, and at the very least, the money spent on its development is at stake, not to mention whatever the users put at stake by depending on it. The bottom line is that there's almost always more at stake than you think.

Nhưng bạn quên rằng viết phần mềm rất tốn kém, và chỉ ít thì số tiền đổ vào việc phát triển nó cũng đang bị đặt cược, chưa kể những gì người dùng đặt cược khi họ phụ thuộc vào nó. Tóm lại, hầu như lúc nào cũng có nhiều thứ bị đặt cược hơn bạn tưởng.

No Harm to Structure — Không gây tổn hại cho cấu trúc

You must not harm the **structure** of the code. You must keep the code clean and well organized.

Bạn không được làm hại cấu trúc của mã. Bạn phải giữ cho mã sạch sẽ và có tổ chức tốt.

Ask yourself why the programmers at Knight Capital did not KNOW that their code could be harmful.

Hãy tự hỏi vì sao các lập trình viên ở Knight Capital lại không BIẾT rằng mã của họ có thể gây tổn hại.

I think the answer is pretty obvious. They had forgotten that the Power-Peg software was still in the system. They had forgotten that the flag they repurposed would activate it. And they assumed that all servers would always have the same software running.

Tôi nghĩ câu trả lời khá hiển nhiên. Họ đã quên rằng phần mềm Power-Peg vẫn còn nằm trong hệ thống. Họ đã quên rằng cái cờ họ đem tái sử dụng sẽ kích hoạt nó. Và họ mặc định rằng mọi máy chủ sẽ luôn chạy cùng một phần mềm.

The reason they did not know their system was harmful is because of the harm they had done to the structure of the system by leaving that dead code in place.

Lý do khiến họ không biết hệ thống của mình có hại chính là vì tổn hại mà họ đã gây ra cho cấu trúc hệ thống khi để nguyên đoạn mã chết đó nằm lại.

And that's one of the big reasons why code structure and cleanliness are so important. The more tangled the structure, the more difficulty in knowing what the code will do. The bigger the **mess**, the more the uncertainty.

Và đó là một trong những lý do lớn khiến cấu trúc và sự sạch sẽ của mã lại quan trọng đến thế. Cấu trúc càng rối, việc biết mã sẽ làm gì càng khó. Mớ hỗn độn càng lớn, sự bất định càng nhiều.

Take the Toyota case, for example. Why didn't they know their software could kill people? Do you think the fact that they had thousands of global variables might have been a factor?

Lấy vụ Toyota làm ví dụ. Vì sao họ không biết phần mềm của mình có thể giết người? Bạn có nghĩ việc họ có hàng nghìn biến toàn cục là một yếu tố góp phần không?

Making a mess in the software undermines your ability to know what the software does, and therefore your ability to prevent harm.

Bày ra một mớ hỗn độn trong phần mềm sẽ bào mòn khả năng biết phần mềm làm gì của bạn, và do đó bào mòn cả khả năng ngăn ngừa tổn hại.

Messy software is harmful software.

Phần mềm bầy hầy là phần mềm gây tổn hại.

Now, some of you might **object** by saying that sometimes a quick-and-dirty patch is necessary to fix a nasty production bug.

Giờ thì hẳn có người sẽ phản đối rằng đôi khi cần một bản vá vội-và-bắn để sửa một con bug khó chịu trên production.

Sure. Of course. If you can fix a production crisis with a quick-and-dirty patch, then you should do it. No question.

Đúng vậy. Tất nhiên rồi. Nếu bạn có thể dập một đám cháy trên production bằng bản vá vội-và-bắn thì bạn nên làm. Không bàn cãi.

A stupid idea that works is not a stupid idea.

Một ý tưởng ngu ngốc mà lại hiệu quả thì không phải là ý tưởng ngu ngốc.

However, you can't leave that quick-and-dirty patch in place without causing harm. The longer that patch remains in the code, the more harm it can do.

Tuy nhiên, bạn không thể để nguyên bản vá vội-và-bắn ấy mà không gây tổn hại. Bản vá đó càng nằm lại lâu trong mã, nó càng có thể gây hại nhiều hơn.

Remember that the Knight Capital debacle would not have happened if the old Power-Peg code had been removed from the codebase. It was that old, defunct code that actually made the bad trades.

Hãy nhớ rằng thảm họa Knight Capital đã không xảy ra nếu đoạn mã Power-Peg cũ được gỡ khỏi codebase. Chính đoạn mã cũ, đã chết ấy mới là thứ đặt ra những lệnh giao dịch tai hại.

So, what do we mean by "Harm to structure?"

Vậy “tổn hại cho cấu trúc” nghĩa là gì?

Clearly, thousands of global variables is a structural flaw. So is dead code left in the codebase.

Rõ ràng, hàng nghìn biến toàn cục là một khiếm khuyết về cấu trúc. Mã chết bị bỏ lại trong codebase cũng vậy.

Structural harm is harm to the organization and content of the source code. It is anything that makes that source code hard to read, hard to understand, hard to change, or hard to reuse.

Tổn hại về cấu trúc là tổn hại gây ra cho cách tổ chức và nội dung của mã nguồn. Nó là bất cứ thứ gì khiến mã nguồn khó đọc, khó hiểu, khó thay đổi, hay khó tái sử dụng.

It is the responsibility of every professional software developer to know the disciplines and standards of **good** software structure. They should know how to **refactor**, how to write tests, how to recognize **bad code**, and how to decouple designs and create appropriate architectural boundaries. They should know and apply the principles of low- and high-level design. And it is the responsibility of every senior developer to make sure that younger developers learn these things, and satisfy them in the code that they write.

Mọi người phát triển phần mềm chuyên nghiệp đều có trách nhiệm nắm được các kỷ luật (discipline) và chuẩn mực của cấu trúc phần mềm tốt. Họ phải biết cách refactor, cách viết test, cách nhận ra mã tệ, cách tách rời (decoupling) các thiết kế và tạo ra những ranh giới kiến trúc (architectural boundary) phù hợp. Họ phải biết và áp dụng các nguyên tắc thiết kế ở cả mức thấp lẫn mức cao. Và mọi senior developer đều có trách nhiệm bảo đảm rằng các developer trẻ hơn học được những điều này, và thể hiện chúng trong mã họ viết ra.

Soft — Mềm

The first word in software is SOFT. Software is supposed to be SOFT. It's supposed to be easy to change. If we didn't want it to be easy to change, we'd have called it HARDware.

Từ đầu tiên trong chữ software là SOFT — mềm. Phần mềm lẽ ra phải MỀM. Nó lẽ ra phải dễ thay đổi. Nếu ta không muốn nó dễ thay đổi thì ta đã gọi nó là HARDware rồi.

It's important to remember why software exists at all. We invented it as a means to make the behavior of machines easy to change. To the extent that our software is hard to change, we have thwarted the very reason that software exists.

Điều quan trọng là phải nhớ vì sao phần mềm tồn tại ngay từ đầu. Chúng ta phát minh ra nó như một cách để làm cho hành vi của máy móc trở nên dễ thay đổi. Phần mềm của ta càng khó thay đổi bao nhiêu, ta càng phá hỏng chính lý do tồn tại của phần mềm bấy nhiêu.

Remember that software has two values. There's the value of its behavior; and then there's the value of its “softness.” Our customers and users expect us to be able to change that behavior easily and without high cost.

Hãy nhớ rằng phần mềm có hai giá trị. Có giá trị nằm ở hành vi của nó; và có giá trị nằm ở “độ mềm” của nó. Khách hàng và người dùng kỳ vọng chúng ta có thể thay đổi hành vi ấy một cách dễ dàng và không tốn kém.

Which of these two values is greater? Which value should we prioritize above the other? We can answer that question with a simple thought experiment.

Trong hai giá trị đó, cái nào lớn hơn? Ta nên ưu tiên giá trị nào? Ta có thể trả lời câu hỏi này bằng một thí nghiệm tưởng tượng đơn giản.

Imagine two programs: One works perfectly but is impossible to change; the other does nothing correctly but is easy to change.

Hãy hình dung hai chương trình: một chương trình chạy hoàn hảo nhưng không thể nào thay đổi được; chương trình kia chẳng làm đúng thứ gì cả nhưng lại rất dễ thay đổi.

Which is more valuable?

Cái nào giá trị hơn?

I hate to be the one to tell you this, but in case you hadn't noticed it, software requirements tend to change, and when they change, that first software will become useless— forever.

Tôi chẳng muốn là người báo tin này đâu, nhưng phòng khi bạn chưa để ý: yêu cầu phần mềm có xu hướng thay đổi, và khi chúng thay đổi, chương trình thứ nhất sẽ trở nên vô dụng — vĩnh viễn.

On the other hand, that second software can be made to work, because it's easy to change. It may take some time and money to get it to work initially, but after that, it'll continue to work forever with minimal effort.

Ngược lại, chương trình thứ hai có thể được làm cho chạy đúng, bởi vì nó dễ thay đổi. Có thể sẽ tốn chút thời gian và tiền bạc để nó chạy được lúc ban đầu, nhưng sau đó nó sẽ tiếp tục chạy tốt mãi mãi với công sức tối thiểu.

And therefore, it is the second of the two values that should be given priority in all but the most urgent of situations.

Và vì thế, chính giá trị thứ hai trong hai giá trị ấy mới là thứ nên được ưu tiên, trừ những tình huống khẩn cấp nhất.

And what do I mean by urgent? I mean a production disaster that's losing the company \$10 million per minute. That's urgent.

Khẩn cấp ở đây nghĩa là gì? Tôi muốn nói tới một thảm họa trên production đang khiến công ty mất 10 triệu đô mỗi phút. Cái đó mới là khẩn cấp.

I do not mean a software startup. A startup is not an urgent situation requiring you to create inflexible software. Indeed, the opposite is true. The one thing that is absolutely certain in a startup is that you are creating the wrong product.

Tôi không nói tới một startup phần mềm. Startup không phải là tình huống khẩn cấp buộc bạn phải tạo ra phần mềm cứng nhắc. Thật ra, điều ngược lại mới đúng. Thứ duy nhất chắc chắn tuyệt đối ở một startup là bạn đang tạo ra sai sản phẩm.

No product survives contact with the user. As soon as you start to put the product into users' hands, you will discover that the product you've built is wrong in a hundred different ways. And if you can't change it without making a mess, you're doomed.

Không sản phẩm nào sống sót sau lần chạm mặt đầu tiên với người dùng. Ngay khi bạn bắt đầu đưa sản phẩm tới tay người dùng, bạn sẽ phát hiện ra sản phẩm mình xây dựng sai theo cả trăm kiểu khác nhau. Và nếu bạn không thể thay đổi nó mà không bày ra một mớ hỗn độn, bạn tiêu đời.

Indeed, this is one of the biggest problems with software startups. They believe they are in an urgent situation requiring them to throw out all the rules and dash to the finish line, leaving a huge mess in

their wake.

Thật vậy, đây là một trong những vấn đề lớn nhất của các startup phần mềm. Họ tin rằng mình đang ở trong tình huống khẩn cấp, buộc phải vứt bỏ hết mọi luật lệ mà lao thẳng tới vạch đích, để lại phía sau một mớ hỗn độn khổng lồ.

Most of the time that huge mess starts to slow them down long before they do their first deployment. They'll go faster, and better, and survive a lot longer, if they keep the structure of the software from harm.

Phần lớn thời gian, cái mớ hỗn độn khổng lồ đó bắt đầu làm họ chậm lại từ rất lâu trước cả lần triển khai đầu tiên. Họ sẽ đi nhanh hơn, tốt hơn, và sống sót lâu hơn nhiều nếu giữ cho cấu trúc phần mềm khỏi bị tổn hại.

When it comes to software, it never pays to rush —Brian Marick

Với phần mềm, vội vàng không bao giờ có lợi. —Brian Marick

Tests — Test

If you follow a comprehensive testing **discipline**, like **TDD**, **test & commit** || **revert** (TCR), or small bundles, you will KNOW, to a high degree of certainty, that the code works.

Nếu bạn theo một kỷ luật kiểm thử toàn diện, như TDD, test & commit || revert (TCR), hay small bundles, bạn sẽ BIẾT, với độ chắc chắn rất cao, rằng mã của mình chạy đúng.

How can you prevent harm to the behavior of your code if you don't have the tests that demonstrate that it works?

Làm sao bạn ngăn được tổn hại cho hành vi của mã nếu bạn không có những test chứng minh rằng nó chạy đúng?

How can you prevent harm to the structure of your code if you don't have the tests that allow you to clean it?

Làm sao bạn ngăn được tổn hại cho cấu trúc của mã nếu bạn không có những test cho phép bạn dọn dẹp nó?

Is a testing discipline really a prerequisite to **professionalism**? Am I really suggesting that you can't be a professional software developer unless you practice such a discipline?

Kỷ luật kiểm thử có thực sự là điều kiện tiên quyết của tính chuyên nghiệp không? Tôi có thật sự đang ngụ ý rằng bạn không thể là một người phát triển phần mềm chuyên nghiệp trừ phi bạn thực hành một kỷ luật như vậy?

Yes, I think that's true. Or rather, it is becoming true. It is true for some of us; and with time it is becoming true for more and more of us. I think the time will come, and relatively soon, when the majority of programmers will agree that practicing a testing discipline is part of the minimum set of disciplines and behaviors that mark a professional developer.

Vâng, tôi nghĩ đúng là như thế. Hay đúng hơn, điều đó đang dần trở thành sự thật. Nó đã đúng với một số người trong chúng ta; và theo thời gian, nó đang đúng với ngày càng nhiều người hơn. Tôi nghĩ sẽ đến lúc — và tương đối sớm thôi — phần lớn lập trình viên sẽ đồng ý rằng thực hành một kỷ luật kiểm thử là một phần trong bộ kỷ luật và hành vi tối thiểu làm nên một developer chuyên nghiệp.

Why do I believe that?

Vì sao tôi tin như vậy?

Because, as I said earlier, we rule the world! We write the rules that make the whole world work.

Bởi vì, như tôi đã nói trước đó, chúng ta cai trị thế giới! Chúng ta viết ra những luật lệ khiến cả thế giới vận hành.

In our society, nothing gets bought or sold without software. Nearly all correspondence is through software. Nearly all documents are written with software. Laws don't get passed or enforced without it. There is virtually no activity of daily life that does not involve software.

Trong xã hội của chúng ta, chẳng có gì được mua hay bán mà không cần phần mềm. Gần như mọi thư từ đều đi qua phần mềm. Gần như mọi tài liệu đều được soạn bằng phần mềm. Luật pháp không được thông qua hay thực thi nếu thiếu nó. Hầu như không có hoạt động thường nhật nào mà không dính dáng tới phần mềm.

Without software, our society does not **function**. Software has become the most important **component** in the infrastructure of our civilization.

Không có phần mềm, xã hội của chúng ta ngừng vận hành. Phần mềm đã trở thành thành phần quan trọng nhất trong hạ tầng của nền văn minh chúng ta.

Society does not understand this yet. We programmers don't really understand it yet either. But the realization is dawning that the software we write is critical. The realization is dawning that too many lives and fortunes depend upon our software. And the realization is dawning that software is being written by people who do not profess a minimum level of discipline.

Xã hội vẫn chưa hiểu điều này. Bản thân giới lập trình viên chúng ta cũng chưa thật sự hiểu. Nhưng nhận thức ấy đang hé rạng: rằng phần mềm chúng ta viết ra là thứ trọng yếu. Nhận thức ấy đang hé rạng: rằng quá nhiều mạng người và gia sản đang phụ thuộc vào phần mềm của chúng ta. Và nhận thức ấy cũng đang hé rạng: rằng phần mềm đang được viết bởi những người không cam kết theo một mức kỷ luật tối thiểu nào.

So, yes, I think TDD, or some discipline very much like it, will eventually be considered a minimum standard behavior for professional software developers. I think our customers and our users will insist upon it.

Vậy nên, vâng, tôi cho rằng TDD, hoặc một kỷ luật nào đó rất giống nó, cuối cùng sẽ được xem là chuẩn mực hành vi tối thiểu của người phát triển phần mềm chuyên nghiệp. Tôi nghĩ khách hàng và người dùng của chúng ta sẽ đòi hỏi điều đó.

Chapter 29. No Defect in Behavior or Structure — Chương 29. Không để lại lỗi trong hành vi lẫn cấu trúc

2. The code that I produce will always be my best work. I will not knowingly allow code that is defective either in **behavior** or in **structure** to accumulate.

2. Mã tôi viết ra sẽ luôn là phần việc tốt nhất tôi có thể làm. Tôi sẽ không cố tình để mã bị lỗi — dù là lỗi về hành vi hay lỗi về cấu trúc — tích tụ lại.

Remember, Kent Beck once said: “First, make it work. Then, make it right.” Getting the program to work is just the first and easiest step. The second and harder step is to clean the code.

Hãy nhớ, Kent Beck từng nói: “Trước hết, làm cho nó chạy. Sau đó, làm cho nó đúng đắn.” Bắt chương trình chạy được chỉ là bước đầu tiên, và là bước dễ nhất. Bước thứ hai, khó hơn nhiều, là dọn dẹp mã.

Unfortunately, too many programmers think they are done once they get a program to work. Once it works, they move on to the next program, and then the next, and the next.

Đáng tiếc là quá nhiều lập trình viên nghĩ rằng mình đã xong việc ngay khi chương trình chạy được. Chạy được rồi là họ nhảy sang chương trình kế tiếp, rồi chương trình kế tiếp nữa, rồi kế tiếp nữa.

In their wake, they leave behind a history of tangled, unreadable code that slows down the whole development team.

Phía sau họ là cả một lịch sử mã rối như tơ vò, không ai đọc nổi, làm chậm chân cả đội phát triển.

They do this because they think their value is in speed. They know they are paid a lot; and so they feel that they must deliver a lot of functionality in a short amount of time.

Họ làm vậy vì họ nghĩ giá trị của mình nằm ở tốc độ. Họ biết mình được trả lương cao; nên họ cảm thấy mình phải giao thật nhiều chức năng trong thật ít thời gian.

But software is hard, and it takes a lot of time, and so they feel like they are going too slowly. They feel like they are failing. And this feeling of failure creates a pressure that causes them to try to go faster. It causes them to rush.

Nhưng phần mềm là chuyện khó, và nó tốn rất nhiều thời gian, nên họ cảm thấy mình đang đi quá chậm. Họ cảm thấy mình đang thất bại. Và chính cảm giác thất bại ấy tạo ra áp lực khiến họ cố đi nhanh hơn. Nó khiến họ hối hả.

And so they rush to get the program working; and then they declare themselves to be done—because, in their minds, it’s already taken them too long.

Thế là họ hối hả bắt chương trình chạy cho được; rồi tuyên bố mình đã xong — bởi trong đầu họ, chuyện này đã kéo dài quá lâu rồi.

The constant tapping of the project manager’s foot doesn’t help, but that’s not the real driver.

Tiếng chân gõ nhịp sốt ruột của quản lý dự án chẳng giúp được gì, nhưng đó không phải là động lực thật sự.

I teach lots of classes. In many of them, I give the programmers small projects to code. My goal is to give them a coding experience in which they can try out new techniques and new disciplines. I don’t care if they actually finish the project.

Tôi dạy rất nhiều lớp. Trong nhiều lớp trong số đó, tôi giao cho các lập trình viên những dự án nhỏ để viết mã. Mục tiêu của tôi là cho họ một trải nghiệm lập trình để thử nghiệm kỹ thuật mới và kỷ luật (discipline) mới. Tôi chẳng bận tâm họ có thực sự hoàn thành dự án hay không.

Indeed, all the code is going to be thrown away.

Thực tế là toàn bộ mã đó rồi cũng sẽ bị vứt đi.

And still I see people rushing. Some stay past the end of the **class** just hammering away at getting something utterly meaningless to work.

Vậy mà tôi vẫn thấy người ta hối hả. Có người ở lại cả sau khi lớp học kết thúc, cắm cúi gõ cho bằng được một thứ hoàn toàn vô nghĩa để nó chạy.

So, although the boss’s pressure doesn’t help, the real pressure comes from inside us. We consider speed of development to be a matter of our own self-worth.

Vậy nên, dù áp lực từ sếp chẳng giúp ích gì, áp lực thật sự lại đến từ bên trong chúng ta. Chúng ta xem tốc độ phát triển là thước đo giá trị bản thân.

Making It Right — Làm cho nó đúng đắn

As we saw in the last chapter, there are two values to software. There is the value of its behavior and the value of its structure. And in the last chapter, I also made the point that the value of structure is more important than the value of behavior. This is because, to have any long-term value, software systems must be able to react to changes in requirements.

Như chúng ta đã thấy ở chương trước, phần mềm có hai giá trị. Có giá trị của hành vi và giá trị của cấu trúc. Và cũng ở chương trước, tôi đã nêu quan điểm rằng giá trị của cấu trúc quan trọng hơn giá trị của hành vi. Lý do là: để có bất kỳ giá trị dài hạn nào, hệ thống phần mềm phải có khả năng phản ứng với những thay đổi trong yêu cầu.

Software that has a structure that makes it hard to change, makes it harder for that software to stay up-to-date with the requirements. Software with a bad structure can quickly get out-of-date.

Phần mềm có cấu trúc khiến nó khó thay đổi thì cũng khó bắt kịp các yêu cầu. Phần mềm với cấu trúc tồi có thể lỗi thời rất nhanh.

So, in order to keep up with requirements, the structure of the software has to be clean enough to allow, and even encourage, change. Software that is easy to change can keep up with changing requirements, allowing it to remain valuable with the least amount of effort. But if you’ve got a

software system that's hard to change, then you're going to have a devil of a time keeping that system working when the requirements change.

Vậy nên, để theo kịp yêu cầu, cấu trúc của phần mềm phải đủ sạch để cho phép — thậm chí khuyến khích — sự thay đổi. Phần mềm dễ thay đổi thì bắt kịp được yêu cầu đang đổi thay, nhờ đó giữ được giá trị với ít công sức nhất. Nhưng nếu bạn có một hệ thống phần mềm khó thay đổi, thì bạn sẽ khốn khổ đủ đường để giữ cho hệ thống ấy còn chạy được khi yêu cầu đổi thay.

When are requirements most likely to change? Requirements are most volatile at the start of a project; just after the users have seen the first few features work. That's because they're getting their first look at what the system actually does, as opposed to what they thought it was going to do.

Khi nào thì yêu cầu dễ thay đổi nhất? Yêu cầu biến động mạnh nhất ở giai đoạn đầu dự án; ngay sau khi người dùng thấy vài tính năng đầu tiên chạy được. Bởi đó là lúc họ lần đầu nhìn thấy hệ thống thực sự làm gì, khác với những gì họ đã tưởng nó sẽ làm.

This means that the structure of the system needs to be clean at the very start, if early development is to proceed quickly. If you make a **mess** at the start, then even the very first release will be slowed down by that mess.

Điều này có nghĩa là cấu trúc của hệ thống phải sạch ngay từ đầu, nếu muốn giai đoạn phát triển ban đầu diễn ra nhanh chóng. Nếu bạn tạo ra một mớ hỗn độn ngay từ đầu, thì đến cả bản phát hành đầu tiên cũng sẽ bị mớ hỗn độn ấy làm chậm lại.

Good structure enables good behavior and bad structure impedes good behavior. The better the structure, the better the behavior. The worse the structure, the worse the behavior. The value of the behavior depends critically on the structure. And therefore, the value of the structure is the more critical of the two values.

Cấu trúc tốt tạo điều kiện cho hành vi tốt, còn cấu trúc tồi cản trở hành vi tốt. Cấu trúc càng tốt, hành vi càng tốt. Cấu trúc càng tệ, hành vi càng tệ. Giá trị của hành vi phụ thuộc một cách sống còn vào cấu trúc. Và do đó, giá trị của cấu trúc là cái quan trọng hơn trong hai giá trị.

And that means that **professional** developers put a higher priority on the structure of the code than on the behavior.

Và điều đó có nghĩa là lập trình viên chuyên nghiệp đặt cấu trúc của mã ở mức ưu tiên cao hơn hành vi.

Yes, first you make it work; but then, you make very sure that you continue to make it right. You keep the system structure as clean as possible throughout the lifetime of the project. From its very beginning to its very end, it must be clean.

Đúng, trước hết bạn làm cho nó chạy; nhưng sau đó, bạn phải chắc chắn rằng mình tiếp tục làm cho nó đúng đắn. Bạn giữ cấu trúc hệ thống sạch nhất có thể trong suốt vòng đời dự án. Từ lúc khởi đầu cho tới lúc kết thúc, nó phải sạch.

What Is Good Structure? — Thế nào là cấu trúc tốt?

Good structure makes the system easy to **test**, easy to change, and easy to reuse. Changes to one part of the code do not break other parts of the code. Changes to one **module** do not force massive recompiles and redeployments. High-level policies are kept separate and independent from low-level details.

Cấu trúc tốt khiến hệ thống dễ test, dễ thay đổi và dễ tái sử dụng. Thay đổi ở một phần của mã không làm hỏng những phần khác. Thay đổi ở một module không buộc phải biên dịch lại và triển khai lại hàng loạt. Các chính sách ở mức cao được giữ tách biệt và độc lập với các chi tiết ở mức thấp.

Poor structure makes a system rigid, fragile, and immobile. These are the traditional design smells.

Cấu trúc kém khiến hệ thống trở nên cứng nhắc, dễ vỡ và bất động. Đó là những code smell (mùi mã) kinh điển về thiết kế.

Rigidity is when relatively minor changes to the system cause large portions of the system to be recompiled, rebuilt, and redeployed. A system is rigid when the effort to integrate a change is much greater than the change itself.

Cứng nhắc (rigidity) là khi những thay đổi tương đối nhỏ trong hệ thống lại buộc phần lớn hệ thống phải biên dịch lại, build lại và triển khai lại. Một hệ thống là cứng nhắc khi công sức để tích hợp một thay đổi lớn hơn nhiều so với bản thân thay đổi đó.

Fragility is when minor changes to the behavior of a system force many corresponding changes in a large number of modules. This creates a high risk that a small change in behavior will silently break some other behaviors in the system that then get inadvertently deployed. When this happens, managers and customers will come to believe that the team has lost control over the software and don't know what they are doing.

Dễ vỡ (fragility) là khi những thay đổi nhỏ về hành vi của hệ thống buộc phải sửa theo ở rất nhiều module. Điều này tạo ra rủi ro cao rằng một thay đổi nhỏ về hành vi sẽ âm thầm làm hỏng vài hành vi khác trong hệ thống, rồi những hỏng hóc đó vô tình được đem đi triển khai. Khi chuyện này xảy ra, quản lý và khách hàng sẽ đi đến kết luận rằng đội phát triển đã mất kiểm soát phần mềm và chẳng biết mình đang làm gì.

Immobility is when a module in an existing system contains a behavior you want to use in a new system but is so tangled up within the existing system that you can't extract it for use in the new system.

Bất động (immobility) là khi một module trong hệ thống hiện có chứa một hành vi mà bạn muốn dùng cho hệ thống mới, nhưng nó lại quấn chặt vào hệ thống hiện có đến mức bạn không thể bóc nó ra để dùng ở nơi khác.

These problems are all problems of structure and not behavior. The system may pass all its tests and meet all its functional requirements. And yet such a system may be close to worthless because it is too difficult to manipulate.

Tất cả những vấn đề này đều là vấn đề của cấu trúc chứ không phải của hành vi. Hệ thống vẫn có thể vượt qua mọi test và đáp ứng mọi yêu cầu chức năng. Vậy mà một hệ thống như thế vẫn có thể gần như vô giá trị, bởi nó quá khó để xoay sở.

There's a certain irony in the fact that so many systems that correctly implement valuable behaviors end up with a structure that is so poor that it negates that value and becomes worthless.

Có một nghịch lý nào đó trong việc quá nhiều hệ thống hiện thực đúng đắn những hành vi giá trị lại kết thúc với một cấu trúc tệ đến mức phủ định luôn giá trị ấy và trở thành vô giá trị.

And worthless is not too strong a word. In Chapter 1, I told you the tale of the grand redesign in the sky. This is when developers tell management that the only way to make progress is to redesign the whole system from scratch; those developers have assessed the current system to be worthless.

Và “vô giá trị” không phải là từ nói quá đâu. Ở Chương 1, tôi đã kể cho bạn nghe câu chuyện về cuộc đại thiết kế lại trên trời. Đó là khi các lập trình viên nói với ban quản lý rằng cách duy nhất để tiến lên là thiết kế lại toàn bộ hệ thống từ con số không; những lập trình viên ấy đã đánh giá hệ thống hiện tại là vô giá trị.

When managers agree to let the developers redesign the system, it simply means that they have agreed with the developers’ assessment that the current system is worthless.

Khi các nhà quản lý đồng ý để lập trình viên thiết kế lại hệ thống, điều đó đơn giản có nghĩa là họ đã đồng tình với đánh giá của lập trình viên rằng hệ thống hiện tại là vô giá trị.

What is it that causes these design smells that lead to worthless systems? Source code and **data** dependencies! And how do we fix those dependencies? **Dependency** management!

Cái gì gây ra những mùi thiết kế dẫn tới các hệ thống vô giá trị ấy? Chính là các phụ thuộc (dependency) trong mã nguồn và trong dữ liệu! Và ta sửa những phụ thuộc đó bằng cách nào? Quản lý phụ thuộc!

And how do we manage dependencies? We use principles of design, like the SOLID 1 principles, to keep the structure of our systems free of the design smells that would make it worthless.

Và ta quản lý phụ thuộc bằng cách nào? Ta dùng các nguyên tắc thiết kế, chẳng hạn các nguyên tắc SOLID 1, để giữ cho cấu trúc hệ thống của mình sạch mùi thiết kế — thứ mùi sẽ biến nó thành vô giá trị.

1 These principles are described in [Clean Arch] and [PPP02].

1 Các nguyên tắc này được mô tả trong [Clean Arch] và [PPP02].

Since the value of structure is greater than the value of behavior, and since the value of structure depends upon good dependency management, and since good dependency management derives from good principles, it follows that the overall value of the system depends upon proper application of those principles.

Vì giá trị của cấu trúc lớn hơn giá trị của hành vi, và vì giá trị của cấu trúc phụ thuộc vào việc quản lý phụ thuộc tốt, và vì quản lý phụ thuộc tốt bắt nguồn từ những nguyên tắc tốt, nên suy ra giá trị tổng thể của hệ thống phụ thuộc vào việc áp dụng đúng đắn những nguyên tắc ấy.

Eisenhower’s Matrix — Ma trận Eisenhower

General Dwight D. Eisenhower once said: “I have two kinds of problems, the urgent and the important. The urgent are not important, and the important are never urgent.”

Đại tướng Dwight D. Eisenhower từng nói: “Tôi có hai loại vấn đề: loại khẩn cấp và loại quan trọng. Việc khẩn cấp thì không quan trọng, còn việc quan trọng thì chẳng bao giờ khẩn cấp.”

There is a deep truth to this statement. A deep truth about engineering. We might even call this “the engineer’s motto”:

Có một sự thật sâu sắc trong câu nói này. Một sự thật sâu sắc về kỹ thuật. Ta thậm chí có thể gọi đây là “châm ngôn của người kỹ sư”:

The greater the urgency, the less the relevance.

Càng khẩn cấp thì càng ít liên quan.

Here is Eisenhower's decision matrix. Urgency is on the **vertical** axis, and importance is on the horizontal axis. The four possibilities are urgent and important, urgent and unimportant, important but not urgent, and neither important nor urgent.

Đây là ma trận ra quyết định của Eisenhower. Tính khẩn cấp nằm trên trục dọc, còn tầm quan trọng nằm trên trục ngang. Bốn khả năng là: khẩn cấp và quan trọng, khẩn cấp nhưng không quan trọng, quan trọng nhưng không khẩn cấp, và chẳng quan trọng cũng chẳng khẩn cấp.

IMPORTANT URGENT	IMPORTANT NOT URGENT	
UNIMPORTANT URGENT	UNIMPORTANT NOT URGENT	

Urgency is about time. Importance is not. Things that are important are long term. Things that are urgent are short term. Structure is long term; therefore, it is important. Behavior is short term; therefore, it is merely urgent.

Tính khẩn cấp là chuyện thời gian. Tầm quan trọng thì không. Những thứ quan trọng là chuyện dài hạn. Những thứ khẩn cấp là chuyện ngắn hạn. Cấu trúc là chuyện dài hạn; do đó, nó quan trọng. Hành vi là chuyện ngắn hạn; do đó, nó chỉ đơn thuần khẩn cấp.

So structure, the important stuff, comes first. Behavior is secondary.

Vậy nên cấu trúc, thứ quan trọng, phải được đặt lên trước. Hành vi chỉ là thứ yếu.

Your boss might not agree with that priority; but that's because it's not your boss's job to worry about structure. It's yours. Your boss simply expects that you will keep the structure clean while you implement the urgent behaviors.

Sếp của bạn có thể không đồng tình với thứ tự ưu tiên đó; nhưng đó là vì lo cho cấu trúc không phải việc của sếp. Đó là việc của bạn. Sếp bạn chỉ đơn giản kỳ vọng rằng bạn sẽ giữ cấu trúc sạch sẽ trong lúc hiện thực (implementation) những hành vi khẩn cấp kia.

This seems to violate Beck's law: "First, make it work. Then, make it right." Now I'm saying that structure is higher priority than behavior. It's a chicken-and-egg dilemma, isn't it?

Chuyện này có vẻ vi phạm định luật Beck: "Trước hết, làm cho nó chạy. Sau đó, làm cho nó đúng đắn." Giờ tôi lại bảo cấu trúc có mức ưu tiên cao hơn hành vi. Đúng là bài toán con gà và quả trứng, phải không?

The reason we make it work first is because structure has to support behavior; so we implement behavior first, then we give it the right structure. But structure is more important than behavior. We give it a higher priority. We deal with structural issues before we deal with behavioral issues.

Lý do ta làm cho nó chạy trước là vì cấu trúc phải nâng đỡ hành vi; nên ta hiện thực hành vi trước, rồi mới trao cho nó cấu trúc đúng đắn. Nhưng cấu trúc quan trọng hơn hành vi. Ta cho nó mức ưu tiên cao hơn. Ta xử lý các vấn đề về cấu trúc trước khi xử lý các vấn đề về hành vi.

We can square this circle by breaking the problem down into tiny units. Let's start with user stories. You get a story to work; and then you get its structure right. And you don't work on the next story until that structure is right. The structure of the current story is higher priority than the behavior of the next story.

Ta có thể hoá giải nghịch lý này bằng cách bẻ nhỏ bài toán thành những đơn vị tí hon. Hãy bắt đầu với user story. Bạn làm cho một story chạy được; rồi bạn chỉnh cho cấu trúc của nó đúng đắn. Và bạn không đụng tới story kế tiếp cho tới khi cấu trúc ấy đã ổn. Cấu trúc của story hiện tại có mức ưu tiên cao hơn hành vi của story kế tiếp.

Except stories are too big. We need to go smaller. Not stories . . . tests. Tests are the perfect size.

Có điều story vẫn còn quá lớn. Ta cần nhỏ hơn nữa. Không phải story... mà là test. Test có kích thước hoàn hảo.

First you get a test to pass, then you fix the structure of the code that passes that test, before you get the next test to pass.

Trước hết bạn làm cho một test pass, rồi bạn sửa cấu trúc của đoạn mã làm test đó pass, trước khi bạn làm cho test kế tiếp pass.

This entire discussion has been the moral foundation for the **Red-Green-Refactor** cycle of a good testing **discipline** like test-driven development (**TDD**).

Toàn bộ cuộc thảo luận này chính là nền tảng đạo lý cho chu trình Red-Green-Refactor của một kỷ luật kiểm thử tốt như test-driven development (TDD).

It is that cycle that helps us prevent **harm** to behavior and harm to structure. It is that cycle that allows us to prioritize structure over behavior. And that's why we consider our testing discipline to be a design technique as opposed to a testing technique.

Chính chu trình ấy giúp ta ngăn tổn hại (harm) cho hành vi và tổn hại cho cấu trúc. Chính chu trình ấy cho phép ta ưu tiên cấu trúc hơn hành vi. Và đó là lý do ta xem kỷ luật kiểm thử của mình là một kỹ thuật thiết kế, chứ không phải một kỹ thuật kiểm thử.

Programmers Are Stakeholders — Lập trình viên cũng là bên liên quan

Remember this. We have a stake in the success of the software. We, programmers, are stakeholders too.

Hãy nhớ điều này. Chúng ta có phần lợi ích trong thành công của phần mềm. Chúng ta, những lập trình viên, cũng là bên liên quan (stakeholder).

Did you ever think about it that way? Did you ever view yourself as one of the stakeholders of the project?

Bạn đã bao giờ nghĩ theo hướng đó chưa? Bạn đã bao giờ xem mình là một trong những bên liên quan của dự án chưa?

But, of course, you are. The success of the project has a direct impact upon your career and reputation. So, yes, you are a **stakeholder**.

Nhưng dĩ nhiên bạn là vậy. Thành công của dự án tác động trực tiếp lên sự nghiệp và danh tiếng của bạn. Nên đúng thế, bạn là một bên liên quan.

And as a stakeholder, you have a say in the way the system is developed and structured. I mean, it's your butt on the line too.

Và với tư cách bên liên quan, bạn có tiếng nói trong cách hệ thống được phát triển và cấu trúc. Ý tôi là, cái ghế của bạn cũng đang đặt cược ở đó mà.

But you are more than just a stakeholder. You are an engineer. You were hired because you know how to build software systems and how to structure those systems so that they last.

Nhưng bạn còn hơn cả một bên liên quan. Bạn là một kỹ sư. Người ta thuê bạn vì bạn biết cách xây dựng hệ thống phần mềm và biết cách cấu trúc những hệ thống ấy sao cho chúng sống lâu.

And with that knowledge comes the responsibility to produce the best product you can.

Và đi kèm với hiểu biết ấy là trách nhiệm tạo ra sản phẩm tốt nhất mà bạn có thể.

So, not only do you have the right as a stakeholder, but you have the duty as an engineer to make sure that the systems you produce do no harm either through bad behavior or through bad structure.

Vậy nên, bạn không chỉ có quyền với tư cách bên liên quan, mà còn có bốn phận với tư cách kỹ sư: bảo đảm rằng những hệ thống bạn tạo ra không gây tổn hại, dù là qua hành vi tồi hay qua cấu trúc tồi.

A lot of programmers don't want that kind of responsibility. They'd rather just be told what to do. And that's a travesty and a shame. It's wholly unprofessional. Programmers who feel that way should be paid minimum wage, because that's what their work output is worth.

Rất nhiều lập trình viên không muốn nhận thứ trách nhiệm đó. Họ thà được bảo phải làm gì thì hơn. Và đó là một trò lố và một điều đáng hổ thẹn. Nó hoàn toàn không chuyên nghiệp. Những lập trình viên nghĩ như vậy nên được trả lương tối thiểu, vì sản phẩm đầu ra của họ chỉ đáng chừng đó.

If you don't take responsibility for the structure of the system, who else will? Your boss?

Nếu bạn không nhận trách nhiệm về cấu trúc của hệ thống, thì còn ai nhận đây? Sếp bạn à?

Does your boss know **the SOLID principles**? How about design patterns? How about **object-oriented design**, and the practice of dependency inversion? Does your boss know the discipline of TDD? Do they know what a self-shunt is, or a test-specific subclass, or a humble object? Does your boss understand that things that change together should be grouped together and that things that change for different reasons should be separated?

Sếp bạn có biết các nguyên tắc SOLID không? Còn design pattern thì sao? Còn thiết kế hướng đối tượng, và thực hành đảo ngược phụ thuộc (dependency inversion)? Sếp bạn có nắm kỹ luật TDD không? Họ có biết self-shunt là gì không, hay test-specific subclass, hay humble object? Sếp bạn có hiểu rằng những thứ thay đổi cùng nhau thì nên được gom lại với nhau, và những thứ thay đổi vì các lý do khác nhau thì nên được tách ra không?

Does your boss understand structure? Or is your boss's understanding limited to behavior?

Sếp bạn có hiểu về cấu trúc không? Hay hiểu biết của sếp chỉ dừng lại ở hành vi?

Structure matters. If you aren't going to care for it, who will? If not you, then who?

Cấu trúc là chuyện hệ trọng. Nếu bạn không định chăm lo cho nó, thì ai sẽ lo? Nếu không phải bạn, thì là ai?

What if your boss specifically tells you to ignore structure and focus entirely on behavior? You refuse. You are a stakeholder. You have rights too. And you are an engineer; you have responsibilities that your boss cannot override.

Sẽ ra sao nếu sếp nói thẳng với bạn rằng hãy phớt lờ cấu trúc và tập trung hoàn toàn vào hành vi? Bạn từ chối. Bạn là một bên liên quan. Bạn cũng có quyền của mình. Và bạn là một kỹ sư; bạn có những trách nhiệm mà sếp bạn không thể phủ quyết.

Perhaps you think that will get you fired. Probably not. Most managers expect to have to fight for things they need and believe in. And they respect those who are willing to do the same.

Có lẽ bạn nghĩ làm thế thì sẽ bị đuổi việc. Chắc là không đâu. Phần lớn các nhà quản lý đều mặc định rằng họ phải đấu tranh cho những gì họ cần và tin tưởng. Và họ tôn trọng những người sẵn sàng làm điều tương tự.

Oh, there will be a struggle; even a confrontation. And it won't be comfortable. But you are a stakeholder and an engineer. You can't just back down and acquiesce. That's not professional.

Ồ, sẽ có giằng co; thậm chí có đối đầu. Và sẽ chẳng dễ chịu gì. Nhưng bạn là bên liên quan và là kỹ sư. Bạn không thể cứ thế lùi bước và xuôi theo. Như vậy là không chuyên nghiệp.

Most programmers do not enjoy confrontation. But dealing with confrontational managers is a skill we have to learn. We have to learn how to fight for what we know is right. Because taking responsibility for the things that matter, and fighting for those things, is how a professional behaves.

Đa số lập trình viên chẳng thích thú gì chuyện đối đầu. Nhưng ứng xử với những nhà quản lý ưa đối đầu là một kỹ năng ta buộc phải học. Ta phải học cách chiến đấu cho điều mà ta biết là đúng. Bối nhận trách nhiệm về những thứ hệ trọng, và chiến đấu vì những thứ đó, chính là cách một người chuyên nghiệp hành xử.

Do Your Best — Hãy làm hết sức mình

This promise of The Oath is about doing your best.

Lời hứa này của Bản Tuyên thệ nói về việc làm hết sức mình.

Clearly, this is a perfectly reasonable promise for a programmer to make. Of course you are going to do your best, and of course you will not knowingly release code that is harmful.

Rõ ràng đây là một lời hứa hoàn toàn hợp lý để một lập trình viên đưa ra. Dĩ nhiên bạn sẽ làm hết sức mình, và dĩ nhiên bạn sẽ không cố tình phát hành thứ mã gây hại.

And of course this is not black and white. There are times when structure must bend to schedule. For example, if, in order to make a trade show, you have to put in some quick-and-dirty fix, then so be it.

Và dĩ nhiên chuyện này không phải trắng đen rạch ròi. Có những lúc cấu trúc phải nhượng bộ tiến độ. Chẳng hạn, nếu để kịp một hội chợ triển lãm mà bạn buộc phải chấp vá một bản sửa nhanh và cẩu thả, thì đành vậy.

The promise doesn't even prevent you from shipping code to customers with less-than-perfect structure. If the structure is close but not quite right, and customers are expecting the release tomorrow, then so be it.

Lời hứa này thậm chí không cấm bạn giao cho khách hàng thứ mã có cấu trúc chưa hoàn hảo. Nếu cấu trúc đã gần đạt nhưng chưa thật chuẩn, mà khách hàng thì đang chờ bản phát hành vào ngày mai, thì đành vậy.

On the other hand, the promise does mean that you will address those issues of behavior and structure before you add more behavior. You will not pile more and more behavior on top of known bad structure. You will not allow those defects to accumulate.

Mặt khác, lời hứa này có nghĩa là bạn sẽ xử lý những vấn đề về hành vi và cấu trúc ấy trước khi bạn thêm hành vi mới. Bạn sẽ không chồng chất thêm hành vi này đến hành vi khác lên trên một cấu trúc mà bạn biết là tồi. Bạn sẽ không để những khiếm khuyết đó tích tụ lại.

What if your boss tells you to anyway? Here's how that conversation should go.

Sẽ ra sao nếu sếp cứ bảo bạn làm vậy? Đây là cách cuộc trò chuyện đó nên diễn ra.

Boss: I want this new feature added by tonight.

Sếp: Tôi muốn tính năng mới này xong trong tối nay.

Programmer: I'm sorry, but I can't do that; I've got some structural cleanup to do before I can add that new feature.

Lập trình viên: Tôi xin lỗi, nhưng tôi không làm được; tôi còn phải dọn dẹp vài chỗ về cấu trúc trước khi có thể thêm tính năng mới đó.

Boss: Do the cleanup tomorrow. Get the feature done by tonight.

Sếp: Mai hãy dọn. Tối nay làm xong tính năng đi.

Programmer: That's what I did for the last feature, and now I have an even bigger mess to clean up. I really have to finish that cleanup before I start on anything new.

Lập trình viên: Đó đúng là điều tôi đã làm ở tính năng lần trước, và giờ tôi có một mớ hỗn độn còn lớn hơn phải dọn. Tôi thật sự phải dọn xong chỗ đó trước khi bắt tay vào bất cứ việc gì mới.

Boss: I don't think you understand. This is business. Either we have a business or we don't have a business. And if we can't get features done, we don't have a business. Now get the feature done.

Sếp: Tôi nghĩ cậu chưa hiểu. Đây là kinh doanh. Hoặc là ta có một doanh nghiệp, hoặc là ta chẳng có gì cả. Và nếu ta không làm xong tính năng, ta chẳng có doanh nghiệp nào hết. Giờ thì làm cho xong tính năng đi.

Programmer: I understand. Really, I do. And I agree. We have to be able to get features done. But if I don't clean up the structural problems that have accumulated over the last few days, we're going to slow down and get even fewer features done.

Lập trình viên: Tôi hiểu. Thật đấy, tôi hiểu. Và tôi đồng ý. Chúng ta phải làm xong được tính năng. Nhưng nếu tôi không dọn những vấn đề cấu trúc đã tích tụ suốt mấy ngày qua, chúng ta sẽ chậm lại và còn làm xong được ít tính năng hơn nữa.

Boss: Y'know, I used to like you. I used to say, that Danny, he's pretty nice. But now I don't think so. You're not being nice at all. Maybe you shouldn't be working with me. Maybe I should fire you.

Sếp: Cậu biết không, tôi từng quý cậu đấy. Tôi từng bảo, thằng Danny ấy, nó cũng được. Nhưng giờ thì tôi không nghĩ vậy nữa. Cậu chẳng dễ chịu chút nào. Có lẽ cậu không nên làm việc với tôi. Có lẽ tôi nên đuổi việc cậu.

Programmer: Well, sir, that's your right. But I'm pretty sure you want features done quickly and done right. And I'm telling you that if I don't do this cleanup tonight, then we're going to start slowing down. And we'll deliver fewer and fewer features.

Lập trình viên: Vâng thưa sếp, đó là quyền của sếp. Nhưng tôi khá chắc rằng sếp muốn tính năng được làm xong nhanh và làm cho đúng. Và tôi đang nói với sếp rằng nếu tối nay tôi không dọn dẹp chỗ này, thì chúng ta sẽ bắt đầu chậm lại. Và chúng ta sẽ giao được ngày càng ít tính năng hơn.

Look, sir, I want to go fast, just like you do. You hired me because I know how to do that. You have to let me do my job. You have to let me do what I know is best.

Sếp này, tôi cũng muốn đi nhanh, y như sếp vậy. Sếp thuê tôi vì tôi biết cách làm điều đó. Sếp phải để tôi làm công việc của mình. Sếp phải để tôi làm điều mà tôi biết là tốt nhất.

Boss: You really think everything will slow down if you don't do this cleanup tonight?

Sếp: Cậu thật sự nghĩ mọi thứ sẽ chậm lại nếu tối nay cậu không dọn dẹp à?

Programmer: I know it will. I've seen it before. And so have you.

Lập trình viên: Tôi biết chắc là vậy. Tôi từng chứng kiến rồi. Và sếp cũng vậy.

Boss: And it has to be tonight?

Sếp: Và nhất định phải là tối nay?

Programmer: I don't feel safe letting the mess get any worse.

Lập trình viên: Tôi thấy không yên tâm khi để mớ hỗn độn này tệ thêm nữa.

Boss: You can give me the feature tomorrow?

Sếp: Ngày mai cậu đưa tôi tính năng đó được chứ?

Programmer: Yes, and it'll be a lot easier to do once the structure is cleaned up.

Lập trình viên: Được, và sẽ dễ làm hơn nhiều một khi cấu trúc đã được dọn sạch.

Boss: OK. Tomorrow. No later. Now get to it.

Sếp: Được. Ngày mai. Không muộn hơn. Giờ bắt tay vào đi.

Programmer: OK. I'll get right on it.

Lập trình viên: Vâng. Tôi làm ngay đây.

Boss: (aside) I like that kid. He's got guts. He's got gumption. He didn't back down even when I threatened to fire him. He's gonna go far, trust me. But don't tell him I said so.

Sếp: (nói riêng) Mình thích thằng nhóc đó. Nó có gan. Nó có chí. Nó không lùi bước ngay cả khi mình dọa đuổi việc. Thằng này rồi sẽ tiến xa, tin mình đi. Nhưng đừng nói với nó là mình bảo vậy đấy.

Chapter 30. Repeatable Proof —

Chương 30. Bằng chứng lặp lại được

3. I will produce, with each release, a quick, sure, and repeatable demonstration that every element of the code works as it should.

3. Với mỗi lần phát hành, tôi sẽ đưa ra một minh chứng nhanh, chắc chắn và lặp lại được rằng mọi thành phần của mã đều chạy đúng như nó phải chạy.

Does that sound unreasonable to you? Does it sound unreasonable to be expected to prove that the code you've written actually works? Allow me to introduce you to Edsger Wybe Dijkstra.

Nghe có vô lý không? Chuyện bị đòi hỏi phải chứng minh rằng đoạn mã mình viết ra thực sự chạy được thì có gì vô lý? Cho phép tôi giới thiệu với bạn một người: Edsger Wybe Dijkstra.

Dijkstra — Dijkstra

Edsger Wybe Dijkstra was born in Rotterdam in 1930. He survived the bombing of Rotterdam and the German occupation of the Netherlands, and in 1948, he graduated high school with the highest possible marks in math, physics, chemistry, and biology.

Edsger Wybe Dijkstra sinh năm 1930 tại Rotterdam. Ông sống sót qua trận ném bom Rotterdam và thời kỳ Đức chiếm đóng Hà Lan, rồi năm 1948 tốt nghiệp trung học với điểm cao nhất có thể ở cả bốn môn toán, lý, hoá và sinh.

In March of 1952, at the age of 21, and just nine months before I would be born, he took a job with the Mathematical Center of Amsterdam as the Netherlands' very first programmer.

Tháng 3 năm 1952, ở tuổi 21, và đúng chín tháng trước khi tôi ra đời, ông nhận việc tại Trung tâm Toán học Amsterdam với tư cách lập trình viên đầu tiên của Hà Lan.

In 1957, he married Maria Debets. In the Netherlands at the time, you had to state your profession as part of the marriage rites. The authorities were unwilling to accept “programmer” as his profession. They'd never heard of such a profession. So he settled for “theoretical physicist.” In 1955, having been a programmer for three years, and while still a student, he concluded that the intellectual challenge of programming was greater than the intellectual challenge of theoretical physics, and as a result chose programming as his long-term career.

Năm 1957, ông cưới Maria Debets. Ở Hà Lan thời đó, khi làm thủ tục kết hôn bạn phải khai nghề nghiệp của mình. Nhà chức trách không chịu chấp nhận “lập trình viên” là một nghề. Họ chưa từng nghe nói tới cái nghề đó bao giờ. Thế là ông đành khai “nhà vật lý lý thuyết”. Năm 1955, sau ba năm làm lập trình viên và khi vẫn còn là sinh viên, ông đi tới

kết luận rằng thách thức trí tuệ của việc lập trình còn lớn hơn thách thức trí tuệ của vật lý lý thuyết, và vì thế đã chọn lập trình làm sự nghiệp lâu dài.

In making this decision, he conferred with his boss, Adriaan van Wijngaarden. Dijkstra was concerned that no one had identified a **discipline** or science of programming, and that he would therefore not be taken seriously. His boss replied that Dijkstra might very well be one of the people who would make it a science.

Khi đưa ra quyết định này, ông có bàn với sếp của mình, Adriaan van Wijngaarden. Dijkstra lo rằng chưa ai xác lập được một kỷ luật (discipline) hay một ngành khoa học nào cho việc lập trình, và do đó ông sẽ không được coi trọng. Sếp của ông đáp rằng rất có thể chính Dijkstra sẽ là một trong những người biến nó thành khoa học.

In pursuit of that goal, Dijkstra was compelled by the idea that software was a formal system, a kind of mathematics. He reasoned that software could become a mathematical **structure** rather like Euclid's Elements—a system of postulates, proofs, theorems, and lemmas. And so he set about to create the language and discipline of software proofs.

Theo đuổi mục tiêu đó, Dijkstra bị cuốn hút bởi ý tưởng rằng phần mềm là một hệ hình thức, một dạng toán học. Ông lập luận rằng phần mềm có thể trở thành một cấu trúc toán học kiểu như bộ *Elements* của Euclid — một hệ thống gồm các tiên đề, chứng minh, định lý và bổ đề. Và thế là ông bắt tay vào xây dựng ngôn ngữ cùng kỷ luật cho việc chứng minh phần mềm.

Proving Correctness — Chứng minh tính đúng đắn

Dijkstra realized that there were only three techniques we could use to prove the correctness of an algorithm: enumeration, induction, and **abstraction**. Enumeration is used to prove that two statements in sequence, or two statements selected by a boolean expression, are correct. Induction is used to prove that a loop is correct. Abstraction is used to break groups of statements into smaller provable chunks.

Dijkstra nhận ra rằng chỉ có ba kỹ thuật để chứng minh tính đúng đắn của một thuật toán: liệt kê (enumeration), quy nạp (induction) và trừu tượng hoá (abstraction). Liệt kê dùng để chứng minh rằng hai câu lệnh nối tiếp nhau, hoặc hai câu lệnh được chọn bởi một biểu thức boolean, là đúng. Quy nạp dùng để chứng minh một vòng lặp là đúng. Trừu tượng hoá dùng để chia các nhóm câu lệnh thành những mẫu nhỏ hơn có thể chứng minh được.

If this sound hard, it is. As an example of just how hard this is, I have included a simple Java program for calculating the remainder of an integer, and my handwritten proof of that algorithm below it. 1
1 This is a translation into Java of a demonstration from Dijkstra's work.

Nếu nghe có vẻ khó thì đúng là nó khó thật. Để bạn thấy nó khó tới mức nào, tôi đưa vào đây một chương trình Java đơn giản tính phần dư của một số nguyên, và bên dưới là phần chứng minh viết tay của tôi cho thuật toán đó. 1 1 Đây là bản chuyển sang Java của một minh hoạ trong công trình của Dijkstra.


```

public static int remainder(int numerator, int denominator) {
    assert(numerator >= 0 && denominator > 0);
    int r = numerator;
    int dd = denominator;
    while (dd <= r)
        dd *= 2;
    while (dd != denominator) {
        dd /= 2;
        if (dd <= r)
            r -= dd;
    }
    return r;
}

```

Don't worry. I don't expect you to read my proof. I just want you to get a feel for what such a proof entails.

Đừng lo. Tôi không mong bạn đọc hết phần chứng minh của tôi. Tôi chỉ muốn bạn cảm nhận được một chứng minh như vậy đòi hỏi những gì.

Expand:

Given $2^x D > N \mid x=0$
 Then $dd = D$
 And while is not entered $\Rightarrow dd = D \times 2^0$

$2^x D > N$

$D > N$
 or
 $2^x D > N > 2^{x-1} D$

Given $2^x D > N \geq 2^{x-1} D \mid x=1$
 Then $dd = D$
 The while is entered: $N \geq dd$
 $dd = 2^1 D$
 The while exits: $2^1 D > N \Rightarrow dd = 2^1 D$

Assume $dd \Rightarrow 2^x \mid 2^x D > N \geq 2^{x-1} D \mid x > 0$

If $2^{x+1} D > N \geq 2^x D$
 Then after the x^{th} loop $dd = 2^x D$ (assumed)
 The while is entered: $dd \leq N$
 $dd = 2^{x+1} D$
 The while exits: $dd > N$

Single Reduction

Given $dd = 2^{x+2} D \mid x \geq 0$
 $0 \leq r < 2^{x+2} D$

$dd \geq 2^x D$

If $dd > r$ Then $0 \leq r < 2^x D \quad r = r - qD \quad q \text{ int} = 0$

If $dd \leq r$

$r = r - dd$ and $0 \leq r < 2^{x+1} D - 2^x D$
 $0 \leq r < 2^x D \quad r = r - qD \quad q \text{ int} > 0$

Reduce:

Given $dd = D \Rightarrow D > N$ by Expand
 $r = N \Rightarrow r \mid r = N - qD \quad q = 0$
 $0 \leq r < D \quad r = N$

Given $dd = 2^1 D \Rightarrow 2D > N \geq D$ by Expand

The loop is entered
 $\Rightarrow 0 \leq r < D \quad r = N - qD \quad q \text{ int}$

Assume $dd = 2^x D \Rightarrow 2^x D > N \geq 2^{x-1} D \mid x > 0$
 $\Rightarrow dd = 2^{x+1} D$
 $0 \leq r < 2^{x+1} D \quad r = N - qD \quad q \geq 0 \text{ int}$

If $dd = 2^{x+2} D \Rightarrow 2^{x+1} D > N \geq 2^x D \mid x > 0$
 The loop is entered: $x > 0$
 $\Rightarrow dd = 2^{x+1} D$
 $0 \leq r < 2^x D \quad r = N - qD \quad q \text{ int}$

Expand:

Given $2^x D > N \mid x=0$
 Then $dd = D$
 And while is not entered $\Rightarrow dd = D \times 2^0$

$2^x D > N$

$D > N$
 or
 $2^x D > N > 2^{x-1} D$

Given $2^x D > N \geq 2^{x-1} D \mid x=1$
 Then $dd = D$
 The while is entered: $N \geq dd$
 $dd = 2^1 D$
 The while exits: $2^1 D > N \Rightarrow dd = 2^1 D$

Assume $dd \Rightarrow 2^x \mid 2^x D > N \geq 2^{x-1} D \mid x > 0$

If $2^{x+1} D > N \geq 2^x D$
 Then after the x^{th} loop $dd = 2^x D$ (assumed)
 The while is entered: $dd \leq N$
 $dd = 2^{x+1} D$
 The while exits: $dd > N$

Single Reduction

Given $dd = 2^{x+1} D \mid x \geq 0$
 $0 \leq r < 2^{x+1} D$

$dd \geq 2^x D$
 If $dd > r$ Then $0 \leq r < 2^x D \quad r = r - qD \quad q \text{ int} = 0$

If $dd \leq r$
 $r = r - dd$ and $0 \leq r < 2^x D - 2^x D$
 $0 \leq r < 2^x D \quad r = r - qD \quad q \text{ int} > 0$

Reduce:

Given $dd = D \Rightarrow D > N$ by Expand
 $r = N \Rightarrow r \mid r = N - qD \quad q = 0$
 $0 \leq r < D \quad r = N$

Given $dd = 2^1 D \Rightarrow 2D > N \geq D$ by Expand

The loop is entered
 $\Rightarrow 0 \leq r < D \quad r = N - qD \quad q \text{ int}$

Assume $dd = 2^x D \Rightarrow 2^x D > N \geq 2^{x-1} D \mid x > 0$
 $\Rightarrow dd = 2^{x-1} D$
 $0 \leq r < 2^{x-1} D \quad r = N - qD \quad q \geq 0 \text{ int}$

If $dd = 2^{x+1} D \Rightarrow 2^{x+1} D > N \geq 2^x D \mid x > 0$
 The loop is entered: $x > 0$
 $\Rightarrow dd = 2^x D$
 $0 \leq r < 2^x D \quad r = N - qD \quad q \text{ int}$

I think you can see the problem with this approach. Indeed, this is something that Dijkstra complained bitterly about:

Tôi nghĩ bạn đã thấy vấn đề của cách tiếp cận này. Quả thật, đây cũng chính là điều Dijkstra than phiền cay đắng:

"Of course I would not dare to suggest (at least at present!) that it is the programmer's duty to supply such a proof whenever he writes a simple loop in his program. If so, he could never write a program of any size at all." 2

“Dĩ nhiên tôi không dám đề xuất (ít nhất là ở thời điểm hiện tại!) rằng lập trình viên có nghĩa vụ đưa ra một chứng minh như thế mỗi khi anh ta viết một vòng lặp đơn giản trong chương trình của mình. Nếu vậy, anh ta sẽ chẳng bao giờ viết nổi một chương trình có kích cỡ đáng kể nào.” 2

2 [SP72].

2 [SP72].

Dijkstra’s hope was that such proofs would become more practical through the creation of a library of theorems, again similar to Euclid’s *Elements*.

Hy vọng của Dijkstra là những chứng minh như thế sẽ trở nên khả thi hơn nhờ việc xây dựng một thư viện định lý, cũng lại giống bộ *Elements* của Euclid.

But Dijkstra did not understand just how prevalent and pervasive software would become. He did not foresee in those early days that computers would outnumber people and that vast quantities of software would be running in the walls of our homes, in our pockets, and on our wrists. Had he known, he would have realized that the library of theorems he envisioned would be far too vast for any mere human to grasp.

Nhưng Dijkstra đã không hình dung nổi phần mềm sẽ trở nên phổ biến và len lỏi đến mức nào. Vào những ngày đầu ấy, ông không lường trước được rằng máy tính sẽ nhiều hơn cả con người, và rằng những khối lượng phần mềm khổng lồ sẽ chạy trong tường nhà ta, trong túi quần ta và trên cổ tay ta. Nếu biết trước, hẳn ông đã nhận ra rằng thư viện định lý mà ông hình dung sẽ đồ sộ tới mức không một con người phạm trần nào nắm nổi.

And so Dijkstra’s dream of explicit mathematical proofs for programs has faded into near oblivion. Oh, there are some holdouts who hope against hope for a resurgence of formal proofs, but their vision has not penetrated very far into the software industry at large.

Và thế là giấc mơ của Dijkstra về những chứng minh toán học tường minh cho chương trình đã phai nhạt đến gần như bị lãng quên. Ô, vẫn còn vài người kiên trì bám trụ, ôm hy vọng mong manh về một cuộc phục hưng của chứng minh hình thức, nhưng tầm nhìn của họ chưa thấm được bao xa vào ngành công nghiệp phần mềm nói chung.

But while the dream may have passed, it drew something deeply profound in its wake. Something that we use today, almost without thinking about it.

Nhưng dù giấc mơ ấy có qua đi, nó vẫn kéo theo sau một thứ vô cùng sâu sắc. Một thứ mà ngày nay chúng ta dùng gần như chẳng cần nghĩ ngợi gì.

Structured Programming — Lập trình có cấu trúc

In the early days of programming, the 1950s and 1960s, we used languages like Fortran. Have you ever seen Fortran? Here, let me show you what it was like.

Vào những ngày đầu của nghề lập trình, thập niên 1950 và 1960, chúng tôi dùng những ngôn ngữ như Fortran. Bạn đã bao giờ thấy Fortran chưa? Đây, để tôi cho bạn xem nó trông ra sao.

```
99      WRITE(4,99)
        FORMAT("  NUMERATOR: ")

        READ(4,100)NN
        WRITE(4,98)
```



```

98      FORMAT ( " DENOMINATOR: " )
      READ ( 4 , 100 ) ND
100     FORMAT ( I6 )
      NR=NN
      NDD=ND
1       IF ( NDD-NR ) 2 , 2 , 3
2       NDD=NDD*2
      GOTO 1
3       IF ( NDD-ND ) 4 , 10 , 4
4       NDD=NDD/2
      IF ( NDD-NR ) 5 , 5 , 6
5       NR=NR-NDD
6       GOTO 3
10      WRITE ( 4 , 20 ) NR
20      FORMAT ( " REMAINDER: " , I6 )
      END

```

This little Fortran program implements the same remainder algorithm as the earlier Java program.

Chương trình Fortran nho nhỏ này hiện thực đúng cái thuật toán tính phần dư như chương trình Java ở trên.

Now, I'd like to draw your attention to those GOTO statements. You probably haven't seen statements like that very often.

Bây giờ, tôi muốn bạn để ý tới những câu lệnh GOTO kia. Chắc bạn cũng không thấy loại câu lệnh đó nhiều lắm.

The reason you haven't seen statements like that very often is that nowadays we look upon them with disfavor. In fact, most modern languages don't even have GOTO statements like that anymore.

Lý do bạn ít thấy loại câu lệnh đó là vì ngày nay chúng ta nhìn chúng bằng con mắt không mấy thiện cảm. Thực tế, phần lớn các ngôn ngữ hiện đại thậm chí còn chẳng còn câu lệnh GOTO kiểu ấy nữa.

Why don't we favor GOTO statements? Why don't our languages support them anymore? Because in 1968, Edsger Dijkstra wrote an article for the Communications of the ACM . The editor, Niklaus Wirth, thought it was so profound that he bypassed the normal review process and published it as a letter to the editor in the March issue. He gave it the title “ Go To Statement Considered Harmful .”

Vì sao chúng ta không ưa câu lệnh GOTO? Vì sao các ngôn ngữ của chúng ta không còn hỗ trợ chúng nữa? Bởi vì năm 1968, Edsger Dijkstra viết một bài báo cho *Communications of the ACM*. Vị biên tập viên, Niklaus Wirth, thấy bài viết sâu sắc đến mức đã bỏ qua quy trình bình duyệt thông thường và đăng nó dưới dạng thư gửi toà soạn trong số tháng 3. Ông đặt cho nó cái tựa: “*Go To Statement Considered Harmful*.”

Why did Dijkstra consider the GOTO statement to be harmful? It all comes back to the three strategies for proving a **function** correct: enumeration, induction, and abstraction.

Vì sao Dijkstra coi câu lệnh GOTO là có hại? Tất cả lại quay về ba chiến lược chứng minh một hàm là đúng: liệt kê, quy nạp và trừu tượng hoá.

Enumeration depends upon the fact that each statement in sequence can be analyzed independently, and that the result of one statement feeds into the next.

Liệt kê dựa trên việc mỗi câu lệnh trong một dãy tuần tự có thể được phân tích độc lập, và kết quả của câu lệnh này chảy vào câu lệnh kế tiếp.

It should be clear to you that in order for enumeration to be an effective technique for proving the correctness of a function, every statement that is enumerated must have a single entry point and a single exit point. Otherwise, we could not be sure of either the inputs or the outputs of a statement.

Hẳn bạn thấy rõ rằng để liệt kê trở thành một kỹ thuật hữu hiệu cho việc chứng minh tính đúng đắn của một hàm, mỗi câu lệnh được liệt kê phải có đúng một điểm vào và đúng một điểm ra. Nếu không, ta chẳng thể chắc chắn về đầu vào lẫn đầu ra của câu lệnh đó.

What's more, induction is simply a special form of enumeration, where we assume the enumerated statement is true for some x , and then prove by enumeration that it is true for $x + 1$.

Hơn nữa, quy nạp chỉ đơn giản là một dạng đặc biệt của liệt kê, ở đó ta giả định câu lệnh được liệt kê là đúng với một x nào đó, rồi chứng minh bằng liệt kê rằng nó cũng đúng với $x + 1$.

Thus, the body of a loop must be enumerable. It must have a single entry and a single exit.

Vì vậy, thân của một vòng lặp phải liệt kê được. Nó phải có đúng một điểm vào và đúng một điểm ra.

GOTO is considered harmful because a GOTO statement can jump into or out of the middle of an enumerated sequence.

GOTO bị xem là có hại bởi vì một câu lệnh GOTO có thể nhảy vào hoặc nhảy ra khỏi giữa một dãy đang được liệt kê.

GOTOs make enumeration intractable, making it impractical to prove an algorithm correct by enumeration or induction. So, in order to keep code provable, Dijkstra recommended that it be constructed out of three standard building blocks.

GOTO làm cho việc liệt kê trở nên bất khả thi, khiến ta không thể chứng minh một thuật toán là đúng bằng liệt kê hay quy nạp. Vậy nên, để giữ cho mã còn chứng minh được, Dijkstra khuyến nghị nó phải được dựng nên từ ba khối xây dựng chuẩn mực.

- Sequence: Depicted as two or more statements ordered in time. This represents nonbranching lines of code.
- Selection: Depicted as two or more statements selected by a predicate. This represents if/else and
 - Tuần tự (Sequence): thể hiện bằng hai câu lệnh trở lên xếp theo thứ tự thời gian. Đây là những dòng mã không rẽ nhánh.
 - Chọn lựa (Selection): thể hiện bằng hai câu lệnh trở lên được chọn bởi một vị từ. Đây là các câu lệnh if/else và

switch/case statements.

switch/case.

- Iteration: Depicted as a statement repeated under the control of a predicate. This represents a while or for loop.
 - Lặp (Iteration): thể hiện bằng một câu lệnh được lặp lại dưới sự điều khiển của một vị từ. Đây là vòng lặp while hoặc for.

Dijkstra knew 3 that any program, no matter how complicated, can be composed of nothing more than these three structures, and that programs structured in that manner are provable.

Dijkstra biết 3 rằng mọi chương trình, dù phức tạp đến đâu, cũng chỉ cần dựng nên từ ba cấu trúc này mà thôi, và rằng những chương trình được cấu trúc theo cách đó là chứng

minh được.

3 Because of the work of Böhm and Jacopini. C. Böhm and G. Jacopini, “Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules,” *Communications of the ACM* 9, no. 5 (May 1966): 366–371.

3 Nhờ công trình của Böhm và Jacopini. C. Böhm và G. Jacopini, “Flow Diagrams, Turing Machines and Languages with Only Two Formation Rules,” *Communications of the ACM* 9, số 5 (tháng 5 năm 1966): 366–371.

He called the technique **structured programming**.

Ông gọi kỹ thuật này là *lập trình có cấu trúc (structured programming)*.

Why is this important if we aren’t going to write those proofs? If something is provable, it means you can reason about it. If something is unprovable, it means you cannot reason about it.

Điều này quan trọng ở chỗ nào, khi mà chúng ta có định viết những chứng minh đó đâu? Nếu một thứ chứng minh được, nghĩa là bạn có thể lập luận về nó. Nếu một thứ không chứng minh được, nghĩa là bạn không thể lập luận về nó.

And if you can’t reason about it, you can’t properly **test** it.

Và nếu bạn không thể lập luận về nó, bạn cũng không thể test nó cho tử tế.

Functional Decomposition — Phân rã theo chức năng

In 1968, Dijkstra’s ideas were not immediately popular. Most of us were using languages that depended upon GOTO . So the idea of abandoning GOTO or imposing discipline on GOTO was abhorrent.

Năm 1968, ý tưởng của Dijkstra không được ưa chuộng ngay. Phần lớn chúng tôi khi đó dùng những ngôn ngữ phụ thuộc vào GOTO. Nên cái ý tưởng vứt bỏ GOTO, hay áp kỷ luật lên GOTO, nghe thật đáng ghê tởm.

The debate over Dijkstra’s ideas raged for several years. We didn’t have an internet in those days, so we didn’t use Facebook memes or flame wars. But we did write letters to the editors of the major software journals of the day.

Cuộc tranh cãi quanh các ý tưởng của Dijkstra kéo dài dử dội suốt vài năm. Ngày ấy chúng tôi chưa có internet, nên chẳng có meme Facebook hay những cuộc khẩu chiến trực tuyến. Nhưng chúng tôi viết thư gửi toà soạn của các tạp chí phần mềm lớn thời đó.

And those letters raged. Some claimed Dijkstra to be a god. Others claimed him to be a fool. Just like social media today; except slower.

Và những lá thư ấy nẩy lửa. Có người tôn Dijkstra là thần thánh. Có người bảo ông là kẻ khờ. Y hết mạng xã hội ngày nay; chỉ là chậm hơn.

But in time, the debate slowed, and Dijkstra’s position gained more and more support. Until nowadays, most of the languages we use simply don’t have a GOTO .

Nhưng theo thời gian, cuộc tranh cãi lắng xuống, và lập trường của Dijkstra ngày càng được ủng hộ. Cho tới ngày nay, phần lớn các ngôn ngữ chúng ta dùng đơn giản là không có GOTO nữa.

Nowadays, we are all structured programmers, because our languages don’t give us a choice.

Ngày nay, tất cả chúng ta đều là lập trình viên có cấu trúc, bởi vì ngôn ngữ chẳng cho ta lựa chọn nào khác.

We all build our programs out of sequence, selection, and iteration. And very few of us make regular use of unconstrained GOTO statements.

Tất cả chúng ta đều dựng chương trình của mình từ tuần tự, chọn lựa và lặp. Và rất ít người trong chúng ta còn thường xuyên dùng những câu lệnh GOTO không bị ràng buộc.

An unintended **side effect** of composing programs from those three structures was a technique called functional decomposition .

Một tác dụng phụ (side effect) ngoài ý muốn của việc dựng chương trình từ ba cấu trúc ấy là một kỹ thuật mang tên *phân rã theo chức năng* (*functional decomposition*).

Functional decomposition is the process whereby you start at the top level of your program and recursively break it down into smaller and smaller provable units. Indeed, this is the reasoning process behind structured programming. Structured programmers reason from the top down through this recursive decomposition into smaller and smaller provable functions.

Phân rã theo chức năng là quá trình bạn bắt đầu từ tầng cao nhất của chương trình rồi đệ quy chẻ nhỏ nó thành những đơn vị ngày càng nhỏ và chứng minh được. Quả thực, đây chính là lối lập luận nằm sau lập trình có cấu trúc. Người lập trình có cấu trúc suy luận từ trên xuống, qua quá trình phân rã đệ quy thành những hàm ngày càng nhỏ và chứng minh được.

This connection between structured programming and functional decomposition was the basis for the structured revolution that took place in the '70s and '80s.

Mối liên hệ giữa lập trình có cấu trúc và phân rã theo chức năng chính là nền tảng cho cuộc cách mạng cấu trúc diễn ra suốt thập niên 70 và 80.

People like Ed Yourdon, Larry Constantine, Tom DeMarco, and Meilir Page-Jones popularized the techniques of structured analysis and structured design during those decades.

Những người như Ed Yourdon, Larry Constantine, Tom DeMarco và Meilir Page-Jones đã phổ biến các kỹ thuật phân tích có cấu trúc và thiết kế có cấu trúc trong hai thập kỷ đó.

Test-Driven Development et al. — Test-Driven Development và những thứ tương tự

Test-driven development (TDD), the red → green → **refactor** cycle, is functional decomposition. After all, you have to write tests against small bits of the problem. That means that you must functionally decompose the problem into testable elements.

Test-driven development (TDD), tức chu trình red → green → refactor, chính là phân rã theo chức năng. Suy cho cùng, bạn phải viết test cho từng mẫu nhỏ của bài toán. Nghĩa là bạn buộc phải phân rã bài toán theo chức năng thành những thành phần test được.

That means that every system built with TDD is built from functionally decomposed elements that conform to structured programming. And that means that the system they compose is provable.

Nghĩa là mọi hệ thống được dựng bằng TDD đều được dựng từ những thành phần đã phân rã theo chức năng, tuân theo lập trình có cấu trúc. Và nghĩa là hệ thống chúng hợp thành là chứng minh được.

And the tests are the proof. Or, rather, the tests are the theory .

Và các test chính là chứng minh. Hay chính xác hơn, các test chính là *lý thuyết*.

The tests created by TDD are not a formal mathematical proof, like Dijkstra wanted. In fact, Dijkstra is famous for saying that tests can only prove a program wrong; they can never prove a program right.

Những test do TDD tạo ra không phải là chứng minh toán học hình thức như Dijkstra mong muốn. Thực tế, Dijkstra nổi tiếng với câu nói rằng test chỉ có thể chứng minh một chương trình là sai; chúng không bao giờ chứng minh được một chương trình là đúng.

This is where Dijkstra missed it, in my opinion. Dijkstra thought of software as a kind of mathematics. He wanted us to build up a superstructure of postulates, theorems, corollaries, and lemmas.

Theo tôi, đây chính là chỗ Dijkstra đã nhìn nhầm. Dijkstra nghĩ về phần mềm như một dạng toán học. Ông muốn chúng ta dựng lên một thượng tầng gồm tiên đề, định lý, hệ quả và bổ đề.

Instead, what we have realized is that software is a kind of science. We validate that science with experiments. We build up a superstructure of theories based upon passing tests. Just like all other sciences do.

Thay vào đó, điều chúng ta nhận ra là phần mềm thực chất là một dạng khoa học. Chúng ta kiểm chứng khoa học đó bằng thực nghiệm. Chúng ta dựng lên một thượng tầng gồm các lý thuyết dựa trên những test đang chạy xanh. Y như mọi ngành khoa học khác vẫn làm.

Have we proven the theory of evolution, or the theory of relativity, or the Big Bang theory, or any of the major theories of science? No. We can't prove them in any mathematical sense.

Chúng ta đã chứng minh được thuyết tiến hoá, hay thuyết tương đối, hay thuyết Big Bang, hay bất kỳ lý thuyết lớn nào của khoa học chưa? Chưa. Ta không thể chứng minh chúng theo bất kỳ nghĩa toán học nào.

But we believe them, within limits, nonetheless. Indeed, every time you get into a car or an airplane, you are betting your life that Newton's laws of motion are correct. Every time you use a GPS you are betting that Einstein's theory of relativity is correct.

Nhưng dù vậy ta vẫn tin chúng, trong một giới hạn nhất định. Quả thực, mỗi lần bước lên ô tô hay máy bay, bạn đang đặt cược mạng sống của mình vào chuyện các định luật chuyển động của Newton là đúng. Mỗi lần dùng GPS, bạn đang đặt cược rằng thuyết tương đối của Einstein là đúng.

The fact that we have not mathematically proven these theories correct does not mean that we don't have sufficient proof to depend upon them, even with our lives.

Việc chúng ta chưa chứng minh được các lý thuyết ấy là đúng về mặt toán học không có nghĩa là ta thiếu bằng chứng để dựa vào chúng, thậm chí dựa vào bằng cả tính mạng.

That's the kind of proof that TDD gives us. Not formal mathematical proof; but experimental empirical proof. The kind of proof we depend upon every day.

Đó chính là loại bằng chứng mà TDD mang lại cho ta. Không phải chứng minh toán học hình thức; mà là bằng chứng thực nghiệm. Đúng cái loại bằng chứng mà ngày nào ta cũng dựa vào.

And that brings us back to the third promise in the Oath:

Và điều đó đưa ta trở lại lời hứa thứ ba trong Lời tuyên thệ:

I will produce, with each release, a quick, sure, and repeatable demonstration that every element of the code works as it should.

Với mỗi lần phát hành, tôi sẽ đưa ra một minh chứng nhanh, chắc chắn và lặp lại được rằng mọi thành phần của mã đều chạy đúng như nó phải chạy.

Quick, sure, and repeatable. Quick means that the **test suite** should run in a very short amount of time. Minutes instead of hours. 4

Nhanh, chắc chắn và lặp lại được. Nhanh nghĩa là bộ test (test suite) phải chạy xong trong một khoảng thời gian rất ngắn. Vài phút thay vì vài giờ. 4

4 James T. Kirk, *Star Trek II: The Wrath of Khan*.

4 James T. Kirk, *Star Trek II: The Wrath of Khan*.

Sure means that when the test suite passes, you know you can ship.

Chắc chắn nghĩa là khi bộ test chạy qua hết, bạn biết mình có thể phát hành.

Repeatable means that those tests can be run by anybody at any time to ensure that the system is working properly. Indeed, we want the tests run many times per day.

Lặp lại được nghĩa là bất kỳ ai cũng có thể chạy những test đó vào bất kỳ lúc nào để bảo đảm hệ thống vẫn đang hoạt động đúng. Quả thực, chúng ta muốn các test được chạy nhiều lần mỗi ngày.

Some may think that it is too much to ask that programmers supply this level of proof. Some may think that programmers should not be held to this high a standard. I, on the other hand, can imagine no other standard that makes any sense.

Có người sẽ nghĩ đòi hỏi lập trình viên cung cấp mức bằng chứng như vậy là quá đáng. Có người sẽ nghĩ không nên áp cho lập trình viên một chuẩn mực cao đến thế. Còn tôi thì ngược lại: tôi chẳng hình dung nổi có chuẩn mực nào khác lại hợp lý cả.

When a customer pays us to develop software for them, aren't we honor bound to prove, to the best of our ability, that the software we've created does what that customer has paid us for?

Khi một khách hàng trả tiền cho chúng ta để phát triển phần mềm cho họ, chẳng lẽ chúng ta không có bốn phạm danh dự là phải chứng minh, bằng hết khả năng của mình, rằng phần mềm ta tạo ra làm đúng cái việc mà khách hàng đã trả tiền để có hay sao?

Of course we are.

Dĩ nhiên là có.

We owe this promise to our customers, our employers, and our teammates. We owe it to our business analysts, our testers, and our project managers.

Chúng ta nợ lời hứa này với khách hàng, với người sử dụng lao động và với đồng đội của mình. Chúng ta nợ nó với các business analyst, các tester và các quản lý dự án của mình.

But mostly we owe this promise to ourselves. For how can we consider ourselves professionals if we cannot prove that the work we have done is the work we have been paid to do?

Nhưng trên hết, chúng ta nợ lời hứa này với chính bản thân mình. Bởi làm sao ta có thể tự coi mình là người chuyên nghiệp nếu ta không chứng minh được rằng công việc mình đã làm đúng là công việc mình đã được trả tiền để làm?

What you owe, when you make that promise, is not the formal mathematical proof that Dijkstra dreamed of; rather, it is the scientific suite of tests that covers all the required **behavior**, runs in seconds or minutes, and produces the same clear pass/fail result every time it is run.

Cái bạn nợ, khi đưa ra lời hứa đó, không phải là chứng minh toán học hình thức mà Dijkstra hằng mơ; mà là bộ test mang tính khoa học phủ hết mọi hành vi được yêu cầu, chạy xong trong vài giây hoặc vài phút, và cho ra cùng một kết quả pass/fail rõ ràng ở mọi lần chạy.

Chapter 31. Small Cycles — Chương 31. Chu kỳ nhỏ

4. I will make frequent, small releases so that I do not impede the progress of others.

4. Tôi sẽ phát hành thường xuyên, mỗi lần một ít, để không cản trở tiến độ của người khác.

Making small releases just means changing a small amount of code for each release. The system may be large, but the change to that system is small.

Phát hành nhỏ chỉ đơn giản là mỗi lần phát hành chỉ thay đổi một lượng mã nhỏ. Hệ thống có thể rất lớn, nhưng thay đổi lên hệ thống đó thì nhỏ.

The History of Source Code Control — Lịch sử của quản lý mã nguồn

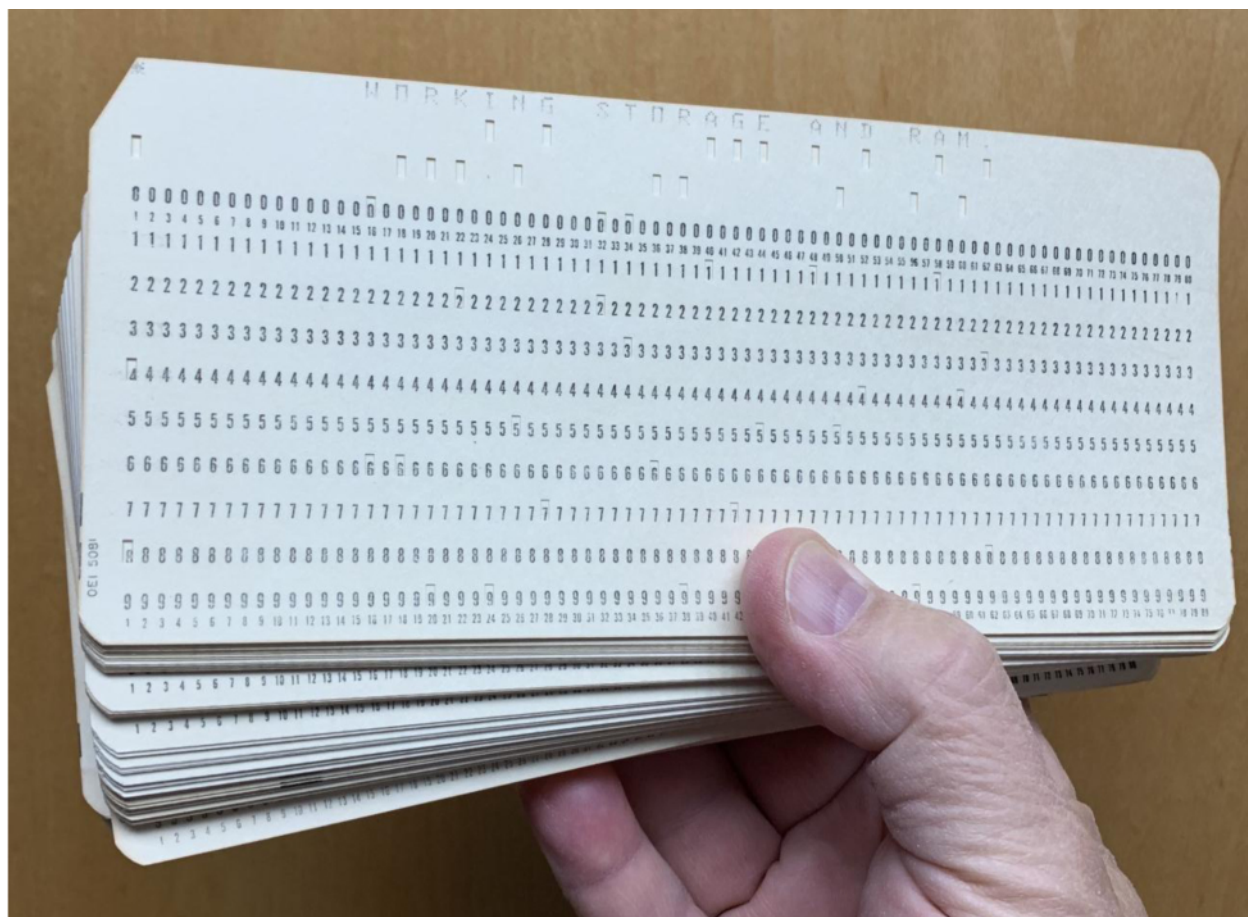
The following is a look back at the history of source code control.

Sau đây là một chuyến nhìn lại lịch sử của việc quản lý mã nguồn.

Punch Cards — Thẻ đục lỗ

Let's go back to the 1960s for a moment. What is your source code control system, when your source code is punched on a deck of cards?

Ta hãy quay về thập niên 1960 một lát. Hệ thống quản lý mã nguồn của bạn là gì, khi mã nguồn của bạn được đục lỗ trên một cỗ bài thẻ?



The source code is not stored on disk. It's not "in the computer." Your source code is, literally, in your hand.

Mã nguồn không nằm trên đĩa. Nó chẳng "ở trong máy tính" gì cả. Mã nguồn của bạn, theo đúng nghĩa đen, nằm trong tay bạn.

What is the source code control system? It is your desk drawer.

Vậy hệ thống quản lý mã nguồn là gì? Là ngăn kéo bàn làm việc của bạn.

When you literally possess the source code, there is no need to "control" it. Nobody else can touch it.

Khi bạn cầm mã nguồn trong tay theo đúng nghĩa đen, chẳng cần "quản lý" nó làm gì. Không ai khác chạm vào được.

And this was the situation throughout much of the '50s and '60s. Nobody even dreamed of something like a source code control system. You simply kept the source code under control by putting it in a drawer or a cabinet.

Và đó là tình cảnh suốt phần lớn thập niên 50 và 60. Chẳng ai buồn mơ tới một thứ như hệ thống quản lý mã nguồn. Bạn giữ mã nguồn trong tầm kiểm soát bằng cách bỏ nó vào ngăn kéo hay tủ hồ sơ.

If anyone wanted to "check out" the source code, they simply went to the cabinet and took it. When they were done, they put it back.

Nếu ai đó muốn "check out" mã nguồn, họ chỉ việc ra tủ và lấy. Xong việc thì bỏ lại chỗ cũ.

We certainly didn't have merge problems. It was physically impossible for two programmers to be making changes to the same modules at the same time.

Chắc chắn chúng tôi không có vấn đề merge. Về mặt vật lý, hai lập trình viên không thể cùng lúc sửa cùng một module.

But things started to change in the '70s. The idea of storing your source code on magnetic tape, or even on disk, was becoming attractive.

Nhưng mọi thứ bắt đầu đổi khác vào thập niên 70. Ý tưởng lưu mã nguồn trên băng từ, thậm chí trên đĩa, ngày càng hấp dẫn.

We wrote line editing programs that allowed us to add, replace, and delete lines in source files on tape.

Chúng tôi viết những chương trình soạn thảo theo dòng, cho phép thêm, thay thế và xóa các dòng trong tệp mã nguồn trên băng.

These weren't screen editors. We punched our add, change, and delete directives on cards. The editor would read the source tape, apply the changes from the edit deck, and write the new source tape.

Đó không phải trình soạn thảo trên màn hình. Chúng tôi đục lỗ các chỉ thị thêm, sửa, xóa lên thẻ. Trình soạn thảo sẽ đọc băng nguồn, áp các thay đổi từ cỗ bài chỉ thị, rồi ghi ra băng nguồn mới.

You may think this sounds awful. Looking back on it, it was. But it was better than trying to manage programs on cards! I mean, 6,000 lines of code on cards weighs 30 pounds. If you dropped it...

Bạn có thể nghĩ nghe thật kinh khủng. Nhìn lại thì đúng là kinh khủng thật. Nhưng vẫn còn hơn cố quản lý chương trình bằng thẻ! Ý tôi là, 6.000 dòng mã trên thẻ nặng chừng 14 ký. Nếu bạn làm rơi nó thì...

If you drop a tape, ..you can just pick it up again.

Còn nếu làm rơi cuộn băng, ..bạn chỉ việc nhặt lên là xong.

Anyway, notice what happened. We start with one source tape, and in the editing process, we wind up with a second, new source tape. But the old tape still exists. If you put that old tape back on the rack, someone else might inadvertently apply their own changes to it, leaving you with a merge problem.

Dù sao đi nữa, hãy để ý điều vừa xảy ra. Ta bắt đầu với một cuộn băng nguồn, và sau khi soạn thảo, ta có thêm cuộn băng nguồn thứ hai. Nhưng cuộn băng cũ vẫn còn đó. Nếu bạn đặt cuộn băng cũ trở lại giá, ai đó có thể vô tình áp thay đổi của họ lên nó, và thế là bạn dính một vấn đề merge.

To prevent that, we simply kept the master source tape in our possession until we were done with our edits and our tests. Then, we put a new master source tape back on the rack. We controlled the source code by keeping possession of the tape.

Để tránh chuyện đó, chúng tôi đơn giản giữ cuộn băng nguồn gốc (master) trong tay cho tới khi soạn thảo và kiểm thử xong. Rồi mới đặt một cuộn băng master mới lên giá. Chúng tôi quản lý mã nguồn bằng cách giữ khư khư cuộn băng.

But now this had to be done by process and convention. A true source code control process had to be used. No software yet, just human rules. But still, the concept of source code control had become separate from the source code itself.

Nhưng bây giờ việc đó phải được thực hiện bằng quy trình và quy ước. Phải áp dụng một quy trình quản lý mã nguồn thực thụ. Chưa có phần mềm nào cả, chỉ có luật lệ giữa người

với người. Dẫu vậy, khái niệm quản lý mã nguồn đã tách khỏi bản thân mã nguồn.

As systems got bigger and bigger, they needed more and more programmers working on the same code at the same time. Grabbing the master tape and holding it became a real nuisance for everyone else. I mean, you could keep the master tape out of circulation for a couple of days or more.

Khi hệ thống ngày một lớn, chúng cần ngày càng nhiều lập trình viên cùng làm trên cùng một mã nguồn, cùng lúc. Việc vồ lấy cuộn băng master rồi ôm khư khư trở thành nỗi phiền toái thực sự cho tất cả những người còn lại. Ý tôi là, bạn có thể giữ cuộn băng master ngoài vòng lưu thông vài ngày liền hoặc hơn.

So then, we decided to extract modules from the master tape. The whole idea of modular programming was pretty new at the time. The notion that a program could be made up of many different source files was revolutionary.

Thế nên chúng tôi quyết định tách các module ra khỏi cuộn băng master. Ý tưởng lập trình theo module hồi đó còn khá mới. Chuyện một chương trình có thể được ghép nên từ nhiều tệp nguồn khác nhau là một điều cách mạng.

So we started to use bulletin boards, like this:

Vậy là chúng tôi bắt đầu dùng bảng ghim, kiểu như thế này:

TRUNK MANAGER

SAC CONTROL

Routine

Data Lib

Parser

Term 10

Validate

Edit

Reticulate Splines

Zombie

Delete MAP

Bufctl

Modem Monitor

Dial

SYN SAK

Fardlesnap

Colligator

Dispatch

Line Test

comutation

Setup



TRUNK MANAGER

SAC CONTROL

Routine

Data Lib

Parser

Term 10

Validate

Edit

Reticulate Splines

Zombie

Delete MAP

Bufctl

Modem Monitor

Dial

SYN SAK

Fardlesnap

Colligator

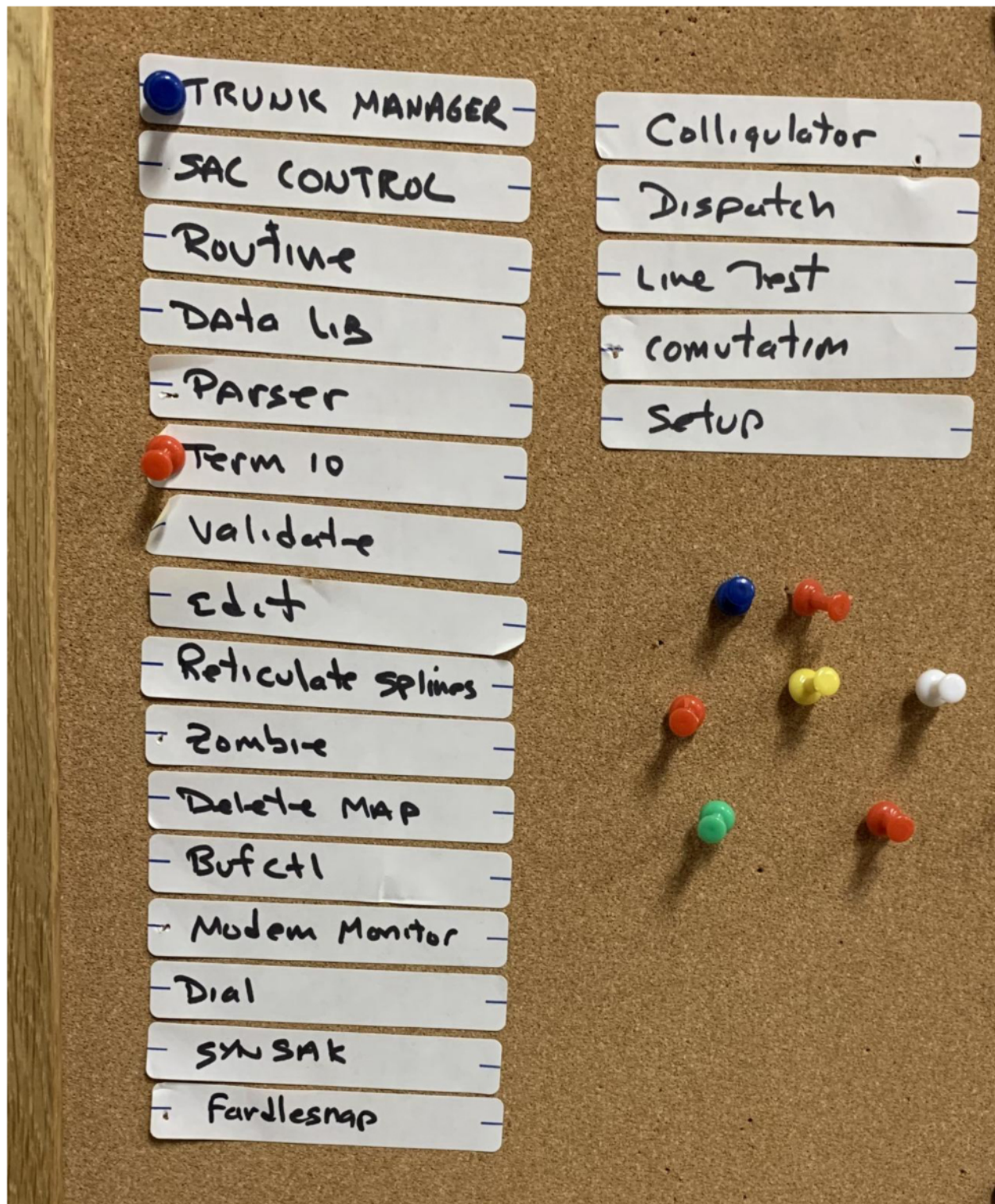
Dispatch

Line Test

comutation

Setup





The bulletin board had labels on it for each of the modules in the system. Each programmer had their own color thumbtack. I was blue. Ken was red. My buddy CK was yellow, and so on.

Trên bảng ghim có nhãn cho từng module trong hệ thống. Mỗi lập trình viên có một màu đinh ghim riêng. Tôi màu xanh dương. Ken màu đỏ. Bạn thân của tôi, CK, màu vàng, và cứ thế.

If I wanted to edit the “Trunk Manager” **module**, I’d look on the bulletin board to see if there was a pin in that module. If not, I put a blue pin in it.

Nếu tôi muốn sửa module “Trunk Manager”, tôi sẽ nhìn lên bảng ghim xem module đó đã có đinh nào chưa. Nếu chưa, tôi cắm một cái đinh xanh dương vào đó.

Then, I took the master tape from the rack and copied it onto a separate tape.

Rồi tôi lấy cuộn băng master từ giá xuống và sao nó sang một cuộn băng riêng.

I would edit the Trunk Manager module, and only the Trunk Manager module, generating a new tape with my changes.

Tôi sẽ sửa module Trunk Manager, và chỉ mỗi module Trunk Manager thôi, tạo ra một cuộn băng mới chứa các thay đổi của mình.

I’d compile, **test**, wash, and repeat until I got my changes to work.

Tôi biên dịch, kiểm thử, rồi lặp lại vòng đó tới khi các thay đổi chạy được.

Then, I’d go get the master tape from the rack, and I would create a new master by copying the current master, but replacing the Trunk Manager module with my changes.

Sau đó, tôi ra giá lấy cuộn băng master, và tạo một master mới bằng cách sao chép master hiện tại nhưng thay module Trunk Manager bằng bản đã sửa của mình.

I’d put the new master back in the rack.

Tôi đặt cuộn master mới trở lại giá.

Finally, I’d remove my blue pin from the bulletin board.

Cuối cùng, tôi rút cái đinh xanh dương của mình khỏi bảng ghim.

This worked; but it only worked because we all knew each other; we all worked in the same office together; we all knew what each other was doing. And we talked to each other all the time.

Cách này chạy được; nhưng nó chạy được chỉ vì tất cả chúng tôi đều biết nhau; chúng tôi làm chung một văn phòng; chúng tôi biết người kia đang làm gì. Và chúng tôi nói chuyện với nhau suốt.

I’d holler across the lab: “Ken, I’m going to change the Trunk Manager module.” He’d say, “Put a pin in it.” I’d say, “I already did.” The pins were just reminders. We all knew the status of the code and who was working on what. And that’s why the system worked.

Tôi hét vọng qua phòng lab: “Ken ơi, tôi sắp sửa module Trunk Manager đấy.” Anh ta đáp: “Cắm đinh vào đi.” Tôi bảo: “Cắm rồi.” Mấy cái đinh chỉ là lời nhắc. Tất cả chúng tôi đều nắm được trạng thái của mã và ai đang làm gì. Và đó là lý do hệ thống ấy chạy được.

Indeed, it worked very well. Knowing what each other was doing meant that we could help each other. We could make suggestions. We could warn each other about problems we’d recently encountered. And we could avoid merges.

Thật ra nó chạy rất tốt. Biết người khác đang làm gì nghĩa là chúng tôi có thể giúp nhau. Chúng tôi có thể góp ý. Chúng tôi có thể cảnh báo nhau về những vấn đề vừa gặp phải. Và chúng tôi tránh được merge.

Back then, merges were not fun.

Hồi đó, merge chẳng vui vẻ gì.

Then, in the ’80s, came the disks. Disks got big, and they got permanent.

Rồi tới thập niên 80, đĩa cứng xuất hiện. Đĩa trở nên lớn, và trở nên thường trực.

By big, I mean hundreds of megabytes. By permanent, I mean that they were permanently mounted—always online.

Lớn ở đây là hàng trăm megabyte. Thường trực ở đây nghĩa là chúng được gắn cố định — luôn trực tuyến.

The other thing that happened is that we got machines like PDP11s and VAXes. We got screen editors, real operating systems, and multiple terminals. More than one person could be editing at exactly the same time.

Chuyện khác nữa là chúng tôi có những cỗ máy như PDP11 và VAX. Chúng tôi có trình soạn thảo trên màn hình, hệ điều hành thực thụ, và nhiều terminal. Nhiều hơn một người có thể ngồi soạn thảo vào đúng cùng một thời điểm.

The era of thumbtacks and bulletin boards had to come to an end.

Kỷ nguyên của đinh ghim và bảng ghim buộc phải kết thúc.

First of all, by then, we had 20 or 30 programmers. There weren't enough thumbtack colors. Second of all, we had hundreds and hundreds of modules. We didn't have enough space on the bulletin board.

Trước hết, đến lúc đó chúng tôi đã có 20 hay 30 lập trình viên. Không đủ màu đinh ghim. Thứ hai, chúng tôi có hàng trăm hàng trăm module. Bảng ghim không còn đủ chỗ.

Fortunately, there was a solution.

May thay, đã có một lời giải.

In 1972, Marc Rochkind wrote the first source code control program. It was called SCCS, and he wrote it in SNOBOL. ¹

Năm 1972, Marc Rochkind viết chương trình quản lý mã nguồn đầu tiên. Nó tên là SCCS, và ông viết bằng SNOBOL. ¹

¹ A lovely little string processing language from the '60s that had many of the pattern matching facilities of our modern languages.

¹ Một ngôn ngữ xử lý chuỗi nhỏ xinh từ thập niên 60, sở hữu nhiều tiện ích khớp mẫu mà các ngôn ngữ hiện đại của chúng ta đang có.

Later he rewrote it in C, and it became part of the Unix distribution on PDP11s. SCCS only worked on one file at a time, but it allowed you to **lock** that file so that someone else couldn't edit it until you were done. It was a lifesaver.

Về sau ông viết lại nó bằng C, và nó trở thành một phần của bản phân phối Unix trên PDP11. SCCS chỉ làm việc với một tệp tại một thời điểm, nhưng nó cho phép bạn khoá tệp đó lại để người khác không sửa được cho tới khi bạn xong việc. Nó đúng là cứu tinh.

In 1982 RCS, the Revision Control System, was created by Walter Tichy. It too was file based and knew nothing of projects, but it was considered an improvement on SCCS and rapidly became the standard source code control system of the day.

Năm 1982, RCS — Revision Control System — được Walter Tichy tạo ra. Nó cũng dựa trên tệp và chẳng biết gì về khái niệm dự án, nhưng được xem là một bước tiến so với SCCS và nhanh chóng trở thành hệ thống quản lý mã nguồn chuẩn mực thời đó.

Then, in 1986, CVS, the Concurrent Versions System, came along. It extended RCS to deal with whole projects, not just individual files. It also introduced the concept of optimistic locking.

Rồi năm 1986, CVS — Concurrent Versions System — xuất hiện. Nó mở rộng RCS để xử lý cả dự án, chứ không chỉ từng tệp riêng lẻ. Nó cũng giới thiệu khái niệm khoá lạc quan (optimistic locking).

Up to this time, most source code control systems worked like my thumbtacks. If you had checked out the module, nobody else could edit it. This is called pessimistic locking .

Cho tới lúc đó, hầu hết hệ thống quản lý mã nguồn đều hoạt động y như mấy cái đinh ghim của tôi. Nếu bạn đã check out module, không ai khác sửa được nó nữa. Cách này gọi là khoá bi quan (pessimistic locking) .

CVS used optimistic locking . Two programmers could check out and change the same file at the same time. CVS would try to merge any nonconflicting changes, and would alert you if it couldn't figure out how to do the merge.

CVS dùng khoá lạc quan . Hai lập trình viên có thể cùng check out và cùng sửa một tệp vào cùng thời điểm. CVS sẽ cố merge những thay đổi không xung đột, và báo cho bạn khi nó không biết phải merge thế nào.

After that, source code control systems exploded and even became commercial products. There were literally hundreds of them. Some used optimistic locking, others used pessimistic locking. Indeed, the locking strategy became a kind of religious divide in the industry.

Sau đó, các hệ thống quản lý mã nguồn bùng nổ, thậm chí thành sản phẩm thương mại. Có tới hàng trăm cái, không nói ngoa. Một số dùng khoá lạc quan, số khác dùng khoá bi quan. Quả thật, chiến lược khoá trở thành một kiểu chia rẽ mang màu sắc tôn giáo trong ngành.

Then, in 2000, Subversion was created. It vastly improved upon CVS and was instrumental in driving the industry away from pessimistic locking once and for all.

Rồi năm 2000, Subversion ra đời. Nó cải thiện CVS một cách vượt bậc và là công cụ then chốt đẩy cả ngành rời bỏ khoá bi quan, dứt khoát và mãi mãi.

Subversion was also the first source code control system to be used in the “cloud.” Does anybody remember Source Forge?

Subversion cũng là hệ thống quản lý mã nguồn đầu tiên được dùng trên “đám mây”. Có ai còn nhớ Source Forge không?

Up to this point, all source code control systems were based on the master tape idea that I used back in my bulletin board days. The source code was maintained in a single central master repository. Source code was checked out from that master repository, and commits were made back into that master repository.

Cho tới thời điểm này, mọi hệ thống quản lý mã nguồn đều dựa trên ý tưởng cuộn băng master mà tôi từng dùng thời còn băng ghim. Mã nguồn được lưu trong một kho master trung tâm duy nhất. Mã nguồn được check out từ kho master ấy, và các commit thì được đẩy ngược trở lại kho master ấy.

But all that was about to change.

Nhưng tất cả những điều đó sắp thay đổi.

Continuous Integration — Tích hợp liên tục

The year is 2005. Multigigabyte disks are in our laptops. Network speeds are fast, and getting faster. Processor clock rates have plateaued at 2.6GHz.

Năm 2005. Đĩa nhiều gigabyte đã nằm trong laptop của chúng ta. Tốc độ mạng nhanh, và ngày càng nhanh hơn. Xung nhịp bộ xử lý chững lại ở mức 2,6GHz.

We are very, very far away from my old bulletin board control system for source code. But we are still using the master tape concept. We still have a central repository that everybody has to check in and out of. Every commit; every reversion; every merge requires network connectivity to the master.

Chúng ta đã đi rất, rất xa so với cái hệ thống quản lý mã nguồn bằng băng ghim ngày xưa của tôi. Nhưng ta vẫn đang dùng khái niệm cuộn băng master. Ta vẫn có một kho trung tâm mà ai cũng phải check in, check out qua đó. Mỗi commit; mỗi lần hoàn tác; mỗi lần merge đều đòi hỏi kết nối mạng tới master.

...and then came git.

...rồi git xuất hiện.

Well, actually, git was presaged by BitKeeper and Monotone, but it was git that caught the attention of the programming world and changed everything.

Thật ra thì git đã được báo trước bởi BitKeeper và Monotone, nhưng chính git mới là thứ hút được sự chú ý của giới lập trình và thay đổi tất cả.

Because, you see, git eliminates the master tape.

Bởi vì, bạn thấy đấy, git xoá sổ cuộn băng master.

Oh, you still need a final authoritative version of the source code. But git does not automatically provide this location for you. Git simply doesn't care about that. Where you decide to put your authoritative version is entirely up to you. Git has nothing whatsoever to do with it.

Ồ, bạn vẫn cần một phiên bản mã nguồn có thẩm quyền cuối cùng. Nhưng git không tự động cung cấp cho bạn chỗ đó. Git đơn giản là chẳng bận tâm. Bạn quyết định đặt phiên bản có thẩm quyền ở đâu là hoàn toàn tùy bạn. Git chẳng dính dáng gì tới chuyện ấy cả.

Git keeps the entire history of the source code on your local machine. Your laptop, if that's what you use. On your machine, you can commit changes, create branches, check out old versions. And generally do anything you could do with a centralized system like Subversion; except you don't need to be connected to some central server.

Git giữ toàn bộ lịch sử mã nguồn ngay trên máy của bạn. Trên laptop, nếu đó là thứ bạn đang dùng. Trên máy của mình, bạn có thể commit thay đổi, tạo nhánh, check out các phiên bản cũ. Và nói chung là làm được mọi thứ bạn làm được với một hệ thống tập trung như Subversion; chỉ khác là bạn không cần kết nối tới một máy chủ trung tâm nào.

Anytime you like, you can connect to another user and push any of the changes you've made to that user. Or you can pull any changes they have made into your local repository. Neither is the master. Both are equal. That's why they call it peer to peer .

Bất cứ lúc nào muốn, bạn có thể kết nối tới một người dùng khác và push bất kỳ thay đổi nào bạn đã làm sang cho họ. Hoặc bạn có thể pull bất kỳ thay đổi nào họ đã làm về kho cục bộ của mình. Không bên nào là master. Cả hai đều ngang hàng. Đó là lý do người ta gọi nó là peer to peer .

And the final authoritative location that you use for making production releases is just another peer that people can push to or pull from, anytime they like.

Còn cái vị trí có thẩm quyền cuối cùng mà bạn dùng để phát hành lên production cũng chỉ là một peer nữa, nơi mọi người có thể push lên hoặc pull về bất cứ lúc nào họ thích.

The end result of this is that you are free to make as many small commits as you like before you push your changes somewhere else. You can commit every 30 seconds if you like. You could make a commit every time you get a **unit test** to pass.

Kết quả cuối cùng là bạn được tự do tạo bao nhiêu commit nhỏ tùy thích trước khi push thay đổi đi nơi khác. Bạn có thể commit mỗi 30 giây nếu muốn. Bạn có thể commit mỗi lần làm cho một unit test chạy qua.

And that brings us to the point of this whole historical discussion.

Và điều đó dẫn ta tới trọng tâm của cả cuộc bàn luận lịch sử này.

If you stand back and look at the trajectory of the evolution of source code control systems, you can see that they have been driven, perhaps unconsciously, by a single underlying imperative.

Nếu bạn lùi lại và nhìn vào quỹ đạo tiến hoá của các hệ thống quản lý mã nguồn, bạn sẽ thấy chúng bị thúc đẩy — có lẽ một cách vô thức — bởi một mệnh lệnh nền tảng duy nhất.

Short Cycles — Chu kỳ ngắn

Consider, again, how we began. How long was the cycle when source code was controlled by physically possessing a deck of cards?

Hãy nghĩ lại một lần nữa về điểm khởi đầu. Chu kỳ dài bao lâu, khi mã nguồn được quản lý bằng việc nắm giữ vật lý một cỗ bài thẻ?

You checked the source code out by taking those cards out of the cabinet. You held on to those cards until you were done with your project. Then, you committed your changes by putting the changed deck of cards back in the cabinet.

Bạn check out mã nguồn bằng cách lấy chồng thẻ đó ra khỏi tủ. Bạn ôm chồng thẻ ấy cho tới khi xong dự án. Rồi bạn commit thay đổi của mình bằng cách bỏ cỗ bài thẻ đã sửa vào lại tủ.

The cycle time was the whole project.

Thời gian một chu kỳ chính là cả dự án.

Then, when we used thumbtacks on the bulletin board, the same rule applied. You kept your thumbtacks in the modules you were changing until you were done with the project you were working on.

Rồi khi chúng tôi dùng đinh ghim trên bảng, luật vẫn y như vậy. Bạn giữ đinh ghim của mình trên những module đang sửa cho tới khi xong dự án đang làm.

Even in the late '70s and into the '80s, when we were using SCCS and RCS, we continued to use this pessimistic locking strategy, keeping others from touching the modules we were changing until we were done.

Thậm chí cuối thập niên 70 và sang thập niên 80, khi đang dùng SCCS và RCS, chúng tôi vẫn tiếp tục dùng chiến lược khoá bi quan này, chặn người khác đụng vào những module mình đang sửa cho tới khi xong việc.

But CVS changed things—at least for some of us. Optimistic locking meant that one programmer could not lock others out of a module. We still only committed when we were done with a project; but others could be working concurrently on the same modules. So the average time between commits on the project shrank drastically. The cost of this, of course, was merges.

Nhưng CVS đã làm mọi thứ khác đi — ít nhất là với một số người trong chúng tôi. Khoá lạc quan nghĩa là một lập trình viên không thể khoá người khác ra khỏi một module. Chúng tôi vẫn chỉ commit khi xong dự án; nhưng người khác có thể làm việc đồng thời trên cùng những module đó. Nhờ vậy, thời gian trung bình giữa các commit trong dự án co lại rất mạnh. Cái giá phải trả, dĩ nhiên, là merge.

And how we hated doing merges.

Và chao ôi chúng tôi ghét merge tới mức nào.

Merges are awful. Especially without unit tests! They are tedious, time-consuming, and dangerous.

Merge thật kinh khủng. Nhất là khi không có unit test! Chúng tẻ nhạt, tốn thời gian và nguy hiểm.

Indeed, our distaste for merges drove us to a new strategy.

Quả thật, nỗi ghê sợ merge đã đẩy chúng tôi tới một chiến lược mới.

Continuous Integration — Tích hợp liên tục

By 2000, even while we were using tools like Subversion, we had begun teaching the **discipline** of committing every few minutes.

Đến năm 2000, ngay cả khi vẫn còn dùng những công cụ như Subversion, chúng tôi đã bắt đầu dạy kỷ luật commit vài phút một lần.

The rationale was simple. The more frequently you commit, the less likely you are to face a merge. And if you do have to merge, the merge will be trivial.

Lý lẽ rất đơn giản. Bạn commit càng thường xuyên, khả năng bạn phải đối mặt với một cuộc merge càng thấp. Và nếu có phải merge, cuộc merge ấy cũng sẽ nhẹ tênh.

We called this discipline **continuous integration**.

Chúng tôi gọi kỷ luật này là tích hợp liên tục (continuous integration).

Of course, this depends critically upon having a very reliable suite of unit tests. I mean, without **good** unit tests, it would be easy to make a merge error and break someone else's code. So continuous integration goes hand in hand with a good testing discipline.

Dĩ nhiên, điều này phụ thuộc sống còn vào việc có một bộ unit test cực kỳ đáng tin cậy. Ý tôi là, không có unit test tốt thì rất dễ merge sai và làm hỏng mã của người khác. Vậy nên tích hợp liên tục đi đôi với một kỷ luật kiểm thử tốt.

And now, with tools like git, there is almost no limit to how small we can shrink the cycle.

Và giờ đây, với những công cụ như git, gần như không còn giới hạn nào cho việc chúng ta có thể rút ngắn chu kỳ tới mức nào.

And that begs the question: Why are we so concerned about shrinking the cycle?

Điều đó dẫn tới câu hỏi: Tại sao chúng ta lại quan tâm tới việc rút ngắn chu kỳ đến thế?

Because long cycles impede the progress of the team.

Bởi vì chu kỳ dài cản trở tiến độ của cả nhóm.

The longer the time between commits, the greater the chance that someone else on the team—perhaps the whole team—will have to wait for you. And that violates the promise.

Thời gian giữa các commit càng dài, khả năng người khác trong nhóm — thậm chí cả nhóm — phải chờ bạn càng lớn. Và như thế là vi phạm lời hứa.

Perhaps you think this is only about production releases. No, it's actually about every other cycle. It's about iterations and sprints. It's about the edit/compile/test cycle. It's about the time between commits. It's about everything.

Có lẽ bạn nghĩ chuyện này chỉ liên quan tới các bản phát hành production. Không đâu, nó thật ra liên quan tới mọi chu kỳ khác. Nó liên quan tới các iteration và sprint. Nó liên quan tới chu kỳ sửa/biên dịch/kiểm thử. Nó liên quan tới khoảng thời gian giữa các commit. Nó liên quan tới mọi thứ.

And remember the rationale: “So that you do not impede the progress of others.”

Và hãy nhớ lý lẽ đằng sau: “Để bạn không cản trở tiến độ của người khác.”

Branches versus Toggles — Nhánh hay toggle

I used to be a branch dogmatist. Back when I was using CVS and Subversion, I refused to allow members on my teams to branch the code. I wanted all changes returned to the main line as frequently as possible.

Tôi từng là một kẻ giáo điều về nhánh. Hồi còn dùng CVS và Subversion, tôi không cho phép thành viên trong nhóm của mình tạo nhánh cho mã nguồn. Tôi muốn mọi thay đổi được trả về nhánh chính càng thường xuyên càng tốt.

My rationale was simple. A branch is simply a long-term checkout. And as we've seen, long-term checkouts impede the progress of others by prolonging the time between integrations.

Lý lẽ của tôi rất đơn giản. Một nhánh chẳng qua là một lần check out kéo dài. Và như ta đã thấy, những lần check out kéo dài sẽ cản trở tiến độ của người khác bằng cách kéo giãn thời gian giữa các lần tích hợp.

But then I switched to git—and everything changed overnight.

Nhưng rồi tôi chuyển sang git — và mọi thứ thay đổi chỉ sau một đêm.

At the time, I was managing the open source FitNesse project, which had a dozen or so people working on it. I had just moved the FitNesse repository from Subversion (Source Forge) to git (GitHub). Suddenly, branches started appearing all over the place.

Thời điểm đó, tôi đang quản lý dự án mã nguồn mở FitNesse, với chừng một tá người tham gia. Tôi vừa mới chuyển kho FitNesse từ Subversion (Source Forge) sang git (GitHub). Đùng một cái, nhánh mọc lên khắp nơi.

For the first few days, these crazy branches in git had me confused. Should I abandon my branch dogmatism stance? Should I abandon continuous integration and simply allow everyone to make branches willy-nilly, forgetting about the cycle time issue?

Vài ngày đầu, đám nhánh loạn xạ trong git khiến tôi rối trí. Liệu tôi có nên từ bỏ lập trường giáo điều về nhánh? Liệu tôi có nên từ bỏ tích hợp liên tục và cứ để ai nấy tạo nhánh vô tội vạ, quên phắt chuyện thời gian chu kỳ đi?

But then, it occurred to me that these “branches” I was seeing were not true named branches. Instead, they were just the stream of commits made by a developer between pushes.

Nhưng rồi tôi chợt nhận ra rằng những “nhánh” mà tôi đang thấy không phải là nhánh có tên thực thụ. Chúng chỉ là dòng commit mà một lập trình viên tạo ra giữa hai lần push.

In fact, all git had really done was to record the actions of the developer between continuous integration cycles.

Thực ra, tất cả những gì git làm chỉ là ghi lại các thao tác của lập trình viên giữa các chu kỳ tích hợp liên tục.

So I resolved to continue my rule restricting branches. It’s just that now it’s not commits that return immediately to the main line. It’s pushes. Continuous integration was preserved.

Thế nên tôi quyết định giữ nguyên luật hạn chế nhánh của mình. Chỉ có điều bây giờ thứ phải quay về nhánh chính ngay lập tức không phải là commit. Mà là push. Tích hợp liên tục vẫn được bảo toàn.

If we are following continuous integration and pushing to the main line every hour or so, then we’ll certainly have a bunch of half-written features on the main line. There are typically two strategies for dealing with that: branches and toggles.

Nếu chúng ta tuân theo tích hợp liên tục và push lên nhánh chính chừng mỗi giờ một lần, thì chắc chắn trên nhánh chính sẽ có một mớ tính năng viết dở. Thường có hai chiến lược để xử lý chuyện đó: nhánh và toggle.

The branching strategy is simple. You create a new branch of the source code in which to develop the feature. You merge it back when the feature is done.

Chiến lược tạo nhánh thì đơn giản. Bạn tạo một nhánh mới của mã nguồn để phát triển tính năng trong đó. Bạn merge nó về khi tính năng hoàn thành.

If you keep the branch out of the main line for days or weeks, then you are likely facing a big merge; and you’ll certainly be impeding the team.

Nếu bạn giữ nhánh đó ngoài nhánh chính hàng ngày hay hàng tuần, nhiều khả năng bạn sẽ đối mặt với một cuộc merge lớn; và chắc chắn bạn đang cản trở cả nhóm.

However, there are cases where the new feature is so isolated from the rest of the code that branching it is not likely to cause a big merge. In those cases, it might be better to let the developers work in peace on the new feature without continuously integrating.

Tuy nhiên, có những trường hợp tính năng mới biệt lập khỏi phần mã còn lại tới mức tách nhánh cho nó khó mà gây ra một cuộc merge lớn. Trong những trường hợp đó, có lẽ tốt hơn là cứ để các lập trình viên yên ổn làm tính năng mới mà không cần tích hợp liên tục.

In fact, we had a case like this with FitNesse a few years ago. We completely rewrote the parser. This was a big project. It took a few person-weeks. And there was no way to do it incrementally. I mean, the parser is the parser.

Thật ra, chúng tôi đã gặp một trường hợp như vậy với FitNesse vài năm trước. Chúng tôi viết lại hoàn toàn bộ parser. Đó là một dự án lớn. Nó tốn vài person-week. Và không có cách nào làm dần từng chút được. Ý tôi là, parser thì là parser thôi.

So we created a branch and kept that branch isolated from the rest of the system until the parser was ready.

Vậy nên chúng tôi tạo một nhánh và giữ nhánh đó tách biệt khỏi phần còn lại của hệ thống cho tới khi parser sẵn sàng.

There was a merge to do at the end, but it wasn't too bad. The parser was isolated well enough from the rest of the system. And fortunately, we had a very comprehensive set of unit and acceptance tests.

Cuối cùng vẫn phải merge, nhưng cũng không đến nỗi tệ. Parser đã được tách biệt đủ tốt khỏi phần còn lại của hệ thống. Và may mắn thay, chúng tôi có một bộ unit test và acceptance test (kiểm thử chấp nhận) rất toàn diện.

Most of the time, however, I think it's better to keep new feature development on the main line, and use toggles to turn those features off until ready.

Tuy nhiên, phần lớn thời gian, tôi cho rằng tốt hơn là giữ việc phát triển tính năng mới ngay trên nhánh chính, và dùng toggle để tắt các tính năng đó cho tới khi chúng sẵn sàng.

Sometimes we use flags for those toggles, but more often we use the Command pattern, the Decorator pattern, and special versions of the Factory pattern to make sure the partially written features cannot be executed in a production environment.

Đôi khi chúng tôi dùng cờ (flag) cho những toggle ấy, nhưng thường thì chúng tôi dùng Command pattern, Decorator pattern, và những biến thể đặc biệt của Factory pattern để bảo đảm các tính năng viết dở không thể chạy được trong môi trường production.

And most of the time, we simply don't give the user the option to use the new feature. I mean, if the button isn't on the Web page, you can't execute that feature...

Và phần lớn thời gian, chúng tôi đơn giản là không cho người dùng lựa chọn dùng tính năng mới. Ý tôi là, nếu cái nút không có trên trang Web, bạn đâu thể chạy tính năng đó...

In many cases, of course, new features will be completed as part of the current iteration—or at least before the next production release. So there's no real need for any kind of toggle.

Tất nhiên, trong nhiều trường hợp, tính năng mới sẽ hoàn thành ngay trong iteration hiện tại — hoặc chỉ ít là trước bản phát hành production kế tiếp. Nên chẳng cần tới toggle nào cả.

You only need a toggle if you are going to be releasing to production while some of the features are unfinished. So how often should that be?

Bạn chỉ cần toggle nếu bạn sắp phát hành lên production trong khi vài tính năng vẫn còn dang dở. Vậy chuyện đó nên xảy ra bao lâu một lần?

Continuous Deployment — Triển khai liên tục (Continuous Deployment)

What if we could eliminate the delays between production releases? What if we could release to production several times per day? After all, delaying production releases impedes others.

Sẽ ra sao nếu ta xóa bỏ được những khoảng trễ giữa các lần phát hành lên production? Sẽ ra sao nếu ta có thể phát hành lên production vài lần mỗi ngày? Suy cho cùng, trì hoãn phát hành lên production là gây cản trở cho người khác.

I want you to be able to release your code to production several times per day. I want you to feel comfortable enough with your work that you could release your code to production on every push.

Tôi muốn bạn có thể đưa mã của mình lên production vài lần mỗi ngày. Tôi muốn bạn yên tâm về công việc của mình đến mức có thể phát hành lên production sau mỗi lần push.

This, of course, depends upon automated testing. Automated tests are written by programmers to cover every line of code. Automated tests are written by business analysts and Q/A testers to cover every desired **behavior**.

Tất nhiên, điều này phụ thuộc vào kiểm thử tự động. Test tự động do lập trình viên viết để phủ mọi dòng mã. Test tự động do business analyst và người kiểm thử Q/A viết để phủ mọi hành vi mong muốn.

Remember the preceding chapter? Those tests are the scientific proof that everything works as it is supposed to. And if everything works as it is supposed to, the next step is to deploy to production.

Còn nhớ chương trước chứ? Những test đó chính là bằng chứng khoa học cho thấy mọi thứ chạy đúng như nó phải chạy. Và nếu mọi thứ chạy đúng như nó phải chạy thì bước tiếp theo là triển khai lên production.

And by the way, that's how you know if your tests are good enough. Your tests are good enough if, when they pass, you feel comfortable deploying. If passing tests don't allow you to deploy, your tests are deficient.

Mà nhân tiện, đó cũng là cách bạn biết bộ test của mình đã đủ tốt hay chưa. Test của bạn đủ tốt nếu khi chúng pass, bạn thấy thoải mái để triển khai. Nếu test pass mà bạn vẫn không dám triển khai thì test của bạn còn thiếu sót.

Perhaps you think that deploying every day, or even several times per day, would lead to chaos. However, the fact that you are ready to deploy does not mean that the business is ready to deploy. As a development team, your standard is to always be ready.

Có lẽ bạn nghĩ rằng triển khai mỗi ngày, thậm chí vài lần mỗi ngày, sẽ dẫn tới hỗn loạn. Tuy nhiên, việc bạn *sẵn sàng* triển khai không có nghĩa là phía kinh doanh cũng *sẵn sàng* triển khai. Là một nhóm phát triển, chuẩn mực của bạn là *luôn luôn* sẵn sàng.

What's more, we want to help the business remove all the impediments to deployment so that the business deployment cycle can be shortened as much as possible.

Hơn nữa, chúng ta muốn giúp phía kinh doanh gỡ bỏ mọi vật cản trên đường triển khai, để chu kỳ triển khai của doanh nghiệp được rút ngắn hết mức có thể.

After all, the more ceremony and ritual the business uses for deployment, the more expensive deployment becomes. Any business would like to shed that expense.

Suy cho cùng, doanh nghiệp càng bày ra nhiều nghi thức và thủ tục quanh việc triển khai thì triển khai càng tốn kém. Doanh nghiệp nào cũng muốn trút bỏ khoản chi phí ấy.

The ultimate goal for any business is continuous, safe, and ceremony-free deployment. Deployment should be as close to a nonevent as possible.

Mục tiêu tối hậu của mọi doanh nghiệp là triển khai liên tục, an toàn và không nghi thức rườm rà. Triển khai nên gần như là một chuyện chẳng có gì đáng kể.

And since deployment is often a lot of work, with servers to configure and databases to load, you need to automate the deployment procedure.

Và vì triển khai thường tốn rất nhiều công sức — nào cấu hình server, nào nạp dữ liệu vào database — bạn cần tự động hoá quy trình triển khai.

And since deployment scripts are part of the system, you write tests for them.

Và vì các script triển khai là một phần của hệ thống, bạn viết test cho chúng.

For many of you, the idea of continuous deployment may be so far away from your current process that it's inconceivable. But that doesn't mean there aren't things you can do to shorten the cycle.

Với nhiều người trong các bạn, ý tưởng triển khai liên tục có lẽ xa vời đến mức không thể tưởng tượng nổi so với quy trình hiện tại. Nhưng điều đó không có nghĩa là chẳng có gì bạn làm được để rút ngắn chu kỳ.

And who knows? If you keep on shortening the cycle, month after month, year after year, perhaps one day you'll find that you are deploying continuously.

Và biết đâu đấy? Nếu bạn cứ rút ngắn chu kỳ, tháng này qua tháng khác, năm này qua năm khác, có lẽ một ngày nào đó bạn sẽ nhận ra mình đang triển khai liên tục.

Continuous Build — Build liên tục (Continuous Build)

Clearly, if you are going to deploy in short cycles, you have to be able to build in short cycles. If you are going to deploy continuously, you have to be able to build continuously.

Rõ ràng, nếu định triển khai theo chu kỳ ngắn thì bạn phải build được theo chu kỳ ngắn. Nếu định triển khai liên tục thì bạn phải build được liên tục.

Perhaps some of you have slow build times. If you do, speed them up.

Có lẽ một số người trong các bạn đang chịu đựng thời gian build chậm. Nếu vậy, hãy làm cho nó nhanh lên.

Seriously, with the memory and speed of modern systems, there is no excuse for slow builds. None. Speed them up. Consider it a design challenge.

Nghiêm túc đấy, với bộ nhớ và tốc độ của máy móc hiện đại, không có lý do gì để bào chữa cho build chậm. Không một lý do nào. Hãy làm cho nó nhanh lên. Hãy coi đó là một bài toán thiết kế.

And then, get yourself a continuous build tool and use it. Make sure that you kick off a build at every push. And do what it takes to ensure that the build never fails.

Rồi sau đó, hãy sắm một công cụ build liên tục và dùng nó. Hãy bảo đảm mỗi lần push đều kích hoạt một lần build. Và hãy làm mọi việc cần thiết để bảo đảm build không bao giờ hỏng.

A build failure is a red alert. It's an emergency. If the build fails, I want emails and text messages sent to every team member. I want sirens going off. I want a flashing red light on the CEO's desk. I want everybody to stop whatever they are doing and deal with the emergency.

Build hỏng là báo động đỏ. Đó là tình trạng khẩn cấp. Nếu build hỏng, tôi muốn email và tin nhắn được gửi tới từng thành viên trong nhóm. Tôi muốn còi hú vang lên. Tôi muốn một ngọn đèn đỏ nhấp nháy ngay trên bàn CEO. Tôi muốn tất cả mọi người bỏ hết mọi việc đang làm để xử lý tình trạng khẩn cấp đó.

Keeping the build from failing is not rocket science. You simply run the build in your local environment, along with all the tests, before you push. You only push the code when all the tests pass.

Giữ cho build không hỏng chẳng phải chuyện gì cao siêu. Bạn chỉ cần chạy build ở môi trường cục bộ của mình, kèm toàn bộ test, trước khi push. Bạn chỉ push mã khi tất cả test đều pass.

If the build fails after that, you've uncovered some environmental issue that needs to be resolved posthaste.

Nếu sau đó build vẫn hỏng thì bạn vừa phát hiện ra một vấn đề về môi trường, và nó cần được giải quyết ngay lập tức.

You never allow the build to go on failing, because if you allow the build to fail, you'll get used to it failing. And if you get used to the failures, you'll start ignoring them.

Bạn không bao giờ được để build hỏng dai dẳng, vì nếu bạn để build hỏng, bạn sẽ quen dần với việc nó hỏng. Và một khi đã quen với những lần hỏng ấy, bạn sẽ bắt đầu phớt lờ chúng.

The more you ignore those failures, the more annoying the failure alerts become. And the more tempted you are to turn the failing tests off until you can fix them—later. You know. Later?

Càng phớt lờ những lần hỏng đó, những cảnh báo hỏng build càng trở nên phiền toái. Và bạn càng bị cám dỗ tắt luôn mấy cái test đang fail đi, cho tới khi sửa được chúng — để sau. Bạn biết đấy. Để sau?

And that's when the tests become lies.

Và đó chính là lúc test biến thành lời nói dối.

With the failing tests removed, the build passes again. And that makes everyone feel good again. But it's a lie.

Gỡ bỏ các test đang fail đi, build lại pass. Và ai nấy lại thấy dễ chịu. Nhưng đó là một lời nói dối.

So build continuously. And never let the build fail.

Vậy nên hãy build liên tục. Và đừng bao giờ để build hỏng.

Chapter 32. Relentless Improvement

— Chương 32. Cải tiến không ngừng

5. I will fearlessly and relentlessly improve my creations at every opportunity. I will never degrade them.

5. Tôi sẽ can đảm và không ngừng cải tiến những gì mình tạo ra, ở mọi cơ hội có được. Tôi sẽ không bao giờ làm chúng tệ đi.

Robert Baden Powell, the father of the Boy Scouts of America, left a posthumous message exhorting the scouts to leave the world a better place than they found it. It was from this statement that the “Boy Scout Rule” was derived.

Robert Baden Powell, cha đẻ của phong trào Hướng đạo sinh Mỹ, để lại một thông điệp sau khi qua đời, nhắc nhở các hướng đạo sinh hãy để lại thế giới này tốt đẹp hơn lúc họ tìm thấy nó. Chính từ câu nói ấy mà “Quy tắc Hướng đạo sinh (the Boy Scout Rule)” ra đời.

Check the code in cleaner than you checked it out.

Hãy check in mã vào sạch hơn lúc bạn check out nó ra.

How? By performing random acts of kindness upon the code every time you check it in. One of those random acts of kindness is to increase **test coverage**.

Bằng cách nào? Bằng cách thực hiện những hành động tử tế ngẫu nhiên với mã mỗi lần bạn check in. Một trong những hành động tử tế ngẫu nhiên đó là tăng độ phủ test (test coverage).

Test Coverage — Độ phủ test

Do you measure how much of your code is covered by your tests? Do you know what percentage of lines are covered? Do you know what percentage of branches are covered? If not, why not?

Bạn có đo xem bao nhiêu phần mã của mình được test phủ tới không? Bạn có biết bao nhiêu phần trăm số dòng được phủ không? Bao nhiêu phần trăm số nhánh được phủ? Nếu không, thì tại sao lại không?

There are plenty of tools that can measure coverage for you. For most of us, those tools come as part of our IDE and are trivial to run. So there’s usually no excuse for not knowing what your coverage numbers are.

Có vô số công cụ đo độ phủ giúp bạn. Với hầu hết chúng ta, những công cụ ấy đi kèm luôn trong IDE và chạy thì dễ như trở bàn tay. Nên thường chẳng có lý do gì để bào chữa cho việc không biết con số độ phủ của mình.

What should you do with those numbers? First, let me tell you what not to do. Don't turn them into management metrics. Don't fail the build if your **test** coverage is too low. Test coverage is a complicated concept that should not be used so naively.

Vậy nên làm gì với những con số đó? Trước hết, để tôi nói cho bạn biết cái *không* nên làm. Đừng biến chúng thành chỉ số quản lý. Đừng làm hỏng build chỉ vì độ phủ test quá thấp. Độ phủ test là một khái niệm phức tạp, không nên đem ra dùng một cách ngây thơ như thế.

Such naive usage sets up perverse incentives to cheat. And it is very easy to cheat test coverage. Remember that coverage tools only measure the amount of code that was executed, not the code that was actually tested. This means that you can drive the coverage number very high by pulling the assertions out of your failing tests. And, of course, that makes the metric useless.

Cách dùng ngây thơ ấy tạo ra động cơ méo mó để gian lận. Mà gian lận độ phủ test thì dễ vô cùng. Hãy nhớ rằng các công cụ đo độ phủ chỉ đo lượng mã *đã được thực thi*, chứ không phải lượng mã thực sự *được kiểm thử*. Nghĩa là bạn có thể đẩy con số độ phủ lên rất cao chỉ bằng cách rút hết assertion ra khỏi những test đang fail. Và dĩ nhiên, làm vậy thì chỉ số ấy trở thành vô dụng.

The best policy is to use the coverage numbers as a developer tool to help you improve the code. You should work to meaningfully drive the coverage toward 100% by writing actual tests.

Chính sách tốt nhất là dùng con số độ phủ như một công cụ của lập trình viên, giúp bạn cải thiện mã. Bạn nên nỗ lực đẩy độ phủ tiến về 100% một cách thực chất, bằng cách viết những test thật sự.

One hundred percent test coverage is always the goal, but it is also an asymptotic goal. Most systems never reach 100%, but that should not deter you from constantly trying to attain it.

Độ phủ test 100% luôn là mục tiêu, nhưng đó là một mục tiêu tiệm cận. Phần lớn hệ thống chẳng bao giờ chạm tới 100%, nhưng điều đó không được phép làm bạn nản lòng mà thôi không ngừng vươn tới nó.

That's what you use the coverage numbers for. You use them as a measurement to help you improve, not as a bludgeon with which to punish the team and fail the build.

Đó mới là công dụng của những con số độ phủ. Bạn dùng chúng như một phép đo để giúp mình tiến bộ, chứ không phải như cây gậy để nện cả nhóm và đánh hỏng build.

Mutation Testing — Mutation testing

One hundred percent test coverage implies that any semantic change to the code should cause a test to fail. **TDD**, TCR, and to a lesser extent, Small Bundles, are **good** disciplines to approximate that goal because, if you follow those disciplines ruthlessly, every line of code, every condition, and every branch is tested.

Độ phủ test 100% hàm ý rằng bất kỳ thay đổi ngữ nghĩa nào trong mã cũng phải làm một test nào đó fail. TDD, TCR, và ở mức độ nhẹ hơn là Small Bundles, là những kỷ luật (discipline) tốt để tiệm cận mục tiêu ấy, bởi nếu bạn theo chúng một cách nghiêm ngặt thì mọi dòng mã, mọi điều kiện và mọi nhánh đều được kiểm thử.

Such ruthlessness, however, is often impractical. Programmers are human, and disciplines are always subject to pragmatics. So the reality is that even the most assiduous test-driven developer will leave gaps in the test coverage.

Tuy nhiên, sự nghiêm ngặt đến mức ấy thường bất khả thi trong thực tế. Lập trình viên cũng là con người, và kỷ luật thì lúc nào cũng phải nhân nhượng với thực dụng. Nên thực tế là ngay cả người làm TDD chăm chỉ nhất cũng sẽ để lại những lỗ hổng trong độ phủ test.

Mutations testing is a way to find those gaps; and there are mutation testing tools that can help.

Mutation testing là một cách để tìm ra những lỗ hổng đó; và đã có sẵn các công cụ mutation testing giúp bạn làm việc này.

A mutation tester runs your **test suite** and measures the coverage. Then, it goes into a loop, mutating your code in some semantic way and then running the test suite with coverage again. The semantic changes are things like changing `>` to `<` or `==` to `!=` or `x=< something>` to `x=null`. Each such semantic change is called a mutation.

Một mutation tester sẽ chạy bộ test (test suite) của bạn và đo độ phủ. Sau đó nó bước vào một vòng lặp: biến đổi mã của bạn theo một cách nào đó về mặt ngữ nghĩa, rồi lại chạy bộ test kèm đo độ phủ. Những thay đổi ngữ nghĩa đó là kiểu như đổi `>` thành `<`, `==` thành `!=`, hay `x=<cái gì đó>` thành `x=null`. Mỗi thay đổi ngữ nghĩa như vậy được gọi là một *mutation*.

The tool expects each mutation to fail the tests. Mutations that do not fail the tests are called surviving mutations. Clearly, the goal is to ensure that there are no surviving mutations.

Công cụ kỳ vọng mỗi mutation đều làm test fail. Những mutation không làm fail test được gọi là *surviving mutation* (mutation sống sót). Rõ ràng, mục tiêu là bảo đảm không còn mutation nào sống sót.

Running a mutation test can be a big investment of time. Even relatively small systems can require hours of runtime. So these kinds of tests are best run over weekends, or at month's end.

Chạy một lượt mutation test có thể tốn rất nhiều thời gian. Ngay cả hệ thống tương đối nhỏ cũng có thể cần hàng giờ chạy máy. Vậy nên loại test này tốt nhất là chạy vào cuối tuần, hoặc vào cuối tháng.

I have frequently been impressed by the kinds of subtle problems these tools have uncovered. So it is definitely worth the occasional effort.

Tôi đã nhiều lần phải trầm trồ trước những vấn đề tinh vi mà các công cụ này moi ra được. Nên chắc chắn thỉnh thoảng bỏ công ra làm là xứng đáng.

Semantic Stability — Ổn định ngữ nghĩa

The goal of test coverage and mutation testing is to create a test suite that ensures semantic stability. The semantics of a system are the required behaviors of that system. A test suite that ensures semantic stability is one that fails whenever a required **behavior** is broken. We use such test suites to eliminate the fear of **refactoring** and cleaning the code. Without a semantically stable test suite, the fear of change is often too great.

Mục tiêu của độ phủ test và mutation testing là tạo ra một bộ test bảo đảm sự ổn định ngữ nghĩa (semantic stability). Ngữ nghĩa của một hệ thống chính là những hành vi mà hệ thống đó buộc phải có. Một bộ test bảo đảm ổn định ngữ nghĩa là bộ test sẽ fail mỗi khi một hành vi bắt buộc bị phá vỡ. Chúng ta dùng những bộ test như thế để xóa đi nỗi sợ refactor và dọn dẹp mã. Không có một bộ test ổn định về ngữ nghĩa, nỗi sợ thay đổi thường lớn đến mức không vượt qua nổi.

Following one of the testing disciplines gives us a good start on a semantically stable test suite; but none of those disciplines is sufficient for full semantic stability. Coverage, mutation testing, and acceptance testing should be used to improve the semantic stability toward completeness.

Theo một trong các kỷ luật kiểm thử sẽ cho ta khởi đầu tốt trên đường tới một bộ test ổn định về ngữ nghĩa; nhưng không kỷ luật nào trong số đó là đủ để đạt ổn định ngữ nghĩa trọn vẹn. Độ phủ, mutation testing và acceptance test (kiểm thử chấp nhận) nên được dùng để đẩy mức ổn định ngữ nghĩa tiến dần tới hoàn chỉnh.

Cleaning — Dọn dẹp

Perhaps the most effective of the random acts of kindness that will improve the code is simple cleaning—refactoring with the goal of improvement.

Có lẽ hành động tử tế ngẫu nhiên hiệu quả nhất để cải thiện mã chính là việc dọn dẹp giản dị — refactor với mục tiêu làm cho mọi thứ tốt hơn.

What kinds of improvements can be made? There is, of course, the obvious elimination of code smells. But I will often **clean code** even when it isn't smelly.

Có thể cải thiện được những gì? Dĩ nhiên có chuyện hiển nhiên là loại bỏ các code smell. Nhưng tôi vẫn thường dọn mã ngay cả khi nó chẳng bốc mùi gì.

I make tiny little improvements in the names, in the **structure**, in the organization. These changes might not be noticed by anyone else. Some folks might even think they make the code less clean. But my goal is not simply the state of the code. By doing minor little cleanings, I learn about the code. I become more familiar and more comfortable with it. Perhaps my cleaning did not actually improve the code in any objective sense; but it improved my understanding of and facility with that code. The cleaning improved me as a developer of that code.

Tôi cải thiện từng chút xíu ở cách đặt tên, ở cấu trúc, ở cách tổ chức. Những thay đổi này có thể chẳng ai khác nhận ra. Vài người thậm chí còn cho rằng chúng làm mã kém sạch đi. Nhưng mục tiêu của tôi không đơn thuần là trạng thái của mã. Bằng cách dọn dẹp lật vật, tôi học được về mã. Tôi trở nên quen thuộc và thoải mái hơn với nó. Có lẽ việc dọn dẹp của tôi chẳng cải thiện mã theo bất kỳ nghĩa khách quan nào; nhưng nó cải thiện hiểu biết của tôi về mã đó và sự thuần thục của tôi với nó. Việc dọn dẹp đã cải thiện chính tôi với tư cách người phát triển đoạn mã ấy.

The cleaning also did something else that should not be understated. By cleaning the code, even in minor ways, I am flexing that code. And one of the best ways to ensure that code stays flexible is to regularly flex it. Every little bit of cleaning I do is actually a test of the code's flexibility. If I find a small cleanup to be a bit difficult, I have detected an area of inflexibility that I can now correct.

Việc dọn dẹp còn làm một chuyện nữa mà ta không nên xem nhẹ. Khi dọn mã, dù chỉ theo những cách nhỏ nhặt, tôi đang *co duỗi* đoạn mã đó. Và một trong những cách tốt nhất để bảo đảm mã luôn dẻo dai là thường xuyên bắt nó co duỗi. Mỗi mẫu dọn dẹp bé tí tôi làm thực ra là một phép thử độ dẻo của mã. Nếu tôi thấy một pha dọn dẹp nhỏ mà hơi vất vả, tức là tôi vừa phát hiện ra một chỗ cứng nhắc, và giờ tôi có thể sửa nó.

Remember, software is supposed to be soft. How do you know that it is soft? By testing that softness on a regular basis. By doing little cleanups and little improvements and feeling how easy or difficult those changes are to make.

Hãy nhớ, phần mềm (software) đúng ra phải *mềm*. Làm sao bạn biết nó mềm? Bằng cách kiểm tra độ mềm ấy một cách đều đặn. Bằng cách dọn dẹp vụn vặt, cải thiện vụn vặt, rồi

cảm nhận xem những thay đổi ấy dễ hay khó.

Creations — Những gì ta tạo ra

The word used in the promise above is creations . In this chapter I have focused mostly on code; but code is not the only thing that programmers create. We create designs, and documents, and schedules, and plans. All these artifacts are creations that should be continuously improved.

Từ được dùng trong lời hứa ở đầu chương là *creations* — những gì ta tạo ra. Trong chương này tôi chủ yếu tập trung vào mã; nhưng mã không phải thứ duy nhất lập trình viên tạo ra. Chúng ta tạo ra thiết kế, tài liệu, lịch trình và kế hoạch. Tất cả những sản phẩm ấy đều là những gì ta tạo ra, và đều đáng được cải tiến liên tục.

We are human beings. Human beings make things better with time. We constantly improve everything we work on.

Chúng ta là con người. Con người làm mọi thứ tốt lên theo thời gian. Chúng ta không ngừng cải tiến mọi thứ mình chạm tay vào.

Chapter 33. Maintain High Productivity — Chương 33. Duy trì năng suất cao

6. I will do all that I can to keep the **productivity** of myself and others as high as possible. I will do nothing that decreases that productivity.

6. Tôi sẽ làm tất cả những gì có thể để giữ năng suất của bản thân và của người khác ở mức cao nhất. Tôi sẽ không làm bất cứ điều gì làm giảm năng suất đó.

Productivity. That's quite a topic, isn't it. How often do you feel that it's the only thing that matters at your job? If you think about it, productivity is what this book, and what all my books on software, is about.

Năng suất. Đúng là một chủ đề đáng bàn, phải không? Bạn có hay cảm thấy rằng đó là thứ duy nhất đáng kể trong công việc của mình không? Nếu ngẫm kỹ, năng suất chính là điều mà cuốn sách này, và tất cả những cuốn sách về phần mềm của tôi, hướng tới.

They're about how to go fast.

Chúng nói về chuyện làm sao để đi nhanh.

And what we've learned over the last eight decades of software is that the way you go fast is to go well.

Và điều chúng ta đã học được suốt tám thập kỷ qua trong ngành phần mềm là: cách để đi nhanh chính là đi cho tử tế.

The only way to go fast is to go well.

Cách duy nhất để đi nhanh là đi cho tử tế.

So you keep your code clean. You keep your designs clean. You write semantically stable tests and keep your coverage high. You know and use appropriate design patterns. You keep your methods small and your names precise.

Vậy nên bạn giữ cho mã của mình sạch. Bạn giữ cho các thiết kế của mình sạch. Bạn viết những test ổn định về mặt ngữ nghĩa và giữ độ phủ test (test coverage) ở mức cao. Bạn biết và dùng đúng các design pattern phù hợp. Bạn giữ cho các method của mình nhỏ và tên gọi thì chính xác.

But those are all indirect methods of achieving productivity. In this chapter, we're going to talk about much more direct ways to keep productivity high:

Nhưng tất cả những thứ đó đều là cách gián tiếp để đạt được năng suất. Trong chương này, chúng ta sẽ bàn về những cách trực tiếp hơn nhiều để giữ năng suất ở mức cao:

- Viscosity: Keeping your development environment efficient
- Managing distractions: Dealing with everyday business and personal life
- Time management: Effectively separating productive time from all the other junk you have to do
 - Độ nhớt (viscosity): Giữ cho môi trường phát triển của bạn hiệu quả
 - Quản lý các thứ gây xao nhãng: Đối phó với chuyện công việc và chuyện đời thường hằng ngày
 - Quản lý thời gian: Tách bạch hiệu quả thời gian làm việc thật sự khỏi mọi thứ tạp nham khác mà bạn phải làm

Viscosity — Độ nhớt

Programmers are often very myopic when it comes to productivity. They view the primary **component** to productivity as their ability to write code quickly.

Lập trình viên thường rất thiếu cận khi nói đến năng suất. Họ xem thành phần chính của năng suất là khả năng viết mã thật nhanh.

But the writing of code is a very small part of the overall process. If you made the writing of code infinitely fast, it would increase overall productivity by only a small amount.

Nhưng việc viết mã chỉ là một phần rất nhỏ trong toàn bộ quy trình. Nếu bạn làm cho việc viết mã nhanh vô hạn, năng suất tổng thể cũng chỉ tăng lên chút xíu.

That's because there's a lot more to the software process than just writing code. There's at least

Đó là bởi quy trình phần mềm còn nhiều thứ khác chứ không chỉ có viết mã. Ít nhất còn có

- Building
- Testing
- Debugging
- Deploying
 - Build
 - Test
 - Debug
 - Deploy

And that doesn't count the requirements, the analysis, the design, the meetings, the research, the infrastructure, the tooling, and all the other stuff that goes into a software project.

Và đó là còn chưa tính đến yêu cầu, phân tích, thiết kế, các cuộc họp, việc nghiên cứu, hạ tầng, công cụ, cùng mọi thứ khác đổ vào một dự án phần mềm.

So, although it is important to be able to write code efficiently, it's not even close to the biggest part of the problem. Let's tackle some of the other issues one at a time.

Vậy nên, dù viết mã hiệu quả là điều quan trọng, nó còn lâu mới là phần lớn nhất của vấn đề. Hãy xử lý từng vấn đề còn lại, mỗi lần một thứ.

Building — Build

If it takes you 30 minutes to build after a 5-minute edit, then you can't be very productive, can you?

Nếu bạn phải mất 30 phút để build sau khi sửa mã trong 5 phút, thì bạn khó mà có năng suất cao được, đúng không?

There is no reason, in the third and subsequent decades of the twenty-first century, that builds should take more than a minute or two.

Chẳng có lý do gì, trong thập niên thứ ba của thế kỷ 21 và những thập niên sau đó, mà một lần build lại phải tốn hơn một hai phút.

Before you **object** to this, think about it. How could you speed up the build? Are you utterly certain, in this age of cloud computing, that there is no way to dramatically speed up your build? Find whatever is causing the build to be slow, and fix it. Consider it a design challenge.

Trước khi bạn phản đối điều này, hãy nghĩ kỹ đã. Làm sao để tăng tốc quá trình build? Bạn có chắc chắn tuyệt đối rằng, trong thời đại điện toán đám mây này, không có cách nào để tăng tốc build một cách đáng kể hay không? Hãy tìm ra bất cứ thứ gì đang làm build chậm, rồi sửa nó. Hãy coi đó là một bài toán thiết kế.

Testing — Test

Is it your tests that are slowing the build? Same answer: Speed up your tests. Here, look at it this way. My poor little laptop has four cores running at a clock rate of 2.8GHz. That means it can execute around 10 billion instructions per second.

Có phải chính các test đang làm chậm build? Câu trả lời vẫn thế: Hãy tăng tốc các test. Đây, hãy nhìn theo cách này. Cái laptop tội nghiệp bé nhỏ của tôi có bốn nhân chạy ở xung nhịp 2,8GHz. Nghĩa là nó có thể thực thi khoảng 10 tỉ lệnh mỗi giây.

Do you even have 10 billion instructions in your whole system? If not, then you should be able to **test** your whole system in less than a second.

Toàn bộ hệ thống của bạn liệu có nổi 10 tỉ lệnh không? Nếu không, thì lẽ ra bạn phải test được cả hệ thống trong chưa đầy một giây.

Unless, of course, you are executing some of those instructions more than once.

Trừ phi, dĩ nhiên, bạn đang thực thi một số lệnh trong đó nhiều hơn một lần.

For example, how many times do you have to test login to know that it works? Generally speaking, once should be sufficient.

Ví dụ, bạn phải test chức năng login bao nhiêu lần thì mới biết là nó chạy được? Nói chung, một lần là đủ.

So how many of your tests go through the login process? Any more than one would be a waste!

Vậy có bao nhiêu test của bạn đi qua quy trình login? Nhiều hơn một là phí phạm rồi!

If login is required before each test, then, when executing tests, you should short-circuit the login process. Use one of the mocking patterns. Or, if you must, remove the login process from systems built for testing.

Nếu bắt buộc phải login trước mỗi test, thì khi chạy test, bạn nên đi tắt qua quy trình login. Hãy dùng một trong các mẫu mocking. Hoặc, nếu buộc phải làm vậy, hãy gỡ hẳn quy trình login khỏi các bản build dựng riêng cho việc test.

The point is, don't tolerate repetition like that in your tests. It can make them horrifically slow.

Vấn đề là: đừng dung thứ cho kiểu lặp lại như thế trong test của bạn. Nó có thể làm test chậm đến kinh hoàng.

As another example, how many times do your tests walk through the navigation and menu **structure** of the user **interface**? How many tests start at the top and then walk through a long chain of links to finally get the system into the state where the test can be run?

Một ví dụ khác: các test của bạn phải đi qua cấu trúc điều hướng và menu của giao diện người dùng bao nhiêu lần? Có bao nhiêu test bắt đầu từ trên cùng rồi lần theo một chuỗi dài các liên kết chỉ để cuối cùng đưa hệ thống về trạng thái mà test có thể chạy được?

Any more than once per navigation pathway would be a waste! So build a special testing API that allows the tests to quickly force the system into the state you need, without logging in, and without navigating.

Nhiều hơn một lần cho mỗi đường điều hướng là phí phạm! Vậy nên hãy dựng một API dành riêng cho việc test, cho phép các test nhanh chóng ép hệ thống về trạng thái bạn cần, không phải login, không phải điều hướng.

How many times do you have to execute a query to know that it works? Once! So **mock** out your databases for the majority of your tests. Don't allow the same queries to be executed over and over and over again.

Bạn phải chạy một truy vấn bao nhiêu lần thì mới biết nó chạy đúng? Một lần! Vậy nên hãy mock database cho phần lớn các test của bạn. Đừng để cùng những truy vấn ấy bị thực thi đi thực thi lại hết lần này đến lần khác.

Peripheral devices are slow. Disks are slow. Web sockets are slow. UI screens are slow. Even SSDs can be slow. Don't let slow things slow down your tests. Mock them out. Bypass them. Get them out of the critical path of your tests.

Thiết bị ngoại vi thì chậm. Đĩa cứng thì chậm. Web socket thì chậm. Màn hình UI thì chậm. Ngay cả SSD cũng có thể chậm. Đừng để những thứ chậm chạp làm chậm test của bạn. Hãy mock chúng đi. Hãy đi vòng qua chúng. Hãy lôi chúng ra khỏi đường găng của các test.

Don't tolerate slow tests. Keep your tests running fast!

Đừng dung thứ cho test chậm. Hãy giữ cho test của bạn chạy thật nhanh!

Debugging — Debug

Does it take a long time to debug things? Why? Why is debugging slow?

Việc debug có mất nhiều thời gian không? Tại sao? Tại sao debug lại chậm?

You are using a testing **discipline** and writing unit tests, aren't you? You are writing acceptance tests too, right? And you are measuring **test coverage** with a **good** coverage analysis tool, right? And you are periodically proving that your tests are semantically stable by using a mutation tester, right?

Bạn đang theo một kỷ luật (discipline) kiểm thử và viết unit test đấy chứ? Bạn cũng viết cả acceptance test (kiểm thử chấp nhận) nữa, phải không? Và bạn đang đo độ phủ test bằng

một công cụ phân tích độ phủ tử tế, đúng không? Và bạn định kỳ chứng minh rằng các test của mình ổn định về mặt ngữ nghĩa bằng cách dùng mutation tester, đúng không?

If you are doing all those things, or even just some of those things, your debug time can be reduced to insignificance.

Nếu bạn làm tất cả những điều đó, hoặc thậm chí chỉ một vài trong số đó, thời gian debug của bạn có thể giảm xuống mức không đáng kể.

Deploying — Deploy

Does deployment take forever? Why? I mean, you are using deployment scripts, right? You aren't deploying manually, are you?

Việc triển khai có kéo dài lê thê không? Tại sao? Ý tôi là, bạn đang dùng script triển khai đấy chứ? Bạn không triển khai bằng tay chứ hả?

Remember, you are programmers. Deployment is a procedure; automate it! And write tests for that procedure too! You should be able to deploy your system, every time, with a single click.

Nhớ nhé, các bạn là lập trình viên. Triển khai là một quy trình; hãy tự động hoá nó! Và hãy viết test cho cái quy trình đó luôn! Bạn phải triển khai được hệ thống của mình, mỗi lần, chỉ bằng một cú nhấp chuột.

Managing Distractions — Quản lý các thứ gây xao nhãng

One of the most pernicious destroyers of productivity is distraction from the job. There are many different kind of distractions. It is important for you to know how to recognize them and defend against them.

Một trong những kẻ huỷ diệt năng suất tai hại nhất là sự xao nhãng khỏi công việc. Có rất nhiều kiểu xao nhãng khác nhau. Điều quan trọng là bạn phải biết cách nhận ra và phòng vệ trước chúng.

Meetings — Họp hành

Are you slowed down by meetings? I have a very simple rule for dealing with meetings. It goes like this:

Bạn có bị họp hành làm chậm lại không? Tôi có một quy tắc rất đơn giản để đối phó với các cuộc họp. Nó như thế này:

When the meeting gets boring, leave.

Khi cuộc họp trở nên nhàm chán, hãy rời đi.

You should be polite about it. Wait a few minutes for a lull in the conversation, and then tell the participants that you believe your input is no longer required and ask them if they would mind if you returned to the rather large amount of work you have to do.

Bạn nên lịch sự khi làm việc đó. Chờ vài phút cho tới lúc câu chuyện chùng xuống, rồi nói với những người tham dự rằng bạn thấy ý kiến của mình không còn cần thiết nữa, và hỏi xem họ có phiền không nếu bạn quay lại với đồng công việc khá đồ sộ đang chờ mình.

Never be afraid to leave a meeting. If you don't figure out how to leave, then some meetings will keep you forever.

Đừng bao giờ ngại rời khỏi một cuộc họp. Nếu bạn không tìm ra cách rời đi, sẽ có những cuộc họp giữ chân bạn mãi mãi.

You would also be wise to decline most meeting invitations. The best way to avoid getting caught in long, boring meetings is to politely refuse the invitation in the first place.

Bạn cũng nên khôn ngoan mà từ chối phần lớn các lời mời họp. Cách tốt nhất để tránh bị mắc kẹt trong những cuộc họp dài lê thê và nhàm chán là lịch sự từ chối lời mời ngay từ đầu.

Don't be seduced by the fear of missing out. If you are truly needed, they'll come get you.

Đừng để nỗi sợ bỏ lỡ dụ dỗ. Nếu người ta thật sự cần bạn, họ sẽ đến tận nơi lôi bạn đi.

When someone invites you to a meeting, make sure they've convinced you that you really need to go. Make sure they understand that you can only afford a few minutes and that you are likely to leave before the meeting is over.

Khi ai đó mời bạn dự họp, hãy chắc rằng họ đã thuyết phục được bạn rằng bạn thật sự cần có mặt. Hãy chắc rằng họ hiểu bạn chỉ có thể dành ra vài phút và nhiều khả năng bạn sẽ rời đi trước khi cuộc họp kết thúc.

And make sure you sit close to the door.

Và nhớ ngồi gần cửa.

And if you are a group leader or a manager, remember that one of your primary duties is to defend your team's productivity by keeping them out of meetings .

Còn nếu bạn là trưởng nhóm hay quản lý, hãy nhớ rằng một trong những nhiệm vụ hàng đầu của bạn là bảo vệ năng suất của đội bằng cách giữ cho họ không phải dự họp.

Music — Âm nhạc

I used to code to music; long, long ago. But I found that listening to music impedes my concentration. Over time, I realized that listening to music only feels like it helps me concentrate, when actually it divides my attention.

Ngày xưa tôi từng vừa nghe nhạc vừa code; lâu, lâu lắm rồi. Nhưng tôi nhận ra rằng nghe nhạc cản trở sự tập trung của mình. Theo thời gian, tôi hiểu ra rằng nghe nhạc chỉ *có cảm giác* như giúp tôi tập trung, còn thực tế thì nó chia nhỏ sự chú ý của tôi.

One day, while looking over some year-old code, I realized that my code was suffering under the lash of the music. There, scattered through the code in a series of comments, were the lyrics to the song I had been listening to.

Một hôm, khi xem lại đoạn mã viết từ năm trước, tôi nhận ra mã của mình đang chịu roi vọt của âm nhạc. Rải rác khắp đoạn mã, trong một loạt comment, là lời của bài hát mà tôi đã nghe lúc đó.

Since then, I've stopped listening to music while I code, and I've found I am much happier with the code I write and with the attention to detail that I can give it.

Từ dạo ấy, tôi thôi nghe nhạc trong lúc code, và tôi thấy mình hài lòng hơn nhiều với mã mình viết ra cũng như với mức độ tỉ mỉ mà tôi dành cho nó.

Programming is the act of arranging elements of procedure through sequence, selection, and iteration.

Lập trình là việc sắp đặt các thành phần của thủ tục thông qua tuần tự, rẽ nhánh và lặp.

Music is composed of tonal and rhythmic elements arranged through sequence, selection, and iteration.

Âm nhạc được tạo thành từ các thành phần âm điệu và tiết tấu, được sắp đặt thông qua tuần tự, chọn lựa và lặp.

Could it be that listening to music uses the same parts of your brain that programming uses, thereby consuming part of your programming ability? That's my theory, and I'm sticking to it.

Liệu có phải nghe nhạc dùng chính những vùng não mà lập trình dùng, do đó ngốn mất một phần khả năng lập trình của bạn? Đó là giả thuyết của tôi, và tôi vẫn giữ nguyên nó.

You will have to work this out for yourselves. Maybe the music really does help you. But maybe it doesn't. I'd advise you to try coding without music for a week, and see if you don't end up producing more and better code.

Bạn sẽ phải tự mình tìm ra câu trả lời. Có thể âm nhạc thật sự giúp ích cho bạn. Nhưng cũng có thể không. Tôi khuyên bạn thử code không nhạc trong một tuần, xem có phải cuối cùng bạn viết được nhiều mã hơn và tốt hơn không.

Mood — Tâm trạng

It's important to realize that being productive requires that you become skilled at managing your emotional state. Emotional stress can kill your ability to code. It can break your concentration and keep you in a perpetually distracted state of mind.

Cần nhận ra rằng muốn có năng suất thì bạn phải trở nên thành thạo trong việc quản lý trạng thái cảm xúc của mình. Căng thẳng cảm xúc có thể giết chết khả năng code của bạn. Nó có thể phá vỡ sự tập trung và giữ bạn trong trạng thái tâm trí xao nhãng triền miên.

For example, have you ever noticed that you can't code after a huge fight with your significant other? 1 Oh, maybe you type a few random characters in your IDE, but they don't amount to much. Perhaps you pretend to be productive by hanging out in some boring meeting that you don't have to pay much attention to.

Chẳng hạn, bạn có bao giờ để ý rằng mình không thể code nổi sau một trận cãi vã nảy lửa với người thương? 1 Ồ, có lẽ bạn cũng gõ được vài ký tự vu vơ trong IDE, nhưng chẳng ra cái gì. Có khi bạn còn giả vờ làm việc năng suất bằng cách la cà trong một cuộc họp nhằm chán nào đó mà bạn chẳng cần chú ý mấy.

1 One of the wisest attitudes I've encountered regarding relationships came from the movie *The Comancheros* (1961). John Wayne's character said: "Me and my wife used to fight like a couple of wildcats with only one tree between us. But you soon find out that it doesn't make a tinker's damn who's got the upper hand. A few years roll by and you kinda settle down to being at ease with each other. Then life gets worth living."

1 Một trong những thái độ khôn ngoan nhất về chuyện tình cảm mà tôi từng gặp đến từ bộ phim *The Comancheros* (1961). Nhân vật của John Wayne nói: "Tôi với vợ tôi từng cãi nhau như hai con mèo rừng chỉ có độc một cái cây để tranh. Nhưng rồi anh sẽ sớm nhận ra rằng ai thắng thế cũng chẳng đáng một xu. Vài năm trôi qua, anh dần lắng lại và thấy thoải mái bên nhau. Rồi thì cuộc đời mới đáng sống."

Here's what I've found works best to restore me to productivity.

Đây là những gì tôi thấy hiệu quả nhất để đưa mình trở lại trạng thái làm việc năng suất.

Act. Act on the root of the emotion. Don't try to code. Don't try to cover the feelings with music or meetings. It won't work. Act to resolve the emotion.

Hãy hành động. Hành động vào gốc rễ của cảm xúc. Đừng cố code. Đừng cố phủ lấp cảm xúc bằng âm nhạc hay họp hành. Không ăn thua đâu. Hãy hành động để giải toả cảm xúc đó.

If you find yourself at work too sad or depressed to code because of a fight with your spouse, then call them to try to resolve the issue. Even if you don't actually get the issue resolved, you'll find that the action to attempt a resolution will sometimes clear your mind well enough to code.

Nếu bạn thấy mình ở chỗ làm mà buồn bã hay chán nản đến mức không code nổi vì vừa cãi nhau với bạn đời, thì hãy gọi cho họ để cố gắng giải quyết chuyện đó. Ngay cả khi bạn không thật sự giải quyết được, bạn sẽ thấy rằng chính hành động cố gắng hoà giải đôi khi cũng đủ làm đầu óc bạn quang đãng để mà code.

You don't actually have to solve the problem. All you have to do is convince yourself that you've taken enough appropriate action. I usually find that's enough to let me redirect my thoughts to the code I have to write.

Bạn không cần thật sự giải quyết được vấn đề. Tất cả những gì bạn cần là thuyết phục được chính mình rằng mình đã hành động đủ và đúng mực. Tôi thường thấy chừng đó là đủ để hướng suy nghĩ của mình trở lại với đoạn mã cần viết.

The Flow — Trạng thái flow

There's an altered state of mind that many programmers enjoy. It's that hyper-focused, tunnel vision state where the code seems to pour out of every orifice of your body. It can make you feel superhuman.

Có một trạng thái tinh thần khác thường mà nhiều lập trình viên rất khoái. Đó là trạng thái siêu tập trung, tầm nhìn thu hẹp như đường hầm, khi mã dường như tuôn ra từ mọi lỗ chân lông trên người bạn. Nó có thể khiến bạn thấy mình như siêu nhân.

Despite the euphoric sensation, I've found over the years that the code I produce in that altered state tends to be pretty bad. The code is not nearly as well considered as code I write in a normal state of attention and focus.

Bất chấp cảm giác lâng lâng ấy, qua nhiều năm tôi nhận ra rằng mã tôi viết ra trong trạng thái khác thường đó thường khá tệ. Nó không được cân nhắc kỹ càng bằng mã tôi viết trong trạng thái chú ý và tập trung bình thường.

So nowadays, I resist getting into the flow.

Vậy nên ngày nay, tôi kháng cự lại việc rơi vào trạng thái flow.

Pairing is a very good way to stay out of the flow. The very fact that you must communicate and collaborate with someone else seems to interfere with the flow.

Pair programming là một cách rất tốt để tránh xa trạng thái flow. Chính việc bạn phải giao tiếp và cộng tác với người khác dường như đã cản trở flow.

Avoiding music also helps me stay out of the flow, because it allows the actual environment to keep me grounded in the real world.

Tránh nghe nhạc cũng giúp tôi ở ngoài trạng thái flow, vì nó cho phép môi trường thật xung quanh neo giữ tôi lại với thế giới thực.

If I find that I'm starting to hyper focus, I'll break away and do something else for a while.

Nếu tôi thấy mình bắt đầu siêu tập trung, tôi sẽ dứt ra và làm chuyện gì khác một lúc.

Time Management — Quản lý thời gian

One of the most important ways to manage distraction is to employ a time management discipline. The one I like best is the Pomodoro Technique. 2

Một trong những cách quan trọng nhất để quản lý sự xao nhãng là áp dụng một kỷ luật quản lý thời gian. Cách tôi thích nhất là Pomodoro Technique. 2

2 [Pomodoro].

2 [Pomodoro].

Pomodoro is Italian for tomato. Indeed, English-speaking teams tend to use the word tomato instead. But you'll have better luck with Google if you search for "Pomodoro Technique."

Pomodoro trong tiếng Ý nghĩa là quả cà chua. Quả thật, các đội nói tiếng Anh thường dùng luôn từ *tomato*. Nhưng bạn sẽ tra Google dễ hơn nếu tìm "Pomodoro Technique".

The aim of the technique is to help you manage your time and focus during a regular workday. It doesn't concern itself with anything beyond that.

Mục tiêu của kỹ thuật này là giúp bạn quản lý thời gian và sự tập trung trong một ngày làm việc bình thường. Nó không quan tâm đến bất cứ điều gì ngoài phạm vi đó.

At its core, the idea is quite simple. Before you begin to work, you set a timer (traditionally a kitchen timer in the shape of a tomato) for 25 minutes.

Về cốt lõi, ý tưởng khá đơn giản. Trước khi bắt đầu làm việc, bạn hẹn giờ (theo truyền thống là một chiếc đồng hồ bếp hình quả cà chua) 25 phút.

Next, you work. And you work until the timer rings.

Tiếp theo, bạn làm việc. Và bạn làm cho tới khi chuông reo.

Then, you break for five minutes, clearing your mind and body.

Rồi bạn nghỉ năm phút, để đầu óc và cơ thể được thanh thoi.

Then, you start again. Set the timer for 25 minutes, work until the timer rings, and then break for 5. And you do this over and over.

Rồi bạn bắt đầu lại. Hẹn giờ 25 phút, làm việc cho tới khi chuông reo, rồi nghỉ 5 phút. Và cứ thế lặp đi lặp lại.

There's nothing magical about 25 minutes. I'd say anything between 15 and 45 minutes is reasonable. But once you choose a time, stick with that time for a while.

Chẳng có gì huyền diệu ở con số 25 phút cả. Tôi cho rằng khoảng nào từ 15 đến 45 phút đều hợp lý. Nhưng một khi đã chọn một mốc thời gian, hãy giữ nguyên mốc đó một thời gian.

Of course, if I were 30 seconds away from getting a test to pass when the timer rang, I'd finish the test. On the other hand, maintaining the discipline is important. I wouldn't go more than a minute beyond.

Dĩ nhiên, nếu chuông reo lúc tôi chỉ còn cách việc làm cho một test pass đúng 30 giây, tôi sẽ làm nốt cái test đó. Mặt khác, giữ được kỷ luật vẫn là điều quan trọng. Tôi sẽ không kéo dài quá một phút.

So far, this sounds mundane, but handling interruptions, like phone calls, is where this technique shines. The rule is to Defend the Tomato!

Đến đây nghe vẫn còn tầm thường, nhưng chỗ kỹ thuật này toả sáng là ở cách xử lý những lần bị ngắt quãng, chẳng hạn các cuộc gọi điện thoại. Quy tắc là: Hãy bảo vệ Quả Cà Chua!

Tell whoever is trying to interrupt you that you'll get back to them within 25 minutes—or whatever the length of your tomato is. Dispatch the interruption as quickly as possible, and then return to work.

Hãy nói với người đang định cắt ngang bạn rằng bạn sẽ quay lại với họ trong vòng 25 phút — hoặc bất kỳ độ dài nào mà quả cà chua của bạn có. Hãy giải quyết cú ngắt quãng đó nhanh nhất có thể, rồi quay lại làm việc.

Then, after your break, handle the interruption.

Rồi, sau giờ nghỉ, hãy xử lý chuyện bị ngắt quãng đó.

This means that the time between tomatoes will sometimes get pretty long, because people who interrupt you often require a lot of time.

Điều này có nghĩa là khoảng cách giữa các quả cà chua đôi khi sẽ khá dài, bởi những người cắt ngang bạn thường tốn khá nhiều thời gian.

Again, that's the beauty of this technique. Because, at the end of the day, you count the number of tomatoes you completed. And that gives you a measure of your productivity.

Một lần nữa, đó chính là cái hay của kỹ thuật này. Bởi vì, đến cuối ngày, bạn đếm số quả cà chua mình đã hoàn thành. Và con số đó cho bạn một thước đo về năng suất của mình.

Once you've gotten good at breaking your day up into tomatoes like this, and defending the tomatoes from interruptions, then you can start planning your day by allocating tomatoes to it. You may even begin estimating your tasks in terms of tomatoes and planning your meetings and lunches around them.

Một khi bạn đã quen với việc chia ngày làm việc thành những quả cà chua như thế, và bảo vệ chúng khỏi bị cắt ngang, bạn có thể bắt đầu lên kế hoạch cho ngày của mình bằng cách phân bổ số quả cà chua cho nó. Bạn thậm chí có thể bắt đầu ước lượng (estimate) các đầu việc theo đơn vị quả cà chua, và sắp xếp các cuộc họp cùng bữa trưa xoay quanh chúng.

Chapter 34. Work as a Team —

Chương 34. Làm việc như một đội

7. I will continuously ensure that others can cover for me and that I can cover for them.

7. Tôi sẽ liên tục bảo đảm rằng người khác có thể gánh việc thay tôi, và tôi có thể gánh việc thay họ.

Imagine a team of soccer players moving the ball down the field toward the goal. Imagine that, during a play, one of the players trips and falls down. What do his teammates do? They step into the breach, cover the hole, and keep the ball moving down the field .

Hãy hình dung một đội bóng đang đưa bóng tiến về phía khung thành. Hãy hình dung giữa pha bóng đó, một cầu thủ vấp ngã. Đồng đội của anh ta làm gì? Họ lấp vào chỗ trống, bịt lỗ hổng, và tiếp tục đưa bóng lên phía trước.

The sailors on a navy vessel each have a job to do, but they also know how to do someone else's job. Why? Because in the heat of a battle, a sailor might go down. Another sailor must be able to step into the breach and keep the ship in the battle .

Trên một chiến hạm, mỗi thủy thủ đều có phần việc của mình, nhưng họ cũng biết cách làm việc của người khác. Tại sao? Bởi giữa lúc chiến trận nóng bỏng, một thủy thủ có thể gục xuống. Một thủy thủ khác phải lấp được vào chỗ trống đó để con tàu vẫn trụ được trong trận đánh.

It is no different on a software team. Indeed, that's why we call it a team . If one of the members of the team goes down for any reason, others on the team must be able to step into the breach, fill that hole, and keep the project on track.

Trong một đội phần mềm cũng chẳng khác gì. Thật vậy, đó chính là lý do ta gọi nó là một *đội*. Nếu một thành viên vì bất cứ lý do gì mà gục xuống, những người còn lại phải lấp được vào chỗ trống ấy, bịt lỗ hổng đó, và giữ cho dự án đi đúng hướng.

Collaborative Programming — Lập trình cộng tác

Segregation of knowledge into silos is extremely detrimental to a team and an organization. The loss of an individual can mean the loss of an entire segment of knowledge. It can paralyze the team and the organization. It also means that the individuals on the team don't have sufficient context to understand each other. Often, they wind up talking past each other.

Việc chia cắt tri thức thành những ốc đảo riêng biệt gây hại vô cùng cho cả đội lẫn tổ chức. Mất một cá nhân có thể đồng nghĩa với mất trọn một mảng tri thức. Nó có thể làm tê liệt cả

đội và cả tổ chức. Nó còn khiến các cá nhân trong đội không có đủ bối cảnh để hiểu nhau. Thường thì họ nói mà chẳng ai chạm được vào ý của ai.

The cure for that is to spread knowledge through the team. Make sure each team member knows a lot about the work that other team members are performing.

Thuốc chữa cho căn bệnh đó là lan toả tri thức khắp đội. Hãy chắc chắn rằng mỗi thành viên đều biết khá nhiều về công việc mà các thành viên khác đang làm.

And the best way to spread that knowledge is to work together —to pair , or mob .

Và cách tốt nhất để lan toả tri thức ấy là *làm việc cùng nhau* — hãy *pair*, hoặc *mob*.

The truth is that there's hardly any better way to improve the **productivity** of a team than collaborative programming. Because a team that knows the deep connections among the work that's being done can't help but be much more productive than a group of silos.

Sự thật là gần như không có cách nào cải thiện năng suất của một đội tốt hơn lập trình cộng tác. Bởi một đội hiểu được những mối liên hệ sâu xa giữa các phần việc đang được làm thì không thể nào không năng suất hơn hẳn một nhóm gồm toàn ốc đảo.

Open/Virtual Office — Văn phòng mở / văn phòng ảo

It is also important that the members of the team see and interact with each other very frequently. The best way to achieve this is to put them into a room together.

Cũng quan trọng không kém: các thành viên trong đội phải nhìn thấy và tương tác với nhau thật thường xuyên. Cách tốt nhất để đạt được điều này là xếp họ vào chung một phòng.

In the early 2000s I owned a company that helped organizations adopt Agile development. We would send a group of instructors and coaches to those companies and guide them through the change. Before each engagement began, we told the managers to rearrange the office space so that the teams we would be coaching worked together in their own team rooms.

Đầu những năm 2000, tôi sở hữu một công ty chuyên giúp các tổ chức áp dụng phát triển Agile. Chúng tôi cử một nhóm giảng viên và huấn luyện viên tới các công ty đó và dẫn dắt họ qua quá trình thay đổi. Trước khi mỗi hợp đồng bắt đầu, chúng tôi dặn các quản lý sắp xếp lại không gian văn phòng sao cho những đội mà chúng tôi sắp huấn luyện được làm việc cùng nhau trong phòng riêng của đội mình.

It happened more than once that, before we arrived to begin coaching, the managers told us that the teams were already much more productive, just because they were working together in the same room.

Đã không ít lần, trước cả khi chúng tôi đến để bắt đầu huấn luyện, các quản lý đã báo rằng những đội ấy năng suất hơn hẳn rồi — chỉ vì họ được làm việc cùng nhau trong cùng một căn phòng.

I wrote the first version of this chapter in the first quarter of

Tôi viết phiên bản đầu tiên của chương này vào quý I năm

2021. The COVID-19 pandemic was beginning to wane and we were all looking forward to a return to normal life. But the pandemic has left, in its wake, a large number of software teams that work remotely.

2021. Đại dịch COVID-19 khi ấy đang bắt đầu lắng xuống và tất cả chúng ta đều mong ngóng ngày trở lại cuộc sống bình thường. Nhưng đại dịch đã để lại phía sau nó một số lượng lớn các đội phần mềm làm việc từ xa.

Working remotely can never be quite as productive as working together in the same room. Even with the best electronic help, seeing each other on screens is just not as **good** as seeing each other in person. Still, the electronic systems for collaboration are very good nowadays. So, if you are working remotely, use them .

Làm việc từ xa không bao giờ có thể năng suất bằng làm việc cùng nhau trong cùng một căn phòng. Ngay cả với sự hỗ trợ điện tử tốt nhất, nhìn nhau qua màn hình vẫn không thể sánh với nhìn nhau ngoài đời. Dù vậy, các hệ thống cộng tác điện tử ngày nay rất tốt. Nên nếu bạn đang làm việc từ xa, hãy *dùng chúng*.

Create a virtual team room. Keep everyone's face in view. Keep everyone's audio channel as open as feasible. Your goal is to create the illusion of a team room, with everyone in it working together.

Hãy tạo một phòng đội ảo. Giữ khuôn mặt của mọi người luôn hiện trên màn hình. Giữ kênh âm thanh của mọi người mở đến mức khả thi nhất. Mục tiêu của bạn là tạo ra ảo giác về một phòng đội, với tất cả mọi người ở trong đó cùng làm việc với nhau.

Pairing and mobbing enjoy a lot of electronic support nowadays. It is relatively easy to share screens and program together over distances. While you do that, keep the faces and the audio up and running. You want to be able to see each other while you are collaborating on code.

Pairing và mobbing ngày nay được hỗ trợ điện tử rất nhiều. Chia sẻ màn hình và cùng lập trình qua khoảng cách địa lý là chuyện tương đối dễ. Trong lúc làm việc đó, hãy giữ cho khuôn mặt và âm thanh luôn bật. Bạn muốn nhìn thấy nhau trong khi cùng nhau viết mã.

Remote teams should try, as hard as they can, to maintain the same working hours. This is very difficult when there is a huge East-West distribution of programmers. So try to keep the number of time zones in each team as small as possible, and try very hard to have at least six contiguous hours per day when everyone can be together in the virtual team room.

Các đội làm việc từ xa nên cố hết sức duy trì cùng một khung giờ làm việc. Điều này rất khó khi các lập trình viên trải dài theo trục Đông-Tây. Vậy nên hãy cố giữ số múi giờ trong mỗi đội càng ít càng tốt, và hãy nỗ lực hết mức để có ít nhất sáu giờ liền mạch mỗi ngày mà tất cả mọi người đều có mặt trong phòng đội ảo.

Have you ever noticed how easy it is to yell at another driver while driving your car? This is the windscreen effect. When you are sitting behind a windscreen, it's easy to see other people as fools, imbeciles, and even enemies. It is easy to dehumanize them. This effect happens, to a lesser degree, behind computer screens.

Bạn có bao giờ để ý rằng khi lái xe, quát tháo một tài xế khác dễ đến thế nào không? Đó là *hiệu ứng kính chắn gió*. Khi ngồi sau tấm kính chắn gió, thật dễ để nhìn người khác như những kẻ ngu ngốc, những gã đần độn, thậm chí như kẻ thù. Thật dễ để phi nhân hoá họ. Hiệu ứng này cũng xảy ra, ở mức độ nhẹ hơn, phía sau màn hình máy tính.

So teams should get together in the same physical room several times per year. I recommend one week each quarter.

Vì thế các đội nên gặp nhau trong cùng một căn phòng thật vài lần mỗi năm. Tôi khuyến nghị một tuần mỗi quý.

This will help the team congeal and maintain itself as a team. It is very hard to fall into the windscreen trap with someone you ate lunch with, and physically collaborated with, two weeks ago.

Điều này sẽ giúp cả đội gắn kết lại và duy trì được bản chất một đội. Rất khó rơi vào cái bẫy kính chắn gió với một người mà bạn vừa ăn trưa cùng và ngồi cạnh nhau viết mã cách đây hai tuần.

Chapter 35. Estimate Honestly and Fairly — Chương 35. Ước lượng trung thực và công bằng

8. I will produce estimates that are honest both in magnitude and precision. I will not make promises without reasonable certainty.

8. Tôi sẽ đưa ra những ước lượng (estimate) trung thực cả về độ lớn lẫn độ chính xác. Tôi sẽ không hứa hẹn khi chưa có đủ mức chắc chắn hợp lý.

In this chapter, we're going to talk about estimating projects and large tasks that take many days or weeks to accomplish. This is not the same as the Agile practice of estimating small tasks and stories, which I describe in the book *Clean Agile*. ¹

Trong chương này, chúng ta sẽ bàn về việc ước lượng các dự án và những đầu việc lớn kéo dài nhiều ngày hoặc nhiều tuần. Chuyện này không giống với thực hành Agile là ước lượng các task và story nhỏ — phần đó tôi đã trình bày trong cuốn *Clean Agile*. ¹

¹ [Clean Agile].

¹ [Clean Agile].

Knowing how to **estimate** is an essential skill for every software developer; and one that most of us are very, very bad at.

Biết cách ước lượng là một kỹ năng thiết yếu với mọi lập trình viên; và cũng là kỹ năng mà phần lớn chúng ta rất, rất tệ.

The skill is essential because every business needs to know, roughly, how much something is going to cost before they commit resources to it.

Kỹ năng này thiết yếu vì mọi doanh nghiệp đều cần biết, một cách đại khái, thứ họ định làm sẽ tốn bao nhiêu trước khi cam kết nguồn lực cho nó.

Unfortunately, our failure to understand what estimates actually are, and how to create them, has led to an almost catastrophic loss of trust between programmers and businesses.

Đáng tiếc là việc chúng ta không hiểu ước lượng thực chất là gì, và tạo ra nó như thế nào, đã dẫn tới một sự đổ vỡ niềm tin gần như thảm họa giữa lập trình viên và giới kinh doanh.

The landscape is littered with billions of dollars in software failures. Often, those failures are due to poor estimation. It is not uncommon for estimates to be off by a factor of 2, 3, or even 4 or 5. But why? Why are estimates so hard to get right?

Khắp nơi ngổn ngang những dự án phần mềm thất bại trị giá hàng tỉ đô la. Nhiều thất bại trong số đó đến từ việc ước lượng kém. Chuyện ước lượng lệch đi 2, 3, thậm chí 4 hay 5 lần là hoàn toàn không hiếm. Nhưng tại sao? Vì sao ước lượng lại khó đúng đến thế?

Mostly it's because we don't understand what estimates actually are, and how to create them.

Chủ yếu là vì chúng ta không hiểu ước lượng thực chất là gì, và làm sao để tạo ra nó.

You see, in order for estimates to be useful, they must be honest. They must be honestly accurate, and they must be honestly precise. But most estimates are neither. Indeed, most estimates are lies.

Bạn thấy đấy, để ước lượng có ích, nó phải trung thực. Nó phải trung thực về độ đúng (accuracy), và trung thực về độ chính xác (precision). Nhưng phần lớn ước lượng chẳng đạt được điều nào cả. Thật ra, phần lớn ước lượng là những lời nói dối.

Lies — Những lời nói dối

Most estimates are lies because most estimates are constructed backward from a known end date.

Phần lớn ước lượng là nói dối, bởi phần lớn ước lượng được dựng ngược lại từ một ngày kết thúc đã biết trước.

Consider H ealthcare.gov , for example. The president of the United State signed a bill into law that mandated a specific date when that software system was to be turned on.

Chẳng hạn, hãy nhìn vào Healthcare.gov. Tổng thống Hoa Kỳ đã ký ban hành một đạo luật ấn định ngày cụ thể mà hệ thống phần mềm ấy phải được bật lên chạy.

The depth of that illogic is nausea inducing. I mean, how absurd. Nobody was asked to estimate the end date; they were just told what the end date had to be—by law!

Mức độ phi lý của chuyện đó đủ khiến người ta buồn nôn. Ý tôi là, thật quá vô lý. Chẳng ai được hỏi để ước lượng ngày hoàn thành cả; người ta chỉ đơn giản bị áp cho ngày hoàn thành phải là ngày nào — bằng luật!

So, of course, all estimates associated with that mandated date were lies. How could they be anything else?

Thế nên tất nhiên, mọi ước lượng gắn với cái ngày được ấn định đó đều là nói dối. Chúng còn có thể là gì khác được nữa?

It reminds me of a team I was consulting for about twenty years ago. I remember being in the project room with them, when the project manager walked in.

Chuyện này làm tôi nhớ tới một nhóm mà tôi tư vấn cho khoảng hai mươi năm trước. Tôi nhớ mình đang ngồi trong phòng dự án với họ thì ông quản lý dự án bước vào.

He was a young fellow; perhaps 25. He'd just returned from a meeting with his boss. He was visibly agitated.

Anh ta còn trẻ; chừng 25 tuổi. Anh vừa họp với sếp về. Trông anh ta rõ ràng là đang bồn chồn.

He told the team how important the end date was. He said: "We really have to make that date. I mean, we really have to make that date." Of course, the rest of the team just rolled their eyes and shook their heads. The need to make the date was not a solution for making the date. The young manager offered no solution.

Anh ta nói với cả nhóm rằng cái ngày hoàn thành quan trọng đến thế nào. Anh nói: “Chúng ta thực sự phải kịp cái ngày đó. Ý tôi là, chúng ta thực sự phải kịp cái ngày đó.” Tất nhiên, cả nhóm chỉ đảo mắt và lắc đầu. Việc cần phải kịp ngày không phải là giải pháp để kịp ngày. Anh quản lý trẻ chẳng đưa ra được giải pháp nào.

Of course, estimates in an environment like that are just lies that support the plan.

Tất nhiên, ước lượng trong một môi trường như thế chỉ là những lời nói dối để chống đỡ cho kế hoạch.

And that reminds me of another client of mine who had a huge software production plan on the wall—full of circles and arrows and labels and tasks.

Và điều đó làm tôi nhớ tới một khách hàng khác của mình, người có treo trên tường một kế hoạch sản xuất phần mềm khổng lồ — đầy những vòng tròn, mũi tên, nhãn và đầu việc.

The programmers referred to it as The Laugh Track .

Đám lập trình viên gọi nó là The Laugh Track (Băng tiếng cười).

In this chapter, we’re going to talk about real, worthwhile, honest, accurate, and precise estimates. The kinds of estimates that professionals create.

Trong chương này, chúng ta sẽ nói về những ước lượng thật sự, đáng giá, trung thực, đúng và chính xác. Loại ước lượng mà những người chuyên nghiệp tạo ra.

Honesty, Accuracy, Precision — Trung thực, đúng, chính xác

The most important aspect of an estimate is honesty. Estimates don’t do anybody any **good** unless they are honest.

Khía cạnh quan trọng nhất của một ước lượng là tính trung thực. Ước lượng chẳng đem lại lợi ích gì cho ai trừ phi nó trung thực.

The most honest estimate you can give is: “I don’t know.” But that estimate is not particularly accurate or precise. After all, you do know something about the estimate. So the challenge is to quantify what you do and don’t know.

Ước lượng trung thực nhất mà bạn có thể đưa ra là: “Tôi không biết.” Nhưng ước lượng đó chẳng đúng mà cũng chẳng chính xác cho lắm. Suy cho cùng, bạn có biết một điều gì đó về ước lượng ấy. Nên thách thức nằm ở chỗ lượng hoá được cái bạn biết và cái bạn không biết.

First, your estimate must be accurate. That doesn’t mean you give a firm date—you must not dare be that precise. It just means that you name a range of dates that you feel confident about.

Thứ nhất, ước lượng của bạn phải đúng. Điều đó không có nghĩa là bạn đưa ra một ngày chắc nịch — bạn không được liều lĩnh chính xác đến mức ấy. Nó chỉ có nghĩa là bạn nêu ra một khoảng thời gian mà bạn thấy tự tin.

So, for example, “Sometime between now and 10 years from now” is a pretty accurate estimate for how long it would take you to write a hello world program. But it lacks precision.

Chẳng hạn, “Đâu đó trong khoảng từ bây giờ tới 10 năm nữa” là một ước lượng khá đúng cho việc bạn cần bao lâu để viết một chương trình hello world. Nhưng nó thiếu độ chính xác.

On the other hand, “Yesterday at 2:15 a.m.” is a very precise estimate, but probably not very accurate if you haven’t started yet.

Ngược lại, “Hôm qua lúc 2 giờ 15 sáng” là một ước lượng rất chính xác, nhưng chắc là chẳng đúng chút nào nếu bạn còn chưa bắt tay vào làm.

Do you see the difference? When you give an estimate, you want it to be honest both in accuracy and in precision. To be accurate, you name a range of dates within which you are confident. To be precise, you narrow that range down to the level of your confidence.

Bạn thấy khác biệt chứ? Khi đưa ra một ước lượng, bạn muốn nó trung thực cả về độ đúng lẫn độ chính xác. Để đúng, bạn nêu ra một khoảng thời gian mà bạn tự tin. Để chính xác, bạn thu hẹp khoảng đó lại đúng bằng mức tự tin của mình.

And for both of these operations, brutal honesty is the only option.

Và với cả hai thao tác ấy, sự trung thực đến tận xương tủy là lựa chọn duy nhất.

Lessons from Me — Bài học từ chính tôi

To be honest about these things, you have to have some idea of how wrong you might be. So let me tell you two stories about how wrong I once was.

Muốn trung thực về những chuyện này, bạn phải có chút khái niệm mình có thể sai đến mức nào. Vậy hãy để tôi kể hai câu chuyện về việc tôi đã từng sai đến thế nào.

Story #1: Vectors — Chuyện số 1: Vector

The year was 1978. I was working at a company named Teradyne, in Deerfield, Illinois. We built automated **test** equipment for the telephone company.

Năm đó là 1978. Tôi làm việc tại một công ty tên Teradyne, ở Deerfield, Illinois. Chúng tôi chế tạo thiết bị kiểm thử tự động cho công ty điện thoại.

I was a young programmer, 26 years old. And I was working on the firmware for an embedded measurement device that bolted into racks in telephone central offices. This device was called a COLT —Central Office Line Tester.

Tôi khi ấy là một lập trình viên trẻ, 26 tuổi. Và tôi đang làm firmware cho một thiết bị đo nhúng được bắt bu-lông vào các rack trong tổng đài điện thoại. Thiết bị đó tên là COLT — Central Office Line Tester.

The processor in the COLT was an Intel 8085—an early 8-bit microprocessor. We had 32K of solid-state RAM and another 32K of ROM. The ROM was based on the Intel 2708 chip, which stored $1K \times 8$. So we used 32 of those chips.

Bộ xử lý trong COLT là một con Intel 8085 — vi xử lý 8-bit đời đầu. Chúng tôi có 32K RAM bán dẫn và thêm 32K ROM. Phần ROM dựa trên chip Intel 2708, mỗi con lưu $1K \times 8$. Nên chúng tôi dùng 32 con chip như vậy.

Those chips were plugged into sockets on our memory boards. Each board could hold 12 chips,; so we used three boards.

Những con chip đó được cắm vào các socket trên bo mạch nhớ. Mỗi bo chứa được 12 chip; nên chúng tôi dùng ba bo.

The software was written in 8085 assembler. The source code was held in a set of source files that were compiled as a single unit. The output of the compiler was a single binary file somewhat less than 32K in length.

Phần mềm được viết bằng assembler 8085. Mã nguồn nằm trong một tập các file nguồn được biên dịch như một khối duy nhất. Đầu ra của trình biên dịch là một file nhị phân duy nhất, dài hơi ít hơn 32K một chút.

We took that file and cut it up into thirty-two 1K chunks. Each 1K chunk was then burned onto one of the ROM chips, which were then inserted into the sockets on the ROM boards.

Chúng tôi lấy file đó và cắt nó thành ba mươi hai mẫu 1K. Mỗi mẫu 1K sau đó được nạp vào một trong các chip ROM, rồi các chip này được cắm vào socket trên bo ROM.

As you can imagine, you had to get the right chip into the right socket on the right board. So we were very careful to label them.

Như bạn có thể hình dung, bạn phải cắm đúng chip vào đúng socket trên đúng bo mạch. Vậy nên chúng tôi rất cẩn thận trong việc dán nhãn chúng.

We sold hundreds of these devices. And they were installed in telephone central offices all over the country.

Chúng tôi bán được hàng trăm thiết bị như thế. Và chúng được lắp đặt trong các tổng đài điện thoại khắp cả nước.

So what do you think happened when we changed that program? Just a one-line change?

Vậy bạn nghĩ chuyện gì xảy ra khi chúng tôi sửa chương trình đó? Chỉ sửa một dòng thôi?

If we added or removed a line, then all the addresses of all the subroutines after that point changed. And since those subroutines were called by other routines earlier in the code, every chip was affected. We had to reburn all 32 chips for a one-line change!

Nếu chúng tôi thêm hoặc bớt một dòng, thì địa chỉ của mọi chương trình con nằm sau điểm đó đều đổi. Và vì những chương trình con ấy lại được gọi bởi các routine khác nằm trước trong mã, nên mọi con chip đều bị ảnh hưởng. Chúng tôi phải nạp lại cả 32 con chip chỉ vì một dòng sửa!

This was a nightmare. We had to burn hundreds of sets of chips and ship them to all the field service reps, all around the world. Then, those reps would have to drive hundreds of miles to get to all the central offices in their district. They'd have to open our units, pull out all the memory boards, remove all 32 of the old chips, insert the 32 new chips, and reinsert the boards.

Đó là một cơn ác mộng. Chúng tôi phải nạp hàng trăm bộ chip và gửi chúng tới tất cả các kỹ thuật viên hiện trường, khắp thế giới. Rồi những người đó phải lái xe hàng trăm dặm để tới mọi tổng đài trong địa bàn của mình. Họ phải mở thiết bị của chúng tôi ra, rút hết các bo mạch nhớ, tháo cả 32 con chip cũ, cắm 32 con chip mới vào, rồi lắp bo trở lại.

Now, I don't know if you know this, but the act of removing and inserting a chip into a socket is not entirely reliable. The little pins on the chips tend to bend and break in frustratingly silent ways. So the poor field service folks had to have lots of spares of each of the 32 chips; and suffer through the inevitable debugging by removing and reinserting chips until they could get a unit to work.

Có thể bạn không biết chuyện này, nhưng thao tác tháo và cắm một con chip vào socket không phải lúc nào cũng đáng tin cậy. Mấy cái chân nhỏ xíu trên chip rất hay bị cong và gãy theo những cách âm thầm đến phát bực. Thế nên đám kỹ thuật viên hiện trường tội nghiệp phải mang theo cả đồng chip dự phòng cho từng con trong số 32 con; rồi chịu trận

với màn debug không thể tránh khỏi bằng cách tháo ra cắm vào cho tới khi một thiết bị chịu chạy.

So my boss came to me one day and told me that we had to solve this problem by making each chip **independently deployable**. He didn't use those words, of course, but that was the intent. Each chip needed to be turned into an independently compilable and deployable unit. This would allow us to make changes to the program without forcing all 32 chips to be reburned. Indeed, in most cases, we could simply redeploy a single chip—the chip that was changed.

Rồi một hôm sếp tôi tới và bảo rằng chúng tôi phải giải quyết vấn đề này bằng cách làm cho mỗi con chip *triển khai độc lập được*. Tất nhiên ông không dùng đúng những từ đó, nhưng ý là vậy. Mỗi con chip cần được biến thành một đơn vị biên dịch và triển khai độc lập. Như thế sẽ cho phép chúng tôi thay đổi chương trình mà không buộc phải nạp lại cả 32 con chip. Thật vậy, trong hầu hết trường hợp, chúng tôi chỉ cần triển khai lại một con chip duy nhất — con chip đã thay đổi.

I won't bore you with the details of the **implementation**. Suffice it to say that it involved vector tables, indirect calls, and the partitioning of the program into independent chunks of less than 1K each. 2

Tôi sẽ không làm bạn chán bằng các chi tiết hiện thực. Chỉ cần nói rằng nó liên quan tới bảng vector, lời gọi gián tiếp, và việc phân hoạch chương trình thành các mẫu độc lập, mỗi mẫu nhỏ hơn 1K. 2

2 In other words, each chip was turned into a polymorphic **object**.

2 Nói cách khác, mỗi con chip được biến thành một đối tượng (object) đa hình (polymorphism).

My boss and I talked through the strategy, and then he asked me how long it would take me to get this done.

Sếp tôi và tôi bàn xong chiến lược, rồi ông hỏi tôi sẽ mất bao lâu để làm xong việc này.

I told him two weeks.

Tôi bảo ông hai tuần.

But I wasn't done in two weeks. I wasn't done in four weeks. Nor was I done in six, eight, or ten weeks.

Nhưng hai tuần tôi chưa xong. Bốn tuần tôi cũng chưa xong. Sáu, tám, rồi mười tuần cũng vẫn chưa xong.

The job took me 12 weeks to complete. It was a lot more complicated than I thought.

Công việc ấy tốn của tôi 12 tuần mới hoàn thành. Nó phức tạp hơn tôi nghĩ rất nhiều.

So I was off by a factor of six. Six!

Vậy là tôi đã lệch đi sáu lần. Sáu lần!

Fortunately, my boss didn't get mad. He saw me working on it every day. He got regular status updates from me. He understood the complexities I was dealing with.

May thay, sếp tôi không nổi giận. Ông thấy tôi làm việc đó mỗi ngày. Ông nhận báo cáo tiến độ đều đặn từ tôi. Ông hiểu những rắc rối mà tôi đang phải xử lý.

But still. Six? How could I have been so wrong?

Nhưng dù sao đi nữa. Sáu lần cơ đấy? Sao tôi lại có thể sai đến thế?

Story #2: pCCU — Chuyện số 2: pCCU

Then there was that time, in the early '80s, that I had to work a miracle. You see, we had promised a new product to our customer. It was called CCU-CMU .

Rồi có lần, hồi đầu thập niên 80, tôi phải làm một phép màu. Chuyện là chúng tôi đã hứa với khách hàng về một sản phẩm mới. Nó tên là CCU-CMU.

Copper is a precious metal. It's rare and expensive. The phone company decided to harvest the huge network of copper wires that it had installed all over the country during the last century. They replaced it with a much cheaper, high-bandwidth network of coaxial cable and fiber carrying digital signals. This was known as digital switching .

Đồng là một kim loại quý. Nó hiếm và đắt. Công ty điện thoại quyết định thu hồi mạng lưới dây đồng khổng lồ mà họ đã lắp đặt khắp cả nước suốt thế kỷ trước. Họ thay nó bằng một mạng lưới rẻ hơn nhiều, băng thông cao, gồm cáp đồng trục và cáp quang truyền tín hiệu số. Việc này được gọi là *digital switching* (chuyển mạch số).

The CCU-CMU was a complete rearchitecture of our measurement technology that fit within the new digital switching **architecture** of the phone company.

CCU-CMU là một cuộc tái kiến trúc toàn diện công nghệ đo lường của chúng tôi sao cho vừa khớp với kiến trúc chuyển mạch số mới của công ty điện thoại.

Now, we had promised the CCU-CMU to the phone company a couple of years before. We knew it was going to take us one person-year or so to build the software. But then, we just never quite got around to building it.

Bấy giờ, chúng tôi đã hứa giao CCU-CMU cho công ty điện thoại từ vài năm trước. Chúng tôi biết sẽ mất chừng một người-năm để xây phần mềm ấy. Nhưng rồi chúng tôi cứ mãi không bắt tay vào làm.

You know how it goes. The phone company delayed their deployment, so we delayed our development. There were always lots of other, more urgent issues to deal with.

Bạn biết chuyện đó diễn ra thế nào rồi đấy. Công ty điện thoại hoãn triển khai, nên chúng tôi hoãn phát triển. Lúc nào cũng có cả đồng vốn đề khác cấp bách hơn cần xử lý.

So one day, my boss calls me into his office and says that they had forgotten about one small customer who had already installed an early digital switch. That customer was now expecting a CCU/CMU within the next month—as promised.

Thế rồi một hôm, sếp gọi tôi vào phòng và nói rằng họ đã quên mất một khách hàng nhỏ vốn đã lắp đặt một tổng đài số đời đầu. Khách hàng đó giờ đang trông đợi nhận CCU/CMU trong vòng một tháng tới — đúng như đã hứa.

So now I had to create a person-year of software in less than a month.

Vậy là giờ tôi phải tạo ra một người-năm phần mềm trong chưa đầy một tháng.

So I told my boss that this was impossible. There was no way that I could get a fully functioning CCU/CMU built in one month.

Nên tôi báo sếp rằng chuyện này là bất khả thi. Không đời nào tôi dựng được một CCU/CMU chạy đầy đủ chức năng trong một tháng.

Then, he looked at me with a sneaky grin and said that there was a way to cheat.

Rồi ông nhìn tôi với nụ cười ranh mãnh và nói rằng có một cách để gian lận.

You see, this was a very small customer. Their installation was literally the smallest possible configuration for a digital switch. What's more, the configuration of their equipment just happened—to eliminate virtually all of the complexity that the CCU/CMU solved.

Chuyện là, đây là một khách hàng rất nhỏ. Hệ thống của họ đúng nghĩa là cấu hình nhỏ nhất có thể có của một tổng đài số. Hơn nữa, cấu hình thiết bị của họ tình cờ — đúng là tình cờ — loại bỏ gần như toàn bộ những phức tạp mà CCU/CMU sinh ra để giải quyết.

Long story short: I got a special-purpose, one-of-a-kind unit up and running for the customer in two weeks. We called it the pCCU .

Nói ngắn gọn: tôi đã dựng xong và chạy được một thiết bị chuyên dụng, độc nhất vô nhị cho khách hàng đó trong hai tuần. Chúng tôi gọi nó là pCCU.

The Lesson — Bài học

Those two stories are examples of the huge range that estimates can have. On the one hand, I underestimated the vectoring of the chips by a factor of six. On the other, we found a solution to the CCU/CMU in one twentieth the expected time.

Hai câu chuyện đó là ví dụ cho khoảng dao động khổng lồ mà ước lượng có thể có. Một mặt, tôi đã ước lượng thiếu việc vector hoá đám chip tới sáu lần. Mặt khác, chúng tôi tìm ra lời giải cho CCU/CMU chỉ trong một phần hai mươi thời gian dự kiến.

This is where honesty comes in. Because, honestly, when things go wrong, they can go very, very wrong. And when things go right, they can sometimes go very, very right.

Đây chính là chỗ mà sự trung thực bước vào. Bởi vì, nói thật lòng, khi mọi thứ trở nên tồi tệ, chúng có thể tồi tệ đến mức rất, rất kinh khủng. Và khi mọi thứ suôn sẻ, đôi khi chúng suôn sẻ đến mức rất, rất khó tin.

And this makes estimating one hell of a challenge.

Và điều đó khiến việc ước lượng trở thành một thách thức kinh khủng.

Accuracy and Precision — Độ đúng và độ chính xác

It should be clear by now that an estimate for a project cannot be a date. A single date is far too precise for a process that can be off by as much as a factor of six, or even twenty.

Đến giờ hẳn đã rõ rằng ước lượng cho một dự án không thể là một cái ngày. Một ngày duy nhất thì chính xác quá mức so với một quá trình có thể lệch tới sáu lần, thậm chí hai mươi lần.

That variation means that every estimate is likely wrong. That's why we call them estimates . But even though an estimate is wrong, it may not be very wrong. So part of the job of estimation is to estimate how wrong the estimate probably is.

Sự biến thiên đó có nghĩa là mọi ước lượng đều có khả năng sai. Chính vì thế mà ta gọi chúng là *ước lượng*. Nhưng dù một ước lượng có sai, nó có thể không sai nhiều lắm. Nên một phần của công việc ước lượng là ước lượng xem ước lượng ấy nhiều khả năng sai đến mức nào.

A single date is too precise to be honest, so estimates are ranges; they must be probability distributions. Probability distributions have a mean, and a width—sometimes called the standard deviation or the sigma. We need to be able to express our estimates with both a mean and a sigma.

Một ngày duy nhất thì chính xác quá mức để có thể trung thực, nên ước lượng phải là những khoảng; chúng phải là *phân phối xác suất* (probability distribution). Phân phối xác suất có giá trị trung bình, và có độ rộng — đôi khi gọi là *độ lệch chuẩn* hay *sigma*. Chúng ta cần diễn đạt được ước lượng của mình bằng cả trung bình lẫn sigma.

My favorite technique for estimating the probability distribution of an estimate is to estimate three numbers: the best case, the worst case, and the normal case.

Kỹ thuật ưa thích của tôi để ước lượng phân phối xác suất của một ước lượng là ước lượng ba con số: trường hợp tốt nhất, trường hợp xấu nhất, và trường hợp bình thường.

The normal case is how long you think the task would take you if the average number of things go wrong—if things go the way they usually do. Think of it as the date that you've got a 50% chance of making.

Trường hợp bình thường là quãng thời gian bạn nghĩ đầu việc sẽ ngắn của mình nếu số chuyện trục trặc ở mức trung bình — nếu mọi thứ diễn ra như nó vẫn thường diễn ra. Hãy xem đó là cái ngày mà bạn có 50% cơ hội đạt được.

The worst-case estimate is the Murphy's law estimate. It assumes that anything that can go wrong will go wrong. It is deeply pessimistic. Think of it as the date that you've got a 95% chance of making.

Ước lượng trường hợp xấu nhất là ước lượng theo định luật Murphy. Nó giả định rằng bất cứ thứ gì có thể hỏng thì sẽ hỏng. Nó bi quan tận cùng. Hãy xem đó là cái ngày mà bạn có 95% cơ hội đạt được.

The best-case estimate is when everything goes right. You eat the right breakfast cereal every morning. When you get into work, your coworkers are all polite and friendly. There are no disasters in the field, no meetings, no telephone calls, no distractions.

Ước lượng trường hợp tốt nhất là khi mọi thứ đều thuận lợi. Sáng nào bạn cũng ăn đúng loại ngũ cốc. Khi tới chỗ làm, đồng nghiệp ai cũng lịch sự và thân thiện. Ngoài hiện trường chẳng có thảm họa nào, không họp hành, không điện thoại, không thứ gì làm bạn phân tâm.

Your chances of hitting the best-case estimate are 5%: one in twenty.

Cơ hội bạn chạm được ước lượng trường hợp tốt nhất là 5%: một phần hai mươi.

OK, so now we have three numbers. The best case which has a 5% chance of success. The nominal case which has a 50% chance of success. And the worst case, which has a 95% chance of success, represents a normal curve—a probability distribution. It is this probability distribution that is your actual estimate.

Được rồi, giờ ta đã có ba con số. Trường hợp tốt nhất với 5% cơ hội thành công. Trường hợp danh định với 50% cơ hội thành công. Và trường hợp xấu nhất với 95% cơ hội thành công — cả ba tạo nên một *đường cong chuẩn*, tức một phân phối xác suất. Chính phân phối xác suất này mới là ước lượng thật sự của bạn.

Notice that this is not a date. We don't know the date. We don't know when we are really going to be done. All we really have is a crude idea of the probabilities.

Hãy để ý rằng đây không phải một cái ngày. Chúng ta không biết ngày nào. Chúng ta không biết thực sự khi nào mình sẽ xong. Tất cả những gì ta thực sự có là một ý niệm thô sơ về

xác suất.

Without certain knowledge, probabilities are the only logical way to estimate.

Khi không có tri thức chắc chắn, xác suất là cách hợp lý duy nhất để ước lượng.

If your estimate is a date, you are making a commitment, not an estimate. And if you make a commitment, you must succeed.

Nếu ước lượng của bạn là một cái ngày, thì bạn đang đưa ra một cam kết, chứ không phải một ước lượng. Và nếu đã cam kết, bạn buộc phải thành công.

Sometimes you have to make commitments. But the thing about commitments is that you absolutely must succeed. You must never promise to make a date that you aren't sure you can make. To do so would be deeply dishonest.

Đôi khi bạn phải cam kết. Nhưng vấn đề của cam kết là bạn tuyệt đối phải thành công. Bạn không bao giờ được hứa hẹn một cái ngày mà bạn không chắc mình đáp ứng nổi. Làm vậy là bất lương tận gốc.

So, if you don't know—and I mean know—that you can make a certain date, then you don't offer that date as an estimate. You offer a range of dates instead. Offering a range of dates with probabilities is much more honest.

Vậy nên, nếu bạn không biết — và ý tôi là *biết* thật sự — rằng mình có thể kịp một ngày nào đó, thì đừng đưa ngày đó ra như một ước lượng. Thay vào đó hãy đưa ra một khoảng ngày. Đưa ra một khoảng ngày kèm xác suất thì trung thực hơn nhiều.

Aggregation — Tổng hợp

OK, so now let's say that you've got a whole project full of tasks that have been described in terms of Best- (B), Normal- (N), and Worst- (W) case estimates. How do you aggregate them all into a single estimate for the whole project?

Được rồi, giờ giả sử bạn có cả một dự án đầy các đầu việc đã được mô tả bằng ước lượng trường hợp Tốt nhất (B — Best), Bình thường (N — Normal) và Xấu nhất (W — Worst). Làm sao gộp tất cả chúng lại thành một ước lượng duy nhất cho toàn dự án?

You simply represent the probability of each task, and then accumulate those probabilities using standard statistical methods.

Bạn chỉ cần biểu diễn xác suất của từng đầu việc, rồi tích lũy các xác suất đó bằng những phương pháp thống kê chuẩn.

The first thing we want to do is represent each task in terms of its expected completion time and the standard deviation.

Việc đầu tiên cần làm là biểu diễn mỗi đầu việc theo thời gian hoàn thành kỳ vọng và độ lệch chuẩn của nó.

Now, remember six standard deviations (three on each side of the mean) corresponds to a probability of better than 99%. So we're going to set our standard deviation, our sigma, to Worst minus Best over 6.

Bây giờ, hãy nhớ rằng sáu độ lệch chuẩn (ba về mỗi phía của giá trị trung bình) tương ứng với xác suất hơn 99%. Nên chúng ta sẽ đặt độ lệch chuẩn, tức sigma, bằng Worst trừ Best chia 6.

The expected completion time (Mu) is a bit trickier. Notice that N is probably not equal to $(W - B)$, the midpoint. Indeed, the midpoint is probably well past N. This is because it is much more likely for a project to go long than to go short. So it's probably best to use a weighted average, like this: $Mu = (2N + (B + W) \div 2) \div 3$.

Thời gian hoàn thành kỳ vọng (Mu) thì hơi rắc rối hơn một chút. Hãy để ý rằng N nhiều khả năng không bằng $(W - B)$, tức điểm giữa. Thật vậy, điểm giữa nhiều khả năng nằm khá xa về phía sau N. Lý do là một dự án dễ bị kéo dài hơn nhiều so với việc kết thúc sớm. Nên tốt nhất có lẽ nên dùng trung bình có trọng số, như thế này: $Mu = (2N + (B + W) \div 2) \div 3$.

Now you have calculated the Mu and sigma for a set of tasks. So the expected completion time for the whole project is just the sum of all the Mus. The sigma for the project is the square root of the sum of the squares of all the sigmas.

Giờ bạn đã tính được Mu và sigma cho một tập các đầu việc. Vậy thời gian hoàn thành kỳ vọng cho toàn dự án chỉ đơn giản là tổng của tất cả các Mu. Sigma của dự án là căn bậc hai của tổng bình phương tất cả các sigma.

This is just basic statistical mathematics.

Đây chỉ là toán thống kê cơ bản.

What I've just described is the estimation procedure invented back in the late 1950s to manage the Polaris Fleet Ballistic Missile program. It has been used successfully on many thousands of projects since.

Thứ tôi vừa mô tả chính là quy trình ước lượng được phát minh hồi cuối thập niên 1950 để quản lý chương trình tên lửa đạn đạo hạm đội Polaris. Kể từ đó nó đã được dùng thành công trên hàng nghìn dự án.

It is called PERT —Program Evaluation and Review Technique.

Nó có tên là PERT — Program Evaluation and Review Technique.

Honesty — Trung thực

So we started with honesty. Then, we talked about accuracy and precision. Now it's time to come back to honesty.

Vậy là chúng ta bắt đầu bằng tính trung thực. Rồi bàn về độ đúng và độ chính xác. Giờ đã đến lúc quay lại với tính trung thực.

The kind of estimating we are talking about here is intrinsically honest. It is a way of communicating, to those who need to know, the level of your uncertainty.

Kiểu ước lượng mà chúng ta đang bàn ở đây tự bản chất là trung thực. Nó là một cách truyền đạt, tới những người cần biết, mức độ bất định của bạn.

This is honest because you truly are uncertain. And those with the responsibility to manage the project must be aware of the risks they are taking so that they can manage those risks.

Điều này trung thực bởi bạn thật sự đang bất định. Và những người có trách nhiệm quản lý dự án phải nắm được các rủi ro mà họ đang gánh, để có thể quản lý chúng.

But uncertainty is something that people don't like. Your customers and managers will almost certainly press you to be more certain.

Nhưng bất định là thứ người ta không ưa. Khách hàng và các quản lý của bạn gần như chắc chắn sẽ ép bạn phải chắc chắn hơn.

We've already talked about the cost of increasing certainty. The only way to truly increase certainty is to do parts of the project. You can only get perfect certainty if you do the entire project.

Chúng ta đã bàn về cái giá của việc tăng độ chắc chắn. Cách duy nhất để thực sự tăng độ chắc chắn là làm thật một phần dự án. Bạn chỉ đạt được độ chắc chắn hoàn hảo khi làm xong toàn bộ dự án.

So part of what you have to tell your customers and managers is the cost of increasing certainty.

Vậy nên một phần điều bạn phải nói với khách hàng và quản lý chính là cái giá của việc tăng độ chắc chắn.

Sometimes, however, your superiors may ask you to increase certainty using a different tactic. They may ask you to commit.

Tuy nhiên, đôi khi cấp trên của bạn có thể yêu cầu bạn tăng độ chắc chắn bằng một chiến thuật khác. Họ có thể yêu cầu bạn cam kết.

You need to recognize this for what it is. They are trying to manage their risk, by putting it onto you. By asking you to commit, they are asking you to take on the risk that it is their job to manage.

Bạn cần nhận ra bản chất của chuyện này. Họ đang tìm cách quản lý rủi ro của mình bằng cách đẩy nó sang bạn. Khi yêu cầu bạn cam kết, họ đang yêu cầu bạn gánh lấy rủi ro mà lẽ ra quản lý nó là việc của họ.

Now, there's nothing wrong with this. Managers have a perfect right to do this. And there are many situations in which you should comply. But—and I stress this—only if you are reasonably certain you can comply.

Nào, chuyện này chẳng có gì sai cả. Các quản lý hoàn toàn có quyền làm vậy. Và có nhiều tình huống bạn nên đồng ý. Nhưng — và tôi nhấn mạnh điều này — chỉ khi bạn tương đối chắc chắn rằng mình làm được.

If your boss comes to you and asks if you can get something done by Friday, you should think very hard about whether that is reasonable. And if it is reasonable and probable, then by all means, say yes!

Nếu sếp tới hỏi bạn có thể xong việc gì đó trước thứ Sáu không, bạn nên suy nghĩ thật kỹ xem điều đó có hợp lý hay không. Và nếu nó hợp lý và khả thi, thì cứ việc, nói vâng!

But under no circumstances should you say yes if you are not sure.

Nhưng tuyệt đối không được nói vâng nếu bạn không chắc.

If you are not sure, then you must say no, and then describe your uncertainty as we've described. It is perfectly OK to say: "I can't promise Friday. It might take as long as the following Wednesday." In fact, it's absolutely critical that you say no to commitments that you are not sure of. Because if you say yes, you set up a long domino chain of failures for you, your boss, and many others. They'll be counting on you, and you'll let them down.

Nếu bạn không chắc, thì bạn phải nói không, rồi mô tả mức bất định của mình theo cách chúng ta đã bàn. Nói thế này là hoàn toàn ổn: "Tôi không thể hứa thứ Sáu. Có thể phải tới thứ Tư tuần sau mới xong." Thật ra, việc nói không với những cam kết mà bạn không chắc là tuyệt đối thiết yếu. Bởi nếu bạn nói vâng, bạn dựng lên một chuỗi domino thất bại dài dằng dặc cho chính bạn, cho sếp bạn, và cho rất nhiều người khác. Họ sẽ trông cậy vào bạn, và bạn sẽ làm họ thất vọng.

So, when you are asked to commit, and you are sure you can comply, then say yes. But if you aren't sure, then say no, and describe your uncertainty.

Vậy nên, khi được yêu cầu cam kết mà bạn chắc rằng mình làm được, hãy nói vâng. Nhưng nếu bạn không chắc, thì hãy nói *không*, và mô tả mức bất định của mình.

And be willing to discuss options and workarounds. Be willing to hunt for ways to say yes. Never be eager to say no. But also, never be afraid to say no.

Và hãy sẵn lòng bàn bạc về các phương án và cách xoay xở. Hãy sẵn lòng đi tìm những cách để có thể nói vâng. Đừng bao giờ hăm hở nói *không*. Nhưng cũng đừng bao giờ sợ phải nói không.

You see, you were hired for your ability to say no. Anybody can say yes. But only people with skill and knowledge know when and how to say no.

Bạn thấy đấy, bạn được tuyển vào chính vì khả năng nói không của mình. Ai cũng có thể nói vâng. Nhưng chỉ những người có kỹ năng và hiểu biết mới biết khi nào và nói không như thế nào.

One of the prime values you bring to the organization is your ability to know when the answer must be no. By saying no at those times, you will save your company untold grief and money.

Một trong những giá trị hàng đầu bạn mang lại cho tổ chức là khả năng biết khi nào câu trả lời buộc phải là không. Bằng cách nói không vào những lúc ấy, bạn sẽ giúp công ty mình tránh được vô số phiền toái và tiền bạc.

Pressure — Áp lực

One last thing: Often a manager will try to cajole you into committing—into saying yes. Watch out for this.

Một điều cuối cùng: sếp thường sẽ tìm cách dỗ ngọt để bạn cam kết — để bạn nói “được”. Hãy coi chừng chuyện đó.

They might tell you that you aren't being a team player, or that other people have more commitment than you do. Don't be fooled by those games.

Họ có thể bảo rằng bạn chẳng có tinh thần đồng đội, hoặc rằng người khác tận tâm hơn bạn. Đừng để bị mắc lừa bởi mấy trò ấy.

Be willing to work with them to find solutions, but don't let them bully you into saying yes when you know you shouldn't.

Hãy sẵn lòng cùng họ tìm giải pháp, nhưng đừng để họ bắt nạt bạn đến mức phải nói “được” trong khi bạn thừa biết là không nên.

And be very careful with the word try. Your boss might say something reasonable, like, “Well, will you at least try?” The answer to this is:

Và hãy hết sức cẩn thận với chữ *thử* (try). Sếp bạn có thể nói một câu nghe rất chi là hợp tình hợp lý, đại loại: “Thôi thì, ít nhất cậu cứ thử xem sao?” Câu trả lời cho chuyện này là:

NO! I am already trying. How dare you suggest that I am not? I am trying as hard as I can; and there is no way I can try harder. There are no magic beans in my pocket with which I can work miracles.

KHÔNG! Tôi đang thử rồi đấy chứ. Sao anh dám ám chỉ là tôi không thử? Tôi đang cố hết sức mình; và chẳng còn cách nào để cố hơn được nữa. Trong túi tôi làm gì có hạt đậu thần nào để mà làm phép màu.

You might not want to use those exact words, but that's exactly what you should be thinking.

Có thể bạn không muốn nói ra đúng từng chữ ấy, nhưng đó chính xác là điều bạn nên nghĩ trong đầu.

And remember this: If you say “Yes, I’ll try,” then you are lying. Because you have no idea how you are going to succeed. You don’t have any plan to change your **behavior**. You said yes just to get rid of them. And that is the most dishonest thing of all.

Và hãy nhớ điều này: nếu bạn nói “Vâng, tôi sẽ thử,” thì bạn đang nói dối. Bởi bạn chẳng có mảy may ý niệm nào về việc mình sẽ thành công bằng cách nào. Bạn không có kế hoạch nào để thay đổi cách mình đang làm. Bạn nói “được” chỉ cốt để họ buông tha cho bạn. Và đó mới là điều bất lương nhất trong tất cả.

Chapter 36. Respect for Fellow Programmers — Chương 36. Tôn trọng đồng nghiệp lập trình viên

9. I will respect my fellow programmers for their ethics, standards, disciplines, and skill. No other attribute or characteristic will be a factor in my regard for my fellow programmers.

9. Tôi sẽ tôn trọng các đồng nghiệp lập trình viên của mình vì đạo đức, chuẩn mực, kỷ luật (discipline) và kỹ năng của họ. Không một thuộc tính hay đặc điểm nào khác sẽ là yếu tố chi phối cách tôi nhìn nhận đồng nghiệp của mình.

We, software professionals, accept the weighty burden of our **craft**. We brave folks are men and women, straight and gay, black, brown, yellow, and white, republicans, democrats, religious believers, and atheists. We are humans, in all the many forms and varieties that humans come in. We are a community—a brotherhood—of mutual respect.

Chúng ta, những người làm phần mềm chuyên nghiệp, chấp nhận gánh nặng nặng nề của nghề mình. Những con người dũng cảm chúng ta là đàn ông và phụ nữ, dị tính và đồng tính, da đen, da nâu, da vàng và da trắng, người Cộng hoà, người Dân chủ, người có tín ngưỡng và người vô thần. Chúng ta là con người, trong tất cả muôn hình vạn trạng mà con người vốn có. Chúng ta là một cộng đồng — một tình anh em — của sự tôn trọng lẫn nhau.

The only qualifications for entry into our brotherhood, and for receiving the acceptance and the respect of each and every member of that brotherhood, are the skills, disciplines, standards, and ethics of our profession. No other human attribute is worthy of consideration. No discrimination on any other basis can be tolerated.

Điều kiện duy nhất để gia nhập tình anh em ấy, và để nhận được sự chấp nhận cùng lòng tôn trọng của từng thành viên trong đó, là kỹ năng, kỷ luật, chuẩn mực và đạo đức nghề nghiệp của chúng ta. Không một thuộc tính con người nào khác đáng được đem ra cân nhắc. Không một sự phân biệt đối xử nào trên bất kỳ cơ sở nào khác có thể được dung thứ.

Within our profession, we are programmers first.

Trong nghề của chúng ta, trước hết chúng ta là những lập trình viên.

Chapter 37. Never Stop Learning —

Chương 37. Không bao giờ ngừng học

10. I will never stop learning and improving my **craft**.

10. Tôi sẽ không bao giờ ngừng học hỏi và trau dồi tay nghề của mình.

A programmer never stops learning.

Một lập trình viên thì không bao giờ ngừng học.

I'm sure you've heard it said that you should learn a new language every year. Well, you should. A **good** programmer should know a dozen languages or so.

Tôi chắc là bạn từng nghe người ta nói rằng mỗi năm bạn nên học một ngôn ngữ mới. Đúng thế, bạn nên làm vậy. Một lập trình viên giỏi nên biết khoảng chục ngôn ngữ.

And not just a dozen varieties of the same language. Not just C, C++, Java, and C#. Rather, you should know languages from many different families.

Và không phải chục biến thể của cùng một ngôn ngữ. Không phải chỉ C, C++, Java và C#. Đúng hơn, bạn nên biết những ngôn ngữ thuộc nhiều họ khác nhau.

You should know a statically typed language like Java or C#. You should know a procedural language like C or Pascal. You should know a logic language like Prolog. You should know a stack language like Forth. You should know a dynamically typed language like Ruby. You should know a functional language like Clojure or Haskell.

Bạn nên biết một ngôn ngữ định kiểu tĩnh như Java hay C#. Bạn nên biết một ngôn ngữ thủ tục như C hay Pascal. Bạn nên biết một ngôn ngữ logic như Prolog. Bạn nên biết một ngôn ngữ stack như Forth. Bạn nên biết một ngôn ngữ định kiểu động như Ruby. Bạn nên biết một ngôn ngữ hàm như Clojure hay Haskell.

You should also know several different frameworks. And several different design methodologies, and several different development processes.

Bạn cũng nên biết vài framework khác nhau. Rồi vài phương pháp luận thiết kế khác nhau, và vài quy trình phát triển khác nhau.

I don't mean to say you should be an expert in all these things; but you should make a point to expose yourself to them at significantly more than a cursory level.

Tôi không có ý bảo rằng bạn phải là chuyên gia trong tất cả những thứ đó; nhưng bạn nên chủ tâm tiếp xúc với chúng ở mức sâu hơn hẳn kiểu cưỡi ngựa xem hoa.

Indeed, the list of things you should similarly expose yourself to is virtually endless. Our industry has experienced rapid change over the decades, and that change is likely to continue for some time. You have to keep up with it.

Thật ra, danh sách những thứ bạn nên tiếp xúc như vậy gần như là vô tận. Ngành của chúng ta đã trải qua thay đổi chóng mặt suốt mấy thập kỷ, và sự thay đổi đó có lẽ còn tiếp diễn thêm một thời gian nữa. Bạn phải theo kịp nó.

And that means you have to keep on learning. Keep reading books and blogs. Keep watching videos. Keep going to conferences and user groups. Keep going to training courses. Keep learning.

Và điều đó có nghĩa là bạn phải tiếp tục học. Cứ tiếp tục đọc sách và blog. Cứ tiếp tục xem video. Cứ tiếp tục dự hội thảo và các nhóm người dùng. Cứ tiếp tục tham gia các khoá huấn luyện. Cứ tiếp tục học.

Pay attention to the treasured works of the past. The books written in the '60s and '70s and '80s are wonderful sources of insight and information. Don't fall into the trap of thinking that all that old stuff is out of date. There is not much in our industry that actually goes out of date. Respect the effort and accomplishments of those who came before you, and study their advice and conclusions.

Hãy để mắt tới những công trình quý giá của quá khứ. Những cuốn sách viết trong thập niên 60, 70 và 80 là nguồn kiến giải và thông tin tuyệt vời. Đừng sa vào cái bẫy nghĩ rằng mấy thứ cũ kỹ đó đều đã lỗi thời. Trong ngành của chúng ta chẳng có mấy thứ thực sự lỗi thời cả. Hãy tôn trọng công sức và thành tựu của những người đi trước, và nghiên cứu lời khuyên cùng kết luận của họ.

And don't fall into the trap of thinking that it is your employer's job to train you. This is your career; you have to take responsibility for it. It is your job to learn. It is your job to figure out what you should be learning.

Và cũng đừng sa vào cái bẫy nghĩ rằng đào tạo bạn là việc của người sử dụng lao động. Đây là sự nghiệp của bạn; bạn phải chịu trách nhiệm về nó. Học là việc của bạn. Tự tìm ra mình nên học cái gì cũng là việc của bạn.

If you are lucky enough to work for a company that will buy you books and send you to conferences and training classes, then take full advantage of that. If not, then pay for those books, conferences, and courses yourself.

Nếu bạn đủ may mắn để làm cho một công ty chịu mua sách và cử bạn đi hội thảo, đi học, thì hãy tận dụng tối đa. Nếu không, hãy tự bỏ tiền túi ra cho những cuốn sách, những hội thảo và những khoá học ấy.

And plan to spend some time at this. Time every week.

Và hãy lên kế hoạch dành thời gian cho việc này. Thời gian mỗi tuần.

You owe your employer 35 to 40 hours per week. You owe your career another 10 to 20.

Bạn nợ người sử dụng lao động 35 đến 40 giờ mỗi tuần. Bạn nợ sự nghiệp của mình thêm 10 đến 20 giờ nữa.

That's what a **professional** does. A professional puts in the time to groom and maintain their career. And that means you should be working 50 to 60 hours per week total. Mostly at work, but a lot at home too.

Người chuyên nghiệp là làm như vậy. Người chuyên nghiệp bỏ thời gian ra để chăm chút và giữ gìn sự nghiệp của mình. Và điều đó có nghĩa là tổng cộng bạn nên làm việc 50 đến 60 giờ mỗi tuần. Phần lớn ở chỗ làm, nhưng cũng khá nhiều ở nhà.

Afterword — Lời bạt



Justin Michael Martin

Justin Michael Martin

“When you’re a carpenter making a beautiful chest of drawers, you’re not going to use a piece of plywood on the back, even though it faces the wall and nobody will ever see it. You’ll know it’s there, so you’re going to use a beautiful piece of wood on the back. For you to sleep well at night, the aesthetic, the quality, has to be carried all the way through.”

“Khi bạn là thợ mộc đang đóng một chiếc tủ ngăn kéo thật đẹp, bạn sẽ không dùng một tấm ván ép cho mặt lưng, dù mặt đó quay vào tường và chẳng ai nhìn thấy bao giờ. Bạn biết nó ở đó, nên bạn sẽ dùng một tấm gỗ đẹp cho mặt lưng. Để đêm về ngủ ngon, cái đẹp, cái chất lượng, phải được giữ trọn vẹn từ đầu đến cuối.”

—Steve Jobs

—Steve Jobs

Aspiring to avoid a colleague quoting The Hitchhiker’s Guide to the Galaxy’s “...probably the oddest thing in the universe” when talking about your code is likely a noble pursuit. This was, of course, in reference to the legendary Babel Fish, which could translate any alien language directly into your ear.

Cố sao cho không đồng nghiệp nào phải trích câu “...có lẽ là thứ kỳ quặc nhất trong vũ trụ” của The Hitchhiker’s Guide to the Galaxy khi nói về mã của bạn — đó hẳn là một mục tiêu cao quý. Câu ấy, dĩ nhiên, nói về con Babel Fish huyền thoại, sinh vật có thể dịch thẳng mọi ngôn ngữ ngoài hành tinh vào tai bạn.

I am often reminded of The Hitchhiker’s Guide to the Galaxy when thinking of or reading **Clean Code** —not just because my father loves the book series and encouraged all of his children to read

it, not only because he references it in Clean Code along with his other books, and not only because Clean Code inspires deep thought. I am reminded of The Hitchhiker's Guide to the Galaxy because Clean Code is the must-have companion guide to navigating the ever-changing universe of software development.

Tôi thường nhớ tới The Hitchhiker's Guide to the Galaxy mỗi khi nghĩ về hay đọc Clean Code — không chỉ vì cha tôi mê bộ sách ấy và khuyến khích tất cả các con đọc nó, cũng không chỉ vì ông nhắc đến nó trong Clean Code cùng những cuốn sách khác của mình, và cũng không chỉ vì Clean Code khiến người ta phải nghĩ sâu. Tôi nhớ tới The Hitchhiker's Guide to the Galaxy vì Clean Code chính là cuốn cẩm nang không thể thiếu để định hướng trong cái vũ trụ luôn đổi thay của phát triển phần mềm.

This is not to say that this book will automatically allow you to understand any code. On the contrary, Clean Code is the inverted Babel Fish that you place at your fingertips so that other developers don't need a Babel Fish to understand your code!

Nói vậy không có nghĩa cuốn sách này sẽ tự động giúp bạn hiểu được mọi đoạn mã. Ngược lại, Clean Code là con Babel Fish đảo ngược mà bạn đặt ngay dưới đầu ngón tay mình, để những lập trình viên khác không cần tới bất kỳ Babel Fish nào mới hiểu được mã của bạn!

When I was in seventh grade, my dad had me code one of my first games: Hunt the Wumpus . It is a simple text-based adventure game whereby you traverse a cave to find and terminate the wumpus while avoiding traps, pits, and whatever else a clever developer can code to challenge you.

Hồi tôi học lớp bảy, cha bảo tôi lập trình một trong những trò chơi đầu tiên của đời mình: Hunt the Wumpus. Đó là một trò phiêu lưu dạng văn bản đơn giản, trong đó bạn đi xuyên một hang động để tìm và hạ gục con wumpus, đồng thời né tránh bẫy, hố sâu, và bất cứ thứ gì mà một lập trình viên tinh quái có thể viết ra để thách thức bạn.

Years later, I was volunteering at the Agile Conference in

Nhiều năm sau, tôi làm tình nguyện viên tại Agile Conference năm

2007. A workshop room had been set up for attendees to sit down and pair-program on a Hunt the Wumpus game built in Java. After hearing some great talks, I was eager to do some coding, so I took a sticky note off a community-managed Kanban-style whiteboard and sat down to start adding a new “fire arrow” feature. Soon after, none other than Ward Cunningham sat down beside me to start pairing. Being the young tyke that I was, I didn't recognize I was now coding with the man who built the first wiki.

2007. Người ta bố trí hẳn một phòng workshop để người tham dự ngồi xuống pair-program trên một bản Hunt the Wumpus viết bằng Java. Sau khi nghe vài bài nói rất hay, tôi nôn nóng muốn code, nên tôi bóc một tờ giấy nhớ khỏi tấm bảng trắng kiểu Kanban do cộng đồng tự quản rồi ngồi xuống bắt tay vào thêm tính năng mới “fire arrow”. Ngay sau đó, không ai khác ngoài Ward Cunningham ngồi xuống bên cạnh để pair cùng tôi. Là một thằng nhóc non nớt, tôi chẳng nhận ra mình đang code cùng người đã dựng nên wiki đầu tiên trên đời.

Unabashed, I charged forward and explained the feature to my new pair, and my plan to create an **inheritance structure** that would allow us to add not only “fire arrows” but also “ice arrows” or “arcane arrows” and so on. He nodded his head with grandiose solemnity and encouraged me to proceed.

Chẳng chút ngượng ngùng, tôi lao tới giải thích tính năng cho người bạn pair mới của mình, kèm kế hoạch tạo một cấu trúc kế thừa (inheritance) cho phép chúng tôi thêm không chỉ

“fire arrow” mà cả “ice arrow” hay “arcane arrow” và vân vân. Ông gật đầu với vẻ trịnh trọng hết mực rồi động viên tôi cứ tiếp tục.

I started by writing a **test** for a new Arrow **class**, which would later be the parent for our FireArrow class. I had already written this game, so I totally knew what I was doing. Once I had my failing test, I passed Ward the keyboard. He dutifully made the test pass, but then paused with a meaningful “hrmmpph” and proceeded to query whether we really ought to introduce such complexity so early. He posited that rather than jumping to our end-state **architecture**, we might try to add the functionality gradually and allow the test-driven process to reveal an appropriate shape. Frustrated because I knew how it ought to be architected, I begrudgingly agreed to follow his lead. I was here to learn, after all.

Tôi bắt đầu bằng việc viết một test cho class Arrow mới, thứ sau này sẽ là lớp cha của class FireArrow. Tôi từng viết trò này rồi, nên tôi thừa biết mình đang làm gì. Khi đã có test fail trong tay, tôi đưa bàn phím cho Ward. Ông ngoan ngoãn làm cho test pass, nhưng rồi khựng lại với một tiếng “hừmmm” đầy hàm ý và hỏi liệu chúng tôi có thật sự nên đưa vào ngần ấy độ phức tạp sớm đến thế không. Ông đặt vấn đề rằng thay vì nhảy thẳng tới kiến trúc (architecture) đích, có lẽ chúng tôi nên thêm chức năng từng chút một và để quy trình test-driven tự bộc lộ hình hài phù hợp. Bực bội vì tôi biết rõ nó phải được kiến trúc ra sao, tôi miễn cưỡng đồng ý làm theo ông. Dù sao thì tôi đến đây là để học.

His next move shocked me. He opened up the class with the code that launched the arrows and proceeded to **comment** a bunch of it out! Aghast, I righteously protested, thinking we were going backward rather than forward! In response, he showed how “everything is still green,” all the tests were passing, and that we might be able to get away with a little **refactoring**. The next 30 minutes passed in a series of moments of awe.

Nước cờ kế tiếp của ông làm tôi choáng váng. Ông mở class chứa mã bắn mũi tên ra rồi comment bỏ đi một mảng lớn trong đó! Kinh hãi, tôi phản đối đây chính nghĩa, bụng nghĩ chúng tôi đang đi lùi chứ không phải đi tới! Đáp lại, ông chỉ cho tôi thấy “mọi thứ vẫn xanh”, tất cả test đều pass, và rằng có lẽ chúng tôi có thể tranh thủ refactor một chút. Ba mươi phút sau đó trôi qua trong hết cơn kinh ngạc này đến cơn kinh ngạc khác.

First, we did a number of subtle refactorings, adding a few tests and renaming/extracting some functions. I remember being struck by the fluidity in which he moved through the phases of red, green, **refactor**. It became abundantly clear that this wasn’t just a process he used while coding. It was a principle to him—a way of thinking, problem-solving, and producing clean and clear results so ingrained into his workflow that the tests and the code were really one and the same.

Trước hết, chúng tôi thực hiện một loạt refactoring tinh tế, thêm vài test rồi đổi tên và extract vài hàm. Tôi vẫn nhớ mình đã sững sờ trước sự trôi chảy khi ông đi qua các pha red, green, refactor. Rõ mồn một rằng đây không đơn thuần là một quy trình ông dùng khi code. Với ông, đó là một nguyên tắc — một lối tư duy, một cách giải quyết vấn đề và tạo ra kết quả sạch sẽ, sáng sủa, ăn sâu vào nếp làm việc đến mức test và mã thực sự là một.

Second, we began adding the new functionality for a “fire arrow.” Now that everything was neatly laid out, this was simple. He wrote the failing test. I made it pass in what couldn’t have been more than a line or two of code. I had finished making it pass before even realizing we had actually completed the feature. When it dawned on me, he nodded appreciatively.

Thứ hai, chúng tôi bắt đầu thêm chức năng mới cho “fire arrow”. Giờ khi mọi thứ đã được bày biện gọn gàng, việc này hoá ra đơn giản. Ông viết test fail. Tôi làm nó pass bằng đúng một, cùng lắm là hai dòng mã. Tôi làm nó pass xong mà còn chưa kịp nhận ra rằng chúng tôi đã hoàn thành xong tính năng. Khi tôi vỡ lẽ, ông gật gù tán thưởng.

Next, he prompted me to grab the next feature. I seem to recall grabbing “ice arrow” but then being instructed toward greater ambition, and grabbed “crossbow bolt” instead, as it was going to be more challenging. However, after writing the failing test and passing Ward the keyboard, I knew my assumption was wrong, and it was remarkably simple to add this totally new weaponization against the wumpus.

Kế đó, ông giục tôi nhận tính năng tiếp theo. Hình như tôi định lấy “ice arrow” nhưng rồi được khuyên hãy tham vọng hơn, nên tôi lấy “crossbow bolt” vì nó hứa hẹn khó khăn hơn. Vậy mà sau khi viết test fail và chuyển bàn phím cho Ward, tôi biết mình đã đoán sai: thêm thứ vũ khí hoàn toàn mới này để đối phó con wumpus lại dễ đến bất ngờ.

Finally, after crushing the new crossbow bolt feature, he asked me what the code looked like. I think he might have used verbiage like “what does it smell like?” but I can’t be certain. After pondering for a moment, thinking about “ice arrows” and “arcane bolts,” the code spoke to me. We had already outlined an **interface** for Projectile, which could encompass a whole range of new weaponry for the would-be wumpus hunter. I proceeded to extract out the new Projectile interface, and all the tests were still a beautiful shimmering green. He nodded satisfactorily, stood, said it was fun, and thanked me for pairing before leaving the workspace. It was a long while before I escaped my rumination and casually added in several new projectiles.

Cuối cùng, sau khi xử đẹp tính năng crossbow bolt mới, ông hỏi tôi thấy mã bây giờ trông thế nào. Tôi ngờ rằng ông đã dùng chữ đại loại như “nó bốc mùi gì?”, nhưng không dám chắc. Ngẫm nghĩ một lát về “ice arrow” và “arcane bolt”, mã tự cất tiếng nói với tôi. Chúng tôi đã phác sẵn một interface Projectile, đủ sức bao trùm cả một dải vũ khí mới cho tay thợ săn wumpus tương lai. Tôi tiến hành extract ra interface Projectile mới, và tất cả test vẫn xanh lấp lánh tuyệt đẹp. Ông gật đầu hài lòng, đứng dậy, bảo rằng vui đấy, cảm ơn tôi đã pair cùng rồi rời khỏi chỗ làm việc. Phải khá lâu sau tôi mới thoát khỏi cơn trầm ngâm và thông thả thêm vào vài loại projectile mới.

Shortly thereafter, my dad walked over and asked me how I enjoyed pairing with Ward Cunningham, the creator of the wiki. Suddenly, everything made sense. I had been coding with a legend, and legends don’t frolic with principles—they live and breathe them. Within about a year, my dad codified many of these same principles in the first edition of Clean Code .

Ngay sau đó, cha tôi đi tới và hỏi tôi thấy pair với Ward Cunningham, cha đẻ của wiki, thế nào. Đột nhiên mọi thứ sáng tỏ. Tôi vừa code cùng một huyền thoại, mà huyền thoại thì không đùa nghịch với các nguyên tắc — họ sống và thở bằng chúng. Chỉ khoảng một năm sau, cha tôi đã hệ thống hoá phần nhiều trong số chính những nguyên tắc ấy vào ấn bản đầu tiên của Clean Code.

I tell this story to highlight the profound impact that can be made when the disciplines, strategies, and principles within this book are taken to heart.

Tôi kể câu chuyện này để làm nổi bật tác động sâu sắc có thể tạo ra khi những kỷ luật (discipline), chiến lược và nguyên tắc trong cuốn sách này được thấm vào tận tim.

Over the years, I have founded or worked at over a dozen early-stage startups. Oftentimes, I’d join a small team of developers who already had a product out the door serving a vast user base. More often than not, the driving principle was to go fast at all costs. The website/app is too slow? Throw more hardware at it. Product development is going slow? Hire more developers and product managers to speed things up. Move fast and break things.

Nhiều năm qua, tôi đã sáng lập hoặc làm việc tại hơn một chục startup giai đoạn đầu. Thường thì tôi gia nhập một nhóm lập trình viên nhỏ đã kịp đưa sản phẩm ra thị trường phục vụ lượng người dùng khổng lồ. Hầu như lúc nào nguyên tắc chủ đạo cũng là chạy

thật nhanh bằng mọi giá. Website/app chạy chậm quá? Ném thêm phần cứng vào. Phát triển sản phẩm ỉ ạch? Tuyển thêm lập trình viên và product manager cho nhanh. Move fast and break things.

Gradually, however, as the industry matured, teams started to focus more and more on having a robust test base, continuous deployment, code reviews, and, critically, clean code. In almost every case, the teams that followed the disciplines and principles laid out in this book were the ones with the highest product velocity. At first, I served as a steady reminder for everyone to write tests, use clear naming, and clean up or refactor the code. These days, it is often a given that the team will already be practicing these principles. Why? Because they work. Much of the startup world has already adopted these lessons because it is a winning strategy. The goal is still to go as fast as possible, but as my dad is fond of saying, “The only way to go fast is to go well.”

Tuy nhiên, dần dần, khi ngành này trưởng thành hơn, các nhóm bắt đầu chú trọng ngày một nhiều vào việc có một nền tảng test vững chắc, continuous deployment, code review, và — quan trọng bậc nhất — clean code. Gần như trong mọi trường hợp, những nhóm tuân theo các kỷ luật và nguyên tắc trình bày trong cuốn sách này chính là những nhóm có tốc độ ra sản phẩm cao nhất. Ban đầu, tôi đóng vai người nhắc nhở đều đặn rằng ai cũng phải viết test, đặt tên rõ ràng, dọn dẹp hoặc refactor mã. Ngày nay, chuyện cả nhóm vốn đã thực hành những nguyên tắc này thường là điều đương nhiên. Vì sao? Vì chúng hiệu quả. Phần lớn thế giới startup đã tiếp thu các bài học đó, bởi lẽ đây là một chiến lược chiến thắng. Mục tiêu vẫn là đi nhanh nhất có thể, nhưng như cha tôi hay nói: “Cách duy nhất để đi nhanh là đi cho tử tế.”

The world of software development is changing fast. Advanced AI tools like Claude, Cursor, GPT-o, and Grok have made it easier than ever to produce working code. Developers are already using them to kick off projects, debug tricky errors, or even whip up fully tested versions of Hunt the Wumpus . This trend isn’t slowing down—it’s accelerating.

Thế giới phát triển phần mềm đang đổi thay chóng mặt. Những công cụ AI tiên tiến như Claude, Cursor, GPT-o và Grok khiến việc tạo ra mã chạy được dễ hơn bao giờ hết. Lập trình viên đã dùng chúng để khởi động dự án, gỡ những lỗi hóc búa, thậm chí dựng luôn một phiên bản Hunt the Wumpus có đầy đủ test. Xu hướng này không hề chậm lại — nó đang tăng tốc.

It is for this reason that Clean Code principles are more crucial now than ever. AI can churn out code like a factory, but it’s often like raw dough—tasty maybe, but not a fully baked pie. Clean Code is the recipe that turns it into something polished, whether you’re refactoring by hand or tweaking AI prompts to guide the process. It’s about crafting software that doesn’t just work, but lasts.

Chính vì vậy mà các nguyên tắc Clean Code giờ đây còn thiết yếu hơn bao giờ hết. AI có thể tuôn ra mã như một nhà máy, nhưng thứ nó tuôn ra thường giống cục bột sống — có thể ngon đấy, nhưng chưa phải một chiếc bánh nướng chín tới. Clean Code là công thức biến nó thành thứ hoàn chỉnh, dù bạn refactor bằng tay hay tinh chỉnh prompt cho AI để dẫn dắt quá trình. Cốt lõi là làm ra phần mềm không chỉ chạy được, mà còn sống được lâu.

Whether you are using AI to write code or using AI as a personal Babel Fish to understand someone else’s code, if the intent is not inherent in the code, it is likely to be lost. Picture this: Grok spits out some code, you clean it up with Cursor and Claude, and it runs fine—until you realize it’s a tangled **mess**. So you ask GPT-o to sort it out, only to end up with a knot of interdependencies. Without Clean Code as a compass, AI’s output can quickly become a maintenance nightmare. The **craft** of writing clear, readable code remains essential.

Dù bạn dùng AI để viết mã hay dùng AI như một con Babel Fish riêng để hiểu mã của

người khác, nếu ý định không nằm sẵn trong chính mã thì nó rất dễ thất lạc. Hãy hình dung: Grok phun ra một đoạn mã, bạn dọn dẹp nó bằng Cursor và Claude, và nó chạy ngon lành — cho tới khi bạn nhận ra đó là một mớ bong bóng. Thế là bạn nhờ GPT-o gỡ rối, để rồi nhận về một búi phụ thuộc (dependency) chẳng chịt. Không có Clean Code làm la bàn, đầu ra của AI có thể nhanh chóng biến thành cơn ác mộng bảo trì. Tay nghề viết mã sáng sủa, dễ đọc vẫn là điều thiết yếu.

As AI keeps evolving, it might one day churn out better code on its own—but don't bet on it yet. Developers who stick to Clean Code principles will be the ones delivering high-quality, maintainable software no matter how smart AI gets. Clean Code is the guide that ensures you or your team's work isn't just functional—it's something to be proud of.

Khi AI tiếp tục tiến hoá, có thể một ngày nào đó nó sẽ tự tuôn ra mã tốt hơn — nhưng đừng vội đặt cược vào chuyện đó. Những lập trình viên bám chắc các nguyên tắc Clean Code sẽ là những người làm ra phần mềm chất lượng cao, dễ bảo trì, bất kể AI có thông minh đến đâu. Clean Code là kim chỉ nam bảo đảm rằng công việc của bạn hay của nhóm bạn không chỉ chạy được — mà còn là thứ đáng để tự hào.

Appendix. The Clean Code Debate — Phụ lục. Cuộc tranh luận về Clean Code



(Uncle) Bob Martin

(Uncle) Bob Martin



Dr. John Kenneth Ousterhout

Dr. John Kenneth Ousterhout

This document is the result of a series of discussions, some online and some in person, held between Robert “Uncle Bob” Martin and John Ousterhout between September 2024 and February 2025. If you would like to **comment** on anything in this discussion, we recommend that you do so on the Google group associated with APOSD. 1

Tài liệu này là kết quả của một loạt cuộc trao đổi, một số diễn ra trực tuyến và một số gặp mặt trực tiếp, giữa Robert “Uncle Bob” Martin và John Ousterhout trong khoảng từ tháng 9 năm 2024 đến tháng 2 năm 2025. Nếu bạn muốn góp ý về bất cứ điều gì trong cuộc thảo luận này, chúng tôi khuyến khích bạn góp ý trên nhóm Google gắn với APOSD. 1

1 [APOSD].

1 [APOSD].

Introductions — Lời giới thiệu

JOHN:

JOHN:

Hi (Uncle) Bob! You and I have each written books on software design. We agree on some things, but there are some pretty big differences of opinion between my recent book, *A Philosophy of Software Design* (hereafter “APOSD”), and your classic book **Clean Code**. Thanks for agreeing to discuss those differences here.

Chào (Uncle) Bob! Anh và tôi, mỗi người đều đã viết sách về thiết kế phần mềm. Chúng ta đồng ý với nhau ở một số điểm, nhưng giữa cuốn sách gần đây của tôi, *A Philosophy of Software Design* (từ đây gọi là “APOSD”), và cuốn sách kinh điển *Clean Code* của anh vẫn có những khác biệt quan điểm khá lớn. Cảm ơn anh đã nhận lời cùng bàn về những khác biệt đó ở đây.

UB:

UB:

My pleasure, John. Before we begin, let me say that I’ve carefully read through your book, and I found it very enjoyable and full of valuable insights. There are some things I disagree with you on, such as **TDD** and **abstraction**-first incrementalism, but overall I enjoyed it a lot.

Rất hân hạnh, John. Trước khi bắt đầu, tôi phải nói rằng tôi đã đọc kỹ cuốn sách của anh, và tôi thấy nó vừa thú vị vừa đầy những nhận định giá trị. Có vài chỗ tôi không đồng tình với anh, chẳng hạn về TDD và lối làm tăng dần theo hướng trừu tượng-trước, nhưng nhìn chung tôi rất thích nó.

JOHN:

JOHN:

I’d like to discuss three topics with you: **method** length, comments, and **test**-driven development. But before getting into these, let’s start by comparing overall philosophies. When you hear about a new idea related to software design, how do you decide whether or not to endorse that idea?

Tôi muốn bàn với anh ba chủ đề: độ dài của method, comment, và phát triển hướng kiểm thử. Nhưng trước khi đi vào những chuyện đó, hãy bắt đầu bằng việc so sánh triết lý tổng

thể của hai bên. Khi nghe về một ý tưởng mới liên quan tới thiết kế phần mềm, anh dựa vào đâu để quyết định có ủng hộ ý tưởng đó hay không?

I'll go first. For me, the fundamental goal of software design is to make it easy to understand and modify the system. I use the term complexity to refer to things that make it hard to understand and modify a system. The most important contributors to complexity relate to information.

Tôi xin đi trước. Với tôi, mục tiêu căn bản của thiết kế phần mềm là làm cho hệ thống dễ hiểu và dễ sửa. Tôi dùng từ *độ phức tạp* để chỉ những thứ khiến hệ thống trở nên khó hiểu và khó sửa. Những yếu tố đóng góp lớn nhất vào độ phức tạp đều liên quan tới thông tin.

- How much information must a developer have in their head in order to carry out a task?
- How accessible and obvious is the information that the developer needs?
 - Một lập trình viên phải giữ bao nhiêu thông tin trong đầu mới hoàn thành được một công việc?
 - Những thông tin mà lập trình viên đó cần có dễ tiếp cận và hiển nhiên đến mức nào?

The more information a developer needs to have, the harder it will be for them to work on the system. Things get even worse if the required information isn't obvious. The worst case is when there is a crucial piece of information hidden in some faraway piece of code that the developer has never heard of.

Lập trình viên càng cần nắm nhiều thông tin thì việc làm trên hệ thống càng khó khăn. Mọi thứ còn tệ hơn nữa nếu những thông tin cần thiết ấy lại không hiển nhiên. Trường hợp tệ nhất là khi có một mẫu thông tin then chốt nằm giấu trong một đoạn mã xa lắc mà lập trình viên chưa từng nghe nói tới.

When I'm evaluating an idea related to software design, I ask whether it will reduce complexity. This usually means either reducing the amount of information a developer has to know, or making the required information more obvious.

Khi đánh giá một ý tưởng về thiết kế phần mềm, tôi hỏi xem nó có làm giảm độ phức tạp hay không. Điều đó thường có nghĩa là hoặc giảm lượng thông tin mà lập trình viên phải biết, hoặc làm cho những thông tin cần thiết trở nên hiển nhiên hơn.

Now over to you: Are there general principles that you use when deciding which ideas to endorse?

Giờ đến lượt anh: Anh có nguyên tắc chung nào khi quyết định ủng hộ ý tưởng nào không?

UB:

UB:

I agree with your approach. A **discipline** or technique should make the job of programmers easier. I would add that the programmer we want to help most is not the author. The programmer whose job we want to make easier is the programmer who must read and understand the code written by others (or by themselves a week later). Programmers spend far more hours reading code than writing code, so the activity we want to ease is that of reading.

Tôi đồng ý với cách tiếp cận của anh. Một kỷ luật (discipline) hay một kỹ thuật thì phải làm cho công việc của lập trình viên dễ dàng hơn. Tôi xin bổ sung rằng người lập trình mà ta muốn giúp nhất không phải là tác giả đoạn mã. Người lập trình mà ta muốn làm nhẹ gánh chính là người phải đọc và hiểu đoạn mã do người khác viết (hoặc do chính mình viết một tuần trước). Lập trình viên dành nhiều giờ đọc mã hơn hẳn số giờ viết mã, nên hoạt động mà ta cần làm cho dễ chịu hơn chính là việc đọc.

Method Length — Độ dài của method

JOHN:

JOHN:

Our first area of disagreement is method length. On page 34 of *Clean Code*, you say: “The first rule of functions is that they should be small. The second rule of functions is that they should be smaller than that.” Later on, you say, “Functions should hardly ever be 20 lines long” and suggest that functions should be “just two, three, or four lines long.” On page 35, you say: “Blocks within if statements, else statements, while statements, and so on should be one line long. Probably that line should be a **function** call.” I couldn’t find anything in *Clean Code* to suggest that a function could ever be too short.

Điểm bất đồng đầu tiên của chúng ta là độ dài của method. Ở trang 34 của *Clean Code*, anh viết: “Quy tắc đầu tiên của hàm là chúng phải nhỏ. Quy tắc thứ hai của hàm là chúng phải *nhỏ hơn thế nữa*.” Sau đó anh viết tiếp: “Hàm hầu như không bao giờ nên dài tới 20 dòng” và gợi ý rằng hàm nên “chỉ dài hai, ba, hoặc bốn dòng.” Ở trang 35, anh viết: “Các khối bên trong câu lệnh if, câu lệnh else, câu lệnh while, v.v. nên dài đúng một dòng. Nhiều khả năng dòng đó nên là một lời gọi hàm.” Tôi không tìm thấy chỗ nào trong *Clean Code* gợi ý rằng một hàm có thể *quá ngắn*.

I agree that dividing up code into relatively small units (modular design) is one of the most important ways to reduce the amount of information a programmer has to keep in their mind at once. The idea, of course, is to take a complex chunk of functionality and encapsulate it in a separate method with a simple **interface**. Developers can then harness the functionality of the method (or read code that invokes the method) without learning the details of how the method is implemented; they only need to learn its interface. The best methods are those that provide a lot of functionality but have a very simple interface: They replace a large cognitive load (reading the detailed **implementation**) with a much smaller cognitive load (learning the interface). I call these methods deep.

Tôi đồng ý rằng chia mã thành những đơn vị tương đối nhỏ (*thiết kế theo module*) là một trong những cách quan trọng nhất để giảm lượng thông tin mà lập trình viên phải giữ trong đầu cùng lúc. Ý tưởng, tất nhiên, là lấy một mảng chức năng phức tạp rồi đóng gói nó vào một method riêng với interface đơn giản. Nhờ đó lập trình viên có thể tận dụng chức năng của method (hoặc đọc đoạn mã gọi method đó) mà không cần tìm hiểu chi tiết method được hiện thực (implementation) ra sao; họ chỉ cần nắm interface của nó. Những method tốt nhất là những method cung cấp thật nhiều chức năng nhưng có interface rất đơn giản: Chúng thay một gánh nặng nhận thức lớn (đọc phần hiện thực chi tiết) bằng một gánh nặng nhận thức nhỏ hơn nhiều (học interface). Tôi gọi những method như thế là *sâu* (deep).

However, like most ideas in software design, decomposition can be taken too far. As methods get smaller and smaller, there is less and less benefit to further subdivision. The amount of functionality hidden behind each interface drops, while the interfaces often become more complex. I call these interfaces shallow: They don’t help much in terms of reducing what the programmer needs to know. Eventually, the point is reached where someone using the method needs to understand every aspect of its implementation. Such methods are usually pointless.

Tuy nhiên, như hầu hết mọi ý tưởng trong thiết kế phần mềm, việc phân rã có thể bị đẩy đi quá xa. Khi các method ngày một nhỏ đi, lợi ích của việc chia nhỏ thêm cũng ngày một ít. Lượng chức năng được giấu sau mỗi interface tụt xuống, trong khi bản thân các interface lại thường phức tạp hơn. Tôi gọi những interface như vậy là *nông* (shallow): Chúng chẳng giúp được bao nhiêu trong việc giảm bớt những thứ lập trình viên cần biết. Rốt cuộc sẽ

đến lúc người dùng method phải hiểu mọi khía cạnh trong phần hiện thực của nó. Những method như thế thường là vô nghĩa.

Another problem with decomposing too far is that it tends to result in entanglement . Two methods are entangled (or conjoined in APOSD terminology) if, in order to understand how one of them works internally, you also need to read the code of the other. If you've ever found yourself flipping back and forth between the implementations of two methods as you read code, that's a red flag that the methods might be entangled. Entangled methods are hard to read because the information you need to have in your head at once isn't all in the same place. Entangled methods can usually be improved by combining them so that all the code is in one place.

Một vấn đề nữa của việc phân rã quá đà là nó có xu hướng dẫn tới *sự bện rối* (entanglement). Hai method bị bện rối (hay *conjoined* theo thuật ngữ của APOSD) nếu, để hiểu một trong hai cái hoạt động ra sao bên trong, bạn buộc phải đọc cả mã của cái kia. Nếu bạn từng thấy mình phải lật qua lật lại giữa phần hiện thực của hai method trong lúc đọc mã, đó là dấu hiệu đỏ cho thấy hai method đó có thể đang bện rối vào nhau. Method bện rối thì khó đọc, vì những thông tin bạn cần giữ trong đầu cùng một lúc lại không nằm chung một chỗ. Method bện rối thường có thể được cải thiện bằng cách gộp chúng lại để toàn bộ mã nằm ở một nơi.

The advice in Clean Code on method length is so extreme that it encourages programmers to create teeny-tiny methods that suffer from both shallow interfaces and entanglement. Setting arbitrary numerical limits such as two to four lines in a method and a single line in the body of an if or while statement exacerbates this problem.

Lời khuyên trong *Clean Code* về độ dài method cực đoan đến mức nó khuyến khích lập trình viên tạo ra những method bé tí xiu, vừa mang interface nông vừa bện rối vào nhau. Việc đặt ra những giới hạn số học tùy tiện như hai đến bốn dòng cho một method và một dòng cho thân của câu lệnh if hay while lại càng làm vấn đề trầm trọng thêm.

UB:

UB:

While I do strongly recommend very short functions, I don't think it's fair to say that the book sets arbitrary numerical limits. The two-to-four-line functions that you referred to on page 34 were part of the Sparkle applet that Kent Beck and I wrote together in 1999 as an exercise for learning TDD. I thought it was remarkable that most of the functions in that applet were two to four lines long because it was a Swing program; and Swing programs tend to have very long methods.

Dù tôi thực sự khuyến nghị viết hàm rất ngắn, tôi không nghĩ là công bằng khi nói cuốn sách đặt ra những giới hạn số học tùy tiện. Những hàm dài hai đến bốn dòng mà anh nhắc tới ở trang 34 là một phần của applet Sparkle mà Kent Beck và tôi cùng viết năm 1999 như một bài tập để học TDD. Tôi thấy đáng chú ý là phần lớn hàm trong applet đó chỉ dài hai đến bốn dòng, bởi vì nó là một chương trình Swing; mà chương trình Swing thì thường có những method rất dài.

As for setting limits, on page 13 I make clear that although the recommendations in the book have worked well for me and the other authors, they might not work for everyone. I claimed no final authority, nor even any absolute "rightness." They are offered for consideration.

Còn về chuyện đặt giới hạn, ở trang 13 tôi đã nói rõ rằng dù những khuyến nghị trong sách đã phát huy tác dụng tốt với tôi và các tác giả khác, chúng có thể không phù hợp với tất cả mọi người. Tôi không hề tuyên bố mình là thẩm quyền tối hậu, cũng chẳng khẳng định một sự "đúng đắn" tuyệt đối nào. Chúng được đưa ra để bạn đọc cân nhắc.

JOHN:

JOHN:

I think these problems will be easiest to understand if we look at specific code examples. But before we do that, let me ask you, Bob: Do you believe that it's possible for code to be over-decomposed, or is smaller always better? And, if you believe that over-decomposition is possible, how do you recognize when it has occurred?

Tôi nghĩ những vấn đề này sẽ dễ hiểu nhất nếu ta nhìn vào các ví dụ mã cụ thể. Nhưng trước khi làm vậy, cho tôi hỏi anh, Bob: Anh có tin rằng mã có thể bị *phân rã quá mức* không, hay cứ nhỏ hơn thì luôn tốt hơn? Và nếu anh tin rằng phân rã quá mức là chuyện có thật, thì làm sao anh nhận ra khi nó xảy ra?

UB:

UB:

It is certainly possible to over-decompose code. Here's an example:

Chắc chắn là mã có thể bị phân rã quá mức. Đây là một ví dụ:

```
void doSomething() {doTheThing()} // over-decomposed.
```

The strategy that I use for deciding how far to take decomposition is the old rule that a method should do One Thing . If I can meaningfully extract one method from another, then the original method did more than one thing. “Meaningfully” means that the extracted functionality can be given a descriptive name, and that it does less than the original method.

Chiến lược tôi dùng để quyết định phân rã tới đâu là quy tắc cũ rằng một method chỉ nên làm *One Thing* — một việc duy nhất. Nếu tôi có thể trích xuất một method ra khỏi method khác một cách *có ý nghĩa*, thì method ban đầu đã làm nhiều hơn một việc. “Có ý nghĩa” tức là phần chức năng được trích ra có thể đặt cho một cái tên mô tả được, và nó làm ít việc hơn method ban đầu.

JOHN:

JOHN:

Unfortunately, the One Thing approach will lead to over-decomposition:

Đáng tiếc là cách tiếp cận One Thing sẽ dẫn tới phân rã quá mức:

The term one thing is vague and easy to abuse. For example, if a method has two lines of code, isn't it doing two things?

Cụm *một việc* mơ hồ và dễ bị lạm dụng. Chẳng hạn, nếu một method có hai dòng mã, chẳng phải nó đang làm hai việc sao?

You haven't provided any useful guardrails to prevent over-decomposition. The example you gave is too extreme to be useful, and the “can it be named” qualification doesn't help: Anything can be named.

Anh không đưa ra được rào chắn hữu ích nào để ngăn phân rã quá mức. Ví dụ anh đưa ra cực đoan tới mức chẳng dùng được, còn điều kiện “có đặt tên được không” thì chẳng giúp gì: Cái gì mà chẳng đặt tên được.

The One Thing approach is simply wrong in many cases. If two things are closely related, it might well make sense to implement them in a single method. For example, any **thread-safe** method will

first have to acquire a **lock**, then carry out its function. These are two “things,” but they belong in the same method.

Cách tiếp cận One Thing đơn giản là sai trong nhiều trường hợp. Nếu hai thứ gắn bó chặt chẽ với nhau, hoàn toàn có lý khi hiện thực chúng trong một method duy nhất. Ví dụ, bất kỳ method an toàn luồng (thread-safe) nào cũng phải giành lock trước rồi mới thực hiện chức năng của nó. Đó là hai “việc,” nhưng chúng thuộc về cùng một method.

UB:

UB:

Let me tackle the last thing first. You suggested that locking the **thread** and performing a critical section should be together in the same method. However, I would be tempted to separate the locking from the critical section.

Để tôi xử lý ý cuối trước. Anh cho rằng việc khoá luồng (thread) và việc thực thi critical section nên nằm chung một method. Tuy nhiên, tôi lại có xu hướng tách phần khoá ra khỏi critical section.

```
void concurrentOperation() {  
    lock()  
    criticalSection();  
    unlock()  
}
```

This decouples the critical section from the lock and allows it to be called at times when locking isn’t necessary (e.g., in single-thread mode) or when the lock has already been set by someone else.

Cách này tách rời (decoupling) critical section khỏi lock và cho phép gọi nó vào những lúc không cần khoá (ví dụ, ở chế độ đơn luồng) hoặc khi lock đã được ai đó thiết lập sẵn.

Now, on to the “ease of abuse” **argument**. I don’t consider that to be a significant concern. If statements are easy to abuse. Switch statements are easy to abuse. Assignment statements are easy to abuse. The fact that something is easy to abuse does not mean that it should be avoided or suppressed. It simply means people should take appropriate care. There will always be this thing called judgment .

Bây giờ, sang lập luận “dễ bị lạm dụng.” Tôi không coi đó là mối lo đáng kể. Câu lệnh if cũng dễ bị lạm dụng. Câu lệnh switch cũng dễ bị lạm dụng. Câu lệnh gán cũng dễ bị lạm dụng. Việc một thứ dễ bị lạm dụng không có nghĩa là nó phải bị né tránh hay dẹp bỏ. Nó chỉ có nghĩa là người ta nên cẩn trọng cho phải phép. Sẽ luôn tồn tại một thứ gọi là *óc phán đoán*.

So, when faced with this snippet of code in a larger method:

Vậy nên, khi gặp đoạn mã sau đây nằm trong một method lớn hơn:

```
...  
amountOwed=0;  
totalPoints=0;  
...
```

it would be poor judgment to extract them as follows, because the extraction is not meaningful. The implementation is not more deeply detailed than the interface.

sẽ là phán đoán kém nếu trích xuất chúng như dưới đây, bởi việc trích xuất đó không mang ý nghĩa gì. Phần hiện thực không chi tiết sâu hơn phần interface.

```
void clearAmountOwed() {
    amountOwed=0;
}

void clearTotalPoints() {
    totalPoints=0;
}
```

However, it may be **good** judgment to extract them as follows because the interface is abstract, and the implementation has deeper detail.

Tuy nhiên, có thể là phán đoán tốt nếu trích xuất chúng như sau, bởi interface mang tính trừu tượng, còn phần hiện thực có chi tiết sâu hơn.

```
void clearTotals() {
    amountOwed=0;
    totalPoints=0;
}
```

The latter has a nice descriptive name that is abstract enough to be meaningful without being redundant. And the two lines together are strongly related so as to qualify for doing one thing : initialization.

Cách sau có một cái tên mô tả dễ chịu, đủ trừu tượng để có ý nghĩa mà không thừa thãi. Và hai dòng đó gắn bó chặt với nhau đến mức đủ tư cách để coi là làm *một việc*: khởi tạo.

JOHN:

JOHN:

Of course, anything can be abused. But the best approaches to design encourage people to do things the right way and discourage abuse. Unfortunately, the One Thing rule encourages abuse for the reasons I gave above.

Dĩ nhiên là cái gì cũng có thể bị lạm dụng. Nhưng những cách tiếp cận thiết kế tốt nhất là những cách khuyến khích người ta làm đúng và cản trở việc lạm dụng. Đáng tiếc, quy tắc One Thing lại khuyến khích lạm dụng, vì những lý do tôi vừa nêu.

And of course, software designers will need to use judgment: It isn't possible to provide precise recipes for software design. But good judgment requires principles and guidance. The Clean Code arguments about decomposition, including the One Thing rule, are one-sided. They give strong, concrete, quantitative advice about when to chop things up, with virtually no guidance for how to tell you've gone too far. All I could find is a two-sentence example on page 36 about Listing 3-3 (which is pretty trivial), buried in the middle of exhortations to "chop, chop, chop."

Và tất nhiên, người thiết kế phần mềm sẽ phải dùng óc phán đoán: Không thể đưa ra công thức chính xác cho thiết kế phần mềm. Nhưng phán đoán tốt đòi hỏi nguyên tắc và chỉ dẫn. Những lập luận của *Clean Code* về phân rã, kể cả quy tắc One Thing, đều một chiều. Chúng đưa ra lời khuyên mạnh mẽ, cụ thể, có định lượng về *khi nào nên chặt nhỏ*, mà gần như chẳng có chỉ dẫn nào giúp bạn nhận ra mình đã đi quá xa. Tất cả những gì tôi tìm được là một ví dụ dài hai câu ở trang 36 về Listing 3-3 (khá tầm thường), bị chôn giữa những lời hô hào "chặt, chặt, chặt nữa."

One of the reasons I use the deep/shallow characterization is that it captures both sides of the trade-off; it will tell you when a decomposition is good and also when decomposition makes things worse.

Một trong những lý do tôi dùng cặp khái niệm sâu/nông là vì nó nắm bắt được cả hai phía của sự đánh đổi; nó cho bạn biết khi nào một phép phân rã là tốt, và cũng cho biết khi nào

phân rã làm mọi thứ tệ đi.

UB:

UB:

You make a good point that I don't talk much in the book about how to make the judgment call. Back in 2008, my concern was breaking the habit of the very large functions that were common in those early days of the Web. I have been more balanced in the second edition.

Anh nói rất đúng rằng trong sách tôi không bàn nhiều về chuyện làm sao đưa ra phán đoán. Hồi năm 2008, mối bận tâm của tôi là phá bỏ thói quen viết hàm cực dài vốn rất phổ biến trong những ngày đầu của Web. Ở lần tái bản thứ hai này, tôi đã cân bằng hơn.

Still, if I must err, I'd rather err on the side of decomposition. There is value in considering and visualizing decompositions. They can always be inlined if we judge them to have gone too far.

Dù vậy, nếu buộc phải sai, tôi thà sai về phía phân rã. Việc cân nhắc và hình dung ra các phương án phân rã tự nó đã có giá trị. Lúc nào ta cũng có thể inline chúng trở lại nếu thấy mình đã đi quá xa.

JOHN:

JOHN:

Coming back to your clearTotals example:

Trở lại ví dụ clearTotals của anh:

- The clearTotals method seems to contradict the One Thing rule: The variables amountOwed and totalPoints don't seem particularly related, so initializing them both is doing two things, no? You say that both statements are performing initialization, which makes it just one thing (initialization). Does that mean it would also be OK to have a single method that initializes two completely independent objects with nothing in common? I suspect not. It feels like you are struggling to create a clean framework for applying the One Thing rule; that makes me think it isn't a good rule.
- Without seeing more context, I'm skeptical that the
 - Method clearTotals có vẻ mâu thuẫn với chính quy tắc One Thing: Hai biến amountOwed và totalPoints trông chẳng liên quan gì mấy tới nhau, nên khởi tạo cả hai là làm hai việc, đúng không? Anh nói cả hai câu lệnh đều đang thực hiện việc khởi tạo, và như thế nó chỉ là một việc (khởi tạo). Vậy có nghĩa là cũng không sao nếu có một method khởi tạo hai đối tượng (object) hoàn toàn độc lập, chẳng có gì chung với nhau? Tôi ngờ là không. Cảm giác là anh đang vật lộn để dựng lên một khung sạch sẽ cho việc áp dụng quy tắc One Thing; điều đó khiến tôi nghĩ rằng nó không phải một quy tắc tốt.
 - Nếu chưa thấy thêm ngữ cảnh, tôi vẫn hoài nghi rằng

clearTotals method makes sense.

method clearTotals là hợp lý.

UB:

UB:

I hope you agree that between these two examples, the former is a bit better.

Tôi hy vọng anh đồng ý rằng giữa hai ví dụ này, cái trước có khá hơn một chút.

```

public String makeStatement() {
    clearTotals();
    return makeHeader() + makeRentalDetails() + makeFooter();
}
public String makeStatement() {
    amountOwed=0;
    totalPoints=0;
    return makeHeader() + makeRentalDetails() + makeFooter();
}

```

JOHN:

JOHN:

Well, actually, no. The second example is completely clear and obvious: I don't see anything to be gained by splitting it up.

Ừm, thực ra là không. Ví dụ thứ hai hoàn toàn sáng rõ và hiển nhiên: Tôi chẳng thấy được lợi lộc gì khi chẻ nó ra.

SPOCK (a.k.a. UB):

SPOCK (tức UB):

Fascinating.

Kỳ thú thay.

JOHN:

JOHN:

I think it will be easier to clarify our differences if we consider a nontrivial code example. Let's look at the

Tôi nghĩ sẽ dễ làm rõ khác biệt giữa chúng ta hơn nếu ta xét một ví dụ mã không tầm thường. Hãy nhìn vào

PrimeGenerator **class** from Clean Code , which is Listing 10-8 on pages 145–146. This Java class generates the first N prime numbers:

class PrimeGenerator trong *Clean Code*, tức Listing 10-8 ở trang 145–146. Class Java này sinh ra N số nguyên tố đầu tiên:

```

package literatePrimes;

import java.util.ArrayList;
public class PrimeGenerator {
    private static int[] primes;
    private static ArrayList<Integer> multiplesOfPrimeFactors;
    protected static int[] generate(int n) {
        primes = new int[n];
        multiplesOfPrimeFactors = new ArrayList<Integer>();
        set2AsFirstPrime();
        checkOddNumbersForSubsequentPrimes();
        return primes;
    }
    private static void set2AsFirstPrime() {
        primes[0] = 2;
    }
}

```

```

        multiplesOfPrimeFactors.add(2);
    }
    private static void checkOddNumbersForSubsequentPrimes() {
        int primeIndex = 1;
        for (int candidate = 3;
            primeIndex < primes.length;
            candidate += 2) {
            if (isPrime(candidate))
                primes[primeIndex++] = candidate;
        }
    }
    private static boolean isPrime(int candidate) {
        if (isLeastRelevantMultipleOfLargerPrimeFactor(candidate))
            multiplesOfPrimeFactors.add(candidate);
        return false;
    }
    return isNotMultipleOfAnyPreviousPrimeFactor(candidate);
}
private static boolean
isLeastRelevantMultipleOfLargerPrimeFactor(int candidate) {
    int nextLargerPrimeFactor =
        primes[multiplesOfPrimeFactors.size()];
    int leastRelevantMultiple =
        nextLargerPrimeFactor * nextLargerPrimeFactor;
    return candidate == leastRelevantMultiple;
}
private static boolean
isNotMultipleOfAnyPreviousPrimeFactor(int candidate) {
    for (int n = 1; n < multiplesOfPrimeFactors.size(); n++) {
        if (isMultipleOfNthPrimeFactor(candidate, n))
            return false;
    }
    return true;
}
private static boolean
isMultipleOfNthPrimeFactor(int candidate, int n) {
    return
        candidate ==
        smallestOddNthMultipleNotLessThanCandidate(candidate, n)
}
private static int
smallestOddNthMultipleNotLessThanCandidate(int candidate, int
{
    int multiple = multiplesOfPrimeFactors.get(n);
    while (multiple < candidate)
        multiple += 2 * primes[n];
    multiplesOfPrimeFactors.set(n, multiple);
    return multiple;
}

```

}

Before we dive into this code, I'd encourage everyone reading this article to take time to read over the code and draw your own conclusions about it. Did you find the code easy to understand? If so, why? If not, what makes it complex?

Trước khi lao vào đoạn mã này, tôi muốn khuyến khích mọi độc giả của bài viết này dành thời gian đọc kỹ đoạn mã và tự rút ra kết luận của riêng mình. Bạn có thấy đoạn mã dễ hiểu không? Nếu có, vì sao? Nếu không, điều gì khiến nó phức tạp?

Also, Bob, can you confirm that you stand by this code (i.e., the code properly exemplifies the design philosophy of Clean Code and this is the way you believe the code should appear if it were used in production)?

Và Bob này, anh có thể xác nhận rằng anh vẫn bảo lưu đoạn mã này không (tức là nó thể hiện đúng triết lý thiết kế của *Clean Code*, và đây là cách anh cho rằng đoạn mã nên trông như vậy nếu nó được dùng trong production)?

UB:

UB:

Ah, yes. The PrimeGenerator . This code comes from the 1982 paper “Literate Programming” 2 written by Donald Knuth.

Ồ, đúng rồi. PrimeGenerator. Đoạn mã này lấy từ bài báo “Literate Programming” 2 do Donald Knuth viết năm 1982.

The program was originally written in Pascal, and was automatically generated by Knuth’s WEB system into a single, very large method, which I translated into Java.

Chương trình ban đầu được viết bằng Pascal, và được hệ thống WEB của Knuth tự động sinh ra thành một method duy nhất rất lớn, mà tôi đã dịch sang Java.

2 <https://www.cs.tufts.edu/~nr/cs257/archive/literate-programming/01-knuth-lp.pdf>

2 <https://www.cs.tufts.edu/~nr/cs257/archive/literate-programming/01-knuth-lp.pdf>

Of course, this code was never meant for production. Both Knuth and I used it as a pedagogical example. In Clean Code , it appears in a chapter named “Classes.” The lesson of the chapter is that a very large method will often contain many different sections of code that are better decomposed into independent classes.

Tất nhiên, đoạn mã này chưa bao giờ được viết ra để chạy trong production. Cả Knuth lẫn tôi đều dùng nó như một ví dụ mang tính sư phạm. Trong *Clean Code*, nó xuất hiện ở chương mang tên “Classes.” Bài học của chương đó là một method rất lớn thường chứa nhiều mảng mã khác nhau, và những mảng ấy sẽ tốt hơn nếu được phân rã thành các class độc lập.

In the chapter, I extracted three classes from that function:

Trong chương đó, tôi đã trích xuất ba class ra từ hàm ấy:

PrimePrinter, RowColumnPagePrinter, and PrimeGenerator .

One of those extracted classes was the PrimeGenerator . It had the following code (which I did not publish in the book). The **variable** names and the overall **structure** are Knuth’s.

Một trong những class được trích xuất chính là PrimeGenerator. Nó có đoạn mã sau (đoạn mà tôi đã không in trong sách). Tên biến và cấu trúc tổng thể là của Knuth.


```

public class PrimeGenerator {
    protected static int[] generate(int n) {
        int[] p = new int[n];
        ArrayList<Integer> mult = new ArrayList<Integer>();
        p[0] = 2;
        mult.add(2);
        int k = 1;
        for (int j = 3; k < p.length; j += 2) {
            boolean jprime = false;
            int ord = mult.size();
            int square = p[ord] * p[ord];
            if (j == square) {
                mult.add(j);
            } else {
                jprime=true;
                for (int mi = 1; mi < ord; mi++) {
                    int m = mult.get(mi);
                    while (m < j)
                        m += 2 * p[mi];
                    mult.set(mi, m);
                    if (j == m) {
                        jprime = false;
                        break;
                    }
                }
            }
            if (jprime)
                p[k++] = j;
        }
        return p;
    }
}

```

Even though I was done with the lesson of the chapter, I didn't want to leave that method looking so outdated. So I cleaned it up a bit as an afterthought. My goal was not to describe how to generate prime numbers. I wanted my readers to see how large methods, that violate the Single Responsibility Principle, can be broken down into a few smaller, well-named classes containing a few smaller, well-named methods.

Dù đã xong bài học của chương, tôi vẫn không muốn để method đó trông cũ kỹ đến thế. Nên tôi dọn dẹp nó một chút, coi như chuyện làm thêm cho vui. Mục tiêu của tôi không phải là mô tả cách sinh số nguyên tố. Tôi muốn bạn đọc thấy được rằng những method lớn, vốn vi phạm Single Responsibility Principle, có thể được bẻ nhỏ thành vài class nhỏ hơn với tên gọi khéo, mỗi class chứa vài method nhỏ hơn cũng với tên gọi khéo.

JOHN:

JOHN:

Thanks for the background. Even though the details of that code weren't the main point of the chapter, presumably the code represents what you think is the "right" and "cleanest" way to do things, given the algorithm at hand. And that's where I disagree.

Cảm ơn anh về phần bối cảnh. Dù chi tiết của đoạn mã đó không phải trọng tâm của chương, ta vẫn có thể giả định rằng đoạn mã ấy thể hiện điều anh cho là cách làm “đúng” và “sạch nhất”, với thuật toán đang có trong tay. Và đó chính là chỗ tôi không đồng tình.

There are many design problems with PrimeGenerator , but for now I'll focus on method length. The code is chopped up so much (eight teeny-tiny methods) that it's difficult to read. For starters, consider the isNotMultipleOfAnyPreviousPrimeFactor method. This method invokes isMultipleOfNthPrimeFactor , which

PrimeGenerator có rất nhiều vấn đề về thiết kế, nhưng bây giờ tôi sẽ chỉ tập trung vào độ dài method. Đoạn mã bị chặt nhỏ đến mức (tám method bé tí xiu) khó mà đọc nổi. Để mở màn, hãy xét method isNotMultipleOfAnyPreviousPrimeFactor. Method này gọi isMultipleOfNthPrimeFactor, mà

invokes smallestOddNthMultipleNotLessThanCandidate . These

methods are shallow and entangled: In order to understand

method đều nông và bện rối vào nhau: Để hiểu được

isNot... , you have to read the other two methods and load all of that code into your mind at once. For example, isNot... has side effects (it modifies multiplesOfPrimeFactors), but you can't see that unless you read all three methods.

isNot..., bạn phải đọc cả hai method kia và nạp toàn bộ đoạn mã ấy vào đầu cùng một lúc. Chẳng hạn, isNot... có tác dụng phụ (side effect) — nó sửa đổi multiplesOfPrimeFactors — nhưng bạn không thể thấy điều đó trừ phi đọc cả ba method.

UB:

UB:

I think you have a point. Eighteen years ago, when I was in the throes of this **refactoring**, the names and structure made perfect sense to me. They make sense to me now, too—but that's because I once again understand the algorithm. When I returned to the algorithm for the first time a few days ago, I struggled with the names and structure. Once I understood the algorithm, the names and structure made perfect sense.

Tôi nghĩ anh có lý. Mười tám năm trước, khi tôi đang mải mê với cuộc refactoring này, các tên gọi và cấu trúc đó hoàn toàn hợp lý với tôi. Bây giờ chúng vẫn hợp lý với tôi — nhưng chỉ vì tôi đã hiểu lại thuật toán một lần nữa. Khi tôi quay lại với thuật toán này lần đầu cách đây vài hôm, tôi đã vật lộn với đám tên gọi và cấu trúc ấy. Một khi đã hiểu thuật toán, tên gọi và cấu trúc lại trở nên hoàn toàn hợp lý.

JOHN:

JOHN:

Those names are problematic even for someone who understands the algorithm; we'll talk about them a bit later, when discussing comments. And if code no longer makes sense to the writer when the writer returns to the code later, that means the code is problematic. The fact that code can eventually be understood (with great pain and suffering) does not excuse its entanglement.

Những cái tên đó gây rắc rối ngay cả với người đã hiểu thuật toán; ta sẽ bàn về chúng sau một chút, khi nói tới comment. Và nếu chính người viết quay lại mà đoạn mã không còn hợp lý với họ nữa, thì điều đó có nghĩa là đoạn mã có vấn đề. Việc rốt cuộc rồi cũng hiểu được đoạn mã (sau bao đau đớn và khổ sở) không hề bào chữa cho sự bện rối của nó.

UB:

UB:

Would that we had such a crystal ball that we could help our future selves avoid such “ great pain and suffering .” ;-)

Ước gì chúng ta có được quả cầu tiên tri để giúp bản thân trong tương lai tránh khỏi những phen “đau đớn và khổ sở” như thế. ;-)

JOHN:

JOHN:

There is no need for a crystal ball. The problems with

Chẳng cần quả cầu tiên tri nào cả. Những vấn đề của

PrimeGenerator are pretty obvious, such as the entanglement and interface complexity; maybe you were surprised that it is hard to understand, but I am not. Said another way, if you are unable to predict whether your code will be easy to understand, there are problems with your design methodology.

PrimeGenerator khá hiển nhiên, chẳng hạn sự bện rối và độ phức tạp của interface; có thể anh ngạc nhiên khi thấy nó khó hiểu, nhưng tôi thì không. Nói cách khác, nếu bạn không dự đoán nổi liệu đoạn mã của mình có dễ hiểu hay không, thì phương pháp luận thiết kế của bạn đang có vấn đề.

UB:

UB:

Fair enough. I will say, however, that I had equal “ pain and suffering ” interpreting your rewrite (below). So, apparently, neither of our methodologies were sufficient to rescue our readers from such struggles.

Cũng có lý. Tuy vậy, tôi phải nói rằng tôi cũng “đau đớn và khổ sở” y hệt khi cố hiểu bản viết lại của anh (ở dưới). Vậy nên, hoá ra chẳng phương pháp luận nào của hai chúng ta đủ sức cứu bạn đọc khỏi những phen vật lộn như thế.

JOHN:

JOHN:

Going back to my introductory remarks about complexity, splitting up isNot... into three methods doesn't reduce the amount of information you have to keep in your mind. It just spreads it out, so it isn't as obvious that you need to read all three methods together. And it's harder to see the overall structure of the code because it's split up: Readers have to flip back and forth between the methods, effectively reconstructing a monolithic version in their minds. Because the pieces are all related, this code will be easiest to understand if it's all together in one place.

Quay lại nhận xét mở đầu của tôi về độ phức tạp: việc chẻ isNot... thành ba method không hề làm giảm lượng thông tin bạn phải giữ trong đầu. Nó chỉ trải thông tin đó ra, khiến việc bạn phải đọc cả ba method cùng nhau trở nên bất hiển nhiên. Và cũng khó thấy được cấu trúc tổng thể của đoạn mã hơn, vì nó bị chẻ nhỏ: Người đọc phải lật qua lật lại giữa các method, thực chất là dựng lại một phiên bản liền khối trong đầu mình. Vì các mảnh đều liên quan tới nhau, đoạn mã này sẽ dễ hiểu nhất khi tất cả nằm chung một chỗ.

UB:

UB:

```
I disagree. Here is isNotMultipleOfAnyPreviousPrimeFactor.
private static boolean
isNotMultipleOfAnyPreviousPrimeFactor(int candidate) {
    for (int n = 1; n < multiplesOfPrimeFactors.size(); n++) {
        if (isMultipleOfNthPrimeFactor(candidate, n))
            return false;
    }
    return true;
}
```

If you trust the `isMultipleOfNthPrimeFactor` method, then this method stands alone quite nicely. I mean, we loop through all N previous primes and see if the candidate is a multiple. That's pretty straightforward.

Nếu bạn tin tưởng method `isMultipleOfNthPrimeFactor`, thì method này đứng một mình khá ổn. Ý tôi là, ta lặp qua toàn bộ N số nguyên tố trước đó và xem candidate có phải bội của chúng không. Chuyện đó khá là thẳng băng.

Now, it would be fair to ask the question how we determine whether the candidate is a multiple, and in that case, you'd want to inspect the `isMultiple...` method.

Tất nhiên, sẽ là hợp lý nếu hỏi rằng ta xác định candidate có phải là bội hay không bằng cách nào, và khi đó bạn sẽ muốn ngó vào method `isMultiple...`.

JOHN:

JOHN:

This code does appear to be simple and obvious. Unfortunately, this appearance is deceiving. If a reader trusts the name `isMultipleOfNthPrimeFactor` (which suggests a predicate with no side effects) and doesn't bother to read its code, they will not realize that it has side effects, and that candidate the side effects create a constraint on the argument to `isNot...` (it must be monotonically non-decreasing from invocation to invocation). To understand these behaviors, you have to read both `isMultiple...` and

Đoạn mã này *trông có vẻ* đơn giản và hiển nhiên. Đáng tiếc, vẻ ngoài ấy đánh lừa. Nếu người đọc tin vào cái tên `isMultipleOfNthPrimeFactor` (cái tên gợi ý một vị từ không có tác dụng phụ) và không buồn đọc mã của nó, họ sẽ không nhận ra rằng nó có tác dụng phụ, và rằng những tác dụng phụ đó tạo ra một ràng buộc lên đối số candidate truyền cho `isNot...` (nó phải không giảm đơn điệu qua từng lần gọi). Để hiểu được những hành vi này, bạn phải đọc cả `isMultiple...` lẫn

`smallestOdd...`. The current decomposition hides this important information from the reader.

`smallestOdd....` Cách phân rã hiện tại giấu đi thông tin quan trọng này khỏi người đọc.

If there is one thing more likely to result in bugs than not understanding code, it's thinking you understand it when you don't.

Nếu có một thứ còn dễ sinh ra bug hơn cả việc không hiểu mã, thì đó là việc tưởng mình hiểu trong khi thực ra không hiểu.

UB:

UB:

That's a valid concern. However, it is tempered by the fact that the functions are presented in the order they are called. Thus, we can expect that the reader has already seen the candidate main loop and understands that increases by two each iteration.

Đó là một mối lo chính đáng. Tuy nhiên, nó được xoa dịu bởi thực tế là các hàm được trình bày theo đúng thứ tự chúng được gọi. Nhờ vậy, ta có thể trông đợi rằng người đọc đã thấy vòng lặp chính duyệt candidate và hiểu rằng nó tăng thêm hai sau mỗi lần lặp.

The **side effect** buried down in `smallestOddNth...` is a bit more problematic. Now that you've pointed it out, I don't like it much. Still, that side effect should not confound the basic understanding of `isNot...`

Tác dụng phụ (side effect) chôn sâu dưới `smallestOddNth...` thì rắc rối hơn một chút. Giờ khi bạn đã chỉ ra, tôi cũng chẳng ưa nó lắm. Dù vậy, tác dụng phụ đó không nên làm rối cách hiểu cơ bản về `isNot...`

In general, if you trust the names of the methods being called, then understanding the caller does not require understanding the callee. For example:

Nói chung, nếu bạn tin vào tên của các method được gọi thì hiểu bên gọi không đòi hỏi phải hiểu bên bị gọi. Chẳng hạn:

```
for (Employee e : employees)
    if (e.shouldPayToday())
        e.pay();
```

This would not be made more understandable if we replaced those two method calls with their implementations. Such a replacement would simply obscure the intent.

Đoạn này sẽ chẳng dễ hiểu hơn nếu ta thay hai lời gọi method kia bằng phần hiện thực (implementation) của chúng. Việc thay thế như vậy chỉ làm ý định bị che mờ đi.

JOHN:

JOHN:

This example works because the called methods are relatively independent of the parent. Unfortunately, that is not the case for `isNot...`

Ví dụ đó chạy được là vì các method được gọi tương đối độc lập với method cha. Đáng tiếc, `isNot...` lại không như vậy.

In fact, `isNot...` is not only entangled with the methods it calls, it's also entangled with its callers. `isNot...` only works if it is invoked in a loop where candidate increases monotonically. To convince yourself that it works, you have to find the code that invokes `isNot...` and make sure that

Thực tế, `isNot...` không chỉ đan xen (entangled) với các method mà nó gọi, nó còn đan xen với những kẻ gọi nó. `isNot...` chỉ chạy đúng nếu được gọi trong một vòng lặp mà ở đó candidate tăng đơn điệu. Muốn tự thuyết phục mình rằng nó chạy đúng, bạn phải đi tìm đoạn mã gọi `isNot...` và bảo đảm rằng

candidate never decreases from one call to the next.

candidate không bao giờ giảm từ lời gọi này sang lời gọi kế tiếp.

Separating `isNot...` from the loop that invokes it makes it harder for readers to convince themselves that it works.

Việc tách `isNot...` ra khỏi vòng lặp gọi nó khiến người đọc khó tự thuyết phục mình rằng nó chạy đúng.

UB:

UB:

Which, as I said before, is why the methods are ordered the way they are. I expect that by the time you get to `isNot...` ,

Mà như tôi đã nói ở trên, đó chính là lý do các method được sắp theo thứ tự như vậy. Tôi cho rằng đến lúc đọc tới `isNot...` ,

you've already read `checkOddNumbersForSubsequentPrimes` and know that candidate increases by twos.

bạn đã biết rằng candidate tăng theo bước hai.

JOHN:

JOHN:

Let's discuss this briefly, because it's another area where I disagree with Clean Code . If methods are entangled, there is no clever ordering of the method definitions that will fix the problem.

Ta hãy bàn nhanh chuyện này, vì đây lại là một điểm nữa tôi bất đồng với Clean Code. Nếu các method đã đan xen với nhau thì chẳng có cách sắp xếp khéo léo nào cho các định nghĩa method có thể chữa được vấn đề.

In this particular situation, two other methods intervene between the loop in `checkOdd...` and `isNot...` , so readers will have forgotten the loop context before they get to `isNot...` . Furthermore, the actual code that creates a **dependency** on the loop isn't in `isNot...` : it's in `smallestOdd..` , which is even farther away from `checkOdd...` .

Trong tình huống cụ thể này, có hai method khác chen vào giữa vòng lặp trong `checkOdd...` và `isNot...` , nên người đọc sẽ quên mất bối cảnh vòng lặp trước khi kịp đọc tới `isNot...` . Hơn nữa, đoạn mã thực sự tạo ra sự phụ thuộc vào vòng lặp lại không nằm trong `isNot...` : nó nằm trong `smallestOdd...` , còn xa `checkOdd...` hơn nữa.

UB:

UB:

I sincerely doubt anyone is going to forget that candidate is being increased by twos. It's a pretty obvious way to avoid waste.

Tôi thật lòng nghi ngờ chuyện có ai đó lại quên mất rằng candidate đang được tăng theo bước hai. Đó là một cách tránh lãng phí khá hiển nhiên.

JOHN:

JOHN:

In my opening remarks, I talked about how it's important to reduce the amount of information people have to keep in their minds at once. In this situation, readers have to remember that loop while they read four intervening methods that are mostly unrelated to the loop. You apparently think this will be easy and natural (I disagree). But it's even worse than that. There is no indication which parts of `checkOdd...` will be important later on, so the only safe approach is to remember everything , from every method, until you have encountered every other method that could possibly descend from it. And to make the connection between the pieces, readers must also reconstruct the call graph to notice that, even through four layers of method call, the code in `smallestOdd...` places constraints on the loop in `checkOdd...` . This is an unreasonable cognitive burden to place on readers.

Trong phần mở đầu, tôi đã nói về tầm quan trọng của việc giảm bớt lượng thông tin mà người ta phải giữ trong đầu cùng một lúc. Ở tình huống này, người đọc phải nhớ vòng lặp đó trong lúc đọc bốn method chen vào giữa, mà phần lớn chẳng liên quan gì tới vòng lặp. Bạn có vẻ nghĩ chuyện đó sẽ dễ dàng và tự nhiên (tôi thì không đồng ý). Nhưng còn tệ hơn thế nữa. Chẳng có dấu hiệu nào cho biết phần nào của `checkOdd...` sẽ quan trọng về sau, nên cách an toàn duy nhất là nhớ mọi thứ, từ mọi method, cho tới khi bạn đã gặp đủ mọi method khác có thể nằm dưới nó trong chuỗi lời gọi. Và để nối được các mảnh lại với nhau, người đọc còn phải dựng lại đồ thị lời gọi (call graph) mới nhận ra rằng, dù cách nhau tới bốn tầng lời gọi method, đoạn mã trong `smallestOdd...` vẫn đặt ràng buộc lên vòng lặp trong `checkOdd...`. Đó là một gánh nặng nhận thức vô lý đặt lên vai người đọc.

If two pieces of code are tightly related, the solution is to bring them together. Separating the pieces, even in physically adjacent methods, makes the code harder to understand.

Nếu hai mẫu mã gắn bó chặt với nhau thì giải pháp là đưa chúng lại gần nhau. Tách rời các mẫu ra, kể cả khi các method nằm sát nhau về mặt vật lý, cũng làm mã khó hiểu hơn.

To me, all of the methods in `PrimeGenerator` are entangled: In order to understand the class, I had to load all of them into my mind at once. I was constantly flipping back and forth between the methods as I read the code. This is a red flag indicating that the code has been over-decomposed.

Với tôi, mọi method trong `PrimeGenerator` đều đan xen vào nhau: để hiểu được class đó, tôi phải nạp tất cả chúng vào đầu cùng một lúc. Tôi liên tục lật qua lật lại giữa các method trong khi đọc mã. Đây là dấu hiệu cảnh báo cho thấy mã đã bị phân rã quá mức (over-decomposed).

Bob, can you help me understand why you divided the code into such tiny methods? Is there some benefit to having so many methods that I have missed?

Bob này, bạn giúp tôi hiểu tại sao bạn lại chia mã thành những method bé xíu như thế được không? Có lợi ích nào của việc có ngần ấy method mà tôi đã bỏ sót không?

UB:

UB:

I think you and I are just going to disagree on this. In general, I believe in the principle of small, well-named methods and the separation of concerns. Generally speaking, if you can break a large method into several well-named smaller methods with different concerns, and by doing so expose their interfaces and the high-level functional decomposition, then that's a good thing.

Tôi nghĩ ở điểm này thì bạn và tôi sẽ cứ bất đồng thôi. Nói chung, tôi tin vào nguyên tắc method nhỏ, đặt tên tử tế, và tách bạch các mối quan tâm (separation of concerns). Nói một cách tổng quát, nếu bạn có thể chẻ một method lớn thành vài method nhỏ hơn với tên hay và những mối quan tâm khác nhau, và qua đó phơi bày interface của chúng cùng cách phân rã chức năng ở mức cao, thì đó là điều tốt.

- Looping over the odd numbers is one concern.
- Determining primality is another.
 - Duyệt qua các số lẻ là một mối quan tâm.
 - Xác định tính nguyên tố là mối quan tâm khác.
- Marking off the multiples of primes is yet another.
 - Đánh dấu loại bỏ các bội của số nguyên tố lại là một mối quan tâm khác nữa.

It seems to me that separating and naming those concerns helps to expose the way the algorithm works—even at the expense of some entanglement.

Với tôi, việc tách bạch và đặt tên cho những mối quan tâm đó giúp phơi bày cách thuật toán vận hành — kể cả khi phải trả giá bằng một chút đản xen.

In your solution, which we are soon to see below, you break the algorithm up in a similar way. However, instead of separating the concerns into functions, you separate them into sections with comments above them.

Trong lời giải của bạn, mà ta sắp thấy ngay dưới đây, bạn cũng chia thuật toán ra theo kiểu tương tự. Có điều, thay vì tách các mối quan tâm thành hàm, bạn tách chúng thành từng đoạn với comment đặt phía trên.

You mentioned that, in my solution, readers will have to keep the loop context in mind while reading the other functions. I suggest that, in your solution, readers will have to keep the loop context in mind while reading your explanatory comments. They may have to “flip back and forth” between the sections in order to establish their understanding.

Bạn có nhắc rằng trong lời giải của tôi, người đọc sẽ phải giữ bối cảnh vòng lặp trong đầu khi đọc các hàm khác. Tôi xin nói rằng trong lời giải của bạn, người đọc cũng sẽ phải giữ bối cảnh vòng lặp trong đầu khi đọc những comment giải thích của bạn. Họ cũng có thể phải “lật qua lật lại” giữa các đoạn để hình thành cách hiểu của mình.

Now, perhaps you are concerned that, in my solution, the “flipping” is a longer distance (in lines) than in yours. I’m not sure that’s a significant point, since they all fit on the same screen (at least they do on my screen) and the landmarks are pretty obvious.

Giờ, có lẽ bạn lo rằng trong lời giải của tôi, quãng đường “lật qua lật lại” ấy dài hơn (tính theo số dòng) so với của bạn. Tôi không chắc đó là điểm đáng kể, vì tất cả đều nằm gọn trên cùng một màn hình (ít nhất là trên màn hình của tôi) và các cột mốc thì khá rõ ràng.

Method Length Summary — Tóm lược về độ dài method

JOHN:

JOHN:

It sounds like it’s time to wrap up this section. Is this a reasonable summary of where we agree and disagree?

Có vẻ đã đến lúc khép lại phần này. Tóm tắt sau đây về chỗ ta đồng ý và bất đồng có hợp lý không?

- We agree that modular design is a good thing.
- We agree that it is possible to over-decompose, and that Clean Code, First Edition doesn’t provide much guidance on how to recognize over-decomposition.
 - Ta đồng ý rằng thiết kế mô-đun là điều tốt.
 - Ta đồng ý rằng hoàn toàn có thể phân rã quá mức, và rằng Clean Code, First Edition không đưa ra mấy chỉ dẫn về cách nhận ra tình trạng phân rã quá mức.
- We disagree on how far to decompose: You recommend decomposing code into much smaller units than I do. You believe that the additional decomposition you recommend makes code easier to understand; I believe that it goes too far and actually makes code more difficult to understand.
- You believe that the One Thing rule, applied with judgment, will lead to appropriate decompositions. I believe it lacks guardrails and will lead to over-decomposition.
- We agree that the internal decomposition of

- Ta bất đồng về mức độ phân rã: bạn khuyến nghị chẻ mã thành những đơn vị nhỏ hơn nhiều so với tôi. Bạn tin rằng phân rã thêm mà bạn khuyến nghị làm mã dễ hiểu hơn; tôi tin rằng nó đi quá xa và thực ra làm mã khó hiểu hơn.
- Bạn tin rằng quy tắc One Thing, nếu áp dụng có cân nhắc, sẽ dẫn tới cách phân rã hợp lý. Tôi tin nó thiếu rào chắn an toàn và sẽ dẫn tới phân rã quá mức.
- Ta đồng ý rằng việc phân rã nội bộ của

PrimeGenerator into methods is problematic. You point out that your main goal in writing PrimeGenerator was to show how to decompose into classes, not so much how to decompose a class internally into methods.

PrimeGenerator thành các method là có vấn đề. Bạn chỉ ra rằng mục tiêu chính của bạn khi viết PrimeGenerator là cho thấy cách phân rã thành các class, chứ không hẳn là cách phân rã nội bộ một class thành các method.

- Entanglement between methods in a class doesn't bother you as much as it bothers me. You believe that the benefits of decomposing methods can compensate for problems caused by entanglement. I believe they can't: When decomposed methods are entangled, they are harder to read than if they were not decomposed, and this defeats the whole purpose of decomposition.
- You believe that ordering the methods in a class can help to compensate for entanglement between methods; I don't.
 - Sự đan xen giữa các method trong một class không làm bạn khó chịu nhiều như nó làm tôi khó chịu. Bạn tin rằng lợi ích của việc phân rã method có thể bù lại những rắc rối do đan xen gây ra. Tôi tin là không: khi các method đã phân rã lại đan xen với nhau, chúng khó đọc hơn cả khi chưa phân rã, và điều đó đánh đổ toàn bộ mục đích của việc phân rã.
 - Bạn tin rằng sắp xếp thứ tự các method trong một class có thể bù đắp phần nào cho sự đan xen giữa chúng; tôi thì không.

UB:

UB:

I think this is a fair assessment of our agreements and disagreements. We both value decomposition, and we both avoid entanglement; but we disagree on the relative weighting of those two values.

Tôi nghĩ đó là bản đánh giá công bằng về những chỗ ta đồng ý và bất đồng. Cả hai ta đều coi trọng việc phân rã, và cả hai đều tránh sự đan xen; chỉ có điều ta bất đồng về trọng số tương đối của hai giá trị ấy.

Comments — Comment

JOHN:

JOHN:

Let's move on to the second area of disagreement: comments. In my opinion, the Clean Code approach to commenting results in code with inadequate documentation, which increases the cost of software development. I'm sure you disagree, so let's discuss.

Ta chuyển sang vùng bất đồng thứ hai: comment. Theo tôi, cách tiếp cận comment của Clean Code dẫn tới mã có tài liệu không đầy đủ, và điều đó làm tăng chi phí phát triển phần mềm. Tôi chắc là bạn không đồng ý, nên ta hãy bàn xem.

Here is what Clean Code says about comments (page 54):

Đây là điều Clean Code nói về comment (trang 54):

“The proper use of comments is to compensate for our failure to express ourselves in code. Note that I use the word failure . I meant it. Comments are always failures. We must have them because we cannot always figure out how to express ourselves without them, but their use is not a cause for celebration... Every time you write a comment you should grimace and feel the failure of your ability of expression.”

“Cách dùng comment đúng đắn là để bù đắp cho việc ta thất bại trong việc diễn đạt bằng mã. Lưu ý là tôi dùng từ thất bại. Tôi cố ý dùng từ đó. Comment luôn là những thất bại. Ta buộc phải có chúng vì không phải lúc nào ta cũng nghĩ ra cách diễn đạt mà không cần tới chúng, nhưng việc phải dùng chúng chẳng phải chuyện đáng ăn mừng... Mỗi lần viết một comment, bạn nên nhăn mặt và cảm nhận sự thất bại trong khả năng diễn đạt của mình.”

I have to be honest: I was horrified when I first read this text, and it still makes me cringe. This stigmatizes writing comments. Junior developers will think “if I write comments, people may think I’ve failed, so the safest thing is to write no comments.”

Tôi phải thành thật: tôi đã kinh hoàng khi lần đầu đọc đoạn này, và tới giờ nó vẫn làm tôi rùng mình. Nó bôi xấu việc viết comment. Các lập trình viên non tay sẽ nghĩ “nếu mình viết comment, người ta có thể nghĩ mình đã thất bại, nên an toàn nhất là đừng viết comment nào cả.”

UB:

UB:

That chapter begins with these words:

Chương đó mở đầu bằng những lời này:

“Nothing can be quite so helpful as a well-placed comment.”

“Không gì hữu ích bằng một comment được đặt đúng chỗ.”

It goes on to say that comments are a necessary evil.

Và nó nói tiếp rằng comment là một cái ác cần thiết.

The only way a reader could infer that they should write no comments is if they hadn’t actually read the chapter. The chapter walks through a series of comments, some bad, some good.

Cách duy nhất để một người đọc suy ra rằng họ không nên viết comment nào là họ đã không thực sự đọc chương đó. Chương này dẫn bạn đi qua một loạt comment, có cái tệ, có cái tốt.

JOHN:

JOHN:

Clean Code focuses a lot more on the “evil” aspects of comments than the “necessary” aspects. The sentence you quoted above is followed by two sentences criticizing comments. Chapter 4 spends 4 pages talking about good comments, followed by 15 pages talking about bad comments. There are snubs like “the only truly good comment is the comment you found a way not to write.” And “Comments are always failures” is so catchy that it’s the one thing readers are most likely to remember from the chapter.

Clean Code tập trung vào khía cạnh “cái ác” của comment nhiều hơn hẳn khía cạnh “cần thiết”. Câu bạn trích ở trên còn được nối tiếp bằng hai câu chê comment. Chương 4 dành 4 trang nói về comment tốt, rồi 15 trang nói về comment tệ. Có những câu dè bủ kiểu

“comment thực sự tốt duy nhất là comment mà bạn tìm được cách để khỏi phải viết.” Còn “Comment luôn là những thất bại” thì bắt tai đến mức đó chính là điều người đọc nhớ nhất từ chương ấy.

UB:

UB:

The difference in page count is because there are just a few ways to write good comments and so many more ways to write bad ones.

Chênh lệch số trang là vì chỉ có vài cách để viết comment tốt, còn cách viết comment tệ thì nhiều hơn hẳn.

JOHN:

JOHN:

I disagree; this illustrates your bias against comments. If you look at Chapter 13 of APOSD, it finds a lot more constructive ways to use comments than Clean Code . And if you compare the tone of Chapter 13 of APOSD with Chapter 4 of Clean Code , the hostility of Clean Code towards comments becomes pretty clear.

Tôi không đồng ý; điều đó cho thấy định kiến của bạn với comment. Nếu bạn xem Chương 13 của APOSD, sẽ thấy nó tìm ra nhiều cách dùng comment mang tính xây dựng hơn Clean Code rất nhiều. Và nếu bạn so giọng điệu của Chương 13 trong APOSD với Chương 4 của Clean Code, thái độ thù địch của Clean Code với comment hiện ra khá rõ.

UB:

UB:

I'll leave you to balance that last comment with the initial statement, and the final example, in the “Comments” chapter. They do not communicate “hostility.”

Tôi để bạn tự cân bằng nhận xét vừa rồi với câu mở đầu và ví dụ cuối cùng trong chương “Comment”. Chúng chẳng hề toát ra “sự thù địch” nào.

I'm not hostile to comments in general. I am very hostile to gratuitous comments.

Tôi không thù địch với comment nói chung. Tôi rất thù địch với comment thừa thãi.

You and I likely both survived through a time when comments were absolutely necessary. In the '70s and '80s, I was an assembly language programmer. I also wrote a bit of FORTRAN. Programs in those languages that had no comments were impenetrable.

Bạn và tôi hẳn đều đã sống qua thời mà comment là thứ tuyệt đối cần thiết. Thập niên 70 và 80, tôi là lập trình viên hợp ngữ. Tôi cũng viết chút ít FORTRAN. Chương trình viết bằng những ngôn ngữ đó mà không có comment thì không tài nào xuyên thủng nổi.

As a result, it became conventional wisdom to write comments by default. And indeed, computer science students were taught to write comments uncritically. Comments became pure good .

Kết quả là viết comment theo mặc định trở thành lẽ thường. Và quả thật, sinh viên khoa học máy tính được dạy viết comment mà không cần dẫn đo. Comment trở thành thứ thuần túy tốt đẹp.

In Clean Code , I decided to fight that mindset. Comments can be really bad as well as good.

Trong Clean Code, tôi quyết định chống lại lối nghĩ đó. Comment có thể rất tệ chứ không chỉ có tốt.

JOHN:

JOHN:

I don't agree that comments are less necessary today than they were 40 years ago. Comments are crucially important and add enormous value to software. The problem is that there is a lot of important information that simply cannot be expressed in code. By adding comments to fill in this missing information, developers can make code dramatically easier to read. This is not a "failure of their ability to express themselves," as you put it.

Tôi không đồng ý rằng comment ngày nay ít cần thiết hơn so với 40 năm trước. Comment cực kỳ quan trọng và mang lại giá trị to lớn cho phần mềm. Vấn đề là có rất nhiều thông tin quan trọng đơn giản là không thể diễn đạt bằng mã. Bằng cách thêm comment để lấp đầy khoảng thiếu hụt ấy, lập trình viên có thể làm mã dễ đọc hơn hẳn. Đây không phải là "thất bại trong khả năng diễn đạt của họ" như cách bạn nói.

UB:

UB:

It's very true that there is important information that is not, or cannot be, expressed in code. That's a failure. A failure of our languages, or of our ability to use them to express ourselves. In every case, a comment is a failure of our ability to use our languages to express our intent.

Rất đúng là có những thông tin quan trọng không được, hoặc không thể, diễn đạt bằng mã. Đó là một thất bại. Thất bại của các ngôn ngữ của ta, hoặc của khả năng dùng chúng để diễn đạt. Trong mọi trường hợp, một comment là một thất bại trong khả năng dùng ngôn ngữ của ta để diễn đạt ý định.

And we fail at that very frequently, and so comments are a necessary evil—or, if you prefer, an unfortunate necessity. If we had the perfect programming language,TM we would never write another comment.

Và ta thất bại ở việc đó rất thường xuyên, nên comment là một cái ác cần thiết — hoặc, nếu bạn thích, một sự cần thiết đáng tiếc. Nếu ta có ngôn ngữ lập trình hoàn hảo,TM ta sẽ chẳng bao giờ phải viết thêm một comment nào nữa.

JOHN:

JOHN:

I don't agree that a perfect programming language would eliminate the need for comments. Comments and code serve very different purposes, so it's not obvious to me that we should use the same language for both. In my experience, English works quite well as a language for comments. Why do you feel that information about a program should be expressed entirely in code, rather than using a combination of code and English?

Tôi không đồng ý rằng một ngôn ngữ lập trình hoàn hảo sẽ xoá bỏ nhu cầu viết comment. Comment và mã phục vụ những mục đích rất khác nhau, nên với tôi chẳng có gì hiển nhiên rằng ta nên dùng cùng một ngôn ngữ cho cả hai. Theo kinh nghiệm của tôi, tiếng Anh hoạt động khá tốt trong vai trò ngôn ngữ cho comment. Vì sao bạn cho rằng thông tin về một chương trình nên được diễn đạt hoàn toàn bằng mã, thay vì dùng kết hợp mã và tiếng Anh?

UB:

UB:

I bemoan the fact that we must sometimes use a human language instead of a programming language. Human languages are imprecise and full of ambiguities. Using a human language to describe something as precise as a program is very hard, and fraught with many opportunities for error and inadvertent misinformation.

Tôi than phiền cái sự thật rằng đôi khi ta buộc phải dùng ngôn ngữ của con người thay cho ngôn ngữ lập trình. Ngôn ngữ con người thiếu chính xác và đầy nhập nhằng. Dùng ngôn ngữ con người để mô tả một thứ chính xác như chương trình là việc rất khó, và đầy rẫy cơ hội cho sai sót cùng thông tin sai lệch ngoài ý muốn.

JOHN:

JOHN:

I agree that English isn't always as precise as code, but it can still be used in precise ways, and comments typically don't need the same degree of precision as code. Comments often contain qualitative information such as why something is being done, or the overall idea of something. English works better for these than code because it is a more expressive language.

Tôi đồng ý rằng tiếng Anh không phải lúc nào cũng chính xác bằng mã, nhưng nó vẫn có thể được dùng một cách chính xác, và comment thường không cần mức độ chính xác ngang với mã. Comment thường chứa thông tin mang tính định tính, chẳng hạn vì sao một việc được làm, hay ý tưởng tổng thể của một thứ gì đó. Tiếng Anh làm những chuyện này tốt hơn mã, vì nó là một ngôn ngữ giàu sức biểu đạt hơn.

UB:

UB:

I have no argument with that statement.

Tôi chẳng có gì để phản bác câu đó.

JOHN:

JOHN:

Are you concerned that comments will be incorrect or misleading and that this will slow down software development? I often hear people complain about stale comments (usually as an excuse for writing no comments at all), but I have not found them to be a significant problem over my career. Incorrect comments do happen, but I don't encounter them very often, and when I do, they rarely cost me much time. In contrast, I waste enormous amounts of time because of inadequate documentation; it's not unusual for me to spend 50% to 80% of my development time **wading** through code to figure out things that would be obvious if the code was properly commented.

Có phải bạn lo rằng comment sẽ sai hoặc gây hiểu lầm, và điều đó sẽ làm chậm việc phát triển phần mềm? Tôi vẫn thường nghe người ta than về comment lỗi thời (thường là để biện minh cho việc chẳng viết comment nào cả), nhưng suốt sự nghiệp của mình tôi không thấy đó là vấn đề đáng kể. Comment sai đúng là có xảy ra, nhưng tôi ít khi gặp, và khi gặp thì chúng hiếm khi ngốn của tôi nhiều thời gian. Ngược lại, tôi lãng phí lượng thời gian khổng lồ vì tài liệu không đầy đủ; chuyện tôi tiêu 50% đến 80% thời gian phát triển để lội bì bõm trong mã nhằm tìm ra những thứ lẽ ra đã hiển nhiên nếu mã được comment tử tế là chuyện không hiếm.

UB:

UB:

You and I have had some very different experiences. I have certainly been helped by well-placed comments. I have also, just as certainly (and within this very document), been distracted and confused by a comment that was incorrect, misplaced, gratuitous, or otherwise just plain bad.

Bạn và tôi đã có những trải nghiệm rất khác nhau. Tôi chắc chắn từng được một comment đặt đúng chỗ giúp đỡ. Nhưng tôi cũng chắc chắn không kém (và ngay trong chính tài liệu này) từng bị phân tâm và rối trí bởi một comment sai, đặt sai chỗ, thừa thãi, hoặc đơn giản là dở tệ.

JOHN:

JOHN:

I invite everyone reading this to ask yourself the following questions.

Tôi mời tất cả những ai đang đọc phần này tự hỏi mình mấy câu sau.

- How much does your software development speed suffer because of incorrect comments?
- How much does your software development speed suffer because of missing comments?
 - Tốc độ phát triển phần mềm của bạn thiệt hại bao nhiêu vì những comment sai?
 - Tốc độ phát triển phần mềm của bạn thiệt hại bao nhiêu vì những comment còn thiếu?

For me, the cost of missing comments is easily 10 to 100 times the cost of incorrect comments. That is why I cringe when I see things in Clean Code that discourage people from writing comments. Let's consider the PrimeGenerator class. There is not a single comment in that code; does this seem appropriate to you?

Với tôi, cái giá của comment còn thiếu dễ dàng gấp 10 đến 100 lần cái giá của comment sai. Đó là lý do tôi rùng mình khi thấy trong Clean Code những điều làm người ta ngại viết comment. Hãy xem class PrimeGenerator. Trong đoạn mã đó không có lấy một comment nào; bạn thấy như vậy là hợp lý sao?

UB:

UB:

I think it was appropriate for the purpose for which I wrote it. It was an adjunct to the lesson that very large methods can be broken down into smaller classes containing smaller methods. Adding lots of explanatory comments would have detracted from that point.

Tôi nghĩ nó hợp lý với mục đích mà tôi viết ra nó. Nó là phần phụ trợ cho bài học rằng những method rất lớn có thể được chẻ nhỏ thành các class nhỏ chứa các method nhỏ. Thêm hàng đống comment giải thích sẽ làm loãng mất điểm đó.

In general, however, the commenting style I used in Listing 4-8 is more appropriate. That listing, at the very end of the “Comments” chapter, describes yet another PrimeGenerator with a slightly different algorithm and a better set of comments.

Tuy nhiên, nói chung thì phong cách comment mà tôi dùng ở Listing 4-8 mới là phù hợp hơn. Listing đó, ở ngay cuối chương “Comment”, mô tả một PrimeGenerator khác với thuật toán hơi khác và một bộ comment tốt hơn.

JOHN:

JOHN:

I disagree that adding comments would have distracted from your point, and I think Listing 4-8 is also woefully under-commented. But let's not argue about either of those issues. Instead, let's discuss what comments the

Tôi không đồng ý rằng thêm comment sẽ làm loãng mất điều bạn muốn nói, và tôi cho rằng Listing 4-8 cũng thiếu comment đến mức đáng thương. Nhưng thôi, ta đừng tranh cãi về cả hai chuyện đó. Thay vào đó, hãy bàn xem đoạn mã

PrimeGenerator code should have if it were used in production. I will make some suggestions, and you can agree or disagree.

PrimeGenerator nên có những comment nào nếu nó được dùng trong production. Tôi sẽ đưa ra vài đề xuất, còn bạn có thể đồng ý hoặc không.

For starters, let's discuss your use of megasyllabic names

Trước hết, hãy bàn về chuyện bạn dùng những cái tên nhiều âm tiết kiểu megasyllabic như

`likeisLeastRelevantMultipleOfLargerPrimeFactor`. My

understanding is that you advocate using names like this instead of using shorter names augmented with descriptive comments: You're effectively moving the comments into code. To me, this approach is problematic.

cách tôi hiểu thì bạn ủng hộ việc dùng những cái tên như vậy thay cho tên ngắn hơn kèm comment mô tả: thực chất bạn đang dời comment vào trong mã. Với tôi, cách làm này có vấn đề.

- Long names are awkward. Developers effectively have to retype the documentation for a method every time they invoke it, and the long names waste horizontal space and trigger line wraps in the code. The names are also awkward to read: My mind wants to parse every syllable every time I read it, which slows me down. Notice that both you and I resorted to abbreviating names in this discussion: That's an indication that the long names are awkward and unhelpful.
 - Tên dài thì vướng víu. Lập trình viên thực chất phải gõ lại toàn bộ tài liệu của một method mỗi lần gọi nó, và những cái tên dài ngốn không gian ngang, gây xuống dòng trong mã. Chúng cũng khó đọc: đầu tôi cứ muốn phân tích từng âm tiết mỗi lần đọc, và điều đó làm tôi chậm lại. Để ý là cả bạn và tôi đều phải viết tắt tên trong cuộc thảo luận này: đó là dấu hiệu cho thấy những cái tên dài vừa vướng víu vừa chẳng giúp ích.
- The names are hard to parse and don't convey information as effectively as a comment. When students read PrimeGenerator, one of the first things they complain about is the long names (students can't make sense of them). For example, the name above is vague and cryptic: What does "least relevant" mean, and what is a "larger prime factor"? Even with a complete understanding of the code in the method, it's hard for me to make sense of the name. If this name is going to eliminate the need for a comment, it needs to be even longer.
 - Những cái tên đó khó phân tích và không truyền tải thông tin hiệu quả bằng một comment. Khi sinh viên đọc PrimeGenerator, một trong những điều đầu tiên họ than phiền chính là những cái tên dài (sinh viên chẳng hiểu nổi chúng nói gì). Chẳng hạn, cái tên ở trên vừa mơ hồ vừa bí hiểm: "least relevant" nghĩa là gì, và "larger prime factor" là cái gì? Ngay cả khi đã hiểu trọn vẹn đoạn mã bên trong method, tôi vẫn thấy khó lòng hiểu nổi cái tên ấy. Nếu cái tên này định thay thế cho một comment, thì nó còn phải dài hơn nữa.

In my opinion, the traditional approach of using shorter names with descriptive comments is more convenient and conveys the required information more effectively. What advantage is there in the approach you advocate?

Theo tôi, cách làm truyền thống là dùng tên ngắn hơn kèm comment mô tả vừa tiện hơn vừa truyền tải thông tin cần thiết hiệu quả hơn. Cách bạn cổ vũ có lợi thế gì?

UB:

UB:

“Megasyllabic”: great word!

“Megasyllabic”: từ hay đấy!

I like my method names to be sentence fragments that fit nicely with keywords and assignment statements. It makes the code a bit more natural to read.

Tôi thích tên method của mình là những mảnh câu ghép ăn khớp với từ khoá và câu lệnh gán. Nó làm mã đọc lên tự nhiên hơn một chút.

```
if (isTooHot)
    cooler.turnOn();
```

I also follow a simple rule about the length of names. The larger the **scope** of a method, the shorter its name should be, and vice versa: The shorter the scope, the longer the name. The private methods I extracted in this case live in very small scopes, and so have longish names. Methods like this are typically called from only one place, so there is no burden on the programmer to remember a long name for another call.

Tôi cũng theo một quy tắc đơn giản về độ dài của tên. Phạm vi (scope) của một method càng lớn thì tên của nó càng phải ngắn, và ngược lại: phạm vi càng nhỏ thì tên càng dài. Các private method mà tôi tách ra trong trường hợp này sống trong những phạm vi rất nhỏ, nên có tên hơi dài. Những method kiểu đó thường chỉ được gọi từ đúng một chỗ, nên lập trình viên chẳng phải mang gánh nặng nhớ một cái tên dài cho lần gọi nào khác.

JOHN:

JOHN:

Names like isTooHot are totally fine by me. My concern is

Những cái tên như isTooHot thì với tôi hoàn toàn ổn. Điều tôi bận tâm là
about names like isLeastRelevantMultipleOfLargerPrimeFactor.

It’s interesting that, as methods get smaller and narrower, you recommend longer names. What this says to me is that the interfaces for those functions are more complex, so it takes more words to describe them. This provides supporting evidence for my **assertion** a while back that the more you split up a method, the shallower the resulting methods will be.

Điều thú vị là khi method càng nhỏ và càng hẹp thì bạn lại khuyến nghị đặt tên dài hơn. Với tôi, điều đó nói lên rằng interface của những hàm ấy phức tạp hơn, nên phải cần nhiều chữ hơn để mô tả. Đây chính là bằng chứng ủng hộ khẳng định tôi đưa ra ban nãy: càng chẻ nhỏ một method thì các method thu được càng nông.

UB:

UB:

It’s not the functions that get smaller. It’s the scope that gets smaller. A private function has a smaller scope than the public function that calls it. A function called by that private function has an even smaller scope. As we descend in scope, we also descend in situational detail. Describing such detail often requires a long name, or a long comment. I prefer to use a name.

Không phải hàm nhỏ đi. Chính phạm vi mới nhỏ đi. Một private function có phạm vi nhỏ hơn public function gọi nó. Một hàm được private function đó gọi lại có phạm vi nhỏ hơn nữa. Càng đi xuống trong phạm vi, ta càng đi xuống trong chi tiết tình huống. Mô tả những chi tiết như vậy thường đòi hỏi một cái tên dài, hoặc một comment dài. Tôi thích dùng một cái tên hơn.

As for long names being hard to parse, that's a matter of practice. Code is full of things that take practice to get used to.

Còn chuyện tên dài khó phân tích, đó là vấn đề luyện tập. Mã nguồn đầy rẫy những thứ phải luyện tập mới quen được.

JOHN:

JOHN:

I don't accept this. Code may be full of things that take practice to get used to, but that doesn't excuse it. Approaches that require more practice are worse than those that require less. If it's going to take a lot of work to get comfortable with the long names, then there had better be some compensating benefit; so far, I'm not seeing any. And I don't see any reason to believe that practice will make those names easier to digest.

Tôi không chấp nhận lập luận này. Mã nguồn có thể đầy rẫy những thứ phải luyện tập mới quen, nhưng điều đó không bào chữa cho nó. Cách làm đòi hỏi luyện tập nhiều hơn thì tệ hơn cách làm đòi hỏi ít hơn. Nếu phải tốn nhiều công sức mới thấy thoải mái với những cái tên dài, thì tốt hơn hết là phải có lợi ích nào đó bù lại; cho tới giờ, tôi chẳng thấy lợi ích nào. Và tôi cũng chẳng thấy lý do gì để tin rằng luyện tập sẽ khiến những cái tên ấy dễ tiêu hoá hơn.

In addition, your comment above violates one of my fundamental rules, which is “complexity is in the eye of the reader.” If you write code that someone else thinks is complicated, then you must accept that the code is probably complicated (unless you think the reader is completely incompetent). It is not OK to make excuses or suggest that it is really the reader's problem (“you just don't have enough practice”). I'm going to have to live by this same rule a bit later in our discussion.

Thêm nữa, nhận xét của bạn ở trên vi phạm một trong những quy tắc nền tảng của tôi, đó là “độ phức tạp nằm trong mắt người đọc”. Nếu bạn viết mã mà người khác thấy phức tạp, thì bạn phải chấp nhận rằng mã đó nhiều khả năng là phức tạp thật (trừ phi bạn cho rằng người đọc hoàn toàn bất tài). Không được phép bao biện hay ngụ ý rằng đó mới thực là vấn đề của người đọc (“tại bạn luyện chưa đủ đó thôi”). Lát nữa trong cuộc thảo luận này, chính tôi cũng sẽ phải sống theo đúng quy tắc ấy.

UB:

UB:

Fair enough. As for the meaning of `leastRelevant`, that's a much larger problem that you and I will encounter shortly. It has to do with the intimacy that the author has with the solution, and the reader's lack of that intimacy.

Được thôi. Còn về ý nghĩa của `leastRelevant`, đó là một vấn đề lớn hơn nhiều mà bạn và tôi sẽ gặp lại chẳng bao lâu nữa. Nó liên quan tới sự thân thuộc mà tác giả có với lời giải, và việc người đọc thiếu đi sự thân thuộc đó.

JOHN:

JOHN:

You still haven't answered my question: Why is it better to use super-long names rather than shorter names augmented with descriptive comments?

Anh vẫn chưa trả lời câu hỏi của tôi: tại sao dùng những cái tên dài lê thê lại tốt hơn là dùng tên ngắn kèm theo comment mô tả?

UB:

UB:

It's a matter of preference for me. I prefer long names to comments. I don't trust comments to be maintained, nor do I trust that they will be read. Have you ever noticed that many IDEs paint comments in light gray so that they can be easily ignored? It's harder to ignore a name than a comment.

Với tôi thì đó là chuyện sở thích. Tôi thích tên dài hơn là comment. Tôi không tin comment sẽ được bảo trì, cũng chẳng tin là chúng sẽ được đọc. Anh có để ý rằng nhiều IDE tô comment bằng màu xám nhạt để người ta dễ dàng phớt lờ chúng không? Phớt lờ một cái tên khó hơn phớt lờ một comment nhiều.

(BTW, I have my IDE paint comments in bright fire-engine red.)

(Nhân tiện, tôi cho IDE của mình tô comment bằng màu đỏ chói như xe cứu hoả.)

JOHN:

JOHN:

I don't see why a monster name is more likely to be "maintained" than a comment, and I don't agree that IDEs encourage people to ignore comments (this is your bias coming out again). My current IDE (VSCode) doesn't use a lighter color for comments. My previous one (NetBeans) did, but the color scheme didn't hide the comments; it distinguished them from the code in a way that made both code and comments easier to read.

Tôi không hiểu tại sao một cái tên khổng lồ lại có nhiều khả năng được "bảo trì" hơn một comment, và tôi cũng không đồng ý rằng các IDE khuyến khích người ta phớt lờ comment (lại là định kiến của anh lộ ra đấy). IDE tôi đang dùng (VSCode) không tô comment bằng màu nhạt hơn. IDE trước đây của tôi (NetBeans) thì có, nhưng bằng màu ấy không hề giấu comment đi; nó phân biệt comment với mã theo cách khiến cả mã lẫn comment đều dễ đọc hơn.

Now that we've discussed the specific issue of comments versus long method names, let's talk about comments in general. I think there are two major reasons why comments are needed. The first reason for comments is abstraction. Simply put, without comments, there is no way to have abstraction or modularity.

Giờ khi đã bàn xong chuyện cụ thể là comment so với tên method dài, ta hãy nói về comment nói chung. Tôi cho rằng có hai lý do chính khiến ta cần comment. Lý do thứ nhất là trừu tượng hoá (abstraction). Nói đơn giản: không có comment thì không thể có trừu tượng hoá hay tính module.

Abstraction is one of the most important components of good software design. I define an abstraction as "a simplified way of thinking about something that omits unimportant details." The most obvious example of an abstraction is a method. It should be possible to use a method without reading its code. The way we achieve this is by writing a header comment that describes the method's interface (all the information someone needs in order to invoke the method). If the method is well designed, the interface will be much simpler than the code of the method (it omits implementation details), so the comments reduce the amount of information people must have in their heads.

Trừu tượng hoá là một trong những thành phần quan trọng nhất của thiết kế phần mềm tốt. Tôi định nghĩa trừu tượng hoá là “một cách nghĩ đơn giản hoá về một sự vật, bỏ qua những chi tiết không quan trọng.” Ví dụ hiển nhiên nhất về trừu tượng hoá là một method. Ta phải có thể dùng một method mà không cần đọc mã của nó. Cách ta đạt được điều đó là viết một comment mở đầu mô tả interface của method (toàn bộ thông tin mà người ta cần để gọi nó). Nếu method được thiết kế tốt, interface sẽ đơn giản hơn nhiều so với mã của method (nó lược bỏ chi tiết hiện thực), nên comment làm giảm lượng thông tin người ta phải giữ trong đầu.

UB:

UB:

Long ago, in a 1995 book, I defined abstraction as “The amplification of the essential and the elimination of the irrelevant.”

Từ lâu rồi, trong một cuốn sách năm 1995, tôi đã định nghĩa trừu tượng hoá là “khuếch đại cái cốt yếu và loại bỏ cái không liên quan.”

I certainly agree that abstraction is of importance to good software design. I also agree that well-placed comments can enhance the ability of readers to understand the abstractions we are attempting to employ. I disagree that comments are the only , or even the best , way to understand those abstractions. But sometimes they are the only option.

Tôi chắc chắn đồng ý rằng trừu tượng hoá rất quan trọng với thiết kế phần mềm tốt. Tôi cũng đồng ý rằng những comment đặt đúng chỗ có thể giúp người đọc hiểu rõ hơn các trừu tượng hoá mà ta đang cố dùng. Điều tôi không đồng ý là comment là cách *duy nhất*, hay thậm chí là cách *tốt nhất*, để hiểu những trừu tượng hoá đó. Nhưng đôi khi chúng là lựa chọn duy nhất.

But consider:

Nhưng hãy xem thử:

```
addSongToLibrary(String title, String[] authors, int durationInSeco
```

This seems like a very nice abstraction to me, and I cannot imagine how a comment might improve it.

Với tôi đây có vẻ là một trừu tượng hoá rất đẹp, và tôi không hình dung nổi một comment có thể cải thiện nó ra sao.

JOHN:

JOHN:

Our definitions of abstraction are very similar; that’s good to see. However, the addSongToLibrary declaration is not (yet) a good abstraction, because it omits information that is essential. In order to use addSongToLibrary , developers need answers to the following questions.

Định nghĩa về trừu tượng hoá của chúng ta rất giống nhau; thấy vậy cũng mừng. Tuy nhiên, khai báo addSongToLibrary (chưa) phải là một trừu tượng hoá tốt, vì nó bỏ sót những thông tin thiết yếu. Để dùng được addSongToLibrary, lập trình viên cần câu trả lời cho những câu hỏi sau.

- Is there any expected format for an author string, such as “LastName, FirstName”?
- Are the authors expected to be in alphabetical order? If not, is the order significant in some other way?

- What happens if there is already a song in the library with the given title but different authors? Is it replaced with the new one, or will the library keep multiple songs with the same title?
- How is the library stored (e.g., is it entirely in memory? saved on disk?)? If this information is documented somewhere else, such as the overall class documentation, then it need not be repeated here.
 - Có định dạng nào được kỳ vọng cho chuỗi author không, chẳng hạn “LastName, FirstName”?
 - Danh sách tác giả có phải xếp theo thứ tự bảng chữ cái không? Nếu không, thứ tự có mang ý nghĩa nào khác không?
 - Chuyện gì xảy ra nếu trong thư viện đã có một bài hát cùng tiêu đề nhưng khác tác giả? Nó bị thay bằng bài mới, hay thư viện sẽ giữ nhiều bài hát trùng tiêu đề?
 - Thư viện được lưu trữ thế nào (ví dụ: hoàn toàn trong bộ nhớ? lưu xuống đĩa?)? Nếu thông tin này đã được ghi ở nơi khác, chẳng hạn trong tài liệu chung của class, thì không cần lặp lại ở đây.

Thus, `addSongToLibrary` needs quite a few comments. Sometimes the signature of a method (names and types of the method, its arguments, and its return value) contains all the information needed to use it, but this is pretty rare. Just skim through the documentation for your favorite library **package**: In how many cases could you understand how to use a method with only its signature?

Vì vậy, `addSongToLibrary` cần khá comment. Đôi khi chữ ký của một method (tên và kiểu của method, của các đối số và của giá trị trả về) đã chứa đủ mọi thông tin cần để dùng nó, nhưng chuyện đó khá hiếm. Cứ thử lướt qua tài liệu của package thư viện bạn ưa thích mà xem: có được bao nhiêu trường hợp bạn hiểu được cách dùng một method chỉ nhờ chữ ký của nó?

UB:

UB:

Yes, there are times when the signature of a method is an incomplete abstraction and a comment is required. This is especially true when the interface is part of a public API, or an API intended for use by a separate team of developers. Within a single development team, however, long descriptive comments on interfaces are often more of an impediment than a help. The team has intimate knowledge of the internals of the system, and will generally be able to understand an interface simply from its signature.

Đúng, có những lúc chữ ký của một method là một trừu tượng hoá chưa đầy đủ và cần đến comment. Điều này đặc biệt đúng khi interface là một phần của public API, hoặc một API dành cho một nhóm lập trình viên khác dùng. Tuy nhiên, trong nội bộ một nhóm phát triển, những comment mô tả dài dòng trên interface thường gây cản trở hơn là giúp ích. Cả nhóm đã nắm rất rõ ruột gan của hệ thống, và nhìn chung sẽ hiểu được một interface chỉ từ chữ ký của nó.

JOHN:

JOHN:

In one of our in-person discussions, you argued that interface comments are unnecessary because, when a group of developers is working on a body of code, they can collectively keep the entire code “loaded” in their minds, so comments are unnecessary: If you have a question, just ask the person who is familiar with that code. This creates a huge cognitive load to keep all that code mentally loaded, and it’s hard for me to imagine that it would actually work. Maybe your memory is better than mine, but I find that I quickly forget code that I wrote just a few weeks ago. In a project of any size, I think

your approach would result in developers spending large amounts of time reading code to re-derive the interfaces, and probably making mistakes along the way. Spending a few minutes to document the interfaces would save time, reduce cognitive load, and reduce bugs.

Trong một lần chúng ta trao đổi trực tiếp, anh lập luận rằng comment cho interface là không cần thiết, vì khi một nhóm lập trình viên cùng làm trên một khối mã, họ có thể chung nhau giữ toàn bộ mã “nạp sẵn” trong đầu, nên không cần comment: có thắc mắc thì cứ hỏi người quen thuộc với phần mã đó. Cách này tạo ra một tải nhận thức khổng lồ để giữ toàn bộ mã trong đầu, và tôi khó mà tưởng tượng nổi nó thực sự chạy được. Có thể trí nhớ của anh tốt hơn tôi, nhưng riêng tôi thì quên rất nhanh những đoạn mã mình mới viết vài tuần trước. Trong một dự án cỡ nào cũng vậy, tôi nghĩ cách làm của anh sẽ khiến lập trình viên tốn hàng đồng thời gian đọc mã để suy ra lại các interface, và nhiều khả năng còn suy sai. Bỏ ra vài phút viết tài liệu cho interface sẽ tiết kiệm thời gian, giảm tải nhận thức và giảm bug.

UB:

UB:

I think that certain interfaces need comments, even if they are private to the team. But I think it is more often the case that the team is familiar enough with the system that well-named methods and arguments are sufficient.

Tôi cho rằng có những interface cần comment, kể cả khi chúng chỉ dùng nội bộ trong nhóm. Nhưng tôi nghĩ trường hợp thường gặp hơn là cả nhóm đã đủ quen thuộc với hệ thống đến mức method và đối số được đặt tên tốt là đủ.

JOHN:

JOHN:

Let's consider a specific example from PrimeGenerator : the

Hãy xét một ví dụ cụ thể trong PrimeGenerator: method

isMultipleOfNthPrimeFactor method. When someone reading the code encounters the call to isMultiple... in isNot... , they need to understand enough about how isMultiple ... works in order to see how it fits into the code of isNot... . The method name does not fully document the interface, so if there is no header comment, then readers will have to read the code of

isMultipleOfNthPrimeFactor. Khi người đọc mã gặp lời gọi isMultiple... bên trong isNot..., họ cần hiểu đủ về cách isMultiple... hoạt động để thấy nó khớp vào mã của isNot... như thế nào. Tên method không mô tả trọn vẹn interface, nên nếu không có comment mở đầu, người đọc sẽ buộc phải đọc mã của

isMultiple . This will force readers to load more information into their heads, which makes it harder to work in the code.

isMultiple. Điều đó bắt họ nạp thêm thông tin vào đầu, khiến việc làm việc với mã khó khăn hơn.

Here is my first attempt at a header comment for isMultiple :

Đây là thử nghiệm đầu tiên của tôi cho comment mở đầu của isMultiple:

```
/**
 * Returns true if candidate is a multiple of primes[n], false othe
 * May modify multiplesOfPrimeFactors[n].
 * @param candidate
```



```

*      Number being tested for primality; must be at least as
*      large as any value passed to this method in the past.
* @param n
*      Selects a prime number to test against; must be
*      <= multiplesOfPrimeFactors.size().
*/

```

What do you think of this?

Anh nghĩ sao về nó?

UB:

UB:

I think it's accurate. I wouldn't delete it if I encountered it. I don't think it should be a Javadoc.

Tôi thấy nó chính xác. Nếu gặp nó, tôi sẽ không xoá. Nhưng tôi không nghĩ nó nên là một Javadoc.

The first sentence is redundant with the name

Câu đầu tiên trùng lặp với cái tên

isMultipleOfNthPrimeFactor and so could be deleted. The warning of the side effect is useful.

isMultipleOfNthPrimeFactor nên có thể xoá đi. Còn lời cảnh báo về tác dụng phụ thì hữu ích.

JOHN:

JOHN:

I agree that the first sentence is largely redundant with the name, and I debated with myself about whether to keep it. I decided to keep it because I think it is a bit more precise than the name; it's also easier to read. You propose to eliminate the redundancy between the comment and the method name by dropping the comment; I would eliminate the redundancy by shortening the method name.

Tôi đồng ý là câu đầu phần lớn trùng với tên hàm, và tôi cũng đã tự tranh luận với chính mình xem có nên giữ nó không. Tôi quyết định giữ vì tôi nghĩ nó chính xác hơn cái tên một chút; nó cũng dễ đọc hơn. Anh đề xuất khử phần trùng lặp giữa comment và tên method bằng cách bỏ comment; còn tôi thì khử phần trùng lặp đó bằng cách rút ngắn tên method.

By the way, you complained earlier about comments being less precise than code, but in this case, the comment is more precise (the method name can't include text like

Nhân tiện, ở trên anh có phàn nàn rằng comment kém chính xác hơn mã, nhưng trong trường hợp này comment lại chính xác hơn (tên method không thể chứa đoạn chữ kiểu như

primes[n]).

UB:

UB:

Fair enough. There are times when precision is better expressed in a comment.

Cũng hợp lý. Có những lúc sự chính xác được diễn đạt trong comment thì tốt hơn.

Continuing with my critique of your comment above: The name candidate is synonymous with "Number being tested for primality."

Nói tiếp về phần tôi chèn comment của anh ở trên: cái tên candidate đã đồng nghĩa với “Number being tested for primality.”

In the end, however, all the words in a comment are just going to have to sit in my brain until I understand why they are there. I’m also going to have to worry if they are accurate. So I’m going to have to read the code to understand and validate the comment.

Nhưng rốt cuộc, mọi con chữ trong một comment đều sẽ phải nằm lì trong đầu tôi cho tới khi tôi hiểu vì sao chúng ở đó. Tôi còn phải lo liệu chúng có chính xác hay không. Thế nên cuối cùng tôi vẫn phải đọc mã để hiểu và kiểm chứng cái comment ấy.

JOHN:

JOHN:

Whoa. That loud sound you just heard was my jaw hitting the floor. Help me understand this a bit better: Approximately what fraction of comments that you encounter in practice are you willing to trust without reading the code to verify them?

Khoan đã. Cái tiếng động lớn anh vừa nghe thấy là hàm tôi rơi xuống sàn đấy. Giúp tôi hiểu rõ hơn chút: trên thực tế, khoảng bao nhiêu phần trăm comment mà anh gặp là anh sẵn sàng tin mà không cần đọc mã để kiểm chứng?

UB:

UB:

I look at every comment as potential misinformation. At best, they are a way to crosscheck the author’s intent against the code. The amount of credence I give to a comment depends a lot on how easy they make that crosscheck. When I read a comment that does not cause me to crosscheck, then I consider it to be of no value. When I see a comment that causes me to crosscheck, and when that crosscheck turns out to be valuable, then that’s a really good comment.

Tôi nhìn mọi comment như một nguồn thông tin sai lệch tiềm tàng. Tốt nhất thì chúng là một cách để đối chiếu chéo ý định của tác giả với mã. Mức độ tin cậy tôi dành cho một comment phụ thuộc rất nhiều vào việc nó khiến phép đối chiếu chéo ấy dễ đến đâu. Khi đọc một comment mà nó chẳng khiến tôi phải đối chiếu chéo, tôi coi nó là vô giá trị. Khi thấy một comment khiến tôi phải đối chiếu chéo, và phép đối chiếu đó hoá ra đáng giá, thì đó là một comment thực sự tốt.

Another way to say this is that the best comments tell me something surprising and verifiable about the code. The worst are those that waste my time telling me something obvious or incorrect.

Nói cách khác, những comment hay nhất cho tôi biết điều gì đó bất ngờ và kiểm chứng được về mã. Những comment tệ nhất là loại làm phí thời gian của tôi để nói một điều hiển nhiên hoặc sai bét.

JOHN:

JOHN:

It sounds like your answer is 0%: You don’t trust any comment unless it has been verified against the code. This makes no sense to me. As I said above, the vast majority of comments are correct. It’s not hard to write comments; the students in my software design class are doing this pretty well within a few weeks. It’s also not hard to keep comments up-to-date as code evolves. Your refusal to trust comments is another sign of your irrational bias against comments.

Nghe như câu trả lời của anh là 0%: anh không tin bất kỳ comment nào trừ khi nó đã được đối chiếu với mã. Với tôi điều đó thật vô lý. Như tôi đã nói ở trên, tuyệt đại đa số comment

là đúng. Viết comment không khó; sinh viên trong lớp thiết kế phần mềm của tôi làm khá tốt việc này chỉ sau vài tuần. Giữ comment luôn cập nhật khi mã tiến hoá cũng không khó. Việc anh từ chối tin comment lại là một dấu hiệu nữa cho định kiến phi lý của anh với comment.

Refusing to trust comments incurs a very high cost. In order to understand how to invoke a method, you will have to read all of the code of that method; if the method invokes other methods, you will also have to read them, and the methods they invoke, recursively. This is an enormous amount of work in comparison to reading (and trusting) a simple interface comment like the one I wrote above.

Từ chối tin comment phải trả một cái giá rất đắt. Để hiểu cách gọi một method, anh sẽ phải đọc toàn bộ mã của method đó; nếu nó gọi các method khác, anh cũng phải đọc chúng, rồi lại đọc các method mà chúng gọi, cứ thế đệ quy. Đó là một khối lượng công việc khổng lồ so với việc đọc (và tin) một comment interface đơn giản như cái tôi viết ở trên.

If you choose not to write an interface comment for methods, then you leave the interface of that method undefined. Even if someone reads the code of the method, they won't be able to tell which parts of the implementation are expected to remain the same and which parts may change (there is no way to specify this "contract" in code). This will result in misunderstanding and more bugs.

Nếu anh chọn không viết comment interface cho các method, thì anh để interface của method đó ở trạng thái không được định nghĩa. Ngay cả khi ai đó đọc mã của method, họ cũng không thể biết phần nào của hiện thực được kỳ vọng giữ nguyên và phần nào có thể thay đổi (không có cách nào đặc tả "hợp đồng" ấy bằng mã). Kết quả là hiểu nhầm và thêm bug.

UB:

UB:

Well, I guess I've just been burned more than you have. I've gone down too many false comment-induced rabbit holes and wasted too much time on worthless word salads.

Chà, chắc tôi đã bị bỏng nhiều hơn anh. Tôi đã chui xuống quá nhiều hang thỏ do comment sai dẫn lối, và phí quá nhiều thời gian cho những mớ chữ nghĩa hổ lốn vô dụng.

Of course, my trust in comments is not a binary thing. I read them if they are there; but I don't implicitly trust them. The more gratuitous I feel the author was, or the less adept at English the author is, the less I trust the comments.

Tất nhiên, niềm tin của tôi với comment không phải chuyện có hoặc không. Có comment thì tôi đọc; nhưng tôi không mặc nhiên tin chúng. Tôi càng cảm thấy tác giả viết thừa thãi, hoặc tác giả càng kém tiếng Anh, thì tôi càng ít tin comment.

As I said above, our IDEs tend to paint comments in an ignorable color. I have my IDE paint comments in bright fire-engine red because, when I write a comment, I intend for it to be read.

Như tôi đã nói ở trên, các IDE của chúng ta có xu hướng tô comment bằng một màu dễ phớt lờ. Tôi cho IDE của mình tô comment bằng màu đỏ chói như xe cứu hoả, bởi vì khi tôi viết một comment, tôi có ý muốn nó được đọc.

By the same token, I use long names as a substitute for comments because I intend for those long names to be read; and it is very hard for a programmer to ignore names.

Cũng vì lẽ đó, tôi dùng tên dài để thay cho comment, vì tôi muốn những cái tên dài ấy được đọc; và lập trình viên thì rất khó phớt lờ một cái tên.

JOHN:

JOHN:

I mentioned earlier that there are two general reasons why comments are needed. So far, we've been discussing the first reason (abstraction). The second general reason for comments is for important information that is not obvious from the code. The algorithm in PrimeGenerator is very nonobvious, so quite a few comments are needed to help readers understand what is going on and why. Most of the algorithm's complexity arises because it is designed to compute primes efficiently.

Ở trên tôi có nhắc rằng có hai lý do chung khiến ta cần comment. Từ này đến giờ chúng ta bàn về lý do thứ nhất (trừu tượng hoá). Lý do chung thứ hai là để ghi lại những thông tin quan trọng nhưng không hiển nhiên từ mã. Thuật toán trong PrimeGenerator rất không hiển nhiên, nên cần khá nhiều comment để giúp người đọc hiểu chuyện gì đang diễn ra và tại sao. Phần lớn độ phức tạp của thuật toán này nảy sinh vì nó được thiết kế để tính số nguyên tố một cách hiệu quả.

- The algorithm goes out of its way to avoid divisions, which were quite expensive when Knuth wrote his original version (they aren't that expensive nowadays).
 - Thuật toán cố hết sức để tránh phép chia, vốn khá đắt đỏ vào thời Knuth viết phiên bản gốc (ngày nay thì chúng chẳng đắt đến thế).
- The first multiple for each new prime number is computed by squaring the prime, rather than multiplying it by 3. This is mysterious: Why is it safe to skip the intervening odd multiples? Furthermore, it might seem that this optimization only has a small impact on performance, but in fact, it makes an enormous difference (orders of magnitude). Using the square has the side-effect that when testing a candidate, only primes up to the square root of the candidate are tested. If $3x$ were used as the initial multiple, primes within a factor of 3 of the candidate would be tested; that's a lot more tests. This implication of using the square is so nonobvious that I only realized it while preparing material for this discussion; it never occurred to me in the many times I have discussed the code with students.
 - Bội số đầu tiên của mỗi số nguyên tố mới được tính bằng cách bình phương số nguyên tố đó, thay vì nhân nó với 3. Chỗ này rất bí ẩn: vì sao bỏ qua các bội lẻ nằm giữa lại an toàn? Hơn nữa, thoạt nhìn có vẻ tối ưu này chỉ tác động nhỏ tới hiệu năng, nhưng thực tế nó tạo ra khác biệt khổng lồ (chênh nhau nhiều bậc độ lớn). Việc dùng bình phương có tác dụng phụ là khi kiểm tra một ứng viên, chỉ những số nguyên tố tới căn bậc hai của ứng viên đó mới được đem ra thử. Nếu dùng $3x$ làm bội số khởi đầu, thì những số nguyên tố nằm trong khoảng một phần ba của ứng viên cũng sẽ bị đem thử; nhiều phép thử hơn hẳn. Hệ quả này của việc dùng bình phương không hiển nhiên đến mức tôi chỉ nhận ra khi đang chuẩn bị tài liệu cho cuộc thảo luận này; suốt bao nhiêu lần bàn về đoạn mã ấy với sinh viên, tôi chưa từng nghĩ tới.

Neither of these issues is obvious from the code; without comments, readers are left to figure them out on their own. The students in my class are generally unable to figure out either of them in the 30 minutes I give them, but I think that comments would have allowed them to understand in a few minutes. Going back to my introductory remarks, this is an example where information is important, so it needs to be made available.

Không điểm nào trong hai điểm trên là hiển nhiên từ mã cả; không có comment thì người đọc phải tự mò ra. Sinh viên lớp tôi nói chung không tìm ra được điểm nào trong 30 phút tôi cho họ, nhưng tôi tin rằng comment sẽ giúp họ hiểu chỉ trong vài phút. Quay lại những nhận định mở đầu của tôi: đây là ví dụ mà thông tin thực sự quan trọng, nên nó cần được cung cấp.

Do you agree that there should be comments to explain each of these two issues?

people have to stare at something for a while to figure it out. I don't agree with this approach: It results in wasted time, misunderstandings, and bugs. I think software should be totally obvious, where readers don't need to be clever or "stare at this for some time" to figure things out. Suffering followed by catharsis is great for Greek tragedies, but not for reading code. Every question a reader might have should be naturally answered, either in the code or in comments. Key ideas and important conclusions should be stated explicitly, not left for the reader to deduce. Ideally, even if a reader is in a hurry and doesn't read the code very carefully, their first guesses about how things work (and why) should be correct. To me, that's clean code.

Có lẽ ở đây chúng ta khác nhau về triết lý ở tầng nền tảng. Tôi có cảm giác anh vui vẻ ném cho người đọc vài manh mối rồi để họ tự ghép lại. Có lẽ anh không ngại chuyện người ta phải nhìn chăm chăm vào cái gì đó một hồi mới hiểu ra. Tôi không đồng tình với cách tiếp cận đó: nó dẫn tới lãng phí thời gian, hiểu nhầm và bug. Tôi cho rằng phần mềm nên *hoàn toàn hiển nhiên*, để người đọc không cần thông minh xuất chúng hay phải "nhìn chăm chăm một hồi" mới vỡ lẽ. Đau khổ rồi thanh tẩy thì hay cho bi kịch Hy Lạp, chứ không hay cho việc đọc mã. Mọi câu hỏi người đọc có thể đặt ra đều nên được trả lời một cách tự nhiên, hoặc trong mã hoặc trong comment. Các ý then chốt và kết luận quan trọng phải được nói ra tường minh, chứ không để người đọc tự suy diễn. Lý tưởng nhất là ngay cả khi người đọc đang vội và không đọc mã thật kỹ, những phỏng đoán đầu tiên của họ về cách mọi thứ vận hành (và vì sao) vẫn đúng. Với tôi, đó mới là clean code.

UB:

UB:

I don't disagree with your sentiment. Good, clean code should be as easy as possible to understand. I want to give my readers as many clues as possible so that the code is intuitive to read.

Tôi không phản đối tinh thần đó. Mã tốt, mã sạch thì nên dễ hiểu nhất có thể. Tôi muốn cho người đọc càng nhiều manh mối càng tốt để mã đọc lên thấy trực giác.

That's the goal. As we are about to see, that can be a tough goal to achieve.

Đó là mục tiêu. Và như ta sắp thấy, đó có thể là một mục tiêu khó đạt.

JOHN:

JOHN:

In that case, do you still stand by the "picture" you painted above? It doesn't seem consistent with what you just said. And if you really wanted to give your readers as many clues as possible, you'd include a lot more comments.

Nếu vậy thì anh có còn bảo vệ "bức tranh" anh vừa vẽ ở trên không? Nó có vẻ không nhất quán với điều anh vừa nói. Và nếu anh thật sự muốn cho độc giả càng nhiều manh mối càng tốt, hẳn anh đã viết thêm rất nhiều comment.

UB:

UB:

I stand by the picture as far as its accuracy is concerned. And I think it makes a good crosscheck. I have no illusions that it is easy to understand.

Tôi bảo vệ bức tranh đó ở khía cạnh nó chính xác. Và tôi nghĩ nó là một phép đối chiếu chéo tốt. Tôi không hề ảo tưởng rằng nó dễ hiểu.

This algorithm is challenging and will require work to comprehend. I finally understood it when I drew this picture in my mind while on that bike ride. When I got home, I drew it for real and presented it in hopes that it might help someone willing to do the work to understand it.

Thuật toán này khó nhằn và đòi hỏi phải bỏ công mới hiểu được. Tôi chỉ thực sự hiểu nó khi vẽ bức tranh này trong đầu lúc đang đạp xe. Về đến nhà, tôi vẽ nó ra thật và đưa vào đây với hy vọng nó giúp được ai đó chịu bỏ công ra để hiểu.

Comments Summary — Tổng kết về comment

JOHN:

JOHN:

Let's wrap up this section of the discussion. Here is my summary of where we agree and disagree.

Hãy khép lại phần này của cuộc thảo luận. Đây là bản tóm tắt của tôi về những chỗ chúng ta đồng ý và bất đồng.

- Our overall views of comments are fundamentally different. I see more value in comments than you do, and I believe that they play a fundamental and irreplaceable role in system design. You agree that there are places where comments are necessary, but that comments don't always make it easier to understand code, so you see far fewer places where comments are needed.
- I would probably write 5 to 10 times more lines of comments for a given piece of code than you would.
- I believe that missing comments are a much greater cause of lost **productivity** than erroneous or unhelpful comments; you believe that comments are a net negative, as generally practiced: Bad comments cost more time than good comments save.
- You view it as problematic that comments are written in English rather than a programming language. I don't see this as particularly problematic and think that, in many cases, English works better.
- You recommend that developers should take information that I would represent as comments and recast it into code if at all possible. One example of this is super-long method names. I believe that super-long names are awkward and hard to understand, and that it would be better to use shorter names supplemented with comments.
 - Quan điểm tổng thể của chúng ta về comment khác nhau về căn bản. Tôi thấy comment có giá trị hơn nhiều so với anh, và tôi tin chúng đóng một vai trò nền tảng, không thể thay thế trong thiết kế hệ thống. Anh đồng ý rằng có những chỗ comment là cần thiết, nhưng cho rằng comment không phải lúc nào cũng làm mã dễ hiểu hơn, nên anh thấy số chỗ cần comment ít hơn hẳn.
 - Với cùng một đoạn mã, tôi có lẽ sẽ viết nhiều hơn anh từ 5 đến 10 lần số dòng comment.
 - Tôi tin rằng comment còn thiếu là nguyên nhân gây mất năng suất lớn hơn nhiều so với comment sai hoặc comment vô ích; còn anh tin rằng comment, theo cách người ta vẫn thường viết, là một khoản âm ròng: comment tẻ ngốn nhiều thời gian hơn phần thời gian mà comment tốt tiết kiệm được.
 - Anh coi việc comment được viết bằng tiếng Anh chứ không phải bằng ngôn ngữ lập trình là một vấn đề. Tôi không thấy điều đó đặc biệt phiền, và cho rằng trong nhiều trường hợp tiếng Anh còn diễn đạt tốt hơn.
 - Anh khuyến nghị lập trình viên hãy lấy những thông tin mà tôi sẽ thể hiện bằng comment và đúc lại thành mã nếu có thể. Một ví dụ là những cái tên method siêu dài. Tôi tin rằng tên siêu dài thì lúng túng và khó hiểu, và rằng dùng tên ngắn hơn kèm

comment bổ trợ sẽ tốt hơn.

- I believe that it is not possible to define interfaces and create abstractions without a lot of comments. You agree for public APIs, but see little need to comment interfaces that are internal to the team.
- You are unwilling to trust comments until you have read code to verify them. I generally trust comments; by doing so, I don't need to read as much code as you do. You think this exposes me to too much risk.
- We agree that implementation code only needs comments when the code is nonobvious. Although neither of us argues for a large number of implementation comments, I'm more likely to see value in them than you do.
 - Tôi tin rằng không thể định nghĩa interface và tạo ra các trừu tượng hoá nếu không có thật nhiều comment. Anh đồng ý với public API, nhưng thấy ít cần comment cho những interface nội bộ trong nhóm.
 - Anh không chịu tin comment cho tới khi đã đọc mã để kiểm chứng. Tôi thì nhìn chung tin comment; nhờ vậy tôi không phải đọc nhiều mã như anh. Anh cho rằng làm thế khiến tôi hứng chịu quá nhiều rủi ro.
 - Chúng ta đồng ý rằng mã hiện thực chỉ cần comment khi nó không hiển nhiên. Dù cả hai đều không cổ vũ cho một lượng lớn comment trong phần hiện thực, tôi vẫn dễ thấy giá trị ở chúng hơn anh.

Overall, we struggled to find areas of agreement on this topic.

Nhìn chung, chúng ta đã rất chật vật để tìm ra điểm chung trong chủ đề này.

UB:

UB:

This is a fair assessment of our individual positions; which I assume are based on our different individual experiences. Over the years, I have found the vast majority of comments, as generally practiced in the industry, to be unhelpful. You seem to have found more help in the comments you have encountered.

Đây là một đánh giá công bằng về lập trường của mỗi người; mà tôi cho là bắt nguồn từ trải nghiệm khác nhau của mỗi chúng ta. Nhiều năm qua, tôi thấy tuyệt đại đa số comment, theo cách ngành này vẫn thường viết, là vô ích. Còn anh thì có vẻ đã tìm được nhiều trợ giúp hơn từ những comment mình gặp.

John's Rewrite of — Bản viết lại của John

JOHN:

JOHN:

I mentioned that I ask the students in my software design class to rewrite PrimeGenerator to fix all of its design problems. Here is my rewrite (note: This was written before we began our discussion; given what I have learned during the discussion, I would now change several of the comments, but I have left this in its original form):

Tôi có nhắc rằng mình yêu cầu sinh viên trong lớp thiết kế phần mềm viết lại PrimeGenerator để khắc phục toàn bộ vấn đề thiết kế của nó. Đây là bản viết lại của tôi (lưu ý: bản này được viết trước khi chúng tôi bắt đầu cuộc thảo luận; với những

gì đã học được trong quá trình thảo luận, giờ tôi sẽ sửa vài comment, nhưng tôi vẫn để nguyên bản gốc):

```
package literatePrimes;
import java.util.ArrayList;
public class PrimeGenerator2 {
    /**
     * Computes the first prime numbers; the return value contains
     * computed primes, in increasing order of size.
     * @param n
     *      How many prime numbers to compute.
     */
    public static int[] generate(int n) {
        int[] primes = new int[n];
        // Used to test efficiently (without division) whether a ca
        // is a multiple of a previously-encountered prime number.
        // Each entry here contains an odd multiple of the correspo
        // entry in primes. Entries increase monotonically.
        int[] multiples = new int[n];
        // Index of the last value in multiples that we need to co
        // when testing candidates (all elements after this are gre
        // than our current candidate, so they don't need to be
        // considered).
        int lastMultiple = 0;
        // Number of valid entries in primes.
        int primesFound = 1;

        primes[0] = 2;
        multiples[0] = 4;
        // Each iteration through this loop considers one candidate
        // the even numbers, since they can't be prime.
        candidates:
        for (int candidate = 3; primesFound < n; candidate += 2) {
            if (candidate >= multiples[lastMultiple]) {
                lastMultiple++;
            }
            // Each iteration of this loop tests the candidate agai
            // potential prime factor. Skip the first factor (2) si
            // only consider odd candidates.
            for (int i = 1; i <= lastMultiple; i++) {
                while (multiples[i] < candidate) {
                    multiples[i] += 2*primes[i];
                }
                if (multiples[i] == candidate) {
                    continue candidates;
                }
            }
            primes[primesFound] = candidate;
            // Start with the prime's square here, rather than 3x t
            // prime. This saves time and is safe because all of t
            // intervening multiples will be detected by smaller p
```

```

        // numbers. As an example, consider the prime 7: the va
        // in multiples will start at 49; 21 will be ruled out
        // multiple of 3, and 35 will be ruled out as a multipl
        // so 49 is the first multiple that won't be ruled out
        // smaller prime.
        multiples[primesFound] = candidate*candidate;
        primesFound++;
    }
    return primes;
}
}

```

Everyone can read this and decide for themselves whether they think it is easier to understand than the original. I'd like to mention a couple of overall things.

Ai cũng có thể đọc đoạn này và tự quyết định xem nó có dễ hiểu hơn bản gốc hay không. Tôi muốn nói vài điểm tổng quát.

- There is only one method. I didn't subdivide it, because I felt the method already divides naturally into pieces that are distinct and understandable. It didn't seem to me that pulling out methods would improve readability significantly. When students rewrite the code, they typically have two or three methods, and those are usually OK too.
- There are a lot of comments. It's extremely rare for me to write code with this density of comments. Most methods I write have no comments in the body, just a header comment describing the interface. But this code is subtle and tricky, so it needs a lot of comments to make the subtleties clear to readers. The long length of some of the comments is a red flag indicating that I struggled to find a clear and simple explanation for the code. Even with all the additional explanatory material, this version is a bit shorter than the original (65 lines versus 70).
- Chỉ có đúng một method. Tôi không chia nhỏ nó ra, vì tôi thấy method này vốn đã tự chia thành những mảnh riêng biệt và dễ hiểu. Với tôi, việc tách method ra không giúp cải thiện đáng kể tính dễ đọc. Khi sinh viên viết lại đoạn mã này, họ thường dùng hai hoặc ba method, và như vậy cũng thường là ổn.
- Có rất nhiều comment. Cực kỳ hiếm khi tôi viết mã với mật độ comment dày như thế này. Phần lớn method tôi viết không có comment nào trong thân hàm, chỉ có một comment ở đầu mô tả interface. Nhưng đoạn mã này tinh vi và hóc hiểm, nên nó cần nhiều comment để làm rõ những chỗ tinh tế cho người đọc. Việc một số comment dài lê thê chính là cớ để cho thấy tôi đã vật lộn để tìm ra một lời giải thích rõ ràng và đơn giản cho đoạn mã. Ngay cả khi đã thêm hết đồng chú giải đó, phiên bản này vẫn ngắn hơn bản gốc một chút (65 dòng so với 70).

UB:

UB:

I presume this is a complete rewrite. My guess is that you worked to understand the algorithm from Clean Code and then wrote this from scratch. If that's so, then fair enough.

Tôi đoán đây là một bản viết lại hoàn toàn. Tôi ngờ rằng anh đã cố hiểu thuật toán từ Clean Code rồi viết lại từ đầu. Nếu vậy thì cũng công bằng thôi.

In Clean Code, I refactored Knuth's algorithm in order to give it a little structure. That's not the same as a complete rewrite.

Trong Clean Code, tôi refactor thuật toán của Knuth để cho nó chút cấu trúc. Việc đó không giống với viết lại hoàn toàn.

Having said that, your version is much better than either Knuth's or mine.

Nói vậy rồi, thì phiên bản của anh hay hơn hẳn cả bản của Knuth lẫn bản của tôi.

I wrote that chapter 18 years ago, so it's been a long time since I saw and understood this algorithm. When I first saw your challenge, I thought: "Oh, I can figure out my own code!" But no. I could see all the moving parts, but I could not figure out why those moving parts generated a list of prime numbers.

Tôi viết chương đó cách đây 18 năm, nên đã rất lâu rồi tôi mới nhìn lại và hiểu được thuật toán này. Lần đầu nhìn thấy thử thách của anh, tôi nghĩ: "Ồ, mã của chính mình thì mình hiểu được chứ!" Nhưng không. Tôi thấy rõ mọi bộ phận chuyển động, nhưng lại không hiểu nổi tại sao những bộ phận đó lại sinh ra được danh sách các số nguyên tố.

So then, I looked at your code. I had the same problem. I could see all the moving parts, all with comments, but I still could not figure out why those moving parts generated a list of prime numbers.

Thế là tôi quay sang xem mã của anh. Tôi gặp đúng vấn đề đó. Tôi thấy hết mọi bộ phận chuyển động, mỗi cái đều có comment, mà vẫn không hiểu nổi tại sao chúng lại sinh ra được danh sách số nguyên tố.

Figuring that out required a lot of staring at the ceiling, closing my eyes, visualizing, and riding my bike.

Để hiểu ra được, tôi phải nhìn trần nhà rất lâu, nhắm mắt lại, hình dung trong đầu, và đạp xe.

Among the problems I had were the comments you wrote. Let's take them one at a time.

Trong số những rắc rối tôi gặp phải có chính những comment anh viết. Ta hãy xét từng cái một.

```
/**
 * Computes the first prime numbers; the return value contains the
 * computed primes, in increasing order of size.
 * @param n
 * How many prime numbers to compute
```

```
    How many prime numbers to compute.
 */
public static int[] generate(int n) {
```

It seems to me that this would be better as:

Với tôi thì viết thế này sẽ tốt hơn:

```
public static int[] generateNPrimeNumbers(int n) {
or, if you must:
```

hoặc, nếu anh nhất định phải viết:

```
//Return the first n prime numbers
public static int[] generate(int n) {
```

I'm not opposed to Javadocs as a rule; but I write them only when absolutely necessary. I also have an aversion for descriptions and @param statements that are perfectly obvious from the method signature.

Về nguyên tắc, tôi không phản đối Javadoc; nhưng tôi chỉ viết chúng khi thật sự cần thiết. Tôi cũng dị ứng với những phần mô tả và những dòng @param mà nhìn chữ ký method là đã thấy quá rõ.

The next comment cost me a good 20 minutes of puzzling things out.

Cái comment tiếp theo ngón của tôi ngon lành 20 phút vò đầu bứt tai.

```
// Used to test efficiently (without division) whether a candidate
// is a multiple of a previously-encountered prime number. Each ent
// here contains an odd multiple of the corresponding entry in
// primes. Entries increase monotonically.
```

First of all, I'm not sure why the "division" statement is necessary. I'm old-school, so I expect that everyone knows to avoid division in inner loops if it can be avoided. But maybe I'm wrong about that...

Trước hết, tôi không rõ tại sao câu về "phép chia" lại cần thiết. Tôi thuộc trường phái cũ, nên tôi mặc định ai cũng biết là phải tránh phép chia trong vòng lặp trong cùng nếu có thể tránh. Nhưng cũng có thể tôi sai về chuyện đó...

Also, the Sieve of Eratosthenes does not do division, and is a lot easier to understand and explain than this algorithm. So why this particular algorithm? I think Knuth was trying to save memory —and in 1982, saving memory was important. This algorithm uses a lot less memory than the sieve.

Ngoài ra, sàng Eratosthenes (Sieve of Eratosthenes) không dùng phép chia, và dễ hiểu, dễ giải thích hơn nhiều so với thuật toán này. Vậy tại sao lại chọn đúng thuật toán này? Tôi nghĩ Knuth muốn tiết kiệm bộ nhớ — và vào năm 1982 thì tiết kiệm bộ nhớ là chuyện quan trọng. Thuật toán này dùng ít bộ nhớ hơn hẳn so với sàng.

Then came the phrase: Each entry here contains an odd multiple... . I looked at that, and then at the code, and I saw

Rồi tới cụm từ: Each entry here contains an odd multiple... (Mỗi phần tử ở đây chứa một bội số lẻ...). Tôi nhìn dòng đó, rồi nhìn vào mã, và tôi thấy

```
multiples[0] = 4; .
```

"That's not odd," I said to myself. "So maybe he meant even."

"Cái đó có lẻ đâu," tôi tự nhủ. "Vậy chắc anh ấy muốn nói là chẵn."

So then, I looked down and saw `multiples[i] += 2*primes[i]; .`

Thế là tôi nhìn xuống dưới và thấy `multiples[i] += 2*primes[i]; .`

"That's adding an even number!" I said to myself. "I'm pretty sure he meant to say 'even' instead of 'odd'."

"Đây là cộng thêm một số chẵn!" tôi tự nhủ. "Tôi khá chắc là anh ấy định viết 'even' chứ không phải 'odd'."

I hadn't yet worked out what the multiples array was. So I thought it was perfectly reasonable that it would have even numbers in it, and that your comment was simply an understandable word transposition. After all, there's no compiler for comments, so they suffer from the kinds of mistakes that humans often make with words.

Lúc đó tôi vẫn chưa hiểu ra mảng multiples là cái gì. Nên tôi thấy hoàn toàn hợp lý khi cho rằng nó chứa các số chẵn, và comment của anh chỉ đơn giản là một vụ nhầm chữ dễ thông cảm. Suy cho cùng, chẳng có compiler nào cho comment cả, nên chúng dính đủ loại lỗi mà con người vẫn thường mắc với ngôn từ.

It was only when I got to `multiples[primesFound] = candidate*candidate;` that I started to question things. If the

Mãi đến khi tới dòng `multiples[primesFound] = candidate*candidate;` tôi mới bắt đầu đặt câu hỏi. Nếu

`candidate` is prime, shouldn't `prime*prime` be odd in every case beyond 2? I had to do the math in my head to prove that: $(2n + 1)(2n + 1) = 4n^2 + 4n + 1$... Yeah, that's odd.

`candidate` là số nguyên tố, thì chẳng phải `prime*prime` luôn là số lẻ trong mọi trường hợp từ 2 trở đi hay sao? Tôi phải nhẩm toán trong đầu để chứng minh điều đó: $(2n + 1)(2n + 1) = 4n^2 + 4n + 1$... Ủ, đúng là số lẻ.

OK, so the multiples array is full of odd multiples, except for the first element, since it will be multiples of 2.

Được rồi, vậy mảng multiples chứa toàn bội số lẻ, trừ phần tử đầu tiên, vì đó là các bội của 2.

So perhaps that comment should be:

Nên có lẽ comment kia phải là:

```
// multiples of corresponding prime.
```

Or perhaps we should change the name of the array to something like `primeMultiples` and drop the comment altogether.

Hoặc có lẽ ta nên đổi tên mảng thành đại loại như `primeMultiples` rồi bỏ hẳn cái comment đi.

Moving on to the next comment:

Chuyển sang comment tiếp theo:

```
// Each iteration of this loop tests the candidate against one  
// potential prime factor. Skip the first factor (2) since we  
// only consider odd candidates.
```

That doesn't make a lot of sense. The code it's talking about is:

Câu đó chẳng mấy hợp lý. Đoạn mã mà nó nói tới là:

```
for (int i = 1; i <= lastMultiple; i++) {  
    while (multiples[i] < candidate) {
```

The multiples array, as we have now learned, is an array of multiples of prime numbers. This loop is not testing the candidate against prime factors ; it's testing it against the current prime multiples .

Mảng multiples, như ta vừa biết, là mảng các bội số của các số nguyên tố. Vòng lặp này không kiểm tra `candidate` với các thừa số nguyên tố (prime factors); nó kiểm tra `candidate` với các bội số nguyên tố hiện tại (prime multiples).

Fortunately for me, the third or fourth time I read this comment I realized that you really meant to use the word multiples . But the only way for me to know that was to understand the algorithm. And when I understand the algorithm, why do I need the comment?

May cho tôi là đến lần đọc thứ ba hay thứ tư cái comment này, tôi nhận ra rằng ý anh thật ra là dùng từ multiples. Nhưng cách duy nhất để tôi biết được điều đó là hiểu thuật toán. Mà một khi đã hiểu thuật toán rồi, thì tôi cần cái comment để làm gì nữa?

That left me with one final question: what the deuce was the reason behind:

Còn lại đúng một câu hỏi cuối: cái quái gì đứng sau dòng này vậy:

```
 multiples[primesFound] = candidate*candidate;
```

Why the square? That makes no sense. So I changed it to:

Sao lại bình phương? Chẳng có nghĩa lý gì cả. Thế là tôi đổi thành:

```
 multiples[primesFound] = candidate;
```

And it worked just fine. So this must be an optimization of some kind.

Và nó chạy ngon lành. Vậy chỗ này hẳn phải là một tối ưu nào đó.

Your comment to explain this is:

Comment anh viết để giải thích chuyện này là:

```
// Start with the prime's square here, rather than 3x the prime.
// This saves time and is safe because all of the intervening
// multiples will be detected by smaller prime numbers. As an
// example, consider the prime 7: the value in multiples will
// start at 49; 21 will be ruled out as a multiple of 3, and
// 35 will be ruled out as a multiple of 5, so 49 is the first
// multiple that won't be ruled out by a smaller prime.
```

The first few times I read this it made no sense to me at all. It was just a jumble of numbers.

Vài lần đọc đầu, tôi chẳng hiểu gì cả. Nó chỉ là một mớ số lộn xộn.

I stared at the ceiling, and closed my eyes to visualize. I couldn't see it. So I went on a long, contemplative bike ride, during which I realized that the prime multiples of 2 will at one point contain 23 *and then* 25. So the multiples array will at some point contain multiples of primes larger than the prime they represent. And it clicked!

Tôi nhìn chăm chăm lên trần nhà, rồi nhắm mắt lại để hình dung. Tôi không thấy được. Thế là tôi đi đạp xe một chuyến dài để ngẫm nghĩ, và trong lúc đó tôi nhận ra rằng các bội số nguyên tố của 2 sẽ có lúc chứa 23 rồi 25. Nghĩa là đến một thời điểm nào đó, mảng multiples sẽ chứa bội của những số nguyên tố lớn hơn chính số nguyên tố mà phần tử ấy đại diện. Và thế là mọi thứ khớp lại!

Suddenly it all made sense. I realized that the multiples array was the equivalent of the array of booleans we use in the Sieve of Eratosthenes—but with a really interesting twist. If you were to do the sieve on a whiteboard, you could erase every number less than the candidate, and only cross out the numbers that were the next multiples of all the previous primes.

Đột nhiên tất cả đều sáng tỏ. Tôi nhận ra rằng mảng multiples chính là tương đương của mảng boolean mà ta dùng trong sàng Eratosthenes—nhưng với một cú xoay rất thú vị. Nếu bạn làm sàng trên bảng trắng, bạn có thể xóa đi mọi số nhỏ hơn candidate, và chỉ gạch bỏ những số là bội tiếp theo của tất cả các số nguyên tố trước đó.

That explanation makes perfect sense to me now, but I'd be willing to bet that those who are reading it are puzzling over it. The idea is just hard to explain.

Lời giải thích đó giờ với tôi hoàn toàn hợp lý, nhưng tôi dám cá là những ai đang đọc nó vẫn đang vò đầu. Ý tưởng này đơn giản là khó giải thích.

Finally, I went back to your comment and could see what you were saying.

Cuối cùng, tôi quay lại comment của anh và đã thấy được anh muốn nói gì.

A Tale of Two Programmers — Chuyện hai người lập trình

UB:

UB:

The bottom line here is that you and I both fell into the same trap. I refactored that old algorithm 18 years ago, and I thought all those method and variable names would make my intent clear— because I understood that algorithm.

Mấu chốt ở đây là cả anh và tôi đều rơi vào cùng một cái bẫy. Tôi refactor thuật toán cũ kỹ đó 18 năm trước, và tôi tưởng rằng tất cả những tên method, tên biến kia sẽ làm rõ ý định của mình — bởi vì tôi hiểu thuật toán đó.

You wrote that code awhile back and decorated it with comments that you thought would explain your intent— because you understood that algorithm.

Anh viết đoạn mã đó cách đây một thời gian và trang trí nó bằng những comment mà anh tưởng sẽ giải thích được ý định của mình — bởi vì anh hiểu thuật toán đó.

But my names didn't help me 18 years later. They didn't help you or your students either. And your comments didn't help me.

Nhưng những cái tên của tôi chẳng giúp được gì cho chính tôi 18 năm sau. Chúng cũng chẳng giúp được anh hay sinh viên của anh. Và những comment của anh cũng chẳng giúp được tôi.

We were inside the box trying to communicate to those who stood outside and could not see what we saw.

Chúng ta đứng bên trong cái hộp và cố giao tiếp với những người đứng bên ngoài, những người không nhìn thấy điều ta nhìn thấy.

The bottom line is that it is very difficult to explain something to someone who is not intimate with the details you are trying to explain. Often, our explanations make sense only after the reader has worked out the details for themselves.

Mấu chốt là: giải thích một điều gì đó cho người chưa thân thuộc với những chi tiết mà bạn đang cố giải thích là chuyện cực kỳ khó. Thường thì lời giải thích của ta chỉ trở nên có nghĩa sau khi người đọc đã tự mình mò ra hết các chi tiết.

JOHN:

JOHN:

There's a lot of stuff in your discussion above, but I think it all boils down to one thing: You don't like the comments that I wrote. As I mentioned earlier, complexity is in the eye of the reader: If you say that my comments were confusing or didn't help you to understand the code, then I have to take that seriously.

Trong phần thảo luận của anh ở trên có rất nhiều thứ, nhưng tôi nghĩ tất cả đều quy về một điều: anh không thích những comment tôi viết. Như tôi đã nói trước đó, độ phức tạp nằm ở mắt người đọc: nếu anh bảo rằng comment của tôi gây khó hiểu hoặc không giúp anh hiểu được đoạn mã, thì tôi phải xem chuyện đó là nghiêm túc.

At the same time, you have made it clear that you don't see much value in comments in general. Your preference is to have essentially no comments for this code (or any code). You argue above that there is simply nothing that comments can do to make the code easier to understand; the only way to understand the code is to read the code. That is a cop-out.

Nhưng đồng thời, anh cũng nói rõ rằng nhìn chung anh không thấy comment có mấy giá trị. Anh thích đoạn mã này (hay bất kỳ đoạn mã nào) về cơ bản là không có comment nào cả. Ở trên anh lập luận rằng chẳng có gì mà comment làm được để khiến đoạn mã dễ hiểu hơn; cách duy nhất để hiểu mã là đọc mã. Đó là lối thoái thác.

UB:

UB:

Sorry to interrupt you, but I think you are overstating my position. I certainly never said that comments can never be helpful. Sometimes, of course, they are. What I said was that I only trust them if the code validates them. Sometimes a comment will make that validation a lot easier.

Xin lỗi phải ngắt lời anh, nhưng tôi nghĩ anh đang thổi phồng quan điểm của tôi. Tôi chắc chắn chưa bao giờ nói rằng comment không bao giờ hữu ích. Đôi khi, dĩ nhiên, chúng hữu ích. Điều tôi nói là tôi chỉ tin chúng khi đoạn mã xác nhận chúng. Đôi khi một comment sẽ khiến việc xác nhận đó dễ hơn nhiều.

JOHN:

JOHN:

You keep saying that you sometimes find use for comments, but the reality is that “sometimes” almost never occurs in your code. We'll see this when we look at your revision of my code.

Anh cứ nói mãi rằng đôi khi anh thấy comment có ích, nhưng thực tế là cái “đôi khi” đó gần như không bao giờ xuất hiện trong mã của anh. Ta sẽ thấy điều này khi xem bản anh sửa lại mã của tôi.

Now back to my point. In order to write our various versions of the code, you and I had to accumulate a lot of knowledge about the algorithm, such as why it's OK for the first multiple of a prime to be its square. Unfortunately, not all of that knowledge can be represented in the code. It is our **professional** responsibility to do the best we can to convey that knowledge in comments so that readers do not have to reconstruct it over and over. Even if the resulting comments are imperfect, they will make the code easier to understand.

Giờ quay lại luận điểm của tôi. Để viết được các phiên bản khác nhau của đoạn mã này, cả anh và tôi đều phải tích lũy rất nhiều kiến thức về thuật toán, chẳng hạn tại sao bội đầu tiên của một số nguyên tố lại có thể là bình phương của chính nó. Đáng tiếc là không phải toàn bộ kiến thức đó đều biểu diễn được trong mã. Trách nhiệm nghề nghiệp của chúng ta là làm hết sức để truyền tải kiến thức ấy qua comment, sao cho người đọc không phải dựng lại nó hết lần này đến lần khác. Ngay cả khi những comment thu được chưa hoàn hảo, chúng vẫn khiến đoạn mã dễ hiểu hơn.

If a situation like this occurred in real life, I would work with you and others to improve my comments. For example, I would ask you questions to get a better sense of why the “squared prime” comment didn't seem to help you.

Nếu một tình huống như thế này xảy ra ngoài đời thực, tôi sẽ cùng anh và những người khác cải thiện comment của mình. Ví dụ, tôi sẽ hỏi anh vài câu để hiểu rõ hơn vì sao cái comment về “bình phương số nguyên tố” lại có vẻ không giúp được gì cho anh.

- Are there things in the comment that are misleading or confusing?
- Is there some important piece of information you acquired on your bike ride that suddenly made things clear?
 - Trong comment có chỗ nào gây hiểu lầm hay khó hiểu không?
 - Có mẫu thông tin quan trọng nào anh có được trong chuyến đạp xe khiến mọi thứ bỗng sáng tỏ không?

I would also show the comment to a few other people to get their takes on it. Then, I would rework the comment to improve it.

Tôi cũng sẽ đưa comment đó cho vài người khác xem để nghe cảm nhận của họ. Rồi tôi sẽ viết lại comment cho tốt hơn.

Given your fundamental disbelief in comments, I think it's likely that you would still see no value in the comment, even after my reworking. In this case, I would show the comment to other people, particularly those who have a more positive view of comments in general, and get their input. As long as the comment is not misleading and at least a few people found it helpful, I would retain it.

Với việc anh về cơ bản không tin vào comment, tôi nghĩ nhiều khả năng anh vẫn sẽ chẳng thấy cái comment đó có giá trị gì, kể cả sau khi tôi viết lại. Trong trường hợp ấy, tôi sẽ đưa comment cho những người khác xem, đặc biệt là những ai nhìn chung có cái nhìn tích cực hơn về comment, và lấy ý kiến của họ. Chừng nào comment không gây hiểu lầm và ít nhất vài người thấy nó hữu ích, tôi sẽ giữ nó lại.

Now let me discuss two few specific comments that you objected to. The first comment was the one for the multiples variable:

Giờ hãy để tôi bàn về hai comment cụ thể mà anh phản đối. Comment thứ nhất là comment cho biến multiples:

```
// Used to test efficiently (without division) whether a candidate
// is a multiple of a previously-encountered prime number. Each ent
// here contains an odd multiple of the corresponding entry in
// primes. Entries increase monotonically.
```

There is a bug in this comment that you exposed (the first entry is not odd); good catch! You then argued that most of the information in the comment is unnecessary and proposed this as an alternative:

Comment này có một lỗi mà anh đã vạch ra (phần tử đầu tiên không phải số lẻ); bắt bài hay lắm! Rồi anh lập luận rằng phần lớn thông tin trong comment là không cần thiết và đề xuất phương án thay thế này:

```
// multiples of corresponding prime.
```

You have left out too much useful information here. For example, I don't think it is safe to assume that readers will figure out that the motivation is avoiding divisions. It's always better to state these assumptions and motivations clearly so that there will be no confusion. And I think it's helpful for readers to know that these entries never decrease. I would simply fix the bug, leaving all of the information intact:

Ở đây anh đã lược bỏ quá nhiều thông tin hữu ích. Chẳng hạn, tôi không nghĩ có thể yên tâm giả định rằng người đọc sẽ tự suy ra được động cơ là tránh phép chia. Luôn tốt hơn

khi nêu rõ những giả định và động cơ này để không còn chỗ cho nhầm lẫn. Và tôi nghĩ việc người đọc biết rằng các phần tử này không bao giờ giảm cũng có ích. Tôi sẽ chỉ sửa cái lỗi kia thôi, giữ nguyên toàn bộ thông tin:

```
// Used to test efficiently (without division) whether a candidate
// is a multiple of a previously-encountered prime number. Each ent
// (except the first, which is never used) contains an odd multiple
// the corresponding entry in primes. Entries increase monotonically
```

The second comment was this one, for the for loop:

Comment thứ hai là cái này, cho vòng for:

```
// Each iteration of this loop tests the candidate against one
// potential prime factor. Skip the first factor (2) since we
// only consider odd candidates.
```

You objected to this comment because the code of the loop doesn't actually test the candidate against the prime factor; it tests it against a multiple. When I write implementation comments like this, my goal is not to restate the code; comments like that don't usually provide much value. The goal here was to say what the code is doing in a logical sense, not how it does it. In that sense, the comment is correct.

Anh phản đối comment này vì mã trong vòng lặp thật ra không kiểm tra candidate với thừa số nguyên tố; nó kiểm tra candidate với một bội số. Khi tôi viết những comment ở mức hiện thực như thế này, mục tiêu của tôi không phải là diễn đạt lại đoạn mã; những comment kiểu đó thường chẳng mang lại mấy giá trị. Mục tiêu ở đây là nói xem đoạn mã đang làm gì về mặt logic, chứ không phải nó làm điều đó như thế nào. Theo nghĩa đó, comment này là đúng.

However, if a comment causes confusion in the reader, then it is not a good comment. Thus, I would rewrite this comment to make it clear that it describes the abstract function of the code, not its precise **behavior**:

Tuy nhiên, nếu một comment khiến người đọc bối rối thì nó không phải là comment tốt. Vì vậy, tôi sẽ viết lại comment này để nói rõ rằng nó mô tả chức năng trừu tượng của đoạn mã, chứ không phải hành vi chính xác của nó:

```
// Each iteration of this loop considers one existing prime, ruling
// out the candidate if it is a multiple of that prime. Skip the
// first prime (2) since we only consider odd candidates.
```

To conclude, I agree with your assertion "it is very difficult to explain something to someone who is not intimate with the details you are trying to explain." And yet, it is our responsibility as programmers to do exactly that.

Để khép lại, tôi đồng ý với khẳng định của anh rằng "giải thích một điều gì đó cho người chưa thân thuộc với những chi tiết mà bạn đang cố giải thích là chuyện cực kỳ khó". Vậy mà, làm đúng điều đó lại chính là trách nhiệm của chúng ta với tư cách lập trình viên.

UB:

UB:

I'm glad we agree. We also agree about getting others to review the code and make recommendations on the code and the comments.

Tôi mừng vì chúng ta đồng ý với nhau. Chúng ta cũng đồng ý về việc nhờ người khác review đoạn mã và đưa ra khuyến nghị cho cả mã lẫn comment.

Bob's Rewrite of — Bob viết lại

UB:

UB:

When I saw your solution, and after I gained a good understanding of it, I refactored it just a bit. I loaded it into my IDE, wrote some simple tests, and extracted a few simple methods.

Khi xem giải pháp của anh, và sau khi đã hiểu nó tương đối tường tận, tôi refactor nó một chút. Tôi nạp nó vào IDE, viết vài test đơn giản, rồi tách ra (extract method) vài method nhỏ.

I also got rid of that awful labeled continue statement. And I added 3 to the primes list so that I could mark the first element as irrelevant and give it a value of -1 . (I think I was still reeling from the even/odd confusion.)

Tôi cũng vứt bỏ luôn cái câu lệnh continue có nhãn (labeled continue) kinh khủng kia. Và tôi thêm 3 vào danh sách primes để có thể đánh dấu phần tử đầu tiên là không liên quan và gán cho nó giá trị -1 . (Chắc lúc ấy tôi vẫn còn choáng váng vì vụ lẫn lộn chẵn/lẻ.)

I like this because the implementation of the

Tôi thích cách này vì phần hiện thực (implementation) của

generateFirstNPrimes method describes the moving parts in a way that hints at what is going on. It's easy to read that implementation and get a glimpse of the mechanism. I'm not at all sure that the comment helps.

method generateFirstNPrimes mô tả các bộ phận chuyển động theo cách gợi ý được chuyện gì đang diễn ra. Đọc phần hiện thực đó rất dễ, và ta thoáng thấy được cơ chế bên trong. Tôi hoàn toàn không chắc là comment giúp được gì.

I think it is just the reality of this algorithm that the effort required to properly explain it, and the effort required for anyone else to read and understand that explanation, is roughly equivalent to the effort needed to read the code and go on a bike ride.

Tôi cho rằng đây đơn giản là thực tế của thuật toán này: công sức bỏ ra để giải thích nó cho hàng hoàng, cộng với công sức để người khác đọc và hiểu lời giải thích đó, xấp xỉ bằng công sức để đọc mã rồi đi đạp xe một vòng.

```
package literatePrimes;
public class PrimeGenerator3 {
    private static int[] primes;

    private static int[] primeMultiples;
    private static int lastRelevantMultiple;
    private static int primesFound;
    private static int candidate;
    // Lovely little algorithm that finds primes by predicting
    // the next composite number and skipping over it. That predict
    // consists of a set of prime multiples that are continuously
    // increased to keep pace with the candidate.
```

```

public static int[] generateFirstNPrimes(int n) {
    initializeTheGenerator(n);
    for (candidate = 5; primesFound < n; candidate += 2) {
        increaseEachPrimeMultipleToOrBeyondCandidate();
        if (candidateIsNotOneOfThePrimeMultiples()) {
            registerTheCandidateAsPrime();
        }
    }
    return primes;
}

private static void initializeTheGenerator(int n) {
    primes = new int[n];
    primeMultiples = new int[n];
    lastRelevantMultiple = 1;
    // prime the pump. (Sorry, couldn't resist.)

    primesFound = 2;
    primes[0] = 2;
    primes[1] = 3;
    primeMultiples[0] = -1; // irrelevant
    primeMultiples[1] = 9;
}

private static void increaseEachPrimeMultipleToOrBeyondCandidate() {
    if (candidate >= primeMultiples[lastRelevantMultiple])
        lastRelevantMultiple++;
    for (int i = 1; i <= lastRelevantMultiple; i++)
        while (primeMultiples[i] < candidate)
            primeMultiples[i] += 2 * primes[i];
}

private static boolean candidateIsNotOneOfThePrimeMultiples() {
    for (int i = 1; i <= lastRelevantMultiple; i++)
        if (primeMultiples[i] == candidate)
            return false;
    return true;
}

private static void registerTheCandidateAsPrime() {
    primes[primesFound] = candidate;
    primeMultiples[primesFound] = candidate * candidate;
    primesFound++;
}
}

```

JOHN:

JOHN:

This version is a considerable improvement over the version in Clean Code . Reducing the number of methods made the code easier to read and resulted in cleaner interfaces. If it were properly commented, I think this version would be about as easy to read as my version (the additional methods you created didn't particularly help, but they didn't hurt either). I suspect that if we polled readers, some would like your version better and some would prefer mine.

Phiên bản này tốt hơn đáng kể so với phiên bản trong Clean Code. Giảm số lượng method làm mã dễ đọc hơn và cho ra những interface sạch hơn. Nếu được comment tử tế, tôi nghĩ phiên bản này cũng dễ đọc ngang phiên bản của tôi (mấy method anh tạo thêm chẳng giúp được gì đặc biệt, nhưng cũng chẳng hại gì). Tôi đoán nếu đem hỏi ý bạn đọc, sẽ có người thích bản của anh hơn và có người thích bản của tôi hơn.

Unfortunately, this revision of the code creates a serious performance **regression**: I measured a factor of three to four times slowdown compared to either of the earlier revisions. The problem is that you changed the processing of a particular candidate from a single loop to two loops (the

Đáng tiếc, bản sửa này gây ra một hồi quy (regression) hiệu năng nghiêm trọng: tôi đo được nó chậm hơn ba đến bốn lần so với cả hai bản trước đó. Vấn đề nằm ở chỗ anh đã đổi việc xử lý một candidate cụ thể từ một vòng lặp thành hai vòng lặp (hai method

increaseEach... and candidateIsNot... methods). In the loop from earlier revisions, and in the candidateIsNot method, the loop aborts once the candidate is disqualified (and most candidates are quickly eliminated). However, increaseEach... must examine every entry in primeMultiples . This results in 5 to 10 times as many loop iterations and a three-to-four-times overall slowdown.

increaseEach... và candidateIsNot...). Trong vòng lặp ở các bản trước, cũng như trong method candidateIsNot, vòng lặp thoát ngay khi candidate bị loại (và phần lớn candidate bị loại rất nhanh). Nhưng increaseEach... thì buộc phải duyệt mọi phần tử trong primeMultiples. Kết quả là số lần lặp tăng gấp 5 đến 10 lần và toàn bộ chương trình chậm đi ba đến bốn lần.

Given that the whole reason for the current algorithm (and its complexity) is to maximize performance, this slowdown is unacceptable. The two methods must be combined.

Xét rằng toàn bộ lý do tồn tại của thuật toán hiện tại (và cả độ phức tạp của nó) là để tối đa hoá hiệu năng, mức chậm này là không thể chấp nhận được. Hai method đó phải được gộp lại.

I think what happened here is that you were so focused on something that isn't actually all that important (creating the tiniest possible methods) that you dropped the ball on other issues that really are important. We have now seen this twice. In the original version of PrimeGenerator , you were so determined to make tiny methods that you didn't notice that the code was becoming incomprehensible. In this version, you were so eager to chop up my single method that you didn't notice that you were blowing up the performance.

Tôi nghĩ chuyện xảy ra ở đây là anh mải tập trung vào một thứ thực ra chẳng quan trọng đến thế (tạo ra những method nhỏ hết mức có thể) đến mức buông rơi những vấn đề thực sự quan trọng. Chúng ta đã thấy điều này hai lần rồi. Ở phiên bản PrimeGenerator gốc, anh quyết tâm làm method thật nhỏ đến mức không nhận ra rằng mã đang trở nên khó hiểu. Ở phiên bản này, anh hăng hái băm nhỏ method duy nhất của tôi đến mức không nhận ra mình đang thổi bay hiệu năng.

I don't think this was just an unfortunate combination of oversights. One of the most important things in software design is to identify what is important and focus on that; if you focus on things that are unimportant, you're likely to **mess** up the things that are important.

Tôi không cho rằng đây chỉ là sự trùng hợp xui xẻo của vài lần sơ suất. Một trong những điều quan trọng nhất trong thiết kế phần mềm là nhận ra cái gì quan trọng rồi tập trung vào đó; nếu anh tập trung vào những thứ không quan trọng, nhiều khả năng anh sẽ làm hỏng những thứ quan trọng.

The code in your revision is still under-commented. You believe that there is no meaningful way for comments to assist the reader in understanding the code. I think this stems from your general disbelief in the value of comments; you are quick to throw in the towel. This algorithm is unusually difficult to explain, but I still believe that comments can help. For example, I believe you must make some attempt to help readers understand why the first multiple for a prime is the square of the prime. You have taken a lot of time to develop your understanding of this; surely there must be some way to convey that understanding to others? If you had included that information in your original version of the code, you could have saved yourself that long bike ride. Giving up on this is an abdication of professional responsibility.

Mã trong bản sửa của anh vẫn thiếu comment. Anh tin rằng comment không có cách nào giúp người đọc hiểu mã một cách có ý nghĩa. Tôi cho rằng điều đó bắt nguồn từ việc anh nói chung không tin vào giá trị của comment; anh vứt khăn đầu hàng quá nhanh. Thuật toán này khó giải thích một cách bất thường, nhưng tôi vẫn tin comment có thể giúp được. Chẳng hạn, tôi tin anh phải cố gắng ít nhiều để giúp người đọc hiểu vì sao bội số đầu tiên của một số nguyên tố lại là bình phương của chính nó. Anh đã bỏ rất nhiều thời gian để tự mình hiểu ra điều đó; chắc chắn phải có cách nào truyền đạt hiểu biết ấy cho người khác chứ? Nếu anh đưa thông tin đó vào phiên bản mã ban đầu, anh đã tiết kiệm được cho chính mình cả chuyến đạp xe dài kia. Bỏ cuộc ở điểm này là chối bỏ trách nhiệm nghề nghiệp.

The few comments that you included in your revision are of little value. The first comment is too cryptic to provide much help: I can't make any sense of the phrase "predicting the next composite number and skipping over it," even though I completely understand the code it purports to explain. One of the comments is just a joke; I was surprised to see this, given your opposition to extraneous comments.

Vài comment ít ỏi anh đưa vào bản sửa cũng chẳng có mấy giá trị. Comment đầu tiên quá bí hiểm nên giúp được rất ít: tôi không tài nào hiểu nổi cụm "dự đoán hợp số kế tiếp rồi nhảy qua nó", dù tôi hiểu tường tận đoạn mã mà nó tự nhận là đang giải thích. Một trong các comment thì chỉ là câu đùa; tôi khá bất ngờ khi thấy điều này, xét việc anh vốn phản đối những comment thừa thãi.

Clearly, you and I live in different universes when it comes to comments.

Rõ ràng, khi bàn đến comment, anh và tôi sống ở hai vũ trụ khác nhau.

Finally, I don't understand why you are offended by the labeled continue statement in my code. This is a clean and elegant solution to the problem of escaping from nested loops. I wish more languages had this feature; the alternative is awkward code where you set a variable, then exit one level of loop, then check the variable and exit the next level.

Cuối cùng, tôi không hiểu vì sao anh lại thấy chướng mắt với câu lệnh continue có nhãn trong mã của tôi. Đó là một giải pháp sạch sẽ và thanh lịch cho bài toán thoát khỏi các vòng lặp lồng nhau. Tôi ước gì có nhiều ngôn ngữ hỗ trợ tính năng này hơn; lựa chọn thay thế là một đoạn mã lóng ngóng, trong đó anh đặt một biến, thoát một mức vòng lặp, rồi kiểm tra biến đó và thoát tiếp mức nữa.

UB:

UB:

Good catch! I would have caught that too had I thought to profile the solution. You are right that separating the two loops added some unnecessary iteration. I found a nice way to solve that problem without using the horrible continue . My updated version is now faster than yours! A million primes

in 440ms as opposed to yours, which takes 561ms. ;-) Below are just the changes.

Bắt lỗi hay lắm! Tôi cũng đã phát hiện ra nếu nghĩ tới chuyện profile giải pháp. Anh nói đúng, việc tách thành hai vòng lặp đã thêm vào một số lần lặp không cần thiết. Tôi đã tìm được một cách khá đẹp để giải quyết vấn đề đó mà không cần dùng đến cái continue kinh dị kia. Phiên bản cập nhật của tôi giờ còn nhanh hơn của anh! Một triệu số nguyên tố trong 440ms, so với 561ms của anh. ;-) Dưới đây chỉ là những phần thay đổi.

```
public static int[] generateFirstNPrimes(int n) {
    initializeTheGenerator(n);
    for (candidate = 5; primesFound < n; candidate += 2)
        if (candidateIsPrime())
            registerTheCandidateAsPrime();
    return primes;
}
private static boolean candidateIsPrime() {
    if (candidate >= primeMultiples[lastRelevantMultiple])
        lastRelevantMultiple++;
    for (int i = 1; i <= lastRelevantMultiple; i++) {
        while (primeMultiples[i] < candidate)
            primeMultiples[i] += 2 * primes[i];
        if (primeMultiples[i] == candidate)
            return false;
    }
    return true;
}
```

JOHN:

JOHN:

Yep, that fixes the problem. I note that you are now down to four methods, from eight in the Clean Code version.

Đúng vậy, thế là hết vấn đề. Tôi để ý là giờ anh đã rút xuống còn bốn method, so với tám method trong phiên bản Clean Code.

Test-Driven Development — Test-Driven Development

JOHN:

JOHN:

Let's move on to our third area of disagreement, which is test-driven development. I am a huge fan of unit testing. I believe that unit tests are an indispensable part of the software development process and pay for themselves over and over. I think we agree on this.

Chuyển sang khu vực bất đồng thứ ba của chúng ta: test-driven development. Tôi là fan cuồng của unit test. Tôi tin unit test là phần không thể thiếu của quy trình phát triển phần mềm và nó tự hoàn vốn hết lần này đến lần khác. Điểm này tôi nghĩ chúng ta đồng ý với nhau.

However, I am not a fan of test-driven development (TDD), which dictates that tests must be written before code and that code must be written and tested in tiny increments. This approach has serious

problems, without any compensating advantages that I have been able to identify.

Tuy nhiên, tôi không phải là fan của test-driven development (TDD), thứ quy định rằng test phải được viết trước mã và mã phải được viết cùng kiểm thử theo từng bước tăng tiến tí hon. Cách tiếp cận này có những vấn đề nghiêm trọng, mà tôi không nhận ra được lợi ích nào bù lại.

UB:

UB:

As I said at the start, I have carefully read A Philosophy of Software Design . I found it to be full of worthwhile insights, and I strongly agree with most of the points you make.

Như tôi đã nói ngay từ đầu, tôi đã đọc kỹ A Philosophy of Software Design. Tôi thấy nó đầy ắp những nhận định đáng giá, và tôi rất đồng tình với phần lớn các luận điểm của anh.

So I was surprised to find, on page 157, that you wrote a very short, dismissive, pejorative, and inaccurate section on test-driven development. Sorry for all the adjectives, but I think that's a fair characterization. So my goal here is to correct the misconceptions that led you to write the following:

Nên tôi khá bất ngờ khi phát hiện, ở trang 157, anh viết một mục rất ngắn, đầy vẻ gạt phăng, mang tính miệt thị và thiếu chính xác về test-driven development. Xin lỗi vì cả một tràng tính từ, nhưng tôi nghĩ đó là mô tả công bằng. Vậy nên mục tiêu của tôi ở đây là đính chính những hiểu lầm đã dẫn anh đến chỗ viết ra đoạn sau:

“Test-driven development is an approach to software development where programmers write unit tests before they write code. When creating a new class, the developer first writes unit tests for the class, based on its expected behavior. None of these tests pass, since there is no code for the class. Then the developer works through the tests one at a time, writing enough code for that test to pass. When all of the tests pass, the class is finished.”

“Test-driven development là cách tiếp cận phát triển phần mềm trong đó lập trình viên viết unit test trước khi viết mã. Khi tạo một class mới, lập trình viên trước hết viết unit test cho class đó, dựa trên hành vi mong đợi của nó. Không test nào chạy đậu cả, vì class chưa có mã. Rồi lập trình viên xử lý từng test một, viết vừa đủ mã để test đó đậu. Khi tất cả test đều đậu, class coi như hoàn tất.”

This is just wrong. TDD is quite considerably different from what you describe. I describe it using three laws.

Điều này đơn giản là sai. TDD khác khá xa những gì anh mô tả. Tôi mô tả nó bằng ba định luật.

1. You are not allowed to write any production code until you have first written a **unit test** that fails because that code does not exist.
2. You are not allowed to write more of a unit test than is sufficient to fail, and failing to compile is failing.
3. You are not allowed to write more production code than is sufficient to make the currently failing test pass.
 1. Bạn không được phép viết bất kỳ dòng mã production nào cho tới khi đã viết trước một unit test thất bại vì mã đó chưa tồn tại.
 2. Bạn không được phép viết một unit test dài hơn mức đủ để nó thất bại, và không biên dịch được cũng là thất bại.
 3. Bạn không được phép viết nhiều mã production hơn mức đủ để làm test đang thất bại chạy đậu.

A little thought will convince you that these three laws will lock you into a cycle that is just a few seconds long. You'll write a line or two of a test that will fail, you'll write a line or two of production code that will pass, around and around every few seconds.

Chỉ cần nghĩ một chút là bạn sẽ thấy ba định luật này khoá bạn vào một chu kỳ chỉ dài vài giây. Bạn viết một hai dòng test cho nó thất bại, rồi viết một hai dòng mã production cho nó đậu, cứ thế vòng đi vòng lại vài giây một lần.

A second layer of TDD is the **Red-Green-Refactor** loop. This loop is several minutes long. It is comprised of a few cycles of the three laws, followed by a period of reflection and refactoring. During that reflection, we pull back from the intimacy of the quick cycle and look at the design of the code we've just written. Is it clean? Is it well structured? Is there a better approach? Does it match the design we are pursuing? If not, should it?

Lớp thứ hai của TDD là vòng lặp Red-Green-Refactor. Vòng lặp này dài vài phút. Nó gồm vài chu kỳ của ba định luật, tiếp đó là một quãng ngẫm nghĩ và refactor. Trong lúc ngẫm nghĩ đó, ta lùi ra khỏi sự thân mật của chu kỳ chớp nhoáng và nhìn vào thiết kế của đoạn mã vừa viết. Nó có sạch không? Có được cấu trúc tốt không? Có cách nào hay hơn không? Nó có khớp với thiết kế ta đang theo đuổi không? Nếu không thì nó có nên khớp không?

JOHN:

JOHN:

Oops! I plead “guilty as charged” to inaccurately describing TDD. I will fix this in the next revision of APOSD. That said, your definition of TDD does not change my concerns.

Ồi! Tôi xin nhận “có tội như cáo buộc” vì đã mô tả TDD thiếu chính xác. Tôi sẽ sửa điều này trong lần tái bản tới của APOSD. Dù vậy, định nghĩa TDD của anh không làm thay đổi những lo ngại của tôi.

Let's discuss the potential advantages and disadvantages of TDD; then readers can decide for themselves whether they think TDD is a good idea overall.

Hãy bàn về những ưu và nhược điểm tiềm tàng của TDD; rồi bạn đọc có thể tự quyết định xem nhìn chung TDD có phải là ý hay hay không.

Before we start that discussion, let me clarify the approach I prefer as an alternative to TDD. In your online videos, you describe the alternative to TDD as one where a developer writes the code, gets it fully working (presumably with manual tests), then goes back and writes the unit tests. You argue that this approach would be terrible: Developers lose interest once they think code is working, so they wouldn't actually write the tests. I agree with you completely. However, this isn't the only alternative to TDD.

Trước khi bắt đầu cuộc thảo luận đó, để tôi làm rõ cách tiếp cận mà tôi ưa thích như một lựa chọn thay cho TDD. Trong các video trực tuyến của anh, anh mô tả lựa chọn thay thế TDD là: lập trình viên viết mã, làm cho nó chạy được đầy đủ (chắc là bằng test thủ công), rồi mới quay lại viết unit test. Anh lập luận rằng cách đó sẽ rất tệ: lập trình viên mất hứng ngay khi nghĩ rằng mã đã chạy, nên họ sẽ chẳng thực sự viết test. Tôi hoàn toàn đồng ý với anh. Tuy nhiên, đó không phải lựa chọn thay thế duy nhất cho TDD.

The approach I prefer is one where the developer works in somewhat larger units than in TDD, perhaps a few methods or a class. The developer first writes some code (anywhere from a few tens of lines to a few hundred lines), then writes unit tests for that code. As with TDD, the code isn't considered to be “working” until it has comprehensive unit tests.

Cách tôi ưa thích là lập trình viên làm việc theo những đơn vị lớn hơn TDD một chút, có lẽ là vài method hoặc một class. Trước hết họ viết một ít mã (từ vài chục đến vài trăm dòng), rồi viết unit test cho đoạn mã đó. Cũng như với TDD, mã chưa được coi là “chạy được” cho tới khi nó có bộ unit test toàn diện.

UB:

UB:

How about if we call this technique “bundling” for purposes of this document? This is the term I use in Clean Code, Second Edition.

Hay là trong tài liệu này ta gọi kỹ thuật đó là “bundling” nhé? Đây là thuật ngữ tôi dùng trong Clean Code, Second Edition.

JOHN:

JOHN:

Fine by me. The reason for working in larger units is to encourage design thinking so that a developer can think about a collection of related tasks and do a bit of planning to come up with a good overall design where the pieces fit together well. Of course, the initial design ideas will have flaws and refactoring will still be necessary, but the goal is to center the development process around design, not tests.

Tôi đồng ý. Lý do làm việc theo đơn vị lớn hơn là để khuyến khích tư duy thiết kế, để lập trình viên có thể nghĩ về một cụm nhiệm vụ liên quan với nhau và lên kế hoạch đôi chút nhằm đưa ra một thiết kế tổng thể tốt, nơi các mảnh ghép ăn khớp với nhau. Dĩ nhiên, những ý tưởng thiết kế ban đầu sẽ có khiếm khuyết và vẫn cần refactor, nhưng mục tiêu là đặt thiết kế — chứ không phải test — vào trung tâm của quy trình phát triển.

To start our discussion, can you make a list of the advantages you think that TDD provides over the approach I just described?

Để mở màn cuộc thảo luận, anh có thể liệt kê những lợi thế mà anh cho rằng TDD mang lại so với cách tiếp cận tôi vừa mô tả không?

UB:

UB:

The advantages I usually attribute to TDD are

Những lợi thế tôi thường quy cho TDD là

- Very little need for debugging. After all, if you just saw everything working a minute or two ago, there’s not much to debug.
- A stream of reliable, low-level documentation, in the form of very small and isolated unit tests. Those tests describe the low-level structure and operation of every facet of the system. If you want to know how to do something in the system, there are tests that will show you how.
- A less-coupled design, which results from the fact that every small part of the system must be designed to be testable, and testability requires **decoupling**.
- A suite of tests that you trust with your life, and therefore supports fearless refactoring.
 - Gần như không cần debug. Suy cho cùng, nếu mới một hai phút trước bạn còn thấy mọi thứ chạy ngon lành thì có gì nhiều để debug đâu.
 - Một dòng chảy tài liệu mức thấp đáng tin cậy, dưới dạng những unit test rất nhỏ và tách biệt. Những test đó mô tả cấu trúc và cách vận hành ở mức thấp của mọi góc

ngách trong hệ thống. Nếu bạn muốn biết làm một việc gì đó trong hệ thống ra sao, sẽ có test chỉ cho bạn.

- Một thiết kế ít kết dính hơn, xuất phát từ việc mọi phần nhỏ của hệ thống đều phải được thiết kế sao cho test được, mà muốn test được thì phải tách rời (decoupling).
- Một bộ test (test suite) mà bạn tin đến mức dám đánh cược cả mạng sống, và vì thế nó hậu thuẫn cho việc refactor không sợ hãi.

However, you asked me which of these advantages TDD might have over your preferred method. That depends on how big you make those larger units you described. The important thing to me is to keep the cycle time short, and to prevent entanglements that block testability.

Tuy nhiên, anh hỏi tôi trong số đó lợi thế nào TDD có mà cách anh ưa thích thì không. Điều đó phụ thuộc vào việc anh làm cho những đơn vị lớn hơn ấy lớn đến mức nào. Với tôi, điều quan trọng là giữ cho thời gian mỗi chu kỳ thật ngắn, và ngăn những mớ rối bòng bong chặn mất khả năng test.

It seems to me that working in small units, and then immediately writing after-the-fact tests, can give you all the above advantages, so long as you are very careful to test every aspect of the code you just wrote. I think a disciplined programmer could effectively work that way. Indeed, I think such a programmer would produce code that I could not distinguish from code written by another programmer following TDD.

Với tôi, có vẻ như làm việc theo đơn vị nhỏ rồi viết test ngay sau đó cũng cho bạn tất cả những lợi thế trên, miễn là bạn hết sức cẩn thận test mọi khía cạnh của đoạn mã vừa viết. Tôi nghĩ một lập trình viên có kỷ luật (discipline) hoàn toàn có thể làm việc hiệu quả theo cách đó. Thật ra, tôi nghĩ mã của một lập trình viên như vậy sẽ không thể phân biệt được với mã của một người theo TDD.

Above you suggested that bundling is to encourage design. I think encouraging design is a very good thing. My question for you is: Why do you think that TDD does not encourage design? My own experience is that design comes from strategic thought, which is independent of the tactical behavior of either TDD or bundling. Design is taking one step back from the code and envisioning structures that address a larger set of constraints and needs.

Ở trên anh nói bundling là để khuyến khích thiết kế. Tôi nghĩ khuyến khích thiết kế là điều rất tốt. Câu hỏi của tôi dành cho anh là: vì sao anh cho rằng TDD không khuyến khích thiết kế? Kinh nghiệm của riêng tôi là thiết kế đến từ tư duy chiến lược, vốn độc lập với hành vi chiến thuật của cả TDD lẫn bundling. Thiết kế là lùi lại một bước khỏi mã và hình dung ra những cấu trúc đáp ứng một tập ràng buộc và nhu cầu rộng hơn.

Once you have that vision in your head, it seems to me bundling and TDD will yield similar results.

Một khi đã có tầm nhìn đó trong đầu, theo tôi bundling và TDD sẽ cho ra kết quả tương tự nhau.

JOHN:

JOHN:

First, let me address the four advantages you listed for TDD.

Trước hết, để tôi nói về bốn lợi thế anh vừa liệt kê cho TDD.

- Very little need for debugging? I think any form of unit testing can reduce debugging work, but not for the reason you suggested. The benefit comes because unit tests expose bugs earlier and in an environment where they are easier to track down. A relatively simple bug to fix in development can be very painful to track down in production. I'm not convinced by your

argument that there's less debugging because "you just saw everything working a minute ago": It's easy to make a tiny change that exposes a really gnarly bug that has existed for a long time but hasn't yet been triggered. Hard-to-debug problems arise from the accumulated complexity of the system, not from the size of the code increments.

- Gần như không cần debug? Tôi nghĩ mọi hình thức unit test đều có thể giảm công sức debug, nhưng không phải vì lý do anh đưa ra. Lợi ích đến từ chỗ unit test phơi bày bug sớm hơn và trong một môi trường dễ truy vết hơn. Một bug tương đối dễ sửa lúc phát triển có thể trở thành cơn ác mộng khi phải truy vết trên production. Tôi không bị thuyết phục bởi lập luận rằng có ít việc debug hơn vì "một phút trước bạn vừa thấy mọi thứ chạy ngon": rất dễ xảy ra chuyện một thay đổi tí hon lại phơi bày một bug cực kỳ hiểm hóc, đã tồn tại từ lâu mà chưa bị kích hoạt. Những vấn đề khó debug sinh ra từ độ phức tạp tích tụ của hệ thống, chứ không phải từ kích thước của các bước tăng tiến mã.

UB:

UB:

True. However, when the cycles are very short, then the causes of even the gnarliest of bugs have the best chance of being tracked down. The shorter the cycles, the better the chances.

Đúng vậy. Tuy nhiên, khi các chu kỳ rất ngắn thì ngay cả nguyên nhân của những bug hiểm hóc nhất cũng có cơ hội tốt nhất để bị truy ra. Chu kỳ càng ngắn, cơ hội càng cao.

JOHN:

JOHN:

This is only true up to a point. I think you believe that making units smaller and smaller continues to provide benefits, with almost no limit to how small they can get. I think that there is a point of diminishing returns, where making things even smaller no longer helps and actually starts to hurt. We saw this disagreement over method length, and I think we're seeing it again here.

Điều đó chỉ đúng đến một mức nào đó. Tôi nghĩ anh tin rằng làm cho các đơn vị càng lúc càng nhỏ thì lợi ích cứ thế tăng thêm, gần như không có giới hạn nào cho độ nhỏ. Tôi thì cho rằng có một điểm mà lợi ích bắt đầu giảm dần, nơi việc làm mọi thứ nhỏ hơn nữa không còn giúp ích mà thực sự bắt đầu gây hại. Chúng ta đã thấy bất đồng này ở chuyện độ dài của method, và tôi nghĩ nó lại xuất hiện ở đây.

- Low-level documentation? I disagree: Unit tests are a poor form of documentation. Comments are a much more effective form of documentation, and you can put them right next to the relevant code. Trying to learn a method's interface by reading a bunch of unit tests seems much more difficult than just reading a couple of sentences of English text.
 - Tài liệu mức thấp? Tôi không đồng ý: unit test là một dạng tài liệu tồi. Comment là dạng tài liệu hiệu quả hơn nhiều, và anh có thể đặt chúng ngay cạnh đoạn mã liên quan. Cố học interface của một method bằng cách đọc cả đống unit test có vẻ khó hơn nhiều so với chỉ đọc vài câu tiếng Anh.

UB:

UB:

Nowadays, it's very easy to find the tests for a function by using the "where-used" feature of the IDE. As for comments being better, if that were true, then no one would publish example code. A less-coupled design? Possibly, but I haven't experienced this myself. It's not clear to me that designing for

testability will produce the best design. Generally the decoupling arises because the test requires a **mock** of some kind. Mocks tend to force abstractions that might otherwise not exist.

Ngày nay, tìm test cho một hàm rất dễ nhờ tính năng “where-used” của IDE. Còn chuyện comment tốt hơn, nếu điều đó đúng thì chẳng ai xuất bản mã ví dụ làm gì. Thiết kế ít kết dính hơn ư? Có thể, nhưng bản thân tôi chưa trải nghiệm điều đó. Tôi không chắc thiết kế để test được sẽ cho ra thiết kế tốt nhất. Nói chung việc tách rời nảy sinh vì test cần một loại mock nào đó. Mà mock có xu hướng ép ra những trừu tượng hoá (abstraction) mà lẽ ra có thể đã không tồn tại.

JOHN:

JOHN:

In my experience, mocking virtually never changes interfaces; it just provides replacements for existing (typically immovable) interfaces.

Theo kinh nghiệm của tôi, mock gần như không bao giờ làm thay đổi interface; nó chỉ cung cấp thứ thay thế cho những interface sẵn có (và thường là bất di bất dịch).

UB:

UB:

Our experiences differ.

Trải nghiệm của chúng ta khác nhau.

- Enabling fearless refactoring? Bingo! This is the where almost all of the benefits from unit testing come from, and it is a really, really big deal.
 - Cho phép refactor không sợ hãi? Trúng phóc! Đây chính là nơi gần như toàn bộ lợi ích của unit test đến từ, và đó là chuyện cực kỳ, cực kỳ lớn.

UB:

UB:

Agreed.

Đồng ý.

JOHN:

JOHN:

I agree with your conclusion that TDD and bundling are about the same in terms of providing these benefits. Now let me explain why I think TDD is likely to result in bad designs. The fundamental problem with TDD is that it forces developers to work too tactically, in units of development that are too small; it discourages design thinking. With TDD, the basic unit of development is one test: First the test is written, then the code to make that test pass. However, the natural units for design are larger than this: a class or method, for example. These units correspond to multiple test cases. If a developer thinks only about the next test, they are only considering part of a design problem at any given time. It's hard to design something well if you don't think about the whole design problem at once. TDD explicitly prohibits developers from writing more code than is needed to pass the current test; this discourages the kind of strategic thinking needed for good design.

Tôi đồng ý với kết luận của anh rằng TDD và bundling đem lại những lợi ích này ở mức tương đương nhau. Giờ để tôi giải thích vì sao tôi cho rằng TDD nhiều khả năng dẫn tới thiết kế tồi. Vấn đề căn bản của TDD là nó ép lập trình viên làm việc quá chiến thuật, theo

những đơn vị phát triển quá nhỏ; nó làm nản lòng tư duy thiết kế. Với TDD, đơn vị phát triển cơ bản là một test: viết test trước, rồi viết mã cho test đó chạy. Nhưng đơn vị tự nhiên của thiết kế thì lớn hơn thế: một class hoặc một method chẳng hạn. Những đơn vị đó ứng với nhiều test case. Nếu lập trình viên chỉ nghĩ đến test kế tiếp, thì tại mỗi thời điểm họ chỉ đang xem xét một phần của bài toán thiết kế. Rất khó thiết kế tốt một thứ nếu bạn không nghĩ về toàn bộ bài toán thiết kế cùng một lúc. TDD cấm tiết lập trình viên viết nhiều mã hơn mức cần để test hiện tại; điều này làm nản lòng đúng kiểu tư duy chiến lược cần cho một thiết kế tốt.

TDD does not provide adequate guidance to encourage design. You mentioned the Red-Green-**Refactor** loop, which recommends refactoring after each step, but there's almost no guidance for refactoring. How should developers decide when and what to refactor? This seems to be left purely to their own judgment. For example, if I am writing a method that requires multiple iterations of the TDD loop, should I refactor after every iteration (which sounds pretty tedious) or wait until after several iterations so that I can look at a bigger chunk of code when refactoring and hence be more strategic? Without guidance, it will be tempting for developers to keep putting off refactoring.

TDD không đưa ra chỉ dẫn đủ để khuyến khích thiết kế. Anh có nhắc tới vòng lặp Red-Green-Refactor, trong đó khuyến nghị refactor sau mỗi bước, nhưng gần như không có chỉ dẫn nào về việc refactor. Lập trình viên phải quyết định khi nào refactor và refactor cái gì như thế nào? Chuyện này có vẻ được phó mặc hoàn toàn cho phán đoán của họ. Chẳng hạn, nếu tôi đang viết một method cần nhiều vòng lặp TDD, tôi nên refactor sau mỗi vòng (nghe khá nhọc) hay đợi qua vài vòng để có thể nhìn vào một mảng mã lớn hơn khi refactor và nhờ đó chiến lược hơn? Không có chỉ dẫn, lập trình viên sẽ dễ bị cám dỗ cứ hoãn refactor mãi.

TDD is similar to the One Thing rule we discussed earlier in that it is biased: It provides very strong and clear instructions pushing developers in one direction (in this case, acting tactically) with only vague guidance in the other direction (designing more strategically). As a result, developers are likely to err on the side of being too tactical.

TDD giống với quy tắc One Thing mà chúng ta bàn ở phần trước ở chỗ nó thiên lệch: nó đưa ra những chỉ dẫn rất mạnh và rất rõ đẩy lập trình viên về một hướng (ở đây là hành động chiến thuật), trong khi hướng còn lại chỉ được chỉ dẫn mơ hồ (thiết kế chiến lược hơn). Hệ quả là lập trình viên nhiều khả năng sẽ lệch về phía quá chiến thuật.

TDD guarantees that developers will initially write **bad code**. If you start writing code without thinking about the whole design problem, the first code you write will almost certainly be wrong. Design only happens after a bunch of bad code has accumulated. I watched your video on TDD, and you repeatedly wrote the wrong code, then fixed it later. If the developer refactors conscientiously (as you did), they can still end up with good code, but this works against human nature. With TDD, that bad code will actually work (there are tests to prove it!), and it's human nature not to want to change something that works. If the code I'm developing is nontrivial, I will probably have to accumulate a lot of bad code with TDD before I have enough code in front of me to understand what the design should have been. It will be very difficult for me to force myself to throw away all that work.

TDD bảo đảm rằng ban đầu lập trình viên sẽ viết ra mã tệ. Nếu bạn bắt đầu viết mã mà không nghĩ về toàn bộ bài toán thiết kế, thì đoạn mã đầu tiên bạn viết gần như chắc chắn sẽ sai. Thiết kế chỉ xuất hiện sau khi một đống mã tệ đã tích tụ. Tôi có xem video của anh về TDD, và anh liên tục viết ra mã sai rồi sửa lại sau đó. Nếu lập trình viên refactor một cách có lương tâm (như anh đã làm), họ vẫn có thể kết thúc với mã tốt, nhưng điều đó đi ngược bản chất con người. Với TDD, đoạn mã tệ ấy thực sự chạy được (có test chứng minh

mà!), và bản chất con người là không muốn thay đổi thứ đang chạy tốt. Nếu đoạn mã tôi đang phát triển không phải loại tầm thường, có lẽ tôi sẽ phải tích tụ rất nhiều mã tệ với TDD trước khi có đủ mã trước mặt để hiểu ra thiết kế lẽ ra phải như thế nào. Và sẽ rất khó để tôi ép mình vứt bỏ toàn bộ công sức đó.

It's easy for a developer to believe they are doing TDD correctly while working entirely tactically, layering on hack after hack with an occasional minor refactor, without ever thinking about the overall design.

Một lập trình viên rất dễ tin rằng mình đang làm TDD đúng cách trong khi thực chất làm việc hoàn toàn theo kiểu chiến thuật, chông hack lên hack với đôi lần refactor nhỏ lẻ, mà chẳng bao giờ nghĩ tới thiết kế tổng thể.

I believe that the bundling approach is superior to TDD because it focuses the development process around design: Design first, then code, then write unit tests. Of course, refactoring will still be required: It's almost never possible to get the design right the first time. But starting with design will reduce the amount of bad code you write and get you to a good design sooner. It is possible to produce equally good designs with TDD; it's just harder and requires a lot more discipline.

Tôi tin cách tiếp cận bundling ưu việt hơn TDD vì nó đặt thiết kế vào trung tâm quy trình phát triển: thiết kế trước, rồi viết mã, rồi viết unit test. Dĩ nhiên vẫn cần refactor: gần như không bao giờ có chuyện thiết kế đúng ngay lần đầu. Nhưng bắt đầu bằng thiết kế sẽ giảm lượng mã tệ bạn viết ra và đưa bạn tới một thiết kế tốt sớm hơn. Với TDD cũng có thể tạo ra thiết kế tốt ngang vậy; chỉ là khó hơn và đòi hỏi nhiều kỷ luật hơn hẳn.

UB:

UB:

I'll address your points one at a time.

Tôi sẽ trả lời từng điểm một.

- I haven't found that the scale of TDD is so tactical that it discourages thinking. Every programmer, regardless of their testing discipline, writes code one line at a time. That's immensely tactical and yet does not discourage design. So why would one test at a time discourage it?
- The literature on TDD strongly discourages delaying refactoring. While thinking about design is strongly encouraged, both are integral parts of the discipline..
- We all write bad code at the start. The discipline of TDD gives us the opportunity, and the safety, to continuously clean it. Design insights arise from those kinds of cleaning activities. The discipline of refactoring allows bad designs to be transformed, one step at a time, into better designs.
- It's not clear to me why the act of writing tests late is a better design choice. There's nothing in TDD that prevents me from thinking through a design long before I write the very first tested code.
 - Tôi không thấy quy mô của TDD chiến thuật đến mức làm nản lòng việc suy nghĩ. Mọi lập trình viên, bất kể kỷ luật kiểm thử của họ là gì, đều viết mã từng dòng một. Chuyện đó cực kỳ chiến thuật, vậy mà đâu có làm nản lòng thiết kế. Vậy sao mỗi lần một test lại làm nản lòng nó?
 - Các tài liệu về TDD phản đối rất mạnh việc trì hoãn refactor. Việc suy nghĩ về thiết kế cũng được khuyến khích mạnh mẽ; cả hai đều là phần không tách rời của kỷ luật này.
 - Tất cả chúng ta đều viết mã tệ lúc ban đầu. Kỷ luật TDD cho ta cơ hội, và sự an toàn, để liên tục dọn sạch nó. Những nhận thức về thiết kế nảy sinh chính từ các hoạt động

dọn dẹp kiểu đó. Kỷ luật refactor cho phép biến những thiết kế tệ, từng bước một, thành những thiết kế tốt hơn.

- Tôi không rõ vì sao viết test muộn lại là lựa chọn thiết kế tốt hơn. Chẳng có gì trong TDD ngăn tôi nghĩ thấu đáo về một thiết kế từ rất lâu trước khi viết dòng mã có test đầu tiên.

JOHN:

JOHN:

You say there is nothing about TDD that stops developers from thinking ahead about design. This is only partly true. Under TDD, I can think ahead, but I can't actually write my ideas down in the form of code, since that would violate TDD Rule 1. This is a significant discouragement.

Anh nói chẳng có gì trong TDD ngăn lập trình viên nghĩ trước về thiết kế. Điều đó chỉ đúng một phần. Dưới TDD, tôi có thể nghĩ trước, nhưng tôi không thể thực sự viết ý tưởng của mình ra dưới dạng mã, vì như vậy là vi phạm Định luật 1 của TDD. Đó là một sự cản trở đáng kể.

You claim that “thinking about design is strongly encouraged” in TDD, but I haven't seen this in your discussions of TDD. I watched your video example of using TDD for computing bowling scores, and design is never even mentioned after the first minute or two (ironically, one of the conclusions of this example is that the brief initial design turned out to be useless). There is no suggestion of thinking ahead in the video; it's all about cleaning up messes after the fact. In all of the TDD materials you have shown me, I have not seen any warnings about the dangers of becoming so tactical with TDD that design never occurs (perhaps you don't even view this as a serious risk?).

Anh khẳng định rằng TDD “khuyến khích mạnh mẽ việc suy nghĩ về thiết kế”, nhưng tôi chưa thấy điều đó trong những gì anh trình bày về TDD. Tôi đã xem video ví dụ của anh về việc dùng TDD để tính điểm bowling, và sau một hai phút đầu thì thiết kế thậm chí không được nhắc tới lần nào nữa (trở trêu thay, một trong những kết luận của ví dụ đó là bản thiết kế sơ bộ ban đầu hoá ra vô dụng). Trong video không hề có gợi ý nào về việc nghĩ trước; tất cả chỉ xoay quanh việc dọn dẹp mớ hỗn độn sau khi đã bày ra. Trong toàn bộ tài liệu về TDD anh cho tôi xem, tôi chưa thấy lời cảnh báo nào về nguy cơ trở nên quá chiến thuật với TDD đến mức thiết kế chẳng bao giờ xảy ra (có lẽ anh thậm chí không xem đó là rủi ro nghiêm trọng?).

UB:

UB:

I usually use an abbreviated form of UML to capture my early design decisions. I have no objection to capturing them in pseudocode, or even real code. However, I would not commit any such prewritten code. I would likely hold it in a text file, and consult it while following the TDD cycle. I might feel safe enough to copy and paste from the text file into my IDE in order to make a failing test pass.

Tôi thường dùng một dạng UML rút gọn để ghi lại các quyết định thiết kế ban đầu. Tôi không phản đối việc ghi chúng bằng mã giả, hay thậm chí mã thật. Tuy nhiên, tôi sẽ không commit bất kỳ đoạn mã viết sẵn nào như vậy. Nhiều khả năng tôi sẽ giữ nó trong một file văn bản và tham khảo trong lúc theo chu kỳ TDD. Tôi thậm chí có thể thấy đủ an tâm để copy-paste từ file đó vào IDE nhằm làm một test đang thất bại chạy được.

The Bowling game is an example of how wildly our initial design decisions can deviate from our eventual solutions. It's true that introductory videos often do not expose the depth of a discipline.

Bowling game là ví dụ cho thấy các quyết định thiết kế ban đầu của chúng ta có thể lệch xa đến mức nào so với giải pháp cuối cùng. Đúng là các video nhập môn thường không

phơi bày hết chiều sâu của một kỷ luật.

JOHN:

JOHN:

As I was watching your TDD video for the second time, you said something that jumped out at me:

Khi xem lại video TDD của anh lần thứ hai, có một câu anh nói khiến tôi giật mình:

“Humans consider things that come first to be important and things that come at the end to be less important and somehow optional; that’s why they are at the end, so we can leave them out if we have to.”

“Con người coi những thứ đến trước là quan trọng, còn những thứ đến sau cùng thì kém quan trọng và có phần tùy chọn; chính vì thế mà chúng nằm ở cuối, để nếu cần thì ta có thể bỏ chúng đi.”

This captures perfectly my concern about TDD. TDD insists that tests must come first, and design, if it happens at all, comes at the end, after code is working. I believe that good design is the most important thing, so it must be the top priority. I don’t consider tests optional, but delaying them is safer than delaying design. Writing tests isn’t particularly difficult; the most important thing is having the discipline to do it. Getting a good design is really hard, even if you are very disciplined; that’s why it needs to be the center of attention.

Câu đó nắm bắt hoàn hảo mối lo của tôi về TDD. TDD khẳng định rằng test phải đến trước, còn thiết kế, nếu có xảy ra, thì đến sau cùng, sau khi mã đã chạy được. Tôi tin thiết kế tốt là thứ quan trọng nhất, nên nó phải là ưu tiên hàng đầu. Tôi không coi test là tùy chọn, nhưng hoãn test vẫn an toàn hơn hoãn thiết kế. Viết test không khó lắm; điều quan trọng nhất là có đủ kỷ luật để làm. Có được một thiết kế tốt mới thực sự khó, kể cả khi bạn rất kỷ luật; đó là lý do nó phải là tâm điểm chú ý.

UB:

UB:

TDD is a coding discipline. Of course, design comes before coding—I don’t know anyone who thinks otherwise. Even the Bowling Game video made that point. But as we saw in the Bowling Game video, sometimes the code will take you in a very different direction.

TDD là một kỷ luật viết mã. Dĩ nhiên thiết kế đến trước việc viết mã — tôi chẳng biết ai nghĩ khác cả. Ngay cả video Bowling Game cũng nói rõ điều đó. Nhưng như ta đã thấy trong chính video Bowling Game, đôi khi mã sẽ đưa bạn đi theo một hướng rất khác.

That difference doesn’t imply that the design shouldn’t have been done. It just implies that designs are speculative and may not always survive reality.

Sự khác biệt đó không hàm ý rằng lẽ ra không nên thiết kế. Nó chỉ hàm ý rằng thiết kế mang tính phỏng đoán và không phải lúc nào cũng sống sót trước thực tế.

As Eisenhower once said:

Như Eisenhower từng nói:

“In preparing for battle, I have always found that plans are useless, but planning is indispensable.”

“Khi chuẩn bị cho trận đánh, tôi luôn thấy rằng các kế hoạch thì vô dụng, nhưng việc lập kế hoạch thì không thể thiếu.”

JOHN:

JOHN:

You ask why writing tests later is a better design choice. It isn't. The benefit of the bundled approach doesn't come from writing tests later; it comes from doing design sooner. Writing tests (a bit) later is a consequence of this choice. The tests are still written pretty early on with the bundled approach, so I don't think the delay causes significant problems.

Anh hỏi vì sao viết test muộn hơn lại là lựa chọn thiết kế tốt hơn. Nó không tốt hơn. Lợi ích của cách tiếp cận bundling không đến từ việc viết test muộn hơn; nó đến từ việc thiết kế sớm hơn. Viết test muộn (một chút) chỉ là hệ quả của lựa chọn đó. Với bundling, test vẫn được viết khá sớm, nên tôi không nghĩ độ trễ đó gây ra vấn đề gì đáng kể.

UB:

UB:

I think we simply disagree that TDD discourages design. The practice of TDD does not discourage me from design, because I value design. I would suggest that those who do not value design will not design, no matter what discipline they practice.

Tôi nghĩ đơn giản là chúng ta bất đồng về chuyện TDD có làm nản lòng thiết kế hay không. Việc thực hành TDD không làm tôi nản lòng thiết kế, bởi tôi coi trọng thiết kế. Tôi cho rằng người nào không coi trọng thiết kế thì sẽ không thiết kế, dù họ thực hành kỷ luật nào đi nữa.

JOHN:

JOHN:

You claim that the problems I worry about with TDD simply don't happen in practice. Unfortunately, I have heard contrary claims from senior developers that I trust. They complain about horrible code produced by TDD-based teams, and they believe that the problems were caused by TDD. Of course, horrible code can be produced with any design approach. And maybe those teams didn't implement TDD properly, or maybe those cases were outliers. But the problems reported to me line up exactly with what I would expect to happen, given the tactical nature of TDD.

Anh khẳng định những vấn đề tôi lo ngại về TDD đơn giản là không xảy ra trong thực tế. Đáng tiếc, tôi lại nghe những khẳng định ngược lại từ các lập trình viên kỳ cựu mà tôi tin tưởng. Họ than phiền về đồng mã kinh khủng do các đội áp dụng TDD tạo ra, và họ tin rằng chính TDD gây ra những vấn đề đó. Dĩ nhiên, mã kinh khủng có thể sinh ra từ bất kỳ cách tiếp cận thiết kế nào. Và có thể những đội đó đã triển khai TDD không đúng cách, hoặc có thể đó chỉ là những trường hợp cá biệt. Nhưng những vấn đề được kể cho tôi khớp chính xác với điều tôi dự đoán sẽ xảy ra, xét bản chất chiến thuật của TDD.

UB:

UB:

My experience differs. I've worked on many projects where TDD has been used effectively and profitably. I'm sure the senior developers that you trust are telling you the truth about their experience. Having never seen TDD lead to such bad outcomes myself, I sincerely doubt that the blame can be traced to TDD.

Kinh nghiệm của tôi thì khác. Tôi đã làm nhiều dự án mà TDD được dùng một cách hiệu quả và có lợi. Tôi chắc rằng những lập trình viên kỳ cựu anh tin tưởng đang nói thật về trải nghiệm của họ. Nhưng vì bản thân chưa bao giờ thấy TDD dẫn tới kết cục tệ đến vậy, tôi thực lòng nghi ngờ việc truy trách nhiệm về TDD.

JOHN:

JOHN:

You ask me to trust your extensive experience with TDD, and I admit that I have no personal experience with TDD. On the other hand, I have a lot of experience with tactical programming, and I know that it rarely ends well. TDD is one of the most extreme forms of tactical programming I've encountered. In general, if "making it work" is the #1 priority instead of "develop a clean design," code turns to spaghetti. I don't see enough safeguards in your approach to TDD to prevent the disaster scenarios; I don't even see a clear recognition of the risk.

Anh bảo tôi hãy tin vào kinh nghiệm dày dạn của anh với TDD, và tôi thừa nhận mình không có kinh nghiệm cá nhân nào với TDD. Mặt khác, tôi có rất nhiều kinh nghiệm với lập trình chiến thuật, và tôi biết nó hiếm khi kết thúc êm đẹp. TDD là một trong những dạng lập trình chiến thuật cực đoan nhất tôi từng gặp. Nói chung, nếu "làm cho nó chạy" là ưu tiên số 1 thay vì "phát triển một thiết kế sạch", mã sẽ biến thành mì spaghetti. Tôi không thấy trong cách tiếp cận TDD của anh có đủ hàng rào bảo vệ để ngăn những kịch bản thảm họa; tôi thậm chí không thấy anh thừa nhận rõ ràng rủi ro đó.

Overall, TDD is in a bad place on the risk-reward spectrum. In comparison to the bundling approach, the downside risks for poor code quality in TDD are huge, and I don't see enough upside reward (if any) to compensate.

Nhìn tổng thể, TDD nằm ở vị trí xấu trên phổ rủi ro-phần thưởng. So với cách tiếp cận bundling, rủi ro về chất lượng mã kém của TDD là khổng lồ, còn phần thưởng thì tôi không thấy đủ (nếu có) để bù lại.

UB:

UB:

All I can say to that is that your opinion is based on a number of false impressions and speculations, and not upon direct experience.

Tất cả những gì tôi có thể nói là quan điểm của anh dựa trên một loạt ấn tượng sai lệch và suy đoán, chứ không phải dựa trên trải nghiệm trực tiếp.

JOHN:

JOHN:

Now let me ask you a couple of questions. First, at a microscopic level, why on earth does TDD prohibit developers from writing more code than needed to pass the current test? How does enforcing myopia make systems better?

Giờ để tôi hỏi anh vài câu. Thứ nhất, ở mức vi mô, tại sao trên đời TDD lại cấm lập trình viên viết nhiều mã hơn mức cần để test hiện tại đâu? Việc cưỡng ép người ta cận thị như vậy làm hệ thống tốt lên bằng cách nào?

UB:

UB:

The goal of the discipline is to make sure that everything is tested. One good way to do that is to refuse to write any code unless it is to make a failing test pass. Also, working in such short cycles provides insights into the way the code is working. Those insights often lead to better design decisions.

Mục tiêu của kỷ luật này là bảo đảm mọi thứ đều được test. Một cách hay để làm điều đó là từ chối viết bất kỳ đoạn mã nào trừ khi nó phục vụ việc làm một test đang thất bại chạy

đầu. Ngoài ra, làm việc theo những chu kỳ ngắn như vậy mang lại nhận thức về cách mã đang vận hành. Những nhận thức đó thường dẫn tới các quyết định thiết kế tốt hơn.

JOHN:

JOHN:

I agree that seeing code (partially) working can provide insights. But surely that benefit can be had without such a severe restriction on how developers think?

Tôi đồng ý rằng thấy mã chạy (dù chỉ một phần) có thể mang lại nhận thức. Nhưng chắc chắn lợi ích đó có thể đạt được mà không cần đến một hạn chế khắc nghiệt đến vậy lên cách lập trình viên suy nghĩ chứ?

Second, at a broader level, do you think TDD is likely to produce better designs than approaches that are more design centric, such as the bundling approach I described? If so, can you explain why?

Thứ hai, ở mức rộng hơn, anh có nghĩ TDD nhiều khả năng cho ra thiết kế tốt hơn so với những cách tiếp cận lấy thiết kế làm trung tâm, chẳng hạn cách bundling tôi vừa mô tả không? Nếu có, anh giải thích được vì sao không?

UB:

UB:

My guess is that someone adept at bundling and someone adept at TDD would produce very similar designs, with very similar **test coverage**. I would also venture to guess that the TDDer would be somewhat more productive than the bundler if for no reason other than that the TDDer finds and fixes problems earlier than the bundler.

Tôi đoán rằng một người thành thạo lối gom cụm (bundling) và một người thành thạo TDD sẽ cho ra những thiết kế rất giống nhau, với độ phủ test (test coverage) cũng rất giống nhau. Tôi cũng mạo muội đoán rằng người theo TDD sẽ có năng suất nhanh hơn người gom cụm, nếu không vì lý do nào khác thì cũng vì người theo TDD phát hiện và sửa vấn đề sớm hơn.

JOHN:

JOHN:

I think that the bundling approach will result in a better design because it actually focuses on design, rather than focusing on tests and hoping that a good design will magically emerge. I think it's really hard to argue that the best way to achieve one thing is to focus your attention on something else. And the bundling approach will make progress faster because the early thinking about design will reduce the amount of bad code you end up having to throw away under TDD. Overall, I'd argue that the best-case outcomes for the two approaches will be about the same, but average and (especially) worst-case outcomes will be far worse for TDD.

Tôi cho rằng cách gom cụm sẽ cho ra thiết kế tốt hơn, bởi nó thực sự tập trung vào thiết kế, thay vì tập trung vào test rồi hy vọng một thiết kế tốt sẽ hiện ra như có phép màu. Tôi nghĩ rất khó biện hộ cho ý tưởng rằng cách tốt nhất để đạt được một thứ là dồn sự chú ý vào một thứ khác. Và cách gom cụm sẽ tiến nhanh hơn, vì việc suy nghĩ sớm về thiết kế sẽ giảm bớt lượng mã tệ mà rốt cuộc bạn phải vứt đi khi làm TDD. Nhìn chung, tôi cho rằng kết quả trong trường hợp tốt nhất của hai cách là xấp xỉ nhau, nhưng kết quả trung bình và (nhất là) kết quả tệ nhất thì TDD kém xa.

JOHN:

JOHN:

I don't think we're going to resolve our disagreements on TDD. To do that, we'd need empirical **data** about the frequency of good and bad outcomes from TDD. Unfortunately, I'm not aware of any such data. Thus, readers will have to decide for themselves whether the potential benefits of TDD outweigh the risks.

Tôi không nghĩ chúng ta sẽ giải quyết được những bất đồng về TDD. Muốn vậy, ta cần dữ liệu thực nghiệm về tần suất TDD dẫn tới kết quả tốt và kết quả tệ. Tiếc là tôi không biết có dữ liệu nào như thế. Bởi vậy, độc giả sẽ phải tự quyết định xem lợi ích tiềm năng của TDD có xứng với rủi ro hay không.

For anyone who chooses to use TDD, I urge you to do so with extreme caution. Your primary goal must not be just working code, but rather, a clean design that will allow you to develop quickly in the future. TDD will not lead you naturally to the best design, so you will need to do significant and continuous refactoring to avoid spaghetti code. Ask yourself repeatedly: "Suppose that I knew everything I know now when I first started on this project; would I have chosen the current structure for the code?" When the answer is no (which will happen frequently), stop and refactor. Recognize that TDD will cause you to write more bad code than you may be used to, so you must be prepared to throw out and rewrite more than you are used to. Take time to plan ahead and think about the overall design, rather than just making the next test work. If you do all of these things diligently, I think it is possible to mitigate the risks of TDD and produce well-designed code.

Với ai chọn dùng TDD, tôi tha thiết khuyên bạn hãy hết sức thận trọng. Mục tiêu chính của bạn không được chỉ là mã chạy được, mà phải là một thiết kế sạch cho phép bạn phát triển nhanh trong tương lai. TDD sẽ không tự nhiên dẫn bạn tới thiết kế tốt nhất, nên bạn sẽ phải refactor nhiều và liên tục để tránh mã rối như mì spaghetti. Hãy tự hỏi đi hỏi lại: "Giả sử ngay từ khi bắt đầu dự án này tôi đã biết mọi thứ tôi biết bây giờ; liệu tôi có chọn cấu trúc mã hiện tại không?" Khi câu trả lời là không (mà điều đó sẽ xảy ra thường xuyên), hãy dừng lại và refactor. Hãy nhận ra rằng TDD sẽ khiến bạn viết ra nhiều mã tệ hơn mức bạn quen, nên bạn phải sẵn sàng vứt đi và viết lại nhiều hơn mức bạn quen. Hãy dành thời gian lập kế hoạch trước và suy nghĩ về thiết kế tổng thể, thay vì chỉ chăm chăm làm cho test kế tiếp chạy được. Nếu bạn làm tất cả những điều đó một cách nghiêm túc, tôi nghĩ hoàn toàn có thể giảm thiểu rủi ro của TDD và tạo ra mã có thiết kế tốt.

UB:

UB:

Let's just say that I agree with all that advice, but disagree with your assertion that TDD might be the cause of bad code.

Cứ cho là tôi đồng ý với toàn bộ lời khuyên đó, nhưng không đồng ý với khẳng định của anh rằng TDD có thể là nguyên nhân sinh ra mã tệ.

TDD Summary — Tóm tắt về TDD

JOHN:

JOHN:

Here is my attempt to summarize our thoughts on test-driven development: We agree that unit tests are an essential element in software development. They allow developers to make significant changes to a system without fear of breaking something. We agree that it is possible to use TDD to produce systems with good designs.

Đây là nỗ lực của tôi nhằm tóm tắt những suy nghĩ của hai chúng ta về phát triển hướng kiểm thử: Chúng ta nhất trí rằng unit test là một thành phần thiết yếu trong phát triển phần mềm. Chúng cho phép lập trình viên thay đổi hệ thống ở quy mô lớn mà không sợ làm hỏng thứ gì. Chúng ta cũng nhất trí rằng hoàn toàn có thể dùng TDD để tạo ra những hệ thống có thiết kế tốt.

- I believe that TDD discourages good design and can easily lead to very bad code. You do not believe that TDD discourages good design and don't see much of a risk of bad code.
- I believe that there are better approaches than TDD for producing good unit test suites, such as the bundling approach discussed above. You agree that bundling can produce outcomes just as good as TDD but think it may lead to somewhat less test coverage.
- I believe that TDD and bundling have similar best-case outcomes, but that the average and worst-case outcomes will be much worse for TDD. You disagree and believe that, if anything, TDD may produce marginally better outcomes than bundling. You also think that preference and personality are larger factors in making the choice between the two.
 - Tôi tin rằng TDD làm nản lòng thiết kế tốt và rất dễ dẫn tới mã cực tệ. Anh thì không tin TDD làm nản lòng thiết kế tốt và không thấy có mấy rủi ro sinh ra mã tệ.
 - Tôi tin rằng có những cách tốt hơn TDD để tạo ra bộ test (test suite) đơn vị tốt, chẳng hạn cách gom cụm đã bàn ở trên. Anh đồng ý rằng gom cụm có thể cho kết quả tốt ngang TDD, nhưng cho rằng nó có thể dẫn tới độ phủ test thấp hơn đôi chút.
 - Tôi tin rằng TDD và gom cụm có kết quả tốt nhất tương đương nhau, nhưng kết quả trung bình và tệ nhất của TDD thì kém hơn nhiều. Anh không đồng ý và tin rằng nếu có khác biệt thì TDD có lẽ còn cho kết quả nhỉnh hơn gom cụm một chút. Anh cũng cho rằng sở thích và tính cách mới là yếu tố lớn hơn khi chọn giữa hai cách.

UB:

UB:

This is a fair summary of our discussion. We seem to disagree over the best application of discipline. I prefer a disciplined approach to keep the code covered by tests written first in very short cycles. You prefer a disciplined approach of writing relatively longer bundles of code and then writing tests for those bundles. We disagree on the risks and rewards of these two disciplines.

Đây là bản tóm tắt công bằng cho cuộc thảo luận của chúng ta. Có vẻ chúng ta bất đồng về cách áp dụng kỷ luật (discipline) sao cho tốt nhất. Tôi thích một cách làm có kỷ luật nhằm giữ cho mã luôn được phủ bởi những test được viết trước, trong những chu kỳ rất ngắn. Anh thích một cách làm có kỷ luật là viết những cụm mã tương đối dài hơn rồi mới viết test cho chúng. Chúng ta bất đồng về rủi ro và phần thưởng của hai kỷ luật này.

Closing Remarks — Lời kết

JOHN:

JOHN:

First, I'd like to thank you for tolerating (and responding to) the arguments I have made about some of the key ideas in Clean Code . I hope this discussion will provide food for thought for readers.

Trước hết, tôi muốn cảm ơn anh đã chịu đựng (và hồi đáp) những luận điểm tôi nêu ra về một số ý tưởng cốt lõi trong Clean Code. Tôi hy vọng cuộc thảo luận này sẽ cho độc giả thứ gì đó để ngẫm nghĩ.

We have covered a lot of topics and subtopics in this discussion, but I think that most of my concerns result from two general errors made by Clean Code : failure to focus on what is important, and failure to balance design trade-offs.

Chúng ta đã đi qua rất nhiều chủ đề lớn nhỏ trong cuộc thảo luận này, nhưng tôi cho rằng phần lớn những băn khoăn của tôi đều bắt nguồn từ hai sai lầm chung của Clean Code: không tập trung vào thứ thực sự quan trọng, và không cân bằng được các đánh đổi trong thiết kế.

In software design (and probably in any design environment), it is essential to identify the things that really matter and focus on those. If you focus your attention on things that are unimportant, you are unlikely to achieve the things that really are important. Unfortunately, Clean Code repeatedly focuses on things that don't really matter, such as:

Trong thiết kế phần mềm (và có lẽ trong bất kỳ môi trường thiết kế nào), điều thiết yếu là nhận ra đâu là những thứ thực sự quan trọng rồi tập trung vào đó. Nếu bạn dồn sự chú ý vào những thứ không quan trọng, bạn khó lòng đạt được những thứ thực sự quan trọng. Đáng tiếc, Clean Code cứ liên tục tập trung vào những thứ chẳng mấy quan trọng, chẳng hạn:

- Dividing 10-line methods into 5-line methods and dividing 5-line methods into 2- or 3-line methods
- Eliminating the use of comments written in English
- Writing tests before code and making the basic unit of development a test rather than an abstraction
 - Chẻ method 10 dòng thành method 5 dòng, rồi chẻ method 5 dòng thành method 2 hoặc 3 dòng
 - Loại bỏ việc dùng comment viết bằng tiếng Anh
 - Viết test trước khi viết mã và lấy đơn vị phát triển cơ bản là một test thay vì một trừu tượng hoá (abstraction)

None of these provides significant value, and we have seen how they distract from producing the best possible designs.

Không điều nào trong số đó mang lại giá trị đáng kể, và chúng ta đã thấy chúng làm ta xao nhãng khỏi việc tạo ra thiết kế tốt nhất có thể như thế nào.

Conversely, Clean Code fundamentally undervalues comments, which are essential and irreplaceable. This comes at a huge cost. Without interface comments, the specifications for interfaces are incomplete. This is guaranteed to result in confusion and bugs. Without implementation comments, readers are forced to re-derive knowledge and intentions that were in the mind of the original developer. This wastes time and leads to more bugs.

Ngược lại, Clean Code về căn bản đánh giá quá thấp comment, thứ vốn thiết yếu và không gì thay thế được. Cái giá phải trả cho chuyện này là khổng lồ. Không có comment ở tầng interface, đặc tả cho các interface sẽ không đầy đủ. Điều đó chắc chắn dẫn tới nhầm lẫn và bug. Không có comment ở tầng hiện thực (implementation), người đọc buộc phải suy diễn lại từ đầu những hiểu biết và ý định vốn nằm trong đầu người viết mã ban đầu. Việc này tốn thời gian và sinh thêm bug.

In my opening remarks, I said that systems become complex when important information is not accessible and obvious to developers. By refusing to write comments, you are hiding important information that you have and that others need.

Trong phần mở đầu, tôi đã nói rằng hệ thống trở nên phức tạp khi thông tin quan trọng

không sẵn có và không hiển nhiên với lập trình viên. Khi từ chối viết comment, bạn đang giấu đi thông tin quan trọng mà bạn có còn người khác thì cần.

The second general error in Clean Code has to do with balance. Design represents a balance between competing concerns. Almost any design idea becomes a bad thing if taken to the extreme. However, Clean Code repeatedly gives very strong advice in one direction without correspondingly strong advice in the other direction or any meaningful guidance about how to recognize when you have gone too far. For example, making methods shorter is often a good thing, but the Clean Code position is so one-sided and extreme that readers are likely to chop things up too much. We saw in the PrimeGenerator example how this resulted in code that was nearly incomprehensible. Similarly, the Clean Code position on TDD is one-sided, failing to recognize any possible weakness and encouraging readers to take this to a tactical extreme where design is completely squeezed out of the development process.

Sai lầm chung thứ hai của Clean Code liên quan tới sự cân bằng. Thiết kế là sự cân bằng giữa những mối quan tâm xung khắc nhau. Gần như mọi ý tưởng thiết kế đều trở thành thứ tai hại nếu bị đẩy tới cực đoan. Vậy mà Clean Code cứ liên tục đưa ra lời khuyên rất mạnh theo một hướng, mà không có lời khuyên mạnh tương ứng theo hướng ngược lại, cũng chẳng có chỉ dẫn nào có ý nghĩa để nhận biết khi nào bạn đã đi quá xa. Chẳng hạn, làm cho method ngắn lại thường là điều tốt, nhưng lập trường của Clean Code phiến diện và cực đoan tới mức độc giả rất dễ bầm mọi thứ ra quá vụn. Chúng ta đã thấy trong ví dụ PrimeGenerator rằng chuyện đó dẫn tới thứ mã gần như không thể hiểu nổi. Tương tự, lập trường của Clean Code về TDD cũng phiến diện, không thừa nhận bất kỳ điểm yếu khả dĩ nào và khuyến khích độc giả đẩy nó tới mức cực đoan về mặt chiến thuật, nơi thiết kế bị vắt kiệt hoàn toàn khỏi quy trình phát triển.

UB:

UB:

John, I'd like to thank you for participating in this project. This was a lot of fun for me. I love disagreement and debate with smart people. I also think that we share far more values than separate us.

John, tôi muốn cảm ơn anh đã tham gia dự án này. Với tôi đây là một trải nghiệm rất thú vị. Tôi thích tranh luận và bất đồng với những người thông minh. Tôi cũng nghĩ rằng những giá trị chúng ta chia sẻ với nhau nhiều hơn hẳn những gì chia rẽ chúng ta.

For my part, I'll just say that I have given due consideration to the points you've made, and while I disagree with your closing remarks, I have integrated several of your better ideas, as well as this entire document, into the second edition of Clean Code .

Về phần mình, tôi chỉ xin nói rằng tôi đã cân nhắc thoả đáng những điểm anh nêu, và dù không đồng ý với lời kết của anh, tôi đã đưa vài ý tưởng hay nhất của anh — cùng với toàn bộ văn bản này — vào ấn bản thứ hai của Clean Code.

Thanks again, and give my best to your students.

Cảm ơn anh một lần nữa, và cho tôi gửi lời chúc tốt đẹp nhất tới các sinh viên của anh.

Bibliography

[A+DS=P]

Niklaus Wirth, Algorithms + Data Structures = Programs , Prentice Hall, 1976.

[APOS D]

John K. Ousterhout, A Philosophy of Software Design , Yaknyam Press, 2018.

[BeckTDD]

Kent Beck, Test Driven Development , Addison-Wesley, 2003.

[Clean Agile]

Robert C. Martin, Clean Agile , Pearson Education Inc., 2020.

[Clean Arch]

Robert C. Martin, Clean Architecture , Pearson Education Inc.,
2018.

[Clean Craftsmanship]

Robert C. Martin, Clean Craftsmanship: Disciplines, Standards, and Ethics , Pearson Education Inc.,
2022.

[DDD]

Eric Evans, Domain Driven Design , Addison-Wesley, 2004.

[DOET]

Don Norman, The Design of Everyday Things , Basic Books,
2002.

[Functional Design]

Robert C. Martin, Functional Design , Pearson Education Inc.,
2024.

[GOF95]

Gamma, Helm, Johnson, Vlissides, Design Patterns , Addison- Wesley, 1995.

[Hex]

Alistair Cockburn and Juan Manuel Garrido de Paz, Hexagonal Architecture Explained , Humans and
Technology Inc., 2024.

[IMP]

Kent Beck, Implementation Patterns , Addison-Wesley, 2007.

[KP78]

Kernighan and Plaugher, The Elements of Programming Style, Second Edition , McGraw-Hill, 1978.

[Lea99]

Doug Lea, Concurrent Programming in Java: Design Principles and Patterns , Second Edition , Prentice Hall,

1999.

[OOSC]

Bertrand Meyer, Object Oriented Software Construction , Prentice Hall, 1988.

[Pomodoro] Francesco Cirillo, The Pomodoro Technique , Crown Publishing Group, 2006.

[PPEDC]

M. V. Wilkes, D. J. Wheeler, and S. Gill, The Preparation of Programs for an Electronic Digital Computer , Addison- Wesley, 1951.

[PPP02]

Robert C. Martin, Agile Software Development: Principles, Patterns, and Practices , Prentice Hall, 2002.

[Prag]

Andrew Hunt and Dave Thomas, The Pragmatic Programmer , Addison-Wesley, 2000.

[Refactoring]

Martin Fowler, Refactoring, Second Edition , Pearson Education Inc., 2019.

[RSpec]

Aslak Hellesøy and David Chelimsky, RSpec: Behavior Driven Development for Ruby Programmers , Pragmatic Bookshelf, 2008.

[Singularity]

Ray Kurzweil, The Singularity is Nearer , Viking, 2024.

[SP72]

O.-J. Dahl, E. W. Dijkstra, and C. A. R. Hoare, Structured Programming , Academic Press, 1972.

[Tidy]

Kent Beck, Tidy First? , O'Reilly, 2023.

[We Programmers]

Robert C. Martin, We, Programmers , Pearson Education Inc.,

2025.

[WELC]

Michael Feathers, Working Effectively with Legacy Code , Pearson Education Inc., 2004.